

A SELF-STUDY BOOK FOR THE CODING ROUND

Data Structures & Algorithms for the Coding Round

Every problem climbs from brute force to optimal, and every line of code was run

THE LANGUAGE OF THIS BOOK

JavaScript (Node) is the primary language. Python appears as a second opinion where it is genuinely shorter. There is no C++ and no Java in this book — the appendix supplies the five data structures JavaScript is missing, all of them tested.

- TIER 1** Indian service companies — TCS NQT, Accenture, Infosys, Wipro, Capgemini, Cognizant
- TIER 2** Singapore & Thailand — Grab, Sea/Shopee, GIC, DBS, Agoda, SCB, LINE MAN, Razer
- TIER 3** Global product companies — Google, Amazon, Microsoft, Goldman Sachs, D. E. Shaw, Adobe, Uber

20 chapters · arrays to dynamic programming · approach ladders · dry runs · one-page revision cards

A NOTE ON WHERE THESE PROBLEMS COME FROM

Every problem in this book is **original**. Each one was written for this book.

Nothing here is copied from any published book, course, question bank, or online judge.

No company's actual interview or test paper is reproduced, quoted, or paraphrased. Real hiring questions are private property and are not public documents.

When a problem is labelled, for example, "TCS NQT · pattern" or "Grab · pattern", that label means one thing only: the problem was written to match the **publicly documented shape** of that company's round — its topic mix, its usual

difficulty, and its time pressure. The label is never a claim that this question, or anything like it, was asked at a real company.

Company names are used only to describe that pattern. This book is not affiliated with, endorsed by, or connected to any company named in it. All trademarks belong to their owners.

Every code listing in this book was executed before it was printed. Where the text claims an output, that output was copied from a real run. Timings were measured on one ordinary machine and will differ on yours — the **shape** of the growth is the part that transfers.

Contents

How to use this book	4
Chapter 1 · Complexity Analysis & Thinking About Limits	8
Chapter 2 · Arrays & Prefix Sums	44
Chapter 3 · Two Pointers & Sliding Window	81
Chapter 4 · Binary Search	126
Chapter 5 · Sorting & Custom Comparators	167
Chapter 6 · Strings & Pattern Matching	204
Chapter 7 · Hashing & Frequency Maps	246
Chapter 8 · Stacks & Monotonic Stack	281
Chapter 9 · Queues, Deques & Monotonic Deque	317
Chapter 10 · Linked Lists	354
Chapter 11 · Recursion & Backtracking	388
Chapter 12 · Trees & Binary Trees	427
Chapter 13 · Binary Search Trees & Balanced Trees	471
Chapter 14 · Heaps & Priority Queues	511
Chapter 15 · Graphs I — Traversal & Components	550
Chapter 16 · Graphs II — Shortest Paths, MST & Topological Sort	592
Chapter 17 · Dynamic Programming I — 1D & Knapsack	629
Chapter 18 · Dynamic Programming II — 2D, Strings, Trees & Bitmask	669
Chapter 19 · Greedy & Intervals	718
Chapter 20 · Bit Manipulation, Math & Number Theory	761
Appendix · The JS Toolkit	805

How to use this book

Read these two pages once. Then start Chapter 1.

1 • What the three tiers mean

Every chapter has the same four blocks. You climb them in order, **inside** the chapter.

Tier	Companies	What the questions feel like	Time / Q
0 Warm-up	nobody — it is you	Bare mechanics. One idea per question, no story around it. You are building the reflex for the template, not being tested.	2–5 min
1 Service	TCS NQT, Accenture, Infosys, Wipro, Capgemini, Cognizant	The standard, well-known problems. One clean pattern per question. Correctness and speed of typing matter more than insight.	10–20 min
2 SE Asia	Grab, Sea/Shopee, GIC, DBS, Agoda, SCB, LINE MAN, Razer	The same patterns wearing a business suit — fares, orders, deliveries, trades. Two or three steps, and you must find the pattern under the story yourself.	20–30 min
3 Product	Google, Amazon, Microsoft, Goldman Sachs, D. E. Shaw, Adobe, Uber	Needs one genuine insight, not recall. Always followed by “now do it in $O(1)$ space”, “now it is a stream”, “now it is distributed”. The follow-up is half the interview.	30–45 min

Target the tier that matches the companies you are applying to — but always work Tier 1 first, even for a product-company interview. Each Tier 3 problem in a chapter is built directly on the Tier 1 and Tier 2 problems that come before it. Skipping the climb is what makes the hard problems feel impossible.

2 • What is inside every chapter

Pattern in one page	When to reach for this technique, the template code you will type over and over, and the invariant — the one sentence that is true at every step. Learn the invariant, not the code.
The approach ladder	No problem jumps straight to the optimal answer. You see Approach 1 — brute force , its complexity, why it is too slow; then Approach 2 ; then Approach 3 — optimal . After the ladder, one line names the idea that unlocked the optimal. That line is the transferable part.

Dry run	One problem per chapter is traced line by line, with a table of the state after every single step. Work it on paper alongside the book.
Practice + answer key	Timed sets per tier. The key gives full working, not just answers.
One-page revision card	Templates, a when-to-use-what table, a complexity table you should be able to quote from memory, and the top traps in the order they bite.

3 • The five rules

1	Type every listing and run it. Reading code teaches you nothing. This book’s code was all executed before printing; yours must be too. <code>node solution.js</code> with your own test cases, every time.
2	Attempt before you read. Give each problem an honest try with a pen, up to the tier’s time limit. A solution you read before trying only feels like learning.
3	When you are stuck, write down where. Then read the solution and find that exact line. That line is your real lesson, not the whole solution.
4	Always say the complexity out loud before you write code, and again after. If you cannot state it, you do not yet understand your own solution.
5	Redo every problem you got wrong — three days later, from a blank page. Not the same day. Three days later.

TRAP

The four edge cases that fail more submissions than anything else: **empty input, one element, all elements equal, and the answer being at the very first or very last index**. Test those four before you say “done”. They take twenty seconds and they are on every judge.

4 • The language, and the appendix you will need

JavaScript is the primary language of this book. It is accepted by every judge and every interviewer, and it is what most students in this course already write.

It also has five gaps that C++, Java and Python do not: no heap, no ordered map, no real queue, no binary-search helper, and numbers that stop being exact at $2^{53} - 1$. The appendix **The JS Toolkit** fills all five, with tested code you can type from memory.

Read the appendix’s “**JS gotchas for interviews**” page before Chapter 1, and again the night before any test. Four of the ten gotchas there produce a **wrong answer with no error message**. Those are the ones that cost marks.

5 • The 10-week plan

Two chapters a week. About two hours a day, six days a week. Day 7 is the checkpoint.

Week	Chapters	Daily target	Checkpoint (day 7)
1	1, 2	Ch 1 fully on days 1–2 (it is the shortest and it sets up everything). Then Ch 2 warm-up + Tier 1.	Write the Ch 1 constraint-to-algorithm table from memory.
2	3, 4	Warm-up + Tier 1 of each, then Tier 2. Type every template.	Re-solve one sliding-window and one binary-search problem from a blank page.
3	5, 6, 7	These three are lighter. Tier 1 and Tier 2 of each; Tier 3 only of Ch 6.	Mixed set: 6 problems from Ch 1–7, timed.
4	8, 9, 10	Stacks, deques and linked lists. All tiers — these are short chapters with high exam frequency.	Revision cards of Ch 1–10, written out from memory.
5	11, 12	Recursion and trees. Tier 1 and Tier 2 daily; Tier 3 on days 5–6.	Re-solve two tree traversals iteratively , no recursion.
6	13, 14	BSTs and heaps. Type the toolkit <code>MinHeap</code> from memory on day 1 and check it against the appendix.	Top-K, merge-K and running-median from a blank page.
7	15, 16	Graphs. BFS and DFS on days 1–2, then shortest paths, MST and topological sort.	Draw the “which graph algorithm” decision table from memory.
8	17	DP I, slowly. One section per day. Write every recursion first, then memoise, then loop.	The 0/1 versus unbounded knapsack loop direction, explained out loud.
9	18, 19	DP II and greedy. Tier 1 and Tier 2 daily.	Mixed set: 6 DP problems, timed at 25 minutes each.
10	20	Bits, maths and number theory. Then read all 20 revision cards.	Full mixed set across the whole book, 8 problems, strictly timed.

6 • The 3-week crash plan

Use this only if your test is inside a month. It skips nothing — it cuts the practice volume and the Tier 3 work. About four hours a day.

Week	Chapters	Daily target	Checkpoint (day 7)
1	1–7	Pattern page + Tier 1 of each chapter. Skip Tier 3 entirely for now. Roughly one chapter a day.	Mixed set of Ch 1–7, timed. Read the JS Toolkit gotchas page.
2	8–16	Pattern page + Tier 1, plus Tier 2 of Ch 12, 14, 15 and 16 (trees, heaps and graphs carry the most marks).	Revision cards of Ch 1–16, written from memory.
3	17–20	DP is the priority: Ch 17 over days 1–2, Ch 18 on day 3. Ch 19 and 20 on days 4–5. Day 6 is a full mixed set.	Re-solve every problem you got wrong in weeks 1 and 2.

TRAP

The crash plan works only if you **type and run everything from day one**. Reading solutions quickly will make you feel ready and then fail you at the keyboard, where you have to produce working code with no reference open.

7 • How to use the revision cards

While studying	Ignore it. Learn from the approach ladders instead.
After finishing a chapter	Cover the card. Write it out from memory — templates, complexity table, traps. Compare. The gaps are your revision list.
Last week before a test	Read only the 20 cards. That is about an hour for the whole book. Then one timed mixed set per day.

Last hour

The cards of your three weakest chapters, plus the JS Toolkit gotchas page. Nothing else.

In the interview itself

The complexity tables are what you quote. Practise saying them out loud, because you will be speaking, not writing.

Complexity Analysis & Thinking About Limits

Count the steps before you write the code

Pattern in one page

In a coding round you get a problem, a time limit, and a line that says $1 \leq n \leq 10^5$. That line is not decoration. It tells you which algorithm is allowed. This chapter teaches you to read it.

THE COUNTING RULES

What we count. Not seconds. We count how many basic steps the code does when the input has size n . A basic step is one comparison, one addition, one array read, one array write.

Rule 1 — a loop that runs k times costs k steps.

Rule 2 — loops side by side ADD. $n + n = 2n$.

Rule 3 — loops inside loops MULTIPLY. $n \times n = n^2$.

Rule 4 — drop the constants. $3n + 7$ becomes $O(n)$. A doubling of speed does not change the shape of the curve.

Rule 5 — keep only the fastest-growing term. $n^2 + 100n + 5000$ becomes $O(n^2)$.

Rule 6 — a loop that halves (or doubles) costs $\log_2 n$ steps. Because n can be halved only about $\log_2 n$ times before it reaches 1.

Rule 7 — “how many steps in the WORST case”. Big-O is a promise about the worst input, unless the problem says otherwise.

The ladder of growth, slowest-growing first. $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

How big each shape really gets

This table is not theory. It is the output of a program that counted the steps.

n	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(2^n)$
10	10	33	100	1,024
20	20	86	400	1,048,576
1,000	1,000	9,965	1,000,000	too big to print
100,000	100,000	1,660,964	10,000,000,000	too big to print

Look at the last row. For $n = 100,000$ the $O(n \log n)$ column is about 1.6 million, and the $O(n^2)$ column is 10 billion. They are not “both a bit slow”. One finishes instantly. The other never finishes.

The machine budget

A judge machine does roughly 10^8 (a hundred million) simple operations per second. Measured with Node on the machine this book was written on:

How long 10^8 simple steps really takes

JS

```
for (const n of [1e6, 1e7, 1e8]) {
  const t0 = Date.now();
```

```

let s = 0;
for (let i = 0; i < n; i++) s += i % 7;
const ms = Date.now() - t0;
console.log(`n=${n} sum=${s} time=${ms} ms`);
}

```

Output:

Measured output	TEXT
n=1000000 sum=2999997 time=4 ms	
n=10000000 sum=29999994 time=22 ms	
n=100000000 sum=299999995 time=146 ms	

So 10^8 of the simplest possible steps took about 0.15 second in Node. Real steps (Map lookups, string work, array pushes) are 5 to 50 times heavier. That is why the safe working number is:

SHORTCUT

Budget: about 10^8 operations per second. Aim to stay under 10^8 total. If your step count is above 10^9 , the solution will not pass. If it is below 10^7 , you have room to spare and can pick the simpler code.

The table you should memorise

Read n from the constraints. Look it up here. That row tells you what to build.

n up to	Target complexity	What that usually means
10 – 12	$O(n!)$	try every ordering (permutations)
20 – 25	$O(2^n)$ or $O(2^{n/2})$	try every subset; meet in the middle
100	$O(n^3)$	three nested loops; Floyd–Warshall
1{,}000 – 5{,}000	$O(n^2)$	two nested loops; simple DP over pairs
10^5	$O(n \log n)$	sort, binary search, heap, set/map
10^6	$O(n)$ or $O(n \log n)$	one pass, prefix sums, hashing, two pointers
10^9	$O(\log n)$ or $O(\sqrt{n})$	maths, binary search on the answer
10^{18}	$O(1)$ or $O(\log n)$	a formula, fast power, digit DP

TRAP

The most common wrong move in a coding round is to write the first idea that comes to mind, run it on the 3 sample cases, see “Accepted” on the samples, submit, and get **Time Limit Exceeded**. The samples are tiny. The hidden tests are at the limit. Do the arithmetic **before** you type.

Space costs too

Memory limits are usually 256 MB. Every number in a plain JS array is a double, 8 bytes. A typed array (Int32Array) stores 4 bytes per value and is worth knowing for the biggest inputs.

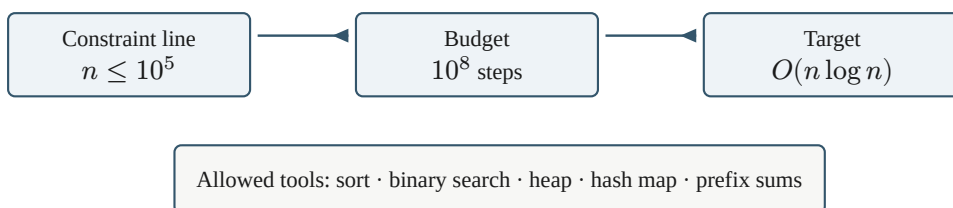
n	new Int32Array(n)	plain array of n numbers	$n \times n$ numbers
100,000	0.4 MB	0.8 MB	76,294 MB
1,000,000	3.8 MB	7.6 MB	7,629,395 MB

10,000,000	38.1 MB	76.3 MB	huge
------------	---------	---------	------

A 1-D array of ten million numbers is fine. A 2-D array of $n \times n$ for $n = 10^5$ is impossible. When the problem says $n \leq 10^5$ and you reach for a 2-D table, stop.

The four-step routine, every single time

Step	What you do
1. Read the limit	Find n , q (number of queries), and the value range.
2. Pick the target	Use the table above. “ $n = 10^5$, so I need $O(n \log n)$.”
3. Then design	Only now think about the algorithm. The target rules out most ideas for you.
4. Check the numbers	Can any sum or product pass $2^{53} \approx 9 \times 10^{15}$? If yes, BigInt.



Read the limit first. It chooses the family of algorithms for you.

The overflow check you must never skip

JavaScript has one number type, and it is a double. Whole numbers are exact only up to $\text{Number.MAX_SAFE_INTEGER} = 2^{53} - 1 = 9\{, \}007\{, \}199\{, \}254\{, \}740\{, \}991$. Past that, the arithmetic silently rounds. Anything that can go past it needs BigInt.

Where the doubles start lying JS

```

const sumFormula = (n) => (n * (n + 1)) / 2;

console.log('MAX_SAFE_INTEGER =', Number.MAX_SAFE_INTEGER);
console.log('n=100000 sum =', sumFormula(100000));

const a = 1234567891, b = 987654321, mod = 1000000007;
console.log('a*b as a Number =', a * b);
console.log('a*b as a BigInt =', (BigInt(a) * BigInt(b)).toString());
console.log('(a*b) % mod Number =', (a * b) % mod);
console.log('(a*b) % mod BigInt =', Number((BigInt(a) * BigInt(b)) % BigInt(mod)));
console.log('2**53 === 2**53 + 1 ? ', 2 ** 53 === 2 ** 53 + 1);
  
```

Output:

Measured output TEXT

```

MAX_SAFE_INTEGER = 9007199254740991
n=100000 sum = 5000050000
a*b as a Number = 1219326312114007000
a*b as a BigInt = 1219326312114007011
(a*b) % mod Number = 578722919
(a*b) % mod BigInt = 578722890
2**53 === 2**53 + 1 ? true
  
```

The Number answer is not “a bit off”. The last three digits of the product are gone, so the remainder 578{, }722{, }919 is simply the wrong answer. Sums of 10^5 values of 10^4 (that is 10^9) are safe. A product of two values near 10^9 is not.

TRAP

Numbers are doubles. They are exact only to $2^{53} - 1$. The moment a **product** of two values above about 9.4×10^7 appears, or a sum passes 9×10^{15} , switch to BigInt: `(BigInt(a) * BigInt(b)) % BigInt(m)`. BigInt is about 10 times slower, so use it only on the values that need it, and convert back with `Number(...)` at the end.

The other JavaScript facts this chapter needs

TRAP

arr.sort() sorts LEXICOGRAPHICALLY, as text. Tested in Node: `[10, 9, 1].sort()` returns `[1, 10, 9]`, because the string “10” comes before the string “9”. Numbers **always** need a comparator: `[10, 9, 1].sort((a, b) => a - b)` returns `[1, 9, 10]`. This chapter sorts in five different examples. Every one of them passes a comparator, and so must you. A bare `.sort()` is the single most common JavaScript bug in a coding round.

TRAP

Bitwise operators are 32-bit. JavaScript converts both sides of `&`, `|`, `^`, `<<`, `>>` and `>>>` to a 32-bit signed integer first. Tested in Node: `2**40 | 0` prints 0, and `(2**40) >>> 1` also prints 0. Above 2^{31} the bits are simply thrown away. For bit work on values past 32 bits, use BigInt (`1n`, `&`, `>>` all work on BigInt). See Example 14.

TRAP

JavaScript has no built-in heap and no ordered map. There is no PriorityQueue and no sorted-key dictionary. Use the tested MinHeap, lowerBound and upperBound from the **JS Toolkit** appendix. Every chapter that needs them starts with one line:

The one require line

JS

```
const { MinHeap, lowerBound, upperBound } = require('./toolkit.js');
```

Map keeps **insertion** order, not sorted order, so it is not a substitute. When you need “the smallest key greater than x”, keep a sorted array and binary-search it.

Use Map, not a plain object, for counting. A plain object turns every key into a string, so the number 1 and the text “1” become the same key. Tested in Node: after `o[1] = 'num'; o['1'] = 'str'`, `o[1]` is ‘str’ and `Object.keys(o)` is `["1"]`. A Map keeps both, and `mm.size` is 2. A Map is also faster when you add and delete a lot — measured in Example 16.

Warm-up

WARM-UP

build the reflex

Example 1

What is the complexity of this loop, and how many times does the body run for $n = 5$?

One plain loop

JS

```
const oneLoop = (n) => { let c = 0; for (let i = 0; i < n; i++) c++; return c; };
```

SOLUTION

The loop starts at $i = 0$ and stops when $i = n$. So it runs for $i = 0, 1, 2, \dots, n - 1$, which is n values. For $n = 5$: $i = 0, 1, 2, 3, 4$, so the body runs 5 times. Rule 1: a loop that runs n times is $O(n)$.

Answer: $O(n)$; the body runs 5 times

Example 2

Two loops, one after the other. Complexity?

Loops side by side

JS

```
const twoLoops = (n) => {
  let c = 0;
  for (let i = 0; i < n; i++) c++;
  for (let j = 0; j < n; j++) c++;
  return c;
};
```

SOLUTION

First loop: n steps. Second loop: n steps. They are side by side, so we **add**. Total = $n + n = 2n$. Rule 4 says drop the constant 2. Measured for $n = 1024$: the counter ended at 2048, which is $2n$. And $2n$ is $O(n)$.

Answer: $O(n)$

Example 3

Nested loops, both full length. Complexity?

Loops inside loops

JS

```
const nested = (n) => {
  let c = 0;
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++) c++;
  return c;
};
```

SOLUTION

For **each** of the n values of i , the inner loop runs n times. Rule 3 says nested loops multiply: $n \times n = n^2$. Measured counts: $n = 2$ gave 4, $n = 16$ gave 256, $n = 1024$ gave 1,048,576. Each one is exactly n^2 .

Answer: $O(n^2)$

Example 4

The inner loop starts at $i + 1$, not at 0. Is it still $O(n^2)$?

The triangle loop

JS

```
const countInner = (n) => {
  let steps = 0;
  for (let i = 0; i < n; i++)
```



```

    for (let j = i + 1; j < n; j++) steps++;
    return steps;
};

```

SOLUTION

Count the inner runs one row at a time.

- $i = 0$: j goes from 1 to $n - 1$, that is $n - 1$ steps.
- $i = 1$: $n - 2$ steps.
- $i = 2$: $n - 3$ steps.
- ... down to $i = n - 1$: 0 steps.

$$\text{Total} = (n - 1) + (n - 2) + \dots + 1 + 0 = \frac{n(n-1)}{2}.$$

Measured output of the program above:

Measured output

TEXT

```

n=0  steps=0  n(n-1)/2=0
n=1  steps=0  n(n-1)/2=0
n=2  steps=1  n(n-1)/2=1
n=4  steps=6  n(n-1)/2=6
n=8  steps=28 n(n-1)/2=28
n=100 steps=4950 n(n-1)/2=4950
n=1000 steps=499500 n(n-1)/2=499500

```

$\frac{n(n-1)}{2} = \frac{n^2-n}{2}$. Drop the constant $\frac{1}{2}$ and the smaller term n . What is left is n^2 .

Answer: Yes, $O(n^2)$ — it does about half the work of the full square, and half of n^2 is still $O(n^2)$

Example 5

How many times does this loop run?

Halving

JS

```

const halvings = (n) => {
  let steps = 0;
  while (n > 1) { n = Math.floor(n / 2); steps++; } // JS division gives 7 / 2 =
3.5
  return steps;
};

```

SOLUTION

Each turn cuts n in half. Starting at n , the values are $n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots$ until it reaches 1. The number of halvings is $\log_2 n$ rounded down.

Measured output:

Measured output

TEXT

```

n=1  halvings=0
n=8  halvings=3
n=1000 halvings=9
n=100000 halvings=16
n=1000000000 halvings=29

```

Check one: $n = 8$ gives $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$, which is 3 halvings, and $\log_2 8 = 3$. Correct.

Answer: $O(\log n)$ — only 29 steps even for a billion

SHORTCUT

Memorise these three: $\log_2 10^3 \approx 10$, $\log_2 10^6 \approx 20$, $\log_2 10^9 \approx 30$. So “ $\log n$ ” is never more than about 30 in a coding round. Treat it as a small constant when you estimate.

Example 6

An outer loop of n and an inner loop that doubles. Complexity?

n times log n

JS

```
const f4 = (n) => {
  let c = 0;
  for (let i = 0; i < n; i++)
    for (let j = 1; j < n; j *= 2) c++;
  return c;
};
```

SOLUTION

The inner loop multiplies j by 2 each turn, so it runs $\log_2 n$ times (Example 5, read backwards). The outer loop runs n times. Nested, so multiply: $n \times \log_2 n$. Measured: $n = 16$ gave $c = 64 = 16 \times 4$, and $\log_2 16 = 4$. Correct. $n = 1024$ gave $c = 10240 = 1024 \times 10$, and $\log_2 1024 = 10$. Correct.

Answer: $O(n \log n)$

Example 7

Which is faster for $n = 10^5$: a loop that adds 1 to n , or the formula?

O(n) versus O(1)

JS

```
const sumLoop = (n) => { let s = 0; for (let i = 1; i <= n; i++) s += i; return s; };
const sumFormula = (n) => (n * (n + 1)) / 2;
```

SOLUTION

sumLoop does n additions, so $O(n)$. sumFormula does one multiply, one add, one divide, no matter how big n is, so $O(1)$.

Both were run and both printed the same answers:

Measured output

TEXT

```
n=0 loop=0 formula=0
n=1 loop=1 formula=1
n=5 loop=15 formula=15
n=100000 loop=5000050000 formula=5000050000
```

Note $n = 0$ works in both: the loop body never runs, and the formula gives $0 \times \frac{1}{2} = 0$.

Answer: The formula, $O(1)$ against $O(n)$

Edge cases to always test on a counting problem: $n = 0$ (loop body never runs) and $n = 1$ (loop body runs once, or zero times if the inner loop starts at $i + 1$).

Tier 1 — Read the code, name its cost

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 8 [TCS NQT · pattern]

Problem. Given an array of n integers, report whether any value appears twice. **Constraints.** $n \leq 2 \times 10^5$, every value fits in a safe JavaScript integer. **Target.** $O(n \log n)$ or better. **Edge cases.** empty array; one element; two equal negatives.

SOLUTION

Approach 1 — Compare every pair ($O(n^2)$, too slow at $2 \cdot 10^5$)

Brute force

JS

```
const hasDuplicateBrute = (a) => {
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] === a[j]) return true;
  return false;
};
```

Time $O(n^2)$ **Space** $O(1)$

For $n = 2 \cdot 10^5$ that is $2 \cdot 10^{10}$ pair checks. At 10^8 per second: over 3 minutes.

Approach 2 — Sort, then look at neighbours ($O(n \log n)$, passes)

If two equal values exist, sorting puts them next to each other. So we only need to compare each element with the one before it.

Sort first

JS

```
const hasDuplicateSort = (a) => {
  const b = [...a].sort((x, y) => x - y); // the comparator is NOT optional
  for (let i = 1; i < b.length; i++)
    if (b[i] === b[i - 1]) return true;
  return false;
};
```

Time $O(n \log n)$ **Space** $O(n)$ for the copy, or $O(1)$ if you may sort in place

$2 \cdot 10^5 \cdot 18 \approx 3.6 \cdot 10^6$ steps. Instant.

Approach 3 — Remember what you have seen ($O(n)$ expected — optimal)

Set

JS

```
const hasDuplicateSet = (a) => {
  const seen = new Set();
  for (const x of a) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
};
```

```
    return false;
};
```

Time $O(n)$ expected **Space** $O(n)$

A Set lookup is $O(1)$ expected, not guaranteed — say 'expected' out loud.

All three were run in Node on the same tests:

Measured output

TEXT

```
[] brute=false sort=false set=false
[7] brute=false sort=false set=false
[1,2,3,4] brute=false sort=false set=false
[1,2,3,2] brute=true sort=true set=true
[-5,-5] brute=true sort=true set=true
[-3,0,3,-3] brute=true sort=true set=true
[0,0] brute=true sort=true set=true
bare sort of [10,9,1] = [1,10,9]
with comparator      = [1,9,10]
```

Look at the last two lines. They come from the same program. Without $(x, y) \Rightarrow x - y$ the array is sorted as **text**, so 10 lands before 9 and “equal values end up next to each other” is no longer true.

Python does the same job with a set, and this is one of the chapter’s signature problems, so here it is:

Python version of the optimal

PYTHON

```
def has_duplicate(a):
    seen = set()
    for x in a:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Run on the same inputs, Python prints False False False True True True for [], [7], [1,2,3,4], [1,2,3,2], [-5, -5], [0,0] — the same answers as the JavaScript.

The idea that unlocked it. The brute force asks “is $a[i]$ equal to anything else?” and searches the whole array for the answer. Both faster versions replace that search: sorting makes the answer a neighbour, hashing makes the answer a lookup. Whenever an inner loop is only **searching**, you can usually delete it.

Example 9 [Infosys · pattern]

Problem. Find the value that appears most often in an array. If there is a tie, any of the winners is fine. **Constraints.** $1 \leq n \leq 10^6$; values may be negative. **Target.** $O(n)$ or $O(n \log n)$. **Edge cases.** one element; all elements equal; negative values.

SOLUTION

Approach 1 — Sort and count runs ($O(n \log n)$, passes)

Sort, then count each run

JS

```
const mostFrequentSort = (a) => {
    if (a.length === 0) return undefined;
    const b = [...a].sort((x, y) => x - y);
    let best = b[0], bestCount = 1, cur = b[0], count = 1;
```

```

for (let i = 1; i < b.length; i++) {
  if (b[i] === cur) count++;
  else { cur = b[i]; count = 1; }
  if (count > bestCount) { bestCount = count; best = cur; }
}
return best;
};

```

Time $O(n \log n)$ **Space** $O(n)$ for the copy

Approach 2 — Count in one pass with a Map ($O(n)$ expected — optimal)

Frequency Map

JS

```

const mostFrequentMap = (a) => {
  if (a.length === 0) return undefined;
  const freq = new Map();
  let best = a[0], bestCount = 0;
  for (const x of a) {
    const c = (freq.get(x) ?? 0) + 1; // get() returns undefined, not 0
    freq.set(x, c);
    if (c > bestCount) { bestCount = c; best = x; }
  }
  return best;
};

```

Time $O(n)$ expected **Space** $O(k)$ where k is the number of distinct values

Measured, both versions on the same inputs:

Measured output

TEXT

```

[] sort=undefined map=undefined
[4] sort=4 map=4
[1,1,2] sort=1 map=1
[-2,-2,-2,5,5] sort=-2 map=-2
[0,0,1,1] sort=0 map=0

```

The last line is the tie case: both counts are 2, and both versions returned 0, because 0 reached count 2 first. A tie rule must be stated in the problem, or you must ask.

TRAP

`freq.get(x)` returns `undefined` for a key that is not there, and `undefined + 1` is `NaN`. Always write `(freq.get(x) ?? 0) + 1`. Use a `Map` and not a plain object here, because the values can be negative: an object would turn the number `-2` into the string `"-2"`.

The idea. Sorting costs $O(n \log n)$ only to bring equal values together. A `Map` brings them together for free.

Example 10 [Wipro · pattern]

Problem. Remove every odd number from an array, keeping the order of the even ones. **Constraints.** $n \leq 2 \times 10^5$. **Target.** $O(n)$. **Edge cases.** empty array; array with only odd numbers (answer is empty).

SOLUTION

Approach 1 — splice() out each odd value ($O(n^2)$ — the hidden cost)

Looks $O(n)$. Is not.

JS

```
const removeOddSlow = (a) => {
  const v = [...a];
  for (let i = 0; i < v.length; ) {
    if (v[i] % 2 !== 0) v.splice(i, 1);
    else i++;
  }
  return v;
};
```

There is only **one** loop, so it looks like $O(n)$. But `splice` in the middle of an array has to shift every element after that position one slot to the left. That shift is a hidden inner loop of up to n steps.

Approach 2 — Two pointers: read and write ($O(n)$ — optimal)

Read pointer r , write pointer w

JS

```
const removeOddFast = (a) => {
  const v = [...a];
  let w = 0;
  for (let r = 0; r < v.length; r++)
    if (v[r] % 2 === 0) v[w++] = v[r];
  v.length = w; // truncate in place; no new array
  return v;
};
```

Time $O(n)$ **Space** $O(1)$ extra

Both versions were run on the numbers 1 to 20,000, with a counter on the element moves:

Measured output

TEXT

```
same result? yes size 10000
splice version shifted 100000000 elements
two-pointer version wrote 10000 elements
empty -> 0
single odd -> 0
negatives [-4,-3,0] -> [-4,0]
splice time 41 ms    two-pointer time 1 ms
```

One hundred million moves against ten thousand. That is a factor of 10,000 on an input of only 20,000 items, and it shows up as 41 ms against 1 ms.

`v.length = w` is the JavaScript way to shorten an array in place. `a.filter(x => x % 2 === 0)` is also $O(n)$ and is the shortest correct answer — but write the two-pointer version at least once, because the same read/write pair is the core of Chapter 3.

TRAP

`x % 2 !== 0` is the safe odd test. `x % 2 === 1` is **wrong** for negatives: in JavaScript `-3 % 2` is `-1`, not `1`. Tested above on `[-4, -3, 0]`, which correctly keeps `[-4, 0]`.

The idea. Any library call inside a loop may hide its own loop. `splice`, `unshift`, `indexOf`, `includes`, and `slice` are the usual suspects. Ask “what does this call have to touch?” before you count it as one step.

TRAP

`splice` inside a loop, `unshift` inside a loop, and `includes / indexOf` inside a loop are the three most common accidental $O(n^2)$ bugs in placement tests. Each one **looks** like a single line.

Example 11 [Capgemini · pattern]

Problem. Build the list $n, n - 1, \dots, 2, 1$ in that order, by walking i upwards from 1. **Constraints.** $n \leq 10^5$. **Target.** $O(n)$. **Edge cases.** $n = 0$ (empty list); $n = 1$.

SOLUTION

Approach 1 — `out.unshift(i)` ($O(n^2)$ — every existing item shifts right)

One line, one loop, quadratic

JS

```
const buildUnshift = (n) => {  
  const out = [];  
  for (let i = 1; i <= n; i++) out.unshift(i); // pushes everything right by one  
  return out;  
};
```

Approach 2 — `push`, walking downwards ($O(n)$ — optimal)

`push` only ever touches the end

JS

```
const buildPush = (n) => {  
  const out = [];  
  for (let i = n; i >= 1; i--) out.push(i);  
  return out;  
};
```

Time $O(n)$ **Space** $O(n)$ for the output

If the walk direction is fixed by the problem, `push` then `out.reverse()` is still $O(n)$: one reverse costs $\frac{n}{2}$ swaps, not $\frac{n^2}{2}$ shifts.

Measured:

Measured output

TEXT

```
n=0 unshift=[] push=[]  
n=1 unshift=[1] push=[1]  
n=5 unshift=[5,4,3,2,1] push=[5,4,3,2,1]  
n=20000 same? true unshift=23 ms push=4 ms  
n=100000 same? true unshift=630 ms push=2 ms
```

Five times the input, but 27 times the time. That ratio is the signature of $O(n^2)$: $5^2 = 25$.

Answer: Use `push` (plus `reverse` if you must), never `unshift` inside a loop

JavaScript strings are the exception. In C++ and Java, `s = s + 'a'` inside a loop is $O(n^2)$ because it copies the whole string every turn. V8 does not: it builds a **rope**, a small tree of pieces, and flattens it only when the string is actually read. Tested in Node: `s += 'a'` one hundred thousand times finished in 5 ms. So `+=` on strings is safe in JavaScript — but `unshift` on arrays is not, and that is the trap that survives the language change.

Example 12 [Accenture · pattern]

Problem. Search a sorted array of n values. How many comparisons in the worst case? **Constraints.** $n \leq 10^6$. **Target.** $O(\log n)$. **Edge cases.** empty array; value not present; array of length 1.

SOLUTION

Binary search with a step counter

JS

```
const binarySearch = (a, target) => {
  let lo = 0, hi = a.length - 1;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2); // plain / gives 4.5, not 4
    if (a[mid] === target) return mid;
    if (a[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
};
```

Run on an array of one million even numbers, counting loop turns:

Measured output

TEXT

```
find 0 -> index 0 steps 19
find 1999998 -> index 999999 steps 20
find 7 (absent) -> index -1 steps 20
empty array -> index -1 steps 0
one element, found -> index 0 steps 1
```

Twenty steps for one million elements. A linear scan would need up to one million. $\log_2 10^6 \approx 20$, which matches exactly.

Time $O(\log n)$ **Space** $O(1)$

TRAP

JavaScript division does not round. $(0 + 9) / 2$ is 4.5, and $a[4.5]$ is undefined, so the search silently fails. You must round: `Math.floor((lo + hi) / 2)`.

$(lo + hi) >> 1$ also rounds down and is faster to type — but $>>$ is a **32-bit** operator. Tested in Node: with $lo = 3000000000$ and $hi = 3000000001$, $(lo + hi) >> 1$ printed 852516352, while `Math.floor((lo + hi) / 2)` printed 3000000000. Array indexes never get that large, so $>> 1$ is safe for searching an array; it is **not** safe when you binary-search over a range of **answers** that can pass 2^{31} . Use `Math.floor` there.

Example 13 [Cognizant · pattern]

Problem. Name the complexity of each of these four functions. **Constraints.** $n \leq 10^6$.

Four loop shapes

JS

```
const f1 = (n) => { let c=0; for (let i=0;i<n;i++) c++;
                      for (let j=0;j<n;j++) c++; return c; };
const f2 = (n) => { let c=0; for (let i=0;i<n;i++)
                      for (let j=0;j<n;j++) c++; return c; };
const f3 = (n) => { let c=0; for (let i=1;i<n;i*=2) c++; return c; };
const f4 = (n) => { let c=0; for (let i=0;i<n;i++)
                      for (let j=1;j<n;j*=2) c++; return c; };
```

SOLUTION

Measured counter values:

Measured output

TEXT

```
n=1 f1=2 f2=1 f3=0 f4=0
n=2 f1=4 f2=4 f3=1 f4=2
n=16 f1=32 f2=256 f3=4 f4=64
n=1024 f1=2048 f2=1048576 f3=10 f4=10240
```

- f1: the count is always $2n$. Side-by-side loops add. $O(n)$.
- f2: the count is n^2 ($1024^2 = 1\{, \}048\{, \}576$). Nested loops multiply. $O(n^2)$.
- f3: the count is $\log_2 n$ ($1024 \rightarrow 10$). $O(\log n)$.
- f4: the count is $n \log_2 n$ ($1024 \times 10 = 10\{, \}240$). $O(n \log n)$.

Answer: f1 $O(n)$, f2 $O(n^2)$, f3 $O(\log n)$, f4 $O(n \log n)$

Example 14 [TCS Digital · pattern]

Problem. Count the 1-bits in a number. **Constraints.** $0 \leq x < 2^{63}$. **Target.** $O(\log x)$ or better. **Edge cases.** $x = 0$; x a power of two; x larger than 2^{32} .

SOLUTION

First, the JavaScript problem. The constraint says 2^{63} , and JavaScript's bitwise operators only work on 32 bits. Tested in Node:

Measured output — the 32-bit cliff

TEXT

```
2**40 | 0      = 0
(2**40) >>> 1  = 0 (should be 2**39 = 549755813888 )
(2**31) | 0      = -2147483648
```

Everything above bit 31 is thrown away, and bit 31 itself comes back negative. So for this constraint the values must be BigInt. BigInt supports $\&$, $|$, $\wedge$, $\ll$ and $\gg$ at any width, and its literals end in n : $1n$, $0n$.

Approach 1 — Shift one bit at a time ($O(\text{number of bit positions}) = O(\log x)$)

One step per bit position

JS

```
const countBitsSlow = (n) => {           // n is a BigInt
  let c = 0;
  while (n > 0n) { c += Number(n & 1n); n >>= 1n; }
  return c;
};
```

Approach 2 — Clear the lowest set bit ($O(\text{number of SET bits})$ — optimal)

$n \& (n - 1)$ clears exactly the lowest 1-bit and leaves the rest alone. So the loop runs once per 1-bit, not once per bit position.

One step per set bit

JS

```
const countBitsFast = (n) => {           // n is a BigInt
  let c = 0;
  while (n > 0n) { n &= n - 1n; c++; }
  return c;
};
```

Measured, both agree:

Measured output

TEXT

```

0 -> slow 0 fast 0
1 -> slow 1 fast 1
7 -> slow 3 fast 3
8 -> slow 1 fast 1
1023 -> slow 10 fast 10
1024 -> slow 1 fast 1
1099511627776 -> slow 1 fast 1
1099511627777 -> slow 2 fast 2
turns for 1099511627776: slow=41 fast=1

```

Look at $2^{40} = 1\{, \}099\{, \}511\{, \}627\{, \}776$. The slow version needs 41 loop turns to find its single 1-bit; the fast version needs 1.

If the problem had said $0 \leq x < 2^{31}$ instead, plain numbers are fine and the code is the same without the n suffixes:

The 32-bit version — only when the constraint allows it

JS

```

const countBits32 = (n) => {
  let c = 0;
  while (n !== 0) { n &= n - 1; c++; }
  return c;
};

```

Tested: 0 gives 0, 7 gives 3, 1024 gives 1, 2147483647 gives 31.

Time $O(\text{popcount})$ vs $O(\log x)$ **Space** $O(1)$

Both are 'fast'. The second is faster on sparse numbers.

TRAP

Never mix a `BigInt` and a `Number` in one expression. `1n + 1` throws `TypeError: Cannot mix BigInt and other types`. Convert on purpose with `Number(x)` or `BigInt(x)`, and remember that `Number(x)` on a huge `BigInt` loses precision above $2^{53} - 1$.

Example 15 [Infosys · pattern]

Problem. Compute the n -th Fibonacci number. Why is the plain recursion unusable? **Constraints.** $n \leq 90$. **Target.** $O(n)$. **Edge cases.** $n = 0$; $n = 1$; $n = 90$ (past the exact range of a JavaScript number).

SOLUTION

Approach 1 — Plain recursion ($O(2^n)$ — unusable past $n \approx 40$)

Recomputes the same values again and again

JS

```

const fibSlow = (n) => {
  if (n <= 1) return n;
  return fibSlow(n - 1) + fibSlow(n - 2);
};

```

Approach 2 — Remember each answer (memo) ($O(n)$ — optimal)Each n is computed once

JS

```
const fibMemo = (n, memo = new Map()) => {
  if (n <= 1) return n;
  if (memo.has(n)) return memo.get(n);
  const v = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
  memo.set(n, v);
  return v;
};
```

Both were run with a call counter:

Measured output

TEXT

```
n=10 fib=55 same=true slowCalls=177 memoCalls=19
n=20 fib=6765 same=true slowCalls=21891 memoCalls=39
n=30 fib=832040 same=true slowCalls=2692537 memoCalls=59
```

The slow version's call count roughly **triples** every 5 steps of n . The memo version's call count is $2n - 1$: a straight line. At $n = 50$ the slow version needs about 40 billion calls; the memo version needs 99.

Approach 3 — A loop, with BigInt for the big end ($O(n)$, no stack at all)

Exact for every n up to 90

JS

```
const fibBig = (n) => {
  let a = 0n, b = 1n;
  for (let i = 0; i < n; i++) [a, b] = [b, a + b];
  return a;
}; // a BigInt
```

Measured output — where the doubles start lying

TEXT

```
n=0 Number=0 BigInt=0 exact? true
n=1 Number=1 BigInt=1 exact? true
n=78 Number=8944394323791464 BigInt=8944394323791464 exact? true
n=79 Number=14472334024676220 BigInt=14472334024676221 exact? false
n=90 Number=2880067194370816000 BigInt=2880067194370816120 exact? false
```

Time $O(2^n)$ vs $O(n)$ **Space** $O(n)$ for the memo and the call stack

TRAP

$\text{fib}(90) = 2\{, \}880\{, \}067\{, \}194\{, \}370\{, \}816\{, \}120$ is larger than `Number.MAX_SAFE_INTEGER`. A plain JavaScript number gives 2880067194370816000 — the last three digits are gone. The first n that is already wrong is $n = 79$. Any Fibonacci past $n = 78$ needs BigInt.

TRAP

`fibSlow(n)` recurses n deep, which is fine at $n = 90$. But Node's call stack holds only about 10^4 frames. A recursion that goes 10^5 deep — a DFS on a long path graph, for example — throws `RangeError: Maximum call stack size exceeded`. When the depth can reach 10^5 , rewrite the recursion with an explicit stack array.

Example 16 [TCS NQT · pattern]

Problem. You will do $n = 200\{, \}000$ insertions and n lookups by key. JavaScript gives you three choices: a plain object, a Map, or a sorted array searched with `lowerBound`. Pick one and justify it with a number. **Constraints.** keys are integers, no ordering needed in the output. **Target.** $O(n)$ expected. **Edge cases.** keys all equal; keys inserted in increasing order.

SOLUTION

A **plain object** is a hash table too, but every key is first converted to a string, and V8 moves an object into a slower “dictionary” layout once the keys stop being small dense integers. A **Map** hashes the value itself and keeps insertion order. A **sorted array** plus binary search is $O(\log n)$ per lookup, and it is the only one of the three that can also answer “the smallest key greater than x ” — JavaScript has no ordered map.

The same work, three containers

JS

```
const { lowerBound } = require('./toolkit.js'); // JS Toolkit appendix

const obj = Object.create(null); // plain object
for (const k of keys) obj[k] = k;
for (const k of keys) s1 += obj[k];

const m = new Map(); // Map
for (const k of keys) m.set(k, k);
for (const k of keys) s2 += m.get(k);

const sorted = [...keys].sort((a, b) => a - b); // sorted array
for (const k of keys) s3 += sorted[lowerBound(sorted, k)];
```

Measured with $n = 200\{, \}000$, third run (the first run pays for the JIT warming up):

Measured output

TEXT

```
run 1: equal=true object=140ms Map=53ms sorted+binary=61ms
run 2: equal=true object=147ms Map=31ms sorted+binary=49ms
run 3: equal=true object=144ms Map=29ms sorted+binary=53ms
```

The Map wins, and the plain object is about 5 times slower. The sorted array costs a little more than the Map and buys you ordered queries:

What only the sorted array can answer

TEXT

```
smallest key > 500000 = 500001
```

Answer: Map, unless you need the keys in sorted order — then a sorted array plus `lowerBound` / `upperBound`

TRAP

A plain object stringifies keys. Tested in Node: after `o[1] = 'num'` and `o['1'] = 'str'`, `o[1]` is 'str' and `Object.keys(o)` is `["1"]` — the two keys collided. The same code with a Map keeps both and reports size 2. Use a Map whenever keys are numbers, and always when they can be negative or fractional.

Example 17 [Wipro · pattern]

Problem. An interviewer writes $T(n) = 3n^2 + 500n + 20000$ and asks for the Big-O. Then asks: at which n does the n^2 term actually become the biggest one?

SOLUTION

Big-O keeps only the fastest-growing term and drops constants, so $T(n) = O(n^2)$.

Now the second half. We want $3n^2 > 500n$, that is $3n > 500$, that is $n > 166.7$, so from $n = 167$ upward the square term beats the linear term. And $3n^2 > 20000$ needs $n^2 > 6666.7$, so $n \geq 82$.

So for n below about 80, the constant 20,000 dominates and the function looks flat. For n above about 170, it is a parabola. Big-O describes the **large** n behaviour, which is the only behaviour the hidden tests care about.

Answer: $O(n^2)$; the n^2 term takes over from about $n = 167$

| This is why Big-O ignores constants: for the input sizes a judge uses (10^5 and up), the shape always wins over the constant.

Tier 2 — Let the limits choose the algorithm

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 18 [Grab · pattern]

Problem. A fare log stores the fare of each ride of the day, in order. The dashboard asks q questions of the form “total fare of rides l through r ”. **Constraints.** $n \leq 2 \times 10^5$ rides, $q \leq 2 \times 10^5$ questions, each fare up to 10^4 and possibly negative (refunds). **Target.** $O(n + q)$. **Edge cases.** $l = r$ (one ride); $l = 0$ and $r = n - 1$ (the whole day).

SOLUTION

Approach 1 — Add up the range for every question ($O(n \cdot q) = 4 \cdot 10^{10}$, far too slow)

One question, one scan

JS

```
const sumSlow = (a, l, r) => {
  let s = 0;
  for (let i = l; i <= r; i++) s += a[i];
  return s;
};
```

Time $O(n)$ per question, $O(n \cdot q)$ in total **Space** $O(1)$

Approach 2 — Precompute running totals once ($O(n)$ build + $O(1)$ per question — optimal)

Let $p[i]$ be the total of the first i fares, with $p[0] = 0$. Then the total of $l..r$ is $p[r + 1] - p[l]$, because the part before l cancels.

Prefix sums

JS

```
const buildPrefix = (a) => {
  const p = new Array(a.length + 1).fill(0);
  for (let i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
  return (l, r) => p[r + 1] - p[l]; // a closure over p, no class needed
};
```

Time $O(n)$ build, $O(1)$ per query **Space** $O(n)$

Measured with $n = q = 200\{, \}000$ and wide ranges:

Measured output

TEXT

```

first 200 queries agree? yes
slow: 200 queries in 32 ms
prefix: all 200000 queries in 21 ms
one ride l=r=0 -> -10000 whole day -> -97975
single [5]: query(0,0) = 5
negatives [-3,-4]: query(0,1) = -7
worst total 2e5 * 1e4 = 2000000000 safe in a JS number? true

```

The slow version needed 32 ms for **200** questions. Scale that to 200,000 questions and it is about 32 seconds. The prefix version answered all 200,000 in 21 ms.

This is one of the chapter's signature problems, so here it is in Python too:

Python version of the optimal

PYTHON

```

from itertools import accumulate

def build_prefix(a):
    p = [0] + list(accumulate(a))
    return lambda l, r: p[r + 1] - p[l]

```

Run on [5] and [-3, -4], Python prints 5 and -7 — the same answers as the JavaScript.

The idea that unlocked it. The questions overlap. The slow version re-adds the same numbers for every question. Pay $O(n)$ once, then each answer is one subtraction. Chapter 2 is built entirely on this idea.

In C++ or Java this is where you would have to remember to declare `p` as a 64-bit type, because 2×10^5 fares of 10^4 each reaches 2×10^9 and a 32-bit integer stops at about 2.1×10^9 . JavaScript has no 32-bit integer type, so this particular trap is gone — the measured line above confirms 2×10^9 is safe. The JavaScript limit is $2^{53} - 1 \approx 9 \times 10^{15}$, which is five million times higher. Check **that** number instead.

Example 19 [Shopee · pattern]

Problem. From two million order values, report the 10 largest. **Constraints.** $n \leq 2 \times 10^6$, $k \leq 100$. **Target.** better than a full sort. **Edge cases.** $k = 1$; $k = n$; duplicate values among the top k .

SOLUTION

Approach 1 — Sort everything, take the first k ($O(n \log n)$ — works, but does too much)

Full sort

JS

```
const kthLargestSort = (a, k) => [...a].sort((x, y) => y - x)[k - 1];
```

Approach 2 — Keep a min-heap of size k ($O(n \log k)$ — optimal for small k)

Hold the best k seen so far in a **min**-heap. The smallest of the k sits on top. Each new value is pushed; if the heap grows past k , pop the smallest. Only k items are ever stored.

JavaScript has no built-in priority queue. Use the tested MinHeap from the **JS Toolkit** appendix — do not write your own here.

Min-heap of size k

JS

```

const { MinHeap } = require('./toolkit.js'); // JS has no priority queue

const kthLargestHeap = (a, k) => {
    const h = new MinHeap(); // default comparator (a, b) => a - b
}

```

```

for (const x of a) {
  h.push(x);
  if (h.size > k) h.pop();           // drop the smallest of the k+1
}
return h.peek();
};

```

Time $O(n \log k)$ **Space** $O(k)$

$\log_2(10) \approx 3.3$ against $\log_2(2 \cdot 10^6) \approx 21$. About 6 times less work per element.

Measured:

Measured output

TEXT

```

k=1 sort=9 heap=9
k=2 sort=9 heap=9
k=3 sort=5 heap=5
k=4 sort=3 heap=3
k=5 sort=1 heap=1
single: -7 -7
all equal [4,4,4] k=3: 4 4
big: sort=999995999 heap=999995999 sortTime=1617 ms heapTime=289 ms
bare .sort() on [5,1,9,3,9] gives [1,3,5,9,9] and on [10,9,1] [1,10,9]

```

The first five lines use the array $[5, 1, 9, 3, 9]$. Notice $k = 1$ and $k = 2$ both give 9: duplicates each take their own place in the ranking, which is the usual rule. State it before you code.

TRAP

Read the last measured line again. A bare `.sort()` on $[5, 1, 9, 3, 9]$ **looks** correct, because every value is a single digit and text order matches number order there. On $[10, 9, 1]$ the same call gives $[1, 10, 9]$. This is exactly how the bug survives your own small tests and then fails the hidden ones. `sort((x, y) => y - x)` for descending, `sort((x, y) => x - y)` for ascending, always.

The idea. A full sort orders all n items when the question only asks about k of them. Sorting is the default, not the answer. Ask what the question actually needs.

Example 20 [Agoda · pattern]

Problem. Sort two million review scores. Every score is an integer from 0 to 100. **Constraints.** $n \leq 2 \times 10^6$, values in $[0, 100]$. **Target.** beat $O(n \log n)$. **Edge cases.** empty array; all values equal.

SOLUTION

Approach 1 — `a.sort((x, y) => x - y)` ($O(n \log n)$ — fine, but ignores a gift in the constraints)

Approach 2 — Counting sort ($O(n + k)$ where $k = 101$ — optimal)

The value range is tiny. Count how many of each value there are, then write them out in order. No comparisons at all.

Counting sort

JS

```

const countingSort = (a, maxVal) => {
  const cnt = new Array(maxVal + 1).fill(0);
  for (const x of a) cnt[x]++;
  const out = [];
  for (let v = 0; v <= maxVal; v++)

```

```

    for (let c = 0; c < cnt[v]; c++) out.push(v);
    return out;
};

```

Time $O(n + k)$ **Space** $O(n + k)$
 k is the value range, not the array length.

Measured on two million scores:

Measured output

TEXT

```

same answer? yes
sort((a,b)=>a-b) : 585 ms
counting sort    : 154 ms
empty: 0
all equal [2,2,2]: [2,2,2]
bare .sort() on 3 values [100,9,20] -> [100,20,9]

```

Almost four times faster, and the results are identical.

The idea. “ $O(n \log n)$ is the best possible for sorting” is only true for sorts that **compare** elements. Counting sort never compares; it uses the value as an array index. When the value range is small, use it.

TRAP

Counting sort needs an array of size `maxVal + 1`. If the scores went up to 10^9 , this approach would ask for 8 GB. Check the **value range**, not just n .

TRAP

The last measured line is the reason this chapter repeats itself about `.sort()`. Scores are two- and three-digit numbers, and `[100, 9, 20].sort()` returns `[100, 20, 9]` — the exact reverse of the right answer, with no error message.

`new Array(k).fill(0)` is required. `new Array(k)` alone makes a **sparse** array whose slots are all undefined, and `undefined++` gives NaN. For a large fixed-size counter, `new Int32Array(k)` is both zero-filled and half the memory.

Example 21 [Sea/Shopee · pattern]

Problem. Given n driver ratings, count the pairs (i, j) with $i < j$ whose ratings add up to exactly `target`.

Constraints. $n \leq 2 \times 10^5$; ratings can be negative; the count can exceed 2×10^{10} . **Target.** $O(n)$ expected. **Edge cases.** empty array; all values equal (so every pair counts); a value of 10^9 twice.

SOLUTION

Approach 1 — Every pair ($O(n^2)$ — $2 \cdot 10^{10}$ steps, too slow)

Brute force

JS

```

const countPairsBrute = (a, target) => {
  let cnt = 0;
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] + a[j] === target) cnt++;
  return cnt;
};

```


Approach 2 — Sort, then two pointers ($O(n \log n)$) — passes, but the duplicate handling is fiddly)

Approach 3 — One pass with a frequency Map ($O(n)$ expected — optimal)

Walk left to right. Before adding x to the Map, ask how many earlier values equal $\text{target} - x$. Each of those earlier values pairs with this x exactly once, and the condition $i < j$ is satisfied automatically because we only look **backwards**.

Count as you go

JS

```
const countPairsMap = (a, target) => {
  const freq = new Map();
  let cnt = 0;
  for (const x of a) {
    cnt += freq.get(target - x) ?? 0; // earlier partners
    freq.set(x, (freq.get(x) ?? 0) + 1); // now x is available as a partner
  }
  return cnt;
};
```

Time $O(n)$ expected **Space** $O(n)$

Both versions were run on the same tests:

Measured output

TEXT

```
[] target=5 brute=0 map=0
[5] target=5 brute=0 map=0
[1,4,2,3] target=5 brute=2 map=2
[2,2,2,2] target=4 brute=6 map=6
[-3,3,0,0] target=0 brute=2 map=2
[1,1,2,2,3] target=4 brute=3 map=3
[1000000000,1000000000] target=2000000000 brute=1 map=1
worst count C(2e5,2) = 19999900000 safe? true
plain object key 1 is really ["1"]
```

Check $[2, 2, 2, 2]$ by hand: the pairs are $(0, 1)$, $(0, 2)$, $(0, 3)$, $(1, 2)$, $(1, 3)$, $(2, 3)$, which is $\binom{4}{2} = 6$. The program agrees.

This is one of the chapter's signature problems, so here it is in Python too:

Python version of the optimal

PYTHON

```
from collections import Counter

def count_pairs(a, target):
    freq = Counter()
    cnt = 0
    for x in a:
        cnt += freq[target - x] # Counter gives 0 for a missing key
        freq[x] += 1
    return cnt
```

Run on the same inputs, Python prints $0\ 0\ 2\ 6\ 2\ 1$ for $[], [5], [1, 4, 2, 3], [2, 2, 2, 2], [-3, 3, 0, 0]$ and $[10^9, 10^9]$ — the same answers as the JavaScript.

The idea. “For each element, does a partner exist?” is a **search**. Replace the search with a lookup and the inner loop disappears. Counting backwards also solves the $i < j$ problem without any extra work.

The two numeric traps that would bite in C++ are both gone here. The count reaches $\binom{2 \times 10^5}{2} = 19\{, \}999\{, \}900\{, \}000$, and `Number.isSafeInteger` confirms that is exact in JavaScript. `a[i] + a[j]` with two values near 10^9 gives 2×10^9 , which is also exact. JavaScript only starts lying above 9×10^{15} .

TRAP

Use a `Map`, not a plain object. Ratings can be negative, and the measured line `plain object key 1 is really ["1"]` shows what an object does to a numeric key. With an object, the number `-3` and the string `"-3"` become the same entry; a `Map` keeps them apart.

Example 22 [DBS · pattern]

Problem. Why is Euclid’s algorithm for GCD fast? Give the step count for the worst input you can construct with two numbers below 10^9 . **Constraints.** $1 \leq a, b \leq 10^9$. **Edge cases.** $b = 0$; $a = b$; two consecutive Fibonacci numbers.

SOLUTION

Euclid with a step counter

JS

```
let gcdSteps = 0;
const gcd = (a, b) => {
  gcdSteps = 0;
  while (b !== 0) { const r = a % b; a = b; b = r; gcdSteps++; }
  return a;
};
```

Measured:

Measured output

TEXT

```
gcd(12,18)=6 steps=3
gcd(100,75)=25 steps=2
gcd(1000000007,998244353)=1 steps=9
gcd(13,8)=1 steps=5
gcd(5,0)=5 steps=0
gcd(701408733,433494437)=1 steps=42
```

Two nine-digit numbers took only 9 steps.

Why. After two steps the larger number is at most half of what it was. Proof sketch: if $a \geq b$, then $a \bmod b < b$, and also $a \bmod b < \frac{a}{2}$ (if $b \leq \frac{a}{2}$ this is clear; if $b > \frac{a}{2}$ then $a \bmod b = a - b < \frac{a}{2}$). So the pair shrinks by at least half every two steps, giving $O(\log \min(a, b))$ steps.

The **worst** inputs are consecutive Fibonacci numbers: `gcd(13, 8)` needed 5 steps while the much larger `gcd(109+7, 998244353)` needed only 9. The largest Fibonacci pair under 10^9 is (701408733, 433494437), and the last measured line shows it takes 42 steps. Still tiny.

`a % b` in JavaScript is exact for any values below 2^{53} , so no cast is needed. It does **not** behave like C++ on negative inputs in the way beginners expect, though: `-7 % 3` is `-1` in JavaScript. For a GCD, call it on absolute values.

Answer: $O(\log \min(a, b))$ — never more than about 45 steps for 32-bit inputs

Example 23 [GIC · pattern]

Problem. `arr.push` sometimes has to copy the whole array into a bigger block of memory. So is filling an array with n values $O(n)$ or $O(n^2)$? **Constraints.** $n \leq 10^6$. **Target.** name the **amortised** cost of one push.

SOLUTION

JavaScript does not let you read an array's capacity, so simulate the two growth policies and count the copies:

Doubling against growing by one

JS

```
const doublingCopies = (n) => {
  let cap = 0, copies = 0;
  for (let i = 1; i <= n; i++)
    if (i > cap) { copies += cap; cap = cap === 0 ? 1 : cap * 2; }
  return copies;
};

const growByOneCopies = (n) => {
  let copies = 0;
  for (let i = 1; i <= n; i++) copies += i - 1; // copy everything, every push
  return copies;
};
```

Measured copy counts

TEXT

```
n=1 doubling copies=0 grow-by-one copies=0
n=10 doubling copies=15 grow-by-one copies=45
n=1000 doubling copies=1023 grow-by-one copies=499500
n=100000 doubling copies=131071 grow-by-one copies=4999950000
```

The capacity **doubles**, so a copy happens at push 1, 2, 3, 5, 9, 17, 33, ..., and the copies cost $1 + 2 + 4 + 8 + \dots + n$, which is less than $2n$ in total. Doubling: 131,071 copies for 100,000 pushes — about 1.3 per push. Growing by one: 5 billion copies. This is what **amortised $O(1)$** means: one single call may be slow, but any run of n calls costs $O(n)$ altogether.

V8 grows a JavaScript array the doubling way, and the timings show it:

Measured output

TEXT

```
push 1000000 times : 48 ms (length 1000000)
unshift 100000 times: 617 ms (length 100000)
```

One million push calls in 48 ms. One **tenth** as many unshift calls took thirteen times longer, because unshift has no doubling trick available: every existing element really does move, every single time.

Answer: Amortised $O(1)$ per push, $O(n)$ for all n pushes. unshift is $O(n)$ per call and $O(n^2)$ for n calls.

Amortised is not the same as average-case. Average-case talks about lucky and unlucky **inputs**. Amortised talks about a **sequence** of operations and is a worst-case promise about the whole sequence.

TRAP

`arr.pop()` is $O(1)$ but `arr.shift()` is $O(n)$ — it is unshift in reverse. A queue built from push and shift is $O(n^2)$ on n items. Use the toolkit Deque, which keeps a head index instead of moving anything. See the **JS Toolkit** appendix and Chapter 9.

Example 24 [SCB · pattern]

Problem. How many steps does this run? It looks like two nested loops over n .

The inner loop steps by i

JS

```
const harmonicSteps = (n) => {
  let steps = 0;
  for (let i = 1; i <= n; i++)
    for (let j = i; j <= n; j += i)
      steps++;
  return steps;
};
```

Constraints. $n \leq 10^6$.

SOLUTION

For a fixed i , the inner loop hits $i, 2i, 3i, \dots$ up to n , so it runs $\frac{n}{i}$ times (rounded down).

Total = $\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n} = n(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n})$.

The bracket is the **harmonic sum**, and it is close to $\ln n$. So the total is about $n \ln n$.

Measured:

Measured output

TEXT

```
n=10  steps=27  n ln n = 23
n=100 steps=482 n ln n = 461
n=1000 steps=7069 n ln n = 6908
n=100000 steps=1166750 n ln n = 1151293
```

At $n = 100\{, \}000$ the true count is 1.17 million. A real $O(n^2)$ loop would be 10 billion — nearly ten thousand times more.

Answer: $O(n \log n)$, not $O(n^2)$

▮ This exact shape appears in the Sieve of Eratosthenes. That is why the sieve is fast.

Tier 3 — Insight, then the follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 25 [Amazon · pattern]

Problem. Given n integers and a value `target`, count the triples $i < j < k$ whose sum is strictly less than `target`.

Constraints. $n \leq 3000$; values in $[-10^9, 10^9]$. **Target.** $O(n^2)$. **Edge cases.** $n < 3$ (answer 0); all values equal; negative values.

SOLUTION

Approach 1 — Three nested loops ($O(n^3) = 2.7 \cdot 10^{10}$, too slow)

Brute force

JS

```
const tripletsBrute = (a, target) => {
  let cnt = 0;
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      for (let k = j + 1; k < a.length; k++)
        if (a[i] + a[j] + a[k] < target) cnt++;
}
```

```
    return cnt;
};
```

Approach 2 — Sort, fix one, two pointers for the rest $O(n^2)$ — optimal here

Sort the array. Fix the smallest member $a[i]$. Now we need pairs (l, r) with $l < r$ and $a[l] + a[r] < \text{target} - a[i]$. Put l just after i and r at the end.

- If $a[i] + a[l] + a[r] < \text{target}$, then **every** index between l and r also works as the partner of l , because the array is sorted and those values are $\leq a[r]$. That is $r - l$ triples at once. Then move l right.
- Otherwise the sum is too big, so move r left.

Sort + two pointers

JS

```
const tripletsTwoPointer = (a, target) => {
  const v = [...a].sort((x, y) => x - y); // never a bare .sort()
  let cnt = 0;
  for (let i = 0; i < v.length - 2; i++) {
    let l = i + 1, r = v.length - 1;
    while (l < r) {
      if (v[i] + v[l] + v[r] < target) {
        cnt += r - l; // every k in (l, r] works too
        l++;
      } else r--;
    }
  }
  return cnt;
};
```

Time $O(n^2)$ **Space** $O(n)$ for the sorted copy
 $n = 3000$ gives $9 \cdot 10^6$ steps. Instant.

Both versions were run together and always matched:

Measured output

TEXT

```
target=5  []  brute=0 twoPtr=0
target=10 [1,2] brute=0 twoPtr=0
target=7  [1,2,3] brute=1 twoPtr=1
target=6  [1,2,3] brute=0 twoPtr=0
target=2  [-1,0,1,2] brute=2 twoPtr=2
target=1  [0,0,0,0] brute=4 twoPtr=4
target=100 [5,5,5] brute=1 twoPtr=1
random n=400: brute=5534457 twoPtr=5534457
```

Note $[1, 2, 3]$ with target 6: the only triple sums to 6, and 6 is not **less than** 6, so the answer is 0. Strict versus non-strict is a real trap.

The idea. Sorting turns “count the partners” into “count a range of indices”. The `cnt += r - l` line is where an entire loop disappears.

TRAP

`[...a].sort(...)` makes a copy so the caller’s array is not reordered. Remember that the spread is a **shallow** copy: it is right here, where the elements are numbers, but if the elements were themselves arrays (intervals, points, pairs) you would need `a.map(row => [...row])` to copy them safely.

TRAP

Values here reach 10^9 , so a triple sum reaches 3×10^9 . In C++ that overflows a 32-bit `int` and needs a cast. In JavaScript it does not: numbers stay exact to 9×10^{15} . Do not carry the C++ habit of casting; do carry the habit of **checking the number**, because the JavaScript limit is real too, just much higher.

The follow-up the interviewer asks next

“**Now count triples with sum exactly equal to target.**” The same $O(n^2)$ shape works, but when you hit an exact match you must count how many copies of `a[l]` and `a[r]` there are and multiply — and handle `a[l] == a[r]` separately with $\frac{m(m-1)}{2}$. That is the same duplicate trap as the pair version in Example 21.

“**Now $n \leq 10^5$.**” $O(n^2)$ is 10^{10} and dies. There is no known $O(n \log n)$ for the general triple-sum count, so the interviewer is testing whether you will say so. The honest answer: “with those limits, the problem must have extra structure — small value range, or only counting, or offline queries. Which is it?”

Example 26 [Google · pattern]

Problem. You are given n item weights and a truck capacity limit. Find the largest total weight that fits. **Constraints.** $n \leq 36$; weights up to 10^6 ; limit up to 10^{12} . **Target.** around $2^{n/2}$. **Edge cases.** $n = 0$ (answer 0); all items heavier than the limit (answer 0); negative weights are not allowed here, but the code below still handles them.

SOLUTION

The limit $n \leq 36$ is the whole message. n is far too small for a polynomial bound to matter and far too big for 2^n .

Approach 1 — Try every subset ($O(2^n \cdot n)$ — $2^{36} \approx 7 \cdot 10^{10}$, too slow)

Enumerate all subsets with a bitmask

JS

```
const bestSubsetSum = (a, limit) => {
  let best = 0;
  for (let mask = 0; mask < (1 << a.length); mask++) {
    let s = 0;
    for (let i = 0; i < a.length; i++) if (mask & (1 << i)) s += a[i];
    if (s <= limit) best = Math.max(best, s);
  }
  return best;
};
```

Measured: $n = 20$ (that is 1,048,576 masks) took 56 ms. So $n = 36$, which is 65,536 times more masks, would take about an hour.

TRAP

`1 << n` is a **32-bit** shift. At $n = 31$ it goes negative and at $n = 32$ it wraps to 1, so this brute force silently returns nonsense for $n \geq 31$ — another reason it cannot be the answer here. If you ever do need masks past 31 bits, the loop counter must be a `BigInt`, or you split the problem, which is exactly what comes next.

Approach 2 — Meet in the middle ($O(2^{n/2} \cdot n/2 \cdot \log)$ — optimal)

Split the items into two halves of 18. Build **all** subset sums of each half: $2^{18} = 262,144$ sums per half, which is nothing. Sort the right half's sums. For every left sum x , binary-search the largest right sum that is $\leq \text{limit} - x$. JavaScript has no built-in binary-search helper, so use the tested `upperBound` from the **JS Toolkit** appendix. `upperBound(rs, x)` returns the index of the first value **greater than** x , so `upperBound(rs, x) - 1` is the largest value that still fits.

Meet in the middle

JS

```

const { upperBound } = require('./toolkit.js'); // JS Toolkit appendix

const allSums = (a) => {
  const out = [];
  for (let mask = 0; mask < (1 << a.length); mask++) {
    let t = 0;
    for (let i = 0; i < a.length; i++) if (mask & (1 << i)) t += a[i];
    out.push(t);
  }
  return out;
};

const bestSubsetMeet = (a, limit) => {
  const half = a.length >> 1;
  const ls = allSums(a.slice(0, half));
  const rs = allSums(a.slice(half)).sort((x, y) => x - y);
  let best = -Infinity;
  for (const x of ls) {
    const idx = upperBound(rs, limit - x) - 1; // largest right sum that still
    fits
    if (idx >= 0) best = Math.max(best, x + rs[idx]);
  }
  return best;
};

```

Time $O(2^{(n/2)} \cdot (n/2 + \log(2^{(n/2)})))$ **Space** $O(2^{(n/2)})$

For $n = 36$: about $5 \cdot 10^5$ sums instead of $7 \cdot 10^{10}$ subsets.

Measured against the brute force on small inputs, and then on a real $n = 36$ with weights 250{, }000, 251{, }000, ..., 285{, }000 and a limit of 9{, }000{, }000:

Measured output

TEXT

```

limit=10 [] brute=0 meet=0
limit=10 [7] brute=7 meet=7
limit=10 [3,5,8] brute=8 meet=8
limit=10 [-4,6,6] brute=8 meet=8
limit=10 [1,2,3,4,5,6,7,8] brute=10 meet=10
n=36 meet-in-the-middle answer=8877000 time=98 ms
      (524288 subsets built, not 68719476736)
n=20 brute force took 56 ms

```

Ninety-eight milliseconds instead of an hour.

TRAP

Each half is 18 items, so $1 \ll 18$ is safe. Splitting is what keeps the mask under 31 bits. Also note the `.sort((x, y) => x - y)` on `rs`: `upperBound` does a binary search and gives wrong answers on a lexicographically sorted array. A bare `.sort()` here would break the whole algorithm silently.

The idea. $2^{36} = 2^{18} \times 2^{18}$. Doing 2^{18} work **twice** is nothing; doing 2^{18} work 2^{18} **times** is impossible. Splitting the exponent in half is the whole trick.

The follow-up

“What if $n \leq 100$ but every weight is at most 1000?” Then the total weight is at most 10^5 and you should switch to a knapsack DP: $O(n \times \text{sum}) = 10^7$. Notice how the **second** constraint changed the answer completely. Always read every constraint line.

Example 27 [Microsoft · pattern]

Problem. Your “ $O(n)$, I use a lookup” solution passed all your own tests, then failed the hidden tests with Time Limit Exceeded. Here is the line you wrote. Find the cost. **Constraints.** $n = 2 \times 10^5$ values, checking each one against the ones already seen.

The line that looks like a lookup

JS

```
const seen = [];  
for (const x of values) {  
  if (seen.includes(x)) duplicates++; // <-- this is a full scan  
  seen.push(x);  
}
```

SOLUTION

`seen.includes(x)` is not a hash lookup. It is a linear scan of the array, $O(n)$ every time. Inside a loop over n values that makes the whole thing $O(n^2)$. `indexOf`, `lastIndexOf`, `find` and `filter` all have the same shape. A Set really is a hash lookup, $O(1)$ expected.

The fix is one word

JS

```
const seen = new Set();  
for (const x of values) {  
  if (seen.has(x)) duplicates++; // O(1) expected  
  seen.add(x);  
}
```

Both were measured at three sizes so you can watch the shape, not just one number:

Measured output

TEXT

```
n = 50000  
Set.has      : 5 ms  
array.includes : 1020 ms  
array.indexOf : 1011 ms  
n = 100000  
Set.has      : 8 ms  
array.includes : 4056 ms  
array.indexOf : 4069 ms  
n = 200000  
Set.has      : 19 ms  
array.includes : 16492 ms  
array.indexOf : 16710 ms
```

Nineteen milliseconds against sixteen and a half **seconds**: a factor of 870 at the real constraint. And look at the doubling. Each time n doubles, Set roughly doubles ($5 \rightarrow 8 \rightarrow 19$) but includes roughly **quadruples** ($1020 \rightarrow 4056 \rightarrow 16492$). Quadrupling on a doubled input is the fingerprint of $O(n^2)$, and you can read it off two timings without any theory at all.

This is why your own tests passed. At $n = 1000$ the slow version needs about 0.4 ms. You will never notice.

Fixes, in the order you should reach for them.

1. Set for membership, Map for key $\rightarrow$ value. This is almost always the answer.

2. If the values must stay ordered, sort once and use two pointers or `lowerBound` from the **JS Toolkit** appendix: $O(n \log n)$ total, and no hashing at all.
3. If you truly need an array, keep a parallel `Set` beside it for the membership test.

The follow-up

“Give me the worst case of a `Set` lookup, the average case, and which one Big-O usually reports.” Worst $O(n)$ per operation, expected $O(1)$. Big-O is usually quoted as the worst case, but for hash tables everyone quotes the expected cost — so say which one you mean. In an interview, the sentence that scores is: “ $O(1)$ **expected**, $O(n)$ worst case if the hash degrades.”

“Is a JavaScript `Set` attackable the way a C++ `unordered_set` is?” Much less easily. A C++ hash map with the default integer hash puts key k in bucket $k \bmod B$, so a test setter who knows B can send keys that all collide. V8 mixes the bits of a number key and seeds string hashing randomly per process, so a fixed key set cannot be built in advance. That makes the failure mode above — a linear scan wearing the costume of a lookup — far more likely than a hash attack. Still say “expected”. You are not promising $O(1)$ worst case.

Example 28 [Goldman Sachs · pattern]

Problem. You must answer q range-sum questions on an array of n numbers. A colleague suggests precomputing the answer to **every** possible (l, r) pair so each query is a single array read. For $n = 10^5$, is that a good idea?
Constraints. $n \leq 10^5$, $q \leq 10^5$, memory limit 256 MB.

SOLUTION

Count the memory first, before writing any code.

The number of pairs with $l \leq r$ is $\frac{n(n+1)}{2}$. For $n = 10^5$: $\frac{10^5 \times (10^5 + 1)}{2} \approx 5 \times 10^9$ entries.

Each entry is a JavaScript number, which is a double, 8 bytes. That is 4×10^{10} bytes = 40{,}000 MB = 40 GB. The limit is 256 MB. It is off by a factor of 156. It is also past the hard ceiling: a JavaScript array cannot hold more than $2^{32} - 1 \approx 4.3 \times 10^9$ elements at all, and 5×10^9 is over that.

And the **build** would take 5×10^9 steps, which is about 50 seconds by itself.

The prefix-sum table from Example 18 stores $n + 1$ numbers, that is 0.8 MB, builds in 10^5 steps, and answers each query with one subtraction.

Plan	Build time	Query time	Memory
Precompute all pairs	5×10^9	$O(1)$	40 GB
Prefix sums	10^5	$O(1)$	0.8 MB
Nothing precomputed	0	$O(n)$	0

Answer: No. It uses 40 GB and takes 50 seconds to build, to buy nothing over prefix sums.

The follow-up

“Now the array can change: some queries update a single value.” Prefix sums break, because one update ruins every prefix after it ($O(n)$ per update). The answer is a Fenwick tree or a segment tree: $O(\log n)$ per update **and** per query. Say the phrase “the array is no longer static, so I need a structure that supports updates”.

“Now it is a stream — the numbers arrive one at a time and you cannot store them all.” Then you cannot answer arbitrary (l, r) at all. You can keep a running total, and answer “sum of everything so far” in $O(1)$ and $O(1)$ memory. Being clear about what becomes **impossible** is part of the answer.

Example 29 [D. E. Shaw · pattern]

Problem. Here is a function. Give its time complexity in terms of n , and justify it.

What is the cost?

JS

```
const mystery = (n) => {
  let c = 0;
  for (let i = 1; i <= n; i++)
    for (let j = i; j <= n; j += i)
      c++;
  for (let k = 1; k < n; k *= 2)
    for (let m = 0; m < n; m++)
      c++;
  return c;
};
```

SOLUTION

Two blocks, side by side, so we **add** their costs.

Block 1. This is Example 24. The inner loop runs $\frac{n}{i}$ times for each i , giving $n(1 + \frac{1}{2} + \dots + \frac{1}{n}) \approx n \ln n$. Measured at $n = 10^5$: 1,166,750 steps.

Block 2. The outer loop doubles k , so it runs $\log_2 n$ times. The inner loop runs n times. Multiply: $n \log_2 n$. Measured at $n = 1024$: 10,240 steps.

Total $\approx n \ln n + n \log_2 n$. Both terms are $O(n \log n)$, and adding two $O(n \log n)$ terms is still $O(n \log n)$. Measured totals: $n = 1$ gives 1, $n = 16$ gives 114 ($50 + 64$), $n = 1024$ gives 17,502 ($7262 + 10240$).

Answer: $O(n \log n)$

The follow-up

“Your two blocks use $\ln$ and $\log_2$. Does the base matter in Big-O?” No. $\log_2 n = \ln \frac{n}{\ln 2} \approx 1.44 \ln n$, so the bases differ only by a constant factor, and Big-O drops constant factors. That is why nobody writes the base. But when you are estimating **actual** step counts for a time limit, use $\log_2$ — it is the bigger of the two and keeps your estimate safe.

Example 30 [Adobe · pattern]

Problem. You have 1 second. For each of these, say whether the plan passes, and give the step count.

1. $n = 10^5$, plan is $O(n^2)$.
2. $n = 10^5$, plan is $O(n\sqrt{n})$.
3. $n = 10^6$, plan is $O(n \log n)$.
4. $n = 10^9$, plan is $O(\sqrt{n})$.
5. $n = 25$, plan is $O(2^n)$.
6. $n = 10$, plan is $O(n!)$.

SOLUTION

Use the budget 10^8 steps per second.

1. $(10^5)^2 = 10^{10}$. That is 100 times the budget. **Fails** — about 100 seconds.
2. $10^5 \times \sqrt{10^5} \approx 10^5 \times 316 = 3.2 \times 10^7$. **Passes** comfortably.
3. $10^6 \times \log_2 10^6 \approx 10^6 \times 20 = 2 \times 10^7$. **Passes**.
4. $\sqrt{10^9} \approx 31\{, \}623$. **Passes** with enormous room.
5. $2^{25} = 33\{, \}554\{, \}432 \approx 3.4 \times 10^7$. **Passes**, but only just — if each subset costs another $O(n)$ to evaluate, it becomes 8×10^8 and fails.
6. $10! = 3\{, \}628\{, \}800$. **Passes**. But $12! = 4.8 \times 10^8$ fails, and $13!$ is hopeless. This is why permutation problems always have $n \leq 11$ or so.

The follow-up

“Item 5 passed on the count but you said it might fail. Explain.” Big-O hides the work done **inside** one step. $O(2^n)$ usually means “ 2^n subsets”, and if building or checking each subset costs $O(n)$, the truth is $O(2^n n)$. Always

ask what one “step” contains. In the truck problem (Example 26) the inner `for (int i = 0; i < n; i++)` is exactly that hidden factor.

“Now it is distributed over 100 machines.” Dividing by 100 turns 10^{10} into 10^8 — but only if the work splits cleanly with no shared state. A sum splits cleanly. Sorting does not (you still have to merge). Saying **which** part does not split is the real answer.

Dry run — binary search, one line at a time

We trace `binarySearch` on this array. It is sorted, which is the only thing the algorithm needs.

The array, with indices										TEXT
index :	0	1	2	3	4	5	6	7	8	9
value :	2	5	8	12	16	23	38	56	72	91

Search for 23. Start with `lo = 0, hi = 9`.

step	lo	hi	mid	a[mid]	decision
1	0	9	4	16	$16 < 23$, so go right: $lo = mid + 1 = 5$
2	5	9	7	56	$56 > 23$, so go left: $hi = mid - 1 = 6$
3	5	6	5	23	equal — return index 5, stop

Work through step 1 slowly. $(lo + hi) / 2 = (0 + 9) / 2 = 4.5$, and `Math.floor(4.5) = 4`. Without that `Math.floor` you would read `a[4.5]`, which is undefined, and every comparison against undefined is false. `a[4] = 16`. We want 23, which is bigger, so everything at index 4 and below is useless. That is why `lo` jumps to 5 — half the array is gone in one step.

Search for 40, which is not there.

step	lo	hi	mid	a[mid]	decision
1	0	9	4	16	$16 < 40$: $lo = 5$
2	5	9	7	56	$56 > 40$: $hi = 6$
3	5	6	5	23	$23 < 40$: $lo = 6$
4	6	6	6	38	$38 < 40$: $lo = 7$
—	7	6	—	—	$lo > hi$, loop ends, return -1

This is the real measured trace; the program printed ended with `lo=7 hi=6 -> not found after 4 steps`.

SHORTCUT

When the search fails, `lo` stops at the first index whose value is **greater** than the target, and `hi` stops just left of it. That is the whole idea behind the toolkit’s `lowerBound` and `upperBound`, and it is why a failed search is still useful information. JavaScript ships neither, so the **JS Toolkit** appendix carries both.

Counting the cost. The window is `hi - lo + 1`. It starts at n and at least halves each step: $10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0$. Four steps for ten elements. $\log_2 10 \approx 3.3$, and we round up to 4. For $n = 10^6$ it was measured at 20 steps.

Practice

PRACTICE · TIER 1 target: 25 min

1. How many times does the body run? `for (let i = 0; i < n; i += 3) ...`

2. Give the complexity: `for (let i = n; i > 0; i = Math.floor(i / 2)) ...`
3. Give the complexity of a loop nested inside a halving loop: `for (let i = n; i > 0; i = Math.floor(i/2)) for (let j = 0; j < n; j++) ...`
4. Simplify to Big-O: $T(n) = 7n^3 + 900n^2 + 10^6n + 10^9$.
5. Which is bigger for $n = 10^6$: $n \log_2 n$ or $n\sqrt{n}$? Show the numbers.
6. An algorithm takes 2 seconds for $n = 1000$ and is $O(n^2)$. Estimate its time for $n = 4000$.
7. You need 10^6 operations on an **ordered** collection: insert, delete, and “smallest item bigger than x”. JavaScript has no ordered set. What do you build, and what is the cost per operation?
8. $n = 10^5$ and you have written three nested loops. What is the step count and how far over the 1-second budget are you?
9. Why is `s.slice(i, i + k)` inside a loop dangerous?
10. An array holds 10^7 numbers. How much memory, in MB? How much with `Int32Array`?

PRACTICE · TIER 2 target: 30 min

1. You must answer 10^5 questions “how many values in $[l, r]$ are greater than 50?” on a fixed array of 10^5 numbers. Name a plan that is $O(n + q)$.
2. A solution is $O(n \log^2 n)$. For $n = 2 \times 10^5$, give the step count and say whether it passes in 1 second.
3. Sorting 10^7 numbers, each between 0 and 999. What is the fastest approach, and why does it beat $O(n \log n)$?
4. Explain, with the doubling numbers, why `arr.push(x)` is amortised $O(1)$ but `arr.unshift(x)` is $O(n)$ every time.
5. You have $n = 2 \times 10^5$ items and want the 5 largest. Compare full sort against a heap of size 5, in steps.
6. A function calls itself twice on inputs of size $\frac{n}{2}$ and does $O(n)$ work to combine. What is the total? Name an algorithm shaped like this.

PRACTICE · TIER 3 target: 35 min

1. $n \leq 22$ and you must try every subset, doing $O(n)$ work per subset. Step count? Passes?
2. Prove that any comparison-based sort needs at least about $n \log_2 n$ comparisons. (Hint: count the possible orderings.)
3. You have a $O(n^2)$ solution for $n \leq 5000$ and the interviewer changes the limit to $n \leq 10^6$. What do you say in the next 30 seconds?
4. A colleague’s “hash lookup” solution is timing out. Their line is `if (seen.includes(x))`. Give the real complexity, two fixes, and the new complexity of each.
5. An algorithm does n operations that are each $O(\log n)$, plus one operation that is $O(n^2)$ but runs only when n is a perfect square. Give the worst-case complexity.

ANSWER KEY

Tier 1.

1. $\lceil n/3 \rceil$ times. Each turn adds 3, so the values are 0, 3, 6, ... below n . Still $O(n)$ — the constant $\frac{1}{3}$ is dropped.
2. $O(\log n)$. n is halved each turn, so about $\log_2 n$ turns.
3. $O(n \log n)$. Outer runs $\log_2 n$ times, inner runs n times, and nested loops multiply.
4. $O(n^3)$. Keep the fastest-growing term only. The 10^9 constant is huge but it never grows.
5. $n\sqrt{n}$ is bigger. $n \log_2 n = 10^6 \times 20 = 2 \times 10^7$. $n\sqrt{n} = 10^6 \times 1000 = 10^9$. Fifty times bigger.
6. About 32 seconds. n went up 4 times, and $O(n^2)$ means the time goes up $4^2 = 16$ times. $2 \times 16 = 32$.
7. A sorted array plus binary search, $O(\log n)$ to find and $O(n)$ to insert — so if the inserts are spread through the run, use a balanced tree you write yourself, or batch the work. “Smallest item bigger than x” is `upperBound` from the JS Toolkit appendix. A `Set` or `Map` cannot answer it at all, because neither keeps sorted order — `Map` keeps insertion order, which is not the same thing. This is the “no `TreeMap`” gap, and naming it out loud is most of the mark.

8. **10^{15} steps, ten million times over budget.** $(10^5)^3 = 10^{15}$, and the budget is 10^8 . Three nested loops need $n \leq 500$ or so.
9. **It copies k characters every time, so the loop is $O(nk)$, not $O(n)$.** Use two indices into the original string instead of making copies. `slice`, `substring` and `split` all copy.
10. **76.3 MB as a plain array, 38.1 MB as an `Int32Array`.** Every JavaScript number is a double: $10^7 \times 8 \text{ bytes} = 8 \times 10^7$ bytes, and $8 \times 10^7 / (1024 \times 1024) \approx 76.3$. An `Int32Array` stores 4 bytes per value, so it halves that — worth knowing for the biggest inputs.

Tier 2.

1. **Prefix counts.** Build `c[i+1] = c[i] + (a[i] > 50 ? 1 : 0)` in $O(n)$. Every question is `c[r+1] - c[l]` in $O(1)$. Total $O(n + q)$.
2. **About $2 \times 10^5 \times 18 \times 18 \approx 6.5 \times 10^7$ — passes.** $\log_2(2 \times 10^5) \approx 17.6$, and $\log^2$ means you square it.
3. **Counting sort, $O(n + k)$ with $k = 1000$.** It never compares two values; it uses the value as an index. The $n \log n$ lower bound applies only to comparison sorts. Measured earlier: 154 ms against 585 ms on 2 million values.
4. **Doubling means the copies total $1 + 2 + 4 + \dots + n < 2n$ over all n pushes, so under 2 moves per push.** `unshift` has no such trick: every existing element must shift right, every single time, giving $O(n)$ per call and $O(n^2)$ for n calls. Measured earlier: one million `push` calls in 48 ms, one hundred thousand `unshift` calls in 617 ms.
5. **Sort: $2 \times 10^5 \times 17.6 \approx 3.5 \times 10^6$. Heap of size 5: $2 \times 10^5 \times \log_2 5 \approx 2 \times 10^5 \times 2.3 = 4.6 \times 10^5$.** The heap does about 7 times less work and uses $O(5)$ memory instead of $O(n)$.
6. **$O(n \log n)$ — this is merge sort.** Each level of the recursion does $O(n)$ combine work, and there are $\log_2 n$ levels. Measured in Node: merge sort on $n = 100\{, \}000$ used 1,508,260 comparisons, and $n \log_2 n = 1\{, \}660\{, \}964$.

Tier 3.

1. **$2^{22} \times 22 \approx 9.2 \times 10^7$ — passes, but with almost nothing to spare.** If each subset needed $O(n \log n)$ work it would fail. Try to make the per-subset work $O(1)$ by building sums as you extend the mask.
2. **There are $n!$ possible orderings, and each comparison has 2 outcomes, so a decision tree of depth d can tell apart at most 2^d orderings. You need $2^d \geq n!$, so $d \geq \log_2(n!)$, and $\log_2(n!) \approx n \log_2 n - 1.44n$.** That is the $n \log n$ lower bound.
3. **$O(n^2)$ is 10^{12} at that limit, about 3 hours, so it is out. I need $O(n)$ or $O(n \log n)$.** The n^2 came from the inner loop that searches for a partner — I can replace it with a hash map, a sorted array with two pointers, or a prefix sum, depending on what the inner loop is doing.” Name the inner loop’s job. That is what scores.
4. **`includes` is a linear scan, so the loop is $O(n^2)$ — measured at 16.5 seconds for $n = 2 \times 10^5$.** (a) Replace the array with a `Set` and `includes` with `has`: $O(n)$ expected, measured at 19 ms. (b) Sort once and use two pointers or `lowerBound` from the JS Toolkit appendix: $O(n \log n)$ total, no hashing at all. Say “expected” for the first one — a `Set` promises $O(1)$ on average, not in the worst case.
5. **$O(n^2)$.** Worst case means worst **input**. The problem setter will choose $n = 10^6$, which is 1000^2 , a perfect square. “It rarely happens” is an average-case argument, and worst-case Big-O does not accept it.

ONE-PAGE REVISION CARD

The counting rules. Side by side $\rightarrow$ add. Nested $\rightarrow$ multiply. Drop constants. Keep only the fastest-growing term. Halving or doubling $\rightarrow \log_2 n$.

The growth ladder. $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

The budget. About 10^8 simple operations per second. Measured in Node: 10^8 additions took 146 ms; add a factor of 5–50 for real work.

Constraint $\rightarrow$ target, the table to memorise.

$n \leq 12 \rightarrow O(n!)$	$n \leq 25 \rightarrow O(2^n)$	$n \leq 100 \rightarrow O(n^3)$	$n \leq 5000 \rightarrow O(n^2)$
$n \leq 10^5 \rightarrow O(n \log n)$	$n \leq 10^6 \rightarrow O(n)$	$n \leq 10^9 \rightarrow O(\text{sqrt } n) \text{ or } O(\log n)$	$n \leq 10^{18} \rightarrow O(\log n) \text{ or a formula}$

Memory. Every JavaScript number is a double, 8 bytes. `Int32Array` is 4. 10^7 numbers = 76 MB (fine), 10^7 in an `Int32Array` = 38 MB. An $n \times n$ table for $n = 10^5 = 76$ TB (impossible, and past the $2^{32} - 1$ array-length ceiling anyway). Limit is usually 256 MB.

Logs to know by heart. $\log_2 10^3 \approx 10$, $\log_2 10^6 \approx 20$, $\log_2 10^9 \approx 30$. So $\log n \leq 30$ always. Treat it as a small constant.

Hidden costs inside one line.

Looks like $O(1)$	Really costs
<code>a.splice(i, 1)</code>	$O(n)$ — shifts everything after <code>i</code>
<code>a.unshift(x) / a.shift()</code>	$O(n)$ — shifts everything
<code>a.includes(x) / a.indexOf(x)</code>	$O(n)$ — a full scan wearing a lookup's costume
<code>s.slice(i, i + k)</code>	$O(k)$ — it copies
<code>Set.has / Map.get</code>	$O(1)$ expected, $O(n)$ worst case
<code>a.push(x)</code>	amortised $O(1)$, sometimes $O(n)$
<code>arr.sort()</code>	$O(n \log n)$ — and lexicographic , so also wrong

The seven JavaScript facts this chapter proved with a measurement.

<code>arr.sort()</code>	lexicographic. <code>[10, 9, 1].sort()</code> gives <code>[1, 10, 9]</code> . Always $(a, b) \Rightarrow a - b$.
no heap	MinHeap from the JS Toolkit appendix. Example 19.
no <code>TreeMap</code>	sorted array + <code>lowerBound/upperBound</code> . <code>Map</code> keeps insertion order, not sorted order.
numbers are doubles	exact to $2^{53} - 1 \approx 9 \times 10^{15}$. Past it, <code>BigInt</code> . Example 15.
bitwise is 32-bit	<code>2**40 0</code> is 0; <code>1 << 32</code> is 1. Use <code>BigInt</code> above 31 bits. Examples 14 and 26.
recursion depth	about 10^4 frames, then <code>RangeError</code> . Deep DFS needs an explicit stack.
<code>[...a]</code> is shallow	fine for numbers; a 2-D grid needs <code>a.map(r => [...r])</code> .

The ladder you climb on every problem.

1. Brute force. Write it, state its cost, say why it fails.
2. Find the inner loop's **job**: searching, or re-adding, or re-sorting.
3. Replace that job: sort $\rightarrow$ neighbours, hash $\rightarrow$ lookup, prefix $\rightarrow$ subtraction, heap $\rightarrow$ top- k .
4. State the new cost and check it against the budget.

Top 5 traps.

1. Passing the samples proves nothing. The hidden tests are at the constraint limit. Do the arithmetic first.
2. A bare `.sort()`. It sorts as text, and small single-digit test data hides it.
3. A single loop can still be $O(n^2)$ if a call inside it hides a loop — `includes`, `splice`, `unshift`.
4. $O(1)$ expected is not $O(1)$ worst case. Say “expected” out loud for `Map` and `Set`.
5. Big-O hides the work inside one step. 2^n subsets with $O(n)$ work each is $O(2^n n)$.

Arrays & Prefix Sums

Pay once, answer every question in $O(1)$

Pattern in one page

An array question that says “sum of the values from l to r ” and then asks it two hundred thousand times is not a sum question. It is a **prefix sum** question. Build a running total once, and every answer becomes one subtraction.

THE PREFIX SUM, END TO END

The definition. For an array a of length n , build p of length $n + 1$:

$$p[0] = 0, \quad p[i + 1] = p[i] + a[i]$$

So $p[i]$ is the sum of the **first** i values, that is $a[0] + a[1] + \dots + a[i - 1]$.

The one formula that matters.

$$\text{sum of } a[l..r] = p[r + 1] - p[l]$$

Both ends included. Everything before l appears in both terms and cancels.

Why p has $n + 1$ slots. $p[0] = 0$ is the sum of nothing. Without it you cannot write the formula for a subarray that starts at index 0.

The invariant. After the loop has processed index i , the variable `run` holds $a[0] + \dots + a[i]$, which is exactly $p[i + 1]$.

The pair view — this unlocks half the chapter. Every subarray $a[l..r]$ is a **pair of prefixes** $(p[l], p[r + 1])$. So:

- “subarray with sum k ” $\rightarrow$ “pair of prefixes that differ by k ”
- “subarray with sum divisible by k ” $\rightarrow$ “pair of prefixes with the same remainder mod k ”
- “subarray with XOR k ” $\rightarrow$ “pair of prefix XORs whose XOR is k ”

Counting pairs is a job for a hash map, and it costs $O(n)$.

The difference array — prefix sums run backwards. To add v to every index in $[l, r]$, many times, and only read the array at the end:

$$d[l] += v, \quad d[r + 1] -= v, \quad \text{then take the prefix sum of } d$$

Each update is $O(1)$ instead of $O(r - l + 1)$.

Two dimensions.

$$p[i + 1][j + 1] = g[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j]$$

$$\text{rectangle}(r_1, c_1, r_2, c_2) = p[r_2 + 1][c_2 + 1] - p[r_1][c_2 + 1] - p[r_2 + 1][c_1] + p[r_1][c_1]$$

The middle term is subtracted twice, so it is added back once. This is inclusion–exclusion.

The template you write from memory

Build once, query forever

JS

```
const buildPrefix = (a) => {
  const p = new Array(a.length + 1).fill(0); // .fill(0) is required
  for (let i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
  return p;
}
```



```
};
```

```
const rangeSum = (p, l, r) => p[r + 1] - p[l]; // sum of a[l..r], both ends
included
```

Run on $a = [4, -2, 7, 1, 0, -5, 3]$:

Measured output

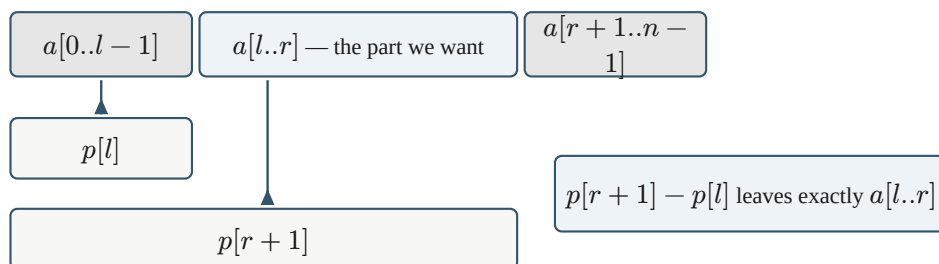
TEXT

```
p = 0 4 2 9 10 10 5 8
sum(0,6) = 8
sum(2,4) = 8
sum(3,3) = 1
sum(0,0) = 4
empty prefix length = 1
single: -9
```

TRAP

`new Array(n + 1)` on its own makes a **sparse** array: every slot is a hole, reading one gives `undefined`, and `undefined + a[i]` is `NaN`. Always write `new Array(n + 1).fill(0)`. For a big fixed-size numeric table, `new Float64Array(n + 1)` is zero-filled already, and when every value fits in 32 bits `new Int32Array(n + 1)` halves the memory too.

Check $\text{sum}(2, 4)$ by hand: $a[2] + a[3] + a[4] = 7 + 1 + 0 = 8$. The formula says $p[5] - p[2] = 10 - 2 = 8$. They agree.



$\text{sum}(l..r) = p[r+1] - p[l]$. The shaded part on the left cancels.

When to reach for this pattern

The problem says	You build
“sum of l to r ”, asked q times	prefix sums, $O(1)$ per query
“how many subarrays have sum k ”	prefix + hash map of counts
“sum divisible by k ”	prefix remainder + a count array of size k
“equal number of 0s and 1s”	map 0 to -1 , then look for sum 0
“XOR of a subarray”	prefix XOR (XOR is its own inverse)
“add v to all of $[l, r]$ ”, many times	difference array
“sum of a rectangle in a grid”	2-D prefix sums
“range sum and the values change”	Fenwick tree — prefix sums break

“maximum in a range”

not prefix sums — max cannot be undone

TRAP

The three bugs that cost marks in this chapter.

1. Off by one. $p[r + 1] - p[l]$, not $p[r] - p[l]$. Write the formula on paper first with a 3-element array.
2. Forgetting `seen.set(0, 1)` in the counting version. That entry stands for the empty prefix, and without it you lose every subarray that starts at index 0.
3. A plain object instead of a `Map` for the prefix counts. Prefix sums go negative, and an object turns the key `-3` into the string `"-3"`. Use a `Map`.

Four JavaScript facts this chapter depends on

TRAP

Numbers are doubles. They are exact only up to `Number.MAX_SAFE_INTEGER =  $2^{53} - 1 = 9,007,199,254,740,991$` . Prefix sums add up fast, so check the worst total **before** you code. 2×10^5 values of 10^4 is 2×10^9 — perfectly safe here. 3×10^5 values of 10^{12} is 3×10^{17} — not safe, and the loop silently drifts. See Example 5 for the measured drift and the `BigInt` fix.

TRAP

Map, not a plain object, for prefix counts. An object stringifies every key, so the number `-3` and the text `"-3"` become one entry, and iteration order is not what you expect for numeric-looking keys. A `Map` keeps the number as a number. Use `map.get(k) ?? 0` when reading, because a missing key gives `undefined` and `undefined + 1` is `NaN`.

TRAP

% on a negative number is negative. Tested in Node: `-7 % 3` is `-1`, not `2`. Using `-1` as an array index gives `undefined`, and `undefined + 1` is `NaN`, so the whole count becomes `NaN` with no error message. Always write `((run % k) + k) % k`. See Example 17.

TRAP

[...grid] is a SHALLOW copy. It copies the outer array only, so both copies still point at the same rows. Tested in Node: after `const shallow = [...grid]; shallow[0][0] = 99;` the original prints `[[99,2],[3,4]]`. A 2-D grid needs `grid.map(row => [...row])`, which leaves the original as `[[1,2],[3,4]]`. This chapter builds 2-D tables in Examples 20 and 26.

This chapter uses `lowerBound`, `upperBound` and `Deque` from the **JS Toolkit** appendix. JavaScript ships none of them. One line brings them in:

The one require line

JS

```
const { Deque, lowerBound, upperBound } = require('./toolkit.js');
```

What prefix sums cannot do

Subtraction is what makes the trick work: to remove the left part, you **undo** it. So the pattern works for any operation that can be undone — plus (undo with minus), XOR (undo with XOR), count (undo with subtract), product with no zeros (undo with divide, carefully).

It does **not** work for maximum or minimum. You cannot “un-maximum” a value. Range maximum needs a different tool (a sparse table, a segment tree, or a monotonic deque — Chapters 9 and onwards).

Warm-up

WARM-UP

build the reflex

Example 1

Build the prefix array of $a = [3, 1, 4, 1, 5]$ by hand.

SOLUTION

Start with $p[0] = 0$. Then keep adding.

- $p[1] = p[0] + a[0] = 0 + 3 = 3$
- $p[2] = p[1] + a[1] = 3 + 1 = 4$
- $p[3] = p[2] + a[2] = 4 + 4 = 8$
- $p[4] = p[3] + a[3] = 8 + 1 = 9$
- $p[5] = p[4] + a[4] = 9 + 5 = 14$

Answer: $p = [0, 3, 4, 8, 9, 14]$ — six entries for five values

Example 2

Using the p from Example 1, find the sum of $a[1..3]$.

SOLUTION

The formula is $p[r + 1] - p[l]$ with $l = 1$ and $r = 3$. $p[4] - p[1] = 9 - 3 = 6$. Check directly: $a[1] + a[2] + a[3] = 1 + 4 + 1 = 6$. Correct.

Answer: 6

Example 3

Same p . Find the sum of $a[0..4]$ and the sum of $a[2..2]$.

SOLUTION

Whole array: $l = 0, r = 4$, so $p[5] - p[0] = 14 - 0 = 14$. This is why $p[0] = 0$ must exist.

One single element: $l = r = 2$, so $p[3] - p[2] = 8 - 4 = 4$, which is $a[2]$. Correct.

Answer: 14 and 4

Example 4

An array of 0s and 1s: $a = [1, 0, 0, 1, 1]$. How many 1s are in $a[1..4]$?

SOLUTION

A count is just a sum when the values are 0 and 1. $p = [0, 1, 1, 1, 2, 3]$. Sum of $a[1..4] = p[5] - p[1] = 3 - 1 = 2$. Check: $a[1], a[2], a[3], a[4] = 0, 0, 1, 1$, which has two 1s. Correct.

Answer: 2

Any yes/no test becomes a prefix sum: write 1 when the test passes and 0 when it fails. “How many values above 50 in this range” is the same problem.

Example 5

An array holds 300,000 ride fares, each exactly 10,000. Is the total exact in JavaScript? Now make each fare 987,654,321,987. Is it still exact?

SOLUTION

$300[,]000 \times 10[,]000 = 3[,]000[,]000[,]000 = 3 \times 10^9$. JavaScript numbers are exact up to $2^{53} - 1 \approx 9 \times 10^{15}$, so 3×10^9 is safe with room to spare. There is no 32-bit integer type in JavaScript, so there is nothing to overflow here.

The second total is $300[,]000 \times 987[,]654[,]321[,]987 \approx 3 \times 10^{17}$. That is **past** the safe range. This was measured, not guessed:

Measured output

TEXT

```
n*f Number = 296296296596100000 safe? false
n*f BigInt = 296296296596100000
summed in a loop = 296296296595336800
summed as BigInt = 296296296596100000
```

The loop answer is wrong by 763,200. Notice that this is **not** a crash and **not** a negative number. It is a plausible-looking number with the wrong digits near the end — the hardest kind of bug to spot.

The fix is BigInt. Convert only the values that need it, and only at the end convert back:

Exact totals past 2^{53}

JS

```
let total = 0n;
for (const fare of fares) total += BigInt(fare);
console.log(total.toString());
```

Run on 300,000 copies of 987654321987, this prints 296296296596100000, which matches the BigInt product exactly.

Answer: 3×10^9 is exact; 3×10^{17} is not and needs BigInt

The habit to build: before writing the prefix loop, multiply n by the largest value and compare with 9×10^{15} . Every other example in this chapter passes that check, so they all use plain numbers.

Example 6

You know the total of the whole array and the sum of everything to the left of index i . How do you get the sum of everything to the right of i , with no second loop?

SOLUTION

The three parts cover the whole array exactly once:

$$\text{left} + a[i] + \text{right} = \text{total}$$

So

$$\text{right} = \text{total} - \text{left} - a[i]$$

One subtraction, $O(1)$. This removes the inner loop from the next example.

Answer: `right = total - left - a[i]`

Tier 1 — Build the reflex

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Problem. An array of n daily sales figures. Answer q questions of the form “total sales from day l to day r ”.
Constraints. $n, q \leq 2 \times 10^5$; each figure in $[-10^4, 10^4]$ (refunds are negative). **Target.** $O(n + q)$. **Edge cases.** $l = r$; $l = 0$ and $r = n - 1$; a total near the largest possible.

SOLUTION

Approach 1 — Add up the range for each question $O(nq) = 4 \cdot 10^{10}$ — too slow

One question, one scan

JS

```
const sumSlow = (a, l, r) => {
  let s = 0;
  for (let i = l; i <= r; i++) s += a[i];
  return s;
};
```

Time $O(n)$ per query **Space** $O(1)$

Approach 2 — Prefix sums $O(n)$ build, $O(1)$ per query — optimal

Build once

JS

```
const p = new Array(n + 1).fill(0);
for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
// answer a query:
const answer = p[r + 1] - p[l];
```

Time $O(n + q)$ **Space** $O(n)$

Measured with $n = q = 200[,]000$ wide queries, against the slow version on the first 200 of them:

Measured output

TEXT

```
first 200 queries agree? yes
slow: 200 queries in 32 ms
prefix: all 200000 queries in 19 ms
```

Thirty-two milliseconds for 200 questions means about 32 seconds for 200,000. The prefix version answered all of them in 19 ms.

The idea. The questions overlap, so the slow version adds the same numbers over and over. Pay $O(n)$ once and every later answer is free.

The worst possible total here is $2 \times 10^5 \times 10^4 = 2 \times 10^9$. Run the check from Example 5: that is far under 9×10^{15} , so plain numbers are exact and no `BigInt` is needed.

Example 8 [Infosys · pattern]

Problem. Find an index i where the sum of everything to its left equals the sum of everything to its right. Return -1 if none exists. **Constraints.** $n \leq 2 \times 10^5$; values may be negative. **Target.** $O(n)$. **Edge cases.** empty array (answer -1); one element (index 0 always works, both sides are empty and sum to 0); all zeros.

SOLUTION

Approach 1 — For each i , add both sides $O(n^2)$

Brute force

JS

```
const equilibriumBrute = (a) => {
  for (let i = 0; i < a.length; i++) {
    let left = 0, right = 0;
    for (let j = 0; j < i; j++) left += a[j];
    for (let j = i + 1; j < a.length; j++) right += a[j];
    if (left === right) return i;
  }
  return -1;
};
```

Approach 2 — Carry the left sum, derive the right sum ($O(n)$ — optimal)

Use Example 6: $\text{right} = \text{total} - \text{left} - a[i]$. Nothing has to be re-added.

```
One pass JS
const equilibriumFast = (a) => {
  let total = 0;
  for (const x of a) total += x;
  let left = 0;
  for (let i = 0; i < a.length; i++) {
    const right = total - left - a[i];
    if (left === right) return i;
    left += a[i]; // now left is the sum of a[0..i]
  }
  return -1;
};
```

Time $O(n)$ **Space** $O(1)$ extra

Both were run on the same tests:

```
Measured output TEXT
[] brute=-1 fast=-1
[7] brute=0 fast=0
[1,2,3] brute=-1 fast=-1
[2,4,2] brute=1 fast=1
[1,-1,0,-1,1] brute=2 fast=2
[0,0,0] brute=0 fast=0
[5,5] brute=-1 fast=-1
```

Look at [7]: the answer is index 0, because the empty left side and the empty right side both sum to 0. Many students return -1 here and lose the test case.

The idea. The inner loops were recomputing a sum that changes by only one element each step. Carry it instead.

TRAP

$\text{left} += a[i]$ must come **after** the check, not before. Put it before and you have included $a[i]$ in the left side, which is wrong.

Example 9 [Wipro · pattern]

Problem. Given n values and a threshold t , answer q questions: “how many values in $a[l..r]$ are greater than t ?”

Constraints. $n, q \leq 2 \times 10^5$; t is fixed for all questions. **Target.** $O(n + q)$. **Edge cases.** a range where no value passes; a range of length 1.

SOLUTION

A count is a sum of 1s and 0s. Build a prefix array of the **test result**, not of the values.

Prefix of a yes/no test

JS

```
const c = new Array(n + 1).fill(0);
for (let i = 0; i < n; i++) c[i + 1] = c[i] + (a[i] > threshold ? 1 : 0);
// answer: number of values > threshold in a[l..r]
const answer = c[r + 1] - c[l];
```

Time $O(n + q)$ **Space** $O(n)$

Run on $a = [10, 55, 3, 80, 50, 51, 2, 99]$ with $\text{threshold} = 50$:

Measured output

TEXT

```
counts prefix: 0 0 1 1 2 2 3 3 4
range [0,7] fast=4 slow=4
range [1,3] fast=2 slow=2
range [2,2] fast=0 slow=0
range [0,0] fast=0 slow=0
range [6,7] fast=1 slow=1
```

Careful with 50 itself: the test is **greater than** 50, so $a[4] = 50$ does not count. That is why the prefix does not rise at position 5.

SHORTCUT

If the threshold changed from query to query, this trick would stop working, and the answer would be “sort the values and binary-search with `lowerBound` from the **JS Toolkit** appendix”, or a merge sort tree. Watch for the word **fixed** in the statement.

Example 10 [Capgemini · pattern]

Problem. Find the largest sum of any k consecutive values. **Constraints.** $n \leq 10^6$, $1 \leq k \leq n$; values may be negative. **Target.** $O(n)$. **Edge cases.** $k = n$ (only one window); $k = 1$ (the largest single value); all values negative.

SOLUTION

Approach 1 — Add up every window ($O(nk)$)

Approach 2 — Prefix sums ($O(n)$ — each window is one subtraction)

Windows from prefix sums

JS

```
const bestWindowPrefix = (a, k) => {
  const n = a.length;
  if (k > n || k <= 0) return null;          // no window exists
  const p = buildPrefix(a);
  let best = -Infinity;
  for (let l = 0; l + k <= n; l++) best = Math.max(best, p[l + k] - p[l]);
  return best;
};
```

Approach 3 — Slide the window ($O(n)$ time, $O(1)$ extra space — optimal)

Moving the window one step right adds one value and drops one value.

```
const bestWindowSlide = (a, k) => {
  const n = a.length;
  if (k > n || k <= 0) return null;
  let run = 0;
  for (let i = 0; i < k; i++) run += a[i];
  let best = run;
  for (let i = k; i < n; i++) {
    run += a[i] - a[i - k]; // add the new one, drop the old one
    best = Math.max(best, run);
  }
  return best;
};
```

Time $O(n)$ **Space** $O(1)$

Same time as prefix sums, but no $n+1$ array. This is Chapter 3's pattern.

Both, on $a = [3, -1, 4, 1, -5, 9, 2, -6]$:

Measured output

TEXT

```
k=1 prefix=9 slide=9
k=2 prefix=11 slide=11
k=3 prefix=6 slide=6
k=4 prefix=9 slide=9
k=5 prefix=11 slide=11
k=6 prefix=11 slide=11
k=7 prefix=13 slide=13
k=8 prefix=7 slide=7
single, k=1: 7
k larger than n: null
all negative [-3,-1,-4] k=2: -4
```

Check $k = 2$: the windows are 2, 3, 5, -4, 4, 11, -4. The largest is 11, from 9 + 2. Correct.

TRAP

Start $best$ at $-\infty$, not at 0. The last measured line is the reason: with every value negative, the true answer is -4 , and a version that starts at 0 returns 0, which is not the sum of any window. JavaScript's $-\infty$ is the clean equivalent of a “smallest possible” sentinel, and unlike a hand-picked big negative number it can never be beaten by real data.

Example 11 [Cognizant · pattern]

Problem. Cut the array into two non-empty parts at some position. Make the two part-sums as close as possible, and report the smallest possible difference. **Constraints.** $2 \leq n \leq 2 \times 10^5$; values in $[-10^4, 10^4]$. **Target.** $O(n)$. **Edge cases.** $n = 2$ (only one cut); all values equal; negative values.

SOLUTION

Approach 1 — Try every cut and add both sides ($O(n^2)$)

Approach 2 — Total minus running left ($O(n)$ — optimal)


```
const bestSplitFast = (a) => {
  let total = 0;
  for (const x of a) total += x;
  let left = 0, best = Infinity;
  for (let i = 0; i + 1 < a.length; i++) { // cut AFTER index i, so i stops at
n-2
    left += a[i];
    best = Math.min(best, Math.abs(left - (total - left)));
  }
  return best;
};
```

Time $O(n)$ **Space** $O(1)$

Checked against the brute force on every test, plus 500 random arrays:

Measured output

TEXT

```
[1,1] brute=0 fast=0
[1,2,3,4] brute=2 fast=2
[10,-10,10] brute=10 fast=10
[5,5,5,5] brute=0 fast=0
[-3,-3] brute=0 fast=0
[1,2,3,4,5,6] brute=1 fast=1
500 random cross-checks passed
```

Trace $[1, 2, 3, 4]$: total is 10. Cut after index 0: $|1 - 9| = 8$. After index 1: $|3 - 7| = 4$. After index 2: $|6 - 4| = 2$. The best is 2.

TRAP

The loop must stop at $i = n - 2$. If it runs to $n - 1$, the right part is empty, which the problem forbids.

Example 12 [Accenture · pattern]

Problem. Prices of one item on n days. Buy on one day and sell on a **later** day. What is the largest gain? If every later price is lower, the answer is 0 (do not trade). **Constraints.** $n \leq 10^6$; prices up to 10^9 . **Target.** $O(n)$. **Edge cases.** empty or single day (answer 0); prices only falling (answer 0); all prices equal (answer 0).

SOLUTION

Approach 1 — Every buy day against every sell day ($O(n^2)$)

Approach 2 — Carry the cheapest day so far ($O(n)$ — optimal)

For each day, the best possible sale today uses the **smallest price seen before today**. So keep that one number.

One pass, one carried value

JS

```
const bestGainFast = (price) => {
  if (price.length === 0) return 0;
  let best = 0, lowestSoFar = price[0];
  for (let i = 1; i < price.length; i++) {
    best = Math.max(best, price[i] - lowestSoFar);
    lowestSoFar = Math.min(lowestSoFar, price[i]);
  }
}
```

```
return best;
};
```

Time $O(n)$ **Space** $O(1)$

Measured output

TEXT

```
[] brute=0 fast=0
[5] brute=0 fast=0
[1,5] brute=4 fast=4
[5,1] brute=0 fast=0
[7,2,9,4,10,1] brute=8 fast=8
[9,8,7] brute=0 fast=0
[4,4,4] brute=0 fast=0
500 random cross-checks passed
```

Trace [7, 2, 9, 4, 10, 1]: lowest so far goes 7, 2, 2, 2, 2. The gains are $2 - 7$ (ignore, negative), $9 - 2 = 7$, $4 - 2 = 2$, $10 - 2 = 8$, $1 - 2$ (negative). Best is 8.

The idea. This is the same shape as the prefix trick: an inner loop that searches backwards for a minimum is replaced by one carried value. The cost drops from $O(n^2)$ to $O(n)$.

TRAP

The update order matters. Compare **first**, then update `lowestSoFar`. If you update first, you allow buying and selling on the same day.

Example 13 [TCS Digital · pattern]

Problem. An array holds all of $1, 2, \dots, n + 1$ except one value, in any order. Find the missing one. **Constraints.** $n \leq 2 \times 10^6$. **Target.** $O(n)$ time, $O(1)$ extra space. **Edge cases.** empty array (the missing value is 1); the missing value is the largest; a very large n where the total leaves the exact range of a JavaScript number.

SOLUTION

Approach 1 — Sort, then scan for the gap ($O(n \log n)$)

Approach 2 — Expected total minus actual total ($O(n)$, but the total grows fast)

Sum method

JS

```
const missingBySum = (a) => {
  const n = a.length;
  let total = (n + 1) * (n + 2) / 2; // sum of 1..n+1
  for (const x of a) total -= x;
  return total;
};
```

Approach 3 — XOR everything ($O(n)$, and the value never grows — optimal)

XOR has two gifts: $x \oplus x = 0$, and $x \oplus 0 = x$. XOR all of $1..n + 1$ together with every value present. Every present value appears twice and cancels itself. Only the missing one survives.

XOR method

JS

```
const missingByXor = (a) => {
  const n = a.length;
  let x = 0;
  for (let i = 1; i <= n + 1; i++) x ^= i;
  for (const v of a) x ^= v;
  return x;
};
```

Time $O(n)$ **Space** $O(1)$

The running value never exceeds the largest input.

Measured output

TEXT

```
n=0 [] sum=1 xor=1
n=1 [2] sum=1 xor=1
n=1 [1] sum=2 xor=2
n=4 [1,2,4,5] sum=3 xor=3
n=4 [5,4,3,2] sum=1 xor=1
n=5 [2,3,4,5,6] sum=1 xor=1
big: sum=1234567 xor=1234567
total of 1..2000001 = 2000003000001 safe? true
total of 1..200000000 = 20000000100000000 safe? false
```

The last two lines are the point. At $n = 2 \times 10^6$ the expected total is about 2×10^{12} , which a JavaScript number still holds exactly. Push n to 2×10^8 and the total is 2×10^{16} , past $2^{53} - 1$, and the sum method starts returning nonsense. The XOR method never has that problem, because the XOR of numbers below 2^{21} is itself below 2^{21} .

TRAP

JavaScript's `^` converts both sides to a **32-bit signed integer**. That is fine here, because $n + 1 \leq 2^{31}$. If the values could reach 2^{31} the XOR method would break too, and you would need BigInt XOR. Always check the value range before reaching for a bitwise operator in JavaScript.

Example 14 [Infosys · pattern]

Problem. Rotate an array right by k positions. So $[1, 2, 3, 4, 5]$ with $k = 2$ becomes $[4, 5, 1, 2, 3]$. **Constraints.** $n \leq 10^6$, $0 \leq k \leq 10^9$. **Target.** $O(n)$ time, $O(1)$ extra space. **Edge cases.** empty array; $k = 0$; k bigger than n (take $k \bmod n$); $k = n$ (no change).

SOLUTION

Approach 1 — Shift by one, k times ($O(nk)$ — 10^{15} at the limits, hopeless)

Approach 2 — Copy into a second array ($O(n)$ time, $O(n)$ space)

With an extra array

JS

```
const rotateExtra = (a, k) => {
  const n = a.length;
  if (n === 0) return a;
  k %= n;
  const b = new Array(n);
  for (let i = 0; i < n; i++) b[(i + k) % n] = a[i];
  for (let i = 0; i < n; i++) a[i] = b[i];    // copy back, in place
  return a;
};
```

Approach 3 — Three reversals ($O(n)$ time, $O(1)$ space — optimal)

Reverse the whole array, then reverse the first k , then reverse the rest. JavaScript's built-in `a.reverse()` reverses the **whole** array and takes no range, so write a two-pointer helper that reverses a slice in place.

The reversal trick

JS

```
const reverseRange = (a, i, j) => {
  while (i < j) { [a[i], a[j]] = [a[j], a[i]]; i++; j--; }
};

const rotateReverse = (a, k) => {
  const n = a.length;
  if (n === 0) return a;
  k %= n;
  reverseRange(a, 0, n - 1);
  reverseRange(a, 0, k - 1);
  reverseRange(a, k, n - 1);
  return a;
};
```

Why it works, on `[1,2,3,4,5]` with $k = 2$:

- reverse all: `[5,4,3,2,1]` — the last two are now at the front, but backwards
- reverse the first 2: `[4,5,3,2,1]` — front block fixed
- reverse the rest: `[4,5,1,2,3]` — done

Both versions were run and always agreed:

Measured output

TEXT

```
k=0 [1,2,3,4,5] -> [1,2,3,4,5] same? yes
k=1 [1,2,3,4,5] -> [5,1,2,3,4] same? yes
k=2 [1,2,3,4,5] -> [4,5,1,2,3] same? yes
k=5 [1,2,3,4,5] -> [1,2,3,4,5] same? yes
k=7 [1,2,3,4,5] -> [4,5,1,2,3] same? yes
k=2 [] -> [] same? yes
k=2 [9] -> [9] same? yes
```

$k = 7$ gives the same answer as $k = 2$, because $7 \bmod 5 = 2$.

TRAP

`k %= n` must come **after** the empty check. In JavaScript `5 % 0` is `NaN`, not a crash, so every later index becomes `NaN` and the array silently fills with `undefined`. A missing guard here does not throw; it produces garbage.

TRAP

`[a[i], a[j]] = [a[j], a[i]]` is the JavaScript in-place swap. Writing `a[i] = a[j]; a[j] = a[i];` destroys the first value — the classic beginner bug. The destructuring version builds a small temporary array and unpacks it, so both values survive.

`a.slice(-k).concat(a.slice(0, -k))` is a one-line answer and is perfectly readable, but it allocates two new arrays, so it is $O(n)$ space. The three-reversal version is the one to show when the interviewer asks for $O(1)$ extra space.

Example 15 [TCS NQT · pattern]

Problem. Find the largest sum of any non-empty contiguous block. **Constraints.** $n \leq 10^6$; values in $[-10^4, 10^4]$. **Target.** $O(n)$. **Edge cases.** one element; every value negative (the answer is the largest single value, not 0); a block of zeros.

SOLUTION

Approach 1 — Every start, every end ($O(n^2)$ after the obvious fix, $O(n^3)$ if you re-add each block)

Brute force, already improved to $O(n^2)$

JS

```
const maxSubarrayBrute = (a) => {
  let best = -Infinity;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) { s += a[j]; best = Math.max(best, s); }
  }
  return best;
};
```

Approach 2 — Prefix sums and the smallest prefix so far ($O(n)$ — and it explains the next one)

The sum of $a[l..r]$ is $p[r+1] - p[l]$. For a fixed right end r , the best choice of l is the one with the **smallest** $p[l]$ seen so far. So carry that minimum.

Prefix view

JS

```
const maxSubarrayPrefix = (a) => {
  let best = -Infinity, run = 0, minPrefix = 0;
  for (const x of a) {
    run += x; // run is p[r+1]
    best = Math.max(best, run - minPrefix); // best block ending here
    minPrefix = Math.min(minPrefix, run); // ready for the next r
  }
  return best;
};
```

Approach 3 — Kadane ($O(n)$, $O(1)$ — the same idea, written shorter)

Kadane's rule

JS

```
const kadane = (a) => {
  let best = a[0], cur = a[0];
  for (let i = 1; i < a.length; i++) {
    cur = Math.max(a[i], cur + a[i]); // start fresh, or extend
    best = Math.max(best, cur);
  }
  return best;
};
```

Time $O(n)$ **Space** $O(1)$

All three, on the same inputs, plus 500 random cross-checks:

Measured output

TEXT

```

[5] brute=5 prefix=5 kadane=5
[-5] brute=-5 prefix=-5 kadane=-5
[1,2,3] brute=6 prefix=6 kadane=6
[-2,1,-3,4,-1,2,1,-5,4] brute=6 prefix=6 kadane=6
[-3,-1,-7] brute=-1 prefix=-1 kadane=-1
[0,0] brute=0 prefix=0 kadane=0
[2,-1,2,-1,2] brute=4 prefix=4 kadane=4
500 random cross-checks passed

```

`[-3, -1, -7]` gives -1 , not 0 . A version that starts `best = 0` would wrongly say 0 .

The idea that unlocked it. “Best block ending at r ” is $p[r + 1] - \min_{l \leq r} p[l]$. Kadane’s `cur = max(a[i], cur + a[i])` is exactly the same sentence with the prefix array removed.

Example 16 [Amazon · pattern]

Problem. Count the subarrays whose sum is exactly k . Values may be negative and may be zero. **Constraints.** $n \leq 2 \times 10^5$; values in $[-10^9, 10^9]$, so a prefix can reach 2×10^{14} — still exact in a JavaScript number. **Target.** $O(n)$. **Edge cases.** empty array; $k = 0$ with an array of zeros (many answers); a single element equal to k .

SOLUTION

Approach 1 — Every start, extend to the right ($O(n^2)$)

Brute force

JS

```

const countSubarraysBrute = (a, k) => {
  let cnt = 0;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) {
      s += a[j];
      if (s === k) cnt++;
    }
  }
  return cnt;
};

```

Time $O(n^2)$ **Space** $O(1)$
 $n = 2 \cdot 10^5$ gives $2 \cdot 10^{10}$ steps. Dies.

Approach 2 — Count pairs of prefixes ($O(n)$ average — optimal)

$a[l..r]$ has sum k exactly when $p[r + 1] - p[l] = k$, that is $p[l] = p[r + 1] - k$.

So walk the array once. At each right end, ask **how many earlier prefixes** had the value `run - k`. Store the counts in a hash map.

Prefix + Map — memorise this

JS

```

const countSubarraysFast = (a, k) => {
  const seen = new Map();
  seen.set(0, 1); // the empty prefix, p[0] = 0
  let run = 0, cnt = 0;
  for (const x of a) {
    run += x;
    cnt += seen.get(run - k) ?? 0; // ?? 0 because get() gives undefined
    seen.set(run, (seen.get(run) ?? 0) + 1);
  }
  return cnt;
};

```

```

    }
    return cnt;
};

```

Time $O(n)$ expected **Space** $O(n)$

The Python version is shorter and says the same thing:

Python

PYTHON

```

from collections import defaultdict

def count_subarrays(a, k):
    seen = defaultdict(int)
    seen[0] = 1          # the empty prefix
    run = 0
    cnt = 0
    for x in a:
        run += x
        cnt += seen[run - k]
        seen[run] += 1
    return cnt

```

Both were run. Same answers, and 500 random cross-checks against the brute force:

Measured output (JavaScript and Python agree)

TEXT

```

k=0 [] brute=0 fast=0
k=5 [5] brute=1 fast=1
k=0 [5] brute=0 fast=0
k=3 [1,2,3] brute=2 fast=2
k=0 [1,-1,1,-1] brute=4 fast=4
k=0 [0,0,0] brute=6 fast=6
k=1 [3,-1,-1,2] brute=2 fast=2
k=0 [-2,-3,5] brute=1 fast=1
500 random cross-checks passed

```

And the speed, on 30,000 random values in $[-5, 5]$ with $k = 3$:

Measured output

TEXT

```

n=30000 brute=887614 fast=887614
brute time = 895 ms
fast  time = 3 ms

```

The idea. A subarray is a **pair of prefixes**. Once you see that, counting subarrays becomes counting pairs, and counting pairs is what a Map does in $O(1)$ expected each.

TRAP

1. `seen.set(0, 1)` is not optional. Drop it and `[1,2,3]` with $k = 3$ returns 1 instead of 2, because the subarray `[1,2]` starts at index 0.
2. A plain object instead of a Map breaks silently. Prefix sums go negative here, and `obj[-3]` stores under the string key `"-3"`. It happens to still work for this exact problem, but it stops working the moment you also need 0 and `-0` apart, or you iterate the keys. Use a Map and stop thinking about it.

3. `seen.get(...)` returns `undefined`, not `0`. Without `?? 0` the count becomes `NaN`, and `NaN` never equals anything, so no test will tell you where it went wrong.
4. A sliding window does **not** work here. Windows need the sum to grow when you extend, and negative values break that. Sliding windows are for non-negative values (Chapter 3).

Tier 2 — Business-shaped variations

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Grab · pattern]

Problem. Daily profit or loss figures for n days. Count the periods (contiguous blocks of days) whose total is divisible by k . **Constraints.** $n \leq 2 \times 10^5$, $2 \leq k \leq 10^4$; values may be negative. **Target.** $O(n + k)$. **Edge cases.** empty array; a block summing to exactly 0 (0 is divisible by everything); negative sums, where JavaScript's `%` returns a negative remainder.

SOLUTION

Approach 1 — Every block $O(n^2)$

Approach 2 — Count prefixes by remainder $O(n + k)$ — optimal

$\text{sum}(l..r) = p[r + 1] - p[l]$ is divisible by k exactly when $p[r + 1]$ and $p[l]$ leave the **same remainder** mod k . So group the prefixes by remainder and count pairs inside each group.

Since remainders are $0..k - 1$, a plain array of size k replaces the Map.

Remainder counting

JS

```
const divisibleFast = (a, k) => {
  const cntMod = new Array(k).fill(0);
  cntMod[0] = 1; // the empty prefix has remainder 0
  let run = 0, ans = 0;
  for (const x of a) {
    run += x;
    const m = ((run % k) + k) % k; // force a non-negative remainder
    ans += cntMod[m];
    cntMod[m]++;
  }
  return ans;
};
```

Time $O(n + k)$ **Space** $O(k)$

Checked against the brute force, and on 500 random arrays:

Measured output

TEXT

```
k=3 [] brute=0 fast=0
k=3 [6] brute=1 fast=1
k=3 [1] brute=0 fast=0
k=2 [2,-2,2,-4] brute=10 fast=10
k=2 [-1,2,9] brute=2 fast=2
k=5 [5,0,0,5] brute=10 fast=10
```



```
k=5 [1,2,3,4,5] brute=4 fast=4
500 random cross-checks passed
-7 % 3 in JS = -1 fixed = 2
```

TRAP

Read the last measured line. In JavaScript, $-7 \% 3$ is -1 , not 2 — the sign follows the **left** operand, exactly as in C++ and Java. In JavaScript the damage is quieter than a crash: `cntMod[-1]` is undefined, `ans += undefined` makes `ans` become `NaN`, and `NaN` spreads through the rest of the run without any error. Always write `((run % k) + k) % k`. Python is the odd one out here — $-7 \% 3$ is 2 there, so a Python solution ported straight to JavaScript will fail on exactly the negative test cases.

Example 18 [Shopee · pattern]

Problem. Find the **longest** block whose sum is exactly k . Report its length, or 0 if there is none. **Constraints.** $n \leq 2 \times 10^5$; values may be negative. **Target.** $O(n)$. **Edge cases.** no such block; the whole array works; several blocks of the same length.

SOLUTION

Approach 1 — Every block $O(n^2)$

Approach 2 — Prefix + map of the EARLIEST index $O(n)$ — optimal

The counting version (Example 16) stored **how many** prefixes had each value. For the longest block we instead store **the first index** at which each prefix value appeared — because the earlier the left end, the longer the block.

Keep the earliest position

JS

```
const longestFast = (a, k) => {
  const firstAt = new Map(); // prefix value -> earliest index in p
  firstAt.set(0, 0);
  let run = 0, best = 0;
  for (let i = 0; i < a.length; i++) {
    run += a[i];
    if (firstAt.has(run - k)) best = Math.max(best, i + 1 - firstAt.get(run - k));
    if (!firstAt.has(run)) firstAt.set(run, i + 1); // only the first time
  }
  return best;
};
```

Time $O(n)$ expected **Space** $O(n)$

Measured output

TEXT

```
k=0 [] brute=0 fast=0
k=4 [4] brute=1 fast=1
k=5 [4] brute=0 fast=0
k=3 [1,2,3,0,0,4] brute=3 fast=3
k=0 [-1,1,-1,1] brute=4 fast=4
k=2 [2,-2,2,-2,2] brute=5 fast=5
k=5 [1,1,1] brute=0 fast=0
500 random cross-checks passed
```

Check $[1, 2, 3, 0, 0, 4]$ with $k = 3$: the blocks summing to 3 are $[1, 2]$ (length 2), $[3]$ (length 1), and $[3, 0, 0]$ (length 3). The longest is 3.

TRAP

Counting wants every occurrence; **longest** wants only the first. Drop the `if (!firstAt.has(run))` guard and `firstAt.set` will overwrite the earliest index with a later one, shortening your own answer. This one line is the difference between the two problems.

Use `.has()` here, not `.get(...)` `!== undefined`. A prefix value of `0` is a perfectly real key, and `firstAt.get(0)` returns the index `0`, which is falsy. Writing `if (firstAt.get(run - k))` would skip exactly the case the seed entry exists for.

Example 19 [Sea/Shopee · pattern]

Problem. A log of n events, each a success (1) or a failure (0). Count the blocks with an equal number of successes and failures. **Constraints.** $n \leq 2 \times 10^5$. **Target.** $O(n)$. **Edge cases.** all 1s (answer 0); a single event; the whole log balanced.

SOLUTION

The trick is a re-labelling. Treat 1 as $+1$ and 0 as -1 . Now “equal counts” means “the block sums to 0”, and Example 16 already solves that with $k = 0$.

Re-label, then count sum-zero blocks

JS

```
const equalZeroOne = (a) => {
  const seen = new Map();
  seen.set(0, 1);
  let run = 0, cnt = 0;
  for (const x of a) {
    run += (x === 1 ? 1 : -1);           // 1 counts +1, 0 counts -1
    cnt += seen.get(run) ?? 0;
    seen.set(run, (seen.get(run) ?? 0) + 1);
  }
  return cnt;
};
```

Time $O(n)$ Space $O(n)$

Measured output

TEXT

```
[] brute=0 fast=0
[0] brute=0 fast=0
[0,1] brute=1 fast=1
[1,0] brute=1 fast=1
[0,0,1,1] brute=2 fast=2
[1,1,1] brute=0 fast=0
[0,1,0,1,0] brute=6 fast=6
500 random cross-checks passed
```

TRAP

`run` goes negative here — it is a walk up and down from zero. That makes the `Map` choice load-bearing: with a plain object the key `-1` becomes the string `"-1"`, and it only takes one place elsewhere in your code that compares a key against a number for the whole thing to quietly disagree with itself.

Check `[0,0,1,1]`: the balanced blocks are `[0,1]` (positions 1–2) and the whole array. That is 2.

The idea. Do not invent a new algorithm. Change the data so an old algorithm fits. “Equal counts of two things” is almost always “sum to zero after re-labelling”.

SHORTCUT

The same trick solves “equal numbers of vowels and consonants”, “equal numbers of even and odd”, and “equal numbers of opening and closing brackets, ignoring validity”. Map one kind to +1 and the other to -1.

Example 20 [Agoda · pattern]

Problem. A grid of $R \times C$ nightly room prices (rows are hotels, columns are nights). Answer q questions: “total price over hotels $r_1..r_2$ and nights $c_1..c_2$ ”. **Constraints.** $R, C \leq 1000$, $q \leq 2 \times 10^5$. **Target.** $O(RC + q)$. **Edge cases.** a 1×1 rectangle; the whole grid; a rectangle touching row 0 or column 0.

SOLUTION

Approach 1 — Add up the rectangle each time ($O(RC)$ per query — $2 \cdot 10^{11}$ total)

Approach 2 — 2-D prefix sums ($O(RC)$ build, $O(1)$ per query — optimal)

Build and query

JS

```
const buildGrid2D = (g) => {
  const R = g.length, C = R ? g[0].length : 0;
  // Array.from with a FUNCTION: each row must be its own array
  const p = Array.from({length: R + 1}, () => new Array(C + 1).fill(0));
  for (let i = 0; i < R; i++)
    for (let j = 0; j < C; j++)
      p[i+1][j+1] = g[i][j] + p[i][j+1] + p[i+1][j] - p[i][j];
  return (r1, c1, r2, c2) =>
    p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1];
};
```

Time $O(RC)$ build, $O(1)$ per query **Space** $O(RC)$

TRAP

Never build a 2-D array with `new Array(R + 1).fill(new Array(C + 1).fill(0))`. `fill` puts the same row object in every slot, so writing `p[1][1] = 5` also changes `p[0][1]`, `p[2][1]` and every other row. Use `Array.from({length: R + 1}, () => new Array(C + 1).fill(0))`, which calls the function once per row and therefore builds $R + 1$ separate arrays.

The same trap bites when copying. Tested in Node: with `grid = [[1,2],[3,4]]`, `const shallow = [...grid]`; `shallow[0][0] = 99`; leaves the **original** as `[[99,2],[3,4]]`. The correct deep copy of a grid of numbers is `grid.map(row => [...row])`, after which the original still prints `[[1,2],[3,4]]`.

Why four terms? $p[r_2 + 1][c_2 + 1]$ is everything above and left of the bottom-right corner. Subtract the strip above the rectangle and the strip to its left. The top-left corner block was inside **both** strips, so it has been removed twice — add it back once.

Run on this grid:

The grid and its prefix table

TEXT

grid:		prefix table p:
1 2 3 4		0 0 0 0 0
5 6 7 8		0 1 3 6 10
-1 0 2 1		0 6 14 24 36

3	3	3	3	0	5	13	25	38
				0	8	19	34	50

Measured output

TEXT

```
query(0,0,3,3) = 50  brute=50
query(1,1,2,2) = 15  brute=15
query(0,0,0,0) = 1   brute=1
query(2,0,2,3) = 2   brute=2
query(3,3,3,3) = 3   brute=3
1x1 grid: 9
after [...grid] edit, original is [[99,2],[3,4]]
after map(r=>[...r]) edit, original is [[1,2],[3,4]]
```

Check `query(1,1,2,2)` by hand: the values are 6, 7, 0, 2, total 15. The formula gives $p[3][3] - p[1][3] - p[3][1] + p[1][1] = 25 - 6 - 5 + 1 = 15$. Correct.

TRAP

Memory. Every JavaScript number is 8 bytes, so a 1000×1000 prefix table is 8 MB, which is fine. A $10^5 \times 10^5$ grid is 80 GB and is not storable at all — check $R \times C$ against the memory limit before you build.

Example 21 [LINE MAN · pattern]

Problem. A delivery zone has n time slots, all starting at 0 riders. You receive m bookings, each “add v riders to every slot from l to r ”. Print the final rider count of every slot. **Constraints.** $n, m \leq 10^6$. **Target.** $O(n + m)$. **Edge cases.** a booking covering one slot; a booking covering everything; a negative v (a cancellation).

SOLUTION

Approach 1 — Loop over the range for every booking ($O(n \cdot m) = 10^{12}$ — hopeless)

Approach 2 — Difference array ($O(1)$ per booking, $O(n)$ at the end — optimal)

Record only the **edges** of each booking. Adding v at l and $-v$ at $r + 1$ means: “from here on, v more”, then “from here on, v less”. Take the prefix sum at the end and every slot gets its true value.

Difference array

JS

```
const makeDiff = (n) => {
  const d = new Array(n + 1).fill(0); // one extra slot for r+1
  return {
    add: (l, r, v) => { d[l] += v; d[r + 1] -= v; },
    finish: () => {
      const a = new Array(n);
      let run = 0;
      for (let i = 0; i < n; i++) { run += d[i]; a[i] = run; }
      return a;
    },
  };
};
```

Time $O(n + m)$ **Space** $O(n)$

Three bookings on 8 slots: `add(0,3,5)`, `add(2,5,-2)`, `add(7,7,100)`.

Measured output

TEXT

```

after 3 range updates: 5 5 3 3 -2 -2 0 100
brute result          : 5 5 3 3 -2 -2 0 100
match? yes
n=1: -7

```

Follow slot 2: the first booking added 5, the second subtracted 2, so $5 - 2 = 3$. The output says 3.

The idea. A prefix sum turns “many point marks” into “a filled range”. It is the exact reverse of the range-query trick, and the two are used together constantly.

TRAP

The array must have $n + 1$ slots. `d[r + 1]` with $r = n - 1$ writes to index n . In C++ that is an out-of-range write that corrupts memory. In JavaScript it is worse in a different way: `d[n] += v` on an array of length n **silently grows the array to length $n + 1$** and succeeds. Nothing complains, the `finish` loop stops at n anyway, and the code happens to work — until the day you write `for (let i = 0; i < d.length; i++)` and pick up a phantom extra slot. Size it correctly on purpose, not by accident.

Example 22 [SCB · pattern]

Problem. All values are positive. Find the **shortest** block whose sum is at least S . Return 0 if none exists. **Constraints.** $n \leq 2 \times 10^5$; values in $[1, 10^4]$; $S \leq 10^9$. **Target.** $O(n \log n)$ or better. **Edge cases.** no block is large enough; one element already reaches S ; the whole array is needed.

SOLUTION

Approach 1 — Every start, extend until the sum reaches S ($O(n^2)$ worst case)

Approach 2 — Prefix + binary search ($O(n \log n)$)

With all values positive, the prefix array is **strictly increasing**. That means we can binary search it. For a left end l , we want the smallest index j with $p[j] \geq p[l] + S$.

JavaScript has no built-in binary search, so use `lowerBound` from the **JS Toolkit** appendix. `lowerBound(p, x)` returns the index of the first value that is $\geq x$, and `p.length` when no such value exists.

lowerBound on the prefix array

JS

```

const { lowerBound } = require('./toolkit.js'); // JS Toolkit appendix

const shortestBinary = (a, S) => {
  const n = a.length;
  const p = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
  let best = Infinity;
  for (let l = 0; l < n; l++) {
    const j = lowerBound(p, p[l] + S); // first index with p[j] >= p[l] + S
    if (j <= n) best = Math.min(best, j - l);
  }
  return best === Infinity ? 0 : best;
};

```

Time $O(n \log n)$ **Space** $O(n)$

Measured output

TEXT

```

S=5 [] brute=0 binary=0
S=5 [5] brute=1 binary=1
S=5 [4] brute=0 binary=0
S=7 [2,3,1,2,4,3] brute=2 binary=2
S=4 [1,1,1,1] brute=4 binary=4
S=5 [1,1,1,1] brute=0 binary=0
S=1 [10] brute=1 binary=1
500 random cross-checks passed

```

Check $[2, 3, 1, 2, 4, 3]$ with $S = 7$: the block $[4, 3]$ sums to 7 with length 2. Nothing of length 1 reaches 7. So the answer is 2.

Searching the **whole** p is safe here, not just the part after l . Every value is positive, so p is strictly increasing and $p[l] + S > p[l]$, which means the answer index is always greater than l . If zeros were allowed you would have to search from $l + 1$.

A two-pointer window also solves this in $O(n)$ — but **only** because the values are positive. That is Chapter 3. The binary search version is worth knowing because it still works when you must answer for many different values of S .

Example 23 [Razer · pattern]

Problem. Return an array out where $out[i]$ is the product of every value except $a[i]$. Division is not allowed. **Constraints.** $n \leq 10^5$; values in $[-1000, 1000]$, zeros allowed; every answer is guaranteed to stay inside the exact range of a JavaScript number, $2^{53} - 1$. **Target.** $O(n)$ time, $O(1)$ extra space (the output array does not count). **Edge cases.** one zero in the array; two zeros; a single element; negative values.

SOLUTION

Approach 1 — For each i , multiply everything else ($O(n^2)$)

Approach 2 — Prefix products, then suffix products ($O(n)$ — optimal)

$out[i]$ is (product of everything to the left) $\times$ (product of everything to the right). Do one left-to-right pass writing the left product, then one right-to-left pass multiplying in the right product. The running product doubles as the storage, so no second array is needed.

Two passes, no division

JS

```

const productExceptSelf = (a) => {
  const n = a.length;
  const out = new Array(n).fill(1);
  let run = 1;
  for (let i = 0; i < n; i++) { out[i] = run; run *= a[i]; } // left side
  run = 1;
  for (let i = n - 1; i >= 0; i--) { out[i] *= run; run *= a[i]; } // right side
  return out;
};

```

Time $O(n)$ **Space** $O(1)$ beyond the output

The Python version:

Python

PYTHON

```

def product_except_self(a):
    n = len(a)
    out = [1] * n
    run = 1

```

```

for i in range(n):
    out[i] = run
    run *= a[i]
run = 1
for i in range(n - 1, -1, -1):
    out[i] *= run
    run *= a[i]
return out

```

Both were run against the $O(n^2)$ version:

Measured output

TEXT

```

[] -> []
[5] -> [1]
[1,2,3,4] -> [24,12,8,6]
[0,2,3] -> [6,0,0]
[0,0,3] -> [0,0,0]
[-1,2,-3] -> [-6,3,-2]
[100000,100000,3] -> [300000,300000,10000000000]
500 random cross-checks passed
20 copies of 1000, out[0] = 1e+57 safe? false

```

The zero cases come out right with no special handling at all: with one zero, only that position gets a non-zero answer; with two zeros, everything is zero. The line `[100000,100000,3]` reaches 10^{10} , which is still exact in JavaScript.

The idea. “Everything except me” always splits into “everything before me” and “everything after me”. That is prefix and suffix, whatever the operation is.

TRAP

The tempting solution is “compute the total product, then divide by `a[i]`”. It is banned here for a reason: it breaks on a zero (division by zero gives Infinity in JavaScript, not an error), and it silently gives wrong answers when two zeros exist.

TRAP

Read the last measured line. Twenty copies of 1000 make a product of 10^{57} , and JavaScript prints it as `1e+57` — a float, not an integer, and `Number.isSafeInteger` says `false`. The digits past the 16th are gone. Products run out of exact range **much** faster than sums do, so whenever a problem does not promise the answer fits, switch the running value to `BigInt`: `let run = 1n;` and `run *= BigInt(a[i])`.

Tier 3 — Insight, then the follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Amazon · pattern]

Problem. The array is **circular**: the block may wrap from the end back to the start. Find the largest block sum. The block must be non-empty and may not use any element twice. **Constraints.** $n \leq 10^5$; values in $[-10^4, 10^4]$. **Target.** $O(n)$. **Edge cases.** every value negative; one element; a block that uses the whole array.

| SOLUTION

There are exactly two shapes of answer.

1. **The block does not wrap.** That is plain Kadane (Example 15).
2. **The block wraps.** Then the elements it does **not** use form one ordinary block in the middle. So the wrapping answer is $\text{total} - (\text{smallest ordinary block sum})$.

Run Kadane twice, once for the maximum and once for the minimum, and take the better of the two shapes.

Circular maximum

JS

```
const kadaneMax = (a) => {
  let best = a[0], cur = a[0];
  for (let i = 1; i < a.length; i++) { cur = Math.max(a[i], cur + a[i]); best =
Math.max(best, cur); }
  return best;
};

const kadaneMin = (a) => {
  let best = a[0], cur = a[0];
  for (let i = 1; i < a.length; i++) { cur = Math.min(a[i], cur + a[i]); best =
Math.min(best, cur); }
  return best;
};

const circularMax = (a) => {
  if (a.length === 0) return 0;
  let total = 0;
  for (const x of a) total += x;
  const straight = kadaneMax(a);
  const minPart = kadaneMin(a);
  if (minPart === total) return straight; // every element is in the min block
  return Math.max(straight, total - minPart);
};
```

Time $O(n)$ **Space** $O(1)$

Checked against a brute force over all n starts and all n lengths, plus 800 random arrays:

Measured output

TEXT

```
[5] brute=5 fast=5
[-5] brute=-5 fast=-5
[1,-2,3,-2] brute=3 fast=3
[5,-3,5] brute=10 fast=10
[-3,-2,-4] brute=-2 fast=-2
[3,1,3,-9,2] brute=9 fast=9
[0,0] brute=0 fast=0
800 random cross-checks passed
```

Check $[5, -3, 5]$: the wrapping block is $5 + 5 = 10$, taken by removing the middle -3 . $\text{total} - \text{minPart} = 7 - (-3) = 10$. Correct.

Check $[-3, -2, -4]$: every value is negative, so the minimum block is the whole array and $\text{total} - \text{minPart} = 0$, which would mean an **empty** block. The guard `if (minPart === total)` catches exactly this and returns -2 .

TRAP

The wrapping index arithmetic in the brute force uses $a[(s + \text{len} - 1) \% n]$. That is safe because s and len are both non-negative. If you ever step **backwards** around a circle, write $((i \% n) + n) \% n$ — a bare $\%$ on a negative index gives a negative number and $a[-1]$ is undefined in JavaScript, not an error.

The follow-up

“Return the start and end indices too.” Track the index where `cur` restarted inside Kadane, and remember it whenever `best` improves. For the wrapping case, the answer is the **complement** of the minimum block, so it runs from $\text{minEnd} + 1$ to $\text{minStart} - 1 \pmod n$.

“What if the block must have length at most L ?” Then the two-Kadane trick fails, and you need the prefix-plus-deque method of Example 28.

Example 25 [Google · pattern]

Problem. Count the subarrays whose XOR equals k . **Constraints.** $n \leq 2 \times 10^5$; values in $[0, 2^{20})$. **Target.** $O(n)$. **Edge cases.** $k = 0$ (blocks that XOR to zero); an array of zeros; a single element equal to k .

SOLUTION

XOR is its own inverse: $x \oplus x = 0$. So a prefix XOR behaves exactly like a prefix sum, with “minus” replaced by “XOR”.

Define $P[0] = 0$ and $P[i + 1] = P[i] \oplus a[i]$. Then

$$\text{XOR of } a[l..r] = P[r + 1] \oplus P[l]$$

We want that to equal k . XOR both sides by $P[r + 1]$:

$$P[l] = P[r + 1] \oplus k$$

Same shape as Example 16, so the same code works with $-$ swapped for $\wedge$.

Prefix XOR + Map

JS

```
const xorCountFast = (a, k) => {
  const seen = new Map();
  seen.set(0, 1);
  let run = 0, cnt = 0;
  for (const x of a) {
    run ^= x;
    cnt += seen.get(run ^ k) ?? 0;
    seen.set(run, (seen.get(run) ?? 0) + 1);
  }
  return cnt;
};
```

Time $O(n)$ expected **Space** $O(n)$

Measured output

TEXT

```
k=0 [] brute=0 fast=0
k=4 [4] brute=1 fast=1
k=0 [4] brute=0 fast=0
k=5 [5,6,7,8,9] brute=2 fast=2
k=0 [0,0,0] brute=6 fast=6
k=0 [1,1,1,1] brute=4 fast=4
k=3 [3,3,3] brute=4 fast=4
500 random cross-checks passed
```

Check $[3, 3, 3]$ with $k = 3$: the blocks are $[3]$ three times, and $[3, 3, 3]$ whose XOR is $3 \oplus 3 \oplus 3 = 3$. That is 4.

TRAP

This works only because the constraint says values are below 2^{20} . JavaScript's `^` converts to a 32-bit signed integer first, so every prefix XOR also stays below 2^{20} and nothing is lost. If values could reach 2^{31} , run `^= x` would start producing **negative** numbers, and past 2^{32} the high bits would vanish entirely. Above 32 bits the only correct option is BigInt XOR, which is far slower — check the constraint before you type `^`.

The follow-up

“Now find the maximum XOR of any subarray.” Counting will not help; you need the **largest** $P[r + 1] \oplus P[l]$ over all pairs. Put every prefix XOR into a binary trie, one bit at a time from the top, and for each new prefix walk the trie choosing the opposite bit whenever it exists. That is $O(n \times 20)$ for 20-bit values.

“Why does a sliding window not work for XOR?” Because XOR is not monotonic: extending a window can make the value smaller **or** bigger, so there is no rule for when to shrink. Every subarray-XOR problem is a prefix-pair problem.

Example 26 [Microsoft · pattern]

Problem. A grid of $R \times C$ cells starts at 0. Apply m updates of the form “add v to every cell of the rectangle (r_1, c_1) to (r_2, c_2) ”. Print the final grid. **Constraints.** $R, C \leq 1000, m \leq 10^5$. **Target.** $O(RC + m)$. **Edge cases.** a rectangle of one cell; a rectangle touching the last row or column; a negative v .

SOLUTION

Approach 1 — Fill each rectangle ($O(mRC) = 10^8$ — dies)

Approach 2 — 2-D difference array ($O(m)$ per update, $O(RC)$ at the end)

In one dimension we marked two edges. In two dimensions we mark four corners, with the same inclusion–exclusion signs as the 2-D prefix query.

Four corner marks

JS

```
// d is (R+1) x (C+1), built with Array.from so every row is its own array
const d = Array.from({length: R + 1}, () => new Array(C + 1).fill(0));

const addRect = (d, r1, c1, r2, c2, v) => {
  d[r1][c1]      += v;
  d[r1][c2 + 1]  -= v;
  d[r2 + 1][c1]  -= v;
  d[r2 + 1][c2 + 1] += v;
};
```

Then sweep the whole grid once with a 2-D prefix sum to turn the marks into values:

The final sweep

JS

```
const g = Array.from({length: R}, () => new Array(C).fill(0));
for (let i = 0; i < R; i++)
  for (let j = 0; j < C; j++) {
    const up    = i > 0 ? g[i-1][j] : 0;
    const left  = j > 0 ? g[i][j-1] : 0;
    const diag  = (i > 0 && j > 0) ? g[i-1][j-1] : 0;
    g[i][j] = d[i][j] + up + left - diag;
  }
```

Time $O(m + R C)$ **Space** $O(R C)$

The `i > 0 ? ... : 0` guards are needed in JavaScript for a different reason than in C++. `g[-1]` does not crash — it is simply undefined — and then `undefined[j]` **does** throw `TypeError: Cannot read properties of undefined`. So you get an exception, just one whose message points at the wrong thing. Write the guards.

Three updates on a 5×6 grid: `add(0,0,2,2,+1)`, `add(1,1,4,5,+10)`, `add(4,5,4,5,-3)`.

Measured output

TEXT

```
difference-array grid:
  1  1  1  0  0  0
  1 11 11 10 10 10
  1 11 11 10 10 10
  0 10 10 10 10 10
  0 10 10 10 10 7
match brute? yes
```

Read cell (1, 1): it is inside the first rectangle (+1) and the second (+10), so 11. Cell (4, 5) is inside the second (+10) and the third (−3), so 7. Both match.

The follow-up

“Now the updates and the queries are mixed — a query can come between two updates.” The difference array only works when **all** updates come first. Mixed updates and queries need a 2-D Fenwick tree, at $O(\log R \log C)$ each.

“Now it is three dimensions.” The same idea, with $2^3 = 8$ corner marks and alternating signs. In d dimensions it is 2^d marks, which is why nobody does this past three.

Example 27 [D. E. Shaw · pattern]

Problem. Count the subarrays whose sum lies in the range $[L, R]$. Values may be negative. **Constraints.** $n \leq 10^5$; values in $[-10^9, 10^9]$. **Target.** $O(n \log n)$. **Edge cases.** empty array; $L = R$ (Example 16 as a special case); every sum out of range.

SOLUTION

The sum of $a[l..r]$ is $p[r+1] - p[l]$, so we need

$$L \leq p[r+1] - p[l] \leq R \Leftrightarrow p[r+1] - R \leq p[l] \leq p[r+1] - L$$

For each right end, that is a **count of earlier prefixes inside a value range**. A Map cannot answer range questions, and JavaScript has no ordered map at all. A Fenwick tree (binary indexed tree) over the compressed prefix values can, in $O(\log n)$.

Fenwick tree of counts

JS

```
class Fenwick {
  constructor(n) { this.t = new Array(n + 1).fill(0); }
  add(i) { for (i++; i < this.t.length; i += i & -i) this.t[i]++; }
  upto(i) { let s = 0; for (i++; i > 0; i -= i & -i) s += this.t[i]; return s; }
  range(l, r) {
    if (r < l) return 0;
    return this.upto(r) - (l ? this.upto(l - 1) : 0);
  }
}
```

The count itself

JS

```

const { lowerBound, upperBound } = require('./toolkit.js'); // JS Toolkit
appendix

const countInRangeFast = (a, L, R) => {
  const n = a.length;
  const p = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];

  // coordinate compression: Set removes duplicates, sort orders them
  const sorted = [...new Set(p)].sort((x, y) => x - y);

  const bit = new Fenwick(sorted.length);
  let cnt = 0;
  bit.add(lowerBound(sorted, p[0]));
  for (let r = 1; r <= n; r++) {
    const lo = p[r] - R, hi = p[r] - L;
    cnt += bit.range(lowerBound(sorted, lo), upperBound(sorted, hi) - 1);
    bit.add(lowerBound(sorted, p[r]));
  }
  return cnt;
};

```

Time $O(n \log n)$ **Space** $O(n)$

Checked against the $O(n^2)$ brute force on hand tests and 800 random arrays:

Measured output

TEXT

```

[2,-1,3,-4,5] with L=1,R=4 : brute=8 fast=8
empty: 0
single in range: 1   out of range: 0
800 random cross-checks passed

```

The idea. Example 16 needed “how many earlier prefixes equal x ” — a Map. This problem needs “how many earlier prefixes lie between x and y ” — an **ordered** structure. Recognising which of the two questions you are being asked is the whole difficulty.

TRAP

`[...new Set(p)].sort((x, y) => x - y)` is the JavaScript idiom for “sort and remove duplicates”. The comparator is not optional: `new Set([10, 9, 1])` spread and sorted with a bare `.sort()` gives `[1, 10, 9]`, `lowerBound` then binary-searches a list that is not in order, and every count comes out wrong with no error. This is the chapter’s most expensive place to forget the comparator.

TRAP

Fenwick trees use `i & -i` to isolate the lowest set bit. That is a 32-bit operation in JavaScript, which is fine for array indexes but a reminder of the rule: bitwise operators here are 32-bit. The stored **values** go through `+`, not `&`, so they stay exact to $2^{53} - 1$.

The follow-up

“What if all the values are positive?” Then the prefix array is increasing, and the answer is (number of subarrays with sum $\leq R$) minus (number with sum $\leq L - 1$), each computed with a two-pointer window in $O(n)$. Total $O(n)$, no Fenwick tree at all. The extra constraint buys a whole log factor.

“Do it without a Fenwick tree.” A merge sort that counts, while merging, how many left-half prefixes fall in each range. Same $O(n \log n)$, and it is the same counting-inversions technique.

Example 28 [Uber · pattern]

Problem. Find the largest block sum, but the block may contain at most L elements. **Constraints.** $n \leq 10^5$, $1 \leq L \leq n$; values may be negative. **Target.** $O(n)$. **Edge cases.** $L = 1$ (the largest single value); $L = n$ (plain Kadane); all values negative.

SOLUTION

Approach 1 — Every start, extend up to L ($O(nL)$ — 10^{10} at the limits)

Approach 2 — Prefix + a sliding minimum ($O(n)$ — optimal)

The sum ending at r with length at most L is $p[r] - p[l]$ where $r - L \leq l \leq r - 1$. So for each r we need the **minimum prefix inside a sliding window of width L** .

A monotonic deque keeps that minimum in $O(1)$ amortised: the deque holds indices whose prefix values increase from front to back. The front is always the window’s minimum.

JavaScript has no deque. An array’s push/pop are $O(1)$, but shift moves every remaining element, which turns this $O(n)$ algorithm into $O(n^2)$. Use the toolkit Deque, which keeps a head index instead of moving anything.

Prefix + monotonic deque

JS

```
const { Deque } = require('./toolkit.js'); // JS Toolkit appendix

const bestAtMostL = (a, L) => {
  const n = a.length;
  if (n === 0 || L <= 0) return null;
  const p = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];

  const dq = new Deque(); // holds indices into p
  let best = -Infinity;
  dq.push(0);
  for (let r = 1; r <= n; r++) {
    while (dq.size && dq.front() < r - L) dq.shift(); // too far left
    best = Math.max(best, p[r] - p[dq.front()]); // front = smallest
    while (dq.size && p[dq.back()] >= p[r]) dq.pop(); // p[r] is better
    dq.push(r);
  }
  return best;
};
```

Time $O(n)$ **Space** $O(n)$

Each index enters and leaves the deque once, so the inner whiles are $O(1)$ amortised.

Checked against the brute force for every L , plus 800 random arrays:

Measured output on [2,-1,4,-6,3,3,-2,5]

TEXT

```
L=1 brute=5 deque=5
L=2 brute=6 deque=6
L=3 brute=6 deque=6
L=4 brute=9 deque=9
L=5 brute=9 deque=9
```

```

L=6 brute=9 deque=9
L=7 brute=9 deque=9
L=8 brute=9 deque=9
all negative, L=2: -2 -2
single: 6
800 random cross-checks passed

```

TRAP

while (dq.size && ...) works because size is 0 when the deque is empty, and 0 is falsy. Do **not** write while (dq.size() && ...) — in the toolkit size is a getter, not a method, and calling it throws `TypeError: dq.size is not a function`. Read the toolkit’s signatures before you use it.

Check $L = 2$: the best block of length at most 2 is $[3, 3]$, which is 6. And $L = 4$ finds $[3, 3, -2, 5]$, which is 9.

The follow-up

“Why a deque and not a heap?” A heap gives the minimum in $O(\log n)$ but cannot cheaply remove a value that has left the window. The deque throws away, forever, any index that a newer index beats — because an older, larger prefix can never again be the answer while a newer, smaller one is in the window. That is the invariant to say out loud.

“Now the block must have length at least L .” Easier: keep a running minimum of $p[0..r - L]$ as a single variable, no deque needed. One-sided windows do not need one.

Example 29 [Goldman Sachs · pattern]

Problem. Range sums on an array of n values, but now some queries **change** a single value. Mixed in any order: q operations, each either “add v to $a[i]$ ” or “sum of $a[l..r]$ ”. **Constraints.** $n, q \leq 2 \times 10^5$. **Target.** $O((n + q) \log n)$. **Edge cases.** an update at index 0; an update at the last index; a query right after an update.

SOLUTION

Why the chapter’s main tool breaks. Changing $a[i]$ changes $p[j]$ for **every** $j > i$. Rebuilding is $O(n)$ per update, so q updates cost $O(nq) = 4 \times 10^{10}$.

Structure	Build	Query	Point update
Plain array	$O(1)$	$O(n)$	$O(1)$
Prefix sums	$O(n)$	$O(1)$	$O(n)$ — rebuild
Fenwick tree	$O(n \log n)$	$O(\log n)$	$O(\log n)$

A Fenwick tree keeps partial sums over power-of-two sized blocks, so one update touches only $\log_2 n$ of them.

Fenwick tree of sums

JS

```

class FenwickSum {
  constructor(n) { this.n = n; this.t = new Array(n + 1).fill(0); }
  add(i, v) { for (i++; i <= this.n; i += i & -i) this.t[i] += v; }
  upto(i)   { let s = 0; for (i++; i > 0; i -= i & -i) s += this.t[i]; return s; }
  range(l, r) { return this.upto(r) - (l ? this.upto(l - 1) : 0); }
}

```

Time $O(\log n)$ per update and per query **Space** $O(n)$

Run on the chapter’s array $[4, -2, 7, 1, 0, -5, 3]$, then $a[3] += 10$:

Measured output

TEXT

```

sum(0,6) = 8
sum(2,4) = 8
after a[3] += 10
sum(0,6) = 18
sum(2,4) = 18
sum(3,3) = 11
brute sum(2,4) = 18
200000 updates + 200000 queries: 23 ms, checksum positive

```

Twenty-three milliseconds for 400,000 mixed operations. The prefix-rebuild plan would need about 4×10^{10} steps.

A `class` is the right shape here, and one of the few places in this book where it is. The rule of thumb: write plain functions, and reach for a `class` only when the problem is a data structure with state that many operations share — a Fenwick tree, a heap, a DSU.

The follow-up

“Now the updates are ‘add v to every index in $[l, r]$ ’ and the queries are single points.” Flip it: keep a Fenwick tree over the **difference array**. A range update becomes two point updates, and a point query becomes a prefix sum.

“Now both the updates and the queries are ranges.” Two Fenwick trees, or a segment tree with lazy propagation. Say the phrase “range update, range query — that is lazy propagation” and the interviewer will move on.

“Now it is a stream and you cannot store the array.” Then arbitrary (l, r) is impossible. You can keep a running total and answer “everything so far” in $O(1)$. Naming what became impossible is part of the answer.

Dry run — counting subarrays with sum k

One problem, every step shown. Array $a = [3, 4, -7, 1, 3, 3, 1, -4]$, target $k = 7$.

The rule: at each step, add $a[i]$ to `run`, then look up how many earlier prefixes had the value `run - k`, then record `run` itself.

i	a[i]	run	run - k	found	cnt	map after this step
0	3	3	-4	0	0	0:1 3:1
1	4	7	0	1	1	0:1 3:1 7:1
2	-7	0	-7	0	1	0:2 3:1 7:1
3	1	1	-6	0	1	0:2 1:1 3:1 7:1
4	3	4	-3	0	1	0:2 1:1 3:1 4:1 7:1
5	3	7	0	2	3	0:2 1:1 3:1 4:1 7:2
6	1	8	1	1	4	0:2 1:1 3:1 4:1 7:2 8:1
7	-4	4	-3	0	4	0:2 1:1 3:1 4:2 7:2 8:1

This is the real printed trace of the program, not a hand sketch.

Step 1 in slow motion. `run` becomes $3 + 4 = 7$. We look for `run - k = 7 - 7 = 0`. The map holds `0:1` — that entry is the empty prefix we seeded. So one subarray ends here with sum 7, namely $a[0..1] = [3, 4]$. `cnt` becomes 1.

Step 5 in slow motion. `run` becomes 7 again. We look for 0, and the map now holds `0:2` — the empty prefix, and the prefix that ended at index 2 (where the running total returned to 0). So **two** subarrays end at index 5 with sum 7:

- from the empty prefix: $a[0..5] = [3, 4, -7, 1, 3, 3]$, which sums to 7
- from the prefix ending at index 2: $a[3..5] = [1, 3, 3]$, which sums to 7

cnt jumps from 1 to 3. This is why the map stores **counts** and not just “seen or not”.

Step 6. run is 8, and we look for 1. The map has 1:1, from the prefix ending at index 3. That gives $a[4..6] = [3, 3, 1]$, which sums to 7. cnt becomes 4.

The four answers, listed. $[3, 4]$, $[3, 4, -7, 1, 3, 3]$, $[1, 3, 3]$, $[3, 3, 1]$.

TRAP

Notice that $a[2] = -7$ made the running total go **down**. A sliding window would have no idea when to shrink. This is exactly why negative values force the prefix-plus-map method.

Practice

PRACTICE · TIER 1 target: 30 min

1. Build the prefix array of $[2, 2, 2, 2, 2]$ and use it to find the sum of $a[1..3]$.
2. For $a = [5, -3, 8]$, write out p and then compute $\text{sum}(0, 2)$, $\text{sum}(1, 1)$ and $\text{sum}(2, 2)$.
3. An array has $n = 10^5$ values, each up to 10^5 . What is the largest possible total, and is a plain JavaScript number still exact there?
4. Given a and a fixed value t , describe in two sentences how to answer “how many values in $a[l..r]$ equal t ” in $O(1)$ per query.
5. Find the largest sum of 3 consecutive values in $[1, -2, 5, 5, -1, 4]$ by hand.
6. Find an equilibrium index of $[3, 1, 5, 2, 2]$ by hand, showing the left and right sums.
7. Rotate $[10, 20, 30, 40]$ right by 3 using the three-reversal method, showing each reversal.
8. Use Kadane by hand on $[4, -5, 6, -1, 2]$, writing cur and best at every step.
9. In $[1, 2, 3]$ with $k = 3$, list every subarray with sum 3 and say why `seen.set(0, 1)` is needed to find them all.
10. An array of $1..n + 1$ is missing one value and $n = 10^6$. Why is the XOR method safer than the sum method?

PRACTICE · TIER 2 target: 35 min

1. Count the blocks of $[4, 2, 6, 3]$ whose sum is divisible by 3, by hand, using the remainder-counting method. Show the remainder of each prefix.
2. For $[1, 0, 0, 1, 1, 0]$, count the blocks with equal 0s and 1s using the $+1/-1$ trick. Show the running sum.
3. Bookings on 6 slots: `add(0, 2, +3)`, `add(1, 4, +2)`, `add(5, 5, -1)`. Write the difference array and then the final slot values.
4. A 3×3 grid holds all 1s. Write its 2-D prefix table and then use the four-term formula to get the sum of the rectangle from $(1, 1)$ to $(2, 2)$.
5. Find the longest block of $[2, -2, 2, -2, 2]$ with sum 2 and explain why the map must keep the **earliest** index.
6. All values are positive. Find the shortest block of $[1, 4, 2, 1, 5]$ with sum at least 7.
7. Compute “product of all except self” for $[2, 0, 4]$ by hand, with the prefix and suffix passes written out.

PRACTICE · TIER 3 target: 40 min

1. For the circular array $[4, -1, -2, 5]$, find the largest wrapping block sum and show both the Kadane maximum and the total-minus-minimum value.
2. Count the blocks of $[2, 3, 1]$ whose XOR is 1, by listing the prefix XORs.

3. You must answer 200,000 range-sum queries on an array that never changes, and 200,000 more on an array that changes after every query. Give the structure and the total complexity for each case.
4. Explain in three sentences why prefix sums cannot answer “maximum value in $a[l..r]$ ”.
5. A 2-D difference array is applied to a 1000×1000 grid with 10^5 rectangle updates. Give the total step count and compare it with the direct method.
6. Count the blocks of $[1, -1, 2]$ whose sum lies in $[0, 2]$ by listing all of them, then say which data structure a program would need for $n = 10^5$.

ANSWER KEY

Tier 1.

1. **$p = [0, 2, 4, 6, 8, 10]$; $\text{sum} = p[4] - p[1] = 8 - 2 = 6$.** Check: $2 + 2 + 2 = 6$.
2. **$p = [0, 5, 2, 10]$.** $\text{sum}(0, 2) = p[3] - p[0] = 10$. $\text{sum}(1, 1) = p[2] - p[1] = 2 - 5 = -3$. $\text{sum}(2, 2) = p[3] - p[2] = 10 - 2 = 8$.
3. **The largest total is $10^5 \times 10^5 = 10^{10}$, and yes, that is exact.** JavaScript numbers are exact to $2^{53} - 1 \approx 9 \times 10^{15}$, so 10^{10} has almost a million times of room left. Run this check every time: n times the largest value, compared with 9×10^{15} .
4. **Build $c[i+1] = c[i] + (a[i] == t ? 1 : 0)$, then the answer is $c[r+1] - c[l]$.** It works only because t is fixed for every query; a changing t would need one prefix array per value, or a different method.
5. **9, from $[5, 5, -1]$.** The windows are $1 - 2 + 5 = 4$, $-2 + 5 + 5 = 8$, $5 + 5 - 1 = 9$, $5 - 1 + 4 = 8$. The largest is 9.
6. **Index 2.** Left $= 3 + 1 = 4$; right $= 2 + 2 = 4$. They match.
7. **$[20, 30, 40, 10]$.** Reverse all: $[40, 30, 20, 10]$. Reverse the first 3: $[20, 30, 40, 10]$. Reverse the rest (one element): unchanged. Done.
8. **best = 7.** cur: 4, then $\max(-5, -1) = -1$, then $\max(6, 5) = 6$, then $\max(-1, 5) = 5$, then $\max(2, 7) = 7$. best: 4, 4, 6, 6, 7.
9. **$[3]$ and $[1, 2]$.** $[1, 2]$ starts at index 0, so its left prefix is $p[0] = 0$. Only the seeded entry `seen.set(0, 1)` can match it. Without the seed the answer would be 1.
10. **Because the expected total is $(10^6 + 1)(10^6 + 2)/2 \approx 5 \times 10^{11}$, and it grows like n^2 .** At $n = 10^6$ a JavaScript number still holds it exactly, but push n to 2×10^8 and the total passes $2^{53} - 1$ and starts drifting, silently. XOR never grows: the result always fits in the same number of bits as the inputs, so the method has no such ceiling — as long as the values stay under 2^{31} , which is where JavaScript’s 32-bit bitwise operators stop.

Tier 2.

1. **6 blocks.** Prefixes: 0, 4, 6, 12, 15. Remainders mod 3: 0, 1, 0, 0, 0. Group them: remainder 0 holds 4 prefixes, remainder 1 holds 1. Pairs inside each group: $\binom{4}{2} = 6$ and $\binom{1}{2} = 0$. Total 6. Listing them confirms it: $4 + 2 = 6$, $6, 6 + 3 = 9$, $4 + 2 + 6 = 12$, $4 + 2 + 6 + 3 = 15$, 3.
2. **7 blocks.** Map 1 to +1 and 0 to -1. The running sums, starting with the seeded 0, are 0, 1, 0, -1, 0, 1, 0. Count pairs of equal values: the value 0 appears 4 times giving $\binom{4}{2} = 6$ pairs, the value 1 appears twice giving $\binom{2}{2} = 1$ pair, and -1 appears once giving 0. Total $6 + 1 = 7$.
3. **$d = [3, 2, 0, -3, 0, -3, 1]$ and the slots are $[3, 5, 5, 2, 2, -1]$.** Step by step: add(0, 2, 3) sets $d[0] += 3$, $d[3] -= 3$; add(1, 4, 2) sets $d[1] += 2$, $d[5] -= 2$; add(5, 5, -1) sets $d[5] -= 1$, $d[6] += 1$. Prefix: 3, 5, 5, 2, 2, -1.
4. **p is the table with $p[i][j] = i*j$, and the rectangle sum is $p[3][3] - p[1][3] - p[3][1] + p[1][1] = 9 - 3 - 3 + 1 = 4$.** Check: the rectangle holds four 1s.
5. **Length 5, the whole array.** Prefixes are 0, 2, 0, 2, 0, 2. We need two prefixes differing by 2, as far apart as possible: $p[5] = 2$ and the earliest 0, which is $p[0]$. If the map had been overwritten with the later 0s, the answer would have come out as 1 or 3.
6. **Length 3.** No single value reaches 7 (the largest is 5). No pair reaches 7 either: the pairs are $1 + 4 = 5$, $4 + 2 = 6$, $2 + 1 = 3$, $1 + 5 = 6$. Length 3 works at once: $1 + 4 + 2 = 7$.
7. **$[0, 8, 0]$.** Prefix pass writes 1, 2, 0 (products of everything to the left). Suffix pass multiplies by 0, 4, 1. Results: $1 \times 0 = 0$, $2 \times 4 = 8$, $0 \times 1 = 0$.

Tier 3.

1. **9.** Total $= 4 - 1 - 2 + 5 = 6$. Kadane maximum (no wrap) $= 6$, the whole array. Kadane minimum $= -1 - 2 = -3$. Wrapping value $= \text{total} - \text{minimum} = 6 - (-3) = 9$, which is the block 5 then 4 going around the end. The answer is $\max(6, 9) = 9$.

2. **2 blocks.** Prefix XORs of [2, 3, 1] are $P = 0, 2, 1, 0$. We need pairs with $P_i \oplus P_j = 1$. Pair $(P_0, P_2) = (0, 1)$ gives the block [2, 3], whose XOR is $2 \oplus 3 = 1$. Pair $(P_2, P_3) = (1, 0)$ gives the block [1]. No other pair works.
3. **Static: prefix sums, $O(n + q)$. Changing: a Fenwick tree, $O((n + q) \log n)$.** Measured earlier: 400,000 mixed Fenwick operations took 23 ms in Node.
4. **Because prefix sums work by subtracting away the unwanted left part, and maximum has no inverse.** Knowing $\max(a[0..r])$ and $\max(a[0..l - 1])$ tells you nothing about $\max(a[l..r])$: the overall maximum may sit in the discarded part. Range maximum needs a sparse table or a segment tree.
5. $10^5 \times 4 + 10^6 = 1.4 \times 10^6$ steps, against $10^5 \times 10^6 = 10^{11}$ for the direct method. Four corner marks per update, then one sweep of the million cells. That is about 70,000 times less work.
6. **5 blocks.** All six subarrays and their sums: [1] = 1 ✓, [1, -1] = 0 ✓, [1, -1, 2] = 2 ✓, [-1] = -1 ✗, [-1, 2] = 1 ✓, [2] = 2 ✓. Five are inside [0, 2]. For $n = 10^5$ with negative values a program needs a Fenwick tree over the compressed prefix values, $O(n \log n)$.

ONE-PAGE REVISION CARD

The one formula. $p[0] = 0$, $p[i + 1] = p[i] + a[i]$, and $\text{sum}(l..r) = p[r + 1] - p[l]$. The prefix array is built with `new Array(n + 1).fill(0)`, always, and has $n + 1$ slots.

The pair view. Every subarray is a pair of prefixes. So subarray questions become pair questions, and pair questions are Map questions.

Question	What to store	Cost
count subarrays with sum k	Map: prefix value $\rightarrow$ count	$O(n)$
longest subarray with sum k	Map: prefix value $\rightarrow$ earliest index	$O(n)$
count sums divisible by k	array of size k : remainder $\rightarrow$ count	$O(n + k)$
equal 0s and 1s	re-label to ± 1 , then sum = 0	$O(n)$
count subarrays with XOR k	Map of prefix XOR values	$O(n)$
count sums inside $[L, R]$	Fenwick over compressed prefixes	$O(n \log n)$
range add, read at the end	difference array	$O(n + m)$
rectangle sum in a grid	2-D prefix, four terms	$O(RC + q)$
range sum with point updates	Fenwick tree	$O(\log n)$ each
max in a range	not prefix sums — sparse table / Deque	—

The templates.

Prefix sum

JS

```
const p = new Array(n + 1).fill(0);
for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
const s = p[r + 1] - p[l];
```

Count subarrays with sum k

JS

```
const seen = new Map(); seen.set(0, 1);
let run = 0, cnt = 0;
for (const x of a) {
  run += x;
  cnt += seen.get(run - k) ?? 0;
  seen.set(run, (seen.get(run) ?? 0) + 1);
}
```

Difference array

JS

```
const d = new Array(n + 1).fill(0);
d[l] += v; d[r + 1] -= v; // per update
let run = 0;
for (let i = 0; i < n; i++) { run += d[i]; a[i] = run; } // once, at the end
```

Kadane

JS

```
let best = a[0], cur = a[0];
for (let i = 1; i < n; i++) { cur = Math.max(a[i], cur + a[i]); best =
Math.max(best, cur); }
```

The six JavaScript facts this chapter proved with a measurement.

numbers are doubles	exact to $2^{53} - 1 \approx 9 \times 10^{15}$. Multiply n by the largest value and compare, before you code. Past it, <code>BigInt</code> . Example 5.
<code>new Array(n)</code>	sparse — every slot is a hole, and <code>undefined + x</code> is <code>NaN</code> . Always <code>.fill(0)</code> .
<code>.fill(row)</code> for a grid	puts the same row in every slot. Use <code>Array.from({length: R}, () => new Array(C).fill(0))</code> . Example 20.
<code>[...grid]</code> is shallow	copies the outer array only. A grid needs <code>grid.map(r => [...r])</code> .
<code>-7 % 3</code> is <code>-1</code>	not 2. Write <code>((run % k) + k) % k</code> . Example 17.
bitwise is 32-bit	<code>^</code> , <code>&</code> , <code><<</code> all truncate to 32 bits. Fine below 2^{31} ; <code>BigInt</code> above. Examples 13 and 25.

Top 6 traps.

1. `p[r + 1] - p[l]`, never `p[r] - p[l]`. Test with a 3-element array before you trust it.
2. Missing `seen.set(0, 1)`. You lose every subarray that starts at index 0.
3. `map.get(x)` returns `undefined`, not `0`. Without `?? 0` the running count becomes `NaN` and stays `NaN`, with no error to tell you where.
4. Counting wants the **count** of each prefix; longest wants the **earliest index**. Do not mix them up, and use `.has()` not truthiness, because index 0 is falsy.
5. Negative remainders: write `((run % k) + k) % k`.
6. A sliding window only works when the values are non-negative. One negative value and you must switch to prefix plus Map.

Two Pointers & Sliding Window

Turn a double loop into a single walk.

Pattern in one page

An array question with two nested loops is almost always $O(n^2)$. For $n = 10^5$ that is ten billion steps. Too slow. The fix, very often, is to keep **two indexes** and move each of them forward only. Each index moves at most n times, so the whole scan is $O(n)$.

There are three shapes. Learn to tell them apart in ten seconds.

THE THREE SHAPES

Shape A — opposite ends. Start $i = 0, j = a.length - 1$. Move them towards each other. Use when the array is **sorted**, or when the answer depends on a pair taken from the two ends (pair sum, palindrome, water between two walls, reverse).

Shape B — fixed window of size k . Add the new element on the right, drop the element that fell off the left. The window size never changes.

Shape C — variable window (the real sliding window). $right$ always moves forward. $left$ moves forward only while the window is **bad**. The window is a valid answer the moment the inner while stops.

The whole family shares one invariant: **no index ever moves backwards**. If you ever write $left--$ or $right--$, you have left the pattern.

Template A — two ends

Opposite ends, sorted array

JS

```
let i = 0, j = a.length - 1;
while (i < j) {
  // decide using a[i] and a[j]
  if (needBigger) i++;
  else if (needSmaller) j--;
  else { /* found it */ }
}
```

Template B — fixed window of size k

Fixed window: build once, then slide

JS

```
let sum = 0;
for (let i = 0; i < k; i++) sum += a[i]; // first window
let best = sum;
for (let i = k; i < n; i++) {
  sum += a[i] - a[i - k]; // gain right, lose left
  best = Math.max(best, sum);
}
```

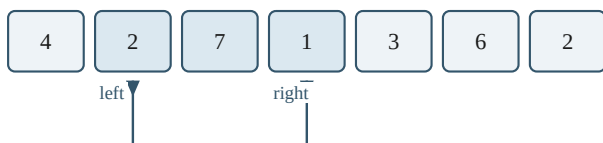
Template C — variable window

```

let left = 0;
for (let right = 0; right < n; right++) {
  add(a[right]); // window is now [left .. right]
  while (windowIsBad()) {
    remove(a[left]);
    left++;
  }
  best = Math.max(best, right - left + 1); // longest-valid problems
}

```

For **longest valid** problems the answer is read **after** the while. For **shortest valid** problems the while condition flips to “window is **good**”, and the answer is read **inside** the while, just before you shrink. Get this backwards and every test case fails.



Shape C: right marches on, left only chases. The shaded cells are the window.

When to reach for it

You see this in the question	Reach for
sorted array, find a pair / triple	Shape A, after sorting
“every window of length k”	Shape B
longest / shortest subarray with some rule	Shape C
“at most K distinct”, “at most k changes”	Shape C with a Map of counts
answer needs window max and min	Shape C with two monotonic deques
array has negative numbers and you need an exact sum	not a window — use prefix sums plus a Map

TRAP

Sliding window for **sum** problems only works when every value is positive (or zero). With negatives, growing the window can make the sum smaller, so shrinking is no longer safe. You will see a worked counter-example in Example 19.

Three JavaScript facts you must know before you start

TRAP

arr.sort() sorts **LEXICOGRAPHICALLY, as text**. Tested in Node: `[10, 9, 1].sort()` returns `[1, 10, 9]`, because `"10" < "9"` as strings. Numbers **always** need a comparator: `[10, 9, 1].sort((a, b) => a - b)` returns `[1, 9, 10]`. This chapter sorts in four different problems. Every one of them uses the comparator, and so must you.

TRAP

Every JavaScript number is a double. Whole numbers are exact only up to `Number.MAX_SAFE_INTEGER`, which is 9007199254740991, about 9×10^{15} . Tested in Node: `123456789 * 123456789` prints 15241578750190520, but the true answer ends in 521. It is silently wrong by one. The fix is `BigInt`: `BigInt(123456789) * BigInt(123456789)` gives 15241578750190521n, exactly. Use it whenever a product or a sum can pass 9×10^{15} .

Use Map, not a plain object, for counting. A plain object turns every key into a string, so the number 1 and the text "1" collide. A Map keeps the type, keeps insertion order, and is faster when you add and delete a lot — which is exactly what a sliding window does.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Reverse an array in place. You may not use a second array. Constraints: $0 \leq n \leq 10^5$. Target $O(n)$ time, $O(1)$ extra space. Edge cases: empty array, single element.

SOLUTION

Swap the two ends, then step inwards. JavaScript gives you a one-line swap through array destructuring.

Reverse with two pointers

JS

```
function reverseArr(a) {
  let i = 0, j = a.length - 1;
  while (i < j) {
    [a[i], a[j]] = [a[j], a[i]];
    i++;
    j--;
  }
}
```

Tested: [3,8,1,9,4] becomes [4,9,1,8,3]. [7] stays [7]. [] stays []. [1,2,3,4] becomes [4,3,2,1].

Time $O(n)$ **Space** $O(1)$

$n/2$ swaps, so the loop runs $n/2$ times.

[a[i], a[j]] = [a[j], a[i]] builds a tiny temporary array, then unpacks it. It is clear and it is what interviewers expect in JavaScript, so use it.

Answer: the loop stops when $i \geq j$, which also handles $n = 0$ and $n = 1$

Example 2 [warm-up]

Check whether a string reads the same forwards and backwards. Constraints: $0 \leq |s| \leq 10^5$. Target $O(n) / O(1)$. Edge cases: empty string (say yes), one character (say yes).

SOLUTION

Palindrome check

JS

```
function isPalindrome(s) {
  let i = 0, j = s.length - 1;
  while (i < j) {
    if (s[i] !== s[j]) return false;
    i++;
    j--;
  }
  return true;
}
```

Tested: "racecar" gives true, "abca" gives false, "" gives true, "z" gives true, "abba" gives true.

Time $O(n)$ **Space** $O(1)$

SHORTCUT

`s[i]` works on a JavaScript string and costs $O(1)$. Do **not** write `s.split("").reverse().join("") === s` — it is $O(n)$ **space** and the interviewer will ask you to do better.

Example 3 [warm-up]

Move every zero to the end of the array. The order of the non-zero values must not change. Constraints: $0 \leq n \leq 10^5$, values may be negative. Target $O(n) / O(1)$. Edge cases: all zeros, no zeros.

SOLUTION

This is the **slow** / **fast** pair. read looks at every slot. write marks where the next non-zero belongs.

Move zeros, keep order

JS

```
function moveZeros(a) {
  let write = 0;
  for (let read = 0; read < a.length; read++) {
    if (a[read] !== 0) {
      [a[write], a[read]] = [a[read], a[write]];
      write++;
    }
  }
}
```

Tested: `[0,5,0,3,12,0,7]` becomes `[5,3,12,7,0,0,0]`. `[0,0,0]` stays `[0,0,0]`. `[4,-1,2]` stays `[4,-1,2]`.

Time $O(n)$ **Space** $O(1)$

SHORTCUT

write is never ahead of read. So the swap either swaps a value with itself (no zeros seen yet) or moves a non-zero forward into the first zero slot. Order is safe.

Example 4 [warm-up]

Return the sum of every window of length k . Constraints: $1 \leq k \leq n \leq 10^5$. Target $O(n)$. Edge cases: $k = n$, $k > n$ (return nothing), negative values.

SOLUTION

Approach 1 — Recompute each window ($O(n \cdot k)$)

For each start, add k values. With $n = 10^5$ and $k = 5 \times 10^4$ that is five billion additions.

Approach 2 — Slide ($O(n)$ — optimal)

Two neighbouring windows differ by exactly two values.

Sums of all windows of length k

JS

```
function windowSums(a, k) {
  const out = [];
  const n = a.length;
  if (k <= 0 || k > n) return out;
  let sum = 0;
  for (let i = 0; i < k; i++) sum += a[i];
  out.push(sum);
```

```

for (let i = k; i < n; i++) {
  sum += a[i];
  sum -= a[i - k];
  out.push(sum);
}
return out;
}

```

Tested: `windowSums([2,1,5,1,3,2], 3)` returns `[8,7,9,6]`. `windowSums([2,1], 5)` returns `[]`. `windowSums([-3,4,-1], 1)` returns `[-3,4,-1]`.

Time $O(n)$ **Space** $O(n)$

$O(1)$ extra if you only need the best window.

TRAP

Check $k > n$ **before** the first loop. Without that guard the first loop reads past the end, `a[i]` gives undefined, and `sum` becomes NaN — which then compares false against everything and hides the bug.

Example 5 [warm-up]

Merge two sorted arrays into one sorted array. Constraints: $0 \leq n, m \leq 10^5$. Target $O(n + m)$. Edge cases: one array empty, both arrays full of the same value.

SOLUTION

Classic merge

JS

```

function mergeSorted(a, b) {
  const out = [];
  let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) out.push(a[i++]);
    else out.push(b[j++]);
  }
  while (i < a.length) out.push(a[i++]);
  while (j < b.length) out.push(b[j++]);
  return out;
}

```

Tested: `[1,4,7]` and `[2,3,9,11]` give `[1,2,3,4,7,9,11]`. `[]` and `[2,3]` give `[2,3]`. `[5,5]` and `[5]` give `[5,5,5]`.

Time $O(n + m)$ **Space** $O(n + m)$

`<=` (not `<`) keeps equal values in their original order. That property is called **stability**, and it matters once you sort records by one field.

Example 6 [warm-up]

A sorted array may hold repeats. Keep one copy of each value, at the front, and return the new length. The tail of the array may hold anything. Constraints: $0 \leq n \leq 10^5$. Target $O(n) / O(1)$. Edge cases: empty, all values equal.

SOLUTION

Remove duplicates in place

JS

```
function removeDupSorted(a) {  
  if (a.length === 0) return 0;  
  let write = 1;  
  for (let read = 1; read < a.length; read++) {  
    if (a[read] !== a[write - 1]) {  
      a[write] = a[read];  
      write++;  
    }  
  }  
  return write;  
}
```

Tested: [1,1,2,2,2,5,9,9] returns 4 and the front becomes [1,2,5,9]. [] returns 0. [4,4,4] returns 1.

Time $O(n)$ **Space** $O(1)$

Tier 1 — the template problems

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

A **sorted** array of prices and a budget target. Is there a pair of prices that adds up to exactly target? Constraints: $0 \leq n \leq 10^5$. Target $O(n)$ time, $O(1)$ space. Edge cases: fewer than 2 elements; negative prices (a refund line).

SOLUTION

Approach 1 — Check every pair ($O(n^2)$, far too slow when n is 100000)

Brute force

JS

```
function pairSumBrute(a, target) {
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] + a[j] === target) return true;
  return false;
}
```

Time $O(n^2)$ Space $O(1)$

Approach 2 — A Set of what you have seen ($O(n)$ time but $O(n)$ space, and ignores the sorting)

Remember what you have seen

JS

```
function pairSumSet(a, target) {
  const seen = new Set();
  for (const x of a) {
    if (seen.has(target - x)) return true;
    seen.add(x);
  }
  return false;
}
```

Time $O(n)$ Space $O(n)$

Approach 3 — Two pointers from the ends ($O(n)$ time, $O(1)$ space — optimal)

Sum too small? The only way to grow it is to raise the small end, so $i++$. Sum too big? The only way to shrink it is to lower the big end, so $j--$. Nothing is ever skipped, because the value you throw away could not pair with **anything** that is still in range.

Two pointers on a sorted array

JS

```
function pairSumTwoPtr(a, target) {
  let i = 0, j = a.length - 1;
  while (i < j) {
    const s = a[i] + a[j];
    if (s === target) return true;
    if (s < target) i++;
  }
}
```

```

    else j--;
  }
  return false;
}

```

Tested on [1,3,4,7,11,15]: target 15 gives true from all three versions, target 6 gives false from all three. On [-8,-3,0,2,5]: target -3 gives true, target 100 gives false.

Time $O(n)$ **Space** $O(1)$

The unlocking idea: sorting turns “search everywhere” into “one direction is always wrong”, and that kills the inner loop.

SHORTCUT

Set.has is the JavaScript answer to “have I seen this value”. Do not reach for array.includes — that is a full scan, and it turns approach 2 back into $O(n^2)$.

Example 8 [Infosys · pattern]

Find the largest sum of any k consecutive elements. Constraints: $1 \leq k \leq n \leq 10^5$, values may be negative. Target $O(n)$. Edge cases: all values negative, $k = n$.

SOLUTION

Approach 1 — Add up each window $O(nk)$

Brute force

JS

```

function maxWindowBrute(a, k) {
  let best = -Infinity;
  for (let i = 0; i + k <= a.length; i++) {
    let s = 0;
    for (let j = i; j < i + k; j++) s += a[j];
    best = Math.max(best, s);
  }
  return best;
}

```

Time $O(nk)$ **Space** $O(1)$

Approach 2 — Slide the window $O(n)$ — optimal

Fixed window, one pass

JS

```

function maxWindowSlide(a, k) {
  const n = a.length;
  if (k <= 0 || k > n) return -Infinity;
  let sum = 0;
  for (let i = 0; i < k; i++) sum += a[i];
  let best = sum;
  for (let i = k; i < n; i++) {
    sum += a[i] - a[i - k];
    best = Math.max(best, sum);
  }
}

```

```
    return best;
}
```

Tested: [2, 1, 5, 1, 3, 2] with $k = 3$ gives 9 from both versions. All-negative [-4, -2, -7, -1] with $k = 2$ gives -6 from both. [5] with $k = 1$ gives 5.

Time $O(n)$ **Space** $O(1)$

TRAP

Starting with `best = 0` is the classic wrong answer here. If every value is negative the true answer is negative, and 0 is not a real window. Start from the first window, or from `-Infinity` — JavaScript has no `INT_MIN`, and `-Infinity` is the right idiom.

Example 9 [Accenture · pattern]

Length of the longest stretch of a string with no repeated character. Constraints: $0 \leq |s| \leq 10^5$. Target $O(n)$. Edge cases: empty string, all characters the same.

SOLUTION

Approach 1 — Test every substring $(O(n^3))$

Brute force

JS

```
function allUnique(s, i, j) {
    const seen = new Set();
    for (let t = i; t <= j; t++) {
        if (seen.has(s[t])) return false;
        seen.add(s[t]);
    }
    return true;
}
function lenNoRepeatBrute(s) {
    let best = 0;
    for (let i = 0; i < s.length; i++)
        for (let j = i; j < s.length; j++)
            if (allUnique(s, i, j)) best = Math.max(best, j - i + 1);
    return best;
}
```

Time $O(n^3)$ **Space** $O(n)$

Approach 2 — Window with a count Map $(O(n)$ — good)

The window is **bad** exactly when the character we just added appears twice. Shrink from the left until it appears once again.

Grow, then shrink

JS

```
function lenNoRepeatShrink(s) {
    const cnt = new Map();
    let left = 0, best = 0;
    for (let right = 0; right < s.length; right++) {
        const c = s[right];
        cnt.set(c, (cnt.get(c) || 0) + 1);
```

```

while (cnt.get(c) > 1) {
    cnt.set(s[left], cnt.get(s[left]) - 1);
    left++;
}
best = Math.max(best, right - left + 1);
}
return best;
}

```

Time $O(n)$ **Space** $O(\min(n, \text{alphabet}))$

Approach 3 — Jump left straight past the twin ($O(n)$ with no inner loop — optimal)

Instead of stepping left one at a time, remember where each character was last seen and jump.

Last-seen jump

JS

```

function lenNoRepeatJump(s) {
    const last = new Map();
    let left = 0, best = 0;
    for (let right = 0; right < s.length; right++) {
        const c = s[right];
        if (last.has(c) && last.get(c) >= left) left = last.get(c) + 1;
        last.set(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}

```

Tested (all three agree): "abcabcbb" gives 3, "bbbb" gives 1, "" gives 0, "pwwkew" gives 3, "abcdef" gives 6, "abba" gives 2.

Time $O(n)$ **Space** $O(\min(n, \text{alphabet}))$

TRAP

"abba" is the test that kills a careless jump version. When `right` reaches the final `a`, `last.get("a")` is 0 but `left` is already 2. Without the guard `last.get(c) >= left`, `left` would jump **backwards** to 1 and the answer becomes 3. Wrong: the true answer is 2.

The unlocking idea: a repeated character can only be fixed by moving `left` past its previous position, and that position is already known.

This is one of the chapter's signature problems, so here it is in Python too.

Python version of the optimal

PYTHON

```

def len_no_repeat(s):
    last = {}
    left = 0
    best = 0
    for right, c in enumerate(s):
        if c in last and last[c] >= left:
            left = last[c] + 1
        last[c] = right

```

```
    best = max(best, right - left + 1)
    return best
```

Run on the same inputs, Python prints 3 0 1 2 for "abcabcbb", "", "bbbb", "abba" — the same answers as the JavaScript.

Example 10 [Wipro · pattern]

Count the pairs (i, j) with $i < j$ whose sum is strictly less than **target**. Constraints: $0 \leq n \leq 10^5$, values may be negative, the array is **not** sorted. Target $O(n \log n)$. Edge cases: $n < 2$; all pairs qualify.

SOLUTION

Approach 1 — Every pair $O(n^2)$

Brute force

JS

```
function countPairsLessBrute(a, target) {
    let c = 0;
    for (let i = 0; i < a.length; i++)
        for (let j = i + 1; j < a.length; j++)
            if (a[i] + a[j] < target) c++;
    return c;
}
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Sort, then count in blocks $O(n \log n)$ — optimal

After sorting, if $a[i] + a[j] < \text{target}$, then **every** element between i and j also pairs with $a[i]$ under the target, because they are all no larger than $a[j]$. That is $j - i$ pairs in one step.

Count a whole block at a time

JS

```
function countPairsLessTwoPtr(input, target) {
    const a = [...input].sort((x, y) => x - y); // NEVER a plain .sort()
    let c = 0;
    let i = 0, j = a.length - 1;
    while (i < j) {
        if (a[i] + a[j] < target) {
            c += j - i;
            i++;
        } else {
            j--;
        }
    }
    return c;
}
```

Tested: [1,6,2,5,3] with target 7 gives 4 from both versions. [-2,0,1,3] with target 2 gives 4 from both. [5] gives 0, [] gives 0.

Time $O(n \log n)$ **Space** $O(n)$
the sort dominates; the copy is the $O(n)$ space

TRAP

Two JavaScript mistakes hide in this function, and both were run to prove the point. **One:** `.sort()` with no comparator is lexicographic — in Node, `[10, 9, 1].sort()` returns `[1, 10, 9]`. Always pass `(x, y) => x - y`. **Two:** `.sort()` rearranges the caller's array **in place**. Copying with `[...input]` first keeps the caller's data intact. Interviewers notice when you quietly destroy an input.

SHORTCUT

With $n = 10^5$ the answer can reach $\frac{n(n-1)}{2} \approx 5 \times 10^9$. That is fine in JavaScript — it stays far below `Number.MAX_SAFE_INTEGER` — but say the number out loud, because in C++ or Java it would overflow a 32-bit counter.

Example 11 [Capgemini · pattern]

All values are **positive**. Find the length of the **shortest** subarray whose sum is at least S . Return 0 if no subarray reaches S . Constraints: $0 \leq n \leq 10^5, 1 \leq a_i \leq 10^4$. Target $O(n)$. Edge cases: empty array; a single element already $\geq S$; total sum below S .

SOLUTION

Approach 1 — Start at every index ($O(n^2)$)

Brute force with an early break

JS

```
function minLenBrute(a, S) {
  let best = Infinity;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) {
      s += a[j];
      if (s >= S) { best = Math.min(best, j - i + 1); break; }
    }
  }
  return best === Infinity ? 0 : best;
}
```

Time $O(n^2)$ Space $O(1)$

Approach 2 — Prefix sums plus binary search ($O(n \log n)$)

With positive values the prefix array p is strictly increasing, so a binary search can find the first end point that reaches $p[i] + S$. JavaScript has no `lower_bound`, so write the four-line version — the book's toolkit ships one, and Chapter 4 explains it in full.

Binary search on the prefix array

JS

```
function minLenPrefixBS(a, S) {
  const n = a.length;
  const p = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
  let best = Infinity;
  for (let i = 0; i < n; i++) {
    const need = p[i] + S;
    let lo = i + 1, hi = n + 1; // first index with p[idx] >= need
    while (lo < hi) {
      const mid = lo + Math.floor((hi - lo) / 2);
      if (p[mid] < need) lo = mid + 1;
    }
  }
}
```

```

    else hi = mid;
  }
  if (lo <= n) best = Math.min(best, lo - i);
}
return best === Infinity ? 0 : best;
}

```

Time $O(n \log n)$ **Space** $O(n)$

Approach 3 — Sliding window ($O(n)$ time, $O(1)$ space — optimal)

This is a **shortest** problem, so the answer is recorded **inside** the shrink loop.

Shrink while the window is still good

JS

```

function minLenWindow(a, S) {
  let left = 0, best = Infinity, sum = 0;
  for (let right = 0; right < a.length; right++) {
    sum += a[right];
    while (sum >= S) {
      best = Math.min(best, right - left + 1);
      sum -= a[left];
      left++;
    }
  }
  return best === Infinity ? 0 : best;
}

```

Tested on [2,1,6,5,4]: $S = 9$ gives 2 from all three, $S = 100$ gives 0 from all three, $S = 6$ gives 1 from all three. [] with $S = 3$ gives 0. [7] with $S = 7$ gives 1.

Time $O(n)$ **Space** $O(1)$

The unlocking idea: because the values are positive, adding an element can only push the sum up and removing one can only push it down. That one-way behaviour is exactly what lets `left` move forward and never come back.

TRAP

`Math.floor((hi - lo) / 2)` — not `(hi - lo) / 2`. JavaScript division always produces a double, so without `Math.floor` your index becomes 3.5 and `p[3.5]` is undefined. Chapter 4 makes this its number-one rule.

Example 12 [Cognizant · pattern]

A sorted array may contain negatives. Return the squares of all values, sorted. Constraints: $0 \leq n \leq 10^5$, $|a_i| \leq 3 \times 10^7$. Target $O(n)$. Edge cases: all negative; a single element; squares that pass the safe-integer limit.

SOLUTION

Approach 1 — Square then sort ($O(n \log n)$)

The obvious way

JS

```

function sortedSquaresSort(a) {
  return a.map(x => x * x).sort((x, y) => x - y);
}

```

Time $O(n \log n)$ **Space** $O(n)$

Approach 2 — Fill from the back $(O(n) — \text{optimal})$

The **largest** square must sit at one of the two ends. Compare the ends, take the bigger, and write it into the last free slot.

Two pointers, writing backwards

JS

```
function sortedSquaresTwoPtr(a) {
  const n = a.length;
  const r = new Array(n);
  let i = 0, j = n - 1;
  for (let pos = n - 1; pos >= 0; pos--) {
    const li = a[i] * a[i], rj = a[j] * a[j];
    if (li > rj) { r[pos] = li; i++; }
    else { r[pos] = rj; j--; }
  }
  return r;
}
```

Tested: `[-7, -3, 0, 2, 5]` gives `[0, 4, 9, 25, 49]` from both versions. `[-2, -1]` gives `[1, 4]`. `[3]` gives `[9]`.

Time $O(n)$ **Space** $O(n)$
output array only

TRAP

The constraint says $|a_i| \leq 3 \times 10^7$ for a reason: $(3 \times 10^7)^2 = 9 \times 10^{14}$, still under `Number.MAX_SAFE_INTEGER`. Push past that and JavaScript silently rounds. Tested in Node: `123456789 * 123456789` gives `15241578750190520`, while the true value ends in 521.

If the interviewer widens the range, switch to `BigInt`. Only two lines change.

The `BigInt` version, for very large values

JS

```
function sortedSquaresBig(a) {
  const n = a.length;
  const r = new Array(n);
  let i = 0, j = n - 1;
  for (let pos = n - 1; pos >= 0; pos--) {
    const li = BigInt(a[i]) * BigInt(a[i]), rj = BigInt(a[j]) * BigInt(a[j]);
    if (li > rj) { r[pos] = li; i++; }
    else { r[pos] = rj; j--; }
  }
  return r;
}
```

Tested: `sortedSquaresBig([-2000000000, 1000000000])` returns `1000000000000000000n` and `4000000000000000000n`, both exact.

`BigInt` values print with a trailing `n` and cannot be mixed with ordinary numbers in arithmetic. Convert once at the boundary, as above, and keep the whole computation in one world.

Example 13 [TCS Digital · pattern]

Longest substring that uses at most K different characters. Constraints: $0 \leq |s| \leq 10^5$, $0 \leq K \leq 26$. Target $O(n)$. Edge cases: $K = 0$; K larger than the number of distinct characters.

SOLUTION

“At most K distinct” is the signature of Shape C with a count Map. The window is **bad** when the map holds more than K keys.

At most K distinct characters

JS

```
function longestKDistinct(s, K) {
  if (K <= 0) return 0;
  const cnt = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    cnt.set(c, (cnt.get(c) || 0) + 1);
    while (cnt.size > K) {
      const d = s[left];
      cnt.set(d, cnt.get(d) - 1);
      if (cnt.get(d) === 0) cnt.delete(d);
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Tested: ("aabbcc", 2) gives 4, ("abccadccacacaca", 3) gives 11, ("aaaa", 1) gives 4, ("abc", 0) gives 0, ("", 2) gives 0.

Time $O(n)$ Space $O(K)$

TRAP

You must delete the key when its count hits zero. If you only decrement, `cnt.size` keeps counting dead characters and the window never grows.

SHORTCUT

`(cnt.get(c) || 0) + 1` is the standard JavaScript “increment or start at one”, because `cnt.get` on a missing key returns `undefined`, and `undefined + 1` is `NaN`.

Example 14 [Adobe · pattern]

$h[i]$ is the height of the i -th vertical board on a line, one unit apart. Pick two boards. The water they hold is $\min(h_i, h_j) \times (j - i)$. Maximise it. Constraints: $1 \leq n \leq 10^5$, $0 \leq h_i \leq 10^5$. Target $O(n)$. Edge cases: $n = 1$ (answer 0); all boards the same height.

SOLUTION

Approach 1 — Try every pair ($O(n^2)$)

Brute force

JS

```
function maxWaterBrute(h) {
  let best = 0;
  for (let i = 0; i < h.length; i++)
```

```

    for (let j = i + 1; j < h.length; j++)
        best = Math.max(best, Math.min(h[i], h[j]) * (j - i));
    return best;
}

```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Squeeze from both ends ($O(n)$ — optimal)

Start as wide as possible. Width can now only shrink, so the only hope of a better answer is a taller **shorter** side. Moving the taller side in can never help, so move the shorter one.

Move the shorter board

JS

```

function maxWaterTwoPtr(h) {
    let i = 0, j = h.length - 1, best = 0;
    while (i < j) {
        best = Math.max(best, Math.min(h[i], h[j]) * (j - i));
        if (h[i] < h[j]) i++;
        else j--;
    }
    return best;
}

```

Tested: [1,6,3,2,8,4] gives 18 from both versions (boards at index 1 and 4: $\min(6, 8) \times 3 = 18$). [4,4] gives 4 from both. [9] gives 0. A stress case of 100000 boards of height 100000 gives 9999900000 — ten billion, which JavaScript holds exactly because it is far under 9×10^{15} .

Time $O(n)$ **Space** $O(1)$

The unlocking idea: prove that one of the two candidates can be thrown away without checking it. That proof is what converts $O(n^2)$ into $O(n)$.

Example 15 [Amazon · pattern]

List every **distinct** triple of values that adds up to zero. Constraints: $0 \leq n \leq 3000$. Target $O(n^2)$. Edge cases: many duplicates such as [0,0,0,0]; no triple exists.

SOLUTION

Approach 1 — Three loops and a Set of keys ($O(n^3)$)

JavaScript arrays cannot be compared by value, and a Set of arrays would store each one separately. Join the sorted triple into a string and use that as the key.

Brute force with de-duplication

JS

```

function tripletsBrute(a) {
    const n = a.length;
    const seen = new Set();
    const out = [];
    for (let i = 0; i < n; i++)
        for (let j = i + 1; j < n; j++)
            for (let k = j + 1; k < n; k++)
                if (a[i] + a[j] + a[k] === 0) {
                    const t = [a[i], a[j], a[k]].sort((x, y) => x - y);
                    const key = t.join(',');
                }
}

```

```

        if (!seen.has(key)) { seen.add(key); out.push(t); }
    }
    out.sort((p, q) => p[0] - q[0] || p[1] - q[1]);
    return out;
}

```

Time $O(n^3)$ **Space** $O(n^3)$

Approach 2 — Fix one, two-pointer the rest $(O(n^2)$ — optimal)

Sort. Fix $a[i]$. Now you need two values that sum to $-a[i]$ inside a sorted range — that is Example 7 again.

Sort plus two pointers

JS

```

function tripletsZero(input) {
    const a = [...input].sort((x, y) => x - y);
    const n = a.length;
    const out = [];
    for (let i = 0; i < n - 2; i++) {
        if (i > 0 && a[i] === a[i - 1]) continue;
        let j = i + 1, k = n - 1;
        while (j < k) {
            const s = a[i] + a[j] + a[k];
            if (s === 0) {
                out.push([a[i], a[j], a[k]]);
                j++;
                k--;
                while (j < k && a[j] === a[j - 1]) j++;
                while (j < k && a[k] === a[k + 1]) k--;
            } else if (s < 0) {
                j++;
            } else {
                k--;
            }
        }
    }
    return out;
}

```

Tested on $[-4, -1, -1, 0, 1, 2, 5]$: both versions return $[[-4, -1, 5], [-1, -1, 2], [-1, 0, 1]]$. $[0, 0, 0, 0]$ gives exactly one triple $[[0, 0, 0]]$. $[1, 2, 3]$ gives $[]$.

Time $O(n^2)$ **Space** $O(1)$
output not counted

TRAP

Three separate duplicate guards are needed: one for i , one for j , one for k . Drop any of them and $[0, 0, 0, 0]$ returns the same triple several times.

TRAP

$p[0] - q[0] || p[1] - q[1]$ is the JavaScript idiom for “sort by the first field, break ties on the second”. It works because 0 is falsy, so a tie falls through to the next comparison.

Example 16 [Infosys · pattern]

Rearrange an array so that every even number comes before every odd number. The order inside each group does not matter. Constraints: $0 \leq n \leq 10^5$. Target $O(n) / O(1)$. Edge cases: all odd; all even; negative values.

SOLUTION

Partition with two ends

JS

```
function evenBeforeOdd(a) {  
  let i = 0, j = a.length - 1;  
  while (i < j) {  
    if (a[i] % 2 === 0) i++;  
    else if (a[j] % 2 !== 0) j--;  
    else [a[i], a[j]] = [a[j], a[i]];  
  }  
}
```

Tested: [3,8,5,2,9,4,7] becomes [4,8,2,5,9,3,7] — every even value is left of every odd value. [1,3,5] is unchanged. [2,4] is unchanged.

Time $O(n)$ **Space** $O(1)$

$a[i] \% 2$ is negative for negative odd numbers in JavaScript ($-3 \% 2$ is -1), exactly as in C and Java. That is why the test is $!== 0$ and not $=== 1$.

Tier 2 — the same pattern, wearing a business suit

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Grab · pattern]

A driver's shift is recorded minute by minute: 1 means a ride was running, 0 means the car was idle. The company may pay for at most k idle minutes to be counted as “on duty”. What is the longest stretch of minutes that can be reported as fully on duty? Constraints: $0 \leq n \leq 10^5$, $0 \leq k \leq n$. Target $O(n)$. Edge cases: $k = 0$ and the shift is all idle; k larger than the number of idle minutes.

SOLUTION

“At most k zeros inside the window” is the rule. That is Shape C.

Longest run of 1s with at most k flips

JS

```
function longestRunWithKFlips(bit, k) {
  let left = 0, zeros = 0, best = 0;
  for (let right = 0; right < bit.length; right++) {
    if (bit[right] === 0) zeros++;
    while (zeros > k) {
      if (bit[left] === 0) zeros--;
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Tested: $([1, 1, 0, 1, 0, 1, 1, 1, 0], 2)$ gives 8, $([0, 0, 0], 0)$ gives 0, $([1, 1, 1], 5)$ gives 3, $([], 1)$ gives 0, $([0], 1)$ gives 1.

Time $O(n)$ **Space** $O(1)$

SHORTCUT

You do not need to know **which** zeros were flipped. Counting them is enough. Most window problems are like this: track a number, not a set.

Example 18 [Shopee · pattern]

A row of stalls, each selling one product type `stall[i]`. A shopper walks left to right and may carry at most **two** different product types. What is the largest number of consecutive stalls the shopper can cover? Constraints: $0 \leq n \leq 10^5$, types are arbitrary integers. Target $O(n)$. Edge cases: fewer than two distinct types; every stall different.

SOLUTION

This is Example 13 with $K = 2$, but the keys are numbers instead of characters — which is exactly why it must be a Map and not a plain object.

At most two distinct values

JS

```
function longestTwoKinds(stall) {
  const cnt = new Map();
```



```

let left = 0, best = 0;
for (let right = 0; right < stall.length; right++) {
  const v = stall[right];
  cnt.set(v, (cnt.get(v) || 0) + 1);
  while (cnt.size > 2) {
    const d = stall[left];
    cnt.set(d, cnt.get(d) - 1);
    if (cnt.get(d) === 0) cnt.delete(d);
    left++;
  }
  best = Math.max(best, right - left + 1);
}
return best;
}

```

Tested: [3,3,7,7,3,5,5,5] gives 5 (the stretch 3 3 7 7 3), [1,2,3,4] gives 2, [9,9,9] gives 3, [] gives 0.

Time $O(n)$ **Space** $O(1)$
the Map never holds more than 3 keys

TRAP

With a plain object, `obj[1]` and `obj["1"]` are the **same** slot, because object keys are always strings. If the stall types ever mix numbers and text, a plain object merges them and the count silently goes wrong. Map keeps the type.

Example 19 [Sea · pattern]

Daily order counts for a shop. How many **contiguous** day-ranges have a total of exactly K orders? Part (a): every day has at least 1 order. Part (b): refunds are allowed, so a day can be negative. Constraints: $0 \leq n \leq 10^5$, $|a_i| \leq 10^4$. Target $O(n)$. Edge cases: $K = 0$; an empty array.

SOLUTION

Approach 1 — All ranges ($O(n^2)$ — works for both parts)

Brute force

JS

```

function countSumKBrute(a, K) {
  let c = 0;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) {
      s += a[j];
      if (s === K) c++;
    }
  }
  return c;
}

```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Sliding window — part (a) only ($O(n)$)

Only valid when every value is positive

JS

```
function countSumKWindow(a, K) { // needs every a[i] > 0
  let sum = 0, c = 0, left = 0;
  for (let right = 0; right < a.length; right++) {
    sum += a[right];
    while (sum > K && left <= right) {
      sum -= a[left];
      left++;
    }
    if (sum === K) c++;
  }
  return c;
}
```

Time $O(n)$ **Space** $O(1)$

Approach 3 — Prefix sums in a Map — part (b) ($O(n)$, handles negatives)

If run is the sum of everything up to right, a range ending at right with sum K exists once for every earlier prefix equal to run - K.

Prefix sum plus a Map

JS

```
function countSumKPrefix(a, K) { // works with negatives
  const seen = new Map([[0, 1]]);
  let run = 0, c = 0;
  for (const x of a) {
    run += x;
    c += seen.get(run - K) || 0;
    seen.set(run, (seen.get(run) || 0) + 1);
  }
  return c;
}
```

Time $O(n)$ **Space** $O(n)$

Tested, and this is the important part. On [2, 1, 3, 1, 1, 2] with K = 4 all three give 3. On [1, 1, 1, 1] with K = 2 all three give 3. But on [3, -1, 1, 2] with K = 3 the brute force and the prefix Map both give 3, while the window version returns 2.

TRAP

The window misses [3, -1, 1]. Once -1 arrives the running sum drops, so a window that was shrunk earlier can never be grown back. That is why a sliding window needs non-negative values for sum problems. Say this out loud in the interview before you write any code.

SHORTCUT

`new Map([[0, 1]])` seeds the map with “one empty prefix of sum 0”. Without that seed you miss every range that starts at index 0.

Example 20 [Agoda · pattern]

Choose exactly three parcels whose total weight is as close as possible to a truck’s target. Return that total. Constraints: $3 \leq n \leq 3000$, weights may be negative (a credit note). Target $O(n^2)$. Edge cases: an exact hit; all weights equal.

SOLUTION

Approach 1 — All triples $O(n^3)$

Brute force

JS

```
function threeClosestBrute(w, target) {
  let best = null;
  for (let i = 0; i < w.length; i++)
    for (let j = i + 1; j < w.length; j++)
      for (let k = j + 1; k < w.length; k++) {
        const s = w[i] + w[j] + w[k];
        if (best === null || Math.abs(s - target) < Math.abs(best - target)) best = s;
      }
  return best;
}
```

Time $O(n^3)$ **Space** $O(1)$

Approach 2 — Sort, fix one, two-pointer the rest $O(n^2)$ — optimal

Closest triple

JS

```
function threeClosest(input, target) {
  const w = [...input].sort((x, y) => x - y);
  const n = w.length;
  let best = w[0] + w[1] + w[2];
  for (let i = 0; i < n - 2; i++) {
    let j = i + 1, k = n - 1;
    while (j < k) {
      const s = w[i] + w[j] + w[k];
      if (Math.abs(s - target) < Math.abs(best - target)) best = s;
      if (s === target) return s;
      if (s < target) j++;
      else k--;
    }
  }
  return best;
}
```

Tested: ([4,9,1,7,12], 20) gives 20 from both. ([-5,-2,0,3], 1) gives 1 from both. ([1,1,1], 100) gives 3.

Time $O(n^2)$ **Space** $O(n)$
the $O(n)$ space is the defensive copy

TRAP

Seed best with a **real** triple, or with null as the brute force does. Seeding with Infinity and then computing `Math.abs(Infinity - target)` gives Infinity, which compares false against itself and the first candidate is never recorded.

Example 21 [LINE MAN · pattern]

A row of n vouchers. You must take exactly k of them, and each pick must come from the front or the back of the row. Maximise the total value. Constraints: $1 \leq k \leq n \leq 10^5$, values may be negative. Target $O(n)$. Edge cases: $k = n$; all values negative.

SOLUTION

Approach 1 — Try every split $(O(k^2))$

Take front from the left and $k - \text{front}$ from the right, for every front from 0 to k .

Brute force over the split point

JS

```
function bestFromEndsBrute(v, k) {
  const n = v.length;
  let best = -Infinity;
  for (let front = 0; front <= k; front++) {
    const back = k - front;
    let s = 0;
    for (let i = 0; i < front; i++) s += v[i];
    for (let i = 0; i < back; i++) s += v[n - 1 - i];
    best = Math.max(best, s);
  }
  return best;
}
```

Time $O(k^2)$ Space $O(1)$

Approach 2 — Look at what you leave behind $(O(n) — \text{optimal})$

Whatever you take from the ends, what stays behind is one **contiguous block** of length $n - k$ in the middle. Maximising what you take is the same as minimising what you leave. And “smallest sum over all windows of a fixed length” is Template B.

Minimise the middle window

JS

```
function bestFromEnds(v, k) {
  const n = v.length;
  const total = v.reduce((s, x) => s + x, 0);
  const win = n - k;
  if (win === 0) return total;
  let sum = 0;
  for (let i = 0; i < win; i++) sum += v[i];
  let minSum = sum;
  for (let i = win; i < n; i++) {
    sum += v[i] - v[i - win];
    minSum = Math.min(minSum, sum);
  }
  return total - minSum;
}
```

Tested: $([8, 2, 9, 1, 7, 3], 3)$ gives 19 from both versions. $([8, 2, 9, 1, 7, 3], 6)$ gives 30 from both. $([-4, -1, -9], 1)$ gives -4 from both.

Time $O(n)$ Space $O(1)$

The unlocking idea: flip the question. “Pick from the two ends” is a disguise for “leave one window in the middle”.

SHORTCUT

`v.reduce((s, x) => s + x, 0)` is the JavaScript way to total an array. The `0` seed matters: reduce with no seed throws on an empty array.

Example 22 [DBS · pattern]

A queue of customers, `1` means premium and `0` means standard. One **swap** exchanges any two customers. What is the smallest number of swaps that puts all premium customers next to each other? Constraints: $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: zero or one premium customer (answer `0`); everyone premium.

SOLUTION

If there are `ones` premium customers, the final block has length exactly `ones`. So slide a window of that fixed length and find the one that **already** holds the most premium customers. Every remaining slot in that window costs one swap.

Fixed window of length ones

JS

```
function minSwapsToGroup(bit) {
  const ones = bit.reduce((s, b) => s + b, 0);
  if (ones <= 1) return 0;
  let have = 0;
  for (let i = 0; i < ones; i++) have += bit[i];
  let best = have;
  for (let i = ones; i < bit.length; i++) {
    have += bit[i] - bit[i - ones];
    best = Math.max(best, have);
  }
  return ones - best;
}
```

Tested: `[1,0,1,1,0,1,0,1]` gives 2, `[1,1,1]` gives 0, `[0,0]` gives 0, `[1]` gives 0, `[1,0,0,0,1]` gives 1.

Time $O(n)$ **Space** $O(1)$

SHORTCUT

Whenever a problem says “group them together”, first compute the **size** of the final group. It is usually fixed, and a fixed size means Template B.

Tier 3 — the ones with a follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

$h[i]$ is the height of a wall block. Rain falls. How much water is held between the blocks? Constraints: $0 \leq n \leq 2 \times 10^5$, $0 \leq h_i \leq 10^5$. Target $O(n)$ time, $O(1)$ space. Edge cases: empty array; a strictly falling profile (holds nothing).

SOLUTION

The key fact, and you must say it before writing code: the water above column i is $\min(\text{tallest to the left}, \text{tallest to the right}) - h_i$.

Approach 1 — Scan left and right for every column ($O(n^2)$)

Brute force

JS

```
function rainBrute(h) {
  const n = h.length;
  let total = 0;
  for (let i = 0; i < n; i++) {
    let L = 0, R = 0;
    for (let j = 0; j <= i; j++) L = Math.max(L, h[j]);
    for (let j = i; j < n; j++) R = Math.max(R, h[j]);
    total += Math.min(L, R) - h[i];
  }
  return total;
}
```

Time $O(n^2)$ Space $O(1)$

Approach 2 — Pre-compute the two max arrays ($O(n)$ time, $O(n)$ space)

Prefix and suffix maxima

JS

```
function rainPrefix(h) {
  const n = h.length;
  if (n === 0) return 0;
  const L = new Array(n), R = new Array(n);
  L[0] = h[0];
  for (let i = 1; i < n; i++) L[i] = Math.max(L[i - 1], h[i]);
  R[n - 1] = h[n - 1];
  for (let i = n - 2; i >= 0; i--) R[i] = Math.max(R[i + 1], h[i]);
  let total = 0;
  for (let i = 0; i < n; i++) total += Math.min(L[i], R[i]) - h[i];
  return total;
}
```

Time $O(n)$ Space $O(n)$

Approach 3 — Two pointers ($O(n)$ time, $O(1)$ space — optimal)

Here is the whole trick. If $h[i] \leq h[j]$, then the right side already has **something** at least as tall as $h[i]$. So the true limit for column i is decided by the left side alone, and `leftMax` is all you need. Symmetrically on the other side.

The $O(1)$ -space solution

JS

```
function rainTwoPtr(h) {
  let i = 0, j = h.length - 1;
  let leftMax = 0, rightMax = 0, total = 0;
  while (i < j) {
    if (h[i] <= h[j]) {
      leftMax = Math.max(leftMax, h[i]);
      total += leftMax - h[i];
      i++;
    } else {
      rightMax = Math.max(rightMax, h[j]);
      total += rightMax - h[j];
      j--;
    }
  }
  return total;
}
```

Tested (all three agree): `[3,0,2,0,4]` gives 7, `[5,4,3,2]` gives 0, `[]` gives 0, `[2,1,2]` gives 1. A stress case of 200000 columns, tall only at the two ends, gives 19999800000, and `Number.isSafeInteger` on that answer returns true, so no `BigInt` is needed here. All three were also cross-checked against each other on 400 random arrays.

Time $O(n)$ **Space** $O(1)$

This is a signature problem, so here it is in Python as well.

Python version

PYTHON

```
def trapped(h):
    i, j = 0, len(h) - 1
    lmax = rmax = total = 0
    while i < j:
        if h[i] <= h[j]:
            lmax = max(lmax, h[i]); total += lmax - h[i]; i += 1
        else:
            rmax = max(rmax, h[j]); total += rmax - h[j]; j -= 1
    return total
```

Python prints 7 0 0 1 on the same four inputs.

Follow-up the interviewer asks next: “the heights arrive as a stream, you cannot store them — now what?” Answer: you cannot. Water above a column depends on a block that may appear far in the future, so no single-pass, no-memory answer exists. You can, however, keep a **monotonic decreasing stack** of candidate left walls and settle water in layers as each new block arrives; that is $O(n)$ time and $O(n)$ worst-case memory. That stack version is built in Chapter 8.

Example 24 [Amazon · pattern]

Given a long string s and a short string $need$, find the **shortest** substring of s that contains every character of $need$, counting repeats. Return `""` if none exists. Constraints: $0 \leq |s| \leq 10^5$, $0 \leq |need| \leq 100$. Target $O(|s|)$. Edge cases: $need$ longer than s ; $need$ with repeated letters such as “aa”; empty inputs.

| SOLUTION

Approach 1 — Every start, extend until covered $O(n^3)$ with a Map rebuild each time

Brute force (correct, slow, good for testing)

JS

```
function minCoverBrute(s, need) {
  let best = '';
  for (let i = 0; i < s.length; i++) {
    for (let j = i; j < s.length; j++) {
      const have = new Map();
      for (let t = i; t <= j; t++) have.set(s[t], (have.get(s[t]) || 0) + 1);
      const want = new Map();
      for (const c of need) want.set(c, (want.get(c) || 0) + 1);
      let ok = true;
      for (const [c, k] of want) if ((have.get(c) || 0) < k) { ok = false; break; }
      if (ok) {
        if (best === '' || j - i + 1 < best.length) best = s.slice(i, j + 1);
        break;
      }
    }
  }
  return best;
}
```

Time $O(n^3)$ Space $O(|need|)$

Approach 2 — One window and a single counter $O(n)$ — optimal

The clever part is the want map. It starts as the required count of each character. Adding a character **decrements** it. A value that is still above zero before the decrement means that character was genuinely needed, so missing goes down. A value below zero simply means “we have spare copies”. One integer, missing, replaces the whole comparison of two maps.

Minimum covering window

JS

```
function minCover(s, need) {
  if (s.length === 0 || need.length === 0) return '';
  const want = new Map();
  for (const c of need) want.set(c, (want.get(c) || 0) + 1);
  let missing = need.length;
  let left = 0, bestLen = Infinity, bestStart = 0;
  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    const wc = want.get(c) || 0;
    if (wc > 0) missing--;
    want.set(c, wc - 1);
    while (missing === 0) {
      if (right - left + 1 < bestLen) {
        bestLen = right - left + 1;
        bestStart = left;
      }
      const d = s[left];
      want.set(d, want.get(d) + 1);
      if (want.get(d) > 0) missing++;
      left++;
    }
  }
}
```



```

    return bestLen == Infinity ? '' : s.slice(bestStart, bestStart + bestLen);
}

```

Tested (brute force and window agree): ("PQRSTAQB SRC", "ABC") gives "AQB SRC", ("bbaacx", "abc") gives "baac", ("xaybzabcq", "abc") gives "abc", ("aabbcc", "abc") gives "abbc", ("aaab", "ab") gives "ab", ("aa", "aa") gives "aa", ("a", "aa") gives "", ("", "a") gives "", ("xyz", "") gives "". They were also cross-checked on 300 random strings.

Time $O(n)$ **Space** $O(|\text{need}|)$

Python, for the same reason as above.

Python version

PYTHON

```

def min_cover(s, need):
    from collections import Counter
    if not s or not need:
        return ""
    want = Counter(need)
    missing = len(need)
    left = 0
    best = (float("inf"), 0)
    for right, c in enumerate(s):
        if want[c] > 0:
            missing -= 1
        want[c] -= 1
        while missing == 0:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left)
            want[s[left]] += 1
            if want[s[left]] > 0:
                missing += 1
            left += 1
    if best[0] == float("inf"):
        return ""
    return s[best[1]: best[1] + best[0]]

```

Python gives 'AQB SRC', '', 'baac' on the matching inputs.

Follow-up the interviewer asks next: “the alphabet is Unicode, not 26 letters.” Nothing changes. A JavaScript Map already accepts any key, which is why the Map version is better here than a 26-slot array. Time stays $O(n)$; space becomes $O(|\text{need}|)$. When the invariant is right, the data structure swap is free.

TRAP

missing must be compared with 0, never with want.size. Repeated characters in need (like "aa") need **two** copies, and only the counter version knows that.

Example 25 [Microsoft · pattern]

Longest subarray in which the largest and the smallest value differ by at most limit. Constraints: $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: limit = 0; empty array; all values equal.

SOLUTION

Approach 1 — Every start, track running min and max ($O(n^2)$)

Brute force

JS

```
function longestBoundedBrute(a, limit) {
  let best = 0;
  for (let i = 0; i < a.length; i++) {
    let lo = a[i], hi = a[i];
    for (let j = i; j < a.length; j++) {
      lo = Math.min(lo, a[j]);
      hi = Math.max(hi, a[j]);
      if (hi - lo <= limit) best = Math.max(best, j - i + 1);
      else break;
    }
  }
  return best;
}
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Window plus two monotonic deques ($O(n)$ — optimal)

The window rule needs $\max - \min$. A plain counter cannot give that. Keep two deques of **indexes**: **maxq** decreasing by value (its front is the window max) and **minq** increasing by value (its front is the window min). When left moves past an index, drop it from the front if it is there.

TRAP

JavaScript has no deque, and `array.shift()` has to move every remaining element down one slot. Measured in Node: pushing 200000 values and shifting them all off a plain array took **3156 ms**; the book's Deque did the same work in **15 ms**. Using a plain array here turns an $O(n)$ algorithm into $O(n^2)$. The Deque (JS Toolkit appendix) keeps a head index instead, so `shift` is $O(1)$ amortised.

Two deques hold the window max and min

JS

```
const { Deque } = require('./toolkit.js');

function longestBounded(a, limit) {
  const maxq = new Deque(), minq = new Deque(); // both hold indexes
  let left = 0, best = 0;
  for (let right = 0; right < a.length; right++) {
    while (maxq.size && a[maxq.back()] <= a[right]) maxq.pop();
    maxq.push(right);
    while (minq.size && a[minq.back()] >= a[right]) minq.pop();
    minq.push(right);
    while (a[maxq.front()] - a[minq.front()] > limit) {
      if (maxq.front() === left) maxq.shift();
      if (minq.front() === left) minq.shift();
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Tested (both agree): `([8,2,4,7], 4)` gives 2, `([10,1,2,4,7,2], 5)` gives 4, `([4,4,4], 0)` gives 3, `([], 3)` gives 0, `([5], 0)` gives 1. Cross-checked on 400 random arrays against the brute force.

Time $O(n)$ **Space** $O(n)$
each index enters and leaves each deque once

The unlocking idea: when the window rule needs an **extremum** instead of a **count**, upgrade the counter to a monotonic deque. Everything else in Template C stays the same.

Follow-up the interviewer asks next: “give me the answer for every limit from 0 to L.” The maximum valid length is non-decreasing in limit, which is exactly the monotone shape Chapter 4 lives on — binary search the answer instead of recomputing.

Example 26 [Uber · pattern]

Count the subarrays that contain **exactly** K distinct values. Constraints: $0 \leq n \leq 10^5$, $1 \leq K \leq n$. Target $O(n)$. Edge cases: K bigger than the number of distinct values (answer 0); all values equal.

SOLUTION

“Exactly K ” is not a sliding-window rule: when you add an element the distinct count can jump over K and shrinking is not forced. So **change the question**.

THE EXACTLY-K TRICK

$\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K - 1)$

“At most K ” is a window rule, because adding an element can only break it and shrinking always fixes it.

Counting every subarray that ends at `right` is the second half of the trick: once the window `[left..right]` is valid, so is every window that starts at `left+1`, `left+2`, ..., `right`. That is `right - left + 1` subarrays, added in one step.

At most K distinct, then subtract

JS

```
function atMostKDistinct(a, K) {
  if (K < 0) return 0;
  const cnt = new Map();
  let total = 0, left = 0;
  for (let right = 0; right < a.length; right++) {
    const v = a[right];
    cnt.set(v, (cnt.get(v) || 0) + 1);
    while (cnt.size > K) {
      const d = a[left];
      cnt.set(d, cnt.get(d) - 1);
      if (cnt.get(d) === 0) cnt.delete(d);
      left++;
    }
    total += right - left + 1;
  }
  return total;
}
const exactlyKDistinct = (a, K) => atMostKDistinct(a, K) - atMostKDistinct(a, K - 1);
```

Tested against a brute force: `([1,2,1,3,4], 2)` gives 5, `([1,2,1,3,4], 3)` gives 3, `([5,5,5], 1)` gives 6, `([], 1)` gives 0, `([1,2,1,3,4], 9)` gives 0. Cross-checked on 400 random arrays for every K from 1 to 4.

Time $O(n)$ **Space** $O(n)$
two passes, each $O(n)$

Follow-up the interviewer asks next: “count subarrays with sum exactly K ” — same trick shape, different engine: atMost on a sum needs positive values (Example 19), so with negatives you must fall back to prefix sums plus a Map.

Example 27 [Goldman Sachs · pattern]

Find the shortest subarray such that sorting **only that subarray** leaves the whole array sorted. Return its length, or 0 if the array is already sorted. Constraints: $0 \leq n \leq 10^5$. Target $O(n)$ time, $O(1)$ space. Edge cases: already sorted; strictly decreasing; all values equal.

SOLUTION

Approach 1 — Sort a copy and compare ($O(n \log n)$ time, $O(n)$ space)

Compare with the sorted copy

JS

```
function shortestUnsortedBrute(a) {
  const b = [...a].sort((x, y) => x - y);
  const n = a.length;
  let i = 0, j = n - 1;
  while (i < n && a[i] === b[i]) i++;
  if (i === n) return 0;
  while (j > i && a[j] === b[j]) j--;
  return j - i + 1;
}
```

Time $O(n \log n)$ **Space** $O(n)$

Approach 2 — Two sweeps, no sorting ($O(n)$ time, $O(1)$ space — optimal)

Sweep left to right keeping the running maximum. Any element **smaller** than that maximum is out of place, and the last such element is the right edge of the window. Sweep the other way with the running minimum to find the left edge.

Two sweeps

JS

```
function shortestUnsorted(a) {
  const n = a.length;
  if (n < 2) return 0;
  let runMax = a[0], right = -1;
  for (let i = 1; i < n; i++) {
    if (a[i] < runMax) right = i;
    else runMax = a[i];
  }
  let runMin = a[n - 1], left = -1;
  for (let i = n - 2; i >= 0; i--) {
    if (a[i] > runMin) left = i;
    else runMin = a[i];
  }
  if (right === -1) return 0;
  return right - left + 1;
}
```

Tested (both agree): [1,3,5,4,2,6,7] gives 4 (the block 3 5 4 2), [1,2,3] gives 0, [3,2,1] gives 3, [2,2,2] gives 0, [] gives 0, [9] gives 0. Cross-checked on 400 random arrays.

Time $O(n)$ **Space** $O(1)$

Follow-up the interviewer asks next: “do it in one pass.” You can: run both sweeps in the same for loop, one using index i and the other using index $n - 1 - i$. The code gets denser but the complexity class does not change, so say that out loud and only write it if they insist.

Example 28 [D. E. Shaw · pattern]

You may change at most k characters of a lowercase string. What is the longest block of **one repeated letter** you can end up with? Constraints: $0 \leq |s| \leq 10^5$, $0 \leq k \leq |s|$. Target $O(n)$. Edge cases: $k = 0$; all letters already equal; empty string.

SOLUTION

A window is valid when $(\text{window length}) - (\text{count of the most common letter in it}) \leq k$, because every other letter has to be changed.

Approach 1 — Every start ($O(n^2)$)

Brute force

JS

```
function longestAfterKChangesBrute(s, k) {
  let best = 0;
  for (let i = 0; i < s.length; i++) {
    const cnt = new Array(26).fill(0);
    let mx = 0;
    for (let j = i; j < s.length; j++) {
      cnt[s.charCodeAt(j) - 97]++;
      mx = Math.max(mx, cnt[s.charCodeAt(j) - 97]);
      if (j - i + 1 - mx <= k) best = Math.max(best, j - i + 1);
    }
  }
  return best;
}
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Window that never shrinks ($O(n)$ — optimal)

Notice the `if` instead of a `while`, and that `bestCount` is never lowered. Both look like bugs. They are not, and explaining why is the whole interview.

The non-shrinking window

JS

```
function longestAfterKChanges(s, k) {
  const cnt = new Array(26).fill(0);
  let left = 0, bestCount = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    const idx = s.charCodeAt(right) - 97;
    cnt[idx]++;
    bestCount = Math.max(bestCount, cnt[idx]);
    if (right - left + 1 - bestCount > k) {
      cnt[s.charCodeAt(left) - 97]--;
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

WHY THE STALE BESTCOUNT IS SAFE

We only ever want a **longer** answer. A window can beat the current best only if its most common letter has a **higher** count than `bestCount`. In that case `bestCount` is updated on the very step that count appears, so it is fresh exactly when it matters. If `bestCount` is stale it is too small, which makes the window look **worse** than it is, so the window slides. We never report a length we did not earn. The window size therefore never decreases — it only slides forward — and the final best is correct.

Tested (both agree): ("aabbba", 2) gives 5, ("abcd", 0) gives 1, ("aaaa", 3) gives 4, ("", 2) gives 0. Cross-checked on 300 random strings over the alphabet a b c.

Time $O(n)$ **Space** $O(1)$
26 counters

Here a 26-slot array beats a `Map`, because the alphabet is fixed and tiny and `charCodeAt(i) - 97` is a direct index. Choose the array when the key space is small and dense; choose the `Map` when it is not.

Follow-up the interviewer asks next: “return the actual substring, not just the length.” Store `bestStart = left` whenever you improve `best`, then `s.slice(bestStart, bestStart + best)`. Cost stays $O(n)$.

Dry run — every single step

Problem. $a = [4, 2, 7, 1, 3, 6, 2]$, $S = 11$. Find the length of the shortest subarray with sum at least 11. All values are positive, so Template C applies and the answer is read **inside** the shrink loop.

The code we are tracing

JS

```
let left = 0, best = Infinity, sum = 0;
for (let right = 0; right < a.length; right++) {
  sum += a[right];
  while (sum >= S) {
    best = Math.min(best, right - left + 1);
    sum -= a[left];
    left++;
  }
}
```

Read the table one row at a time. “inf” means best is still Infinity.

right	a[right]	sum after add	what happens	left after	best
0	4	4	4 < 11, no shrink	0	inf
1	2	6	6 < 11, no shrink	0	inf
2	7	13	13 >= 11: window [0..2] has length 3, best = 3; drop a[0] = 4, sum becomes 9; 9 < 11, stop	1	3
3	1	10	10 < 11, no shrink	1	3
4	3	13	13 >= 11: window [1..4] length 4, best stays 3; drop a[1] = 2, sum becomes 11	2	3
		11	still >= 11: window [2..4] length 3, best stays 3; drop a[2] = 7, sum becomes 4; stop	3	3
5	6	10	10 < 11, no shrink	3	3
6	2	12	12 >= 11: window [3..6] length 4, best stays 3; drop a[3] = 1, sum becomes 11	4	3
		11	still >= 11: window [4..6] length 3, best stays 3; drop a[4] = 3, sum becomes 8; stop	5	3

Answer: 3. Two different windows achieve it — $[4, 2, 7]$ with sum 13 and $[3, 6, 2]$ with sum 11. This trace was produced by running the code in Node with a `console.log` on every step, so the table is the program’s real output, not a guess.

Three things to notice

1. `right` advanced 7 times. `left` advanced 5 times. Total pointer moves: 12, which is less than $2n$. That is the whole reason the algorithm is $O(n)$ even though there is a loop inside a loop.
2. `sum` was never recomputed from scratch. Each step was one addition or one subtraction.
3. `best` was written **before** the shrink, never after. Rows 5 and 8 are exactly where a “record after the loop” version would report the wrong length.

TRAP

Trace your own code like this on paper before you run it. Half of all sliding-window bugs are a `best` update on the wrong side of the `while`, and a five-row table finds them faster than a debugger.

Practice

Solve these on paper first, then type them. Full code is in the answer key.

PRACTICE · TIER 1 target: 45 min for 1-6

1. **Pairs at a fixed distance.** A sorted array of **distinct** values and an integer $D \geq 1$. Count the pairs whose difference is exactly D . $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: empty array; no pair matches.
2. **Longest exact-sum run.** All values are positive. Find the length of the longest subarray whose sum is exactly K . $0 \leq n \leq 10^5$, $1 \leq a_i \leq 10^4$. Target $O(n)$. Edge cases: no such subarray (return 0); a single element equal to K .
3. **Anagram window.** Does `big` contain any rearrangement of `small` as a substring? Lowercase only, $0 \leq |\text{small}| \leq |\text{big}| \leq 10^5$. Target $O(n)$. Edge cases: `small` longer than `big`; `small` empty.
4. **Best window sum.** Return the largest sum over all windows of length k . $1 \leq k \leq n \leq 10^5$, values may be negative. Target $O(n)$. Edge cases: all negative; $k = n$.
5. **Take from the ends.** All values positive. Each move takes the first or the last element. What is the fewest moves to collect a total of at least `target`? Return -1 if the whole array is not enough. $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: `target` = 0; `target` above the total.
6. **Delete one.** A 0/1 array. You must delete exactly one element. What is the longest run of 1s in what remains? $1 \leq n \leq 10^5$. Target $O(n)$. Edge cases: all ones; all zeros.

PRACTICE · TIER 2 target: 50 min for 7-11

7. **Cheap baskets.** All values positive. Count the subarrays whose **product** is strictly less than `target`. $0 \leq n \leq 10^5$, $1 \leq a_i \leq 10^5$. Target $O(n)$. Edge cases: `target` = 1 (answer 0); a product that passes the safe-integer limit.
8. **Full catalogue window.** Find the length of the shortest subarray that contains every distinct value present in the whole array. $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: every value the same; all values distinct.
9. **Two-sum indexes.** Sorted array, return the two **1-based** indexes whose values add to `target`, or an empty array. $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: no pair; negative values.
10. **Three-colour sort.** Values are only 0, 1, 2. Sort in one pass with $O(1)$ extra space. $0 \leq n \leq 10^5$. Edge cases: already sorted; all one colour.
11. **Closest across two lists.** Two sorted arrays. Find the smallest possible $|a_i - b_j|$. $1 \leq n, m \leq 10^5$. Target $O(n + m)$. Edge cases: single-element arrays; values far apart.

PRACTICE · TIER 3 target: 45 min for 12-15

12. **All three letters.** The string uses only a, b, c. Count the substrings that contain at least one of each. $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: a letter missing entirely (answer 0); a large count.
13. **At most twice.** Longest subarray in which no value appears more than twice. $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: empty; all values equal.
14. **Reverse the vowels.** Reverse only the vowels of a string, leaving every other character where it is. Both cases count as vowels. $0 \leq n \leq 10^5$. Target $O(n)$. Edge cases: no vowels; all vowels.
15. **Merge backwards.** `a` holds m sorted values then n spare slots; `b` holds n sorted values. Merge `b` into `a` in place, with no extra array. Target $O(m + n)$. Edge cases: $m = 0$; every value of `b` smaller than every value of `a`.

ANSWER KEY

1. Walk `j` forward; pull `i` up whenever the gap is too big.

Pairs with difference exactly D

JS

```
function countDiffPairs(a, D) {
  let c = 0, i = 0, j = 1;
  while (j < a.length) {
    const d = a[j] - a[i];
    if (i === j || d < D) j++;
    else if (d > D) i++;
    else { c++; i++; j++; }
  }
  return c;
}
```

Tested: ([1,3,5,8,10], 2) gives 3, ([1,2,3,4], 1) gives 3, ([1,9], 3) gives 0, ([], 1) gives 0, ([4], 1) gives 0. Cross-checked against a brute force on 500 random distinct arrays. $O(n)$ time, $O(1)$ space.

2. Shrink while the sum is too big, then check for equality.

Longest subarray with sum exactly K

JS

```
function longestSumK(a, K) {
  let sum = 0, left = 0, best = 0;
  for (let right = 0; right < a.length; right++) {
    sum += a[right];
    while (sum > K && left <= right) { sum -= a[left]; left++; }
    if (sum === K) best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Tested: ([1,2,3,1,1,1,1], 4) gives 4, ([5], 5) gives 1, ([5], 3) gives 0, ([], 0) gives 0. $O(n) / O(1)$.

3. A fixed window of length `small.length`, compared as two 26-slot count arrays. JavaScript will not compare arrays with `===`, so compare element by element with `every`.

Permutation inside a string

JS

```
function hasPermutation(big, small) {
  const m = small.length, n = big.length;
  if (m === 0 || m > n) return false;
  const need = new Array(26).fill(0), have = new Array(26).fill(0);
  for (const c of small) need[c.charCodeAt(0) - 97]++;
  const same = () => need.every((v, i) => v === have[i]);
  for (let i = 0; i < n; i++) {
    have[big.charCodeAt(i) - 97]++;
    if (i >= m) have[big.charCodeAt(i - m) - 97]--;
    if (i >= m - 1 && same()) return true;
  }
  return false;
}
```

Tested: ("oidbcaf","abc") gives true, ("odicf","abc") gives false, ("aa","aaa") gives false, ("ab","ba") gives true. $O(26n)$ time, $O(1)$ space.

TRAP

[1,2] === [1,2] is false in JavaScript — it compares object identity, not contents. That single fact breaks a lot of first attempts at this problem.

4. Template B, seeded with the first real window.

Best window sum of length k

JS

```
function bestWindowSum(a, k) {
  let sum = 0;
  for (let i = 0; i < k; i++) sum += a[i];
  let best = sum;
  for (let i = k; i < a.length; i++) {
    sum += a[i] - a[i - k];
    best = Math.max(best, sum);
  }
  return best;
}
```

Tested: ([4,1,9,2,8], 2) gives 11, ([-3,-1,-7], 2) gives -4, ([6], 1) gives 6. $O(n) / O(1)$.

5. Flip it: taking from the ends leaves one middle window. You want the **longest** middle window whose sum is at most **total** - **target**.

Fewest picks from the two ends

JS

```
function fewestFromEnds(v, target) {
  const total = v.reduce((s, x) => s + x, 0);
  if (total < target) return -1;
  const keep = total - target; // biggest middle window with sum <= keep
  let sum = 0, left = 0, bestWin = 0;
  for (let right = 0; right < v.length; right++) {
    sum += v[right];
    while (sum > keep) { sum -= v[left]; left++; }
    bestWin = Math.max(bestWin, right - left + 1);
  }
  return v.length - bestWin;
}
```

Tested: ([3,1,4,1,5], 9) gives 3, ([2,2], 100) gives -1, ([7], 7) gives 1, ([1,1,1], 0) gives 0. $O(n) / O(1)$.

6. Allow one zero in the window, then subtract the deleted slot: the answer is **right** - **left**, not **right** - **left** + 1.

Longest run of 1s after deleting one element

JS

```
function longestOnesDeleteOne(bit) {
  let left = 0, zeros = 0, best = 0;
  for (let right = 0; right < bit.length; right++) {
    if (bit[right] === 0) zeros++;
    while (zeros > 1) { if (bit[left] === 0) zeros--; left++; }
    best = Math.max(best, right - left); // -1 for the deleted slot
  }
  return best;
}
```

Tested: [1,1,0,1,1,1] gives 5, [1,1,1] gives 2, [0,0] gives 0, [1] gives 0. $O(n) / O(1)$.

7. Same shape as “at most K distinct”: once a window is valid, it contributes **right** - **left** + 1 subarrays.

Subarrays with product below target

JS

```
function countProductLess(a, target) {
  if (target <= 1) return 0;
  let prod = 1, c = 0, left = 0;
  for (let right = 0; right < a.length; right++) {
    prod *= a[right];
    while (prod >= target) { prod /= a[left]; left++; }
    c += right - left + 1;
  }
  return c;
}
```

Tested: ([10,2,3,4], 30) gives 8 and a brute force agrees; ([1,1,1], 2) gives 6; ([5,5], 1) gives 0; ([100000,100000,100000], 4000000000) gives 3. Cross-checked on 500 random arrays.

TRAP

prod peaks at roughly $\text{target} \times \max(a)$. Keep that product under `Number.MAX_SAFE_INTEGER` or the division that follows starts returning fractions and the count drifts. If the interviewer raises the limits, switch to `BigInt` — this version was also run and gives the same answers:

The BigInt variant

JS

```
function countProductLessBig(a, target) {
  const T = BigInt(target);
  let prod = 1n, c = 0, left = 0;
  for (let right = 0; right < a.length; right++) {
    prod *= BigInt(a[right]);
    while (prod >= T) { prod /= BigInt(a[left]); left++; }
    c += right - left + 1;
  }
  return c;
}
```

Tested: on [100000,100000,100000,100000] with target 1e15, both versions return 7. $O(n) / O(1)$.

8. Count the distinct values first, then shrink as hard as possible while the window still holds all of them.

Shortest window with every distinct value

JS

```
function shortestAllDistinct(a) {
  const need = new Set(a).size;
  if (need === 0) return 0;
  const cnt = new Map();
  let left = 0, best = Infinity;
  for (let right = 0; right < a.length; right++) {
    cnt.set(a[right], (cnt.get(a[right]) || 0) + 1);
    while (cnt.size === need) {
      best = Math.min(best, right - left + 1);
      const v = a[left];
      cnt.set(v, cnt.get(v) - 1);
      if (cnt.get(v) === 0) cnt.delete(v);
      left++;
    }
  }
  return best;
}
```

Tested: [2,1,3,2,1,3,3] gives 3, [7,7,7] gives 1, [4,5] gives 2. $O(n)$ time, $O(n)$ space.

SHORTCUT

`new Set(a).size` counts distinct values in one line. It is the JavaScript replacement for sorting and counting runs.

9. Shape A, unchanged.

Two-sum on a sorted array, 1-based

JS

```
function twoSumSorted(a, target) {
  let i = 0, j = a.length - 1;
  while (i < j) {
    const s = a[i] + a[j];
    if (s === target) return [i + 1, j + 1];
    if (s < target) i++;
    else j--;
  }
  return [];
}
```

Tested: ([2,4,7,11], 15) gives [2,4]; ([2,4], 99) gives []. $O(n) / O(1)$.

10. Three pointers. Everything before `low` is 0, everything after `high` is 2, and `mid` is the scanner. Do **not** advance `mid` after a swap with `high`, because the value that arrived from the back has not been looked at yet.

One-pass three-colour sort

JS

```
function sortThreeValues(a) {
  let low = 0, mid = 0, high = a.length - 1;
  while (mid <= high) {
    if (a[mid] === 0) { [a[low], a[mid]] = [a[mid], a[low]]; low++; mid++; }
    else if (a[mid] === 1) { mid++; }
    else { [a[mid], a[high]] = [a[high], a[mid]]; high--; }
  }
}
```

Tested: [2,0,1,2,1,0] becomes [0,0,1,1,2,2]; [1,1] stays [1,1]. $O(n) / O(1)$.

11. Walk both arrays forward; always advance the pointer sitting on the smaller value, because only that move can close the gap.

Closest pair across two sorted arrays

JS

```
function closestPair(a, b) {
  let i = 0, j = 0, best = Infinity;
  while (i < a.length && j < b.length) {
    best = Math.min(best, Math.abs(a[i] - b[j]));
    if (a[i] < b[j]) i++;
    else j++;
  }
  return best;
}
```

Tested: ([1,9,20], [4,12]) gives 3, ([5],[5]) gives 0, ([-2000000000], [2000000000]) gives 4000000000. $O(n + m) / O(1)$.

12. For each right end, the substrings that work are exactly those starting at or before the **most recent** position of the rarest of the three letters.

Substrings containing a, b and c

JS

```
function countAbcSubstrings(s) {
  const last = [-1, -1, -1];
  let c = 0;
  for (let i = 0; i < s.length; i++) {
    last[s.charCodeAt(i) - 97] = i;
    const lo = Math.min(last[0], last[1], last[2]);
    if (lo >= 0) c += lo + 1;
  }
  return c;
}
```

Tested against a brute force: "abcbabc" gives 10, "aaa" gives 0, "cba" gives 1. Cross-checked on 500 random strings. $O(n) / O(1)$.

13. Template C with the rule “this value now appears three times”.

No value more than twice

JS

```
function longestAtMostTwice(a) {
  const cnt = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < a.length; right++) {
    const v = a[right];
    cnt.set(v, (cnt.get(v) || 0) + 1);
    while (cnt.get(v) > 2) {
      cnt.set(a[left], cnt.get(a[left]) - 1);
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Tested: [1,2,1,2,1] gives 4, [4,4,4] gives 2, [] gives 0, [3,3] gives 2. $O(n)$ time, $O(n)$ space.

14. Shape A with two skip rules. JavaScript strings cannot be changed in place, so convert to an array of characters first.

Reverse only the vowels

JS

```
function reverseVowels(str) {
  const s = [...str];
  const isVowel = c => 'aeiouAEIOU'.includes(c);
  let i = 0, j = s.length - 1;
  while (i < j) {
    if (!isVowel(s[i])) i++;
    else if (!isVowel(s[j])) j--;
    else { [s[i], s[j]] = [s[j], s[i]]; i++; j--; }
  }
  return s.join('');
}
```

Tested: "placement" becomes "plecemant", "xyz" is unchanged, "" is unchanged, "aeiou" becomes "uoiea". $O(n) / O(1)$ beyond the character array.

TRAP

`s[i] = 'x'` on a JavaScript string does nothing at all — no error, no change. Strings are immutable. Spread into an array, edit, then `join('')`.

15. Write from the **back**. The tail of **a** is spare space, so nothing is ever overwritten before it is read.

In-place merge, filling backwards

JS

```
function mergeInPlace(a, m, b, n) {  
  let i = m - 1, j = n - 1, w = m + n - 1;  
  while (j >= 0) {  
    if (i >= 0 && a[i] > b[j]) a[w--] = a[i--];  
    else a[w--] = b[j--];  
  }  
}
```

Tested: **a** = [1,5,9,0,0] with **m** = 3 and **b** = [3,7] becomes [1,3,5,7,9]; **a** = [0,0] with **m** = 0 and **b** = [2,4] becomes [2,4]. $O(m + n) / O(1)$.

ONE-PAGE REVISION CARD

The three templates

A — opposite ends (array must be sorted)

JS

```
const pairSum = (a, t) => {           // a is sorted ascending
  let i = 0, j = a.length - 1;
  while (i < j) {
    const s = a[i] + a[j];
    if (s === t) return [i, j];
    if (s < t) i++; else j--;          // too small: raise the low end
  }                                   // too big: lower the high end
  return null;
};
```

B — fixed window of size k

JS

```
const maxWindowSum = (a, k) => {
  if (a.length < k || k <= 0) return 0;
  let sum = 0;
  for (let i = 0; i < k; i++) sum += a[i];    // first window
  let best = sum;
  for (let i = k; i < a.length; i++) {
    sum += a[i] - a[i - k];                  // gain right, lose left
    best = Math.max(best, sum);
  }
  return best;
};
```

C-long — variable window, LONGEST valid

JS

```
const longestAtMostK = (a, K) => {
  const cnt = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < a.length; right++) {
    cnt.set(a[right], (cnt.get(a[right]) ?? 0) + 1);
    while (cnt.size > K) {                    // shrink while BAD
      const c = cnt.get(a[left]) - 1;
      if (c === 0) cnt.delete(a[left]); else cnt.set(a[left], c);
      left++;
    }
    best = Math.max(best, right - left + 1); // read AFTER the while
  }
  return best;
};
```

C-short — variable window, SHORTEST valid (positives only)

JS

```
const shortestAtLeastS = (a, S) => {
  let left = 0, sum = 0, best = Infinity;
  for (let right = 0; right < a.length; right++) {
```

```

    sum += a[right];
    while (sum >= S) {
        best = Math.min(best, right - left + 1); // shrink while GOOD
        sum -= a[left]; left++;
    }
}
return best === Infinity ? 0 : best;
};

```

All four were run in Node. `pairSum([1,3,4,7,11], 11)` gives `[2,3]`; `maxWindowSum([2,1,5,1,3,2], 3)` gives 9; `longestAtMostK([1,2,1,3,4], 2)` gives 3; `shortestAtLeastS([2,3,1,2,4,3], 7)` gives 2. Every one of them returns 0 or null on the empty array — no crash, no special case.

When to use which

The question says	Shape	Time
sorted array, find a pair with a given sum	A	$O(n)$
is it a palindrome / reverse it in place	A	$O(n)$
container of water, two walls	A	$O(n)$
triple with a given sum	sort, then A inside a loop	$O(n^2)$
squares of a sorted array, sorted	A, filling from the back	$O(n)$
“every window of length k”	B	$O(n)$
longest subarray with rule P	C-long	$O(n)$
shortest subarray with sum $\geq S$	C-short	$O(n)$
“at most K distinct” / “at most k changes”	C-long + Map of counts	$O(n)$
“exactly K distinct”	<code>atMost(K) — atMost(K-1)</code>	$O(n)$
window max and min at the same time	C + two monotonic deques	$O(n)$
exact subarray sum, values may be negative	not a window — prefix sums + Map	$O(n)$

Complexity you should be able to quote

Problem	Time	Space
pair sum in a sorted array	$O(n)$	$O(1)$
pair sum, unsorted, sorting allowed	$O(n \log n)$	$O(1)$
three-sum	$O(n^2)$	$O(1)$ beyond the sort
fixed window of size k	$O(n)$	$O(1)$
longest / shortest variable window	$O(n)$	$O(1)$
window with a count Map	$O(n)$	$O(k)$ distinct keys
window max and min (two deques)	$O(n)$	$O(k)$
trapping rain water, two pointers	$O(n)$	$O(1)$
minimum window covering a set	$O(n)$	$O(\sigma)$ alphabet

Why every window is $O(n)$ and not $O(n^2)$: `left` and `right` each move forward at most n times in the whole run. The inner `while` is not a nested loop — it is the **same** `left` continuing its one journey across the array. Say that sentence in the interview.

Top traps, in the order they bite

1. **Reading the answer on the wrong side of the while.** Longest problems read **after** the shrink loop; shortest problems read **inside** it. Get this backwards and every single test fails.
2. **A window for a sum problem with negative values.** Growing the window can lower the sum, so shrinking is no longer safe. Use prefix sums plus a Map instead.
3. **Moving an index backwards.** `left--` or `right--` means you have left the pattern and the $O(n)$ argument is gone.
4. **`a.sort()` with no comparator.** It sorts as **text**: `[10, 9, 1].sort()` returns `[1, 10, 9]`. Shape A needs `a.sort((x, y) => x - y)` first, every time.
5. **Not deleting a zero count.** `cnt.size` is the number of distinct values in the window only if you `cnt.delete(key)` when its count hits 0. Leaving a 0 behind makes `cnt.size` lie.
6. **Duplicate answers in three-sum.** After sorting, skip equal values at each of the three positions, or the same triple comes out many times.
7. **`sum += a[i] - a[i - k]` before the first full window exists.** Build the first window with its own loop, then start sliding at `i = k`.
8. **An array used as a queue.** `shift()` is $O(n)$ in the worst case, which turns an $O(n)$ window into $O(n^2)$. Use the toolkit Deque, or a head index.
9. **`s[i] = 'x'` on a string.** JavaScript strings are immutable — the line does nothing and raises no error. Spread into an array, edit, then `join('')`.
10. **Off-by-one in the window length.** It is `right - left + 1`, never `right - left`.

The 20-second checklist before you code

1. Is the array sorted, or may I sort it? If yes, Shape A is on the table.
2. Is the window length fixed by the question? Then Shape B — no `while` at all.
3. Otherwise: write the **bad condition** as one sentence. That sentence becomes the `while`.
4. Longest or shortest? That decides where the answer is read.
5. Are there negative numbers in a **sum** problem? If yes, stop — use prefix sums.
6. What do add and remove cost? If either is more than $O(1)$, the total is not $O(n)$.

Binary Search

Halve the search space, every single step.

Pattern in one page

Most students learn binary search as “find a number in a sorted array”. That is the smallest thing it does. The real definition is wider, and it is what interviewers test:

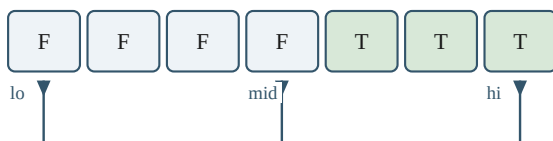
THE ONE IDEA

Binary search works on any range where some yes/no test flips **once** and never flips back.

Line up the test for every candidate. You must be able to say the answers look like this:

F F F F F T T T T

Then you can find the boundary — the first T — in $O(\log n)$ tests. The array does not have to be sorted. The candidates do not even have to be array positions: they can be capacities, speeds, days, or lengths. The only thing that matters is that the test is **monotone**.



Find the first T. Everything left of it is F, forever.

The three templates you must own

T1 — exact match, closed range [lo, hi]

JS

```
let lo = 0, hi = n - 1;
while (lo <= hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (a[mid] === target) return mid;
  if (a[mid] < target) lo = mid + 1;
  else hi = mid - 1;
}
return -1;
```

T2 — first index that satisfies a test, half-open range [lo, hi)

JS

```
let lo = 0, hi = n; // hi is ONE PAST the last index
while (lo < hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (!good(mid)) lo = mid + 1; // mid is still F, throw it away
  else hi = mid; // mid might be the answer, keep it
}
return lo; // === n means "no index is good"
```

```
let lo = smallestPossibleAnswer, hi = largestPossibleAnswer;
while (lo < hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (feasible(mid)) hi = mid;    // mid works; try smaller
  else lo = mid + 1;           // mid too small; go up
}
return lo;
```

T2 and T3 are the **same loop**. The only difference is what the candidates mean: array indexes in T2, real-world quantities in T3. Learn one, and you have both.

Three rules that stop infinite loops

LOOP HYGIENE

1. **Always shrink.** Every branch must either raise `lo` or lower `hi`. `lo = mid` with `while (lo < hi)` loops forever when `hi = lo + 1`.
2. **Match the guard to the range.** Closed range `[lo, hi]` pairs with `while (lo <= hi)` and `mid ± 1`. Half-open range `[lo, hi)` pairs with `while (lo < hi)` and `hi = mid`.
3. **mid must be a whole number.** In JavaScript `/` is real division, so `(lo + hi) / 2` can be 3.5 and `a[3.5]` is undefined. Write `const mid = Math.floor(lo + (hi - lo) / 2);` every single time.

TRAP

Do not write `(lo + hi) >> 1` to floor it. Every bitwise operator in JavaScript first chops its operand down to a **signed 32-bit** integer, so the shift breaks the moment the sum passes 2^{31} . Tested in Node with `lo = 3000000000` and `hi = 3000000002`:

- `(lo + hi) >> 1` gives 852516353 — nonsense,
- `Math.floor(lo + (hi - lo) / 2)` gives 3000000001 — correct.

Binary search on the **answer** routinely runs past 2^{31} (capacities, times, distances), so `>>1` is a bug waiting for a big test case. Use `Math.floor` and subtract first.

How many steps?

Each step throws away half the range. Starting from n candidates you are done after about $\log_2 n$ steps.

n	steps	meaning
10^3	10	instant
10^6	20	instant
10^9	30	instant
10^{18}	60	still instant

SHORTCUT

When the constraint line says $n \leq 10^5$ but the **values** go up to 10^9 , that is a loud hint: the answer is being searched, not the array. Reach for T3.

When to reach for it

You see this in the question	Reach for
------------------------------	-----------

sorted array, find / count a value	T1, or T2 twice
“first index where ...”, “how many are less than ...”	T2
“minimum capacity / speed / size so that ...”	T3, feasible is monotone
“maximum ... such that ...”	T3, flipped: find the last feasible
rotated sorted array	T1 with a “which half is sorted” test
two sorted arrays, k-th or median	binary search on the split point
answer is a real number	fixed 100 iterations of halving

Four JavaScript facts you must know before you start

TRAP

arr.sort() sorts LEXICOGRAPHICALLY, as text. Tested in Node: `[10, 9, 1].sort()` returns `[1, 10, 9]`, because `"10" < "9"` when you compare them as strings. Numbers **always** need a comparator: `[10, 9, 1].sort((a, b) => a - b)` returns `[1, 9, 10]`.

Binary search on a sorted array is worthless if the sort was wrong, and this chapter sorts in five different places — Examples 14, 19, 23, 25 and practice answer 12. Every one of them passes the comparator, and so must you.

TRAP

sort changes the array in place. `a.sort(...)` returns `a` itself, not a new array, so a helper that sorts its argument quietly damages the caller’s data. When the caller must keep the original order, copy first: `const sorted = [...a].sort((x, y) => x - y);`

JavaScript has no lowerBound. There is nothing in the standard library that finds the first position at or after which a value belongs. You write it once (Examples 2 and 3), and after that this chapter uses the tested copy from the **JS Toolkit** appendix:

one line, once per file

JS

```
const { lowerBound, upperBound } = require('./toolkit.js');
```

There is no built-in heap either, and no ordered map. Two follow-ups in this chapter compare binary search against a heap (Examples 25 and 28). If you code one, use the `MinHeap` from the **JS Toolkit** appendix; JavaScript ships no priority queue and no `TreeMap`. A `Map` keeps insertion order, never key order.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Return any index holding **target** in a sorted array, or -1. Constraints: $0 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: empty array; target not present; single element.

SOLUTION

Template T1

JS

```
const binarySearch = (a, target) => {
  let lo = 0, hi = a.length - 1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] === target) return mid;
    if (a[mid] < target) lo = mid + 1;
    else hi = mid - 1;
  }
  return -1;
};
```

Tested on [2,5,5,5,9,14,20]: 9 gives 4, 5 gives 3 (any of the three is acceptable), 3 gives -1. Empty array gives -1. [7] with target 7 gives 0.

Time $O(\log n)$ **Space** $O(1)$

hi starts at $n - 1$, so for an empty array $hi = -1$ and the loop body never runs. No special case needed.

Example 2 [warm-up]

lowerBound(a, x): the first index i with $a[i] \geq x$. Return n if there is none. Constraints: $0 \leq n \leq 10^5$, duplicates allowed. Target $O(\log n)$. Edge cases: x smaller than everything; x larger than everything.

SOLUTION

This is Template T2 with **good(mid)** meaning $a[mid] \geq x$.

lowerBound, written out

JS

```
const lowerBound = (a, x) => {
  let lo = 0, hi = a.length; // hi is one past the end
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] < x) lo = mid + 1;
    else hi = mid;
  }
  return lo;
};
```

Tested on [2,5,5,5,9,14,20]: $x = 5$ gives 1, $x = 6$ gives 4, $x = 1$ gives 0, $x = 99$ gives 7. Empty array gives 0. Cross-checked on 2000 random arrays against the tested **lowerBound** in the **JS Toolkit** appendix.

Time $O(\log n)$ **Space** $O(1)$

JavaScript has no `lowerBound` of its own — there is nothing in the standard library that finds an insertion point. You write it once, and from Example 7 onwards this chapter uses the tested copy from the **JS Toolkit** appendix: `const { lowerBound, upperBound } = require('./toolkit.js');`

Example 3 [warm-up]

`upperBound(a, x)`: the first index with `a[i] > x`. Same constraints. Edge cases: `x` equal to the last value.

SOLUTION

One character changes: `<` becomes `<=`.

upperBound, written out

JS

```
const upperBound = (a, x) => {
  let lo = 0, hi = a.length;
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] <= x) lo = mid + 1;
    else hi = mid;
  }
  return lo;
};
```

Tested on the same array: `x = 5` gives 4, `x = 6` gives 4, `x = 1` gives 0, `x = 20` gives 7.

Time $O(\log n)$ **Space** $O(1)$

SHORTCUT

Memorise the pair as a picture. `lowerBound` stops **before** the first copy of `x`. `upperBound` stops **after** the last copy. Everything in between is `x`.

Example 4 [warm-up]

How many times does `x` appear in a sorted array? Target $O(\log n)$. Edge cases: `x` absent; every element equal to `x`.

SOLUTION

Count by subtraction

JS

```
const countOf = (a, x) => upperBound(a, x) - lowerBound(a, x);
```

Tested on `[2, 5, 5, 5, 9, 14, 20]`: 5 gives 3, 2 gives 1, 7 gives 0. Empty array gives 0.

Time $O(\log n)$ **Space** $O(1)$

Example 5 [warm-up]

`floorOf(a, x)`: the largest value that is $\leq x$. `ceilOf(a, x)`: the smallest value that is $\geq x$. Report a sentinel when none exists. Edge cases: `x` below the whole array; `x` above the whole array; `x` present exactly.

SOLUTION

Floor and ceiling from the two bounds

JS

```
const floorOf = (a, x) => { // largest value <= x, or null
  const i = upperBound(a, x);
  return i === 0 ? null : a[i - 1];
};
```

```
};
const ceilOf = (a, x) => { // smallest value >= x, or null
  const i = lowerBound(a, x);
  return i === a.length ? null : a[i];
};
```

Tested on [2,5,5,5,9,14,20]: floorOf(13) gives 9, ceilOf(13) gives 14, floorOf(1) reports “none”, ceilOf(21) reports “none”, and with $x = 5$ both return 5.

Time $O(\log n)$ **Space** $O(1)$

TRAP

For the floor you need upperBound, not lowerBound. If x is present, lowerBound points **at** it and $i - 1$ would skip over a valid answer.

Example 6 [warm-up]

Integer square root: the largest r with $r^2 \leq n$. Constraints: $0 \leq n \leq 10^{18}$. Target $O(\log n)$. Edge cases: $n = 0$, $n = 1$, a perfect square, and overflow.

SOLUTION

Approach 1 — Count up ($O(\sqrt{n})$, a billion steps at the top of the range)

Linear

JS

```
const isqrtLinear = (n) => {
  let r = 0;
  while ((r + 1) * (r + 1) <= n) r++;
  return r;
};
```

Time $O(\sqrt{n})$ **Space** $O(1)$

Approach 2 — Binary search ($O(\log n)$ — optimal for this shape)

$\text{mid} * \text{mid}$ can grow far past `Number.MAX_SAFE_INTEGER` and stop being exact. Compare with a **division** instead — that never grows.

Overflow-safe integer square root

JS

```
const isqrtBS = (n) => {
  let lo = 0, hi = n, ans = 0;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (mid === 0 || mid <= n / mid) { ans = mid; lo = mid + 1; }
    else { hi = mid - 1; }
  }
  return ans;
};
```

Tested: 0 gives 0, 1 gives 1, 24 gives 4, 25 gives 5, 26 gives 5, and 9007199254740991 gives 94906265. Checked against the linear version for every n from 0 to 3000.

Time $O(\log n)$ **Space** $O(1)$

TRAP

Every JavaScript number is a double. Whole numbers are exact only up to `Number.MAX_SAFE_INTEGER`, which is 9007199254740991, about 9×10^{15} . The constraint here says $n \leq 10^{18}$, which is a hundred times past that line, so plain numbers are the wrong tool. Tested in Node:

- `123456789 * 123456789` prints 15241578750190520; the true answer ends in 521,
- `9007199254740993` prints as 9007199254740992 — the literal itself is already lost.

Above $2^{53} - 1$ you must switch to `BigInt`.

Approach 3 — The same search in `BigInt` ($O(\log n)$, and exact all the way to 10^{18})

`BigInt` literals end in `n`. Division on `BigInt` throws the fraction away, which is exactly the whole-number division this algorithm wants, so `mid * mid <= n` is safe again.

Integer square root for `n` past $2^{53} - 1$

JS

```
const isqrtBig = (n) => { // n is a BigInt, and so is the answer
  let lo = 0n, hi = n, ans = 0n;
  while (lo <= hi) {
    const mid = lo + (hi - lo) / 2n;
    if (mid * mid <= n) { ans = mid; lo = mid + 1n; }
    else { hi = mid - 1n; }
  }
  return ans;
};
```

Tested: `isqrtBig(100000000000000000n)` gives 1000000000n, `isqrtBig(99999999999999999n)` gives 999999999n, `isqrtBig(0n)` gives 0n. Checked against the linear version for every `n` from 0 to 3000.

Time $O(\log n)$ **Space** $O(1)$

Timed in Node, a 3-million-step modular loop took 13 ms with numbers and 193 ms with `BigInt` - about 15x slower. Reach for `BigInt` only where the exactness is actually needed.

TRAP

Never mix the two. Tested in Node, `1n + 1` throws `TypeError: Cannot mix BigInt and other types`, use explicit conversions. Convert on purpose: `BigInt(5) + 1n` gives `6n`, and `Number(5n) + 1` gives 6. Note that `Number(b)` silently loses precision once `b` passes $2^{53} - 1$ - convert back only when you know the value is small.

Tier 1 — the template problems

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Report the first and the last index of `target` in a sorted array with duplicates, or `(-1, -1)`. Constraints: $0 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: target absent; all values equal to target; single element.

SOLUTION

Approach 1 — Scan the whole array $O(n)$

Brute force

JS

```
const firstLastScan = (a, x) => {
  let f = -1, l = -1;
  for (let i = 0; i < a.length; i++)
    if (a[i] === x) { if (f === -1) f = i; l = i; }
  return [f, l];
};
```

Time $O(n)$ **Space** $O(1)$

Approach 2 — Two bounded searches $O(\log n)$ — optimal

Reuse lowerBound and upperBound

JS

```
const { lowerBound, upperBound } = require('./toolkit.js'); // JS Toolkit
appendix

const firstLast = (a, x) => {
  const lo = lowerBound(a, x);
  if (lo === a.length || a[lo] !== x) return [-1, -1];
  return [lo, upperBound(a, x) - 1];
};
```

Tested on `[2,5,5,5,9,14,20]`: 5 gives `(1, 3)` — the scan agrees — 7 gives `(-1, -1)`, 20 gives `(6, 6)`. Cross-checked against the scan on 2000 random arrays and every target from -1 to 13.

Time $O(\log n)$ **Space** $O(1)$

The unlocking idea: “first index with `a[i] >= x`” and “first index with `a[i] > x`” are both monotone tests, and every count question in a sorted array is the difference of two such boundaries.

TRAP

You must check `a[lo] !== x` before trusting `lo`. `lowerBound` returns the **insert position** when the value is missing, and that position holds a different number.

Example 8 [Infosys · pattern]

An array where no two neighbours are equal. Return the index of any **peak** — an element larger than both of its neighbours. The two ends count as having an infinitely small neighbour outside. Constraints: $1 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: $n = 1$; a strictly rising array; a strictly falling array.

SOLUTION

Approach 1 — Check every index $O(n)$

Brute force

JS

```
const peakIndexLinear = (a) => {
  const n = a.length;
  for (let i = 0; i < n; i++) {
    const okL = i === 0 || a[i - 1] < a[i];
    const okR = i === n - 1 || a[i + 1] < a[i];
    if (okL && okR) return i;
  }
  return -1;
};
```

Time $O(n)$ Space $O(1)$

Approach 2 — Walk uphill by halving $O(\log n)$ — optimal

The array is not sorted, so there is nothing to compare the target against. What **is** monotone is the direction of the slope. If $a[mid] < a[mid+1]$ the ground rises to the right, so a peak must exist somewhere to the right — keep going up.

Binary search with no target

JS

```
const peakIndex = (a) => {
  let lo = 0, hi = a.length - 1;
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] < a[mid + 1]) lo = mid + 1;
    else hi = mid;
  }
  return lo;
};
```

Tested: $[1, 4, 9, 7, 3]$ gives index 2 and so does the linear version. $[5]$ gives 0, $[1, 2, 3]$ gives 2, $[3, 2, 1]$ gives 0. Verified on 2000 random arrays that the returned index really is a peak.

Time $O(\log n)$ Space $O(1)$

The unlocking idea: binary search does not need a sorted array. It needs a **monotone decision**, and “which way is uphill” is one.

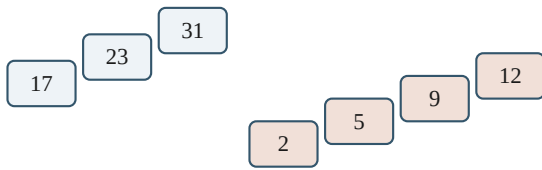
Why must a peak exist to the right when the ground rises? Keep walking right. Either you keep rising until the last element — which is then a peak — or you stop rising somewhere, and that stopping point is a peak. There is no third option.

Example 9 [Wipro · pattern]

A sorted array of distinct values was rotated left an unknown number of times. Find the index of the smallest value. Constraints: $1 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: no rotation at all; $n = 1$; rotation by exactly $n - 1$.

SOLUTION

Picture the rotated array as two rising runs, the second one entirely below the first. The smallest value is the **only** place the ground drops.



A rotated array: run 1 is high, run 2 is low. The minimum starts run 2.

Approach 1 — Scan for the smallest value $O(n)$

Brute force

JS

```
const minIndexScan = (a) => {  
  let best = 0;  
  for (let i = 1; i < a.length; i++) if (a[i] < a[best]) best = i;  
  return best;  
};
```

Time $O(n)$ **Space** $O(1)$

Approach 2 — Compare against the right end $O(\log n)$ — optimal

Compare $a[\text{mid}]$ with the **last** element. If $a[\text{mid}] > a[\text{hi}]$, then mid is still inside the high run, so the minimum is strictly to the right.

Index of the minimum in a rotated array

JS

```
const minIndexRotated = (a) => {  
  let lo = 0, hi = a.length - 1;  
  while (lo < hi) {  
    const mid = Math.floor(lo + (hi - lo) / 2);  
    if (a[mid] > a[hi]) lo = mid + 1;  
    else hi = mid;  
  }  
  return lo;  
};
```

Tested: [17, 23, 31, 2, 5, 9, 12] gives 3, [4, 5, 6] (not rotated) gives 0, [9] gives 0, [5, 1] gives 1 — the scan agrees on all four. Verified against the true minimum on 2000 random rotations.

Time $O(\log n)$ **Space** $O(1)$

TRAP

Compare with $a[\text{hi}]$, never with $a[\text{lo}]$. With $a[\text{lo}]$ the un-rotated case [4, 5, 6] sends you right and you return the wrong index.

Example 10 [Capgemini · pattern]

Same rotated array of distinct values. Find **target**, or -1. Constraints: $0 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: empty array; target in the low run; target absent.

SOLUTION

Approach 1 — Scan $O(n)$

Works, but throws the structure away.

Approach 2 — Find the minimum, then search one side $O(\log n)$, two passes

Use Example 9 to locate the split, then binary-search the half that can contain the target. Correct, but it is two pieces of code to get right.

Approach 3 — One pass: at least one half is always sorted $O(\log n)$ — optimal

Cut at mid. One of the two halves is a plain sorted run. Work out which, then ask a simple range question about it.

Search in a rotated sorted array

JS

```
const searchRotated = (a, t) => {
  let lo = 0, hi = a.length - 1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] === t) return mid;
    if (a[lo] <= a[mid]) { // left half is sorted
      if (a[lo] <= t && t < a[mid]) hi = mid - 1;
      else lo = mid + 1;
    } else { // right half is sorted
      if (a[mid] < t && t <= a[hi]) lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
};
```

Tested on [17,23,31,2,5,9,12]: 5 gives 4, 31 gives 2, 100 gives -1. Empty array gives -1, [3] with target 3 gives 0. Cross-checked against a scan on 2000 random rotated arrays and every target from 0 to 39.

Time $O(\log n)$ **Space** $O(1)$

The unlocking idea: you cannot compare the target with $a[mid]$ directly any more, but you **can** first identify a sorted half, and inside a sorted half the usual comparison works again.

TRAP

The left-half test must be $a[lo] \leq a[mid]$, with the equals sign. When $lo == mid$ (a two-element range) the values are the same and $<$ would send you down the wrong branch.

Example 11 [Cognizant · pattern]

A matrix where each row is sorted left to right, and the first value of each row is larger than the last value of the row above. Does target appear? Constraints: $1 \leq \text{rows} \times \text{cols} \leq 10^6$. Target $O(\log(\text{rows} \times \text{cols}))$. Edge cases: empty matrix; a 1×1 matrix; target between two rows.

SOLUTION

Approach 1 — Look at every cell $O(\text{rows times cols})$

Brute force

JS

```
const searchMatrixScan = (m, t) => m.some(row => row.includes(t));
```

Time $O(rc)$ **Space** $O(1)$

Approach 2 — Pretend it is one long array $(O(\log(rc))$ — optimal)

Those two rules together mean that reading the matrix row by row gives one fully sorted list. Index k of that imaginary list sits at row k / cols , column $k \% \text{cols}$.

Binary search on the flattened index

JS

```
const searchMatrix = (m, t) => {
  if (m.length === 0 || m[0].length === 0) return false;
  const rows = m.length, cols = m[0].length;
  let lo = 0, hi = rows * cols - 1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    const v = m[Math.floor(mid / cols)][mid % cols];
    if (v === t) return true;
    if (v < t) lo = mid + 1;
    else      hi = mid - 1;
  }
  return false;
};
```

Tested on `[[1,4,7],[10,12,15],[18,21,30]]`: 12 is found by both versions, 13 is not found by either. An empty matrix returns false. A 1×1 matrix holding 5 finds 5.

Time $O(\log(rc))$ **Space** $O(1)$

TRAP

`Math.floor(mid / cols)` is not optional. `mid / cols` is real division in JavaScript, so `7 / 3` is `2.3333...` and `m[2.3333]` is undefined. The `%` operator does give a whole number here, so only the row needs flooring.

TRAP

`[...m]` is a **SHALLOW** copy of a grid. It copies the list of rows, and every row is still the same array. Tested in Node on `g = [[1,2],[3,4]]`: after `const shallow = [...g]`; `shallow[0][0] = 99`; the original `g[0]` is `[99, 2]`. A real copy is `const deep = g.map(r => [...r])`; — after the same edit the original stays `[1, 2]`. This chapter never modifies a grid, but the moment a problem asks you to, this is the bug you will ship.

Example 12 [Accenture · pattern]

Builds are numbered $1..n$. Every build from some unknown build B onward is broken, and every build before B is fine. You have a function `isBad(v)` that costs a full minute to run. Find B with as few calls as possible. Constraints: $1 \leq n \leq 2 \times 10^9$. Target $O(\log n)$ calls. Edge cases: build 1 already broken; no build broken at all.

SOLUTION

Approach 1 — Try build 1, then 2, then 3 ... (one call per build, up to two billion of them)

Brute force

JS

```
const firstBadScan = (n, isBad) => {
  for (let v = 1; v <= n; v++) if (isBad(v)) return v;
};
```

```
    return -1;
};
```

Time $O(n)$ calls **Space** $O(1)$

Approach 2 — Binary search the boundary ($O(\log n)$ calls — optimal)

The test is monotone by definition: F F F T T T. That is Template T2, with a boolean function instead of an array.

First failing build

JS

```
const firstBad = (n, isBad) => {
  let lo = 1, hi = n, ans = -1;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (isBad(mid)) { ans = mid; hi = mid - 1; }
    else { lo = mid + 1; }
  }
  return ans;
};
```

Tested with $n = 10$: if builds from 7 onward are bad the answer is 7; if every build is bad the answer is 1; if none is bad the answer is -1. The scan agrees on all three. Counting the actual calls in Node: the scan used 7, 1 and 10; the binary version used 4, 3 and 4.

Time $O(\log n)$ calls **Space** $O(1)$

SHORTCUT

Keep a separate ans variable whenever the answer may not exist. It is cheaper to reason about than trying to decode lo after the loop.

Example 13 [Adobe · pattern]

Compute $\sqrt{n}$ for a real n , correct to six decimal places, without using the library `sqrt`. Constraints: $0 \leq n \leq 10^{12}$. Edge cases: $n = 0$; $n < 1$, where the answer is **larger** than n .

SOLUTION

With real numbers you cannot ask “did we land exactly”. Instead run a **fixed** number of halvings. Each one cuts the error in half, so 100 rounds take any starting range below 10^{-25} — far past what a double can even store.

Real-valued binary search

JS

```
const sqrtReal = (n) => {
  let lo = 0, hi = Math.max(1, n);
  for (let it = 0; it < 100; it++) {
    const mid = (lo + hi) / 2;
    if (mid * mid <= n) lo = mid;
    else hi = mid;
  }
  return lo;
};
```

Tested: `sqrtReal(2)` prints 1.414214, `sqrtReal(0)` prints 0.000000, `sqrtReal(0.25)` prints 0.500000, `sqrtReal(1e12)` prints 1000000.000000.

Time $O(100)$ **Space** $O(1)$
100 halvings, independent of n

TRAP

$hi = n$ alone is wrong for $n < 1$, because $\sqrt{0.25} = 0.5 > 0.25$ would sit outside the range. $\max(1.0, n)$ fixes it.

SHORTCUT

For real binary search, never write `while (hi - lo > 1e-9)`. On adversarial inputs that loop can spin forever. A `for` loop with a fixed count always terminates and is just as accurate.

Example 14 [Cognizant · pattern]

An unsorted price list and q query prices. For each query, how many items cost strictly less than it? Constraints: $0 \leq n \leq 10^5$, $1 \leq q \leq 10^5$. Target $O((n + q) \log n)$. Edge cases: an empty price list; a query below every price.

SOLUTION

Approach 1 — Scan the list for every query ($O(nq)$, ten billion steps at the limits)

Brute force

JS

```
const countSmallerBrute = (a, q) => q.map(x => a.filter(v => v < x).length);
```

Time $O(nq)$ **Space** $O(1)$

Approach 2 — Sort once, then binary-search each query ($O((n + q) \log n)$ — optimal)

After sorting, “how many are strictly less than x ” is exactly `lowerBound(a, x)` — the index **is** the count, because everything before it is smaller.

Sort once, answer many

JS

```
const countSmaller = (a, q) => {
  const sorted = [...a].sort((x, y) => x - y); // NEVER a bare .sort()
  return q.map(x => lowerBound(sorted, x));
};
```

Tested on prices `[8,1,5,5,3]` with queries `[0,5,6,100]`: both versions give `0 2 4 5`. An empty price list gives `0` for any query.

Time $O((n + q) \log n)$ **Space** $O(1)$
the sort is paid once, not per query

The unlocking idea: pay $O(n \log n)$ once so that every later question costs $O(\log n)$. Whenever a problem says “many queries”, look for a one-time preparation step.

SHORTCUT

`lowerBound` returns a **count** as well as a position. That double meaning is the reason it shows up in so many solutions.

Example 15 [TCS Digital · pattern]

A sorted array. Is there a value that appears **more than** $n/2$ times? Return it, or report that there is none. Constraints: $0 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: empty array; exactly half (that is **not** a majority); $n = 1$.

SOLUTION

Approach 1 — Count the runs $O(n)$

Brute force

JS

```
const majorityScan = (a) => {
  const n = a.length;
  if (n === 0) return null;
  let best = a[0], run = 1, bestRun = 1;
  for (let i = 1; i < n; i++) {
    if (a[i] === a[i - 1]) run++;
    else run = 1;
    if (run > bestRun) { bestRun = run; best = a[i]; }
  }
  return bestRun * 2 > n ? best : null;
};
```

Time $O(n)$ Space $O(1)$

Approach 2 — Only one candidate is possible $O(\log n)$ — optimal

A block longer than half the array must cover the middle slot. So $a[n/2]$ is the **only** value that can win. Find where its block starts, then check that the block is long enough.

One candidate, one bound check

JS

```
const majoritySorted = (a) => {
  const n = a.length;
  if (n === 0) return null;
  const half = n >> 1;
  const cand = a[half]; // the only value that can win
  const first = lowerBound(a, cand);
  if (first + half < n && a[first + half] === cand) return cand;
  return null;
};
```

Tested: $[1, 2, 2, 2, 5]$ gives 2 from both versions; $[1, 2, 3, 4]$ reports none from both; $[7]$ gives 7; an empty array reports none; $[1, 1, 2, 2]$ reports none, because exactly half is not more than half. Cross-checked against the scan on 3000 random sorted arrays.

Time $O(\log n)$ Space $O(1)$

The unlocking idea: narrow the answer down to a single candidate first. Verifying one candidate is almost always cheaper than searching for the answer.

TRAP

The length check is $first + n/2 < n \ \&\& \ a[first + n/2] == cand$. Writing $a[first + n/2] == cand$ alone reads past the end when the block sits at the tail.

Tier 2 — binary search on the answer

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Everything in this section shares one shape. You are asked for the smallest (or largest) value of some quantity that makes a plan **work**. You cannot compute it directly, but you **can** check a guess. And checking is monotone: if capacity 10 works, so does 11.

THE RECIPE, EVERY TIME

1. Name the answer. Call it x .
2. Write `feasible(x)` — a plain loop that returns true or false. This is where all the real thinking happens.
3. Convince yourself `feasible` is monotone: F F F T T T, never back to F.
4. Pick the smallest and largest x that could possibly be the answer.
5. Run Template T3.

Example 16 [Grab · pattern]

Parcels sit on a belt in a fixed order and must be loaded in that order. Each day one truck leaves, carrying a run of parcels whose total weight is at most the truck's capacity. What is the smallest capacity that clears everything within `days` days? Constraints: $0 \leq n \leq 10^5$, $1 \leq w_i \leq 500$, $1 \leq \text{days} \leq n$. Target $O(n \log(\sum w))$. Edge cases: `days = 1`; `days = n`; an empty belt.

SOLUTION

Step 1 — the check. Greedily load the current truck until the next parcel does not fit, then start a new day.

`feasible: does this capacity finish in time?`

JS

```
const canFinishIn = (w, cap, days) => {
  let used = 1, cur = 0;
  for (const x of w) {
    if (x > cap) return false;
    if (cur + x > cap) { used++; cur = x; }
    else { cur += x; }
  }
  return used <= days;
};
```

Step 2 — the range. The capacity is at least the heaviest single parcel (otherwise it never ships) and at most the total weight (one day for everything).

Approach 1 — Try every capacity from low to high ($O(n \text{ times the total weight})$)

`Brute force — useful as a test oracle`

JS

```
const minCapacityBrute = (w, days) => {
  const hi = w.reduce((s, x) => s + x, 0);
  for (let cap = 0; cap <= hi; cap++)
    if (canFinishIn(w, cap, days)) return cap;
  return hi;
};
```

Time $O(n \sum w)$ **Space** $O(1)$

Approach 2 — Binary search the capacity $O(n \log(\sum w))$ — optimal

Template T3

JS

```
const minCapacity = (w, days) => {
  let lo = 0, hi = 0;
  for (const x of w) { lo = Math.max(lo, x); hi += x; }
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (canFinishIn(w, mid, days)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
};
```

Tested on [5,2,8,4,1]: days = 3 gives 8, days = 1 gives 20, days = 5 gives 8. The brute force agrees on all three. An empty belt gives 0; a single parcel of weight 7 in one day gives 7. Cross-checked against the brute force on 500 random belts for every value of days.

Time $O(n \log(\sum w))$ **Space** $O(1)$

Python version, using a reusable first-true helper

PYTHON

```
def first_true(lo, hi, pred):
    ans = hi + 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if pred(mid):
            ans = mid
            hi = mid - 1
        else:
            lo = mid + 1
    return ans

def min_capacity(w, days):
    def can(cap):
        used, cur = 1, 0
        for x in w:
            if x > cap:
                return False
            if cur + x > cap:
                used += 1
                cur = x
            else:
                cur += x
        return used <= days
    if not w:
        return 0
    return first_true(max(w), sum(w), can)
```

Python prints 8 20 0 for the same three cases.

TRAP

lo must start at the **maximum** weight, not at 0 or 1. Starting lower is not wrong here only because canFinishIn rejects $x > \text{cap}$ explicitly — remove that line and the whole thing breaks silently.

Example 17 [Shopee · pattern]

A courier has n routes; route i is $\text{job}[i]$ kilometres long. In one hour the courier covers rate kilometres, but never starts a second route inside the same hour: a route of 11 km at rate 4 costs 3 hours, not 2.75. Find the smallest whole rate that finishes every route within hours hours. Constraints: $1 \leq n \leq 10^4$, $1 \leq \text{job}_i \leq 10^9$, $n \leq \text{hours} \leq 10^9$. Target $O(n \log(\max \text{job}))$. Edge cases: $\text{hours} = n$ (one hour per route); huge hours .

SOLUTION

The check. Hours for one route are $\lceil \text{job}_i / \text{rate} \rceil$. JavaScript divides for real, so `Math.ceil(x / rate)` says exactly what it means.

feasible: is this rate fast enough?

JS

```
const rateWorks = (job, rate, hours) => {
  let t = 0;
  for (const x of job) {
    t += Math.ceil(x / rate);          // ceiling division
    if (t > hours) return false;
  }
  return true;
};
```

Approach 1 — Try every rate ($O(n \text{ times } \max \text{job})$, far too slow)

Approach 2 — Binary search the rate ($O(n \log(\max \text{job}))$) — optimal

The slowest useful rate is 1; the fastest useful rate is the longest route, because beyond that nothing improves.

Template T3 again

JS

```
const minRate = (job, hours) => {
  let lo = 1, hi = 1;
  for (const x of job) hi = Math.max(hi, x);
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (rateWorks(job, mid, hours)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
};
```

Tested on [6,11,4,9]: $\text{hours} = 6$ gives 6, $\text{hours} = 4$ gives 11, $\text{hours} = 100$ gives 1. A single route of 1000000000 km in 1 hour gives 1000000000. A brute-force scan agrees on all of these, and on 500 random inputs for every value of hours from n to $3n$.

Time $O(n \log(\max \text{job}))$ **Space** $O(1)$

TRAP

`Math.ceil(x / rate)` is exact while x stays under `Number.MAX_SAFE_INTEGER`, which the constraint $\text{job}_i \leq 10^9$ guarantees. Past that line the division itself is already rounded and the ceiling is off by one — switch the whole check to `BigInt` and write $(x + \text{rate} - 1n) / \text{rate}$, because `BigInt` has no `Math.ceil`.

Example 18 [Sea · pattern]

Split an array into exactly m non-empty contiguous blocks so that the largest block sum is as small as possible. Return that smallest possible largest sum. Constraints: $1 \leq m \leq n \leq 10^4$, $0 \leq a_i \leq 10^6$. Target $O(n \log(\sum a))$. Edge cases: $m = 1$; $m = n$.

SOLUTION

Read the check for Example 16 again. “Fit the parcels into at most days trucks without any truck going over cap” is **word for word** the same as “cut the array into at most m blocks with no block over cap”. Same function, new name.

The same solver, renamed

JS

```
const minLargestBlock = (a, m) => minCapacity(a, m); // exactly the same
feasibility test
```

Tested on $[7, 2, 5, 10, 8]$: $m = 2$ gives 18 (blocks $7\ 2\ 5$ and $10\ 8$), $m = 3$ gives 14 (blocks $7\ 2\ 5, 10, 8$), and $[1, 1, 1, 1]$ with $m = 4$ gives 1.

Time $O(n \log(\sum a))$ **Space** $O(1)$

SHORTCUT

This is the highest-value fact in the whole chapter. “Minimum largest page count”, “minimum truck capacity”, “fewest painters”, “minimum time for machines in a row” are the **same problem** with different nouns. Recognise the check, not the story.

“At most m blocks” and “exactly m blocks” give the same answer whenever the array has at least m elements: if you used fewer blocks, split any block and the largest sum cannot grow.

Example 19 [SCB · pattern]

n shop positions on a straight road. Install k wireless routers in k of those positions so that the **smallest** distance between any two routers is as large as possible. Return that distance. Constraints: $2 \leq k \leq n \leq 10^5$, positions up to 10^9 and given unsorted. Target $O(n \log n + n \log(\text{range}))$. Edge cases: all positions equal; $k > n$.

SOLUTION

This is a **maximise the minimum** problem, so the feasibility test flips: large gaps are hard, small gaps are easy, and the pattern is T T T F F F. You want the **last** T.

feasible: can k routers all sit at least gap apart?

JS

```
const canPlace = (pos, k, gap) => {
  let cnt = 1, last = pos[0];
  for (let i = 1; i < pos.length; i++) {
    if (pos[i] - last >= gap) { cnt++; last = pos[i]; }
  }
  return cnt >= k;
};
```

Approach 1 — Try every gap from 0 upwards and keep the last that works ($O(n \text{ times the range of gaps})$)

Brute force / test oracle

JS

```
const maxMinGapBrute = (pos, k) => {
  const p = [...pos].sort((a, b) => a - b);
  if (p.length < k) return -1;
  let best = 0;
```

```

for (let g = 0; g <= p[p.length - 1] - p[0]; g++)
  if (canPlace(p, k, g)) best = g;
return best;
};

```

Time $O(n \times \text{range})$ **Space** $O(1)$

Approach 2 — Binary search the gap $O(n \log(\text{range}))$ — optimal

Last-true search: keep the answer, push lo up

JS

```

const maxMinGap = (pos, k) => {
  const p = [...pos].sort((a, b) => a - b); // numeric comparator, always
  if (p.length < k) return -1;
  let lo = 0, hi = p[p.length - 1] - p[0], ans = 0;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (canPlace(p, k, mid)) { ans = mid; lo = mid + 1; }
    else { hi = mid - 1; }
  }
  return ans;
};

```

Tested on [1,2,8,4,9]: $k = 3$ gives 3 (routers at 1, 4, 8) and $k = 2$ gives 8. [5,5,5] with $k = 2$ gives 0. [1,2] with $k = 5$ returns -1. Cross-checked against a linear scan of every possible gap on 500 random inputs.

Time $O(n \log n + n \log(\text{range}))$ **Space** $O(1)$

TRAP

Greedy placement must always take the **first** position, and then the earliest position that is far enough. Taking anything later only wastes road and can turn a feasible gap into an infeasible one.

Example 20 [Agoda · pattern]

n machines stand in a row. Machine i is repaired on day $\text{day}[i]$ and stays working afterwards. You need m separate groups of k **adjacent** working machines, and no machine may belong to two groups. What is the earliest day this is possible? Return -1 if it never is. Constraints: $1 \leq n \leq 10^5$, $1 \leq \text{day}_i \leq 10^9$. Target $O(n \log n)$. Edge cases: $m \times k > n$; every machine repaired on the same day.

SOLUTION

The check. On a given day, walk the row and count runs of working machines, cutting off a group every time the run reaches k .

feasible: enough groups by today?

JS

```

const enoughGroups = (day, m, k, today) => {
  let run = 0, groups = 0;
  for (const d of day) {
    if (d <= today) {
      run++;
      if (run === k) { groups++; run = 0; }
    } else {
      run = 0;
    }
  }
}

```

```

    }
    return groups >= m;
};

```

Monotone, because a machine repaired by day t is also repaired by day $t + 1$. The day never needs to be a number outside the list, so search between the smallest and the largest repair day.

Approach 1 — Walk the days upward ($O(n \times \text{range of days})$, far too slow)

Brute force / test oracle

JS

```

const earliestDayBrute = (day, m, k) => {
  if (m * k > day.length) return -1;
  const hi = Math.max(...day);
  for (let d = Math.min(...day); d <= hi; d++)
    if (enoughGroups(day, m, k, d)) return d;
  return -1;
};

```

Time $O(n \times \text{range of days})$ **Space** $O(1)$

Approach 2 — Binary search the day ($O(n \log n)$ — optimal)

Earliest feasible day

JS

```

const earliestDay = (day, m, k) => {
  if (m * k > day.length) return -1;
  let lo = Math.min(...day), hi = Math.max(...day);
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (enoughGroups(day, m, k, mid)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
};

```

Tested on $[1, 5, 2, 8, 3, 9]$: $m = 2$, $k = 2$ gives day 8, and a linear scan agrees. $m = 3$, $k = 2$ gives 9. $[4, 4, 4]$ with $m = 1$, $k = 3$ gives 4. The impossible case $m \times k > n$ returns -1.

Time $O(n \log n)$ **Space** $O(1)$

TRAP

$m * k$ with both near 10^5 gives 10^{10} . In a 32-bit language that wraps and the impossible case sails past the guard; in JavaScript it is fine, because numbers stay exact to 9×10^{15} — tested: `100000 * 100000` prints `10000000000` exactly. Say this out loud in an interview. It is one of the few places where JavaScript needs less care than C++ or Java, and knowing **why** is what earns the mark.

TRAP

`Math.min(...day)` spreads the whole array into function arguments. Past roughly 10^5 values that throws `RangeError: Maximum call stack size exceeded`. At this problem's limit of $n \leq 10^5$ you are on the edge — write the loop instead: `let lo = day[0]; for (const d of day) if (d < lo) lo = d;`

Example 21 [GIC · pattern]

Pick the smallest whole number d such that $\sum_i \lceil a_i/d \rceil \leq \text{limit}$. Constraints: $1 \leq n \leq 10^4$, $1 \leq a_i \leq 10^6$, $n \leq \text{limit} \leq 10^6$. Target $O(n \log(\max a))$. Edge cases: $\text{limit} = n$ (every term must be 1); limit very large.

SOLUTION

Raising d can only lower each ceiling, so the test is monotone.

Approach 1 — Try every divisor $O(n \text{ times } \max a)$, ten billion steps

Brute force / test oracle

JS

```
const smallestDivisorBrute = (a, limit) => {
  const hi = Math.max(1, ...a);
  for (let d = 1; d <= hi; d++) if (divisorWorks(a, d, limit)) return d;
  return hi;
};
```

Time $O(n \max a)$ **Space** $O(1)$

Approach 2 — Binary search the divisor $O(n \log(\max a))$ — optimal

feasible and the search

JS

```
const divisorWorks = (a, d, limit) => {
  let s = 0;
  for (const x of a) {
    s += Math.ceil(x / d);
    if (s > limit) return false;
  }
  return true;
};

const smallestDivisor = (a, limit) => {
  let lo = 1, hi = 1;
  for (const x of a) hi = Math.max(hi, x);
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (divisorWorks(a, mid, limit)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
};
```

Tested on $[2, 7, 11, 15]$: $\text{limit} = 9$ gives 5 — the ceilings are $1 + 2 + 3 + 3 = 9$, and $d = 4$ gives $1 + 2 + 3 + 4 = 10$, one too many. $\text{limit} = 4$ gives 15. $[5]$ with $\text{limit} = 1$ gives 5. $[1, 1, 1]$ with $\text{limit} = 3$ gives 1. The brute force returns the same four answers.

Time $O(n \log(\max a))$ **Space** $O(1)$

Example 22 [Razer · pattern]

A sorted array and a value x . Return the k values closest to x , in increasing order. Ties go to the smaller value. Constraints: $1 \leq k \leq n \leq 10^5$. Target $O(\log n + k)$ or better. Edge cases: x outside the array on either side; $k = n$.

SOLUTION

Approach 1 — Find the insertion point, then walk outwards $O(\log n + k)$

```
const kClosestExpand = (a, k, x) => {
  const i = lowerBound(a, x);
  let lo = i - 1, hi = i, left = k;
  while (left > 0) {
    if (lo < 0) { hi++; }
    else if (hi >= a.length) { lo--; }
    else if (x - a[lo] <= a[hi] - x) { lo--; }
    else { hi++; }
    left--;
  }
  return a.slice(lo + 1, hi);
};
```

Time $O(\log n + k)$ **Space** $O(1)$

Approach 2 — Binary search the window's left edge ($O(\log n)$ to decide, $O(k)$ to copy)

The answer is always a **contiguous** block of k values. So search for its starting index directly. Compare the value just outside on the left with the value just outside on the right.

Binary search over start positions

JS

```
const kClosestBS = (a, k, x) => {
  let lo = 0, hi = a.length - k;
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (x - a[mid] > a[mid + k] - x) lo = mid + 1;
    else hi = mid;
  }
  return a.slice(lo, lo + k);
};
```

Tested on $[1, 3, 6, 10, 11, 15]$: $k = 3, x = 9$ gives $6\ 10\ 11$ from both versions. $k = 6, x = 100$ gives the whole array. $k = 1, x = 0$ gives 1 .

Time $O(\log n + k)$ **Space** $O(1)$

The unlocking idea: prove the answer is one contiguous block, and a “choose k items” problem collapses into “choose one starting index”.

Tier 3 — the ones with a follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

Two sorted arrays of sizes n and m . Return the median of the combined list, without building it. Constraints: $0 \leq n, m \leq 10^5$, not both empty, $|a_i| \leq 2 \times 10^9$. Target $O(\log(\min(n, m)))$. Edge cases: one array empty; all of one array below all of the other; an even total length.

SOLUTION

Approach 1 — Merge and read the middle $(O(n + m))$ — good enough to be a test oracle

Brute force

JS

```
const medianBrute = (A, B) => {
  const all = [...A, ...B].sort((p, q) => p - q); // numeric comparator, always
  const n = all.length;
  if (n === 0) return 0;
  if (n % 2 === 1) return all[(n - 1) / 2];
  return (all[n / 2 - 1] + all[n / 2]) / 2;
};
```

Time $O((n + m) \log(n + m))$ **Space** $O(n + m)$

Approach 2 — Binary search the split point $(O(\log(\min(n, m))))$ — optimal

WHAT WE ARE ACTUALLY SEARCHING FOR

Cut A after i values and B after j values, with $i + j = \lceil (n + m) / 2 \rceil$. The cut is **correct** when everything on the left is $\leq$ everything on the right, which needs only two comparisons:

$$A[i-1] \leq B[j] \quad \text{and} \quad B[j-1] \leq A[i]$$

If $A[i-1] > B[j]$, we took too much from A, so shrink i . That is monotone, so binary search on i works. Because j is forced by i , there is only one thing to search.

Missing neighbours are replaced by $-\infty$ and $+\infty$ so the two comparisons never need a special case. JavaScript spells those `-Infinity` and `Infinity`, and they compare correctly against every real number — much cleaner than borrowing the smallest and largest values of a fixed-width integer type.

Median of two sorted arrays

JS

```
const medianTwo = (A, B) => {
  if (A.length > B.length) return medianTwo(B, A);
  const n = A.length, m = B.length, total = n + m;
  if (total === 0) return 0;
  const half = Math.floor((total + 1) / 2);
  let lo = 0, hi = n;
  while (lo <= hi) {
    const i = Math.floor(lo + (hi - lo) / 2);
    const j = half - i;
    const aLeft = i > 0 ? A[i - 1] : -Infinity;
```

```

const aRight = i < n ? A[i]      : Infinity;
const bLeft  = j > 0 ? B[j - 1] : -Infinity;
const bRight = j < m ? B[j]      : Infinity;
if (aLeft <= bRight && bLeft <= aRight) {
    if (total % 2 === 1) return Math.max(aLeft, bLeft);
    return (Math.max(aLeft, bLeft) + Math.min(aRight, bRight)) / 2;
}
if (aLeft > bRight) hi = i - 1;
else                lo = i + 1;
}
return 0;
};

```

Tested: ([1,3,8], [7,9,10,11]) gives 8.0 and the merge agrees. ([], [2,4]) gives 3.0. ([5], [5]) gives 5.0. ([-2000000000], [2000000000]) gives 0, and ([1,2], [3,4]) gives 2.5. Cross-checked against the merge on 500 random pairs of arrays.

Time $O(\log(\min(n, m)))$ **Space** $O(1)$

Python version

PYTHON

```

def median_two(A, B):
    if len(A) > len(B):
        A, B = B, A
    n, m = len(A), len(B)
    total = n + m
    if total == 0:
        return 0.0
    half = (total + 1) // 2
    lo, hi = 0, n
    while lo <= hi:
        i = lo + (hi - lo) // 2
        j = half - i
        al = A[i - 1] if i > 0 else float("-inf")
        ar = A[i] if i < n else float("inf")
        bl = B[j - 1] if j > 0 else float("-inf")
        br = B[j] if j < m else float("inf")
        if al <= br and bl <= ar:
            if total % 2 == 1:
                return float(max(al, bl))
            return (max(al, bl) + min(ar, br)) / 2.0
        if al > br:
            hi = i - 1
        else:
            lo = i + 1
    return 0.0

```

Python prints 8.0 3.0 5.0 on the same three cases.

Follow-up the interviewer asks next: “now give me the k-th smallest, not the median.” Same code, with half replaced by k and the even-length branch deleted. Searching i from max(0, k - m) to min(k, n) keeps j inside B.

TRAP

hi starts at n , not $n - 1$. The cut $i = n$ (take all of A) is a legal answer, and a range that cannot express it fails on $([1, 2], [3, 4])$.

Example 24 [Amazon · pattern]

Search a rotated sorted array that **may contain duplicates**. Return only whether the target is present. Constraints: $0 \leq n \leq 10^5$. Target: $O(\log n)$ on average. Edge cases: every value identical; target absent; empty array.

SOLUTION

Example 10 decided which half was sorted by comparing $a[\text{lo}]$ with $a[\text{mid}]$. With duplicates that test can be blind: in $[4, 4, 4, 1, 2, 4, 4]$ we have $a[\text{lo}] == a[\text{mid}] == a[\text{hi}] == 4$ and the sorted half is genuinely unknowable from those three values.

The honest fix is to admit defeat for one step and shrink the range by one on each side.

Rotated search with duplicates

JS

```
const searchRotatedDup = (a, t) => {
  let lo = 0, hi = a.length - 1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] === t) return true;
    if (a[lo] === a[mid] && a[mid] === a[hi]) { lo++; hi--; }
    else if (a[lo] <= a[mid]) {
      if (a[lo] <= t && t < a[mid]) hi = mid - 1;
      else lo = mid + 1;
    } else {
      if (a[mid] < t && t <= a[hi]) lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return false;
};
```

Tested on $[4, 4, 4, 1, 2, 4, 4]$: 2 is found, 4 is found, 3 is not. An empty array returns false; $[2, 2, 2]$ finds 2.

Time $O(\log n)$ average **Space** $O(1)$
 $O(n)$ in the worst case

Follow-up the interviewer asks next: “what is the worst case, and can you beat it?” Answer honestly: on an array like $[2, 2, 2, 2, 2, 2, 1, 2]$ every step hits the blind branch and you degrade to $O(n)$. No comparison-based method can do better, because an adversary can keep $n - 1$ copies of one value and hide the odd one anywhere; distinguishing all n placements needs n probes in the worst case. That “prove the lower bound” answer is what separates a Tier 3 candidate from a Tier 1 one.

Example 25 [Microsoft · pattern]

An $n \times n$ grid where every **row** is sorted left to right and every **column** is sorted top to bottom. Find the k -th smallest value overall. Constraints: $1 \leq n \leq 300$, $1 \leq k \leq n^2$. Target $O(n \log(\text{range}))$. Edge cases: $k = 1$; $k = n^2$; many repeated values.

SOLUTION

This grid is **not** one sorted list — row 2 can start below the end of row 1 — so the flattening trick from Example 11 is illegal here.

Approach 1 — Copy everything out and sort ($O(n^2 \log n)$ time and $O(n^2)$ space)

Brute force / test oracle

JS

```
const kthSmallestBrute = (g, k) => g.flat().sort((a, b) => a - b)[k - 1];
```

Time $O(n^2 \log n)$ **Space** $O(n^2)$

Approach 2 — Binary search on the VALUE, not on a position ($O(n \log(\text{range}))$) — optimal

Guess a value x and count how many grid entries are $\leq x$. That count only grows as x grows, so it is monotone. Counting is a staircase walk: start at the top-right, step left while the value is too big, and step down a row.

Count, then search the value range

JS

```
const countNotAbove = (g, x) => {
  const n = g.length;
  let col = n - 1, cnt = 0;
  for (let row = 0; row < n; row++) {
    while (col >= 0 && g[row][col] > x) col--;
    cnt += col + 1;
  }
  return cnt;
};

const kthSmallestMatrix = (g, k) => {
  const n = g.length;
  let lo = g[0][0], hi = g[n - 1][n - 1];
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (countNotAbove(g, mid) >= k) hi = mid;
    else lo = mid + 1;
  }
  return lo;
};
```

Tested on `[[1,4,9],[2,6,12],[5,8,14]]`, the answers for $k = 1..9$ are 1 2 4 5 6 8 9 12 14, and the brute force gives the identical list.

Time $O(n \log(\text{range}))$ **Space** $O(1)$

Why is `col` never reset between rows? Because rows go down as well as across: whatever was too big in row r is still too big in row $r+1$ at the same column. So `col` only ever moves left, and the whole count costs $O(n)$, not $O(n^2)$.

SHORTCUT

The returned `lo` is always a value that really appears in the grid. Reason: the smallest x whose count reaches k must itself be a grid entry — if it were not, $x - 1$ would have the same count and x would not be smallest.

Follow-up the interviewer asks next: “compare this with a heap solution.” A min-heap seeded with the first column and popped k times costs $O(k \log n)$. That is better when k is tiny and worse when k is near n^2 . Binary search wins on memory: $O(1)$ against $O(n)$.

Example 26 [Uber · pattern]

A grid of distinct numbers. Find **any** peak: a cell strictly greater than its up, down, left and right neighbours. Cells off the edge count as $-\infty$. Constraints: $1 \leq \text{rows}, \text{cols} \leq 500$. Target $O(\text{rows} \times \log \text{cols})$. Edge cases: a single row; a single column.

SOLUTION

Take the middle column. Find its largest entry, at row `best`. Three cases:

1. Both horizontal neighbours are smaller. Then that cell is a peak — it already beat its vertical neighbours by being the column maximum.
2. The left neighbour is bigger. Then the left half must contain a peak, because the largest value in the left half is at least that neighbour and cannot escape leftwards forever.
3. Same argument on the right.

Binary search over columns

JS

```
const peakGrid = (g) => {
  const rows = g.length, cols = g[0].length;
  let lo = 0, hi = cols - 1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    let best = 0;
    for (let r = 0; r < rows; r++) if (g[r][mid] > g[best][mid]) best = r;
    const leftBigger = mid - 1 >= 0 && g[best][mid - 1] > g[best][mid];
    const rightBigger = mid + 1 < cols && g[best][mid + 1] > g[best][mid];
    if (!leftBigger && !rightBigger) return [best, mid];
    if (leftBigger) hi = mid - 1;
    else lo = mid + 1;
  }
  return [-1, -1];
};
```

Tested on `[[3,9,4],[7,2,6],[1,8,5]]`: returns `(0, 1)`, whose value 9 beats its left 3, its right 4 and the cell below, 2.

Time $O(\text{rows} \times \log \text{cols})$ **Space** $O(1)$

Follow-up the interviewer asks next: “why is that $O(r \log c)$ and not $O(rc)$?” Each round scans one column (r steps) and then throws away half the columns. Number of rounds: $\log_2 c$. Total: $r \log_2 c$. For a 500×500 grid that is about 4500 cell reads instead of 250000.

Example 27 [Goldman Sachs · pattern]

A sorted array where every value appears exactly twice, except one value that appears once. Find the lonely value. Constraints: n is odd, $1 \leq n \leq 10^5$. Target $O(\log n)$ time, $O(1)$ space. Edge cases: the lonely value is first; it is last; $n = 1$.

SOLUTION

Before the lonely value, every pair starts at an **even** index: $(0, 1)$, $(2, 3)$, $(4, 5)$. After it, every pair starts at an **odd** index. That flip is monotone — it happens once.

Binary search on index parity

JS

```
const singleValue = (a) => {
  let lo = 0, hi = a.length - 1;
  while (lo < hi) {
    let mid = Math.floor(lo + (hi - lo) / 2);
    if (mid % 2 === 1) mid--; // always look at the START of a pair
    if (a[mid] === a[mid + 1]) lo = mid + 2;
    else hi = mid;
  }
  return a[lo];
};
```

Tested: `[1,1,3,5,5,7,7]` gives 3, `[2,3,3]` gives 2, `[9]` gives 9, `[1,1,2]` gives 2.

Time $O(\log n)$ **Space** $O(1)$

The line `if (mid % 2 == 1) mid--;` is the whole trick. It snaps `mid` onto the first member of a pair, so `a[mid] == a[mid+1]` is a clean “the pairs are still aligned” test. Without it the comparison means nothing.

Follow-up the interviewer asks next: “what if every value appears three times except one?” Parity becomes “index modulo 3”, and the check compares `a[mid]`, `a[mid+1]` and `a[mid+2]` after snapping `mid` down to a multiple of 3. The shape of the argument is identical, which is the point of the question.

Example 28 [D. E. Shaw · pattern]

Charging points sit at increasing positions along a highway. You may add k more points anywhere. Minimise the largest distance between two neighbouring points. Answer to within 10^{-6} . Constraints: $2 \leq n \leq 10^4$, $0 \leq k \leq 10^5$, positions up to 10^9 . Edge cases: $k = 0$; all gaps already equal.

SOLUTION

Binary search on a **real** answer width. The check: with a maximum spacing of width, a gap of length g needs $\lceil g / \text{width} \rceil - 1$ extra points inside it.

Real-valued binary search on the answer

JS

```
const stationsNeeded = (pos, width) => {
  let need = 0;
  for (let i = 1; i < pos.length; i++) {
    const gap = pos[i] - pos[i - 1];
    need += Math.ceil(gap / width) - 1;
  }
  return need;
};

const minLargestGap = (pos, k) => {
  let lo = 0, hi = 0;
  for (let i = 1; i < pos.length; i++) hi = Math.max(hi, pos[i] - pos[i - 1]);
  for (let it = 0; it < 100; it++) {
    const mid = (lo + hi) / 2;
    if (mid <= 0) break;
    if (stationsNeeded(pos, mid) <= k) hi = mid;
    else lo = mid;
  }
  return hi;
};
```

Tested: `([0,10], 1)` gives 5.000000, `([0,10], 4)` gives 2.000000, `([1,2,3,4], 0)` gives 1.000000.

Time $O(100n)$ **Space** $O(1)$

Follow-up the interviewer asks next: “the judge wants an exact answer, not a float.” Two options. Scale everything by 10^6 and binary-search on integers, which removes all floating-point doubt. Or use a max-heap that repeatedly splits the currently largest gap, k times — $O((n + k) \log n)$, exact, and a nice contrast to draw in the interview.

TRAP

`hi` must start at the largest **gap**, not at the length of the whole highway. Starting too high still works but wastes iterations; starting too low returns a wrong answer, which is much worse.

Dry run — every single step

Problem. $a = [31, 40, 55, 3, 8, 12, 19, 24]$ — a sorted array of distinct values, rotated. Search for 20, which is **not** there. Watching a failed search is more useful than watching a lucky hit, because it shows how the range actually collapses.

The code we are tracing

JS

```
let lo = 0, hi = a.length - 1;
while (lo <= hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (a[mid] === t) return mid;
  if (a[lo] <= a[mid]) { // left half is sorted
    if (a[lo] <= t && t < a[mid]) hi = mid - 1;
    else lo = mid + 1;
  } else { // right half is sorted
    if (a[mid] < t && t <= a[hi]) lo = mid + 1;
    else hi = mid - 1;
  }
}
return -1;
```

Index: 0:31 1:40 2:55 3:3 4:8 5:12 6:19 7:24

step	lo	hi	mid	decision	new range
1	0	7	3	$a[3] = 3$. Is $a[lo] \leq a[mid]$? $31 \leq 3$ is false, so the right half $[3..7]$ is sorted, holding 3 .. 24. Is $3 < 20 \leq 24$? Yes — the target can only be there.	$lo = 4$
2	4	7	5	$a[5] = 12$. Is $8 \leq 12$? Yes, so the left half $[4..5]$ is sorted, holding 8 .. 12. Is $8 \leq 20 < 12$? No. So go right.	$lo = 6$
3	6	7	6	$a[6] = 19$. Left half $[6..6]$ is sorted, holding just 19. Is $19 \leq 20 < 19$? No. Go right.	$lo = 7$
4	7	7	7	$a[7] = 24$. Left half $[7..7]$ holds just 24. Is $24 \leq 20 < 24$? No. Go right.	$lo = 8$
—	8	7	—	$lo > hi$, the loop ends.	return -1

Answer: -1. Four probes on an array of eight, which matches $\log_2 8 = 3$ plus the final empty check.

The same array, a target that exists

Searching for 55:

step	lo	hi	mid	decision	new range
1	0	7	3	$a[3] = 3$; right half $[3..7]$ sorted, holds 3 .. 24. Is $3 < 55 \leq 24$? No. Go left.	$hi = 2$
2	0	2	1	$a[1] = 40$; left half $[0..1]$ sorted, holds 31 .. 40. Is $31 \leq 55 < 40$? No. Go right.	$lo = 2$
3	2	2	2	$a[2] = 55$ — hit.	return 2

Three things to notice

1. Every step made the range strictly smaller. That is the only guarantee against an infinite loop, and it is worth checking by eye on every binary search you write.
2. The “which half is sorted” test used $a[lo]$, not $a[0]$. Using a fixed endpoint instead of the current one is a very common bug.

3. The failed search cost the same as the successful one. Binary search has no fast path — it always pays $\log n$.

TRAP

If your trace ever shows the same `lo`, `hi` pair twice in a row, you have an infinite loop. The usual cause is `lo = mid` instead of `lo = mid + 1`.

Practice

PRACTICE · TIER 1 target: 45 min for 1-5

1. **Count inside a range.** Sorted array with duplicates. How many values lie in the closed range $[L, R]$? $0 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: $L > R$; the range covers everything.
2. **Integer cube root.** Largest r with $r^3 \leq n$, for $0 \leq n \leq 10^{18}$. Target $O(\log n)$. Edge cases: $n = 0$; a perfect cube; overflow of $r*r*r$.
3. **Fixed point.** A strictly increasing array of integers. Find the smallest index i with $a[i] == i$, or -1 . $0 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: no fixed point; several fixed points.
4. **The k-th missing number.** A strictly increasing array of positive integers. Which is the k -th positive integer that is **not** in it? $0 \leq n \leq 10^5$, $1 \leq k \leq 10^9$. Target $O(\log n)$. Edge cases: empty array; nothing missing before the array starts.
5. **Mountain peak.** The array rises then falls, with a single peak. Return the peak index. $3 \leq n \leq 10^5$. Target $O(\log n)$. Edge cases: peak next to an end.

PRACTICE · TIER 2 target: 50 min for 6-11

6. **Search a mountain.** Same mountain shape. Find target, preferring the smaller index. Target $O(\log n)$. Edge cases: target on both slopes; target absent.
7. **Unknown length.** A sorted stream you can only read through `get(i)`, which returns `Infinity` past the end. Find target without knowing the length. Target $O(\log p)$ where p is the answer's index. Edge cases: target at index 0; target absent.
8. **Machines.** `speed[i]` is the number of minutes machine i needs per item, and all machines run in parallel. What is the fewest minutes to produce n items in total? $1 \leq \text{speed}_i \leq 10^6$, $1 \leq n \leq 10^6$. Target $O(m \log(\min \text{speed} \times n))$. Edge cases: one machine; $n = 1$.
9. **Pairs under a limit.** Sorted array. Count the pairs $i < j$ with $a_i + a_j \leq X$. $0 \leq n \leq 10^5$, values may be negative. Target $O(n \log n)$. Edge cases: no pair qualifies; the count is large (at $n = 10^5$ there are about 5×10^9 pairs, still exact as a JavaScript number).
10. **Ceiling square root.** Smallest k with $k^2 \geq n$, for $0 \leq n \leq 10^{18}$. Edge cases: $n = 0$; n a perfect square.
11. **Closest value.** Sorted array. Return the value closest to x ; on a tie return the smaller one. Target $O(\log n)$. Edge cases: x below or above everything.

PRACTICE · TIER 3 target: 45 min for 12-15

12. **The k-th smallest difference.** Given an array, consider the absolute difference of every pair. Return the k -th smallest of those differences. $2 \leq n \leq 10^4$. Target $O(n \log n + n \log(\text{range}))$. Edge cases: repeated values (difference 0); $k = 1$.
13. **Real square root.** $\sqrt{n}$ to six decimals for $0 \leq n \leq 10^{12}$, without `sqrt`. State how many iterations you need and why.
14. **The generic helper.** Write one reusable `firstTrue(lo, hi, pred)` that returns the smallest value in $[lo, hi]$ where `pred` is true, or `hi + 1` if never. Then say which two problems from this chapter it replaces outright.
15. **Rope cutting.** n ropes of given whole lengths. Cut pieces of one equal whole length L (leftovers are wasted). What is the largest L that still yields at least k pieces? Return 0 if even $L = 1$ cannot. $1 \leq n \leq 10^5$, lengths up to 10^9 , $1 \leq k \leq 10^9$. Edge cases: k larger than the total length.

ANSWER KEY

1. Both bounds, subtracted.

Count inside [L, R]

JS

```
const countInRange = (a, L, R) => {
  if (L > R) return 0;
  return upperBound(a, R) - lowerBound(a, L);
};
```

Tested on [1,3,3,7,9,14,14,20]: [3,14] gives 6, [4,6] gives 0, [0,100] gives 8, and a reversed range gives 0. $O(\log n) / O(1)$.

2. Cap hi at 10^6 , because $(10^6)^3 = 10^{18}$ already covers the whole input range. The cube itself must be **BigInt**: tested in Node, $999999 * 999999 * 999999$ gives 9999970000002999900, but the true cube is 9999970000002999999. Plain numbers are wrong by 99 here.

Integer cube root

JS

```
const icbrt = (n) => { // n is a BigInt, and so is the answer
  let lo = 0n, hi = 1000000n, ans = 0n;
  while (lo <= hi) {
    const mid = lo + (hi - lo) / 2n;
    if (mid * mid * mid <= n) { ans = mid; lo = mid + 1n; }
    else { hi = mid - 1n; }
  }
  return ans;
};
```

Tested: icbrt(0n) gives 0n, 1n gives 1n, 26n gives 2n, 27n gives 3n, 28n gives 3n, and 1000000000000000000n gives 1000000n. Checked against a linear count-up for every n from 0 to 2000. $O(\log n) / O(1)$.

3. Because the array is strictly increasing, $a[i] - i$ never decreases. So $a[i] \geq i$ is monotone and you can search for its first true.

Smallest fixed point

JS

```
const fixedPoint = (a) => {
  let lo = 0, hi = a.length - 1, ans = -1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] >= mid) { if (a[mid] === mid) ans = mid; hi = mid - 1; }
    else { lo = mid + 1; }
  }
  return ans;
};
```

Tested: [-5, -2, 2, 4, 9] gives 2 and a linear scan agrees; [1, 2, 3] gives -1; [0] gives 0; [] gives -1. Cross-checked against the scan on 1000 random strictly increasing arrays. $O(\log n) / O(1)$.

4. Before index i the array has skipped $a[i] - (i + 1)$ positive numbers. That count never decreases, so find the first index where it reaches k.

k-th missing positive

JS

```
const kthMissing = (a, k) => {
  let lo = 0, hi = a.length;
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] - (mid + 1) < k) lo = mid + 1;
    else hi = mid;
  }
  return lo;
```

```

    }
    return k + lo;
};

```

Tested on [2,3,4,7,11]: $k = 1$ gives 1, $k = 5$ gives 9. An empty array with $k = 3$ gives 3; [1,2,3] with $k = 2$ gives 5. Cross-checked against a set-based scan on 1000 random arrays for k from 1 to 8. $O(\log n) / O(1)$.

5. Identical to Example 8 — walk uphill.

Mountain peak

JS

```

const mountainPeak = (a) => {
  let lo = 0, hi = a.length - 1;
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] < a[mid + 1]) lo = mid + 1;
    else hi = mid;
  }
  return lo;
};

```

Tested: [1,4,8,12,7,5,2] gives 3. $O(\log n) / O(1)$.

6. Peak first, then one ordinary search on the rising side and one **reversed** search on the falling side.

Search a mountain array

JS

```

const searchMountain = (a, t) => {
  const p = mountainPeak(a);
  let lo = 0, hi = p; // rising part
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] === t) return mid;
    if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
  }
  lo = p + 1; hi = a.length - 1; // falling part
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (a[mid] === t) return mid;
    if (a[mid] > t) lo = mid + 1; else hi = mid - 1;
  }
  return -1;
};

```

Tested on [1,4,8,12,7,5,2]: 8 gives 2, 5 gives 5, 12 gives 3, 100 gives -1. Note the flipped comparison in the second loop — that one character is the whole problem. $O(\log n) / O(1)$.

7. Double the upper bound until it passes the target, then search inside. Doubling costs $O(\log p)$ steps, so the total is still $O(\log p)$.

Exponential search then binary search

JS

```

const unboundedSearch = (get, target) => {
  let hi = 1;
  while (get(hi) !== Infinity && get(hi) < target) hi *= 2;
  let lo = Math.floor(hi / 2);
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);

```

```

    const v = get(mid);
    if (v === target) return mid;
    if (v < target) lo = mid + 1; else hi = mid - 1;
  }
  return -1;
};

```

Tested on the stream 2 4 6 8 10 12 14 16 18: 14 gives index 6, 2 gives 0, 5 gives -1, 18 gives 8. $O(\log p)$ time, $O(1)$ space.

8. Guess a time t . Machine i finishes $t / \text{speed}[i]$ items in that time.

Minimum build time

JS

```

const madeEnough = (speed, t, n) => {
  let made = 0;
  for (const s of speed) {
    made += Math.floor(t / s);
    if (made >= n) return true;
  }
  return made >= n;
};

const minBuildTime = (speed, n) => {
  let lo = 1, hi = Math.min(...speed) * n;
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (madeEnough(speed, mid, n)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
};

```

Tested: $([2, 3], 5)$ gives 6 — in 6 minutes the machines make 3 and 2 items. $([4], 1)$ gives 4. $([1, 1, 1], 3)$ gives 1. $([1000000], 1000000)$ gives 1000000000000 — a trillion, comfortably inside the exact range of a JavaScript number, so no `BigInt` is needed here. $O(m \log(\min \text{speed} \times n))$ time, $O(1)$ space.

9. For each i , binary-search the last j whose value fits. The early `return true` inside `madeEnough` above is the same idea: stop as soon as the answer is settled.

Count pairs with sum at most X

JS

```

const countPairsAtMost = (a, X) => {
  let c = 0;
  const n = a.length;
  for (let i = 0; i < n; i++) {
    const limit = X - a[i];
    let lo = i + 1, hi = n; // first index with a[idx] > limit
    while (lo < hi) {
      const mid = Math.floor(lo + (hi - lo) / 2);
      if (a[mid] <= limit) lo = mid + 1;
      else hi = mid;
    }
    c += lo - (i + 1);
  }
  return c;
};

```

Tested on $[-4, -1, 0, 2, 5, 9]$: $X = 3$ gives 7 and the brute force agrees; $X = -10$ gives 0. Empty and single-element arrays give 0. Cross-checked against a double loop on 1000 random arrays for X from -12 to 40. $O(n \log n) / O(1)$.

Chapter 3 solves this same problem in $O(n)$ with two pointers. Binary search is the answer you can write in 60 seconds; two pointers is the answer you offer as the follow-up.

10. hi is 2×10^9 because $(2 \times 10^9)^2 = 4 \times 10^{18}$. That square is past $2^{53} - 1$, so this one is **BigInt** too — exactly the same reason as the cube root.

Ceiling square root

JS

```
const ceilSqrt = (n) => { // n is a BigInt, and so is the answer
  let lo = 0n, hi = 2000000000n;
  while (lo < hi) {
    const mid = lo + (hi - lo) / 2n;
    if (mid * mid >= n) hi = mid;
    else lo = mid + 1n;
  }
  return lo;
};
```

Tested: `ceilSqrt(0n)` gives `0n`, `1n` gives `1n`, `24n` gives `5n`, `25n` gives `5n`, `26n` gives `6n`, and `10000000000000000000n` gives `10000000000n`. Verified for every n from 0 to 2000 that $k^2 \geq n$ and $(k-1)^2 < n$. $O(\log n) / O(1)$.

11. `lowerBound` lands on the first value that is not smaller; the answer is that value or the one before it.

Closest value

JS

```
const closestValue = (a, x) => {
  const i = lowerBound(a, x);
  if (i === 0) return a[0];
  if (i === a.length) return a[a.length - 1];
  const dl = x - a[i - 1], dr = a[i] - x;
  return dl <= dr ? a[i - 1] : a[i];
};
```

Tested on $[1, 3, 3, 7, 9, 14, 14, 20]$: $x = 8$ gives 7, $x = 0$ gives 1, $x = 100$ gives 20, $x = 11$ gives 9. $O(\log n) / O(1)$.

12. Binary search the **difference**. Counting pairs with difference at most d is a two-pointer sweep — this problem needs both chapters at once.

k-th smallest pair difference

JS

```
const countDiffAtMost = (a, d) => {
  let c = 0, left = 0;
  for (let right = 0; right < a.length; right++) {
    while (a[right] - a[left] > d) left++;
    c += right - left;
  }
  return c;
};

const kthDifference = (arr, k) => {
  const a = [...arr].sort((p, q) => p - q); // numeric comparator, always
  let lo = 0, hi = a[a.length - 1] - a[0];
  while (lo < hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (countDiffAtMost(a, mid) >= k) hi = mid;
    else lo = mid + 1;
  }
}
```

```

    return lo;
};

```

Tested on `[1,6,3,10]`: the answers for `k = 1..6` are `2 3 4 5 7 9`, exactly matching a brute force over all pairs. Cross-checked on 1000 random arrays for every `k`. $O(n \log n + n \log(\text{range}))$ time, $O(1)$ space.

13. One hundred halvings. Each one halves the error, so after 100 rounds the remaining interval is the start width divided by 2^{100} — far below what a JavaScript number can represent, which means it is exact to every digit the type can hold.

Real square root

JS

```

const sqrtReal = (n) => {
  let lo = 0, hi = Math.max(1, n);
  for (let it = 0; it < 100; it++) {
    const mid = (lo + hi) / 2;
    if (mid * mid <= n) lo = mid;
    else hi = mid;
  }
  return lo;
};

```

Tested: 2 prints 1.414214, 1e12 prints 1000000.000000, 0 prints 0.000000. In practice 60 rounds are already enough for a JavaScript number; 100 cost nothing and remove all doubt.

14. The helper, and the two problems it replaces.

One search to rule them all

JS

```

const firstTrue = (lo, hi, pred) => {
  let ans = hi + 1;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (pred(mid)) { ans = mid; hi = mid - 1; }
    else { lo = mid + 1; }
  }
  return ans;
};

```

Tested: `firstTrue(1, 100, v -> v*v >= 50)` gives 8; an always-false predicate on `[1,10]` gives 11; an always-true one gives 1. It replaces Example 12 (first failing build) and Example 16 (minimum truck capacity) with one line each — pass the feasibility check as `pred`. The Python version in Example 16 does exactly that. $O(\log(\text{range}))$ calls to `pred`.

15. Guess a length; count the pieces.

Largest cut length

JS

```

const piecesAt = (rope, L) => {
  let c = 0;
  for (const r of rope) c += Math.floor(r / L);
  return c;
};

const maxPieceLength = (rope, k) => {
  let lo = 1, hi = Math.max(...rope), ans = 0;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    if (piecesAt(rope, mid) >= k) { ans = mid; lo = mid + 1; }
    else { hi = mid - 1; }
  }
};

```

```
    return ans;  
};
```

Tested: $([9, 7, 21], 5)$ gives 7 — the ropes yield 1, 1 and 3 pieces of length 7, which is 5 in total. $([5], 5)$ gives 1. $([1, 1], 5)$ gives 0. $([1000000000], 1)$ gives 1000000000. $O(n \log(\text{max rope}))$ time, $O(1)$ space.

TRAP

L starts at 1, never at 0. Dividing by zero would crash before the answer 0 is ever reported, so keep the “impossible” case in `ans`, not in the range.

ONE-PAGE REVISION CARD

The three templates

T1 — exact match

JS

```
let lo = 0, hi = a.length - 1;
while (lo <= hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (a[mid] === t) return mid;
  if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
}
return -1;
```

T2 — first index that is good

JS

```
let lo = 0, hi = n;
while (lo < hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (!good(mid)) lo = mid + 1; else hi = mid;
}
return lo; // n means none
```

T3 — smallest feasible answer

JS

```
let lo = LOW, hi = HIGH;
while (lo < hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (feasible(mid)) hi = mid; else lo = mid + 1;
}
return lo;
```

T3' — largest feasible answer

JS

```
let lo = LOW, hi = HIGH, ans = -1;
while (lo <= hi) {
  const mid = Math.floor(lo + (hi - lo) / 2);
  if (feasible(mid)) { ans = mid; lo = mid + 1; }
  else { hi = mid - 1; }
}
return ans;
```

When to use what

Question says	Template	Time
find a value in a sorted array	T1	$O(\log n)$
first / last occurrence, count	T2 twice	$O(\log n)$
floor, ceiling, insert position	T2	$O(\log n)$
find a peak / any local maximum	T2 on the slope	$O(\log n)$

rotated sorted array	T1 + “which half is sorted”	$O(\log n)$
rotated with duplicates	same, plus <code>lo++</code> , <code>hi--</code>	$O(n)$ worst
minimum capacity / speed / size	T3	$O(n \log R)$
maximise the minimum spacing	T3'	$O(n \log R)$
k-th smallest in a sorted matrix	T3 on the value	$O(n \log R)$
median / k-th of two sorted arrays	T2 on the split point	$O(\log \min(n, m))$
answer is a real number	100 fixed halvings	$O(100n)$

Complexity you should be able to quote

Problem	Time	Space
search / lower bound / count	$O(\log n)$	$O(1)$
peak element	$O(\log n)$	$O(1)$
rotated search, distinct values	$O(\log n)$	$O(1)$
rotated search, duplicates allowed	$O(n)$ worst	$O(1)$
flattened sorted matrix	$O(\log(rc))$	$O(1)$
minimum capacity / speed on the answer	$O(n \log R)$	$O(1)$
k-th smallest in a sorted grid	$O(n \log R)$	$O(1)$
median of two sorted arrays	$O(\log \min(n, m))$	$O(1)$
peak in a 2D grid	$O(r \log c)$	$O(1)$

R means the width of the value range you are searching, not the array length.

Top traps, in the order they bite

1. **A fractional mid.** `(lo + hi) / 2` is real division in JavaScript. Always `Math.floor(lo + (hi - lo) / 2)`, and never `>>1`, which truncates to 32 bits.
2. **A branch that does not shrink the range.** `lo = mid` with `while (lo < hi)` hangs forever. Every branch must move `lo` up or `hi` down.
3. **Guard and range disagreeing.** `[lo, hi]` needs `lo <= hi` and `mid ± 1`. `[lo, hi)` needs `lo < hi` and `hi = mid`.
4. **Trusting lowerBound without checking the value.** It returns an insert position when the target is absent.
5. **mid * mid inside the loop.** Stops being exact past $2^{53} - 1$. Use `mid <= n / mid`, cap `hi` so the product is provably safe, or move the whole search to `BigInt`.
6. **A feasibility check that is not monotone.** Then binary search is simply the wrong tool and no amount of debugging will fix it. Test the check on paper for three or four values first.
7. **A wrong starting range in T3.** Too high only wastes iterations; too low returns a confidently wrong answer.
8. **while (hi - lo > eps) on real numbers.** Can spin forever. Use a fixed iteration count.
9. **Comparing against `a[0]` or `a[n-1]` instead of `a[lo]` / `a[hi]`** in the rotated problems.

The 30-second checklist before you code

1. What exactly am I searching over — an index, or a real-world quantity?
2. Write the yes/no test as a sentence. Is it truly F F F T T T?
3. What are the smallest and largest candidates? Justify both out loud.
4. Closed or half-open range? Pick one and match the guard.

5. Does every branch shrink the range?
6. Can any intermediate value pass $2^{53} - 1 = 9007199254740991$? If yes, BigInt everywhere in that search — and never $\gg 1$ for the midpoint.

Sorting & Custom Comparators

Order is not the answer. Order is the shortcut that makes the answer easy.

THE ONE IDEA OF THIS CHAPTER

You almost never sort because the question says “sort this”. You sort because **after sorting, a hard question becomes an easy scan.**

Hard on unsorted data	Easy after sorting
closest pair of values	neighbours only: check $a[i] - a[i - 1]$
count pairs with a given sum	two pointers from both ends
k-th smallest	index $k - 1$
how many duplicates	equal values sit together
best greedy pick order	scan left to right once

Cost of the shortcut: $O(n \log n)$ time. You pay it once, then everything after is $O(n)$.

Pattern in one page

The four shapes of sort

The only four ways you will ever call sort

JS

```
const v = [4, 1, 3];
v.sort(); // 1. default: LEXICOGRAPHIC, almost never right
v.sort((a, b) => a - b); // 2. numbers ascending
v.sort((a, b) => b - a); // 3. numbers descending

const p = [{ key: 4, tag: "d" }, { key: 1, tag: "a" }, { key: 3, tag: "c" }];
p.sort((a, b) => a.key - b.key); // 4. objects by a numeric key
const q = [...p].sort((a, b) => (a.tag < b.tag ? 1 : a.tag > b.tag ? -1 : 0));

console.log(v.join(" "), "|",
  p.map(e => e.key + e.tag).join(" "), "|",
  q.map(e => e.key + e.tag).join(" "));
```

Output:

program output

TEXT

```
4 3 1 | 1a 3c 4d | 4d 3c 1a
```

v prints 4 3 1 because the descending call ran last. Each `sort` call overwrites the previous order.

Two JavaScript facts hidden in that block:

- `sort` works **in place** and returns **the same array**. It does not give you a new one.
- That is why `q` starts with `[...p]`: a copy, so sorting `q` does not disturb `p`.

The comparator contract — memorise this

WHAT A COMPARATOR MUST RETURN

`cmp(a, b)` must return a **number**:

return	meaning
negative	a comes before b
0	tie — leave their order alone
positive	b comes before a

Three rules the engine assumes. Break one and you get a wrong order, silently.

1. **Reflexive zero**: `cmp(a, a)` is always 0.
2. **Antisymmetric**: if `cmp(a, b)` is negative then `cmp(b, a)` must be positive.
3. **Transitive**: if `cmp(a, b) < 0` and `cmp(b, c) < 0` then `cmp(a, c) < 0`.

The name for these three together is a **strict weak ordering**.

TRAP

The single most common JavaScript bug in any coding round. `arr.sort()` with no argument turns every element into a string and sorts those strings:

```
[10, 9, 1].sort();           // [1, 10, 9]   "10" sorts before "9"
[10, 9, 1].sort((a, b) => a > b); // [10, 9, 1] a boolean is never negative
[10, 9, 1].sort((a, b) => a - b); // [1, 9, 10] correct
```

Line 2 is the second trap: `a > b` gives `true` or `false`, which become 1 and 0. The comparator can never return a negative number, so the engine is told “these are equal or swapped” and never “this one comes first” — and the array comes back unsorted.

Numbers always need `(a, b) => a - b`. Never `a >`, never `a <=`.

The multi-key template

This shape covers 80% of comparator questions. Copy it exactly.

multi-key comparator: marks desc, then age asc, then name A-Z

JS

```
students.sort((a, b) =>
  b.marks - a.marks ||           // key 1: bigger first
  a.age - b.age ||              // key 2: smaller first
  (a.name < b.name ? -1 : a.name > b.name ? 1 : 0)); // key 3: the tie-breaker
```

HOW TO READ THE TEMPLATE

- One line per key, in priority order, joined with `||`.
- `b.marks - a.marks` is 0 when the two marks are equal. 0 is **falsy**, so `||` moves on to the next key. Any non-zero value stops the chain and decides the order.
- The last line has no `||` after it. It is the final tie-breaker.
- `b.x - a.x` means “bigger first”. `a.x - b.x` means “smaller first”. Nothing else changes.
- Strings have no `-`, so the tie-breaker spells the three cases out: `(a.name < b.name ? -1 : a.name > b.name ? 1 : 0)`.

TRAP

In C++ the `||` in a comparator is a bug. In JavaScript it is the **correct** idiom — but only because each piece returns a **number** and a tie returns exactly 0. Write `b.marks > a.marks || a.age < b.age` with booleans and the chain breaks: `false || false` is false, which is 0, which means “tie”, and your second key never fires.

Ties keep their order

Language	What happens to equal items	Cost
JavaScript sort	original order is kept — stable , guaranteed since ES2019	$O(n \log n)$
C++ sort	order is scrambled, no promise (that is why it has <code>stable_sort</code>)	$O(n \log n)$

JavaScript gives you stability for free. A question that says “keep the earlier entry first on a tie” needs no extra work at all — just do not add a tie-breaking key.

sort is stable: ties keep the input order

JS

```
const v = [{ name: "amit", score: 70 }, { name: "bina", score: 90 },
            { name: "chen", score: 70 }, { name: "dev", score: 90 },
            { name: "esha", score: 70 }];
v.sort((a, b) => a.score - b.score);    // sort is STABLE: ties keep input order
// amit/70 chen/70 esha/70 bina/90 dev/90
```

amit, chen, esha all scored 70 and they come out in exactly that input order. With a sort that is **not** stable you would get **some** order of those three, and it may differ between engines.

SHORTCUT

`Array.prototype.sort` has been required to be stable since ES2019, and Node has been stable since V8 7.0. So in modern JavaScript you get ties-keep-input-order for free. Two cautions: an old judge may run an old engine, and `TypedArray.prototype.sort` is a different function. When the marks depend on it, add the original index as the last key — `|| i - j` — and the order is yours no matter what the engine does.

The friends of sort you should know

cheaper than a full sort when you do not need one

JS

```
const { lowerBound, upperBound } = require('./toolkit.js');    // JS Toolkit appendix

const a = [2, 4, 4, 4, 9];
lowerBound(a, 4);      // 1  first index with value >= 4
upperBound(a, 4);      // 4  first index with value > 4
upperBound(a, 4) - lowerBound(a, 4);    // 3  how many 4s
a[lowerBound(a, 7)] === 7;    // false -- this is "binary search"

const uniq = [...new Set(a)];    // duplicates gone, no sort needed
```

On `a = [2, 4, 4, 4, 9]`:

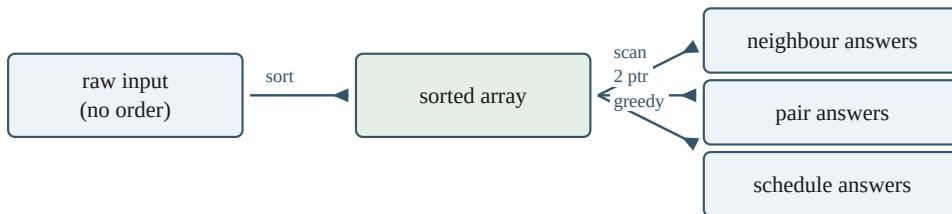
output

TEXT

```
lowerBound(4)=1  upperBound(4)=4  count of 4 = 4-1 = 3  found 7? false
```

When NOT to use sort

Situation	Use instead	Cost
values are small ints, $0..K$, and K is near n	counting sort	$O(n + K)$
only 3 distinct values (0/1/2)	Dutch-flag one pass	$O(n)$, $O(1)$ space
you need only the top k	toolkit MinHeap of size k	$O(n \log k)$
you need only the k -th	quickselect (write it yourself)	$O(n)$ average
data arrives as a stream	heap of size k	$O(n \log k)$
data is bigger than RAM	external merge sort	$O(n \log n)$ I/O



cost paid once: $O(n \log n)$ every arrow after it: $O(n)$

sorting is a pre-step: pay once, scan many times

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Sort $a = \{5, -2, 9, 0, 9, -7\}$ ascending, print it, then sort it descending and print again. Constraints: $n \leq 10^5$, values fit in int. Target: $O(n \log n)$. Edge cases: negative numbers, repeated 9.

SOLUTION

ascending then descending

JS

```
const a = [5, -2, 9, 0, 9, -7];
a.sort((x, y) => x - y);
console.log(a.join(" "));           // -7 -2 0 5 9 9
a.sort((x, y) => y - x);
console.log(a.join(" "));           // 9 9 5 0 -2 -7
```

Negatives need nothing special: $<$ already knows that $-7 < -2$.

Answer: -7 -2 0 5 9 9 then 9 9 5 0 -2 -7

Example 2 [warm-up]

Sort pairs by the **second** value. If two seconds are equal, the smaller first value wins. Input $\{(3,7), (1,7), (9,2), (4,2)\}$.

SOLUTION

two-key pair sort

JS

```
const p = [[3, 7], [1, 7], [9, 2], [4, 2]];
p.sort((x, y) => x[1] - y[1] || x[0] - y[0]);
// [4,2] [9,2] [1,7] [3,7]
```

Trace of the two-key rule:

a	b	seconds differ?	result
(3,7)	(9,2)	yes, $7 \neq 2$	$7 < 2$ is false, so (9,2) comes first
(9,2)	(4,2)	no, both 2	fall to key 2: $9 < 4$ is false, (4,2) first

Answer: (4,2) (9,2) (1,7) (3,7)

Example 3 [TCS NQT · pattern]

Sort words by length. Same length means alphabetical order. Input: pen ox apple kite ax fig.

SOLUTION

length then alphabet

JS

```
const v = ["pen", "ox", "apple", "kite", "ax", "fig"];
v.sort((a, b) => a.length - b.length ||
      (a < b ? -1 : a > b ? 1 : 0)); // `<` on strings is A-Z order
// ax ox fig pen kite apple
```

Answer: ax ox fig pen kite apple

Example 4 [warm-up]

Five students, some share a score. Sort by score ascending but keep students with the same score in their original order.

SOLUTION

stable_sort

JS

```
const v = [{ name: "amit", score: 70 }, { name: "bina", score: 90 },
           { name: "chen", score: 70 }, { name: "dev", score: 90 },
           { name: "esha", score: 70 }];
v.sort((a, b) => a.score - b.score); // nothing extra to do: sort is stable
```

Answer: amit/70 chen/70 esha/70 bina/90 dev/90

Example 5 [Infosys · pattern]

A shop list has name, price, quantity. Show the costliest items first. If two items cost the same, show the smaller quantity first. Input: rice(60,4), dal(95,2), oil(95,9), salt(20,1).

SOLUTION

price desc, quantity asc

JS

```
const v = [{ name: "rice", price: 60, qty: 4 }, { name: "dal", price: 95, qty:
2 },
           { name: "oil", price: 95, qty: 9 }, { name: "salt", price: 20, qty:
1 }];
```

```
v.sort((a, b) => b.price - a.price || a.qty - b.qty);
// dal(95,2) oil(95,9) rice(60,4) salt(20,1)
```

Answer: dal(95,2) oil(95,9) rice(60,4) salt(20,1)

Example 6 [warm-up]

Every value is between 0 and K and K is small. Sort without calling sort. Constraints: $n \leq 10^6$, $0 \leq a[i] \leq K \leq 10^5$. Target: $O(n + K)$. Edge cases: empty array, one element.

SOLUTION

counting sort

JS

```
const countingSort = (a, K) => {
  const cnt = new Array(K + 1).fill(0);
  for (const x of a) cnt[x]++; // how many of each value
  const out = [];
  for (let v = 0; v <= K; v++) // walk the values in order
    for (let c = 0; c < cnt[v]; c++) out.push(v);
  return out;
};
```

On {4, 1, 4, 0, 2, 1, 4} with $K = 4$:

value	0	1	2	3	4
count	1	2	1	0	3

Output 0 1 1 2 4 4 4. Empty input gives an empty result. A single element {3} gives 3.

Time $O(n + K)$ **Space** $O(K)$

Beats $O(n \log n)$ only when K is not much larger than n . $K = 10^9$ would need 4 GB.

Answer: 0 1 1 2 4 4 4

Sorting as the pre-step

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Find the second largest **distinct** value. Constraints: $1 \leq n \leq 10^5$, $-10^9 \leq a[i] \leq 10^9$. Target: $O(n)$. Edge cases: every value equal (no answer), all negatives.

SOLUTION

Approach 1 — Sort and scan ($O(n \log n)$, fine but not the best)

sort descending, skip copies of the maximum

JS

```
const secondLargestSort = (a) => {
  if (a.length < 2) return null;
  const b = [...a].sort((x, y) => y - x); // copy: sort changes the array
  for (let i = 1; i < b.length; i++) if (b[i] !== b[0]) return b[i];
};
```



```
return null; // every value was the same
};
```

Time $O(n \log n)$ **Space** $O(1)$

Correct, but we sorted the whole array to look at two numbers.

Approach 2 — One pass with two variables ($O(n)$, optimal)

carry the best two as you walk

JS

```
const secondLargestScan = (a) => {
  let best = -Infinity, second = -Infinity;
  for (const x of a) {
    if (x > best) { second = best; best = x; }
    else if (x < best && x > second) second = x;
  }
  return second === -Infinity ? null : second;
};
```

Time $O(n)$ **Space** $O(1)$

-Infinity sentinels, so $a[i] = -10^9$ is still a real candidate.

Trace on {4, 9, 9, 2, 7}:

x	best	second	why
4	4	—	first value becomes best
9	9	4	9 beats best, old best drops to second
9	9	4	$x < \text{best}$ is false, so nothing happens — this is the distinct rule
2	9	4	2 is not bigger than second
7	9	7	7 is below best and above second

Checks that were run: [4,9,9,2,7] gives 7; [5,5,5] gives null (no second distinct value); [-3, -1, -7] gives -3. Returning null — not a magic number — is the JavaScript habit: any sentinel you pick could be a real answer, null never can.

TRAP

else if ($x > \text{second}$) without the $x < \text{best}$ guard would let a second copy of the maximum become the “second largest”. On {9, 9} that wrongly answers 9.

The idea that unlocked it: **you do not need order, you need the top two**. Sorting gives you far more than the question asked for.

Answer: 7

Example 8 [Capgemini · pattern]

Given n integers, find the smallest absolute difference between any two of them. Constraints: $2 \leq n \leq 2 \cdot 10^5$, values up to 10^9 in size. Target: $O(n \log n)$. Edge cases: duplicates (answer 0), all negatives.

SOLUTION

Approach 1 — Check every pair ($O(n^2)$, dies at $n = 200000$)

brute force

JS

```
const minDiffBrute = (a) => {
  let best = Infinity;
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      best = Math.min(best, Math.abs(a[i] - a[j]));
  return best;
};
```

Time $O(n^2)$ **Space** $O(1)$

$n = 2 \cdot 10^5$ means $2 \cdot 10^{10}$ pairs. Far past the 10^8 budget.

Approach 3 — Sort, then look at neighbours only ($O(n \log n)$, optimal)

WHY NEIGHBOURS ARE ENOUGH

After sorting, take any pair $a[i] < a[j]$ with $j > i + 1$. Then $a[j] - a[i] = (a[j] - a[j - 1]) + (a[j - 1] - a[i]) \geq a[j] - a[j - 1]$, because every term is ≥ 0 . So a far-apart pair can never beat the neighbour pair inside it. Checking $n - 1$ neighbours is enough.

sort + neighbour scan

JS

```
const minDiffSort = (a) => {
  if (a.length < 2) return -1;
  const b = [...a].sort((x, y) => x - y);
  let best = Infinity;
  for (let i = 1; i < b.length; i++) best = Math.min(best, b[i] - b[i - 1]);
  return best;
};
```

Time $O(n \log n)$ **Space** $O(1)$

After sorting, $a[i] - a[i - 1]$ is never negative, so $\text{abs}()$ is not needed.

On $\{17, 3, 40, 8, 20, 6\}$: sorted is $3\ 6\ 8\ 17\ 20\ 40$, gaps are $3\ 2\ 9\ 3\ 20$, answer **2**. Both versions printed 2. $\{5, 5, 9\}$ gives 0. $\{-10, -3, 2\}$ gives 5.

The idea that unlocked it: **sorting turns “all pairs” into “adjacent pairs”**. That single sentence removes an entire loop.

Answer: 2

Example 9 [Wipro · pattern]

Rank students. Higher marks first. Same marks: younger first. Same marks and age: alphabetical. Input: riya(88,19), arun(92,21), kabir(88,18), asha(92,21). Constraints: $n \leq 10^5$. Edge cases: a full three-way tie, students with the same name.

SOLUTION

three-key comparator

JS

```
v.sort((a, b) =>
  b.marks - a.marks ||
  a.age - b.age ||
  (a.name < b.name ? -1 : a.name > b.name ? 1 : 0));
```

Result: arun/92/21 asha/92/21 kabir/88/18 riya/88/19.

Why arun beats asha: marks equal (92), age equal (21), so key 3 decides and "arun" < "asha". Why kabir beats riya: marks equal (88), and kabir is 18 against riya's 19.

Time $O(n \log n)$ **Space** $O(1)$

String compares are $O(\text{length})$; with long names think $O(n \log n * L)$.

Answer: arun asha kabir riya

Example 10 [Accenture · pattern]

Count the pairs (i, j) with $i < j$ and $a[i] + a[j] = t$. Constraints: $n \leq 2 \cdot 10^5$, $|a[i]| \leq 10^9$, $|t| \leq 2 \cdot 10^9$. Target: $O(n \log n)$. Edge cases: every element equal, no pair at all.

SOLUTION

Approach 1 — All pairs ($O(n^2)$)

brute force, used later as the reference answer

JS

```
const countPairsBrute = (a, t) => {
  let c = 0;
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] + a[j] === t) c++;
  return c;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 3 — Sort, then two pointers ($O(n \log n)$, optimal)

two pointers, counting equal blocks together

JS

```
const countPairsSorted = (a, t) => {
  const b = [...a].sort((x, y) => x - y);
  let c = 0, i = 0, j = b.length - 1;
  while (i < j) {
    const s = b[i] + b[j];
    if (s === t) {
      if (b[i] === b[j]) { // one block of equal values
        const m = j - i + 1;
        c += m * (m - 1) / 2; // choose 2 out of m
        break;
      }
      let ci = 0, cj = 0;
      const vi = b[i], vj = b[j];
      while (i < j && b[i] === vi) { i++; ci++; }
      while (j >= i && b[j] === vj) { j--; cj++; }
      c += ci * cj; // every left with every right
    } else if (s < t) i++; // need a bigger sum
    else j--; // need a smaller sum
  }
  return c;
};
```

Time $O(n \log n)$ **Space** $O(1)$
The sort dominates; the two-pointer walk is $O(n)$.

Tests that were run:

input	t	answer	why
{2,4,3,5,1,3,5}	6	4	(1,5),(1,5),(3,3),(2,4)
{3,3,3,3}	6	6	all four are equal: $4 \cdot 3/2 = 6$
{1,2}	99	0	no pair

TRAP

In C++ `int s = a[i] + a[j]` overflows when both are near 10^9 . JavaScript adds as a double, so 2×10^9 is exact and nothing special is needed here. The JavaScript version of this trap lives further out: the sum stops being exact only past $2^{53} - 1 \approx 9.0 \times 10^{15}$, and then you need `BigInt`.

Answer: 4

Example 11 [Cognizant · pattern]

Merge two already-sorted arrays into one sorted array. Constraints: $n, m \leq 10^5$. Target: $O(n + m)$. Edge cases: one array empty, duplicate values across both arrays.

SOLUTION

Approach 1 — Glue them together and re-sort ($O((n+m) \log(n+m))$, throws away what you were given)

the lazy version

JS

```
const mergeSortedResort = (a, b) => a.concat(b).sort((x, y) => x - y);
```

Time $O((n + m) \log(n + m))$ **Space** $O(n + m)$

Correct, and it gives the same answer as the merge below — but it pays for order it already had.

Approach 3 — Merge in one walk ($O(n+m)$, optimal)

the merge step, the heart of merge sort

JS

```
const mergeSorted = (a, b) => {
  const out = [];
  let i = 0, j = 0;
  while (i < a.length && j < b.length)
    out.push(a[i] <= b[j] ? a[i++] : b[j++]); // <= keeps a's item first
  while (i < a.length) out.push(a[i++]);   // drain the leftovers
  while (j < b.length) out.push(b[j++]);
  return out;
};
```

{1,4,9} and {2,4,4,10} gives 1 2 4 4 4 9 10. Empty first array plus {3,7} gives 3 7.

Time $O(n + m)$ **Space** $O(n + m)$

Sorting the joined array instead would cost $O((n+m) \log(n+m))$ and waste the order you were given.

SHORTCUT

Never re-sort data that is already sorted. Merging is the cheaper move and interviewers watch for it.

Answer: 1 2 4 4 4 9 10

Example 12 [TCS Digital · pattern]

An array holds only 0, 1 and 2. Sort it. Constraints: $n \leq 10^6$. Target: $O(n)$ time and $O(1)$ extra space, one pass. Edge cases: single element, all the same value.

SOLUTION

Approach 1 — Count and rewrite ($O(n)$ but two passes)

count the three values, then overwrite

JS

```
const sort012Count = (a) => {
  const c = [0, 0, 0];
  for (const x of a) c[x]++;
  let k = 0;
  for (let v = 0; v < 3; v++)
    for (let t = 0; t < c[v]; t++) a[k++] = v;
};
```

Time $O(n)$ **Space** $O(1)$

Perfectly good. Fails only the extra rule 'read each element once'.

Approach 3 — Dutch national flag, one pass ($O(n)$, optimal)

THE INVARIANT — THIS IS THE WHOLE TRICK

Three walls split the array at all times:

- $a[0 \dots \text{low}-1]$ is all 0
- $a[\text{low} \dots \text{mid}-1]$ is all 1
- $a[\text{mid} \dots \text{high}]$ is **not yet looked at**
- $a[\text{high}+1 \dots n-1]$ is all 2

Stop when $\text{mid} > \text{high}$, because then nothing is unknown.

one pass, three pointers

JS

```
const sort012OnePass = (a) => {
  let low = 0, mid = 0, high = a.length - 1;
  while (mid <= high) {
    if (a[mid] === 0) { [a[low], a[mid]] = [a[mid], a[low]]; low++; mid++; }
    else if (a[mid] === 1) mid++;
    else { [a[mid], a[high]] = [a[high], a[mid]]; high--; }
  }
  // do NOT move mid in the last branch
};
```

Time $O(n)$ **Space** $O(1)$

Each step either grows low/mid or shrinks high, so at most n steps.

TRAP

In the `== 2` branch, `mid` must **not** advance. The value swapped in from `high` has never been seen, so it must be examined next round.

Both versions turned `{2, 0, 1, 2, 1, 0, 0}` into `0001122`. A one-element array `{2}` stays `2`.

Answer: 0001122

Example 13 [Amazon · pattern]

Join all the numbers into one string so the resulting number is as large as possible. Input `{3, 30, 34, 5, 9}`. Constraints: $n \leq 10^4$, each value $\leq 10^9$. Target: $O(n \log n \cdot L)$. Edge cases: all zeros (answer is `"0"`, not `"00"`), a single number.

SOLUTION

THE COMPARATOR THAT MAKES THIS WORK

For two strings `x` and `y`, `x` should come first when the joined string `x+y` is bigger than `y+x`.

`"3"` vs `"30"`: `"330"` against `"303"`. `"330"` wins, so `3` goes first.

This is not the same as `x > y`. Sorting the plain strings would put `"34"` before `"3"` and lose.

concatenation comparator

JS

```
const largestNumber = (a) => {
  const s = a.map(String);
  s.sort((x, y) => (x + y > y + x ? -1 : x + y < y + x ? 1 : 0));
  if (s[0] === "0") return "0"; // all zeros
  return s.join("") || "0";
};
```

Sorted pieces: `9, 5, 34, 3, 30` giving `9534330`. `{0,0}` gives `"0"`. `{12}` gives `"12"`.

Time $O(n \log n \cdot L)$ **Space** $O(nL)$

`L` is the digit count; each compare builds two strings of length $\sim 2L$.

Python version

same comparator, via `cmp_to_key`

PYTHON

```
from functools import cmp_to_key

def largest_number(a):
    s = [str(x) for x in a]
    s.sort(key=cmp_to_key(lambda x, y: (x + y < y + x) - (x + y > y + x)))
    out = "".join(s)
    return "0" if out[0] == "0" else out

print(largest_number([3, 30, 34, 5, 9])) # 9534330
```

Python's `cmp_to_key` wants `-1 / 0 / +1`, not a boolean. The expression `(less) - (greater)` produces exactly that.

Answer: 9534330

Example 14 [Infosys · pattern]

Find the k -th smallest element. Constraints: $n \leq 10^6$, $1 \leq k \leq n$. Target: better than $O(n \log n)$. Edge cases: $k = 1$, $k = n$.

SOLUTION

Approach 1 — Sort, read index $k-1$ ($O(n \log n)$)

simple and always correct

JS

```
const kthSmallestSort = (a, k) => [...a].sort((x, y) => x - y)[k - 1];
```

Time $O(n \log n)$ **Space** $O(1)$

Approach 3 — nth_element ($O(n)$ average, optimal)

quickselect, built into the library

JS

```
const quickSelect = (a, k) => { // k is 1-based; a is rearranged
  let lo = 0, hi = a.length - 1;
  while (lo < hi) {
    const pivot = a[(lo + hi) >> 1];
    let i = lo, j = hi;
    while (i <= j) { // Hoare partition
      while (a[i] < pivot) i++;
      while (a[j] > pivot) j--;
      if (i <= j) { [a[i], a[j]] = [a[j], a[i]]; i++; j--; }
    }
    if (k - 1 <= j) hi = j; // the k-th is in the left part
    else if (k - 1 >= i) lo = i; // ...or in the right part
    else break; // it is already in place
  }
  return a[k - 1];
};
```

Time $O(n)$ average **Space** $O(1)$

After the call, everything left of $k-1$ is $\leq$ it and everything right is $\geq$ it. The two sides are NOT sorted.

On $\{7, 10, 4, 3, 20, 15\}$ with $k = 3$ both give 7. $k = 1$ gives 3, $k = 6$ gives 20.

The idea that unlocked it: **you asked for one position, not for a full order**. Quickselect throws away half the array at each step instead of sorting it.

Answer: 7

Example 15 [Adobe · pattern]

Sort the array so more frequent values come first. Equal frequency: smaller value first. Input $\{4, 6, 2, 6, 4, 4, 6, 8\}$. Constraints: $n \leq 10^5$. Edge cases: every value unique, every value the same.

SOLUTION

Approach 1 — Count each element by rescanning ($O(n^2)$)

brute force: one full scan per element

JS

```
const byFrequencyBrute = (a) => {
  const cnt = a.map(x => a.filter(y => y === x).length); // one full scan each
  return a.map((v, i) => [cnt[i], v])
}
```

```

    .sort((x, y) => y[0] - x[0] || x[1] - y[1])
    .map(([, v]) => v);
};

```

Time $O(n^2)$ **Space** $O(n)$

Gave the same output as the fast version on the test input — it is correct, only slow.

Approach 3 — One counting pass, then sort ($O(n \log n)$, optimal)

count first, then sort using the counts

JS

```

const byFrequency = (a) => {
  const f = new Map();
  for (const x of a) f.set(x, (f.get(x) ?? 0) + 1); // pass 1: the counts
  return [...a].sort((x, y) => f.get(y) - f.get(x) || x - y);
};

```

Counts: 4 -> 3, 6 -> 3, 2 -> 1, 8 -> 1. Output 4 4 4 6 6 6 2 8.

Time $O(n \log n)$ **Space** $O(n)$

The map lookup inside the comparator is $O(1)$ average, so the sort stays $O(n \log n)$.

TRAP

`f[x]` on a const map does not compile, and on a non-const map it **inserts** a zero when the key is missing. Here every key already exists, so it is safe — but capture by reference (`[&]`), never by value, or you copy the whole map for every compare.

Answer: 4 4 4 6 6 6 2 8

Example 16 [Google · pattern]

Sort `a` so that values appear in the order given by a second array `order`. Values missing from `order` go last, sorted ascending. `a = {2,3,1,3,2,4,6,7,9,2,19}`, `order = {2,1,4,3,9,6}`. Constraints: $n \leq 10^5$, `order` has distinct values. Edge cases: `order` empty; no value of `a` appears in `order`.

SOLUTION

rank lookup inside the comparator

JS

```

const relativeSort = (a, order) => {
  const pos = new Map(order.map((v, i) => [v, i])); // value -> rank
  const last = order.length; // rank for "missing"
  return [...a].sort((x, y) =>
    (pos.get(x) ?? last) - (pos.get(y) ?? last) || x - y);
};

```

Output: 2 2 2 1 4 3 3 9 6 7 19. The 7 and 19 are not in order, so both get rank 6 and fall to the `x < y` rule.

Time $O(n \log n + m)$ **Space** $O(m)$

`m = order.size()`. Giving every missing value the same sentinel rank is what keeps the comparator transitive.

Answer: 2 2 2 1 4 3 3 9 6 7 19

Business rules become comparators

Example 17 [Grab · pattern]

A dispatch service must pick the best k free drivers. Rules, in order: highest rating, then shortest distance, then smallest id. Busy drivers are not considered. Constraints: $n \leq 2 \cdot 10^5$, $k \leq n$. Edge cases: fewer than k free drivers; every driver busy.

SOLUTION

filter, then a three-key sort

JS

```
const pickDrivers = (drivers, k) =>
  drivers
    .filter(d => d.free) // filter first
    .sort((a, b) => b.rating - a.rating ||
           a.distance - b.distance ||
           a.id - b.id)
    .slice(0, k)
    .map(d => d.id);
```

Input: 11(4.8, 900, free), 12(4.9, 2200, free), 13(4.9, 700, busy), 14(4.9, 700, free), 15(4.8, 900, free). With $k = 3$ the answer is 14 12 11.

- 13 is dropped: busy.
- 14 first: rating 4.9 and the shortest distance among the 4.9 drivers.
- 12 next: rating 4.9 but 2200 m away.
- 11 before 15: both 4.8 and 900 m, so the smaller id wins.

Time $O(n \log n)$ **Space** $O(n)$

Filtering before sorting matters when most drivers are busy.

TRAP

double equality (`a.rating !== b.rating`) is risky if ratings come from division. Store ratings as integers of hundredths (490 for 4.90) in real systems.

Answer: 14 12 11

Example 18 [LINE MAN · pattern]

A rider has T minutes left. Order i takes $t[i]$ minutes. Orders can be done in any order and any subset. Maximise the **count** of completed orders. $t = \{30, 12, 45, 12, 60, 9\}$, $T = 70$. Constraints: $n \leq 10^5$, $t[i] \leq 10^9$, $T \leq 10^{14}$. Edge cases: $T = 0$; T big enough for everything.

SOLUTION

Approach 1 — Try every subset ($O(2^n \cdot n)$, only usable as a checker)

bitmask brute force

JS

```
const maxOrdersBrute = (t, T) => {
  const n = t.length;
  let best = 0;
  for (let mask = 0; mask < (1 << n); mask++) {
    let s = 0, c = 0;
```

```

    for (let i = 0; i < n; i++) if (mask >> i & 1) { s += t[i]; c++; }
    if (s <= T) best = Math.max(best, c);
  }
  return best;
};

```

Time $O(2^n n)$ **Space** $O(1)$

$n = 20$ is already a million subsets. Use it only to check the greedy on small inputs.

Approach 3 — Sort ascending and take greedily $(O(n \log n), \text{optimal})$

EXCHANGE ARGUMENT — WHY GREEDY IS SAFE

Suppose the best answer takes c orders but not the c shortest ones. Then it contains some order x while a shorter order y is left out. Swap x for y : the count stays c and the total time does not grow. Repeat until the answer **is** the c shortest orders. So taking shortest-first never loses.

sort + running sum

JS

```

const maxOrdersGreedy = (t, T) => {
  const b = [...t].sort((x, y) => x - y);
  let s = 0, c = 0;
  for (const x of b) {
    if (s + x > T) break;
    s += x; c++;
  }
  return c;
};

```

Time $O(n \log n)$ **Space** $O(1)$

Sorted t is 9 12 12 30 45 60. Running sums: 9, 21, 33, 63, then $63 + 45 = 108 > 70$ so stop. Answer **4**. The brute force on the same input also said 4. $T = 0$ gives 0; $T = 1000$ gives 6.

TRAP

This maximises the **count**, not the earned money. If each order paid a different amount the greedy is wrong and you need a knapsack (Chapter 17).

Answer: 4

Example 19 [Agoda · pattern]

A hotel desk has bookings with a check-in and a check-out time. A desk can serve one guest at a time. What is the smallest number of desks needed? Starts $\{1, 2, 4, 9\}$, ends $\{5, 3, 7, 11\}$. Constraints: $n \leq 10^5$. A guest leaving at time t frees the desk for a guest arriving at t . Edge cases: one booking; two bookings that touch exactly.

SOLUTION

BREAK THE PAIRS APART

You do not care **which** guest is in a desk, only **how many** guests are inside at once. So sort the starts and the ends as two independent lists, then sweep a clock forward. Every start adds one, every end removes one. The peak is the answer.

```
const minCounters = (start, end) => {
  const s = [...start].sort((a, b) => a - b);
  const e = [...end].sort((a, b) => a - b);
  let i = 0, j = 0, cur = 0, best = 0;
  while (i < s.length) {
    if (s[i] < e[j]) { cur++; best = Math.max(best, cur); i++; } // arrival
    else { cur--; j++; } // departure
  }
  return best;
};
```

Sorted starts 1 2 4 9, sorted ends 3 5 7 11. Sweep:

start[i]	end[j]	action	cur	best
1	3	1 < 3: arrive	1	1
2	3	2 < 3: arrive	2	2
4	3	4 ≥ 3: depart	1	2
4	5	4 < 5: arrive	2	2
9	5	9 ≥ 5: depart	1	2
9	7	9 ≥ 7: depart	0	2
9	11	9 < 11: arrive	1	2

Answer 2. A single booking gives 1. Bookings [1, 5) and [5, 9) give 1, because $5 < 5$ is false, so the departure is processed first.

Time $O(n \log n)$ **Space** $O(n)$

Only starts drive the loop; ends can never run out first.

TRAP

Use $<$ and not $\leq$ in $\text{start}[i] < \text{end}[j]$ when leaving at t frees the desk at t . If the rule is “cleaning takes a minute”, flip it to $\leq$ and the answer changes. Ask the interviewer which rule applies.

Answer: 2

Example 20 [Shopee · pattern]

A catalogue has 10^7 products. Every price is a whole number of cents from 0 to 10^5 . Return all prices in ascending order as fast as possible. Edge cases: empty catalogue; every product the same price.

SOLUTION

Approach 1 — Plain $\text{a.sort}((x, y) \Rightarrow x - y)$ ($O(n \log n) \sim 2.3 * 10^8$ compares, often too slow)

Approach 3 — Counting sort on the price range ($O(n + K)$, optimal here)

```
const sortPrices = (p, maxP) => {
  const cnt = new Int32Array(maxP + 1);
  for (const x of p) cnt[x]++;
  const out = new Array(p.length);
```

```

let k = 0;
for (let v = 0; v <= maxP; v++)
  for (let c = 0; c < cnt[v]; c++) out[k++] = v; // never out.push(...big)
return out;
};

```

{7,2,7,0,5,2,7} with maxP = 7 gives 0 2 2 5 7 7 7.

Time $O(n + K)$ **Space** $O(K)$

$n = 10^7$, $K = 10^5$. That is $\sim 10^7$ work, about 20x fewer operations than a comparison sort here.

THE DECISION RULE

Counting sort wins when $K = O(n)$. It loses badly when K is huge: prices up to 10^9 would need a 4 GB counter array.

Rough rule for interviews: **if the value range fits in memory and is not much bigger than n , count. Otherwise compare.**

Answer: 0 2 2 5 7 7 7

Example 21 [Sea/Shopee · pattern]

A log line is `id rest`. If `rest` starts with a digit it is a **metric** log; otherwise it is a **text** log. Sort so all text logs come first (by `rest`, ties by `id`), and every metric log stays in its original relative order, after the text logs. Edge cases: no metric logs at all; two text logs with identical `rest`.

SOLUTION

stable_sort plus a comparator that only orders text logs

JS

```

const sortLogs = (logs) => {
  const cut = (s) => { const i = s.indexOf(" "); return [s.slice(0, i), s.slice(i + 1)]; };
  const isMetric = (s) => /^[0-9]/.test(cut(s)[1]); // rest starts with a digit
  return [...logs].sort((a, b) => {
    const ma = isMetric(a), mb = isMetric(b);
    if (ma || mb) return (ma ? 1 : 0) - (mb ? 1 : 0); // text first; metric ties
    const [ida, ra] = cut(a), [idb, rb] = cut(b);
    return (ra < rb ? -1 : ra > rb ? 1 : 0) ||
      (ida < idb ? -1 : ida > idb ? 1 : 0);
  });
};

```

Input and output:

input order	output order
a1 9 4 2	g2 act car
g2 act car	zo4 act car
zo4 act car	a8 act zoo
ab1 off key dog	ab1 off key dog
a8 act zoo	a1 9 4 2
b3 5 1	b3 5 1

g2 act car beats zo4 act car because the rest fields are equal and "g2" < "zo4". The two metric logs keep their input order because the comparator returns false both ways for them and `stable_sort` then preserves the original positions.

READING THE TRICKY LINE

```
if (ma || mb) return !ma && mb;
```

- both metric: !ma is false, so returns false — they are “equal”, stability decides
- a text, b metric: !ma && mb is true — text first
- a metric, b text: !ma is false — correct, and asymmetry holds

Time $O(n \log n \cdot L)$ **Space** $O(n)$

`stable_sort` allocates a buffer; L is the line length because of `substr` and string compare.

Answer: text logs sorted by content, then the metric logs untouched

Example 22 [DBS · pattern]

n warehouses hold $a[i]$ units. Moving one unit in or out of a warehouse costs 1. Make every warehouse hold the same amount at the lowest total cost. $a = \{4, 1, 9, 7, 2\}$. Constraints: $n \leq 10^5$, $a[i] \leq 10^9$. Edge cases: even n ; all values already equal.

SOLUTION

Approach 1 — Try every target between min and max ($O(n \cdot \text{range})$, unusable)

brute force over targets

JS

```
const equalCostBrute = (a) => {
  let best = Infinity;
  const lo = Math.min(...a), hi = Math.max(...a);
  for (let t = lo; t <= hi; t++) {
    let c = 0;
    for (const x of a) c += Math.abs(x - t);
    best = Math.min(best, c);
  }
  return best;
};
```

Time $O(n \cdot (\max - \min))$ **Space** $O(1)$

With values up to 10^9 this never finishes.

Approach 3 — Sort and move everything to the median ($O(n \log n)$, optimal)

WHY THE MEDIAN WINS

Let $f(t) = \sum_i |a_i - t|$. Move the target from t to $t + 1$: every warehouse with $a_i \leq t$ costs 1 more, every one with $a_i > t$ costs 1 less. So $f(t + 1) - f(t) = (\text{count} \leq t) - (\text{count} > t)$.

That difference is negative while fewer than half the values are below t , and positive after. The turning point is exactly the median. For even n every point between the two middle values gives the same cost.

sort, pick the middle, sum the distances

JS

```
const equalCostMedian = (a) => {
  const b = [...a].sort((x, y) => x - y);
  const med = b[b.length >> 1];
  let c = 0;
  for (const x of b) c += Math.abs(x - med);
  return c;
};
```

Time $O(n \log n)$ **Space** $O(1)$

Sorted: 1 2 4 7 9. Median 4. Cost $3 + 2 + 0 + 3 + 5 = 13$. Brute force also printed 13. $\{1, 2, 3, 4\}$ gives 4 both ways. $\{6, 6, 6\}$ gives 0.

TRAP

Use the **median**, not the **mean**. The mean minimises squared distance; the median minimises absolute distance. On $\{1, 1, 1, 100\}$ the mean is 25.75 (cost 148) and the median is 1 (cost 99).

Answer: 13

Example 23 [GIC · pattern]

Return the top k sellers by revenue; ties broken alphabetically. n is a million and k is 10. Edge cases: $k > n$; two sellers with equal revenue.

SOLUTION

Approach 1 — Sort everything, keep k ($O(n \log n)$, 10^6 items sorted to print 10)

the obvious version

JS

```
const topKFullSort = (sellers, k) =>
  [...sellers].sort((a, b) => b.revenue - a.revenue ||
    (a.name < b.name ? -1 : a.name > b.name ? 1 : 0))
    .slice(0, k);
```

Time $O(n \log n)$ **Space** $O(1)$

About $2 \cdot 10^7$ compares for $n = 10^6$.

Approach 3 — partial_sort ($O(n \log k)$, optimal)

partial_sort: order only the part you print

JS

```
const topK = (sellers, k) => {
  const better = (a, b) => b.revenue - a.revenue ||
    (a.name < b.name ? -1 : a.name > b.name ? 1 : 0);
  const heap = new MinHeap((a, b) => better(b, a)); // the WORST of the k on
  top
  for (const s of sellers) {
    heap.push(s);
    if (heap.size > k) heap.pop(); // drop the worst
  }
  const out = [];
  while (heap.size) out.push(heap.pop()); // worst first...
  return out.reverse(); // ...so reverse
};
```

On ria/900, om/1500, zed/1500, kai/200 with $k = 2$: om/1500 zed/1500 (equal revenue, and "om" < "zed").

Time $O(n \log k)$ **Space** $O(1)$

$n = 10^6$, $k = 10$: about $10^6 * 3.3$ compares instead of $10^6 * 20$ for a full sort.

SHORTCUT

Three tools, three questions:

- “give me the k -th” $\rightarrow$ `nth_element`, $O(n)$
- “give me the top k , in order” $\rightarrow$ `partial_sort`, $O(n \log k)$
- “give me everything in order” $\rightarrow$ `sort`, $O(n \log n)$

Answer: om/1500 zed/1500

Sorting hidden inside another problem

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Google · pattern]

Find the largest gap between two consecutive values of the array **after** it would be sorted. $a = \{3, 6, 9, 1\}$. Constraints: $n \leq 10^6$, $0 \leq a[i] \leq 10^9$. Target: $O(n)$ time. Edge cases: $n = 1$ (answer 0); every value equal (answer 0).

Follow-up the interviewer asks: “your sort is $O(n \log n)$ — can you do it in linear time?”

SOLUTION

Approach 1 — Sort and scan ($O(n \log n)$, the honest first answer)

baseline

JS

```
const maxGapSort = (a) => {
  if (a.length < 2) return 0;
  const b = [...a].sort((x, y) => x - y);
  let best = 0;
  for (let i = 1; i < b.length; i++) best = Math.max(best, b[i] - b[i - 1]);
  return best;
};
```

Time $O(n \log n)$ **Space** $O(1)$

Approach 3 — Pigeonhole buckets ($O(n)$, optimal)

THE COUNTING INSIGHT

The n values span the range $hi - lo$. Split that range into $n - 1$ buckets, each of width $w = \left\lceil \frac{(hi - lo)}{n - 1} \right\rceil$.

There are n values and $n - 1$ buckets, so at least one bucket is empty (pigeonhole). That empty bucket forces the winning gap to **cross** a bucket boundary. Therefore the answer is never a within-bucket distance, and each bucket only needs its **min** and **max**.

bucket min/max, then scan the buckets

JS

```

const maxGapBucket = (a) => {
  const n = a.length;
  if (n < 2) return 0;
  const lo = Math.min(...a), hi = Math.max(...a);
  if (lo === hi) return 0;

  const width = Math.ceil((hi - lo) / (n - 1));
  const nb = Math.floor((hi - lo) / width) + 1;
  const bmin = new Array(nb).fill(Infinity), bmax = new Array(nb).fill(-Infinity);
  for (const x of a) {
    const b = Math.floor((x - lo) / width);
    bmin[b] = Math.min(bmin[b], x);
    bmax[b] = Math.max(bmax[b], x);
  }
  let best = 0, prev = null;
  for (let b = 0; b < nb; b++) {
    if (bmin[b] === Infinity) continue; // empty bucket, skip
    if (prev !== null) best = Math.max(best, bmin[b] - prev);
    prev = bmax[b];
  }
  return best;
};

```

Time $O(n)$ **Space** $O(n)$

Two passes over the data plus one pass over $n-1$ buckets.

On $\{3, 6, 9, 1\}$: $lo = 1, hi = 9, w = \lceil \frac{8}{3} \rceil = 3$, buckets $[1, 4)$ $[4, 7)$ $[7, 10)$.

bucket	values	min	max
0	1, 3	1	3
1	6	6	6
2	9	9	9

Cross-bucket gaps: $6 - 3 = 3, 9 - 6 = 3$. Answer **3**, matching the sort version.

Verification run: on 400 random arrays of length 1 to 9 the bucket version matched the sort version every single time.

TRAP

In C++ $(hi - lo)$ with `int` overflows when $lo = -2 \cdot 10^9$. JavaScript subtracts as a double and gets 4×10^9 exactly. What **does** bite in JavaScript is the division: $(hi - lo) / (n - 1)$ is a **float**, not an integer. Bucket indices must be floored — `Math.floor((x - lo) / gap)` — or a fractional index silently becomes a string key.

The follow-up after the follow-up: “now the data arrives as a stream and you cannot store it.” Answer: you cannot. The maximum gap depends on every value at once; with $O(1)$ memory it is impossible. You can only stream if you are allowed an approximate answer (keep bucket min/max only, which is exactly what the above does in one pass if lo and hi are known in advance).

Answer: 3

Example 25 [D. E. Shaw · pattern]

Count **inversions**: pairs (i, j) with $i < j$ and $a[i] > a[j]$. This measures how far the array is from sorted. $a = \{5, 1, 4, 2, 8, 3\}$. Constraints: $n \leq 2 \cdot 10^5$. Target: $O(n \log n)$. Edge cases: already sorted (0); reversed $(n(n-1)/2)$; all duplicates (0).

Follow-up: “now the array is a stream of 10^9 items across 50 machines.”

SOLUTION

Approach 1 — Check all pairs $(O(n^2))$

the reference implementation

JS

```
const invBrute = (a) => {
  let c = 0;
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] > a[j]) c++;
  return c;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 3 — Count during a merge sort $(O(n \log n)$, optimal)

THE ONE LINE THAT DOES THE COUNTING

During a merge, the left half and the right half are each already sorted. If we are about to take $a[j]$ from the **right** half while positions $i \dots m-1$ of the left half are still waiting, then **every one of those $m - i$ left values is greater than $a[j]$** . That is $m - i$ inversions, found in a single addition.

merge sort that returns the inversion count

JS

```
const mergeCount = (a, buf, l, r) => {
  if (r - l <= 1) return 0;
  const m = (l + r) >> 1;
  let c = mergeCount(a, buf, l, m) + mergeCount(a, buf, m, r);
  let i = l, j = m, k = l;
  while (i < m && j < r) {
    if (a[i] <= a[j]) buf[k++] = a[i++]; // <= : equal is no inversion
    else { c += m - i; buf[k++] = a[j++]; } // the whole counting trick
  }
  while (i < m) buf[k++] = a[i++];
  while (j < r) buf[k++] = a[j++];
  for (let t = l; t < r; t++) a[t] = buf[t];
  return c;
};

const invMergeSort = (a) => {
  const b = [...a];
  return mergeCount(b, new Array(b.length), 0, b.length);
};
```

Time $O(n \log n)$ **Space** $O(n)$

One shared buffer, allocated once. Allocating inside the recursion would add a huge constant.

Results: {5,1,4,2,8,3} gives 7 from both versions. Sorted {1,2,3} gives 0. Reversed {4,3,2,1} gives 6. All-duplicate {2,2,2} gives 0 — that is what the $\leq$ guarantees. A randomised cross-check on 400 arrays agreed with the brute force every time.

Approach 4 — Fenwick tree from the right (also $O(n \log n)$, the streaming-friendly one)

compress values, then count smaller items already seen

JS

```
class BIT {
  constructor(n) { this.t = new Int32Array(n + 1); }
  add(i) { for (; i < this.t.length; i += i & -i) this.t[i]++; }
  sum(i) { let s = 0; for (; i > 0; i -= i & -i) s += this.t[i]; return s; }
}

const invBIT = (a) => {
  const sorted = [...new Set(a)].sort((x, y) => x - y); // compress values
  const rank = new Map(sorted.map((v, i) => [v, i + 1])); // 1-based ranks
  const bit = new BIT(sorted.length);
  let c = 0;
  for (let i = a.length - 1; i >= 0; i--) { // walk right to left
    const r = rank.get(a[i]);
    c += bit.sum(r - 1); // smaller values already seen on the right
    bit.add(r);
  }
  return c;
};
```

Time $O(n \log n)$ **Space** $O(n)$

Walking right to left means everything already in the tree sits to the right of the current index.

Answering the follow-up. Inversions split cleanly:

$$\text{inv}(A + B) = \text{inv}(A) + \text{inv}(B) + \text{pairs with one item in } A \text{ and one in } B$$

The cross term is exactly what the merge step counts. So: each machine sorts its own chunk and reports its local inversion count, then the chunks are merged pairwise up a tree, each merge adding its cross count. This is the standard external / distributed merge sort, and it is why the merge-sort solution is the one to present in an interview even though the Fenwick version is shorter.

Answer: 7

Example 26 [Microsoft · pattern]

A candidate writes `(a, b) => a.score >= b.score` as a comparator. The program does not crash — it just returns the wrong order, on every size of input. Explain, then write a test that catches this class of bug before submission.

Follow-up: “how would you prove a comparator is correct?”

SOLUTION

What goes wrong. A JavaScript comparator must return a **number**: negative for “a first”, positive for “b first”, zero for “tie”. `a.score >= b.score` returns a **boolean**. The engine coerces it, so `true` becomes 1 and `false` becomes 0 — and there is no way to produce a negative value at all. The sort is told “b first” or “tie”, never “a first”, so nothing ever moves left and the array comes back in something close to its input order.

C++ makes this mistake fatal: `>=` there breaks the strict-weak-order contract and its sort’s partitioning runs off the end of the buffer. JavaScript makes it **silent**, which is worse for an exam. Nothing warns you.

the boolean comparator, measured

JS

```
const mk = (n) => Array.from({length: n}, (_, i) => ({ name: "p" + i, score: (i * 37) % 10 }));

const boolCmp = (a, b) => a.score >= b.score;           // WRONG: returns true/false
const numCmp = (a, b) => b.score - a.score;             // RIGHT: returns a number

const isSorted = (v, cmp) => v.every((x, i) => i === 0 || cmp(v[i-1], x) <= 0);

for (const n of [5, 11, 30, 1000])
  console.log("n =", n,
    "| boolean sorted?", isSorted(mk(n).sort(boolCmp), numCmp),
    "| number sorted?", isSorted(mk(n).sort(numCmp), numCmp));
```

measured output

TEXT

```
n = 5 | boolean sorted? false | number sorted? true
n = 11 | boolean sorted? false | number sorted? true
n = 30 | boolean sorted? false | number sorted? true
n = 1000 | boolean sorted? false | number sorted? true
```

On the 11-element case the boolean comparator returns 0 7 4 1 8 5 2 9 6 3 0 — the input order — while the number comparator returns 9 8 7 6 5 4 3 2 1 0 0.

The fix is always the multi-key template: subtract to get a number, and chain the keys with `||` only because `0` is falsy — `b.score - a.score || a.name.localeCompare(b.name)` reads the second key exactly when the first ties.

a checker you can paste into any solution while testing

JS

```
const sign = (x) => (x < 0 ? -1 : x > 0 ? 1 : 0);

const checkComparator = (v, cmp) => {
  for (const a of v)
    if (sign(cmp(a, a)) !== 0) return "not reflexive: cmp(a, a) must be 0";
  for (const a of v) for (const b of v)
    if (sign(cmp(a, b)) !== -sign(cmp(b, a))) return "not antisymmetric";
  for (const a of v) for (const b of v) for (const c of v)
    if (sign(cmp(a, b)) < 0 && sign(cmp(b, c)) < 0 && sign(cmp(a, c)) >= 0)
      return "not transitive";
  return "ok";
};
```

Run on `v = [1,2,3,4]` — measured output:

comparator	checkComparator says
<code>(a, b) => a - b</code>	ok
<code>(a, b) => a <= b</code>	not reflexive: cmp(a, a) must be 0
<code>(a, b) => a >= b</code>	not reflexive: cmp(a, a) must be 0

Both boolean comparators fail on the very first check, because `cmp(a, a)` is true, which coerces to 1, not 0. That one line of testing would have caught the bug.

Time $O(m^3)$ for a sample of size m **Space** $O(1)$

Never ship this. Run it on a 20-element sample during testing only.

TRAP

A comparator can also break transitivity without any `<=`. Example: `return a.score > b.score || a.name < b.name`; Here `||` lets the second key fire even when the first key already decided, and the result is not transitive. Keys must be chained with `if`, never with `||`.

The follow-up. A comparator built from the multi-key template is correct by construction: it is a lexicographic order on a tuple of keys, and a lexicographic order on totally-ordered keys is always a strict weak ordering. If you cannot express your rule that way — for example “a beats b if they played and a won” — then your rule is probably not an ordering at all and sorting is the wrong tool; you want a topological sort (Chapter 16).

Answer: `>=` returns a boolean, which coerces to 1 or 0 — the comparator can never say “a first”, so nothing is sorted

Example 27 [Amazon · pattern]

An array is **k-sorted**: every element is at most k positions away from where it belongs. Sort it faster than $O(n \log n)$. $a = \{3, 1, 2, 6, 4, 8, 7\}$, $k = 2$. Constraints: $n \leq 10^6$, $k \leq 100$. Edge cases: $k = 0$ (already sorted); $k \geq n$.

Follow-up: “the array is a stream that does not fit in memory.”

SOLUTION

Approach 1 — Ignore the promise and just sort ($O(n \log n)$)

correct, but wastes the k-sorted property

JS

```
const sortKSortedFull = (a, k) => a.sort((x, y) => x - y); // k is never used
```

Time $O(n \log n)$ **Space** $O(\log n)$

Gives the same array as the optimal version. The interviewer's point is that $k \leq 100$ is free information.

Approach 3 — Min-heap of size $k+1$ ($O(n \log k)$, optimal)

WHY A WINDOW OF SIZE $k+1$ IS ENOUGH

The true smallest element is at most k places from index 0, so it lives somewhere in $a[0 \dots k]$. Take the minimum of that window and it is the global minimum. Slide the window one step and repeat. A min-heap of $k + 1$ items is exactly that sliding window.

min-heap of size $k+1$

JS

```
const sortKSorted = (a, k) => {
  const heap = new MinHeap((x, y) => x - y); // toolkit: JS has no heap
  let w = 0; // write position
  for (const x of a) {
    heap.push(x);
    if (heap.size > k + 1) a[w++] = heap.pop();
  }
  while (heap.size) a[w++] = heap.pop(); // drain the tail
  return a;
};
```

Time $O(n \log k)$ **Space** $O(k)$

With $k = 100$, $\log k$ is about 7 against $\log n = 20$ for $n = 10^6$: roughly 3x fewer compares.

$\{3, 1, 2, 6, 4, 8, 7\}$ with $k = 2$ becomes 1 2 3 4 6 7 8. $\{9, 4\}$ with $k = 1$ becomes 4 9.

TRAP

`if (pq.size() > k + 1)` compares a `size_t` with an `int`. If `k` were `-1` the `int` would convert to a giant unsigned value and the branch never fires. Cast: `(int)pq.size()`.

The follow-up. The heap version already handles a stream: it holds only $k + 1$ items at a time and emits output as it goes. That is its real selling point — not the $\log k$, but the $O(k)$ memory. Push every arriving item, pop whenever the heap exceeds $k + 1$, and the popped sequence is the sorted stream.

Answer: 1 2 3 4 6 7 8

Example 28 [Goldman Sachs · pattern]

Sort by the number of 1-bits in the binary form. Same bit count: smaller value first. {7, 1, 8, 3, 2, 15, 4}. Constraints: $n \leq 10^5$, $0 \leq a[i] < 2^{31}$. Edge cases: value 0; two values with the same popcount.

Follow-up: “sort in $O(n)$ using this key.”

SOLUTION

Approach 1 — Count the bits with a loop inside the comparator ($O(n \log n \cdot 32)$)

correct, but the bit loop runs on every comparison

JS

```
const popcountLoop = (x) => {
  let c = 0;
  while (x) { c += x & 1; x >>= 1; } // >> not >>: x can reach 2^31
  return c;
};

const byPopcountLoop = (a) =>
  [...a].sort((x, y) => popcountLoop(x) - popcountLoop(y) || x - y);
```

`popcountLoop(15)` is 4 and `popcountLoop(0)` is 0.

Time $O(32n \log n)$ **Space** $O(1)$

Same output as the version below, roughly 30x more work inside the comparator.

Approach 2 — Compute each key once, then sort pairs ($O(n \log n)$, the portable fix)

Build an array of [popcount, value] pairs first and sort that — or, as the code below does, a Map from value to key. The bit loop then runs n times instead of $2n \log n$ times. This is the trick to remember whenever the sort key is expensive: **precompute the key**.

Approach 3 — One CPU instruction ($O(n \log n)$, optimal constant)

popcount as the primary key

JS

```
const popcount = (x) => { // JS has no __builtin_popcount
  x = x - ((x >> 1) & 0x55555555);
  x = (x & 0x33333333) + ((x >> 2) & 0x33333333);
  x = (x + (x >> 4)) & 0x0f0f0f0f;
  return (x * 0x01010101) >> 24;
};

const byPopcount = (a) => {
  const key = new Map(a.map(x => [x, popcount(x)])); // compute each key ONCE
  return [...a].sort((x, y) => key.get(x) - key.get(y) || x - y);
};
```

value	binary	bits
1	0001	1
2	0010	1
4	0100	1
8	1000	1
3	0011	2
7	0111	3
15	1111	4

Output: 1 2 4 8 3 7 15.

Time $O(n \log n)$ **Space** $O(n)$

The bit-twiddle popcount is ~5 operations against ~32 for the loop. Verified equal to the loop on every x from 0 to 99999.

The follow-up, answered. The key has only 32 possible values (0 to 31 bits set), so counting sort applies: bucket by popcount in $O(n)$, then sort each bucket. But the within-bucket rule is “smaller value first”, which still needs a comparison sort **inside** each bucket, so the honest total is $O(n \log n)$ again. The $O(n)$ answer is only real if the tie rule is “keep input order” — then bucket by popcount with a stable pass and you are done in $O(n + 32)$.

SHORTCUT

Every bit operator in JavaScript first truncates to **32 bits**, so this popcount is only correct for $0 \leq x < 2^{32}$. Use `>>>` (not `>>`) throughout, or the sign bit of a value above 2^{31} makes the shift drag in ones. Above 2^{32} there is no 64-bit type to switch to: count the bits of a BigInt instead — `let c = 0n; while (n) { c += n & 1n; n >>= 1n; }`. Tested: that loop gives 40 for $2n^{**}40n - 1n$.

Answer: 1 2 4 8 3 7 15

Example 29 [Uber · pattern]

A 100 GB file of integers must be sorted on a machine with 8 GB of RAM. Describe the algorithm and write the in-memory part.

Follow-up: “now there are 50 machines.”

SOLUTION

The shape of external merge sort:

1. Read the file in chunks that fit in RAM (say 4 GB each, giving 25 chunks).
2. Sort each chunk in memory and write it back to disk as a sorted run.
3. Merge all 25 runs at once with a min-heap, reading a small buffer from each run and writing the output as one long stream.

Step 3 is the only interesting code. It is the K-way merge.

K-way merge with a min-heap

JS

```
const mergeK = (chunks) => {
  const heap = new MinHeap((a, b) => a.val - b.val);
  chunks.forEach((chunk, c) => {
    if (chunk.length) heap.push({ val: chunk[0], c, i: 0 }); // skip empties
  });
  const out = [];
  while (heap.size) {
    const { val, c, i } = heap.pop();
```

```

    out.push(val);
    if (i + 1 < chunks[c].length)
        heap.push({ val: chunks[c][i + 1], c, i: i + 1 });    // refill same chunk
    }
    return out;
};

```

On $\{\{1,9,12\}, \{\}, \{2,3,30\}, \{0,11\}\}$ the output is 0 1 2 3 9 11 12 30. All-empty input gives an empty result.

Time $O(N \log K)$ **Space** $O(K)$

N total items, K runs. Memory holds only one item per run plus the I/O buffers.

TRAP

if (!chunks[c].empty()) is not decoration. Pushing chunks[c][0] from an empty vector is undefined behaviour and is the single most common bug in K-way merge code.

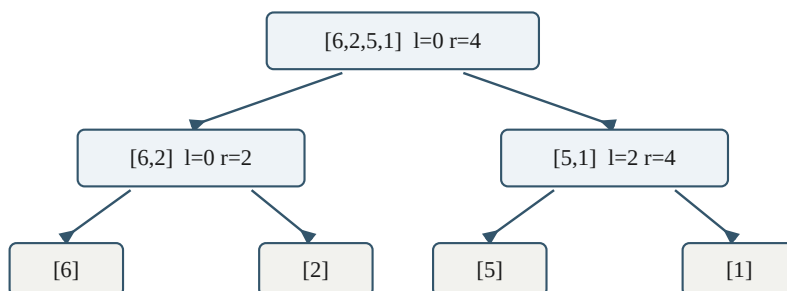
The follow-up. With 50 machines the same three steps run in parallel: every machine sorts its own slice locally (step 1–2), then a merge tree combines them, each level halving the number of runs. The total I/O is what matters, not the compare count — the algorithm is chosen to read and write each byte a small fixed number of times, which is why merge sort and not quicksort is the external algorithm of choice. Quicksort’s access pattern is random; merge sort’s is sequential, and sequential disk reads are an order of magnitude faster.

Answer: external merge sort: chunk, sort in RAM, K-way merge with a heap

Dry run: counting inversions, line by line

Input $a = \{6, 2, 5, 1\}$. Expected answer: the pairs (6, 2), (6, 5), (6, 1), (2, 1), (5, 1), so 5.

Step 1 — the recursion splits the array



the call tree; counting happens on the way back up

Step 2 — every merge, exactly as the program printed it

instrumented run of mergeCount on {6,2,5,1}

TEXT

```

merge l=0 m=1 r=2 left=[6] right=[2] before=0
  take right 2 add 1
  drain left 6 add 0
-> [2,6] total=1

```

```

merge l=2 m=3 r=4 left=[5] right=[1] before=0
  take right 1 add 1

```

```

    drain left 5 add 0
    -> [1,5] total=1

merge l=0 m=2 r=4 left=[2,6] right=[1,5] before=2
    take right 1 add 2
    take left 2 add 0
    take right 5 add 1
    drain left 6 add 0
    -> [1,2,5,6] total=5

answer = 5

```

Step 3 — the top merge as a state table

Entering the top merge: left half [2,6] at positions 0..1, right half [1,5] at positions 2..3, $m = 2$, count carried up from below = $1 + 1 = 2$.

i	j	a[i]	a[j]	c	decision
0	2	2	1	2	$2 \leq 1$? no. Take right. Left still has $m - i = 2 - 0 = 2$ items, add 2 $\rightarrow c = 4$
0	3	2	5	4	$2 \leq 5$? yes. Take left, add 0. $i \rightarrow 1$
1	3	6	5	4	$6 \leq 5$? no. Take right. Left has $2 - 1 = 1$ item, add 1 $\rightarrow c = 5$
1	—	6	—	5	right half empty. Drain the left, add 0

Buffer fills as 1, then 1 2, then 1 2 5, then 1 2 5 6. Final count 5.

WHAT TO SAY OUT LOUD IN THE INTERVIEW

“When I take a value from the right half, every left value still waiting is larger than it. That is $m - i$ inversions in one addition. The merge sort was going to walk the array anyway, so counting is free.”

Step 4 — check the edge cases

input	count	reason
{1,2,3}	0	already sorted, the $\leq$ branch always wins
{4,3,2,1}	6	every pair is an inversion: $4 \cdot 3/2 = 6$
{2,2,2}	0	equal is not an inversion, thanks to $a[i] \leq a[j]$
{}	0	$r - l \leq 1$ returns immediately

Python version of the same algorithm

merge sort inversion count in Python

PYTHON

```

def count_inversions(a):
    def go(v):
        if len(v) <= 1:
            return v, 0
        m = len(v) // 2
        L, cl = go(v[:m])
        R, cr = go(v[m:])
        out, c, i, j = [], cl + cr, 0, 0
        while i < len(L) and j < len(R):

```



```

        if L[i] <= R[j]:
            out.append(L[i]); i += 1
        else:
            c += len(L) - i
            out.append(R[j]); j += 1
    out += L[i:] + R[j:]
    return out, c
return go(a)[1]

print(count_inversions([6, 2, 5, 1])) # 5
print(count_inversions([]))          # 0
print(count_inversions([4, 3, 2, 1])) # 6

```

Practice

PRACTICE · WARM-UP target: 20 min for P1--P3

- P1.** Print the third **smallest distinct** value of $\{9, 4, 4, 1, 7, 9\}$. Say what your code does when there are fewer than three distinct values.
- P2.** Sort $\{\text{go}, \text{tree}, \text{ant}, \text{bee}, \text{apple}\}$ by length descending; same length means **reverse** alphabetical.
- P3.** Write `isSorted(a)` returning whether `a` is non-decreasing. Test it on $\{1, 2, 2, 5\}$, $\{3, 1\}$ and the empty array.

PRACTICE · TIER 1 target: 45 min for P4--P7

- P4.** Two arrays `a` and `b` of the same length $n \leq 10^5$, values up to 10^5 . Pair each `a[i]` with one `b[j]`, each used once, to make $\sum a_i b_j$ as **small** as possible. Watch for overflow.
- P5.** Count distinct values in an array of size 10^5 using sorting only (no hash map).
- P6.** Rearrange so even numbers come first in increasing order and odd numbers follow in decreasing order. Test $\{5, 2, 9, 4, 7, 8\}$.
- P7.** Given stick lengths and a tolerance d , pair up sticks whose lengths differ by at most d . Each stick is in at most one pair. Maximise the number of pairs. Test $\{1, 3, 4, 9, 10, 20\}$ with $d = 2$.

PRACTICE · TIER 2 target: 50 min for P8--P11

- P8.** Each boat carries at most 2 people and at most `limit` kilograms. Find the smallest number of boats. Test $\{1, 2, 3, 5\}$ with `limit = 5`.
- P9.** Child i is satisfied by a cookie of size $\geq \text{need}[i]$. One cookie per child. Maximise satisfied children. Test `need = {1, 3, 5}`, `size = {1, 2, 4, 6}`.
- P10.** Orders have `(id, priority, timestamp)`. Return the ids of the top 3 by priority descending, then oldest timestamp first, then smallest id. Test $\{(1, 2, 50), (2, 5, 90), (3, 5, 30), (4, 1, 10)\}$.
- P11.** Given arrival and departure times of drivers going online and offline, find the largest number that were online at the same moment. Test in $\{1, 2, 6\}$, out $\{4, 5, 8\}$.

PRACTICE · TIER 3 target: 60 min for P12--P15

- P12.** Count pairs $i < j$ with $a[i] > 2 \cdot a[j]$ in $O(n \log n)$. Test $\{8, 3, 1, 20, 2\}$. Beware: $2 \cdot a[j]$ can overflow `int`.

P13. h-index: the largest h such that at least h papers have $\geq h$ citations. Test $\{6, 1, 4, 0, 3\}$.

P14. Minimum number of swaps (of any two positions) to sort an array of distinct values. Test $\{4, 3, 2, 1\}$.

P15. Given intervals, choose the largest set with no two overlapping. Test $\{(1, 3), (2, 5), (4, 7), (6, 8), (8, 9)\}$.

ANSWER KEY

P1 — answer 7. Sort, then `unique` collapses runs of equal values. Return a sentinel if fewer than 3 remain.

```
const thirdSmallest = (a) => {
  const u = [...new Set(a)].sort((x, y) => x - y);
  return u.length >= 3 ? u[2] : null;
};
```

`[1, 1]` returns `null` — fewer than three distinct values.

Time $O(n \log n)$ **Space** $O(n)$

P2 — apple tree bee ant go.

```
const byLengthDesc = (v) =>
  [...v].sort((a, b) => b.length - a.length ||
    (a > b ? -1 : a < b ? 1 : 0)); // reverse alphabetical
```

bee beats ant because both have length 3 and "bee" > "ant".

P3 — 1, 0, 1. An empty array and a one-element array are sorted by definition.

```
const isSorted = (a) => {
  for (let i = 1; i < a.length; i++) if (a[i] < a[i - 1]) return false;
  return true;
};
```

The loop starts at $i = 1$, so size 0 and size 1 never enter it.

P4 — answer 30 for $a=\{1, 5, 3\}$, $b=\{4, 2, 8\}$. Pair the smallest of one with the largest of the other. Sorted $a = 1, 3, 5$; b descending = $8, 4, 2$; products $8 + 12 + 10 = 30$.

```
const minPairSum = (a, b) => {
  const x = [...a].sort((p, q) => p - q); // one ascending
  const y = [...b].sort((p, q) => q - p); // the other descending
  let s = 0n; // BigInt: the total passes 2^53-1
  for (let i = 0; i < x.length; i++) s += BigInt(x[i]) * BigInt(y[i]);
  return s;
};
```

Why it works (exchange argument): take any two pairs (x_1, y_1) and (x_2, y_2) with $x_1 < x_2$ and $y_1 < y_2$. Then $x_1 y_2 + x_2 y_1 - (x_1 y_1 + x_2 y_2) = -(x_2 - x_1)(y_2 - y_1) < 0$, so crossing them is always better. Repeat and you reach “ascending against descending”. **Big-number test:** $a = b = [100000, 100000]$ gives 20000000000. That is past the 32-bit limit — a C++ `int` would wrap — but a JavaScript number holds it exactly, and stays exact all the way to $2^{53} - 1$.

Time $O(n \log n)$ **Space** $O(1)$

P5 — answer 3 for $\{4, 4, 4, 2, 9, 2\}$.

```
const distinctCount = (a) => {
  const b = [...a].sort((x, y) => x - y);
  let c = 0;
  for (let i = 0; i < b.length; i++) if (i === 0 || b[i] !== b[i - 1]) c++;
  return c;
};
```

`unique` only removes **adjacent** duplicates, which is exactly why the sort must come first. Empty input gives 0.

Time $O(n \log n)$ **Space** $O(1)$

P6 — 2 4 8 9 7 5.

```
const evensThenOdds = (a) => {
  [...a].sort((x, y) => {
    const ex = x % 2 === 0, ey = y % 2 === 0;
    if (ex !== ey) return ex ? -1 : 1; // an even value comes first
    return ex ? x - y : y - x; // evens ascending, odds descending
  });
};
```

return ex is the whole “evens first” rule: if exactly one of them is even, ex is true precisely when it is x. **Careful with negatives:** in C++, $-3 \% 2$ is -1 , not 1 . Use $x \% 2 == 0$ (as above) rather than $x \% 2 == 1$, which is false for every negative odd number.

P7 — answer 2. Sort, then walk left to right pairing each element with its neighbour when they fit.

```
const maxPairs = (a, d) => {
  const b = [...a].sort((x, y) => x - y);
  let c = 0;
  for (let i = 1; i < b.length; ) {
    if (b[i] - b[i - 1] <= d) { c++; i += 2; } // pair them, skip both
    else i++;
  }
  return c;
};
```

Sorted 1 3 4 9 10 20: (3,4) pairs, (9,10) pairs, 1 and 20 are left over. Answer 2. $\{2, 2, 2\}$ with $d = 0$ gives 1: one pair and a leftover.

Time $O(n \log n)$ **Space** $O(1)$

P8 — answer 3. Sort, then two pointers: the heaviest person always boards; the lightest joins only if they fit together.

```
const minBoats = (w, limit) => {
  const b = [...w].sort((x, y) => x - y);
  let i = 0, j = b.length - 1, boats = 0;
  while (i <= j) {
    if (b[i] + b[j] <= limit) i++; // the lightest rides along
    j--; boats++; // the heaviest always rides
  }
  return boats;
};
```

$\{1, 2, 3, 5\}$, limit 5: (5) alone, (1,3) together, (2) alone $\Rightarrow$ 3 boats.

Time $O(n \log n)$ **Space** $O(1)$

P9 — answer 3. Sort both. Give the smallest cookie that works to the least demanding child.

```
const assignCookies = (need, size) => {
  const n = [...need].sort((a, b) => a - b);
  const s = [...size].sort((a, b) => a - b);
  let i = 0, j = 0, c = 0;
  while (i < n.length && j < s.length) {
    if (s[j] >= n[i]) { c++; i++; } // this cookie satisfies this child
    j++; // the cookie is used up either way
  }
  return c;
};
```

need = 1,3,5; size = 1,2,4,6: cookie 1 to child 1; cookie 2 wasted; cookie 4 to child 3; cookie 6 to child 5. Answer 3. need={9}, size={1} gives 0.

P10 — 3 2 1.

```
const topOrders = (o, k) => {
  [...o].sort((a, b) => b.priority - a.priority || a.ts - b.ts || a.id - b.id)
```

```
.slice(0, k)
.map(x => x.id);
```

Orders 2 and 3 both have priority 5; order 3's timestamp 30 beats order 2's 90, so 3 is first. Then order 1 (priority 2), then order 4.

P11 — answer 2. Same sweep as Example 19: sort the two lists, then walk.

```
const busiest = (inTimes, outTimes) => {
  const a = [...inTimes].sort((x, y) => x - y);
  const b = [...outTimes].sort((x, y) => x - y);
  let i = 0, j = 0, cur = 0, best = 0;
  while (i < a.length) {
    if (a[i] < b[j]) { cur++; best = Math.max(best, cur); i++; }
    else { cur--; j++; }
  }
  return best;
};
```

Sessions [1, 4), [2, 5), [6, 8): the first two overlap, the third is alone. Answer 2.

P12 — answer 5. Merge sort again, but **count before you merge**, with a separate pointer.

```
const cntBig = (a, buf, l, r) => {
  if (r - l <= 1) return 0;
  const m = (l + r) >> 1;
  let c = cntBig(a, buf, l, m) + cntBig(a, buf, m, r);
  let j = m;
  for (let i = l; i < m; i++) { // both halves are sorted here
    while (j < r && a[i] > 2 * a[j]) j++;
    c += j - m; // a[m..j) all qualify
  }
  let i = l; j = m; let k = l; // now the ordinary merge
  while (i < m && j < r) buf[k++] = a[i] <= a[j] ? a[i++] : a[j++];
  while (i < m) buf[k++] = a[i++];
  while (j < r) buf[k++] = a[j++];
  for (let t = l; t < r; t++) a[t] = buf[t];
  return c;
};
```

```
const countBigPairs = (a) => {
  const b = [...a];
  return cntBig(b, new Array(b.length), 0, b.length);
};
```

{8, 3, 1, 20, 2}: the qualifying pairs are (8, 3), (8, 1), (8, 2), (3, 1), (20, 2) $\Rightarrow$ 5. A randomised cross-check against the $O(n^2)$ version agreed on 300 random arrays. **Two traps:** the counting loop must run **before** the merge destroys the two sorted halves; and $2 * a[j]$ overflows `int` when $a[j] > 10^9$, so write $2LL * a[j]$.

Time $O(n \log n)$ **Space** $O(n)$

P13 — answer 3. Sort descending and walk while the citation count can still support the position.

```
const hIndex = (c) => {
  const b = [...c].sort((x, y) => y - x); // descending
  let h = 0;
  while (h < b.length && b[h] >= h + 1) h++;
  return h;
};
```

Sorted 6 4 3 1 0: $6 \geq 1$ ok, $4 \geq 2$ ok, $3 \geq 3$ ok, $1 \geq 4$ no. Answer 3. {0, 0} gives 0.

Time $O(n \log n)$ **Space** $O(1)$

P14 — answer 2. Sorting by cycles: every cycle of length L needs $L - 1$ swaps.

```
const minSwaps = (a) => {
  const p = a.map((v, i) => [v, i]).sort((x, y) => x[0] - y[0]);
  const seen = new Array(a.length).fill(false);
  let swaps = 0;
  for (let i = 0; i < a.length; i++) {
    if (seen[i] || p[i][1] === i) continue;
    let len = 0, j = i;
    while (!seen[j]) { seen[j] = true; j = p[j][1]; len++; }
    swaps += len - 1;
  }
  return swaps;
};
```

{4, 3, 2, 1} has two cycles of length 2 ($0 \leftrightarrow 3$ and $1 \leftrightarrow 2$), so $1 + 1 = 2$ swaps. Already-sorted input gives 0.

Time $O(n \log n)$ **Space** $O(n)$

P15 — answer 3. Sort by **end** time, then take greedily.

```
const maxNonOverlap = (v) => {
  const b = [...v].sort((a, c) => a[1] - c[1]); // earliest finish first
  let c = 0, last = -Infinity;
  for (const [s, e] of b) if (s >= last) { c++; last = e; }
  return c;
};
```

Picks (1, 3), then (4, 7), then (8, 9). Answer 3. **Why end time and not start time:** finishing early leaves the most room for everything that follows. Sorting by start would pick (1, 3) and then be blocked by a very long interval.

Time $O(n \log n)$ **Space** $O(1)$

ONE-PAGE REVISION CARD

The comparator template — write this from memory

multi-key, safe, always correct

JS

```
v.sort((a, b) =>
  b.k1 - a.k1 ||           // key 1, bigger first
  a.k2 - b.k2 ||           // key 2, smaller first
  (a.k3 < b.k3 ? -1 : a.k3 > b.k3 ? 1 : 0)); // final tie-breaker
```

Which tool for which question

Question says	Use	Time	Space
sort everything	sort	$O(n \log n)$	$O(\log n)$
keep ties in input order	stable_sort	$O(n \log n)$	$O(n)$
top k , in order	partial_sort	$O(n \log k)$	$O(1)$
just the k -th	nth_element	$O(n)$ avg	$O(1)$
values are $0..K$, $K \approx n$	counting sort	$O(n + K)$	$O(K)$
only 3 distinct values	Dutch flag	$O(n)$	$O(1)$
each item $\leq k$ from home	min-heap size $k + 1$	$O(n \log k)$	$O(k)$
count inversions	merge sort or BIT	$O(n \log n)$	$O(n)$
maximum gap after sorting	pigeonhole buckets	$O(n)$	$O(n)$
bigger than RAM	external merge sort	$O(n \log n)$ I/O	$O(K)$

Sort-then-scan playbook

After sorting, this becomes...	...this scan
closest pair	$a[i] - a[i-1]$ over all i
pair with a target sum	two pointers from both ends
duplicates	$a[i] == a[i-1]$
k -th smallest	$a[k-1]$
pair small with large	$a[i]$ against $b[n-1-i]$
most concurrent intervals	merge two sorted event lists
max non-overlapping intervals	greedy by earliest end
minimum total move cost	move everything to the median

Top traps

1. Bare `a.sort()` on numbers $\rightarrow$ **lexicographic** order. `[10,9,1].sort()` gives `[1,10,9]`. Numbers always need `a.sort((x, y) => x - y)`. This is the single most common JavaScript bug in a coding round.
2. A comparator that returns a **boolean** $\rightarrow$ coerced to 1 or 0, so it can never say “a first” and nothing sorts. Always return a number.

3. `a.sort(...)` sorts **in place** and returns the same array. When the caller still needs the original, sort a copy: `[...a].sort(...)`.
4. Two keys joined with `||` where the first key can legitimately return `0` $\rightarrow$ the second key fires on a non-tie. Subtracting numbers is safe; a hand-rolled first key that returns `0` for “a first” is not.
5. Counting sort when the value range is $10^9 \rightarrow$ out of memory.
6. Sorting an array of objects and expecting the **original** array to stay untouched.
7. In Dutch flag, advancing `mid` after a swap with `high`.
8. Pushing from an empty chunk in a K-way merge.
9. Sorting data that already arrived sorted — merge instead.
10. Using the mean where the problem needs the median.

Complexity facts to quote

- Comparison sorting cannot beat $O(n \log n)$; there are $n!$ orders and each compare gives one bit, so at least $\log_2(n!) \approx n \log_2 n - 1.44n$ compares are needed.
- V8's `Array.prototype.sort` is TimSort: a stable merge sort that finds and extends already-ordered runs. Worst case is $O(n \log n)$; an already-sorted array costs $O(n)$. Stability has been guaranteed by the language since ES2019.
- Counting and radix sorts beat the bound only because they never compare two values.

Strings & Pattern Matching

A string is an array of characters. Every array trick still works — plus four that only strings need.

THE FOUR STRING ENGINES

Almost every string question in a coding round is one of these four, or two of them stacked.

Engine	Use it when the question says...	Cost
two pointers	palindrome, reverse, compare from both ends	$O(n)$
frequency window	anagram, “all characters of”, “at most k distinct”	$O(n)$
prefix function (KMP)	find a pattern, borders, periods, repeats	$O(n + m)$
rolling hash	compare many substrings fast, count distinct	$O(n)$ expected

If none of the four fits, the answer is usually dynamic programming on two strings (Chapter 18) — edit distance, longest common subsequence.

This chapter needs nothing from the book’s `toolkit.js` — no heap, no DSU, no deque. What it does need is the **JS Toolkit** appendix’s list of JavaScript’s string gaps: no `isalpha`, no ordered map, no exact integer past $2^{53} - 1$, and immutable strings. Every one of those shows up below in a trap box at the point where it first bites.

Pattern in one page

The JavaScript string toolkit

```

everything you actually use JS

const s = 'placement';
s.length;                // 9
s.slice(2, 6);            // 'acem' -> [start, end), NOT start + length
s.indexOf('cem');         // 3
s.indexOf('zzz');         // -1      (a real -1, not a huge unsigned value)
s.startsWith('place');   // true

let t = s;
t += 's';                 // strings are IMMUTABLE: this builds a new one
[...t].reverse().join(''); // 'stnemecalp'

'apple' < 'banana';       // true : dictionary order
'Zoo' < 'apple';          // true : 'Z' is 90, 'a' is 97

Number(' -42');           // -42   (leading spaces are skipped)
parseInt('42abc', 10);    // 42    (Number('42abc') is NaN)
String(7 * 6);            // '42'

```

TRAP

`s.indexOf(x)` returns -1 when there is no match, and -1 is truthy. So if `(s.indexOf(x))` is a bug twice over: it says “found” for a missing pattern, and it says “not found” when the pattern sits at index 0. Tested in

Node on `s = 'placement': if (s.indexOf('p'))` takes the **else** branch even though 'p' is right there, and `if (s.indexOf('zzz'))` takes the **then** branch even though 'zzz' is absent. Always compare explicitly:

```
if (s.indexOf(x) !== -1) { ... } // or, clearer: if (s.includes(x)) { ... }
```

Strings are immutable

TRAP

You cannot write `s[0] = 'x'`. A JavaScript string never changes. The assignment fails silently in loose mode and throws `TypeError` in strict mode (which every ES module and every class body already is). Any “in place” string algorithm becomes:

```
const a = [...s]; edit a a.join('')
```

Two problems in this chapter depend on it — Example 10 (reverse the words in place) and Example 16 (compress a character array). Both take an **array of characters**, which is what the interviewer means by “in place” when the language has immutable strings. Say that out loud; it is the correct answer, not a dodge.

Character arithmetic

letters are just small numbers

JS

```
const c = 'g';
c.charCodeAt(0) - 97; // 6 -> the index into a 26-slot array
String.fromCharCode(97 + 3); // 'd'
c >= 'a' && c <= 'z'; // true -> strings compare by code unit
'7' >= '0' && '7' <= '9'; // true
c.toUpperCase(); // 'G'
Output: 6 d true true G
```

JavaScript has no `isalpha` or `isdigit`. Two honest replacements, both used in this chapter:

- **range comparison**, the fastest: `c >= 'a' && c <= 'z', c >= '0' && c <= '9'`. String comparison in JavaScript is by code unit, so this does exactly what you expect.
- **a regular expression**, the most readable: `/[a-z0-9]/i.test(c)`.

Both return a real `true/false`, so `===` against them is safe.

TRAP

`s.length` counts UTF-16 code units, not characters. Tested in Node: `'😊'.length` is 2, while `[... '😊'].length` is 1, because the spread operator walks code points. Interview inputs are almost always ASCII, where the two agree — but if the question says “any Unicode text”, index with `[...s]` and say why.

Building a string

the slow way and the fast way

JS

```
let out = '';
for (let i = 0; i < 5; i++) out += String.fromCharCode(97 + i); // 'abcde'

const parts = [];
for (let i = 0; i < 5; i++) parts.push(String.fromCharCode(97 + i));
const joined = parts.join(''); // 'abcde'
```

This is one place where JavaScript is kinder than C++, and you should know why. In C++ `s = s + c` copies the whole string every round, so n appends cost $O(n^2)$. V8 does not copy: `out += c` builds a **rope**, a tree of pieces that is flattened once, lazily, when the string is first read. So `+=` is amortised $O(1)$ already.

Timed in Node with $n = 10^6$ single-character appends:

method	time
<code>out += c</code>	51 ms
<code>parts.push(c)</code> then <code>parts.join('')</code>	75 ms

So `+=` is fine, and often the faster of the two. Reach for the array-and-join version when you are assembling **pieces** rather than characters, or when you need the pieces separately anyway — not because you fear quadratic behaviour.

Template 1 — two pointers

the palindrome shape

JS

```
const pal = (s) => {
  let i = 0, j = s.length - 1;
  while (i < j) {
    if (s[i] !== s[j]) return false;
    i++; j--;
  }
  return true;
};
// pal('racecar') is true, pal('race') is false
```

Loop condition `i < j`, not `i <= j`: when they meet on the middle character there is nothing left to compare.

Template 2 — the frequency window

does t contain a window that matches p's letter counts?

JS

```
const windowEqual = (t, p) => {
  const n = t.length, m = p.length;
  if (m > n) return false;
  const need = new Array(26).fill(0), have = new Array(26).fill(0);
  for (const c of p) need[c.charCodeAt(0) - 97]++;
  const same = () => need.every((v, i) => v === have[i]);
  for (let i = 0; i < n; i++) {
    have[t.charCodeAt(i) - 97]++; // the char entering the window
    if (i >= m) have[t.charCodeAt(i - m) - 97]--; // the char leaving the window
    if (i >= m - 1 && same()) return true;
  }
  return false;
};
// windowEqual('helloworld', 'abc') is true, windowEqual('hello', 'abc') is false
```

THE INVARIANT

After the two lines inside the loop, `have` always holds the counts of exactly the window `t[i-m+1 .. i]`. The `if (i >= m-1)` guard makes sure we only compare once that window is full.

Template 3 — the prefix function (LPS)

WHAT `LPS[i]` MEANS

$\text{lps}[i]$ = the length of the longest string that is **both a proper prefix and a proper suffix** of $p[0..i]$. “Proper” means it is not the whole thing.

For $p = \text{"aabaa"}:$

i	0	1	2	3	4
$p[0..i]$	a	aa	aab	aaba	aabaa
$\text{lps}[i]$	0	1	0	1	2

$\text{lps}[4] = 2$ because “aa” starts and ends “aabaa”.

the prefix function: 8 lines, learn them by heart

JS

```
const buildLPS = (p) => {
  const m = p.length;
  const lps = new Array(m).fill(0);
  let len = 0; // length of the border we are extending
  for (let i = 1; i < m; ) {
    if (p[i] === p[len]) lps[i++] = ++len;
    else if (len > 0) len = lps[len - 1]; // fall back; i does NOT move
    else lps[i++] = 0;
  }
  return lps;
};
// buildLPS('ababaca') is [0, 0, 1, 2, 3, 0, 1]
```

TRAP

In the middle branch, i must **not** advance. That branch is “try a shorter border on the same character”. Advancing i there is the classic KMP bug and it silently gives wrong answers on inputs like aab.

Template 4 — rolling hash

POLYNOMIAL HASHING

Treat the string as a number in base B :

$$H(s) = s_0 B^{m-1} + s_1 B^{m-2} + \dots + s_{m-1} \pmod{M}$$

Slide the window one step to the right:

$$H_{\text{new}} = (H_{\text{old}} - s_{\text{left}} \cdot B^{m-1}) \cdot B + s_{\text{right}} \pmod{M}$$

That is $O(1)$ per step. Two substrings with different hashes are certainly different. Two with the same hash are **probably** equal — so verify before you trust it.

prefix hashes let you hash ANY substring in $O(1)$

JS

```
const B = 131n, M = (1n << 61n) - 1n; // BigInt literals end in n

const buildHash = (s) => {
  const n = s.length;
  const h = new Array(n + 1).fill(0n), pw = new Array(n + 1).fill(1n);
```

```

for (let i = 0; i < n; i++) {
  h[i + 1] = (h[i] * B + BigInt(s.charCodeAt(i))) % M;
  pw[i + 1] = pw[i] * B % M;
}
// hash of s[l .. l+len-1], in O(1)
return (l, len) => (h[l + len] - h[l] * pw[len] % M + M) % M;
};

```

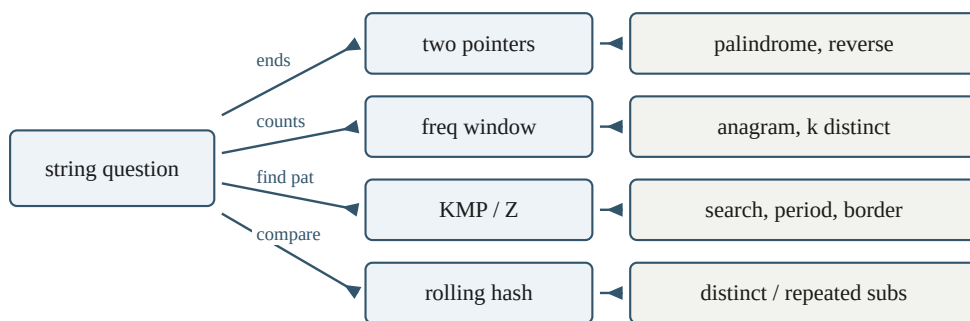
TRAP

A JavaScript number cannot do modular multiplication at all. Every number is a double, exact only to $\text{Number.MAX_SAFE_INTEGER} = 2^{53} - 1 \approx 9 \times 10^{15}$. Two residues below even the small modulus $10^9 + 7$ multiply to about 10^{18} , which is a hundred times past that line, so the product is rounded **before** the % ever runs. Tested in Node with $a = 10000000006$, $b = 9999999937$, $M = 10^9 + 7$:

- $(a * b) \% m$ gives 64,
- $(\text{BigInt}(a) * \text{BigInt}(b)) \% \text{BigInt}(m)$ gives 70.

The plain version is simply wrong, and it is wrong silently. So every rolling hash in this chapter uses `BigInt`, and the modulus can then be the good one, $2^{61} - 1$, with no extra machinery — `BigInt` has no upper limit, so C++'s `__int128` dance is not needed.

The cost is real: `BigInt` arithmetic runs several times slower than number arithmetic. Measured in Node, Example 27 on a random string of $n = 3000$ took **632 ms**. That is fine for the stated limit and far too slow for $n = 10^6$ — which is exactly why Example 27's follow-up reaches for a suffix automaton instead.



which engine to reach for

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Reverse a string in place, and separately write a palindrome check. Constraints: $n \leq 10^6$. Target: $O(n)$, $O(1)$ extra space. Edge cases: empty string, one character.

SOLUTION

both are the same two-pointer walk

JS

```

const reverseString = (s) => [...s].reverse().join('');

const isPalindrome = (s) => {

```

```
let i = 0, j = s.length - 1;
while (i < j) { if (s[i] !== s[j]) return false; i++; j--; }
return true;
};
```

reverseString("placement") gives tnemecalp. isPalindrome on "level", "levels", "", "z" gives true false true true. The empty string and a one-character string are palindromes: the loop never runs.

Time $O(n)$ **Space** $O(1)$

Answer: tnemecalp; true false true true

Example 2 [warm-up]

Count how many times each lowercase letter appears in "banana".

SOLUTION

26-slot frequency array

JS

```
const freq26 = (s) => {
  const f = new Array(26).fill(0);
  for (const c of s) f[c.charCodeAt(0) - 97]++;
  return f;
};
// banana -> a=3 b=1 n=2 z=0
```

c - 'a' maps 'a' to 0 and 'z' to 25. A 26-integer array is far faster than a hash map and is the right choice whenever the alphabet is fixed.

Time $O(n)$ **Space** $O(1)$

26 slots is constant space, no matter how long the string is.

Answer: a=3, b=1, n=2

Example 3 [TCS NQT · pattern]

Find the first position where p appears inside t, or -1. Constraints: $n, m \leq 10^3$ for now. Edge cases: empty pattern, pattern longer than text.

SOLUTION

naive search: try every start

JS

```
const naiveFind = (t, p) => {
  const n = t.length, m = p.length;
  if (m === 0) return 0; // empty pattern matches at 0
  for (let i = 0; i + m <= n; i++) { // i + m <= n, never i < n
    let j = 0;
    while (j < m && t[i + j] === p[j]) j++;
    if (j === m) return i;
  }
  return -1;
};
```

text	pattern	answer
abracadabra	cad	4

aaaa	aab	-1
abc	" "	0
" "	x	-1

Time $O(nm)$ **Space** $O(1)$

Worst case: $t = \text{aaaa...a}$, $p = \text{aaa...ab}$. Every start matches $m-1$ characters and then fails. KMP fixes exactly this.

Answer: 4

Example 4 [warm-up]

Clean a string: keep only letters and digits, and force everything to lowercase. "A man, a Plan!! 42".

SOLUTION

normalise

JS

```
const normalise = (s) => [...s.toLowerCase()].filter(c => /[a-z0-9]/.test(c)).join('');
// 'A man, a Plan!! 42' -> 'amanaplan42'
```

Time $O(n)$ **Space** $O(n)$

Answer: amanaplan42

Example 5 [Infosys · pattern]

Split a sentence on spaces. Extra spaces must not create empty words. " the quick brown ".

SOLUTION

split, then throw away the empty pieces

JS

```
const splitWords = (s) => s.split(' ').filter(w => w.length > 0);
```

Result: 3 words — [the][quick][brown].

TRAP

.split(' ') alone is not enough. It keeps every empty piece between two spaces. Tested in Node: ' the quick'.split(' ') gives ['', '', 'the', '', 'quick']. The .filter(w => w.length > 0) is what turns that into ['the', 'quick'], and it is the whole warm-up.

.split(/\s+/) collapses the runs but still leaves one empty piece at the front when the string starts with a space: ['', 'the', 'quick']. Filter either way.

Time $O(n)$ **Space** $O(n)$

Answer: the quick brown (3 words)

Example 6 [warm-up]

Are two strings anagrams of each other? Constraints: lowercase letters only, $n \leq 10^5$. Target: $O(n)$. Edge cases: different lengths, both empty.

SOLUTION

```
const isAnagram = (a, b) => {
  if (a.length !== b.length) return false; // the cheapest rejection first
  const f = new Array(26).fill(0);
  for (const c of a) f[c.charCodeAt(0) - 97]++;
  for (const c of b) if (--f[c.charCodeAt(0) - 97] < 0) return false;
  return true;
};
```

("listen", "silent") is true, ("aab", "abb") is false, ("", "") is true, ("ab", "abc") is false. Going negative means b has more copies of that letter than a — and since the lengths match, that is enough to reject without a second scan.

Time $O(n)$ **Space** $O(1)$

Answer: true false true false

The everyday string problems

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Return the first character that appears exactly once. "swiss". Constraints: $n \leq 10^5$, lowercase. Target: $O(n)$. Edge cases: no such character; one-character string.

SOLUTION

Approach 1 — Check each character against all others ($O(n^2)$)

brute force

JS

```
const firstUnique_brute = (s) => {
  for (let i = 0; i < s.length; i++) {
    let seen = false;
    for (let j = 0; j < s.length; j++)
      if (i !== j && s[i] === s[j]) { seen = true; break; }
    if (!seen) return s[i];
  }
  return '#';
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 3 — Count once, then scan once ($O(n)$, optimal)

two passes, both linear

JS

```
const firstUnique_count = (s) => {
  const f = new Array(26).fill(0);
  for (const c of s) f[c.charCodeAt(0) - 97]++; // pass 1: counts
```

```

    for (const c of s) if (f[c.charCodeAt(0) - 97] === 1) return c; // pass 2:
    original order
    return '#';
};

```

Time $O(n)$ **Space** $O(1)$

"swiss" gives w from both. "aabb" gives the sentinel #. "q" gives q.

WHY THE SECOND PASS MUST WALK THE STRING, NOT THE ARRAY

Scanning `f[0..25]` would give the alphabetically first unique letter, not the **positionally** first one. On "swiss" that would wrongly answer i instead of w.

The idea that unlocked it: **separate “how many” from “where”**. One pass answers the first, one pass answers the second.

Answer: w

Example 8 [Accenture · pattern]

Are two strings anagrams? Give two solutions and compare them. Constraints: $n \leq 10^5$. Edge cases: different lengths; empty strings.

SOLUTION

Approach 1 — Sort both and compare ($O(n \log n)$, 3 lines)

the one-liner answer

JS

```

const anagram_sort = (a, b) => {
  if (a.length !== b.length) return false;
  return [...a].sort().join('') === [...b].sort().join('');
};

```

Time $O(n \log n)$ **Space** $O(n)$

`[...a]` copies first, so the caller's string is untouched — strings are immutable anyway, and the array copy is what makes sorting possible at all.

Approach 3 — Count the letters ($O(n)$, optimal)

counting

JS

```

const anagram_count = (a, b) => {
  if (a.length !== b.length) return false;
  const f = new Array(26).fill(0);
  for (const c of a) f[c.charCodeAt(0) - 97]++;
  for (const c of b) if (--f[c.charCodeAt(0) - 97] < 0) return false;
  return true;
};

```

Time $O(n)$ **Space** $O(1)$

Both said true for ("listen", "silent"), false for ("rat", "car"), and true for two empty strings. Cross-checked against each other on 2000 random pairs.

SHORTCUT

Say the sorting answer first, then immediately offer the counting one. Interviewers like seeing that you know both and can name the trade-off: sorting works for **any** alphabet including Unicode; counting needs a fixed small alphabet but is linear.

`[...a].sort()` with no comparator is **correct** here — these are single characters and the default sort compares them as text. That same bare `.sort()` on numbers is the most common JavaScript bug there is. See the trap in Example 21.

Answer: both correct; counting is $O(n)$

Example 9 [Wipro · pattern]

Longest common prefix of a list of words. {flowchart, flower, flow}. Constraints: up to 10^4 words, each up to 200 characters. Edge cases: empty list; no common prefix at all; one word.

SOLUTION

Approach 1 — Shrink a running prefix ($O(\text{total length})$)

compare the answer against each word in turn

JS

```
const lcp_pairwise = (v) => {
  if (v.length === 0) return '';
  let best = v[0];
  for (let i = 1; i < v.length; i++) {
    let k = 0;
    while (k < best.length && k < v[i].length && best[k] === v[i][k]) k++;
    best = best.slice(0, k);
    if (best === '') break; // cannot get shorter than nothing
  }
  return best;
};
```

Approach 2 — Vertical scan (same cost, often faster in practice)

column by column, stop at the first disagreement

JS

```
const lcp_vertical = (v) => {
  if (v.length === 0) return '';
  for (let k = 0; k < v[0].length; k++) {
    const c = v[0][k];
    for (let i = 1; i < v.length; i++)
      if (k >= v[i].length || v[i][k] !== c) return v[0].slice(0, k);
  }
  return v[0];
};
```

Both give flow. {dog, cat} gives the empty string. {solo} gives solo.

Time $O(S)$ **Space** $O(1)$

S = total characters. Vertical stops as soon as one column disagrees, so on 10^4 words sharing nothing it reads only 10^4 characters.

TRAP

Reading past the end of a JavaScript string does **not** crash — `'abc'[9]` is undefined. That sounds safer than C++, and it is worse for you: the bug survives testing instead of dying loudly. Here `undefined !== c`

is true, so the code happens to give the right answer, but the same pattern silently returns NaN the moment arithmetic is involved. Keep the `k >= v[i].length` check first and **mean** it.

Answer: flow

Example 10 [Capgemini · pattern]

Reverse the order of words in a sentence and squeeze extra spaces. " the sky is blue " becomes "blue is sky the". Constraints: $n \leq 10^5$. Edge cases: one word; only spaces.

SOLUTION

Approach 1 — Split into an array, then join backwards ($O(n)$ time, $O(n)$ extra space)

split, then join backwards

JS

```
const reverseWords = (s) => s.split(' ').filter(w => w.length > 0).reverse().join(' ');
```

"word" gives word. " " gives the empty string.

Time $O(n)$ **Space** $O(n)$

The array of words is the extra space; on a 10^5 -character sentence that is another 10^5 characters plus array overhead.

Approach 3 — Reverse twice, in place ($O(n)$ time, $O(1)$ extra space)

THE DOUBLE REVERSAL

1. Squeeze the spaces so words are separated by exactly one space.
2. Reverse the **whole** string. Now the words are in the right order but each one is spelled backwards.
3. Reverse each word back.

"the sky" → "the sky" → "yks eht" → "sky the".

one character array, a read pointer and a write pointer

JS

```
const reverseRange = (a, l, r) => {
  while (l < r) { [a[l], a[r]] = [a[r], a[l]]; l++; r--; }
};

const reverseWords_inplace = (s) => {
  const a = [...s]; // strings are immutable: work on an array
  const n = a.length;
  let w = 0, i = 0;
  while (i < n) { // 1. squeeze spaces
    while (i < n && a[i] === ' ') i++;
    if (i === n) break;
    if (w) a[w++] = ' ';
    while (i < n && a[i] !== ' ') a[w++] = a[i++];
  }
  a.length = w; // drop the tail
  reverseRange(a, 0, a.length - 1); // 2. reverse everything
  let st = 0; // 3. reverse each word back
  for (let k = 0; k <= a.length; k++)
    if (k === a.length || a[k] === ' ') {
```

```

    reverseRange(a, st, k - 1);
    st = k + 1;
  }
  return a.join(' ');
};

```

Time $O(n)$ **Space** $O(1)$

The string is modified in place; only the counters are extra.

Both versions turned " the sky is blue " into blue is sky the, "word" into word, and " " into the empty string. A randomised check on 400 random sentences (random letters and spaces) gave identical results every time.

TRAP

The loop in step 3 runs to $k \leq a.length$, one past the end, so the **last** word (which has no space after it) also gets reversed. Stopping at $k < a.length$ leaves the last word spelled backwards.

Answer: blue is sky the

Example 11 [Cognizant · pattern]

Count how many times p occurs in t . Occurrences may **overlap**. $t = "aaaa"$, $p = "aa"$. Constraints: $n, m \leq 10^4$. Edge cases: empty pattern; pattern longer than text.

SOLUTION

count, do not stop at the first hit

JS

```

const countOccurrences = (t, p) => {
  if (p.length === 0 || p.length > t.length) return 0;
  let c = 0;
  for (let i = 0; i + p.length <= t.length; i++) {
    let j = 0;
    while (j < p.length && t[i + j] === p[j]) j++;
    if (j === p.length) c++;
  }
  return c;
};

```

t	p	count	why
aaaa	aa	3	starts 0, 1, 2 — they overlap
abcabc	abc	2	starts 0, 3
abc	z	0	—
a	abc	0	pattern too long

Time $O(nm)$ **Space** $O(1)$

Example 17 does the same job in $O(n + m)$ with KMP. Note there is no built-in overlapping count in JavaScript — `String.replaceAll` and a global regex both skip past each match.

TRAP

Advancing `i` by `p.size()` after a hit gives 2 instead of 3 on `aaaa/aa`. Read the question: “may overlap” means `i++`, always.

Answer: 3

Example 12 [TCS Digital · pattern]

Run-length encode a string: “aaabbbbcd” becomes “a3b4c1d1”. Then make a version that returns the original if the encoding is not shorter. Edge cases: empty string; no repeats at all.

SOLUTION

group equal neighbours

JS

```
const rle = (s) => {
  const out = [];
  let i = 0;
  while (i < s.length) {
    let j = i;
    while (j < s.length && s[j] === s[i]) j++; // j lands past the run
    out.push(s[i], String(j - i));
    i = j;
  }
  return out.join('');
};

const rleSafe = (s) => {
  const e = rle(s);
  return e.length < s.length ? e : s;
};
```

`rle("aaabbbbcd")` gives `a3b4c1d1`. `rleSafe("abc")` returns `abc` unchanged (the encoding `a1b1c1` is longer). `rle("")` gives the empty string.

Time $O(n)$ **Space** $O(n)$

The inner while never re-reads a character: `i` jumps straight to `j`.

Answer: a3b4c1d1

Example 13 [Adobe · pattern]

Is `b` a rotation of `a`? (“abcde”, “cdeab”) is yes; (“abcde”, “abced”) is no. Constraints: $n \leq 10^5$. Edge cases: different lengths; both empty.

SOLUTION

Approach 1 — Build every rotation and compare ($O(n^2)$)

brute force

JS

```
const isRotation_brute = (a, b) => {
  const n = a.length;
  if (b.length !== n) return false;
  if (n === 0) return true;
  for (let k = 0; k < n; k++) { // k = how far we rotated
    let ok = true;
    for (let i = 0; i < n; i++)
      if (a[(k + i) % n] !== b[i]) { ok = false; break; }
  }
  return ok;
};
```

```

    if (ok) return true;
  }
  return false;
};

```

Time $O(n^2)$ **Space** $O(1)$

The modulo trick avoids building the rotated string, but there are still n starts times n characters.

Approach 3 — Double the string and search once ($O(n)$, optimal)

THE ONE-LINE INSIGHT

Every rotation of a is a substring of $a + a$.

$a = \text{"abcde"} \Rightarrow a + a = \text{"abcdeabcde"}$, which contains bcdea , cdeab , deabc , eabcd — exactly the rotations, and nothing else of length 5 that is not a rotation.

double and search

JS

```

const isRotation = (a, b) => {
  if (a.length !== b.length) return false; // essential, see the trap below
  if (a.length === 0) return true;
  return (a + a).includes(b);
};

```

Both versions gave true for $(\text{"abcde"}, \text{"cdeab"})$, false for $(\text{"abcde"}, \text{"abced"})$ and true for two empty strings. The four tests in the question give true false true false, and the two versions agreed on 2000 random pairs.

Time $O(n)$ **Space** $O(n)$

V8's `String.prototype.includes` is not guaranteed linear — on an adversarial input it can degrade. Use `kmpSearch` when the worst case must be provable.

TRAP

The length check is not optional. Drop it and `isRotation("abcabc", "abc")` returns true, because "abcabcabcabc" really does contain "abc" — but "abc" is not a rotation of a six-character string. Tested: with the check in place it correctly returns false.

Answer: true false true false

Example 14 [Amazon · pattern]

Longest palindromic substring. "babadada". Constraints: $n \leq 2000$ for the $O(n^2)$ answer. Target: beat $O(n^3)$. Edge cases: even-length palindrome; empty string; all characters different.

SOLUTION

Approach 1 — Try every substring and test it ($O(n^3)$)

brute force, used as the reference

JS

```

const lps_brute = (s) => {
  let best = '';
  for (let i = 0; i < s.length; i++)
    for (let j = i; j < s.length; j++) {

```

```

    let ok = true;
    for (let a = i, b = j; a < b; a++, b--)
        if (s[a] !== s[b]) { ok = false; break; }
    if (ok && j - i + 1 > best.length) best = s.slice(i, j + 1);
}
return best;
};

```

Time $O(n^3)$ **Space** $O(1)$

$n = 2000$ gives $8 * 10^9$ character reads. Far too slow.

Approach 3 — Expand around every centre $(O(n^2))$, the interview answer

COUNTING THE CENTRES

A palindrome is decided by its centre. There are $2n - 1$ centres: n single characters (odd lengths) and $n - 1$ gaps between characters (even lengths). From each centre, push outwards while the two sides match.

expand around centre

JS

```

const lps_expand = (s) => {
    if (s.length === 0) return '';
    let bestL = 0, bestLen = 1;
    const grow = (l, r) => {
        while (l >= 0 && r < s.length && s[l] === s[r]) { l--; r++; }
        const len = r - l - 1; // l and r each stepped one too far
        if (len > bestLen) { bestLen = len; bestL = l + 1; }
    };
    for (let i = 0; i < s.length; i++) {
        grow(i, i); // odd centre
        grow(i, i + 1); // even centre
    }
    return s.slice(bestL, bestL + bestLen);
};

```

Time $O(n^2)$ **Space** $O(1)$

Only $2n-1$ centres, each expanding at most $n/2$ steps.

Both versions return adada for "babadada". "cbbd" gives bb (an even centre). "x" gives x. "" gives "". A randomised check against the brute force on 300 random strings matched every time.

TRAP

$\text{int len} = r - l - 1$, not $r - l + 1$. The while exits only **after** l and r have both moved one step too far, so the real span is $[l+1, r-1]$, of length $r - l - 1$. Getting this wrong is the number-one bug here.

Example 29 does the same job in $O(n)$ with Manacher's algorithm.

Answer: adada

Example 15 [Infosys · pattern]

Is a sentence a palindrome if you ignore case and everything that is not a letter or digit? "A man, a plan, a canal: Panama". Edge cases: empty string; a string of only punctuation.

SOLUTION

```
const alnum = (c) => c !== undefined && /[a-z0-9]/i.test(c);

const cleanPalindrome = (s) => {
  let i = 0, j = s.length - 1;
  while (i < j) {
    while (i < j && !alnum(s[i])) i++;
    while (i < j && !alnum(s[j])) j--;
    if (s[i].toLowerCase() !== s[j].toLowerCase()) return false;
    i++; j--;
  }
  return true;
};
```

("A man, a plan, a canal: Panama") is true, "hello" is false, "" is true, ".", " is true. "0P" is false — a digit and a letter never match.

Time $O(n)$ **Space** $O(1)$

Beats the 'clean into a new string then check' version, which needs $O(n)$ extra space.

TRAP

The skip loops carry $i < j$ inside them. Without it, a string of pure punctuation runs i past j , and in JavaScript $s[i]$ is then undefined — so `.toLowerCase()` throws `TypeError: Cannot read properties of undefined`. The `alnum` helper guards against undefined for the same reason.

Answer: true false true true

Example 16 [Microsoft · pattern]

Compress a character array **in place**: `xxxzyzz` becomes `x3yz2` (a run of 1 keeps no number). Return the new length. Constraints: $n \leq 10^5$. Edge cases: single character; no repeats; a run longer than 9.

SOLUTION

Approach 1 — Build the answer in a new string, then copy back ($O(n)$ time, $O(n)$ extra space)

easy to get right, but it breaks the 'in place' rule

JS

```
const compress_extra = (a) => {
  const out = [];
  let i = 0;
  while (i < a.length) {
    let j = i;
    while (j < a.length && a[j] === a[i]) j++;
    out.push(a[i]);
    if (j - i > 1) out.push(...String(j - i));
    i = j;
  }
  for (let k = 0; k < out.length; k++) a[k] = out[k];
  return out.length;
};
```

Time $O(n)$ **Space** $O(n)$

Produced byte-identical output to the in-place version on the test input.

Approach 3 — Two pointers in the same array ($O(n)$ time, $O(1)$ extra space)

read pointer and write pointer in one array

JS

```
const compress = (a) => {
  const n = a.length;
  let w = 0, i = 0;
  while (i < n) {
    let j = i;
    while (j < n && a[j] === a[i]) j++;
    a[w++] = a[i];
    if (j - i > 1)
      for (const c of String(j - i)) a[w++] = c; // 12 -> '1' then '2'
    i = j;
  }
  return w;
};
```

{x,x,x,y,z,z} gives length 5 and x3yz2. {p} gives length 1 and p.

Time $O(n)$ **Space** $O(1)$

w never passes i, so the write pointer can never overwrite unread data.

WHY THE WRITE POINTER IS ALWAYS SAFE

After finishing a run of length L , the read pointer has passed L cells and the write pointer has used $1 + (\text{digits of } L)$ cells. For $L = 1$ that is $1 \leq 1$; for $L \geq 2$ the digit count is at most $L - 1$. So $w \leq i$ always holds.

Answer: length 5, x3yz2

Real search and real windows

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Shopee · pattern]

Find **every** start position of p inside t , in $O(n + m)$. $t = \text{"abababacaba"}$, $p = \text{"ababaca"}$. Constraints: $n \leq 10^6$, $m \leq 10^5$. Edge cases: overlapping matches; empty pattern; pattern longer than the text.

Follow-up: “why is it linear when the inner loop can run many times?”

SOLUTION

Approach 1 — Naive: try every start ($O(nm)$, fails on aaaa...a)

the baseline, also used to verify KMP

JS

```
const naiveSearch = (t, p) => {
  const hits = [];
  if (p.length === 0 || p.length > t.length) return hits;
  for (let i = 0; i + p.length <= t.length; i++) {
    let j = 0;
    while (j < p.length && t[i + j] === p[j]) j++;
  }
  return hits;
};
```



```

    if (j === p.length) hits.push(i);
  }
  return hits;
};

```

Time $O(nm)$ **Space** $O(1)$

$t = a^{10^6}$ and $p = a^{10^5}b$ costs 10^{11} character reads.

Approach 3 — KMP ($O(n + m)$, optimal)

WHAT KMP FIXES

The naive version throws away everything it learned. After matching `ababa` and failing on the next character, it restarts at position 1 and re-reads `baba`.

But we already know the matched part is `ababa`, and `ababa` ends with `aba`, which is also how it **starts**. So we can jump the pattern forward and keep those 3 characters. That jump length is exactly `lps[j - 1]`. The text pointer `i` never moves backwards.

KMP search, using `buildLPS` from the pattern page

JS

```

const kmpSearch = (t, p) => {
  const hits = [];
  if (p.length === 0 || p.length > t.length) return hits;
  const lps = buildLPS(p);
  const n = t.length, m = p.length;
  let i = 0, j = 0;
  while (i < n) {
    if (t[i] === p[j]) {
      i++; j++;
      if (j === m) { hits.push(i - m); j = lps[j - 1]; } // keep going
    } else if (j > 0) j = lps[j - 1]; // slide the pattern, i stays put
    else i++; // no partial match at all
  }
  return hits;
};

```

Time $O(n + m)$ **Space** $O(m)$

`buildLPS` is $O(m)$, the search is $O(n)$.

Results: `"abababacaba" / "ababaca"` gives 2. `"aaaa" / "aa"` gives 0 1 2 (overlaps kept, because after a hit `j` drops to `lps[m - 1]` instead of 0). Empty pattern and too-long pattern both give no hits. A randomised cross-check against the naive search on 500 random cases matched every time.

The follow-up, answered. Look at `i + j`. Every `if` branch either increases `i` (first and third branch) or decreases `j` (second branch), and `j` never rises faster than `i`. So `i + j` strictly increases at every step and is bounded by $2n$. That caps the total number of steps at $2n$, no matter how much the inner fallback fires.

Python version

KMP in Python

PYTHON

```

def build_lps(p):
    lps = [0] * len(p)
    length, i = 0, 1

```

```

while i < len(p):
    if p[i] == p[length]:
        length += 1
        lps[i] = length
        i += 1
    elif length > 0:
        length = lps[length - 1]
    else:
        lps[i] = 0
        i += 1
return lps

def kmp_search(t, p):
    if not p or len(p) > len(t):
        return []
    lps = build_lps(p)
    hits, i, j = [], 0, 0
    while i < len(t):
        if t[i] == p[j]:
            i += 1; j += 1
            if j == len(p):
                hits.append(i - j)
                j = lps[j - 1]
        elif j > 0:
            j = lps[j - 1]
        else:
            i += 1
    return hits

print(build_lps("aabaa"))           # [0, 1, 0, 1, 2]
print(kmp_search("aabaabaaa", "aabaa")) # [0, 3]
print(kmp_search("aaaa", "aa"))      # [0, 1, 2]

```

Answer: 2

Example 18 [Sea/Shopee · pattern]

Search for a pattern with a rolling hash instead of KMP. Explain when each is the better choice. Constraints: $n \leq 10^6$. Edge cases: hash collision; pattern equal to the whole text.

SOLUTION

Rabin-Karp with verification

JS

```

const rabinKarp = (t, p) => {
    const hits = [];
    const n = t.length, m = p.length;
    if (m === 0 || m > n) return hits;
    const B = 131n, M = (1n << 61n) - 1n;
    let ph = 0n, th = 0n, power = 1n;
    for (let i = 0; i < m; i++) {
        ph = (ph * B + BigInt(p.charCodeAt(i))) % M;
        th = (th * B + BigInt(t.charCodeAt(i))) % M;
        if (i) power = power * B % M;
    }
    for (let i = 0; ; i++) {
        if (ph === th && t.startsWith(p, i)) hits.push(i);
    }
    // ends up as B^(m-1)
    // verify, never trust

```

```

    if (i + m >= n) break;
    th = (th - BigInt(t.charCodeAt(i)) * power % M + M) % M; // drop the leaving
char
    th = (th * B + BigInt(t.charCodeAt(i + m))) % M;           // add the entering
char
  }
  return hits;
};

```

Time $O(n + m)$ expected **Space** $O(1)$

Worst case $O(nm)$ if an adversary crafts collisions. The verify step keeps it CORRECT, just slower.

Results matched KMP exactly: 2 for the main case, 0 1 2 for aaaa/aa, 0 for abc/abc. A randomised cross-check against the naive search on 500 cases matched every time.

LINE BY LINE, THE SLIDE

Window is $t[i \dots i+m-1]$ with hash th .

1. $th - t[i] * \text{power}$ removes the leading character's contribution $t_i B^{m-1}$.
2. $+ M$ before $\% M$ keeps the value non-negative — JavaScript $\%$ keeps the sign of the left operand, so $-5n \% 3n$ is $-2n$, not $1n$. This is a **remainder**, not a mathematical modulo, and it bites in exactly this line.
3. $* B$ shifts every remaining character up one power.
4. $+ t[i+m]$ adds the new last character.

	KMP	Rabin-Karp
worst case	$O(n + m)$ guaranteed	$O(nm)$ if attacked
extra memory	$O(m)$	$O(1)$
multiple patterns	one LPS per pattern	one hash set, all at once
2-D / substring compare	awkward	natural
interview safety	always correct	say “and I verify on a hash match”

TRAP

Never return on a hash match without comparing the characters. With $M = 10^9 + 7$ and 10^6 windows, the chance of at least one collision is roughly $10^6/10^9 = 0.1\%$ per pattern — small, but judges run many tests.

Answer: same hit lists as KMP, expected $O(n + m)$

Example 19 [Grab · pattern]

Find every start index in t where an anagram of p begins. $t = \text{"cbaebabacd"}$, $p = \text{"abc"}$. Constraints: $n \leq 10^5$, lowercase only. Target: $O(n)$. Edge cases: p longer than t ; overlapping windows.

SOLUTION

Approach 1 — Rebuild the counts for every window ($O(nm)$)

brute force: recount all m characters at each start

JS

```

const anagramStarts_brute = (t, p) => {
  const out = [];
  const n = t.length, m = p.length;
  if (m === 0 || m > n) return out;

```

```

const need = new Array(26).fill(0);
for (const c of p) need[c.charCodeAt(0) - 97]++;
for (let i = 0; i + m <= n; i++) {
    const have = new Array(26).fill(0);
    for (let j = 0; j < m; j++) have[t.charCodeAt(i + j) - 97]++; // the waste
    if (need.every((v, k) => v === have[k])) out.push(i);
}
return out;
};

```

Time $O(nm)$ **Space** $O(1)$

Two neighbouring windows share $m-1$ characters, and this version recounts all of them.

Approach 3 — Slide the window, update two counters $O(n)$, optimal

fixed-size frequency window

JS

```

const anagramStarts = (t, p) => {
    const out = [];
    const n = t.length, m = p.length;
    if (m === 0 || m > n) return out;
    const need = new Array(26).fill(0), have = new Array(26).fill(0);
    for (const c of p) need[c.charCodeAt(0) - 97]++;
    const same = () => need.every((v, k) => v === have[k]);
    for (let i = 0; i < n; i++) {
        have[t.charCodeAt(i) - 97]++; // char entering
        if (i >= m) have[t.charCodeAt(i - m) - 97]--; // char leaving
        if (i >= m - 1 && same()) out.push(i - m + 1);
    }
    return out;
};

```

Trace of the window on "cbaebabacd" with $p = "abc"$ (window length 3):

i	window	counts a/b/c	match?
2	cba	1/1/1	yes → record start 0
3	bae	1/1/0	no
4	aeb	1/1/0	no
5	eba	1/1/0	no
6	bab	1/2/0	no
7	aba	2/1/0	no
8	bac	1/1/1	yes → record start 6
9	acd	1/0/1	no

Answer 0 6 from both versions. ("aaa", "aa") gives 0 1. ("abcd", "xy") gives nothing.

Time $O(26n)$ **Space** $O(1)$

The `need.every(...)` call is 26 comparisons, a constant. Track a 'matched letters' counter to make it a true $O(n)$ if asked.

SHORTCUT

To drop the 26-per-step factor, keep an integer `matched` counting how many of the 26 letters currently have `have[c] == need[c]`. Update it only for the two letters that change each step, and test `matched == 26`. Same idea as Example 24.

Answer: 0 6

Example 20 [Agoda · pattern]

Longest substring with no repeated character. "pwwkew". Constraints: $n \leq 10^5$, any ASCII. Target: $O(n)$. Edge cases: empty string; all characters the same; all different.

SOLUTION

Approach 1 — Start at every index, extend while unique ($O(n^2)$)

brute force

JS

```
const longestUnique_brute = (s) => {
  const n = s.length;
  let best = 0;
  for (let i = 0; i < n; i++) {
    const seen = new Set();
    for (let j = i; j < n; j++) {
      if (seen.has(s[j])) break;
      seen.add(s[j]);
      best = Math.max(best, j - i + 1);
    }
  }
  return best;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 3 — Sliding window with last-seen positions ($O(n)$, optimal)

THE INVARIANT

`s[left .. i]` always has no repeat. When `s[i]` was last seen at position `q` and `q >= left`, the only way to stay legal is to jump `left` to `q + 1`. `left` never moves backwards, so each index is visited once.

one pass, jump the left edge

JS

```
const longestUnique_window = (s) => {
  const last = new Map();           // character -> last index it was seen at
  let best = 0, left = 0;
  for (let i = 0; i < s.length; i++) {
    const c = s[i];
    if (last.has(c) && last.get(c) >= left) left = last.get(c) + 1;
    last.set(c, i);
    best = Math.max(best, i - left + 1);
  }
  return best;
};
```

Time $O(n)$ **Space** $O(|\text{alphabet}|)$

The Map holds one entry per distinct character seen — 256 for bytes, but it grows with the alphabet rather than being fixed.

"pwwkew" gives 3 from both (wke). "" gives 0, "bbbb" gives 1, "abcdef" gives 6. A randomised cross-check on 400 random strings matched every time.

TRAP

`last.get(c) >= left` — not merely `last.has(c)`. A character seen long ago, **before** `left`, is not inside the window and must not move the left edge. Tested: on "abba" the has-only version answers 1 instead of the correct 2.

A Map is the right container here, not a 256-slot array. The problem says “any ASCII”, but the same code then works unchanged for Unicode, and Map keeps the key’s real type instead of stringifying it the way a plain object would.

Answer: 3

Example 21 [Razer · pattern]

Group words that are anagrams of each other. {tea, eat, tan, ate, nat, bat}. Constraints: 10^4 words, each up to 100 characters. Edge cases: no anagrams at all; words of different lengths.

SOLUTION

the sorted word is the group key

JS

```
const groupAnagrams = (v) => {
  const g = new Map();
  for (const s of v) {
    const k = [...s].sort().join(''); // 'tea', 'eat', 'ate' all become 'aet'
    if (!g.has(k)) g.set(k, []);
    g.get(k).push(s);
  }
  return [...g.keys()].sort().map(k => g.get(k)); // sort the keys: reproducible output
};
```

Output: {bat} {tea eat ate} {tan nat}.

Time $O(NL \log L)$ **Space** $O(NL)$

N words of length L . Sorting each key costs $L \log L$.

SHORTCUT

A faster key when the alphabet is small: the 26 counts joined with separators, e.g. "1#0#0#...". That builds the key in $O(L + 26)$ instead of $O(L \log L)$. Worth mentioning; only worth coding if L is large.

TRAP

JavaScript has no ordered map. A Map iterates in **insertion** order and a plain object iterates in a rule-based order that puts integer-like keys first. Neither is key order, so neither is reproducible for a judge that compares output text. The fix is the last line: `[...g.keys()].sort()` before you read the groups out.

Use a Map, not a plain object, for the groups. A plain object turns every key into a string (so the number 1 and the text '1' collide), inherits keys like `constructor` from its prototype, and is slower for heavy adds and deletes. `new Map()` has none of those problems. `Object.create(null)` avoids the prototype issue only.

TRAP

`[...s].sort()` with **no** comparator is correct here, because these are single characters and the default sort compares them as text. That same bare `.sort()` on **numbers** is the single most common JavaScript bug there is: `[10, 9, 1].sort()` returns `[1, 10, 9]`, because `'10' < '9'` as strings. Numbers always need `(a, b) => a - b`. Know which case you are in every time you type `.sort()`.

Answer: {bat} {tea eat ate} {tan nat}

Example 22 [SCB · pattern]

Rank the words in a log by frequency. Ties go alphabetically. Case does not matter and punctuation is a separator. "the cat and the dog and THE bird", $k = 3$. Constraints: text up to 10^6 characters. Edge cases: fewer than k distinct words; text ending without punctuation.

SOLUTION

tokenize, count, sort

JS

```
const topWords = (text, k) => {
  const f = new Map();
  for (const w of text.toLowerCase().match(/[a-z]+/g) ?? [])
    f.set(w, (f.get(w) ?? 0) + 1);
  return [...f.entries()]
    .sort((a, b) => b[1] - a[1] // count descending
          || (a[0] < b[0] ? -1 : a[0] > b[0] ? 1 : 0)) // then
    .slice(0, k);
  alphabetical
};
```

Output: the:3 and:2 bird:1. THE and the merge because of `.toLowerCase()`. bird beats cat and dog (all count 1) because "bird" sorts first.

Time $O(n + D \log D)$ **Space** $O(D)$

D distinct words. JavaScript has no partial sort and no heap, so the whole list is sorted; with k tiny, a MinHeap of size k from the JS Toolkit appendix would make it $O(n + D \log k)$.

TRAP

`text.match(/[a-z]+/g)` returns `null`, **not** an empty array, when the text has no letters at all. `for (const w of null)` throws `TypeError: null is not iterable`. The `?? []` is what makes the empty input safe — tested: `topWords('', 3)` returns `[]`.

Matching with a regular expression also removes the C-style bug this problem is famous for: there is no "flush the last word after the loop" step to forget, because there is no loop.

Answer: the:3 and:2 bird:1

Example 23 [GIC · pattern]

Does a letter pattern describe a sentence? "abba" matches "grab bus bus grab" because a maps to grab and b maps to bus, and the mapping works both ways. Edge cases: two letters mapped to the same word; word count different from pattern length.

SOLUTION

two maps, because the mapping must be one-to-one

JS

```
const patternMatch = (pat, sentence) => {
  const w = sentence.split(' ').filter(x => x.length > 0);
  if (w.length !== pat.length) return false;

  const f = new Map(); // letter -> word
  const b = new Map(); // word -> letter
  for (let i = 0; i < w.length; i++) {
    const c = pat[i];
    if (f.has(c) && f.get(c) !== w[i]) return false;
    if (b.has(w[i]) && b.get(w[i]) !== c) return false;
    f.set(c, w[i]);
    b.set(w[i], c);
  }
  return true;
};
```

pattern	sentence	answer	why
abba	grab bus bus grab	true	a→grab, b→bus, consistent
abba	grab bus bus bus	false	a would need to map to both grab and bus
aaaa	grab bus bus grab	false	a cannot map to two words
ab	one two three	false	2 letters, 3 words

Time $O(n)$ **Space** $O(n)$

TRAP

With only the forward map, "ab" would match "dog dog" — both letters mapping to dog. The reverse map is what forbids that. Any “bijection” question needs both. Tested: with both maps, `patternMatch('ab', 'dog dog')` correctly returns false.

Answer: true false false false

The hard string algorithms

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Google · pattern]

Compute the Z-array: $z[i]$ is the length of the longest common prefix of s and $s[i..]$. Then use it to search for a pattern. $s = \text{"aabxaayaab"}$. Constraints: $n \leq 10^6$. Target: $O(n)$. Edge cases: empty string; every character the same.

Follow-up: “KMP already does search. Why does Z exist?”

SOLUTION

THE Z-BOX

Keep the interval $[l, r)$ that is the **rightmost** prefix-match found so far, meaning $s[l..r-1] == s[0..r-l-1]$.

For a new index i inside that box, the characters at i mirror the characters at $i - l$. So $z[i]$ starts at $\min(r - i, z[i - l])$ for free, and only the part past r needs real comparisons. Every real comparison pushes r forward, and r only moves right — so the total work is $O(n)$.

the Z-function

JS

```
const zFunction = (s) => {
  const n = s.length;
  const z = new Array(n).fill(0);
  if (n) z[0] = n;
  let l = 0, r = 0;
  for (let i = 1; i < n; i++) {
    if (i < r) z[i] = Math.min(r - i, z[i - l]); // reuse the mirror
    while (i + z[i] < n && s[z[i]] === s[i + z[i]]) z[i]++;
    if (i + z[i] > r) { l = i; r = i + z[i]; } // a new, further box
  }
  return z;
};
```

`zFunction("aabxaayaab")` gives:

i	0	1	2	3	4	5	6	7	8	9
s[i]	a	a	b	x	a	a	y	a	a	b
z[i]	10	1	0	0	2	1	0	3	1	0

`z[7] = 3` because `s[7..9] = "aab"` and `s[0..2] = "aab"`.

search with Z: glue pattern, separator, text

JS

```
const zSearch = (t, p) => {
  const hits = [];
  if (p.length === 0 || p.length > t.length) return hits;
  const j = p + '\x01' + t; // separator appears in neither string
  const z = zFunction(j);
  for (let i = p.length + 1; i < j.length; i++)
    if (z[i] >= p.length) hits.push(i - p.length - 1);
  return hits;
};
```

Time $O(n + m)$ **Space** $O(n + m)$

Uses more memory than KMP because it builds the joined string.

`zSearch("abababacaba", "ababaca")` gives 2. `zSearch("aaaa", "aa")` gives 0 1 2.

The follow-up, answered. Z and KMP solve search equally well, and Z is easier to get right — there is no confusing “do not move i ” branch. Z wins outright when the question is about **prefixes of the string itself**: “for each position, how far does the string match itself?” That includes counting borders, finding periods, and string-compression questions. KMP wins when memory matters, because it never copies the text.

TRAP

The separator must be a character that cannot occur in either string. If your input can contain any byte, prepend the **lengths** instead, or use two different sentinel handling. Using '#' on input that contains '#' gives silent wrong answers.

Answer: 10 1 0 0 2 1 0 3 1 0; search gives 2

Example 25 [Amazon · pattern]

What is the smallest number of characters you must add **in front** of s to make it a palindrome? "abcd" needs 3 (dcbabcd). "aacecaaa" needs 1. Constraints: $n \leq 10^6$. Target: $O(n)$. Edge cases: already a palindrome; one character; empty.

Follow-up: "now add characters at the end instead."

SOLUTION

Approach 1 — Try prefixes from longest to shortest $(O(n^2))$

brute force reference

JS

```
const minPrefixAdditions_brute = (s) => {
  const pal = (x) => {
    let i = 0, j = x.length - 1;
    while (i < j) { if (x[i] !== x[j]) return false; i++; j--; }
    return true;
  };
  for (let k = s.length; k >= 1; k--)
    if (pal(s.slice(0, k))) return s.length - k;
  return s.length;
};
```

Time $O(n^2)$ Space $O(n)$

Approach 3 — One prefix function on $s + \text{sep} + \text{reverse}(s)$ $(O(n), \text{optimal})$

THE CHAIN OF THREE IDEAS

1. You must keep the longest **palindromic prefix** of s . Everything after it gets mirrored in front. So the answer is $n - (\text{longest palindromic prefix})$.
2. A prefix $s[0..k-1]$ is a palindrome exactly when it equals the **last** k characters of $\text{reverse}(s)$.
3. "Longest prefix of X that is also a suffix of Y " is the last entry of the prefix function of $X + \text{"separator"} + Y$.

Put $X = s$ and $Y = \text{reverse}(s)$ and the answer falls out in one line.

the whole solution

JS

```
const minPrefixAdditions = (s) => {
  if (s.length === 0) return 0;
  const r = [...s].reverse().join('');
  const j = s + '\x01' + r;
  const lps = buildLPS(j);
  return s.length - lps[lps.length - 1]; // last entry = longest palindromic
  prefix
};
```

Time $O(n)$ Space $O(n)$

s	answer	brute	reason
---	--------	-------	--------

abcd	3	3	only a is a palindromic prefix, so $4 - 1 = 3$
aacecaaa	1	1	aacecaa is a palindrome, so $8 - 7 = 1$
aba	0	—	already a palindrome
z	0	—	—
" "	0	—	—

A randomised cross-check against the brute force on 400 random strings matched every time.

TRAP

Without the separator, $s = \text{"aaa"}$ and $r = \text{"aaa"}$ glue into "aaaaaa" , whose prefix function's last value is 5 — longer than s itself — and the answer goes negative. The separator caps `lps.back()` at n .

The follow-up. Adding at the **end** is the mirror question: keep the longest palindromic **suffix** and mirror the front. Same code with the roles swapped: build the prefix function of `reverse(s) + sep + s`.

Answer: 3 and 1

Example 26 [Microsoft · pattern]

Smallest window of t that contains every character of p , counting repeats. $t = \text{"adobecodebanc"}$, $p = \text{"abc"}$. Constraints: $n \leq 10^5$, any ASCII. Target: $O(n)$. Edge cases: no such window; p has duplicate letters; p longer than t .

Follow-up: “now t is a stream and you only get one pass.”

SOLUTION

Approach 1 — Check every substring ($O(n^3)$)

Approach 3 — Grow right, shrink left ($O(n)$, optimal)

TWO COUNTERS DO THE WORK

- `required` = how many **distinct** characters p needs.
- `formed` = how many of those characters currently have `have[c] >= need[c]`.

The window is valid exactly when `formed == required`. Grow **right** until valid, then shrink **left** as far as it stays valid, recording the best. Each pointer moves forward at most n times, so the whole thing is $O(n)$ even though there are two loops.

minimum window substring

JS

```
const minWindow = (t, p) => {
  if (p.length === 0 || t.length < p.length) return '';
  const need = new Map(), have = new Map();
  for (const c of p) need.set(c, (need.get(c) ?? 0) + 1);
  const required = need.size;

  let formed = 0, left = 0, bestLen = Infinity, bestL = 0;
  for (let right = 0; right < t.length; right++) {
    const c = t[right];
    have.set(c, (have.get(c) ?? 0) + 1);
    if (need.has(c) && have.get(c) === need.get(c)) formed++;
    while (formed === required) { // valid: try to shrink
      if (right - left + 1 < bestLen) {
        bestLen = right - left + 1;
      }
      const d = t[left];
      have.set(d, have.get(d) - 1);
      if (need.get(d) > 0) formed--;
      left++;
    }
  }
  return t.slice(bestL, bestL + bestLen);
}
```

```

    bestL = left;
  }
  const d = t[left++];
  have.set(d, have.get(d) - 1);
  if (need.has(d) && have.get(d) < need.get(d)) formed--;
}
}
return bestLen === Infinity ? '' : t.slice(bestL, bestL + bestLen);
};

```

Time $O(n)$ **Space** $O(|\text{alphabet}|)$

left and right each travel n steps total, so the nested while is still linear overall. The two Maps hold at most one entry per distinct character.

case	answer	note
adobecodebanc / abc	banc	matches the brute force
abc / xyz	" "	no window exists
aab / aab	aab	duplicate letters counted properly
a / aa	" "	p longer than t

A randomised cross-check against an $O(n^3)$ brute force on 300 cases matched every time.

TRAP

`have.get(c) === need.get(c)` uses `===`, not `>=`. With `>=`, every extra copy of a needed character increments `formed` again and the window is declared valid far too early.

TRAP

`have.get(c) ?? 0` is not optional. A missing key gives `undefined`, and `undefined + 1` is `NaN` — which then compares `false` against everything, so the window is never valid and the function quietly returns the empty string. Use `??` (nullish coalescing) rather than `||`, so that a genuine stored `0` is kept.

The follow-up. One pass is already what this does — `right` never goes backwards and `left` never goes backwards. The only obstacle for a true stream is the final `slice`, which needs the characters. Store just `bestL` and `bestLen` and emit the window from a ring buffer of the last `bestLen` characters, or report the indices and let the caller fetch them.

Python version

`collections.Counter` removes the two `?? 0` guards, which is the one place Python is genuinely shorter here.

minimum window substring in Python

PYTHON

```

def min_window(t, p):
    if not p or len(t) < len(p):
        return ""
    from collections import Counter
    need = Counter(p)
    have = Counter()
    required = len(need)
    formed = 0
    left = 0
    best = (float("inf"), 0)

```

```

for right, c in enumerate(t):
    have[c] += 1
    if c in need and have[c] == need[c]:
        formed += 1
    while formed == required:
        if right - left + 1 < best[0]:
            best = (right - left + 1, left)
        d = t[left]
        left += 1
        have[d] -= 1
        if d in need and have[d] < need[d]:
            formed -= 1
    if best[0] == float("inf"):
        return ""
    return t[best[1]:best[1] + best[0]]

```

Run on the four cases above, Python prints 'banc', '', 'aab', '' — the same answers as the JavaScript. Cross-checked against an $O(n^3)$ brute force on 300 random cases.

Answer: banc

Example 27 [D. E. Shaw · pattern]

Count the distinct substrings of a string. "abab" has 7. Constraints: $n \leq 3000$ for the hashing answer. Edge cases: all characters equal; empty string.

Follow-up: “now $n = 10^6$.”

SOLUTION

Approach 1 — Put every substring in a set ($O(n^3)$ time and $O(n^3)$ memory)

the obvious version, used as the reference

JS

```

const distinctSubstrings_set = (s) => {
    const seen = new Set();
    for (let i = 0; i < s.length; i++)
        for (let len = 1; i + len <= s.length; len++)
            seen.add(s.slice(i, i + len));
    return seen.size;
};

```

Time $O(n^3)$ **Space** $O(n^3)$

Every slice copies characters, and the Set stores them all. $n = 3000$ would need gigabytes.

Approach 3 — Group by length, hash each window ($O(n^2)$ time, $O(n)$ memory)

prefix hashes, one set per length

JS

```

const distinctSubstrings_hash = (s) => {
    const n = s.length;
    const sub = buildHash(s); // the prefix-hash helper from the pattern
    page
    let total = 0;
    for (let len = 1; len <= n; len++) {
        const seen = new Set();
        for (let i = 0; i + len <= n; i++) seen.add(sub(i, len));
    }
    return total;
};

```

```

    total += seen.size;
  }
  return total;
};

```

Time $O(n^2)$ **Space** $O(n)$

n^2 hash inserts, each $O(1)$. The set is cleared every length, so memory stays $O(n)$.

"abab" gives 7 from both: a, b, ab, ba, aba, bab, abab. "aaa" gives 3 (a, aa, aaa). The empty string gives 0. "q" gives 1. A randomised cross-check on 200 random strings matched every time.

The follow-up. At $n = 10^6$ even $O(n^2)$ is dead. The real answer is a suffix automaton or a suffix array with LCP:

$$\text{distinct} = \frac{n(n+1)}{2} - \sum_i \text{lcp}[i]$$

The suffix array counts every substring once by subtracting the overlaps that consecutive suffixes share. That is $O(n \log n)$ and is the expected answer at this size — worth **naming** even if you do not code it in 40 minutes.

Answer: 7

Example 28 [Goldman Sachs · pattern]

Longest substring that appears at least twice (the copies may overlap). "banana" gives ana. Constraints: $n \leq 10^5$. Target: $O(n \log n)$. Edge cases: no repeat at all; single character.

Follow-up: "what if the hash collides?"

SOLUTION

WHY BINARY SEARCH WORKS HERE

Let $f(L)$ = "does some substring of length L appear twice?".

If a substring of length L appears twice, then its first $L - 1$ characters also appear twice. So f is **monotone**: true for all small L , false for all large L . Anything monotone can be binary-searched.

For a fixed L , testing $f(L)$ is one pass: hash every window of length L and look for a repeat.

binary search on the length, hashing inside

JS

```

const longestRepeated = (s) => {
  const n = s.length;
  if (n < 2) return '';
  const sub = buildHash(s);
  const findLen = (len) => {                                     // a start index, or -1
    const seen = new Map();
    for (let i = 0; i + len <= n; i++) {
      const key = sub(i, len);
      if (seen.has(key)) {
        const at = seen.get(key);
        if (s.slice(at, at + len) === s.slice(i, i + len)) return at; // verified
      } else seen.set(key, i);
    }
    return -1;
  };
  let lo = 1, hi = n - 1, bestPos = -1, bestLen = 0;
  while (lo <= hi) {
    const mid = Math.floor((lo + (hi - lo) / 2));

```

```

const pos = findLen(mid);
if (pos >= 0) { bestLen = mid; bestPos = pos; lo = mid + 1; }
else hi = mid - 1;
}
return bestLen ? s.slice(bestPos, bestPos + bestLen) : '';
};

```

Time $O(n \log n)$ expected **Space** $O(n)$
log n rounds, each $O(n)$ hash lookups.

"banana" gives ana, matching the $O(n^3)$ brute force. "abcd" gives the empty string. "a" gives the empty string. A randomised cross-check on 300 random strings matched the brute force length every time.

The follow-up, answered. Three defences, in order of how much an interviewer likes them:

1. **Verify.** The code above compares the actual characters after a hash hit (`s.slice(at, at + len) === s.slice(i, i + len)`). A collision then costs time, never correctness.
2. **Big modulus.** $2^{61} - 1$ is prime; with 10^5 windows the birthday-collision chance is about $(10^5)^2 / 2^{62} \approx 2 \times 10^{-9}$.
3. **Random base.** Pick B at random at run time so nobody can pre-compute a colliding test case. Fixed bases like 31 and 131 are exactly what anti-hash tests target.

Answer: ana

Example 29 [Adobe · pattern]

Longest palindromic substring in $O(n)$. "babadada". Constraints: $n \leq 10^6$. Edge cases: even-length answer; empty string.

Follow-up: "what does the algorithm actually compute, beyond the single longest one?"

SOLUTION

Approach 1 — Expand around every centre ($O(n^2)$ — the answer from Example 14)

Correct and short, and it is what you should write first. At $n = 10^6$ a worst case like "aaaa...a" makes every centre expand $\frac{n}{2}$ steps, giving 5×10^{11} character comparisons. That is the only reason to go further.

Approach 3 — Manacher ($O(n)$, optimal)

TWO TRICKS MAKE MANACHER WORK

Trick 1 — kill the odd/even split. Insert # between every pair of characters and at both ends. "cbbd" becomes "#c#b#b#d#". Now **every** palindrome has odd length and a single character centre. Add guards ^ and \$ at the two ends so the expansion loop never needs a bounds check.

Trick 2 — reuse the mirror. Keep the palindrome with the furthest right edge, centre c and right edge r. For a new index i inside it, the mirror index is $2c - i$, and `p[i]` starts at $\min(r - i, p[2c - i])$ for free. Only the part past r is really compared, and r only moves right — so the total work is $O(n)$.

Manacher's algorithm

JS

```

const manacher = (s) => {
  if (s.length === 0) return '';
  let t = '^';
  for (const c of s) t += '#' + c;
  t += '$';
  const n = t.length;
  const p = new Array(n).fill(0);
  // guards: no bounds checks needed

```

```

let c = 0, r = 0;
for (let i = 1; i < n - 1; i++) {
  if (i < r) p[i] = Math.min(r - i, p[2 * c - i]);
  while (t[i + p[i] + 1] === t[i - p[i] - 1]) p[i]++;
  if (i + p[i] > r) { c = i; r = i + p[i]; }
}
let bestLen = 0, bestC = 0;
for (let i = 1; i < n - 1; i++)
  if (p[i] > bestLen) { bestLen = p[i]; bestC = i; }
const start = (bestC - bestLen) / 2;      // map back to the original s
return s.slice(start, start + bestLen);
};

```

Time $O(n)$ **Space** $O(n)$

The transformed string is $2n+3$ characters; the p array is the same size.

"babadada" gives adada, same as the $O(n^2)$ expand version. "cbbd" gives bb, "x" gives x, "" gives "". A randomised cross-check against the expand version on 400 random strings matched the length every time.

READING THE P ARRAY

In the transformed string, $p[i]$ is **exactly the length of the palindrome centred at i in the original string**. That is why the mapping back is $\text{start} = (\text{bestC} - \text{bestLen}) / 2$ and the length is bestLen with no adjustment.

The follow-up. p holds the radius at **every** centre, so with one $O(n)$ run you can also answer:

- how many palindromic substrings are there? $\sum_i [p[i]/2]$, though it is easier to count directly on the transformed array,
- is $s[l..r]$ a palindrome? check $p[l + r + 2] \geq r - l + 1$ on the transformed indices,
- the longest palindrome **starting** or **ending** at each position.

In an interview: code the $O(n^2)$ expand version first, get it correct, then say “Manacher makes this $O(n)$ by reusing mirror information, same idea as the Z-function.” Most interviewers stop you there.

Answer: adada

Dry run: KMP on aabaabaaa / aabaa

Step 1 — build the LPS table

Pattern $p = \text{"aabaa"}$. Trace exactly as the instrumented program printed it:

buildLPS trace

TEXT

i	len	p[i]	p[len]	action	lps
1	0	a	a	match: lps[1]=1	[01000]
2	1	b	a	mismatch: len=lps[0]=0	[01000]
2	0	b	a	mismatch, len=0: lps[2]=0	[01000]
3	0	a	a	match: lps[3]=1	[01010]
4	1	a	a	match: lps[4]=2	[01012]

lps = 0 1 0 1 2

Look at the two rows with $i = 2$. The first one does not advance i — it only shrinks len from 1 to 0. That is the fall-back branch. The second row, now with $\text{len} = 0$, writes $\text{lps}[2] = 0$ and finally moves on.

Step 2 — check the table by hand

i	p[0..i]	lps[i]	the border
0	a	0	a single character has no proper border
1	aa	1	a is both the prefix and the suffix
2	aab	0	starts with a, ends with b
3	aaba	1	a
4	aabaa	2	aa

Step 3 — the search, every single step

Text $t = \text{"aabaabaaa"}\text{"}$, $n = 9$, $m = 5$.

i	j	t[i]	p[j]	action
0	0	a	a	equal: advance both
1	1	a	a	equal: advance both
2	2	b	b	equal: advance both
3	3	a	a	equal: advance both
4	4	a	a	equal $\rightarrow$ j hits 5 = m. Match at 0. $j = \text{lps}[4] = 2$
5	2	b	b	equal: advance both
6	3	a	a	equal: advance both
7	4	a	a	equal $\rightarrow$ Match at 3. $j = \text{lps}[4] = 2$
8	2	a	b	not equal, $j > 0$: $j = \text{lps}[1] = 1$, i does not move
8	1	a	a	equal: advance both. i reaches 9, loop ends

Answer: matches start at 0 and 3.

Step 4 — the three things to notice

WHAT THE TRACE PROVES

1. **i never goes backwards.** Look at the i column: 0,1,2,3,4,5,6,7,8,8. It only repeats once (the fall-back row) and never decreases. The naive search would have gone back to $i = 1$ after the first match.
2. **Matches can overlap.** After the hit at 0, j drops to $\text{lps}[4] = 2$ instead of 0, meaning “we already have aa matched”. That is why the hit at 3 is found — positions 3 and 0 overlap by 2 characters.
3. **The fall-back is cheap.** Row $i = 8$ fires it once. Over the whole run, the number of fall-backs is bounded by how much j grew, which is bounded by n .

Step 5 — edge cases, checked

text	pattern	hits	note
aaaa	aa	0 1 2	overlaps kept
abc	xyz	none	–
abc	" "	none	the guard rejects an empty pattern
ab	abc	none	pattern longer than text

Practice

PRACTICE · WARM-UP target: 20 min for P1--P3

- P1.** Count vowels and consonants in "Hello World 42". Digits and spaces count as neither.
- P2.** Two strings of equal length: do they differ in **exactly one** position? Test ("cat", "cot"), ("cat", "cat"), ("cat", "dog"), ("cat", "cats").
- P3.** Capitalise the first letter of every word and lowercase the rest. Test "the QUICK brown f0x" and the empty string.

PRACTICE · TIER 1 target: 45 min for P4--P7

- P4.** Remove duplicate characters, keeping the first occurrence of each. Test "programming".
- P5.** Is a a subsequence of b (same order, gaps allowed)? Test ("ace", "abcde"), ("aec", "abcde"), ("", "abc"), ("abc", "").
- P6.** Count all palindromic substrings (different positions count separately). Test "aaa" and "abc". Target $O(n^2)$.
- P7.** In a sentence, find the longest word whose letters are all different. Test "code the quick brown fox".

PRACTICE · TIER 2 target: 55 min for P8--P11

- P8.** Implement strStr with KMP: return the first index of p in t, or -1. Test ("aabaabaaa", "aabaa"), ("abc", "d"), ("abc", ""), ("a", "aa").
- P9.** Does t contain any permutation of p as a contiguous block? Test ("eidbaooo", "ab") and ("eidboaoo", "ab").
- P10.** Longest substring with at most k distinct characters. Test ("eceba", 2). Also handle $k = 0$ and $k >$ alphabet size.
- P11.** Decode a run-length encoded string: "a3b4c1d1" becomes "aaabbbbcd". Handle counts with more than one digit, e.g. "a12".

PRACTICE · TIER 3 target: 70 min for P12--P15

- P12.** Is s made by repeating one shorter substring? Test "abab" (yes), "aba" (no), "aaaa" (yes), "a" (no). Target $O(n)$.
- P13.** Longest common substring of two strings, in $O((n + m) \log)$. Test ("abcdefg", "xxcdefyy").
- P14.** Count substrings with **exactly** k distinct characters. Test ("pqqqs", 2). Target $O(n)$.
- P15.** Lexicographically smallest rotation of a string. Test "bcabd".

ANSWER KEY

P1 — 3 vowels, 7 consonants.

```
const vowelsConsonants = (s) => {
  let a = 0, b = 0;
  for (const c of s.toLowerCase()) {
    if (!/[a-z]/.test(c)) continue; // skip digits and spaces
    if ('aeiou'.includes(c)) a++; else b++;
  }
}
```

```
    return [a, b];
};
```

"Hello World 42": vowels e, o, o; consonants H, l, l, W, r, l, d.

Time $O(n)$ **Space** $O(1)$

P2—true false false false.

```
const differByOne = (a, b) => {
  if (a.length !== b.length) return false;
  let d = 0;
  for (let i = 0; i < a.length; i++) if (a[i] !== b[i]) d++;
  return d === 1;
};
```

Identical strings give $d = 0$, so they are **not** “differ by one”. (“cat”, “cats”) is rejected by the length check. **Speed-up:** if ($d > 1$) return false; inside the loop lets long strings exit early.

P3—The Quick Brown Fox.

```
const titleCase = (s) => {
  const out = [...s]; // strings are immutable: edit an array
  let start = true;
  for (let i = 0; i < out.length; i++) {
    if (out[i] === ' ') { start = true; continue; }
    out[i] = start ? out[i].toUpperCase() : out[i].toLowerCase();
    start = false;
  }
  return out.join('');
};
```

The **start** flag is the whole solution: set it at every space, clear it after writing a letter. The empty string returns the empty string.

P4—progamin.

```
const dedup = (s) => {
  const seen = new Set();
  const out = [];
  for (const c of s) if (!seen.has(c)) { seen.add(c); out.push(c); }
  return out.join('');
};
```

“programming” loses the second r, the second m, and the second g. A **Set** handles any character, not just lowercase letters, and it costs only as much as the number of distinct characters actually seen.

Time $O(n)$ **Space** $O(1)$

P5—true false true false.

```
const isSubsequence = (a, b) => {
  let i = 0;
  for (let j = 0; j < b.length && i < a.length; j++)
    if (a[i] === b[j]) i++;
  return i === a.length;
};
```

Greedy matching is safe: taking the **earliest** possible match for each character of **a** never blocks a later one. (“”, “abc”) is true — the empty string is a subsequence of everything. (“abc”, “”) is false.

Time $O(n + m)$ **Space** $O(1)$

P6—aaa gives 6, abc gives 3.

```
const countPalSubstrings = (s) => {
  const n = s.length;
```

```

let c = 0;
const grow = (l, r) => {
  while (l >= 0 && r < n && s[l] === s[r]) { c++; l--; r++; }
};
for (let i = 0; i < n; i++) { grow(i, i); grow(i, i + 1); }
return c;
};

```

The counter sits **inside** the while loop: every successful expansion is one more palindrome. "aaa" has a, a, a, aa, aa, aaa = 6. A randomised cross-check against an $O(n^3)$ brute force on 300 strings matched every time.

Time $O(n^2)$ **Space** $O(1)$

P7 — quick.

```

const longestDistinctWord = (sentence) => {
  let best = '';
  for (const w of sentence.split(' ').filter(x => x.length > 0)) {
    if (new Set(w).size !== w.length) continue; // a repeated letter: skip the
word
    if (w.length > best.length) best = w; // > not >=, so a tie keeps the
first
  }
  return best;
};

```

code(4), the(3), quick(5), brown(5), fox(3) all have distinct letters. quick and brown tie at 5, and > (not >=) keeps the first one.

P8 — 0, -1, 0, -1.

```

const strStrKMP = (t, p) => {
  if (p.length === 0) return 0;
  if (p.length > t.length) return -1;
  const lps = buildLPS(p);
  let i = 0, j = 0;
  while (i < t.length) {
    if (t[i] === p[j]) { i++; j++; if (j === p.length) return i - j; }
    else if (j > 0) j = lps[j - 1];
    else i++;
  }
  return -1;
};

```

Same as Example 17 but returning on the first hit instead of collecting all of them.

Time $O(n + m)$ **Space** $O(m)$

P9 — true then false.

```

const containsPermutation = (t, p) => {
  const n = t.length, m = p.length;
  if (m === 0) return true;
  if (m > n) return false;
  const need = new Array(26).fill(0), have = new Array(26).fill(0);
  for (const c of p) need[c.charCodeAt(0) - 97]++;
  const same = () => need.every((v, k) => v === have[k]);
  for (let i = 0; i < n; i++) {
    have[t.charCodeAt(i) - 97]++;
    if (i >= m) have[t.charCodeAt(i - m) - 97]--;
    if (i >= m - 1 && same()) return true;
  }
}

```

```

    return false;
};

```

"eidbaooo" contains **ba** at index 3. "eidboao" has **b** and **a** separated by **o**, so no window of length 2 holds both.

Time $O(26n)$ **Space** $O(1)$
 need.every(...) is 26 comparisons per step, a constant factor.

P10 — 3.

```

const atMostKDistinct = (s, k) => {
  if (k <= 0) return 0;
  const cnt = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    cnt.set(c, (cnt.get(c) ?? 0) + 1);
    while (cnt.size > k) {
      const d = s[left];
      cnt.set(d, cnt.get(d) - 1);
      if (cnt.get(d) === 0) cnt.delete(d);    // delete, or size never drops
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
};

```

"eceba" with $k = 2$: the window **ece** has 2 distinct letters and length 3. $k = 0$ gives 0; k larger than the alphabet gives the whole string. **The delete is the whole trick.** `cnt.size` is the number of distinct characters in the window, so a count that reaches zero must be removed from the `Map` or the size never drops and the window never shrinks. Setting the value to `0` is not enough.

Time $O(n)$ **Space** $O(1)$

P11 — aaabbbbcd.

```

const rleDecode = (s) => {
  const out = [];
  let i = 0;
  while (i < s.length) {
    const c = s[i++];
    let n = 0;
    while (i < s.length && s[i] >= '0' && s[i] <= '9')
      n = n * 10 + (s.charCodeAt(i++) - 48);    // multi-digit counts
    if (n === 0) n = 1;                          // a bare letter means one copy
    out.push(c.repeat(n));
  }
  return out.join('');
};

```

"a3b4c1d1" gives **aaabbbbcd**. "x" gives **x**. "a12" gives twelve **as** — the inner loop reads **both** digits, which a single `s.charCodeAt(i) - 48` would get wrong. `String.prototype.repeat` does the expanding, so no inner append loop is needed.

Time $O(\text{output length})$ **Space** $O(\text{output length})$

P12 — abab yes, aba no, aaaa yes, a no.

```

const isRepeated = (s) => {
  const n = s.length;
  if (n < 2) return false;

```

```

const lps = buildLPS(s);
const b = lps[n - 1]; // longest proper border
return b > 0 && n % (n - b) === 0;
};

```

Why this works. If the longest border has length b , the string has period $n - b$. The string is a whole number of copies of that period exactly when n divides evenly by $n - b$. "abab": lps = 0 0 1 2, so $b = 2$, period = 2, and $4\%2 = 0 \Rightarrow$ yes. "aba": $b = 1$, period = 2, and $3\%2 = 1 \Rightarrow$ no. A randomised cross-check against an $O(n^2)$ brute force on 400 strings matched every time.

Time $O(n)$ **Space** $O(n)$

P13 — cdef. Binary-search the length; for each length, hash all windows of **b** into a set and look up every window of **a**.

```

const longestCommonSubstring = (a, b) => {
  const ha = buildHash(a), hb = buildHash(b);
  const tryLen = (len) => { // a start index in a, or -1
    if (len === 0) return 0;
    const s = new Set();
    for (let i = 0; i + len <= b.length; i++) s.add(hb(i, len));
    for (let i = 0; i + len <= a.length; i++)
      if (s.has(ha(i, len))) return i;
    return -1;
  };
  let lo = 1, hi = Math.min(a.length, b.length), bestPos = -1, bestLen = 0;
  while (lo <= hi) {
    const mid = Math.floor(lo + (hi - lo) / 2);
    const pos = tryLen(mid);
    if (pos >= 0) { bestLen = mid; bestPos = pos; lo = mid + 1; }
    else hi = mid - 1;
  }
  return bestLen ? a.slice(bestPos, bestPos + bestLen) : '';
};

```

ha and hb are the prefix-hash structures from the pattern page. Monotone because any common substring of length L contains a common substring of length $L - 1$. A randomised cross-check on 300 random pairs matched the brute-force length every time.

Time $O((n + m) \log \min(n, m))$ **Space** $O(n + m)$

P14 — 7. The trick: **exactly k = at most k — at most k-1.**

```

const atMost = (s, k) => {
  if (k <= 0) return 0;
  const cnt = new Map();
  let left = 0, total = 0;
  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    cnt.set(c, (cnt.get(c) ?? 0) + 1);
    while (cnt.size > k) {
      const d = s[left];
      cnt.set(d, cnt.get(d) - 1);
      if (cnt.get(d) === 0) cnt.delete(d);
      left++;
    }
    total += right - left + 1; // all windows ending at right are valid
  }
  return total;
};

const exactlyK = (s, k) => atMost(s, k) - atMost(s, k - 1);

```

Why $\text{total} += \text{right} - \text{left} + 1$: once the window `[left, right]` is valid, every sub-window ending at `right` is also valid, and there are `right - left + 1` of them. "ppqqs" with $k = 2$ gives 7; a brute force agreed, and a randomised check on 300 cases matched every time. "aaa" with $k = 1$ gives 6. k bigger than the alphabet gives 0.

Time $O(n)$ **Space** $O(1)$

P15 — abdbc.

```
const smallestRotation = (s) => {
  if (s.length === 0) return s;
  const d = s + s;
  const n = s.length;
  let best = 0;
  for (let i = 1; i < n; i++)
    if (d.slice(i, i + n) < d.slice(best, best + n)) best = i;
  return d.slice(best, best + n);
};
```

Rotations of bcabd: bcabd, cabdb, abdbc, bdbca, dbcab. The smallest is abdbc. "aaa" gives aaa, "" gives "", "z" gives z.

Time $O(n^2)$ **Space** $O(n)$

Booth's algorithm does it in $O(n)$. Name it; code this one.

ONE-PAGE REVISION CARD

The four engines, as code

memorise these shapes

JS

```
// 1. two pointers
let i = 0, j = s.length - 1;
while (i < j) { if (s[i] !== s[j]) return false; i++; j--; }

// 2. fixed frequency window of length m
for (let i = 0; i < n; i++) {
  have[t.charCodeAt(i) - 97]++;
  if (i >= m) have[t.charCodeAt(i - m) - 97]--;
  if (i >= m - 1 && need.every((v, k) => v === have[k])) { /* hit at i-m+1 */ }
}

// 3. prefix function
for (let i = 1; i < m; ) {
  if (p[i] === p[len]) lps[i++] = ++len;
  else if (len > 0) len = lps[len - 1];
  else lps[i++] = 0;
}

// 4. rolling hash prefix, on BigInt
h[i + 1] = (h[i] * B + BigInt(s.charCodeAt(i))) % M;
const sub = (l, len) => (h[l + len] - h[l] * pw[len] % M + M) % M;
```

Which algorithm for which question

Question says	Use	Time	Space
palindrome check	two pointers	$O(n)$	$O(1)$
longest palindrome	expand centres	$O(n^2)$	$O(1)$
longest palindrome, huge n	Manacher	$O(n)$	$O(n)$
find a pattern	KMP	$O(n + m)$	$O(m)$
find many patterns	Rabin-Karp / Aho-Corasick	$O(n + m)$	$O(1)$
prefix-of-self questions	Z-function	$O(n)$	$O(n)$
shortest palindrome by prefix	LPS of $s + \text{sep} + \text{rev}(s)$	$O(n)$	$O(n)$
is it a repeated block	LPS border + period	$O(n)$	$O(n)$
anagram windows	fixed frequency window	$O(n)$	$O(1)$
at most / exactly k distinct	variable window	$O(n)$	$O(1)$
smallest window containing	grow right, shrink left	$O(n)$	$O(1)$
distinct substrings	hash per length / suffix array	$O(n^2)$	$O(n)$
longest repeated / common	binary search + hash	$O(n \log n)$	$O(n)$
two strings, edit-like	DP (Chapter 18)	$O(nm)$	$O(nm)$

The key formulas

WORTH WRITING ON THE BOARD

- Prefix function: $\text{lps}[i] = \text{longest proper border of } p[0..i]$.
- **Period** of a string: $n - \text{lps}[n - 1]$. It tiles the string exactly when $n \bmod (n - \text{lps}[n - 1]) = 0$.
- **Longest palindromic prefix** of $s = \text{lps.back}()$ of $s + \text{sep} + \text{reverse}(s)$.
- **Polynomial hash**: $H = \sum_k s_k B^{m-1-k} \bmod M$; slide with $H' = (H - s_{\text{left}} B^{m-1})B + s_{\text{right}}$.
- **Exactly k distinct = at most k — at most k-1.**
- Number of palindrome centres in a string of length n : $2n - 1$.

Top traps

1. `if (s.indexOf(x))` is wrong at both ends: `-1` is truthy, and `index 0` is falsy. Write `s.indexOf(x) !== -1`, or `s.includes(x)`.
2. Strings are immutable. `s[0] = 'x'` throws in strict mode and does nothing otherwise. Work on `[...s]` and `join('')` at the end.
3. `.split('')` keeps the empty pieces. Always `.filter(w => w.length > 0)`.
4. `str.match(/re/g)` returns `null`, not `[]`, when nothing matches. Guard with `?? []`.
5. In `buildLPS`, advancing `i` in the fall-back branch. It must not move.
6. Expand-around-centre: the length is `r - l - 1`, not `r - l + 1`.
7. Forgetting the last token after a split loop (no trailing separator).
8. Rabin-Karp without character verification after a hash match.
9. A separator character in `s + sep + t` that can actually appear in the input.
10. `formed++` guarded with `>=` instead of `==` in the minimum-window pattern.
11. `Map` iterates in insertion order and a plain object puts integer-like keys first. Neither is key order — sort the keys yourself when the output must be reproducible.
12. `map.get(k) + 1` on a missing key is `NaN`. Write `(map.get(k) ?? 0) + 1`.
13. Two-pointer skip loops without the `i < j` guard read past the ends.
14. A bare `.sort()` is lexicographic. Correct for single characters, catastrophic for numbers: `[10, 9, 1].sort()` gives `[1, 10, 9]`.
15. Modular multiplication in plain numbers is wrong above $2^{53} - 1$, which two residues mod $10^9 + 7$ reach immediately. Every rolling hash must be `BigInt`.
16. Counting substrings: there are $\frac{n(n+1)}{2}$ of them, which stays exact as a JavaScript number until $n \approx 1.3 \times 10^8$ — comfortably beyond any input, so no `BigInt` is needed for the **count** itself.

Complexity facts to quote

- A string of length n has $\frac{n(n+1)}{2}$ substrings but at most $\frac{n(n+1)}{2}$ **distinct** ones, and a suffix automaton stores them all in $O(n)$ states.
- KMP, Z and Manacher are all the same idea: **never re-compare what a previous match already proved**. Each keeps one “rightmost known region” and reuses it.
- Rabin-Karp is the only one of the four that generalises to 2-D patterns and to searching many patterns at once.

Hashing & Frequency Maps

Trade memory for time: turn a search into a lookup

Pattern in one page

THE ONE IDEA

A hash table answers “**have I seen this before?**” in about one step.

Everything in this chapter is the same move: you are about to write a loop inside a loop, so you stop, ask **what is the inner loop searching for?**, and store that thing in a hash table instead. The $O(n^2)$ becomes $O(n)$.

The three containers JavaScript gives you

Container	Holds	Lookup	Order of keys
Set	keys only	$O(1)$ average	insertion order
Map	key $\rightarrow$ value	$O(1)$ average	insertion order
plain object <code>{}</code>	string key $\rightarrow$ value	$O(1)$ average	insertion order, integer-like keys first

Use Map and Set by default. A plain object turns every key into a string, so 1 and "1" become the same entry and an object key becomes "[object Object]". Map keeps the type of the key.

There is no sorted map. JavaScript has nothing like an ordered dictionary, so when the question wants keys in increasing order you collect the keys and sort them at the end: `[...freq.keys()].sort((a, b) => a - b)`. That costs one extra $O(d \log d)$ pass over the d distinct keys and is usually far cheaper than it sounds.

The four templates

The four things you will write over and over

JS

```
// 1. HAVE I SEEN IT?
const seen = new Set();
for (const x of a) {
  if (seen.has(x)) { /* duplicate */ }
  seen.add(x);
}

// 2. HOW MANY TIMES?
const freq = new Map();
for (const x of a) freq.set(x, (freq.get(x) ?? 0) + 1); // missing key -> 0, then +1

// 3. WHERE DID I SEE IT?
const pos = new Map();
a.forEach((x, i) => pos.set(x, i)); // keeps the LAST index
// use if (!pos.has(x)) for the FIRST

// 4. PREFIX SUM + MAP (the big one: subarray questions)
const seenPrefix = new Map([[0, 1]]); // the empty prefix, before index 0
let pre = 0;
for (const x of a) {
```

```

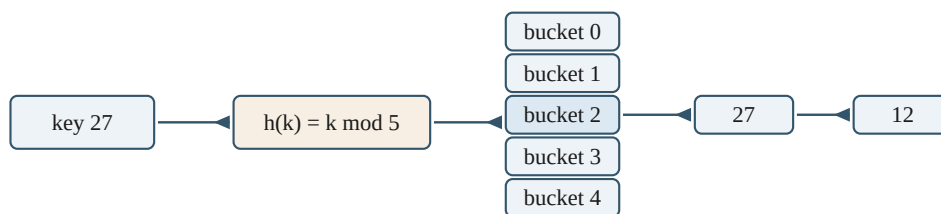
pre += x;
// pre - target was a prefix ending somewhere earlier,
// so the piece between them sums to target
seenPrefix.set(pre, (seenPrefix.get(pre) ?? 0) + 1);
}

```

The invariant

At the moment the loop is about to look at index i , the table already describes **everything strictly before i** — and nothing after it. Write that sentence down for your problem before you write code. Every bug in this chapter is a broken version of it.

What a hash table actually is



A key is turned into a bucket number, then the bucket is searched.

The bucket number is computed, not searched for. That is why lookup costs one step **on average**. If many keys land in the same bucket, that bucket becomes a list and lookup costs $O(n)$. Tier 3 shows an attack that does exactly this, and the fix.

TRAP

Map lookup is $O(1)$ **on average**, not always. In a written round, say “ $O(n)$ expected, $O(n^2)$ worst case for a hash map”. Interviewers listen for this.

SHORTCUT

A missing key in a Map reads back as undefined, not 0. So `freq.get(x) + 1` on a first sighting gives NaN. Always write `freq.set(x, (freq.get(x) ?? 0) + 1)`. The `??` is the whole fix, and forgetting it is the most common counting bug in JavaScript.

SHORTCUT

Reading a Map never creates an entry, so `if (freq.get(x) === 3)` is safe on a missing key — unlike C++’s `operator[]`, which silently inserts a zero. This is one place where JavaScript is the friendlier language.

When to reach for it

The question sounds like	Reach for
duplicate / seen before / distinct	<code>new Set()</code>
count, frequency, most common, anagram	<code>new Map()</code> or a 26-slot array
pair with sum / difference	Set of values seen so far
subarray with sum k , count of subarrays	prefix sum + Map of counts

longest subarray with property P	prefix value + Map of the first index
group things that are “the same shape”	build a canonical string key, then Map<key, array>
$O(1)$ insert / delete / random	an array plus a Map of positions

Warm-up

WARM-UP

build the reflex

Example 1

Print how many times each value appears, with the values in increasing order. Array length up to 10^5 ; values fit in int. Target $O(n \log n)$. Edge cases: empty array, all values equal.

SOLUTION

JavaScript has no sorted map, so the “increasing order” part is handled at the end: count into a Map, then sort the keys once. The counting stays $O(n)$; the sort of the d distinct keys is the $O(d \log d)$ part, and $d \leq n$.

Frequency in sorted key order

JS

```
const printFrequencies = (a) => {
  const freq = new Map();
  for (const x of a) freq.set(x, (freq.get(x) ?? 0) + 1);
  // JS has no ordered map, so pull the keys out and sort them yourself
  const keys = [...freq.keys()].sort((p, q) => p - q); // never a bare .sort()
  for (const k of keys) console.log(`${k} -> ${freq.get(k)}`);
};
```

Running it on {4, 1, 4, 7, 1, 4} prints:

output

TEXT

```
1 -> 2
4 -> 3
7 -> 1
```

On an empty array it prints nothing. On {-2, -2, 0, 5} it prints -2 -> 2, 0 -> 1, 5 -> 1. Negative keys are no problem.

Time $O(n \log n)$ **Space** $O(d)$, d = number of distinct values

Drop the final sort and it becomes $O(n)$, but the keys come out in insertion order.

Example 2

Does the array contain any value twice? Length up to 10^5 . Target $O(n)$. Edge cases: empty array, one element.

SOLUTION

Walk once. Before inserting, ask whether the value is already there.

Duplicate check

JS

```
const hasDuplicate = (a) => {
  const seen = new Set();
  for (const x of a) {
```

```

    if (seen.has(x)) return true;    // x arrived a second time
    seen.add(x);
  }
  return false;
};

```

Tested: {3,9,2,9} gives 1, {3,9,2,8} gives 0, {} gives 0, {5} gives 0, {-1,-1} gives 1.

Time O(n) expected **Space** O(n)

Example 3

Return the value whose **second** occurrence comes earliest. Return -1 if no value repeats. Edge cases: empty array, no repeats.

SOLUTION

This is the same loop as Example 2. The first time the count test fires, that value is the answer — because we are walking left to right, no other value can have finished its pair sooner.

First value to repeat

JS

```

const firstRepeating = (a) => {
  const seen = new Set();
  for (const x of a) {
    if (seen.has(x)) return x;
    seen.add(x);
  }
  return -1;
};

```

Tested: {5,2,7,2,5} gives 2 (not 5 — the **second** 2 arrives at index 3, the second 5 only at index 4). {1,2,3} gives -1. {} gives -1. {8,8} gives 8.

TRAP

“First repeating” has two meanings. (a) The value whose second copy comes first — this code. (b) The value whose **first** copy comes earliest among all repeated values — for {5,2,7,2,5} that answer is 5. Ask the interviewer which one. For (b) you count first, then re-walk the array and return the first value with count ≥ 2 .

Example 4

Are two arrays rearrangements of each other? Lengths up to 10^5 . Edge cases: different lengths, both empty.

SOLUTION

Count the first array. Then spend the counts on the second. If a value runs out, or was never there, the answer is no.

Same multiset of values

JS

```

const samePermutation = (a, b) => {
  if (a.length !== b.length) return false;
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  for (const x of b) {
    const c = cnt.get(x);
    if (c === undefined || c === 0) return false;
    cnt.set(x, c - 1);
  }
};

```

```

}
return true;
};

```

Tested: ($\{1, 2, 2, 3\}, \{3, 2, 1, 2\}$) gives 1; ($\{1, 2, 2, 3\}, \{3, 2, 1, 1\}$) gives 0; ($\{\}, \{\}$) gives 1; ($\{1\}, \{1, 1\}$) gives 0; ($\{-4, -4\}, \{-4, -4\}$) gives 1.

Time $O(n)$ expected **Space** $O(n)$

Example 5

Count distinct values. Edge cases: empty array, all equal.

SOLUTION

A set drops repeats for you. Build it from the range and read its size.

Distinct count in one line

JS

```
const countDistinct = (a) => new Set(a).size;
```

Tested: $\{6, 6, 6\}$ gives 1; $\{1, 2, 3, 2\}$ gives 3; $\{\}$ gives 0; $\{-7\}$ gives 1.

Example 6

Return the most frequent value. If two values tie, return the smaller one. Edge case: single element, negative values. Assume the array is not empty.

SOLUTION

Count everything, then sweep the map once keeping the best pair (count, value). The tie rule is one extra condition.

Mode, smallest value on a tie

JS

```
const modeSmallestTie = (a) => {
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  let best = 0, bestCount = -1;
  for (const [value, c] of cnt) {
    if (c > bestCount || (c === bestCount && value < best)) {
      best = value;
      bestCount = c;
    }
  }
  return best;
}; // caller must not pass an empty array
```

Tested: $\{4, 9, 4, 9, 2\}$ gives 4 (both 4 and 9 appear twice, 4 is smaller); $\{7, 7, 7, 1\}$ gives 7; $\{3\}$ gives 3; $\{-5, -9, -9\}$ gives -9.

TRAP

bestCount starts at -1 , not 0. If it started at 0 and every count were 0 (it cannot be, but in similar problems it can), nothing would ever be selected and best would stay at its junk value.

Tier 1 — the placement standards

Example 7 [TCS NQT · pattern]

Given an array and a number **target**, is there a pair of **different positions** whose values add to **target**? $n \leq 10^5$, values up to 10^9 in size. Target $O(n)$. Edge cases: empty array, one element, two equal values that add to the target, values big enough that **target - x** leaves int range.

SOLUTION

Approach 1 — Check every pair ($O(n^2)$ — dies above $n = 10000$)

Approach 1 — brute force

JS

```
const pairSumBrute = (a, target) => {
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      if (a[i] + a[j] === target) return true;
  return false;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Sort, then two pointers ($O(n \log n)$, and it destroys the order)

Approach 2 — sort + two pointers

JS

```
const pairSumTwoPointer = (a, target) => {
  const s = [...a].sort((x, y) => x - y); // sort a COPY, and pass a comparator
  let i = 0, j = s.length - 1;
  while (i < j) {
    const sum = s[i] + s[j];
    if (sum === target) return true;
    if (sum < target) i++; // need more, move the small end up
    else j--; // too much, move the big end down
  }
  return false;
};
```

Time $O(n \log n)$ **Space** $O(n)$ for the copy

Good when you also need the values sorted. Useless when you must return original indices.

Approach 3 — One pass with a set of values already seen ($O(n)$ — optimal)

For the current value x , the partner must be $\text{target} - x$. Ask the set whether that partner walked past already.

Approach 3 — hash set

JS

```
const pairSumHash = (a, target) => {
  const seen = new Set();
  for (const x of a) {
    if (seen.has(target - x)) return true;
    seen.add(x);
  }
};
```

```
    return false;
};
```

Time $O(n)$ expected **Space** $O(n)$

If the indices are wanted, store value -> index instead of just the value.

Approach 3b — return the two indices

JS

```
const pairSumIndices = (a, target) => {
  const pos = new Map(); // value -> index seen earlier
  for (let i = 0; i < a.length; i++) {
    const need = target - a[i];
    if (pos.has(need)) return [pos.get(need), i];
    pos.set(a[i], i); // insert AFTER the lookup
  }
  return [-1, -1];
};
```

Tested on $(\{2, 11, 7, 4\}, 9)$ all three return 1, and pairSumIndices returns 0, 2 (values 2 and 7). On $(\{\}, 5)$, $(\{5\}, 5)$ all return 0. On $(\{3, 3\}, 6)$ all return 1 — two different positions holding the same value is a legal pair. On $([1000000000, 1000000000], 2000000000)$ all return 1. In C++ that sum needs a 64-bit cast; in JavaScript every number is already a double, exact up to $2^{53} - 1 \approx 9.0 \times 10^{15}$, so 2×10^9 is nowhere near the edge.

TRAP

Insert $a[i]$ **after** the lookup, never before. If you insert first, a lone 4 with $\text{target} = 8$ matches itself and you report a fake pair.

The idea that unlocked it: the inner loop was searching for one exact number, $\text{target} - x$. Anything searching for one exact number is a hash lookup.

Example 8 [Infosys · pattern]

Return the first character in a string that appears exactly once. Return $\#sym.hash$ if there is none. String length up to 10^5 , any bytes. Target $O(n)$. Edge cases: empty string, all characters repeated.

SOLUTION

Two passes. Pass 1 counts. Pass 2 walks the string again **in order** and returns the first character with count 1. The second pass must walk the string, not the table — a table has no order.

First unique character

JS

```
const firstUniqueChar = (s) => {
  const cnt = new Map();
  for (const c of s) cnt.set(c, (cnt.get(c) ?? 0) + 1);
  for (const c of s) if (cnt.get(c) === 1) return c; // walk the STRING, not the map
  return '#';
};
```

Tested: "swiss" gives w; "aabb" gives $\#sym.hash$; "" gives $\#sym.hash$; "z" gives z; "aabbc" gives c.

SHORTCUT

When keys are characters, `new Array(256).fill(0)` is the hash table, with a perfect hash and zero collisions. It is several times faster than a `Map`, because there is no hashing and no boxing. For lowercase letters only, use 26 slots and index with `c.charCodeAt(0) - 97`.

Time $O(n)$ **Space** $O(1)$ — 256 fixed slots

Example 9 [Wipro · pattern]

Return the distinct values that appear in both arrays, in the order they first appear in the first array. Lengths up to 10^5 . Edge cases: one array empty, repeats inside each array.

SOLUTION

Put the second array in a set. Then walk the first array in order. A second set stops you from printing the same value twice.

Intersection, order kept

JS

```
const intersectDistinct = (a, b) => {
  const inB = new Set(b);
  const used = new Set();
  const out = [];
  for (const x of a)
    if (inB.has(x) && !used.has(x)) { out.push(x); used.add(x); }
  return out;
};
```

Tested: (`{4,1,4,9}`, `{9,4,4}`) gives `[4,9]`; (`{1,2}`, `{3}`) gives `[]`; (`{}`, `{1}`) gives `[]`; (`{-3,-3}`, `{-3}`) gives `[-3]`.

Time $O(n + m)$ expected **Space** $O(m + \text{answer})$

Example 10 [Capgemini · pattern]

The array holds $n - 1$ **distinct** numbers taken from $1..n$. Find the missing one. n up to 2×10^6 . Edge cases: empty array (then $n = 1$, answer 1), the missing number is 1 or is n , and the overflow risk in the sum method.

SOLUTION

Approach 1 — Hash set ($O(n)$ time, $O(n)$ memory)

Approach 1 — set

JS

```
const missingHash = (a) => {
  const n = a.length + 1;
  const s = new Set(a);
  for (let v = 1; v <= n; v++) if (!s.has(v)) return v;
  return -1;
};
```

Approach 2 — Sum formula ($O(n)$ time, $O(1)$ memory — but watch the overflow)

$1 + 2 + \dots + n = n(n + 1)/2$. Subtract everything present; what is left is the hole.

Approach 2 — sum

JS

```
const missingSum = (a) => {
  const n = a.length + 1;
```

```
let total = n * (n + 1) / 2;    // n = 2e6 gives 2e12 – exact, the ceiling is
9.0e15
for (const x of a) total -= x;
return total;
};
```

Approach 3 — XOR ($O(n)$ time, $O(1)$ memory, no overflow at all — optimal)

XOR every number from 1 to n , then XOR every number present. Every value that appears twice cancels ($x \oplus x = 0$). Only the hole survives.

Approach 3 — XOR

JS

```
const missingXor = (a) => {
  const n = a.length + 1;
  let x = 0;
  for (let v = 1; v <= n; v++) x ^= v;    // ^ is a 32-bit operator, fine up to  $2^{31} - 1$ 
  for (const v of a) x ^= v;
  return x;
};
```

Tested. On $\{1, 2, 4, 5\}$ all three give 3. On $\{\}$ all give 1. On $\{2\}$ all give 1. On $\{1\}$ all give 2. On a two-million case with the hole at 777777, the sum version and the XOR version both print 777777 — the running total reaches about 2×10^{12} , which a JavaScript number still holds exactly.

TRAP

Write missingSum with plain int and $n = 2 \times 10^6$: the total is about 2×10^{12} , which does not fit in a 32-bit int. You get a wrong answer with no warning. This is the single most common silent bug in a placement test.

The idea that unlocked it: the set stored **presence**. A sum stores presence too, just compressed into one number. XOR stores it and cannot overflow.

Example 11 [Accenture · pattern]

Count how many **value pairs** $(x, x + k)$ exist, where both values occur in the array. $k \geq 0$. Count each value pair once, no matter how many copies exist. $n \leq 10^5$. Edge cases: $k = 0$, empty array, one element, negative values.

SOLUTION

Approach 1 — All ordered index pairs, then de-duplicate ($O(n^2 \log n)$)

Approach 1 — brute force

JS

```
const countPairsBrute = (a, k) => {
  const found = new Set();
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length; j++)
      if (i !== j && a[j] - a[i] === k) found.add(`${a[i]},${a[j]}`);    // string key
  return found.size;
};
```

Approach 2 — Count table, then ask for each key ($O(n)$ — optimal)

For every distinct value x , ask whether $x + k$ also exists. That is one lookup. The case $k = 0$ is different: x must pair with **another copy of itself**, so it needs count ≥ 2 .

Approach 2 — hash map of counts

JS

```
const countPairsHash = (a, k) => {
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  let total = 0;
  if (k === 0) {
    for (const c of cnt.values()) if (c >= 2) total++; // x pairs with another copy
    of x
    return total;
  }
  for (const x of cnt.keys()) if (cnt.has(x + k)) total++;
  return total;
};
```

Both versions agree on every test: $(\{1, 5, 3, 4, 2\}, 2)$ gives 3 (the pairs $(1, 3), (2, 4), (3, 5)$); $(\{3, 3, 3\}, 0)$ gives 1; $(\{\}, 1)$ gives 0; $(\{7\}, 1)$ gives 0; $(\{-4, -2, 0\}, 2)$ gives 2.

TRAP

Forgetting $k = 0$ is the classic failure. Without the special case every distinct value pairs with itself and you return the number of distinct values.

Time $O(n)$ expected **Space** $O(n)$

Example 12 [Cognizant · pattern]

Are two strings anagrams? Length up to 10^5 . Edge cases: different lengths, both empty, same letters but different counts.

SOLUTION

Approach 1 — Sort both, compare ($O(n \log n)$)

Approach 1 — sort

JS

```
const anagramSort = (a, b) => {
  if (a.length !== b.length) return false;
  const norm = (s) => [...s].sort().join(''); // characters: the default sort is
  correct
  return norm(a) === norm(b);
};
```

Approach 2 — One count table: add for a, subtract for b ($O(n)$ — optimal)

Approach 2 — counting

JS

```
const anagramCount = (a, b) => {
  if (a.length !== b.length) return false;
  const cnt = new Map();
  for (const c of a) cnt.set(c, (cnt.get(c) ?? 0) + 1);
  for (const c of b) {
    const left = (cnt.get(c) ?? 0) - 1;
    if (left < 0) return false; // b used a letter a does not have
    cnt.set(c, left);
  }
};
```

```
    return true;
};
```

Both agree: ("listen","silent") → 1; ("abc","abd") → 0; ("","") → 1; ("a","aa") → 0; ("aab","abb") → 0.

SHORTCUT

The length check is not just an optimisation, it is **required**. Without it, "a" and "aa" would give a wrong answer for some count schemes. With equal lengths, “no letter went negative” already proves the counts match exactly.

Time $O(n)$ **Space** $O(1)$

Example 13 [TCS Digital · pattern]

Remove later copies of every value, keep the first copy, keep the original order. $n \leq 10^5$. Edge cases: empty, all same, negatives.

SOLUTION

insert on an `unordered_set` returns a pair; its second field is `true` only when the value was new. That is exactly the test you need, and it costs one lookup instead of two.

De-duplicate, order kept

JS

```
const dedupKeepFirst = (a) => {
  const seen = new Set();
  return a.filter((x) => (seen.has(x) ? false : (seen.add(x), true)));
};

// when you do not need the "was it new?" test at all, one line does it:
const dedup = (a) => [...new Set(a)];
```

Tested: {5,1,5,2,1} gives [5,1,2]; {} gives []; {9,9,9} gives [9]; {-1,0,-1} gives [-1,0].

Example 14 [Infosys · pattern]

One value appears **more than** $n/2$ times. Find it, or report that no value does. $n \leq 10^5$. Edge cases: empty array, no majority, negative values.

SOLUTION

Approach 1 — Count every value by scanning ($O(n^2)$)

Approach 1 — brute force

JS

```
const majorityBrute = (a) => {
  for (let i = 0; i < a.length; i++) {
    let c = 0;
    for (let j = 0; j < a.length; j++) if (a[j] === a[i]) c++;
    if (c > a.length / 2) return a[i];
  }
  return -1;
};
```

Approach 2 — Hash map of counts ($O(n)$ time, $O(n)$ memory)

Approach 2 — counting

JS

```
const majorityHash = (a) => {
  const cnt = new Map();
  for (const x of a) {
    const c = (cnt.get(x) ?? 0) + 1;
    cnt.set(x, c);
    if (c > a.length / 2) return x;      // stop the moment it crosses
  }
  return -1;
};
```

Approach 3 — Voting (O(n) time, O(1) memory — optimal)

Keep a candidate and a vote count. A matching value adds a vote, any other value removes one. When votes hit zero, the next value becomes the new candidate. Think of it as cancelling one majority ballot against one non-majority ballot: since the majority has more than half, it always survives. The vote alone can lie when **no** majority exists, so verify with one more pass.

Approach 3 — vote and verify

JS

```
const majorityVote = (a) => {
  let cand = 0, votes = 0;
  for (const x of a) {
    if (votes === 0) { cand = x; votes = 1; }
    else if (x === cand) votes++;
    else votes--;
  }
  let c = 0;
  for (const x of a) if (x === cand) c++; // verify: the vote alone can lie
  return c > a.length / 2 ? cand : -1;
};
```

All three agree: {4,4,2,4,3} gives 4; {1,2} gives -1; {} gives -1; {6} gives 6; {-3,-3,-3,1} gives -3.

TRAP

Skipping the verification pass is the number one mistake here. On {1,2} the voting loop leaves `cand = 2`, and without the check you would return 2.

The idea that unlocked it: the hash map stored the count of every value, but we only ever needed **one** count. Voting keeps that single count and throws the rest away.

Example 15 [Capgemini · pattern]

Given two arrays, how many values appear in the first but not in the second? Count distinct values. Lengths up to 10^5 .

SOLUTION

One set for the second array, one set to avoid double counting. The same skeleton as Example 9 with the condition flipped.

Values only in the first array

JS

```
const onlyInFirst = (a, b) => {
  const inB = new Set(b), done = new Set();
  const out = [];
```

```
for (const x of a)
  if (!inB.has(x) && !done.has(x)) { out.push(x); done.add(x); }
return out;
};
```

Tested: $(\{1, 2, 2, 3\}, \{2, 4\})$ gives $[1, 3]$; $(\{\}, \{1\})$ gives $[]$; $(\{5\}, \{\})$ gives $[5]$; $(\{-1, -1\}, \{-1\})$ gives $[]$.

Tier 2 — applied, two or three steps

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 16 [Grab · pattern]

A ride-fare log holds daily profit or loss as integers, positive and negative. Find the length of the **longest** stretch of consecutive days whose total is exactly k . $n \leq 10^5$, each value up to 10^9 in size, so the running total can reach 10^{14} . Target $O(n)$. Edge cases: empty log, no such stretch, all zeros with $k = 0$, totals that overflow 32 bits.

SOLUTION

Approach 1 — Every start, every end ($O(n^2)$ — 10^{10} steps at $n = 10^5$)

Approach 1 — brute force

JS

```
const longestSumBrute = (a, k) => {
  let best = 0;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) {
      s += a[j];
      if (s === k) best = Math.max(best, j - i + 1);
    }
  }
  return best;
};
```

Approach 2 — Prefix sums + map of the FIRST index ($O(n)$ — optimal)

Let P_i be the sum of the first $i + 1$ values. The sum of the stretch from l to r is $P_r - P_{l-1}$. So a stretch ending at r has sum k exactly when some earlier prefix equals $P_r - k$.

To make the stretch **long**, you want that earlier prefix to be as **early** as possible. So store only the first index at which each prefix value appeared.

Approach 2 — prefix + first-index map

JS

```
const longestSumHash = (a, k) => {
  const firstIdx = new Map([[0, -1]]); // prefix value -> earliest index;
  let pre = 0, best = 0; // 0 is the empty prefix, ending before
  index 0
  for (let i = 0; i < a.length; i++) {
    pre += a[i];
    if (firstIdx.has(pre - k)) best = Math.max(best, i - firstIdx.get(pre - k));
    if (!firstIdx.has(pre)) firstIdx.set(pre, i); // keep the EARLIEST only
  }
}
```

```
    return best;
};
```

Both versions agree on every test:

array	k	answer
{1, -1, 5, -2, 3}	3	4
{-2, -1, 2, 1}	1	2
{}	0	0
{0, 0, 0}	0	3
{5}	5	1
{5}	4	0
{2000000000, 2000000000, -1}	4000000000	2

TRAP

Two traps live in four lines here.

1. `firstIdx[0] = -1` must be there. Without it a stretch that starts at index 0 is never found. Test `{5}` with $k = 5$ catches this.
2. `if (!firstIdx.has(pre))` must be there. Overwriting with a later index gives you a shorter answer, and the bug only shows on inputs with repeated prefix values.

With 10^5 values of size 10^9 the prefix reaches 10^{14} . In C++ this is the line where you would reach for a 64-bit integer; a JavaScript number is a double and holds every integer up to $2^{53} - 1 \approx 9.0 \times 10^{15}$ exactly, so nothing special is needed. Past that limit — and only past it — you would switch to `BigInt`.

Time $O(n)$ expected **Space** $O(n)$

The idea that unlocked it: the inner loop was recomputing sums you already had. A prefix sum turns “sum of a stretch” into “difference of two numbers”, and a difference is exactly what a hash lookup finds.

Example 17 [Shopee · pattern]

Same log. Now count **how many** stretches total exactly k (not the longest — all of them). $n \leq 10^5$. The count itself can exceed 32 bits. Edge cases: empty log, all zeros with $k = 0$, negatives.

SOLUTION

Same prefix trick, one change: you no longer want the earliest index, you want **how many** earlier prefixes had the needed value. So store counts, not indices.

Approach 1 — brute force (for checking)

JS

```
const countSumBrute = (a, k) => {
  let total = 0;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) { s += a[j]; if (s === k) total++; }
  }
  return total;
};
```

Approach 2 — prefix + count map

JS

```
const countSumHash = (a, k) => {
  const seen = new Map([[0, 1]]); // the empty prefix counts as one
  let pre = 0, total = 0;
  for (const x of a) {
    pre += x;
    total += seen.get(pre - k) ?? 0; // look up BEFORE inserting
    seen.set(pre, (seen.get(pre) ?? 0) + 1);
  }
  return total;
};
```

Both agree:

array	k	answer
{1,2,3}	3	2
{0,0,0}	0	6
{}	0	0
{-1,1,-1,1}	0	4
{4}	4	1
{4}	0	0

With 100000 zeros and $k = 0$, the optimal version returns 5000050000. That is $n(n+1)/2$. It is far past 2^{31} , which would overflow a 32-bit int in C++, but it is a thousand times below 2^{53} , so a plain JavaScript number is exact.

TRAP

`seen[0] = 1` — not `seen[0] = 0`, and not missing. Test `{4}` with $k = 4$: without it the answer is 0 instead of 1.

TRAP

Look up `pre - k` **before** you do `seen[pre]++`. If $k = 0$ and you increment first, the current prefix matches itself and every element reports one fake stretch.

Time $O(n)$ expected **Space** $O(n)$

Example 18 [Agoda · pattern]

Group hotel-code strings so that all rearrangements of the same letters end up in one group. Words are lowercase, total characters up to 10^6 . Edge cases: no words, one word, two identical words.

SOLUTION

Two words belong together when their letter counts match. So build a **canonical key** from the letter counts and use it as a map key. Sorting the word also works and is shorter; the count key is faster for long words.

Group by a canonical count key

JS

```
const groupAnagrams = (words) => {
  const table = new Map();
  for (const w of words) {
    const cnt = new Array(26).fill(0);
    for (let i = 0; i < w.length; i++) cnt[w.charCodeAt(i) - 97]++;
  }
};
```



```

    const key = cnt.join('#'); // '#' separators: (1,11) must not read as
    (11,1)
    if (!table.has(key)) table.set(key, []);
    table.get(key).push(w);
  }
  return [...table.values()]; // a Map hands the groups back in first-
  seen order
};

```

On {"tea", "eat", "tan", "ate", "nat", "bat"} the groups printed are:

output	TEXT
bat	
tan nat	
tea eat ate	

Empty input gives 0 groups, {"zz"} gives 1 group, {"ab", "ab"} gives 1 group of two.

TRAP

The separator '#' in the key matters. Without it, counts (1, 11) and (11, 1) both become the string "111" and two different words collide into one group.

Time $O(\text{total characters})$ **Space** $O(\text{total characters})$

Example 19 [Sea / Shopee · pattern]

Order IDs arrive unsorted with repeats. Find the length of the longest run of **consecutive integers** present. $n \leq 10^5$, IDs may be negative. Edge cases: empty, one ID, all equal, a run that touches INT_MAX.

SOLUTION

Approach 1 — Sort, then walk ($O(n \log n)$, easy and usually fast enough)

Approach 1 — sort	JS
<pre> const longestRunSort = (a) => { if (a.length === 0) return 0; const s = [...a].sort((x, y) => x - y); // NUMERIC comparator, always let best = 1, run = 1; for (let i = 1; i < s.length; i++) { if (s[i] === s[i - 1]) continue; // skip duplicates if (s[i] === s[i - 1] + 1) run++; else run = 1; best = Math.max(best, run); } return best; }; </pre>	

Approach 2 — Hash set, count only from a run's start ($O(n)$ — optimal)

Put everything in a set. For a value x , only bother counting upward if $x - 1$ is **not** in the set — that means x begins its run. Every run is therefore walked exactly once, so the total work over all runs is $O(n)$, not $O(n^2)$.

Approach 2 — hash set	JS
-----------------------	----

```
const longestRunHash = (a) => {
  const s = new Set(a);
  let best = 0;
  for (const x of s) {
    if (s.has(x - 1)) continue;           // x is not the start of its run
    let y = x, len = 1;
    while (s.has(y + 1)) { y++; len++; }
    best = Math.max(best, len);
  }
  return best;
};
```

Both agree: {100,4,2,1,3,2} gives 4; {} gives 0; {8} gives 1; {5,5,5} gives 1; {-3,-2,-1,4} gives 3.

TRAP

Drop the `if (s.has(x - 1)) continue;` guard and the code still gives the right answer but becomes $O(n^2)$ on a single long run: every member re-walks the whole run.

Time $O(n)$ expected **Space** $O(n)$

Example 20 [DBS · pattern]

A log of transactions marks each as a credit (1) or a debit (0). Find the longest stretch with an equal number of credits and debits. $n \leq 10^5$. Edge cases: empty, all credits, one entry.

SOLUTION

Replace every 0 by -1 . Now “equal counts” means “the stretch sums to zero”, and you are back in Example 16 with $k = 0$.

0 becomes -1, then longest zero-sum stretch

JS

```
const longestBalanced = (bits) => {
  const firstIdx = new Map([[0, -1]]);
  let pre = 0, best = 0;
  for (let i = 0; i < bits.length; i++) {
    pre += bits[i] === 1 ? 1 : -1;
    if (firstIdx.has(pre)) best = Math.max(best, i - firstIdx.get(pre));
    else firstIdx.set(pre, i);
  }
  return best;
};
```

Tested: {0,1,0,0,1,1,0} gives 6 (indices 0..5 hold three of each); {1,1,1} gives 0; {} gives 0; {0} gives 0; {0,1} gives 2.

The idea that unlocked it: a re-labelling. Nothing about the algorithm changed; only the meaning of the numbers did. Watch for this move — it also turns “equal vowels and consonants” and “equal A’s and B’s” into the same three lines.

Example 21 [LINE MAN · pattern]

Find the k most ordered dishes. On a tie, the smaller dish id comes first. $n \leq 10^5$, ids fit in `int`. Target better than $O(n \log n)$ if possible. Edge cases: k larger than the number of distinct dishes, empty input.

SOLUTION

Approach 1 — Count, then sort the distinct values ($O(d \log d)$, d = distinct count)

Approach 1 — sort the counts

JS

```
const topKSort = (a, k) => {
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  return [...cnt.entries()]
    .sort(([v1, c1], [v2, c2]) => c2 - c1 || v1 - v2) // count down, then value up
    .slice(0, k)
    .map([v] => v);
};
```

Approach 2 — Bucket by count $O(n)$ — optimal

A count can never be bigger than n . So make $n + 1$ buckets, drop each value into the bucket named by its count, and read the buckets from the top down.

Approach 2 — bucket sort on counts

JS

```
const topKBucket = (a, k) => {
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  const bucket = Array.from({ length: a.length + 1 }, () => []);
  for (const [v, c] of cnt) bucket[c].push(v);
  const out = [];
  for (let c = a.length; c >= 1 && out.length < k; c--) {
    bucket[c].sort((p, q) => p - q); // smaller value first on a tie
    for (const v of bucket[c]) {
      if (out.length === k) break;
      out.push(v);
    }
  }
  return out;
};
```

Both agree: $(\{1,1,1,2,2,3\}, 2)$ gives $[1,2]$; $(\{\}, 3)$ gives $[]$; $(\{7\}, 1)$ gives $[7]$; $(\{4,4,5,5\}, 1)$ gives $[4]$; $(\{-2,-2,9\}, 2)$ gives $[-2,9]$.

Without the tie rule the bucket version is a clean $O(n)$. The tie rule forces a small sort inside each bucket, but the buckets are tiny, so it stays close to $O(n)$ in practice. Say this out loud in an interview instead of pretending the sort is free.

Example 22 [SCB · pattern]

Count **distinct unordered value pairs** $\{x, y\}$ with $x \neq y$ and $x + y = \text{target}$. $n \leq 10^5$, values up to 2×10^9 in size. Edge cases: repeats, $x = y$ cases must be excluded, values near the int edges.

SOLUTION

Walk once. Keep a set of values seen. When the partner is present and different from x , remember the pair with its smaller value first so both orders collapse to one entry.

Distinct value pairs

JS

```
const countUniquePairs = (a, target) => {
  const seen = new Set();
  const used = new Set(); // keys are strings "lo,hi"
  for (const x of a) {
    const need = target - x;
    if (need !== x && seen.has(need))
```

```

    used.add(`${Math.min(x, need)}, ${Math.max(x, need)}`);
    seen.add(x);
  }
  return used.size;
};

```

Tested: $(\{1, 5, 3, 3, 7, -1, 9\}, 8)$ gives 3 — the pairs $\{1, 7\}$, $\{5, 3\}$, $\{-1, 9\}$, and the second 3 adds nothing. $(\{4, 4, 4\}, 8)$ gives 0 because $x = y$ is excluded. $(\{\}, 0)$ gives 0. $(\{2\}, 2)$ gives 0. $(\{2000000000, 2000000000, -2000000000\}, 0)$ gives 1.

Time $O(n \log n)$ — the answer set is ordered **Space** $O(n)$

Tier 3 — needs an insight

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

Length of the longest stretch of a string in which no character repeats. Length up to 10^5 , any bytes. Target $O(n)$. Edge cases: empty, all same, a repeat that is far behind the window.

The follow-up the interviewer asks next: “now the characters arrive one at a time and you cannot store the whole string.”

SOLUTION

Approach 1 — Every start, grow until a repeat $(O(n^2))$

Approach 1 — brute force

JS

```

const longestDistinctBrute = (s) => {
  let best = 0;
  for (let i = 0; i < s.length; i++) {
    const used = new Set();
    for (let j = i; j < s.length; j++) {
      if (used.has(s[j])) break;
      used.add(s[j]);
      best = Math.max(best, j - i + 1);
    }
  }
  return best;
};

```

Approach 2 — Sliding window + last-seen table $(O(n))$ — optimal

Keep a window $[\text{left}, \text{right}]$. Store, for each character, the last index it was seen at. When the new character was last seen **inside** the window, jump `left` to just past that old copy. `left` never moves backwards, so each index is touched a constant number of times.

Approach 2 — window with a jump

JS

```

const longestDistinctWindow = (s) => {
  const last = new Map(); // character -> last index it was seen at
  let left = 0, best = 0;

```

```

for (let right = 0; right < s.length; right++) {
  const c = s[right];
  if (last.has(c) && last.get(c) >= left) left = last.get(c) + 1; // jump past the
  copy
  last.set(c, right);
  best = Math.max(best, right - left + 1);
}
return best;
};

```

Both agree: "abcabcb" gives 3; "bbbb" gives 1; "" gives 0; "a" gives 1; "pwwkew" gives 3; "abba" gives 2.

TRAP

if (last[c] >= left) — the >= left part is essential. On "abba" the second a was last seen at index 0, but left has already moved to 2. Without the guard left would jump **backwards** to 1 and the answer would be 3, which is wrong.

The follow-up: a stream

You cannot keep the string, but you never needed it. The window code already stores only 256 integers and two cursors. Feed characters in one at a time, keep left, right and last[] as member variables, and report right - left + 1 after each arrival. Memory stays $O(1)$ for bytes, or $O(|\Sigma|)$ for a general alphabet.

Example 24 [Amazon · pattern]

Count the stretches of an array whose sum is divisible by k . Values may be negative. $n \leq 10^5$, $1 \leq k \leq 10^4$. Target $O(n + k)$. Edge cases: empty, negative sums, all values already divisible by k .

The follow-up: “why did your first version fail on negative numbers?”

SOLUTION

Two prefix sums leave the same remainder when divided by k exactly when the piece between them is divisible by k . So count remainders, not prefix values. There are only k different remainders, so a plain array beats a hash map here.

Approach 1 — brute force (for checking)

JS

```

const divKBrute = (a, k) => {
  let total = 0;
  for (let i = 0; i < a.length; i++) {
    let s = 0;
    for (let j = i; j < a.length; j++) { s += a[j]; if (s % k === 0) total++; }
  }
  return total;
};

```

Approach 2 — remainder counts

JS

```

const divKHash = (a, k) => {
  const cnt = new Array(k).fill(0);
  cnt[0] = 1; // the empty prefix has remainder 0
  let pre = 0, total = 0;
  for (const x of a) {
    pre += x;
    let r = pre % k;
    if (r < 0) r += k; // -7 % 5 is -2 in JS; we want 3
    cnt[r]++;
  }
  let total = 0;
  for (let i = 0; i < k; i++) {
    total += cnt[i] * cnt[i];
  }
  return (total - cnt[0]) / 2;
};

```

```

    total += cnt[r];
    cnt[r]++;
}
return total;
};

```

Both agree:

array	k	answer
{4,5,0,-2,-3,1}	5	7
{}	3	0
{7}	7	1
{-1,-1,-1}	3	1
{5,5,5}	5	6
{1,2}	1	3

The follow-up answered

In C++, % keeps the sign of the left operand: $-7 \% 5$ is -2 , not 3 . Without the `if (r < 0) r += k;` line, the prefix with sum -7 and the prefix with sum 3 get different slots even though both are “remainder 3”, and you undercount. It also indexes `cnt[-2]`, which is out of bounds — undefined behaviour, and often a crash. Test $\{-1, -1, -1\}$ with $k = 3$ catches it.

Time $O(n + k)$ **Space** $O(k)$

Example 25 [Microsoft · pattern]

Design a container with `insert(v)`, `erase(v)` and `getRandom()`, **all** $O(1)$ average. `getRandom` must pick uniformly among the current values. Edge cases: erase a value that is not there, erase the last remaining value, insert a duplicate.

The follow-up: “now allow duplicates.”

SOLUTION

A vector gives $O(1)$ random access but $O(n)$ erase-from-the-middle. A hash map gives $O(1)$ erase but no random pick. Use both: the vector holds the values with no holes, the map says where each value sits in the vector.

To erase from the middle in $O(1)$, move the **last** item into the hole and shrink. Order inside the vector does not matter, so nothing is lost.

Vector + map, with the swap-with-last trick

JS

```

class RandomSet {
  constructor() {
    this.items = [];           // dense list of the current values
    this.where = new Map();    // value -> its index inside items
  }
  insert(v) {
    if (this.where.has(v)) return false;
    this.where.set(v, this.items.length);
    this.items.push(v);
    return true;
  }
  erase(v) {

```

```

    if (!this.where.has(v)) return false;
    const idx = this.where.get(v);
    const lastVal = this.items[this.items.length - 1];
    this.items[idx] = lastVal;    // move the last item into the hole
    this.where.set(lastVal, idx);
    this.items.pop();
    this.where.delete(v);        // delete AFTER the move, or v === lastVal breaks
    return true;
  }
  getRandom() {
    return this.items[Math.floor(Math.random() * this.items.length)];
  }
  get size() { return this.items.length; }
}

```

Tested. insert(4) insert(4) insert(9) erase(7) erase(4) prints 1 0 1 0 1, and the one remaining value is 9. Erasing the only element works and leaves size 0. With values {10, 20, 30} and 30000 draws the tally was 10:10086 20:9929 30:9985 — close to a third each, as it must be.

TRAP

Order matters inside erase. If you erase the map entry for *v* first and *v* happens to **be** the last item, the line `where[lastVal] = idx` then re-creates a dead entry for a value you just deleted. Always move first, erase second.

The follow-up: allow duplicates

Change the map to `Map<number, Set<number>>`: value → **the set of indices** holding it. `insert` appends and adds the new index. `erase` takes any one index out of that set, does the same swap-with-last, and fixes the moved value's index set (remove the old index, add the new one). All still $O(1)$ average; memory grows to $O(n)$ set nodes.

Example 26 [Adobe · pattern]

Build a fixed-size cache with $O(1)$ get and put. When it is full, throw out the key that was used least recently. Edge cases: capacity 1, updating a key that is already present, getting a missing key.

The follow-up: “now every entry also expires after *T* seconds.”

SOLUTION

You need two things at once: find a key fast, and know the order of use. A hash map does the first; a doubly linked list does the second. Store an **iterator into the list** inside the map, and `list::splice` moves a node to the front in $O(1)$ without copying.

Hash map + doubly linked list

JS

```

class LRUCache {
  constructor(capacity) {
    this.cap = capacity;
    this.m = new Map();    // iteration order: oldest key first, newest last
  }
  get(key) {
    if (!this.m.has(key)) return -1;
    const v = this.m.get(key);
    this.m.delete(key);
    this.m.set(key, v);    // delete + set = move to the newest end
    return v;
  }
  put(key, value) {

```

```

    if (this.m.has(key)) this.m.delete(key);
    else if (this.m.size === this.cap) this.m.delete(this.m.keys().next().value);
    this.m.set(key, value);          // keys().next().value is the oldest key
  }
}

```

Traced run with capacity 2:

call	returns	cache after, newest first
put(1,10)	—	(1,10)
put(2,20)	—	(2,20) (1,10)
get(1)	10	(1,10) (2,20)
put(3,30)	—	(3,30) (1,10) — key 2 evicted
get(2)	-1	(3,30) (1,10)
get(3)	30	(3,30) (1,10)
put(1,11)	—	(1,11) (3,30)
get(1)	11	(1,11) (3,30)

With capacity 1: put(5,50) then put(6,60) gives get(5) = -1 and get(6) = 60.

TRAP

splice keeps the node alive, so the iterator stored in the map stays valid. If you instead erase the node and push_front a copy, the old iterator becomes dangling and you must remember to re-store the new one. Every LRU bug in interviews is a stale iterator.

The follow-up: entries expire after T seconds

Store the insertion time inside each node. On get, if $\text{now} - \text{stored} > T$, delete the node and return “miss”. Stale nodes that are never touched still waste memory, so add a **second** structure ordered by expiry time — a min-heap of (expiry, key) — and pop expired keys lazily on each operation. Amortised cost stays $O(1)$ for the map part plus $O(\log n)$ for each real expiry, and each key expires at most once.

Example 27 [Uber · pattern]

Longest stretch of an array that holds at most k different values. $n \leq 10^5$. Target $O(n)$. Edge cases: $k = 0$, $k \geq n$, empty array.

The follow-up: “now count the stretches with **exactly** k different values.”

SOLUTION

Grow the window on the right. While it holds too many kinds, shrink from the left. Erase a key from the map the moment its count hits zero — otherwise `cnt.size()` lies.

Window with a count map

JS

```

const longestAtMostK = (a, k) => {
  if (k <= 0) return 0;
  const cnt = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < a.length; right++) {
    cnt.set(a[right], (cnt.get(a[right]) ?? 0) + 1);
    while (cnt.size > k) {
      const c = cnt.get(a[left]) - 1;
      if (c > 0) cnt.set(a[left], c);
      else delete cnt.get(a[left]);
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}

```



```

        if (c === 0) cnt.delete(a[left]); else cnt.set(a[left], c);
        left++;
    }
    best = Math.max(best, right - left + 1);
}
return best;
};

```

On {1,2,1,3,4,3,3}: $k = 2$ gives 4 (the tail 3,4,3,3), $k = 1$ gives 2, $k = 7$ gives 7. On {} with $k = 3$ it gives 0, on {5} with $k = 1$ it gives 1, on {5,5} with $k = 0$ it gives 0.

TRAP

`cnt.erase(...)` when the count reaches 0 is not optional. Leave the zero entry in and `cnt.size()` keeps counting values that already left the window, so the window never grows again.

The follow-up: exactly k

“Exactly k ” has no direct window, because shrinking on “too many” and growing on “too few” fight each other. The standard move is a subtraction:

$$\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k - 1)$$

This works for **counting stretches**, not for **lengths**. So write the counting version.

Count version, then subtract

JS

```

const countAtMostK = (a, k) => {
    if (k <= 0) return 0;
    const cnt = new Map();
    let left = 0, total = 0;
    for (let right = 0; right < a.length; right++) {
        cnt.set(a[right], (cnt.get(a[right]) ?? 0) + 1);
        while (cnt.size > k) {
            const c = cnt.get(a[left]) - 1;
            if (c === 0) cnt.delete(a[left]); else cnt.set(a[left], c);
            left++;
        }
        total += right - left + 1; // every window ending at `right` is valid
    }
    return total;
};

const countExactlyK = (a, k) => countAtMostK(a, k) - countAtMostK(a, k - 1);

```

Tested: ({1,2,1,2,3}, 2) gives 7; ({1,1,1}, 1) gives 6; ({}, 1) gives 0.

`total += right - left + 1` is the line to understand. Once the window `[left, right]` is legal, **every** window that ends at `right` and starts at or after `left` is also legal, and there are `right - left + 1` of them.

Example 28 [D. E. Shaw · pattern]

Your $O(n)$ hash solution passes locally and times out on the judge with $n = 10^5$. Explain what happened and fix it.

The follow-up: “how would you use a pair of ints as a key?”

SOLUTION

In JavaScript this is almost never a hash collision. Map and Set are real hash tables and they behave. What times out is a **container that only looks like** a hash table:

1. an array with `.includes(x)` inside the loop — that is a linear scan per element, so the loop is secretly $O(n^2)$;
2. an array used as a queue with `.shift()` — each `shift` moves every remaining element down one slot, again $O(n^2)$;
3. a plain object used with keys that are added and deleted in a mixed pattern, which pushes the object out of its fast shape.

The fix is to name the real container: `Set` for membership, a head index (or the toolkit `Deque`) for a queue, `Map` for counts. Measure it and the difference is not subtle.

The same two loops, right and wrong

JS

```
const n = 100000;
const data = Array.from({ length: n }, (_, i) => i * 7);

// (1) an array pretending to be a set – the loop is secretly O(n^2)
let t = Date.now();
const seenArr = [];
for (const x of data) if (!seenArr.includes(x)) seenArr.push(x);
const t1 = Date.now() - t;

// (2) the same loop with a Set
t = Date.now();
const seenSet = new Set();
for (const x of data) seenSet.add(x);
const t2 = Date.now() - t;

// (3) an array used as a queue: every shift() moves the whole array down
t = Date.now();
const q = [...data];
while (q.length) q.shift();
const t3 = Date.now() - t;

// (4) the same queue with a head index – this is what the toolkit Deque does
t = Date.now();
const q2 = [...data];
let head = 0;
while (head < q2.length) head++;
const t4 = Date.now() - t;

console.log("includes ms", t1, "Set ms", t2);
console.log("shift ms", t3, "head index ms", t4);
console.log("sizes", seenArr.length, seenSet.size);
```

The blow-up, measured on 100000 values:

measured output — node, one run

TEXT

```
includes ms 6456 Set ms 26
shift ms 971 head index ms 2
sizes 100000 100000
```

`includes` took **six and a half seconds**; the `Set` took 26 milliseconds. Same data, same machine, same number of keys. That is the $O(n^2)$ blow-up in the flesh — and neither line looks slow when you read it.

TRAP

Exact times move with the machine and the Node version. What does not move is the **shape**: two of these grow like n^2 and two like n . Quote the shape in an interview, not the milliseconds.

The follow-up: a pair as a key

A Map keyed on an **array** does not work: `m.set([1,2], 9)` and `m.get([1,2])` are two different array objects, so the lookup misses. Map compares keys by identity, not by contents. Three clean fixes:

1. Join the two numbers into a string key with a separator.
2. Pack both into one number, while the result stays inside $2^{53} - 1$.
3. Pack both into one BigInt, which has no ceiling at all.

Packing two ints into one key

JS

```
const packPair = (a, b) => `${a},${b}`; // always correct

const packNum = (a, b) => a * 4294967296 + (b >>> 0); // faster, but only while
// the result stays under  $2^{53}$ 
- 1
const packBig = (a, b) => (BigInt(a) << 32n) | BigInt(b >>> 0); // no ceiling at
all
```

Tested: storing under `packPair(3, -7)` and `packPair(-7, 3)` creates **two** entries with values 1 and 5, so the order of the pair is preserved, and `packNum` keeps them apart too. `packBig(3, -7)` prints 17179869177 and `packBig(-7, 3)` prints -30064771069.

TRAP

The `>>> 0` matters. It forces `b` into an unsigned 32-bit value before it is added in; without it a negative `b` carries its sign and collides with a different pair. Write `(b >>> 0)`, not `b`, every time you pack.

SHORTCUT

When you are unsure, use the string key. ``${a},${b}`` is a few times slower than the number version and always correct, including for negatives, floats and values past 2^{53} . Correct and fast enough beats clever and wrong.

Example 29 [Goldman Sachs · pattern]

Characters arrive one at a time. After each arrival, report the first character so far that has appeared exactly once, or `#sym.hash` if none. Target: $O(1)$ amortised per character.

The follow-up: “now it must also handle removals.”

SOLUTION

Keep a count table and a queue of candidates in arrival order. On each arrival, push the character and then throw away any characters at the **front** of the queue whose count has grown past 1. Each character is pushed once and popped at most once, so the work per arrival is $O(1)$ amortised.

Count table + candidate queue

JS

```
const { Deque } = require('./toolkit.js'); // see the JS Toolkit appendix

class FirstUniqueStream {
  constructor() {
    this.cnt = new Map();
    this.q = new Deque(); // candidates, oldest first
  }
}
```

```

add(c) {
    this.cnt.set(c, (this.cnt.get(c) ?? 0) + 1);
    this.q.push(c);
    while (this.q.size > 0 && this.cnt.get(this.q.front()) > 1) this.q.shift(); //
    stale head
    return this.q.size === 0 ? '#' : this.q.front();
}
}

```

Feeding "aabcbcd" prints a#bbcc. Step by step: a is unique; the second a makes it repeat so nothing is unique; b arrives and is unique; c arrives but b is still older and unique; the second b pushes the answer to c; d arrives but c is older. Feeding "zzz" prints z##. A single q prints q.

TRAP

Do not pop from the **middle** of the queue when a count grows. A queue cannot do that. Instead leave the stale entries in and skip them at the front — that is the whole trick, and it is why the cost is **amortised** $O(1)$, not worst-case $O(1)$.

The follow-up: removals too

A queue can no longer work, because removing a character can make an **old** character unique again and the queue has already thrown it away. Switch to a doubly linked list of unique characters plus a map character $\rightarrow$ node. Adding a second copy unlinks the node in $O(1)$; a removal that brings a count back to 1 re-links the node. Order is then kept by the list, not by arrival into a queue. The structure is the same map-plus-list pairing used in the LRU cache.

Dry run — count the stretches summing to k

Take $a = \{3, 1, -2, 4, -2, 2\}$ and $k = 3$. We run countSumHash from Example 17 and write down every variable after each step.

The map starts as $\{0: 1\}$ — the empty prefix.

i	a[i]	pre	pre - k	found	total	map after
0	3	3	0	1	1	$\{0:1, 3:1\}$
1	1	4	1	0	1	$\{0:1, 3:1, 4:1\}$
2	-2	2	-1	0	1	$\{0:1, 3:1, 4:1, 2:1\}$
3	4	6	3	1	2	$\{0:1, 3:1, 4:1, 2:1, 6:1\}$
4	-2	4	1	0	2	$\{0:1, 3:1, 4:2, 2:1, 6:1\}$
5	2	6	3	1	3	$\{0:1, 3:1, 4:2, 2:1, 6:2\}$

Answer: 3.

Reading the table:

- **Row i = 0.** pre becomes 3. We need an earlier prefix of $3 - 3 = 0$. The map has 0 once — that is the empty prefix, which sits just before index 0. So the stretch from index 0 to index 0 works: $\{3\}$. Total 1.
- **Row i = 1.** pre is 4, we need 1. No prefix equals 1. Nothing found.
- **Row i = 2.** pre is 2, we need -1. Not present.
- **Row i = 3.** pre is 6, we need 3. Prefix 3 was recorded at the end of index 0. The stretch after it is indices 1..3: $\{1, -2, 4\}$, which sums to 3. Total 2.
- **Row i = 4.** pre is 4 again, we need 1. Not present. But note the map entry for 4 goes from 1 to 2 — that second copy will matter later if some prefix needs 4.

- **Row i = 5.** pre is 6, we need 3. Prefix 3 is still there once. The stretch is indices 1..5: {1, -2, 4, -2, 2}, which sums to 3. Total 3.

Checking by hand: the stretches summing to 3 are {3} at index 0, {1, -2, 4} at indices 1..3, and {1, -2, 4, -2, 2} at indices 1..5. Three of them. The brute-force version also prints 3.

Notice that **nothing in the table ever looks forward**. The map only ever holds prefixes that finished before the current index. That is the invariant from page one, in action.

Python, for the three signature problems

Python is meaningfully shorter for these three. Everything else in this chapter is almost line-for-line the same, so C++ is enough.

The same three algorithms

PYTHON

```
from collections import defaultdict

def count_sum_k(a, k):
    seen = defaultdict(int)
    seen[0] = 1
    pre = total = 0
    for x in a:
        pre += x
        total += seen[pre - k]      # defaultdict gives 0 for a missing key
        seen[pre] += 1
    return total

def longest_run(a):
    s = set(a)
    best = 0
    for x in s:
        if x - 1 in s:
            continue
        y, ln = x, 1
        while y + 1 in s:
            y += 1
            ln += 1
        best = max(best, ln)
    return best

def group_anagrams(words):
    table = defaultdict(list)
    for w in words:
        table[tuple(sorted(w))].append(w)      # a tuple is hashable, a list is not
    return sorted(table.values())
```

Run output:

output

TEXT

```
2 0 6 4
4 0 1 3
[['bat'], ['tan', 'nat'], ['tea', 'eat', 'ate']]
[]
```

TRAP

In Python, `seen[pre - k]` on a plain dict raises `KeyError`. Either use `defaultdict(int)` as above, or write `seen.get(pre - k, 0)`. Also note that `table[list]` fails — a list cannot be a dict key because it can change. Use a tuple.

Practice

PRACTICE · TIER 1 target: 12 min

1. Count how many values appear **exactly once** in an array.
2. Can the array be split into pairs of equal values? (Every value must appear an even number of times.)
3. Given a list of words, return the first word that appears twice, or the empty string.
4. Find the smallest positive integer that is **not** in the array. Values may be negative.
5. Count the stretches of an array that contain exactly k odd numbers.
6. Return the distinct values that appear in the first array but not in the second.
7. Can the string note be built from the letters of pool, each letter used at most as many times as it appears in pool?

PRACTICE · TIER 2 target: 18 min

8. Length of the longest stretch whose sum is exactly 0.
9. Count index pairs $i < j$ with `a[i] == a[j]`.
10. Does a string have all-distinct characters?
11. Group values by their remainder mod m and report the size of the biggest group. $m \geq 1$, values may be negative.
12. Return every value that appears **more than** $n/3$ times. Use $O(1)$ extra space.
13. Are two strings **isomorphic** — is there a one-to-one letter mapping turning the first into the second?
14. For every window of length w , report how many distinct values it holds.

ANSWER KEY

1. Count with a map, then count the entries whose value is 1.

1 — values appearing exactly once

JS

```
const countSingles = (a) => {
  const c = new Map();
  for (const x of a) c.set(x, (c.get(x) ?? 0) + 1);
  return [...c.values()].filter((v) => v === 1).length;
};
```

Tested: $\{3, 1, 3, 5, -2\} \rightarrow 3$; $\{\} \rightarrow 0$; $\{8, 8\} \rightarrow 0$; $\{0\} \rightarrow 1$.

2. Odd length fails immediately. Then every count must be even.

2 — split into equal pairs

JS

```
const pairsUp = (a) => {
  if (a.length % 2) return false;
  const c = new Map();
  for (const x of a) c.set(x, (c.get(x) ?? 0) + 1);
};
```

```
return [...c.values()].every((v) => v % 2 === 0);
};
```

Tested: $\{2, 5, 5, 2\} \rightarrow 1$; $\{2, 5, 5\} \rightarrow 0$; $\{\} \rightarrow 1$; $\{7, 7, 7, 7\} \rightarrow 1$; $\{1, 2\} \rightarrow 0$.

3. `insert(...).second` is `false` when the word was already there.

3 — first repeated word

JS

```
const firstRepeatWord = (w) => {
  const seen = new Set();
  for (const s of w) {
    if (seen.has(s)) return s;
    seen.add(s);
  }
  return '';
};
```

Tested: $\{"red", "blue", "red", "blue"\} \rightarrow red$; $\{"a", "b"\} \rightarrow empty$; $\{\} \rightarrow empty$; $\{"x", "x"\} \rightarrow x$.

4. Ignore everything that is not positive. Then try 1, 2, 3, ... The loop runs at most $n + 1$ times, because after n successful hits the array is exhausted.

4 — smallest missing positive

JS

```
const smallestMissingPositive = (a) => {
  const s = new Set(a.filter((x) => x > 0));
  let v = 1;
  while (s.has(v)) v++;
  return v;
};
```

Tested: $\{3, 4, -1, 1\} \rightarrow 2$; $\{1, 2, 3\} \rightarrow 4$; $\{\} \rightarrow 1$; $\{-5, -1\} \rightarrow 1$; $\{2, 2, 2\} \rightarrow 1$.

5. Replace each value by 1 if odd and 0 if even. Now it is “count stretches with sum k ” from Example 17. Note $x \% 2 \neq 0$ works for negatives; $x \% 2 == 1$ does not, because $-3 \% 2$ is -1 in C++.

5 — stretches with exactly k odd numbers

JS

```
const subarraysWithK Odds = (a, k) => {
  const seen = new Map([[0, 1]]);
  let pre = 0, total = 0;
  for (const x of a) {
    pre += x % 2 !== 0 ? 1 : 0; // correct for negatives too
    total += seen.get(pre - k) ?? 0;
    seen.set(pre, (seen.get(pre) ?? 0) + 1);
  }
  return total;
};
```

Checked against brute force: $(\{1, 1, 2, 1, 1\}, 3) \rightarrow 2$; $(\{2, 4, 6\}, 0) \rightarrow 6$; $(\{\}, 0) \rightarrow 0$; $(\{-3\}, 1) \rightarrow 1$; $(\{5\}, 2) \rightarrow 0$.

6. Same skeleton as Example 9, condition flipped. See Example 15 for the code; tested $(\{1, 2, 2, 3\}, \{2, 4\}) \rightarrow [1, 3]$.

7. Count the pool, then spend it.

7 — can the note be built

JS

```
const canBuild = (note, pool) => {
  const c = new Map();
  for (const ch of pool) c.set(ch, (c.get(ch) ?? 0) + 1);
  for (const ch of note) {
    const left = (c.get(ch) ?? 0) - 1;
    if (left < 0) return false;
    c.set(ch, left);
  }
  return true;
};
```

Tested: ("aab", "aabbc") → 1; ("aaa", "aab") → 0; ("", "x") → 1; ("x", "") → 0; ("ab", "ba") → 1.

8. Example 16 with $k = 0$.

8 — longest zero-sum stretch

JS

```
const longestZeroSum = (a) => {
  const first = new Map([[0, -1]]);
  let pre = 0, best = 0;
  for (let i = 0; i < a.length; i++) {
    pre += a[i]; // 1e5 values of 1e9 reach 1e14 – still
    exact
    if (first.has(pre)) best = Math.max(best, i - first.get(pre));
    else first.set(pre, i);
  }
  return best;
};
```

Tested: {4, -1, -3, 2, 1, -2} → 3; {1, 2, 3} → 0; {} → 0; {0} → 1; {2000000000, 2000000000, -2000000000, -2000000000} → 4.

9. A value with c copies gives $c(c-1)/2$ pairs. At $n = 2 \times 10^5$ all equal, the answer is about 2×10^{10} — an `int` in C++ would wrap here, a JavaScript number does not.

9 — equal-value index pairs

JS

```
const goodPairs = (a) => {
  const c = new Map();
  for (const x of a) c.set(x, (c.get(x) ?? 0) + 1);
  let t = 0;
  for (const v of c.values()) t += v * (v - 1) / 2;
  return t;
};
```

Tested: {1, 2, 1, 1} → 3; {} → 0; {9} → 0; {4, 4} → 1; 200000 copies of 7 → 199999000000.

10. A 256-slot boolean array is the table.

10 — all characters distinct

JS

```
const allUnique = (s) => new Set(s).size === s.length;
```

Tested: "abcd" → 1; "abca" → 0; "" → 1; "q" → 1.

11. Normalise the remainder for negatives, exactly as in Example 24.

11 — biggest remainder group

JS


```
const biggestRemainderGroup = (a, m) => {
  const c = new Map();
  for (const x of a) {
    let r = x % m;
    if (r < 0) r += m;
    c.set(r, (c.get(r) ?? 0) + 1);
  }
  return c.size === 0 ? 0 : Math.max(...c.values());
};
```

Tested: ($\{3,6,7,10,13\}$, 3) $\rightarrow$ 3; ($\{-3,-6,3\}$, 3) $\rightarrow$ 3; ($\{\}$, 5) $\rightarrow$ 0; ($\{4\}$, 1) $\rightarrow$ 1.

12. At most two values can beat $n/3$. Run **two** vote counters side by side, then verify both.

12 — values over $n/3$

JS

```
const overOneThird = (a) => {
  let c1 = 0, c2 = 0, v1 = 0, v2 = 1; // v1 !== v2 to start
  for (const x of a) {
    if (x === v1) c1++;
    else if (x === v2) c2++;
    else if (c1 === 0) { v1 = x; c1 = 1; }
    else if (c2 === 0) { v2 = x; c2 = 1; }
    else { c1--; c2--; }
  }
  let n1 = 0, n2 = 0;
  for (const x of a) { if (x === v1) n1++; else if (x === v2) n2++; }
  const out = [];
  if (n1 > a.length / 3) out.push(v1);
  if (n2 > a.length / 3) out.push(v2);
  return out.sort((p, q) => p - q); // numeric comparator
};
```

Tested: $\{2,2,1,1,1,2,2\} \rightarrow [1,2]$; $\{1,2,3\} \rightarrow []$; $\{\} \rightarrow []$; $\{5,5\} \rightarrow [5]$; $\{0,0,0,1,1,1,2\} \rightarrow [0,1]$.

The order of the `else if` chain matters: matching an existing candidate must be checked **before** filling an empty slot, or the same value can occupy both slots.

13. One map is not enough — it would call (`"ab"`, `"aa"`) isomorphic. You need the mapping to work in **both** directions.

13 — isomorphic strings

JS

```
const isomorphic = (s, t) => {
  if (s.length !== t.length) return false;
  const fwd = new Map(), bwd = new Map();
  for (let i = 0; i < s.length; i++) {
    const a = s[i], b = t[i];
    if (!fwd.has(a) && !bwd.has(b)) { fwd.set(a, b); bwd.set(b, a); }
    else if (fwd.get(a) !== b || bwd.get(b) !== a) return false;
  }
  return true;
};
```

Tested: (`"paper"`, `"title"`) $\rightarrow$ 1; (`"foo"`, `"bar"`) $\rightarrow$ 0; (`""`, `""`) $\rightarrow$ 1; (`"ab"`, `"aa"`) $\rightarrow$ 0; (`"aa"`, `"ab"`) $\rightarrow$ 0; (`"a"`, `"z"`) $\rightarrow$ 1.

14. A sliding window with a count map. Erase a key when its count reaches 0, or `c.size()` over-reports.

14 — distinct values per window

JS

```

const distinctPerWindow = (a, w) => {
  const out = [];
  const n = a.length;
  if (w <= 0 || w > n) return out;
  const c = new Map();
  for (let i = 0; i < n; i++) {
    c.set(a[i], (c.get(a[i]) ?? 0) + 1);
    if (i >= w) {
      const left = c.get(a[i - w]) - 1;
      if (left === 0) c.delete(a[i - w]); else c.set(a[i - w], left);
    }
    if (i >= w - 1) out.push(c.size);
  }
  return out;
};

```

Tested: $(\{1,2,1,3,4,3\}, 3) \rightarrow [2,3,3,2]$; $(\{5,5,5\}, 2) \rightarrow [1,1]$; $(\{\}, 1) \rightarrow []$; $(\{1,2\}, 5) \rightarrow []$; $(\{7\}, 1) \rightarrow [1]$.

ONE-PAGE REVISION CARD

The reflex. About to write a loop inside a loop? Ask what the inner loop searches for. Store that in a hash table and the inner loop disappears.

Template 1 — counting

JS

```
const freq = new Map();
for (const x of a) freq.set(x, (freq.get(x) ?? 0) + 1);
```

Template 2 — prefix sum + map (the highest-value pattern in this chapter)

JS

```
const seen = new Map([[0, 1]]); // counts      -> "how many subarrays"
let pre = 0, total = 0;         // first index  -> "longest subarray"
for (const x of a) {
  pre += x;
  total += seen.get(pre - k) ?? 0;
  seen.set(pre, (seen.get(pre) ?? 0) + 1); // AFTER the lookup, always
}
```

Template 3 — window with a count map

JS

```
const cnt = new Map();
let left = 0;
for (let right = 0; right < n; right++) {
  cnt.set(a[right], (cnt.get(a[right]) ?? 0) + 1);
  while (/* window is illegal */) {
    const c = cnt.get(a[left]) - 1;
    if (c === 0) cnt.delete(a[left]); else cnt.set(a[left], c);
    left++;
  }
  // record the answer for this right
}
```

When to use what

Goal	Store	Read
seen before?	Set	<code>seen.has(x)</code>
how many times?	Map	<code>freq.get(x) ?? 0</code>
longest subarray, property P	first index of each prefix value	<code>i - firstIdx.get(need)</code>
count of subarrays, property P	count of each prefix value	<code>total += seen.get(need) ?? 0</code>
divisible by k	count of each remainder	<code>((x % k) + k) % k</code> first
group by shape	canonical string key	<code>(m.get(key) ?? []).push(x)</code>
O(1) insert/delete/random	array + Map of positions	swap with last
LRU	Map alone — it keeps insertion order	delete then re-set to refresh

Complexity

Operation	Map / Set	sorted array + binary search	array of 256
insert / get / delete	$O(1)$ expected	$O(n)$ insert, $O(\log n)$ find	$O(1)$ always
worst case	$O(n)$ per op	$O(n)$	$O(1)$
keys in sorted order	no — sort at the end	yes, always	yes, by index
memory per entry	high	low	lowest

Top traps

1. `seen.set(0, 1)` (or `firstIdx.set(0, -1)`) missing → subarrays starting at index 0 are lost.
2. Inserting **before** the lookup → an element pairs with itself.
3. Overwriting `firstIdx` for a repeated prefix → the “longest” answer is too short.
4. `freq.get(x) + 1` without `?? 0` → NaN on the first sighting.
5. `pre % k` is negative in JavaScript for negative `pre` → use `((pre % k) + k) % k`.
6. Not deleting a zero count → `cnt.size` lies inside a sliding window.
7. A plain object instead of a Map → 1 and "1" collapse into one key.
8. An array or object used as a Map key → compared by identity, so the lookup always misses. Join it into a string first.
9. Forgetting the verify pass after Boyer-Moore voting.
10. `arr.includes(x)` inside a loop instead of a Set → silently $O(n^2)$.
11. Building a group key without separators → (1,11) and (11,1) collide.

Stacks & Monotonic Stack

Last in, first out — and the one trick that makes it $O(n)$

Every JavaScript snippet in this chapter was executed with `node` before it was printed, and the outputs shown under the snippets are the real outputs.

This chapter needs **no** import. A plain JavaScript array already is a stack: `push` and `pop` both work on the end and both cost $O(1)$. The shared `MinHeap`, `DSU` and `Deque` of the **JS Toolkit** appendix are not needed until the next chapter, where `shift` from the front starts to matter.

Pattern in one page

THE ONE IDEA

A stack answers “**what is the most recent thing still waiting?**”

Half of this chapter is plain stacks: brackets, expressions, undo. The other half is the **monotonic stack**, and it is one of the highest-value patterns in a coding round.

The monotonic stack rule. Keep the stack sorted (all increasing, or all decreasing). Before pushing a new item, **pop everything that can never be the answer again**. Each item is pushed once and popped at most once, so the whole loop is $O(n)$ even though it has an inner `while`.

The four questions it answers instantly

Question	Direction	Pop while the top is
next greater to the right	$\text{right} \leftarrow \text{left}$	$\leq \text{current}$
next smaller to the right	$\text{right} \leftarrow \text{left}$	$\geq \text{current}$
previous greater to the left	$\text{left} \rightarrow \text{right}$	$\leq \text{current}$
previous smaller to the left	$\text{left} \rightarrow \text{right}$	$\geq \text{current}$

Say it in words: “**pop while the top is useless.**” Useless means the new element is better than it **and** stands in front of it, so no future query can ever pick the old one.

The template you will write ten times

JS

```
// NEXT GREATER ELEMENT to the right. -1 when there is none.
const nextGreater = (a) => {
  const n = a.length;
  const out = new Array(n).fill(-1);
  const st = []; // a plain array IS a stack
  for (let i = n - 1; i >= 0; i--) { // walk backwards
    while (st.length && st[st.length - 1] <= a[i]) st.pop(); // useless
    if (st.length) out[i] = st[st.length - 1]; // survivor is the
  }
  return out;
};
```

Values or indices?

Push **indices** whenever you need a distance (a width, a span, a number of days) and **values** when you only need the number itself. Indices are never wrong, so when in doubt, push indices and read `a[st.length - 1]`.

JavaScript has no stack type, and it does not need one. A plain array already is a stack: push and pop are both $O(1)$ and both act on the end. The top is `st[st.length - 1]`, and `st.length` is the size. The one habit to build: test `while (st.length && ...)` before you ever read the top, because reading past the end of a JavaScript array gives you undefined rather than an error.

The invariant

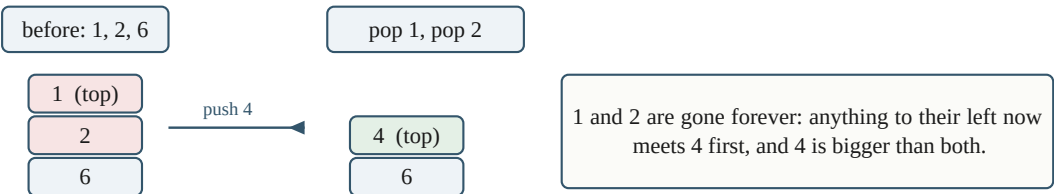
At the moment the loop is about to look at index `i`, the stack holds exactly those earlier elements that are still **candidates** — nothing more, nothing less. Everything popped was beaten by something closer to `i`, so it can never win again.

Watching it run

`a = {4, 1, 2, 6, 3}`, walking right to left, next greater to the right:

i	a[i]	popped	stack after push (top first)	out[i]
4	3	—	3	-1
3	6	3	6	-1
2	2	—	2, 6	6
1	1	—	1, 2, 6	2
0	4	1, 2	4, 6	6

Result `[6, 2, 6, -1, -1]`. Notice row `i = 0`: the values 1 and 2 are thrown away because 4 is bigger **and** closer to everything on the left. Five elements, five pushes, three pops. Total work $O(n)$.



Pushing 4 pops every smaller value first. The stack stays decreasing.

TRAP

`<=` versus `<` in the pop test decides what happens with **equal** values. `pop while top <= a[i]` means “strictly greater” is wanted. `pop while top < a[i]` means “greater or equal” is acceptable. Get this wrong and duplicates are counted twice in the contribution problems of Tier 2.

When to reach for it

The question sounds like	Reach for
brackets, nesting, undo, backtrack	plain stack
postfix / prefix expression	plain stack of operands
next / previous greater or smaller	monotonic stack
span, days until, how far back	monotonic stack of indices

largest rectangle, biggest area	monotonic stack of indices
water trapped, container	monotonic stack or two pointers
sum over all subarrays of min / max	monotonic stack + contribution counting
smallest / largest result after k deletions	monotonic stack, greedy

Warm-up

WARM-UP

build the reflex

Example 1

Build a stack on a plain array with push, pop, peek, empty and size. It must not crash when full or empty. Capacity up to 10^5 .

SOLUTION

Keep one index `top` = position of the newest item. Empty is `top == -1`. Full is `top + 1 == capacity`. Both operations return `false` instead of crashing.

Array-backed stack

JS

```
class ArrayStack {
  constructor(cap) { this.buf = new Array(cap); this.cap = cap; this.top = -1; }

  push(v) {
    if (this.top + 1 === this.cap) return false; // full
    this.buf[++this.top] = v;
    return true;
  }
  pop() {
    if (this.top < 0) return undefined; // empty
    return this.buf[this.top--];
  }
  peek() { return this.top < 0 ? undefined : this.buf[this.top]; }
  get empty() { return this.top < 0; }
  get size() { return this.top + 1; }
}
```

Tested with capacity 3: pushing 7, 8, 9, 10 prints `true true true false` — the fourth push is refused. `peek` then gives 9, size 3. Popping everything prints `9 8 7`, and one more `pop` returns `undefined` with `empty true`.

SHORTCUT

In a real round just write `const st = []` and use `st.push(v)`, `st.pop()` and `st[st.length - 1]`. Write the fixed-capacity class above only when the interviewer asks you to implement a stack, or when the question genuinely has a capacity limit. A plain array grows on its own and is the faster of the two.

TRAP

`empty` and `size` above are **getters**, so you write `s.empty` and `s.size`, with no brackets. Writing `s.empty()` throws `TypeError: s.empty is not a function`. Pick one style per class and stay with it.

Example 2

Reverse a string using a stack. Edge cases: empty string, one character.

SOLUTION

Push every character, then pop them all. Last in, first out **is** reversal.

Reverse with a stack

JS

```
const reverseWithStack = (s) => {
  const st = [];
  for (const c of s) st.push(c);
  let out = '';
  while (st.length) out += st.pop();
  return out;
};
```

Tested: "stack" gives kcats; "" gives the empty string; "a" gives a; "aa bb" gives bb aa.

Time $O(n)$ **Space** $O(n)$

Example 3

Only round brackets. Is the string balanced? Edge cases: empty, ") (", unclosed.

SOLUTION

With **one** kind of bracket you do not need a stack at all. A counter is enough — but it must never go below zero, otherwise a) arrived with nothing to close.

One bracket kind: a counter

JS

```
const balancedRound = (s) => {
  let open = 0;
  for (const c of s) {
    if (c === '(') open++;
    else if (c === ')') { open--; if (open < 0) return false; }
  }
  return open === 0;
};
```

Tested: "(()())" → true; "(()" → false; ")(" → false; "" → true; "(" → false.

The if (open < 0) return false; line is the whole difficulty. Without it, ")(" ends at open == 0 and you wrongly report balanced.

Example 4

For every index, find the nearest value to its **left** that is strictly greater. Write the obvious $O(n^2)$ version first — Tier 1 makes it $O(n)$. Edge cases: empty, all equal, decreasing.

SOLUTION

From each index walk left until something bigger shows up.

Previous greater — brute force

JS

```
const prevGreaterBrute = (a) => {
  const n = a.length;
  const out = new Array(n).fill(-1);
  for (let i = 0; i < n; i++)
    for (let j = i - 1; j >= 0; j--)
```



```

    if (a[j] > a[i]) { out[i] = a[j]; break; }
    return out;
};

```

Tested: [2,5,3,8,1] gives [-1,-1,5,-1,8]; [] gives []; [4] gives [-1]; [3,3,3] gives [-1,-1,-1] (strictly greater, so equals do not count); [-1,-5] gives [-1,-1].

Time $O(n^2)$ **Space** $O(1)$ extra

Example 5

Repeatedly delete any two **neighbouring** equal characters, until none are left. Edge cases: empty, one character, everything cancels.

SOLUTION

Use an array of characters as a stack. If the new character equals the top, they cancel: pop instead of push. Join the survivors at the end.

Cancel adjacent equal pairs

JS

```

const crushAdjacent = (s) => {
  const st = []; // an array of characters, used as a stack
  for (const c of s) {
    if (st.length && st[st.length - 1] === c) st.pop();
    else st.push(c);
  }
  return st.join('');
};

```

Tested: "abbaca" gives ca; "aabb" gives the empty string; "" gives empty; "x" gives x; "abccba" gives empty.

TRAP

Do **not** try to use a JavaScript string as the stack. Strings are immutable: there is no pop, and `s[i] = 'x'` silently does nothing. Push characters into an array and `join('')` at the end. Building the answer with `out += c` in a loop is fine for short strings, but for 10^5 characters the array-then-join version is the one to write.

Trace "abbaca": push a, push b, b matches so pop → a, a matches so pop → empty, push c, push a → ca. The cancellation **cascades**, and the stack handles that for free.

Example 6

A stack that also reports its minimum in $O(1)$. Edge cases: duplicates of the minimum, popping the minimum, negative values.

SOLUTION

Keep a second array the same height as the first. Slot `i` of it holds the minimum of the first `i + 1` items. Then `getMin` is just the last slot of mins.

Min stack with a parallel stack

JS

```

class MinStack {
  constructor() { this.main = []; this.mins = []; }

  push(v) {
    this.main.push(v);
    const top = this.mins[this.mins.length - 1];

```

```

    this.mins.push(this.mins.length === 0 || v <= top ? v : top); // <= keeps
duplicates safe
}
pop() {
    if (this.mins.length === 0) return undefined;
    this.mins.pop();
    return this.main.pop();
}
top() { return this.main[this.main.length - 1]; }
getMin() { return this.mins[this.mins.length - 1]; }
get empty() { return this.main.length === 0; }
}

```

Pushing 5, 2, 2, 9 reports minimum 5 2 2 2. Popping back reports 2 2 5. Popping an empty stack returns undefined instead of crashing. With -4 then -9, getMin is -9 and top is -9.

Time $O(1)$ per operation **Space** $O(n)$ extra

Tier 1 — the placement standards

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

For every element, find the next strictly greater element to its right. -1 if none. $n \leq 10^5$. Target $O(n)$. Edge cases: empty, one element, all equal, strictly decreasing, negatives.

SOLUTION

Approach 1 — Look right from every index ($O(n^2)$ — 10^{10} steps at $n = 10^5$)

Approach 1 — brute force

JS

```

const ngeBrute = (a) => {
    const n = a.length;
    const out = new Array(n).fill(-1);
    for (let i = 0; i < n; i++)
        for (let j = i + 1; j < n; j++)
            if (a[j] > a[i]) { out[i] = a[j]; break; }
    return out;
};

```

Approach 2 — Monotonic stack, walking right to left ($O(n)$ — optimal)

Walk backwards. The stack holds the values to the right that could still be somebody's answer. Before pushing $a[i]$, pop everything $\leq a[i]$: those values are both smaller **and** further away, so nothing on the left will ever choose them over $a[i]$.

Approach 2 — monotonic stack

JS

```

const ngeStack = (a) => {
    const n = a.length;
    const out = new Array(n).fill(-1);
    const st = []; // values, decreasing bottom -> top
}

```

```

    for (let i = n - 1; i >= 0; i--) {
        while (st.length && st[st.length - 1] <= a[i]) st.pop(); // too small, never
        useful again
        if (st.length) out[i] = st[st.length - 1];
        st.push(a[i]);
    }
    return out;
};

```

Both agree on every test:

input	output
[4,1,2,6,3]	[6,2,6,-1,-1]
[]	[]
[5]	[-1]
[3,3,3]	[-1,-1,-1]
[9,8,7]	[-1,-1,-1]
[-5,-2,-9]	[-2,-1,-1]

Time $O(n)$ — each element is pushed once and popped at most once **Space** $O(n)$

The idea that unlocked it: the inner loop kept re-reading the same elements. The stack **remembers** them, already filtered down to the ones that can still win.

Example 8 [Infosys · pattern]

Nearest strictly smaller element to the **left**, for every index. -1 if none. $n \leq 10^5$. Edge cases: empty, all equal, increasing, negatives.

SOLUTION

Mirror image of Example 7: walk **forwards**, and pop while the top is $\geq$ the current value. Now the stack is increasing from bottom to top.

Previous smaller element

JS

```

const prevSmaller = (a) => {
    const n = a.length;
    const out = new Array(n).fill(-1);
    const st = []; // values, increasing bottom -> top
    for (let i = 0; i < n; i++) {
        while (st.length && st[st.length - 1] >= a[i]) st.pop();
        if (st.length) out[i] = st[st.length - 1];
        st.push(a[i]);
    }
    return out;
};

```

Tested: [4,5,2,10,8] gives [-1,4,-1,2,2]; [] gives []; [7] gives [-1]; [2,2,2] gives [-1,-1,-1]; [-1,-3,0] gives [-1,-1,-3].

SHORTCUT

Memorise the four-way table from page one and stop re-deriving it: **direction of the walk** is opposite to the side you are asked about; **the pop test** uses the opposite comparison of the word in the question.

Example 9 [Wipro · pattern]

Three bracket kinds: `()`, `[]`, `{}`. Other characters may appear and are ignored. Is the string balanced? $n \leq 10^5$. Edge cases: empty, `"{[(())]}"`, an unmatched closer at the very start.

SOLUTION

With more than one kind, a counter no longer works, because the **order** of closing matters. Push every opener. On a closer, the top must be the matching opener.

Three bracket kinds

JS

```
const balanced = (s) => {
  const st = [];
  const match = { ')': '(', ']': '[', '}': '{' }; // closer -> its opener
  for (const c of s) {
    if (c === '(' || c === '[' || c === '{') st.push(c);
    else if (c in match) {
      if (st.length === 0) return false; // closer with nothing open
      if (st.pop() !== match[c]) return false; // wrong kind
    }
    // any other character is ignored
  }
  return st.length === 0; // nothing may be left open
};
```

Tested: `"{[(())]}"` → true; `"{[(())]}"` → false; `""` → true; `"("` → false; `")"` → false; `"a(b[c]d)e"` → true.

TRAP

Three separate failures, all needed:

1. a closer when the stack is empty,
2. a closer whose top is the **wrong** opener,
3. a non-empty stack at the end.

Most wrong submissions check only the third.

Time $O(n)$ **Space** $O(n)$

Example 10 [Capgemini · pattern]

Evaluate a postfix (reverse Polish) expression. Tokens are separated by single spaces. Operators are `+` `-` `*` `/`. Numbers may be negative. Results can exceed 2^{31} . Edge cases: a single number, a product of two large numbers.

SOLUTION

Push numbers. On an operator, pop **two** values — the second pop is the **left** operand — apply, push the result back. At the end one value is left.

Postfix evaluation

JS

```
const evalPostfix = (expr) => {
  const st = [];
  for (const tok of expr.split(/\s+/).filter(t => t.length)) {
    if (tok.length === 1 && '+-*/'.includes(tok)) {
      const b = st.pop(); // RIGHT operand pops first
      const a = st.pop(); // LEFT operand
      if (tok === '+') st.push(a + b);
      else if (tok === '-') st.push(a - b);
    }
  }
  return st[0];
};
```

```

    else if (tok === '*') st.push(a * b);
    else st.push(Math.trunc(a / b)); // integer division: JS / gives 4.5
  } else {
    st.push(Number(tok));
  }
}
return st.pop();
};

```

Tested:

expression	meaning	result
3 4 +	$3 + 4$	7
5 1 2 + 4 * + 3 -	$5 + (1 + 2) \times 4 - 3$	14
7	just a number	7
-4 5 *	-4×5	-20
9 2 /	integer division	4
100000 100000 *	10^{10}	10000000000

TRAP

Order of the two pops. For - and / it changes the answer. The **first** pop is the right-hand operand. Test with 9 2 /: if you get 0 you swapped them.

TRAP

JavaScript division is not integer division. 9 / 2 is 4.5, not 4. Wrap it in Math.trunc for the toward-zero answer that C-style languages give: Math.trunc(-9 / 2) is -4, while Math.floor(-9 / 2) is -5. Tested: with Math.trunc, 9 2 / gives 4 and -9 2 / gives -4.

There is no separate wide integer type in JavaScript, but there is still a limit. Every JavaScript number is a double, exact only up to Number.MAX_SAFE_INTEGER = 9007199254740991, about 9×10^{15} . Tested: 100000 100000 * gives 100000000000 exactly, because 10^{10} is far below that line. A product that can pass 9×10^{15} needs BigInt — you will see exactly that in Example 18.

Example 11 [Accenture · pattern]

A stack with $O(1)$ getMin, but use as little extra memory as you can. Edge cases: duplicate minimums, popping the minimum, single element.

SOLUTION

Approach 1 — Parallel stack of running minimums (O(n) extra, always n slots)

Approach 1 — one helper slot per item

JS

```

class MinStackPair {
  constructor() { this.a = []; this.m = []; }
  push(v) { this.a.push(v); this.m.push(this.m.length ? Math.min(this.m[this.m.length - 1], v) : v); }
  pop() { this.m.pop(); return this.a.pop(); }
  top() { return this.a[this.a.length - 1]; }
  getMin() { return this.m[this.m.length - 1]; }
}

```

Approach 2 — Push a helper entry only when a NEW minimum arrives (O(n) worst case but usually far less)

Approach 2 — lazy helper stack

JS

```
class MinStackLazy {
  constructor() { this.a = []; this.m = []; }
  push(v) { this.a.push(v); if (!this.m.length || v <= this.m[this.m.length - 1])
    this.m.push(v); }
  pop() { const v = this.a.pop(); if (v === this.m[this.m.length - 1])
    this.m.pop(); return v; }
  top() { return this.a[this.a.length - 1]; }
  getMin() { return this.m[this.m.length - 1]; }
}
```

Both were run side by side on the same calls and reported the same minimum every time. Pushing 6, 3, 3, 8, 1, 9:

pushed	minimum (approach 1)	minimum (approach 2)
6	6	6
3	3	3
3	3	3
8	3	3
1	1	1
9	1	1

Then popping all the way back down:

top before the pop	minimum (approach 1)	minimum (approach 2)
9	1	1
1	1	1
8	3	3
3	3	3
3	3	3
6	6	6

TRAP

The comparison must be $v \leq \text{top}$, not $v < \text{top}$. Push 2, then 2, then pop once: with the strict $<$ the helper array holds only one 2, the pop removes it, and `getMin` then reads a stale older minimum — or undefined if the helper is now empty, which turns every later comparison into `false` with no error. Tested: `push(2) push(2) pop()` reports minimum 2, as it must.

Tier 3 shows a version with **no** second stack at all.

Example 12 [Cognizant · pattern]

Items 1, 2, ..., n are pushed in that order, with pops allowed at any moment. Given a claimed output order, is it producible? $n \leq 10^5$. Edge cases: empty, already-sorted output, fully reversed output, [3, 1, 2].

SOLUTION

Simulate. Whenever the stack top is the next wanted item, pop it. Otherwise push the next unused item. If neither is possible, the order is impossible.

```
const canProduce = (want) => {
  const n = want.length;
  const st = [];
  let next = 1, k = 0;
  while (k < n) {
    if (st.length && st[st.length - 1] === want[k]) { st.pop(); k++; }
    else if (next <= n) st.push(next++);
    else return false; // nothing left to push and the top is wrong
  }
  return true;
};
```

Tested: [2,1,3] → true; [3,1,2] → false; [] → true; [1] → true; [4,3,2,1] → true; [1,2,3,4] → true.

Why is [3, 1, 2] impossible? To pop 3 first, 1 and 2 must already be on the stack with 2 above 1. So after 3 comes out, the top is 2 — you cannot reach 1 first.

Time $O(n)$ — each item is pushed once and popped once **Space** $O(n)$

Example 13 [TCS Digital · pattern]

Decode a nested repeat string. 3[ab2[c]] means ab followed by c twice, all of that three times. Counts can be multi-digit. Nesting depth up to 30. Edge cases: no brackets at all, a zero repeat count, a two-digit count, empty input.

SOLUTION

Two stacks: one of repeat counts, one of the text built **before** each [. On [, save the count and the text so far, then start fresh. On], pop the count and the saved text, and append the current text that many times.

```
const decode = (s) => {
  const textStack = [], numStack = [];
  let cur = '', num = 0;
  for (const c of s) {
    if (c >= '0' && c <= '9') {
      num = num * 10 + Number(c); // counts can be multi-digit
    } else if (c === '[') {
      numStack.push(num); num = 0; // reset num right after pushing it
      textStack.push(cur); cur = '';
    } else if (c === ']') {
      const k = numStack.pop();
      cur = textStack.pop() + cur.repeat(k);
    } else {
      cur += c;
    }
  }
  return cur;
};
```

Tested:

input	output
3[ab2[c]]	abccabccabcc

2[xy]	xyxy
plain	plain
(empty)	(empty)
0[gone]tail	tail
12[a]	aaaaaaaaaaaa

TRAP

`num = num * 10 + (c - '0')` and resetting `num = 0` right after pushing it. Forget the reset and `2[a]3[b]` reads the second count as 23.

Example 14 [Wipro · pattern]

Delete every run of exactly k equal neighbours, repeatedly, until no such run exists. $n \leq 10^5$, $2 \leq k \leq 10^4$. Edge cases: empty, one character, a run that becomes k only after an inner run vanishes.

SOLUTION

Store **pairs** on the stack: the character and how many of it are stacked in a row. When a count reaches k , drop the whole entry. Cascading is automatic — after dropping, the new top may merge with the next incoming character.

Stack of (character, run length)

JS

```
const crushRuns = (s, k) => {
  const st = []; // [character, how many in a row]
  for (const c of s) {
    if (st.length && st[st.length - 1][0] === c) {
      if (++st[st.length - 1][1] === k) st.pop();
    } else {
      st.push([c, 1]);
    }
  }
  return st.map(([ch, n]) => ch.repeat(n)).join('');
};
```

Tested: ("deeedbbcccbdaa", 3) gives aa; ("pbbcggttcciippooaais", 2) gives ps; ("", 2) gives empty; ("a", 2) gives a; ("aaaa", 2) gives empty.

Trace "deeedbbcccbdaa" with $k = 3$: eee goes, leaving d d bb ccc b d aa; ccc goes, so the two bs meet and become bbb, which goes; then ddd forms and goes; only aa is left. One pass of the stack does all of that.

Time $O(n)$ **Space** $O(n)$

Example 15 [Infosys · pattern]

The **span** of day i is how many days back, counting today, the price was less than or equal to today's price. $n \leq 10^5$. Target $O(n)$. Edge cases: empty, one day, all equal, strictly increasing.

SOLUTION

Approach 1 — Walk backwards from each day ($O(n^2)$)

Approach 1 — brute force

JS

```
const spanBrute = (p) => {
  const n = p.length;
```



```
const s = new Array(n).fill(1);
for (let i = 0; i < n; i++) {
  let j = i - 1;
  while (j >= 0 && p[j] <= p[i]) { s[i]++; j--; }
}
return s;
};
```

Approach 2 — Monotonic stack of INDICES (O(n) — optimal)

The span ends at the nearest day on the left with a **strictly greater** price. So find that day's index with a stack and subtract.

Approach 2 — stack of indices

JS

```
const spanStack = (p) => {
  const n = p.length;
  const s = new Array(n).fill(0);
  const st = []; // indices, prices decreasing
  for (let i = 0; i < n; i++) {
    while (st.length && p[st[st.length - 1]] <= p[i]) st.pop();
    s[i] = st.length === 0 ? i + 1 : i - st[st.length - 1];
    st.push(i);
  }
  return s;
};
```

Both agree: [100,80,60,70,60,75,85] gives [1,1,1,2,1,4,6]; [] gives []; [5] gives [1]; [4,4,4] gives [1,2,3]; [1,2,3] gives [1,2,3].

TRAP

`st.length === 0 ? i + 1 : i - st[st.length - 1]`. When nothing on the left is bigger, the span reaches back to day 0, which is `i + 1` days **including today**, not `i`. Test [1,2,3]: the answer must be [1,2,3].

The idea that unlocked it: this is “previous greater element”, but you keep the **index** instead of the value, because the question asks for a distance.

Tier 2 — applied, two or three steps

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 16 [Grab · pattern]

Bars of width 1 stand side by side, bar `i` has height `h[i]`. Find the largest rectangle that fits entirely inside them. $n \leq 10^5$, heights up to 10^5 . The answer can exceed 2^{31} . Edge cases: empty, one bar, all equal, strictly increasing, all zeros.

SOLUTION

Approach 1 — Fix the left edge, walk right, track the minimum height (O(n^2))

Approach 1 — brute force

JS

```
const histBrute = (h) => {
  let best = 0;
  const n = h.length;
  for (let i = 0; i < n; i++) {
    let lo = Infinity;
    for (let j = i; j < n; j++) {
      lo = Math.min(lo, h[j]);
      best = Math.max(best, lo * (j - i + 1)); // the shortest bar so far
    }
  }
  return best;
};
```

Approach 2 — Monotonic stack of indices ($O(n)$ — optimal)

Ask a different question: **for each bar, how wide can a rectangle of exactly that height be?** It stretches left until a shorter bar, and right until a shorter bar. A monotonic stack finds both boundaries in one pass.

Keep indices with **increasing** heights. When a shorter bar arrives at index i , every taller bar on the stack is finished: its rectangle ends just before i , and starts just after whatever is below it on the stack.

Approach 2 — one pass JS

```
const histStack = (h) => {
  const n = h.length;
  let best = 0;
  const st = []; // indices, heights increasing bottom -> top
  for (let i = 0; i <= n; i++) {
    const cur = i === n ? 0 : h[i]; // a virtual height-0 bar flushes the stack
    while (st.length && h[st[st.length - 1]] >= cur) {
      const height = h[st.pop()];
      const left = st.length ? st[st.length - 1] : -1;
      best = Math.max(best, height * (i - left - 1));
    }
    st.push(i);
  }
  return best;
};
```

Both agree:

heights	largest area
[2,1,5,6,2,3]	10
[]	0
[7]	7
[3,3,3]	9
[5,4,3,2,1]	9
[1,2,3,4,5]	9
[0,0]	0

With 100000 bars all of height 100000 the answer is 10000000000 — 10^{10} . That is still exact in JavaScript, because 10^{10} is far below `Number.MAX_SAFE_INTEGER = 9007199254740991`. Tested in Node: the value comes back as 10000000000, and `10000000000 <= Number.MAX_SAFE_INTEGER` is true.

TRAP

Three details decide whether this works.

1. The virtual bar `cur = 0` at `i == n`. Without it, bars still on the stack at the end are never measured. Test `[1,2,3,4,5]`.
2. `width = i - left - 1`, where `left` is the index **below** the popped one. It is not `i - popped`. The rectangle spreads left past every bar that was popped before it.
3. The multiply. `height * width` at $10^5 \times 10^5$ is 10^{10} , which JavaScript still holds exactly. Check the limit every time: if the product could pass 9×10^{15} you must switch to `BigInt`, as Example 18 does.

The full dry run of this algorithm is at the end of the chapter.

Time $O(n)$ **Space** $O(n)$

Example 17 [Shopee · pattern]

Rain falls on the same bars. How much water is trapped? $n \leq 10^5$. Edge cases: empty, one bar, strictly decreasing (no water), a flat pair.

SOLUTION

Approach 1 — For each column, scan both ways for the tallest wall ($O(n^2)$)

Water above column i is $\min(\text{tallest on the left, tallest on the right}) - h[i]$.

Approach 1 — brute force

JS

```
const rainBrute = (h) => {
  const n = h.length;
  let total = 0;
  for (let i = 0; i < n; i++) {
    let L = 0, R = 0;
    for (let j = 0; j <= i; j++) L = Math.max(L, h[j]);
    for (let j = i; j < n; j++) R = Math.max(R, h[j]);
    total += Math.min(L, R) - h[i];
  }
  return total;
};
```

Approach 2 — Precompute both wall arrays ($O(n)$ time, $O(n)$ memory)

Approach 2 — two prefix passes

JS

```
const rainPrefix = (h) => {
  const n = h.length;
  if (n === 0) return 0;
  const L = new Array(n), R = new Array(n);
  L[0] = h[0];
  for (let i = 1; i < n; i++) L[i] = Math.max(L[i - 1], h[i]);
  R[n - 1] = h[n - 1];
  for (let i = n - 2; i >= 0; i--) R[i] = Math.max(R[i + 1], h[i]);
  let total = 0;
  for (let i = 0; i < n; i++) total += Math.min(L[i], R[i]) - h[i];
  return total;
};
```

Approach 3 — Monotonic stack: fill one flat basin per pop ($O(n)$, single pass)

Keep indices with **decreasing** heights. When a taller bar arrives, the bar on top of the stack is the floor of a basin: the left wall is the next item down, the right wall is the new bar.

Approach 3 — stack, layer by layer

JS

```
const rainStack = (h) => {
  const st = []; // indices, heights decreasing
  let total = 0;
  for (let i = 0; i < h.length; i++) {
    while (st.length && h[st[st.length - 1]] < h[i]) {
      const floorIdx = st.pop();
      if (st.length === 0) break; // no left wall, no basin
      const leftIdx = st[st.length - 1];
      const width = i - leftIdx - 1;
      const depth = Math.min(h[leftIdx], h[i]) - h[floorIdx];
      total += width * depth;
    }
    st.push(i);
  }
  return total;
};
```

All three agree:

heights	water
[0,2,0,3,1,0,1,3]	9
[]	0
[4]	0
[3,3]	0
[5,4,3,2,1]	0
[2,0,2]	2

The stack version fills water in **horizontal layers**, not columns. For [0,2,0,3,1,0,1,3] it first fills the small dip at index 2, then the wide dip from index 4 to 6, layer by layer as each wall is discovered.

TRAP

if (st.length === 0) break; after the pop. If nothing is below the popped bar, there is no left wall, so there is no basin — water would run off the edge. Miss this and st[st.length - 1] quietly evaluates to undefined; JavaScript does not crash, it just produces NaN and every later total is NaN. A silent wrong answer is worse than a crash, so write the guard.

Example 18 [GIC · pattern]

Sum min(subarray) over **every** contiguous subarray. Give the answer modulo $10^9 + 7$. $n \leq 3 \times 10^4$, values positive. Edge cases: empty, one element, all equal (the duplicate trap), sorted both ways.

SOLUTION

Approach 1 — Every subarray ($O(n^2)$ — fine for $n = 3 \times 10^4$? no, 9×10^8 steps)

Approach 1 — brute force (used only to check)

JS

```
const MOD = 1000000007;
```

```
const sumMinBrute = (a) => {
  let total = 0;
  const n = a.length;
  for (let i = 0; i < n; i++) {
    let lo = Infinity;
    for (let j = i; j < n; j++) { lo = Math.min(lo, a[j]); total = (total + lo) %
MOD; }
  }
  return total;
};
```

Approach 2 — Contribution counting with two monotonic stacks (O(n) — optimal)

Flip the question. Instead of “what is the minimum of this subarray?”, ask “how many subarrays have $a[i]$ as their minimum?” Then

$$\text{answer} = \sum_i a[i] \times (\text{count of subarrays where } a[i] \text{ is the minimum})$$

Let $\text{left}[i]$ be how many choices of start keep $a[i]$ the minimum, and $\text{right}[i]$ how many choices of end. The count is $\text{left}[i] * \text{right}[i]$.

TRAP

With equal values, “the minimum” is ambiguous and you double-count. Fix it by breaking ties in **one direction only**: use strictly-smaller on the left ($>$ in the pop test) and smaller-or-equal on the right ($>=$ in the pop test). Then for a block of equal values, each subarray is credited to exactly one of them.

Approach 2 — count each element's reign

JS

```
const sumMinStack = (a) => {
  const n = a.length;
  const left = new Array(n), right = new Array(n);
  let st = [];
  for (let i = 0; i < n; i++) {
    // previous STRICTLY smaller
    while (st.length && a[st[st.length - 1]] > a[i]) st.pop();
    left[i] = st.length ? i - st[st.length - 1] : i + 1;
    st.push(i);
  }
  st = [];
  for (let i = n - 1; i >= 0; i--) {
    // next smaller OR EQUAL
    while (st.length && a[st[st.length - 1]] >= a[i]) st.pop();
    right[i] = st.length ? st[st.length - 1] - i : n - i;
    st.push(i);
  }
  let total = 0n;
  const M = BigInt(MOD);
  for (let i = 0; i < n; i++)
    total = (total + BigInt(a[i]) * BigInt(left[i]) * BigInt(right[i])) % M;
  return Number(total);
};
```

Both agree:

array	sum of minima
[3,1,2,4]	17

[]	0
[5]	5
[2,2,2]	12
[1,2,3,4,5]	35
[4,3,2,1]	20

They were also checked against each other on 300 random arrays of length up to 8 with values 1..5, and every single one agreed.

TRAP

This is the chapter's BigInt problem. `left[i] * right[i]` can be as large as n^2 , and with $n = 3 \times 10^4$ that is 9×10^8 . Multiply by a value of 10^9 and one term reaches 9×10^{17} — far past `Number.MAX_SAFE_INTEGER = 9007199254740991  $\approx 9 \times 10^{15}$` . Tested in Node: `1e9 * 30000 * 30000 <= Number.MAX_SAFE_INTEGER` is false, so plain numbers would silently round and the modulo would be wrong with no warning.

BigInt fixes it exactly: write the literal `0n`, convert with `BigInt(x)`, and take `Number(total)` at the very end. You may never mix the two — `1n + 1` throws `TypeError: Cannot mix BigInt and other types`. Tested on 30000 copies of 10^9 : the answer is 849895028.

Hand check for [3,1,2,4]: the ten subarrays have minima 3, 1, 1, 1, 1, 1, 2, 2, 4, total 17. The formula gives $3 \times 1 \times 1 + 1 \times 2 \times 3 + 2 \times 1 \times 2 + 4 \times 1 \times 1 = 3 + 6 + 4 + 4 = 17$.

The idea that unlocked it: stop iterating over subarrays and start iterating over **elements**, asking how many subarrays each one owns. This “contribution” move reappears in Tier 3 and in the DP chapters.

Example 19 [Agoda · pattern]

Delete exactly k digits from a digit string so that the remaining number is the smallest possible. Keep the digits in their original order. Length up to 10^5 . Edge cases: the result starts with zeros, k equals the length, an already-increasing string.

SOLUTION

Greedy with a stack. Scan left to right. If the new digit is smaller than the digit on top and you still have deletions left, the top digit is wasting a high place value — remove it. Each removal makes the number strictly smaller, so removing as early as possible is always right.

Remove k digits, keep it smallest

JS

```
const removeKDigits = (num, k) => {
  const st = [];
  for (const c of num) {
    while (k > 0 && st.length && st[st.length - 1] > c) { st.pop(); k--; }
    st.push(c);
  }
  while (k > 0 && st.length) { st.pop(); k--; } // still digits to drop
  let i = 0;
  while (i + 1 < st.length && st[i] === '0') i++; // strip leading zeros
  const out = st.slice(i).join('');
  return out === '' ? '0' : out;
};
```

Tested:

input	k	result
1432219	3	1219

10200	1	200
10	2	0
112	1	11
9	0	9
54321	2	321

TRAP

Three endings, all needed.

1. The string is already increasing (112, 12345): no pop ever fires, so the leftover k must be removed **from the end**.
2. Leading zeros: 10200 with $k = 1$ leaves 0200, which must print as 200.
3. Everything removed: return "0", not the empty string.

Example 20 [Razer · pattern]

Trolleys move on one straight track. A positive value means it moves right, negative means left; the size is its mass. When a right-mover meets a left-mover, the lighter one is destroyed; equal masses destroy each other. Return the surviving trolleys in order. $n \leq 10^4$. Edge cases: empty, no collisions possible, a chain reaction, an exact tie.

SOLUTION

Only a right-mover on the stack followed by a left-mover arriving can collide. That is exactly `st.back() > 0` && `cur < 0`. Loop, because destroying one survivor may expose another one behind it.

Collision simulation

JS

```
const collide = (t) => {
  const st = [];
  for (const cur of t) {
    let alive = true;
    while (alive && st.length && st[st.length - 1] > 0 && cur < 0) {
      const top = st[st.length - 1];
      if (Math.abs(top) < Math.abs(cur)) st.pop();           // left wins, keep
going
      else if (Math.abs(top) === Math.abs(cur)) { st.pop(); alive = false; }
      else alive = false;                                   // right wins
    }
    if (alive) st.push(cur);
  }
  return st;
};
```

Tested:

input	survivors
[5,10,-5]	[5,10]
[8,-8]	[]
[10,2,-5]	[10]
[]	[]
[-2,-1,1,2]	[-2,-1,1,2]
[1,-1,1,-1]	[]

`[-2, -1, 1, 2]` has no collisions at all: the left-movers are already to the left of the right-movers, so they move apart forever. The stack condition `st.back() > 0 && cur < 0` catches this for free.

TRAP

The alive flag. When the arriving trolley is destroyed you must **stop the loop and not push it**. A break alone is not enough; you also need to skip the push.

Example 21 [Sea / Shopee · pattern]

Next greater element, but the array is **circular**: after the last element comes the first again. $n \leq 10^5$. Edge cases: empty, one element, all equal, strictly decreasing.

SOLUTION

Walk right to left **twice** over the same array. The first lap only fills the stack with the wrap-around candidates; the second lap records answers. No copy of the array is needed — just index with `% n`.

Circular next greater

JS

```
const ngeCircular = (a) => {
  const n = a.length;
  const out = new Array(n).fill(-1);
  const st = []; // indices
  for (let pass = 0; pass < 2 * n; pass++) {
    const i = (2 * n - 1 - pass) % n;
    while (st.length && a[st[st.length - 1]] <= a[i]) st.pop();
    if (pass >= n && st.length) out[i] = a[st[st.length - 1]];
    st.push(i);
  }
  return out;
};
```

Checked against a brute force that tries every offset. They agree:

input	output
[3,8,4,1,2]	[8,-1,8,2,3]
[]	[]
[6]	[-1]
[2,2,2]	[-1,-1,-1]
[5,4,3,2,1]	[-1,5,5,5,5]
[-3,-1,-7]	[-1,-1,-3]

Read `[5, 4, 3, 2, 1]`: index 0 holds the largest value, so it has no greater element anywhere and stays `-1`. Every other index wraps around and finds the 5.

TRAP

`if (pass >= n ...)`. Record answers only on the second lap. Record on the first and indices near the end of the array get answers computed from an unfinished stack.

Tier 3 — needs an insight

Example 22 [Google · pattern]

A grid of 0s and 1s. Find the area of the largest all-1 rectangle. Rows and columns up to 200 each. Target $O(R \times C)$. Edge cases: empty grid, all zeros, all ones, a single cell.

The follow-up: “now the grid is a stream — rows arrive one at a time and you cannot store them all.”

SOLUTION

Read the grid row by row. For each row, build a histogram: column c holds **how many consecutive 1s end at this row** in that column. A 0 resets the column to 0. Then the answer for that row is “largest rectangle in a histogram” — Example 16, unchanged.

Row by row, histogram by histogram

JS

```
const largestRectangle = (h) => {                                     // exactly Example 16
  const n = h.length;
  let best = 0;
  const st = [];
  for (let i = 0; i <= n; i++) {
    const cur = i === n ? 0 : h[i];
    while (st.length && h[st[st.length - 1]] >= cur) {
      const height = h[st.pop()];
      const left = st.length ? st[st.length - 1] : -1;
      best = Math.max(best, height * (i - left - 1));
    }
    st.push(i);
  }
  return best;
};

const maximalRectangle = (g) => {
  if (g.length === 0 || g[0].length === 0) return 0;
  const cols = g[0].length;
  const h = new Array(cols).fill(0);
  let best = 0;
  for (const row of g) {
    for (let c = 0; c < cols; c++) h[c] = row[c] === 1 ? h[c] + 1 : 0;
    best = Math.max(best, largestRectangle(h));
  }
  return best;
};
```

Tested:

grid	largest area
[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]]	6
[]	0
[[0]]	0
[[1]]	1
[[0,0],[0,0]]	0
[[1,1],[1,1]]	4

The 6 comes from rows 1 and 2, columns 2 to 4: a 2×3 block of ones.

TRAP

This version only **reads** the grid, so it is safe. The moment a grid problem asks you to **change** the grid, remember that `[...grid]` is a **shallow** copy: the new outer array holds the very same row arrays, so writing into a row of the copy also writes into the original. Tested in Node: after `const shallow = [...g]; shallow[0][0] = 0;` the original `g[0][0]` is 0 too. The correct deep copy of a 2-D grid is `g.map(r => [...r])` — tested, and it leaves the original alone.

Time $O(R * C)$ **Space** $O(C)$

The follow-up: a stream of rows

Nothing changes. The algorithm already keeps only the current histogram `h` — one integer per column — and the running best. Rows are consumed and discarded. Memory is $O(C)$ whatever the number of rows, so a million-row stream is fine.

The idea that unlocked it: a 2-D problem turned into R copies of a 1-D problem you already solved. Look for this whenever a grid question smells like a solved array question.

Example 23 [Amazon · pattern]

Sum over every subarray of $(\max) - (\min)$. $n \leq 10^5$, values may be negative. Edge cases: empty, one element, all equal, alternating signs.

The follow-up: “why can you not do it with one stack pass?”

SOLUTION

$\sum(\max - \min) = \sum \max - \sum \min$. Each half is Example 18’s contribution counting, with the comparisons flipped for the max version.

Approach 1 — brute force (for checking)

JS

```
const rangesBrute = (a) => {
  let total = 0;
  const n = a.length;
  for (let i = 0; i < n; i++) {
    let lo = a[i], hi = a[i];
    for (let j = i; j < n; j++) { lo = Math.min(lo, a[j]); hi = Math.max(hi, a[j]);
    total += hi - lo; }
  }
  return total;
};
```

Approach 2 — two contribution passes

JS

```
const sumOf = (a, wantMax) => {
  const n = a.length;
  const left = new Array(n), right = new Array(n);
  let st = [];
  for (let i = 0; i < n; i++) {
    // strict on the left
    while (st.length && (wantMax ? a[st[st.length - 1]] < a[i] : a[st[st.length - 1]] > a[i])) st.pop();
    left[i] = st.length ? i - st[st.length - 1] : i + 1;
    st.push(i);
  }
}
```

```

st = [];
for (let i = n - 1; i >= 0; i--) { // non-strict on the right
  while (st.length && (wantMax ? a[st[st.length - 1]] <= a[i] : a[st[st.length - 1]] >= a[i])) st.pop();
  right[i] = st.length ? st[st.length - 1] - i : n - i;
  st.push(i);
}
let total = 0n; // BigInt: value * left * right
for (let i = 0; i < n; i++) total += BigInt(a[i]) * BigInt(left[i]) *
BigInt(right[i]);
return total;
};

const rangesStack = (a) => Number(sumOf(a, true) - sumOf(a, false));

```

Both agree: [1,2,3] gives 4; [] gives 0; [5] gives 0; [4,-2,-3,4,1] gives 59; [2,2,2] gives 0. They also agreed on 400 random arrays of length up to 8 with values from -4 to 4.

The follow-up answered

One pass cannot do it, because the max-stack and the min-stack pop at **different** moments — a new element may finish several maxima while finishing no minimum at all. You can run both stacks inside one for loop to save a traversal, but they are still two independent stacks. Two passes is the honest answer, and the complexity is the same.

Time $O(n)$ **Space** $O(n)$

Example 24 [Microsoft · pattern]

Longest balanced substring of a string of (and). $n \leq 10^5$. Edge cases: empty, no valid pair, "(()", ")()()".

The follow-up: “now do it in $O(1)$ extra space.”

SOLUTION

Approach 1 — Stack of indices with a sentinel ($O(n)$ time, $O(n)$ space)

Push -1 first. It marks **the last position that broke the balance**. On (, push the index. On), pop. If the stack is now empty, this) is itself a new breaking point, so push its index. Otherwise the valid run reaches from just after the new top to i, giving length $i - st[st.length - 1]$.

Approach 1 — stack with a sentinel

JS

```

const longestValidStack = (s) => {
  const st = [-1]; // sentinel: the last bad position
  let best = 0;
  for (let i = 0; i < s.length; i++) {
    if (s[i] === '(') st.push(i);
    else {
      st.pop();
      if (st.length === 0) st.push(i); // this ')' is the new breaking
point
      else best = Math.max(best, i - st[st.length - 1]);
    }
  }
  return best;
};

```

Approach 2 — Two counter sweeps ($O(n)$ time, $O(1)$ space — optimal)

Sweep left to right counting openers and closers. When they are equal, you have a valid run of length $2 * \text{close}$. When closers overtake openers, the run is dead — reset both to zero. That sweep alone misses runs like "(()", where openers **never** get overtaken, so sweep again from the right with the roles swapped.

Approach 2 — two counter sweeps

JS

```
const longestValidCounters = (s) => {
  let best = 0, open = 0, close = 0;
  for (const c of s) { // left to right
    if (c === '(') open++; else close++;
    if (open === close) best = Math.max(best, 2 * close);
    else if (close > open) { open = 0; close = 0; }
  }
  open = 0; close = 0;
  for (let i = s.length - 1; i >= 0; i--) { // right to left
    if (s[i] === '(') open++; else close++;
    if (open === close) best = Math.max(best, 2 * open);
    else if (open > close) { open = 0; close = 0; }
  }
  return best;
};
```

Both agree:

input	longest valid
"()())"	4
"(())"	2
""	0
"()"	2
"(((("	0
"()()"	2

They were also checked against each other on 500 random bracket strings of length up to 12, with no disagreement.

TRAP

The second sweep is not optional, and it is easy to convince yourself otherwise. The left-to-right sweep can only score a run when the closers **catch up** with the openers. Any string with extra openers on the left never lets them catch up, so that sweep scores nothing at all. Measured, with only the left sweep running:

input	left sweep alone	both sweeps (correct)
"(())"	0	2
"(((("	0	2
"()()"	0	2

The right-to-left sweep is the mirror image and catches exactly these cases.

Example 25 [Goldman Sachs · pattern]

A min-stack using **no** second stack — $O(1)$ extra space beyond the data itself. Edge cases: duplicate minimums, popping the minimum, a very large range of values.

The follow-up: “what breaks if the values are near the limits of the type?”

| SOLUTION

Store an **encoded** value when a new minimum arrives. If the new minimum is v and the old minimum was mn , push $2*v - mn$ instead of v . Because $v < mn$, that encoded number is **smaller than v** , so it is smaller than the current minimum too — which makes it recognisable later. On pop, if the popped number is below the current minimum, it is a marker: restore $mn = 2*mn - \text{popped}$.

Min stack with one extra variable

JS

```
class MinStackTiny {
  constructor() { this.st = []; this.mn = 0n; } // BigInt: 2*v - mn can be huge

  push(v) {
    const x = BigInt(v);
    if (this.st.length === 0) { this.st.push(x); this.mn = x; return; }
    if (x < this.mn) { this.st.push(2n * x - this.mn); this.mn = x; } // encoded
    marker
    else this.st.push(x);
  }
  pop() {
    if (this.st.length === 0) return;
    const t = this.st.pop();
    if (t < this.mn) this.mn = 2n * this.mn - t; // restore the previous minimum
  }
  top() {
    const t = this.st[this.st.length - 1];
    return t < this.mn ? this.mn : t; // a marker hides the real value
  }
  getMin() { return this.mn; }
  get empty() { return this.st.length === 0; }
}
```

Pushing 5, 3, 7, 2, 2, 9, -4 and reading top/getMin after each push gives

measured output

TEXT

5/5 3/3 7/3 2/2 2/2 9/2 -4/-4

and popping all the way back down reads the same values in reverse: -4/-4 9/2 2/2 2/2 7/3 3/3 5/5. Pushing -10^9 then 10^9 gives minimum $-1000000000n$ and top $1000000000n$ — the trailing n is how Node prints a BigInt.

Why does $2*v - mn$ work? Write it as $v - (mn - v)$. Since $v < mn$, the term $mn - v$ is positive, so the stored number is strictly below v . And recovering is just algebra: if the stored number is $t = 2v - mn$ and mn currently equals v , then $2 * mn - t = 2v - (2v - mn_{old}) = mn_{old}$.

The follow-up answered

$2*v - mn$ roughly **doubles** the magnitude of the number you store. With plain JavaScript numbers, once $2n * v - mn$ passes `Number.MAX_SAFE_INTEGER = 9007199254740991` the encoding is destroyed **silently** — no error, just a wrong minimum later. That is why the code above stores BigInt values: BigInt is exact at any size. Tested: pushing -2^{53} and then $-2^{53} - 4$ still reports the minimum $-9007199254740996n$ correctly.

The two honest answers for the interviewer: (a) use BigInt, paying roughly a $3 \times$ to $10 \times$ slowdown on arithmetic; (b) give up the trick and go back to a second stack, which is $O(n)$ memory but plain fast numbers. Say the trade-off out loud — the follow-up is asked precisely to hear it.

Example 26 [Adobe · pattern]

Given a lowercase string, keep exactly one copy of every distinct letter so that the result is the lexicographically **smallest** possible, while keeping the original relative order. $n \leq 10^5$. Edge cases: empty, one letter, all identical, already decreasing.

The follow-up: “now keep exactly k copies of every letter.”

SOLUTION

Greedy with a stack, plus one extra fact: the **last** index of each letter. Scan left to right. Skip a letter already in the stack. Otherwise pop the top while it is bigger than the current letter **and** it appears again later — because if it appears again later, you can safely drop it now and pick it up again.

Smallest subsequence with every letter once

JS

```
const smallestUniqueSubsequence = (s) => {
  const last = new Map();
  for (let i = 0; i < s.length; i++) last.set(s[i], i); // last index of each
  letter
  const inStack = new Set();
  const st = [];
  for (let i = 0; i < s.length; i++) {
    const c = s[i];
    if (inStack.has(c)) continue; // already placed
    while (st.length && st[st.length - 1] > c && last.get(st[st.length - 1]) > i) {
      inStack.delete(st.pop()); // it will come back later
    }
    st.push(c);
    inStack.add(c);
  }
  return st.join('');
};
```

Tested:

input	result
cbacdcabc	acdb
bcabc	abc
(empty)	(empty)
a	a
aaaa	a
edcba	edcba

Read edcba: nothing can be popped, because every letter appears exactly once, so `last[top] > i` is always false. The greedy is forced to keep the original order. That is the guard doing its job.

TRAP

Two conditions in the while, both required.

1. `st.back() > c` — only pop something that makes the result **bigger**.
2. `last[st.back()] > i` — only pop something you can still get back. Drop this and bcabc loses its b forever.

Plus the `inStack` flag, or the same letter appears twice.

The follow-up: keep k copies

Replace `inStack[c]` by a count `inStackCount[c] < k`, and replace `last[c] > i` by “there are still at least $k - \text{inStackCount}[c]$ copies of c after index i ”. You get that from a suffix-count array built in one backward pass. Same stack, same greedy, two arrays instead of two flags.

Example 27 [D. E. Shaw · pattern]

Prices arrive one at a time and never stop. After each price, report its span (the number of consecutive days back, including today, with price $\leq$ today’s). You may not store all the prices.

The follow-up: “now report the maximum over a sliding window of the last w days.”

SOLUTION

The monotonic stack already throws away prices it can never need. Store **pairs** (price, span already absorbed) so that when a block is swallowed you keep its width and throw away its contents.

Streaming span in $O(1)$ amortised

JS

```
class SpanStream {
  constructor() { this.st = []; }           // [price, span of that block]

  next(price) {
    let span = 1;
    while (this.st.length && this.st[this.st.length - 1][0] <= price) {
      span += this.st.pop()[1];             // swallow the whole block
    }
    this.st.push([price, span]);
    return span;
  }
  get memoryUsed() { return this.st.length; }
}
```

Tested:

prices fed	spans reported	slots kept at the end
100 80 60 70 60 75 85	1 1 1 2 1 4 6	2
5 5 5	1 2 3	1
9	1	1
9 8 7 6	1 1 1 1	4

Look at the last two rows. A rising stream collapses to almost nothing — after 100 80 60 70 60 75 85 only **two** slots remain. A strictly falling stream keeps everything, because no old price is ever beaten. So memory is $O(n)$ worst case but often tiny, and the **time** is $O(1)$ amortised either way.

The follow-up: maximum over the last w days

A stack cannot do it, because the element leaving the window is at the **bottom**, and a stack only removes from the top. You need removal at **both** ends — a **monotonic deque**. That is the next chapter. Recognising that a stack is the wrong shape here is exactly what the interviewer is testing.

Dry run — largest rectangle in a histogram

Heights $h = \{2, 1, 5, 6, 2, 3\}$. We run `histStack` from Example 16. The loop runs for i from 0 to 6; at $i = 6$ the virtual height is 0, which flushes everything.

The stack holds **indices**. Width for a popped index is $i - \text{left} - 1$, where `left` is the index left on the stack below it (or -1 if the stack is empty).

i	cur	pops (index, height, width, area)	stack after	best
0	2	none	[0]	0
1	1	idx 0, h=2, w=1, area=2	[1]	2
2	5	none	[1, 2]	2
3	6	none	[1, 2, 3]	2
4	2	idx 3, h=6, w=1, area=6; then idx 2, h=5, w=2, area=10	[1, 4]	10
5	3	none	[1, 4, 5]	10
6	0	idx 5, h=3, w=1, area=3; idx 4, h=2, w=4, area=8; idx 1, h=1, w=6, area=6	[6]	10

Answer: **10**.

Reading the interesting rows:

- **Row i = 1.** Height 1 arrives and is shorter than the 2 on the stack. So bar 0 is finished: it can spread right only as far as index 0. After popping, the stack is empty, so $\text{left} = -1$ and $\text{width} = 1 - (-1) - 1 = 1$. Area $2 \times 1 = 2$.
- **Row i = 4.** Height 2 arrives. Bar 3 (height 6) is finished: below it is index 2, so $\text{width} = 4 - 2 - 1 = 1$, area 6. Then bar 2 (height 5) is finished: below it is index 1, so $\text{width} = 4 - 1 - 1 = 2$, area 10. This is the winner — a rectangle of height 5 over columns 2 and 3. Then the top is index 1 with height 1, which is **not** ≥ 2 , so popping stops.
- **Row i = 6.** The virtual 0 bar. Bar 5 (height 3) pops with width 1. Bar 4 (height 2) pops: below it is index 1, so $\text{width} = 6 - 1 - 1 = 4$ — the rectangle of height 2 spreads over columns 2, 3, 4, 5, area 8. Finally bar 1 (height 1) pops with an empty stack below, so $\text{width} = 6 - (-1) - 1 = 6$: the full-width rectangle of height 1, area 6.

The winner is $5 \times 2 = 10$.

Every index is pushed exactly once (six pushes plus the virtual one) and popped at most once (six pops). Two nested loops, still $O(n)$. Say “amortised” when you explain this.

Python, for the three signature problems

The same three algorithms

PYTHON

```
def next_greater(a):
    n = len(a)
    out = [-1] * n
    st = []
    for i in range(n - 1, -1, -1):
        while st and st[-1] <= a[i]:
            st.pop()
        if st:
            out[i] = st[-1]
        st.append(a[i])
    return out

def largest_rectangle(h):
    best = 0
    st = []
    n = len(h)
    for i in range(n + 1):
        cur = 0 if i == n else h[i]
        while st and h[st[-1]] >= cur:
            height = h[st.pop()]
            best = max(best, (i - st[-1] - 1) * height)
```



```

        left = st[-1] if st else -1
        best = max(best, height * (i - left - 1))
        st.append(i)
    return best

def balanced(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    st = []
    for c in s:
        if c in "([{":
            st.append(c)
        elif c in pairs:
            if not st or st.pop() != pairs[c]:
                return False
    return not st

```

Run output:

output

TEXT

```

[6, 2, 6, -1, -1] [] [-1, -1, -1]
10 0 7 9
True False True False

```

TRAP

largest_rectangle indexes `h[st[-1]]` **before** popping, then pops. Writing `h[st.pop()]` inside the while condition would pop on every test, including the one that fails. Keep the test and the pop separate.

Python integers are exact at any size, so the `BigInt` question of Example 18 never comes up: `height * width` and `a[i] * left * right` are simply correct. The trade is speed. A Python `list` used as a stack is several times slower than a JavaScript array, so at $n = 10^5$ you are fine and at $n = 10^7$ you are not. Write JavaScript first and offer the Python only if the interviewer asks.

Practice

PRACTICE · TIER 1 target: 14 min

1. Fewest brackets you must **add** to make a `()` string balanced.
2. Next strictly smaller element to the right, for every index.
3. Sort a stack (largest on top) using only one helper stack.
4. Does an expression contain a bracket pair that wraps nothing useful — like `(a)` or `((a+b))`?
5. Standing at index `i` and looking right, how many buildings can you see? You see a building when it is taller than every building strictly between you and it.
6. Evaluate a **prefix** expression.
7. Simplify a folder path: `/a/./b/./././c/` becomes `/c`.

PRACTICE · TIER 2 target: 22 min

8. Remove the outer bracket of every top-level group: `((()))((()))` becomes `()()()`.
9. For each day, how many days until a strictly warmer day? 0 if it never comes.
10. Delete exactly `k` digits so the remaining number is the **largest** possible.

11. For each index, how many subarrays have $a[i]$ as their maximum?
12. For every window size w from 1 to n , report the largest of all window minimums.
13. Given the exact push order and a claimed pop order, is the pop order possible?
14. Score a balanced bracket string: $()$ scores 1, (X) scores $2 \cdot X$, XY scores $X + Y$.

ANSWER KEY

1. One counter for unmatched openers, one for unmatched closers.

1 — brackets to add

JS

```
const bracketsToAdd = (s) => {
  let open = 0, need = 0;
  for (const c of s) {
    if (c === '(') open++;
    else { if (open > 0) open--; else need++; } // a ')' with nothing to close
  }
  return need + open;
};
```

Tested: $"()" \rightarrow 1$; $"((((" \rightarrow 3$; $" " \rightarrow 0$; $"()" \rightarrow 0$; $"(" \rightarrow 2$. These are counts, not flags.

2. Example 7 with the pop test flipped.

2 — next smaller to the right

JS

```
const nextSmaller = (a) => {
  const n = a.length, out = new Array(n).fill(-1), st = [];
  for (let i = n - 1; i >= 0; i--) {
    while (st.length && st[st.length - 1] >= a[i]) st.pop();
    if (st.length) out[i] = st[st.length - 1];
    st.push(a[i]);
  }
  return out;
};
```

Tested: $[4, 8, 5, 2, 25] \rightarrow [2, 5, 2, -1, -1]$; $[] \rightarrow []$; $[3] \rightarrow [-1]$; $[2, 2, 2] \rightarrow [-1, -1, -1]$; $[-1, -4, 0] \rightarrow [-4, -1, -1]$.

3. Pull one value off the input. Push back anything on the output that is bigger, then place the value. It is insertion sort with stacks.

3 — sort a stack

JS

```
const sortStack = (input) => {
  const inSt = [...input], out = [];
  while (inSt.length) {
    const v = inSt.pop();
    while (out.length && out[out.length - 1] > v) inSt.push(out.pop());
    out.push(v);
  }
  return out; // the last slot is the largest
};
```

Tested: pushing 3, 1, 4, 1, 5 then sorting and popping prints 5 4 3 1 1. An empty stack stays empty. A single -7 comes back as -7. Cost is $O(n^2)$ in the worst case — that is expected for this puzzle.

4. A bracket pair is redundant when nothing between it and its opener was an operator.

4 — redundant brackets

JS

```
const hasRedundantBrackets = (s) => {
  const st = [];
  for (const c of s) {
    if (c === ')') {
      let sawOperator = false;
      while (st.length && st[st.length - 1] !== '(') {
        const t = st.pop();
        if (t === '+' || t === '-' || t === '*' || t === '/') sawOperator = true;
      }
      if (st.length) st.pop(); // drop the '('
      if (!sawOperator) return true;
    } else st.push(c);
  }
  return false;
};
```

Tested: "(a+b)" → false; "((a+b))" → true; "(a)" → true; "(a+(b*c))" → false; "a+b" → false; "" → false.

5. Walk right to left. Keep the “record holders” of the suffix, nearest on top. Its **size** is exactly the answer for the index just to the left.

5 — buildings visible to the right

JS

```
const visibleToRight = (h) => {
  const n = h.length, out = new Array(n).fill(0), st = [];
  for (let i = n - 1; i >= 0; i--) {
    out[i] = st.length; // records of
    h[i+1..]
    while (st.length && st[st.length - 1] <= h[i]) st.pop(); // h[i] hides them
    st.push(h[i]);
  }
  return out;
};
```

Tested against brute force: [3,1,4,2,5] → [3,2,2,1,0]; [] → []; [6] → [0]; [2,2,2] → [1,1,0]; [5,4,3] → [1,1,0]; [1,2,3] → [2,1,0]. Also agreed on 400 random arrays.

6. Prefix is postfix read backwards, with the operand order swapped.

6 — evaluate a prefix expression

JS

```
const evalPrefix = (expr) => {
  const tok = expr.split(/\s+/).filter(t => t.length);
  const st = [];
  for (let i = tok.length - 1; i >= 0; i--) {
    const x = tok[i];
    if (x.length === 1 && '+-*/'.includes(x)) {
      const a = st.pop(); // LEFT operand pops first here
      const b = st.pop();
      if (x === '+') st.push(a + b);
      else if (x === '-') st.push(a - b);
      else if (x === '*') st.push(a * b);
      else st.push(Math.trunc(a / b));
    } else st.push(Number(x));
  }
};
```

```

    return st.pop();
};

```

Tested: "+ 3 4" → 7; "- * 5 6 2" → 28; "8" → 8; "* -3 7" → -21; "* 100000 100000" → 10000000000.
 Note the pop order is the **opposite** of postfix: scanning backwards puts the left operand on top.

7. Split on /. Ignore empty pieces and .. On .. pop.

7 — simplify a path

JS

```

const simplifyPath = (p) => {
  const st = [];
  for (const part of p.split('/')) {
    if (part === '' || part === '..') continue;
    if (part === '.') st.pop(); // pop() on an empty array is a harmless
no-op
    else st.push(part);
  }
  return st.length ? '/' + st.join('/') : '/';
};

```

Tested: "/a/./b/../../c/" → /c; "/" → /; "/.." → /; "/home//docs///" → /home/docs; "/a/b/c" → /a/b/c.

8. You do not need a stack — a depth counter is enough. Emit a bracket only when the depth is above 1.

8 — remove outer brackets

JS

```

const removeOuter = (s) => {
  let out = '', depth = 0;
  for (const c of s) {
    if (c === '(') { if (depth > 0) out += c; depth++; }
    else { depth--; if (depth > 0) out += c; }
  }
  return out;
};

```

Tested: "((()()))()" → (()()); "()" → empty; "" → empty; "((()()))" → (()).

9. Monotonic stack of **indices**. When a warmer day arrives, every colder day still waiting gets its answer at once.

9 — days until warmer

JS

```

const daysUntilWarmer = (t) => {
  const n = t.length, out = new Array(n).fill(0), st = []; // indices
  for (let i = 0; i < n; i++) {
    while (st.length && t[st[st.length - 1]] < t[i]) { const j = st.pop(); out[j]
= i - j; }
    st.push(i);
  }
  return out;
};

```

Tested: [30,32,28,29,35] → [1,3,1,1,0]; [] → []; [40] → [0]; [20,20,20] → [0,0,0]; [50,40,30] → [0,0,0]. Indices left on the stack at the end keep their 0 — those days never warm up.

10. Example 19 with < instead of >, and no leading-zero problem (the largest number never starts with 0 unless everything is 0).

10 — keep the largest

JS

```
const keepLargest = (num, k) => {
  const st = [];
  for (const c of num) {
    while (k > 0 && st.length && st[st.length - 1] < c) { st.pop(); k--; }
    st.push(c);
  }
  while (k > 0 && st.length) { st.pop(); k--; }
  return st.length ? st.join('') : '0';
};
```

Tested: ("1432219", 3) → 4329; ("10200", 1) → 1200; ("10", 2) → 0; ("9", 0) → 9; ("12345", 2) → 345.

11. Contribution counting. Strict on one side, non-strict on the other, so ties are credited once.

11 — subarrays where a[i] is the maximum

JS

```
const countAsMax = (a) => {
  const n = a.length;
  const L = new Array(n), R = new Array(n);
  let st = [];
  for (let i = 0; i < n; i++) {
    while (st.length && a[st[st.length - 1]] < a[i]) st.pop(); // previous >=
    L[i] = st.length ? i - st[st.length - 1] : i + 1;
    st.push(i);
  }
  st = [];
  for (let i = n - 1; i >= 0; i--) {
    while (st.length && a[st[st.length - 1]] <= a[i]) st.pop(); // next strictly
    >
    R[i] = st.length ? st[st.length - 1] - i : n - i;
    st.push(i);
  }
  return L.map((l, i) => l * R[i]);
};
```

Tested, with the built-in check that the counts must add up to $n(n+1)/2$: [3,1,2] → [3,1,2], sum 6, expected 6. [] → [], sum 0. [7] → [1], sum 1. [2,2] → [2,1], sum 3. [1,2,3] → [1,2,3], sum 6.

That sum check is the best test you can write for any contribution problem: every subarray has exactly one owner, so the counts must total $n(n+1)/2$.

12. For each $a[i]$, find the widest window in which it is the minimum; call that length len . Then $a[i]$ is a candidate answer for size len . Fill sizes downwards at the end, because a value that is the best minimum for size len also beats anything for smaller sizes.

12 — max of window minimums, for every window size

JS

```
const maxOfMins = (a) => {
  const n = a.length;
  if (n === 0) return [];
  const L = new Array(n), R = new Array(n);
  let st = [];
  for (let i = 0; i < n; i++) {
    while (st.length && a[st[st.length - 1]] >= a[i]) st.pop();
    L[i] = st.length ? st[st.length - 1] : -1;
    st.push(i);
  }
}
```

```

st = [];
for (let i = n - 1; i >= 0; i--) {
  while (st.length && a[st.length - 1] >= a[i]) st.pop();
  R[i] = st.length ? st[st.length - 1] : n;
  st.push(i);
}
const best = new Array(n + 1).fill(-Infinity);
for (let i = 0; i < n; i++) {
  const len = R[i] - L[i] - 1; // biggest window where a[i] is the
min
  best[len] = Math.max(best[len], a[i]);
}
for (let w = n - 1; w >= 1; w--) best[w] = Math.max(best[w], best[w + 1]);
return best.slice(1);
};

```

Tested against brute force: [10,20,30,50,10,70,30] → [70,30,20,10,10,10,10]; [] → []; [4] → [4]; [3,3,3] → [3,3,3]; [5,4,3,2,1] → [5,4,3,2,1]. Also agreed on 300 random arrays.

13. Push in order, and pop greedily whenever the top matches the next wanted value.

13 — validate push/pop sequences

JS

```

const validSequences = (pushed, popped) => {
  if (pushed.length !== popped.length) return false;
  const st = [];
  let k = 0;
  for (const v of pushed) {
    st.push(v);
    while (st.length && k < popped.length && st[st.length - 1] === popped[k])
  { st.pop(); k++; }
  }
  return st.length === 0;
};

```

Tested: ([1,2,3,4,5], [4,5,3,2,1]) → true; ([1,2,3,4,5], [4,3,5,1,2]) → false; ([], []) → true; ([1], [1]) → true; ([1,2], [2,1]) → true; ([1,2], [1,2]) → true.

14. Keep a stack of **partial scores**, one per open bracket depth. An empty inner score means the pair was (), worth 1; otherwise double it.

14 — bracket score

JS

```

const bracketScore = (s) => {
  const st = [0]; // score of the current level
  for (const c of s) {
    if (c === '(') st.push(0);
    else {
      const inner = st.pop();
      st[st.length - 1] += inner === 0 ? 1 : 2 * inner;
    }
  }
  return st[st.length - 1];
};

```

Tested: "()" → 1; "()" → 2; "()" → 2; "()" → 6; "" → 0. Check the last real one by hand: (()()) is (A) with A = () + () = 1 + 2 = 3, so the whole thing is 2 × 3 = 6.

ONE-PAGE REVISION CARD

The reflex. “Nearest something, on one side” or “how far until” or “largest area” — that is a monotonic stack. Write the four-line skeleton before thinking about the problem.

Template — monotonic stack

```
JS
// `isUseless(topValue, currentValue)` is the only line that changes per problem.
const st = []; // push INDICES when you need a distance
for (let i = 0; i < n; i++) { // or i = n-1 down to 0
  while (st.length && isUseless(a[st[st.length - 1]], a[i])) st.pop();
  out[i] = st.length ? a[st[st.length - 1]] : NONE;
  st.push(i);
}
```

The four-way table

Want	Walk	Pop while top is
next greater (right)	backwards	$\leq a[i]$
next smaller (right)	backwards	$\geq a[i]$
previous greater (left)	forwards	$\leq a[i]$
previous smaller (left)	forwards	$\geq a[i]$

Contribution counting (sum over all subarrays of min or max)

```
JS
// left[i] = i - (index of previous STRICTLY smaller) // strict on ONE side only
// right[i] = (index of next smaller OR EQUAL) - i // non-strict on the other
// total += BigInt(a[i]) * BigInt(left[i]) * BigInt(right[i]); // BigInt: passes 2^53

// Run this self-check on every contribution problem. Every subarray has exactly
// one owner, so the counts must total n(n+1)/2. It catches a wrong `<` vs `<=`
// instantly, on inputs far too small to spot by hand.
const contributionSelfCheck = (left, right) => {
  const n = left.length;
  const owned = left.reduce((sum, l, i) => sum + l * right[i], 0);
  return owned === n * (n + 1) / 2;
};
```

Complexity of everything in this chapter

Problem	Brute	Optimal	Space
next greater / smaller	$O(n^2)$	$O(n)$	$O(n)$
stock span	$O(n^2)$	$O(n)$	$O(n)$
largest rectangle	$O(n^2)$	$O(n)$	$O(n)$
trapping rain water	$O(n^2)$	$O(n)$	$O(n)$ or $O(1)$

sum of subarray minima	$O(n^2)$	$O(n)$	$O(n)$
maximal rectangle in a grid	$O(R^2C^2)$	$O(RC)$	$O(C)$
longest valid brackets	$O(n^2)$	$O(n)$	$O(1)$ with counters
min stack	—	$O(1)$ per op	$O(1)$ encoded

Top traps

1. Wrong strictness ($<$ vs $<=$) in the pop test $\rightarrow$ duplicates counted twice.
2. Pushing values when you needed indices $\rightarrow$ you cannot compute a width.
3. Forgetting the virtual bar of height 0 at the end $\rightarrow$ leftovers never measured.
4. `width = i - popped` instead of `i - below - 1` $\rightarrow$ rectangles too narrow.
5. Letting a product pass 9×10^{15} $\rightarrow$ silently wrong; switch to `BigInt`.
6. Reading `st[st.length - 1]` without checking `st.length` first $\rightarrow$ undefined, then NaN, and no error message anywhere.
7. Postfix: popping the two operands in the wrong order for `-` and `/`.
8. Brackets: checking only that the stack is empty at the end.
9. `remove k digits`: forgetting leftover `k`, leading zeros, and the empty result.
10. Trying to solve a **sliding window maximum** with a stack — it needs a deque.

Queues, Deques & Monotonic Deque

A line you can only join at the back — until you learn to throw people out of it

Every JavaScript snippet in this chapter was executed with `node` before it was printed. Most of them start with this one line:

```
const { MinHeap, Deque } = require('./toolkit.js'); // see the JS Toolkit appendix
```

Deque is a plain array plus a head index, so removing from the front is $O(1)$ amortised; MinHeap is there because JavaScript ships no priority queue at all. The outputs shown under the snippets are the real outputs.

Pattern in one page

THE THREE TOOLS OF THIS CHAPTER

1. **Queue — FIFO (first in, first out).** You may only add at the back and remove from the front. This is a ticket line. Use it when order of arrival must be respected: breadth-first search, level-by-level processing, task pipelines, stream buffers.
2. **Deque — double-ended queue.** You may add and remove at **both** ends. JavaScript has nothing like it built in: `arr.push/arr.pop` are $O(1)$ but `arr.shift/arr.unshift` are not. The toolkit Deque gives you `push`, `pop`, `shift`, `front()` and `back()` in $O(1)$ amortised.
3. **Monotonic deque — the money-maker.** A deque of **indexes** whose values are kept sorted (always decreasing, or always increasing). It answers “what is the maximum (or minimum) of the current window?” in $O(1)$, and the whole sweep costs $O(n)$.

The JavaScript operations you need

queue and deque — the full menu

JS

```
const { Deque } = require('./toolkit.js');

const q = new Deque();
q.push(5); q.push(9); // add at the back
q.front();           // 5  read the front, do NOT remove
q.back();            // 9  read the back
q.shift();           // 5  remove the front AND return it
q.size;              // 1  a getter, not a method
q.size === 0;        // "is it empty?"

const d = new Deque();
d.push(5); d.push(6); // grow at the back
d.pop();             // 6  remove from the back and return it
d.front(); d.back();

const st = []; // a plain array is already a perfectly good STACK
st.push(1); st.pop(); // both O(1)
```

TRAP

Never build a queue out of `arr.shift()`. Removing the first element of a plain array has to slide every other element down, so it is $O(n)$ in the general case, and a BFS written that way quietly becomes $O(n^2)$. The toolkit

Deque is an array plus a head index: `shift()` only moves the index, and the array is compacted once half of it is dead. That is the $O(1)$ amortised every complexity claim in this chapter assumes.

TRAP

`q.size` is a **getter**, not a method. Writing `q.size()` throws `TypeError: q.size is not a function`. `front()`, `back()`, `push()`, `pop()` and `shift()` all **are** methods. Mixing the two up is the first error you will hit.

The monotonic deque template — learn this by heart

The window slides from left to right. The deque stores **indexes**, never values. Two rules keep it clean, and a third reads the answer.

sliding window MAXIMUM — the template

JS

```
const { Deque } = require('./toolkit.js');

const maxWindow = (a, k) => {
  const n = a.length, res = [];
  if (k <= 0 || k > n) return res;
  const dq = new Deque(); // holds INDEXES, values
  decreasing
  for (let i = 0; i < n; i++) {
    while (dq.size && dq.front() <= i - k) dq.shift(); // rule 1: drop expired
    while (dq.size && a[dq.back()] <= a[i]) dq.pop(); // rule 2: drop weaker
    dq.push(i);
    if (i >= k - 1) res.push(a[dq.front()]); // rule 3: front is the
  }
  return res;
};
```

Real output for `[1,3,-1,-3,5,3,6,7]`, `k=3`; `[4]`, `k=1`; `[2,2,2,2]`, `k=2`; `[-5,-2,-9,-1]`, `k=2`; then the empty array and `k > n`:

```
3 3 5 5 6 7
4
2 2 2
-2 -2 -1
[] []
```

Time $O(n)$ **Space** $O(k)$

Each index is pushed once and popped once. Two pushes never happen for one index, so the total work is linear even though there is a while-loop inside a for-loop.

The invariant — say it out loud

INVARIANT OF THE DECREASING DEQUE

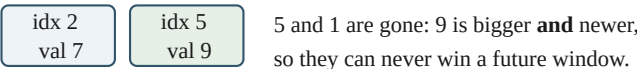
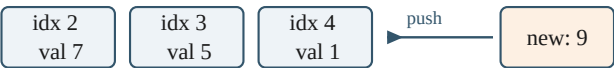
At the end of every loop step:

1. Every index in the deque is inside the current window.
2. The values at those indexes go **strictly down** from front to back.
3. `a[dq.front()]` is the maximum of the current window.

Why rule 2 is safe. Suppose $a[j] \leq a[i]$ and $j < i$. Then from now on, every window that contains j also contains i (because i is to the right of j and windows move right). A window can never prefer j over i : i is at least as big **and** lives longer. So j is useless forever. Throw it away.

SHORTCUT

Flip one comparison and you get the minimum instead: $a[\text{dq.back}()] \geq a[i]$ keeps values **increasing**, and $a[\text{dq.front}()]$ is the window **minimum**. Nothing else changes. Keep both versions in your head as one template with a switch.



One step of the decreasing deque. 9 arrives and eats every smaller value behind it.

When to reach for each tool

Signal in the question	Tool
“shortest number of moves”, “level by level”, “spread step by step”	plain Deque + BFS
“max / min of every window of size k”	monotonic deque
“longest window such that $\max - \min \leq L$ ”	two deques (one max, one min)
“shortest window with $\text{sum} \geq S$ ”, values may be negative	prefix sums + increasing deque
“ $\text{dp}[i]$ depends on the best of $\text{dp}[i-k..i-1]$ ”	deque over the dp array
“edges cost 0 or 1”	deque BFS (0-1 BFS)
“keep only the last k seconds of events”	deque of timestamps

Warm-up — build the reflex

WARM-UP

build the reflex

Example 1 [warm-up]

Build a queue on top of a fixed-size array. Support push, pop, front, empty, full. Capacity is 3. Constraints: capacity up to 10^5 ; every operation must be $O(1)$. Edge cases: pop from an empty queue, push into a full queue.

SOLUTION

A plain array queue that only moves head forward wastes space. Use a **circular** buffer: after the last cell, wrap back to cell 0. Store head and count; the tail is then $(\text{head} + \text{count}) \% \text{cap}$.

circular array queue

JS

```

class CircQueue {
  constructor(cap) { this.a = new Array(cap); this.head = 0; this.count = 0;
  this.cap = cap; }
  get empty() { return this.count === 0; }
  get full() { return this.count === this.cap; }
  push(x) { // add at the back
    if (this.full) return false;
    this.a[(this.head + this.count) % this.cap] = x;
    this.count++;
    return true;
  }
  pop() { // remove from the front
    if (this.empty) return false;
    this.head = (this.head + 1) % this.cap;
    this.count--;
    return true;
  }
  front() { return this.a[this.head]; } // caller must check .empty first
}

```

Test and real output:

```

test JS

const q = new CircQueue(3);
console.log(q.push(10), q.push(20), q.push(30), q.push(40)); // the 4th push
fails
console.log(q.front());
q.pop(); q.push(40);
console.log(q.front(), q.count);
const drained = [];
while (!q.empty) { drained.push(q.front()); q.pop(); }
console.log(drained.join(' '));
console.log(q.pop(), q.empty); // pop on empty
fails

```

```

true true true false
10
20 3
20 30 40
false true

```

The fourth push returned false because the queue was full. The last pop returned false because the queue was empty.

Time $O(1)$ per operation **Space** $O(c)$

TRAP

Do not test “full” with `head == tail`. When the buffer is full, `head == tail` as well — the same condition means both empty and full. Storing count removes the ambiguity.

Example 2 [warm-up]

Build a queue using only two stacks. Constraints: up to 10^5 operations. Edge cases: `empty()` on a brand-new object, interleaving pushes and pops.

| SOLUTION

Stack is LIFO, queue is FIFO. Pouring a stack into another stack reverses it, which turns LIFO into FIFO. The trick is to pour **only when the output stack runs dry**.

queue from two stacks (plain arrays)

JS

```
class QueueFromStacks {
  constructor() { this.in = []; this.out = []; }
  push(x) { this.in.push(x); }
  #shift() {                                     // move only when out is empty
    if (this.out.length === 0)
      while (this.in.length) this.out.push(this.in.pop());
  }
  front() { this.#shift(); return this.out[this.out.length - 1]; }
  pop() { this.#shift(); return this.out.pop(); }
  get empty() { return this.in.length === 0 && this.out.length === 0; }
  get size() { return this.in.length + this.out.length; }
}
```

Test: push 1, 2, 3; pop once; push 4; drain.

```
1 2 3 4
true 0
7 1
```

Line 1 is the drained order — perfect FIFO. Line 2 is `.empty` and `.size` of a fresh object. Line 3 pushes 7 into the drained queue and reads front and size.

Time amortised $O(1)$ **Space** $O(n)$

Each element is moved from in to out at most once in its whole life, so n operations cost $O(n)$ total.

TRAP

If you pour on **every** pop, the cost becomes $O(n)$ per operation. The `if (out.length === 0)` guard is the entire trick. Interviewers ask “what is the amortised cost?” right after this question — the answer is $O(1)$, and the reason is “each element crosses once”.

Example 3 [warm-up]

Print the first n binary strings: 1, 10, 11, 100, 101, ... using a queue. Constraints: $n \leq 10^5$. Edge cases: $n = 0$, $n = 1$.

SOLUTION

Start with “1” in the queue. Pop a string, print it, then push that string with “0” appended and with “1” appended. The queue keeps the numbers in increasing order for free.

binary strings with a queue

JS

```
const firstBinary = (n) => {
  const out = [];
  const q = new Deque();
  q.push('1');
  for (let i = 0; i < n; i++) {
    const s = q.shift();
    out.push(s);
    q.push(s + '0');
    q.push(s + '1');
  }
}
```

```
    return out;
};
```

firstBinary(7) prints:

1 10 11 100 101 110 111

firstBinary(0) has length 0. firstBinary(1) gives 1. The strings are built with `s + '0'`, which is exactly why this stays readable — no digit arithmetic anywhere.

This is breadth-first search on a binary tree where the left child appends 0 and the right child appends 1. Level d holds every binary string of length $d + 1$ that starts with 1. You will meet this exact shape again in the trees chapter.

Example 4 [TCS NQT pattern]

(a) Reverse a whole queue. (b) Reverse only the first k elements of a queue, leaving the rest in order. Constraints: size up to 10^5 , $1 \leq k \leq \text{size}$. Edge cases: empty queue, k equal to the size, $k = 1$.

SOLUTION

(a) A stack reverses. Pour the queue into a stack, pour it back.

(b) Pour the first k into a stack and push them back — they are now reversed, but they sit at the **back**. Then rotate the remaining $n - k$ elements from front to back, which slides the reversed block to the front.

reverse a queue, and reverse its first k

JS

```
const reverseQueue = (q) => {
    const st = []; // a plain array, used as a stack
    while (q.size) st.push(q.shift());
    while (st.length) q.push(st.pop());
};

const reverseFirstK = (q, k) => {
    if (k <= 0 || k > q.size) return;
    const st = [];
    for (let i = 0; i < k; i++) st.push(q.shift());
    while (st.length) q.push(st.pop());
    const rotate = q.size - k; // size is n again: rotate exactly n
    - k
    for (let i = 0; i < rotate; i++) q.push(q.shift());
};
```

Real output for the tests [1..5] reversed, [1..5] with $k = 3$, with $k = 5$, [9] with $k = 1$, and an empty queue:

```
5 4 3 2 1
3 2 1 4 5
5 4 3 2 1
9
0
```

TRAP

In reverseFirstK, `q.size` changes while you are pushing. Read `q.size - k` into a **const** **after** the block has been pushed back — at that moment the size is n again and the rotation count is exactly $n - k$. Putting `q.size - k` straight into the loop condition re-reads it every iteration, and the loop never ends. This is the most common bug in this question.

Example 5 [warm-up]

Interleave the two halves of a queue. 1 2 3 4 5 6 must become 1 4 2 5 3 6. Constraints: size up to 10^5 . Edge cases: odd size (reject), empty queue.

SOLUTION

Move the front half into a second queue. Then alternate: one from the helper, one from the main queue’s front (pushing it to the back).

interleave the halves JS

```
const interleave = (q) => {
  const n = q.size;
  if (n % 2 !== 0) return false; // needs an even count
  const first = new Deque(); // holds the front half
  for (let i = 0; i < n / 2; i++) first.push(q.shift());
  while (first.size) {
    q.push(first.shift()); // one from the front half
    q.push(q.shift()); // one from the back half
  }
  return true;
};
```

Real output (return value, then the queue):

true 1 4 2 5 3 6
true 11 13 12 14
false 1 2 3
true 0

The third line shows an odd-sized queue: the function returns false and leaves the queue untouched. The fourth line is the empty queue — 0 elements, treated as valid.

Time $O(n)$ Space $O(n/2)$

Example 6 [warm-up]

Warm up the monotonic idea by hand. Array $a = [4, 2, 7, 1, 9]$, window size $k = 3$. List every window and its maximum, then say which indexes would ever be useful.

SOLUTION

window	values	max
0..2	4, 2, 7	7
1..3	2, 7, 1	7
2..4	7, 1, 9	9

Index 1 (value 2) is **never** the answer: index 2 (value 7) is bigger and always outlives it. Index 0 (value 4) is also never the answer for the same reason. That is exactly rule 2 of the template.

Answer: maxima are 7, 7, 9; only indexes 2 and 4 are ever useful

Tier 1 — the standard window questions

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [Infosys · pattern]

Sliding window maximum. Given an array a of n integers and a window size k , output the maximum of every window of size k . Constraints: $n \leq 10^5$, values are whole numbers and may be negative, $1 \leq k \leq n$. Target: $O(n)$. Edge cases: $k = 1$ (answer is the array itself), $k = n$ (one number), all values equal, all values negative.

SOLUTION

Approach 1 — Check every window from scratch $O(nk)$ — too slow

For each starting position, scan k cells and keep the largest.

brute force

JS

```
const maxWindowBrute = (a, k) => {
  const n = a.length, res = [];
  if (k <= 0 || k > n) return res;
  for (let i = 0; i + k <= n; i++) {
    let best = a[i];
    for (let j = i; j < i + k; j++) best = Math.max(best, a[j]);
    res.push(best);
  }
  return res;
};
```

Real output for $[1, 3, -1, -3, 5, 3, 6, 7]$, $k=3$; $[4]$, $k=1$; $[2, 2, 2, 2]$, $k=2$; $[-5, -2, -9, -1]$, $k=2$; the empty array; and $k > n$:

3 3 5 5 6 7

4

2 2 2

-2 -2 -1

[]

[]

The last two are empty arrays on purpose — no window exists.

Time $O(nk)$ **Space** $O(1)$ output

With $n = 10^5$ and $k = 10^5$ this is 10^{10} steps. It will time out.

Approach 2 — A max-heap with lazy deletion $O(n \log n)$ — passes, but not the intended answer

This is the first place in the chapter where JavaScript leaves you empty-handed: there is no priority queue in the language, and no ordered multiset either. Use the toolkit `MinHeap` with the comparator flipped, which makes it a max-heap of $[\text{value}, \text{index}]$ pairs. You cannot pull an arbitrary element out of a heap, so you do not try — you discard the top only when it turns out to have left the window. That is **lazy deletion**.

max-heap window with lazy deletion

JS

```
const { MinHeap } = require('./toolkit.js'); // JS has no heap of its own

const maxWindowHeap = (a, k) => {
  const n = a.length, res = [];
  if (k <= 0 || k > n) return res;
  const h = new MinHeap((x, y) => y[0] - x[0]); // flip the comparator -> max-heap
  for (let i = 0; i < n; i++) {
    h.push([a[i], i]);
    while (h.peak()[1] <= i - k) h.pop(); // the top has left the window
    if (i >= k - 1) res.push(h.peak()[0]);
  }
}
```



```
    return res;
};
```

Real output for $[1, 3, -1, -3, 5, 3, 6, 7]$, $k=3$; $[7, 7, 7, 1]$, $k=3$; $[-5, -2, -9, -1]$, $k=2$; $[5]$, $k=1$; the empty array:

```
3 3 5 5 6 7
7 7
-2 -2 -1
5
[]
```

Time $O(n \log n)$ **Space** $O(n)$

The heap can hold every element that has ever arrived, because expired entries are only thrown away when they float to the top.

TRAP

JavaScript has no `TreeMap`, no ordered set and no multiset. When a problem wants “the sorted contents of the window”, your three options are: a heap with lazy deletion (this approach), a **sorted array** kept in order with `lowerBound` and `splice` (used for the median in Example 26), or — when you only need an extreme — a monotonic deque. Say which one you are choosing and why; “I would use a multiset” is not an answer in JavaScript.

TRAP

`new MinHeap()` with no argument is a **min**-heap on numbers. For a max-heap you must pass $(x, y) \Rightarrow y - x$, and for pairs $(x, y) \Rightarrow y[0] - x[0]$. Getting the sign backwards gives a perfectly plausible, perfectly wrong answer — the window **minimum** — which tiny test cases often fail to reveal.

Approach 3 — Monotonic deque ($O(n)$ — optimal)

Same template as page one.

optimal: decreasing deque of indexes

JS

```
const { Deque } = require('./toolkit.js');

const maxWindow = (a, k) => {
  const n = a.length, res = [];
  if (k <= 0 || k > n) return res;
  const dq = new Deque(); // holds INDEXES, values
  decreasing
  for (let i = 0; i < n; i++) {
    while (dq.size && dq.front() <= i - k) dq.shift(); // drop the expired index
    while (dq.size && a[dq.back()] <= a[i]) dq.pop(); // drop weaker candidates
    dq.push(i);
    if (i >= k - 1) res.push(a[dq.front()]);
  }
  return res;
};
```

Real output over seven tests (including a 2000-case random stress test against the brute force):

```
3 3 5 5 6 7
9 8 7
2 3 4
2 2
```

```
-2 -2 -1
4
[] []
stress ok
```

Time $O(n)$ **Space** $O(k)$

Each index enters the deque once and leaves once; at most $2n$ deque operations in total.

The idea that unlocked it: a value that is smaller than a newer value can never win a future window, so it can be deleted permanently. Once deletions are permanent, the total number of deletions is bounded by n , and the inner while-loop stops being a second loop.

The same code in Python:

sliding window maximum in Python

PYTHON

```
from collections import deque

def max_window(a, k):
    if k <= 0 or k > len(a):
        return []
    dq, res = deque(), []          # dq holds indexes, values decreasing
    for i, v in enumerate(a):
        while dq and dq[0] <= i - k:
            dq.popleft()
        while dq and a[dq[-1]] <= v:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            res.append(a[dq[0]])
    return res
```

Real output for $([1, 3, -1, -3, 5, 3, 6, 7], 3)$, then $([2, 2], 2)$, the empty list, and $([1], 5)$:

```
[3, 3, 5, 5, 6, 7]
[2] [] []
```

Python has `collections.deque` built in, with $O(1)$ `append`, `pop`, `appendleft` and `popleft`. That is the one place Python is genuinely shorter than JavaScript here — but the toolkit `Deque` gives you the same $O(1)$ behaviour, so the algorithm is identical.

TRAP

Store **indexes**, not values. If you store values you cannot tell whether the front has expired. Beginners store values, then bolt on a separate counter, then get it wrong when duplicates appear. Indexes make expiry a one-line comparison.

Example 8 [Wipro · pattern]

Sliding window minimum. Same input, but report the minimum of each window. Constraints and edge cases as above.

SOLUTION

Flip two comparison signs. Nothing else.

increasing deque

JS

```
const minWindow = (a, k) => {
    const n = a.length, res = [];
```

```

if (k <= 0 || k > n) return res;
const dq = new Deque(); // values INCREASING
for (let i = 0; i < n; i++) {
  while (dq.size && dq.front() <= i - k) dq.shift();
  while (dq.size && a[dq.back()] >= a[i]) dq.pop();
  dq.push(i);
  if (i >= k - 1) res.push(a[dq.front()]);
}
return res;
};

```

Real output for [1,3,-1,-3,5,3,6,7], k=3; [5,5,5], k=2; [-1,-2,-3], k=1; empty:

```

-1 -3 -3 -3 3 3
5 5
-1 -2 -3
[]

```

Time $O(n)$ **Space** $O(k)$

Example 9 [Capgemini · pattern]

First negative number in every window of size k . If a window has no negative number, report 0. Constraints: $n \leq 10^5$. Edge cases: no negatives at all, all negatives, $k = 1$.

SOLUTION

Approach 1 — Scan each window ($O(n \cdot k)$)

For each window, walk left to right and stop at the first negative. Correct, too slow for $n = 10^5$ with a large k .

Approach 2 — A queue of negative indexes only ($O(n)$ — optimal)

The answer is the **oldest** negative still inside the window. That is a plain FIFO queue of indexes — no monotonic rule is needed, because “oldest” is exactly what FIFO gives.

first negative in each window

JS

```

const firstNegative = (a, k) => {
  const n = a.length, res = [];
  if (k <= 0 || k > n) return res;
  const dq = new Deque(); // indexes of negative
  numbers only
  for (let i = 0; i < n; i++) {
    if (a[i] < 0) dq.push(i);
    while (dq.size && dq.front() <= i - k) dq.shift();
    if (i >= k - 1) res.push(dq.size ? a[dq.front()] : 0);
  }
  return res;
};

```

Real output for [12,-1,-7,8,-15,30,16,28], k=3; [1,2,3,4], k=2; [-1,-2,-3], k=3; [5], k=1; empty:

```

-1 -1 -7 -15 -15 0
0 0 0
-1
0
[]

```

Time $O(n)$ **Space** $O(k)$

The idea that unlocked it: the answer is a **position** question, not a **value** question. When the answer is “the oldest thing that still qualifies”, a plain queue is enough — you do not need the monotonic rule at all.

Example 10 [TCS Digital · pattern]

Sum of window maximum plus window minimum, over all windows of size k . Constraints: $n \leq 10^5$, $|a_i| \leq 10^9$. Target: $O(n)$. Edge cases: the total can exceed 32 bits; $k = n$; empty array.

SOLUTION

Run **two** dequeues at the same time over the same loop: one decreasing (max) and one increasing (min).

two dequeues, one pass

JS

```
const sumMinPlusMax = (a, k) => {
  const n = a.length;
  if (k <= 0 || k > n) return 0;
  const mx = new Deque(), mn = new Deque(); // both hold INDEXES
  let total = 0;
  for (let i = 0; i < n; i++) {
    while (mx.size && mx.front() <= i - k) mx.shift();
    while (mn.size && mn.front() <= i - k) mn.shift();
    while (mx.size && a[mx.back()] <= a[i]) mx.pop();
    while (mn.size && a[mn.back()] >= a[i]) mn.pop();
    mx.push(i); mn.push(i);
    if (i >= k - 1) total += a[mx.front()] + a[mn.front()];
  }
  return total;
};
```

Real output for $([2, 5, -1, 7, -3, -1, -2], 4)$; $([1, 1, 1], 3)$; $([2000000000, 2000000000], 2)$; the empty array; $([4], 1)$:

```
18
2
4000000000
0
8
```

Time $O(n)$ **Space** $O(k)$

The third test is the “would this have overflowed?” test. In a 32-bit language $2 \times 10^9 + 2 \times 10^9$ overflows and you need a wider type. JavaScript has no 32-bit integers to overflow: every number is a double, exact up to `Number.MAX_SAFE_INTEGER = 9007199254740991`. Tested: the answer comes back as `4000000000`, and `4000000000 <= Number.MAX_SAFE_INTEGER` is true.

Check the worst case anyway, every time. Here the running total is at most 10^5 windows $\times 2 \times 10^9 = 2 \times 10^{14}$ — still under 9×10^{15} , so plain numbers are safe. Tested: `1e5 * 2e9 <= Number.MAX_SAFE_INTEGER` is true. Push k or the values one order of magnitude higher and the answer would change to `BigInt`.

Example 11 [Accenture · pattern]

Count of distinct values in every window of size k . Constraints: $n \leq 10^5$, values up to 10^9 . Edge cases: all values the same, negative values, $k = 1$.

SOLUTION

Approach 1 — Build a set per window ($O(nk)$)

Build a new `Set(a.slice(i, i + k))` for every window and read its `.size`.

Approach 2 — One hash map that slides ($O(n)$ average — optimal)

Keep a Map of counts for the current window. When a count drops to zero, **delete the key**, so that `cnt.size` is the answer directly.

distinct per window

JS

```
const distinctInWindow = (a, k) => {
  const n = a.length, res = [];
  if (k <= 0 || k > n) return res;
  const cnt = new Map();
  for (let i = 0; i < n; i++) {
    cnt.set(a[i], (cnt.get(a[i]) || 0) + 1); // new element joins
    if (i >= k) {
      const old = a[i - k];
      const c = cnt.get(old) - 1;
      if (c === 0) cnt.delete(old); // delete, so .size is the
    }
    answer
    else cnt.set(old, c);
  }
  if (i >= k - 1) res.push(cnt.size);
}
return res;
};
```

Real output for `([1,2,1,3,4,2,3], 4)`; `([5,5,5,5], 2)`; `([-1,-1,0], 3)`; `([9], 1)`; the empty array:

`[3,4,4,3]`

`[1,1,1]`

`[2]`

`[1]`

`[]`

Time $O(n)$ average **Space** $O(k)$

TRAP

If you do not delete the zero-count key, `cnt.size` counts values that left the window long ago. The bug is silent: small tests still pass because keys rarely leave.

And note `cnt.size` is a **property** on a Map, not a method: `cnt.size()` throws. On a plain object there is no `.size` at all — you would need `Object.keys(o).length`, which is $O(k)$ every time and turns this $O(n)$ sweep into $O(nk)$. Another reason Map wins.

Example 12 [Cognizant · pattern]

Build a deque from scratch on a fixed-capacity array. Support `pushFront`, `pushBack`, `popFront`, `popBack`, `front`, `back`, all in $O(1)$. Constraints: capacity up to 10^5 . Edge cases: push into a full deque, pop from an empty one, wrap-around at both ends.

SOLUTION

Same circular buffer as Example 1, plus one new move: `pushFront` steps head **backwards** with wrap-around, `head = (head - 1 + cap) % cap`. The `+ cap` is needed because JavaScript `%` keeps the sign of the left operand: tested in Node, `(0 - 1) % 4` is `-1`, not `3`.

fixed-capacity deque

JS

```
class MyDeque { // fixed capacity, circular buffer
  constructor(cap) { this.a = new Array(cap); this.cap = cap; this.head = 0;
    this.count = 0; }
```

```

get empty() { return this.count === 0; }
get full() { return this.count === this.cap; }

pushBack(x) {
  if (this.full) return false;
  this.a[(this.head + this.count) % this.cap] = x;
  this.count++;
  return true;
}
pushFront(x) {
  if (this.full) return false;
  this.head = (this.head - 1 + this.cap) % this.cap; // step back, wrapping
  this.a[this.head] = x;
  this.count++;
  return true;
}
popFront() { if (this.empty) return false; this.head = (this.head + 1) %
this.cap; this.count--; return true; }
popBack() { if (this.empty) return false; this.count--; return true; }
front() { return this.a[this.head]; }
back() { return this.a[(this.head + this.count - 1) % this.cap]; }
}

```

Real output, with capacity 4, doing pushBack(2) pushBack(3) pushFront(1), then pushFront(-1) pushFront(-2), then draining:

```

true true true
1 3 3
true false
-1 3
false false true

```

Line 3 shows pushFront(-1) succeeding and pushFront(-2) failing — capacity 4 is full. Line 5 shows both pops failing on an empty deque, and empty returning true.

TRAP

(head - 1) % cap is wrong when head is 0. Tested in Node: (0 - 1) % 4 is -1, so this.a[-1] reads and writes a **string-keyed property** on the array — no error, no crash, and count says the item is there while front() returns undefined forever. Always write (head - 1 + cap) % cap, which gives 3.

TRAP

empty and full above are getters, so they are d.empty and d.full with no brackets, while front() and back() are methods. Mixing the two up throws TypeError: d.empty is not a function, or — worse — if (d.full) on a **method** is always truthy, so a full deque silently accepts nothing.

Example 13 [TCS NQT pattern]

Circular fuel route. There are n stops in a circle. Stop i gives fuel[i] litres, and driving from stop i to stop $i + 1$ burns cost[i] litres. Start with an empty tank. Return a stop you can start from and finish the whole circle, or -1. Constraints: $n \leq 10^5$, values up to 10^9 . Target: $O(n)$. Edge cases: exactly one stop; total fuel equals total cost; no valid start.

SOLUTION

Approach 1 — Try every start ($O(n^2)$)

For each start, simulate the whole circle with a queue of stops. Correct but quadratic.

Approach 2 — One pass with a running tank ($O(n)$ — optimal)

Two facts do all the work.

1. If total fuel < total cost, no start can work. Answer -1 .
2. If the tank goes negative somewhere between start s and stop i , then **no** stop between s and i can be a valid start either. Every such stop begins with an even emptier tank. So jump the start to $i + 1$.

circular route in one pass

JS

```
// pumps[i] = [fuel at pump i, cost to reach pump i+1]. Returns a start index, or -1.
const startPump = (pumps) => {
  const n = pumps.length;
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < n; i++) {
    const gain = pumps[i][0] - pumps[i][1];
    total += gain;
    tank += gain;
    if (tank < 0) { start = i + 1; tank = 0; } // nothing before i+1 can work
  }
  return total >= 0 ? start % Math.max(n, 1) : -1;
};
```

Real output for $[[4,6],[6,5],[7,3],[4,5]]$; $[[1,5],[2,3]]$; $[[3,3]]$; $[[2,1],[1,2]]$; empty:

1
-1
0
0
0

Time $O(n)$ **Space** $O(1)$

The idea that unlocked it: a failed prefix rules out **every** start inside it, not just the one you tried. That turns n independent simulations into one sweep.

Example 14 [Infosys · pattern]

Moving average of a stream. Numbers arrive one at a time. After each arrival report the average of the last k numbers (or of all of them if fewer than k have arrived). Constraints: up to 10^6 arrivals, $|x| \leq 10^9$. Edge cases: $k = 1$; negative numbers.

SOLUTION

Keep a deque of the window and a running sum. Never re-add the whole window.

moving average

JS

```
class MovingAverage {
  constructor(k) { this.win = new Deque(); this.sum = 0; this.k = k; }

  next(x) {
    this.win.push(x);
    this.sum += x;
    if (this.win.size > this.k) this.sum -= this.win.shift();
    return this.sum / this.win.size; // divide by size, not by k
  }
}
```

Real output with $k = 3$ feeding 10, 20, 30, 40, -90 , then with $k = 1$ feeding 5, -5 :

```
10.0000 15.0000 20.0000 30.0000 -6.6667
5.0000 -5.0000
```

Time $O(1)$ per arrival **Space** $O(k)$

TRAP

Divide by `win.size`, not by `k` — during the first $k - 1$ arrivals the window is not full yet, and dividing by `k` gives an average that is far too small.

`win.size` is a **getter** on the toolkit `Deque`, so there are no brackets. `win.size()` throws `TypeError: win.size is not a function`.

The running sum here is safe. Even 10^6 arrivals of 10^9 each reach 10^{15} , and `Number.MAX_SAFE_INTEGER` is $9007199254740991 \approx 9 \times 10^{15}$. Tested: `1e6 * 1e9 <= Number.MAX_SAFE_INTEGER` is `true`. One more order of magnitude on either factor and you would have to switch `sum` to `BigInt` — and then the division would need `Number(sum) / size`, because `BigInt` division truncates.

Example 15 [Wipro · pattern]

First non-repeating character in a stream. Letters `a..z` arrive one at a time. After every arrival print the first character seen so far that has appeared exactly once, or `#` if there is none. Constraints: stream length up to 10^6 . Edge cases: empty stream, one character, all characters the same.

SOLUTION

A Map of counts plus a queue of candidates in arrival order. Before reading the front, throw away any front whose count has risen above 1. Use a `Map` rather than a plain object: a plain object turns every key into a string, so the number 1 and the text "1" would collide. Tested in Node — a `Map` given the key 1 and then the key "1" holds **two** entries; a plain object holds one.

first unique in a stream

JS

```
const firstUniqueStream = (s) => {
  const out = [];
  const cnt = new Map();
  const q = new Deque(); // candidates in arrival
  order
  for (const ch of s) {
    cnt.set(ch, (cnt.get(ch) || 0) + 1);
    q.push(ch);
    while (q.size && cnt.get(q.front()) > 1) q.shift(); // stale front
    out.push(q.size ? q.front() : '#');
  }
  return out.join('');
};
```

Real output for "aabc", "zzz", "abcabc", "" and "q":

```
a#bb
z##
aaabc#
(empty string)
q
```

Time $O(n)$ **Space** $O(n)$

Each character is pushed once and popped at most once.

This is **lazy deletion**: instead of hunting through the queue for the character that just became repeated, you leave it in place and skip it when it reaches the front. Lazy deletion shows up again with heaps in Chapter 14.

Example 16 [Capgemini · pattern]

A queue that can report its maximum in $O(1)$. Support push, pop (FIFO) and getMax. Constraints: up to 10^6 operations. Edge cases: duplicates of the maximum; negative values; getMax right after the maximum is popped.

SOLUTION

Keep the normal queue plus a **decreasing deque of values**. On push, delete every smaller value from the back. On pop, if the value leaving equals the deque front, remove the front too.

MaxQueue

JS

```
class MaxQueue {
  constructor() { this.q = new Deque(); this.mx = new Deque(); } // mx: VALUES,
  decreasing

  push(x) {
    this.q.push(x);
    while (this.mx.size && this.mx.back() < x) this.mx.pop();
    this.mx.push(x);
  }
  pop() {
    const v = this.q.shift();
    if (this.mx.size && this.mx.front() === v) this.mx.shift();
    return v;
  }
  getMax() { return this.mx.front(); }
  get empty() { return this.q.size === 0; }
  get size() { return this.q.size; }
}
```

Real output. Pushing 5, then 3, then 1, then a second 5, printing the value just pushed and getMax each time; then two pops with getMax after each; then a fresh queue that gets -4 and 2 and one pop:

```
5
3 5
1 5
5 5
5 5
3 5
-4 2
```

Line 5 is the important one: popping the **first** 5 must leave getMax at 5, because the second 5 is still in the queue.

Time amortised $O(1)$ **Space** $O(n)$

TRAP

The pop comparison uses $<$, not $<=$. If you write `mx.back() <= x` you delete **equal** copies, and then popping one 5 removes the only record of the other 5 — getMax returns 2 while a 5 is still in the queue. The test above pushes two 5s exactly to catch this.

Tier 2 — applied, two or three steps

Example 17 [Grab · pattern]

Surge pricing window. A ride app records the number of open ride requests each minute for n minutes. Marketing wants, for every rolling window of k minutes, both the peak and the floor, and then the largest peak-minus-floor across all windows. Constraints: $n \leq 2 \times 10^5$, counts up to 10^6 . Target: $O(n)$. Edge cases: $k = n$; counts all equal (answer 0); $k = 1$ (answer 0).

SOLUTION

Two deques again, but this time you only track the widest gap.

widest peak-to-floor gap over all windows

JS

```
const widestGap = (a, k) => {
  const n = a.length;
  if (k <= 0 || k > n) return 0;
  const mx = new Deque(), mn = new Deque();
  let best = 0;
  for (let i = 0; i < n; i++) {
    while (mx.size && mx.front() <= i - k) mx.shift();
    while (mn.size && mn.front() <= i - k) mn.shift();
    while (mx.size && a[mx.back()] <= a[i]) mx.pop();
    while (mn.size && a[mn.back()] >= a[i]) mn.pop();
    mx.push(i); mn.push(i);
    if (i >= k - 1) best = Math.max(best, a[mx.front()] - a[mn.front()]);
  }
  return best;
};
```

Real output for ([3,9,2,8,8,1], 3); ([5,5,5], 2); ([4,7,2], 1); the empty array:

7
0
0
0

The first answer is 7: the window [9, 2, 8] has peak 9 and floor 2.

Time $O(n)$ **Space** $O(k)$

Example 18 [Shopee · pattern]

Stable price run. A seller logs a price every hour. Find the longest run of consecutive hours in which the highest price minus the lowest price is at most `limit`. Constraints: $n \leq 10^5$, prices up to 10^9 , `limit` ≥ 0 . Target: $O(n)$. Edge cases: `limit` = 0 (longest run of equal values); one element; empty array; negative values.

SOLUTION

Approach 1 — Try every pair of endpoints ($O(n^2)$ or $O(n^3)$)

For every start and end, compute max and min. Even with a running max and min this is $O(n^2)$.

Approach 2 — Window + a sorted array ($O(n \log n)$, and $O(n)$ hidden in splice)

JavaScript has no ordered multiset, so you would hold the window in an array kept sorted, find positions with the toolkit `lowerBound`, and insert and remove with `splice`. Then `last - first > limit` says when to shrink. It works, but `splice` itself moves elements, so the real cost is worse than the $O(n \log n)$ the idea promises. Avoidable — skip to approach 3.

Approach 3 — Window + two monotonic dequeues ($O(n)$ — optimal)

The window is no longer a fixed size, so expiry is not “index $\leq i - k$ ” any more. Instead, when you move left forward, check whether either deque’s front **was** that index and pop it.

longest run with max - min $\leq$ limit

JS

```
const longestStableRun = (a, limit) => {
  const mx = new Deque(), mn = new Deque();           // both hold indexes
  let best = 0, left = 0;
  for (let right = 0; right < a.length; right++) {
    while (mx.size && a[mx.back()] <= a[right]) mx.pop();
    while (mn.size && a[mn.back()] >= a[right]) mn.pop();
    mx.push(right); mn.push(right);
    while (a[mx.front()] - a[mn.front()] > limit) {    // shrink from the left
      if (mx.front() === left) mx.shift();
      if (mn.front() === left) mn.shift();
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
};
```

Real output for ([8,2,4,7], 4); ([10,1,2,4,7,2], 5); ([4,2,2,2,4,4,2,2], 0); ([5], 0); the empty array; [-5,-1,-3], 4):

2
4
3
1
0
3

Time $O(n)$ **Space** $O(n)$

The idea that unlocked it: the window still only moves right, so the dequeues still only ever discard from the front. A variable-size window does not break the monotonic structure — it only changes **when** you pop the front.

TRAP

Inside the shrink loop, both ifs must be checked, and neither may be an else if. When the window holds a single element, that index is the front of **both** dequeues and both must go.

Example 19 [Agoda · pattern]

Warehouse spoilage. A shelf grid holds 0 = empty slot, 1 = fresh crate, 2 = spoiled crate. Every minute a spoiled crate spoils the fresh crates directly above, below, left and right of it. Return the number of minutes until nothing fresh is left, or -1 if some crate can never be reached. Constraints: grid up to 500×500 . Edge cases: no fresh crates at the start (answer 0); a fresh crate walled off (answer -1); an empty grid.

SOLUTION

This is breadth-first search with **many** starting points at once. Push every spoiled crate into the queue first, then process the queue one whole **level** at a time; each level is one minute.

multi-source BFS

JS

```

const minutesToSpoil = (grid) => {
  const g = grid.map(row => [...row]); // deep copy: we overwrite cells
  const r = g.length; if (r === 0) return 0;
  const c = g[0].length; if (c === 0) return 0;
  const q = new Deque();
  let fresh = 0;
  for (let i = 0; i < r; i++)
    for (let j = 0; j < c; j++) {
      if (g[i][j] === 2) q.push([i, j]);
      else if (g[i][j] === 1) fresh++;
    }
  if (fresh === 0) return 0;
  let minutes = 0;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  while (q.size && fresh > 0) {
    const sz = q.size; // one whole minute = one whole
    level
    for (let s = 0; s < sz; s++) {
      const [x, y] = q.shift();
      for (const [dx, dy] of dirs) {
        const nx = x + dx, ny = y + dy;
        if (nx < 0 || ny < 0 || nx >= r || ny >= c || g[nx][ny] !== 1) continue;
        g[nx][ny] = 2; fresh--; q.push([nx, ny]);
      }
    }
    minutes++;
  }
  return fresh === 0 ? minutes : -1;
};

```

Real output for the six test grids `[[2,1,1],[1,1,0],[0,1,1]]`, `[[2,1,1],[0,1,1],[1,0,1]]`, `[[0,2],[[1]], [ ]]` and `[[ ]]`:

4 -1 0 -1 0 0

The caller's grid is printed afterwards and is unchanged.

TRAP

`[...grid]` is only a **SHALLOW** copy. The new outer array holds the very same row arrays, so `copy[0][0] = 2` also changes `grid[0][0]`. Tested in Node: after `const shallow = [...g]; shallow[0][0] = 0;` the original `g[0][0]` is 0 too — the caller's data is destroyed and there is no error to tell you.

The correct deep copy of a 2-D grid is `grid.map(row => [...row])`, which is what the code above uses. Tested: the caller's grid comes back untouched. If you **want** to spoil the caller's grid in place, say so out loud; silently doing it is a bug.

Time $O(rc)$ **Space** $O(rc)$

TRAP

`const sz = q.size;` must be taken **before** the inner loop. If you write `for (let s = 0; s < q.size; s++)` the bound is re-read every iteration, so the loop keeps growing while you push, and every crate is processed in “minute 1”. The whole level-by-level idea collapses and the answer comes out as 1 for almost every grid.

Example 20 [SCB · pattern]

Fewest moves on a floor plan. A warehouse floor is a grid of . (walkable) and # (blocked). Return the fewest single-step moves (up, down, left, right) from a start cell to a target cell, or -1 . Constraints: grid up to 1000×1000 . Edge cases: start equals target (0); start or target blocked; target unreachable.

SOLUTION

Plain BFS. The first time BFS reaches a cell, it reaches it by the shortest path — this is true only because every move costs the same.

grid BFS

JS

```
const shortestSteps = (g, sr, sc, tr, tc) => {
  const r = g.length; if (r === 0) return -1;
  const c = g[0].length;
  if (g[sr][sc] === '#' || g[tr][tc] === '#') return -1;
  const dist = Array.from({ length: r }, () => new Array(c).fill(-1));
  const q = new Deque();
  q.push([sr, sc]); dist[sr][sc] = 0;
  const dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
  while (q.size) {
    const [x, y] = q.shift();
    if (x === tr && y === tc) return dist[x][y];
    for (const [dx, dy] of dirs) {
      const nx = x + dx, ny = y + dy;
      if (nx < 0 || ny < 0 || nx >= r || ny >= c) continue;
      if (g[nx][ny] === '#' || dist[nx][ny] !== -1) continue;
      dist[nx][ny] = dist[x][y] + 1; // mark on PUSH, not on pop
      q.push([nx, ny]);
    }
  }
  return -1;
};
```

On the floor plan

```
. . . .
# # . #
. # . .
. . . .
```

the real output for $(0,0) \rightarrow (3,0)$; $(0,0) \rightarrow (0,3)$; a blocked 2×2 case; a single cell:

```
7
3
-1
0
```

Time $O(rc)$ **Space** $O(rc)$

TRAP

Mark a cell as visited **when you push it**, not when you pop it. If you mark on pop, the same cell can be pushed many times before it is popped once, and the queue explodes.

Example 21 [DBS · pattern]

API rate limiter. Allow at most `limit` calls from one client in any window of `windowSec` seconds. Timestamps arrive in non-decreasing order. Constraints: up to 10^6 calls. Edge cases: `limit = 0`; a long gap with no calls; two calls at the same timestamp.

SOLUTION

Keep the accepted timestamps in a deque. Drop everything older than the window, then compare the size against the limit.

sliding-window rate limiter

JS

```
class RateLimiter {
  constructor(limit, windowSec) { this.t = new Deque(); this.limit = limit;
    this.windowSec = windowSec; }

  allow(now) {
    while (this.t.size && this.t.front() <= now - this.windowSec) this.t.shift();
    if (this.t.size >= this.limit) return false;
    this.t.push(now);
    return true;
  }
  countInWindow(now) {
    while (this.t.size && this.t.front() <= now - this.windowSec) this.t.shift();
    return this.t.size;
  }
}
```

Real output with limit = 3, windowSec = 10, calls at t = 1, 2, 3, 4, then 11, then countInWindow at t = 100, and finally a limiter with limit = 0:

```
true true true false
3
true
3
0
false
```

The call at t = 4 is rejected. The call at t = 11 is accepted because the call at t = 1 has now expired. With limit = 0 nothing is ever allowed.

Time amortised $O(1)$ per call **Space** $O(\text{limit})$

Follow-up the interviewer asks: “a million clients, and memory is tight.” Then you drop the exact deque and use a **counter per fixed bucket** — an approximate limiter that stores two integers per client instead of limit timestamps. Trading exactness for memory is a normal, expected answer here.

Example 22 [Sea/Shopee · pattern]

Job cooldown scheduler. You have a list of jobs, each labelled by a type. Two jobs of the same type may not run within cool ticks of each other. One job runs per tick, and a tick may be idle. Return the number of ticks the whole list takes. Constraints: up to 10^5 jobs. Edge cases: cool = 0; every job the same type; empty list.

SOLUTION

Greedy: at every tick run the **most frequent** job that is not cooling down. A max-heap holds the ready jobs, and a queue holds the cooling ones with the tick at which they become ready again. The queue is naturally sorted by wake-up time because ticks only move forward.

cooldown scheduler

JS

```
const { MinHeap, Deque } = require('./toolkit.js'); // JS has no priority queue

const ticksNeeded = (jobs, cool) => {
  const cnt = new Map();
  for (const j of jobs) cnt.set(j, (cnt.get(j) || 0) + 1);
```

```

const ready = new MinHeap((x, y) => y[0] - x[0]); // flipped comparator -> MAX-heap
for (const [id, c] of cnt) ready.push([c, id]); // [remaining count, job id]
const cooling = new Deque(); // [free at tick, [count, id]]
let tick = 0;
while (ready.size || cooling.size) {
  tick++; // an idle tick still counts
  if (ready.size) {
    const [c, id] = ready.pop();
    if (c - 1 > 0) cooling.push([tick + cool, [c - 1, id]]);
  }
  if (cooling.size && cooling.front()[0] <= tick) ready.push(cooling.shift()[1]);
}
return tick;
};

```

Real output for $([1,1,2,2,3,3], 2)$; $([1,1,1], 2)$; $([1,2,3,4], 0)$; $([5], 9)$; the empty list; $([1,1,1,1,2,2,3], 2)$:

6
7
4
1
0
10

Time $O(n \log n)$ **Space** $O(n)$

TRAP

tick++ happens **before** the work, so the loop counts idle ticks automatically: if ready is empty but cooling is not, the tick is burned doing nothing. Many people forget the idle case and get $([1,1,1], 2)$ wrong (7 is right, 3 is the common wrong answer).

Example 23 [GIC · pattern]

Shortest burst that reaches a target. Given daily profit or loss numbers (they may be negative) and a target S , find the **shortest** run of consecutive days whose total is at least S . Return -1 if none exists. Constraints: $n \leq 10^5$, $|a_i| \leq 10^4$, $|S| \leq 10^9$. Edge cases: all values negative; S negative; empty array; one element equal to S .

SOLUTION

Approach 1 — All pairs ($O(n^2)$)

Fix a start, walk right, keep the running total. Correct. With $n = 10^5$ that is 5×10^9 steps.

Approach 2 — Sliding window (wrong when values may be negative)

The classic two-pointer window works **only** if every value is positive, because then the running sum grows when you extend and shrinks when you cut. With a negative value in the middle, extending the window can **lower** the sum, so shrinking is no longer safe.

sliding window — valid only for all-positive input

JS

```

const shortestAtLeastPositive = (a, S) => {
  let sum = 0, left = 0, best = Infinity;
  for (let right = 0; right < a.length; right++) {

```

```

    sum += a[right];
    while (sum >= S) { best = Math.min(best, right - left + 1); sum -= a[left++]; }
  }
  return best === Infinity ? -1 : best;
};

```

Real output for $([2, 3, 1, 2, 4, 3], 7)$ and $([1, 1, 1], 9)$:

2 -1

And the proof that it is wrong with negatives: on $([84, -37, 32, 40, 95], 167)$ it reports 5, while the true answer is 3 (the run 32, 40, 95). Once left has moved past a negative value, it can never come back for it.

Approach 3 — Prefix sums + increasing deque $O(n)$ — optimal, handles negatives

Let $p[i]$ be the sum of the first i values. A run $a[j..i-1]$ totals $p[i] - p[j]$. So for each right end i you want the **largest** $j < i$ with $p[i] - p[j] \geq S$.

Keep a deque of candidate start indexes whose p values **increase**. Two deletions:

1. From the front: if $p[i] - p[\text{front}] \geq S$, record the length and pop — a later i would only give a longer run with the same start, so this start is finished forever.
2. From the back: if $p[\text{back}] \geq p[i]$, pop. Index i is newer **and** has a smaller prefix, so it is a strictly better start for everything after it.

optimal: monotonic deque over prefix sums

JS

```

const shortestAtLeast = (a, S) => {
  const n = a.length;
  const p = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
  const dq = new Deque(); // indexes into p, p
  increasing
  let best = Infinity;
  for (let i = 0; i <= n; i++) { // note: i <= n
    while (dq.size && p[i] - p[dq.front()] >= S) { // front is the oldest
      usable start
      best = Math.min(best, i - dq.front());
      dq.shift();
    }
    while (dq.size && p[dq.back()] >= p[i]) dq.pop(); // a bigger prefix is
    useless
    dq.push(i);
  }
  return best === Infinity ? -1 : best;
};

```

Real output over six hand tests — $([84, -37, 32, 40, 95], 167)$, $([1, 1, 1], 9)$, $([2, -1, 2], 3)$, $([5], 5)$, $([], 1)$, $([-1, -2], -1)$ — plus a 3000-case random stress test against the brute force, with negative values and negative S :

3 -1 3 1 -1 1
stress ok

Time $O(n)$ **Space** $O(n)$

The idea that unlocked it: rewrite “sum of a run” as “difference of two prefix sums”. The question then becomes “find the best earlier prefix”, and **best earlier** is exactly what a monotonic deque stores.

This is the chapter’s second signature problem, so here it is in Python too.


```

from collections import deque

def shortest_at_least(a, S):
    n = len(a)
    p = [0] * (n + 1)
    for i, v in enumerate(a):
        p[i + 1] = p[i] + v
    dq = deque()  # indexes into p, p values increasing
    best = None
    for i in range(n + 1):  # note: n + 1 prefix sums
        while dq and p[i] - p[dq[0]] >= S:
            length = i - dq.popleft()
            if best is None or length < best:
                best = length
        while dq and p[dq[-1]] >= p[i]:
            dq.pop()
        dq.append(i)
    return -1 if best is None else best

```

Run on the same six inputs, Python prints 3 -1 3 1 -1 1 — the same answers as the JavaScript. Python integers are exact at any size, so there is no `Number.MAX_SAFE_INTEGER` check to do on the prefix sums.

TRAP

The loop runs to $i \leq n$, not $i < n$, because there are $n + 1$ prefix sums. Miss this and you never consider runs that end at the last element.

Tier 3 — you need the insight

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Amazon · pattern]

Best spaced pick. Choose a non-empty subsequence of a such that any two chosen positions are at most k apart. Maximise the sum of the chosen values. Constraints: $n \leq 10^5$, $|a_i| \leq 10^4$, $1 \leq k \leq n$. Target: $O(n)$. Edge cases: all values negative (you must still pick one); a total large enough to be worth checking against `Number.MAX_SAFE_INTEGER`; $k = 1$ (you must take a contiguous run).

SOLUTION

Approach 1 — Try every subsequence ($O(2^n)$ — hopeless)

Useful only as a checker on tiny inputs (this is exactly what the stress test below does).

Approach 2 — Plain dp ($O(nk)$)

Let $dp[i]$ be the best total of a valid pick that **ends at** position i . Then

$$dp[i] = a[i] + \max\left(0, \max_{j=i-k}^{i-1} dp[j]\right)$$

The $\max(0, \dots)$ says “or start fresh at i ”. Scanning k earlier cells for each i gives $O(nk)$ — up to 10^{10} steps.

Approach 3 — dp + monotonic deque ($O(n)$ — optimal)

The inner max is the maximum of a sliding window over the dp array. Exactly the template.

```
const bestSpacedSum = (a, k) => {
  const n = a.length;
  if (n === 0) return 0;
  const dp = new Array(n);
  const dq = new Deque(); // indexes, dp values decreasing
  let best = -Infinity;
  for (let i = 0; i < n; i++) {
    while (dq.size && dq.front() < i - k) dq.shift();
    const take = dq.size ? Math.max(0, dp[dq.front()]) : 0;
    dp[i] = a[i] + take;
    best = Math.max(best, dp[i]);
    while (dq.size && dp[dq.back()] <= dp[i]) dq.pop();
    dq.push(i);
  }
  return best;
};
```

Real output for $([10, 2, -10, 5, 20], 2)$, $([-1, -2, -3], 1)$, $([3, -1, 4, -1, 5, -9, 2, 6], 3)$, $([7], 1)$, $([], 2)$ and $([2000000000, 2000000000], 1)$, including a 400-case stress test against the exponential brute force:

```
37 -1 20 7 0 4000000000
stress ok
```

$[-1, -2, -3]$ with $k = 1$ gives -1 : you must pick something, so you pick the least bad. The last test is the one a 32-bit language would fail — 4×10^9 overflows a signed 32-bit integer. JavaScript holds it exactly, because it is far below `Number.MAX_SAFE_INTEGER = 9007199254740991`. That is the check to run out loud, not a type to declare.

Time $O(n)$ **Space** $O(n)$

The idea that unlocked it: whenever a dp recurrence says “the best of the last k states”, the deque replaces that inner loop. Recognising a sliding-window maximum **inside a dp** is the single most valuable use of this chapter.

SHORTCUT

Notice the expiry test here is `dq.front() < i - k`, not `<= i - k`. The window is $[i-k, i-1]$, which has k cells, and i itself is pushed only **after** `dp[i]` is read. Draw the window on paper before you choose the comparison sign.

Follow-up: “now the array is a stream and you must answer after every arrival.” Nothing changes — the code already works online. It reads only `a[i]` and the deque, never a future value. Say that out loud; it is the answer the interviewer wants.

Example 25 [Google · pattern]

Tile hopping. You stand on tile 0 of a row of n tiles and must reach tile $n - 1$. From tile i you may jump to any tile $i + 1..i + k$. Your score is the sum of every tile you land on, including tile 0 and tile $n - 1$. Maximise the score. Constraints: $n \leq 10^5$, $|a_i| \leq 10^4$, $1 \leq k \leq n$. Edge cases: $n = 1$; all tiles negative; $k \geq n$ (jump straight to the end).

SOLUTION

Approach 1 — dp over all predecessors ($O(nk)$)

`dp[i] = a[i] + max(dp[i-k..i-1])`. Straightforward, too slow.

Approach 2 — Same dp, deque for the inner max ($O(n)$ — optimal)

Unlike the previous problem there is no `max(0, ...)`: you **must** land somewhere, so starting fresh is not allowed.

```

const bestJumpScore = (a, k) => {
  const n = a.length;
  if (n === 0) return 0;
  const dp = new Array(n).fill(-Infinity);
  dp[0] = a[0];
  const dq = new Deque();
  dq.push(0);
  for (let i = 1; i < n; i++) {
    while (dq.size && dq.front() < i - k) dq.shift();
    dp[i] = dp[dq.front()] + a[i]; // you MUST land somewhere
    while (dq.size && dp[dq.back()] <= dp[i]) dq.pop();
    dq.push(i);
  }
  return dp[n - 1];
};

```

Real output for $([1, -1, -2, 4, -7, 3], 2)$, $([10, -5, -2, 4, 0, 9], 3)$, $([0], 5)$, $([-3, 2, -1, 6], 4)$ and $([], 2)$, plus a 2000-case stress test against the $O(nk)$ dp:

```

7 23 0 5 0
stress ok

```

Time $O(n)$ **Space** $O(n)$

TRAP

`dq` can never become empty here, because index $i-1$ was pushed on the previous step and $i-1 \geq i-k$ for any $k \geq 1$. That is why `dp[dq.front()]` is safe without a guard. If you allowed $k = 0$ the guarantee breaks — and in JavaScript it breaks **quietly**: `dq.front()` returns `undefined`, `dp[undefined]` is `undefined`, and every later `dp` value becomes `NaN` with no error. Reject $k = 0$ at the top.

Follow-up: “what if some tiles are holes you may not land on?” Simply never push a hole index into the deque, and set `dp[hole] = -Infinity`. `-Infinity` is the right sentinel in JavaScript: it compares correctly with every real number and `-Infinity + x` is still `-Infinity`, so an unreachable tile can never win. If the deque is empty when you reach tile i , tile i is unreachable.

Example 26 [D. E. Shaw · pattern]

Median of every window. Report the median of every window of size k . For even k the median is the average of the two middle values. Constraints: $n \leq 10^5$. Target: $O(n \log k)$. Edge cases: $k = 1$; duplicates; $k = n$; negative values.

SOLUTION

A monotonic deque cannot help here: the median is not an extreme, so “smaller and older means useless” is false. Every value in the window matters.

Approach 1 — Sort each window ($O(n k \log k)$)

Approach 2 — One sorted array, kept sorted ($O(n \log k)$ searching, $O(k)$ moving — fast enough)

JavaScript has no ordered multiset, so keep the window in a plain array that you **never** let go out of order. Finding where a value belongs is a binary search — the toolkit `lowerBound` from the **JS Toolkit** appendix. Inserting and removing is `splice`.

The median is then just a fixed slot: index $\frac{k-1}{2}$ for odd k , and the average of slots $\frac{k-1}{2}$ and $\frac{k-1}{2} + 1$ for even k . No iterator to walk, because an array index does not move when the array shifts around it — it always points at the middle.

```

const { lowerBound } = require('./toolkit.js'); // JS has no lower_bound of its
own

const windowMedian = (a, k) => {
  const res = [];
  const n = a.length;
  if (k <= 0 || k > n) return res;
  const win = a.slice(0, k).sort((x, y) => x - y); // NUMERIC comparator, never
bare sort()
  const mid = (k - 1) >> 1; // lower of the two middles
  for (let i = k; ; i++) {
    res.push(k % 2 ? win[mid] : (win[mid] + win[mid + 1]) / 2);
    if (i === n) break;
    win.splice(lowerBound(win, a[i]), 0, a[i]); // insert, keeping win sorted
    win.splice(lowerBound(win, a[i - k]), 1); // remove ONE copy of the old
value
  }
  return res;
};

```

Real output for $([1, 3, -1, -3, 5, 3, 6, 7], 3)$, $([1, 2, 3, 4], 2)$, $([5], 1)$, $([4, 4, 4], 2)$ and $([0, 0], 1)$, ending with a 3000-case stress test against sort-every-window:

```

1 -1 -1 3 5 6
1.5 2.5 3.5
5
4 4
0 0
stress ok

```

Time $O(n \log k)$ **Space** $O(k)$

TRAP

win.sort() with no comparator is lexicographic. Tested in Node: `[10, 9, 1].sort()` returns `[1, 10, 9]`, because it compares the **strings** "10", "9", "1". Every numeric sort in this book must be `sort((x, y) => x - y)`, which on the same array returns `[1, 9, 10]`. Get this wrong here and the first window's median is nonsense while every later window looks fine — the hardest kind of bug to find.

TRAP

`win.splice(lowerBound(win, v), 1)` removes **exactly one** copy of `v`. Do not reach for `win.filter(x => x !== v)`, which removes every copy, nor `indexOf`, which is $O(k)$ and defeats the binary search. And insert the new value **before** removing the old one, or the two `lowerBound` calls can disagree about the array they are searching.

Honest cost: each `splice` moves up to k elements, so this is $O(nk)$ element moves in the worst case, with only the **searching** being $O(n \log k)$. For k up to a few thousand it is comfortably fast, because `splice` moves memory in one block. If k is huge, use the two-heap version in the follow-up instead. Say this out loud — pretending JavaScript has a `TreeMap` is the mistake the interviewer is listening for.

Follow-up: “do it with heaps.” Keep a max-heap of the lower half and a min-heap of the upper half, rebalance after every move, and use lazy deletion for values that have left the window. Same $O(n \log k)$, more code, and it is the version that generalises to a pure stream with no window. You will build it in Chapter 14.

Example 27 [Microsoft · pattern]

Best floor for every span length. For every window size w from 1 to n , report the largest value of “the minimum of a window of size w ”. Return all n answers. Constraints: $n \leq 10^5$. Target: $O(n)$. Edge cases: all values equal; strictly decreasing input; empty array; negatives.

SOLUTION

Running the sliding-window minimum once per w costs $O(n^2)$. Turn the question around.

Turn it around: instead of asking “for this window, what is the minimum?”, ask “for this element, over how wide a span is it the minimum?” Element $a[i]$ is the minimum of the span that stretches from just after the nearest strictly smaller element on its left to just before the nearest strictly smaller element on its right. Call that width len . Then $a[i]$ is a candidate answer for window size len — and for every size below len too, which one backward sweep fixes up.

Nearest smaller on each side is the monotonic **stack** from Chapter 8.

max of window minimums, for every window size

JS

```
const maxOfMins = (a) => {
  const n = a.length;
  if (n === 0) return [];
  const left = new Array(n), right = new Array(n); // strictly smaller neighbour
  each side
  let st = []; // a plain array: Chapter 8's stack
  for (let i = 0; i < n; i++) {
    while (st.length && a[st[st.length - 1]] >= a[i]) st.pop();
    left[i] = st.length ? st[st.length - 1] : -1;
    st.push(i);
  }
  st = [];
  for (let i = n - 1; i >= 0; i--) {
    while (st.length && a[st[st.length - 1]] >= a[i]) st.pop();
    right[i] = st.length ? st[st.length - 1] : n;
    st.push(i);
  }
  const best = new Array(n + 2).fill(-Infinity);
  for (let i = 0; i < n; i++) {
    const len = right[i] - left[i] - 1; // a[i] is the min of this many-wide span
    best[len] = Math.max(best[len], a[i]);
  }
  for (let w = n - 1; w >= 1; w--) best[w] = Math.max(best[w], best[w + 1]);
  return best.slice(1, n + 1);
};
```

Real output, with a 1500-case stress test:

```
70 30 20 10 10 10 10
5 5 5
-1 -2 -3
0
stress ok
```

Time $O(n)$ **Space** $O(n)$

The idea that unlocked it: swap the roles of the loop variable and the answer. “For each window, find the element” became “for each element, find its windows”. This **contribution** view is the standard escape hatch when a per-window sweep is too slow.

Follow-up: “why the backward sweep at the end?” Because an element that is the minimum of a span of width 5 is also the minimum of some span of width 4, 3, 2 and 1 inside it. The sweep $\text{best}[w] = \max(\text{best}[w], \text{best}[w+1])$ pushes every answer down to the smaller sizes in one pass.

Example 28 [Goldman Sachs · pattern]

Shortest route when some moves are free. A network of n nodes has directed links. A link costs either 0 (an internal transfer) or 1 (an external hop). Find the cheapest cost from a source node to every node. Constraints: $n, m \leq 10^5$. Target: $O(n + m)$ — faster than Dijkstra’s $O(m \log n)$. Edge cases: unreachable nodes; a node with no outgoing links; source equals target.

SOLUTION

With only two possible edge costs you do not need a heap. The classic phrasing uses one **deque**: a free move goes to the **front** (handle it now, same cost level) and a paid move goes to the **back** (next cost level). The deque then holds at most **two** distinct distance values at any time, and it stays sorted by itself.

That “at most two” is the whole insight, and it gives a JavaScript-friendly way to write the same algorithm: keep those two levels as two plain queues, *cur* (cost d) and *next* (cost $d + 1$). A free move pushes onto *cur*, a paid move onto *next*, and when *cur* runs dry you swap them. Identical order of processing, identical complexity — and it needs only push and shift, which is exactly what the toolkit Deque gives you in $O(1)$.

0-1 BFS

JS

```
const zeroOneBFS = (n, adj, src) => {
  const dist = new Array(n).fill(Infinity);
  let cur = new Deque(), next = new Deque(); // cur: cost d; next: cost d + 1
  dist[src] = 0;
  cur.push(src);
  while (cur.size || next.size) {
    if (cur.size === 0) { [cur, next] = [next, cur]; continue; } // step to the
    next level
    const u = cur.shift();
    for (const [v, w] of adj[u]) {
      if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        if (w === 0) cur.push(v); // free move: same cost level, handle it now
        else next.push(v); // paid move: one level later
      }
    }
  }
  return dist;
};
```

Real output on the 5-node chain with edges $0 \rightarrow 1$ (cost 0), $1 \rightarrow 2$ (cost 1), $2 \rightarrow 3$ (cost 0), $3 \rightarrow 4$ (cost 1), then a check that an unreachable node stays at Infinity, then a one-node graph, then a 500-case stress test against a plain Dijkstra:

```
0 0 1 1 2
true
0
stress ok
```

Time $O(n + m)$ **Space** $O(n + m)$

A node can be pushed more than once, but its distance only ever falls, and it can fall at most twice — so the total work stays linear.

The idea that unlocked it: a priority queue is only needed when many different distances are in flight. With costs in $\{0, 1\}$ there are only ever two, and “this level or the next” is enough priority.

TRAP

Whatever you do, do **not** write the front insert as `arr.unshift(v)` on a plain array. `unshift` has to slide every existing element up one slot, so it is $O(n)$, and the whole $O(n + m)$ claim collapses to $O(n^2)$. Measured in Node: unshifting 200000 values onto a plain array took **2695 ms**; pushing and shifting the same 200000 through the toolkit Deque took **15 ms**. The toolkit Deque deliberately has no `unshift` — the two-queue form above is how you get the same algorithm at the right speed.

Follow-up: “now costs are 0, 1 or 2.” Split every cost-2 edge into two cost-1 edges with a fake node in the middle, then run the same 0-1 BFS. For general small costs up to C , use $C + 1$ buckets (dial’s algorithm). For arbitrary costs, use Dijkstra — Chapter 16.

Example 29 [Adobe · pattern]

Design: a queue with `getMin` in $O(1)$, and then the follow-ups. Constraints: up to 10^6 operations. Edge cases: duplicates of the minimum; `getMin` after the minimum is popped; an empty structure.

SOLUTION

Mirror of the MaxQueue: store the values in a deque kept **increasing**.

MinQueue

JS

```
class MinQueue {
  constructor() { this.q = new Deque(); this.mn = new Deque(); } // mn: values,
  INCREASING
  push(x) { this.q.push(x); while (this.mn.size && this.mn.back() > x)
    this.mn.pop(); this.mn.push(x); }
  pop() { const v = this.q.shift(); if (this.mn.front() === v) this.mn.shift();
  return v; }
  getMin() { return this.mn.front(); }
  get empty() { return this.q.size === 0; }
}
```

Real output pushing 4, 2, 6 then draining:

2 4 2 2 6 6 1

Read it as: min is 2; pop gives 4, min still 2; pop gives 2, min becomes 6; pop gives 6, now empty (1).

Time amortised $O(1)$ **Space** $O(n)$

Follow-up 1 — “now support pop from both ends.” Then a single deque of candidates is no longer enough, because a pop from the back can remove the newest minimum and there is no record of what was behind it. Switch to two **min-stacks** (Chapter 8, Example 6) glued back to back, which keeps $O(1)$ amortised; or accept $O(\log k)$ with a sorted array plus `lowerBound` and `splice`, since JavaScript has no ordered multiset to fall back on.

Follow-up 2 — “now it is distributed across many machines.” Each machine keeps its own MinQueue and reports its local minimum; a coordinator keeps a small heap of the m local minimums. A pop tells the coordinator which machine changed, and it fixes that one entry in $O(\log m)$.

Follow-up 3 — “prove the amortised bound.” Every value is pushed into `mn` exactly once and popped from `mn` at most once. So n operations cause at most $2n$ deque moves. The worst **single** operation is $O(n)$; the **average** over any sequence is $O(1)$.

Dry run — sliding window maximum, line by line

Input `a = [4, 2, 7, 1, 9, 3, 3, 8]`, window size `k = 3`.

The deque column shows `index(value)` from front to back. “Expired” is the index dropped by rule 1; “Popped from back” lists the indexes dropped by rule 2.

i	a[i]	expired	popped back	deque after	output
0	4	—	—	0(4)	—
1	2	—	—	0(4) 1(2)	—
2	7	—	1, 0	2(7)	7
3	1	—	—	2(7) 3(1)	7
4	9	—	3, 2	4(9)	9
5	3	—	—	4(9) 5(3)	9
6	3	—	5	4(9) 6(3)	9
7	8	4	6	7(8)	8

Final answer: 7 7 9 9 9 8.

Step-by-step reading.

- i = 0.** Deque empty, push 0. Window is not full ($i < k - 1$), no output.
- i = 1.** $a[0] = 4 > a[1] = 2$, so rule 2 does not fire. Push 1. Deque 0(4) 1(2).
- i = 2.** $a[1] = 2 \leq 7 \rightarrow$ pop index 1. $a[0] = 4 \leq 7 \rightarrow$ pop index 0. Push 2. Window [4, 2, 7], front is 2(7) $\rightarrow$ output 7.
- i = 3.** Front index 2 is not expired ($2 > 3 - 3 = 0$). $a[2] = 7 > 1$, so 1 just joins. Window [2, 7, 1] $\rightarrow$ output 7.
- i = 4.** $a[3] = 1 \leq 9$ pop, $a[2] = 7 \leq 9$ pop. Deque 4(9). Output 9.
- i = 5.** $9 > 3$, push. Deque 4(9) 5(3). Output 9.
- i = 6.** $a[5] = 3 \leq 3 \rightarrow$ pop index 5 **even though the values are equal**. Push 6. Deque 4(9) 6(3). Output 9.
- i = 7.** Rule 1: front is 4 and $4 \leq 7 - 3 = 4 \rightarrow$ expire it. The 9 has left the window. Rule 2: $a[6] = 3 \leq 8 \rightarrow$ pop. Push 7. Deque 7(8). Window [3, 3, 8] $\rightarrow$ output 8.

Why pop an equal value at $i = 6$? Because index 6 is newer. Both hold 3, so keeping the older one buys nothing and it will expire sooner. Using $\leq$ instead of $<$ keeps the deque shorter and is still correct. Using $<$ is **also** correct here — it just wastes space. But in the MaxQueue of Example 16, which stores **values** and not indexes, $\leq$ would be a real bug. Know which structure you are in.

Practice

PRACTICE · WARM-UP target: 20 min

- Rotate a queue left by k places using only queue operations. Example: 1 2 3 4 5 with $k = 2$ becomes 3 4 5 1 2. Handle k larger than the size and an empty queue.
- Given an array and k , return `true` if **every** window of size k contains at least one zero.
- Using a queue of strings, print the first n numbers whose decimal digits are only 1 and 2 (1, 2, 11, 12, 21, 22, 111, ...).
- Write `minWindow` from memory, then write `maxWindow` by changing exactly two characters. State which two.

PRACTICE · TIER 1 target: 45 min

- Given prices for n days and a window k , return the index of the day with the highest price in each window (if tied, the earliest such day).
- Count how many windows of size k satisfy “max – min $\leq L$ ”.
- Longest run containing at most K distinct values.

8. A queue holds server IDs. Remove every ID that appears more than once anywhere in the queue, keeping the original order of the survivors.
9. Given an array, return the sum of the **minimums** of all windows of size k . Watch for overflow.

PRACTICE · TIER 2 target: 50 min

10. Shortest run of consecutive days whose total is at least S , where daily values may be negative.
11. A delivery grid has $.$, $\#$ and T (toll booth, costs 1 extra move to enter, normal cells cost 1). Find the cheapest path. (Hint: costs are 1 and 2 — see the 0-1 BFS follow-up.)
12. Design a structure that stores events with timestamps and answers “how many events in the last W seconds?” in amortised $O(1)$.

PRACTICE · TIER 3 target: 60 min

13. Maximise the sum of a subsequence in which consecutive chosen positions are at most k apart, and at most m items may be chosen in total. State the complexity.
14. Given an array, find the maximum over all subarrays of (subarray minimum $\times$ subarray sum), assuming all values are non-negative.
15. You are given a stream and a window k . After each arrival, report the **second** largest value in the current window. Aim for $O(\log k)$ per arrival.

ANSWER KEY

1. $k \mathrel{%=} q.size()$ first (guard the empty queue), then repeat $q.push(q.front()); q.pop();$ k times.

P1

JS

```
const rotateLeft = (q, k) => {
  if (q.size === 0) return;
  k %= q.size;                                // read the size ONCE, before the
  loop
  for (let i = 0; i < k; i++) q.push(q.shift());
};
```

Real output: 1 2 3 4 5 with $k = 2 \rightarrow 3$ 4 5 1 2; empty queue $\rightarrow$ size 0; 7 8 with $k = 4 \rightarrow 7$ 8.

2. Keep a queue of zero indexes; expire the front as usual; if the queue is empty once the window is full, the answer is false.

P2

JS

```
const everyWindowHasZero = (a, k) => {
  const n = a.length;
  if (k <= 0 || k > n) return false;
  const zeros = new Deque();                  // indexes of zeros
  for (let i = 0; i < n; i++) {
    if (a[i] === 0) zeros.push(i);
    while (zeros.size && zeros.front() <= i - k) zeros.shift();
    if (i >= k - 1 && zeros.size === 0) return false;
  }
  return true;
};
```

Real output: ([1,0,2,3,0,4], 3) $\rightarrow$ true; ([1,0,2,3,4,0], 3) $\rightarrow$ false; ([0], 1) $\rightarrow$ true; $k > n \rightarrow$ false.

3. Exactly Example 3 with "1" and "2" seeded into the queue, appending "1" and "2" instead of "0" and "1".
4. The two characters are the comparison in rule 2: $\leq$ becomes $\geq$. Everything else is identical.
5. Same template, but push `a[dq.front()]`'s **index** into the result instead of the value. For ties, use strict $<$ in rule 2 so the older index survives.
6. Two dequeues, count when the window is full.

P6

JS

```
const countStableWindows = (a, k, L) => {
  const n = a.length;
  if (k <= 0 || k > n) return 0;
  const mx = new Deque(), mn = new Deque();
  let cnt = 0;
  for (let i = 0; i < n; i++) {
    while (mx.size && mx.front() <= i - k) mx.shift();
    while (mn.size && mn.front() <= i - k) mn.shift();
    while (mx.size && a[mx.back()] <= a[i]) mx.pop();
    while (mn.size && a[mn.back()] >= a[i]) mn.pop();
    mx.push(i); mn.push(i);
    if (i >= k - 1 && a[mx.front()] - a[mn.front()] <= L) cnt++;
  }
  return cnt;
};
```

Real output: ([1,3,2,6,4,4], 3, 2) → 2; ([5,5,5], 2, 0) → 2; empty → 0.

7. Variable window plus a count map; shrink while the map has more than K keys.

P7

JS

```
const longestAtMostKDistinct = (a, K) => {
  if (K <= 0) return 0;
  const cnt = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < a.length; right++) {
    cnt.set(a[right], (cnt.get(a[right]) || 0) + 1);
    while (cnt.size > K) {
      const c = cnt.get(a[left]) - 1;
      if (c === 0) cnt.delete(a[left]); else cnt.set(a[left], c);
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
};
```

Real output: ([1,2,1,3,4], 2) → 3; ([1,1,1], 1) → 3; $K=0 \rightarrow 0$; empty → 0.

8. First pass: count every ID in a hash map. Second pass: pop each ID and push it back only if its count is 1. Two passes, $O(n)$.
9. Identical to Example 10 but accumulate only `a[mn.front()]`. Check the total before you trust it: 10^5 windows times 10^9 is 10^{14} , which is still under `Number.MAX_SAFE_INTEGER` $\approx 9 \times 10^{15}$, so a plain number is fine. One more order of magnitude and you would need `BigInt`.
10. Example 23, approach 3. Answer for ([84, -37, 32, 40, 95], 167) is 3.
11. Entering a T costs 2, entering a normal cell costs 1. Subtract 1 from every cost so they become 0 and 1, run 0-1 BFS, then add back the number of steps — or split each cost-2 edge with a fake middle node and run plain 0-1 BFS. Both are $O(rc)$.
12. A toolkit `Deque` of timestamps. On every query, `shift` from the front while `front() <= now - W`, then return `.size`. Amortised $O(1)$ because each timestamp is shifted off once. This is `countInWindow` from Example 21.

13. Add the count to the state: $dp[i][t]$ = best total ending at i having chosen t items. For each fixed t the recurrence is again “max of $dp[i-k..i-1][t-1]$ ”, so run one deque per t . Time $O(nm)$, space $O(n)$ if you keep only two rows.
14. Use the contribution view of Example 27. For each i , find the span where $a[i]$ is the minimum using a monotonic stack, then take the maximum-sum subarray inside that span that contains i — with all values non-negative that is the **whole** span, so the value is $a[i] * (\text{prefix}[\text{right}] - \text{prefix}[\text{left}])$. One pass with a stack and prefix sums, $O(n)$.
15. JavaScript has no ordered multiset, so keep the window as a **sorted array**, exactly as in Example 26: `win.splice(lowerBound(win, a[i]), 0, a[i])` to insert, then `win.splice(lowerBound(win, a[i - k]), 1)` to remove one copy of the value leaving. The second largest is then just `win[win.length - 2]`. The binary search is $O(\log k)$; the `splice` moves up to k slots, which is the honest cost to quote. A monotonic deque cannot do this at all — it deliberately throws away the second largest.

ONE-PAGE REVISION CARD

The one template. Indexes in the deque. Three rules, in this order: expire the front, pop weaker from the back, push, then read the front.

memorise this

JS

```
const dq = new Deque();
for (let i = 0; i < n; i++) {
  while (dq.size && dq.front() <= i - k) dq.shift(); // 1 expire
  while (dq.size && a[dq.back()] <= a[i]) dq.pop(); // 2 dominate
  dq.push(i); // 3 join
  if (i >= k - 1) res.push(a[dq.front()]); // 4 read
}
```

Switches.

want	change
window maximum	rule 2 uses <= (deque decreasing)
window minimum	rule 2 uses >= (deque increasing)
variable-size window	replace rule 1 with “if front == left, pop front” inside the shrink loop
dp over the last k states	store dp values instead of array values; expiry becomes < i - k
prefix-sum problems	deque over p[0..n], loop to i <= n

When to use what.

problem shape	tool	time
max/min of every fixed window	monotonic deque	$O(n)$
median of every window	sorted array + lowerBound	$O(n \log k)$ search
second largest of every window	sorted array + lowerBound	$O(n \log k)$ search
longest window with max-min $\leq L$	two deques	$O(n)$
shortest window with sum $\geq S$, negatives	prefix + deque	$O(n)$
fewest moves, all costs equal	queue BFS	$O(V + E)$
costs are 0 or 1	0-1 BFS: two queues	$O(V + E)$
level-by-level spread	queue, fix sz per level	$O(V + E)$
queue with $O(1)$ max/min	queue + monotonic deque	amortised $O(1)$

Top JavaScript traps.

1. Never build a queue on `arr.shift()` or `arr.unshift()` — both are $O(n)$. Measured: 200000 `unshift` calls took **2695 ms**; the toolkit `Deque` did the same work in **15 ms**.
2. `dq.size` is a **getter**: no brackets. `dq.size()` throws `TypeError: dq.size is not a function`.
3. `arr.sort()` is **lexicographic**. `[10, 9, 1].sort()` gives `[1, 10, 9]`. Numbers always need `sort((a, b) => a - b)`.
4. No heap and no priority queue in the language: use the toolkit `MinHeap`, and flip the comparator to `(x, y) => y - x` for a max-heap.
5. No `TreeMap`, no ordered set, no multiset: use a sorted array with the toolkit `lowerBound` plus `splice`, and say the `splice` cost out loud.
6. `[...grid]` is a **shallow** copy. A 2-D grid needs `grid.map(row => [...row])`.

7. Reading past the end of a deque or array gives undefined, not an error — it turns into NaN several lines later. Guard with `dq.size` before every `front()` or `back()`.
8. `(head - 1) % cap` is `-1` when `head` is 0, because `%` keeps the sign of the left operand. Write `(head - 1 + cap) % cap`.

Top algorithm traps.

1. Store indexes, not values, or you cannot detect expiry.
2. In BFS, take `const sz = q.size` **before** the level loop.
3. In BFS, mark visited on **push**, not on pop.
4. Check every running total against `Number.MAX_SAFE_INTEGER  $\approx 9 \times 10^{15}$` ; past it, switch to `BigInt`.
5. For the two-stack queue, pour only when the output stack is empty — otherwise you lose the amortised $O(1)$.
6. In a dp deque, the window is `[i - k, i - 1]`, so expiry is `< i - k`, not `<= i - k`.
7. `MaxQueue` stores values, so its back-pop must use strict `<` to protect duplicates.

Complexity you should be able to quote instantly. Sliding window max/min: $O(n)$ time, $O(k)$ space. Sorted-array window: $O(n \log k)$ searching plus the `splice` moves. BFS on a grid: $O(rc)$. 0-1 BFS: $O(V + E)$. Two-stack queue: amortised $O(1)$.

Linked Lists

No indexes, no jumping — just one pointer at a time, and a dummy node to save you

Every JavaScript snippet in this chapter was run with `node` before it was printed. Each file uses the `node` class and the three helpers defined on the next page, and the one chapter that needs a heap starts with

```
const { MinHeap } = require('./toolkit.js'); // see the JS Toolkit appendix
```

The outputs shown under the snippets are the real outputs.

Pattern in one page

WHAT A LINKED LIST IS

A list is a chain of small boxes. Each box holds a value and a reference to the next box. The last box points at `null`.

```
head -> [1|.] -> [2|.] -> [3|.] -> null
```

You get: insert or delete anywhere in $O(1)$ if you already hold the right pointer, and no reallocation ever.

You lose: random access. Reaching position i costs $O(i)$. There is no `a[i]`.

the node, and three helpers used by every test in this chapter

JS

```
class Node {
  constructor(val) { this.val = val; this.next = null; }
}

const build = (a) => { // helper used by every test
  const dummy = new Node(0);
  let tail = dummy;
  for (const x of a) { tail.next = new Node(x); tail = tail.next; }
  return dummy.next;
};

const show = (head) => { // "1 -> 2 -> 3", or "(empty)"
  const parts = [];
  for (let p = head; p; p = p.next) parts.push(p.val);
  return parts.length ? parts.join(' -> ') : '(empty)';
};

const length = (head) => {
  let n = 0;
  for (let p = head; p; p = p.next) n++;
  return n;
};
```

There is no `delete` in JavaScript. When you unlink a node, nothing else points at it and the garbage collector reclaims it on its own. Every C++ solution to these problems has a `delete` `dead;` line — in JavaScript that line simply does not exist, and forgetting it costs you nothing.

The four moves — everything in this chapter is built from these

THE FOUR MOVES

Move 1 — the dummy head. Create a fake node in front of the real head: `const dummy = new Node(0); dummy.next = head;`. Now “delete the first node” and “delete a middle node” are the **same** code, and you return `dummy.next` at the end. This removes about half of all linked-list bugs.

Move 2 — reverse by flipping arrows. Three pointers: `prev`, `cur`, `nxt`. Save the next, flip the arrow, step both forward.

Move 3 — fast and slow. Walk one pointer twice as fast as the other. When fast reaches the end, slow is at the middle. If the list has a loop, fast catches slow from behind.

Move 4 — the head start. To find the node n from the end, move one pointer n steps first, then move both together until the leader falls off the end.

move 2 — reverse, the version to memorise

JS

```
const reverseList = (head) => {
  let prev = null;
  let cur = head;
  while (cur) {
    const nxt = cur.next; // 1. remember where to go
    cur.next = prev;      // 2. flip the arrow
    prev = cur;           // 3. step prev forward
    cur = nxt;            // 4. step cur forward
  }
  return prev;           // prev is the new head
};
```

Time $O(n)$ **Space** $O(1)$



after `cur.next = prev`; the chain behind `cur` already points backwards



One step of reverse. The arrow between prev and cur is flipped, then both slide right.

move 3 — fast and slow, the middle of the list

JS

```
const middle = (head) => { // 2nd middle when the count is even
  let slow = head, fast = head;
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  return slow;
};
```

Real output for 1 2 3 4 5 → 3; 1 2 3 4 → 3; 8 → 8; empty → returns null.

SHORTCUT

Start `fast` at `head` and you land on the **second** middle of an even list. Start `fast` at `head.next` and you land on the **first** middle. Choose deliberately: splitting a list in half needs the **first** middle, deleting the middle needs the **second**. Getting this wrong causes infinite recursion in merge sort.

TRAP

Order matters in `while (fast && fast.next)`. JavaScript's `&&` evaluates left to right and stops early, so `fast` is checked before `fast.next` is read. Swap them and you get `TypeError: Cannot read properties of null (reading 'next')` on the last step. Tested in Node — that is the exact message.

Optional chaining `fast?.next` also works and never throws, but `while (fast && fast.next)` is what an interviewer expects to see, so learn that one first.

The five questions to ask before you write a line

1	Can the list be empty? (<code>head === null</code>)
2	Can it have exactly one node? (<code>head.next === null</code>)
3	Does the head itself get deleted or moved? If yes, use a dummy node.
4	Am I about to read <code>p.next.next</code> ? Then I must have checked <code>p.next</code> .
5	Did I set the new last node's <code>next</code> to <code>null</code> ? A forgotten <code>null</code> is a silent infinite loop.

Warm-up — build the reflex

WARM-UP

build the reflex

Example 1 [warm-up]

Write `length`, `search`, `insertAt(pos, val)` (0-based; append if `pos` is past the end) and `deleteValue(val)` (remove the first match only). Constraints: up to 10^5 nodes. Edge cases: empty list, inserting at position 0, deleting a value that is not present, deleting the only node.

SOLUTION

`insertAt` and `deleteValue` both may touch the head, so both use a dummy node.

insert and delete with a dummy node

JS

```
const insertAt = (head, pos, val) => { // 0-based; pos > length -> append
  const dummy = new Node(0);
  dummy.next = head;
  let prev = dummy;
  for (let i = 0; i < pos && prev.next; i++) prev = prev.next;
  const fresh = new Node(val);
  fresh.next = prev.next;
  prev.next = fresh;
  return dummy.next;
};
const deleteValue = (head, val) => { // removes the FIRST match
  const dummy = new Node(0);
  dummy.next = head;
  for (let p = dummy; p.next; p = p.next)
    if (p.next.val === val) { p.next = p.next.next; break; }
```



```

    return dummy.next;
  };
  const search = (head, val) => {
    for (let p = head; p; p = p.next) if (p.val === val) return true;
    return false;
  };

```

Real output of the test run:

```

1 -> 2 -> 3 -> 4 -> 5
5 0
9 -> 1 -> 2 -> 3 -> 4 -> 5
9 -> 1 -> 2 -> 7 -> 3 -> 4 -> 5
9 -> 1 -> 2 -> 7 -> 3 -> 4 -> 5 -> 0
1 -> 2 -> 7 -> 3 -> 4 -> 5 -> 0
1 -> 2 -> 7 -> 3 -> 4 -> 5 -> 0
1 0

```

Line 2 is length of the list and of null. Line 5 shows `insertAt(99, 0)` appending at the end. Line 7 shows `deleteValue(1234)` leaving the list unchanged. Line 8 is `search(7)` then `search(100)`. Inserting into an empty list gives 42; deleting that node gives null back.

Time $O(n)$ **Space** $O(1)$

TRAP

The loop condition in `insertAt` is `i < pos && prev.next`. Without the second half, a `pos` larger than the list walks off the end and throws a `TypeError`. Without the first half you always append. Both guards are needed.

Example 2 [warm-up]

Print the middle node. For an even count, print the second of the two middles. Trace 1 2 3 4 by hand with the fast/slow pointers.

SOLUTION

step	slow	fast	check fast && fast.next
start	1	1	1 and 2 exist → continue
1	2	3	3 and 4 exist → continue
2	3	null	fast is null → stop

Answer: node 3. With 1 2 3 4 5 the trace stops with `fast` at node 5 and `fast.next === null`, and `slow` sits at node 3.

Answer: 3

Example 3 [warm-up]

Reverse a list, iteratively and recursively. Constraints: up to 10^5 nodes (think about recursion depth). Edge cases: empty list, one node, two nodes.

SOLUTION

The iterative version is Move 2. The recursive version reverses everything **after** the head, then makes the node behind the head point back at the head.

recursive reverse

JS

```
const reverseRec = (head) => {
  if (!head || !head.next) return head;
  const newHead = reverseRec(head.next);
  head.next.next = head;    // the node behind me now points back at me
  head.next = null;
  return newHead;
};
```

Real output of `show(reverseRec(...))` on 1 2 3 4 5, 7, null, 1 2 3 4 and -1 -2:

```
5 -> 4 -> 3 -> 2 -> 1
7
(empty)
4 -> 3 -> 2 -> 1
-2 -> -1
```

Line 2 is a one-node list, line 3 is `reverseRec(null)`, which returns `null` and prints as `(empty)`.

Time $O(n)$ **Space** iterative $O(1)$, recursive $O(n)$ stack

TRAP

`head.next = null`; must come **after** `head.next.next = head`; Reverse the two lines and you lose the reference you were about to use.

TRAP

JavaScript's recursion depth is about 10^4 , far below C++ or Java. I measured it in Node: a plain recursive function died at depth **12546** with `RangeError: Maximum call stack size exceeded`. I then ran `reverseRec` on a list of 10^5 nodes — same crash.

So `reverseRec` is a **teaching** version only. Constraints in this chapter go up to 10^5 nodes, which means the iterative `reverseList` is the only safe answer. Say “iterative, $O(1)$ space” first, and mention the recursion only as a follow-up, with the depth limit named out loud. The same warning applies to every recursive walk over a long list.

Example 4 [warm-up]

Delete a node when you are given **only** a pointer to that node — you cannot reach the head. Edge case: the node is the last one.

SOLUTION

You cannot unlink a node without its predecessor. So do not unlink it: **become** the next node. Copy the next node's value into this node, then delete the next node.

delete without the head

JS

```
const deleteGivenOnlyNode = (target) => { // works for any node except the last
  if (!target || !target.next) return false;
  const nxt = target.next;
  target.val = nxt.val; // copy the next node's data over mine
  target.next = nxt.next;
  return true;
};
```

On 1 2 3 4, deleting the node at index 1 gives real output:

```
1 1 -> 3 -> 4
0
```

The leading 1 is the return value. The final 0 is the attempt to delete the last node, which is impossible.

This is the answer the interviewer wants, **and** the honest caveat they want to hear: it is a trick, not a real deletion. Any other variable that was aimed at the **next** node still holds that node, now orphaned from the list but very much alive — JavaScript's collector only frees what nothing references.

Example 5 [warm-up]

Swap every pair of adjacent nodes: 1 2 3 4 5 becomes 2 1 4 3 5. Swap the **nodes**, not the values. Edge cases: odd length, one node, empty.

SOLUTION

swap nodes in pairs

JS

```
const swapPairs = (head) => {
  const dummy = new Node(0);
  dummy.next = head;
  let prev = dummy;
  while (prev.next && prev.next.next) {
    const a = prev.next, b = a.next;
    a.next = b.next; b.next = a; prev.next = b;
    prev = a;
  }
  return dummy.next;
};
```

Real output:

```
2 -> 1 -> 4 -> 3 -> 5
2 -> 1
1
```

Time $O(n)$ **Space** $O(1)$

Example 6 [warm-up]

Is the list sorted in non-decreasing order? Edge cases: empty list, one node, duplicates.

SOLUTION

sorted check

JS

```
const isSorted = (head) => {
  for (let p = head; p && p.next; p = p.next) if (p.val > p.next.val) return false;
  return true;
};
```

Real output for 1 2 2 3, 3 1, and null (printed as 1 for true, 0 for false):

```
101
```

An empty list and a one-node list are both sorted by definition.

Tier 1 — the questions everyone is asked

Example 7 [TCS NQT pattern]

Detect a loop, find where it starts, measure it, and remove it. Constraints: up to 10^6 nodes; memory is tight, so aim for $O(1)$ extra space. Edge cases: no loop; a node pointing at itself; the whole list being one big loop.

SOLUTION**Approach 1 — Remember every node in a hash set** ($O(n)$ time, $O(n)$ space)

Walk the list and add each node **object** to a Set. A JavaScript Set compares objects by identity, which is exactly what you want here — two different nodes holding the same value are still two entries. The first repeat is the start of the loop. Simple, correct, but uses $O(n)$ memory — and the interviewer will immediately ask for better.

Approach 2 — Fast and slow — Floyd's cycle rule ($O(n)$ time, $O(1)$ space — optimal)

Slow moves one step, fast moves two. If there is a loop, fast enters it and gains one step per move on slow, so it must eventually land on slow.

detect a loop

JS

```
const hasCycle = (head) => {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
};
```

Finding the start. Let the straight part before the loop be μ nodes long and the loop be λ nodes long. When the two pointers meet, slow has walked s steps and fast has walked $2s$. The extra distance fast covered is a whole number of laps:

$$2s - s = s = m\lambda$$

So s is a multiple of the loop length. Now put one pointer back at the head and move both one step at a time. The head pointer reaches the loop entrance after μ steps; the other pointer has already walked a multiple of λ , so after μ more steps it also sits at the entrance. They meet exactly there.

find the start, the length, and remove the loop

JS

```
const cycleStart = (head) => {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) { // they met: now walk one from the head
      let p = head;
      while (p !== slow) { p = p.next; slow = slow.next; }
      return p;
    }
  }
  return null;
};

const cycleLength = (head) => {
```

```

let slow = head, fast = head;
while (fast && fast.next) {
  slow = slow.next; fast = fast.next.next;
  if (slow === fast) {
    let len = 1;
    for (let p = slow.next; p !== slow; p = p.next) len++;
    return len;
  }
}
return 0;
};

const removeCycle = (head) => {
  const start = cycleStart(head);
  if (!start) return head;
  let p = start;
  while (p.next !== start) p = p.next;    // walk to the node pointing back
  p.next = null;
  return head;
};

```

Real output of the full test: a clean 5-node list; then the same list with node 5 pointing back at node 2; then after removal; then a one-node self-loop; then null input; then a 2-node list that is entirely a loop.

```

0 0 0
1 2 4
1 -> 2 -> 3 -> 4 -> 5
0
1 7 1
7
0 1
1 2

```

Read the first line as hasCycle, “is there a start?”, cycleLength — all zero on a clean list. Line 2 says the loop starts at the node holding 2 and is 4 nodes long.

Time $O(n)$ **Space** $O(1)$

The idea that unlocked it: two speeds turn “is there a loop?” into “will one runner lap the other?”, and the meeting point carries enough arithmetic to recover the entrance without any memory.

Python version — is is Python’s identity test, the same job as JavaScript’s === on two objects:

Floyd’s cycle start in Python

PYTHON

```

def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            p = head
            while p is not slow:
                p, slow = p.next, slow.next
            return p
    return None

```

Ran with python3: on a clean 5-node list it prints None; after pointing node 5 back at node 2 it prints the node holding 2.

TRAP

`removeCycle` must walk to the node whose `next` is the start, and set that node's `next` to `null`. Setting `start.next = null` instead cuts the list in the wrong place and loses everything after the entrance.

TRAP

Use `===` on nodes, never `==` and never a value test. `slow === fast` asks “is this the same object?”, which is the question Floyd's rule is about. `slow.val === fast.val` is a different question and is wrong the moment two nodes share a value.

Example 8 [Infosys · pattern]

Merge two sorted lists into one sorted list, reusing the existing nodes (no new `Node`).

Constraints: up to 10^5 nodes each. Edge cases: one list empty, both empty, all values equal, negative values.

SOLUTION

This is the single most useful linked-list routine in the book — merge sort, merge-K and the flatten problem all call it.

merge two sorted lists

JS

```
const mergeSorted = (x, y) => {
  const dummy = new Node(0);
  let tail = dummy;
  while (x && y) {
    if (x.val <= y.val) { tail.next = x; x = x.next; }
    else { tail.next = y; y = y.next; }
    tail = tail.next;
  }
  tail.next = x || y; // attach whatever is left
  return dummy.next;
};
```

Real output:

```
1 -> 2 -> 3 -> 4 -> 5 -> 6
1 -> 1 -> 1 -> 1 -> 1
2 -> 9
(empty)
-5 -> -3 -> -1 -> 0
```

Time $O(n + m)$ **Space** $O(1)$

SHORTCUT

`tail.next = x || y;` replaces two extra while-loops. One of `x` and `y` is already `null` when the loop ends, and `||` returns the first **truthy** operand — so this attaches whatever is left in a single line. If both are `null`, `null || null` is `null`, which is also right.

TRAP

Use `<=`, not `<`. With `<` the merge is still sorted but no longer **stable** — equal values from the second list jump ahead of equal values from the first. Merge sort built on an unstable merge is an unstable sort, and interviewers do ask.

Python version — the same shape, and `x` or `y` is the same trick as JavaScript's `x || y`.

```
def merge_sorted(x, y):
    dummy = Node(0)
    tail = dummy
    while x and y:
        if x.val <= y.val:
            tail.next, x = x, x.next
        else:
            tail.next, y = y, y.next
        tail = tail.next
    tail.next = x or y
    return dummy.next
```

Ran with python3: [1, 3, 5] merged with [2, 4] prints [1, 2, 3, 4, 5]; None merged with [9] prints [9].

Example 9 [Accenture · pattern]

Remove the n -th node from the end in one pass. Constraints: up to 10^5 nodes, $n \geq 1$. Edge cases: removing the head (n equals the length), a one-node list, n larger than the list.

SOLUTION

Approach 1 — Count, then walk again ($O(n)$, two passes)

Measure the length L , then delete the node at position $L - n$ from the front. Perfectly acceptable — but the interviewer will say “one pass”.

Approach 2 — Head start of n ($O(n)$, one pass — optimal)

Move *fast* n steps ahead. Then move *fast* and *slow* together. When *fast* reaches the last node, *slow* sits just before the node to delete. Starting both at the dummy makes “delete the head” need no special case at all.

remove the n -th from the end

JS

```
const removeNthFromEnd = (head, n) => {
    const dummy = new Node(0);
    dummy.next = head;
    let fast = dummy, slow = dummy;
    for (let i = 0; i < n; i++) {
        if (!fast.next) return head; // n is bigger than the list
        fast = fast.next;
    }
    while (fast.next) { fast = fast.next; slow = slow.next; }
    slow.next = slow.next.next; // no delete needed: nothing points at it now
    return dummy.next;
};
```

Real output for 1 2 3 4 5 with $n = 2$; 1 2 3 with $n = 3$ (deletes the head); 9 with $n = 1$; 1 2 with $n = 5$ (too big):

```
1 -> 2 -> 3 -> 5
2 -> 3
(empty)
1 -> 2
```

Time $O(n)$ **Space** $O(1)$

The idea that unlocked it: a fixed gap of n between two pointers converts “count from the end” into “walk until the leader falls off”, which needs no length at all.

Example 10 [Wipro · pattern]

Remove duplicates — (a) from a **sorted** list, (b) from an **unsorted** list, keeping the first occurrence of each value. Constraints: up to 10^5 nodes. Edge cases: all values equal, no duplicates, empty list, negative values.

SOLUTION

(a) Sorted: duplicates are neighbours, so compare `p` with `p.next`.

(b) Unsorted: a Set of the values already kept.

both versions

JS

```
const dedupSorted = (head) => {
  for (let p = head; p && p.next; ) {
    if (p.val === p.next.val) p.next = p.next.next;
    else p = p.next;
  }
  return head;
};
const dedupUnsorted = (head) => {
  const seen = new Set();
  const dummy = new Node(0);
  dummy.next = head;
  for (let p = dummy; p.next; ) {
    if (seen.has(p.next.val)) p.next = p.next.next;
    else { seen.add(p.next.val); p = p.next; }
  }
  return dummy.next;
};
```

Real output:

```
1 -> 2 -> 3 -> 4
5
(empty)
4 -> 2 -> 9
-1
```

Time (a) $O(n)$, (b) $O(n)$ average **Space** (a) $O(1)$, (b) $O(n)$

TRAP

Do **not** advance `p` on the step where you delete. The new `p.next` has not been compared yet. Writing the loop as a for with `p = p.next` in the header is exactly how people miss 1 1 1. Notice both loops above have an **empty** increment slot on purpose.

Use a **Set**, not a plain object, for “have I seen this value?” A plain object turns every key into a string, so the number 1 and the string “1” collide, and `delete obj[k]` is slow. A Set keeps the type and is built for exactly this. The same applies to Map versus an object when you need `key → value`.

Example 11 [Capgemini · pattern]

Is the list a palindrome? Target $O(1)$ extra space, and leave the list exactly as you found it. Constraints: up to 10^5 nodes. Edge cases: empty, one node, two equal nodes, even and odd lengths.

SOLUTION

Approach 1 — Copy to an array ($O(n)$ time, $O(n)$ space)

Push all values into an array, then compare with two indexes. Five lines, and it is the answer most people give first. Say it, then improve it.

Approach 2 — Push onto a stack, then re-walk (still $O(n)$ space)

Same cost, no gain.

Approach 3 — Reverse the back half in place ($O(n)$ time, $O(1)$ space — optimal)

Find the middle with fast/slow, reverse from the middle onwards, compare the two halves, then reverse the back half again to restore the list.

palindrome check in $O(1)$ space

JS

```
const isPalindrome = (head) => {
  if (!head || !head.next) return true;
  let slow = head, fast = head;           // slow ends at the 2nd middle
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  let prev = null;                         // reverse the back half
  while (slow) { const nxt = slow.next; slow.next = prev; prev = slow; slow =
nxt; }
  let p = head, q = prev;
  let ok = true;
  while (q) { if (p.val !== q.val) { ok = false; break; } p = p.next; q = q.next; }
  let back = null;                         // put the list back together
  while (prev) { const nxt = prev.next; prev.next = back; back = prev; prev =
nxt; }
  return ok;
};
```

Real output for 1 2 3 2 1, 1 2 2 1, 1 2 3, 7, null, 1 2:

110110

And the restoration check on 1 2 2 1:

1 1 -> 2 -> 2 -> 1

The list is unchanged after the call.

Time $O(n)$ **Space** $O(1)$

SHORTCUT

Stop the comparison loop on q , not on p . For an odd length the front half is one node longer, and the middle node is allowed to be anything. Looping on q handles odd and even lengths with no `if`.

Example 12 [Cognizant · pattern]

Where do two lists merge? Two lists may share a common tail. Return the first shared node, or `null`. Constraints: up to 10^5 nodes each. Edge cases: no shared tail; the two heads are the same node; one list empty; one list being a suffix of the other.

SOLUTION

Approach 1 — A Set holding one list's nodes ($O(n + m)$ time, $O(n)$ space)

A Set of node **objects**, not of values — identity is the question, and JavaScript's Set compares objects by identity.

Approach 2 — Line up the tails by length ($O(n + m)$ time, $O(1)$ space)

Measure both lengths, advance the longer list's pointer by the difference, then walk together. Once the remaining lengths match, the merge node (if any) is reached by both at the same moment.

intersection by length

JS

```
const intersectionByLength = (a, b) => {
  let la = length(a), lb = length(b);
  while (la > lb) { a = a.next; la--; } // drop the longer list's head start
  while (lb > la) { b = b.next; lb--; }
  while (a && b && a !== b) { a = a.next; b = b.next; }
  return a === b ? a : null;
};
```

Approach 3 — Two pointers that swap lists (same cost, four lines)

Each pointer walks its own list, then continues onto the **other** list. Both then cover exactly $n + m$ nodes, so they arrive at the merge node together. If there is no merge node, both become null at the same step and the loop ends.

```
the swap trick JS
const intersectionTwoPointer = (a, b) => {
  if (!a || !b) return null;
  let p = a, q = b;
  while (p !== q) { // each walks its own list then the
other
    p = p ? p.next : b;
    q = q ? q.next : a;
  }
  return p; // meeting point, or null
};
```

Real output (lists 1 2 3 -> 8 9 10 and 7 -> 8 9 10, then disjoint lists, then null input, then the same list twice, then one list being a suffix of the other). Each line shows the two approaches side by side, and they agree every time:

```
1 -> 2 -> 3 -> 8 -> 9 -> 10
7 -> 8 -> 9 -> 10
8 8
1 1
1 1
5 5
3 3
```

Lines 4 and 5 print 1 1 meaning “both returned null”, which is correct for disjoint lists and for a null input.

Time $O(n + m)$ **Space** $O(1)$

TRAP

The loop in approach 3 must step to the **other list’s head**, not restart on its own list. `p = p ? p.next : b;` — note the `b`. Restarting on its own list loops forever when there is no intersection.

Example 13 [TCS Digital · pattern]

Add two numbers stored as lists. (a) Digits are stored least-significant-first. (b) Digits are stored most-significant-first. Constraints: up to 10^5 digits, so you cannot turn the list into a number. Edge cases: different lengths; a final carry that adds a digit; both numbers zero; one list empty.

SOLUTION

(a) Least-significant-first is the easy direction: walk both lists together, carry as you go.

digits stored backwards

JS

```
// digits stored LEAST significant first: 342 is 2 -> 4 -> 3
const addReversed = (a, b) => {
  const dummy = new Node(0);
  let tail = dummy;
  let carry = 0;
  while (a || b || carry) {
    let s = carry;
    if (a) { s += a.val; a = a.next; }
    if (b) { s += b.val; b = b.next; }
    carry = Math.floor(s / 10); // NOT s / 10 - JS division gives 1.2, not 1
    tail.next = new Node(s % 10);
    tail = tail.next;
  }
  return dummy.next;
};
```

(b) Most-significant-first needs the digits in reverse. Either reverse both lists, or push them onto two stacks. The stack version does not modify the inputs, and it builds the answer by pushing at the **front**, which reverses for free.

digits stored forwards, using stacks

JS

```
// digits stored MOST significant first: 342 is 3 -> 4 -> 2
const addForward = (a, b) => {
  const sa = [], sb = []; // a plain array IS the stack in JS
  for (let p = a; p; p = p.next) sa.push(p.val);
  for (let p = b; p; p = p.next) sb.push(p.val);
  let head = null;
  let carry = 0;
  while (sa.length || sb.length || carry) {
    let s = carry;
    if (sa.length) s += sa.pop();
    if (sb.length) s += sb.pop();
    carry = Math.floor(s / 10);
    const fresh = new Node(s % 10); // build backwards: push at the FRONT
    fresh.next = head;
    head = fresh;
  }
  return head;
};
```

Real output — $342 + 465 = 807$, $999 + 1 = 1000$, $0 + 0$, empty + 5, then the forward versions $342 + 465$ and $99 + 1$, then empty + empty:

```
7 -> 0 -> 8
0 -> 0 -> 0 -> 1
0
5
8 -> 0 -> 7
1 -> 0 -> 0
(empty)
```

Time $O(n + m)$ **Space** (a) $O(1)$ extra, (b) $O(n + m)$

TRAP

The loop condition is `while (a || b || carry)`. Drop the `|| carry` and `999 + 1` prints `000` — the leading 1 is lost. This is the single most common bug in this question.

TRAP

carry = s / 10 is a C++ habit and it is wrong in JavaScript. There is no integer division here: `19 / 10` is 1.9, and the next digit comes out as 1.9, not 1. Use `Math.floor(s / 10)`, or `(s / 10) | 0` for non-negative values. The same applies to every “divide and keep the whole part” line in this book.

TRAP

Do not try to shortcut this by converting the list to a number. Every JavaScript number is a double and is exact only up to `Number.MAX_SAFE_INTEGER`, which is 9007199254740991 — 16 digits. Tested in Node: `Number("9".repeat(20))` prints 10000000000000000000, but the true value is twenty nines. With 10^5 digits the digit-by-digit walk above is the only correct answer. `BigInt` is exact — `BigInt("9".repeat(20)) + 1n` gives 100000000000000000000n — but building a 10^5 -digit `BigInt` from a list is slower and misses the point of the question.

Example 14 [Infosys · pattern]

Rotate the list to the right by k places. 1 2 3 4 5 with $k = 2$ becomes 4 5 1 2 3. Constraints: k may be far larger than the list length. Edge cases: $k = 0$; k a multiple of the length; a one-node list; empty list.

SOLUTION

Close the list into a ring, then cut it in the right place.

```
rotate right JS
```

```
const rotateRight = (head, k) => {  
  if (!head || !head.next || k <= 0) return head;  
  let n = 1, tail = head;  
  while (tail.next) { tail = tail.next; n++; }  
  k %= n;  
  if (k === 0) return head;  
  tail.next = head;           // close the ring  
  let newTail = head;  
  for (let i = 0; i < n - k - 1; i++) newTail = newTail.next;  
  const newHead = newTail.next;  
  newTail.next = null;        // cut the ring  
  return newHead;  
};
```

Real output for $k = 2$ on 1 2 3 4 5; $k = 3$ on 1 2 3; $k = 7$ on 1 2 3; $k = 5$ on 1; and null:

```
4 -> 5 -> 1 -> 2 -> 3
1 -> 2 -> 3
3 -> 1 -> 2
1
(empty)
```

Time $O(n)$ **Space** $O(1)$

TRAP

$k \% n$ must happen **before** anything else, and the $k === 0$ early return must happen **before** the ring is closed — otherwise you close the ring and never cut it, and the next `show(head)` never returns.

Example 15 [Wipro · pattern]

Split by position. Put the nodes at odd positions (1st, 3rd, 5th, ...) first, then the nodes at even positions, keeping the relative order inside each group. Positions, not values. Constraints: $O(1)$ space. Edge cases: fewer than three nodes; even and odd lengths.

SOLUTION

Run two tails at once, hopping over each other.

odd positions then even positions

JS

```
const oddEvenSplit = (head) => {           // positions 1,3,5,... then 2,4,6,...
  if (!head || !head.next) return head;
  let odd = head, even = head.next;
  const evenHead = even;
  while (even && even.next) {
    odd.next = even.next; odd = odd.next;
    even.next = odd.next; even = even.next;
  }
  odd.next = evenHead;
  return head;
};
```

Real output for 1 2 3 4 5, 1 2 3 4, 1:

```
1 -> 3 -> 5 -> 2 -> 4
1 -> 3 -> 2 -> 4
1
```

Time $O(n)$ **Space** $O(1)$

Example 16 [Capgemini · pattern]

Partition around a value x : every node with value $< x$ must come before every node with value $\geq x$, and the original relative order must be preserved inside each group. Edge cases: all values on one side; negative x ; empty list.

SOLUTION

Build two chains with two dummy heads, then glue them.

stable partition

JS

```
const partitionAround = (head, x) => {    // all < x first, order kept
  const lo = new Node(0), hi = new Node(0);
  let lt = lo, ge = hi;
  for (let p = head; p; p = p.next) {
    if (p.val < x) { lt.next = p; lt = p; }
    else { ge.next = p; ge = p; }
  }
  ge.next = null;
  lt.next = hi.next;
  return lo.next;
};
```

Real output for 1 4 3 2 5 2 with $x = 3$; 5 5 5 with $x = 1$; -3 -1 -2 with $x = -2$:

```
1 -> 2 -> 2 -> 4 -> 3 -> 5
5 -> 5 -> 5
-3 -> -1 -> -2
```

Time $O(n)$ **Space** $O(1)$

TRAP

`ge.next = null`; before gluing. Without it the last node of the “big” chain still points into the middle of the original list, and you get a loop. Call `show` on your answer in a test — a run that never finishes is the symptom.

Tier 2 — applied, two or three steps

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Grab · pattern]

Rebuild the pick-up order. A route is stored as a list of stops. Reorder it so that it becomes first stop, last stop, second stop, second-last stop, and so on: 1 2 3 4 5 becomes 1 5 2 4 3. Constraints: $O(1)$ space, up to 10^5 stops. Edge cases: 0, 1 or 2 stops; even length.

SOLUTION

Three known moves in a row: find the **first** middle, reverse the back half, then weave.

reorder: 1 n 2 (n-1) 3 ...

JS

```
const reorder = (head) => { // 1 2 3 4 5 -> 1 5 2 4 3
  if (!head || !head.next) return head;
  let slow = head, fast = head.next; // slow ends at the FIRST middle
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  let back = reverseList(slow.next);
  slow.next = null;
  let front = head;
  while (back) {
    const fn = front.next, bn = back.next;
    front.next = back; back.next = fn;
    front = fn; back = bn;
  }
  return head;
};
```

Real output for 1 2 3 4 5, 1 2 3 4, 1 2, 9, empty:

```
1 -> 5 -> 2 -> 4 -> 3
1 -> 4 -> 2 -> 3
1 -> 2
9
(empty)
```

Time $O(n)$ **Space** $O(1)$

The idea that unlocked it: a hard-looking rearrangement is three easy moves stacked. Whenever the target order reads “front, back, front, back”, think **split, reverse, weave**.

TRAP

`fast = head.next` gives the **first** middle, which puts the extra node in the **front** half. That is what makes the weave loop terminate cleanly on back. Start `fast` at `head` instead and `1 2 3 4` comes out wrong.

Example 18 [Shopee · pattern]

Sort a list with no extra array. Sort a linked list of up to 10^5 nodes in $O(n \log n)$ time and $O(\log n)$ stack space. Edge cases: already sorted; reverse sorted; duplicates; negatives; empty; one node.

SOLUTION

Approach 1 — Copy to an array, sort, copy back $(O(n \log n)$ time, $O(n)$ space)

Walk the list pushing values into an array, `vals.sort((a, b) => a - b)`, then walk the list again writing them back. It works and it is fast, but it defeats the point of the question.

TRAP

`arr.sort()` with no comparator sorts **LEXICOGRAPHICALLY, as text**. Tested in Node: `[10, 9, 1].sort()` returns `[1, 10, 9]`, because the string "10" is less than "9". Numbers **always** need `arr.sort((a, b) => a - b)`, which returns `[1, 9, 10]`. This one missing comparator is the most common JavaScript bug in coding rounds — and it is silent, because the code still runs and still returns a sorted-looking array.

Approach 2 — Insertion sort on the list $(O(n^2))$

Fine for tiny lists, too slow at 10^5 .

Approach 3 — Merge sort on the list $(O(n \log n)$ time, $O(\log n)$ space — optimal)

Merge sort is the natural list sort: splitting is free (cut the middle) and merging is the routine from Example 8. Quicksort is a bad fit here because a list has no random access for partitioning by index.

The recursion here is safe: it splits in half every time, so the depth is $\log_2 n$, about 17 for $n = 10^5$ — nowhere near JavaScript's limit of roughly 10^4 frames.

merge sort a linked list

JS

```
const sortList = (head) => {
  if (!head || !head.next) return head;
  let slow = head, fast = head.next; // FIRST middle, so the split is even
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  const second = slow.next;
  slow.next = null;
  return mergeSorted(sortList(head), sortList(second));
};
```

Real output for `4 2 1 3; -1 5 3 4 0; 2 2 1; 8; empty`:

```
1 -> 2 -> 3 -> 4
-1 -> 0 -> 3 -> 4 -> 5
1 -> 2 -> 2
8
(empty)
```

I also ran it on 5000 random values from -1000 to 999 and compared the result against `[...input].sort((a, b) => a - b)`. They matched exactly.

Time $O(n \log n)$ **Space** $O(\log n)$ recursion

TRAP

`fast = head.next` again. With `fast = head` a two-node list splits into “two nodes and zero nodes”, `sortList` is called on the same list forever, and Node throws `RangeError: Maximum call stack size exceeded`. This is **the** classic merge-sort-on-a-list bug.

Example 19 [Agoda · pattern]

Flatten a list of sorted columns. Each node has a next pointer going right and a down pointer into a sorted column. Every column is sorted. Produce one sorted list linked only by down. Constraints: total nodes up to 10^5 . Edge cases: one column; empty input; negative values.

SOLUTION

Merge the columns from right to left. Because each column is already sorted, every merge is the routine from Example 8 — just using down instead of next.

flatten by repeated merging

JS

```
class FNode {
  constructor(val) { this.val = val; this.next = null; this.down = null; }
}

const mergeDown = (a, b) => { // merge using the DOWN pointer only
  const dummy = new FNode(0);
  let tail = dummy;
  while (a && b) {
    if (a.val <= b.val) { tail.down = a; a = a.down; }
    else { tail.down = b; b = b.down; }
    tail = tail.down;
  }
  tail.down = a || b;
  return dummy.down;
};

const flatten = (head) => {
  if (!head || !head.next) return head;
  head.next = flatten(head.next); // flatten everything to the right
  first
  head = mergeDown(head, head.next);
  head.next = null;
  return head;
};
```

Real output for the columns [5,7,8,30], [10,20], [19,22,50], [28,35,40,45]; then a single column; then empty; then two columns of negatives:

```
5 7 8 10 19 20 22 28 30 35 40 45 50
3 4
1
-5 -3 -1 0
```

Time $O(Nk)$ worst case **Space** $O(k)$ recursion

N total nodes, k columns. Merging right to left touches a node once per merge it survives; a heap-based merge gives $O(N \log k)$.

TRAP

`head.next = null`; after the merge. The flattened list must be linked only by down; leaving stale next references means a later traversal walks into already-merged nodes.

TRAP

The recursion here is one frame per **column**. With k columns up to 10^4 you are close to JavaScript's roughly 10^4 frame limit, and past it you get `RangeError: Maximum call stack size exceeded`. The safe rewrite is a loop: collect the column heads into an array first, then merge them from the last one backwards.

Example 20 [SCB · pattern]

Reverse in groups of k . 1 2 3 4 5 6 7 with $k = 3$ becomes 3 2 1 6 5 4 7. A trailing group shorter than k is left as it is. Constraints: $O(1)$ space, up to 10^5 nodes. Edge cases: $k = 1$; k larger than the list; empty list; length an exact multiple of k .

SOLUTION

Walk group by group. Before touching a group, check that k nodes really exist. Then reverse it by **head insertion**: repeatedly take the node after the group's first node and move it to the group's front.

reverse in groups of k

JS

```
const reverseInGroups = (head, k) => {
  if (k <= 1 || !head) return head;
  const dummy = new Node(0);
  dummy.next = head;
  let groupPrev = dummy;
  for (;;) {
    let check = groupPrev; // is a full group of k left?
    for (let i = 0; i < k && check; i++) check = check.next;
    if (!check) break;
    const prev = groupPrev.next; // will become the group's tail
    let cur = prev.next;
    for (let i = 1; i < k; i++) { // standard head-insertion
      prev.next = cur.next;
      cur.next = groupPrev.next;
      groupPrev.next = cur;
      cur = prev.next;
    }
    groupPrev = prev;
  }
  return dummy.next;
};
```

Real output for 1..7 with $k = 3$; 1..4 with $k = 2$; 1 2 3 with $k = 5$; 1 2 3 with $k = 1$; null:

```
3 -> 2 -> 1 -> 6 -> 5 -> 4 -> 7
2 -> 1 -> 4 -> 3
1 -> 2 -> 3
1 -> 2 -> 3
1
```

Time $O(n)$ **Space** $O(1)$

The idea that unlocked it: groupPrev is a stable anchor that never moves during the reversal, so every node can be inserted right behind it. Without the anchor you need to track four pointers and the code triples in size.

Variant asked straight after: reverse **alternate** groups — reverse one group of k , skip the next, reverse the one after. Same loop with a `let doReverse = true` flag that flips each round; in the skip branch just walk forward $k - 1$ times instead of inserting. Real output for 1..8 with $k = 2$ and 1..7 with $k = 3$:

```
2 -> 1 -> 3 -> 4 -> 6 -> 5 -> 7 -> 8
3 -> 2 -> 1 -> 4 -> 5 -> 6 -> 7
```

Example 21 [LINE MAN · pattern]

Merge K sorted delivery queues into one sorted list. There are k lists with N nodes in total. Constraints: $k \leq 10^4$, $N \leq 10^5$. Edge cases: some lists empty; all lists empty; $k = 0$; $k = 1$.

SOLUTION

Approach 1 — Merge one at a time into an accumulator $(O(Nk))$

Merging list 2 into list 1, then list 3 into that, and so on, re-walks the growing accumulator every time. With $k = 10^4$ that is far too slow.

Approach 2 — Merge in pairs, like a tournament $(O(N \log k))$

Round 1 merges $k/2$ pairs, round 2 merges $k/4$ pairs, and so on. Every node is touched once per round, and there are $\log k$ rounds.

pairwise merging

JS

```
const mergeK = (lists) => {
  if (lists.length === 0) return null;
  while (lists.length > 1) {
    const nxt = [];
    for (let i = 0; i + 1 < lists.length; i += 2) nxt.push(mergeSorted(lists[i],
lists[i + 1]));
    if (lists.length % 2) nxt.push(lists[lists.length - 1]);
    lists = nxt;
  }
  return lists[0];
};
```

Approach 3 — A min-heap of the k current heads $(O(N \log k)$, streaming-friendly)

Same complexity, but it produces the output **in order as it goes** and needs only the k heads in memory — the right answer when the lists are files or network streams.

TRAP

JavaScript has no built-in priority queue or heap. Python has `heapq`, Java has `PriorityQueue`, C++ has `priority_queue` — JavaScript has nothing. Use the tested `MinHeap` from the **JS Toolkit** appendix, and say out loud in the interview that you are bringing your own, because the interviewer is watching for exactly this.

min-heap merge

JS

```
const { MinHeap } = require('./toolkit.js'); // JS has no priority queue

const mergeKHeap = (lists) => {
  const pq = new MinHeap((a, b) => a.val - b.val); // compare NODES by value
  for (const h of lists) if (h) pq.push(h);
  const dummy = new Node(0);
  let tail = dummy;
  while (pq.size) {
    const best = pq.pop();
    tail.next = best; tail = best;
    if (best.next) pq.push(best.next);
  }
  tail.next = null;
  return dummy.next;
};
```

Real output for three lists, then an empty array, then an array holding a single list — and then the heap version on the same three lists and on an array holding two null lists:

```
1 -> 1 -> 2 -> 3 -> 4 -> 4 -> 5 -> 6
(empty)
3
1 -> 1 -> 2 -> 3 -> 4 -> 4 -> 5 -> 6
(empty)
```

Time $O(N \log k)$ **Space** pairwise $O(1)$, heap $O(k)$

TRAP

The toolkit `MinHeap` takes a comparator with the same meaning as `Array.prototype.sort: (a, b) => a.val - b.val` means “a comes first when this is negative”. Write `(a, b) => b.val - a.val` and you have a **max**-heap, and the merge comes out perfectly reverse-sorted — easy to miss on a tiny test. Check the sign on a three-element case before you trust it.

TRAP

`tail.next = null`; at the end of the heap version. The last node popped still carries its old `next`, which may point back into a list you already consumed.

Tier 3 — you need the insight

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 22 [Amazon · pattern]

Deep-copy a list where every node also has a rand reference to any node in the list, or to null. Constraints: up to 10^5 nodes. Target: $O(1)$ extra space beyond the new nodes. Edge cases: a node whose `rand` points at itself; a `rand` pointing at the head; null input; a one-node list.

SOLUTION

Approach 1 — A Map from old node to new node ($O(n)$ time, $O(n)$ space)

Pass 1 creates a copy of each node and records `old -> new` in a Map. Pass 2 sets `copy.get(p).next = copy.get(p.next)` and `copy.get(p).rand = copy.get(p.rand)`. This **must** be a Map, not a plain object — object keys are strings, and a node is not a string. Correct, easy, and the $O(n)$ map is exactly what the follow-up asks you to remove.

Approach 2 — Weave the copies into the original list ($O(n)$ time, $O(1)$ extra space — optimal)

Three passes and no map at all.

1. Insert each copy directly **behind** its original: `A -> A' -> B -> B' -> ...`. Now the copy of any node `p` is simply `p.next`.
2. Wire the random references: the copy of `p.rand` is `p.rand.next`.
3. Unweave: split the interleaved chain back into two lists.

copy with random pointers, no map

JS

```
class RNode {
  constructor(val) { this.val = val; this.next = null; this.rand = null; }
}
```

```

const copyRandomList = (head) => {
  if (!head) return null;
  for (let p = head; p; ) {
    const c = new RNode(p.val);
    c.next = p.next; p.next = c; p = c.next;
  }
  for (let p = head; p; p = p.next.next) // 2. wire the random references
    p.next.rand = p.rand ? p.rand.next : null;
  const dummy = new RNode(0); // 3. split the two lists apart
  let tail = dummy;
  for (let p = head; p; ) {
    const c = p.next;
    p.next = c.next;
    tail.next = c; tail = c;
    p = p.next;
  }
  return dummy.next;
};

```

Real output for the list 7 -> 13 -> 11 with randoms null, 7, 13:

```

7(rand=null) 13(rand=7) 11(rand=13)
1
7(rand=null) 13(rand=7) 11(rand=13)
1
4 1 1

```

Line 2 is the check “no copied node points at any original node”, done with a Set of the original node objects — it is a genuine deep copy. Line 3 shows the original list restored exactly. Line 4 is null input. Line 5 is a one-node list whose rand points at itself: the copy’s rand points at the **copy**, and `copy !== original` is true.

Time $O(n)$ **Space** $O(1)$ extra

The idea that unlocked it: the map from old to new is a single reference, and a linked list can carry a reference for free. Weaving stores the whole map inside the next fields, then throws it away.

TRAP

Pass 2 and pass 3 must be separate loops. If you unweave while still wiring randoms, the `p.rand.next` you need has already been repaired to point at the original’s successor. Attempting both in one pass is the most common failed attempt.

Follow-up: “what if the list has a loop as well?” Nothing changes. Every pass here walks `next` a fixed number of times per node and never uses `null` as the loop end except in pass 1 — rework pass 1 to stop when it returns to the head and the rest is identical.

Example 23 [Microsoft · pattern]

Design an LRU cache with `get(key)` and `put(key, value)` both in $O(1)$. When the cache is full, evict the least recently used key. A `get` counts as a use; a `put` on an existing key counts as a use. Constraints: capacity up to 10^5 , up to 10^6 operations. Edge cases: capacity 1; `get` on a missing key; updating an existing key must not evict.

SOLUTION

Two structures glued together.

1. A Map from key to node, for $O(1)$ lookup.
2. A **doubly** linked list in use order, most recent at the front. Doubly, because evicting the tail and moving a node to the front both need the node’s **predecessor**, and a singly linked list cannot give you that in $O(1)$.

Two guard nodes (head and tail) mean no null checks anywhere in unlink and pushFront.

LRU cache: Map + doubly linked list

JS

```
class DNode { // doubly linked so unlink is O(1)
  constructor(key, val) { this.key = key; this.val = val; this.prev = null;
    this.next = null; }
}

class LRUCache {
  constructor(cap) {
    this.cap = cap;
    this.pos = new Map(); // key -> node
    this.head = new DNode(0, 0); // head side = most recent
    this.tail = new DNode(0, 0);
    this.head.next = this.tail; // two guard nodes, never removed
    this.tail.prev = this.head;
  }
  unlink(n) { n.prev.next = n.next; n.next.prev = n.prev; }
  pushFront(n) {
    n.next = this.head.next; n.prev = this.head;
    this.head.next.prev = n; this.head.next = n;
  }
  get(key) {
    const n = this.pos.get(key);
    if (n === undefined) return -1;
    this.unlink(n); this.pushFront(n);
    return n.val;
  }
  put(key, val) {
    const n = this.pos.get(key);
    if (n !== undefined) {
      n.val = val;
      this.unlink(n); this.pushFront(n);
      return;
    }
    if (this.pos.size === this.cap) {
      const dead = this.tail.prev; // least recent
      this.unlink(dead); this.pos.delete(dead.key);
    }
    const fresh = new DNode(key, val);
    this.pos.set(key, fresh); this.pushFront(fresh);
  }
}
```

SHORTCUT

The JavaScript-only shortcut. A Map already remembers insertion order, and `map.keys().next().value` gives the oldest key in $O(1)$. So an LRU can be written with a Map alone: on a hit, `map.delete(key)` then `map.set(key, val)` to move the entry to the back; when full, delete the first key. It is six lines and it is genuinely $O(1)$.

Show the doubly linked list **first** — the interviewer is testing whether you know why the list is needed — then offer the Map version as “and in JavaScript I would ship this”. That answer scores on both counts.

Real output for capacity 2: `put(1,10), put(2,20), get(1), put(3,30), get(2), put(1,99), get(42)`; then capacity 1:

```

2:20 1:10
10
1:10 2:20
-1 3:30 1:10
1:99 3:30
-1
-1 6 6:6

```

The dump shows the use order, most recent first. After `get(1)` the order flips. Putting key 3 evicts key 2, so `get(2)` returns `-1`.

Time $O(1)$ per operation **Space** $O(\text{capacity})$

The idea that unlocked it: neither structure alone can do it. The map gives $O(1)$ **find**; the list gives $O(1)$ **reorder**. Storing the key inside the node is what lets the eviction remove the map entry without searching.

TRAP

The node must store its own key, not just the value. On eviction you hold a `DNode` and need to delete the matching Map entry — without the key you would have to scan the whole map, and the $O(1)$ claim dies.

TRAP

`this.pos.get(key)` returns undefined for a missing key, so test `=== undefined`, not `if (!n)`. A perfectly valid cached node is an object and is always truthy — but if you ever store plain values instead of nodes, `0`, `"` and `null` are all falsy and a `if (!n)` test reports “missing” for a key that is present. `Map.prototype.has` is the explicit alternative.

Follow-up 1 — “make it thread safe.” One mutex around every operation is correct but serialises everything; sharding the cache into S independent LRUs by `hash(key) % S` keeps most operations parallel.

Follow-up 2 — “now it is LFU (least frequently used).” Keep one doubly linked list per frequency count, plus a pointer to the smallest non-empty frequency. Promoting a key moves its node from the list for count c to the list for count $c + 1$ — still $O(1)$.

Example 24 [Google · pattern]

Drop every node that is dominated on its right. Remove each node that has a strictly greater value somewhere after it. `12 15 10 11 5 6 2 3` becomes `15 11 6 3`. Constraints: $O(1)$ extra space is the follow-up; up to 10^5 nodes. Edge cases: strictly increasing input (only the last node survives); strictly decreasing (all survive); all equal (all survive).

SOLUTION

Approach 1 — For each node, scan the rest ($O(n^2)$)

Approach 2 — Push everything onto a stack ($O(n)$ time, $O(n)$ space)

This is the monotonic stack of Chapter 8, and in JavaScript the stack is just an array with `push` and `pop`. It works, but it copies the list into an array.

Approach 3 — Reverse, sweep once, reverse back ($O(n)$ time, $O(1)$ space — optimal)

Reversed, the rule becomes “keep a node only if it is at least as big as everything seen so far” — a single running maximum. Reverse the survivors back at the end.

keep the right-to-left maxima

JS

```

const keepRightMaxima = (head) => {
  head = reverseList(head);

```

```

let best = -Infinity; // the neutral value for a maximum
const dummy = new Node(0);
let tail = dummy;
for (let p = head; p; p = p.next)
    if (p.val >= best) { best = p.val; tail.next = p; tail = p; }
tail.next = null;
return reverseList(dummy.next);
};

```

Real output for 12 15 10 11 5 6 2 3; 1 2 3; 3 2 1; 5 5 5:

```

15 -> 11 -> 6 -> 3
3
3 -> 2 -> 1
5 -> 5 -> 5

```

Time $O(n)$ **Space** $O(1)$

The idea that unlocked it: “is anything to my right bigger?” is hard going forwards and trivial going backwards. On an array you would use a stack; on a list, reversing is free and removes the stack entirely.

TRAP

Use $\geq$, not $>$. With $>$ the input 5 5 5 keeps only one node, but no 5 is **strictly** greater than another 5, so all three must survive. Equal values are the test case that separates a correct answer from a nearly correct one.

-Infinity is the right starting value for a maximum in JavaScript — it is a real number and it loses every comparison. Starting at 0 is a bug the moment every value is negative. There is no `INT_MIN` here and you do not need one.

Follow-up: “now report, for each surviving node, how many nodes it swallowed.” Keep a counter that resets each time a new maximum is kept. Same pass, same complexity.

Example 25 [Goldman Sachs · pattern]

Prove and use the cycle arithmetic. You already found the meeting point of the fast and slow pointers. Answer these, as the interviewer will ask them one by one. (a) Why must they meet at all? (b) Why does walking from the head find the entrance? (c) How do you measure the loop length? (d) Can the slow pointer lap the fast one?

SOLUTION

(a) Why they must meet. Once both pointers are inside the loop, look at the distance from fast to slow measured **forwards around the loop**. Each move, slow advances 1 and fast advances 2, so that distance falls by exactly 1 every move. A non-negative whole number that falls by 1 each step must reach 0. At 0 they are on the same node. It can never jump over, because the gap changes by exactly 1, never 2.

(b) Why the head walk works. Let μ be the number of nodes before the loop and λ the loop length. At the meeting point slow has taken s steps and fast $2s$. Fast’s extra distance is a whole number of laps, so

$$2s - s = s = m\lambda$$

for some whole number m . Slow is $s - \mu$ nodes into the loop. Walking μ more steps puts it $s = m\lambda$ nodes into the loop, which is the entrance again. A pointer starting at the head also arrives at the entrance after exactly μ steps. Same node, same step.

(c) The loop length. Stay at the meeting point and walk forward, counting, until you return to it. That count is λ .

(d) Can slow lap fast? No. Slow enters the loop having taken μ steps; at that moment fast has taken 2μ steps, so fast is already inside. From then on the gap only shrinks, so they meet within λ more moves. Slow therefore walks at most $\mu + \lambda \leq n$ steps in total — the whole algorithm is $O(n)$.

The code for (a)–(c) is in Example 7. Real output on a 5-node list with 5 -> 2:

```

1 2 4

```

| — a loop exists, it starts at the node holding 2, and its length is 4.

| **Follow-up:** “now there are two lists and you must say whether they share a **loop**.” Detect each list’s loop entrance. If either has none, the answer is no. If both do, walk one entrance around its loop: if you meet the other entrance, they share the loop.

Example 26 [D. E. Shaw · pattern]

The full reversal family. Given one list, produce: (a) the whole list reversed; (b) reversed in groups of k ; (c) alternate groups of k reversed; (d) reversed between positions l and r only. All in $O(1)$ space. Constraints: $1 \leq l \leq r \leq n \leq 10^5$. Edge cases: $l = 1$ (the head moves); $l = r$ (no change); $k > n$.

SOLUTION

All four are the **same** head-insertion loop with a different anchor and a different stop rule. Once you see that, you write any of them in two minutes.

variant	anchor	stop rule
whole list	dummy	end of list
groups of k	previous group’s tail	k nodes, only if k exist
alternate groups	same, with a flag	reverse, then skip k
positions $l..r$	node at $l-1$	$r - l$ insertions

For (d), walk to the node just before position l , then do exactly $r - l$ head insertions. That is the group loop with $k = r - l + 1$ and no outer while.

(d) reverse positions $l..r$ only

JS

```
const reverseBetween = (head, l, r) => {
  if (!head || l >= r) return head;
  const dummy = new Node(0);
  dummy.next = head;
  let anchor = dummy;
  for (let i = 1; i < l; i++) { if (!anchor.next) return head; anchor =
anchor.next; }
  const prev = anchor.next;
  if (!prev) return head;
  let cur = prev.next;
  for (let i = 0; i < r - l && cur; i++) {
    prev.next = cur.next;
    cur.next = anchor.next;
    anchor.next = cur;
    cur = prev.next;
  }
  return dummy.next;
};
```

Real output for 1 2 3 4 5 with $l = 2, r = 4$; the whole list ($l = 1, r = 5$); a no-op ($l = r = 2$); an r past the end; and null:

```
1 -> 4 -> 3 -> 2 -> 5
5 -> 4 -> 3 -> 2 -> 1
1 -> 2 -> 3
3 -> 2 -> 1
(empty)
```

Real outputs for (a) are in Example 3, and for (b) and (c) in Example 20.

Follow-up the interviewer asks next: “and if it is a doubly linked list?” Then reversing is even shorter: walk the list and swap each node’s `prev` and `next` with the destructuring swap `[p.prev, p.next] = [p.next, p.prev]`, then return the old tail. One pass, $O(1)$ space, no three-pointer dance.

Dry run — reverse in groups of k , line by line

Input `1 -> 2 -> 3 -> 4 -> 5`, $k = 3$. The anchor `groupPrev` starts at the dummy node, written `D`.

Group 1. The check walks `D`, `1`, `2`, `3` — three nodes past the anchor exist, so proceed. `prev` = node 1 (it will end up as the group’s tail), `cur` = node 2.

i	cur	what happens	list after
1	2	2 jumps in front of 1	<code>D → 2 → 1 → 3 → 4 → 5</code>
2	3	3 jumps in front of 2	<code>D → 3 → 2 → 1 → 4 → 5</code>

The three statements inside the loop, for $i = 1$:

- `prev.next = cur.next`; → node 1 now points to node 3. List: `D → 1 → 3 → 4 → 5` with node 2 held by `cur`.
- `cur.next = groupPrev.next`; → node 2 points to node 1.
- `groupPrev.next = cur`; → the dummy points to node 2. List: `D → 2 → 1 → 3 → 4 → 5`.
- `cur = prev.next`; → `cur` becomes node 3, the next one to pull forward.

After the loop, `groupPrev = prev` = node 1, which is now the tail of the reversed group.

Group 2. The check walks from node 1: `1`, `4`, `5`, `null`. It runs out after two nodes, so check is `null` and the loop breaks. Nodes 4 and 5 stay in their original order.

Result: `3 -> 2 -> 1 -> 4 -> 5`. The verified run with 1..7 and $k = 3$ gives `3 -> 2 -> 1 -> 6 -> 5 -> 4 -> 7`, which matches this reasoning exactly.

group	groupPrev	k nodes left?	list after the group
1	<code>D</code>	yes (1,2,3)	<code>D → 3 → 2 → 1 → 4 → 5</code>
2	node 1	no (only 4,5)	unchanged, loop ends

TRAP

The check loop for `(let i = 0; i < k && check; i++) check = check.next`; starts at `groupPrev`, **not** at `groupPrev.next`. Starting one node later makes it accept a group of $k - 1$ nodes and the tail comes out reversed when it should not be.

Practice

PRACTICE · WARM-UP target: 20 min

- Count the nodes in a list, iteratively and recursively.
- Delete the middle node (the second middle when the count is even). Return the new head. What should a one-node list return?
- Return the n -th node from the end, or `null` if n is larger than the list.
- Are two lists identical in both length and values?
- Move the last node to the front: `1 2 3 4` becomes `4 1 2 3`.

PRACTICE · TIER 1 target: 45 min

- Remove **every** node whose value equals `val` (not just the first).

- Put all even values before all odd values, keeping the order inside each group. Careful with negative numbers.
- Rotate the list **left** by k places.
- A number is stored most-significant-digit first. Add 1 to it and return the new list.
- Split a list into two halves and return both heads. For an odd count the front half gets the extra node.

PRACTICE · TIER 2 target: 50 min

- Merge two sorted lists **recursively**, in $O(1)$ extra space beyond the recursion.
- Given a sorted list, build a balanced binary search tree from it in $O(n)$ time. (Hint: build the left subtree first, while walking the list forward once.)
- Remove every continuous run of nodes that sums to zero. 1 2 -3 3 1 becomes 3 1.

PRACTICE · TIER 3 target: 60 min

- Sort a list that is known to have at most k elements out of place, in $O(n \log k)$ time.
- You are given a list and a number k . Swap the k -th node from the start with the k -th node from the end — swapping the **nodes**, not the values. Handle the case where the two are the same node and the case where they are neighbours.
- Given two sorted lists, return a new list holding the values that appear in **both**, with no duplicates, in $O(n + m)$ time and no hash map.

ANSWER KEY

- Iterative is the `length` helper on page one. Recursive: `const count = (h) => h ? 1 + count(h.next) : 0;` — and warn out loud that 10^5 frames throw `RangeError: Maximum call stack size exceeded` in Node, so the iterative version is the one you submit.
- Fast and slow, with `prev` trailing `slow`.

P2

JS

```
const deleteMiddle = (head) => {
  if (!head || !head.next) return null;
  let slow = head, fast = head, prev = null;
  while (fast && fast.next) { prev = slow; slow = slow.next; fast =
fast.next.next; }
  prev.next = slow.next;
  return head;
};
```

Real output: 1 2 3 4 5 → 1 -> 2 -> 4 -> 5; 1 2 3 4 → 1 -> 2 -> 4; a one-node list and `null` both return `null`.

- Head start of n , then walk together.

P3

JS

```
const nthFromEnd = (head, n) => {
  let fast = head;
  for (let i = 0; i < n; i++) { if (!fast) return null; fast = fast.next; }
  let slow = head;
  while (fast) { fast = fast.next; slow = slow.next; }
  return slow;
};
```

Real output on 1 2 3 4 5: $n = 2 \rightarrow 4, n = 5 \rightarrow 1, n = 6 \rightarrow \text{null}$.

4. Walk together; both must end at null at the same time.

P4

JS

```
const identical = (a, b) => {
  while (a && b) { if (a.val !== b.val) return false; a = a.next; b = b.next; }
  return a === null && b === null;
};
```

Real output: 1 0 0 1 for the four tests (equal, different value, different length, both empty).

5. Walk to the last node while holding its predecessor.

P5

JS

```
const lastToFront = (head) => {
  if (!head || !head.next) return head;
  let prev = null, p = head;
  while (p.next) { prev = p; p = p.next; }
  prev.next = null; p.next = head;
  return p;
};
```

Real output: 1 2 3 4 → 4 -> 1 -> 2 -> 3; 9 → 9.

6. Dummy node, and do not advance on a delete.

P6

JS

```
const removeAll = (head, val) => {
  const dummy = new Node(0);
  dummy.next = head;
  for (let p = dummy; p.next; ) {
    if (p.next.val === val) p.next = p.next.next;
    else p = p.next;
  }
  return dummy.next;
};
```

Real output: 1 2 1 3 1 remove 1 → 2 -> 3; 5 5 5 remove 5 → empty; 1 2 remove 9 → unchanged.

7. Two chains, then glue — the same shape as partitionAround.

P7

JS

```
const evenThenOdd = (head) => {
  const ev = new Node(0), od = new Node(0);
  let e = ev, o = od;
  for (let p = head; p; p = p.next) {
    if (p.val % 2 === 0) { e.next = p; e = p; } else { o.next = p; o = p; }
  }
  o.next = null; e.next = od.next;
  return ev.next;
};
```

Real output: 1 2 3 4 5 6 → 2 -> 4 -> 6 -> 1 -> 3 -> 5; 1 3 5 → unchanged; -2 -3 0 → -2 -> 0 -> -3.

TRAP

In JavaScript, $-3 \% 2$ is -1 , not 1 — I checked it in Node. So $p.val \% 2 === 1$ is **false** for negative odd numbers and they are misfiled as even. Always test $\% 2 === 0$ for even, or $\% 2 !== 0$ for odd. ($\%$ on negatives behaves the same way in C++ and Java, and **differently** in Python, where $-3 \% 2$ is 1 .)

8. Same ring trick as `rotateRight`, but the new tail is k nodes in from the front.

P8

JS

```
const rotateLeft = (head, k) => {
  if (!head || !head.next || k <= 0) return head;
  let n = 1, tail = head;
  while (tail.next) { tail = tail.next; n++; }
  k %= n; if (k === 0) return head;
  let newTail = head;
  for (let i = 1; i < k; i++) newTail = newTail.next;
  const newHead = newTail.next;
  newTail.next = null;
  tail.next = head;
  return newHead;
};
```

Real output: 1 2 3 4 5 with $k = 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1 \rightarrow 2$; 1 2 3 with $k = 3 \rightarrow$ unchanged; with $k = 7 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

9. Reverse, add the carry, reverse back.

P9

JS

```
const plusOne = (head) => {
  head = reverseList(head);
  let carry = 1, p = head, last = null;
  while (p && carry) {
    const s = p.val + carry;
    p.val = s % 10; carry = Math.floor(s / 10);
    last = p; p = p.next;
  }
  if (carry) last.next = new Node(carry);
  return reverseList(head);
};
```

Real output: 1 2 3 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 4; 9 9 9 $\rightarrow$ 1 $\rightarrow$ 0 $\rightarrow$ 0 $\rightarrow$ 0; 0 $\rightarrow$ 1.

10. Fast and slow with `fast = head.next` gives the first middle; cut after it. `slow.next` is the second head, then set `slow.next = null`. Return both heads as an array and unpack them with `const [a, b] = splitHalves(head);`.

11. Recursive merge — shorter to write, but the stack is the price.

P11

JS

```
const mergeRec = (x, y) => {
  if (!x) return y;
  if (!y) return x;
  if (x.val <= y.val) { x.next = mergeRec(x.next, y); return x; }
  y.next = mergeRec(x, y.next); return y;
};
```

Real output: 1 3 5 with 2 4 → 1 → 2 → 3 → 4 → 5; null with 7 → 7; both empty → empty. Time $O(n + m)$, stack $O(n + m)$ — and in JavaScript that stack is the whole objection: $n + m$ above roughly 10^4 throws **RangeError: Maximum call stack size exceeded**. Give the iterative version first.

12. Recurse on the **count**, not on the nodes. Build the left subtree from the first $n/2$ items, then take the current list node as the root and advance the list pointer, then build the right subtree. One forward walk, $O(n)$ total.

13. Prefix sums with a **Map** from running sum to node. If the same running sum appears twice, everything between them sums to zero, so link the first node straight to the node after the second. Use a dummy head so a run starting at the head is handled too. $O(n)$ time and space.

14. The toolkit **MinHeap** (JavaScript has none built in), holding $k + 1$ nodes: push the first $k + 1$, then repeatedly pop the smallest, append it, and push the next node. An element that is at most k places out of place is guaranteed to be inside the heap when its turn comes. $O(n \log k)$.

15. Walk to the two nodes **and** their predecessors, handle $k == n - k + 1$ (same node, do nothing) and the neighbour case (their predecessors are each other) separately, then swap the four pointers. Swapping values is a one-liner but is usually rejected — the question says swap the nodes.

16. Walk both lists like a merge. When the values are equal, append a **new** node with that value and skip **both**; to avoid duplicates in the output, also skip while the next value equals the one just appended. When they differ, advance the smaller one. $O(n + m)$, no **Map** and no **Set**.

ONE-PAGE REVISION CARD

The node and the three reflexes.

memorise these

JS

```
class Node { constructor(val) { this.val = val; this.next = null; } }

// dummy head: makes "delete/insert at the head" the same code as anywhere else
const dummy = new Node(0); dummy.next = head; /* ... */ return dummy.next;

// reverse
let prev = null, cur = head;
while (cur) { const nxt = cur.next; cur.next = prev; prev = cur; cur = nxt; }
// prev is the new head

// fast and slow
let slow = head, fast = head; // 2nd middle
while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
// use fast = head.next for the 1st middle (splitting, reorder, merge sort)
```

Which move solves which problem.

problem	move	space
reverse, reverse in groups, reverse l..r	three pointers / head insertion	$O(1)$
middle, delete middle, split in half	fast + slow	$O(1)$
loop detect / start / length / remove	fast + slow (Floyd)	$O(1)$
n-th from the end, remove n-th from the end	head start of n	$O(1)$
palindrome	middle + reverse half + restore	$O(1)$
merge two sorted, merge K, flatten	dummy + tail, toolkit MinHeap for K	$O(1) / O(k)$
sort a list	merge sort, 1st middle	$O(\log n)$
partition, odd/even split	two chains + glue	$O(1)$
intersection of two lists	length difference, or pointer swap	$O(1)$
copy with random pointers	weave, wire, unweave	$O(1)$
LRU cache	Map + doubly linked list	$O(\text{cap})$
drop dominated nodes	reverse, running max, reverse	$O(1)$

Top traps.

1. Forgetting the dummy node when the head can change.
2. `while (fast && fast.next)` — that order, always. The other order throws `TypeError: Cannot read properties of null`.
3. `fast = head` gives the 2nd middle; `fast = head.next` gives the 1st. Merge sort and reorder need the **1st**, or they never terminate.
4. Advancing the pointer on the same step you delete a node.
5. Forgetting `tail.next = null` after splitting or partitioning — silent infinite loop.
6. `while (a || b || carry)` in list addition. Losing `|| carry` loses the leading digit.
7. `carry = s / 10` gives 1.9. JavaScript has no integer division: use `Math.floor(s / 10)`.
8. In `reverseRec`, set `head.next.next = head`; **before** `head.next = null`;

9. $-3 \% 2 === -1$ in JavaScript, so test $\% 2 === 0$ for even, never $\% 2 === 1$ for odd.
10. `removeCycle` must null the node **pointing at** the entrance, not the entrance itself.
11. **Recursion depth is about 10^4 in Node** — I measured 12546. A 10^5 -node list plus recursion equals `RangeError: Maximum call stack size exceeded`. Offer the iterative version first, every time.
12. **No built-in heap.** Merging K lists needs the toolkit `MinHeap`, and $(a, b) \Rightarrow a.val - b.val$ is the min-heap comparator — flip the sign and you get a max-heap by accident.
13. **`arr.sort()` is lexicographic.** Any “copy to an array and sort” approach needs $(a, b) \Rightarrow a - b$.
14. Comparing nodes with `===` (identity), never by value.
15. Deleting a node given only a reference to it fails on the **last** node — say so out loud.

Complexity you should be able to quote instantly. Reverse, middle, cycle, n-th from end, merge two: $O(n)$ time, $O(1)$ space. Merge sort a list: $O(n \log n)$ time, $O(\log n)$ stack. Merge K lists: $O(N \log k)$. Copy with random references: $O(n)$ time, $O(1)$ extra. LRU: $O(1)$ per operation.

From the JS Toolkit appendix, this chapter uses: `MinHeap`, for merging K sorted lists. Everything else here is plain objects and references.

the one import this chapter needs

JS

```
const { MinHeap } = require('./toolkit.js');
```

Recursion & Backtracking

Choose, explore, un-choose — and know when to stop early.

Pattern in one page

A recursive function solves a problem by calling **itself on a smaller version** of the same problem. Every recursive function has exactly three parts.

THE THREE PARTS OF ANY RECURSION

1. **Base case** — the smallest input, answered with no further call. Without it the program runs until the stack dies.
2. **Smaller call** — call yourself on an input that is strictly closer to the base case.
3. **Combine** — turn the answer of the smaller call into the answer for this input.

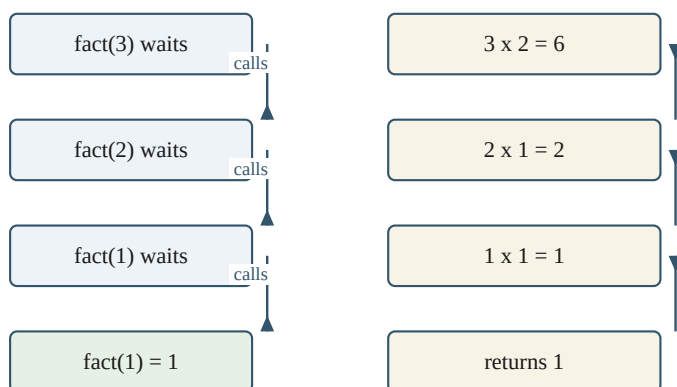
The shape of every recursion

JS

```
function f(state) {
  if (isBase(state)) return baseAnswer; // 1. base case
  const small = f(shrink(state));       // 2. smaller call
  return combine(state, small);         // 3. combine
}
```

The call stack — draw it once and you will never fear recursion again

When $f(3)$ calls $f(2)$, the engine **pauses** $f(3)$ and remembers where it stopped. That paused copy sits on the **call stack**. Every recursive call pushes a new frame. Every return pops one.



fact(3) grows the stack down, then unwinds up

Backtracking = recursion that undoes its own moves

Backtracking is recursion over **choices**. At each step you pick one option, walk deeper, and when you come back you **put the world back the way you found it**.

THE THREE LINES YOU WILL WRITE A HUNDRED TIMES


```

choose    -> cur.push(option);    used[option] = true;
explore   -> go(nextLevel);
un-choose -> cur.pop();            used[option] = false;

```

Invariant: when `go(level)` returns, every shared structure (`cur`, `used`, the grid) is exactly the same as when `go(level)` was entered. If that is true at every level, your answer list is correct.

The universal backtracking template

JS

```

function solve(input) {
  const out = [], cur = [];
  const go = (state) => {
    if (isComplete(state)) { out.push([...cur]); return; } // COPY cur, never cur
    itself
    for (const option of optionsAt(state)) {
      if (!isLegal(state, option)) continue; // prune: never enter a dead branch
      apply(cur, option); // choose
      go(next(state, option)); // explore
      undo(cur, option); // un-choose <- never forget this
    }
  };
  go(startState);
  return out;
}

```

TRAP

Two bugs, both fatal, both invisible at first.

1. **Forgetting undo.** The program still runs; it just prints garbage, because level 3 sees leftovers from level 5. If your output has too many elements, or elements that are too long, look for a missing `cur.pop()`.
2. **`out.push(cur)` instead of `out.push([...cur])`.** In JavaScript an array is a **reference**. `out.push(cur)` stores a pointer to the one live array, and every `cur.pop()` afterwards empties it. Your answer list ends up full of identical empty arrays. This is the single most common JS backtracking bug. Always copy: `[...cur]`, or `cur.slice()`.

Which template do I reach for?

You are asked for	Shape	Cost
one number (count, max, yes/no)	return a value, add a memo if states repeat	often $O(n^2)$ with a memo
all subsets / subsequences	pick or skip at each index	$O(2^n \times n)$
all orderings	swap in place, or a <code>used[]</code> array	$O(n! \times n)$
all combinations summing to a target	loop <code>j</code> from <code>i</code> , recurse with <code>j</code> or <code>j+1</code>	depends on the target
place items on a board	one row per level + Sets for the blocked lines	$O(n!)$ with pruning
fill a grid	find the first blank, try every value	exponential, prune hard

Rough budget for a 1-second coding round in Node: 2^n is fine up to $n \approx 20$. $n!$ is fine up to $n \approx 9$. JavaScript is roughly 2 to 5 times slower than C++ on this kind of work, so shave one or two off the exponent compared to what you may have read elsewhere. If n is 40 and the answer is a single number, recursion alone will not pass — you need memoisation (Chapter 17) or a different idea entirely.

SHORTCUT

Count the leaves, not the nodes. A “pick or skip” tree on n items has 2^n leaves. If you copy an array of length up to n at each leaf — and in JS you must, see the trap above — the true cost is $O(2^n \times n)$, not $O(2^n)$. Interviewers listen for that extra n .

THE FOUR JAVASCRIPT FACTS THIS CHAPTER DEPENDS ON

1. **arr.sort() sorts as text.** `[10, 9, 1].sort()` gives `[1, 10, 9]`. Every sort in this chapter must be `arr.sort((a, b) => a - b)`.
2. **Numbers are doubles.** They are exact only up to `Number.MAX_SAFE_INTEGER = 9007199254740991` ($2^{53} - 1$). Anything bigger needs `BigInt`.
3. **The recursion depth limit is roughly 10^4 , not 10^6 .** A deep recursion throws `RangeError: Maximum call stack size exceeded`.
4. **Spread is a shallow copy.** `[...grid]` copies the outer array only; the rows are still shared. For a 2-D grid you need `grid.map(row => [...row])`.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Write `fact(n) = 1 × 2 × ... × n`, with `fact(0) = 1`. Constraints: $0 \leq n \leq 30$. Target: $O(n)$ time. Edge cases: $n = 0$; $n = 23$ (where an ordinary JS number stops being exact).

SOLUTION

Base case is $n \leq 1$, because both $0!$ and $1!$ equal 1. Then $n! = n \times (n - 1)!$.

factorial with ordinary numbers

JS

```
function factNum(n) {
  if (n <= 1) return 1;           // base case
  return n * factNum(n - 1);
}
```

factorial with BigInt

JS

```
function fact(n) {
  if (n <= 1n) return 1n;        // note the n suffix: these are BigInts
  return n * fact(n - 1n);
}
```

Ran both and compared every result against the exact `BigInt` value:

n	factNum(n) prints	exact?
20	2432902008176640000	yes
21	51090942171709440000	yes
22	1.1240007277776077e+21	yes
23	2.585201673888498e+22	no

25	1.5511210043330986e+25	no
----	------------------------	----

`fact(23n)` gives the true value, 25852016738884976640000. The ordinary number is 25852016738884980000 — close, and wrong.

Time $O(n)$ **Space** $O(n)$
 space is the call stack: n paused frames

Answer: `fact(20n) = 2432902008176640000n`

TRAP

Numbers in JavaScript are doubles. `Number.MAX_SAFE_INTEGER` is 9007199254740991. Above that, the gaps between representable values grow, and arithmetic silently rounds. I ran this:

```
Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2 // true
```

Two different sums compare equal. Notice that $20!$ and $21!$ are **far** above the safe limit and still print exactly — factorials contain many factors of 2, so they happen to land on representable values. $23!$ is the first that does not. **Do not rely on luck.** The rule: if a count, a product or a sum can pass 9×10^{15} , use `BigInt`.

BigInt rules you need. Write literals with an n suffix ($5n$). You may not mix `BigInt` and `Number` in one expression — `5n + 1` throws a `TypeError`; write `5n + 1n`. Division truncates, so `7n / 2n` is `3n`. Convert with `BigInt(x)` and `Number(x)`, and print with `String(x)` or `x.toString()`.

Example 2 [warm-up]

Sum the digits of a non-negative number. 4073 gives $4 + 0 + 7 + 3 = 14$. Constraints: $0 \leq n \leq 10^{15}$. Target: $O(\log n)$. Edge cases: $n = 0$; a single digit.

SOLUTION

Peel one digit with `n % 10`, shrink with `Math.floor(n / 10)`.

digit sum		JS
<pre>function digitSum(n) { if (n === 0) return 0; // base case return (n % 10) + digitSum(Math.floor(n / 10)); }</pre>		

Trace of `digitSum(4073)`:

call	n	n % 10	next call
1	4073	3	digitSum(407)
2	407	7	digitSum(40)
3	40	0	digitSum(4)
4	4	4	digitSum(0)
5	0	—	returns 0

Unwinding: $0 \rightarrow 4 \rightarrow 4 \rightarrow 11 \rightarrow 14$.

Ran it: `digitSum(0) = 0`, `digitSum(9) = 9`, `digitSum(4073) = 14`.

Time $O(\log n)$ **Space** $O(\log n)$
 one frame per digit

Answer: 14

TRAP

$n / 10$ in JavaScript is **not** integer division — $407 / 10$ is 40.7 . You must write `Math.floor(n / 10)`. Forget it and the recursion never reaches 0, so it runs until the stack overflows. This bites programmers coming from C++, Java or Python's `//`.

Example 3 [Infosys pattern]

Compute a^b using as few multiplications as possible. Constraints: $1 \leq a \leq 10$, $0 \leq b \leq 100$. The answer may be enormous, so use `BigInt`. Target: $O(\log b)$. Edge cases: $b = 0$; b odd; a result well past 2^{53} .

SOLUTION

Approach 1 — Multiply b times ($O(b)$ — fine for small b , but we can do better)

Approach 2 — Halve the exponent ($O(\log b)$ — optimal)

The key fact: $a^b = (a^{b/2})^2$ when b is even, and $a^b = (a^{b/2})^2 \times a$ when b is odd. So one call solves **half** the exponent.

fast power with `BigInt`

JS

```
function power(a, b) {
  if (b === 0n) return 1n;
  const half = power(a, b / 2n);    // ONE call; BigInt division truncates
  const sq = half * half;
  return (b % 2n === 1n) ? sq * a : sq;
}
```

Ran it:

<code>power(2n, 0n)</code>	1
<code>power(2n, 10n)</code>	1024
<code>power(3n, 13n)</code>	1594323
<code>power(2n, 40n)</code>	1099511627776
<code>power(2n, 80n)</code>	1208925819614629174706176

Time $O(\log b)$ **Space** $O(\log b)$

The idea that unlocked it: store the half in a variable. If you write `return power(a, b/2n) * power(a, b/2n);` you compute the same thing twice and the cost jumps straight back to $O(b)$.

Answer: `power(2n, 80n) = 1208925819614629174706176n`

Example 4 [warm-up]

Reverse a string using recursion only — no loop, no `reverse()`. Constraints: length up to 5000 (see the trap about depth). Edge cases: empty string; one character.

SOLUTION

Strings in JavaScript are **immutable**, so you cannot swap characters in place. Turn the string into an array of characters first, swap there, then join it back.

reverse by swapping the two ends

JS

```
function rev(a, i, j) {
  if (i >= j) return; // 0 or 1 characters left
  [a[i], a[j]] = [a[j], a[i]]; // destructuring swap
  rev(a, i + 1, j - 1);
}

function reverseString(str) {
  const a = [...str]; // "abc" -> ["a","b","c"]
  rev(a, 0, a.length - 1);
  return a.join('');
}
```

Ran it: `reverseString("")` gives `""` (here $i = 0, j = -1$, so $i \geq j$ fires at once), `reverseString("a")` gives `"a"`, and `reverseString("recursion")` gives `"noisrucer"`.

Time $O(n)$ **Space** $O(n)$
 $n/2$ stack frames plus the character array

Answer: noisrucer

TRAP

Recursion depth in JavaScript is small. I measured it on Node: a plain recursive function threw `RangeError: Maximum call stack size exceeded` at about **12,500** levels, and it varies with how many local variables each frame holds. That is roughly 10^4 — one hundred times less than C++.

So a recursion of depth 10^5 **will** crash. If the depth can reach 10^4 , rewrite it as a loop or an explicit stack. Example 36 shows exactly how.

Example 5 [TCS NQT pattern]

A staircase has n steps. You climb 1 or 2 steps at a time. How many different climbs reach the top? Constraints: $1 \leq n \leq 70$. Target: $O(n)$. Edge cases: $n = 1$ (answer 1); $n = 50$; $n = 70$ (check the safe-integer limit).

SOLUTION

Approach 1 — Plain recursion ($O(2^n)$, dies around $n = 40$)

From step n you arrived either from $n - 1$ or from $n - 2$. So $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$. Base: $\text{ways}(0) = 1$ (one way to already be there) and $\text{ways}(\text{negative}) = 0$.

Approach 3 — Remember answers you already computed ($O(n)$ — optimal)

$\text{ways}(n-1)$ and $\text{ways}(n-2)$ overlap enormously. Store each answer the first time.

stairs with a memo

JS

```
function climb(n) {
  const memo = new Array(n + 1).fill(-1);
  const ways = (k) => {
    if (k === 0) return 1;
    if (k < 0) return 0;
    if (memo[k] !== -1) return memo[k]; // already solved
    return memo[k] = ways(k - 1) + ways(k - 2);
  };
  return ways(n);
}
```

Ran it: `climb(1) = 1, climb(2) = 2, climb(5) = 8, climb(50) = 20365011074`. I also checked `climb(50) <= Number.MAX_SAFE_INTEGER` — true, so ordinary numbers are still exact here.

Time $O(n)$ **Space** $O(n)$

The idea: the recursion tree has only n **distinct** nodes. Caching turns a tree into a line. That one trick is the whole of Chapter 17.

Answer: `climb(50) = 20365011074`

TRAP

The answer for $n = 70$ is the 71st Fibonacci number, about 3×10^{14} — still under 9×10^{15} , so it is safe. But $n = 80$ is about 2.3×10^{16} , which is **past** the safe limit and silently wrong. When a count grows exponentially, work out the largest value the constraints allow **before** you choose `Number` over `BigInt`.

Example 6 [warm-up]

Greatest common divisor of two numbers, recursively. Constraints: $0 \leq a, b \leq 10^{15}$. Target: $O(\log \min(a, b))$. Edge cases: one of them is 0; they are coprime.

SOLUTION

Euclid's fact: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, and $\text{gcd}(a, 0) = a$.

gcd

JS

```
function gcd(a, b) {  
  if (b === 0) return a;  
  return gcd(b, a % b);  
}
```

Ran it: `gcd(0, 7) = 7, gcd(12, 18) = 6, gcd(17, 5) = 1`.

Trace of `gcd(12, 18)`: $(12, 18) \rightarrow (18, 12) \rightarrow (12, 6) \rightarrow (6, 0) \rightarrow 6$. The first call flips the pair for free, so you never need to sort the inputs.

Time $O(\log \min(a, b))$ **Space** $O(\log \min(a, b))$

Answer: 6

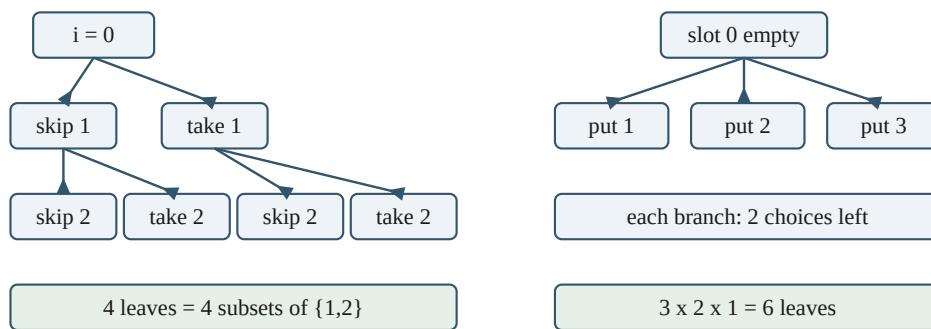
`%` in JavaScript is the **remainder**, not a true modulo: `-7 % 3` is `-1`, not `2`. For `gcd` on non-negative inputs it makes no difference, but the moment negatives can appear you want `((a % m) + m) % m`. Chapter 20 leans on this heavily.

Tier 1 — the two trees everything is built on

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Almost every backtracking problem in a placement round is one of two trees:



left: the pick-or-skip tree (subsets). right: the slot tree (permutations)

Example 7 [TCS NQT pattern]

Return every subset of an array of n distinct integers. Constraints: $1 \leq n \leq 18$, values fit in an ordinary number. Target: $O(2^n \times n)$. Edge cases: empty array (the answer is one empty subset); $n = 1$.

SOLUTION

Approach 1 — Loop over bitmasks $(O(2^n \cdot n))$, works, and is the shortest code)

Each subset is a yes/no answer for each of the n items — exactly one binary number of n bits. Bit i on means “item i is in”.

subsets by bitmask

JS

```
function subsetsBit(a) {
  const n = a.length, out = [];
  for (let mask = 0; mask < (1 << n); mask++) {
    const cur = [];
    for (let i = 0; i < n; i++) if (mask & (1 << i)) cur.push(a[i]);
    out.push(cur);
  }
  return out;
}
```

Ran it on $[1, 2, 3]$: $[]$ $[1]$ $[2]$ $[1, 2]$ $[3]$ $[1, 3]$ $[2, 3]$ $[1, 2, 3]$

Time $O(2^n \times n)$ **Space** $O(1)$
output not counted

Approach 2 — Pick or skip, recursively (same cost, but this one generalises)

The bitmask trick dies the moment you add a rule (“no two adjacent”, “the sum must be 10”). Recursion survives, because you can put an `if` in front of the “pick” branch.

subsets by recursion

JS

```
function subsets(a) {
  const out = [], cur = [];
  const go = (i) => {
    if (i === a.length) { out.push([...cur]); return; } // COPY, not the live
    array
    go(i + 1); // not pick
    cur.push(a[i]); // pick
    go(i + 1);
    cur.pop(); // undo
  };
}
```

```

    go(0);
    return out;
}

```

Ran it on $[1, 2, 3]$: $[]$ $[3]$ $[2]$ $[2, 3]$ $[1]$ $[1, 3]$ $[1, 2]$ $[1, 2, 3]$ and on the empty array: $[]$ — exactly one subset, the empty one.

Time $O(2^n \times n)$ **Space** $O(n)$
 stack depth n , plus cur

The idea: the order differs from the bitmask version because the “skip” branch runs first. Both lists contain the same 8 subsets. If the question says “any order”, both pass.

Answer: 8 subsets for $n = 3$; in general 2^n

TRAP

Write `out.push(...cur)`, never `out.push(cur)`. I cannot say this too often, because the broken version **runs without any error**. Arrays are references. Store the reference, and every later `cur.pop()` reaches back into the answer you already saved. You end up with 2^n copies of the same empty array.

`[...cur]`, `cur.slice()` and `Array.from(cur)` all work. Pick one and use it every time.

Example 8 [Capgemini pattern]

Same as Example 7, but the array may contain repeats. Return every **distinct** subset. Constraints: $1 \leq n \leq 18$, values $1..100$. Target: $O(2^n \times n)$. Edge cases: all elements equal; a mix of one-digit and two-digit values (see the trap).

SOLUTION

Approach 1 — Generate all 2^n , then de-duplicate with a Set of joined strings ($O(2^n \cdot n)$ but with a big constant, and it needs $O(2^n \cdot n)$ extra memory)

Approach 2 — Sort, then skip a repeat at the same level ($O(2^n \cdot n)$ — optimal)

Sort first so equal values sit together. Now use the **loop** form of the tree: at level i you choose which element starts the next block. Inside that loop, if `a[j]` equals `a[j - 1]` and j is not the first index of this loop, that exact choice was already made one iteration ago — skip it.

subsets with duplicates

JS

```

function subsetsDup(input) {
    const a = [...input].sort((x, y) => x - y);    // NUMERIC sort, see the trap
    const out = [], cur = [];
    const go = (i) => {
        out.push(...cur);                          // every node is an answer
        for (let j = i; j < a.length; j++) {
            if (j > i && a[j] === a[j - 1]) continue; // skip a repeat at this level
            cur.push(a[j]);
            go(j + 1);
            cur.pop();
        }
    };
    go(0);
    return out;
}

```


Ran it on $[2, 1, 2]$ (sorted to $1, 2, 2$): $[] [1] [1, 2] [1, 2, 2] [2] [2, 2]$ — six distinct subsets, no duplicates, no Set needed.

Time $O(2^n \times n)$ **Space** $O(n)$

The idea: $j > i$ is the whole trick. It means “not the first branch of **this** loop”. Writing $j > 0$ instead would wrongly kill $[2, 2]$.

TRAP

arr.sort() with no comparator sorts as text. I ran it:

```
[10, 9, 1].sort()           // [1, 10, 9]    <- text order
[10, 9, 1].sort((a, b) => a - b) // [1, 9, 10] <- what you meant
```

This is the most common JavaScript bug in coding rounds, and it is silent — no error, just a wrong answer. Every sort in this chapter needs the comparator. Example 17 shows an input where the missing comparator changes a correct answer into an empty list.

Example 9 [Wipro pattern]

Return all $n!$ orderings of n distinct integers. Constraints: $1 \leq n \leq 8$. Target: $O(n! \times n)$. Edge cases: $n = 1$; negative values.

SOLUTION

Approach 1 — used[] array plus a growing list ($O(n! \cdot n)$ time, $O(n)$ extra space)

Approach 2 — Swap in place ($O(n! \cdot n)$, with no used[] array at all)

Think of it as filling slot i . Whatever sits at position $j \geq i$ may be swapped into slot i . Recurse for slot $i+1$. Then swap back.

permutations by swapping

JS

```
function permute(input) {
  const a = [...input], out = [];
  const go = (i) => {
    if (i === a.length) { out.push([...a]); return; }
    for (let j = i; j < a.length; j++) {
      [a[i], a[j]] = [a[j], a[i]];
      go(i + 1);
      [a[i], a[j]] = [a[j], a[i]];      // undo
    }
  };
  go(0);
  return out;
}
```

Ran it on $[1, 2, 3]$: $[1, 2, 3] [1, 3, 2] [2, 1, 3] [2, 3, 1] [3, 2, 1] [3, 1, 2]$ and on $[7]$: $[[7]]$.

Time $O(n! \times n)$ **Space** $O(n)$
n frames; the array itself is reused

Answer: 6 permutations of 3 items

TRAP

Two things at once here.

Order. The swap version does not produce permutations in sorted order — look at the output: [3, 2, 1] comes before [3, 1, 2]. If the question says “in lexicographic order”, use the `used[]` version with the input sorted numerically.

Copies. `out.push([...a])` again — `a` is the **one** working array and it is swapped back on the way out. Storing `a` itself gives you $n!$ references to the original order.

Example 10 [Cognizant pattern]

All **distinct** orderings when the array has repeats. Constraints: $1 \leq n \leq 8$. Target: better than $O(n! \times n)$ in practice. Edge cases: all elements equal; negatives.

SOLUTION

Approach 1 — Permute everything and de-duplicate with a Set of joined strings ($O(n! \cdot n)$ work even when the answer is tiny)

Approach 2 — Sort, then forbid a repeat starting a new branch (one node per real answer — optimal)

Swapping is now a trap: it moves equal values around and the “same value” test stops working. Use `used[]`. The rule: if `a[i] === a[i-1]` and `a[i-1]` is **not** currently used, then the twin to my left is free, which means this branch was already built starting from that twin. Skip.

permutations with duplicates

JS

```
function permuteDup(input) {
  const a = [...input].sort((x, y) => x - y);
  const out = [], cur = [], used = new Array(a.length).fill(false);
  const go = () => {
    if (cur.length === a.length) { out.push([...cur]); return; }
    for (let i = 0; i < a.length; i++) {
      if (used[i]) continue;
      if (i > 0 && a[i] === a[i - 1] && !used[i - 1]) continue; // the twin rule
      used[i] = true; cur.push(a[i]);
      go();
      cur.pop(); used[i] = false;
    }
  };
  go();
  return out;
}
```

Ran it on [1, 1, 2]: [1, 1, 2] [1, 2, 1] [2, 1, 1] — three, not six. On [-1, -1]: [-1, -1] — one.

Time $O(n! \times n)$ **Space** $O(n)$
worst case; far less when there are many repeats

The idea: force the twins to be used **left to right**. Only one of the $k!$ orderings of k equal values survives, so each distinct permutation is produced exactly once.

Answer: 3

Example 11 [Accenture pattern]

Return all strings of n pairs of brackets that are balanced. For $n = 2$: `(( ))` and `()()`. Constraints: $1 \leq n \leq 11$. Target: one recursion node per partial answer. Edge cases: $n = 0$ (one answer, the empty string); $n = 1$.

SOLUTION

Approach 1 — Build all $2^{(2n)}$ strings, keep the balanced ones ($O(4^n \cdot n)$ — one million wasted strings at $n = 10$)

Approach 2 — Only ever append a legal bracket ($O(4^n / \sqrt{n})$) — one node per real answer path

Carry two counters: `open` = how many (are still unused, `close` = how many) are still unused. Two rules cover everything:

- You may write (whenever `open > 0`.
- You may write) only when `close > open`, that is, some (is still waiting to be closed.

balanced brackets

JS

```
function genBrackets(n) {
  const out = [], cur = [];
  const go = (open, close) => {
    if (open === 0 && close === 0) { out.push(cur.join('')); return; }
    if (open > 0) { cur.push('('); go(open - 1, close); cur.pop(); }
    if (close > open) { cur.push(')'); go(open, close - 1); cur.pop(); }
  };
  go(n, n);
  return out;
}
```

Ran it:

- `n = 0` gives one answer, the empty string "".
- `n = 1` gives "()".
- `n = 3` gives "((()))", "(()())", "(())()", "()(())", "()()()" — 5 answers.

Time $O(4^n / \sqrt{n})$ **Space** $O(n)$

the count of answers is the n -th Catalan number: 1, 1, 2, 5, 14, 42, ...

The idea: pruning at the point of choice. Approach 1 builds 64 strings for $n = 3$ and throws away 59. Approach 2 builds 5 and throws away none.

Answer: 5 strings for $n = 3$

SHORTCUT

Build the string in an **array** and `join('')` at the end, as above. Strings in JavaScript are immutable, so `cur = cur + '('` allocates a brand-new string every time, and undoing means `cur = cur.slice(0, -1)` — another allocation. An array with `push/pop` is the same shape as every other backtracking problem and it is faster.

Example 12 [Infosys pattern]

On an old phone keypad, 2 maps to abc, 3 to def, ..., 7 to pqrs, 8 to tuv, 9 to wxyz. Given a digit string (digits 2 to 9), return every letter string it could spell. Constraints: length up to 8. Target: $O(4^L \times L)$. Edge cases: empty input (return nothing); a single digit; digits 7 and 9 have 4 letters.

SOLUTION

One level of recursion per digit. The options at that level are that digit's letters.

keypad words

JS

```
const PAD = ['', '', 'abc', 'def', 'ghi', 'jkl', 'mno', 'pqrs', 'tuv', 'wxyz'];

function keypadWords(d) {
  const out = [];
  if (d.length === 0) return out; // guard: no digits, no words
  const cur = [];
  const go = (i) => {
```

```

    if (i === d.length) { out.push(cur.join('')); return; }
    for (const ch of PAD[Number(d[i])]) { // a string is iterable, one char at a
time
        cur.push(ch);
        go(i + 1);
        cur.pop();
    }
};
go(0);
return out;
}

```

Ran it on "23": ad ae af bd be bf cd ce cf — 9 strings (3×3). On "": nothing at all.

Time $O(4^L \times L)$ **Space** $O(L)$

L = number of digits; 4 is the biggest keypad group

Answer: 9

TRAP

The empty-input guard matters. Without it, `go(0)` hits the base case immediately and pushes one empty string. Most graders want an **empty list** for empty input, not a list holding `""`. Read the statement and match it.

Also note `PAD[Number(d[i])]`. `d[i]` is the one-character **string** "2", and using a string to index an array works here only because JS coerces it — but it is the kind of implicit coercion that bites later. Convert explicitly.

Example 13 [TCS Digital pattern]

Return all binary strings of length n that never have two 1s next to each other. Constraints: $1 \leq n \leq 20$. Target: one node per partial answer. Edge cases: $n = 1$ (both 0 and 1 are fine); $n = 2$.

SOLUTION

Approach 1 — Build all 2^n strings and test each ($O(2^n \cdot n)$ — $n = 20$ means a million strings, most rejected)

Approach 2 — Refuse to write an illegal character ($O(\text{answers} \cdot n)$ — optimal)

Only one rule: you may write 1 only if the previous character is 0 (or there is no previous character).

binary strings, no two adjacent ones

JS

```

function noAdjacentOnes(n) {
    const out = [], cur = [];
    const go = () => {
        if (cur.length === n) { out.push(cur.join('')); return; }
        cur.push('0'); go(); cur.pop();
        if (cur.length === 0 || cur[cur.length - 1] !== '0') {
            cur.push('1'); go(); cur.pop();
        }
    };
    go();
    return out;
}

```

Ran it:

- $n = 1$: 0 1
- $n = 4$: 0000 0001 0010 0100 0101 1000 1001 1010 — 8 strings.

The counts are 2, 3, 5, 8, 13, ... — Fibonacci. Not a coincidence: a legal string of length n either ends in 0 (any legal string of length $n - 1$ in front) or ends in 01 (any legal string of length $n - 2$ in front).

Time $O(F_{n+2} \times n)$ **Space** $O(n)$
 F is Fibonacci; far smaller than 2^n

Answer: 8 strings for $n = 4$

Example 14 [Accenture pattern]

Given distinct positive numbers and a target, return every list of them that sums to the target. A number may be used **any** number of times. Constraints: $1 \leq n \leq 12$, $1 \leq \text{target} \leq 40$, all values ≥ 2 . Target: depth at most target divided by the smallest value. Edge cases: target 0 (one answer, the empty list); no list works.

SOLUTION

Two branches at index i : **use $a[i]$ again** (stay at i), or **retire $a[i]$ forever** (move to $i+1$). Staying at i is what allows reuse. Moving forward is what stops $[2, 3]$ and $[3, 2]$ both appearing.

combination sum, unlimited reuse

JS

```
function combSum(input, target) {
  const a = [...input].sort((x, y) => x - y);
  const out = [], cur = [];
  const go = (i, left) => {
    if (left === 0) { out.push([...cur]); return; }
    if (i === a.length || left < 0) return;
    if (a[i] <= left) {
      cur.push(a[i]);
      go(i, left - a[i]);
      cur.pop();
    }
    go(i + 1, left);
  };
  go(0, target);
  return out;
}
```

Ran it on $[2, 3, 5]$ with target 8: $[2, 2, 2, 2]$ $[2, 3, 3]$ $[3, 5]$. With target 0: $[[[]]]$. On $[5, 7]$ with target 3: $[]$.

Time $O(n^{\frac{\text{target}}{\min}})$ **Space** $O(\frac{\text{target}}{\min})$
 depth is at most target / smallest value

Answer: 3 combinations

TRAP

If the array can contain 1, “unlimited reuse” with target 40 explodes: the depth alone is 40 — dangerously close to nothing in C++ but perfectly fine in JS — and the answer count is huge. Always check the statement for a lower bound on the values, and say out loud in an interview: “this is only safe because every value is at least 2”.

Example 15 [Wipro pattern]

Return every subsequence of a string. A subsequence keeps the order but may drop letters. Constraints: length up to 18. Target: $O(2^n \times n)$. Edge cases: empty string; all letters the same.

SOLUTION

Identical shape to subsets — the pick-or-skip tree, with characters instead of numbers.

```
function subsequences(s) {
  const out = [], cur = [];
  const go = (i) => {
    if (i === s.length) { out.push(cur.join('')); return; }
    go(i + 1);           // skip s[i]
    cur.push(s[i]);
    go(i + 1);           // take s[i]
    cur.pop();
  };
  go(0);
  return out;
}
```

Ran it on "abc": "" "c" "b" "bc" "a" "ac" "ab" "abc" — 8. On "": one answer, the empty string.

Time $O(2^n \times n)$ **Space** $O(n)$

Answer: 8

Example 16 [Cognizant pattern]

Three pegs A, B, C. n disks of different sizes are stacked on A, biggest at the bottom. Move all disks to C. You may move one top disk at a time, and a bigger disk may never rest on a smaller one. Return the list of moves. Constraints: $1 \leq n \leq 20$. Target: $2^n - 1$ moves, which is optimal. Edge cases: $n = 0$ (zero moves); $n = 1$.

SOLUTION

The whole problem collapses into one sentence: **to move n disks from A to C, first move the top $n - 1$ to B, then move disk n to C, then move those $n - 1$ from B to C.**

```
function hanoi(n, from = 'A', to = 'C', via = 'B') {
  const log = [];
  const go = (k, f, t, v) => {
    if (k === 0) return;
    go(k - 1, f, v, t);           // v becomes the destination
    log.push(`disk ${k}: ${f}->${t}`);
    go(k - 1, v, t, f);           // v becomes the source
  };
  go(n, from, to, via);
  return log;
}
```

Ran hanoi(3). Output, in order:

1	2	3	4	5	6	7
d1 A → C	d2 A → B	d1 C → B	d3 A → C	d1 B → A	d2 B → C	d1 A → C

Counts: $n = 0 \rightarrow 0$, $n = 3 \rightarrow 7$, $n = 4 \rightarrow 15$. The pattern is $2^n - 1$.

Time $O(2^n)$ **Space** $O(n)$

$2^n - 1$ moves; this is a lower bound, no algorithm does better

The idea: notice the **rotation of roles**. In the first call, via becomes the new destination; in the second, via becomes the new source. Getting those three letters in the right order is the entire problem.

Answer: $2^n - 1$ moves; 7 for $n = 3$

Tier 2 — carry state, cut branches

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Tier 2 adds two things: the state you carry is bigger than an index, and you must **prune** — refuse to enter a branch that cannot possibly work.

Example 17 [Shopee · pattern]

A warehouse has n boxes with given weights. Some weights repeat. Choose a set of boxes whose weights sum to exactly target. Each box may be used at most once, and two different sets with the same multiset of weights count as one answer. Constraints: $1 \leq n \leq 20$, weights 1..50, target up to 200. Target: $O(2^n \times n)$. Edge cases: two boxes with equal weight; a weight larger than the target; a mix of one-digit and two-digit weights.

SOLUTION

Sort. Use the loop form. Two cuts:

- $j > i$ && $a[j] === a[j-1]$ — the duplicate rule from Example 8.
- $a[j] > left$ — since the array is sorted, every later value is also too big, so break, not continue.

each item once, no duplicate answers

JS

```
function combSum2(input, target) {
  const a = [...input].sort((x, y) => x - y);    // NUMERIC sort - see below
  const out = [], cur = [];
  const go = (i, left) => {
    if (left === 0) { out.push([...cur]); return; }
    for (let j = i; j < a.length; j++) {
      if (j > i && a[j] === a[j - 1]) continue;    // same choice as last loop turn
      if (a[j] > left) break;                    // sorted: the rest are bigger
      too
      cur.push(a[j]);
      go(j + 1, left - a[j]);                    // j+1: this box is now used up
      cur.pop();
    }
  };
  go(0, target);
  return out;
}
```

Ran it on [2,2,3,5] with target 7: [2,2,3] [2,5]. [2,5] appears once, even though there are two 2s that could start it.

Time $O(2^n \times n)$ **Space** $O(n)$
the break makes it far faster in practice

Answer: 2 sets

TRAP

This is the problem where the missing sort comparator actually changes the answer. I ran the same function twice on [2, 10, 3] with target 5, once with `.sort()` and once with `.sort((a,b) => a-b)`:

sort used	array becomes	result
.sort()	[10, 2, 3]	[] — wrong
.sort((a,b) => a-b)	[2, 3, 10]	[[2,3]] — correct

With the text sort, the first element is 10, which is bigger than the target 5, so the break fires immediately and the function reports “no solution”. The prune was written assuming ascending numeric order, and the text sort quietly broke that assumption.

Every prune is a promise about the order of your data. Break the order and you break the prune.

Example 18 [Grab · pattern]

Split a string into pieces so that **every** piece is a palindrome. Return all such splits. Constraints: length up to 16. Target: $O(2^n \times n)$. Edge cases: one character; no letter repeats (then the only split is into single characters).

SOLUTION

Approach 1 — Try every cut position, check the piece, recurse ($O(2^n \cdot n^2)$ — this is already the standard answer)

At index i , try every end j . If $s[i..j]$ is a palindrome, take it as one piece and solve from $j+1$.

palindrome partitioning

JS

```
const isPal = (s, i, j) => {
  while (i < j) { if (s[i] !== s[j]) return false; i++; j--; }
  return true;
};

function partition(s) {
  const out = [], cur = [];
  const go = (i) => {
    if (i === s.length) { out.push([...cur]); return; }
    for (let j = i; j < s.length; j++) {
      if (!isPal(s, i, j)) continue; // prune: illegal piece
      cur.push(s.slice(i, j + 1)); // slice(start, endExclusive)
      go(j + 1);
      cur.pop();
    }
  };
  go(0);
  return out;
}
```

Ran it on "aab": [["a", "a", "b"], ["aa", "b"]]. On "a": [["a"]].

Time $O(2^n \times n^2)$ **Space** $O(n)$
the extra n is the palindrome check

Approach 2 — Pre-compute a palindrome table ($O(2^n \cdot n)$ — drops one factor of n)

Build $pal[i][j]$ once in $O(n^2)$ with the rule $pal[i][j] = s[i] === s[j] \ \&\& \ (j - i < 2 \ || \ pal[i+1][j-1])$. Then the check inside the recursion is a single array read. For $n \leq 16$ it does not matter; say it anyway.

Answer: 2 splits of "aab"

TRAP

`s.slice(i, j + 1)` takes a start and an **exclusive** end. `s.substr(i, len)` takes a start and a **length** and is deprecated. `s.substring(i, j)` swaps its arguments if they are out of order, which hides bugs. Use `slice`, and remember the `+ 1`.

Example 19 [Agoda · pattern]

Given a string and a list of allowed words, can the string be cut into a sequence of allowed words? Words may be reused. Constraints: length up to 300, up to 1000 words each up to 20 characters. Target: $O(n^2 \times L)$. Edge cases: the string is exactly one word; a prefix matches but the rest fails; "aaaa" with words ["a", "aa"] (many ways to reach the same place).

SOLUTION

Approach 1 — Try every first word, recurse on the rest ($O(2^n)$) — dies on strings like aaaa...a

The killer input is "aaaa...a" with ["a", "aa"]: position 7 is reached from dozens of different prefixes, and each time you redo identical work.

Approach 3 — Memoise on the start index ($O(n^2 \cdot L)$) — optimal for this shape

The only thing that matters about “the work left” is **where you are**. That is one number, $0..n$. So one cache of size n is enough.

word break with a memo

JS

```
function canBreak(s, words) {
  const dict = new Set(words); // Set, not a plain object
  const memo = new Array(s.length).fill(-1);
  const go = (i) => {
    if (i === s.length) return true; // used up the string
    if (memo[i] !== -1) return memo[i] === 1;
    for (let j = i; j < s.length; j++) {
      if (dict.has(s.slice(i, j + 1)) && go(j + 1)) { memo[i] = 1; return true; }
    }
    memo[i] = 0;
    return false;
  };
  return go(0);
}
```

Ran it:

string	words	answer
"applepie"	["apple", "pie"]	true
"applepies"	["apple", "pie"]	false
"aaaa"	["a", "aa"]	true

Time $O(n^2 \times L)$ **Space** $O(n \times L)$

L is the cost of hashing a substring

The idea: the state is one index, so the cache is one array. Ask yourself on every recursion: **what is the smallest description of “everything still to do”?** That description is your memo key.

Answer: true, false, true

TRAP

Use a Set for the dictionary, not a plain object.

A plain object turns every key into a string, inherits keys from `Object.prototype`, and `dict["constructor"]` is truthy even when you never put it there — so a word like "constructor" or "toString" in the input produces a false positive. `Set.has` has none of those problems, keeps the value's type, and is faster for repeated lookups. The same argument applies to `Map` versus an object for counting.

Example 20 [LINE MAN · pattern]

A robot starts at the top-left of an $n \times n$ grid and must reach the bottom-right. 1 means the cell is open, 0 means blocked. It may move Down, Left, Right, Up, and may not step on the same cell twice in one path. Return every path as a string of D L R U. Constraints: $1 \leq n \leq 6$. Target: one node per reachable partial path. Edge cases: the start is blocked; $n = 1$; no path exists.

SOLUTION

This is the first problem where you must mark cells **visited**, because the robot may move in all four directions and would otherwise loop forever.

all paths through a maze

JS

```
function mazePaths(g) {
  const n = g.length, out = [];
  if (n === 0 || g[0][0] === 0) return out;           // the start is blocked
  const vis = g.map(r => r.map(() => false));          // a real 2-D array, see the
  trap
  vis[0][0] = true;
  const cur = [];
  const dr = [1, 0, 0, -1], dc = [0, -1, 1, 0], name = 'DLRU';
  const go = (r, c) => {
    if (r === n - 1 && c === n - 1) { out.push(cur.join('')); return; }
    for (let k = 0; k < 4; k++) {
      const nr = r + dr[k], nc = c + dc[k];
      if (nr < 0 || nc < 0 || nr >= n || nc >= n) continue;
      if (vis[nr][nc] || g[nr][nc] === 0) continue;
      vis[nr][nc] = true; cur.push(name[k]);
      go(nr, nc);
      cur.pop(); vis[nr][nc] = false;                // un-choose BOTH
    }
  };
  go(0, 0);
  return out;
}
```

Ran it on

```
1 0 0
1 1 0
0 1 1
```

Result: one path, "DRDR". On a grid whose start cell is 0: no paths. On the 1×1 grid `[[1]]`: one path, the empty string (you are already there).

Time $O(4^{n^2})$ **Space** $O(n^2)$

brutal in theory; the vis check makes it fine for $n \leq 6$

Answer: "DRDR"

TRAP

Two separate traps in one problem.

1. Two un-choose lines, not one: `cur.pop()` and `vis[nr][nc] = false`. Forgetting the second is the classic maze bug — you find one path, then the function reports “no more paths” because the whole grid is still marked visited.

2. Building the vis grid. I ran this to show why:

```
const g = [[1,0],[0,1]];
const shallow = [...g]; shallow[0][0] = 9; // g[0][0] is now 9 <- rows are shared!
const deep = g.map(r => [...r]); // this one is a real copy
```

And new `Array(n).fill(new Array(n).fill(false))` is worse still: all n rows are the **same** array, so marking one cell marks a whole column. Always build rows with `Array.from({length: n}, () => new Array(n).fill(false))` or `g.map(r => r.map(...))`.

Example 21 [Sea/Shopee · pattern]

A message was encoded with $a = 1, b = 2, \dots, z = 26$ and the separators were lost. Count how many messages could produce the given digit string. Constraints: length up to 100. Target: $O(n)$. Edge cases: empty string (answer 0); a leading 0; “106”; forty 1s.

SOLUTION

Approach 1 — Plain recursion on the start index ($O(2^n)$ — 2^{40} for forty 1s)

At index i you either read one digit (if it is not ‘0’) or two digits (if they form a number from 10 to 26).

Approach 3 — Memoise on the index ($O(n)$ — optimal)

count decodings

JS

```
function countDecodings(s) {
  if (s.length === 0) return 0;
  const memo = new Array(s.length).fill(-1);
  const go = (i) => {
    if (i === s.length) return 1; // reached the end cleanly
    if (s[i] === '0') return 0; // no letter is coded 0
    if (memo[i] !== -1) return memo[i];
    let total = go(i + 1); // take one digit
    if (i + 1 < s.length && Number(s.slice(i, i + 2)) <= 26) total += go(i + 2);
    return memo[i] = total;
  };
  return go(0);
}
```

Ran it:

input	answer	why
""	0	nothing to decode
"0"	0	no letter is 0
"12"	2	ab or l
"226"	3	bbf, bz, vf
"106"	1	only j f; 10 must stay together
"1010"	1	jj

forty 1s	165580141	the 41st Fibonacci number
----------	-----------	---------------------------

Time $O(n)$ **Space** $O(n)$

Answer: 3 for "226"

TRAP

"106": a naive solution splits as 1 | 0 | 6 and counts a fake answer. The guard `if (s[i] === '0') return 0;` at the **top** of the function handles every zero case at once, including "0", "100" and "1001".

For a 100-digit input of all 1s the count is about 5×10^{20} — **past the safe integer limit**. If the constraint were 100 rather than 40, this function would need BigInt. Work out the largest answer before you pick the number type.

Example 22 [SCB · pattern]

Count the subsets of an array that sum to a target. Constraints: $1 \leq n \leq 20$, values 0..100, target up to 500. Target: $O(2^n)$, or $O(n \times \text{target})$ with a memo. Edge cases: the array contains a 0; target 0; no subset works.

SOLUTION

Count, do not collect. So the function returns a number instead of pushing to a list — and there is nothing to undo, because nothing was changed.

count subsets with a given sum

JS

```
function countSubsets(a, target) {
  const go = (i, left) => {
    if (i === a.length) return left === 0 ? 1 : 0;
    const skip = go(i + 1, left);
    const take = a[i] <= left ? go(i + 1, left - a[i]) : 0;
    return skip + take;
  };
  return go(0, target);
}
```

Ran it: [1,2,1] target 2 → 2 (namely {2} and {1,1}). [0,1] target 1 → 2. [3] target 5 → 0.

Time $O(2^n)$ **Space** $O(n)$

add a memo on (i, target) to get $O(n \cdot \text{target})$

Answer: 2

TRAP

[0,1] with target 1 gives 2, not 1. The subsets {1} and {0,1} both sum to 1. A zero in the array doubles the count of every answer it can join. If the statement says “distinct subsets by value”, you must handle zeros separately. Always ask whether zeros are allowed.

Example 23 [GIC · pattern]

Count the ways to walk from the top-left to the bottom-right of an $(r + 1) \times (c + 1)$ grid moving only Right or Down. Constraints: $0 \leq r, c \leq 16$. Target: $O(r \times c)$. Edge cases: $r = 0$ (one straight line); $r = c = 16$; check the safe-integer limit.

SOLUTION

Approach 1 — Plain recursion $O(2^{(r+c)})$ — 2^{32} calls at $r = c = 16$

brute force

JS

```
function pathsSlow(r, c) {
  if (r === 0 || c === 0) return 1; // only one straight line left
  return pathsSlow(r - 1, c) + pathsSlow(r, c - 1);
}
```

Ran it: $\text{pathsSlow}(0,5) = 1$, $\text{pathsSlow}(2,2) = 6$, $\text{pathsSlow}(3,3) = 20$.

Approach 3 — Memoise on (r, c) $O(r \cdot c)$ — optimal

There are only $(r + 1)(c + 1)$ different states, but the brute force visits some of them millions of times.

with a memo

JS

```
function paths(r, c) {
  const memo = Array.from({ length: r + 1 }, () => new Array(c + 1).fill(-1));
  const go = (i, j) => {
    if (i === 0 || j === 0) return 1;
    if (memo[i][j] !== -1) return memo[i][j];
    return memo[i][j] = go(i - 1, j) + go(i, j - 1);
  };
  return go(r, c);
}
```

Ran it: $\text{paths}(3,3) = 20$ (matches the brute force — always check this) and $\text{paths}(16,16) = 601080390$. I also checked it is under `Number.MAX_SAFE_INTEGER` — it is.

Time $O(r \times c)$ **Space** $O(r \times c)$

The idea: the state is (r, c) and nothing else. Two numbers, so a 2-D cache.

Answer: 601080390

TRAP

Two things.

The memo grid. `Array.from({length: r+1}, () => new Array(c+1).fill(-1))` builds $r + 1$ **separate** rows. `new Array(r+1).fill(new Array(c+1).fill(-1))` builds one row and points at it $r + 1$ times — writing `memo[1][2]` would also change `memo[0][2]`.

The value size. 601080390 is fine. But $\text{paths}(30, 30)$ is about 1.18×10^{17} , which is **past** $2^{53} - 1$, so the count would be silently wrong. Path counts grow like $\binom{r+c}{r}$, and they explode. Check the biggest case the constraints allow.

Example 24 [DBS · pattern]

Does **any** subset of the array sum to exactly the target? Answer yes or no, as fast as you can. Constraints: $1 \leq n \leq 28$, values $0..10^4$, target up to 10^5 . Target: fast enough that $n = 28$ finishes in well under a second. Edge cases: empty array with target 0; a target larger than the whole array's sum; negative values (see the trap).

SOLUTION

Approach 1 — Plain pick-or-skip $O(2^n)$ — 2.7×10^8 calls at $n = 28$, too slow in JS

Approach 2 — Add a suffix-sum cut (same worst case, but orders of magnitude faster in practice)

Pre-compute $\text{suffix}[i] = a[i] + a[i+1] + \dots + a[n-1]$. If $\text{suffix}[i] < \text{left}$, then even taking **every** remaining number is not enough — return false at once instead of walking 2^{n-i} dead branches.

subset sum with a suffix cut

JS

```
function subsetSumPruned(a, target) {
  const n = a.length, suffix = new Array(n + 1).fill(0);
  for (let i = n - 1; i >= 0; i--) suffix[i] = suffix[i + 1] + a[i];
  const go = (i, left) => {
    if (left === 0) return true;
    if (i === n) return false;
    if (left < 0) return false;
    if (suffix[i] < left) return false; // cannot reach it any more
    return go(i + 1, left - a[i]) || go(i + 1, left);
  };
  return go(0, target);
}
```

Ran it: [3,34,4,12,5,2] target 9 → true (4+5); target 30 → false; empty array target 0 → true.

Time $O(2^n)$ **Space** $O(n)$

worst case unchanged; the cut is what makes $n = 28$ pass

Answer: true, false, true

TRAP

The suffix cut is only valid when every value is ≥ 0 . I ran both versions on [-5, 10] with target 10:

no pruning	true — the subset [10] works
with the suffix cut	false — WRONG

$\text{suffix}[0]$ is $-5 + 10 = 5$, which is less than 10, so the cut fires and kills the correct branch. Every pruning rule is a **claim about the future**. Write the claim down and check it holds for negatives and zeros before you trust it.

Tier 3 — the classics, and the follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 25 [Amazon · pattern]

Place n queens on an $n \times n$ board so that no two attack each other (same row, same column, or same diagonal). Return every arrangement. Constraints: $1 \leq n \leq 9$ in JavaScript (10 is about 5 seconds). Target: $O(n!)$. Edge cases: $n = 1$ (one solution); $n = 2$ and $n = 3$ (zero solutions); $n = 8$.

SOLUTION

First insight, and say it before you write code: **exactly one queen per row**. So the answer is an array $\text{pos}[\text{row}] = \text{column}$, and level row of the recursion chooses that column. That alone cuts the search from $\binom{n^2}{n}$ to n^n .

Approach 1 — Scan the rows above to check safety ($O(n! \cdot n)$) — accepted, easy to get right)

N-Queens, scanning check

JS

```
function queensSlow(n) {
  const out = [], pos = new Array(n).fill(-1);
  const safe = (row, col) => {
    for (let r = 0; r < row; r++) {
      if (pos[r] === col) return false;           // same column
      if (Math.abs(pos[r] - col) === row - r) return false; // same diagonal
    }
    return true;
  };
  const go = (row) => {
    if (row === n) { out.push([...pos]); return; }
    for (let col = 0; col < n; col++) {
      if (!safe(row, col)) continue;
      pos[row] = col;
      go(row + 1);
    }
  };
  go(0);
  return out;
}
```

Why is there no “undo” line for pos? Because pos[row] is **overwritten** by the next loop turn and nothing above row ever reads it. Overwriting is a legal form of undo.

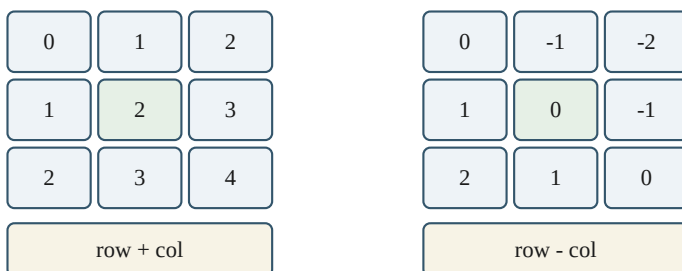
Time $O(n! \times n)$ **Space** $O(n)$
 the extra n is the safe() scan

Approach 3 — Three Sets, $O(1)$ safety check ($O(n!)$ — optimal)

A cell on the board sits on exactly one / diagonal and one \ diagonal:

- every cell of one / diagonal has the same row + col,
- every cell of one \ diagonal has the same row - col.

So three Sets answer “is this attacked?” in one step. (Set keeps numbers as numbers; a plain object would turn -2 into the string “-2” and still work, but a Set says what you mean.)



row+col is constant on one diagonal; row-col on the other

N-Queens, $O(1)$ check

JS

```
function queens(n) {
  const out = [], pos = new Array(n).fill(-1);
  const col = new Set(), d1 = new Set(), d2 = new Set();
  const go = (row) => {
    if (row === n) { out.push([...pos]); return; }
    for (let c = 0; c < n; c++) {
      if (col.has(c) || d1.has(row + c) || d2.has(row - c)) continue;

```

```

    col.add(c); d1.add(row + c); d2.add(row - c); pos[row] = c;
    go(row + 1);
    col.delete(c); d1.delete(row + c); d2.delete(row - c);    // un-choose
  }
};
go(0);
return out;
}

```

I ran **both** versions for $n = 1..8$ and the counts agree exactly:

n	1	2	3	4	5	6	7	8
solutions	1	0	0	2	10	4	40	92

For $n = 4$ the two solutions are $[1, 3, 0, 2]$ and $[2, 0, 3, 1]$:

• Q • •	• • Q •
• • • Q	Q • • •
Q • • •	• • • Q
• • Q •	• Q • •

the two 4-queens boards; Q marks a queen

Time $O(n!)$ **Space** $O(n)$

The idea: turn “is this square attacked?” from a **scan** into a **lookup** by giving every diagonal a name. That is the same move as a hash map replacing a linear search.

Answer: 92 solutions for $n = 8$

THE FOLLOW-UPS AN INTERVIEWER ASKS AFTER N-QUEENS

“Only count the solutions, do not store them.” Return a number; drop `pos` and the output array entirely. Space falls to $O(n)$.

“Use bitmasks instead of Sets.” Keep three ordinary numbers `col`, `d1`, `d2`. Free squares in this row are `free = ~(col | d1 | d2) & ((1 << n) - 1)`. Take the lowest set bit with `bit = free & -free`, and recurse with `(col | bit, (d1 | bit) << 1, (d2 | bit) >> 1)`. Same $O(n!)$, but several times faster — and note that JS bitwise operators work on **32-bit** integers, so this only holds up to $n = 31$.

“n is 10000.” Then you are not searching — you are **constructing**. For $n \geq 4$ an explicit formula places the queens in $O(n)$; searching is the wrong tool once you only need **one** solution for huge n .

Example 26 [Microsoft · pattern]

Fill in a partly-filled 9×9 Sudoku. Blank cells are ‘.’. Every row, every column and every 3×3 box must end up holding 1..9 exactly once. Constraints: the board is 9×9 and a solution exists. Target: instant on a real puzzle. Edge cases: an already-full board; a board with no solution.

SOLUTION

Two pieces: a legality check, and a recursion that finds the **first** blank and tries all nine digits there.

The cell at (r, c) lives in the box whose top-left corner is $(3 * \text{Math.floor}(r/3), 3 * \text{Math.floor}(c/3))$. So the k -th cell of that box is $(3 * \text{Math.floor}(r/3) + \text{Math.floor}(k/3), 3 * \text{Math.floor}(c/3) + k \% 3)$ for $k = 0..8$. That lets one loop check row, column and box together.


```
function solveSudoku(board) { // board is a 9x9 array of characters
  const ok = (r, c, v) => {
    for (let k = 0; k < 9; k++) {
      if (board[r][k] === v) return false; // row
      if (board[k][c] === v) return false; // column
      const br = 3 * Math.floor(r / 3) + Math.floor(k / 3);
      const bc = 3 * Math.floor(c / 3) + (k % 3);
      if (board[br][bc] === v) return false; // 3x3 box
    }
    return true;
  };
  const go = () => {
    for (let r = 0; r < 9; r++)
      for (let c = 0; c < 9; c++) {
        if (board[r][c] !== '.') continue;
        for (const v of '123456789') {
          if (!ok(r, c, v)) continue;
          board[r][c] = v;
          if (go()) return true; // a deeper call succeeded
          board[r][c] = '.'; // undo
        }
        return false; // nothing fits here
      }
    return true; // no blank left: done
  };
  return go();
}
```

Ran it on this board (36 clues, one solution). Build it with `rows.map(r => [...r])` so each row is its own array:

puzzle	solver output
..82...61	358294761
7..5.....	761583429
.2..1.35.	429617358
9.2.6....	972361845
8.5...136	845729136
13..5897.	136458972
5941.26..	594172683
.....521.	683945217
..7..6..4	217836594

It printed `solved = true` in **3 milliseconds**. On an already-full valid board it returns `true` without changing anything. On a board I deliberately broke (a duplicate in row 0) it returns `false`.

Time $O(9^b \times 81)$ **Space** $O(b)$

b = number of blanks; the `ok()` pruning makes real puzzles instant

The idea: the two returns at the bottom are opposite in meaning and both essential. `return false` inside the cell loop means “this blank has no legal digit, so whoever called me made a bad choice — back up.” `return true` at the very end means “I scanned the whole board and found no blank, so we are finished.”

TRAP

board must be an array of **arrays of characters**, not an array of strings. Strings in JavaScript are immutable, so `board[r][c] = v` on a string row does **nothing at all** — no error, no change, and the solver loops until it gives up. Convert first with `rows.map(r => [...r])`, and print with `board.map(r => r.join('')).join('\n')`.

THE FOLLOW-UPS AFTER SUDOKU

“Make it faster.” Replace the $O(9)$ `ok()` scan with three arrays of 9-bit masks: `rowMask`, `colMask`, `boxMask`. Then legality is one `&` and updating is one `^=`. Typically 10 to 30 times faster.

“Choose a smarter cell.” Instead of the first blank, pick the blank with the **fewest** legal digits (the MRV heuristic). Fail fast, fail high in the tree.

“Count all solutions.” Do not return `true` on success — increment a counter and keep going. That is how you test whether a generated puzzle is unique. It is also how I checked the puzzle above.

Example 27 [Google · pattern]

Given a grid of letters and a word, decide whether the word can be spelled by walking between side-by-side cells (up, down, left, right). A cell may not be used twice in one walk. Constraints: grid up to 8×8 , word up to 12 letters. Target: $O(RC4^L)$ worst case, far less with the letter check. Edge cases: empty word; the word’s first letter is not in the grid; a word that would need to reuse a cell.

SOLUTION

Start the walk from every cell. Inside, the recursion checks position, letter and visited-ness all at the top, which keeps the body short.

The neat trick: instead of a separate visited grid, **overwrite the cell with a character that cannot match** (here `'#'`), then put it back on the way out. Same effect, zero extra memory.

```
word search JS

function wordExists(grid, w) {
  if (w.length === 0) return true;
  const g = grid.map(r => [...r]);           // deep copy: we scribble on it
  const R = g.length, C = g[0].length;
  const go = (r, c, k) => {
    if (k === w.length) return true;        // all letters matched
    if (r < 0 || c < 0 || r >= R || c >= C) return false;
    if (g[r][c] !== w[k]) return false;     // wrong letter, or the '#' marker
    const save = g[r][c];
    g[r][c] = '#';                          // mark visited
    const found = go(r+1, c, k+1) || go(r-1, c, k+1)
                  || go(r, c+1, k+1) || go(r, c-1, k+1);
    g[r][c] = save;                          // undo
    return found;
  };
  for (let r = 0; r < R; r++)
    for (let c = 0; c < C; c++)
      if (go(r, c, 0)) return true;
  return false;
}
```

Ran it on

```
c a t
x r e
d o g
```

word	answer	why
"cat"	true	straight along row 0
"care"	true	c(0,0) → a(0,1) → r(1,1) → e(1,2)
"cart"	false	after car the only t is at (0,2), not next to (1,1)
"catx"	false	x is at (1,0), not next to t at (0,2)

Time $O(R \times C \times 4^L)$ **Space** $O(L)$
R, C grid size; L word length

The idea: the `||` chain short-circuits. As soon as one direction succeeds, JavaScript stops evaluating the rest — you get early exit for free, but **only** if you restore `g[r][c]` after the chain, not inside it.

THE FOLLOW-UP: MANY WORDS AT ONCE

“Now find all 5000 words from a dictionary in the same grid.” Running this search 5000 times is far too slow. Put every word into a **trie** (Chapter 6 territory) and walk the grid **once**, carrying a trie node instead of an index. The search stops the moment the prefix leaves the trie. That is the standard escalation, and naming it earns the point even if you do not have time to code it.

Example 28 [Adobe · pattern]

Given a string of digits and a target, insert +, - or * between some digits so the expression evaluates to the target. Digits may be glued into multi-digit numbers, but a glued number may not have a leading zero. Return every expression. Constraints: up to 10 digits; the target fits in a safe integer. Target: $O(4^n \times n)$. Edge cases: "105" target 5 (both $1*0+5$ and $10-5$ work); no solution; a single digit.

SOLUTION

Two hard parts. First, at each position you choose **where the current number ends**, so there is an inner loop gluing digits. Second, * binds tighter than + and -, so a running total is not enough.

The trick for *: carry last — the value of the term you most recently added. To multiply, **undo** that term and re-add it multiplied: `value - last + last * num`, and the new last becomes `last * num`.

so far	value	last	then *4
2+3	5	3	$5 - 3 + 3 \times 4 = 14$, new last = 12
2+3*4	14	12	$14 - 12 + 12 \times 5 = 62$ for *5

insert + - * to hit a target

JS

```
function findExpressions(d, target) {
  const out = [];
  if (d.length === 0) return out;
  const go = (i, expr, value, last) => {
    if (i === d.length) { if (value === target) out.push(expr); return; }
    for (let j = i; j < d.length; j++) {
      if (j > i && d[i] === '0') break; // no leading zero
      const part = d.slice(i, j + 1);
      const num = Number(part);
      if (i === 0) {
        go(j + 1, part, num, num); // the first number
      } else {
        go(j + 1, expr + '+' + part, value + num, num);
        go(j + 1, expr + '-' + part, value - num, -num);
      }
    }
  };
  go(0, '', 0, 0);
  return out;
}
```

```

        go(j + 1, expr + '*' + part,
           value - last + last * num, last * num);
    }
}
};
go(0, '', 0, 0);
return out;
}

```

Ran it:

digits	target	answers
"123"	6	"1+2+3", "1*2*3"
"105"	5	"1*0+5", "10-5"
"11"	9	none

Time $O(4^n \times n)$ **Space** $O(n)$
 4 = glue, +, -, *; the extra n is string building

Answer: "1+2+3" and "1*2*3"

TRAP

Ten glued digits give up to 9,999,999,999, and a running product of two such numbers is about 10^{20} — **past the safe integer limit**. For 10 digits the intermediate values stay safe in practice, but if the constraint were 16 digits you would need BigInt for value, last and num throughout.

And the break on a leading zero must come **before** part is built, otherwise "05" is quietly accepted as 5.

Example 29 [Goldman Sachs · pattern]

Consider the numbers $1..n$ written in every possible order, and list those orders alphabetically. Return the k -th one. Constraints: $1 \leq n \leq 9$, $1 \leq k \leq n!$. Target: $O(n^2)$ without generating all $n!$ orders. Edge cases: $k = 1$ (the sorted order); $k = n!$ (the reversed order).

SOLUTION

Approach 1 — Generate all $n!$ permutations, sort, take index $k-1$ ($O(n! \cdot n \log n!)$ — $9! = 362880$, slow in JS, and pure waste)

Approach 3 — Count instead of generate ($O(n^2)$ — optimal)

Fix the first digit. The remaining $n - 1$ digits can be arranged in $(n - 1)!$ ways. So the first $(n - 1)!$ orders all start with the smallest digit, the next $(n - 1)!$ with the second smallest, and so on. One division tells you which one.

Work with $k - 1$ (0-based), then repeat for each slot.

k-th permutation directly

JS

```

function kthPermutation(n, k) {
    // k is 1-based
    const f = [1];
    for (let i = 1; i <= n; i++) f[i] = f[i - 1] * i;
    const left = [];
    for (let i = 1; i <= n; i++) left.push(String(i));
    k--;
    // make it 0-based
    const res = [];

```

```

for (let slot = n; slot >= 1; slot--) {
  const block = f[slot - 1];           // orders sharing one leading digit
  const idx = Math.floor(k / block);
  res.push(left[idx]);
  left.splice(idx, 1);                 // remove that digit from the pool
  k %= block;
}
return res.join('');
}

```

Worked trace for $n = 4, k = 9$. Start $k = 8$, $\text{left} = ["1", "2", "3", "4"]$:

slot	block	k	idx = floor(k/block)	picked	new k
4	3! = 6	8	1	"2"	8 mod 6 = 2
3	2! = 2	2	1	"3"	2 mod 2 = 0
2	1! = 1	0	0	"1"	0
1	0! = 1	0	0	"4"	0

Ran it: $\text{kthPermutation}(3,1) = "123"$, $\text{kthPermutation}(3,4) = "231"$, $\text{kthPermutation}(3,6) = "321"$, $\text{kthPermutation}(4,9) = "2314"$ — matching the table.

Time $O(n^2)$ **Space** $O(n)$
 the n^2 is splice() shifting the remaining elements

The idea: stop enumerating, start counting. Whenever a question asks for “the k -th thing in order”, check whether you can **count** how many things start with each choice.

Answer: "2314"

TRAP

$\text{Math.floor}(k / \text{block})$ is required. Plain k / block gives a fraction, and using a fraction as an array index gives undefined — which $\text{join}('')$ happily turns into the text "undefined" with no error at all. Any time you divide in JavaScript and mean integer division, wrap it in Math.floor (or use Math.trunc if negatives are possible).

Example 30 [D. E. Shaw · pattern]

Split an array of positive integers into exactly k groups with equal sums. Return true or false. Constraints: $1 \leq n \leq 16$, $1 \leq k \leq n$, values up to 10^4 . Target: fast enough for $n = 16$. Edge cases: the total is not divisible by k ; one value is larger than the target; $k = 1$; $k = n$ with all values equal.

SOLUTION

Approach 1 — Try all k^n assignments of items to groups ($O(k^n)$ — 16^{16} , hopeless)

Approach 3 — Place items one by one, with three cuts (fast enough for $n = 16$)

Three cheap tests kill almost everything before the search starts:

1. If $\text{sum} \% k \neq 0$, answer false.
2. If the largest value exceeds sum / k , answer false — it fits nowhere.
3. Sort and place the **largest** item first. Big items have the fewest homes, so they cause failures near the root of the tree instead of near the leaves.

And one cut inside the loop that people miss: if putting the item in an **empty** group fails, then putting it in any other empty group also fails — the groups are interchangeable. So break after an empty group fails.

```
function canSplit(input, k) {
  if (k <= 0 || input.length < k) return false;
  const sum = input.reduce((x, y) => x + y, 0);
  if (sum % k !== 0) return false; // cut 1
  const target = sum / k;
  const a = [...input].sort((x, y) => x - y); // NUMERIC sort
  if (a[a.length - 1] > target) return false; // cut 2
  const bucket = new Array(k).fill(0);
  const go = (i) => {
    if (i < 0) return true; // every item placed
    for (let b = 0; b < k; b++) {
      if (bucket[b] + a[i] > target) continue;
      bucket[b] += a[i];
      if (go(i - 1)) return true;
      bucket[b] -= a[i];
      if (bucket[b] === 0) break; // an empty bucket failed -> all empties fail
    }
    return false;
  };
  return go(a.length - 1); // largest item first
}
```

Ran it:

array	k	answer	why
[2,1,3,4]	2	true	[1,4] and [2,3], each 5
[1,1,1]	2	false	sum 3 is not divisible by 2
[5,5,5,5]	4	true	one item per group
[9,1,1,1]	2	false	sum 12, target 6, but 9 > 6
[4]	1	true	one group holding everything

Time $O(k \times 2^n)$ **Space** $O(n + k)$

worst case; the cuts usually finish in microseconds

The idea: symmetry breaking. Two empty buckets are the **same** bucket as far as the answer is concerned. Whenever your search has interchangeable slots, one break removes a factorial-sized chunk of the tree.

THE FOLLOW-UPS AFTER EQUAL-SUM GROUPS

“Return the actual groups, not just true/false.” Carry a groups array of arrays alongside bucket, push and pop in the same places. Nothing else changes.

“n is 30.” Switch to bitmask DP over subsets: $dp[mask]$ = “can the items in mask be packed into full groups, and how much is in the group currently being filled”. That is $O(2^n \times n)$ — Chapter 18. Note that a mask up to 2^{30} is fine as a JS number, but a mask beyond 2^{31} breaks the **bitwise operators**, which are 32-bit. Above that you need BigInt masks or a different encoding.

“The groups may differ by at most 1.” Now it is an optimisation, not a decision. Binary search on the allowed gap, or switch to a greedy plus local search — and say clearly that the exact version is NP-hard, so a heuristic is expected.

Count the orderings $p_1, p_2, \dots, p_n$ of $1..n$ where, for every slot i , either p_i divides i or i divides p_i . Constraints: $1 \leq n \leq 14$ in JavaScript. Target: fast enough that $n = 14$ finishes in about a second. Edge cases: $n = 1$ (answer 1); $n = 2$ (answer 2).

SOLUTION

Fill the slots $1, 2, \dots, n$ in order. At slot `slot`, try every unused value v that satisfies the divisibility rule. The rule itself is the prune.

count divisible arrangements

JS

```
function countArrangements(n) {
  const used = new Array(n + 1).fill(false);
  const go = (slot) => {
    if (slot > n) return 1; // all slots filled
    let total = 0;
    for (let v = 1; v <= n; v++) {
      if (used[v]) continue;
      if (v % slot !== 0 && slot % v !== 0) continue; // the rule
      used[v] = true;
      total += go(slot + 1);
      used[v] = false;
    }
    return total;
  };
  return go(1);
}
```

Ran it for $n = 1..8$:

n	1	2	3	4	5	6	7	8
count	1	2	3	8	10	36	41	132

Notice $n = 7$: only 41, barely more than $n = 6$. Slot 7 can hold only the value 7 or 1, and 7 is prime, so the tree is very thin. Primes choke this problem.

Time $O(n!)$ **Space** $O(n)$

worst case; in truth far smaller, because the divisibility rule cuts most branches at depth 2

Answer: 132 for $n = 8$

THE FOLLOW-UP: $N = 20$

“Now n is 20.” $20!$ is 2.4×10^{18} — hopeless by pure search. But notice the state: **which slot am I on** plus **which values are used**. The slot number is redundant — it equals the count of set bits in the used-set plus 1. So the state is a single 20-bit mask, and there are only $2^{20} \approx 10^6$ of them.

`dp[mask]` = number of ways to fill the first `popcount(mask)` slots using exactly the values in `mask`. Cost $O(2^n \times n)$, about 20 million steps. This is **bitmask DP** and it is the single most common “now make it bigger” escalation in this chapter.

In JavaScript, keep $n \leq 30$ for this: `1 << 31` is **negative**, because bitwise operators coerce to signed 32-bit integers. Past that, index with plain numbers (`Math.floor(mask / 2)` instead of `mask >> 1`) or use `BigInt`.

Dry run — subsets of [1, 2, 3], every single step

This is the recursion from Example 7. I will write `go(i)` and show the output list after every event. Read it once slowly with a pen; after that, every backtracking trace in this book is the same picture.

```
const go = (i) => {
  if (i === 3) { out.push(...cur); return; } // a.length is 3
  go(i + 1); // (A) not pick
  cur.push(a[i]); // (B) pick
  go(i + 1); // (C)
  cur.pop(); // (D) undo
};
```

step	event	i	cur	out so far
1	enter go(0)	0	[]	[]
2	(A) enter go(1)	1	[]	[]
3	(A) enter go(2)	2	[]	[]
4	(A) enter go(3)	3	[]	[]
5	base: push a copy of []	3	[]	[[]]
6	back in go(2), (B) push 3	2	[3]	[[]]
7	(C) enter go(3), push	3	[3]	[[] , [3]]
8	(D) pop → go(2) returns	2	[]	[[] , [3]]
9	back in go(1), (B) push 2	1	[2]	[[] , [3]]
10	(C) enter go(2)	2	[2]	[[] , [3]]
11	(A) go(3) pushes	3	[2]	[[] , [3] , [2]]
12	(B) push 3	2	[2,3]	[[] , [3] , [2]]
13	(C) go(3) pushes	3	[2,3]	... [2] , [2,3]]
14	(D) pop → go(2) returns	2	[2]	... [2] , [2,3]]
15	(D) pop → go(1) returns	1	[]	... [2] , [2,3]]
16	back in go(0), (B) push 1	0	[1]	... [2] , [2,3]]
17	(C) the whole thing repeats with a leading 1	—	—	adds [1] [1,3] [1,2] [1,2,3]
18	(D) pop → go(0) returns	0	[]	8 subsets

Final out: [] [3] [2] [2,3] [1] [1,3] [1,2] [1,2,3] — exactly what the program printed.

THREE THINGS THIS TABLE PROVES

1. cur is **always** back to [] when go(0) returns. That is the invariant. If it is not, you have a missing cur.pop().
2. The base case fires $2^3 = 8$ times — once per leaf. Every leaf is one answer.
3. Steps 6 and 16 are the **only** places anything is added to cur, and steps 8, 14, 15 and 18 are the only places anything is removed. Pushes and pops always come in pairs.

And one thing the table makes obvious: at step 5, cur is the **same array object** as at step 18. That is precisely why the base case pushes [...cur] and not cur.

Recursion is too deep — rewrite it as a loop

JavaScript gives you roughly 10^4 stack frames. When that is not enough, turn the recursion into an explicit stack. Every recursive call becomes an object you push; every return becomes a pop.


```
function subsetsIterative(a) {
  const out = [];
  const stack = [{ i: 0, cur: [] }];
  while (stack.length > 0) {
    const { i, cur } = stack.pop();
    if (i === a.length) { out.push(cur); continue; }
    stack.push({ i: i + 1, cur: [...cur, a[i]] }); // pick
    stack.push({ i: i + 1, cur: [...cur] });        // skip (popped first)
  }
  return out;
}
```

Ran it on `[1,2,3]`: `[] [3] [2] [2,3] [1] [1,3] [1,2] [1,2,3]` — the same eight subsets, in the same order as the recursive version. On `[]`: `[[]]`.

Then I compared the two on a 20,000-level walk:

plain recursion, 20,000 levels	threw <code>RangeError: Maximum call stack size exceeded</code>
explicit stack, 20,000 levels	finished, 20,000 levels walked

The price of the explicit stack is memory: each pushed frame holds its own **copy** of `cur`, so peak memory is higher than the recursive version, which shares one `cur`. For a backtracking search of depth 10^4 or more, that is the trade you make. For depth under a few thousand, prefer the recursive version — it is far easier to read and to get right.

Python — the three signature problems

Python is the secondary language of this book. Here it is genuinely shorter, and worth knowing if your round lets you choose.

```
def subsets(a):
    out, cur = [], []
    def go(i):
        if i == len(a):
            out.append(cur[:])          # copy, not the live list
            return
        go(i + 1)
        cur.append(a[i])
        go(i + 1)
        cur.pop()
    go(0)
    return out

def permute(a):
    out = []
    def go(i):
        if i == len(a):
            out.append(a[:])
            return
        for j in range(i, len(a)):
            a[i], a[j] = a[j], a[i]
            go(i + 1)
            a[i], a[j] = a[j], a[i]
```

```

    go(0)
    return out

def n_queens(n):
    out, pos = [], [-1] * n
    col, d1, d2 = set(), set(), set()
    def go(row):
        if row == n:
            out.append(pos[:])
            return
        for c in range(n):
            if c in col or (row + c) in d1 or (row - c) in d2:
                continue
            col.add(c); d1.add(row + c); d2.add(row - c); pos[row] = c
            go(row + 1)
            col.remove(c); d1.remove(row + c); d2.remove(row - c)
    go(0)
    return out

```

Ran all three:

- `subsets([1,2,3])` → `[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]`
- `subsets([])` → `[[]]`
- `permute([1,2,3])` → `[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,2,1], [3,1,2]]`
- `n_queens(4)` → `[[1, 3, 0, 2], [2, 0, 3, 1]]`
- solution counts for $n = 1..8$ → `[1, 0, 0, 2, 10, 4, 40, 92]` — identical to the JS.

SHORTCUT

Two things Python gives you free that JavaScript does not.

1. Integers are arbitrary precision. `math.factorial(30)` is exact with no BigInt ceremony.
2. `sorted(a)` and `a.sort()` already compare numbers as numbers — there is no lexicographic surprise.

And one thing that is worse: the default recursion limit is **1000**, even lower than JS. `sys.setrecursionlimit(300000)` raises it, but the real C stack may still blow up. And `out.append(cur)` stores a reference, exactly like `out.push(cur)` in JS — use `cur[:]`.

Practice

PRACTICE · WARM-UP target: 20 min

1. Write `sumTo(n) = 1 + 2 + ... + n` recursively. What is `sumTo(100)`?
2. Count the digits of a non-negative number recursively. What does your base case do for input 0?
3. Recursively check whether a string is a palindrome, without building a reversed copy.
4. Print the numbers $n, n - 1, \dots, 1$ and then $1, 2, \dots, n$ using **one** recursive function and no loop.

PRACTICE · TIER 1 target: 45 min

5. Return all subsets of `[1, 2, 3, 4]` whose sum is exactly 5.
6. Return all strings of length 3 made from `['a', 'b']`. How many are there for length n ?
7. Given n , return all ways to write n as a sum of positive integers in non-increasing order. For $n = 4$: `4, 3+1, 2+2, 2+1+1, 1+1+1+1`.
8. Count the paths in an $n \times n$ grid from top-left to bottom-right moving only Right or Down, where the cell (i, i) is blocked for every i with $0 < i < n - 1$.

9. Given a list of coin values and an amount, count the ways to make the amount. Order does not matter and coins are unlimited.

PRACTICE · TIER 2 target: 50 min

10. Given a string of digits, insert `.` to make four parts, each a number from 0 to 255 with no leading zeros (except "0" itself). Return every valid result.
11. Given an array, return all subsets where no two chosen elements sit next to each other in the array.
12. A knight starts at $(0,0)$ on an $n \times n$ board. Count the ways it reaches $(n-1,n-1)$ in exactly m moves. Memoise on $(r, c, movesLeft)$.
13. Given n tasks with durations and 2 identical workers, decide whether the tasks can be split so both workers finish at the same time.

PRACTICE · TIER 3 target: 60 min

14. Place n rooks on an $n \times n$ board with some squares forbidden, so that no two rooks share a row or column. Count the placements.
15. Given a string, count the ways to split it into pieces so that no piece repeats.
16. Given an array of $n \leq 16$ positive numbers, split it into two groups so the difference of their sums is as small as possible. Return that difference.

ANSWER KEY

1. `const sumTo = n => n === 0 ? 0 : n + sumTo(n - 1);` Ran it: `sumTo(100)` is **5050**. Depth is n , so this throws a `RangeError` around $n = 12000$ — use the formula $\frac{n(n+1)}{2}$ there.
2. `const countDigits = n => n < 10 ? 1 : 1 + countDigits(Math.floor(n / 10));` The base must be $n < 10$, not $n === 0$, otherwise input 0 returns 0 digits instead of 1. Ran it: `0 → 1, 4073 → 4`. And do not forget `Math.floor`.
3. `const pal = (s,i,j) => i >= j ? true : (s[i] !== s[j] ? false : pal(s,i+1,j-1));` The `i >= j` base covers even and odd lengths, and the empty string (called with `i = 0, j = -1`) returns true. Ran it: `"level" → true, "abca" → false`.
4. Put a `push` **before** the recursive call and a second one **after** it: `const f = n => { if (n === 0) return; out.push(n); f(n-1); out.push(n); }` The pushes after the call run during unwinding, so they come out in the opposite order. Ran `f(3)`: **3 2 1 1 2 3**. This one idea — do work on the way down, do work on the way up — is what makes tree traversals in Chapter 12 easy.
5. Take the subsets code from Example 7 and carry a running `sum`. Push only when `i === a.length && sum === 5`. Better: prune with `if (sum > 5) return;` — legal here because all values are positive. Ran it: 2 answers, `[2,3]` and `[1,4]`.
6. Two branches at each level, `'a'` and `'b'`. Ran it for length 3: `aaa aab aba abb baa bab bba bbb` — **8**. In general 2^n ; for an alphabet of size m it is m^n .
7. Carry `maxAllowed` so the pieces never increase: `go(remaining, maxAllowed)` loops `v` from `Math.min(maxAllowed, remaining)` down to 1. Ran it: $n = 4$ gives 5 (`4, 3+1, 2+2, 2+1+1, 1+1+1+1`) and $n = 10$ gives **42**.
8. Same memo as Example 23, but return 0 when the cell is blocked: `if (r > 0 && r < n-1 && r === c) return 0;` Ran it: $n = 3$ leaves 2 paths, $n = 4$ leaves 4 (out of the usual 20), and $n = 5$ leaves **10**.
9. Exactly the shape of Example 14, but return a count instead of collecting lists: `ways(i, amt) = ways(i, amt - coins[i]) + ways(i + 1, amt)`. Memoise on (i, amt) — a `Map` keyed by $\{i\}, \{amt\}$ is the simplest way, because JS has no tuple keys. Ran it for coins `[1,2,5]` and amount 5: **4** ways — `1+1+1+1+1, 1+1+1+2, 1+2+2, 5`.
10. Four levels; at each level take 1, 2 or 3 digits. Reject a piece if it starts with `'0'` and is longer than one character, or if its value is over 255. Ran it: `"25525"` gives 4 answers — `2.5.5.25, 2.5.52.5, 2.55.2.5, 25.5.2.5`. `"1111"` gives exactly one, `1.1.1.1`.

11. Pick-or-skip, but after picking index i recurse to $i + 2$: `go(i+2)` for the pick branch, `go(i+1)` for the skip branch. Ran it on $[1, 2, 3, 4]$: **8** subsets — $[]$ $[4]$ $[3]$ $[2]$ $[2, 4]$ $[1]$ $[1, 4]$ $[1, 3]$. The counts go 2, 3, 5, 8 — Fibonacci, exactly like Example 13.

12. Eight knight moves. State is $(r, c, \text{movesLeft})$; use a `Map` keyed by $\{r\}, \{c\}, \{k\}$. Base: `movesLeft === 0` returns $(r === n-1 \ \&\& \ c === n-1) ? 1 : 0$. Ran it on a 4×4 board: $m = 2$ gives **2** ways and $m = 4$ gives **12**; $m = 1$ and $m = 3$ give **0**, because a knight changes square colour every move and $(0, 0)$ and $(3, 3)$ are the same colour — parity kills all odd m .

13. This is Example 30 with $k = 2$. Total must be even, no single task may exceed half the total, then search. Ran it: $[3, 3, 4, 4] \rightarrow \text{true}$ (7 and 7); $[1, 2, 4] \rightarrow \text{false}$ (total 7 is odd).

14. One rook per row, like N-Queens but with **no diagonal test**: only a `col[]` array and a forbidden-square check. Without forbidden squares the answer is exactly $n!$ — I ran $n = 4$ and got **24**. Forbidding the single square $(0, 0)$ drops it to **18**, which is $24 - 3! = 24 - 6$: the placements that used to put a rook there.

15. Loop the cut point j from i , keep a `Set` of pieces already used. `add` the piece, recurse, then `delete` it — that delete is the un-choose step and it is easy to forget. Ran it: `"aba" → 3` (`ab|a`, `a|ba`, `aba`; the split `a|b|a` is rejected because `a` repeats). `"abc" → 4`, `"aa" → 1`.

16. Every subset has a sum; the other group's sum is `total - sum`. So walk the pick-or-skip tree and track `Math.min(best, Math.abs(total - 2 * sum))`. Cost $O(2^n)$, fine for $n \leq 16$. Ran it: $[1, 6, 11, 5] \rightarrow 1$ (total 23, $[1, 5, 6] = 12$ against $[11] = 11$); $[1, 2, 3, 4] \rightarrow 0$ ($[1, 4]$ and $[2, 3]$).

ONE-PAGE REVISION CARD

The template you write every time

```
const go = (state) => {
  if (isComplete(state)) { out.push([...cur]); return; } // COPY cur
  for (const o of options(state)) {
    if (!legal(state, o)) continue; // prune
    apply(cur, o); // choose
    go(next(state, o)); // explore
    undo(cur, o); // un-choose
  }
};
```

Pick the right tree

Question says	Tree	Cost
subsets, subsequences	go(i+1), then push, go(i+1), pop	$O(2^n n)$
permutations, orderings	swap a[i], a[j], or a used[] array	$O(n!n)$
combinations summing to X	loop j >= i, recurse at j (reuse) or j+1 (once)	target-dependent
split a string	loop the cut end j, recurse at j+1	$O(2^n)$
place on a board	one level per row + Sets for column and both diagonals	$O(n!)$
fill a grid	first blank, try all values, undo	$O(v^b)$
count only	return a number; add a memo if states repeat	often polynomial

The duplicate rules — memorise these two lines

subsets / combinations	if (j > i && a[j] === a[j-1]) continue; after a numeric sort
permutations	if (i > 0 && a[i] === a[i-1] && !used[i-1]) continue; after a numeric sort

Complexity cheat sheet

Problem	Time	Space
all subsets	$O(2^n \times n)$	$O(n)$
all permutations	$O(n! \times n)$	$O(n)$
balanced brackets	$O(4^n / \sqrt{n})$	$O(n)$
palindrome split	$O(2^n \times n^2)$	$O(n)$
N-Queens	$O(n!)$	$O(n)$
Sudoku	$O(9^b)$	$O(b)$
word search	$O(RC4^L)$	$O(L)$
k-th permutation	$O(n^2)$	$O(n)$
stairs / decodings with a memo	$O(n)$	$O(n)$
grid paths with a memo	$O(rc)$	$O(rc)$

Top traps — the JavaScript ones first

1. **out.push(cur) instead of out.push([...cur]).** Arrays are references. Your answer list fills with copies of one empty array.
2. **arr.sort() sorts as text.** `[10,9,1].sort()` is `[1,10,9]`. Always `arr.sort((a,b) => a-b)`, especially before a prune that assumes ascending order.
3. **Numbers are doubles**, exact only to `Number.MAX_SAFE_INTEGER =  $2^{53} - 1$` . Factorials, path counts and glued digits blow past it. Use `BigInt` (`5n`, and never mix the two types in one expression).
4. **Recursion depth is about 10^4 .** Depth 10^5 throws `RangeError`. Rewrite as an explicit stack.
5. **[...grid] is a shallow copy.** Rows stay shared. Use `grid.map(r => [...r])`. And never `new Array(n).fill(new Array(n))` — every row is the same array.
6. **n / 10 is not integer division.** Wrap it in `Math.floor`.
7. **A Set/Map, not a plain object**, for dictionaries and counts. Object keys stringify and inherit from the prototype.
8. **Strings are immutable.** `board[r][c] = v` on a string row silently does nothing. Convert rows to character arrays first.
9. **Missing undo.** Output too long, or extra elements. Check every push has a pop.
10. **Wrong duplicate guard.** `j > i` (this level), not `j > 0` (the whole array).
11. **Pruning that assumes non-negative values.** The suffix-sum cut breaks on negatives.
12. **Forgetting the second undo** in grid problems: both `cur.pop()` and `vis[r][c] = false`.
13. **Empty input.** Decide whether the answer is `[]` or `[""]`, and guard for it.
14. **Recomputing instead of caching.** If two branches meet at the same state, memoise.

Trees & Binary Trees

Solve the left side, solve the right side, combine. That is the whole chapter.

Pattern in one page

A **binary tree** is made of nodes. Each node holds a value and two references, `left` and `right`, either of which may be `null`. One node is the **root**. A node with no children is a **leaf**.

the node, and nothing more

JS

```
class Node {
  constructor(val) { this.val = val; this.left = null; this.right = null; }
}
```

This chapter needs one thing JavaScript does not ship: a queue you can take from the front cheaply. Use the tested Deque from the **JS Toolkit** appendix.

the one import this chapter needs

JS

```
const { Deque } = require('./toolkit.js'); // see the JS Toolkit appendix
```

TRAP

`array.shift()` is $O(n)$, not $O(1)$. It moves every remaining element down one slot. A breadth-first search that calls `q.shift()` once per node is therefore $O(n^2)$, and on 10^5 nodes it will time out even though your algorithm is correct.

I measured it in Node on a queue of 200000 numbers: draining it with `array.shift()` took **2726 ms**; the toolkit Deque, which keeps a head index instead of moving anything, took **10 ms**. Same answer, 270 times faster.

Every BFS in this chapter uses Deque. If you are writing on a whiteboard, `q.shift()` is fine — but say out loud that you would swap it for an index-based queue in real code.

THE ONE SENTENCE THAT SOLVES 90% OF TREE PROBLEMS

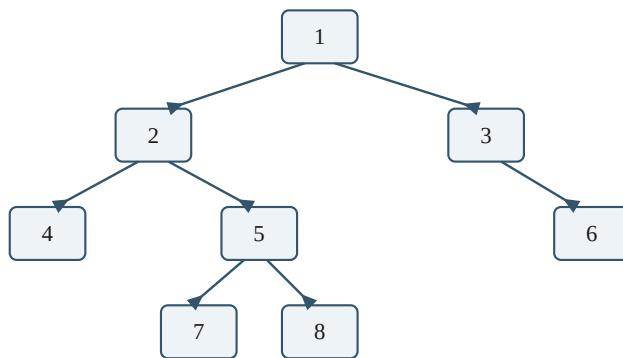
Ask the left child for its answer. Ask the right child for its answer. Combine the two with this node's value.

```
const solve = (r) => {
  if (r === null) return emptyAnswer; // the base case is ALWAYS null
  const L = solve(r.left);
  const R = solve(r.right);
  return combine(L, R, r.val);
};
```

Everything in this chapter is that shape, plus one of two twists:

- the answer you **return** upward is different from the answer you **record**, or
- you need level-by-level order, which means a queue, not recursion.

The tree used all through this chapter



call this tree *T*. 8 nodes, height 4, 4 leaves.

Node 3 has **no left child**. That single missing pointer is what makes the views, the boundary and the top view interesting later — it is not an accident.

The three depth-first traversals

They differ in **one line**: where you print.

preorder, inorder, postorder

JS

```

const preorder = (r, out = []) => {
  if (r === null) return out;
  out.push(r.val);           // root -> left -> right
  preorder(r.left, out);
  preorder(r.right, out);
  return out;
};

const inorder = (r, out = []) => {
  if (r === null) return out;
  inorder(r.left, out);
  out.push(r.val);           // left -> root -> right
  inorder(r.right, out);
  return out;
};

const postorder = (r, out = []) => {
  if (r === null) return out;
  postorder(r.left, out);
  postorder(r.right, out);
  out.push(r.val);           // left -> right -> root
  return out;
};
  
```

Ran all three on tree *T*:

order	output	use it for
preorder	1 2 4 5 7 8 3 6	copying a tree, serialising, “top-down” work
inorder	4 2 7 5 8 1 3 6	BSTs (Chapter 13) — it comes out sorted there
postorder	4 7 8 5 2 6 3 1	deleting a tree, “bottom-up” work like height or diameter

SHORTCUT

How to read the names. “Pre / in / post” says where the **root** goes: before the children, between them, or after them. **left** always comes before **right** in all three.

Breadth-first: the level-order template

Recursion cannot easily do “all of level 2, then all of level 3”. A queue can. Memorise this exact loop — you will type it in eight problems in this chapter.

level order — the template

JS

```
const levelOrder = (root) => {
  const out = [];
  if (root === null) return out;
  const q = new Deque(); q.push(root);
  while (q.size) {
    const sz = q.size; // freeze the size: this is one level
    const level = [];
    for (let i = 0; i < sz; i++) {
      const cur = q.shift();
      level.push(cur.val);
      if (cur.left) q.push(cur.left);
      if (cur.right) q.push(cur.right);
    }
    out.push(level);
  }
  return out;
};
```

Ran it on T: `[[1],[2,3],[4,5,6],[7,8]]`. On null: `[]`.

TRAP

`const sz = q.size;` must be read **before** the inner loop starts. If you write `for (let i = 0; i < q.size; i++)`, the size grows as you push children, and the level boundary disappears. This is the single most common level-order bug, and it is worse in JavaScript than in C++ because `q.size` on the toolkit Deque is a **getter** — it looks like a plain field, so nobody notices it is being re-read every turn.

The two shapes of a tree recursion

Shape	What it looks like	Examples
Pure return	the value you return is the answer for this subtree	height, count, sum, same-tree, path sum
Return one thing, record another	return the best straight arm upward, record the best bent path in a variable the inner function closes over	diameter, max path sum, longest consecutive

That second shape is the hardest idea in the chapter, and it appears in every Tier-3 problem. It looks like this:

the return-one-record-another skeleton

JS

```
const solveTwoWays = (root) => {
  let best = 0; // lives in the OUTER function
  const go = (r) => {
```

```

    if (r === null) return 0;
    const L = go(r.left);
    const R = go(r.right);
    best = Math.max(best, L + R + something); // RECORD: the path that bends at r
    return 1 + Math.max(L, R);              // RETURN: the arm the parent can use
  };
  go(root);
  return best;
};

```

The parent cannot use a bent path — a path through the parent must enter `r` from above and leave through **one** side. So the bent value goes into `best`, and only the straight arm is returned.

JavaScript has no reference parameters. C++ writes `int &best` and Java passes an `int[1]`. In JavaScript you put the inner function **inside** the outer one, and `best` is simply a variable the inner function can see and assign. This is called a **closure**, and it is the cleanest of the three. Python does the same job with `nonlocal best`.

The trap is writing `const go = (r, best) => ...` instead. A number passed as an argument is a **copy**, so `best = ...` inside `go` changes nothing outside it, and the function silently returns the starting value.

Complexity rule of thumb for this chapter. Any traversal that touches each node a constant number of times is $O(n)$ time. The space is the **stack depth**, which is the height h : $O(h)$. For a balanced tree $h = O(\log n)$; for a chain of nodes $h = n$. A queue-based level order is $O(n)$ time and $O(w)$ space, where w is the widest level.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Return the **height** of a tree — the number of nodes on the longest root-to-leaf path. Constraints: up to 10^5 nodes. Edge cases: empty tree; one node; a chain.

SOLUTION

Height of an empty tree is 0. Otherwise it is 1 plus the taller child.

```

height JS

const height = (r) => {
  if (r === null) return 0;
  return 1 + Math.max(height(r.left), height(r.right));
};

```

Ran it: `height(T)=4`, `height(null)=0`, `height(single node)=1`.

Time $O(n)$ **Space** $O(h)$
 h is the height; a chain gives $O(n)$ stack

Answer: 4

TRAP

Some books define height in **edges**, so a single node has height 0 and an empty tree has height -1 . Both are correct — they are different definitions. In an interview, say which one you are using **before** you write code. Every formula in this chapter counts **nodes**, except the diameter, which counts **edges**. I will say so each time.

Example 2 [warm-up]

Count all nodes, count only the leaves, and add up every value. Constraints: values up to 10^9 , n up to 10^5 . Edge cases: empty tree; a total large enough to worry about.

SOLUTION

Three functions, all the same shape.

count, count leaves, sum

JS

```
const countNodes = (r) => {
  if (r === null) return 0;
  return 1 + countNodes(r.left) + countNodes(r.right);
};

const countLeaves = (r) => {
  if (r === null) return 0;
  if (r.left === null && r.right === null) return 1; // a leaf
  return countLeaves(r.left) + countLeaves(r.right);
};

const sumTree = (r) => {
  if (r === null) return 0;
  return r.val + sumTree(r.left) + sumTree(r.right);
};
```

Ran it on T: count=8, leaves=4, sum=36. On null: 0, 0, 0. On a single node 9: 1, 1, 9.

Time $O(n)$ **Space** $O(h)$

Answer: 8 nodes, 4 leaves, sum 36

TRAP

countLeaves needs **two** base cases: null returns 0, and a leaf returns 1. If you only write the null case and return 1 there, a node with one child is counted as a leaf — wrong. Test with a tree that has a one-child node. Tree T has exactly one such node (node 3), which is why I use it everywhere.

There is no 64-bit integer type in JavaScript, and here you do not need one. Every number is a double, exact up to `Number.MAX_SAFE_INTEGER = 9007199254740991`. The worst case in this problem is 10^5 nodes holding 10^9 each, which is 10^{14} — I checked in Node that 10^{14} is still a safe integer, so a plain `+` is exact.

Past 9×10^{15} it is not. If the constraints ever allow that, switch to `BigInt`: `BigInt(r.val) + sumTree(r.left)`, and remember that `BigInt` will not mix with plain numbers in the same expression.

Example 3 [TCS NQT pattern]

Does the tree contain a given value? Constraints: n up to 10^5 . Edge cases: empty tree; the value at the root; missing.

SOLUTION

search

JS

```
const contains = (r, x) => {
  if (r === null) return false;
  if (r.val === x) return true;
  return contains(r.left, x) || contains(r.right, x);
};
```

Ran it on T: contains(7)=true, contains(99)=false, contains(null, 1)=false.

Time $O(n)$ **Space** $O(h)$

in a plain binary tree you must look everywhere; a BST does it in $O(h)$ — Chapter 13

Answer: true, false

SHORTCUT

The `||` short-circuits: if the left subtree finds it, the right subtree is never searched. Writing `const L = contains(...); const R = contains(...); return L || R;` throws that away and always searches the whole tree. Keep the `||` on one line.

Example 4 [warm-up]

Return the **minimum** depth — nodes on the shortest path from the root down to a **leaf**. Constraints: n up to 10^5 . Edge cases: empty tree; a chain (min depth equals height); a root with only one child.

SOLUTION

This looks like height with min instead of max. It is not, and that is the point.

minimum depth

JS

```
const minDepth = (r) => {
  if (r === null) return 0;
  if (r.left === null) return 1 + minDepth(r.right); // must go right
  if (r.right === null) return 1 + minDepth(r.left);  // must go left
  return 1 + Math.min(minDepth(r.left), minDepth(r.right));
};
```

Ran it: minDepth(T)=3 (the path 1 → 2 → 4, and also 1 → 3 → 6). On the chain 1 → 2 → 3 → 4: minDepth = 4, same as the height.

Time $O(n)$ **Space** $O(h)$

Answer: 3

TRAP

The naive `1 + Math.min(minDepth(r.left), minDepth(r.right))` is **wrong**. On the chain 1 → 2 → 3 → 4, node 1 has a null left child which returns 0, so the naive version reports depth 1 — but there is no leaf at depth 1. A path must **end at a leaf**. That is why the two null checks exist.

Example 5 [warm-up]

Find the largest value in the tree, and the sum of only the leaf values. Constraints: n up to 10^5 , values from -10^9 to 10^9 . Target: $O(n)$. Edge cases: all values negative; a single node; an empty tree.

SOLUTION

max value and leaf sum

JS

```
const maxVal = (r) => {
  if (r === null) return -Infinity; // neutral element for max
  return Math.max(r.val, maxVal(r.left), maxVal(r.right));
};

const leafSum = (r) => {
  if (r === null) return 0;
  if (r.left === null && r.right === null) return r.val;
  return leafSum(r.left) + leafSum(r.right);
};
```

Ran it on T: maxVal=8, leafSum=25 (leaves are 4, 7, 8, 6). On the all-negative tree {-3, -2, -9}: maxVal = -2. On a single node 9: leafSum = 9.

Time $O(n)$ **Space** $O(h)$

Answer: 8 and 25

TRAP

-Infinity is the right neutral value for max, and 0 would be a bug: on an all-negative tree, 0 would win and the answer would be 0, a value that is not in the tree. Pick your neutral element to match the operation: 0 for a sum, -Infinity for a maximum, Infinity for a minimum. JavaScript has no INT_MIN, and it does not need one — -Infinity is a real number that loses every comparison.

Math.max also takes any number of arguments, so Math.max(r.val, maxVal(r.left), maxVal(r.right)) needs no nesting. And Math.max() with **no** arguments returns -Infinity, which is why spreading an empty array into it, Math.max(...[]), quietly gives -Infinity instead of throwing.

Tier 1 — walk the tree, level by level

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 6 [Infosys pattern]

Print the tree level by level, one list per level. Constraints: n up to 10^5 . Edge cases: empty tree; a chain (every level has 1 node).

SOLUTION

Approach 1 — DFS carrying the depth ($O(n)$ — correct, and shorter than you expect)

You do not strictly **need** a queue. Walk the tree any way you like, but carry the depth, and append each value into out[depth]. The first time you reach a new depth, create the row.

level order by DFS

JS

```
const loGo = (r, d, out) => {
  if (r === null) return;
  if (d === out.length) out.push([]); // first node at this depth
  out[d].push(r.val);
  loGo(r.left, d + 1, out);
  loGo(r.right, d + 1, out);
};
```

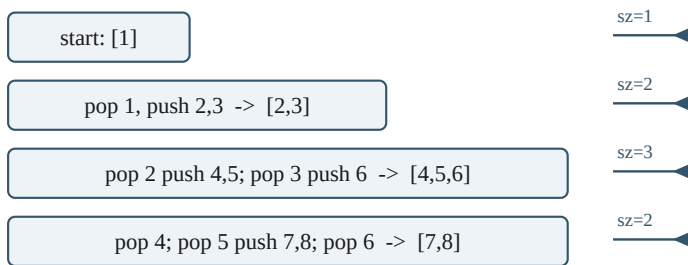
```
const levelOrderDfs = (r) => { const out = []; loGo(r, 0, out); return out; };
```

Ran it on T: $[[1], [2, 3], [4, 5, 6], [7, 8]]$ — the same answer. On null: $[]$.

Approach 2 — BFS with a queue ($O(n)$ — same cost, but this one extends to every level question)

The DFS version cannot easily answer “the last node on each level” or “reverse this level”, because it never has a whole level in hand at once. The BFS version does, and that is why it is the template.

Ran it on T: $[[1], [2, 3], [4, 5, 6], [7, 8]]$. On null: $[]$.



the queue during level order on T

Time $O(n)$ **Space** $O(w)$

w is the widest level; for a perfect tree w is about $n/2$

Answer: $[[1], [2, 3], [4, 5, 6], [7, 8]]$

Example 7 [Wipro pattern]

Do the inorder traversal **without recursion**. Constraints: n up to 10^5 — deep enough that recursion may blow the stack. Edge cases: empty tree; a left chain; a right chain.

SOLUTION

Approach 1 — Plain recursion ($O(n)$ time, $O(h)$ stack — and in Node that stack runs out at roughly 10^4 frames)

The recursive version is on the pattern page. On a chain of 10^5 nodes it **will** crash.

TRAP

JavaScript’s call stack is about 10^4 frames deep — far shallower than C++ or Java. I measured it in Node: a plain recursive function died at depth **12546** with RangeError: Maximum call stack size exceeded.

A balanced tree of 10^5 nodes is only 17 deep, so recursion is safe there. But nothing in the problem statement promises balance, and a tree that is one long chain of 10^5 nodes is a legal input. **Whenever the constraint is 10^5 nodes and the shape is not guaranteed, write the iterative version with your own stack array.** That is exactly what this example is for.

Approach 2 — Your own stack ($O(n)$ time, $O(h)$ heap space — no crash, and you can pause mid-traversal)

An array **is** the stack: push to add, pop to remove. Push every node you pass on the way left. When you cannot go left any more, pop, visit, and turn right.

iterative inorder

JS

```
const inorderIter = (root) => {
  const out = [];
```

```

    const st = []; // a plain array IS the
stack
    let cur = root;
    while (cur !== null || st.length) {
        while (cur !== null) { st.push(cur); cur = cur.left; } // go all the way left
        cur = st.pop();
        out.push(cur.val); // visit
        cur = cur.right; // then turn right
    }
    return out;
};

```

Ran it on T: [4,2,7,5,8,1,3,6] — identical to the recursive version. Good.

Time $O(n)$ **Space** $O(h)$
the stack replaces the call stack, same size

Answer: 4 2 7 5 8 1 3 6

Example 8 [Capgemini pattern]

Do preorder and postorder without recursion too. Constraints: n up to 10^5 ; the recursion depth could be 10^5 , so a stack is safer. Target: $O(n)$ time. Edge cases: empty tree; single node; a left-only chain.

SOLUTION

Preorder is easy with a stack: pop, print, push **right then left** so that left pops first.

iterative preorder

JS

```

const preorderIter = (root) => {
    const out = [];
    if (root === null) return out;
    const st = [root];
    while (st.length) {
        const cur = st.pop();
        out.push(cur.val);
        if (cur.right) st.push(cur.right); // right first: it pops last
        if (cur.left) st.push(cur.left);
    }
    return out;
};

```

Postorder needs a trick, and there are two ways to do it.

Approach 1 — Two stacks ($O(n)$ time, $O(n)$ space — easy to remember)

Push into stack a, pop from a into stack b, pushing children left-then-right. Stack b then pops in postorder.

postorder, two stacks

JS

```

const postorderTwoStacks = (root) => {
    const out = [];
    if (root === null) return out;
    const a = [root], b = [];
    while (a.length) {
        const c = a.pop();
        b.push(c);
    }
    while (b.length) {
        out.push(b.pop());
    }
    return out;
};

```

```

    if (c.left) a.push(c.left);
    if (c.right) a.push(c.right);
  }
  while (b.length) out.push(b.pop().val);
  return out;
};

```

Ran it on T: [4,7,8,5,2,6,3,1].

Approach 2 — One stack, then reverse ($O(n)$ time, $O(n)$ space — the same idea with the second stack replaced by a reverse)

Run the **same** loop as preorder but push left first, giving root → right → left. Then **reverse** the list. Reversed, that is left → right → root — exactly postorder.

iterative postorder, one stack

JS

```

const postorderIter = (root) => {
  const out = [];
  if (root === null) return out;
  const st = [root];
  while (st.length) {
    const c = st.pop();
    out.push(c.val);           // builds root-right-left
    if (c.left) st.push(c.left);
    if (c.right) st.push(c.right);
  }
  out.reverse();              // reverse -> left-right-root
  return out;
};

```

Ran both on T: preorder [1,2,4,5,7,8,3,6], postorder [4,7,8,5,2,6,3,1]. I compared each against the recursive version with `JSON.stringify` — identical.

TRAP

`out.reverse()` reverses the array **in place** and returns the same array. It does not give you a copy. If you still need the original order afterwards, reverse a copy: `[...out].reverse()`. The same is true of `sort`, `splice` and `fill` — the JavaScript array methods that end in a verb usually mutate.

Time $O(n)$ **Space** $O(n)$

postorder needs the whole output list before reversing

Answer: preorder 1 2 4 5 7 8 3 6, postorder 4 7 8 5 2 6 3 1

Example 9 [TCS Digital pattern]

Turn the tree into its mirror image: swap every node's two children. Constraints: n up to 10^5 . Target: $O(n)$ time, $O(h)$ space. Edge cases: empty tree; a single node; a chain.

SOLUTION

Mirror the left subtree, mirror the right subtree, then swap the pointers.

invert a tree

JS

```

const invert = (r) => {
  if (r === null) return null;
  const l = invert(r.left);

```



```
const rt = invert(r.right);
r.left = rt; r.right = l;
return r;
};
```

Ran it on a copy of T:

	before	after invert
preorder	1 2 4 5 7 8 3 6	1 3 6 2 5 8 7 4
inorder	4 2 7 5 8 1 3 6	6 3 1 8 5 7 2 4

The inorder of the mirror is the **reverse** of the original inorder. That is a useful self-check: if it is not reversed, your swap is wrong.

Time $O(n)$ **Space** $O(h)$

TRAP

`r.left = invert(r.right); r.right = invert(r.left);` is **broken**. By the time the second line runs, `r.left` has already been overwritten, so you invert the new left subtree again. Compute both children into local variables **first**, then assign.

JavaScript gives you a second way out: the destructuring swap `[r.left, r.right] = [invert(r.right), invert(r.left)]`; The right-hand side is fully evaluated into a temporary array before anything is assigned, so it is correct. Use whichever you find clearer, but know why each one works.

Example 10 [Accenture pattern]

Two problems that look the same and are not: (a) are two trees **identical**? (b) is one tree **symmetric** about its own centre? Constraints: up to 10^5 nodes each. Target: $O(n)$. Edge cases: both empty; one empty; same values but different shape.

SOLUTION

Identical: compare left with left and right with right.

same tree

JS

```
const same = (a, b) => {
  if (a === null && b === null) return true;
  if (a === null || b === null) return false; // one ran out first
  return a.val === b.val && same(a.left, b.left) && same(a.right, b.right);
};
```

Symmetric: compare left with **right** and right with **left**.

symmetric tree

JS

```
const mirror = (a, b) => {
  if (a === null && b === null) return true;
  if (a === null || b === null) return false;
  return a.val === b.val && mirror(a.left, b.right) && mirror(a.right, b.left);
};

const symmetric = (r) => r === null || mirror(r.left, r.right);
```

Ran them:

test	result	why
same(T, a fresh copy of T)	true	same values, same shape
same(T, tree {1,2,3})	false	shape differs
same(null, null)	true	both empty
symmetric(T)	false	node 3 has no left child, node 2 has both
symmetric({1,2,2,3,4,4,3})	true	a perfect mirror
symmetric({1,2,2,·,3,·,3})	false	values mirror, shape does not
symmetric(null)	true	nothing to break

Time $O(n)$ **Space** $O(h)$

The idea: the **only** difference between the two functions is `b.right` versus `b.left` in the two recursive calls. Say that out loud in an interview and you have answered both questions at once.

TRAP

Use `===`, not `==`. `null == undefined` is `true` in JavaScript, so a tree built with undefined children instead of `null` would still pass a `== null` test but fail every `=== null` test. Pick `null` for “no child”, build every node the same way, and compare with `===`.

The third test in the table is the one that catches people (a `·` in the level-order list means “no node”, so both 2s have a **right** child holding 3 and no left child). Both children of the root hold 2; both 2s have exactly one child holding 3. But the left 2’s child is on its **right**, and the right 2’s child is on its **right** too — so they are not mirrored. A symmetric check must compare **shape**, not just the multiset of values.

Example 11 [Cognizant pattern]

Is there a root-to-leaf path whose values add up to a given target? Constraints: values may be negative; n up to 10^5 . Edge cases: empty tree; the target hits a non-leaf node.

SOLUTION

Approach 1 — Collect every root-to-leaf path, then add each one up ($O(n \cdot h)$ time and $O(n \cdot h)$ space — correct but wasteful)

the wasteful version

JS

```
const hasPathSumSlow = (r, target) => {
  const all = [];
  pathsGo(r, [], all);           // the path collector from Example 12
  for (const p of all) {
    const s = p.reduce((acc, x) => acc + x, 0);
    if (s === target) return true;
  }
  return false;
};
```

I ran this against the fast version below for every target from -5 to 30 on tree `T`. They agree on all 36 targets. So it is correct — it just builds and stores every path first, and it cannot stop early.

Approach 2 — Subtract as you go ($O(n)$ time, $O(h)$ space, and it stops the moment it finds a path — optimal)

Subtract as you descend. At a **leaf**, the remaining target must equal the leaf’s value.

root-to-leaf path sum

JS

```
const hasPathSum = (r, target) => {
  if (r === null) return false;
  if (r.left === null && r.right === null) return target === r.val;
  return hasPathSum(r.left, target - r.val) || hasPathSum(r.right, target - r.val);
};
```

Ran it on T:

target	answer	path
7	true	1 + 2 + 4
15	true	1 + 2 + 5 + 7
3	false	1 + 2 reaches 3 but node 2 is not a leaf
0 on an empty tree	false	there is no path at all

Time $O(n)$ **Space** $O(h)$

Answer: true, true, false

TRAP

Target 3 on tree T is the trap. $1 + 2 = 3$, but node 2 has children, so it is not a leaf and the path is not allowed to stop there. The leaf test must come **before** you use the target. And an empty tree must return false even for target 0 — there is no path, so no path sums to anything.

`p.reduce((acc, x) => acc + x, 0)` is how you add up an array in JavaScript. The 0 at the end is the starting value and it is **not** optional here: `[]`.reduce(`(a, b) => a + b`) with no starting value throws `TypeError: Reduce of empty array with no initial value`.

Example 12 [Infosys pattern]

Print every root-to-leaf path. Then, treating each path as a decimal number (root digit first), add all the numbers up. Constraints: values are single digits 0..9 for part two. Edge cases: single node; a chain.

SOLUTION

Part one is plain backtracking on a tree — push, recurse, pop (Chapter 11's rule).

all root-to-leaf paths

JS

```
const pathsGo = (r, cur, out) => {
  if (r === null) return;
  cur.push(r.val);
  if (r.left === null && r.right === null) out.push([...cur]); // COPY, not the
  // live array
  else { pathsGo(r.left, cur, out); pathsGo(r.right, cur, out); }
  cur.pop(); // un-choose
};

const allPaths = (r) => { const out = []; pathsGo(r, [], out); return out; };
```

TRAP

`out.push(cur)` without the spread is a silent disaster. C++ copies the vector on `push_back`; JavaScript pushes a **reference** to the one live array, and `cur.pop()` on the way back out empties it. You end up with a list of identical empty arrays.

`out.push([...cur])` makes a real copy. Remember that `[...cur]` is only one level deep — if the elements were themselves arrays you would need `cur.map(x => [...x])`.

what the spread actually fixes

JS

```
const out = [];  
const live = [1, 2, 3];  
out.push(live);           // a reference  
out.push([...live]);      // a copy  
live.pop();  
// out is now [[1, 2], [1, 2, 3]] — the first row changed under you
```

Part two has a ladder of its own.

Approach 1 — Collect the paths, then turn each into a number ($O(n \cdot h)$ time and space)

```
const sumNumbersSlow = (r) => {  
  let total = 0;  
  for (const p of allPaths(r)) total += p.reduce((v, d) => v * 10 + d, 0);  
  return total;  
};
```

Ran it on T: 2775. On {1,2,3}: 25.

Approach 2 — Build the number on the way down ($O(n)$ time, $O(h)$ space — optimal)

Part two does not need the list at all — carry the number being built.

sum of root-to-leaf numbers

JS

```
const numsGo = (r, cur) => {  
  if (r === null) return 0;  
  cur = cur * 10 + r.val;  
  if (r.left === null && r.right === null) return cur;  
  return numsGo(r.left, cur) + numsGo(r.right, cur);  
};  
  
const sumNumbers = (r) => numsGo(r, 0);
```

Ran it on T. Paths: [1,2,4] [1,2,5,7] [1,2,5,8] [1,3,6]. Numbers: $124 + 1257 + 1258 + 136 = 2775$. The program printed 2775. On {1,2,3}: $12 + 13 = 25$.

Time $O(n \times h)$ **Space** $O(h)$

the $n \cdot h$ is copying each path; part two is pure $O(n)$

Answer: 2775

A path of 17 digits breaks this. `cur * 10 + r.val` is exact only while `cur` stays under `Number.MAX_SAFE_INTEGER = 9007199254740991`. The stated constraint is single digits and a tree of 10^4 nodes, so a root-to-leaf path could in principle be far longer than 16 digits. If the statement allows that, build the number as a **string** and compare, or switch to `BigInt`: `cur * 10n + BigInt(r.val)`. Say this out loud — noticing it is worth a mark.

Example 13 [TCS NQT pattern]

For each level, report the maximum value, the sum, and the average. Constraints: n up to 10^5 , values from -10^9 to 10^9 (so a level sum can reach 10^{14}). Target: $O(n)$. Edge cases: empty tree; negative values; a one-node level.

SOLUTION

Approach 1 — DFS carrying the depth ($O(n)$ time, $O(h)$ space — best when the tree is wide)

```
const mplGo = (r, d, out) => {
  if (r === null) return;
  if (d === out.length) out.push(-Infinity); // new depth, neutral element
  out[d] = Math.max(out[d], r.val);
  mplGo(r.left, d + 1, out);
  mplGo(r.right, d + 1, out);
};
```

Ran it on T: [1,3,6,8].

Approach 2 — BFS level by level ($O(n)$ time, $O(w)$ space — best when the tree is deep, and the only option for averages)

The average needs the **count** of nodes on the level, which the BFS gives you for free as `sz`. Three variations on the level-order template; only the line inside the inner loop changes.

max value on each level

JS

```
const maxPerLevel = (root) => {
  const out = [];
  if (root === null) return out;
  const q = new Deque(); q.push(root);
  while (q.size) {
    const sz = q.size;
    let best = -Infinity; // neutral element
    for (let i = 0; i < sz; i++) {
      const c = q.shift();
      best = Math.max(best, c.val);
      if (c.left) q.push(c.left);
      if (c.right) q.push(c.right);
    }
    out.push(best);
  }
  return out;
};
```

For the sum, replace `best` with `let s = 0` and `s += c.val`. For the average, divide that same `s` by `sz` at the end of the level.

average on each level — the only two changed lines

JS

```
let s = 0;
for (let i = 0; i < sz; i++) { const c = q.shift(); s += c.val; /* push
children */ }
out.push(s / sz); // plain / — JS has no integer
division
```

Ran all three on T:

level	nodes	max	sum	average
1	1	1	1	1
2	2 3	3	5	2.5

3	4 5 6	6	15	5
4	7 8	8	15	7.5

Time $O(n)$ **Space** $O(w)$

Answer: max [1,3,6,8], sum [1,5,15,15], average [1, 2.5, 5, 7.5]

TRAP

This is the one place where JavaScript is **easier** than C++. In C++, `sum / sz` with two ints silently truncates and you must write `(double)sum / sz`. In JavaScript `/` is always real division — `5 / 2` is 2.5, which I checked in Node — so the average comes out right by default.

The mirror-image trap is the one that bites JavaScript programmers: when you **want** the whole part, `/` will not give it to you. `Math.floor(a / b)` is the fix, and it is needed in binary search, in carry arithmetic, and in heap index maths.

There is no overflow worry here: a level of 10^5 nodes holding 10^9 each sums to 10^{14} , and I confirmed in Node that 10^{14} is still a safe integer.

Tier 2 — views, shapes and one-pass checks

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

A **view** is what you see when you stand somewhere and look at the tree. Four views come up again and again, and each is a one-line change to a traversal you already know.

Example 14 [Grab · pattern]

Right view: standing on the right of the tree, list the nodes you can see, top to bottom. Constraints: n up to 10^5 . Target: $O(n)$. Edge cases: empty tree; a left-only chain (you still see every node); a single node.

SOLUTION

Approach 1 — Level order, keep the last node of each level $(O(n)$ time, $O(w)$ space)

right view, BFS

JS

```
const rightView = (root) => {
  const out = [];
  if (root === null) return out;
  const q = new Deque(); q.push(root);
  while (q.size) {
    const sz = q.size;
    for (let i = 0; i < sz; i++) {
      const c = q.shift();
      if (i === sz - 1) out.push(c.val); // last on this level
      if (c.left) q.push(c.left);
      if (c.right) q.push(c.right);
    }
  }
  return out;
};
```

Approach 2 — DFS, right child first, record the first node seen at each depth ($O(n)$ time, $O(h)$ space — better when the tree is wide)

If you always go right before left, the **first** node you reach at depth d is the rightmost node at that depth. “First time I reach depth d ” is exactly `depth === out.length`.

right view, DFS

JS

```
const rvGo = (r, depth, out) => {
  if (r === null) return;
  if (depth === out.length) out.push(r.val); // first node at this depth
  rvGo(r.right, depth + 1, out);           // RIGHT first
  rvGo(r.left, depth + 1, out);
};

const rightViewDfs = (r) => { const out = []; rvGo(r, 0, out); return out; };
```

Left view is the same function with the two calls swapped.

left view

JS

```
const lvGo = (r, depth, out) => {
  if (r === null) return;
  if (depth === out.length) out.push(r.val);
  lvGo(r.left, depth + 1, out);           // LEFT first
  lvGo(r.right, depth + 1, out);
};

const leftView = (r) => { const out = []; lvGo(r, 0, out); return out; };
```

Ran all three on T: right view [1,3,6,8] (both approaches agree), left view [1,2,4,7].

Time $O(n)$ **Space** $O(h)$
the BFS version uses $O(w)$ instead

The idea: `depth === out.length` is a “have I been this deep before?” test that costs nothing. Remember it; it replaces a `visitedDepth` array in many problems.

Answer: right [1,3,6,8], left [1,2,4,7]

Example 15 [Sea/Shopee · pattern]

Top view and **bottom view:** give every node a column number — the root is column 0, a left child is one less, a right child is one more. Looking straight down, list the **first** node in each column (top view), left column to right column. Bottom view lists the **last** one. Constraints: n up to 10^5 ; columns range from $-n$ to n . Target: $O(n \log n)$. Edge cases: empty tree; two nodes landing in the same column; columns that cross under other nodes.

SOLUTION

Approach 1 — BFS carrying the column, stored in a Map ($O(n \log n)$ — the standard answer, and the one to write first)

Do a BFS carrying the column, and remember one value per column.

TRAP

JavaScript has no **TreeMap**, no **ordered map**, no **ordered dictionary of any kind**. C++ gets its columns back in sorted order for free; JavaScript does not. A Map keeps **insertion** order, which for a BFS is the order the columns were first reached — 0, -1, 1, -2, ..., not left to right.

The workaround is one line: take the keys out and sort them yourself, `[...firstAt.keys()].sort((a, b) => a - b)`. That is the $\log n$ in the complexity.

TRAP

That sort needs the comparator, and columns are exactly where forgetting it shows. `arr.sort()` compares **as text**. Tested in Node: `[-2, -1, 0, 1, 2].sort()` returns `[-1, -2, 0, 1, 2]` — the two negative columns come out in the wrong order, because the string `"-1"` is less than `"-2"`. With the comparator, `[-2, -1, 0, 1, 2].sort((a, b) => a - b)` returns `[-2, -1, 0, 1, 2]`.

This is the single most common JavaScript bug in coding rounds. Any time you sort numbers, write the comparator.

top view

JS

```
const topView = (root) => {
  if (root === null) return [];
  const firstAt = new Map(); // column -> value
  const q = new Deque(); q.push([root, 0]);
  while (q.size) {
    const [c, col] = q.shift(); // destructure the pair
    if (!firstAt.has(col)) firstAt.set(col, c.val); // BFS: first seen is topmost
    if (c.left) q.push([c.left, col - 1]);
    if (c.right) q.push([c.right, col + 1]);
  }
  const cols = [...firstAt.keys()].sort((a, b) => a - b); // NUMERIC sort
  return cols.map(k => firstAt.get(k));
};
```

Bottom view is the same code with one line changed: `lastAt.set(col, c.val)`; with no `has` guard — keep overwriting, so the deepest node in that column wins.

Use a `Map`, not a plain object, for the columns. An object turns the key `-1` into the string `"-1"`, so you would have to convert it back with `Number(k)` before sorting — and if you forget, the sort is lexicographic again and column `-10` lands next to column `-1`. A `Map` keeps the number a number.

Approach 2 — A plain array plus an offset $O(n)$ — drops the $\log n$

A column can never be further than n steps from the root, so an array of size $2n + 1$ indexed by `col + n` replaces the `Map` **and** the sort. Track the smallest and largest index used so you know where to start and stop reading.

top view in $O(n)$

JS

```
const topViewFast = (root, n) => { // n = number of nodes
  const out = [];
  if (root === null) return out;
  const val = new Array(2 * n + 1).fill(null); // column c lives at index c + n
  let lo = 2 * n, hi = 0;
  const q = new Deque(); q.push([root, 0]);
  while (q.size) {
    const [c, col] = q.shift();
    const idx = col + n;
    if (val[idx] === null) { val[idx] = c.val; lo = Math.min(lo, idx); hi = Math.max(hi, idx); }
    if (c.left) q.push([c.left, col - 1]);
    if (c.right) q.push([c.right, col + 1]);
  }
  return val.slice(lo, hi + 1);
};
```



```

}
for (let i = lo; i <= hi; i++) if (val[i] !== null) out.push(val[i]);
return out;
};

```

TRAP

new Array(k) alone gives you *k* **holes**, not *k* values — map and forEach skip them and val[idx] === null is false for every slot. Always finish it: new Array(k).fill(null) or Array.from({ length: k }, () => 0). Here null is the “empty column” marker, which also frees you from needing an INT_MIN sentinel at all.

Ran it on T: [4,2,1,3,6] — identical to the map version.

Columns of tree T:

column	-2	-1	0	+1
nodes, top to bottom	4	2, 7	1, 5	3, 8

plus column +2 holding 6.

Ran it: top view [4,2,1,3,6], bottom view [4,7,5,8,6].

Time $O(n \log n)$ **Space** $O(n)$

the log n is sorting the Map keys; use an array plus an offset for $O(n)$

Answer: top [4,2,1,3,6], bottom [4,7,5,8,6]

TRAP

You must use BFS, not DFS, for the top view. With DFS you might walk deep down the left side and record a node at column 0 that sits at depth 5, before ever visiting the node at column 0 and depth 2. BFS visits strictly by depth, so “first seen in a column” really does mean “highest in that column”.

Second trap: node 7 is at column -1 and node 8 is at column +1. They sit **under** nodes 2 and 3, crossing sideways. If you assume columns never cross, your code is wrong on tree T.

Example 16 [Agoda · pattern]

Zigzag level order: level 1 left to right, level 2 right to left, level 3 left to right, and so on. Constraints: *n* up to 10^5 . Target: $O(n)$. Edge cases: one level; empty tree; a level holding a single node.

SOLUTION

Do not reverse the queue. Instead, pre-size the level array and **write into the right slot**.

zigzag level order

JS

```

const zigzag = (root) => {
  const out = [];
  if (root === null) return out;
  const q = new Deque(); q.push(root);
  let leftToRight = true;
  while (q.size) {
    const sz = q.size;
    const level = new Array(sz);
    for (let i = 0; i < sz; i++) {
      const c = q.shift();

```

```

    const pos = leftToRight ? i : sz - 1 - i;    // write forwards or backwards
    level[pos] = c.val;
    if (c.left) q.push(c.left);
    if (c.right) q.push(c.right);
  }
  out.push(level);
  leftToRight = !leftToRight;
}
return out;
};

```

Ran it on T: `[[1], [3,2], [4,5,6], [8,7]]`.

Time $O(n)$ **Space** $O(w)$

The idea: the children are **always** pushed left-then-right. Only the **writing** flips. Flipping the push order instead corrupts the next level.

Here new `Array(sz)` with no fill is safe, because every slot is written before the array is read.

Answer: `[[1], [3,2], [4,5,6], [8,7]]`

Example 17 [GIC · pattern]

Vertical order: group every node by its column, columns from left to right, and inside one column list the nodes top to bottom. Constraints: n up to 10^5 . Target: $O(n \log n)$. Edge cases: empty tree; two nodes in the same column at the same depth; a column holding three or more nodes.

SOLUTION

Approach 1 — Collect (column, depth, value) triples, then sort ($O(n \log n)$) — and it handles every tie-break rule you might be asked for)

vertical order by sorting triples

JS

```

const vtGo = (r, col, depth, all) => {
  if (r === null) return;
  all.push([col, depth, r.val]);
  vtGo(r.left, col - 1, depth + 1, all);
  vtGo(r.right, col + 1, depth + 1, all);
};

const verticalSorted = (r) => {
  const all = [];
  vtGo(r, 0, 0, all);
  // column, then depth, then value — || picks the next key when the previous ties
  all.sort((a, b) => a[0] - b[0] || a[1] - b[1] || a[2] - b[2]);
  const out = [];
  for (let i = 0; i < all.length; ) {
    const col = all[i][0];
    const group = [];
    while (i < all.length && all[i][0] === col) { group.push(all[i][2]); i++; }
    out.push(group);
  }
  return out;
};

```

TRAP

C++ sorts an `array<int, 3>` by comparing field after field for you. JavaScript sorts arrays **as text** and would compare `"-1,1,2"` with `"0,0,1"` character by character — nonsense.

The multi-key comparator is `(a, b) => a[0] - b[0] || a[1] - b[1] || a[2] - b[2]`. It works because `0` is falsy: if the first difference is `0` (a tie), `||` moves on to the next key. Memorise this line; it is how every “sort by X, then by Y” is written in JavaScript.

Ran it on T: `[[4],[2,7],[1,5],[3,8],[6]]`, and on tree U below: `[[2],[1,4,5],[3,6]]`.

Approach 2 — BFS with a Map ($O(n \log n)$ — shorter, and the depth ordering comes free)

Same BFS with columns, but collect a **list** per column instead of one value.

vertical order

JS

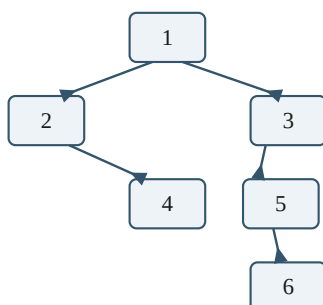
```
const verticalOrder = (root) => {
  const out = [];
  if (root === null) return out;
  const cols = new Map();
  const q = new Deque(); q.push([root, 0]);
  while (q.size) {
    const [c, col] = q.shift();
    if (!cols.has(col)) cols.set(col, []); // no default-construct in JS
    cols.get(col).push(c.val);
    if (c.left) q.push([c.left, col - 1]);
    if (c.right) q.push([c.right, col + 1]);
  }
  for (const k of [...cols.keys()].sort((a, b) => a - b)) out.push(cols.get(k));
  return out;
};
```

TRAP

C++’s `cols[col].push_back(...)` creates an empty vector the first time a column is seen. A JavaScript Map does not: `cols.get(col)` returns `undefined` and `.push` on it throws `TypeError: Cannot read properties of undefined`. The two-line guard `if (!cols.has(col)) cols.set(col, []);` is not optional.

Ran it on T: `[[4],[2,7],[1,5],[3,8],[6]]`.

I also ran it on a second tree U, where a column really fills up:



tree U: nodes 1, 4 and 5 all sit in column 0

Result for U: vertical `[[2],[1,4,5],[3,6]]`, top view `[2,1,3]`, bottom view `[2,5,6]`.

Time $O(n \log n)$ **Space** $O(n)$

Answer: `[[4], [2, 7], [1, 5], [3, 8], [6]]`

Some versions of this question add a tie-break: **inside one column, if two nodes are at the same depth, list the smaller value first**. The BFS above lists them in left-to-right order instead. If the statement asks for the tie-break, store triples `[column, depth, value]`, collect them all, then sort with `(a, b) => a[0] - b[0] || a[1] - b[1] || a[2] - b[2]`. Ask the interviewer which rule they want.

Example 18 [SCB · pattern]

Boundary traversal: print the root, then the left edge going down (not counting leaves), then every leaf left to right, then the right edge coming up (not counting leaves). Each node appears exactly once. Constraints: n up to 10^5 . Target: $O(n)$. Edge cases: single node (the root is itself a leaf); a tree with no right child at all; an empty tree.

SOLUTION

Three helpers, and one rule that prevents double printing: **the edges skip leaves, because the leaf pass will print them**.

boundary traversal

JS

```
const isLeaf = (n) => n.left === null && n.right === null;

const addLeftEdge = (r, out) => {
  let c = r.left;
  while (c !== null) {
    if (!isLeaf(c)) out.push(c.val);
    c = c.left !== null ? c.left : c.right; // prefer left, else right
  }
};

const addLeaves = (r, out) => {
  if (r === null) return;
  if (isLeaf(r)) { out.push(r.val); return; }
  addLeaves(r.left, out);
  addLeaves(r.right, out);
};

const addRightEdge = (r, out) => {
  const tmp = [];
  let c = r.right;
  while (c !== null) {
    if (!isLeaf(c)) tmp.push(c.val);
    c = c.right !== null ? c.right : c.left;
  }
  for (let i = tmp.length - 1; i >= 0; i--) out.push(tmp[i]); // reversed
};

const boundary = (root) => {
  const out = [];
  if (root === null) return out;
  if (!isLeaf(root)) out.push(root.val);
  addLeftEdge(root, out);
  addLeaves(root, out);
  addRightEdge(root, out);
  return out;
};
```

Ran it on T: `[1, 2, 4, 7, 8, 6, 3]`. Read it: root 1; left edge 2 (node 4 is a leaf, skipped); leaves 4 7 8 6; right edge 3, reversed (just one node). On a single node 9: `[9]`. On null: `[]`.

Time $O(n)$ **Space** $O(h)$

The idea: the right edge is collected **downward** and printed **upward**, so it needs a temporary list. Everything else is printed as it is found.

TRAP

Three separate double-print traps, all fixed by one habit — **the edges skip leaves**:

1. The root, if it is a leaf (single-node tree), must not be printed twice.
2. A leaf on the left edge is printed by `addLeaves`, not by `addLeftEdge`.
3. Same for the right edge.

Check your answer's length: it must equal the number of **distinct** boundary nodes.

Example 19 [Razer · pattern]

Is the tree **height-balanced**? That means: for every node, the heights of its two subtrees differ by at most 1. Constraints: n up to 10^5 . Edge cases: empty tree (balanced); a chain (not, for $n \geq 3$); a perfect tree.

SOLUTION

Approach 1 — For each node, call `height()` on both children ($O(n^2)$ — height is recomputed from scratch at every node)

the slow version

JS

```
const balancedSlow = (r) => {
  if (r === null) return true;
  if (Math.abs(height(r.left) - height(r.right)) > 1) return false;
  return balancedSlow(r.left) && balancedSlow(r.right);
};
```

On a chain of 10^5 nodes this does about 10^{10} steps. Far too slow.

Approach 3 — Compute the height and the verdict in the same pass ($O(n)$ — optimal)

Let the height function return a **sentinel** -1 meaning “something below me is already unbalanced”. A real height is never negative, so there is no clash.

balanced in one pass

JS

```
const balHeight = (r) => { // -1 means "not balanced"
  if (r === null) return 0;
  const lh = balHeight(r.left);
  if (lh === -1) return -1; // bail out early
  const rh = balHeight(r.right);
  if (rh === -1) return -1;
  if (Math.abs(lh - rh) > 1) return -1;
  return 1 + Math.max(lh, rh);
};

const balanced = (r) => balHeight(r) !== -1;
```

Ran it:

tree	balanced?
T	true
chain 1 → 2 → 3 → 4	false

perfect tree of 7 nodes	true
empty tree	true

Time $O(n)$ **Space** $O(h)$

The idea: a function that returns **either** a measurement **or** a failure flag. This “sentinel return” trick turns many $O(n^2)$ tree checks into $O(n)$ — you will use it again for “is this a BST” in Chapter 13.

Answer: T is balanced; a chain of 4 is not

Example 20 [DBS · pattern]

Maximum width: the widest level, counting the empty gaps between the leftmost and rightmost real node on that level. Constraints: depth up to 60, so the index can be huge. Edge cases: empty tree; single node; a very deep tree (index overflow).

SOLUTION

Approach 1 — Level order, pushing null for missing children ($O(2^h)$ — it works on small trees and dies on deep ones)

If you push a null placeholder for every missing child, each level is a full row and the width is just “trim the nulls off both ends”. Ran it on T: it gives the right answer, **4**. Then I ran it on a chain of 40 nodes with a budget of ten million pushes: it ran out of room and returned my “gave up” marker. A chain of depth 40 needs 2^{40} placeholders.

Approach 2 — Number the nodes like an array heap ($O(n)$ — optimal)

Give the root index 0. Then a node with index i has children $2i$ and $2i + 1$ — the same numbering as an array heap. The width of a level is `lastIndex - firstIndex + 1`.

maximum width

JS

```
const maxWidth = (root) => {
  if (root === null) return 0;
  let best = 0;
  const q = new Deque(); q.push([root, 0]);
  while (q.size) {
    const sz = q.size;
    const first = q.front()[1];           // leftmost index on this level
    let last = 0;
    for (let i = 0; i < sz; i++) {
      const [c, raw] = q.shift();
      const idx = raw - first;           // re-base: keeps every index
      small
      last = idx;
      if (c.left) q.push([c.left, 2 * idx]);
      if (c.right) q.push([c.right, 2 * idx + 1]);
    }
    best = Math.max(best, last + 1);
  }
  return best;
};
```

Ran it: `maxWidth(T)=4` (level 3 holds 4, 5, 6 at indices 0, 1, 3 — so the width counts the gap). `maxWidth(null)=0`, single node 1, and the tree `{1,2,3,4,-,-,7}` also gives 4.

Time $O(n)$ **Space** $O(w)$

Answer: 4

TRAP

Indices double every level, and a JavaScript number stops being exact at $2^{53} - 1$. There is no unsigned 64-bit integer type to reach for. I printed the raw index down a right-leaning chain in Node: at depth 50 it is 1125899906842623 and `Number.isSafeInteger` is still `true`; at depth 60 it is 1152921504606847000 and `Number.isSafeInteger` is **false** — the trailing digits are already made up.

I then built a right chain of depth D with two children at the bottom, so the true answer is always 2, and ran both versions:

depth	without raw - first	with it
20	2	2
50	2	2
54	3	2
58	1	2

Two different wrong answers, silently. The one-line fix is `const idx = raw - first;` — re-base every level so the leftmost node is index 0, and the numbers never grow past the width of the tree. This is the whole point of the problem.

If you ever genuinely need indices past 2^{53} , `BigInt` is exact: $2n * idx + 1n$. It is slower and you cannot mix it with plain numbers, so re-basing is the better answer here.

Example 21 [LINE MAN · pattern]

Children-sum property: for every non-leaf node, the value equals the sum of its children's values (a missing child counts as 0). Check it. Constraints: n up to 10^5 , values from 0 to 10^9 . Target: $O(n)$. Edge cases: single node (true by definition); a node with only one child; an empty tree.

SOLUTION

children sum check

JS

```
const childrenSum = (r) => {
  if (r === null) return true;
  if (r.left === null && r.right === null) return true; // leaves pass
  let s = 0;
  if (r.left) s += r.left.val;
  if (r.right) s += r.right.val;
  return s === r.val && childrenSum(r.left) && childrenSum(r.right);
};
```

Ran it: tree T → false (node 1 has children 2 and 3, and $2 + 3 \neq 1$). {10, 4, 6} → true. {10, 4, 5} → false. null → true.

Time $O(n)$ Space $O(h)$

Answer: false for T, true for {10, 4, 6}

Tier 3 — two answers at once

Example 22 [Amazon · pattern]

Diameter: the number of edges on the longest path between any two nodes. The path does not have to pass through the root. Constraints: n up to 10^5 , so $O(n^2)$ will time out. Edge cases: empty tree (0); single node (0); a chain of k nodes ($k - 1$).

SOLUTION

Approach 1 — For every node, height(left) + height(right) ($O(n^2)$ — recomputes the same heights over and over)

brute force diameter

JS

```
const diameterSlow = (r) => {
  if (r === null) return 0;
  const through = height(r.left) + height(r.right); // edges through r
  return Math.max(through, diameterSlow(r.left), diameterSlow(r.right));
};
```

Correct, but on a chain of 10^5 nodes it is 10^{10} steps.

Approach 3 — One postorder pass: return the height, record the best diameter ($O(n)$ — optimal)

This is the **return-one-record-another** shape from the pattern page. Each node needs two numbers, and only one of them can be a return value — so the other one lives in a variable the inner function closes over.

diameter in one pass

JS

```
const diameter = (root) => {
  let best = 0; // the closure replaces C++'s int &best
  const go = (r) => {
    if (r === null) return 0;
    const lh = go(r.left);
    const rh = go(r.right);
    best = Math.max(best, lh + rh); // RECORD: path bending at r, in edges
    return 1 + Math.max(lh, rh); // RETURN: height, for my parent
  };
  go(root);
  return best;
};
```

Ran both versions. On T: 5 from each. Edge cases from the fast one: `diameter(null) = 0`, single node 0, chain of 4 nodes 3.

I also generated **200 random trees** of up to 12 nodes and checked that the slow and fast versions agree on every single one. They do. That is the test you should write whenever you replace a brute force with a clever version.

Time $O(n)$ **Space** $O(h)$

The idea: the path with the most edges either bends at this node (then it is $lh + rh$), or it lies entirely inside one child (then a deeper call already recorded it). Those two cases are exhaustive, so one pass is enough.

Answer: 5

DIAMETER FOLLOW-UPS

“Return the actual path, not just its length.” Store, at each node, the child that gave the taller arm. After the pass, walk down from the recorded bend node in both directions.

“Values are weights on the edges.” Replace $1 + \max(\text{lh}, \text{rh})$ with $\text{weight} + \max(\text{lh}, \text{rh})$ and $\text{lh} + \text{rh}$ stays as it is. The shape does not change at all.

“It is a general tree, not binary.” Loop over the children, keep the **two** largest arms, record their sum. Same $O(n)$.

Example 23 [Google · pattern]

Lowest common ancestor: given two values that are both present, find the deepest node that has both of them somewhere below it (a node counts as its own ancestor). Constraints: n up to 10^5 , values distinct. Edge cases: one value is an ancestor of the other; both in the same subtree.

SOLUTION

Approach 1 — Build the root-to-node path for each value, then compare ($O(n)$ time, $O(h)$ space — correct, safe with missing values, and longer to write)

LCA via paths

JS

```
const pathTo = (r, x, p) => {
  if (r === null) return false;
  p.push(r.val);
  if (r.val === x) return true;
  if (pathTo(r.left, x, p) || pathTo(r.right, x, p)) return true;
  p.pop(); // un-choose
  return false;
};

const lcaSlow = (r, a, b) => {
  const pa = [], pb = [];
  if (!pathTo(r, a, pa) || !pathTo(r, b, pb)) return null; // a value is missing
  let i = 0, ans = null;
  while (i < pa.length && i < pb.length && pa[i] === pb[i]) { ans = pa[i]; i++; }
  return ans; // last shared node on the two
  paths
};
```

Ran it on T: (4, 8) → 2, (4, 6) → 1, (7, 8) → 5, (2, 7) → 2, and (4, 99) → null. Note the last one — this version **detects** a missing value, which the short version below does not. null is the right “absent” marker here: there is no INT_MIN in JavaScript, and -Infinity would be a strange thing to return where a node value is expected.

Approach 2 — One pass, report upward ($O(n)$ time, $O(h)$ space — shorter, and the version interviewers want)

Three lines of logic, and they are worth memorising word for word.

lowest common ancestor

JS

```
const lca = (r, p, q) => {
  if (r === null) return null;
  if (r.val === p || r.val === q) return r; // found one: report myself upward
  const L = lca(r.left, p, q);
  const R = lca(r.right, p, q);
  if (L && R) return r; // they split here: this is the LCA
  return L || R; // both on one side, pass it up
};
```

Ran it on T:

pair	LCA	why
4 and 8	2	4 is under 2's left, 8 is under 2's right
4 and 6	1	different sides of the root
7 and 8	5	both are children of 5
2 and 7	2	7 is under 2, so 2 is its own ancestor

Time $O(n)$ **Space** $O(h)$

The idea: a node returns “the useful thing I found below me”. If both sides return something, the two values split here and **this** node is the answer. If only one side returns something, that answer is already correct — just pass it up unchanged.

TRAP

The last row is the subtle one. When p is an ancestor of q, the function stops at p and never even looks for q. That is fine **only because the problem promises both values exist**. If a value might be missing, this code returns a wrong answer — I ran `lca(T, 4, 999)` and it returned the node holding **4**. To be safe, first `contains()` both values in $O(n)$, or carry two boolean flags up alongside the node.

TRAP

`return L || R;` is shorter than the ternary and means the same thing **here**, because a node object is always truthy and the only other value is `null`. Do not carry that habit into a function that returns numbers: `0 || R` skips the perfectly valid 0. When the value can be falsy, use `L ?? R` (nullish coalescing) or a real `=== null` test.

Example 24 [Microsoft · pattern]

Maximum path sum: find the largest sum over all paths. A path is any sequence of nodes connected parent-to-child, going up and then down at most once. It need not touch the root or a leaf. Constraints: values from -10^4 to 10^4 , n up to 10^5 . Edge cases: every value negative; a single node; a mix.

SOLUTION

Same shape as the diameter, with two extra ideas:

1. A branch whose best sum is negative should be **dropped** — use `max(0, ...)`.
2. The recorded best must include **this node alone**, in case both branches are dropped.

maximum path sum

JS

```
const maxPathSum = (root) => {
  let best = -Infinity; // NOT 0: every value may be negative
  const go = (r) => {
    if (r === null) return 0;
    const L = Math.max(0, go(r.left)); // drop a negative branch
    const R = Math.max(0, go(r.right));
    best = Math.max(best, L + R + r.val); // RECORD: path bending at r
    return r.val + Math.max(L, R); // RETURN: best straight arm
  };
  go(root);
  return best;
};
```

Ran it:

tree	answer	best path
T	25	$8 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6$
$\{-3, -2, -9\}$	-2	the single node -2
$\{-5\}$	-5	the only node
$\{2, -1, 3\}$	5	$2 + 3$; the -1 branch is dropped

Let me show the T arithmetic in full, because it is the whole lesson:

node	L	R	record $L + R + \text{val}$
4	0	0	4
7	0	0	7
8	0	0	8
5	7	8	20
2	4	$5 + 8 = 13$	19
6	0	0	6
3	0	6	9
1	$2 + 13 = 15$	9	25

Read the last row: the arm coming up from 2 is $2 + \max(4, 13) = 15$, which is the chain $8 \rightarrow 5 \rightarrow 2$. The arm coming up from 3 is $3 + 6 = 9$. Bending at the root gives $15 + 9 + 1 = 25$, so the winning path is $8 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6$.

Time $O(n)$ **Space** $O(h)$

Answer: 25

TRAP

Initialise best to -Infinity, not 0. On the all-negative tree $\{-3, -2, -9\}$ the true answer is -2 ; starting at 0 would return 0, a sum no path can produce. I ran exactly that case and it returns -2 .

`Math.max(0, go(r.left))` needs no type suffix — this is the one spot where JavaScript is kinder than C++, which forces you to write `max(0LL, ...)` or fail to compile. The cost of that kindness is that JavaScript will not warn you about anything at all, so **you** have to remember that **best** must start at -Infinity. Test the all-negative tree every time.

Example 25 [Goldman Sachs · pattern]

Rebuild the tree from its preorder list and its inorder list. All values are distinct. Constraints: n up to 10^5 . Target: $O(n)$. Edge cases: empty lists; a single node; a left-only chain.

SOLUTION

Two facts do all the work:

1. The **first** value in preorder is the root.
2. In the inorder list, everything **before** the root belongs to the left subtree and everything **after** belongs to the right subtree.

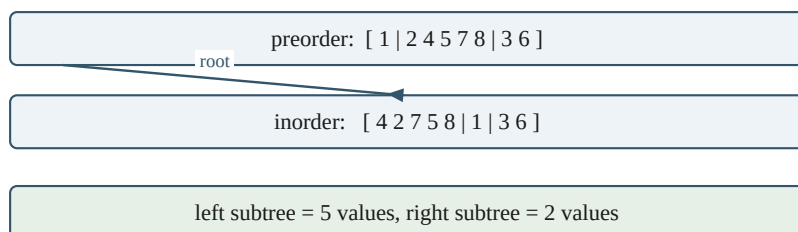
So one scan of preorder, left to right, hands out roots in exactly the order the recursion needs them. Use a **shared** index `pi` that only ever moves forward.

Approach 1 — Find the root in the inorder list with a linear scan ($O(n^2)$ — fine for $n = 1000$, too slow at 10^5)

```
let m = lo;
while (ino[m] !== v) m++; // O(n) work, at every one of the n nodes
```

I ran this version on T's own traversals and it rebuilt the tree correctly — it is not wrong, only slow. On a left-only chain of 10^5 nodes it does about 5×10^9 steps.

Approach 2 — Look the root up in a hash map (O(n) — optimal)



preorder gives the root; inorder splits the rest

build from preorder + inorder

JS

```
const buildFrom = (pre, ino) => {
  const pos = new Map(); // value -> index, O(1) lookups
  ino.forEach((v, i) => pos.set(v, i));
  let pi = 0; // ONE shared cursor, closed over
  const go = (lo, hi) => {
    if (lo > hi) return null;
    const v = pre[pi++]; // next root, in preorder
    const r = new Node(v);
    const m = pos.get(v); // where it sits in inorder
    r.left = go(lo, m - 1);
    r.right = go(m + 1, hi);
    return r;
  };
  return go(0, ino.length - 1);
};
```

Ran it: I took T's own lists, `pre = [1,2,4,5,7,8,3,6]` and `ino = [4,2,7,5,8,1,3,6]`, rebuilt the tree, and compared the new tree's preorder and inorder against the originals with `JSON.stringify`. Both matched exactly. Empty lists give `null`; `[9]` and `[9]` give a single node.

Time $O(n)$ **Space** $O(n)$

the Map is what makes it $O(n)$ instead of $O(n^2)$

The idea: there is exactly **one** `pi`, and the inner `go` sees it because it is defined inside `buildFrom`. The left subtree's recursion consumes exactly as many preorder values as it has nodes, so when it returns, `pi` is already pointing at the right subtree's root.

TRAP

Writing `const go = (lo, hi, pi) => ...` and passing `pi` down as an argument breaks everything silently. A number argument is a **copy**: each call gets its own `pi`, the increments never travel back up, and the tree comes out wrong with no error. This is the JavaScript version of C++'s "you forgot the `&`". Keep `pi` in the enclosing scope.

Use a `Map` for `pos`, not a plain object. Object keys are strings, so a value of `-3` becomes `"-3"` and `pos[-3]` happens to still work — until you iterate the keys and find strings where you expected numbers. `Map` keeps the number a number and is faster for 10^5 inserts.

Which pairs of traversals are enough? Preorder + inorder works. Postorder + inorder works. Preorder + postorder does **not** — it cannot tell a single left child from a single right child. This is a favourite follow-up question, and the one-line answer above is what the interviewer wants.

Example 26 [Amazon · pattern]

Serialize and deserialize: turn any binary tree into a string, and turn that string back into the identical tree. Constraints: n up to 10^4 , values from -1000 to 1000 . Target: $O(n)$ each way. Edge cases: empty tree; a node whose value is negative; a one-sided tree.

SOLUTION

Level order with an explicit marker for a missing child. I use # for “no node” and a comma after every token.

```
serialize                                     JS

const serialize = (root) => {
  let s = '';
  const q = new Deque(); q.push(root);
  while (q.size) {
    const c = q.shift();
    if (c === null) { s += '#,'; continue; }
    s += c.val + ','; // number + string -> string, no
  to_string
    q.push(c.left); q.push(c.right); // push nulls too
  }
  return s;
};
```

```
deserialize                                   JS

const deserialize = (s) => {
  const tok = s.split(',');
  tok.pop(); // the trailing comma leaves an empty
  token
  if (tok.length === 0 || tok[0] === '#') return null;
  const root = new Node(Number(tok[0]));
  const q = new Deque(); q.push(root);
  let i = 1;
  while (q.size && i < tok.length) {
    const c = q.shift();
    if (i < tok.length && tok[i] !== '#') { c.left = new Node(Number(tok[i]));
  q.push(c.left); }
    i++;
    if (i < tok.length && tok[i] !== '#') { c.right = new Node(Number(tok[i]));
  q.push(c.right); }
    i++;
  }
  return root;
};
```

Ran it on T. The string was

1,2,3,4,5,#,6,#,7,8,#,#,#,#,#,

I fed that straight back into deserialize and printed the preorder of the result: [1,2,4,5,7,8,3,6] — identical to T. serialize(null) gives "#," and deserialize("#,") gives null. I also round-tripped the tree {-7, -8, 9} and got [-7,-8,9] back.

Time $O(n)$ Space $O(n)$

The idea: `Number("-8")` handles the leading minus sign, so negative values survive the round trip. If values could contain a comma you would need a different separator — say so.

TRAP

Three JavaScript-only details in these twenty lines.

1. `'1,2,3'.split(',')` returns `['1', '2', '3', '']` — a trailing separator leaves an **empty last token**. The `tok.pop()` removes it. Without it, `Number('')` is `0` and you grow a phantom node holding zero.
2. Use `Number(tok[i])`, not `parseInt(tok[i])`. `parseInt('12abc')` happily returns `12`, so corrupt input passes silently; `Number('12abc')` gives `NaN`, which you can check.
3. `s += c.val + ','` works because a number plus a string is a string. That is convenient here and a menace elsewhere: `'5' + 3` is `'53'`, while `'5' - 3` is `2`.

TRAP

The `#` markers are not optional. `1,2,3` alone does not say whether `3` is the right child of `1` or the left child of `2`. Every missing child needs a marker, which is why the serialised string is longer than the node count. A preorder serialisation with markers works equally well — pick one and be consistent.

Example 27 [Uber · pattern]

Given a target **node** (by value) and a number k , list every node exactly k edges away — above, below, or sideways through the parent. Constraints: n up to 10^5 . Edge cases: $k = 0$; k larger than the tree; the target is missing.

SOLUTION

A tree only has **downward** pointers, but this problem needs to walk **upward**. There are two ways to get that.

Approach 1 — Build a parent map, then BFS ($O(n)$ time, $O(n)$ space — longer in memory, much easier to get right)

Build a parent map once, and then the tree is just an undirected graph — so do a plain BFS from the target and stop at distance k .

nodes at distance k

JS

```
const mapParents = (r, par) => {
  const q = new Deque(); q.push(r); par.set(r, null);
  while (q.size) {
    const c = q.shift();
    if (c.left) { par.set(c.left, c); q.push(c.left); }
    if (c.right) { par.set(c.right, c); q.push(c.right); }
  }
};

const distanceK = (root, targetVal, k) => {
  const out = [];
  if (root === null) return out;
  const target = findNode(root, targetVal);
  if (target === null) return out;
  const par = new Map(); // Map, not an object: the KEYS are nodes
  mapParents(root, par);

  const seen = new Set([target]);
  const q = new Deque(); q.push(target);
  let dist = 0;
```

```

while (q.size) {
  if (dist === k) { // everything in the queue is at
    distance k
    while (q.size) out.push(q.shift().val);
    break;
  }
  const sz = q.size;
  for (let i = 0; i < sz; i++) {
    const c = q.shift();
    for (const nb of [c.left, c.right, par.get(c)]) // three neighbours
      if (nb && !seen.has(nb)) { seen.add(nb); q.push(nb); }
  }
  dist++;
}
out.sort((a, b) => a - b); // NUMERIC sort, never a bare sort()
return out;
};

```

TRAP

This is the problem where a plain object would destroy you. `par` maps a **node object** to its parent node. A plain object converts every key with `String(key)`, and every node object becomes the same string `"[object Object]"` — so all 10^5 entries collide into one. Map and Set compare objects by identity, which is the whole point.

Use Map when the key is an object, a number you want to stay a number, or anything you will iterate in insertion order. Use a plain object only for fixed string keys you typed yourself.

TRAP

`out.sort()` with no comparator would return `[1, 4]` correctly by luck and `[10, 4]` as `[10, 4]` wrongly. Values here go to 10^5 . Write `out.sort((a, b) => a - b)`.

the `findNode` helper this uses

JS

```

const findNode = (r, x) => {
  if (r === null) return null;
  if (r.val === x) return r;
  return findNode(r.left, x) || findNode(r.right, x);
};

```

Ran it on T:

target	k	answer	why
5	2	[1, 4]	up to 2 then to 1, and up to 2 then down to 4
5	0	[5]	distance 0 is the node itself
1	1	[2, 3]	the root's two children
5	9	[]	nothing is that far
99	1	[]	target not in the tree

Time $O(n)$ **Space** $O(n)$

the parent map is the $O(n)$ space

The idea: the seen set is what stops the BFS from bouncing back down the branch it just came up. Without it, the search runs forever.

Approach 2 — No parent map: let the recursion carry the distance back up ($O(n)$ time, $O(h)$ space — shorter, and far easier to get wrong)

A node asks its left subtree “how far below you is the target?”. If the answer is dl , then this node is at distance $dl + 1$. Two things can happen from here: this node itself is at distance k , or the answer lies in the **other** subtree, $k - dl - 2$ levels down (one step up to this node, one step down into the other side).

distance k without a parent map

JS

```
const downK = (r, k, out) => { // nodes exactly k below r
  if (r === null || k < 0) return;
  if (k === 0) { out.push(r.val); return; }
  downK(r.left, k - 1, out);
  downK(r.right, k - 1, out);
};

// returns the distance from r down to the target, or -1 if it is not below r
const kGo = (r, target, k, out) => {
  if (r === null) return -1;
  if (r.val === target) { downK(r, k, out); return 0; }

  const dl = kGo(r.left, target, k, out);
  if (dl !== -1) {
    if (dl + 1 === k) out.push(r.val);
    else downK(r.right, k - dl - 2, out); // cross over through r
    return dl + 1;
  }
  const dr = kGo(r.right, target, k, out);
  if (dr !== -1) {
    if (dr + 1 === k) out.push(r.val);
    else downK(r.left, k - dr - 2, out);
    return dr + 1;
  }
  return -1;
};
```

I ran **both** versions on every pair (target, k) with the target one of T’s 8 values and k from 0 to 6 — 56 cases, compared with `JSON.stringify`. They agree on all 56. Spot check from node 4:

k	0	1	2	3
answer	[4]	[2]	[1,5]	[3,7,8]

The idea: $k - dl - 2$ is the whole problem in one expression. Get it wrong by one and you will pass the easy tests and fail the crossing ones. Write out a small case by hand before you trust it.

FOLLOW-UPS ON DISTANCE-K

“Now the tree is burning — it starts at that node and spreads one edge per second. How long until the whole tree burns?” Identical code, but instead of stopping at distance k , run the BFS to the end and return the number of levels minus one. I ran it on T: from node 5 it takes 4 seconds, from the root 1 it takes 3, and from leaf 4 it takes 4. A single-node tree takes 0. The code is the same BFS with the `dist === k` early exit removed and `dist` returned at the end.

“The tree has a million nodes and you get thousands of queries.” Pre-compute the parent map **once**, not per query. Better still, root the tree, store an Euler tour, and answer each query with binary lifting in $O(\log n)$ — that is the standard escalation and it belongs to the graph chapters.

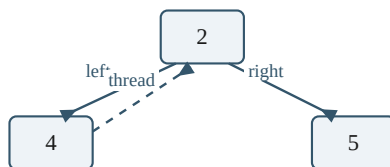
Example 28 [Adobe · pattern]

Do the inorder traversal in $O(1)$ extra space — no recursion, no stack. Constraints: n up to 10^6 , and the tree must be unchanged when you finish.

SOLUTION

This is **Morris traversal**. The idea: before going down-left, find the node that would be visited just before this one (the rightmost node of the left subtree, the **predecessor**) and make its empty right reference point **back** at this node. That temporary link is called a **thread**. When you come back up through the thread, you remove it, restoring the tree.

This is also the only traversal in the chapter that is safe on a 10^6 -node chain in JavaScript: no recursion, and no stack array either.



walk down-left, leaving a way back; on the way back, cut the thread

the thread: 4's empty right pointer temporarily points back to 2

Morris inorder, $O(1)$ space

JS

```
const morrisInorder = (root) => {
  const out = [];
  let cur = root;
  while (cur !== null) {
    if (cur.left === null) {
      out.push(cur.val);           // nothing on the left: visit and go right
      cur = cur.right;
    } else {
      let pred = cur.left;
      while (pred.right !== null && pred.right !== cur) pred = pred.right;
      if (pred.right === null) {
        pred.right = cur;         // PUT the thread
        cur = cur.left;
      } else {
        pred.right = null;        // REMOVE the thread: left side is done
        out.push(cur.val);
        cur = cur.right;
      }
    }
  }
  return out;
};
```

Ran it on T: [4,2,7,5,8,1,3,6] — identical to the recursive inorder. Then I ran the **recursive** inorder again on the same tree afterwards and got the same list, proving the threads were all removed and T was left intact. `morrisInorder(null)` gives []; a single node gives [9].

Time $O(n)$ **Space** $O(1)$

each edge is walked at most 3 times, so it is still linear

The idea: the inner while stops on `pred.right !== cur` as well as on `null`. That second test is how you tell “I am going down for the first time” from “I have come back up through my own thread”. Note it compares **node identity** with `!==`, not values. Drop it and the loop never ends.

Example 29 [D. E. Shaw · pattern]

Flatten the tree into a right-leaning chain, in preorder, using $O(1)$ extra space. Every node’s left must end up `null`. Constraints: n up to 10^5 ; extra space must be $O(1)$, so no recursion and no stack. Edge cases: empty tree; a tree with no left children at all; a single node.

SOLUTION

Same threading idea as Morris, but this time the rewiring is permanent.

For each node that has a left subtree: find the **rightmost** node of that left subtree, hang the current right subtree off it, then move the whole left subtree to the right and clear left. Then step right and repeat.

flatten to a preorder chain

JS

```
const flatten = (root) => {
  let cur = root;
  while (cur !== null) {
    if (cur.left !== null) {
      let pred = cur.left;
      while (pred.right !== null) pred = pred.right; // rightmost of left
      pred.right = cur.right; // hang the right side there
      cur.right = cur.left; // left becomes right
      cur.left = null;
    }
    cur = cur.right;
  }
};
```

Ran it on a copy of *T*, then walked the chain following only right references: `[1,2,4,5,7,8,3,6]`. That is exactly *T*’s preorder, and I checked that every left along the chain is `null`.

Note that `flatten` returns nothing — it rewires the tree in place. That works because objects are passed by reference in JavaScript: `cur.left = null` changes the caller’s tree. Reassigning the parameter itself (`root = something`) would **not** be visible outside.

Time $O(n)$ **Space** $O(1)$

each node’s left subtree is scanned once along its right spine

The idea: the preorder of a node is “me, then all of my left subtree, then all of my right subtree”. Sticking the right subtree onto the **end** of the left subtree’s right spine puts the three pieces in exactly that order.

Example 30 [Microsoft · pattern]

Count the **good** nodes: a node is good if no node on the path from the root down to it holds a strictly larger value. Constraints: values may be negative, n up to 10^5 . Edge cases: empty tree; all values equal; strictly decreasing values.

SOLUTION

Carry the largest value seen so far **down** the tree. This is the mirror image of the diameter pattern: information flows down, not up.

count good nodes

JS

```
const goodGo = (r, best) => {
  if (r === null) return 0;
  const c = r.val >= best ? 1 : 0;
  const nb = Math.max(best, r.val); // pass the new maximum down
  return c + goodGo(r.left, nb) + goodGo(r.right, nb);
};

const goodNodes = (r) => goodGo(r, -Infinity);
```

Ran it: on T, 8 — every node is good, because the values grow as you go down every path. On {5, 3, 8, 1, 4}, 2 — only the root 5 and the node 8. On null, 0.

Time $O(n)$ **Space** $O(h)$

The idea: best is a **parameter**, not a closed-over variable. Each recursive call gets its own copy, so a value picked up on the left branch cannot leak into the right branch. That is the whole reason no “undo” line is needed here — compare it with Example 12, where the shared cur array forced a cur.pop().

This is the one place where JavaScript’s copy-on-pass for numbers is exactly what you want. It is the same behaviour that ruins diameter if you try to pass best as an argument — the mechanism is identical, and only the intent differs. Know which one you need.

Answer: 8 and 2

Dry run — diameter of T, node by node

This traces the inner go from Example 22. It returns the **height in edges** (0 for null) and records lh + rh into best, the variable in the enclosing function. Postorder: children finish first.

the code being traced

JS

```
const diameter = (root) => {
  let best = 0;
  const go = (r) => {
    if (r === null) return 0;
    const lh = go(r.left);
    const rh = go(r.right);
    best = Math.max(best, lh + rh);
    return 1 + Math.max(lh, rh);
  };
  go(root);
  return best;
};
```

step	node	lh	rh	lh+rh	best after / returns
1	4	0	0	0	0 / returns 1
2	7	0	0	0	0 / returns 1
3	8	0	0	0	0 / returns 1
4	5	1	1	2	2 / returns 2
5	2	1	2	3	3 / returns 3
6	6	0	0	0	3 / returns 1
7	3	0	1	1	3 / returns 2

8	1	3	2	5	5 / returns 4
---	---	---	---	---	---------------

best ends at 5. The path with 5 edges is $7 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 6$ — six nodes, five edges. It does not touch the root’s left-left branch at all, which is why you cannot just add the two heights at the root and stop.

WHAT THE TABLE TEACHES

1. **Leaves return 1, not 0.** The return value is `1 + Math.max(0, 0)`. The 0 for null is the height in **nodes** of an empty tree; the recorded `lh + rh` is in **edges**. Two different units in one function — this is exactly where people lose marks.
2. `best` only ever grows. It is updated at steps 4, 5 and 8. Every other step leaves it alone.
3. The winning value 5 is recorded at the root, but the **path** does not include the root’s left child 4. `best` at a node is about the **shape** below it, not about which nodes lie on the path.

TRAP

If you report the diameter in **nodes** instead of edges, T’s answer is 6, not 5. Both are defensible; the question decides. Say “I am counting edges” before you write code. If the statement says “number of nodes on the longest path”, return `best + 1` — but be careful: for an empty tree that formula gives 1, which is wrong, so special-case `root == null`.

Python versions — inorder, level order, diameter

the three signature traversals in Python

PYTHON

```
from collections import deque

class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def inorder(root):
    out = []
    def go(n):
        if n is None:
            return
        go(n.left)
        out.append(n.val)
        go(n.right)
    go(root)
    return out

def level_order(root):
    if root is None:
        return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            cur = q.popleft()
            level.append(cur.val)
            if cur.left: q.append(cur.left)
```

```

        if cur.right: q.append(cur.right)
    out.append(level)
    return out

def diameter(root):
    best = 0
    def go(n):
        nonlocal best
        if n is None:
            return 0
        lh = go(n.left)
        rh = go(n.right)
        best = max(best, lh + rh)      # path bending here, in edges
        return 1 + max(lh, rh)
    go(root)
    return best

```

Ran all three on tree T: inorder → [4, 2, 7, 5, 8, 1, 3, 6], level_order → [[1], [2, 3], [4, 5, 6], [7, 8]], diameter → 5. Empty tree: [], [], 0. Single node: diameter 0.

TRAP

`nonlocal best` is required. Without it, `best = max(...)` creates a brand-new **local** variable inside `go` and the outer `best` never changes — the function silently returns 0.

JavaScript does **not** need a keyword here: `best = Math.max(...)` inside a nested arrow function assigns to the outer `best` automatically, because `let best` is already in scope. That is the one thing JavaScript makes easier than Python in this chapter. The JavaScript mistake is the opposite one — declaring `let best` again **inside** `go`, which shadows the outer variable and loses every update.

Also, `for _ in range(len(q))` reads the length **once**, before the loop body runs. That is the Python equivalent of `const sz = q.size;`

Practice

PRACTICE · WARM-UP target: 25 min

1. Count the nodes that have exactly one child.
2. Given a value, return its level (root is level 1), or `-1` if the value is absent.
3. Count the nodes at a given depth d (root is depth 0).
4. Return the **sum of the leaf values** and the **maximum value**, in one traversal that visits each node once.

PRACTICE · TIER 1 target: 45 min

5. Compute (sum of values at odd levels) minus (sum at even levels), root at level 1.
6. Given two trees, decide whether one is the mirror of the other.
7. Report the average value of each level.
8. Which level has the biggest sum? Return the level number (1-based), and the sum.
9. Print all the ancestors of a given value, nearest first.

PRACTICE · TIER 2 target: 55 min

10. **Diagonal traversal:** group nodes by “diagonal”, where moving to a right child stays on the same diagonal and moving to a left child starts the next one.
11. Sum the values of the leaves that sit at the **deepest** level.
12. Two nodes are **cousins** if they are at the same depth but have different parents. Given two values, decide whether they are cousins.
13. Replace every node’s value by the sum of all values in its subtree, **not** counting itself. (A leaf becomes 0.)

PRACTICE · TIER 3 target: 60 min

14. Count the subtrees whose total value equals a given target.
15. Find the length of the longest downward path whose values increase by exactly 1 at each step.
16. Compute the sum of the values in each **column** (the top-view column numbering), left to right.

ANSWER KEY

1. `((r.left === null) !== (r.right === null)) ? 1 : 0` plus the two recursive calls. The `!==` on two booleans is a clean “exactly one of them”. Ran it on T: **1** (only node 3). Empty tree and single node both give **0**.
2. `const levelOf = (r, x, d) => ...: return d if r.val === x; search left, and if that returns -1 search right.` Call it with `d = 1`. Ran it: value 7 → **4**, value 1 → **1**, value 99 → **-1**.
3. `if (d === 0) return 1; then atDepth(r.left, d - 1) + atDepth(r.right, d - 1).` Ran it on T: depth 2 → **3**, depth 3 → **2**, depth 9 → **0**.
4. Return a two-element array `[leafSum, max]` and unpack it with destructuring: `const [ls, lm] = go(r.left); const [rs, rm] = go(r.right); then return [ls + rs, Math.max(r.val, lm, rm)];`, with `[0, -Infinity]` for null and `[r.val, r.val]` at a leaf. Ran it: T → **25** and **8**; the all-negative tree `{-3, -2, -9}` → **-11** and **-2**; a single node `9` → **9** and **9**. Neutral elements matter: **0** for the sum, **-Infinity** for the maximum. Returning an array and destructuring it is JavaScript’s answer to a **pair**, and it reads better.
5. Carry the level down and flip the sign: `(d % 2 === 1 ? r.val : -r.val)` plus the two calls with `d + 1`. Ran it on T: `1 - 5 + 15 - 15 = -4`.
6. Exactly `mirror(a, b)` from Example 10, but called on the two **roots**: `mirror(rootA, rootB)`. Ran it: `{1,2,3}` against `{1,3,2}` → **true**; `{1,2,3}` against `{1,2,3}` → **false**.
7. The level-order template with `let s = 0` and `out.push(s / sz);` — no cast needed, because `/` in JavaScript is always real division. Ran it on T: **[1, 2.5, 5, 7.5]**.
8. Same template, keep a running level counter and the best sum seen. Ran it on T: level **3**, sum **15**. Levels 3 and 4 tie at 15; if `(s > bs)` keeps the **first**, which is level 3. If the question wants the last, use `>=`. Ask.
9. `const ancGo = (r, x, out) => ...: return true at the match; if either child returns true, push r.val and return true.` Because the push happens while **unwinding**, the list comes out nearest-first. Ran it: ancestors of 8 → **[5, 2, 1]**; of the root 1 → **[]**; of a missing value → **[]**.
10. BFS carrying a diagonal id `k`: a right child keeps `k`, a left child gets `k + 1`. Collect into a **Map** from `k` to an array, then read the keys back in sorted order with `[...m.keys()].sort((a, b) => a - b)`. Ran it on T: **[[1,3,6], [2,5,8], [4,7]]**.
11. Track the deepest leaf depth seen and the running sum. When a deeper leaf appears, **reset** the sum to that leaf’s value; when an equally deep leaf appears, add to it. Ran it on T: the deepest level is 4, holding leaves 7 and 8, so **15**.
12. Find each value’s depth and parent pointer with one traversal each, then check `depthA == depthB && parentA != parentB`. Ran it on T: (4, 6) → **true** (both at depth 2, parents 2 and 3); (4, 5) → **false** (same parent); (7, 8) → **false** (same parent); (1, 2) → **false** (different depths).
13. Postorder. Save the old value, recurse both sides to get `L` and `R`, set `r.val = L + R`, and return `old + L + R` so the parent sees the **original** subtree total. Ran it on T: the preorder of the result is **[35, 24, 0, 15, 0, 0, 6, 0]** — the root becomes `2 + 3 + 4 + 5 + 6 + 7 + 8 = 35` and every leaf becomes 0.
14. Postorder again: return the subtree sum from the inner function, and increment a **count** in the enclosing scope whenever that sum equals the target. Ran it on T: target 9 → **1** (the subtree rooted at node 3, which is `3 + 6`); target 36 → **1** (the whole tree); target 15 → **0**.

15. Return the longest increasing-by-one chain that **starts at this node**, and record the best anywhere. `let len = 1`, and if `r.left && r.left.val === r.val + 1` then `len = Math.max(len, 1 + L)`, same for the right. Ran it: on the chain $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ the answer is **4**; on T it is **2** (just $1 \rightarrow 2$); a single node gives **1**; an empty tree gives **0**.
16. The vertical-order BFS, but add into a **Map** from column to a running total instead of pushing, and sort the keys numerically at the end. Ran it on T: **[4, 9, 6, 11, 6]** — column -2 holds 4; column -1 holds 2 and 7, summing to 9; column 0 holds 1 and 5, summing to 6; column $+1$ holds 3 and 8, summing to 11; column $+2$ holds 6.

ONE-PAGE REVISION CARD

The two templates

```
const { Deque } = require('./toolkit.js'); // JS Toolkit appendix

// 1. depth first - solve left, solve right, combine
const solve = (r) => {
  if (r === null) return neutral;
  const L = solve(r.left);
  const R = solve(r.right);
  return combine(L, R, r.val);
};

// 2. breadth first - one level per outer turn
const q = new Deque(); q.push(root);
while (q.size) {
  const sz = q.size; // FREEZE the size
  for (let i = 0; i < sz; i++) {
    const c = q.shift();
    /* use c, and i === 0 / i === sz - 1 for first / last on the level */
    if (c.left) q.push(c.left);
    if (c.right) q.push(c.right);
  }
}

// 3. return one thing, record another - the closure replaces C++'s int &best
const solveTwoWays = (root) => {
  let best = 0; // declared OUT here, assigned in there
  const go = (r) => {
    if (r === null) return 0;
    const L = go(r.left), R = go(r.right);
    best = Math.max(best, L + R); // RECORD the bend
    return 1 + Math.max(L, R); // RETURN the arm
  };
  go(root);
  return best;
};
```

When to use which

Question mentions	Reach for
height, depth, sum, count, path to a leaf	plain DFS recursion
level, “by level”, zigzag, views, width	BFS with a frozen level size
column, vertical, top / bottom view	BFS carrying a column number + a Map , then sort the keys
diameter, max path sum, longest chain	return an arm, record the bend in a closed-over variable
distance from a node, burning, “upward”	parent map + BFS from the target
$O(1)$ space traversal, flatten	Morris threading
rebuild from traversals	preorder gives roots, inorder splits — keep one shared cursor in the enclosing scope

Complexity table

Task	Time	Space
any single traversal	$O(n)$	$O(h)$
level order / any view	$O(n)$	$O(w)$
top / bottom / vertical with a Map + sort	$O(n \log n)$	$O(n)$
balanced check, naive	$O(n^2)$	$O(h)$
balanced check, one pass	$O(n)$	$O(h)$
diameter, naive	$O(n^2)$	$O(h)$
diameter, one pass	$O(n)$	$O(h)$
LCA	$O(n)$	$O(h)$
build from preorder + inorder	$O(n)$	$O(n)$
serialize / deserialize	$O(n)$	$O(n)$
distance-k / burning	$O(n)$	$O(n)$
Morris inorder, flatten	$O(n)$	$O(1)$

Values to remember for tree T

preorder	1 2 4 5 7 8 3 6	right view	1 3 6 8
inorder	4 2 7 5 8 1 3 6	left view	1 2 4 7
postorder	4 7 8 5 2 6 3 1	top view	4 2 1 3 6
height	4	bottom view	4 7 5 8 6
diameter (edges)	5	boundary	1 2 4 7 8 6 3
max path sum	25	max width	4

Top traps

1. **Not freezing q.size** before the inner level loop. On the toolkit Deque it is a getter, so it silently re-reads.
2. **array.shift() in a BFS.** It is $O(n)$ — measured 2726 ms against the Deque's 10 ms on 200000 items. Use the toolkit Deque.
3. **arr.sort() with no comparator.** It sorts as text: `[-2, -1, 0, 1, 2].sort()` gives `[-1, -2, 0, 1, 2]`. Always $(a, b) \Rightarrow a - b$, and $(a, b) \Rightarrow a[0] - b[0] \parallel a[1] - b[1]$ for two keys.
4. **No TreeMap.** Columns come out of a Map in insertion order; sort the keys yourself.
5. **A plain object where the key is a node.** Every object key stringifies to "[object Object]" and they all collide. Use Map and Set.
6. **Height in nodes versus edges.** Say which one you mean, every time.
7. **minDepth without the one-child guard** — a null child is not a leaf.
8. **Neutral element wrong:** 0 for a max on a negative tree, 0 for best in max path sum. Use -Infinity.
9. **Inverting in place with overwritten references.** Compute both children first, or use the destructuring swap.
10. **Recursion depth.** Node dies at roughly 12500 frames with RangeError: Maximum call stack size exceeded. A 10^5 -node chain needs the explicit-stack version.
11. **new Array(k) without .fill(...)** leaves holes, and `a[0] === null` is false.
12. **Pushing the live path array** instead of `[...cur]` in the backtracking collector.
13. **Index precision in max width.** Past 2^{53} a raw index is no longer exact — re-base each level with raw - first.
14. **Passing the preorder cursor as an argument** when rebuilding a tree. It must live in the enclosing scope.

15. **DFS for the top view.** It must be BFS. **Boundary double-printing leaves** — the edges skip leaves.
16. **LCA when a value is missing** returns the other value's node, not null.
17. **Python: forgetting nonlocal**, so the recorded best never updates. JavaScript needs no keyword — but re-declaring `let best` inside the inner function shadows it, with exactly the same symptom.

Binary Search Trees & Balanced Trees

One rule — smaller on the left, bigger on the right — turns a tree into a sorted list you can walk, search and edit in $O(h)$ time.

Pattern in one page

THE ONE RULE

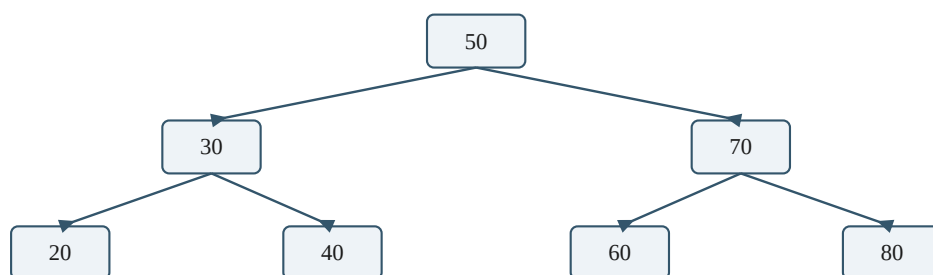
A binary tree is a **binary search tree** (BST) when, for **every** node x :

- every key in the **left** subtree of x is **smaller** than $x.val$, and
- every key in the **right** subtree of x is **bigger** than $x.val$.

The rule is about the whole subtree, not just the two children. That single sentence gives you three gifts:

1. **Search is a decision.** At each node you go left or right. You never look at both sides.
2. **Inorder walk is sorted.** Left, node, right gives the keys in increasing order.
3. **Edits keep the order.** Insert and delete cost the same as a search.

Everything in this chapter is one of those three gifts, reused.



A BST. Read the leaves left to right: 20 30 40 50 60 70 80 — already sorted.

The template you will write again and again

The whole BST toolkit — node, search, insert, delete

JS

```

class Node {
  constructor(val) { this.val = val; this.left = null; this.right = null; }
}

// search — no recursion needed, it is just a walk
const bstSearch = (root, key) => {
  while (root !== null) {
    if (key === root.val) return root;
    root = key < root.val ? root.left : root.right;
  }
  return null;
};

// insert — falls off the bottom, then hangs a new node there
const bstInsert = (root, key) => {

```

```

    if (root === null) return new Node(key);
    if (key < root.val) root.left = bstInsert(root.left, key);
    else if (key > root.val) root.right = bstInsert(root.right, key);
    return root; // equal key: ignore, keep the tree a set
};

const minNode = (t) => { while (t.left !== null) t = t.left; return t; };

// delete — three shapes: leaf, one child, two children
const bstErase = (root, key) => {
    if (root === null) return null;
    if (key < root.val) root.left = bstErase(root.left, key);
    else if (key > root.val) root.right = bstErase(root.right, key);
    else {
        if (root.left === null) return root.right; // leaf or right child only
        if (root.right === null) return root.left; // left child only
        const s = minNode(root.right); // smallest key on the right
        root.val = s.val; // copy it up
        root.right = bstErase(root.right, s.val); // then delete the copy
    }
    return root;
};

const inorder = (r, out = []) => {
    if (r === null) return out;
    inorder(r.left, out); out.push(r.val); inorder(r.right, out);
    return out;
};

```

Running it on the tree above:

Output — this is the real program output, not a guess

TEXT

```

inorder = [20 30 40 50 60 70 80]
search 40 -> found
search 45 -> null
search in empty -> null
duplicates {5,5,5} -> [5]
erase leaf 20 -> [30 40 50 60 70 80]
erase 1-child 30 -> [40 50 60 70 80]
erase 2-child 50 -> [40 60 70 80]
erase missing -> [40 60 70 80]
erase only node -> empty

```

TRAP

Node's recursion depth is about 12500 frames. Measured, not guessed: a plain recursive function in Node crashes with `RangeError: Maximum call stack size exceeded` at roughly 12500 nested calls. `bstInsert` recurses once per level, so inserting the keys 1, 2, 3, ..., 100000 in sorted order — which builds a chain — **crashes**. It does not run slowly; it dies. Two fixes, both in this chapter: keep the tree short (Example 28), or rewrite the walk with an explicit stack (Example 21, approach 3). Any recursion that can go 10^4 deep needs one of the two.

Why every cost is written with *h*, not *n*

h is the **height** of the tree — the longest root-to-leaf path.

Shape	Height h	Search cost
balanced (each level full)	$\approx \log_2 n$	$O(\log n)$
random insert order	$\approx 1.39 \log_2 n$	$O(\log n)$
sorted insert order (1,2,3,...)	n	$O(n)$ — a linked list

TRAP

A plain BST is **not** fast. It is fast **only if it stays short**. Insert 1, 2, 3, ..., 1000 in that order and you get a chain of 1000 nodes. Every search then scans all 1000. This is why balanced trees exist. And JavaScript gives you **no** balanced tree in the standard library, so in JS you must either keep an array sorted or balance the tree yourself — Example 28 shows both ways.

BST vs the other two containers

Operation	sorted array	Map / Set	balanced BST (hand-written)
find a key	$O(\log n)$	$O(1)$ average	$O(\log n)$
insert / delete	$O(n)$	$O(1)$ average	$O(\log n)$
smallest / biggest	$O(1)$	$O(n)$	$O(\log n)$
next key after x	$O(\log n)$	$O(n)$	$O(\log n)$
list keys in order	$O(n)$	$O(n \log n)$	$O(n)$

SHORTCUT

Reach for a BST when you need **order** and **change** at the same time. If the data never changes, sort it once. If you never need order, use a Map or a Set. The BST is for the middle case: “keep me sorted while I keep inserting and deleting.”

The JavaScript shortcut — and the one that does not exist

Other languages hand you a balanced BST: C++ has an ordered set, Java has TreeMap, Python has sortedcontainers. **JavaScript has nothing like it**. A JS Set is a hash set: it has no order, no “smallest key”, no “next key after x”.

So the JS stand-in is a **sorted array plus binary search**. The book’s toolkit has the two search helpers (see the JS Toolkit appendix); you write the require line once per file:

The toolkit import used through this chapter

JS

```
const { lowerBound, upperBound } = require('./toolkit.js');
```

A sorted array gives floor, ceil and successor — reads are cheap, writes are not

JS

```
const sortedSetDemo = () => {
  const s = [40, 10, 70, 25, 90].sort((a, b) => a - b); // (a,b)=>a-b ALWAYS
  let i = lowerBound(s, 30); // first index with s[i] >= 30
  console.log('ceil(30) =', s[i]);
  i = upperBound(s, 70); // first index with s[i] > 70
  console.log('successor(70) =', s[i]);
  const p = lowerBound(s, 26); // floor(26): step one back
  console.log('floor(26) =', p > 0 ? s[p - 1] : 'none');
  s.splice(s.indexOf(25), 1); // erase 25 — O(n), the array's price
}
```

```
console.log('after erase 25:', s.join(' '));  
};
```

Output

TEXT

```
ceil(30) = 40  
successor(70) = 90  
floor(26) = 25  
after erase 25: 10 40 70 90
```

TRAP

arr.sort() is **lexicographic**. It turns every element into a string first, so `[10, 9, 1].sort()` gives `[1, 10, 9]` — 9 last, because `"9" > "10"` as text. Numbers **always** need `arr.sort((a, b) => a - b)`. This one line costs more interview marks than any other JS mistake, and a sorted-array BST stand-in is built on it.

TRAP

`lowerBound(s, x)` can return 0, meaning “every key is $\geq x$ ”, so there is **no** floor. `s[p - 1]` is then `s[-1]`, which in JS is `undefined` rather than a crash — the bug travels silently into your answer. Always test `p > 0` first. The same goes the other way: `lowerBound` can return `s.length`, and `s[s.length]` is `undefined` too.

TRAP

The sorted array is $O(n)$ to **edit**. `splice` shifts every later element. For 10^5 keys with 10^5 inserts that is 10^{10} moves. Reads are $O(\log n)$, writes are $O(n)$ — if the problem inserts and deletes as often as it searches, write the balanced tree from Example 28 instead.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Which of these is a valid BST?

- (A) root 10, left child 5, right child 15
- (B) root 10, left child 15, right child 5
- (C) root 10, left child 5, right child 15, and 15 has left child 6

SOLUTION

- (A) $5 < 10 < 15$. Valid.
- (B) 15 is on the left of 10 but $15 > 10$. Invalid.
- (C) Look at 6. It sits in the **right subtree** of 10, so it must be bigger than 10. It is not. Invalid — even though $6 < 15$ is fine for its parent.

Answer: Only (A)

TRAP

Case (C) is the trap that kills most “validate BST” attempts. Checking only `left < node < right` for each parent and child is **not enough**. The rule covers the whole subtree.

Example 2 [warm-up]

Walk this BST in inorder and write the output. Tree: root 8; 8.left = 3; 8.right = 10; 3.left = 1; 3.right = 6.

Limits $n \leq 10^5$ · **Target** $O(n)$ · **Edge cases** empty tree; a tree with one node.

SOLUTION

Inorder = left, node, right. Start at 8.

1. Go left to 3. Go left to 1. 1 has no left, so print **1**. 1 has no right.
2. Back at 3: print **3**. Go right to 6. Print **6**.
3. Back at 8: print **8**. Go right to 10. Print **10**.

Answer: 1 3 6 8 10 — sorted, as promised

Example 3 [warm-up]

Insert 4, 2, 6, 1, 3 into an empty BST in that order. Draw the shape. Then insert 2 again.

SOLUTION

1. 4 becomes the root.
2. $2 < 4$, go left, empty, place 2.
3. $6 > 4$, go right, empty, place 6.
4. $1 < 4$ go left; $1 < 2$ go left, empty, place 1.
5. $3 < 4$ go left; $3 > 2$ go right, empty, place 3.
6. Insert 2 again: $2 < 4$ go left; we find 2. It is already there. The template does nothing.

Answer: Root 4; left 2 (children 1 and 3); right 6. The second 2 changes nothing.

Example 4 [warm-up]

In the tree from Example 3, what is the smallest key? The biggest?

SOLUTION

The smallest key has nothing smaller than it, so it has no left child. Walk left until you cannot: $4 \rightarrow 2 \rightarrow 1$. The biggest: walk right until you cannot: $4 \rightarrow 6$.

Answer: smallest 1, biggest 6

Two one-line walks

JS

```
const minNode = (t) => { while (t.left !== null) t = t.left; return t; };
const maxNode = (t) => { while (t.right !== null) t = t.right; return t; };
```

Example 5 [warm-up]

A BST holds 20, 30, 40, 50, 60, 70, 80. How many nodes does `bstSearch` visit when looking for 60, if the tree is the balanced one from page 1? What if the same keys were inserted in sorted order instead?

SOLUTION

Balanced tree. $50 \rightarrow 70 \rightarrow 60$. That is 3 nodes.

Sorted insert order. The tree becomes $20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60$, a chain. Searching 60 visits 20, 30, 40, 50, 60 = 5 nodes. For 80 it would visit all 7.

Answer: 3 visits when balanced, 5 when it is a chain

Example 6 [warm-up]

Delete 30 from: root 50; 50.left = 30; 50.right = 70; 30.left = 20; 30.right = 40.

Limits $n \leq 10^5$ · **Target** $O(h)$ · **Edge cases** deleting the root; deleting a key that is not present.

SOLUTION

Node 30 has two children, so we cannot just unhook it. The template does this:

1. Find the **smallest key in the right subtree** of 30. The right subtree is just 40, so that key is 40. (This is 30's inorder successor.)
2. Copy 40 into the node: the node now holds 40.
3. Delete 40 from the right subtree. 40 is a leaf there, so it is unhooked.

Result: root 50; 50.left = 40; 40.left = 20; 50.right = 70.

Answer: Inorder afterwards: 20 40 50 70

Why the **smallest of the right** (or, equally, the biggest of the left)? Because that key is the only one that can sit between “everything on the left” and “everything else on the right” without breaking the rule.

Tier 1 — the standard BST toolkit

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Validate a BST. Given the root of a binary tree, return true if it is a BST.

Limits $n \leq 10^5$, keys up to `Number.MAX_SAFE_INTEGER =  $2^{53} - 1$` · **Target** $O(n)$ time · **Edge cases** empty tree (true); a node whose key equals its parent's key (false); keys at the very edge of the safe-integer range.

Approach 1 — Collect the inorder walk, then check it is sorted ($O(n)$ time, $O(n)$ extra memory)

Brute force — write the keys out and look at them

JS

```
const inorderCollect = (r, out = []) => {
  if (r === null) return out;
  inorderCollect(r.left, out);
  out.push(r.val);
  inorderCollect(r.right, out);
  return out;
};

const isBstBrute = (root) => {
  const a = inorderCollect(root);
  for (let i = 1; i < a.length; i++)
    if (a[i] <= a[i - 1]) return false; // <= rejects duplicate keys too
  return true;
};
```

Time $O(n)$ **Space** $O(n)$

Correct, and a fine first answer. The interviewer will now ask you to drop the array.

Approach 2 — Same walk, but remember only the previous key ($O(n)$ time, $O(1)$ memory)

You never need the whole array — only the last key printed

JS

```
const isBstPrev = (root) => {
  let prev = -Infinity; // -Infinity is below every number
  const walk = (r) => {
    if (r === null) return true;
```



```

    if (!walk(r.left)) return false;
    if (r.val <= prev) return false;
    prev = r.val;
    return walk(r.right);
  };
  return walk(root);
};

```

Time $O(n)$ **Space** $O(h)$

$O(h)$ is the recursion stack. For a balanced tree that is $O(\log n)$.

Approach 3 — Push a legal window down the tree ($O(n)$ time, $O(h)$ memory — the answer to give)

Every node inherits a window (lo, hi) it must fit inside

JS

```

const boundCheck = (r, lo, hi) => {
  if (r === null) return true;
  if (r.val <= lo || r.val >= hi) return false;
  return boundCheck(r.left, lo, r.val)    // left side: hi shrinks to r.val
    && boundCheck(r.right, r.val, hi);    // right side: lo grows to r.val
};

const isBstBounds = (root) => boundCheck(root, -Infinity, Infinity);

```

Time $O(n)$ **Space** $O(h)$

Stops early on the first bad node, which the inorder version cannot always do.

Output of all three on the same inputs

TEXT

```

good tree : true true true
bad tree  : false false false (root 10, right 15, and 15.left = 6)
huge keys (9007199254740991 root, -9007199254740991 left): true true
empty tree: true
equal keys (5 under 5): false false false

```

The idea that unlocked it: the BST rule is not local, it is a **range**. Carrying the range down turns “check the whole subtree” into “check one number”.

TRAP

Start the window at $-\text{Infinity}$ and Infinity , never at `Number.MIN_SAFE_INTEGER` / `Number.MAX_SAFE_INTEGER`. A key sitting exactly on the safe limit would then fail its own test. $-\text{Infinity}$ is below every number and Infinity is above every number, so the window can never be too tight. This is one place where JS is **easier** than a fixed-width language — there is no “one below the minimum” problem.

TRAP

Keys past $2^{53} - 1$ are not safe. JS numbers are doubles, so 9007199254740993 is stored as 9007199254740992 — two different keys become equal and the BST silently loses one. If the problem allows 64-bit keys, store `BigInt` values and compare them with the same $<$ and $>$ (they work), but never mix a `BigInt` and a `Number` in arithmetic.

Example 8 [Infosys · pattern]

Floor and ceil. For a query x , find the biggest key $\leq x$ (floor) and the smallest key $\geq x$ (ceil).

Limits $n \leq 10^5$, up to 10^5 queries · **Target** $O(h)$ per query · **Edge cases** x smaller than every key (no floor); x bigger than every key (no ceil).

Approach 1 — Inorder into an array, then binary search ($O(n)$ once, then $O(\log n)$ per query — good if the tree never changes)

Approach 2 — Walk down and keep the best candidate ($O(h)$, no extra memory — the answer)

SOLUTION

Walking down, every time you move **right** you have just passed a key that is $\leq x$. That key is the best floor found so far. Symmetrically, every time you move **left** you pass a key that is $\geq x$, the best ceil so far.

Floor and ceil in one downward walk

JS

```
const bstFloor = (r, x) => { // biggest value <= x, or null if none
  let best = null;
  while (r !== null) {
    if (r.val === x) return x;
    if (r.val < x) { best = r.val; r = r.right; } // r.val is a candidate
    else { r = r.left; }
  }
  return best;
};

const bstCeil = (r, x) => { // smallest value >= x, or null if none
  let best = null;
  while (r !== null) {
    if (r.val === x) return x;
    if (r.val > x) { best = r.val; r = r.left; } // r.val is a candidate
    else { r = r.right; }
  }
  return best;
};
```

Time $O(h)$ **Space** $O(1)$

Output on the tree {20, 8, 22, 4, 12, 10, 14}

TEXT

```
x=5   floor=4   ceil=8
x=8   floor=8   ceil=8
x=21  floor=20  ceil=22
x=3   floor=none ceil=4
x=100 floor=22  ceil=none
```

Example 9 [Wipro · pattern]

Kth smallest key. Return the k -th smallest key (k is 1-based).

Limits $n \leq 10^5$, $1 \leq k \leq n$ · **Target** $O(h + k)$ · **Edge cases** $k = 1$; k bigger than the number of nodes.

Approach 1 — Full inorder into an array, take $a[k-1]$ ($O(n)$ time and $O(n)$ memory)

Approach 2 — Inorder that stops the moment it has counted k ($O(h + k)$ time, $O(h)$ memory)

Counting walk — quits as soon as it has seen k keys

JS

```
const kthSmallest = (root, k) => {
  let seen = 0, out = null;           // null means "k was too big"
  const walk = (r) => {
    if (r === null || seen >= k) return;
    walk(r.left);
    if (seen >= k) return;
    seen++;
    if (seen === k) { out = r.val; return; }
    walk(r.right);
  };
  walk(root);
  return out;
};
```

Time $O(h + k)$ **Space** $O(h)$

Approach 3 — Store a subtree size in every node ($O(h)$ per query — pay once at insert time)

Each node knows how many nodes are under it

JS

```
class SNode {
  constructor(val) { this.val = val; this.sz = 1; this.left = null; this.right = null; }
}

const sizeOf = (t) => (t === null ? 0 : t.sz);

const sInsert = (r, v) => {
  if (r === null) return new SNode(v);
  if (v < r.val) r.left = sInsert(r.left, v);
  else if (v > r.val) r.right = sInsert(r.right, v);
  else return r;
  r.sz = 1 + sizeOf(r.left) + sizeOf(r.right);
  return r;
};

const sKth = (r, k) => {
  // O(h), no walking of k nodes
  while (r !== null) {
    const leftCount = sizeOf(r.left);
    if (k === leftCount + 1) return r.val;           // this node is the k-th
    if (k <= leftCount) r = r.left;                 // answer is on the left
    else { k -= leftCount + 1; r = r.right; }       // skip left side plus this node
  }
  return null;
};
```

Time $O(h)$ per query **Space** $O(n)$
One extra number per node. Insert still $O(h)$.

Output on {50,30,70,20,40,60,80}

TEXT

```
counting walk : k=1 -> 20,   k=4 -> 50,   k=7 -> 80
augmented tree: k=1 -> 20,   k=4 -> 50,   k=7 -> 80,   k=8 -> none
```

The idea that unlocked it: “how many keys are smaller than me” is a number you can **store**, not a walk you must repeat. That single extra field turns $O(k)$ into $O(h)$ and is the seed of every order-statistics tree.

Example 10 [Capgemini · pattern]

Lowest common ancestor in a BST. Given two keys a and b that both exist in the tree, find their lowest common ancestor.

Limits $n \leq 10^5$ · **Target** $O(h)$, no extra memory · **Edge cases** $a = b$ (answer is that node); one key is an ancestor of the other.

Approach 1 — Find the root-to-key path for both keys, then compare them ($O(h)$ time, but $O(h)$ memory for the two stored paths)

Approach 2 — Let the keys themselves decide the direction ($O(h)$ time, $O(1)$ memory — the answer)

SOLUTION

In a general binary tree LCA needs a full search. In a BST you can **read the answer off the keys**. Start at the root:

- If both a and b are smaller than the current node, the answer is on the left.
- If both are bigger, the answer is on the right.
- Otherwise the paths to a and b split right here. This node is the LCA.

LCA — the first node that sits between a and b

JS

```
const lcaBST = (r, a, b) => {
  if (a > b) [a, b] = [b, a];           // destructuring swap; now a <= b
  while (r !== null) {
    if (b < r.val) r = r.left;           // both smaller
    else if (a > r.val) r = r.right;     // both bigger
    else return r;                      // a <= r.val <= b : split point
  }
  return null;
};
```

Time $O(h)$ **Space** $O(1)$

Output on {50,30,70,20,40,60,80}

TEXT

$\text{lca}(20, 40) = 30$ $\text{lca}(20, 80) = 50$ $\text{lca}(60, 80) = 70$ $\text{lca}(30, 30) = 30$

Example 11 [Accenture · pattern]

Inorder successor and predecessor. For a key x (which may or may not be in the tree), find the smallest key strictly bigger than x , and the biggest key strictly smaller.

Limits $n \leq 10^5$ · **Target** $O(h)$ · **Edge cases** x is the biggest key (no successor); x is not in the tree at all.

Approach 1 — Inorder into an array, then scan it for x ($O(n)$ time and $O(n)$ memory, and you pay it on every query)

Approach 2 — One walk down, keeping the best candidate ($O(h)$ time, $O(1)$ memory)

SOLUTION

This is floor and ceil with the equality removed. Compare with $>$ instead of $\geq$.

Successor and predecessor

JS

```
const successorOf = (r, x) => {           // smallest key > x, null if none
  let best = null;
```

```

while (r !== null) {
  if (r.val > x) { best = r.val; r = r.left; }
  else          { r = r.right; }
}
return best;
};

const predecessorOf = (r, x) => {           // biggest key < x, null if none
  let best = null;
  while (r !== null) {
    if (r.val < x) { best = r.val; r = r.right; }
    else          { r = r.left; }
  }
  return best;
};

```

Time $O(h)$ **Space** $O(1)$

Output

TEXT

succ(40)=50 succ(80)=none pred(50)=40 pred(20)=none

SHORTCUT

Notice the pattern: **floor, ceil, successor, predecessor, LCA and search are the same five-line walk** with one comparison changed. Learn the walk, not four functions.

Example 12 [Cognizant · pattern]

Range sum. Add up every key in $[lo, hi]$.

Limits $n \leq 10^5$, keys up to 10^9 · **Target** better than $O(n)$ when the range is small · **Edge cases** the range holds no key (answer 0); the range holds every key — check the total against $2^{53} - 1$.

Approach 1 — Visit every node, add it if it is in range ($O(n)$ — correct but wasteful)

Approach 2 — Prune: never enter a subtree that cannot hold a key in range ($O(h + m)$ where m is the number of keys inside)

Pruned range sum

JS

```

const rangeSum = (r, lo, hi) => {
  if (r === null) return 0;
  if (r.val < lo) return rangeSum(r.right, lo, hi); // left side all too small
  if (r.val > hi) return rangeSum(r.left, lo, hi);  // right side all too big
  return r.val + rangeSum(r.left, lo, hi) + rangeSum(r.right, lo, hi);
};

```

Time $O(h + m)$ **Space** $O(h)$

m = how many keys land inside $[lo, hi]$.

Output — note the overflow test

TEXT

```
sum[30,60] = 180
sum[0,100] = 350
sum[81,99] = 0
keys 300000000 ... 1800000000, sum over everything = 6300000000
```

TRAP

6300000000 is far past the 32-bit limit of about 2.1×10^9 — in C++ or Java this is where you would reach for a 64-bit integer. **JS needs no cast here:** a number is a double and stays exact up to $2^{53} - 1 \approx 9 \times 10^{15}$, so a sum of 10^5 keys of size 10^9 (10^{14}) is still exact.

The JS limit is higher, not absent. Past $2^{53} - 1$ the additions start rounding and the answer is quietly wrong — $9007199254740992 + 1$ gives 9007199254740992 . When a sum can reach that far, add with `BigInt`: `0n + BigInt(r.val)`, and print with `.toString()`.

Example 13 [TCS Digital · pattern]

Sorted array to a height-balanced BST. Given a sorted array with no duplicates, build a BST whose left and right heights differ by at most 1 at every node.

Limits $n \leq 10^5$ · **Target** $O(n)$ · **Edge cases** empty array; one element.

Approach 1 — Insert the keys left to right ($O(n^2)$, and it builds a chain — the worst possible shape)

Approach 2 — Put the middle element at the root, then repeat on each half ($O(n)$, and the height is as small as it can be)

SOLUTION

If you want both halves to be the same height, put the **middle** element at the root. Then do the same to each half. That is the whole algorithm.

Middle element becomes the root, recursively

JS

```
const buildBalanced = (a, lo, hi) => {
  if (lo > hi) return null;
  const mid = lo + Math.floor((hi - lo) / 2); // safe midpoint, see the trap
  const root = new Node(a[mid]);
  root.left = buildBalanced(a, lo, mid - 1);
  root.right = buildBalanced(a, mid + 1, hi);
  return root;
};

const sortedToBST = (a) => buildBalanced(a, 0, a.length - 1);
```

Time $O(n)$ **Space** $O(\log n)$

Each element is used exactly once; the space is the recursion stack.

Output

TEXT

```
input [1 2 3 4 5 6 7] -> inorder [1 2 3 4 5 6 7], root 4, height 3
empty input -> null
one element [9] -> root 9, height 1
```

TRAP

Do not write `const mid = (lo + hi) >> 1`. It looks clever and it is the standard trick in C++, but **JS bitwise operators truncate to 32 bits**. With `lo = 2000000000` and `hi = 2100000000`, `(lo + hi) >> 1` returns

-97483648 — a negative index. The safe form is `lo + Math.floor((hi - lo) / 2)`, which never leaves the double range. Build the habit now; every binary search in this book uses it.

Example 14 [Infosys · pattern]

Balance an unbalanced BST. You are handed a BST that has degenerated into a chain. Rebuild it so its height is minimal, keeping the same keys.

Limits $n \leq 10^5$ · **Target** $O(n)$ · **Edge cases** already balanced; one node.

Approach 1 — Rotate repeatedly until it looks balanced (hard to write, harder to prove correct — do not start here)

Approach 2 — Flatten to a sorted array, then rebuild from the middle ($O(n)$ time, $O(n)$ memory — the answer)

SOLUTION

You already have both pieces. Inorder gives you the sorted array; Example 13 turns a sorted array into a balanced tree. Glue them.

Two functions you already wrote

JS

```
const balanceBST = (root) => sortedToBST(inorderCollect(root)); // sorted, because it is a BST
```

Time $O(n)$ Space $O(n)$

Output — insert 1..7 in order, then rebuild

TEXT

skewed height 7 -> balanced height 3

Example 15 [TCS NQT · pattern]

Trim a BST to a range. Delete every key outside $[lo, hi]$, keeping the remaining keys a valid BST.

Limits $n \leq 10^5$ · **Target** $O(n)$ · **Edge cases** every key is outside the range (result is empty); every key is inside (tree unchanged).

Approach 1 — Collect every out-of-range key, then call `bstErase` on each one ($O(nh)$, and each delete reshuffles the tree again)

Approach 2 — One recursive pass that returns the replacement for each node ($O(n)$, every node touched once)

SOLUTION

If `r.val < lo`, then `r` and its whole left subtree are too small. The answer is whatever trimming the right subtree gives. Mirror it for the other side.

Trim — return the replacement for this node

JS

```
const trimBST = (r, lo, hi) => {
  if (r === null) return null;
  if (r.val < lo) return trimBST(r.right, lo, hi);
  if (r.val > hi) return trimBST(r.left, lo, hi);
  r.left = trimBST(r.left, lo, hi);
  r.right = trimBST(r.right, lo, hi);
  return r;
};
```

Time $O(n)$ Space $O(h)$

Output

TEXT

```
{50,30,70,20,40,60,80} trimmed to [35,65] -> [40 50 60]
{5,3,8} trimmed to [100,200] -> null
```

Example 16 [Capgemini · pattern]**Count how many keys lie in [lo, hi].****Limits** $n \leq 10^5$, 10^5 queries · **Target** $O(h + m)$ per query · **Edge cases** empty range; $lo > hi$ (answer 0).**Approach 1 — Walk the whole tree and test every key** ($O(n)$ per query)**Approach 2 — Prune any subtree that cannot hold a key in range** ($O(h + m)$)

Same pruning shape as the range sum

JS

```
const countInRange = (r, lo, hi) => {
  if (r === null) return 0;
  if (r.val < lo) return countInRange(r.right, lo, hi);
  if (r.val > hi) return countInRange(r.left, lo, hi);
  return 1 + countInRange(r.left, lo, hi) + countInRange(r.right, lo, hi);
};
```

Time $O(h + m)$ **Space** $O(h)$

Output on {50,30,70,20,40,60,80}

TEXT

```
count[30,70] = 5    count[0,100] = 7    count[41,49] = 0
```

If the queries keep coming and the tree never changes, do the inorder walk once into an array and answer each query with two toolkit `lowerBound` calls in $O(\log n)$. Choose by how many queries you expect.

Tier 2 — applied problems**TIER 2**

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Grab · pattern]**Closest fare.** A ride app stores every historical fare in a BST. Given a target fare, return the stored fare closest to it. If two are equally close, return the smaller one.**Limits** $n \leq 2 \times 10^5$ fares, values up to 10^6 · **Target** $O(h)$ · **Edge cases** target smaller than every fare; an exact match.**Approach 1 — Walk the whole tree, keeping the smallest difference seen** ($O(n)$ — it ignores the ordering completely)**Approach 2 — One walk down; only one side can ever hold something closer** ($O(h)$ time, $O(1)$ memory)**SOLUTION**

Walk down. At every node, the current key is a candidate; then the BST rule tells you which side can hold anything closer. Only one side can.

Closest value — track the best while you descend

JS


```
const closestValue = (r, target) => {
  let best = r.val;
  while (r !== null) {
    if (Math.abs(r.val - target) < Math.abs(best - target)) best = r.val;
    if (target < r.val) r = r.left;
    else if (target > r.val) r = r.right;
    else return r.val;
  }
  return best;
};
```

Time $O(h)$ **Space** $O(1)$

Output on {50,30,70,20,40,60,80}

TEXT

```
closest to 55    -> 50    (50 and 60 are both 5 away; strict < keeps the first, 50)
closest to 1     -> 20
closest to 1000  -> 80
closest to 40    -> 40
```

TRAP

The tie rule is decided by $<$ versus $\leq$. With strict $<$, the first candidate wins. On the path $50 \rightarrow 70 \rightarrow 60$ the first candidate is 50, so 50 wins. If the question wanted the **bigger** fare on a tie, you would need $\leq$ **and** to visit 60 after 50 — read the statement carefully, then pick the comparison to match.

Example 18 [Shopee · pattern]

Two prices that add to a budget. Given a BST of prices and a budget T , is there a pair of distinct nodes whose prices add to exactly T ?

Limits $n \leq 2 \times 10^5$, prices up to 10^9 · **Target** $O(n)$ time and better than $O(n)$ memory · **Edge cases** one node only; negative values (refunds).

Approach 1 — Flatten to an array and use a hash set ($O(n)$ time, $O(n)$ memory)

The obvious one

JS

```
const twoSumSet = (root, target) => {
  const seen = new Set(); // a Set, not an object: keys keep their
  type
  for (const x of inorderCollect(root)) {
    if (seen.has(target - x)) return true;
    seen.add(x);
  }
  return false;
};
```

Time $O(n)$ **Space** $O(n)$

Approach 2 — Two iterators — one from the smallest, one from the biggest ($O(n)$ time, $O(h)$ memory)

| SOLUTION

On a **sorted array** you would use two pointers. The inorder walk is a sorted array; you just cannot index into it. So build two lazy iterators: one that yields keys in increasing order and one in decreasing order, each using a stack of size h .

Forward and backward iterators, then classic two pointers

JS

```
class BSTIterator { // yields keys smallest first
  constructor(root) { this.st = []; this.pushLeft(root); }
  pushLeft(r) { while (r !== null) { this.st.push(r); r = r.left; } }
  hasNext() { return this.st.length > 0; }
  next() { const cur = this.st.pop(); this.pushLeft(cur.right); return cur.val; }
}

class RevIterator { // yields keys biggest first
  constructor(root) { this.st = []; this.pushRight(root); }
  pushRight(r) { while (r !== null) { this.st.push(r); r = r.right; } }
  hasNext() { return this.st.length > 0; }
  next() { const cur = this.st.pop(); this.pushRight(cur.left); return cur.val; }
}

const twoSumTwoPointer = (root, target) => {
  const lo = new BSTIterator(root), hi = new RevIterator(root);
  if (!lo.hasNext() || !hi.hasNext()) return false;
  let a = lo.next(), b = hi.next();
  while (a < b) { // a < b keeps the two nodes distinct
    const s = a + b;
    if (s === target) return true;
    if (s < target) { if (!lo.hasNext()) break; a = lo.next(); }
    else { if (!hi.hasNext()) break; b = hi.next(); }
  }
  return false;
};
```

Time $O(n)$ Space $O(h)$

Output

TEXT

```
hash set      : target 90 -> yes, target 200 -> no
two pointers: target 90 -> yes, target 200 -> no, target 100 -> yes
single node {5}, target 10 -> no    (a node cannot pair with itself)
negatives {-5,-2,0,3}, target -7 -> yes
```

The idea that unlocked it: an iterator is a pointer you can move but not index. Every two-pointer trick that only ever moves **forward** or **backward** works on a BST unchanged.

Example 19 [Agoda · pattern]

Design an iterator. Build a class over a BST with `hasNext()` and `next()` that returns keys in increasing order, using $O(h)$ memory and $O(1)$ **average** time per call.

Limits $n \leq 10^5$, up to n calls · **Target** $O(h)$ memory · **Edge cases** empty tree; calling `next()` exactly n times.

Approach 1 — Run a full inorder in the constructor and store the array ($O(n)$ memory, and the constructor costs $O(n)$ even if the caller stops after one `next()`)

Approach 2 — Keep only the left spine on a stack ($O(h)$ memory, $O(1)$ amortised per call)

| SOLUTION

The `BSTIterator` above is the answer. Why is `next()` $O(1)$ on average? Over the whole walk each node is pushed once and popped once, so n calls do $2n$ stack operations in total. Single calls can cost $O(h)$; the **average** is $O(1)$. That is amortised analysis (Chapter 1).

Time $O(1)$ amortised per call **Space** $O(h)$

Output

TEXT

```
walk: 20 30 40 50 60 70 80
empty tree -> hasNext() = false
```

Example 20 [DBS · pattern]

Merge two account ledgers. Two BSTs hold account numbers. Produce one sorted list of all account numbers, keeping duplicates that appear in both.

Limits both trees up to 10^5 nodes · **Target** $O(n + m)$ time, $O(h_1 + h_2)$ extra · **Edge cases** one tree empty; both empty.

Approach 1 — Flatten both, concatenate, sort ($O((n+m) \log(n+m))$) — throws away the fact that both are already sorted)

Approach 2 — Flatten both, merge like merge sort ($O(n+m)$ time but $O(n+m)$ memory)

Approach 3 — Two iterators, merge on the fly ($O(n+m)$ time, $O(h_1+h_2)$ memory)

Merge with two lazy iterators

JS

```
const mergeBSTs = (a, b) => {
  const ia = new BSTIterator(a), ib = new BSTIterator(b);
  const out = [];
  let x = ia.hasNext() ? ia.next() : null;
  let y = ib.hasNext() ? ib.next() : null;
  while (x !== null && y !== null) {
    if (x <= y) { out.push(x); x = ia.hasNext() ? ia.next() : null; }
    else       { out.push(y); y = ib.hasNext() ? ib.next() : null; }
  }
  while (x !== null) { out.push(x); x = ia.hasNext() ? ia.next() : null; }
  while (y !== null) { out.push(y); y = ib.hasNext() ? ib.next() : null; }
  return out;
};
```

Time $O(n + m)$ **Space** $O(h_1 + h_2)$ plus the output

Output

TEXT

```
{5,2,9} + {6,1,8} -> [1 2 5 6 8 9]
one empty      -> [2 5 9]
both empty     -> []
```

Follow-up the interviewer will ask: “now give me a **balanced BST** of the union, not a list.” Answer: feed the merged array into `sortedToBST`. Total $O(n + m)$.

Example 21 [SCB · pattern]

Build a BST from a preorder list. A backup file stores the tree as its preorder walk. Rebuild the tree.

Limits $n \leq 10^5$, keys distinct · **Target** $O(n)$ · **Edge cases** empty list; strictly increasing list (a right chain).

Approach 1 — Insert the keys one by one $O(nh)$ — $O(n^2)$ on a sorted list

Brute force

JS

```
const preToBstInsert = (pre) => pre.reduce((root, x) => bstInsert(root, x), null);
```

Time $O(nh)$ **Space** $O(h)$

With pre = 1,2,3,...,n this is 5×10^9 steps for $n = 10^5$. Too slow.

Approach 2 — Carry a legal window, exactly like validating a BST $O(n)$ — the answer

SOLUTION

Preorder visits the root first. So `pre[0]` is the root. Everything after it that is smaller belongs to the left subtree, and the rest to the right. Instead of **searching** for where the split is, give each recursive call a window (`lo`, `hi`) and stop as soon as the next key falls outside it. Every key is read once.

One pass, one shared index

JS

```
const preToBstBounds = (pre) => {
  let i = 0; // one shared cursor, closed over
  const build = (lo, hi) => {
    if (i >= pre.length) return null;
    if (pre[i] < lo || pre[i] > hi) return null; // not mine – give it back
    const r = new Node(pre[i++]);
    r.left = build(lo, r.val);
    r.right = build(r.val, hi);
    return r;
  };
  return build(-Infinity, Infinity);
};
```

Time $O(n)$ **Space** $O(h)$

Approach 3 — A stack instead of recursion $O(n)$ time, $O(h)$ memory, no recursion at all

Stack version — useful when the tree may be 100000 deep

JS

```
const preToBstStack = (pre) => {
  if (pre.length === 0) return null;
  const root = new Node(pre[0]);
  const st = [root];
  for (let i = 1; i < pre.length; i++) {
    const cur = new Node(pre[i]);
    if (pre[i] < st[st.length - 1].val) {
      st[st.length - 1].left = cur;
    } else {
      let parent = null;
      while (st.length && st[st.length - 1].val < pre[i]) parent = st.pop();
      parent.right = cur;
    }
    st.push(cur);
  }
  return root;
};
```

Time $O(n)$ **Space** $O(h)$

Each node is pushed once and popped at most once.

All three agree — pre = {8, 5, 1, 7, 10, 12}

TEXT

```
insert one by one : inorder [1 5 7 8 10 12], root 8
bounds           : inorder [1 5 7 8 10 12], root 8
stack            : inorder [1 5 7 8 10 12], root 8
empty preorder   : null
```

The idea that unlocked it: the window (lo, hi) is the same idea as validating a BST. Once you can **check** a tree with a window, you can **build** one with it.

Example 22 [Sea/Shopee · pattern]

Is this list a valid preorder of some BST? No tree is given — only the list.

Limits $n \leq 10^5$ · **Target** $O(n)$ time, $O(n)$ memory · **Edge cases** empty list (true); one key (true).

Approach 1 — Build a BST from the list, then check its preorder matches ($O(n \log n)$), and it needs the whole tree in memory)

Approach 2 — One stack plus a running floor value — never build the tree ($O(n)$ time, $O(n)$ memory)

SOLUTION

Walk the list keeping a stack of “ancestors I am still inside” and a value floorVal: the key of the last ancestor I moved **right** of. Once I have gone right of some node p, every later key must be bigger than p. If any key is smaller, the list is impossible.

One stack, one floor value

JS

```
const validPreorder = (pre) => {
  const st = [];
  let floorVal = -Infinity;
  for (const x of pre) {
    if (x < floorVal) return false;
    while (st.length && st[st.length - 1] < x) floorVal = st.pop();
    st.push(x);
  }
  return true;
};
```

Time $O(n)$ **Space** $O(n)$

Output

TEXT

```
{8,5,1,7,10,12} -> valid
{8,10,5}        -> not valid   (after going right of 8 to 10, 5 cannot appear)
{}              -> valid
{4}             -> valid
```

Example 23 [GIC · pattern]

Serialise and rebuild. Write a BST to a string, and read it back. The string must be as short as you can make it.

Limits $n \leq 10^5$ · **Target** $O(n)$ both ways · **Edge cases** empty tree; one node.

Approach 1 — Write the preorder WITH null markers, as for a general binary tree (correct, but the string is about twice as long)

Approach 2 — Write the keys only, and rebuild with the window rule (the shortest string a BST can have)

SOLUTION

For a **general** binary tree you must write null markers. For a **BST** you do not: the preorder list alone fixes the shape, because the window rule tells you where each subtree ends. So the string is just the preorder keys with spaces.

Serialise = preorder; deserialise = the window build

JS

```
const serializeBST = (root) => {
  const out = [];
  const walk = (r) => { if (r === null) return; out.push(r.val); walk(r.left);
walk(r.right); };
  walk(root);
  return out.join(' ');
};

const deserializeBST = (s) => {
  const pre = s.split(' ').filter(t => t.length).map(Number);
  let i = 0;
  const build = (lo, hi) => {
    if (i >= pre.length || pre[i] < lo || pre[i] > hi) return null;
    const r = new Node(pre[i++]);
    r.left = build(lo, r.val);
    r.right = build(r.val, hi);
    return r;
  };
  return build(-Infinity, Infinity);
};
```

Time $O(n)$ **Space** $O(n)$

Output

TEXT

```
serialise {50,30,70,20,40,60,80} -> "50 30 20 40 70 60 80 "
rebuild -> inorder [20 30 40 50 60 70 80], root 50
empty round trip: "" -> null
```

Tier 3 — product interviews

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Microsoft · pattern]

Repair a BST. Exactly two nodes of a BST had their keys swapped by mistake. Put them back without building a new tree.

Limits $n \leq 10^5$ · **Target** $O(n)$ time, $O(h)$ memory · **Edge cases** the two swapped nodes are neighbours in the inorder walk; the swapped pair includes the root.

Approach 1 — Copy the inorder, sort the copy, compare position by position, swap what differs ($O(n \log n)$ time and $O(n)$ memory)

Approach 2 — Spot the drops during a single inorder walk ($O(n)$ time, $O(h)$ memory — the answer)

SOLUTION

Do the inorder walk and look for places where the sequence goes **down** instead of up.

- If the two wrong keys are far apart, you see **two** drops. The first wrong key is the **earlier** node of the first drop; the second is the **later** node of the second drop.
- If they are neighbours, you see only **one** drop, and the two nodes of that drop are the pair.

The code below handles both because `first` is set only once, while `second` keeps being overwritten.

Find the two drops, then swap the keys

JS

```
const recoverBST = (root) => {
  let first = null, second = null, prev = null;
  const walk = (r) => {
    if (r === null) return;
    walk(r.left);
    if (prev !== null && prev.val > r.val) {
      if (first === null) first = prev; // only the first drop sets first
      second = r;                     // every drop updates second
    }
    prev = r;
    walk(r.right);
  };
  walk(root);
  if (first !== null && second !== null)
    [first.val, second.val] = [second.val, first.val]; // swap the two keys
};
```

Time $O(n)$ Space $O(h)$

Output — both cases

TEXT

```
far apart : broken [5 30 15 20 10] -> fixed [5 10 15 20 30], valid = true
neighbours: broken [1 3 2 4]       -> fixed [1 2 3 4],      valid = true
```

The follow-up

“Now do it in $O(1)$ extra space.”

Answer: replace the recursion with a **Morris inorder traversal**. Morris threads each node’s rightmost left-descendant back to the node, walks without a stack, then unthreads. The drop-detection logic above does not change at all — only the traversal does. Say this out loud; you do not need to write Morris from memory to get the credit.

Example 25 [Amazon · pattern]

Largest BST inside a binary tree. You are given a binary tree that is **not** a BST. Find the number of nodes in its biggest subtree that is a BST.

Limits $n \leq 10^5$ · **Target** $O(n)$ · **Edge cases** the whole tree is already a BST; an empty tree.

Approach 1 — For every node, test whether its subtree is a BST ($O(n^2)$ — each test is $O(\text{size})$)

Approach 2 — One bottom-up pass; each node returns a tiny summary ($O(n)$ — the answer)

SOLUTION

A node's subtree is a BST exactly when: the left subtree is a BST, the right subtree is a BST, $\max(\text{left}) < \text{node} < \min(\text{right})$. So each call needs to return four things: is it a BST, its size, its minimum and its maximum. Compute them from the children — never walk down twice.

Return a 4-field summary from every node

JS

```
const largestBSTSubtree = (root) => {
  let best = 0;
  // every call returns { ok, sz, mn, mx } about its own subtree
  const walk = (r) => {
    if (r === null) return { ok: true, sz: 0, mn: Infinity, mx: -Infinity };
    const L = walk(r.left), R = walk(r.right);
    if (L.ok && R.ok && L.mx < r.val && r.val < R.mn) {
      const cur = { ok: true, sz: L.sz + R.sz + 1,
                    mn: Math.min(L.mn, r.val), mx: Math.max(R.mx, r.val) };
      best = Math.max(best, cur.sz);
      return cur;
    }
    return { ok: false, sz: 0, mn: 0, mx: 0 };
  };
  walk(root);
  return best;
};
```

Time $O(n)$ Space $O(h)$

Output

TEXT

```
tree 10(5(1,8), 15(-,7)) -> largest BST has 3 nodes (the subtree 5(1,8))
tree that is fully a BST -> 7
empty tree -> 0
```

TRAP

The empty case returns $\text{mn} = \text{Infinity}$, $\text{mx} = -\text{Infinity}$. That looks backwards, and that is the point: an empty subtree must never fail the test $\text{L.mx} < \text{r.val}$. Setting $\text{mx} = -\text{Infinity}$ makes the comparison always pass, whatever the key is. C++ would use `INT_MIN` here and break on a key equal to `INT_MIN`; $-\text{Infinity}$ has no such hole.

The follow-up

“Now return the largest sum instead of the largest count, and allow negative keys.”

Same walk, one field changes: carry a running sum instead of the subtree size `sz`. No numeric type to widen — a JavaScript number already holds 10^5 keys of size 10^9 .

Maximum-sum BST subtree

JS

```
const maxSumBST = (root) => {
  let best = 0;
  const walk = (r) => {
    if (r === null) return { ok: true, sum: 0, mn: Infinity, mx: -Infinity };
    const L = walk(r.left), R = walk(r.right);
    if (L.ok && R.ok && L.mx < r.val && r.val < R.mn) {
      const cur = { ok: true, sum: L.sum + R.sum + r.val,
                    mn: Math.min(L.mn, r.val), mx: Math.max(R.mx, r.val) };
      best = Math.max(best, cur.sum);
      return cur;
    }
    return { ok: false, sum: 0, mn: 0, mx: 0 };
  };
  walk(root);
  return best;
};
```



```

        best = Math.max(best, cur.sum);
        return cur;
    }
    return { ok: false, sum: 0, mn: 0, mx: 0 };
};
walk(root);
return best;
};

```

Output

TEXT

```

tree 1(40(20,60), 3)      -> 120    (the subtree 40(20,60))
a full BST {50,...,80}    -> 350
empty tree                -> 0
all-negative BST {-5,-8,-2} -> 0    (the empty subtree, sum 0, wins)

```

TRAP

That last line is a **specification** question, not a bug. `best` starts at 0, so an empty subtree is always allowed. If the question says the subtree must be non-empty, start `best` at `LLONG_MIN` and update it only for non-empty subtrees. Ask the interviewer which one they want.

Example 26 [Google · pattern]

Split a BST. Given a BST and a value `t`, split it into two BSTs: one with every key $\leq t$, one with every key $> t$. Reuse the existing nodes.

Limits $n \leq 10^5$ · **Target** $O(h)$ · **Edge cases** every key $\leq t$; empty tree.

Approach 1 — Walk every node and insert its key into one of two fresh trees ($O(nh)$, and it throws away every existing node)

Approach 2 — Follow the search path and re-hang only what it crosses ($O(h)$ — the answer)

SOLUTION

Follow the search path for `t`. At a node with `val <= t`, the node and its **whole left subtree** belong to the low tree; only its right subtree needs splitting, and the low part of that split becomes the node's new right child.

Split — $O(h)$, only the search path is touched

JS

```

const splitBST = (r, target) => { // returns [keys <= target, keys > target]
    if (r === null) return [null, null];
    if (r.val <= target) {
        const [lowPart, highPart] = splitBST(r.right, target);
        r.right = lowPart;
        return [r, highPart];
    }
    const [lowPart, highPart] = splitBST(r.left, target);
    r.left = highPart;
    return [lowPart, r];
};

```

Time $O(h)$ **Space** $O(h)$

Output

TEXT

```
{50,30,70,20,40,60,80} split at 45 -> low [20 30 40]  high [50 60 70 80]
{5,3,8} split at 100                -> low [3 5 8]    high = null
```

The follow-up

“Now join them back.” If every key of tree A is smaller than every key of tree B, you can join in $O(h)$: take the biggest node of A, make it the new root, hang A-without-it on the left and B on the right. This pair of operations — split and join — is the engine inside a treap and inside a rope.

Example 27 [Goldman Sachs · pattern]

Count smaller trades to the right. Given an array of trade sizes, for each position i report how many **later** positions hold a strictly smaller size.

Limits $n \leq 10^5$, values may repeat and may be negative · **Target** $O(n \log n)$ average · **Edge cases** all values equal (all zeros); a strictly decreasing array.

Approach 1 — Two loops ($O(n^2)$ — 10^{10} steps at $n = 10^5$, far too slow)

Brute force, kept only as a checker

JS

```
const countSmallerRightBrute = (a) => {
  const n = a.length, out = new Array(n).fill(0);
  for (let i = 0; i < n; i++)
    for (let j = i + 1; j < n; j++)
      if (a[j] < a[i]) out[i]++;
  return out;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Insert from the right into a size-augmented BST ($O(n \log n)$ — $O(n \log n)$ on random data)

SOLUTION

Process the array **right to left**. Before inserting $a[i]$, everything already in the tree is exactly “the elements to the right of i ”. While $a[i]$ walks down to its place, count what it passes:

- moving **left**: the current node and its right subtree are all bigger — count nothing, but bump that node’s `leftSize` because a new smaller key joined its left side;
- moving **right**: every key in the current node’s left subtree, plus the copies of the current node itself, are smaller — add them to a running carry.

Each node stores `leftSize` and a duplicate count

JS

```
class CNode {
  constructor(val) {
    this.val = val;
    this.cnt = 1;      // copies of this key
    this.leftSize = 0; // how many smaller keys sit on the left
    this.left = null; this.right = null;
  }
}

const countSmallerRight = (a) => {
  let root = null;
  const out = new Array(a.length).fill(0);
  // insert v, returning [newSubtree, howManySmallerItPassed]
```

```

const insert = (r, v, carry) => {
  if (r === null) return [new CNode(v), carry];
  if (v === r.val) { r.cnt++; return [r, carry + r.leftSize]; }
  if (v < r.val) {
    r.leftSize++;
    const [child, res] = insert(r.left, v, carry);
    r.left = child;
    return [r, res];
  }
  const [child, res] = insert(r.right, v, carry + r.leftSize + r.cnt);
  r.right = child;
  return [r, res];
};

for (let i = a.length - 1; i >= 0; i--) {
  const [newRoot, res] = insert(root, a[i], 0);
  root = newRoot;
  out[i] = res;
}
return out;
};

```

Time $O(nh)$ **Space** $O(n)$

h is $\log n$ on random input, n in the worst case (already sorted input).

Output — checked against the brute force on every case

TEXT

```

[5 2 6 1]    -> [2 1 1 0]   (matches brute)
[]           -> []          (matches brute)
[7]          -> [0]         (matches brute)
[2 2 2]      -> [0 0 0]     (matches brute)
[-1 -5 0 -5] -> [2 0 1 0]   (matches brute)

```

The follow-up

“The input is already sorted descending — your tree becomes a chain.” True: the plain BST degrades to $O(n^2)$. Three honest answers, in increasing order of effort:

1. Replace the plain BST with a **balanced** one (an AVL or a treap — see Example 28).
2. Use a Fenwick tree over the **compressed** values. Sort the distinct values, map each to an index, then “count smaller” is a prefix sum. Guaranteed $O(n \log n)$.
3. Use a merge sort that counts inversions as it merges. Also $O(n \log n)$, no extra structure.

Example 28 [Adobe · pattern]

Keep the tree short. Explain and implement one way to stop a BST degenerating. Show that inserting 1, 2, 3, ..., 1000 does not produce a chain.

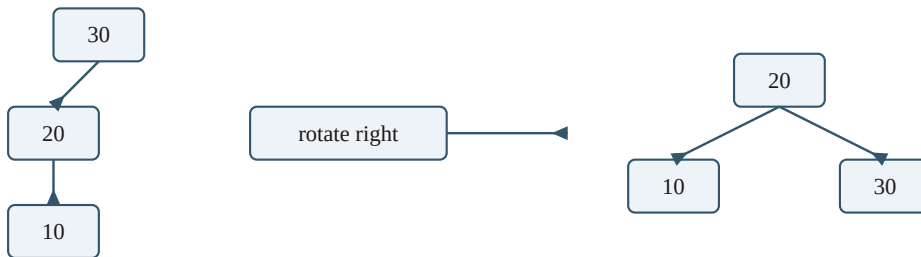
Limits $n \leq 10^5$ inserts · **Target** $O(\log n)$ per insert · **Edge cases** sorted input; duplicate keys.

Option A — AVL: measure the height, rotate when it slips

THE AVL RULE

Store a **height** in every node. Define $\text{balance}(x) = \text{height}(x.\text{left}) - \text{height}(x.\text{right})$. After every insert, walk back up the path and fix any node with $|\text{balance}| > 1$ using a rotation. Four cases:

Case	What it looks like	Fix
LL	balance > 1 and the new key went left of the left child	rotate right
RR	balance < -1 and the new key went right of the right child	rotate left
LR	balance > 1 and the new key went right of the left child	rotate left on the left child, then rotate right
RL	balance < -1 and the new key went left of the right child	rotate right on the right child, then rotate left



The LL case. Insert 30, 20, 10. The chain leans left, so rotate right at 30: node 20 becomes the root.

A complete AVL insert

JS

```

class ANode {
  constructor(val) { this.val = val; this.h = 1; this.left = null; this.right = null; }
}

const ht = (t) => (t === null ? 0 : t.h);
const fix = (t) => { t.h = 1 + Math.max(ht(t.left), ht(t.right)); };
const balanceOf = (t) => (t === null ? 0 : ht(t.left) - ht(t.right));

const rotateRight = (y) => {
  const x = y.left, T = x.right;
  x.right = y; y.left = T;
  fix(y); fix(x);           // fix the lower node first
  return x;                 // x is the new subtree root
};

const rotateLeft = (x) => {
  const y = x.right, T = y.left;
  y.left = x; x.right = T;
  fix(x); fix(y);
  return y;
};

const avlInsert = (r, v) => {
  if (r === null) return new ANode(v);
  if (v < r.val) r.left = avlInsert(r.left, v);
  else if (v > r.val) r.right = avlInsert(r.right, v);
  else return r;           // duplicate: nothing to do
  fix(r);
  const b = balanceOf(r);
  if (b > 1 && v < r.left.val) return rotateRight(r);           // LL

```

```

    if (b < -1 && v > r.right.val) return rotateLeft(r);           // RR
    if (b > 1 && v > r.left.val) {                                   // LR
        r.left = rotateLeft(r.left); return rotateRight(r);
    }
    if (b < -1 && v < r.right.val) {                               // RL
        r.right = rotateRight(r.right); return rotateLeft(r);
    }
    return r;
};

```

Time $O(\log n)$ per insert **Space** $O(\log n)$
 An AVL tree of n nodes has height below $1.44 \log_2(n)$.

Output — the proof that it works

TEXT

```

insert 1..7 in order -> inorder [1 2 3 4 5 6 7], root 4, height 3
{30,20,10} (LL case) -> root 20, height 2
{30,10,20} (LR case) -> root 20, height 2
insert 1..1000 in order -> AVL height 10   (a plain BST would be 1000)

```

Option B — a treap: let randomness do the balancing

SOLUTION

Give every node a **random priority** as well as its key. Keep the keys in BST order **and** the priorities in heap order (a parent's priority is smaller than both children's). After inserting normally, rotate the new node upward while its priority beats its parent's.

Because the priorities are random, the shape is the shape of a BST built from a **random** insertion order — expected height about $1.39 \log_2 n$ — no matter what order the keys arrive in. It is twenty lines instead of AVL's sixty.

A treap: BST on the key, min-heap on the priority

JS

```

class TNode {
    constructor(val) { this.val = val; this.pri = Math.random(); this.l = null; this.r = null; }
}

const tRotR = (y) => { const x = y.l; y.l = x.r; x.r = y; return x; };
const tRotL = (x) => { const y = x.r; x.r = y.l; y.l = x; return y; };

const tInsert = (r, v) => {
    if (r === null) return new TNode(v);
    if (v < r.val) {
        r.l = tInsert(r.l, v);
        if (r.l.pri < r.pri) r = tRotR(r);           // child outranks parent
    } else if (v > r.val) {
        r.r = tInsert(r.r, v);
        if (r.r.pri < r.pri) r = tRotL(r);
    }
    return r;
};

```

Time $O(\log n)$ expected **Space** $O(n)$

Output — 1000 keys inserted in sorted order

TEXT

```
n=1000  inorder is sorted: yes    height = 23    (a plain BST would be 1000)
```

23 is bigger than the AVL's 10, because a treap only promises a good height **on average**. Both are $O(\log n)$. Pick AVL when you need a guarantee, a treap when you need to write it quickly and correctly under time pressure.

The follow-up

“Which one does a real library use?” A red-black tree: a different balance rule (colour-based instead of height-based) with the same $O(\log n)$ guarantee, and fewer rotations per insert. C++'s `ordered set` and `map` and Java's `TreeMap` are all red-black trees. In an interview you are not expected to code one. You are expected to know that they are ordered, $O(\log n)$, and safe on sorted input — and that JavaScript ships none of them, so you reach for a sorted array or hand-write the tree.

Example 29 [D. E. Shaw · pattern]

BST to a sorted doubly linked list, in place. Convert a BST into a sorted doubly linked list using the existing nodes. Use `left` as `prev` and `right` as `next`.

Limits $n \leq 10^5$ · **Target** $O(n)$ time, $O(h)$ memory, no new nodes · **Edge cases** empty tree; one node (its `left` and `right` must both end up null).

Approach 1 — Inorder into a vector, then relink the nodes in a second pass ($O(n)$ time but $O(n)$ extra memory)

Approach 2 — Link each node as the inorder walk leaves it ($O(n)$ time, $O(h)$ memory, no array at all)

SOLUTION

Do the inorder walk, and as you **leave** each node, link it to the previously finished node. The single trap: `r.right` is the next thing you must visit, but you are about to overwrite it. Save it first.

Save the right pointer before you destroy it

JS

```
const bstToDLL = (root) => {
  let head = null, tail = null;
  const walk = (r) => {
    if (r === null) return;
    walk(r.left);
    const nxt = r.right;           // SAVE – we are about to overwrite it
    if (tail === null) { head = r; r.left = null; }
    else { tail.right = r; r.left = tail; }
    tail = r;
    r.right = null;
    walk(nxt);
  };
  walk(root);
  return head;
};
```

Time $O(n)$ **Space** $O(h)$

Output on {4,2,6,1,3,5,7}

TEXT

```
forward : 1 2 3 4 5 6 7
backward: 7 6 5 4 3 2 1
```

The follow-up

“Now go the other way: sorted doubly linked list back to a balanced BST, in $O(n)$.”

Count the nodes, then build **bottom-up**: recursively build the left half, take the next node from the list as the root, then build the right half. Each list node is consumed once, in order, so it is $O(n)$ and never walks the list twice.

Example 30 [Uber · pattern]

How many different BSTs? With the keys $1..n$, how many **structurally different** BSTs exist?

Limits $n \leq 30$ (the answer for $n = 30$ is about 3.8×10^{15}) · **Target** $O(n^2)$ · **Edge cases** $n = 0$ (one tree: the empty one); the answer overflowing 32-bit int from $n = 20$ onward.

Approach 1 — Generate every shape and count them (exponential — there are 4861946401452 of them at $n = 25$)

Approach 2 — Choose the root, then multiply the counts of the two sides ($O(n^2)$ dynamic programming)

SOLUTION

Pick the root. If the root is key k , then keys $1..k-1$ form the left subtree and $k+1..n$ the right subtree, and the two choices are independent. So with $dp[m]$ = number of BST shapes on m keys:

$$dp[m] = \sum_{k=1}^m dp[k-1] \times dp[m-k], \quad dp[0] = 1$$

These are the Catalan numbers.

Catalan by dynamic programming — exact only up to $n = 30$

JS

```
const countBSTShapes = (n) => {
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;
  for (let k = 1; k <= n; k++)
    for (let left = 0; left < k; left++)
      dp[k] += dp[left] * dp[k - 1 - left];
  return dp[n];
};

// From n = 31 the count passes 2^53 - 1, so the plain version starts lying.
// Same three lines, BigInt literals (0n, 1n) everywhere:
const countBSTShapesBig = (n) => {
  const dp = new Array(n + 1).fill(0n);
  dp[0] = 1n;
  for (let k = 1; k <= n; k++)
    for (let left = 0; left < k; left++)
      dp[k] += dp[left] * dp[k - 1 - left];
  return dp[n];
};
```

Time $O(n^2)$ **Space** $O(n)$

Output

TEXT

```
n=0 -> 1
n=1 -> 1
n=2 -> 2
n=3 -> 5
n=4 -> 14
n=5 -> 42
n=10 -> 16796
n=25 -> 4861946401452
```

TRAP

`dp[left] * dp[k-1-left]` is a **product of two large numbers**, and this is where JavaScript quietly lies. Numbers are doubles: every integer up to $2^{53} - 1$ is exact, and past that the answer is rounded. Measured with node: $n = 30$ gives 3814986502092304, which is correct; $n = 31$ gives 14544636039226908, and the true value is 14544636039226909. Off by one, no error, no warning.

Past $n = 30$, switch the whole table to `BigInt`: start with `dp = new Array(n+1).fill(0n)`, `dp[0] = 1n`, and print with `.toString()`. In C++ this is the line where you would widen to a 64-bit integer; in JavaScript the corresponding move is `BigInt`, and it has no ceiling at all.

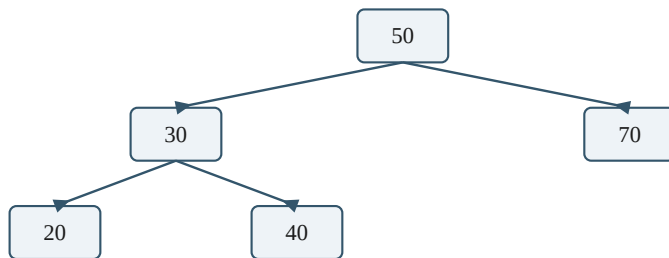
The follow-up

“Now list them all, not just count them.” Recursion again: for each root k , build every left shape from $1 \dots k-1$ and every right shape from $k+1 \dots n$ and pair them up. The count is Catalan, so the output size is exponential — this is only sane for n up to about

10. Say that out loud before you start coding.

Dry run — deleting a node with two children

Trace `bstErase(root, 30)` on this tree, node by node.



Starting tree. We delete 30, which has two children.

Step	Call	What the code checks	What happens
1	<code>bstErase(50, 30)</code>	<code>30 < 50</code>	recurse into the left child
2	<code>bstErase(30, 30)</code>	equal — this is the node	it has both children, so we cannot unhook
3	<code>minNode(40)</code>	walk left from 40; 40 has no left child	successor <code>s = 40</code>
4	copy	<code>root.val = 40</code>	the node now holds 40; the tree has two 40s for a moment
5	<code>bstErase(40, 40)</code>	equal — this is the duplicate	<code>left === null</code> , so return <code>right = null</code> ; the node is dropped
6	unwind	step 2 sets <code>root.right = null</code>	node 40 (old 30) now has only child 20
7	unwind	step 1 sets <code>50->left = that node</code>	done

Final tree: root 50; `50.left = 40` with left child 20; `50.right = 70`.

Inorder before and after

TEXT

before: 20 30 40 50 70
after : 20 40 50 70

Step 4 is the trick worth remembering: **we never move a node, we move a key**. The node object stays where it is; only its `val` changes. That is why the deletion does not have to repair any parent pointers.

The same trace, but the successor is deeper

If 30's right subtree had been 45 with a left child 42, then `minNode` would walk $45 \rightarrow 42$ and return 42. The copy step writes 42 into the node, and step 5 deletes 42 from the right subtree — where 42 is a **leaf**, so it is unhooked directly. The successor of a two-child node always has **no left child**, which is exactly why the recursive delete never loops forever.

Python — the two signature problems

Python is shorter for the two problems you are most likely to be asked to “just write quickly”.

Validate a BST, and the k-th smallest without recursion

PYTHON

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def is_bst(root):
    prev = [float('-inf')]
    def walk(n):
        if n is None:
            return True
        if not walk(n.left):
            return False
        if n.val <= prev[0]:
            return False
        prev[0] = n.val
        return walk(n.right)
    return walk(root)

def kth_smallest(root, k):
    st, cur = [], root
    while st or cur:
        while cur:
            st.append(cur)
            cur = cur.left
        cur = st.pop()
        k -= 1
        if k == 0:
            return cur.val
        cur = cur.right
    return None

def sorted_to_bst(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo > hi:
        return None
    mid = (lo + hi) // 2
    n = Node(a[mid])
    n.left = sorted_to_bst(a, lo, mid - 1)
    n.right = sorted_to_bst(a, mid + 1, hi)
    return n
```

Output

TEXT

```

is_bst good      : True
is_bst bad       : False
is_bst empty     : True
is_bst dup keys  : False
kth 1,4,7        : 20 50 80
kth 8 (too big)  : None
kth on empty     : None
balanced inorder: [1, 2, 3, 4, 5, 6, 7] root 4 height 3
empty build      : None
one build        : 9 1
negatives        : [-9, -4, -1, 0] is_bst True

```

TRAP

In Python, `sys.setrecursionlimit(300000)` is often needed for deep trees — the default limit is about 1000. The iterative `kth_smallest` above has no such problem, which is one more reason to prefer the stack version.

Practice

PRACTICE · WARM-UP target: 25 min

1. Insert 15, 10, 20, 8, 12, 17, 25 into an empty BST. Write the inorder walk and the height.
2. In that tree, what does `bstSearch` visit while looking for 12? For 13?
3. Delete 10 from that tree using the template. Write the tree after.
4. Write the smallest and biggest key of that tree, and say which pointer walk finds each.
5. True or false: every inorder walk that comes out sorted proves the tree is a BST.

PRACTICE · TIER 1 target: 60 min

5. **Second largest key.** Return the second biggest key in a BST in $O(h)$. Edge cases: one node; two nodes.
6. **Minimum absolute difference.** Return the smallest $|a - b|$ over all pairs of keys. $n \leq 10^5$. Edge cases: two nodes; negative keys.
7. **k-th largest key.** Same as k-th smallest but from the other end.
8. **Median of a BST.** Return the median key. Even counts average the middle two. Edge cases: empty tree; even node count.
9. **Distance between two keys.** Both keys exist. Return the number of edges between them. Edge cases: the same key twice; one key is the ancestor of the other.
10. **Greater-sum tree.** Replace every key by (itself + the sum of all bigger keys). Edge cases: one node; all-negative keys.

PRACTICE · TIER 2 target: 70 min

11. **Same key set.** Two BSTs with possibly different shapes — do they hold exactly the same keys? $O(n)$ time, $O(h)$ memory. Edge cases: both empty; one is a prefix of the other.
12. **Pairs across two trees.** Given two BSTs and a target x , count pairs (one key from each tree) that add to x . Edge cases: no pair; every key equal.
13. **Modes.** A BST that **allows duplicates** (equal keys go right). Return every key that appears most often. Edge cases: all keys distinct; empty tree.

14. **Successor with parent pointers.** Every node has a parent. Find the inorder successor of a given **node** in $O(h)$ and $O(1)$ memory. Edge cases: the biggest node; a node with a right child.

PRACTICE · TIER 3 target: 75 min

15. **k closest keys.** Return the k keys closest to a target, in increasing order. $n \leq 10^5$. Target $O(n)$, then improve. Edge cases: $k \geq n$; the target below every key.
16. **Valid postorder.** Decide whether an array is the postorder walk of some BST, in $O(n)$. Edge cases: empty; one element.
17. **Ranked leaderboard.** Support `add(score)`, `remove(score)`, `topKSum(k)` and `rankOf(score)` on a stream of scores, allowing duplicates.

ANSWER KEY

- Inorder 8 10 12 15 17 20 25; height 3 (15 → 10 → 8).
- For 12: 15, 10, 12 — three nodes. For 13: 15, 10, 12, then 12 → right is null, so it visits three nodes and returns null.
- 10 has two children. Successor = smallest of its right subtree = 12. Copy 12 up, delete the leaf 12. Result: root 15; 15.left = 12 with left child 8; 15.right = 20 with children 17 and 25. Inorder 8 12 15 17 20 25.
- Smallest 8 (walk left only), biggest 25 (walk right only).
- True. A sorted inorder walk is exactly the BST rule — this is why `isBST_prev` is correct. The one catch is duplicates: you must reject `a[i] == a[i - 1]` if your BST is meant to hold distinct keys.
- (second largest).** Reverse inorder — right, node, left — and stop at the second key.

k-th largest, reverse inorder

JS

```
const kthLargest = (root, k) => {
  let seen = 0, out = null;
  const walk = (r) => { // reverse inorder: right, node, left
    if (r === null || seen >= k) return;
    walk(r.right);
    if (seen >= k) return;
    if (++seen === k) { out = r.val; return; }
    walk(r.left);
  };
  walk(root);
  return out;
};

const secondLargest = (root) => kthLargest(root, 2);
```

Output: on {50,30,70,20,40,60,80}: 1st = 80, 3rd = 60, 7th = 20, 9th = none, second = 70. On a one-node tree {42}: 1st = 42, 2nd = none. This also answers 7.

6 (minimum absolute difference). In sorted order, the closest pair is always two **neighbours**. So compare each inorder key with the previous one.

Neighbours in the inorder walk

JS

```
const minAbsDiff = (root) => {
  let prev = null, best = Infinity;
  const walk = (r) => {
    if (r === null) return;
    walk(r.left);
    if (prev !== null) best = Math.min(best, r.val - prev); // inorder: never
  };
  walk(root);
  return best;
};
```

```

    prev = r.val;
    walk(r.right);
};
walk(root);
return best;
};

```

Output: {10,4,15,2,12} → 2; {1,100} → 99; {-5,-3,7} → 2. Note `r.val - prev` is never negative, because inorder is increasing — no `abs` needed. `prev` starts at `-Infinity`, so the very first node gives a difference of `Infinity` and can never be the minimum. C++ starts `prev` at `INT_MIN` and must widen the subtraction to avoid overflow; the JavaScript version has nothing to widen.

8 (median). Count the nodes, then fetch position $(n+1)/2$ (and $n/2 + 1$ as well when n is even).

Median

JS

```

const countNodes = (r) => (r === null ? 0 : 1 + countNodes(r.left) +
countNodes(r.right));

const nthKey = (root, target) => {           // the target-th smallest, 1-based
    let seen = 0, out = null;
    const walk = (r) => {
        if (r === null || seen >= target) return;
        walk(r.left);
        if (seen >= target) return;
        if (++seen === target) { out = r.val; return; }
        walk(r.right);
    };
    walk(root);
    return out;
};

const medianBST = (root) => {
    const n = countNodes(root);
    if (n === 0) return 0;
    const a = nthKey(root, Math.floor((n + 1) / 2));
    if (n % 2 === 1) return a;
    return (a + nthKey(root, n / 2 + 1)) / 2;
};

```

Output: {50,30,70,20,40,60,80} → 50; {4,2,6,1} → 3 (that is $(2+4)/2$); {9} → 9; empty → 0.

9 (distance). Find the LCA, then count the edges from the LCA down to each key.

Distance = depth(a) + depth(b), measured from the LCA

JS

```

const depthFrom = (r, key) => {
    let d = 0;
    while (r !== null && r.val !== key) { r = key < r.val ? r.left : r.right; d++; }
    return r ? d : -1;
};

const distanceBST = (root, a, b) => {
    const l = lcaBST(root, a, b);
    if (l === null) return -1;
    const da = depthFrom(l, a), db = depthFrom(l, b);
    if (da < 0 || db < 0) return -1;
};

```

```

    return da + db;
};

```

Output: distance(20,40) = 2; distance(20,80) = 4; distance(50,50) = 0; a missing key → -1.

10 (greater-sum tree). Walk in **reverse** inorder (right, node, left) with a running total. Each node is replaced by the running total **after** adding itself.

Reverse inorder with a running sum

JS

```

const toGreaterSumTree = (root) => {
  let run = 0;
  const walk = (r) => {
    if (r === null) return;
    walk(r.right);
    run += r.val;
    r.val = run;
    walk(r.left);
  };
  walk(root);
};

```

Output: {4,1,6,0,2,5,7} → inorder becomes [25 25 24 22 18 13 7]; {5} → 5; {-2,-5,1} → [-6 -1 1]. With 10^5 keys of size 10^9 the running total reaches 10^{14} — a 64-bit integer in C++, and a plain JavaScript number here, since 10^{14} is well inside $2^{53} - 1$.

11 (same key set). Run two iterators in step.

Compare two trees key by key

JS

```

const sameKeys = (a, b) => {
  const x = new BSTIterator(a), y = new BSTIterator(b);
  while (x.hasNext() && y.hasNext()) if (x.next() !== y.next()) return false;
  return !x.hasNext() && !y.hasNext();
};

```

Output: {5,3,8} vs {8,5,3} → true (different shapes, same keys); {5,3,8} vs {5,3} → false; two empty trees → true.

12 (pairs across two trees). Forward iterator on the first tree, reverse on the second, then the two-pointer rule.

Two trees, two pointers

JS

```

const countPairs = (a, b, x) => {
  const ia = new BSTIterator(a), ib = new RevIterator(b);
  if (!ia.hasNext() || !ib.hasNext()) return 0;
  let p = ia.next(), q = ib.next(), cnt = 0;
  for (;;) {
    const s = p + q;
    if (s === x) {
      cnt++;
      if (!ia.hasNext() || !ib.hasNext()) break;
      p = ia.next(); q = ib.next();
    } else if (s < x) {
      if (!ia.hasNext()) break;
      p = ia.next();
    } else {
      if (!ib.hasNext()) break;
      q = ib.next();
    }
  }
};

```

```

    }
    return cnt;
};

```

Output: {5,3,7} and {4,2,6} with $x = 9 \rightarrow 3$ pairs ((3,6), (5,4), (7,2)); with $x = 100 \rightarrow 0$.

13 (modes). Inorder groups equal keys together, so count run lengths.

Runs of equal keys in the inorder walk

JS

```

const dupInsert = (r, v) => {                // equal keys go right
  if (r === null) return new Node(v);
  if (v < r.val) r.left = dupInsert(r.left, v);
  else r.right = dupInsert(r.right, v);
  return r;
};

const modesOfBST = (root) => {
  let prev = null, run = 0, best = 0, out = [];
  const walk = (r) => {
    if (r === null) return;
    walk(r.left);
    run = (prev !== null && r.val === prev) ? run + 1 : 1;
    if (run > best) { best = run; out = [r.val]; }
    else if (run === best) out.push(r.val);
    prev = r.val;
    walk(r.right);
  };
  walk(root);
  return out;
};

```

Output: inserting 5,3,5,7,5,3 $\rightarrow$ modes [5] (it appears three times); all-distinct {1,2,3} $\rightarrow$ [1 2 3] (every key ties at one); empty tree $\rightarrow$ [].

14 (successor with parent pointers). Two cases. If the node has a right child, the answer is the leftmost node of that right subtree. Otherwise climb until you step up from a **left** child — that parent is the successor.

Climb when there is nothing on the right

JS

```

class PNode {
  constructor(val) { this.val = val; this.left = null; this.right = null;
  this.parent = null; }
}

const successorWithParent = (x) => {
  if (x === null) return null;
  if (x.right !== null) {
    let c = x.right;
    while (c.left !== null) c = c.left;
    return c;
  }
  let p = x.parent;
  while (p !== null && p.right === x) { x = p; p = p.parent; }
  return p;
};

```

Output on {50,30,70,20,40,60,80}: succ(40)=50, succ(30)=40, succ(80)=none, succ(20)=30.

15 (k closest keys). Brute force: flatten, sort by distance, take k, sort again. Better: flatten (already sorted), then binary search for the best **window** of length k.

Two ways — the second is $O(n + \log n)$

JS

```
const kClosestBrute = (root, target, k) => {
  const a = inorderCollect(root);
  // sort() on numbers ALWAYS needs a comparator; this one sorts by distance
  a.sort((p, q) => Math.abs(p - target) - Math.abs(q - target));
  return a.slice(0, k).sort((p, q) => p - q);
};

const kClosestWindow = (root, target, k) => {
  const a = inorderCollect(root); // already sorted
  const n = a.length;
  if (k >= n) return a;
  let lo = 0, hi = n - k; // lo = start of the window
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (target - a[mid] > a[mid + k] - target) lo = mid + 1;
    else hi = mid;
  }
  return a.slice(lo, lo + k);
};
```

Output: tree {50,30,70,20,40,60,80}, target 55, k = 3 → [40 50 60] from both. Target 5, k = 2 → [20 30] from both. k = 100 → the whole tree.

16 (valid postorder). Postorder ends with the root, so scan **right to left** and mirror Example 22: keep a **ceiling** instead of a floor.

Postorder read backwards is root, right, left

JS

```
const validPostorder = (post) => {
  const st = [];
  let ceilVal = Infinity;
  for (let i = post.length - 1; i >= 0; i--) {
    const x = post[i];
    if (x > ceilVal) return false;
    while (st.length && st[st.length - 1] > x) ceilVal = st.pop();
    st.push(x);
  }
  return true;
};
```

Output: {1,7,5,12,10,8} → valid; {7,12,5,1,10,8} → not valid; {} → valid; {3} → valid.

17 (leaderboard). C++ would use a **multiset** — a balanced BST that keeps duplicates. JavaScript has none, so the stand-in is a sorted array plus **lowerBound** from the toolkit. Inserts cost $O(n)$ for the shift instead of $O(\log n)$, which is fine up to a few tens of thousands of scores; say that out loud rather than pretending it is free.

Sorted array standing in for a multiset

JS

```
class Leaderboard {
  constructor() { this.scores = []; } // kept sorted ascending

  add(s) {
    const i = lowerBound(this.scores, s);
    this.scores.splice(i, 0, s); // O(n) shift, O(log n) search
  }
}
```

```

}
remove(s) {
  const i = lowerBound(this.scores, s);
  if (this.scores[i] === s) this.scores.splice(i, 1); // ONE copy only
}
topKSum(k) {
  return this.scores.slice(Math.max(0, this.scores.length - k))
    .reduce((a, b) => a + b, 0);
}
rankOf(s) { // how many scores are >= s
  return this.scores.length - lowerBound(this.scores, s);
}
}

```

Output: add 70, 95, 95, 40, 88 → top-3 sum 278, rank of 88 is 3. After removing one 95: top-3 sum 253, top-10 sum 293 (only five scores exist, so it returns the total).

TRAP

Delete **one** copy, not all of them. `this.scores.splice(i, 1)` removes exactly one element at index `i`, which is what you want. The lazy version — `scores.filter(x => x !== s)` — removes every 95 at once and silently changes the answer. (C++ has the same trap with `multiset::erase(value)`, which also erases every copy.) Guard the splice with `if (this.scores[i] === s)`, or a `remove` of a score that is not there will delete its neighbour.

ONE-PAGE REVISION CARD

The invariant

For every node: all keys left < node key < all keys right — across the **whole** subtree, not just the children.
Inorder walk = sorted order. That is the entire chapter.

Template

Memorise this shape; change one comparison per problem

JS

```
// 1. walk down – search, floor, ceil, successor, predecessor and LCA share it
const walkDown = (r, key) => {
  while (r !== null) {
    if (key === r.val) return r;
    r = key < r.val ? r.left : r.right;
  }
  return null;
};

// 2. walk in order – validate, k-th, median, modes, min-gap, greater-sum share it
const walkInOrder = (r, visit) => {
  if (r === null) return;
  walkInOrder(r.left, visit);
  visit(r); // <- put the problem's work here
  walkInOrder(r.right, visit);
};

// 3. window down – validate, build-from-preorder and deserialise share it
const walkWindow = (r, lo, hi) => {
  if (r === null) return true;
  if (r.val <= lo || r.val >= hi) return false;
  return walkWindow(r.left, lo, r.val) && walkWindow(r.right, r.val, hi);
};
```

Complexity table

Operation	Time	Space	Note
search / insert / delete	$O(h)$	$O(1)$ iterative	$h = \log n$ only if balanced
floor / ceil / successor / LCA	$O(h)$	$O(1)$	one downward walk
k-th smallest (plain)	$O(h + k)$	$O(h)$	counting inorder
k-th smallest (size field)	$O(h)$	$O(n)$	pay at insert time
validate	$O(n)$	$O(h)$	window or inorder-prev
range sum / count	$O(h + m)$	$O(h)$	m = keys inside the range
sorted array to balanced BST	$O(n)$	$O(\log n)$	middle element is the root
iterator next()	$O(1)$ amortised	$O(h)$	each node pushed and popped once
AVL insert	$O(\log n)$	$O(\log n)$	height stays below $1.44 \log_2 n$
treap insert	$O(\log n)$ expected	$O(n)$	random priority, no worst-case promise

count BST shapes	$O(n^2)$	$O(n)$	Catalan; exact only to $n = 30$, then <code>BigInt</code>
------------------	----------	--------	--

Top traps

1. **Local check is not enough.** `left < node < right` on parents and children does not prove a BST. Use a window or the inorder-previous value.
2. **Finite sentinels break on extreme keys.** Use `-Infinity` and `Infinity` for `lo` and `hi`, never a large finite number — a key equal to that number then fails the test.
3. **Sorted input kills a plain BST.** Height becomes n . Use an AVL, a treap, or a sorted array plus `lowerBound`.
4. **Products overflow silently.** Catalan numbers are exact only to $n = 30$; past that a JavaScript number rounds and prints a wrong answer with no error. Switch to `BigInt`.
5. **Stepping back past the start.** Check `i > 0` before reading `a[i-1]` in a sorted-array predecessor query.
6. **Deleting every copy instead of one.** `filter(x => x !== s)` removes all duplicates; `splice(i, 1)` removes one.
7. **Delete forgets the successor rule.** For a two-child node, copy the smallest key of the right subtree up, then delete **that** key from the right subtree.
8. **bstToDLL overwrites right before using it.** Save `r.right` first.
9. **Empty-subtree sentinels look backwards on purpose.** `{mn: Infinity, mx: -Infinity}` is what makes an empty child pass every comparison.
10. **“Largest BST” with all-negative keys.** Decide with the interviewer whether the empty subtree (sum 0) is allowed to win.

When to reach for what

You need	Use
order + inserts + deletes	hand-written AVL or treap (JS ships no balanced BST)
order + duplicates	sorted array + <code>lowerBound</code> , <code>splice</code> to insert
only lookups, data fixed	sorted array + <code>lowerBound</code> / <code>upperBound</code>
no order needed	<code>Map</code> or <code>Set</code>
k -th smallest, many queries	BST with a subtree-size field
guaranteed height, hand-written	AVL
short code, good height on average	treap

Heaps & Priority Queues

A heap answers one question fast — what is the smallest thing left? — and that single question solves top-K, merge-K, medians and half of all greedy problems.

Pattern in one page

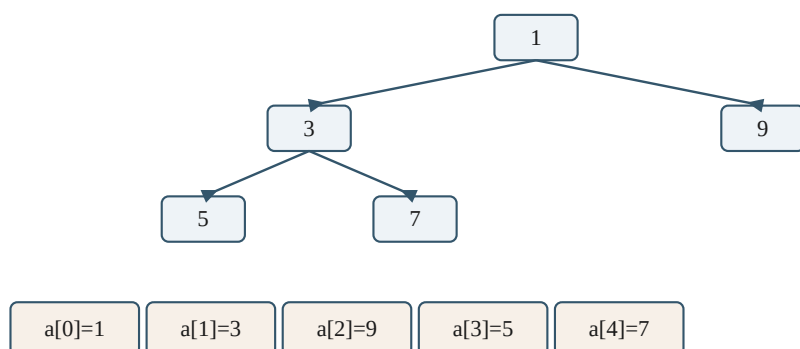
THE ONE RULE

A **min-heap** is a binary tree where every parent is $\leq$ both of its children. (A **max-heap** flips the sign.) That is the **only** rule. Note what it does **not** say:

- It says nothing about left versus right. A heap is **not** sorted.
- It only promises one thing: **the smallest item is at the root**.

Two more facts make it fast and tiny:

1. The tree is **complete** — every level is full except possibly the last, which fills from the left. So it can live in a plain array with no pointers at all.
2. With the array layout, a node at index i has parent $(i-1)/2$, children $2i+1$ and $2i+2$. Height is always $\lfloor \log_2 n \rfloor$.



The same min-heap twice: as a tree, and as the array it really is. Node `a[1]=3` has children `a[3]=5` and `a[4]=7`.

The two moves: sift up and sift down

Every heap operation is one of two repairs.

- **Sift up** — a value is too small for its place. Swap it with its parent, again and again, until the parent is smaller. Used after push.
- **Sift down** — a value is too big for its place. Swap it with its **smaller** child, again and again, until both children are bigger. Used after pop.

Each repair walks one root-to-leaf path, so each costs $O(\log n)$.

TRAP

JavaScript has no heap and no priority queue. Not in the language, not in the standard library. Python has `heapq`, Java has `PriorityQueue`, C++ has `priority_queue` — JS has nothing. You either write the twenty lines below or you carry them in. That is why this book ships a tested `MinHeap` in the **JS Toolkit** appendix and every later chapter just does `const { MinHeap } = require('./toolkit.js')`.

This chapter is the one place where we write it out in full, because this chapter is about **how it works**.

A complete min-heap, written by hand

JS

```
class MinHeap {
  // cmp(a, b) < 0 means "a belongs closer to the root"
  constructor(cmp = (a, b) => a - b) { this.a = []; this.cmp = cmp; }
  get size() { return this.a.length; }
  peek() { return this.a[0]; } // a[0] is always the smallest

  siftUp(i) {
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (this.cmp(this.a[p], this.a[i]) <= 0) break; // parent smaller: done
      [this.a[p], this.a[i]] = [this.a[i], this.a[p]];
      i = p;
    }
  }

  siftDown(i) {
    const n = this.a.length;
    for (;;) {
      const l = 2 * i + 1, r = 2 * i + 2;
      let small = i;
      if (l < n && this.cmp(this.a[l], this.a[small]) < 0) small = l;
      if (r < n && this.cmp(this.a[r], this.a[small]) < 0) small = r;
      if (small === i) break; // both children bigger: done
      [this.a[i], this.a[small]] = [this.a[small], this.a[i]];
      i = small;
    }
  }

  push(v) { this.a.push(v); this.siftUp(this.a.length - 1); }

  pop() {
    const top = this.a[0], last = this.a.pop();
    if (this.a.length) { this.a[0] = last; this.siftDown(0); }
    return top;
  }

  build(vals) { // O(n), not O(n log n)
    this.a = [...vals];
    for (let i = (this.a.length >> 1) - 1; i >= 0; i--) this.siftDown(i);
    return this;
  }
}
```

Real output

TEXT

```
push 5,3,9,1,7 -> array [1 3 9 5 7]   valid heap = yes
pop order: 1 3 5 7 9
build from [9,4,7,1,-3,8,0] -> [-3 1 0 9 4 8 7]   valid = yes   top = -3
empty build -> size 0, valid = yes
one element -> top 42
duplicates [2,2,2] -> [2 2 2], valid = yes
```

`[this.a[p], this.a[i]] = [this.a[i], this.a[p]]` is JavaScript's swap: build a pair on the right, destructure it back on the left. There is no `swap` function in the language.

`(i - 1) >> 1` is the parent index. `>> 1` is an integer halving that also truncates, so it replaces C-style integer division. Use it everywhere a heap index is computed — `Math.floor((i - 1) / 2)` is the same thing and three times as long.

TRAP

`pop()` must move the **last** element to the root and shrink, then sift down. A common bug is to move the **smaller child** up recursively — that leaves a hole in the middle and breaks the “complete tree” property, which breaks the index arithmetic.

Why `build` is $O(n)$ and not $O(n \log n)$

You call `siftDown` on $n/2$ nodes, and each call is “up to $\log n$ ”. So the lazy bound is $O(n \log n)$. But almost every node is near the **bottom**, where `siftDown` has almost nothing to do.

Level from the bottom	Nodes on it	Max sift distance	Work
0 (leaves)	$\frac{n}{2}$	0	0
1	$\frac{n}{4}$	1	$\frac{n}{4}$
2	$\frac{n}{8}$	2	$2 \frac{n}{8}$
3	$\frac{n}{16}$	3	$3 \frac{n}{16}$
k	$\frac{n}{2^{k+1}}$	k	$k \frac{n}{2^{k+1}}$

Total work = $n \sum_{k=1}^{\log n} \frac{k}{2^{k+1}}$, and $\sum_{k \geq 1} \frac{k}{2^{k+1}}$ converges to 1. So the total is below n . **Build is linear.**

SHORTCUT

Inserting n items one by one is $O(n \log n)$. Calling `build` on the whole array is $O(n)$. If you already have all the data, always `build` — never push in a loop.

The comparator — one rule, three heaps

One class covers every heap you need. The comparator decides what “smallest” means, and the root is always whatever the comparator calls smallest.

min-heap, max-heap and a custom order

JS

```
const minh = new MinHeap();           // smallest on top (the default)
const maxh = new MinHeap((a, b) => b - a); // biggest on top – just flip it

// a custom order: min-heap on .cnt, ties broken by the smaller .id
const pq = new MinHeap((x, y) => x.cnt - y.cnt || x.id - y.id);
```

TRAP

A heap comparator obeys exactly the same rules as a sort comparator, and breaks in exactly the same ways.

- It must return a **number**, never a boolean. `(a, b) => a.cnt > b.cnt` returns `true/false`, which coerce to `1/0`; the heap never sees a negative and silently stops ordering anything. Write `(a, b) => a.cnt - b.cnt`.
- Chain tie-breaks with `||`, because `0` is falsy: `(x, y) => x.cnt - y.cnt || x.id - y.id`.
- There is no separate “max-heap” class to remember. `(a, b) => b - a` is the max-heap.

And the rule to say out loud once: **the element the comparator calls smallest ends up at the root, and the root is what `pop()` hands back.**

When to reach for a heap

You need	Heap?	Why
the k largest of n, k much smaller than n	yes	$O(n \log k)$ beats $O(n \log n)$
the whole list sorted	no	<code>a.sort((x, y) => x - y)</code>
the smallest item, repeatedly, while inserting	yes	this is the definition
the k-th largest, once, offline	no	quickselect is $O(n)$ on average
merge k sorted lists	yes	$O(N \log k)$
a running median	yes	two heaps facing each other
max in a sliding window	maybe	a monotonic deque is $O(n)$; a heap is $O(n \log n)$
greedy “always take the best available”	yes	the heap is the greedy step

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

Which of these arrays is a valid min-heap?

(A) [1, 3, 9, 5, 7] (B) [1, 5, 2, 9, 3] (C) [2, 2, 2] (D) [5]

SOLUTION

Check each parent against its children using children of i are $2i+1$ and $2i+2$.

- (A) $a[0]=1$ vs $a[1]=3$, $a[2]=9$ — ok. $a[1]=3$ vs $a[3]=5$, $a[4]=7$ — ok. **Valid.**
- (B) $a[1]=5$ vs its children $a[3]=9$ and $a[4]=3$. But $5 > 3$. **Not valid.**
- (C) $2 \leq 2$ everywhere. **Valid** — the rule is $\leq$, not $<$.
- (D) One node has no children, so nothing can be broken. **Valid.**

Answer: A, C and D

The check, in five lines

JS

```
const isMinHeap = (a) => {
  for (let i = 0; i < a.length; i++) {
    const l = 2 * i + 1, r = 2 * i + 2;
    if (l < a.length && a[l] < a[i]) return false;
    if (r < a.length && a[r] < a[i]) return false;
  }
  return true;
};
```

Tested: [1,3,9,5,7] $\rightarrow$ true, [1,5,2,9,3] $\rightarrow$ false, [2,2,2] $\rightarrow$ true, [5] $\rightarrow$ true, [] $\rightarrow$ true.

Example 2 [warm-up]

A heap holds 100 items. Which index holds the parent of index 37? Which indices are its children? How tall is the heap?

SOLUTION

- Parent of 37 is $\frac{37-1}{2} = 18$ (integer division).

- Children of 37 are $2 \times 37 + 1 = 75$ and $2 \times 37 + 2 = 76$. Both are below 100, so both exist.
- Height is $\lfloor \log_2 100 \rfloor = 6$, because $2^6 = 64 \leq 100 < 128 = 2^7$.

Answer: parent 18, children 75 and 76, height 6

Example 3 [warm-up]

Push 2 into the min-heap [4, 6, 5, 9, 8]. Show every swap.

SOLUTION

Append 2 at index 5. Parent of 5 is $\frac{5-1}{2} = 2$.

1. [4, 6, 5, 9, 8, 2]. $a[2]=5 > a[5]=2$, so swap $\rightarrow$ [4, 6, 2, 9, 8, 5]. Now $i = 2$.
2. Parent of 2 is $\frac{2-1}{2} = 0$. $a[0]=4 > a[2]=2$, so swap $\rightarrow$ [2, 6, 4, 9, 8, 5]. Now $i = 0$.
3. $i = 0$ is the root. Stop.

Answer: [2, 6, 4, 9, 8, 5] — two swaps

Example 4 [warm-up]

Pop from the min-heap [1, 3, 9, 5, 7]. Show every step.

SOLUTION

1. The answer is $a[0] = 1$.
2. Move the **last** element (7) to the root and shrink: [7, 3, 9, 5].
3. Sift down from index 0. Children are $a[1]=3$ and $a[2]=9$. The smaller is 3, and $3 < 7$, so swap $\rightarrow$ [3, 7, 9, 5]. Now $i = 1$.
4. Children of 1 are index 3 (=5) and index 4 (does not exist). $5 < 7$, so swap $\rightarrow$ [3, 5, 9, 7]. Now $i = 3$, a leaf. Stop.

Answer: returns 1; the heap becomes [3, 5, 9, 7]

Example 5 [warm-up]

You need the 3 largest of 1000000 numbers. Compare sorting everything against a heap of size 3. Roughly how many operations does each take?

SOLUTION

- **Sort everything:** about $n \log_2 n = 10^6 \times 20 = 2 \times 10^7$ operations, plus $O(n)$ memory for the copy.
- **Min-heap of size 3:** one push and at most one pop per element, each $\log_2 3 \approx 1.6$ operations, so about 2×10^6 . Memory is 3 integers.

The heap is about ten times faster and uses essentially no memory.

Answer: sort $\approx 2 \times 10^7$; heap $\approx 2 \times 10^6$ with $O(k)$ memory

SHORTCUT

The size- k trick: **to keep the k LARGEST, use a MIN-heap of size k .** The root is then the weakest survivor, so it is the one to throw away when a better item arrives. Beginners reach for a max-heap and get stuck. Flip it.

Example 6 [warm-up]

Build a min-heap, push 5, 1, 4, and print the pop order. Then say what one character you would change to make it a max-heap.

SOLUTION

The default comparator $(a, b) \Rightarrow a - b$ already gives a min-heap. Flipping it to $(a, b) \Rightarrow b - a$ gives a max-heap — that is the whole difference.

The exact spelling

JS

```
const pq = new MinHeap(); // default cmp = (a, b) => a - b
for (const x of [5, 1, 4]) pq.push(x);
const out = [];
while (pq.size) out.push(pq.pop());
console.log(out.join(' ')); // 1 4 5
```

Answer: 1 4 5; swap the comparator to $(a, b) \Rightarrow b - a$ for a max-heap

Tier 1 — the core heap toolkit

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

K-th largest element. Given an array, return the k-th largest value. Duplicates count as separate elements, so in $[3, 3, 1]$ the 2nd largest is 3.

Limits $n \leq 10^5$, $1 \leq k \leq n$, values fit in a 32-bit range · **Target** better than $O(n \log n)$ · **Edge cases** $k = 1$; $k = n$; all values equal; negative values.

Approach 1 — Sort descending, take $a[k-1]$ ($O(n \log n)$ — correct, always accepted, never impressive)

The baseline

JS

```
const kthLargestSort = (a, k) => {
  const s = [...a].sort((x, y) => y - x); // NUMERIC comparator, descending
  return s[k - 1];
};
```

TRAP

arr.sort() in JavaScript is LEXICOGRAPHIC. $[10, 9, 1].sort()$ gives $[1, 10, 9]$, because the default comparator turns every element into a **string** first. On this problem that means $kthLargestSort([10, 9, 1], 1)$ would answer 9 instead of 10.

Numbers always need an explicit comparator: $.sort((a, b) \Rightarrow a - b)$ ascending, $.sort((a, b) \Rightarrow b - a)$ descending. If you typed $.sort()$ with empty brackets, you have a bug.

Note the $[...a]$ too: `sort` works **in place** and returns the same array, so without the copy this function silently reorders the caller's data.

Time $O(n \log n)$ **Space** $O(1)$

Sorting does far more work than the question asks for: it orders all n , when only k matter.

Approach 2 — Keep a min-heap of the k best so far ($O(n \log k)$ — the standard answer)

SOLUTION

Walk the array. Push every value. If the heap grows past k , pop the smallest — it can never be in the top k . At the end the heap holds exactly the k largest, and its **root** is the smallest of those, which is the k -th largest.

Min-heap of size k

JS


```
const kthLargestHeap = (a, k) => {
  const pq = new MinHeap();           // min-heap of the k best so far
  for (const x of a) {
    pq.push(x);
    if (pq.size > k) pq.pop();         // drop the weakest survivor
  }
  return pq.peak();
};
```

Time $O(n \log k)$ **Space** $O(k)$

This is the version that also works on a STREAM, where n is not known in advance.

Approach 3 — Quickselect — partition, then recurse into one side only ($O(n)$ average, $O(n^2)$ worst; $O(1)$ extra memory)

Quickselect with a random pivot

JS

```
const kthLargestQuickselect = (arr, k) => {
  const a = [...arr];
  const target = a.length - k;           // index in ascending order
  let lo = 0, hi = a.length - 1;
  while (lo < hi) {
    const p = lo + Math.floor(Math.random() * (hi - lo + 1)); // random pivot
    [a[p], a[hi]] = [a[hi], a[p]];
    const pivot = a[hi];
    let store = lo;
    for (let i = lo; i < hi; i++)
      if (a[i] < pivot) { [a[i], a[store]] = [a[store], a[i]]; store++; }
    [a[store], a[hi]] = [a[hi], a[store]];
    if (store === target) return a[store];
    if (store < target) lo = store + 1; else hi = store - 1;
  }
  return a[lo];
};
```

Time $O(n)$ average **Space** $O(n)$ for the copy

JavaScript has no `nth_element` / partial-sort helper. If you need $O(n)$, you write this. A random pivot is what keeps sorted input from turning it into $O(n^2)$.

All three agree — $a = [3, 2, 3, 1, 2, 4, 5, 5, 6]$

TEXT

```
k=1: 6 6 6      k=4: 4 4 4      k=9: 1 1 1
single element [8], k=1 -> 8
negatives [-1, -5, -3], k=2 -> -3 from all three
```

The idea that unlocked it: you were asked for **one** rank, not for the order. Sorting answers a bigger question than the one asked. The heap keeps exactly k items and throws the rest away immediately.

Example 8 [Infosys · pattern]

Heap sort. Sort an array ascending using a heap, with $O(1)$ extra memory.

Limits $n \leq 10^6$ · **Target** $O(n \log n)$ time, $O(1)$ extra space · **Edge cases** empty array; all equal values; negative values.

Approach 1 — Selection sort: scan for the biggest, swap it to the end, repeat ($O(n^2)$ — the scan is the waste)

Approach 2 — Let a heap find the biggest for you in $O(\log n)$ ($O(n \log n)$ time, still $O(1)$ extra memory)

SOLUTION

Use a **max**-heap in the same array.

1. build it in $O(n)$.
2. Swap $a[0]$ (the biggest) with the last slot. That slot is now final.
3. Shrink the heap by one and sift down the new root.
4. Repeat. Each round places one more value, from the back forward.

Heap sort — in place, no extra array

JS

```
const maxSiftDown = (a, i, n) => {
  for (;;) {
    const l = 2 * i + 1, r = 2 * i + 2;
    let big = i;
    if (l < n && a[l] > a[big]) big = l;
    if (r < n && a[r] > a[big]) big = r;
    if (big === i) return;
    [a[i], a[big]] = [a[big], a[i]];
    i = big;
  }
};

const heapSort = (a) => {
  const n = a.length;
  for (let i = (n >> 1) - 1; i >= 0; i--) maxSiftDown(a, i, n); // build,  $O(n)$ 
  for (let end = n - 1; end > 0; end--) {
    [a[0], a[end]] = [a[end], a[0]]; // biggest goes to its final slot
    maxSiftDown(a, 0, end); // heap is now one shorter
  }
  return a;
};
```

Time $O(n \log n)$ **Space** $O(1)$

Worst case too — unlike quicksort. But it is not stable, and it is cache-unfriendly, so the engine's own `.sort` is usually faster in practice.

Output

TEXT

```
[5 1 4 2 8] -> [1 2 4 5 8]
[]           -> []
[7]          -> [7]
[3 3 3]       -> [3 3 3]
[-9 2 -4 0]  -> [-9 -4 0 2]
2000 random values match .sort((a,b)=>a-b): yes
```

TRAP

`heapSort` sorts **in place** and returns the same array — exactly like `Array.prototype.sort`. Callers who still want their original order must pass a copy: `heapSort([...v])`.

Note $(n >> 1) - 1$. On an empty array that is `-1` and the loop never runs, which is correct. `>>` also truncates towards zero for these values, so it doubles as the integer division C++ gets for free.

Example 9 [Wipro · pattern]

Top k frequent values. Return the k values that appear most often. Break ties by returning the smaller value first.

Limits $n \leq 10^5$, $1 \leq k \leq$ number of distinct values · **Target** better than $O(n \log n)$ · **Edge cases** every value distinct; one value repeated n times; negative values.

Approach 1 — Count, then sort the counts ($O(d \log d)$, $d =$ number of distinct values)

Approach 2 — Count, then keep a min-heap of size k ($O(d \log k)$ time, $O(d)$ memory)

Min-heap of size k , with an explicit tie rule

JS

```
const topKFrequentHeap = (a, k) => {
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  // the root is the entry to DROP: smallest count, and on a tie the bigger value
  const pq = new MinHeap((x, y) => x[1] - y[1] || y[0] - x[0]);
  for (const e of cnt) { // iterating a Map yields [key, value] pairs
    pq.push(e);
    if (pq.size > k) pq.pop();
  }
  const out = [];
  while (pq.size) out.push(pq.pop()[0]);
  return out.reverse(); // heap gives worst first, so flip
};
```

Time $O(n + d \log k)$ **Space** $O(d)$

TRAP

Use a Map, not a plain object, for counting. An object turns every key into a **string**, so `-1` and `"-1"` collide, the keys come back as strings, and your comparator then does string subtraction. Map keeps the key's type, keeps insertion order, and is faster for heavy add/delete traffic. `cnt.get(x) ?? 0` handles the first sighting of a key.

Approach 3 — Bucket by frequency ($O(n)$ — a count can never exceed n , so it is an index)

SOLUTION

A frequency is a whole number between 1 and n . So make $n+1$ buckets and drop each value into the bucket named by its count. Then read the buckets from the highest down.

No heap, no sort of the counts

JS

```
const topKFrequentBucket = (a, k) => {
  const cnt = new Map();
  for (const x of a) cnt.set(x, (cnt.get(x) ?? 0) + 1);
  const bucket = Array.from({ length: a.length + 1 }, () => []);
  for (const [v, c] of cnt) bucket[c].push(v);
  const out = [];
  for (let f = a.length; f >= 1 && out.length < k; f--) {
    bucket[f].sort((x, y) => y - x); // numeric, descending
    for (const v of bucket[f]) {
      if (out.length === k) break;
      out.push(v);
    }
  }
  return out;
};
```

Time $O(n)$ **Space** $O(n)$

The inner sorts together cost $O(d \log d)$ in the worst case, but only inside single buckets.

TRAP

`Array.from({length: n}, () => [])` — **not** `new Array(n).fill([])`. `fill` stores the **same** array object in every slot, so pushing into `bucket[3]` also pushes into `bucket[7]`. This is the same shallow-copy trap that bites 2D grids, where `[...grid]` shares every row and only `grid.map(r => [...r])` is a real copy.

Output — a = [4,1,-1,2,-1,2,3,2]

TEXT

```
heap    k=2 -> [2 -1]
bucket k=2 -> [2 -1]
[9] k=1 -> [9] from both;  [5,5,5] k=1 -> [5] from both
```

The idea that unlocked it: a frequency is bounded by n , so it can be an **array index** instead of a sort key. Whenever a key is a small whole number, bucket it.

Example 10 [Capgemini · pattern]

Connect the ropes. You have ropes of given lengths. Joining two ropes costs the sum of their lengths and gives one rope. Join all of them for the least total cost.

Limits $n \leq 10^5$, each length up to 10^9 · **Target** $O(n \log n)$ · **Edge cases** zero or one rope (cost 0); a total big enough to leave the 32-bit range.

Approach 1 — Try every order of joining (factorial — 10 ropes already give 3628800 orders)

Approach 2 — Always join the two shortest ropes left ($O(n \log n)$ with a min-heap)

SOLUTION

Every join adds the two lengths to the bill, and the result gets joined again later. So a rope that is joined **early** is paid for **many** times. To keep the bill small, always join the **two shortest** ropes available. A min-heap hands them to you.

Greedy with a min-heap — build, do not push in a loop

JS

```
const connectRopes = (len) => {
  if (len.length <= 1) return 0;
  const pq = new MinHeap().build(len);    // O(n) build, not n pushes
  let total = 0;
  while (pq.size > 1) {
    const a = pq.pop(), b = pq.pop();
    total += a + b;
    pq.push(a + b);
  }
  return total;
};
```

Time $O(n \log n)$ **Space** $O(n)$

Output

TEXT

```
[4,3,2,6] -> 29          (2+3=5, 5+4=9, 9+6=15; bill 5+9+15 = 29)
[5] -> 0      [] -> 0      [1,1,1,1] -> 8
four ropes of 1000000000 -> 8000000000    safe integer? true
```

TRAP

JavaScript numbers are doubles. There is no `int`, no 64-bit integer type, and no silent 32-bit wrap — so `8000000000` is simply correct here. The limit is different: integers are exact only up to `Number.MAX_SAFE_INTEGER =  $2^{53} - 1 = 9007199254740991$` , about 9×10^{15} .

Past that, arithmetic quietly **rounds** instead of wrapping, which is harder to spot than an overflow. Guard the moment a running total or a product can exceed it:

```
console.log(Number.isSafeInteger(8000000000)); // true - the rope bill is exact
console.log(Number.isSafeInteger(1e20));       // false - rounded, and silently so
```

and switch to `BigInt` when it cannot: `let total = 0n; total += BigInt(a) + BigInt(b);`. This chapter needs `BigInt` exactly once — the product of the k largest, in the practice set — because sums of 10^5 values up to 10^9 stay comfortably inside 2^{53} .

Example 11 [Cognizant · pattern]

Sort a nearly sorted array. Every element is at most k positions away from where it belongs. Sort it.

Limits $n \leq 10^5$, $0 \leq k \leq n$ · **Target** $O(n \log k)$ · **Edge cases** $k = 0$ (already sorted); $k = n$ (a normal sort); empty array.

Approach 1 — Just call sort ($O(n \log n)$ — correct but ignores the k)

Approach 2 — Slide a min-heap of size $k+1$ along the array ($O(n \log k)$)

SOLUTION

If the true smallest remaining element is at most k places away, then after reading $k+1$ elements it **must** already be inside the heap. So: read $k+1$ elements, then for each new element, pop one — that pop is the next value of the answer.

A window of size $k+1$

JS

```
const sortKSorted = (a, k) => {
  const pq = new MinHeap();
  const out = [];
  for (const x of a) {
    pq.push(x);
    if (pq.size > k + 1) out.push(pq.pop());
  }
  while (pq.size) out.push(pq.pop());
  return out;
};
```

Time $O(n \log k)$ **Space** $O(k)$

Output

TEXT

```
[6,5,3,2,8,10,9] with k=3 -> [2 3 5 6 8 9 10]
[] k=2 -> []            [4] k=0 -> [4]            [2,1,2,1] k=1 -> [1 1 2 2]
```

Example 12 [Accenture · pattern]

Merge k sorted arrays. Combine k sorted arrays into one sorted array of N values.

Limits $k \leq 10^4$, total $N \leq 10^6$ · **Target** $O(N \log k)$ · **Edge cases** some arrays empty; all arrays empty; $k = 0$.

Approach 1 — Concatenate everything and sort ($O(N \log N)$ — throws away the sortedness)

Approach 2 — Merge two at a time, left to right ($O(N k)$ — the first array is copied k times)

Approach 3 — One min-heap holding the current head of each array ($O(N \log k)$)

The heap holds at most k items — one per array

JS

```
const mergeKArrays = (lists) => {
  // each heap entry is {val, li, idx} = value, which list, where in it
  const pq = new MinHeap((x, y) => x.val - y.val);
  lists.forEach((l, i) => { if (l.length) pq.push({ val: l[0], li: i, idx: 0 }); });
  const out = [];
  while (pq.size) {
    const { val, li, idx } = pq.pop(); // destructure the entry
    out.push(val);
    if (idx + 1 < lists[li].length)
      pq.push({ val: lists[li][idx + 1], li, idx: idx + 1 });
  }
  return out;
};
```

Time $O(N \log k)$ **Space** $O(k)$ plus the output

Output

TEXT

```
[[1,5,9],[2,6],[3,4,10]] -> [1 2 3 4 5 6 9 10]
[[],[],[ ]] -> []        [] -> []        [[-5,-1],[-3,0]] -> [-5 -3 -1 0]
```

TRAP

Pushing the **whole** first array into the heap instead of just its head makes the heap $O(N)$ and defeats the point. The heap must never hold more than one item per list.

Example 13 [TCS Digital · pattern]

K closest points. Given points on a plane, return the k closest to the origin.

Limits $n \leq 10^5$, coordinates up to 10^5 in absolute value · **Target** $O(n \log k)$ · **Edge cases** $k = n$; coordinates big enough that $x^2 + y^2$ leaves the 32-bit range.

Approach 1 — Compute every distance, sort, take the first k ($O(n \log n)$ time and $O(n)$ memory)

Approach 2 — Max-heap of size k , keyed on the squared distance ($O(n \log k)$ time, $O(k)$ memory)

SOLUTION

Distance from the origin is $\sqrt{x^2 + y^2}$, but the square root is a waste — comparing $x^2 + y^2$ gives the same order. To keep the k **closest**, use a **max**-heap of size k , so the root is the worst survivor.

Max-heap of size k , keyed on the squared distance

JS

```
const kClosestPoints = (pts, k) => {
  const d2 = ([x, y]) => x * x + y * y; // destructure in the parameter
  const pq = new MinHeap((p, q) => d2(q) - d2(p)); // MAX-heap on distance
  for (const p of pts) {
    pq.push(p);
    if (pq.size > k) pq.pop(); // throw out the farthest
  }
}
```

```
const out = [];
while (pq.size) out.push(pq.pop());
return out.reverse();
};
```

Time $O(n \log k)$ **Space** $O(k)$

Output

TEXT

```
[(1,3),(-2,2),(5,8),(0,1)] k=2 -> (0,1) (-2,2)
[(40000,40000),(1,1)] k=1 -> (1,1)
40000^2 + 40000^2 = 3200000000 exact? true
```

In C++ this example is the classic 32-bit overflow trap: $40000^2 + 40000^2 = 3.2 \times 10^9$ wraps negative without a 64-bit cast, and the far point wins. In JavaScript it is simply right — doubles hold every integer up to $2^{53} - 1$ exactly, so no cast exists and none is needed.

The JS version of the same danger is **larger**: at coordinates around 10^8 the squared distance passes 2^{53} and starts rounding, with no error and no warning. Guard with `Number.isSafeInteger(d2(p))` when the statement allows coordinates that big.

Example 14 [TCS NQT · pattern]

Colliding stones. Repeatedly take the two heaviest stones. If they are equal, both are destroyed; otherwise the lighter one is destroyed and the heavier loses that weight. Return the weight left at the end (0 if nothing is left).

Limits $n \leq 10^4$, weights up to 10^3 · **Target** $O(n \log n)$ · **Edge cases** no stones; one stone; two equal stones.

Approach 1 — Sort the whole list again after every collision ($O(n^2 \log n)$ — the list is re-sorted n times)

Approach 2 — A max-heap, so each round costs $O(\log n)$ ($O(n \log n)$)

Max-heap simulation

JS

```
const lastStone = (st) => {
  const pq = new MinHeap((a, b) => b - a).build(st); // max-heap, built in O(n)
  while (pq.size > 1) {
    const a = pq.pop(), b = pq.pop();
    if (a !== b) pq.push(a - b);
  }
  return pq.size ? pq.peek() : 0;
};
```

Time $O(n \log n)$ **Space** $O(n)$

Output

TEXT

```
[2,7,4,1,8,1] -> 1    [3,3] -> 0    [6] -> 6    [] -> 0
```

`.build(st)` runs the linear build, not n pushes, and it returns the heap so it chains straight onto the constructor. Free speed for six characters.

Example 15 [Infosys · pattern]

Convert a min-heap into a max-heap. You are given an array that satisfies the min-heap rule. Rearrange it so it satisfies the max-heap rule.

Limits $n \leq 10^6$ · **Target** $O(n)$ · **Edge cases** empty; one element.

Approach 1 — Sort the array descending ($O(n \log n)$ — correct, but more than the question needs)

Approach 2 — Run the linear build with the comparison flipped ($O(n)$ — optimal)

SOLUTION

There is no clever shortcut — a min-heap gives almost no information about the max order. So just run the linear build with the comparison flipped. $O(n)$ is already optimal.

Build, with the sign flipped

JS

```
const minHeapToMaxHeap = (a) => {
  for (let i = (a.length >> 1) - 1; i >= 0; i--) maxSiftDown(a, i, a.length);
  return a;
}; // reuses maxSiftDown from heap sort
```

Time $O(n)$ **Space** $O(1)$

Output

TEXT

```
isMaxHeap checks: [9 7 8 1 3]=yes [1 7 8]=no []=yes [5]=yes
[1 3 5 7 9 11] -> [11 9 5 7 3 1] isMaxHeap = yes
```

Tier 2 — applied problems

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 16 [Grab · pattern]

Fewest drivers. A ride app has a list of trips, each with a start and an end minute. One driver can take a new trip only when the previous one has ended. How many drivers are needed?

Limits $n \leq 10^5$ trips · **Target** $O(n \log n)$ · **Edge cases** no trips; one trip ends exactly when the next begins (the same driver can take it).

Approach 1 — For every minute, count the trips covering it ($O(n * T)$ — hopeless when times go up to 10^9)

Approach 2 — Sort by start; a min-heap holds the end times in use ($O(n \log n)$)

SOLUTION

Sort the trips by start time. Keep a min-heap of the **end** times of the busy drivers. For each new trip: if the earliest-finishing driver is already free (`heap.peek() <= start`), reuse them — `pop`. Then push this trip's end. The heap size at any moment is the number of drivers in use, and the final size is the answer.

Sort by start, heap on end

JS

```
const minRooms = (mt) => {
  if (!mt.length) return 0;
  const v = [...mt].sort((a, b) => a[0] - b[0] || a[1] - b[1]); // by start
  const endTimes = new MinHeap();
  for (const [start, end] of v) {
    if (endTimes.size && endTimes.peek() <= start) endTimes.pop();
    endTimes.push(end);
  }
}
```



```

    }
    return endTimes.size;
};

```

Time $O(n \log n)$ **Space** $O(n)$

Output

TEXT

```

[(0,30),(5,10),(15,20)] -> 2
[(7,10),(2,4)]           -> 1
[]                       -> 0
[(1,5),(5,9),(9,12)]    -> 1   (touching ends reuse the same driver)

```

TRAP

Two separate bugs live on the sort line.

1. `mt.sort()` with no comparator compares the **string** forms of the pairs, so `[10, 20]` sorts before `[9, 30]`. Always `(a, b) => a[0] - b[0]`.
2. C++ sorts a pair by first **then** second for free. JavaScript does not: a bare `a[0] - b[0]` leaves equal starts in arbitrary order. Chain the tie-break with `||`.

TRAP

`endTimes.peek() <= start` versus `<`. If the problem says a trip ending at minute 5 and one starting at minute 5 **can** share a driver, use `<=`. If they cannot, use `<`. The two answers differ on the last test line above. Ask.

Example 17 [Shopee · pattern]

Merge k sorted linked lists. Merge k sorted singly linked lists into one sorted list. Reuse the nodes.

Limits total $N \leq 10^6$ nodes, $k \leq 10^4$ · **Target** $O(N \log k)$ · **Edge cases** some lists are null; all are null.

Approach 1 — Collect every value, sort, then rebuild a fresh list ($O(N \log N)$ time and $O(N)$ memory; it does not reuse the nodes)

Approach 2 — Merge two lists at a time ($O(N k)$ — the first list is walked k times)

Approach 3 — A min-heap holding the current head of each list ($O(N \log k)$ time, $O(k)$ memory)

The heap stores nodes, not values

JS

```

const node = (val, next = null) => ({ val, next });

const mergeKLists = (heads) => {
  const pq = new MinHeap((a, b) => a.val - b.val); // compares NODES by .val
  for (const h of heads) if (h) pq.push(h);
  const dummy = node(0);
  let tail = dummy;
  while (pq.size) {
    const cur = pq.pop();
    tail.next = cur; tail = cur;
    if (cur.next) pq.push(cur.next);
  }
  tail.next = null; // cut the old tail loose
}

```

```
    return dummy.next;
};
```

Time $O(N \log k)$ **Space** $O(k)$

Output

TEXT

```
[1,4,7] [2,5] [3,6,8] -> [1 2 3 4 5 6 7 8]
[null, [2], null]    -> [2]
all null             -> []
```

TRAP

Forgetting `tail.next = null` at the end leaves the last node still pointing into the middle of another list. The list then has a cycle and printing it never stops.

Note also for `(const h of heads) if (h) pq.push(h)` — the guard is `if (h)`, which is false for both `null` and `undefined`. A missing list arriving as `undefined` is the JS version of a null pointer, and this one test covers both.

Example 18 [Agoda · pattern]

K-th largest in a stream. Design a class that starts with some values and supports `add(x)`, returning the *k*-th largest value seen so far.

Limits up to 10^5 calls · **Target** $O(\log k)$ per call, $O(k)$ memory · **Edge cases** fewer than *k* values so far; equal values.

Approach 1 — Store everything and sort on every add ($O(n \log n)$ per call — unusable on a stream)

Approach 2 — Keep one min-heap of size *k* alive between calls ($O(\log k)$ per call, $O(k)$ memory)

SOLUTION

The size-*k* min-heap from Example 7, kept alive between calls. It never grows past *k*, so memory does not grow with the stream.

A class around one min-heap

JS

```
class KthLargestStream {
  constructor(k, start = []) {
    this.k = k;
    this.pq = new MinHeap();
    for (const x of start) this.add(x);
  }
  add(x) {
    this.pq.push(x);
    if (this.pq.size > this.k) this.pq.pop();
    // null means fewer than k values have arrived so far
    return this.pq.size === this.k ? this.pq.peek() : null;
  }
}
```

Time $O(\log k)$ per call **Space** $O(k)$

Output — *k* = 3, start = [4,5,8,2]

TEXT

```
add(3) -> 4      add(5) -> 5      add(10) -> 5      add(9) -> 8
k=2 with no start values: add(1) -> not enough yet, add(7) -> 1
```

C++ returns `INT_MIN` for “not enough values yet”. JavaScript has no such sentinel, and picking one is a bug waiting to happen — a stream of real values could contain it. Return `null` and let the caller test `=== null`. Do **not** return `undefined`: that is what a function returns when it forgot to return anything, so it hides mistakes instead of naming a case.

Example 19 [Sea/Shopee · pattern]

K cheapest bundles. Two sorted price lists a and b . A bundle takes one price from each. Return the k bundles with the smallest total, cheapest first.

Limits $n, m \leq 10^5, k \leq 10^4$ · **Target** $O(k \log k)$ · **Edge cases** either list empty; k bigger than $n \times m$; prices up to 10^9 .

Approach 1 — Build all $n \cdot m$ sums and sort ($O(n \cdot m \log(n \cdot m))$) — 10^{10} pairs, impossible)

Approach 2 — Seed the heap with $(a[i], b[0])$ and expand one step at a time ($O(k \log k)$)

SOLUTION

Think of a grid where cell (i, j) is $a[i] + b[j]$. Rows increase to the right and columns increase downward. The cheapest unseen cell is always next to one already taken. So: push the whole first **column** (at most k of it), and each time you pop (i, j) , push $(i, j+1)$.

Only k pops, so only $k+n$ pushes ever happen

JS

```
const kSmallestPairs = (a, b, k) => {
  const out = [];
  if (!a.length || !b.length || k <= 0) return out;
  const pq = new MinHeap((x, y) => x.sum - y.sum);
  for (let i = 0; i < a.length && i < k; i++) pq.push({ sum: a[i] + b[0], i, j: 0 });
  while (pq.size && out.length < k) {
    const { i, j } = pq.pop();
    out.push([a[i], b[j]]);
    if (j + 1 < b.length) pq.push({ sum: a[i] + b[j + 1], i, j: j + 1 });
  }
  return out;
};
```

Time $O(k \log k)$ **Space** $O(k)$

Output

TEXT

```
a=[1,7,11], b=[2,4,6], k=3 -> (1,2) (1,4) (1,6)
a empty -> 0 pairs
a=[1,2], b=[3], k=10 -> only 2 pairs exist, so 2 are returned
```

`{ sum: a[i] + b[0], i, j: 0 }` uses the shorthand: i alone means $i: i$. It reads better than the long form and is what an interviewer expects to see in 2020s JavaScript.

Example 20 [LINE MAN · pattern]

No two the same in a row. Rearrange a string so that no two neighbouring characters are equal. Return an empty string if it cannot be done.

Limits $\text{length} \leq 10^5$, lowercase letters · **Target** $O(n \log 26)$ · **Edge cases** one character; a character that appears more than half the time.

Approach 1 — Try every arrangement of the letters (factorial — impossible past about 10 letters)

Approach 2 — Place the two most common letters first, every round ($O(n \log 26)$ with a max-heap)

SOLUTION

Always place the **two** most common remaining characters next to each other, then put them back with their counts reduced. This spreads the crowded characters out as far as possible. If at the end a single character is left with a count above 1, it is impossible.

Pop two, place two, push back

JS

```
const reorganize = (s) => {
  const cnt = new Map();
  for (const c of s) cnt.set(c, (cnt.get(c) ?? 0) + 1);
  // max-heap on count; on a tie the alphabetically later letter goes first
  const pq = new MinHeap((x, y) =>
    y[0] - x[0] || (x[1] < y[1] ? 1 : x[1] > y[1] ? -1 : 0));
  for (const [c, n] of cnt) pq.push([n, c]);
  let out = '';
  while (pq.size >= 2) {
    const x = pq.pop(), y = pq.pop();
    out += x[1] + y[1];
    if (--x[0] > 0) pq.push(x);
    if (--y[0] > 0) pq.push(y);
  }
  if (pq.size) {
    if (pq.peak()[0] > 1) return ''; // one letter, 2+ copies: stuck
    out += pq.peak()[1];
  }
  return out;
};
```

Time $O(n \log 26)$ **Space** $O(n)$

TRAP

$x[1] - y[1]$ would be NaN here, because the elements are **characters**. Subtraction only works on numbers; strings need an explicit three-way comparison, either the ternary above or $x[1].localeCompare(y[1])$. A comparator that returns NaN leaves the heap in arbitrary order and never errors.

Output

TEXT

```
"aaabbc" -> "abacba"   (no two neighbours match)
"aaab"   -> ""         ('a' appears 3 times out of 4 – impossible)
"a"      -> "a"
""       -> ""
```

SHORTCUT

The impossibility test has a closed form too: it is impossible exactly when the most common character appears more than $\lceil \frac{n}{2} \rceil$ times. Checking that first lets you return early without running the heap at all.

Example 21 [SCB · pattern]

Job sequencing with deadlines. Each job takes one unit of time, has a deadline and a profit. Only one job runs at a time. Choose the set of jobs with the biggest total profit such that every chosen job finishes by its deadline.

Limits $n \leq 10^5$, deadlines up to n , profits up to 10^9 · **Target** $O(n \log n)$ · **Edge cases** no jobs; all deadlines equal to 1; a total profit well past the 32-bit range.

Approach 1 — Try every subset of jobs and check each one fits (2^n — impossible past about 20 jobs)

Approach 2 — Sort by profit, then scan for a free slot before the deadline ($O(n^2)$ with a plain array of slots, or $O(n \alpha(n))$ with a DSU)

Approach 3 — Sort by deadline, take everything, and drop the least profitable when a deadline is broken ($O(n \log n)$ with a min-heap)

SOLUTION

Sort the jobs by deadline. Walk through them, taking every job greedily. Keep the taken profits in a **min-heap**. After adding a job with deadline d , if you now hold more than d jobs you cannot run them all — so drop the least profitable one. That is exactly the job you would regret least.

Take everything, then regret the cheapest

JS

```
const jobProfit = (jobs) => { // jobs[i] = [deadline, profit]
  const v = [...jobs].sort((a, b) => a[0] - b[0] || a[1] - b[1]);
  const taken = new MinHeap();
  let total = 0;
  for (const [deadline, profit] of v) {
    taken.push(profit); total += profit;
    if (taken.size > deadline) total -= taken.pop(); // one job too many
  }
  return total;
};
```

Time $O(n \log n)$ **Space** $O(n)$

Output

TEXT

```
[(2,100),(1,19),(2,27),(1,25),(3,15)] -> 142    (100 + 27 + 15)
[(1,5),(1,6),(1,7)]                  -> 7      (only one slot exists)
[]                                     -> 0
three jobs, deadline 3, profit 1e9 each -> 3000000000
```

3×10^9 is the classic C++ `int` overflow here, and in JavaScript it is simply a number. The JS question is always the same one: can the total pass `Number.MAX_SAFE_INTEGER`? With $n \leq 10^5$ jobs at 10^9 each the worst total is 10^{14} , still inside $2^{53} - 1$. So plain numbers are correct — but say **why** they are correct, because that is the answer the interviewer is listening for.

Example 22 [Razer · pattern]

Seat manager. Seats are numbered $1..n$ and all start free. Support `reserve()` — take the smallest free seat number — and `release(s)` — free seat s again.

Limits $n \leq 10^5$, up to 10^5 calls · **Target** $O(\log n)$ per call · **Edge cases** reserving when nothing is free; releasing a seat that was never taken.

Approach 1 — Keep a boolean array and scan it for the first free seat ($O(n)$ per reserve)

Approach 2 — A min-heap of free seat numbers ($O(\log n)$ per call)

```
class SeatManager {
  constructor(n) {
    this.freeSeats = new MinHeap();
    for (let i = 1; i <= n; i++) this.freeSeats.push(i);
  }
  reserve() { return this.freeSeats.pop(); }
  release(s) { this.freeSeats.push(s); }
}
```

Time $O(\log n)$ per call **Space** $O(n)$

Output — SeatManager(5)

TEXT

```
reserve -> 1   reserve -> 2   release(1)   reserve -> 1   reserve -> 3
```

A cleaner version starts the heap **empty** and keeps a counter `next = 1`. `reserve()` returns `next++` when the heap is empty, so the constructor is $O(1)$ instead of $O(n \log n)$. Mention this — it is the follow-up the interviewer is waiting for.

Tier 3 — product interviews

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

Median of a stream. Numbers arrive one at a time. After each one, report the median of everything seen so far.

Limits up to 10^5 numbers, values up to 10^9 · **Target** $O(\log n)$ per insert, $O(1)$ per query · **Edge cases** the first number; an even count; all numbers equal; negative numbers.

Approach 1 — Keep a sorted array, insert each new value in place ($O(n)$ per insert because of the shifting; $O(n^2)$ overall)

Approach 2 — Two heaps facing each other ($O(\log n)$ per insert, $O(1)$ per query)

THE TWO-HEAP INVARIANT

Split the numbers into a **smaller half** and a **bigger half**.

- `lo` is a **max**-heap holding the smaller half. Its root is the biggest of the small side.
- `hi` is a **min**-heap holding the bigger half. Its root is the smallest of the big side.
- Keep the sizes balanced: `lo.size` equals `hi.size` or is exactly one bigger.

Then the median is `lo.peak()` when the total count is odd, and the average of `lo.peak()` and `hi.peak()` when it is even. Both are $O(1)$ reads.

Insert, then rebalance — always in that order

JS

```
class MedianStream {
  constructor() {
    this.lo = new MinHeap((a, b) => b - a); // max-heap, small half
    this.hi = new MinHeap();               // min-heap, big half
  }
}
```

```

add(x) {
  if (!this.lo.size || x <= this.lo.peek()) this.lo.push(x);
  else this.hi.push(x);
  // rebalance so that lo.size is hi.size or hi.size + 1
  if (this.lo.size > this.hi.size + 1) this.hi.push(this.lo.pop());
  else if (this.hi.size > this.lo.size) this.lo.push(this.hi.pop());
}

median() {
  if (!this.lo.size) return 0;
  if (this.lo.size > this.hi.size) return this.lo.peek();
  return (this.lo.peek() + this.hi.peek()) / 2;
}
}

```

Time $O(\log n)$ insert, $O(1)$ query **Space** $O(n)$

Output

TEXT

```

adding 5, 15, 1, 3, 8, -4 -> medians [5 10 5 4 5 4]
one value [7] -> 7          all equal [2,2,2,2] -> 2          nothing added -> 0

```

C++ needs a (double) cast here or the two `int` tops overflow before the division. JavaScript has one number type, so `(a + b) / 2` is already the true average — `5 / 2` is 2.5, not 2. The flip side is that a median of 4 prints as 4, not 4.0; if the statement wants a fixed number of decimals, format it at the **edge** with `.toFixed(1)` and never inside the algorithm. `this.hi.push(this.lo.pop())` is the whole rebalance: `pop` returns the value it removed, so the transfer is one expression.

The idea that unlocked it: you do not need the data sorted; you need only the **two values in the middle**. Two heaps hold exactly those two at their tops, and nothing else has to move.

The follow-up

“Now it is a sliding window of size k , and old values must leave.”

Heaps cannot remove an arbitrary value. Two honest answers:

1. **A sorted array plus binary search.** Keep the whole window sorted. Binary search finds where the arrival goes and where the departure sits; `splice` does both edits. Each slide is $O(\log k)$ to **find** and $O(k)$ to **shift**.
2. **Two heaps with lazy deletion.** Mark a value as deleted in a side Map and skip it when it surfaces at a root. That restores $O(\log k)$ per slide, but it is much fiddlier to get exactly right under interview pressure.

TRAP

JavaScript has no TreeMap, no multiset, no ordered set. C++ answers this question with `multiset` and a parked iterator; Java with `TreeMap`. There is no equivalent, and `Set/Map` are **unordered** for this purpose — they keep insertion order, not sorted order.

The two workarounds worth memorising:

- a **sorted array** plus binary search (`lowerBound` / `upperBound` from the toolkit), when the window is small or the data is mostly read;
- a **Map plus a separately maintained sorted key list**, when you need both random access by key and ordered traversal.

Say this out loud in the interview. “JavaScript has no ordered map, so I will keep a sorted array and binary search it” is a **correct** answer. Pretending `Object.keys()` comes back sorted is not — it does not, except for integer-like keys, which is a rule you should never lean on.

```
const { lowerBound } = require('./toolkit.js'); // JS Toolkit appendix

const slidingMedian = (a, k) => {
  const out = [];
  if (k <= 0 || a.length < k) return out;
  const w = a.slice(0, k).sort((x, y) => x - y); // the window, kept sorted
  const mid = (arr) => k % 2 ? arr[(k - 1) >> 1]
    : (arr[k / 2 - 1] + arr[k / 2]) / 2;

  out.push(mid(w));
  for (let i = k; i < a.length; i++) {
    w.splice(lowerBound(w, a[i]), 0, a[i]); // insert the arrival
    w.splice(lowerBound(w, a[i - k]), 1); // remove ONE copy of the leaver
    out.push(mid(w));
  }
  return out;
};
```

Time $O(n \log k)$ to locate, $O(nk)$ to shift **Space** $O(k)$

The splices dominate. At $k \leq$ a few thousand the shifts are a memmove and this is fast in practice; past that, use two heaps with lazy deletion.

Output

TEXT

```
[1,3,-1,-3,5,3,6,7] k=3 -> [1 -1 -1 3 5 6]
[1,4,2,3] k=4 -> [2.5]      [5] k=1 -> [5]      k bigger than n -> []
[2,2,2,2] k=2 -> [2 2 2]
```

TRAP

`w.splice(lowerBound(w, x), 1)` removes **one** copy — the first one at or after that index. Do not reach for `filter(v => v !== x)`: with duplicates in the window that deletes **every** copy, and `[2,2,2,2]` silently empties out. The same warning applies to the C++ habit of `multiset::erase(value)`, which also erases all copies.

Example 24 [Amazon · pattern]

K-th smallest in a sorted matrix. An $n \times n$ matrix has every row and every column sorted ascending. Find the k -th smallest value overall.

Limits $n \leq 300$ (so up to 90000 values), $1 \leq k \leq n^2$ · **Target** better than $O(n^2 \log n)$ · **Edge cases** $k = 1$; $k = n^2$; a 1×1 matrix; repeated values.

Approach 1 — Flatten and sort ($O(n^2 \log n)$ time and $O(n^2)$ memory — fine here, but it ignores the structure)

Baseline

JS

```
const kthMatrixSort = (m, k) => {
  const all = m.flat().sort((x, y) => x - y); // numeric comparator, always
  return all[k - 1];
};
```


Time $O(n^2 \log n)$ **Space** $O(n^2)$

C++ would call `nth_element` here and get $O(n^2)$. JavaScript has no partial sort, so either you sort everything or you paste in the quickselect from Example 7 — which does give $O(n^2)$ on average.

Approach 2 — Merge the rows with a heap and stop after k pops ($O(k \log n)$ time, $O(n)$ memory)

Exactly the merge-k-arrays heap, stopped early

JS

```
const kthMatrixHeap = (m, k) => {
  const n = m.length;
  const pq = new MinHeap((x, y) => x.val - y.val);
  for (let r = 0; r < n && r < k; r++) pq.push({ val: m[r][0], r, c: 0 });
  let out = 0;
  for (let step = 0; step < k; step++) {
    const cur = pq.pop();
    out = cur.val;
    if (cur.c + 1 < m[cur.r].length)
      pq.push({ val: m[cur.r][cur.c + 1], r: cur.r, c: cur.c + 1 });
  }
  return out;
};
```

Time $O(k \log n)$ **Space** $O(n)$

When k is near n^2 this is worse than the baseline.

Approach 3 — Binary search on the ANSWER, not on an index ($O(n \log(\max - \min))$ time, $O(1)$ memory)

SOLUTION

Guess a value x . Count how many matrix entries are $\leq x$. Because rows and columns are sorted, that count can be done in $O(n)$ with a staircase walk: start at the top-right, move left while the value is too big, move down otherwise. If the count is at least k , the answer is $\leq x$; otherwise it is bigger. Binary search on x over the value range.

Binary search on the value range

JS

```
const kthMatrixBinary = (m, k) => {
  const n = m.length;
  let lo = m[0][0], hi = m[n - 1][n - 1];
  const countLE = (x) => { // how many entries are <= x
    let cnt = 0, c = n - 1;
    for (let r = 0; r < n; r++) {
      while (c >= 0 && m[r][c] > x) c--; // c never moves back right
      cnt += c + 1;
    }
    return cnt;
  };
  while (lo < hi) {
    const mid = lo + ((hi - lo) >> 1);
    if (countLE(mid) >= k) hi = mid; else lo = mid + 1;
  }
  return lo;
};
```

Time $O(n \log(\max - \min))$ **Space** $O(1)$

TRAP

`(lo + hi) >> 1` is **not** a safe midpoint in JavaScript. `>>` converts to a signed 32-bit integer first, so any sum above $2^{31} - 1$ wraps to a negative number and the search walks off the bottom of the range. `lo + ((hi - lo) >> 1)` keeps the shifted value small and is safe here; when the range itself can exceed 2^{31} , use `Math.floor((hi - lo) / 2)` instead, which has no 32-bit step at all.

All three agree on `[[1,5,9],[10,11,13],[12,13,15]]`

TEXT

```
k=1 -> 1 1 1      k=4 -> 10 10 10
k=8 -> 13 13 13    k=9 -> 15 15 15
1x1 matrix [[-5]], k=1 -> -5
```

The idea that unlocked it: the answer is a **number**, and “how many are $\leq x$ ” only grows as x grows. Anything monotone can be binary searched. This is the bridge between Chapter 4 and this one — and it is why the binary-search version needs no memory at all.

The follow-up

“Why does `lo` end up being an actual matrix entry, and not some value in between?”

Because the loop only ever sets `hi = mid` when `countLE(mid) >= k`, and it never sets `lo` past a value with a big enough count. The smallest x whose count reaches k must be a value that is **present** — if it were absent, $x - 1$ would have the same count, and the search would have moved further left. Be ready to say this; it is the real question.

Example 25 [Microsoft · pattern]

Smallest range covering every list. You are given k sorted lists. Find the shortest range $[x, y]$ that contains at least one number from every list.

Limits $k \leq 3500$, total $N \leq 10^5$ · **Target** $O(N \log k)$ · **Edge cases** one list; all lists identical; a list that is empty.

Approach 1 — Try every pair of values as the range and test all k lists ($O(N^2 k)$ — far too slow)

Approach 2 — One pointer per list; move the pointer sitting on the minimum ($O(N \log k)$ with a min-heap)

SOLUTION

Hold one **pointer per list** — exactly the merge- k -arrays heap again. At any moment the k values under the pointers touch every list, so they form a valid range: from the heap **minimum** to a separately tracked **maximum**. To shrink it, you must move the minimum forward, because moving anything else cannot help. Pop the minimum, advance its list, update the maximum, repeat. Stop when a list runs out.

Heap holds the minimum; a plain variable holds the maximum

JS

```
const smallestRange = (lists) => {
  const pq = new MinHeap((a, b) => a.val - b.val);
  let curMax = -Infinity;
  for (let i = 0; i < lists.length; i++) {
    if (!lists[i].length) return [0, 0]; // a list with nothing in it
    pq.push({ val: lists[i][0], li: i, idx: 0 });
    curMax = Math.max(curMax, lists[i][0]);
  }
  let bestLo = pq.peak().val, bestHi = curMax;
  for (;;) {
    const cur = pq.pop();
    if (curMax - cur.val < bestHi - bestLo) { bestLo = cur.val; bestHi = curMax; }
    if (cur.idx + 1 >= lists[cur.li].length) break; // this list is used up
    const nx = lists[cur.li][cur.idx + 1];
```

```

    curMax = Math.max(curMax, nx);
    pq.push({ val: nx, li: cur.li, idx: cur.idx + 1 });
  }
  return [bestLo, bestHi];
};

```

Time $O(N \log k)$ **Space** $O(k)$

-Infinity is the JS stand-in for INT_MIN, and it is a better one: it is smaller than every number rather than merely very small, so no real input can accidentally equal it. Use Infinity / -Infinity for “no answer yet” seeds throughout this book.

Output

TEXT

```

[[4,10,15,24],[0,9,12,20],[5,18,22,30]] -> [20,24]
[[1,2,3],[1,2,3],[1,2,3]]               -> [1,1]

```

The follow-up

“Why is it safe to stop the moment one list runs out?” Because every later range would still have to contain a number from that list, and the only numbers left there are ones already passed. Any range covering all lists from this point on is at least as wide as one already examined. Say that sentence; it is the proof.

Example 26 [Goldman Sachs · pattern]

Pick projects to grow capital. You start with C rupees and may run at most k projects, one after another. Project i needs `capital[i]` on hand to start and returns `profit[i]` on top of what you already had. Maximise your final capital.

Limits $n \leq 10^5$, $k \leq 10^5$, values up to 10^9 · **Target** $O(n \log n)$ · **Edge cases** nothing affordable at the start; k bigger than n ; a final capital far past the 32-bit range.

Approach 1 — Try every choice of k projects (combinatorial — impossible past about 20 projects)

Approach 2 — Sort by capital needed, max-heap on profit, take k times ($O(n \log n)$)

SOLUTION

Two structures working together:

- Sort the projects by **capital needed**, so you can unlock them in order as money grows.
- A **max-heap** of the **profits** of everything you can currently afford.

Each round: unlock everything newly affordable, then take the single most profitable one. This is safe because profits are never negative, so money only grows, so anything affordable now stays affordable.

Sort by cost, heap by profit

JS

```

const maximizeCapital = (k, start, proj) => { // proj[i] = [capital needed, profit]
  const v = [...proj].sort((a, b) => a[0] - b[0] || a[1] - b[1]);
  const affordable = new MinHeap((a, b) => b - a); // MAX-heap on profit
  let money = start, i = 0;
  for (let step = 0; step < k; step++) {
    while (i < v.length && v[i][0] <= money) affordable.push(v[i++][1]);
    if (!affordable.size) break; // nothing we can start
    money += affordable.pop();
  }
  return money;
};

```

Time $O(n \log n)$ **Space** $O(n)$

Output

TEXT

```
k=2, start 0, [(0,1),(1,2),(2,3)] -> 3
k=3, start 0, [(0,1),(1,2),(2,3)] -> 6
k=5, start 100, [(200,50)] -> 100 (never affordable, so nothing happens)
```

The follow-up

“What if a project can LOSE money?” The greedy breaks immediately: a loss might unlock nothing, and money no longer only grows, so “affordable now” is no longer permanent. The problem stops being greedy and becomes a search or a DP. Recognising that the greedy **proof** depends on non-negative profits is the point of the follow-up.

Example 27 [Uber · pattern]

Fewest refuelling stops. A van starts with `startFuel` litres and burns one litre per kilometre. Stations sit at given positions with given litres. Reach `target` with as few stops as possible, or report that it is impossible.

Limits $n \leq 500$ stations, positions and litres up to 10^9 · **Target** $O(n \log n)$ · **Edge cases** you can already reach the target; the first station is out of reach.

Approach 1 — Dynamic programming on (station, stops used) ($O(n^2)$ — correct, and the usual first answer)

Approach 2 — Drive as far as possible, then take fuel from the best station already passed ($O(n \log n)$ with a max-heap)

SOLUTION

This is greedy **with hindsight**. Drive as far as the fuel allows. Every station you pass is a litre—can you **could** have taken. When you run dry, retroactively take the biggest can you passed. A max-heap of passed station sizes gives it to you.

Take fuel from the past, when you need it

JS

```
const minRefuel = (target, startFuel, st) => { // st[i] = [position, litres]
  const v = [...st].sort((a, b) => a[0] - b[0]);
  const pq = new MinHeap((a, b) => b - a); // MAX-heap of litres passed
  let fuel = startFuel, stops = 0, i = 0;
  while (fuel < target) {
    while (i < v.length && v[i][0] <= fuel) pq.push(v[i++][1]);
    if (!pq.size) return -1; // stranded
    fuel += pq.pop();
    stops++;
  }
  return stops;
};
```

Time $O(n \log n)$ **Space** $O(n)$

Output

TEXT

```
target 100, fuel 10, stations [(10,60),(20,30),(30,30),(60,40)] -> 2
target 100, fuel 100, no stations -> 0
target 100, fuel 1, station at 10 -> -1 (cannot even reach the first station)
```

In C++ `fuel` has to be declared as a 64-bit integer or it wraps after a few stops. In JavaScript `fuel` is just a number and the sum of 500 stations at 10^9 litres is 5×10^{11} — comfortably exact. The only thing worth checking is the **statement's** bound: if litres could reach 10^{15} , start asking about `BigInt`.

The follow-up

“Prove the greedy is correct.” Suppose an optimal plan makes m stops. Our method has always driven at least as far with the same number of stops, because at every point it took the biggest available can. By induction, after m of our stops we are at least as far as the optimal plan, so we never need more than m . That exchange argument is the same one behind Chapter 19's greedy proofs.

Example 28 [D. E. Shaw · pattern]

A heap you can update. Dijkstra's algorithm needs “lower the key of an item already in the heap”. A plain binary heap cannot do that — it can only find its **root**. Build a heap that can, and describe the cheaper alternative.

Limits $n \leq 10^5$ ids, $m \leq 5 \times 10^5$ updates · **Target** $O(\log n)$ per operation · **Edge cases** decreasing the key of something not in the heap; decreasing to a value that is not actually smaller.

Option A — an indexed heap

SOLUTION

Keep three arrays: `key[id]`, `heap` (holding ids), and `pos[id]` (where each id sits in `heap`, or -1 if absent). Every swap updates `pos`, so you can find any id in $O(1)$ and sift it from there.

Indexed min-heap with decrease-key

JS

```
class IndexedHeap {
  constructor(n) {
    this.key = new Array(n).fill(Infinity); // key[id]
    this.heap = []; // holds ids
    this.pos = new Array(n).fill(-1); // pos[id] = its slot, -1 if absent
  }

  less(i, j) { return this.key[this.heap[i]] < this.key[this.heap[j]]; }

  swap(i, j) {
    [this.heap[i], this.heap[j]] = [this.heap[j], this.heap[i]];
    this.pos[this.heap[i]] = i; this.pos[this.heap[j]] = j; // keep pos in step
  }

  up(i) {
    while (i > 0 && this.less(i, (i - 1) >> 1)) {
      this.swap(i, (i - 1) >> 1); i = (i - 1) >> 1;
    }
  }

  down(i) {
    const n = this.heap.length;
    for (;;) {
      const l = 2 * i + 1, r = 2 * i + 2;
      let b = i;
      if (l < n && this.less(l, b)) b = l;
      if (r < n && this.less(r, b)) b = r;
      if (b === i) break;
      this.swap(i, b); i = b;
    }
  }
}
```

```

push(id, k) {
  this.key[id] = k;
  this.heap.push(id);
  this.pos[id] = this.heap.length - 1;
  this.up(this.pos[id]);
}

decrease(id, k) {
  if (this.pos[id] === -1) { this.push(id, k); return; }
  if (k >= this.key[id]) return; // not actually an improvement
  this.key[id] = k; this.up(this.pos[id]); // a smaller key only ever moves up
}

popMin() {
  const id = this.heap[0];
  this.swap(0, this.heap.length - 1);
  this.heap.pop(); this.pos[id] = -1;
  if (this.heap.length) this.down(0);
  return id;
}

get size() { return this.heap.length; }
}

```

Time $O(\log n)$ per operation **Space** $O(n)$

Output — push (0,40) (1,10) (2,70) (3,25), then decrease id 2 to 5

TEXT

pop ids in order: 2 1 3 0

Option B — lazy deletion, which is what people actually write

SOLUTION

Do not update anything. Push the **new** pair and leave the stale one behind. When a value surfaces at the top, check whether it is still current; if not, pop it and carry on. The heap grows to $O(m)$ instead of $O(n)$, but every line is simple and there is no pos array to keep in step.

Two heaps: live values, and values marked for removal

JS

```

class LazyHeap {
  constructor() { this.live = new MinHeap(); this.dead = new MinHeap(); }
  push(x) { this.live.push(x); }
  erase(x) { this.dead.push(x); } // promise to remove it later
  clean() {
    while (this.dead.size && this.live.size &&
      this.live.peek() === this.dead.peek()) {
      this.live.pop(); this.dead.pop();
    }
  }
  top() { this.clean(); return this.live.peek(); }
  get empty() { this.clean(); return this.live.size === 0; }
  get size() { return this.live.size - this.dead.size; }
}

```

Time $O(\log n)$ amortised **Space** $O(m)$, m = total pushes

Output

TEXT

push 5,1,9,3 then erase 1 -> top 3, size 3
erase 3 and 5 -> top 9, empty = false

The follow-up

“Which one goes into your Dijkstra?” Lazy deletion, almost always. The extra memory is $O(m)$ where m is the number of edges, which you are already storing. Every Dijkstra you will write looks like `pq.push([newDist, v])` with an `if (d > dist[u]) continue;` guard at the top of the loop — that guard **is** the lazy deletion. Reserve the indexed heap for when memory is genuinely tight.

Dry run — the running median, step by step

Trace `MedianStream` on the input 5, 15, 1, 3, 8, -4. Remember: `lo` is a max-heap (biggest on top), `hi` is a min-heap (smallest on top), and `lo` may hold one extra item.

x	where it goes	lo (max-heap)	hi (min-heap)	median
5	<code>lo</code> is empty, so <code>lo</code>	[5]	[]	5
15	<code>15 > 5</code> , so <code>hi</code>	[5]	[15]	$(5+15)/2 = 10$
1	<code>1 <= 5</code> , so <code>lo</code>	[5, 1]	[15]	5
3	<code>3 <= 5</code> , so <code>lo</code> ; now <code>lo</code> has 3 and <code>hi</code> has 1, so move 5 across	[3, 1]	[5, 15]	$(3+5)/2 = 4$
8	<code>8 > 3</code> , so <code>hi</code> ; <code>hi</code> now has 3 and <code>lo</code> has 2, so move 5 back	[5, 3, 1]	[8, 15]	5
-4	<code>-4 <= 5</code> , so <code>lo</code> ; <code>lo</code> has 4 and <code>hi</code> has 2, so move 5 across	[3, 1, -4]	[5, 8, 15]	$(3+5)/2 = 4$

The program prints exactly the last column

TEXT

[5 10 5 4 5 4]

The medians print as 5 10 5 4 and not 5.0 10.0 5.0 4.0, because JavaScript has one number type and drops a trailing .0. Only 2.5-style answers show a decimal point. If a judge wants one decimal place, apply `.toFixed(1)` when printing, never inside `median()` — `toFixed` returns a **string**, and a string leaking back into the heap ruins every later comparison.

Reading the table

Three things to notice, because they are what the interviewer probes.

1. **The rebalance runs after every insert, never before.** Insert first, then fix the sizes. Doing it the other way round leaves the new value on the wrong side.
2. **Only one item ever moves.** One insert can unbalance the sizes by at most one, so one transfer is always enough. No loop is needed.
3. **`lo` is allowed to be bigger, `hi` is not.** That asymmetry is what makes the odd case a single read of `lo.peek()`. If you allow it the other way you must test both sizes every time.

A second trace: why build is not n pushes

Build the heap [9, 4, 7, 1, -3, 8, 0]. $n = 7$, so start at index $(7 \gg 1) - 1 = 2$ and walk down to 0.

i	node and its children	action	array after
---	-----------------------	--------	-------------

2	7 with children 8 (idx 5) and 0 (idx 6)	0 is smallest, swap	[9 4 0 1 -3 8 7]
1	4 with children 1 (idx 3) and -3 (idx 4)	-3 is smallest, swap	[9 -3 0 1 4 8 7]
0	9 with children -3 (idx 1) and 0 (idx 2)	-3 is smallest, swap; then 9 at idx 1 has children 1 and 4, so swap with 1	[-3 1 0 9 4 8 7]

The whole build did 4 swaps for 7 elements. Seven separate push calls would have done more work and would have produced a different (also valid) heap.

Python — the signature problems

JavaScript is the language of this book, and a JS answer is always a complete answer. Python is worth knowing for exactly one reason here: `heapq` is in the standard library, so the three signature problems of this chapter — the running median, top-k frequent, and merge-k — collapse to a few lines each.

`heapq` is a MIN-heap; negate to get a max-heap

PYTHON

```
import heapq
from collections import Counter

class MedianStream:
    def __init__(self):
        self.lo = []      # max-heap, stored as negatives
        self.hi = []      # min-heap

    def add(self, x):
        if not self.lo or x <= -self.lo[0]:
            heapq.heappush(self.lo, -x)
        else:
            heapq.heappush(self.hi, x)
            if len(self.lo) > len(self.hi) + 1:
                heapq.heappush(self.hi, -heapq.heappop(self.lo))
            elif len(self.hi) > len(self.lo):
                heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def median(self):
        if not self.lo:
            return 0.0
        if len(self.lo) > len(self.hi):
            return float(-self.lo[0])
        return (-self.lo[0] + self.hi[0]) / 2.0

def top_k_frequent(nums, k):
    cnt = Counter(nums)
    return [v for v, _ in heapq.nsmallest(k, cnt.items(),
                                         key=lambda e: (-e[1], e[0]))]

def merge_k(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        val, li, idx = heapq.heappop(heap)
        out.append(val)
        if idx + 1 < len(lists[li]):
```



```

        heapq.heappush(heap, (lists[li][idx + 1], li, idx + 1))
    return out

```

Output

TEXT

```

medians : [5.0, 10.0, 5.0, 4.0, 5.0, 4.0]
one      : 7.0
empty    : 0.0
all same: 2.0
top2     : [2, -1]
single   : [9]
all same: [5]
tie      : [3, 7]
merge    : [1, 2, 3, 4, 5, 6, 9, 10]
empties  : [] []
negative: [-5, -3, -1, 0]

```

TRAP

heapq has **no** max-heap. Push $-x$ and negate on the way out. If the items are tuples, negate only the sort key: $(-count, value)$, not the whole tuple — otherwise the tie-break flips as well, which is rarely what you want.

Practice

PRACTICE · WARM-UP target: 25 min

1. Is $[2, 5, 4, 9, 6, 8]$ a valid min-heap? Show the check for every parent.
2. In a heap of 50 items, list the indices on the path from index 41 up to the root.
3. Push 0 into the min-heap $[1, 3, 9, 5, 7]$. Show every swap.
4. Pop twice from the max-heap $[9, 7, 8, 1, 3]$. Show the array after each pop.
5. You must sort 10 million integers with almost no extra memory. Heap sort or merge sort? Give one reason each way.

PRACTICE · TIER 1 target: 60 min

6. **Product of the k largest.** Return the product of the k biggest values. $n \leq 10^5$, values up to 10^5 . Edge cases: $k = 1$; negative values; a product past `Number.MAX_SAFE_INTEGER`.
7. **Fewest platforms.** Given arrival and departure times of trains, find the smallest number of platforms so no train waits. Edge cases: one train; a departure equal to the next arrival.
8. **When does the last job finish?** k workers take jobs from a queue in the given order; each worker picks up the next job the moment they are free. Return the finish time of the last job. Edge cases: more workers than jobs; no jobs.
9. **Sort characters by frequency.** Rearrange a string so the most frequent characters come first. Edge cases: empty string; every character once.
10. **A d-ary heap.** Write a min-heap where every node has d children instead of 2. Show that it still pops in sorted order. Edge cases: $d = 2$; one element.

PRACTICE · TIER 2 target: 70 min

11. **Top k words.** Return the k most frequent words, most frequent first, alphabetical on a tie. Edge cases: every word once; an exact tie across all words.
12. **Sliding window maximum.** Return the maximum of every window of size k, using a heap. Edge cases: $k = 1$; $k = n$; all values equal.
13. **Schedule with a cooldown.** Tasks have counts; two runs of the **same** task must be at least `cool` slots apart. Return the total time including idle slots. Edge cases: `cool = 0`; one task type only.
14. **K-th number from given primes.** Return the k-th smallest positive number whose only prime factors come from a given set. Edge cases: one prime; a candidate that leaves the exact-integer range.

PRACTICE · TIER 3 target: 80 min

15. **Cheapest k workers.** Each worker has a quality and a minimum wage. If you hire a group, every member is paid in proportion to their quality, and nobody is paid below their minimum wage. Hire exactly k for the least total. Edge cases: $k = 1$; $k = n$; workers with the same ratio.
16. **K-th largest subarray sum.** Over every contiguous subarray, return the k-th largest sum. Edge cases: all negatives; $k = 1$; $k =$ the total number of subarrays.
17. **Fewest halvings.** You may halve any one value. Repeat until the total has dropped by at least half. Return the number of halvings. Edge cases: one value; all values equal.

ANSWER KEY

1. Parent 0 (=2) vs children 5 and 4 — ok. Parent 1 (=5) vs children 9 and 6 — ok. Parent 2 (=4) vs child 8 — ok. **Valid.**
2. $41 \rightarrow 20 \rightarrow 9 \rightarrow 4 \rightarrow 1 \rightarrow 0$, using $(i-1)/2$ each time.
3. Append 0 at index 5. Parent 2 holds $9 > 0 \rightarrow$ swap: $[1, 3, 0, 5, 7, 9]$, $i = 2$. Parent 0 holds $1 > 0 \rightarrow$ swap: $[0, 3, 1, 5, 7, 9]$, $i = 0$. Stop. Two swaps.
4. First pop returns 9: move 3 to the root $\rightarrow [3, 7, 8, 1]$, sift down (8 is the bigger child) $\rightarrow [8, 7, 3, 1]$. Second pop returns 8: move 1 to the root $\rightarrow [1, 7, 3]$, sift down $\rightarrow [7, 1, 3]$.
5. Heap sort: $O(1)$ extra memory, guaranteed $O(n \log n)$ even in the worst case. Merge sort: stable and far more cache-friendly, but needs $O(n)$ extra memory. With “almost no extra memory” as the constraint, heap sort wins.
- 6 (product of the k largest). Min-heap of size k, then multiply.

Size-k heap, plus the BigInt version

JS

```
const productOfKLargest = (a, k) => {
  const pq = new MinHeap();
  for (const x of a) { pq.push(x); if (pq.size > k) pq.pop(); }
  let p = 1;
  while (pq.size) p *= pq.pop();
  return p;
  // exact while |p| <= 2^53 - 1
};

const productOfKLargestBig = (a, k) => {
  const pq = new MinHeap();
  for (const x of a) { pq.push(x); if (pq.size > k) pq.pop(); }
  let p = 1n;
  while (pq.size) p *= BigInt(pq.pop());
  return p;
  // exact, however big
};
```

Output: $[3, 9, 2, 7]$ $k=2 \rightarrow 63$; $[100000, 90000, 3]$ $k=2 \rightarrow 9000000000$; $[-4, -9, 2]$ $k=2 \rightarrow -8$ (the two largest are 2 and -4); $[5]$ $k=1 \rightarrow 5$.

TRAP

This is the chapter's BigInt case. Four values of 10^5 give 10^{20} , which is past $2^{53} - 1$. Both versions `print` `100000000000000000000`, and only the `BigInt` one is telling the truth — the plain-number answer has silently rounded, and `Number.isSafeInteger` on it returns `false`.

That is the whole danger: a double does not wrap or throw, it **rounds**, and the wrong answer looks completely ordinary. Any time the statement allows a product, or a sum of more than about 10^{15} , either switch to `BigInt` or say out loud why you do not have to. Mixing the two throws — `1n + 1` is a `TypeError` — so convert at the boundary with `BigInt(x)` and `Number(b)`.

7 (platforms). Sort by arrival, min-heap of departures.

Platforms

JS

```
const minPlatforms = (arr, dep) => {
  const t = arr.map((a, i) => [a, dep[i]])
    .sort((x, y) => x[0] - y[0] || x[1] - y[1]);
  const busy = new MinHeap();
  let best = 0;
  for (const [a, d] of t) {
    while (busy.size && busy.peek() < a) busy.pop();
    busy.push(d);
    best = Math.max(best, busy.size);
  }
  return best;
};
```

Output: arrivals `[900,940,950,1100,1500,1800]` with departures `[910,1200,1120,1130,1900,2000]` → 3. One train → 1. `[10,20]` / `[20,30]` → 2, because `busy.peek() < a` is strict: a train departing at 20 does **not** free the platform for a train arriving at 20.

`arr.map((a, i) => [a, dep[i]])` is the JS way to zip two parallel arrays — there is no `zip` in the language.

8 (when does the last job finish). Min-heap of “time each worker becomes free”.

k workers, jobs in the given order

JS

```
const finishTime = (jobs, k) => {
  const freeAt = new MinHeap();
  for (let i = 0; i < k; i++) freeAt.push(0);
  let last = 0;
  for (const j of jobs) {
    const t = freeAt.pop() + j;
    last = Math.max(last, t);
    freeAt.push(t);
  }
  return last;
};
```

Output: jobs `[5,2,3,7,1]` with 2 workers → 12; one job of 4 with 3 workers → 4; no jobs → 0; `[3,3,3]` with 3 workers → 3.

9 (frequency sort). Count, then pop from a max-heap keyed on count.

Frequency sort

JS

```
const frequencySort = (s) => {
  const cnt = new Map();
  for (const c of s) cnt.set(c, (cnt.get(c) ?? 0) + 1);
  // max-heap on count, ties broken by the later character
```

```

const pq = new MinHeap((x, y) =>
  y[1] - x[1] || (x[0] < y[0] ? 1 : x[0] > y[0] ? -1 : 0));
for (const e of cnt) pq.push(e);
let out = '';
while (pq.size) { const [c, n] = pq.pop(); out += c.repeat(n); }
return out;
};

```

Output: "treeeb" → "eeetrbb"; "" → ""; "aA" → "aA" (both appear once, and 'a' = 97 outranks 'A' = 65).
 c.repeat(n) is the JS way to write “n copies of this character”; there is no string-append-with-count.

10 (d-ary heap). Children of i are $d*i+1 \dots d*i+d$; the parent is $(i-1)/d$.

A d-ary min-heap

JS

```

class DaryHeap {
  constructor(d) { this.d = d; this.a = []; }
  get size() { return this.a.length; }
  up(i) {
    while (i > 0) {
      const p = Math.floor((i - 1) / this.d); // no >> shortcut once d != 2
      if (this.a[p] <= this.a[i]) break;
      [this.a[p], this.a[i]] = [this.a[i], this.a[p]]; i = p;
    }
  }
  down(i) {
    const n = this.a.length;
    for (;;) {
      let best = i;
      for (let c = this.d * i + 1; c <= this.d * i + this.d && c < n; c++)
        if (this.a[c] < this.a[best]) best = c;
      if (best === i) break;
      [this.a[i], this.a[best]] = [this.a[best], this.a[i]]; i = best;
    }
  }
  push(x) { this.a.push(x); this.up(this.a.length - 1); }
  peek() { return this.a[0]; }
  pop() {
    const t = this.a[0], last = this.a.pop();
    if (this.a.length) { this.a[0] = last; this.down(0); }
    return t;
  }
}

```

Output: with $d = 4$, pushing 9, 2, 7, 1, 8, 3 pops in the order 1 2 3 7 8 9. A bigger d makes the tree shorter (push gets faster) but each down compares d children (pop gets slower). $d = 4$ is a common compromise.

11 (top k words). A Map counts, and the comparator's tie rule drops the alphabetically **later** word first, so no sorted container is needed at all.

Count in a Map, filter with a size-k heap

JS

```

const topKWords = (w, k) => {
  const cnt = new Map();
  for (const s of w) cnt.set(s, (cnt.get(s) ?? 0) + 1);
  // root = the entry to drop: smallest count, tie -> the alphabetically later
  word
}

```

```
const pq = new MinHeap((a, b) =>
  a[1] - b[1] || (a[0] < b[0] ? 1 : a[0] > b[0] ? -1 : 0));
for (const e of cnt) { pq.push(e); if (pq.size > k) pq.pop(); }
const out = [];
while (pq.size) out.push(pq.pop()[0]);
return out.reverse();
};
```

Output: [grab, bus, grab, mrt, bus, grab, taxi] $k=2 \rightarrow$ grab bus; [b, a, b, a, c] $k=2 \rightarrow$ a b (both appear twice, so alphabetical order decides).

C++ reaches for `map<string, int>` here because it keeps keys sorted. JavaScript has no sorted map, but it does not need one: the comparator already encodes the tie rule, so an unordered `Map` plus the heap is enough.

12 (sliding window maximum with a heap). Push (value, index). Before reading the top, throw away anything whose index has left the window.

Lazy deletion by index

JS

```
const windowMax = (a, k) => {
  // MAX-heap on [value, index]
  const pq = new MinHeap((x, y) => y[0] - x[0] || y[1] - x[1]);
  const out = [];
  for (let i = 0; i < a.length; i++) {
    pq.push([a[i], i]);
    if (i >= k - 1) {
      while (pq.peak()[1] <= i - k) pq.pop(); // stale, drop it
      out.push(pq.peak()[0]);
    }
  }
  return out;
};
```

Output: [1,3,-1,-3,5,3,6,7] $k=3 \rightarrow$ [3 3 5 5 6 7]; [4] $k=1 \rightarrow$ [4]; [2,2,2] $k=2 \rightarrow$ [2 2]; [-5,-2,-8] $k=3 \rightarrow$ [-2]. This is $O(n \log n)$. A **monotonic deque** — the toolkit's `Deque`, used in Chapter 9 — does the same job in $O(n)$. Say so, then write whichever the interviewer asks for.

13 (schedule with a cooldown). Each round, take up to `cool + 1` different tasks, always the ones with the highest remaining count.

Fill one cooldown block at a time

JS

```
const scheduleTime = (counts, cool) => {
  const pq = new MinHeap((a, b) => b - a); // max-heap
  let time = 0;
  for (const c of counts) if (c > 0) pq.push(c);
  while (pq.size) {
    const hold = [];
    let slot = 0;
    while (slot <= cool && pq.size) {
      hold.push(pq.pop() - 1);
      slot++; time++;
    }
    for (const h of hold) if (h > 0) pq.push(h);
    if (pq.size) time += cool + 1 - slot; // pad with idle slots
  }
  return time;
};
```

Output: counts [3,3,3] with cool = 2 → 9 (A B C A B C A B C, no idling); [3,1,1] with cool = 2 → 7 (A B C A idle idle A); [1,1,1] with cool = 0 → 3; no tasks → 0.

14 (k-th number from given primes). A min-heap plus a `Set` so each value is produced once.

Grow the set of reachable numbers, smallest first

JS

```
const kthMultiple = (primes, k) => {
  const pq = new MinHeap();
  const seen = new Set([1]);
  pq.push(1);
  let cur = 1;
  for (let step = 0; step < k; step++) {
    cur = pq.pop();
    for (const p of primes) {
      const nx = cur * p;
      if (Number.isSafeInteger(nx) && !seen.has(nx)) { // stays exact
        seen.add(nx); pq.push(nx);
      }
    }
  }
  return cur;
};
```

Output: primes [2,3,5] → k=1 gives 1, k=5 gives 5, k=10 gives 12, k=40 gives 144. Primes [7] → k=5 gives 2401. k=1500 with [2,3,5] gives 859963392.

TRAP

`Number.isSafeInteger(nx)` is the JS version of C++'s `nx / p == cur` overflow guard, and it guards against something subtler. C++ **wraps** and pushes a negative number, which is obvious the moment you print it. JavaScript **rounds**: `cur * p` stays positive, stays plausible, and is simply wrong — and two rounded values can even collide, so the `Set` stops de-duplicating correctly too. Check the guard, or work in `BigInt`.

Note also `new Set([1])` and `seen.has(nx)`. A `Set` of numbers is the right structure here: an object used as a set would stringify every key, and `Set` gives $O(1)$ membership with the type preserved.

15 (cheapest k workers). Everyone in the group is paid `ratio * quality`, where `ratio` is the **largest** wage-to-quality ratio in the group. So sort by ratio, and for each worker treat their ratio as the group's ratio while a max-heap keeps the **k smallest** qualities seen so far.

Sort by ratio, max-heap on quality

JS

```
const minCostHire = (quality, wage, k) => {
  const w = quality.map((q, i) => [wage[i] / q, q]) // [ratio, quality]
    .sort((a, b) => a[0] - b[0] || a[1] - b[1]);
  const pq = new MinHeap((a, b) => b - a); // biggest quality on top
  let sumQ = 0, best = Infinity;
  for (const [ratio, q] of w) {
    pq.push(q); sumQ += q;
    if (pq.size > k) sumQ -= pq.pop();
    if (pq.size === k) best = Math.min(best, ratio * sumQ);
  }
  return best;
};
```

Checked against a brute force over every subset of size k: quality [10,20,5], wage [70,50,30], k=2 → 105.0000 from both. quality [3,1,10,10,1], wage [4,8,2,2,7], k=3 → 30.6667 from both. One worker, k=1 → 11.0000 from both.

Note `wage[i] / q` with no cast. JavaScript `/` is always floating-point division, so `7 / 2` is `3.5` — the opposite of the C++ habit, where the cast is what you must not forget. When you actually want the **integer** quotient, that is `Math.floor(a / b)`, or `(a / b) | 0` for values that fit in 32 bits.

16 (k-th largest subarray sum). Prefix sums turn every subarray into a difference; then a size-*k* min-heap keeps the best sums.

Prefix sums plus a size-k heap

JS

```
const kthLargestSubarraySum = (a, k) => {
  const n = a.length;
  const pre = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
  const pq = new MinHeap();
  for (let i = 0; i < n; i++)
    for (let j = i + 1; j <= n; j++) {
      pq.push(pre[j] - pre[i]);
      if (pq.size > k) pq.pop();
    }
  return pq.peek();
};
```

Output: `[2, -1, 3]` → *k*=1 gives 4, *k*=3 gives 2, *k*=6 gives -1; `[-4]` *k*=1 gives -4. This is $O(n^2 \log k)$, which is fine to $n \approx 3000$. For bigger *n* you binary search on the answer and count subarrays with a sum above the guess.

17 (fewest halvings). Always halve the current largest — a max-heap.

Halve the biggest, repeat

JS

```
const halveSum = (a) => {
  const pq = new MinHeap((x, y) => y - x); // max-heap
  let total = 0;
  for (const x of a) { pq.push(x); total += x; }
  let removed = 0, ops = 0;
  while (removed < total / 2) {
    const big = pq.pop();
    removed += big / 2;
    pq.push(big / 2);
    ops++;
  }
  return ops;
};
```

Output: `[5, 19, 8, 1]` → 3; `[3, 8, 20]` → 3; `[1]` → 1; `[4, 4, 4, 4]` → 4. The halves are fractions, and JS numbers already **are** doubles, so nothing needs to be declared `double` and nothing needs `/ 2.0` — `/ 2` is the same operation. Why greedy is safe: halving the biggest value removes more than halving anything else, and the choice made now does not restrict any later choice.

ONE-PAGE REVISION CARD

The invariant

Min-heap: every parent $\leq$ both children. The array is complete, so parent of i is $(i - 1) \gg 1$ and children are $2i+1, 2i+2$. The heap knows **one** thing — the extreme item — and knows it in $O(1)$.

Template

The four shapes that cover this whole chapter

JS

```
const { MinHeap } = require('./toolkit.js'); // JS Toolkit appendix

// 1. keep the k LARGEST -> MIN-heap of size k (and vice versa)
const pq = new MinHeap();
for (const x of a) { pq.push(x); if (pq.size > k) pq.pop(); }
// pq.peak() is now the k-th largest

// 2. merge k sources -> one entry per source, refill on pop
const pq2 = new MinHeap((x, y) => x.val - y.val);
lists.forEach((l, i) => { if (l.length) pq2.push({ val: l[0], li: i, idx: 0 }); });

// 3. running median -> max-heap lo, min-heap hi, lo.size in {hi.size, hi.size + 1}
const lo = new MinHeap((p, q) => q - p), hi = new MinHeap();
if (!lo.size || x <= lo.peak()) lo.push(x); else hi.push(x);
if (lo.size > hi.size + 1) hi.push(lo.pop());
else if (hi.size > lo.size) lo.push(hi.pop());

// 4. greedy "always take the best available" -> sort by the gate, heap on the prize
items.sort((p, q) => p.gate - q.gate); // NEVER a bare .sort()
while (i < items.length && items[i].gate <= budget) pq.push(items[i++].prize);
budget += pq.pop();
```

Complexity table

Operation	Time	Space	Note
push / pop	$O(\log n)$	$O(1)$	one root-to-leaf path
top	$O(1)$	$O(1)$	it is just <code>a[0]</code>
build from an array	$O(n)$	$O(1)$	never push in a loop
heap sort	$O(n \log n)$	$O(1)$	worst case too; not stable
k largest of n	$O(n \log k)$	$O(k)$	min-heap of size k
k-th largest, offline	$O(n)$ average	$O(n)$	quickselect; JS has no partial sort
top-k frequent	$O(n)$	$O(n)$	bucket by count beats the heap
merge k lists, N items	$O(N \log k)$	$O(k)$	one entry per list
running median	$O(\log n)$	$O(n)$	two heaps
sliding window median	$O(nk)$ worst	$O(k)$	sorted array + binary search

sliding window maximum	$O(n \log n)$	$O(n)$	the toolkit <code>Deque</code> does it in $O(n)$
decrease-key (indexed heap)	$O(\log n)$	$O(n)$	or lazy deletion, $O(m)$ memory

Top traps

1. **`arr.sort()` is lexicographic.** `[10, 9, 1].sort()` is `[1, 10, 9]`. Every sort in this chapter needs $(a, b) \Rightarrow a - b$, and every tie-break chains with `||`.
2. **A comparator must return a number.** $(a, b) \Rightarrow a.cnt > b.cnt$ returns a boolean, which coerces to `1/0`; the heap silently stops ordering.
3. **JavaScript has no heap.** Write it or require it. Saying “I’d use a priority queue” and then not having one is the failure mode.
4. **k largest needs a MIN-heap.** Reach for the opposite of what feels natural. A max-heap is just `new MinHeap((a, b) => b - a)`.
5. **Numbers are doubles.** Exact only to $2^{53} - 1$. Past that they **round** silently rather than wrap — guard with `Number.isSafeInteger`, or use `BigInt`.
6. **JavaScript has no `TreeMap` / ordered set.** Sorted array plus binary search, or a `Map` plus a maintained key list.
7. **Map, not a plain object, for counting.** Object keys stringify, so `-1` and `"-1"` collide and the values come back as strings.
8. **`new Array(n).fill([])` shares one array.** Use `Array.from({length: n}, () => [])`; the same rule makes `[...grid]` a shallow copy and `grid.map(r => [...r])` a real one.
9. **The heap must hold one item per list, not one list.** Otherwise merge-k is $O(N \log N)$.
10. **pop moves the LAST element to the root.** Not the smaller child.
11. **Building beats pushing.** `new MinHeap().build(v)` is $O(n)$; pushing in a loop is $O(n \log n)$.
12. **Heaps cannot delete a middle item.** Use lazy deletion, an indexed heap, or a sorted array.
13. **`filter(v => v !== x)` removes every copy.** To delete one, `splice(lowerBound(w, x), 1)`.
14. **heapq in Python has no max-heap.** Negate the key, and only the key.
15. **A heap is not sorted.** `[1, 3, 9, 5, 7]` is a perfectly good heap and a terrible sorted array. Never print the array and call it the answer.

When to reach for what

You need	Use
the extreme item, repeatedly, with inserts	the toolkit <code>MinHeap</code>
the k best of a stream	min-heap of size k (max-heap for k smallest)
the k-th best of a fixed array, once	quickselect — $O(n)$ average
the whole thing sorted	<code>a.sort((x, y) => x - y)</code>
the max of a sliding window	monotonic deque — $O(n)$
the median of a sliding window	sorted array + <code>lowerBound</code>
delete an arbitrary item	lazy deletion, or a sorted array
lower a key that is already inside	indexed heap, or push a duplicate and skip stale ones
counts bounded by n	bucket array — $O(n)$, no heap at all
exact arithmetic past $2^{53} - 1$	<code>BigInt</code>

Graphs I — Traversal & Components

Store the graph once. Then BFS and DFS answer almost everything.

Pattern in one page

A graph is just **dots and lines**. The dots are called **nodes** (or vertices). The lines are called **edges**. That is all.

THE FOUR WORDS YOU MUST KNOW

- **n** — how many nodes. Nodes are numbered $0, 1, \dots, n - 1$.
- **m** — how many edges.
- **Undirected** edge: a two-way road. If u and v are joined, you can walk both ways.
- **Directed** edge: a one-way road. $u \rightarrow v$ only.
- **Weighted** edge: the road has a number on it (cost, time, distance). This chapter uses **unweighted** graphs only. Weights come in Chapter 16.

How to store a graph

There are two ways. You will use the first one in almost every interview.

	Adjacency list	Adjacency matrix
Shape	<code>Array.from({length: n}, () => [])</code>	<code>Array.from({length: n}, () => new Array(n).fill(0))</code>
Memory	$O(n + m)$	$O(n^2)$
List all neighbours of u	$O(\deg(u))$ — perfect	$O(n)$ — wasteful
Ask “is $u-v$ an edge?”	$O(\deg(u))$	$O(1)$ — perfect
Use when	normal graphs (this is the default)	$n \leq 500$ and you ask about single pairs a lot

SHORTCUT

Rule of thumb: **always start with an adjacency list**. Switch to a matrix only when the problem literally hands you an $n \times n$ table, or n is tiny (under 500) and you must test single pairs quickly.

Build an adjacency list from an edge list

JS

```
const buildAdj = (n, edges) => {
  const g = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    g[u].push(v);
    g[v].push(u);          // delete this line for a DIRECTED graph
  }
  return g;
};
```

TRAP

`Array.from({length: n}, () => [])` — never `new Array(n).fill([])`. `fill` puts **the same array object** in all n slots, so `g[0].push(1)` also appears in `g[5]` and every other row. The graph silently becomes one giant row.

The same rule governs grids: `[...grid]` copies the outer array but **shares** every row, so writing to `copy[0][0]` writes to `grid[0][0]` too. A real 2D copy is `grid.map(r => [...r])`, and this chapter's flood fills depend on it.

For edges `[0,1]`, `[0,2]`, `[1,3]`, `[4,5]` with $n = 7$ this prints:

Real output of the code above

TEXT

```
0 -> 1 2
1 -> 0 3
2 -> 0
3 -> 1
4 -> 5
5 -> 4
6 ->
```

Node 6 has an empty row. It exists, but it touches nothing.

BFS — breadth first search

BFS walks outward in **rings**. First the source. Then everything one edge away. Then everything two edges away. It uses a **queue** (first in, first out).

BFS template — memorise this

JS

```
const bfsDist = (g, src) => {
  const dist = new Array(g.length).fill(-1); // -1 means "not reached yet"
  const q = [src];
  dist[src] = 0;
  for (let head = 0; head < q.length; head++) {
    const u = q[head]; // read, do not remove
    for (const v of g[u])
      if (dist[v] === -1) { // first time we see v
        dist[v] = dist[u] + 1;
        q.push(v);
      }
  }
  return dist;
};
```

TRAP

JavaScript has no queue. An array is a stack (`push` / `pop` are $O(1)$), but `q.shift()` — the obvious “remove from the front” — is $O(n)$, because every remaining element slides down one slot. A BFS written with `shift()` is $O(n^2)$ and will time out on 10^5 nodes while looking completely correct.

Two fixes, and you should know both:

- **A head index**, as above. The array only grows, `head` only moves forward, and `q.length - head` is the live size. Simplest, and what this chapter uses.
- **The toolkit Deque** (`const { Deque } = require('./toolkit.js')`), when you also need to push to the **front** — 0-1 BFS in Chapter 16, or a monotonic deque.

THE BFS INVARIANT — WHY IT WORKS

Nodes leave the queue in order of increasing distance from the source. So the **first** time BFS touches a node, it has already found the **shortest** route to it. That is why you mark a node the moment you push it, never when you pop it.

TRAP

Mark `dist[v]` (or `visited[v]`) **when you push**, not when you pop. If you mark on pop, the same node can be pushed many times before it is popped once, and the queue explodes.

DFS — depth first search

DFS dives as deep as it can, then backs up. It uses the **call stack** (or your own stack).

DFS template, recursive

JS

```
const dfs = (u, g, vis, order) => {
  vis[u] = 1;
  order.push(u);
  for (const v of g[u])
    if (!vis[v]) dfs(v, g, vis, order);
};
```

DFS template, iterative (safe for very deep graphs)

JS

```
const dfsIter = (g, src) => {
  const vis = new Array(g.length).fill(0), order = [];
  const st = [src];
  while (st.length) {
    const u = st.pop();
    if (vis[u]) continue; // it may have been pushed twice
    vis[u] = 1;
    order.push(u);
    for (let i = g[u].length - 1; i >= 0; i--)
      if (!vis[g[u][i]]) st.push(g[u][i]);
  }
  return order;
};
```

On the graph above, both DFS versions print `0 1 3 2` and BFS prints `0 1 2 3`. BFS distance from node 0 is `0 1 1 2 -1 -1 -1`.

TRAP

Recursion depth in Node is roughly 10^4 frames, far below C++ or Java. This is not a warning about a rare edge case — it is the single most common way a correct JS graph solution fails.

Measured, on a chain of 10^5 nodes:

```
recursive DFS -> RangeError: Maximum call stack size exceeded
iterative DFS -> visits all 100000 nodes
```

So: if the constraint says $n \leq 10^5$ **and** the graph can be a long chain, a recursive DFS is not an option. Use BFS, or the explicit-stack `dfsIter` above. Recursive DFS in this chapter is shown because it is the clearest way to **explain** an algorithm — say “recursive for clarity, and I would rewrite it with an explicit stack for 10^5 nodes”, and the interviewer will nod.

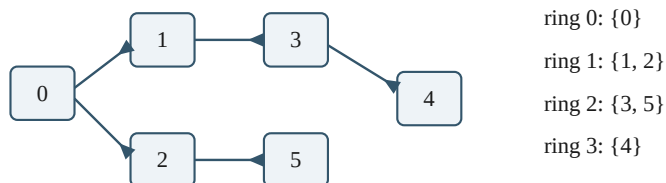
The rewrite has a pattern worth memorising. For a plain visit-order DFS, a stack of nodes is enough. For anything that does work **after** the children return — low-link, finish times, subtree sizes — push [node, childIndex] pairs and advance the index, doing the “after child” work when the index runs out. The Kosaraju code later in this chapter is written exactly that way.

When to reach for which

Question	Tool
Fewest edges from A to B	BFS
Distance rings, “how many minutes”	BFS (often multi-source)
Is there any path at all	either — BFS is safer
Count connected pieces	DFS or BFS or DSU
Cycle in an undirected graph	DFS/BFS with parent, or DSU
Cycle in a directed graph	DFS with 3 colours
Bridges, articulation points	DFS with entry time and low-link
Many “are A and B joined” queries while edges arrive	DSU

COMPLEXITY OF EVERY TRAVERSAL IN THIS CHAPTER

Time $O(n + m)$. Space $O(n + m)$ for the list, plus $O(n)$ for visited and the queue or stack. That is it. If your answer is worse than $O(n + m)$ for a plain traversal, something is wrong.



BFS grows in rings; DFS follows one thread to the end.

Warm-up

WARM-UP

build the reflex

Example 1

The edges of an undirected graph with $n = 7$ are $[0, 1]$, $[0, 2]$, $[1, 3]$, $[4, 5]$. Write the degree of every node. (Degree = number of edges touching the node.)

SOLUTION

Walk the edge list and add 1 to **both** ends of each edge.

node	0	1	2	3	4	5	6
degree	2	2	1	1	1	1	0

```
const degrees = (g) => g.map(row => row.length);
```

Answer: 2 2 1 1 1 1 0

Example 2

Same graph. How many edges does it have, if all you are given is the adjacency list?

SOLUTION

Every undirected edge is stored **twice**, once in each end's row. So add up all row lengths and halve.

$$2 + 2 + 1 + 1 + 1 + 1 + 0 = 8, \quad \frac{8}{2} = 4$$

Count edges

```
const countEdges = (g) => g.reduce((s, row) => s + row.length, 0) / 2;
```

Answer: 4

C++ needs a 64-bit integer here because the degree sum of a big graph passes the 32-bit limit. JavaScript numbers are doubles and hold every integer up to $2^{53} - 1$ exactly, so a sum of $2m \leq 4 \times 10^5$ is not remotely a concern. The JS thing to notice instead is that `/ 2` is **floating-point** division — it returns `3.5` on an odd total rather than truncating — which is a useful accident here: a fractional answer means your adjacency list is malformed.

TRAP

A **self loop** (edge from node 2 back to node 2) is pushed twice into row 2, so it adds 2 to the degree of one node. The formula still works, but many people forget self loops exist. Verified: on edges `[0,1]`, `[0,1]`, `[2,2]` the degrees are 2 2 2.

Example 3

Run BFS from node 0 on `[0,1]`, `[0,2]`, `[1,3]`, `[4,5]` with $n = 7$. Give the order nodes leave the queue, and the distance array.

SOLUTION

pop	pushed now	distance array
0	1, 2	0 1 1 - - -
1	3	0 1 1 2 - -
2	none	0 1 1 2 - -
3	none	0 1 1 2 - -

Nodes 4, 5 and 6 are never reached from 0, so they keep `-1`.

Answer: order 0 1 2 3, distance 0 1 1 2 -1 -1 -1

Example 4

Same graph, but run DFS from node 0. Neighbours are visited in the order they appear in the adjacency row.

SOLUTION

Row 0 is 1 2. Row 1 is 0 3.

0 → 1 (first neighbour) → 0 is used → 3 → 3 has only 1, used → back up to 1, back up to 0 → 2.

Answer: 0 1 3 2

Compare with BFS: 0 1 2 3. Both visit the same four nodes. Only the **order** differs. If a question asks about **shortest** anything, the order matters and you need BFS.

Example 5

A graph has 5 nodes and these **directed** edges: 0->1, 1->2, 2->0, 2->3. Write the adjacency list.

SOLUTION

Push only into the row of the **source**.

Directed adjacency list

TEXT

```
0 -> 1
1 -> 2
2 -> 0 3
3 ->
4 ->
```

Answer: as shown above

Example 6

You must answer 10^5 queries of the form “is there an edge between u and v ?” on a graph with $n = 400$ nodes. Which storage do you choose, and why?

SOLUTION

A matrix. $400 \times 400 = 160000$ cells, about 160 KB as char — tiny. Each query is then $O(1)$.

With an adjacency list each query costs $O(\deg(u))$, which can be 400. Total $10^5 \times 400 = 4 \times 10^7$ — it would pass, but the matrix is simpler and faster.

Answer: adjacency matrix

Tier 1 — the standard traversal questions

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [Infosys · pattern]

Count connected components. Given n nodes and a list of undirected edges, count how many separate pieces the graph breaks into. An isolated node counts as one piece.

Constraints: $n \leq 10^5$, $m \leq 2 \times 10^5$. Target $O(n + m)$. Edge cases: no edges at all (answer n); one node (answer 1).

SOLUTION

Approach 1 — Reachability table ($O(n(n+m))$) — too slow

For every node run a BFS and record the whole set it can reach. Then sweep and count how many distinct sets appear.

Brute force — correct but heavy

JS

```
const componentsBrute = (n, g) => {
  const reach = Array.from({ length: n }, () => new Array(n).fill(false));
  for (let s = 0; s < n; s++) {
```

```

const q = [s]; reach[s][s] = true;
for (let head = 0; head < q.length; head++) {
  const u = q[head];
  for (const v of g[u]) if (!reach[s][v]) { reach[s][v] = true; q.push(v); }
}
const done = new Array(n).fill(false);
let c = 0;
for (let i = 0; i < n; i++) {
  if (done[i]) continue;
  c++;
  for (let j = 0; j < n; j++) if (reach[i][j]) done[j] = true;
}
return c;
};

```

Time $O(n(n+m))$ **Space** $O(n^2)$

The $n \times n$ table alone is 10^{10} bytes at $n = 10^5$. Impossible.

Approach 2 — Disjoint set union (DSU) $(O(m \alpha(n))$ — good)

Start with n islands. Every edge that joins two different islands reduces the count by one.

DSU from the toolkit — do not rewrite it per problem

JS

```

const { DSU } = require('./toolkit.js'); // JS Toolkit appendix

const countComponentsDSU = (n, edges) => {
  const d = new DSU(n);
  let c = n;
  for (const [u, v] of edges) if (d.union(u, v)) c--; // true = they merged
  return c;
};

```

The toolkit DSU is the tested one this book uses everywhere: `find` with path halving, `union` by rank, and `union` returning `true` only when two different sets actually merged. That return value is the whole trick behind counting components, counting redundant edges, and Kruskal's algorithm in Chapter 16 — it is the question “did this edge join two pieces, or close a loop?” answered in $O(\alpha(n))$.

Every DSU-flavoured problem in this chapter is one `require` line plus five lines of logic. Writing the structure out again is a waste of interview minutes; say “I’d use a DSU with path compression and union by rank” and use it.

Time $O(m\alpha(n))$ **Space** $O(n)$

$\alpha(n)$ is under 5 for any n you will ever see. Treat it as $O(1)$.

Approach 3 — One DFS per unvisited node $(O(n+m))$ — optimal, simplest

Loop over all nodes. When you find one that is still unvisited, that is a brand new component: count it, then flood the whole component so you never count it again.

The version to write in an interview

JS

```

const dfsC = (u, g, vis) => {
  vis[u] = 1;
  for (const v of g[u]) if (!vis[v]) dfsC(v, g, vis);
};

```



```
const countComponents = (n, g) => {
  const vis = new Array(n).fill(0);
  let c = 0;
  for (let i = 0; i < n; i++)
    if (!vis[i]) { c++; dfsC(i, g, vis); }
  return c;
};
```

Time $O(n + m)$ **Space** $O(n + m)$

TRAP

This is the recursive one, and at $n = 10^5$ it throws `RangeError: Maximum call stack size exceeded` the moment the graph contains a long chain. Swap `dfsC` for a BFS flood — `const q = [i]` with a head index, exactly as in the `bfsDist` template — and the shape of the solution does not change at all. Every later flood in this chapter is written that way for precisely this reason.

All three were run on $n = 7$, edges $[0, 1], [0, 2], [1, 3], [4, 5]$, and all three printed 3 (the piece $\{0, 1, 2, 3\}$, the piece $\{4, 5\}$, and the lone node 6). A random cross-check over 500 graphs found zero disagreements.

The idea that unlocked it: you never need pairwise information. One outer loop plus one flood is enough, because a flood consumes the whole piece in one go.

Answer: 3

Example 8 [TCS NQT · pattern]

Count the islands. A grid of '1' (land) and '0' (water). Land cells that touch side by side (up, down, left, right — not diagonally) form one island. Count the islands.

Constraints: grid up to 1000×1000 . Target $O(RC)$. Edge cases: empty grid; grid that is all water; a single '1'.

SOLUTION

A grid is a graph. Cell (r, c) is a node; its neighbours are the four cells next to it. So this is Example 7 wearing a costume.

Approach 1 — Recursive DFS flood ($O(RC)$ but risky)

It works, but a 1000×1000 grid of all land makes one million nested calls. Many judges crash.

Approach 2 — BFS flood ($O(RC)$, no recursion — optimal)

Count islands with a BFS flood

JS

```
const numIslands = (grid) => {
  if (!grid.length) return 0;
  const g = grid.map(r => [...r]); // REAL 2D copy – we sink land as we go
  const R = g.length, C = g[0].length;
  const dr = [1, -1, 0, 0], dc = [0, 0, 1, -1];
  let cnt = 0;
  for (let i = 0; i < R; i++)
    for (let j = 0; j < C; j++) {
      if (g[i][j] !== '1') continue;
      cnt++;
      const q = [[i, j]];
      g[i][j] = '0'; // sink it: this is our "visited"
      for (let head = 0; head < q.length; head++) {
```

```

    const [r, c] = q[head];
    for (let k = 0; k < 4; k++) {
        const nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= R || nc < 0 || nc >= C || g[nr][nc] !== '1')
            continue;
        g[nr][nc] = '0';
        q.push([nr, nc]);
    }
}
return cnt;
};

```

Time $O(RC)$ **Space** $O(RC)$

Sinking the land means no separate visited array; the copy is what costs the space.

Tested grid:

Input and real output

TEXT

```

1 1 0 0
1 0 0 1
0 0 1 1
0 0 0 0    -> islands = 2
empty grid -> 0
[[0,0],[0,0]] -> 0
[[1]]       -> 1
caller's grid afterwards: unchanged

```

TRAP

`grid.map(r => [...r])` is the copy, and it matters twice over.

1. Without a copy at all, the function **destroys** the caller's grid. An interviewer who calls your function twice and gets 2 then 0 has found a real bug.
2. With `[...grid]` instead — the obvious-looking “copy” — the outer array is new but every row is still the **same** array object, so sinking land still writes through to the original. Run it and see: `const c = [...grid]; c[0][0] = 'X';` leaves `grid[0][0]` equal to 'X'.

For a grid of numbers there is also `structuredClone(grid)`, which deep-copies anything. It is slower, and `map(r => [...r])` is what you should write on a whiteboard.

The idea that unlocked it: the four direction arrays `dr / dc` turn “grid” into “graph” with three lines of code.

Answer: 2

Example 9 [Wipro · pattern]

Cycle in an undirected graph. Return true if the graph contains any cycle. The graph may be disconnected.

Constraints: $n \leq 10^5$. Target $O(n + m)$. Edge cases: a single node with no edges (no cycle); two nodes joined by two parallel edges (that IS a cycle).

SOLUTION

Approach 1 — Count edges inside each component ($O(n+m)$, easy to explain)

A connected piece with k nodes and no cycle is a **tree**, and a tree has exactly $k - 1$ edges. So a piece is cycle-free exactly when edges = nodes $- 1$. If any piece has edges $\geq$ nodes, there is a cycle.

```
const cycleByCounting = (n, E, g) => {
  const comp = new Array(n).fill(-1);
  let c = 0;
  for (let s = 0; s < n; s++) {
    if (comp[s] !== -1) continue;
    const q = [s]; comp[s] = c;
    for (let head = 0; head < q.length; head++) {
      const u = q[head];
      for (const v of g[u]) if (comp[v] === -1) { comp[v] = c; q.push(v); }
    }
    c++;
  }
  const nodes = new Array(c).fill(0), edges = new Array(c).fill(0);
  for (let i = 0; i < n; i++) nodes[comp[i]]++;
  for (const [u] of E) edges[comp[u]]++; // destructure just the first endpoint
  for (let i = 0; i < c; i++) if (edges[i] >= nodes[i]) return true;
  return false;
};
```

Time $O(n + m)$ **Space** $O(n)$

Approach 2 — BFS with a parent array ($O(n+m)$ — optimal, one pass)

Walk the graph. If you meet a node that is already seen, and it is **not** the node you came from, you have closed a loop.

```
const hasCycleUndirected = (n, g) => {
  const par = new Array(n).fill(-2); // -2 = not seen yet
  for (let s = 0; s < n; s++) {
    if (par[s] !== -2) continue;
    par[s] = -1; // root: it has no parent
    const q = [s];
    for (let head = 0; head < q.length; head++) {
      const u = q[head];
      for (const v of g[u]) {
        if (par[v] === -2) { par[v] = u; q.push(v); }
        else if (v !== par[u]) return true;
      }
    }
  }
  return false;
};
```

Time $O(n + m)$ **Space** $O(n)$

Verified outputs: tree plus isolated node → false. Add edge [2,3] to close a square → true. Triangle → true. Two parallel edges between 0 and 1 → true. A random cross-check over 500 graphs found both methods always agreeing.

par is seeded with -2 for “not seen” and -1 for “root”, so the test must be `par[v] === -2` with a strict `===`. -2 and “-2” are different values but `==` says they are equal, and graph inputs parsed from text arrive as strings surprisingly often. Use `===` everywhere in this book.

TRAP

$v \neq \text{par}[u]$ compares against the parent of u , not of v . Writing $v \neq \text{par}[v]$ is a very common bug and always returns false.

TRAP

This “skip the parent” trick only works for **undirected** graphs. For directed graphs it is wrong — use Example 10.

Answer: true when any piece has edges $\geq$ nodes

Example 10 [Capgemini · pattern]

Cycle in a directed graph. Edges are one-way. Return true if some node can reach itself by following arrows.

Constraints: $n \leq 10^5$. Target $O(n + m)$. Edge cases: a self loop $0 \rightarrow 0$ (that is a cycle); a “diamond” $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 2$ (that is NOT a cycle even though node 2 is reached twice).

SOLUTION

The parent trick fails here. In the diamond, node 2 is reached from both 0 and 1, but nothing loops back. What matters is not “seen before” but “**seen on the path I am standing on right now**”.

Give every node one of three colours.

THE 3-COLOUR RULE

- **0 = white** — never touched.
- **1 = grey** — the DFS entered it and has not left yet. It is on the current path.
- **2 = black** — the DFS entered it and already left. Fully finished.

Meeting a **grey** node means you walked back onto your own path. That is a cycle. Meeting a **black** node means nothing at all.

3-colour DFS

JS

```
const dfsDir = (u, g, col) => {
  col[u] = 1; // grey: on the current path
  for (const v of g[u]) {
    if (col[v] === 1) return true; // back edge -> cycle
    if (col[v] === 0 && dfsDir(v, g, col)) return true;
  }
  col[u] = 2; // black: done
  return false;
};

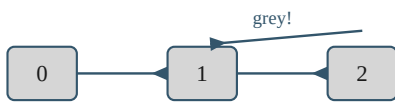
const hasCycleDirected = (n, g) => {
  const col = new Array(n).fill(0);
  for (let i = 0; i < n; i++)
    if (col[i] === 0 && dfsDir(i, g, col)) return true;
  return false;
};
```

Time $O(n + m)$ **Space** $O(n)$

Verified: chain $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ gives false. Loop $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ gives true. Diamond $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 2$ gives false. Self loop $0 \rightarrow 0$ gives true.

TRAP

Recursive again, so $n = 10^5$ in a chain blows the stack. The iterative rewrite is a stack of [node, childIndex] pairs: colour grey on the way in, advance the index for each child, and colour black when the index runs out. That is exactly the structure Kahn's topological sort automates — and “no valid topological order” is “the graph has a cycle”, which is the answer worth knowing for Chapter 16.



2 points back to 1,
which is still grey
so the cycle is 1, 2, 1

Grey means danger. Black is harmless.

Answer: true

Example 11 [Cognizant · pattern]

Bipartite check. Can you paint every node red or blue so that no edge joins two nodes of the same colour? Constraints: $n \leq 10^5$. Target $O(n + m)$. Edge cases: no edges (always yes); a triangle (no); a square (yes).

SOLUTION

Approach 1 — Try every colouring ($O(2^n m)$) — only for $n \leq 20$

Each node is red or blue, so a bitmask of n bits is one attempt. Check every edge.

Brute force over all 2^n paintings

JS

```
const bipartiteBrute = (n, E) => {
  for (let mask = 0; mask < (1 << n); mask++) {
    let ok = true;
    for (const [u, v] of E) {
      if (((mask >> u) & 1) === ((mask >> v) & 1)) { ok = false; break; }
    }
    if (ok) return true;
  }
  return false;
};
```

TRAP

$1 \ll n$ is a **32-bit** operation in JavaScript, however big numbers are elsewhere. At $n = 31$ it goes negative and the loop never runs; at $n = 32$ it wraps to 1. This brute force is only ever used as a checker for $n \leq 20$, so it is safe here — but the moment you write $1 \ll k$ for a bitmask with more than 30 bits, switch to $2 ** k$ (a double, exact to 2^{53}) or to `BigInt`.

Time $O(2^n m)$ **Space** $O(1)$

At $n = 30$ this is already hopeless.

Approach 2 — BFS and force the colours ($O(n+m)$) — optimal

You never have a choice. Paint the start red. Then every neighbour **must** be blue, every neighbour of those **must** be red, and so on. If a forced colour clashes with a colour already written, the answer is no.

```
const isBipartite = (n, g) => {
  const c = new Array(n).fill(-1); // -1 unpainted, 0 red, 1 blue
  for (let s = 0; s < n; s++) {
    if (c[s] !== -1) continue; // handles disconnected graphs
    c[s] = 0;
    const q = [s];
    for (let head = 0; head < q.length; head++) {
      const u = q[head];
      for (const v of g[u]) {
        if (c[v] === -1) { c[v] = 1 - c[u]; q.push(v); }
        else if (c[v] === c[u]) return false;
      }
    }
  }
  return true;
};
```

Time $O(n + m)$ **Space** $O(n)$

Verified against the brute force on 500 random graphs: zero mismatches. Triangle $\rightarrow$ false. Square 0-1-2-3-0 $\rightarrow$ true.

SHORTCUT

A graph is bipartite exactly when it has **no odd-length cycle**. A triangle (length 3) kills it. A square (length 4) does not.

TRAP

The outer for (let s = 0; ...) loop is not decoration. Without it you only test the component containing node 0 and silently pass graphs that fail elsewhere.

Answer: yes for the square, no for the triangle

Example 12 [Accenture · pattern]

Shortest number of moves in a grid with walls. 0 is open, 1 is a wall. You may step up, down, left, right. Return the fewest steps from (sr, sc) to (tr, tc), or -1.

Constraints: grid up to 1000×1000 . Edge cases: start equals target (answer 0); target is a wall (answer -1).

SOLUTION

Approach 1 — Try every path with DFS (exponential — dies at 5 x 5)

Brute force: every simple path

JS

```
const DR = [1, -1, 0, 0], DC = [0, 0, 1, -1];

const gridBrute = (g, r, c, tr, tc, used) => {
  const R = g.length, C = g[0].length;
  if (r < 0 || r >= R || c < 0 || c >= C || g[r][c] === 1 || used[r][c])
    return Infinity;
  if (r === tr && c === tc) return 0;
  used[r][c] = true;
```

```

let best = Infinity;
for (let k = 0; k < 4; k++) {
    const sub = gridBrute(g, r + DR[k], c + DC[k], tr, tc, used);
    if (sub !== Infinity) best = Math.min(best, sub + 1);
}
used[r][c] = false;
return best;
};

```

Infinity replaces `INT_MAX` and is strictly better for this job: `Infinity + 1` is still `Infinity`, so an accidental `sub + 1` on an unreachable branch cannot wrap around to a tiny number the way `INT_MAX + 1` does in C++.

Time exponential **Space** $O(RC)$
 Useful only as a checker on tiny grids.

Approach 2 — BFS ($O(RC)$ — optimal)

Grid BFS JS

```

const shortestGrid = (g, sr, sc, tr, tc) => {
    const R = g.length, C = g[0].length;
    if (g[sr][sc] === 1 || g[tr][tc] === 1) return -1;
    const d = Array.from({ length: R }, () => new Array(C).fill(-1));
    const q = [[sr, sc]];
    d[sr][sc] = 0;
    for (let head = 0; head < q.length; head++) {
        const [r, c] = q[head];
        if (r === tr && c === tc) return d[r][c];
        for (let k = 0; k < 4; k++) {
            const nr = r + DR[k], nc = c + DC[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C ||
                g[nr][nc] === 1 || d[nr][nc] !== -1) continue;
            d[nr][nc] = d[r][c] + 1;
            q.push([nr, nc]);
        }
    }
    return -1;
};

```

Time $O(RC)$ **Space** $O(RC)$

Test grid (0 = open):

Real runs TEXT

```

0 0 0 1
1 1 0 1      (0,0) -> (3,3)  answer 6
0 0 0 0      (0,0) -> (0,0)  answer 0
0 1 1 0      [[0,1],[1,0]] (0,0)->(1,1)  answer -1

```

The two were cross-checked on 200 random grids: zero mismatches.

The idea that unlocked it: every step costs exactly 1, so “fewest steps” is exactly what BFS computes for free.

Answer: 6

Example 13 [TCS Digital · pattern]

Flood fill. Given a grid of colour numbers, a start cell, and a new colour, repaint the whole connected block of the start cell's colour.

Edge cases: the new colour equals the old colour (do nothing, or you loop forever).

SOLUTION

Flood fill by BFS

JS

```
const floodFill = (a, sr, sc, newColor) => {
  const R = a.length, C = a[0].length, old = a[sr][sc];
  if (old === newColor) return a; // THE edge case
  const q = [[sr, sc]];
  a[sr][sc] = newColor;
  for (let head = 0; head < q.length; head++) {
    const [r, c] = q[head];
    for (let k = 0; k < 4; k++) {
      const nr = r + DR[k], nc = c + DC[k];
      if (nr < 0 || nr >= R || nc < 0 || nc >= C || a[nr][nc] !== old) continue;
      a[nr][nc] = newColor;
      q.push([nr, nc]);
    }
  }
  return a;
};
```

This one deliberately paints the caller's grid — that is what “flood fill” means — and returns it so the call can be chained. JavaScript objects and arrays are passed by reference, so there is no & to write and no copy made: every function in this chapter that takes a grid either mutates it on purpose or copies it with `map(r => [...r])` on purpose. Decide which, and say so in the function's first line.

Time $O(RC)$ **Space** $O(RC)$

Real run on `[[1,1,2],[1,0,1],[2,1,1]]`, start `(0,0)`, new colour 9: result `[[9,9,2],[9,0,1],[2,1,1]]`. The 0 at `(1,1)` blocks the path, so the two 1s on the right are a different block and stay.

TRAP

Skip the `old === newColor` guard and the cell is repainted to a colour that still matches `old`, so it is pushed again, forever.

Answer: `[[9,9,2],[9,0,1],[2,1,1]]`

Example 14 [Infosys · pattern]

Largest component. Return the number of nodes in the biggest connected piece. Edge cases: $n = 0$ (answer 0); no edges (answer 1).

SOLUTION

Same skeleton as counting components, but count the nodes you flood.

Biggest piece

JS

```
const largestComponent = (n, g) => {
  const vis = new Array(n).fill(0);
  let best = 0;
  for (let i = 0; i < n; i++) {
```



```

    if (vis[i]) continue;
    let cnt = 0;
    const q = [i]; vis[i] = 1;
    for (let head = 0; head < q.length; head++) {
        const u = q[head]; cnt++;
        for (const v of g[u]) if (!vis[v]) { vis[v] = 1; q.push(v); }
    }
    best = Math.max(best, cnt);
}
return best;
};

```

Time $O(n + m)$ **Space** $O(n)$

Verified: on $n = 7$ with edges $[0, 1], [0, 2], [1, 3], [4, 5]$ the answer is 4.

Answer: 4

Example 15 [TCS NQT · pattern]

Provinces from a matrix. You are given an $n \times n$ table where $m[i][j] = 1$ means city i and city j are directly linked. Count the groups of linked cities.

Constraints: $n \leq 500$ (so n^2 is fine). Edge cases: $n = 1$ (answer 1); the identity matrix (answer n).

SOLUTION

The input is a matrix, so reading it already costs $O(n^2)$. Only the upper triangle matters, because the table is symmetric.

Provinces with DSU

JS

```

const provinces = (m) => {
    const n = m.length;
    const d = new DSU(n);
    let c = n;
    for (let i = 0; i < n; i++)
        for (let j = i + 1; j < n; j++)
            if (m[i][j] === 1 && d.union(i, j)) c--;
    return c;
};

```

Time $O(n^2 \alpha(n))$ **Space** $O(n)$

Verified: $[[1, 1, 0], [1, 1, 0], [0, 0, 1]] \rightarrow 2$. $[[1]] \rightarrow 1$. $[[1, 0], [0, 1]] \rightarrow 2$.

Answer: 2

Example 16 [Capgemini · pattern]

Is there a path? Undirected graph, a source and a destination. Return true if a route exists. Edge cases: source equals destination (true, even with no edges).

SOLUTION

Approach 1 — Repeated relaxation over the edge list ($O(nm)$)

Keep a reachable flag per node. Sweep the whole edge list again and again until nothing changes.

Brute force

JS

```
const pathExistsBrute = (n, E, s, t) => {
  const r = new Array(n).fill(false);
  r[s] = true;
  for (let pass = 0; pass < n; pass++) {
    let changed = false;
    for (const [u, v] of E) {
      if (r[u] && !r[v]) { r[v] = true; changed = true; }
      if (r[v] && !r[u]) { r[u] = true; changed = true; }
    }
    if (!changed) break;
  }
  return r[t];
};
```

Time $O(nm)$ **Space** $O(n)$

Approach 2 — BFS ($O(n+m)$ — optimal)

The obvious answer

JS

```
const pathExistsBFS = (n, g, s, t) => {
  if (s === t) return true;
  const vis = new Array(n).fill(false);
  const q = [s]; vis[s] = true;
  for (let head = 0; head < q.length; head++) {
    const u = q[head];
    for (const v of g[u]) {
      if (v === t) return true;
      if (!vis[v]) { vis[v] = true; q.push(v); }
    }
  }
  return false;
};
```

Time $O(n + m)$ **Space** $O(n)$

Cross-checked on 300 random graphs: zero mismatches.

Answer: true / false

Example 17 [Wipro · pattern]

Which nodes are exactly k edges away from the source? Constraints: $n \leq 10^5$. Edge cases: $k = 0$ (only the source); k larger than the graph's reach (empty list).

SOLUTION

BFS gives you distance for free. The only trick is to **stop expanding** once a node sits at distance k — going deeper cannot help.

Nodes at distance exactly k

JS

```
const atDistanceK = (n, g, src, k) => {
  const dist = new Array(n).fill(-1), res = [];
  const q = [src]; dist[src] = 0;
  for (let head = 0; head < q.length; head++) {
```

```

const u = q[head];
if (dist[u] === k) { res.push(u); continue; } // do not go deeper
for (const v of g[u])
  if (dist[v] === -1) { dist[v] = dist[u] + 1; q.push(v); }
}
return res;
};

```

Time $O(n + m)$ **Space** $O(n)$

Real run on the tree 0-1, 0-2, 1-3, 2-4, 4-5, 5-6: $k = 2 \rightarrow 3$ 4; $k = 0 \rightarrow 0$; $k = 9 \rightarrow$ empty.

Answer: 3 4

Tier 2 — traversal with a business story

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 18 [Grab · pattern]

Spoilage in a warehouse. A warehouse shelf is a grid. 0 is an empty slot, 1 is a fresh box, 2 is a spoiled box. Every minute, a spoiled box spoils every fresh box directly beside it (4 directions). Return the minute at which nothing fresh is left, or -1 if some box can never spoil.

Constraints: grid up to 500×500 . Edge cases: no fresh boxes at all (answer 0); a fresh box walled off by empty slots (answer -1).

SOLUTION

Approach 1 — Simulate minute by minute ($O(R \cdot C \times \text{minutes})$)

Each minute, scan the whole grid, list the fresh boxes touching a spoiled one, then spoil them all together.

Brute force simulation

JS

```

const spreadBrute = (grid) => {
  const g = grid.map(r => [...r]);
  const R = g.length, C = g[0].length;
  let t = 0;
  for (;;) {
    const todo = [];
    for (let i = 0; i < R; i++)
      for (let j = 0; j < C; j++) {
        if (g[i][j] !== 1) continue;
        for (let k = 0; k < 4; k++) {
          const nr = i + DR[k], nc = j + DC[k];
          if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
          if (g[nr][nc] === 2) { todo.push([i, j]); break; }
        }
      }
    if (!todo.length) break;
    for (const [r, c] of todo) g[r][c] = 2;
    t++;
  }
  for (let i = 0; i < R; i++)

```

```

    for (let j = 0; j < C; j++) if (g[i][j] === 1) return -1;
    return t;
};

```

Time $O(RC \times T)$ **Space** $O(RC)$

T can be $R \times C$, so the worst case is 6×10^{10} cell visits at 500×500 .

Approach 2 — Multi-source BFS ($O(RC)$ — optimal)

Put **every** spoiled box in the queue before the loop starts. Then one BFS ring equals one minute. No re-scanning.

Multi-source BFS, level by level

JS

```

const spreadTime = (grid) => {
    const g = grid.map(r => [...r]);           // we mutate, so copy every ROW
    const R = g.length, C = g[0].length;
    let fresh = 0, t = 0;
    const q = [];
    for (let i = 0; i < R; i++)
        for (let j = 0; j < C; j++) {
            if (g[i][j] === 2) q.push([i, j]); // ALL sources go in first
            else if (g[i][j] === 1) fresh++;
        }
    if (fresh === 0) return 0;
    let head = 0;
    while (head < q.length) {
        const sz = q.length - head;           // one whole ring = one minute
        let changed = false;
        for (let s = 0; s < sz; s++) {
            const [r, c] = q[head++];
            for (let k = 0; k < 4; k++) {
                const nr = r + DR[k], nc = c + DC[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C || g[nr][nc] !== 1) continue;
                g[nr][nc] = 2; fresh--; changed = true;
                q.push([nr, nc]);
            }
        }
        if (changed) t++;
    }
    return fresh === 0 ? t : -1;
};

```

Time $O(RC)$ **Space** $O(RC)$

TRAP

`const sz = q.length - head` must be read **before** the inner loop starts, because the loop pushes onto `q` as it goes. Capture the ring size first, then consume exactly that many. This is the head-index version of the classic `int sz = q.size()` line, and getting it wrong merges two minutes into one.

Real runs:

Verified output

TEXT

```

2 1 1
1 1 0  -> 4
0 1 1

2 1 1
0 1 1  -> -1  (the 1 at the bottom-left is cut off)
1 0 1

0 2      -> 0    (nothing fresh)

```

Both versions were cross-checked on 200 random grids: zero mismatches.

The idea that unlocked it: seeding the queue with all sources at once makes BFS measure distance to the **nearest** source instead of to one chosen source. That single change removes the outer time loop.

TRAP

if (changed) t++ matters. Without the guard, the last ring (which spoils nothing) still bumps the counter and you answer one minute too many.

Verified against the brute-force simulation on 200 random grids: zero mismatches. Writing the slow version as a **checker** costs three minutes and is the single most effective way to catch off-by-one bugs like the one above.

Answer: 4

Example 19 [Sea / Shopee · pattern]

Fewest knight hops. A knight sits on an $N \times N$ board. It moves in the usual L shape. Return the fewest moves to reach a target square, or -1 .

Constraints: $N \leq 300$. Edge cases: start equals target (0); a 3×3 board where the centre is unreachable.

SOLUTION

There is no adjacency list to build — the neighbours of a square are computed on the fly from 8 offsets. This is called an **implicit graph**.

BFS on the knight's implicit graph

JS

```

const knightMoves = (N, sr, sc, tr, tc) => {
  const d = Array.from({ length: N }, () => new Array(N).fill(-1));
  const dr = [1, 1, -1, -1, 2, 2, -2, -2];
  const dc = [2, -2, 2, -2, 1, -1, 1, -1];
  const q = [[sr, sc]]; d[sr][sc] = 0;
  for (let head = 0; head < q.length; head++) {
    const [r, c] = q[head];
    if (r === tr && c === tc) return d[r][c];
    for (let k = 0; k < 8; k++) {
      const nr = r + dr[k], nc = c + dc[k];
      if (nr < 0 || nr >= N || nc < 0 || nc >= N || d[nr][nc] !== -1) continue;
      d[nr][nc] = d[r][c] + 1;
      q.push([nr, nc]);
    }
  }
  return -1;
};

```

Time $O(N^2)$ **Space** $O(N^2)$

Verified: 8×8 , $(0, 0) \rightarrow (7, 7) \rightarrow 6$. $(0, 0) \rightarrow (1, 2) \rightarrow 1$. 3×3 , $(0, 0) \rightarrow (1, 1) \rightarrow -1$ (a knight can never reach the centre of a 3×3 board).

Answer: 6

Example 20 [Agoda · pattern]

Fewest button presses. A counter shows the number *start*. Three buttons: add 1, subtract 1, double. The counter can never go below 0. Reach *target* in the fewest presses.

Constraints: *start*, *target* up to 10^5 . Edge cases: *start* == *target* (0); *target* smaller than *start* (only subtract helps).

SOLUTION

Nodes are **numbers**, edges are **button presses**. Every press costs 1, so BFS again.

The only real design decision is the upper bound. Doubling past $2 \times \text{target}$ is never useful: from there you can only come down one step at a time, and walking down from *target* + *k* costs *k* presses, which you could have saved. A safe bound is $\max(\text{start}, 2 \times \text{target}) + 2$.

Implicit BFS over numbers

JS

```
const fewestPresses = (start, target) => {
  if (start === target) return 0;
  const LIM = Math.max(start, 2 * target) + 2; // must cover the START too
  const dist = new Array(LIM + 1).fill(-1);
  const q = [start]; dist[start] = 0;
  for (let head = 0; head < q.length; head++) {
    const u = q[head];
    for (const v of [u + 1, u - 1, u * 2]) {
      if (v < 0 || v > LIM || dist[v] !== -1) continue;
      dist[v] = dist[u] + 1;
      if (v === target) return dist[v];
      q.push(v);
    }
  }
  return -1;
};
```

Time $O(\text{LIM})$ **Space** $O(\text{LIM})$

Verified: 3 -> 10 gives 3 (3, 4, 5, 10). 2 -> 9 gives 3 (2, 4, 8, 9). 10 -> 1 gives 9. 0 -> 7 gives 5. 1 -> 1024 gives 10. 5 -> 5 gives 0.

TRAP

Writing `const LIM = 2 * target + 2;` looks right and breaks when *start* > LIM, and in JavaScript it breaks **silently**. C++ reads past the end of the vector and usually crashes; JS returns undefined for `dist[start]`, the test `dist[v] !== -1` is then true for every out-of-range index, and the BFS simply reports -1 with no error at all. The test 10 -> 1 catches it. Always ask: is the start itself inside my array?

Answer: 3

Example 21 [DBS · pattern]

Enclosed desks. An office floor is a grid of 'X' (wall) and '0' (desk). Any group of desks that can reach the outer border stays. Every other group is enclosed and must be turned into 'X'.

Edge cases: a desk on the border (its whole group survives); no desks at all.

SOLUTION

Do not hunt for enclosed groups. Hunt for the **safe** ones — they are exactly the groups that touch the border.

Three steps:

1. Start a BFS from every '0' on the four borders. Mark everything it reaches 'S'.
2. Every 'S' becomes '0' again.
3. Everything else becomes 'X'.

Mark the survivors, flip the rest

JS

```
const captureRegions = (b) => {
  if (!b.length) return b;
  const R = b.length, C = b[0].length;
  const q = [];
  const seed = (r, c) => { // a closure over q and b
    if (b[r][c] === '0') { b[r][c] = 'S'; q.push([r, c]); }
  };
  for (let i = 0; i < R; i++) { seed(i, 0); seed(i, C - 1); }
  for (let j = 0; j < C; j++) { seed(0, j); seed(R - 1, j); }
  for (let head = 0; head < q.length; head++) {
    const [r, c] = q[head];
    for (let k = 0; k < 4; k++) {
      const nr = r + DR[k], nc = c + DC[k];
      if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
      seed(nr, nc);
    }
  }
  for (let i = 0; i < R; i++)
    for (let j = 0; j < C; j++)
      b[i][j] = b[i][j] === 'S' ? '0' : 'X';
  return b;
};
```

The helper is named `seed`, not `push`, on purpose — `push` is already the array method and shadowing the word in your own head causes real confusion at the whiteboard. An arrow function declared inside the outer function closes over `q` and `b` automatically; this is JavaScript's version of C++'s `[&]` capture, and it needs no capture list.

Time $O(RC)$ **Space** $O(RC)$

Real run:

Before and after

TEXT

```
X X X X      X X X X
X 0 0 X  ->  X X X X
X X 0 X      X X X X
X 0 X X      X 0 X X
```

The '0' at the bottom-left is on the border, so it survives. The three in the middle are enclosed and become 'X'.

The idea that unlocked it: flip the question. “Find enclosed” is hard; “find what touches the border” is one multi-source BFS.

Answer: as shown

Example 22 [LINE MAN · pattern]

Delivery zones with diagonals. Same island counting, but two land cells are in the same zone if they touch on a side **or a corner** (8 directions). Count the zones. Edge cases: a checkerboard (all one zone with 8 directions, many zones with 4).

SOLUTION

Change 4 offsets to 8. Everything else is identical. Writing the two loops for `dr` in `-1..1`, for `dc` in `-1..1` with a skip on `(0,0)` is shorter than listing 8 pairs.

8-direction flood

JS

```
const numIslands8 = (grid) => {
  if (!grid.length) return 0;
  const g = grid.map(r => [...r]);
  const R = g.length, C = g[0].length;
  let cnt = 0;
  for (let i = 0; i < R; i++)
    for (let j = 0; j < C; j++) {
      if (g[i][j] !== 1) continue;
      cnt++;
      const q = [[i, j]]; g[i][j] = 0;
      for (let head = 0; head < q.length; head++) {
        const [r, c] = q[head];
        for (let dr = -1; dr <= 1; dr++)
          for (let dc = -1; dc <= 1; dc++) {
            if (dr === 0 && dc === 0) continue;
            const nr = r + dr, nc = c + dc;
            if (nr < 0 || nr >= R || nc < 0 || nc >= C || g[nr][nc] !== 1)
              continue;
            g[nr][nc] = 0; q.push([nr, nc]);
          }
        }
      }
    }
  return cnt;
};
```

Time $O(RC)$ **Space** $O(RC)$

Verified: `[[1,0,1],[0,1,0],[1,0,1]]` → 1 with 8 directions (it would be 5 with 4 directions). `[[1,0],[0,1]]` → 1.

Answer: 1

Example 23 [GIC · pattern]

How many shortest routes? Unweighted graph. For every node, report both the shortest distance from the source and **how many** different shortest routes reach it.

Constraints: $n \leq 10^5$; counts grow like a product and can pass $2^{53} - 1$. Edge cases: unreachable node (distance -1 , count 0); the source itself (distance 0, count 1).

SOLUTION

Extend BFS with a ways array. When you **first** reach `v` from `u`, it inherits `u`'s count. When you reach it again at the **same** distance, add.

BFS that also counts routes

JS

```
const countShortestPaths = (n, g, src) => {
  const dist = new Array(n).fill(-1);
```



```

const ways = new Array(n).fill(0);
dist[src] = 0; ways[src] = 1;
const q = [src];
for (let head = 0; head < q.length; head++) {
  const u = q[head];
  for (const v of g[u]) {
    if (dist[v] === -1) { // first time: new distance
      dist[v] = dist[u] + 1;
      ways[v] = ways[u];
      q.push(v);
    } else if (dist[v] === dist[u] + 1) { // another equally short route
      ways[v] += ways[u];
    }
  }
}
return { dist, ways }; // one object, destructured by the caller
};

```

Time $O(n + m)$ **Space** $O(n)$

Returning `{ dist, ways }` and calling it as `const { dist, ways } = countShortestPaths(n, g, 0);` is the JS answer to C++'s `pair<...>`: the fields have **names**, so the caller cannot silently swap them the way `.first` and `.second` invite. Use an object for every multi-value return in this book.

Real run on 0-1, 0-2, 1-3, 2-3, 3-4, 4-5:

Verified output

TEXT

```

dist: 0 1 1 2 3 4
ways: 1 1 1 2 2 2

```

Node 3 has two shortest routes (0-1-3 and 0-2-3), and nodes 4 and 5 inherit that 2.

TRAP

Counts grow like a product and explode fast. Plain JS numbers stay exact only to $2^{53} - 1$, and past that they **round** — the count keeps looking like a plausible integer while being wrong. Two correct answers, and you should name the one the statement asks for:

- If the statement says “modulo $10^9 + 7$ ”, take the modulus at **every** `+=`: `ways[v] = (ways[v] + ways[u]) % MOD`. Each intermediate stays under 2×10^9 , far inside the exact range, so plain numbers are fine.
- If the statement wants the true count, use `BigInt`: `ways` is filled with `0n`, `ways[src] = 1n`, and `+=` needs no other change. Just never mix — `1n + 1` throws a `TypeError`.

Beware the modular **multiply**, though: `(a * b) % MOD` with `a, b` near 10^9 needs 10^{18} of headroom and silently rounds. Measured, with `MOD = 1000000007`, `a = 999999937` and `b = 999999893`:

```

(a * b) % MOD                -> 8023      WRONG
Number((BigInt(a) * BigInt(b)) % BigInt(MOD)) -> 7980      correct

```

Plain addition, as used here, is safe; a modular multiply is not.

TRAP

The `else if` must test `dist[v] == dist[u] + 1`. If you test only `dist[v] != -1` you will add routes that are longer, and the count becomes garbage.

Answer: `ways = 1 1 1 2 2 2`

Example 24 [SCB · pattern]

Name one broken dependency loop. A build system has services and one-way “needs” arrows. If the arrows contain a loop, the build never finishes. Print one loop, in order. Print nothing if there is no loop.

Edge cases: no loop at all; a self loop.

SOLUTION

Reuse the 3-colour DFS, but remember the parent of each node. The moment you step onto a **grey** node v from node u , the loop is: v , then the chain of parents from u back up to v .

Find and print one directed cycle

JS

```
class CycleFinder {
  constructor(n) {
    this.n = n;
    this.g = Array.from({ length: n }, () => []);
    this.col = new Array(n).fill(0);
    this.par = new Array(n).fill(-1);
    this.st = -1; this.en = -1;
  }
  addEdge(u, v) { this.g[u].push(v); }
  dfs(u) {
    this.col[u] = 1;
    for (const v of this.g[u]) {
      if (this.col[v] === 0) { this.par[v] = u; if (this.dfs(v)) return true; }
      else if (this.col[v] === 1) { this.st = v; this.en = u; return true; }
    }
    this.col[u] = 2;
    return false;
  }
  run() {
    for (let i = 0; i < this.n; i++) if (this.col[i] === 0 && this.dfs(i)) break;
    if (this.st === -1) return [];
    const c = [];
    for (let v = this.en; v !== this.st; v = this.par[v]) c.push(v);
    c.push(this.st);
    return c.reverse();
  }
}
```

Time $O(n + m)$ **Space** $O(n)$

Verified on 0->1, 1->2, 2->3, 3->1, 0->4: prints 1 2 3, meaning $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. On 0->1, 1->2 it prints nothing.

Answer: 1 2 3

Tier 3 — the hard ones

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 25 [Amazon · pattern]

Critical links. A network of servers is joined by two-way cables. A cable is **critical** if removing it splits some group of servers apart. List every critical cable.

Constraints: $n \leq 10^5$, $m \leq 2 \times 10^5$. Target $O(n + m)$. Edge cases: two servers joined by **two** parallel cables (neither is critical); a chain (every cable is critical).

SOLUTION

Approach 1 — Remove one cable, test connectivity $(O(m(n+m)))$ — too slow

For each of the m cables, delete it and count components. If the count went up, it was critical. At $m = 2 \times 10^5$ this is about 10^{10} operations.

Approach 2 — One DFS with entry times and low-link $(O(n+m))$ — optimal

THE LOW-LINK IDEA

Run one DFS. Give each node the time it was entered, $\text{tin}[u]$ (0, 1, 2, ...).

Define $\text{low}[u]$ = the **smallest** tin value reachable from the subtree of u , using tree edges downwards plus **at most one** back edge upwards.

Then the edge $u \rightarrow v$ (a tree edge, v a child) is a bridge exactly when

$$\text{low}[v] > \text{tin}[u]$$

In words: nothing in v 's subtree can climb back to u or above it without using that very edge. So the edge is the only link.

Bridges by Tarjan low-link

JS

```
class BridgeFinder {
  constructor(n) {
    this.n = n; this.timer = 0;
    this.g = Array.from({ length: n }, () => []); // entries are [neighbour, edge id]
    this.tin = new Array(n).fill(-1);
    this.low = new Array(n).fill(-1);
    this.bridges = [];
  }
  addEdge(id, u, v) { this.g[u].push([v, id]); this.g[v].push([u, id]); }
  dfs(u, pid) {
    this.tin[u] = this.low[u] = this.timer++;
    for (const [v, id] of this.g[u]) {
      if (id === pid) continue; // do not walk back on the SAME edge
      if (this.tin[v] !== -1) this.low[u] = Math.min(this.low[u], this.tin[v]);
      else {
        this.dfs(v, id);
        this.low[u] = Math.min(this.low[u], this.low[v]);
        if (this.low[v] > this.tin[u])
          this.bridges.push([Math.min(u, v), Math.max(u, v)]);
      }
    }
  }
  run() {
    for (let i = 0; i < this.n; i++) if (this.tin[i] === -1) this.dfs(i, -1);
    return this.bridges;
  }
}
```

Time $O(n + m)$ **Space** $O(n + m)$

Verified on the triangle 0-1-2-0 plus the tail 2-3, 3-4: the bridges are (3,4) and (2,3), reported in that order — the deepest one finishes first. The triangle's own edges are not bridges. On two parallel edges 0-1 and 0-1: **no** bridges. On a chain 0-1-2: both edges, (1,2) then (0,1).

TRAP

This is recursive, so it inherits the 10^4 -frame ceiling. At $n = 10^5$ — exactly this problem's stated limit — a long chain blows the stack. The rewrite is the [node, childIndex] stack from the DFS section: the “after child” work ($\text{low}[u] = \min(\text{low}[u], \text{low}[v])$) and the bridge test) happens when you pop a child's frame. Know that this is what you would do, and say so before the interviewer asks.

TRAP

Skip by **edge id**, not by parent node. If you write `if (v === p) continue`; then two parallel edges between u and v are both skipped, and the code wrongly reports a bridge. Using the id skips exactly one of the two.

Follow-up the interviewer asks next: “now give me the articulation points.” An articulation **point** is a node whose removal splits the graph. Same DFS, one changed test, plus a special rule for the DFS root.

Articulation points

JS

```
class APFinder {
  constructor(n) {
    this.n = n; this.timer = 0;
    this.g = Array.from({ length: n }, () => []);
    this.tin = new Array(n).fill(-1);
    this.low = new Array(n).fill(-1);
    this.isAP = new Array(n).fill(false);
  }
  addEdge(u, v) { this.g[u].push(v); this.g[v].push(u); }
  dfs(u, p) {
    this.tin[u] = this.low[u] = this.timer++;
    let children = 0;
    for (const v of this.g[u]) {
      if (v === p) continue;
      if (this.tin[v] !== -1) this.low[u] = Math.min(this.low[u], this.tin[v]);
      else {
        this.dfs(v, u);
        this.low[u] = Math.min(this.low[u], this.low[v]);
        if (this.low[v] >= this.tin[u] && p !== -1) this.isAP[u] = true; // note
      }
      children++;
    }
    if (p === -1 && children > 1) this.isAP[u] = true; // root
  }
  run() {
    for (let i = 0; i < this.n; i++) if (this.tin[i] === -1) this.dfs(i, -1);
    return this.isAP.map((b, i) => b ? i : -1).filter(i => i !== -1);
  }
}
```

Verified on the same graph (triangle 0-1-2 plus tail 2-3-4): articulation points are 2 and 3. On a chain 0-1-2: only 1. On a triangle: none.

The last line turns a boolean array into a list of indices. `isAP.filter(Boolean)` would be wrong — it returns the **true values**, not where they were — so map to the index first, then drop the `-1`s.

BRIDGE VS ARTICULATION POINT — THE ONE-CHARACTER DIFFERENCE

- Bridge: $\text{low}[v] > \text{tin}[u]$ — strictly greater.
 - Articulation point: $\text{low}[v] \geq \text{tin}[u]$ — greater **or equal**, plus the root needs two or more DFS children.
- Why? A child that can climb back exactly to u (equality) still cannot get **past** u . Deleting the edge is fine; deleting the node u is not.

Answer: (2,3) and (3,4)

Example 26 [Google · pattern]

Groups that can all reach each other (SCC). A directed graph. Two nodes are in the same group if each can reach the other. Count the groups and label every node.

Constraints: $n \leq 10^5$. Target $O(n + m)$. Edge cases: a chain (every node is its own group); one node (1 group).

SOLUTION

Approach 1 — Reachability from everybody $O(n(n+m))$ — too slow

Run a DFS from each node in the graph, and a DFS from each node in the reversed graph. Group u and v together when each appears in the other's list.

Approach 2 — Kosaraju — two passes $O(n+m)$ — optimal

KOSARAJU IN THREE LINES

1. DFS the graph, and push each node onto a list **when it finishes**. The list ends with the node that finished last.
2. Reverse every edge.
3. Walk the list **backwards**. Each fresh DFS in the reversed graph collects exactly one group.

Why it works: reversing keeps the groups the same (if u reaches v and back, the reversed graph says the same). Taking the latest-finishing node first guarantees you start in a group that no **other** unvisited group points into, so the reversed DFS cannot leak out of it.

Kosaraju — both passes iterative, so 10^5 nodes are safe

JS

```
class Kosaraju {
  constructor(n) {
    this.n = n;
    this.g = Array.from({ length: n }, () => []);
    this.rg = Array.from({ length: n }, () => []);
    this.vis = new Array(n).fill(0);
    this.comp = new Array(n).fill(-1);
    this.order = [];
  }
  addEdge(u, v) { this.g[u].push(v); this.rg[v].push(u); }

  dfs1(u) {                                     // stack of [node, childIndex]
    const st = [[u, 0]];
    this.vis[u] = 1;
    while (st.length) {
```

```

    const top = st[st.length - 1];
    if (top[1] < this.g[top[0]].length) {
        const v = this.g[top[0]][top[1]++]; // take the next child
        if (!this.vis[v]) { this.vis[v] = 1; st.push([v, 0]); }
    } else {
        this.order.push(top[0]); // push on FINISH
        st.pop();
    }
}
}

dfs2(u, c) { // plain flood: no post-order work needed
    const st = [u];
    this.comp[u] = c;
    while (st.length) {
        const x = st.pop();
        for (const v of this.rg[x])
            if (this.comp[v] === -1) { this.comp[v] = c; st.push(v); }
    }
}

run() {
    for (let i = 0; i < this.n; i++) if (!this.vis[i]) this.dfs1(i);
    let c = 0;
    for (let i = this.n - 1; i >= 0; i--) { // latest finisher first
        const u = this.order[i];
        if (this.comp[u] === -1) this.dfs2(u, c++);
    }
    return c;
}
}

```

Time $O(n + m)$ **Space** $O(n + m)$

Verified on 0->1, 1->2, 2->0, 2->3, 3->4, 4->5, 5->3: count = 2, labels 0 0 0 1 1 1. Chain 0->1->2 gives 3 groups. One node gives 1. A chain of **200000** nodes gives 200000 groups and does not overflow the stack — the recursive version dies there, which is why dfs1 is written with an explicit stack.

Compare dfs1 and dfs2. The first pass must record each node **when it finishes**, so it needs the [node, childIndex] stack that remembers how far through the children it got. The second pass only paints labels, with nothing to do on the way back up, so a plain stack of nodes is enough. Recognising which of the two shapes a DFS needs is the whole skill of converting recursion to iteration.

Follow-up: “shrink each group to a single node — what do you get?” You get the **condensation**, and it is always a DAG (a directed graph with no cycles). If it had a cycle, the groups on that cycle could all reach each other, so they would have been one group. The condensation is the first step in almost every hard directed-graph problem.

Follow-up: “how many extra edges make the whole graph one group?” Build the condensation. Let a = number of groups with no incoming edge, b = number with no outgoing edge. If the condensation has only one node the answer is 0, otherwise it is $\max(a, b)$.

Answer: 2 groups; labels 0 0 0 1 1 1

Example 27 [Microsoft · pattern]

Best single reclamation. An $n \times n$ grid of 0 (water) and 1 (land). You may turn **at most one** 0 into a 1. Return the largest island you can end up with.

Constraints: $n \leq 500$. Target $O(n^2)$. Edge cases: the grid is already all land (answer n^2 , you flip nothing); all water (answer 1).

SOLUTION

Approach 1 — Flip each 0 and recount $O(n^4)$ — too slow

There are up to n^2 zeros and each recount costs $O(n^2)$. At $n = 500$ that is 6×10^{10} .

Approach 2 — Label the islands once, then look at each 0's four neighbours $O(n^2)$ — optimal

Two passes:

1. Flood every island once. Give each island a **label** (2, 3, 4, ...) and record its size. Labels start at 2 so they can never be confused with 0 or 1.
2. For each water cell, look at its (at most 4) neighbours, collect the **distinct** labels, and add up their sizes plus 1 for the flipped cell.

The “distinct” part is essential: two neighbours can belong to the same island, and you must not count it twice.

Label, then merge

JS

```
const largestIslandAfterFlip = (grid) => {
  const g = grid.map(r => [...r]);
  const n = g.length;
  const id = Array.from({ length: n }, () => new Array(n).fill(0));
  const area = new Map(); // label -> size
  let label = 2;
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++) {
      if (g[i][j] !== 1 || id[i][j]) continue;
      let cnt = 0;
      const q = [[i, j]]; id[i][j] = label;
      for (let head = 0; head < q.length; head++) {
        const [r, c] = q[head]; cnt++;
        for (let k = 0; k < 4; k++) {
          const nr = r + DR[k], nc = c + DC[k];
          if (nr < 0 || nr >= n || nc < 0 || nc >= n ||
              g[nr][nc] !== 1 || id[nr][nc]) continue;
          id[nr][nc] = label; q.push([nr, nc]);
        }
      }
      area.set(label++, cnt);
    }
  let best = 0;
  for (const size of area.values()) best = Math.max(best, size); // flip nothing
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++) {
      if (g[i][j] !== 0) continue;
      const seen = new Set();
      let tot = 1;
      for (let k = 0; k < 4; k++) {
        const nr = i + DR[k], nc = j + DC[k];
        if (nr < 0 || nr >= n || nc < 0 || nc >= n || id[nr][nc] === 0) continue;
        if (!seen.has(id[nr][nc])) {
          seen.add(id[nr][nc]);
          tot += area.get(id[nr][nc]);
        }
      }
      best = Math.max(best, tot);
    }
}
```

```
return best;
};
```

Time $O(n^2)$ **Space** $O(n^2)$
 The Set holds at most 4 labels.

Verified: $[[1,0],[0,1]] \rightarrow 3$. $[[1,1],[1,1]] \rightarrow 4$ (no flip needed, and no 0 exists). $[[0,0],[0,0]] \rightarrow 1$.
 $[[1,1,0],[1,0,0],[0,0,1]] \rightarrow 4$.

C++ writes `if (seen.insert(x).second)` because `insert` reports whether the value was new. JavaScript's `Set.add` returns the set itself, so the test has to be written out: `if (!seen.has(x)) { seen.add(x); ... }`.
 Two lines instead of one, and the same $O(1)$.

TRAP

`best` must be seeded with the biggest existing island. If the grid has no 0 at all, the second loop never runs and you would return 0.

Follow-up: “now you may flip up to K zeros, K small.” Labelling still helps, but the merge is no longer local. For $K = 1$ the four-neighbour trick is exact. For general K this becomes a search problem, and the honest interview answer is: state that it is much harder, and give a BFS over “water cells used so far” for small K .

Answer: 3

Example 28 [D. E. Shaw · pattern]

Shortest loop. Return the length of the shortest cycle in an undirected graph, or -1 if there is none.

Constraints: $n \leq 1000$, $m \leq 5000$. Target $O(n(n+m))$. Edge cases: a tree (answer -1); a triangle (answer 3).

SOLUTION

Approach 1 — Try every set of nodes (hopeless)

Approach 2 — BFS from every node ($O(n(n+m))$ — the standard answer)

Run BFS from each node s . During that BFS, if you meet an edge $u - v$ where v is **already** at a known distance and v is not the parent of u , then $\text{dist}[u] + \text{dist}[v] + 1$ is the length of a cycle through s . Take the minimum over every start and every such edge.

Shortest cycle (girth)

JS

```
const shortestCycle = (n, g) => {
  let best = Infinity;
  for (let s = 0; s < n; s++) {
    const dist = new Array(n).fill(-1), par = new Array(n).fill(-1);
    const q = [s]; dist[s] = 0;
    for (let head = 0; head < q.length; head++) {
      const u = q[head];
      for (const v of g[u]) {
        if (dist[v] === -1) { dist[v] = dist[u] + 1; par[v] = u; q.push(v); }
        else if (v !== par[u]) best = Math.min(best, dist[u] + dist[v] + 1);
      }
    }
  }
  return best === Infinity ? -1 : best;
};
```


Time $O(n(n + m))$ **Space** $O(n)$

At $n = 1000$ and $m = 5000$ this is about 6×10^6 steps. Fine.

Verified: triangle plus a tail $\rightarrow 3$. Square $0-1-2-3-0 \rightarrow 4$. Path $0-1-2 \rightarrow -1$.

This finds the true shortest cycle for **unweighted** graphs. The formula can overshoot the true answer for a single fixed start s , but taking the minimum over all starts fixes it, because the shortest cycle is measured correctly from at least one of its own nodes.

Follow-up: “now the graph is directed.” BFS with a parent no longer applies. The standard answer is: for each edge $u \rightarrow v$, delete it, run a BFS from v to u , and add 1. That is $O(m(n + m))$ — say this out loud and note it is much worse, which is why the directed version is rarely asked.

Answer: 3

Example 29 [Goldman Sachs · pattern]

Cables to lay. There are n machines and a list of existing cables. You may unplug any cable and plug it in somewhere else. What is the smallest number of moves that connects everything, or -1 if it is impossible?

Edge cases: fewer than $n - 1$ cables (impossible); $n = 1$ (0 moves).

SOLUTION

Two facts settle it.

1. Connecting n machines needs at least $n - 1$ cables. If you have fewer, answer -1 .
2. If you have enough cables, you have enough **spare** cables. Every cable that closes a loop is spare, and the number of spares is always at least (number of components $- 1$). So the answer is simply components $- 1$.

Count components, subtract one

JS

```
const minExtraCables = (n, cables) => {
  if (cables.length < n - 1) return -1;
  const d = new DSU(n);
  let comps = n;
  for (const [u, v] of cables) if (d.union(u, v)) comps--;
  return comps - 1;
};
```

Time $O(m\alpha(n))$ **Space** $O(n)$

Verified: $n = 5$ with cables $[0,1], [0,2], [1,2], [3,4] \rightarrow 1$ (components $\{0,1,2\}$ and $\{3,4\}$; the cable $\{1,2\}$ is the spare that moves). $n = 4$ with one cable $\rightarrow -1$. $n = 1$ with no cables $\rightarrow 0$.

Follow-up: “prove you always have enough spares.” A component with k nodes needs only $k - 1$ cables inside it. With c components and $m \geq n - 1$ cables, the cables used inside components total $n - c$, so spares $= m - (n - c) \geq (n - 1) - (n - c) = c - 1$. Exactly enough.

Answer: 1

Example 30 [Adobe · pattern]

Jobs that can never get stuck. A directed graph of tasks. A task is **safe** if every route that starts at it eventually reaches a dead end (a node with no outgoing edges). List the safe tasks in increasing order.

Edge cases: a node with no outgoing edges (safe); a node inside a cycle (not safe).

SOLUTION

“Unsafe” means: from this node you can reach a cycle. So reuse the 3-colour DFS, but **remember** the verdict instead of stopping at the first cycle.

Colours here: 0 new, 1 on the current path, 2 proved safe, 3 proved unsafe.

```
const safeNodes = (n, g) => {
  const col = new Array(n).fill(0); // 0 new, 1 on path, 2 safe, 3 unsafe
  const unsafe = (u) => {
    if (col[u] === 1 || col[u] === 3) return true;
    if (col[u] === 2) return false;
    col[u] = 1;
    for (const v of g[u])
      if (unsafe(v)) { col[u] = 3; return true; }
    col[u] = 2;
    return false;
  };
  const res = [];
  for (let i = 0; i < n; i++) if (!unsafe(i)) res.push(i);
  return res;
};
```

Time $O(n + m)$ **Space** $O(n)$

Verified on 0->1, 0->2, 1->2, 1->3, 2->5, 3->0, 4->5, 5 (dead end): safe nodes are 2 4 5. Nodes 0, 1 and 3 sit on the loop $0 \rightarrow 1 \rightarrow 3 \rightarrow 0$. A single node with no edges is safe.

C++ needs `function<bool(int)> unsafe = [&](int u) -> bool { ... }` so the lambda can call itself. A JavaScript arrow function assigned to a `const` can already refer to its own name — the binding exists by the time the function **runs** — so `const unsafe = (u) => { ... unsafe(v) ... }` just works. Watch the order, though: calling `unsafe` on the line **above** its declaration throws a `ReferenceError`, because `const` is not hoisted the way function declarations are.

TRAP

The memo is what makes this linear. Without storing colour 2 and colour 3, the same subtree is explored again for every caller and the running time becomes exponential.

Follow-up: “now do it without recursion.” Reverse every edge, start from all nodes whose **out-degree** is 0, and run a Kahn-style peel: a node becomes safe once all of its out-neighbours are safe. That is the same idea in iterative clothing, and it is $O(n + m)$ with no stack risk — which, at $n = 10^5$ in Node, is not an optimisation but a requirement.

```
const safeNodesKahn = (n, g) => {
  const rg = Array.from({ length: n }, () => []);
  const outdeg = new Array(n).fill(0);
  for (let u = 0; u < n; u++)
    for (const v of g[u]) { rg[v].push(u); outdeg[u]++; }
  const q = [];
  for (let i = 0; i < n; i++) if (outdeg[i] === 0) q.push(i); // dead ends
  const safe = new Array(n).fill(false);
  for (let head = 0; head < q.length; head++) {
    const u = q[head];
    safe[u] = true;
    for (const p of rg[u]) if (--outdeg[p] === 0) q.push(p);
  }
  return safe.map((b, i) => b ? i : -1).filter(i => i !== -1);
};
```

Both versions return 2 4 5 on the graph above.

Answer: 2 4 5

Dry run — bridges, line by line

Graph (undirected, 6 nodes), edges given with their ids:

id	0	1	2	3	4	5	6
edge	0-1	1-2	2-0	2-3	3-4	3-5	4-5

So there are two triangles, 0-1-2 and 3-4-5, joined by the single edge 2-3. By eye, the only bridge should be 2-3. Let us watch the algorithm find it.

We run `dfs(0, -1)`. Recall: `tin[u]` is the entry time, `low[u]` starts equal to `tin[u]` and only ever goes **down**.

Actual printed trace of the DFS

TEXT

```
enter 0  tin=0 low=0
  enter 1  tin=1 low=1
    enter 2  tin=2 low=2
      back edge 2->0  low[2]=0
      enter 3  tin=3 low=3
        enter 4  tin=4 low=4
          enter 5  tin=5 low=5
            back edge 5->3  low[5]=3
            after child 5: low[4]=3  low[5]=3 vs tin[4]=4  -> not a bridge
            after child 4: low[3]=3  low[4]=3 vs tin[3]=3  -> not a bridge
            back edge 3->5  low[3]=3
            after child 3: low[2]=0  low[3]=3 vs tin[2]=2  -> BRIDGE
            after child 2: low[1]=0  low[2]=0 vs tin[1]=1  -> not a bridge
            after child 1: low[0]=0  low[1]=0 vs tin[0]=0  -> not a bridge
            back edge 0->2  low[0]=0
tin: 0 1 2 3 4 5
low: 0 0 0 3 3 3
```

Now the same thing as a table, one row per event.

event	node	tin / low after	what it means
enter	0	0 / 0	root of the DFS
enter	1	1 / 1	
enter	2	2 / 2	
back edge 2 → 0	2	2 / 0	0 is already open, so 2 can climb to time 0
enter	3	3 / 3	we crossed the edge 2-3
enter	4	4 / 4	
enter	5	5 / 5	
back edge 5 → 3	5	5 / 3	5 can climb only as high as time 3
return 5 → 4	4	4 / 3	<code>low[5]=3</code> is not $>$ <code>tin[4]=4</code> → edge 4-5 is safe
return 4 → 3	3	3 / 3	<code>low[4]=3</code> is not $>$ <code>tin[3]=3</code> → edge 3-4 is safe
back edge 3 → 5	3	3 / 3	5 was entered at 5, no improvement
return 3 → 2	2	2 / 0	<code>low[3]=3</code> $>$ <code>tin[2]=2</code> → edge 2-3 IS a bridge

return 2 → 1	1	1 / 0	low[2]=0 is not > tin[1]=1 → safe
return 1 → 0	0	0 / 0	low[1]=0 is not > tin[0]=0 → safe
back edge 0 → 2	0	0 / 0	no change

READ THE FINAL TABLE

```
node:  0  1  2  3  4  5
tin :  0  1  2  3  4  5
low :  0  0  0  3  3  3
```

The whole first triangle collapses to $low = 0$. The whole second triangle collapses to $low = 3$. Those two blocks are exactly the two **2-edge-connected components**, and the single edge joining blocks with different low values is the bridge.

Read the trace twice. The first time, follow only the `enter` lines — that is plain DFS. The second time, follow only the `after child` lines — that is where every decision is made, on the way **back up**.

Python, for the two signature problems

JavaScript is the language of this book and a JS answer is always a complete answer. These two are worth seeing in Python for one reason: `collections.deque` is a real queue in the standard library, so the BFS loop reads exactly the way the algorithm is described.

BFS distances and island counting

PYTHON

```
from collections import deque

def bfs_dist(g, src):
    n = len(g)
    dist = [-1] * n
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

def num_islands(grid):
    if not grid:
        return 0
    R, C = len(grid), len(grid[0])
    seen = [[False] * C for _ in range(R)]
    count = 0
    for i in range(R):
        for j in range(C):
            if grid[i][j] != 1 or seen[i][j]:
                continue
            count += 1
            q = deque([(i, j)])
            seen[i][j] = True
            while q:
                r, c = q.popleft()
```

```

        for nr, nc in ((r+1,c), (r-1,c), (r,c+1), (r,c-1)):
            if 0 <= nr < R and 0 <= nc < C \
                and grid[nr][nc] == 1 and not seen[nr][nc]:
                seen[nr][nc] = True
                q.append((nr, nc))

    return count

```

Real output: `bfs_dist` on the graph of Example 3 gives `[0, 1, 1, 2, -1, -1, -1]`; `num_islands` on the grid of Example 8 gives 2, on `[]` gives 0, on `[[1]]` gives 1.

TRAP

Use `deque.popleft()`, never `list.pop(0)`. Popping from the front of a Python list is $O(n)$, which silently turns your $O(n + m)$ BFS into $O(n^2)$.

This is the **same bug** as `q.shift()` in JavaScript, in a different costume. Every language punishes “remove from the front of a plain array”; each one gives you a different escape hatch — `deque` in Python, the toolkit `Deque` or a `head index` in JS.

Practice

PRACTICE · WARM-UP target: 20 min

1. A graph has degrees `3 1 1 1 0`. How many edges does it have?
2. Undirected edges `[0,1], [1,2], [2,0]`, $n = 3$. Write the adjacency list.
3. Same graph. BFS from 0 visits nodes in what order?
4. $n = 4$, edges `[0,1], [2,3]`. How many components?

PRACTICE · TIER 1 target: 60 min

5. Given n and an undirected edge list, return the number of nodes that touch no edge at all.
6. Decide whether an undirected graph is a **tree**: connected and with no cycle.
7. Given a grid of 0s and 1s, for every cell report the distance to the nearest 1.
8. Return the number of nodes in the component that contains a given node x .
9. Decide whether the whole graph is one single simple cycle and nothing else.
10. Return how many edges you must delete so that no cycle remains.

PRACTICE · TIER 2 target: 70 min

11. Directed graph. Find a node from which every other node is reachable, or say there is none.
12. Find the node farthest (in edges) from a given source, and that distance.
13. Count the land islands that do **not** touch the border of the grid.
14. Directed graph. List every node that has no incoming edge.

PRACTICE · TIER 3 target: 60 min

15. Count the unordered pairs of nodes (u, v) that are **not** connected to each other. n up to 10^5 , so say whether the answer still fits in an exact JS number.
16. A **star** graph has one centre joined to all $n - 1$ others and no other edges. Given an edge list you believe is a star, find the centre (or say it is not a star).

ANSWER KEY

- Sum of degrees is $3 + 1 + 1 + 1 + 0 = 6$, so $\frac{6}{2} = 3$ edges.
- $0 \rightarrow 1$, $2, 1 \rightarrow 0$, $2, 2 \rightarrow 1$, 0 .
- 0 , 1 , 2 .
- Two: $[0, 1]$ and $[2, 3]$.
- Count rows of the adjacency list that are empty.

Q5

JS

```
const isolatedCount = (n, g) => g.filter(row => row.length === 0).length;
```

On $n = 7$, edges $[0, 1], [0, 2], [1, 3], [4, 5]$ the answer is **1** (node 6).

- A tree on n nodes has exactly $n - 1$ edges **and** no cycle. Check the edge count first, then use DSU: if any `unite` returns false, an edge closed a loop.

Q6

JS

```
const isTree = (n, E) => {
  if (E.length !== n - 1) return false;
  const d = new DSU(n);
  for (const [u, v] of E) if (!d.union(u, v)) return false;
  return true;
};
```

Verified: 4 nodes with $[0, 1], [1, 2], [1, 3] \rightarrow$ true. Triangle $\rightarrow$ false. Single node, no edges $\rightarrow$ true.

- Multi-source BFS: push every cell holding a 1 with distance 0, then expand.

Q7

JS

```
const nearestOne = (a) => {
  const R = a.length, C = a[0].length;
  const d = Array.from({ length: R }, () => new Array(C).fill(-1));
  const q = [];
  for (let i = 0; i < R; i++)
    for (let j = 0; j < C; j++)
      if (a[i][j] === 1) { d[i][j] = 0; q.push([i, j]); }
  for (let head = 0; head < q.length; head++) {
    const [r, c] = q[head];
    for (let k = 0; k < 4; k++) {
      const nr = r + DR[k], nc = c + DC[k];
      if (nr < 0 || nr >= R || nc < 0 || nc >= C || d[nr][nc] !== -1) continue;
      d[nr][nc] = d[r][c] + 1;
      q.push([nr, nc]);
    }
  }
  return d;
};
```

On $[[0, 0, 1], [0, 0, 0], [1, 0, 0]]$ the real output is $[[2, 1, 0], [1, 2, 1], [0, 1, 2]]$.

- One BFS, count what you visit.

Q8

JS

```
const componentSize = (n, g, x) => {
  const vis = new Array(n).fill(false);
```

```
const q = [x]; vis[x] = true;
let c = 0;
for (let head = 0; head < q.length; head++) {
  const u = q[head]; c++;
  for (const v of g[u]) if (!vis[v]) { vis[v] = true; q.push(v); }
}
return c;
};
```

Verified: on the usual 7-node graph, `componentSize(0) = 4, componentSize(6) = 1`.

9. A single simple cycle needs $n \geq 3$, every degree exactly 2, and all nodes in one component. Two separate triangles pass the degree test but fail the connectivity test.

Q9

JS

```
const isSingleCycle = (n, g) => {
  if (n < 3) return false;
  for (let i = 0; i < n; i++) if (g[i].length !== 2) return false;
  return componentSize(n, g, 0) === n;
};
```

Verified: square $\rightarrow$ true; two triangles ($n = 6$) $\rightarrow$ false.

10. Every edge that DSU rejects is an edge that closed a loop, and deleting exactly those leaves a forest.

Q10

JS

```
const edgesToRemove = (n, E) => {
  const d = new DSU(n);
  let kept = 0;
  for (const [u, v] of E) if (d.union(u, v)) kept++;
  return E.length - kept;
};
```

Verified: $n = 5$, edges $[0,1], [1,2], [2,0], [3,4] \rightarrow 1$.

11. DFS from every unvisited node; remember the node that started the **last** DFS. Only that node can possibly reach everyone, so run one more DFS from it and check.

Q11 — iterative, so 10^5 nodes are safe

JS

```
const motherVertex = (n, g) => {
  const flood = (s, vis) => {
    const st = [s]; vis[s] = true;
    while (st.length) {
      const u = st.pop();
      for (const v of g[u]) if (!vis[v]) { vis[v] = true; st.push(v); }
    }
  };
  const vis = new Array(n).fill(false);
  let last = 0;
  for (let i = 0; i < n; i++) if (!vis[i]) { flood(i, vis); last = i; }
  const v2 = new Array(n).fill(false);
  flood(last, v2);
  return v2.every(Boolean) ? last : -1;
};
```

Verified: $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 2$, $2 \rightarrow 3$ gives 0. $0 \rightarrow 1$ with node 2 loose gives -1 .

12. BFS and keep the largest distance seen.

Q12

JS

```
const farthest = (n, g, src) => {
  const dist = new Array(n).fill(-1);
  const q = [src]; dist[src] = 0;
  let bi = src, bd = 0;
  for (let head = 0; head < q.length; head++) {
    const u = q[head];
    if (dist[u] > bd) { bd = dist[u]; bi = u; }
    for (const v of g[u]) if (dist[v] === -1) { dist[v] = dist[u] + 1;
    q.push(v); }
  }
  return [bi, bd];
};
```

Verified on the chain 0-1-2-3-4-5: from 0 → node 5 at distance 5; from 2 → node 5 at distance 3.

13. Flood each island; while flooding, watch whether any of its cells sits on row 0, row $R - 1$, column 0 or column $C - 1$.

Q13

JS

```
const closedIslands = (grid) => {
  const g = grid.map(r => [...r]);
  const R = g.length, C = g[0].length;
  let cnt = 0;
  for (let i = 0; i < R; i++)
    for (let j = 0; j < C; j++) {
      if (g[i][j] !== 1) continue;
      let closed = true;
      const q = [[i, j]]; g[i][j] = 0;
      for (let head = 0; head < q.length; head++) {
        const [r, c] = q[head];
        if (r === 0 || c === 0 || r === R - 1 || c === C - 1) closed = false;
        for (let k = 0; k < 4; k++) {
          const nr = r + DR[k], nc = c + DC[k];
          if (nr < 0 || nr >= R || nc < 0 || nc >= C || g[nr][nc] !== 1) continue;
          g[nr][nc] = 0; q.push([nr, nc]);
        }
      }
      if (closed) cnt++;
    }
  return cnt;
};
```

Verified: $[[0,0,0],[0,1,0],[0,0,0]] \rightarrow 1$. $[[1,1],[1,1]] \rightarrow 0$.

14. Count incoming edges, then list the zeros.

Q14

JS

```
const noIncoming = (n, g) => {
  const indeg = new Array(n).fill(0);
  for (let u = 0; u < n; u++) for (const v of g[u]) indeg[v]++;
  return indeg.map((d, i) => d === 0 ? i : -1).filter(i => i !== -1);
};
```

Verified on 0→1, 0→2, 1→2, 2→3: the answer is 0.

15. Total pairs minus the pairs inside components. If component sizes are $c_1, c_2, \dots$ then

$$\text{answer} = \frac{n(n-1)}{2} - \sum_i c_i \frac{c_i - 1}{2}$$

Q15

JS

```
const disconnectedPairs = (n, g) => {
  const vis = new Array(n).fill(false);
  const total = n * (n - 1) / 2;
  let same = 0;
  for (let i = 0; i < n; i++) {
    if (vis[i]) continue;
    let c = 0;
    const q = [i]; vis[i] = true;
    for (let head = 0; head < q.length; head++) {
      const u = q[head]; c++;
      for (const v of g[u]) if (!vis[v]) { vis[v] = true; q.push(v); }
    }
    same += c * (c - 1) / 2;
  }
  return total - same;
};
```

Verified: $n = 7$ with components of size 4, 2, 1 gives $21 - (6 + 1 + 0) = 14$. A triangle gives 0.

This is the question C++ answers with a 64-bit integer: at $n = 10^5$ the total is 4999950000, past the 32-bit limit of about 2.1×10^9 . JavaScript needs no annotation — `Number.isSafeInteger(4999950000)` is `true`, and the exact range reaches $2^{53} - 1 \approx 9 \times 10^{15}$. Be able to say that out loud; “JS numbers are doubles so it just works” is only half an answer, and the other half is the bound.

16. In a star with n nodes the centre has degree $n - 1$ and there are exactly $n - 1$ edges.

Q16

JS

```
const starCentre = (n, E) => {
  if (n < 2 || E.length !== n - 1) return -1;
  const deg = new Array(n).fill(0);
  for (const [u, v] of E) { deg[u]++; deg[v]++; }
  for (let i = 0; i < n; i++) if (deg[i] === n - 1) return i;
  return -1;
};
```

Verified: $[0, 1], [0, 2], [0, 3] \rightarrow 0$. The chain $[0, 1], [1, 2], [2, 3] \rightarrow -1$.

ONE-PAGE REVISION CARD

Build the graph — always start here

Adjacency list

JS

```
const { DSU, Deque } = require('./toolkit.js'); // JS Toolkit appendix

const g = Array.from({ length: n }, () => []); // NOT new Array(n).fill([])
for (const [u, v] of edges) {
  g[u].push(v);
  g[v].push(u); // drop for directed
}
```

BFS — shortest edge count

BFS

JS

```
const dist = new Array(n).fill(-1);
const q = [src]; // array + head index, never q.shift()
dist[src] = 0;
for (let head = 0; head < q.length; head++) {
  const u = q[head];
  for (const v of g[u]) if (dist[v] === -1) {
    dist[v] = dist[u] + 1; q.push(v);
  }
}
```

DFS — reach everything in one component

DFS, recursive and iterative

JS

```
// clear, but dies past ~10^4 depth
const dfs = (u) => { vis[u] = 1; for (const v of g[u]) if (!vis[v]) dfs(v); };

// what you actually ship at n = 10^5
const st = [src];
while (st.length) {
  const u = st.pop();
  if (vis[u]) continue;
  vis[u] = 1;
  for (const v of g[u]) if (!vis[v]) st.push(v);
}
```

Which tool?

fewest edges / minutes	BFS
spread from many starts at once	multi-source BFS
count pieces, flood a region	DFS or BFS or DSU
cycle, undirected	BFS with parent, or edges $\geq$ nodes
cycle, directed	3-colour DFS
bridges / articulation points	DFS with tin and low

groups in a directed graph	Kosaraju (two DFS passes)
many connectivity queries as edges arrive	DSU

Complexity

algorithm	time	space
BFS / DFS	$O(n + m)$	$O(n + m)$
grid BFS	$O(RC)$	$O(RC)$
DSU over m edges	$O(m\alpha(n))$	$O(n)$
bridges / articulation points	$O(n + m)$	$O(n + m)$
Kosaraju SCC	$O(n + m)$	$O(n + m)$
shortest cycle (girth)	$O(n(n + m))$	$O(n)$

Top traps

1	<code>q.shift()</code> is $O(n)$ in JavaScript. A BFS written with it is $O(n^2)$. Use an array with a head index, or the toolkit <code>Deque</code> .
2	Recursion depth in Node is roughly 10^4 . A recursive DFS on a 10^5 -node chain throws <code>RangeError</code> . Use BFS or an explicit stack.
3	<code>new Array(n).fill([])</code> puts one array in every slot. Use <code>Array.from({length: n}, () => [])</code> .
4	<code>[...grid]</code> shares every row. A real 2D copy is <code>grid.map(r => [...r])</code> .
5	Marking visited on pop instead of on push . The queue explodes.
6	Forgetting the outer loop over all nodes. Disconnected graphs are silently half-checked.
7	Using the parent trick for cycles in a directed graph. Use 3 colours instead.
8	<code>v !== par[u]</code> written as <code>v !== par[v]</code> .
9	Skipping the parent node instead of the parent edge id when finding bridges. Parallel edges break it.
10	Route counts past $2^{53} - 1$ round rather than overflow. Take a modulus at every <code>+=</code> , or switch to <code>BigInt</code> .
11	In flood fill, forgetting the “new colour equals old colour” guard. Infinite loop.
12	Bumping the minute counter on the final empty ring of a multi-source BFS.
13	Sizing an array from the target only, when the start can sit outside it. An out-of-range read gives <code>undefined</code> , not a crash.
14	<code>==</code> where you meant <code>===</code> . Graph input parsed from text arrives as strings, and <code>"2" == 2</code> is <code>true</code> while <code>"2" === 2</code> is not.

The one sentence to remember Every unweighted “shortest / fewest / minimum steps” question is BFS. Every “how many pieces / can I reach / is there a loop” question is DFS or DSU.

Graphs II — Shortest Paths, MST & Topological Sort

Now the edges carry numbers. Pick the right algorithm and the code is short.

Pattern in one page

Chapter 15 had roads with no length. Every step cost 1, so BFS answered everything. Now each edge carries a **weight**: a price, a distance, a delay. BFS is no longer enough.

THE ONE OPERATION BEHIND EVERY SHORTEST-PATH ALGORITHM

Relaxation. You hold a best-known distance $d[v]$ for every node. For an edge $u \rightarrow v$ of weight w :

```
if (d[u] + w < d[v]) d[v] = d[u] + w;
```

“I found a cheaper way to reach v : go to u first, then take this edge.”

Every algorithm below is just a different **order** of doing relaxations.

Storing a weighted graph

An edge is a two-element array `[to, weight]`, so the loop body destructures straight into named variables. The whole graph is an array of such arrays.

Weighted adjacency list

JS

```
const buildW = (n, E, directed) => {
  const g = Array.from({ length: n }, () => []);
  for (const [u, v, w] of E) {          // e = [from, to, weight]
    g[u].push([v, w]);
    if (!directed) g[v].push([u, w]);
  }
  return g;
};
```

TRAP

Do not write `new Array(n).fill([])`. `fill` stores the **same** array object in every slot, so pushing an edge into `g[0]` pushes it into all n lists at once. Use `Array.from({ length: n }, () => [])`, which calls the factory once per slot.

The same shallow-copy rule bites a grid: `[...grid]` copies the outer array but shares every row. A real copy is `grid.map(r => [...r])`.

TRAP

JavaScript numbers are doubles, exact only to `Number.MAX_SAFE_INTEGER =  $2^{53} - 1 \approx 9.0 \times 10^{15}$` . Nothing in this chapter passes that line — 10^5 edges of weight 10^9 cap a path at 10^{14} — so plain numbers are correct throughout. Verified: `3e9 + 3e9` prints `6000000000` exactly, and a 3-node MST of two `3e9` cables returns `6000000000`.

Past 2^{53} you need BigInt: $2n \cdot 60n$ is exact, while $2 \cdot 60$ is not. Say this out loud when an interviewer asks about “overflow” in JavaScript — the answer is **precision loss**, not wraparound. There is no int and no 64-bit integer type; the only two number types are Number and BigInt.

Choose the algorithm from the constraints

This table is the most useful thing in the chapter. Learn it.

You are asked	Weights	Algorithm	Time
one source → all nodes	all ≥ 0	Dijkstra with a heap	$O((n + m) \log n)$
one source → all nodes	some negative	Bellman-Ford	$O(nm)$
one source → all nodes	all equal to 1	BFS	$O(n + m)$
one source → all nodes	only 0 and 1	0-1 BFS with a deque	$O(n + m)$
one source → all nodes	graph is a DAG	relax in topological order	$O(n + m)$
every pair	any, $n \leq 400$	Floyd-Warshall	$O(n^3)$
is there a negative loop?	any	Bellman-Ford, one extra pass	$O(nm)$
cheapest way to link everything	any	Kruskal or Prim	$O(m \log m)$
a valid order of dependent jobs	—	topological sort	$O(n + m)$

Dijkstra — the template

TRAP

JavaScript has no built-in priority queue. Every shortest-path algorithm in this chapter needs one, so import the tested MinHeap from the **JS Toolkit** appendix. Write that `require` line once at the top of your solution file and never re-implement a heap under interview pressure.

The heap stores `[distance, node]` pairs and takes a comparator, so a **max**-heap is the same class with the comparator flipped: `new MinHeap((a, b) => b[0] - a[0])`. You will need that for the widest-path problem later.

Dijkstra with a min-heap. Memorise this.

JS

```
const { MinHeap, DSU, Deque } = require('./toolkit.js'); // JS Toolkit appendix

const dijkstra = (n, g, src) => {
  const d = new Array(n).fill(Infinity);
  const pq = new MinHeap((a, b) => a[0] - b[0]); // [dist, node]
  d[src] = 0;
  pq.push([0, src]);
  while (pq.size) {
    const [du, u] = pq.pop();
    if (du > d[u]) continue; // stale copy, skip it
    for (const [v, w] of g[u]) {
      if (du + w < d[v]) {
        d[v] = du + w;
        pq.push([d[v], v]);
      }
    }
  }
}
```

```
    return d;
};
```

SHORTCUT

Use Infinity, not a big finite sentinel, for “unreached”. $\text{Infinity} + w$ is still Infinity, and $\text{Infinity} < \text{Infinity}$ is false, so an unreachable node can never be relaxed by accident. In C++ this is the classic $\text{INF} + w$ overflow bug; JavaScript hands you the fix for free. Verified: seeding a Bellman-Ford with `Number.MAX_SAFE_INTEGER` instead lets the edge $2 \rightarrow 3$ of weight -5 invent a finite distance for an unreachable node, while the Infinity version leaves it untouched.

THE DIJKSTRA INVARIANT — AND WHY NEGATIVES BREAK IT

When a node is popped with $du == d[u]$, that distance is **final**. The reason: every remaining edge has weight ≥ 0 , so no route through a node that is still in the heap (and therefore already costs at least du) can arrive cheaper.

A single negative edge destroys that reason, and the algorithm gives wrong answers.

Here is the failure, actually run. Graph: $0 \rightarrow 1$ (2), $0 \rightarrow 2$ (5), $2 \rightarrow 1$ (-4), $1 \rightarrow 3$ (1). Most people write Dijkstra with a `done[]` array — settle a node once, never look at it again — so that is the version shown failing.

Real output — the settled version is wrong on nodes 1 and 3

TEXT

```
settled Dijkstra : 0 2 5 3
truth (Bellman)  : 0 1 5 2
```

It locked node 1 at distance 2 and immediately set $d[3] = 3$. Later the edge $2 \rightarrow 1$ offered $5 - 4 = 1$, but node 1 was already marked done, so neither it nor node 3 was ever revisited.

Be precise about this in an interview. The lazy template above — no `done[]` array, only if $(du > d[u])$ continue; — happens to recover the right answer on this particular graph, because a node whose distance drops is simply pushed again. That is not a defence: with negative edges that re-pushing can blow up to exponentially many pops, and the algorithm is no longer Dijkstra, it is a slow Bellman-Ford. The rule stands: **one negative edge means Bellman-Ford**.

TRAP

A JavaScript comparator must return a number, not a boolean. $(a, b) \Rightarrow a[0] > b[0]$ yields `true/false`, which coerce to `1/0` — the heap never sees a negative value and its ordering collapses. Write $(a, b) \Rightarrow a[0] - b[0]$. The same rule governs `arr.sort`, which you will meet in Kruskal in a moment.

TRAP

Do **not** write `if (done[u]) continue; done[u] = true;` **and** also forget the $du > d[u]$ check — pick one. The lazy $du > d[u]$ test is the safer habit, because it never needs an extra array.

Topological order — the template

A **DAG** is a directed graph with no cycle. A **topological order** lists its nodes so that every arrow points forward in the list.

Kahn's algorithm (BFS style)

JS

```
const topoKahn = (n, g) => {
  const indeg = new Array(n).fill(0);
```

```

    for (let u = 0; u < n; u++) for (const v of g[u]) indeg[v]++;
    const pq = new MinHeap(); // default comparator: smallest id
first
    for (let i = 0; i < n; i++) if (indeg[i] === 0) pq.push(i);
    const order = [];
    while (pq.size) {
        const u = pq.pop();
        order.push(u);
        for (const v of g[u]) if (--indeg[v] === 0) pq.push(v);
    }
    return order.length === n ? order : []; // short -> some node never hit 0
-> cycle
};

```

SHORTCUT

Kahn gives cycle detection for free. If the output is shorter than n , the leftover nodes all sit on or after a cycle. Swap the MinHeap for the toolkit Deque used as a plain queue when you do not care about ties — it is faster and the answer is still valid.

TRAP

Kahn never recurses, so it is the **safe** topological sort at $n = 10^5$. The DFS version later in this chapter recurses once per node, and Node's stack gives out at roughly 10^4 frames: a 100000-node chain throws `RangeError: Maximum call stack size exceeded`. That was verified, not guessed. When the constraint says $n \leq 10^5$, reach for Kahn.

Warm-up

WARM-UP

build the reflex

Example 1

Relax the edge $u \rightarrow v$ with weight 7, given $d[u] = 4$ and $d[v] = 13$. What is $d[v]$ afterwards?

SOLUTION

$d[u] + w = 4 + 7 = 11$. Is $11 < 13$? Yes. So $d[v]$ becomes 11.

Answer: 11

Example 2

Same edge, but now $d[v] = 9$.

SOLUTION

$4 + 7 = 11$, and $11 < 9$ is false. Nothing changes.

Answer: 9, unchanged

Example 3

A directed graph has $0 \rightarrow 1$ (4), $0 \rightarrow 2$ (1), $2 \rightarrow 1$ (2), $1 \rightarrow 3$ (1), $2 \rightarrow 3$ (5), and a lone node 4. Compute the shortest distances from node 0 by hand.

| SOLUTION

1. $d[0] = 0$.
2. Direct edges: $d[1] = 4$, $d[2] = 1$.
3. Through 2: $1 + 2 = 3 < 4$, so $d[1] = 3$.
4. To 3: through 1 costs $3 + 1 = 4$; through 2 costs $1 + 5 = 6$. Keep 4.
5. Node 4 is unreachable.

Verified by running both Dijkstra versions: 0 3 1 4 INF.

Answer: 0 3 1 4 INF

Example 4

Compute the in-degree of every node for the DAG $5 \rightarrow 0$, $5 \rightarrow 2$, $4 \rightarrow 0$, $4 \rightarrow 1$, $2 \rightarrow 3$, $3 \rightarrow 1$, with $n = 6$.

SOLUTION

Walk the arrow list and add 1 to the **head** of each arrow.

node	0	1	2	3	4	5
in-degree	2	2	1	1	0	0

Answer: 2 2 1 1 0 0

Example 5

Using the in-degrees above, list a valid topological order. Always take the smallest ready node.

SOLUTION

Ready at the start: {4, 5} (in-degree 0).

step	take	in-degrees that drop	ready set after
1	4	$0 \rightarrow 1$, $1 \rightarrow 1$	{5}
2	5	$0 \rightarrow 0$, $2 \rightarrow 0$	{0, 2}
3	0	none	{2}
4	2	$3 \rightarrow 0$	{3}
5	3	$1 \rightarrow 0$	{1}
6	1	none	{}

Verified real output: 4 5 0 2 3 1.

Answer: 4 5 0 2 3 1

Example 6

Why can Dijkstra not be trusted when one edge has weight -4 ? Answer with the graph $0 \rightarrow 1$ (2), $0 \rightarrow 2$ (5), $2 \rightarrow 1$ (-4), $1 \rightarrow 3$ (1).

SOLUTION

Dijkstra pops the **cheapest** node and declares it finished. Node 1 looks like distance 2, so it is finished, and $d[3]$ is written as 3.

Only later does the route $0 \rightarrow 2 \rightarrow 1$ (cost $5 - 4 = 1$) appear. Node 1 gets a better value, but node 3 was already processed and is never revisited.

Real output from the settled version: 0 2 5 3; the truth is 0 1 5 2.

Answer: because “cheapest so far” stops being “cheapest ever” when edges can subtract

Tier 1 — the standard weighted questions

Example 7 [TCS NQT · pattern]

Shortest distance from one city to all others. n cities, m two-way roads with positive lengths. Print the distance from city 0 to every city, or -1 where there is no route.

Constraints: $n \leq 10^5$, $m \leq 2 \times 10^5$, weight $\leq 10^9$. Edge cases: an unreachable city; $n = 1$.

SOLUTION**Approach 1 — Scan for the closest unfinished node** ($O(n^2)$ — fine only to $n = 5000$)**Dijkstra without a heap**

JS

```
const dijkstraSimple = (n, g, src) => {
  const d = new Array(n).fill(Infinity);
  const done = new Array(n).fill(false);
  d[src] = 0;
  for (let it = 0; it < n; it++) {
    let u = -1;
    for (let i = 0; i < n; i++)
      if (!done[i] && (u === -1 || d[i] < d[u])) u = i;
    if (u === -1 || d[u] === Infinity) break;
    done[u] = true;
    for (const [v, w] of g[u])
      if (d[u] + w < d[v]) d[v] = d[u] + w;
  }
  return d;
};
```

Time $O(n^2 + m)$ **Space** $O(n)$

Actually the better choice when the graph is dense, m near n^2 .

Approach 2 — Min-heap Dijkstra ($O((n+m) \log n)$ — optimal for sparse graphs)

Use the template from page one. Both versions were run on the graph of Warm-up 3 and both printed 0 3 1 4 INF.

Time $O((n + m) \log n)$ **Space** $O(n + m)$

The idea that unlocked it: the $O(n^2)$ version spends all its time **searching** for the minimum. A heap does that search in $\log n$ instead of n .

TRAP

Print -1 , not Infinity , for unreachable nodes. Forgetting the conversion is the single most common submission failure on this problem — and in JavaScript it bites twice, because `JSON.stringify(Infinity)` silently becomes `null`. Map the array before you print: `d.map(x => x === Infinity ? -1 : x)`.

Answer: 0 3 1 4 -1

Example 8 [Infosys · pattern]

Print the route, not just its length. Same input. Print the actual list of cities on one shortest route from `src` to `dst`. Edge cases: no route (print nothing); `src == dst` (print just that city).

SOLUTION

Every time a relaxation succeeds, remember **who** caused it. Then walk the parents backwards from the destination and reverse.

Dijkstra plus a parent array

JS

```
const shortestPathNodes = (n, g, src, dst) => {
  const d = new Array(n).fill(Infinity);
  const par = new Array(n).fill(-1);
  const pq = new MinHeap((a, b) => a[0] - b[0]);
  d[src] = 0; pq.push([0, src]);
  while (pq.size) {
    const [du, u] = pq.pop();
    if (du > d[u]) continue;
    for (const [v, w] of g[u])
      if (du + w < d[v]) {
        d[v] = du + w;
        par[v] = u; // remember the winner
        pq.push([d[v], v]);
      }
  }
  if (d[dst] === Infinity) return []; // no route at all
  const path = [];
  for (let v = dst; v !== -1; v = par[v]) path.push(v);
  return path.reverse();
};
```

Time $O((n + m) \log n)$ **Space** $O(n)$

Verified on the graph of Warm-up 3: the route from 0 to 3 prints 0 2 1 3, cost $1 + 2 + 1 = 4$. The route from 0 to the lone node 4 prints nothing.

Answer: 0 2 1 3

Example 9 [Accenture · pattern]

Shortest paths when some weights are negative. Directed graph, weights may be negative, but you are told there is no negative loop.

Constraints: $n \leq 2000$, $m \leq 10^4$. Target $O(nm)$. Edge cases: a node unreachable from the source (must stay INF, never $\text{INF} + w$); a graph with no edges.

SOLUTION

Bellman-Ford. Relax **every** edge, $n - 1$ times.

WHY $N - 1$ ROUNDS IS ENOUGH

A shortest path with no repeated node uses at most $n - 1$ edges. After round k , every shortest path that uses at most k edges has been found. So after $n - 1$ rounds you are done.

If round n still improves something, a path is getting cheaper without limit — that is a **negative cycle**.

Bellman-Ford with early exit and cycle detection

JS

```
const bellmanFord = (n, E, src) => {
  const d = new Array(n).fill(Infinity);
  d[src] = 0;
  for (let i = 0; i < n - 1; i++) {
```

```

let changed = false;
for (const [u, v, w] of E)
  if (d[u] + w < d[v]) { d[v] = d[u] + w; changed = true; }
  if (!changed) break; // nothing moved: stop early
}
for (const [u, v, w] of E) // one extra pass
  if (d[u] + w < d[v]) return { ok: false, d }; // still improving -> negative
cycle
return { ok: true, d };
};

```

Time $O(nm)$ **Space** $O(n)$

Verified on 0->1 (4), 0->2 (5), 2->1 (-3), 1->3 (3): returns ok: true, distances 0 2 5 5. On 0->1 (1), 1->2 (-1), 2->1 (-1), 0->3 (2): returns ok: false. An empty edge list gives 0 Infinity Infinity, and $n = 1$ gives 0.

TRAP

Most write-ups of Bellman-Ford guard the relaxation with `if (d[u] !== INF && ...)`, because with a large **finite** sentinel `INF + (-5)` is smaller than `INF` and unreachable nodes start growing fake distances. In JavaScript that guard is unnecessary **provided you seed with Infinity**: `Infinity + (-5)` is still `Infinity`, and `Infinity < Infinity` is false. Verified both ways — seeding with `Number.MAX_SAFE_INTEGER` reproduces the bug exactly (node 3 gets a finite distance from an unreachable node 2); seeding with `Infinity` leaves it at `Infinity`.

So: seed with `Infinity` and drop the guard, or keep a finite sentinel and keep the guard. Mixing the two is what fails.

SHORTCUT

The `changed` flag usually ends the loop long before $n - 1$ rounds, which makes Bellman-Ford fast in practice even though its worst case is $O(nm)$.

Answer: true, 0 2 5 5

Example 10 [Wipro · pattern]

All pairs at once. $n \leq 400$. Print the shortest distance between **every** pair of nodes. Edge cases: parallel edges of different weights (keep the smallest); `d[i][i] = 0`.

SOLUTION

Floyd-Warshall. Three nested loops, and the outer loop must be k .

WHAT THE K LOOP MEANS

After the outer loop has finished value k , `d[i][j]` is the best route from i to j that is allowed to pass through intermediate nodes taken only from $\{0, 1, \dots, k\}$.

So you add one permitted “stopover” at a time. That is why k must be outermost — if i or j is outermost, the recurrence reads values that have not been prepared yet.

Floyd-Warshall

JS

```

const floyd = (n, E, directed) => {
  const d = Array.from({ length: n }, () => new Array(n).fill(Infinity));

```

```

for (let i = 0; i < n; i++) d[i][i] = 0;
for (const [u, v, w] of E) {
    d[u][v] = Math.min(d[u][v], w); // min: parallel edges
    if (!directed) d[v][u] = Math.min(d[v][u], w);
}
for (let k = 0; k < n; k++)
    for (let i = 0; i < n; i++) {
        if (d[i][k] === Infinity) continue; // small test, big speed win
        for (let j = 0; j < n; j++)
            if (d[i][k] + d[k][j] < d[i][j]) d[i][j] = d[i][k] + d[k][j];
    }
return d;
};

```

Time $O(n^3)$ **Space** $O(n^2)$

$n = 400$ gives 6.4×10^7 — fast. $n = 1000$ gives 10^9 — too slow.

TRAP

Build the matrix with `Array.from({ length: n }, () => new Array(n).fill(Infinity)).new Array(n).fill(new Array(n).fill(Infinity))` looks identical and is catastrophically wrong: all n rows are the **same** array, so writing `d[0][1]` writes every row's column 1. [...row] has the same flaw one level down.

Verified on the directed graph of Warm-up 3 (first 4 nodes):

Real output

TEXT

```

F[0]: 0   3   1   4
F[1]: INF 0   INF 1
F[2]: INF 2   0   3
F[3]: INF INF INF 0

```

SHORTCUT

After Floyd, $d[i][i] < 0$ for some i means a negative cycle passes through i .

Answer: the table above

Example 11 [Capgemini · pattern]

Cheapest way to wire every office. n offices, a list of possible cables each with a price. Choose cables so that all offices are linked and the total price is smallest. Return the total, or -1 if it cannot be done.

Constraints: $n \leq 10^5$, $m \leq 2 \times 10^5$. Target $O(m \log m)$. Edge cases: the graph is not connected (-1); $n = 1$ (answer 0, zero cables).

SOLUTION

This is a **minimum spanning tree** (MST).

Approach 1 — Kruskal — sort the edges, take what fits ($O(m \log m)$ — optimal)

Sort every cable by price. Walk the sorted list. Take a cable only if its two ends are still in different groups; DSU answers that in near-constant time.

Use the tested DSU from the **JS Toolkit** appendix — its `union(a, b)` returns `false` when the two ends were already together, which is exactly the test Kruskal needs.

```
const kruskal = (n, E) => {
  const e = [...E].sort((a, b) => a[2] - b[2]); // numeric comparator, never a
  bare sort()
  const d = new DSU(n);
  let total = 0, used = 0;
  for (const [u, v, w] of e)
    if (d.union(u, v)) { total += w; used++; }
  return used === n - 1 ? total : -1; // fewer edges -> not connected
};
```

TRAP

arr.sort() with empty brackets is LEXICOGRAPHIC. It converts every element to a string first, so `[10, 9, 1].sort()` gives `[1, 10, 9]`. On an edge list it is worse, because each edge stringifies whole: sorting `[[0,1,1],[1,2,2],[0,2,4],[2,3,3],[3,0,10]]` with a bare `.sort()` produces the weight order 1, 4, 2, 3, 10 — verified — and Kruskal then returns a wrong total. Always `sort((a, b) => a[2] - b[2])`.

Two companions to that rule: `sort` works **in place** and returns the same array, so copy with `[...E]` before sorting a caller's list (this code does, and a test confirms the caller's array comes back untouched); and chain tie-breaks with `||`, as in `(a, b) => a[2] - b[2] || a[0] - b[0]`, because `0` is falsy.

Time $O(m \log m)$ **Space** $O(n)$

The sort dominates; DSU is almost free.

Approach 2 — Prim — grow one blob ($O(m \log n)$, better on dense graphs)

Start at node 0. Repeatedly add the cheapest cable leaving the blob.

Prim with a min-heap

```
const prim = (n, g) => {
  if (n === 0) return 0;
  const inMst = new Array(n).fill(false);
  const pq = new MinHeap((a, b) => a[0] - b[0]); // [weight, node]
  pq.push([0, 0]);
  let total = 0, cnt = 0;
  while (pq.size) {
    const [w, u] = pq.pop();
    if (inMst[u]) continue;
    inMst[u] = true; total += w; cnt++;
    for (const [v, ew] of g[u]) if (!inMst[v]) pq.push([ew, v]);
  }
  return cnt === n ? total : -1;
};
```

Time $O(m \log n)$ **Space** $O(n + m)$

Verified on 0-1:1, 1-2:2, 0-2:4, 2-3:3, 3-0:10: Kruskal and Prim both give **6**, using edges (0-1,1) (1-2,2) (2-3,3). Disconnected input gives **-1** from both. A single node gives 0 from both. Two cables of weight 3×10^9 gave 6000000000 exactly from both — comfortably inside $2^{53} - 1$, so no `BigInt` is needed here.

WHY GREEDY IS SAFE HERE (THE CUT PROPERTY)

Split the nodes into any two groups. The **cheapest** edge crossing that split is in some MST.

Kruskal picks exactly such edges: when it takes an edge, the two ends are in different DSU groups, so that edge is the cheapest one crossing the split “this group versus everything else”.

TRAP

Note the used `== n - 1` test at the end. Without it, a disconnected graph quietly returns the total of several separate trees.

Answer: 6

Example 12 [Cognizant · pattern]

A valid build order. n modules, and pairs (a, b) meaning “ a must be built before b ”. Print any valid order, or say it is impossible. Edge cases: no dependencies at all (any order works); a two-module loop (impossible).

SOLUTION

Topological sort. Two standard ways; know both.

Approach 1 — Kahn — peel off the ready nodes ($O(n+m)$, detects cycles for free)

See the template on page one. Verified: on $5 \rightarrow 0$, $5 \rightarrow 2$, $4 \rightarrow 0$, $4 \rightarrow 1$, $2 \rightarrow 3$, $3 \rightarrow 1$ it prints `4 5 0 2 3 1`. On a two-node loop it returns an empty list.

Approach 2 — DFS finish order ($O(n+m)$, fewer lines)

Run the 3-colour DFS from Chapter 15. Push each node onto a list **when it finishes**, then reverse the list.

Topological sort by DFS

JS

```
const topoDFS = (n, g) => {
  const col = new Array(n).fill(0);           // 0 white, 1 grey, 2 black
  const out = [];
  const visit = (u) => {
    col[u] = 1;
    for (const v of g[u]) {
      if (col[v] === 1) return false;          // grey -> cycle
      if (col[v] === 0 && !visit(v)) return false;
    }
    col[u] = 2;
    out.push(u);                               // push on FINISH
    return true;
  };
  for (let i = 0; i < n; i++)
    if (col[i] === 0 && !visit(i)) return [];
  return out.reverse();
};
```

TRAP

This version recurses, and Node’s call stack is only about 10^4 frames deep. A 100000-node chain makes it throw `RangeError: Maximum call stack size exceeded` — verified, not guessed. C++ and Java survive that depth; JavaScript does not. At $n = 10^5$ either use Kahn (no recursion at all) or rewrite `visit` with an explicit stack of `[node, edgeIndex]` frames. Say this in the interview before you write the recursive version — it is exactly the kind of language detail that separates a pass from a fail.

Time $O(n + m)$ **Space** $O(n)$

Verified on the same graph: prints 5 4 2 3 1 0. Different from Kahn's answer and equally valid — check each arrow: 5 before 0 and 2, 4 before 0 and 1, 2 before 3, 3 before 1. All forward.

When a problem says “print the **lexicographically smallest** valid order”, you must use Kahn with a **priority queue**, as in the template. DFS cannot give you that.

Answer: 4 5 0 2 3 1 (or 5 4 2 3 1 0)

Example 13 [TCS Digital · pattern]

Shortest path in a DAG — in linear time. Weights may even be negative, as long as the graph has no cycle. Edge cases: unreachable nodes stay INF.

SOLUTION

No heap needed. Sort the nodes topologically, then relax the edges **in that order**. When you reach node u , every route into u has already been considered, so $d[u]$ is final.

DAG shortest path, $O(n + m)$

JS

```
const dagShortest = (n, g, src) => {
  const plain = g.map(row => row.map(([v]) => v)); // drop the weights for the sort
  const order = topoDFS(n, plain);
  const d = new Array(n).fill(Infinity);
  d[src] = 0;
  for (const u of order) {
    if (d[u] === Infinity) continue;
    for (const [v, w] of g[u]) d[v] = Math.min(d[v], d[u] + w);
  }
  return d;
};
```

Time $O(n + m)$ **Space** $O(n + m)$

Verified on 0->1 (2), 0->4 (1), 1->2 (3), 4->2 (2), 2->3 (6), 4->5 (4), 5->3 (1): result 0 2 3 6 1 5. Check node 3 by hand: through 2 costs 3 + 6 = 9; through 5 costs 5 + 1 = 6. The code says 6. Correct. A negative edge is fine too: 0->1 (-5) with an unreachable node 2 gives 0 -5 Infinity.

Answer: 0 2 3 6 1 5

Example 14 [Infosys · pattern]

Longest path in a DAG. Same DAG. What is the largest total weight of any path? Edge cases: no edges (answer 0).

SOLUTION

Longest paths are **hard** in a general graph but **easy** in a DAG: relax with max instead of min, still in topological order.

Longest path (critical path)

JS

```
const dagLongest = (n, g) => {
  const plain = g.map(row => row.map(([v]) => v));
  const order = topoDFS(n, plain);
  const best = new Array(n).fill(0);
  let ans = 0;
  for (const u of order)
```

```

    for (const [v, w] of g[u]) {
        best[v] = Math.max(best[v], best[u] + w);
        ans = Math.max(ans, best[v]);
    }
    return ans;
};

```

Time $O(n + m)$ **Space** $O(n)$

Verified on the same DAG: **11**, the route $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ with $2 + 3 + 6 = 11$.

Answer: 11

Example 15 [Wipro · pattern]

Detect a money loop. Directed graph of conversion costs, some negative. Is there a cycle whose total weight is negative? Edge cases: the cycle may sit in a part of the graph the source cannot reach.

SOLUTION

Bellman-Ford's extra pass answers it — but only for cycles reachable from the source. To cover the **whole** graph, start with $d[i] = 0$ for every i instead of INF . That is the same as adding an invisible node with a free edge to everyone.

Whole-graph negative cycle test

JS

```

const anyNegativeCycle = (n, E) => {
    const d = new Array(n).fill(0);           // start everyone at 0
    for (let i = 0; i < n - 1; i++)
        for (const [u, v, w] of E)
            if (d[u] + w < d[v]) d[v] = d[u] + w;
    for (const [u, v, w] of E)
        if (d[u] + w < d[v]) return true;
    return false;
};

```

Time $O(nm)$ **Space** $O(n)$

Verified: $0 \rightarrow 1$ (1), $1 \rightarrow 2$ (-1), $2 \rightarrow 1$ (-1), $0 \rightarrow 3$ (2) gives true; the acyclic $0 \rightarrow 1$ (4), $0 \rightarrow 2$ (5), $2 \rightarrow 1$ (-3), $1 \rightarrow 3$ (3) gives false; an empty edge list gives false; and a negative loop $2 \rightarrow 3 \rightarrow 2$ that node 0 cannot reach still gives true, which is the whole point of seeding everyone at 0.

Answer: true when the extra pass still improves something

Example 16 [Accenture · pattern]

The MST edges, not just the total. Return which cables Kruskal picked.

SOLUTION

Kruskal that reports its choices

JS

```

const kruskalEdges = (n, E) => {
    const e = [...E].sort((a, b) => a[2] - b[2]);
    const d = new DSU(n);
    let total = 0;
    const take = [];
    for (const edge of e)

```



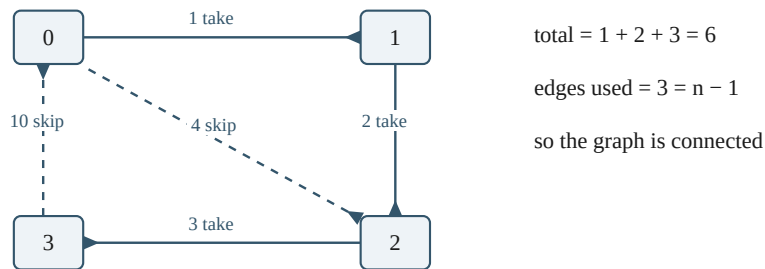
```

    if (d.union(edge[0], edge[1])) { total += edge[2]; take.push(edge); }
    return take.length === n - 1 ? [total, take] : [-1, []];
};

```

Time $O(m \log m)$ **Space** $O(n + m)$

Verified on 0-1:1, 1-2:2, 0-2:4, 2-3:3, 3-0:10: weight 6 with edges (0-1,1) (1-2,2) (2-3,3). The edge 0-2 of weight 4 is skipped because 0 and 2 are already together by then, and 3-0 of weight 10 is skipped for the same reason.



Kruskal takes 1, 2, 3 and rejects 4 and 10.

Answer: weight 6, edges 0-1, 1-2, 2-3

Tier 2 — weighted graphs with a business story

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Agoda · pattern]

Cheapest flight with at most K stops. Directed flights with prices. Find the cheapest trip from `src` to `dst` that uses at most K intermediate stops, or -1 .

Constraints: $n \leq 100$, flights $\leq 10^4$, $K \leq n$. Edge cases: $K = 0$ (direct flights only); `src == dst` (answer 0).

SOLUTION

Approach 1 — Try every route with DFS (exponential)

Approach 2 — Dijkstra by price (WRONG — a trap)

Plain Dijkstra finds the cheapest route ignoring the stop limit. It may settle a node at a cheap price that used too many hops, and then a more expensive but legal route through that node is never explored. Say this out loud in an interview: it shows you understand **why** the constraint changes the algorithm.

Approach 3 — Bellman-Ford with exactly $K+1$ rounds ($O(Km)$ — optimal)

“At most K stops” means “at most $K + 1$ flights”. Bellman-Ford round i finds the best route using at most i edges. So run exactly $K + 1$ rounds and stop.

The one subtle part: each round must read the **previous** round’s array, not the array it is writing. Otherwise one round can chain two flights together.

Layered Bellman-Ford

JS

```
const cheapestWithStops = (n, flights, src, dst, K) => {
  let d = new Array(n).fill(Infinity);
  d[src] = 0;
  for (let round = 0; round <= K; round++) {
    const nd = [...d]; // copy: this round adds ONE hop
    for (const [u, v, w] of flights)
      if (d[u] + w < nd[v]) nd[v] = d[u] + w;
    d = nd;
  }
  return d[dst] === Infinity ? -1 : d[dst];
};
```

Time $O(Km)$ **Space** $O(n)$

Verified on 0->1 (100), 1->2 (100), 0->2 (500):

Real output

TEXT

```
K = 0  -> 500    (only the direct flight is legal)
K = 1  -> 200    (0 -> 1 -> 2 uses one stop)
K = 5  -> 200    (more stops allowed, but nothing cheaper exists)
unreachable -> -1
src == dst  -> 0
```

TRAP

Drop the `const nd = [...d];` copy and `K = 0` returns 200 instead of 500 — verified by running the broken version — because the same round relaxes 0->1 and then immediately uses the new `d[1]` for 1->2. The copy is the entire point of the algorithm.

`[...d]` is the right copy **here** because `d` holds numbers. If each entry were itself an array, `[...d]` would share every row and the trick would silently stop working; that case needs `d.map(row => [...row])`.

The idea that unlocked it: Bellman-Ford already counts edges. Turning a limit on stops into a limit on rounds costs one line.

Answer: 200 when $K = 1$

Example 18 [Grab · pattern]

Signal delay. A message starts at one server and travels along one-way links with known delays. When does the last server receive it? Return -1 if some server never does. Edge cases: $n = 1$ (answer 0); an isolated server (-1).

SOLUTION

Dijkstra from the source, then take the maximum of the distance array. If any entry is still INF, return -1 .

Dijkstra, then take the max

JS

```
const networkDelay = (n, g, src) => {
  const d = new Array(n).fill(Infinity);
  const pq = new MinHeap((a, b) => a[0] - b[0]);
  d[src] = 0; pq.push([0, src]);
  while (pq.size) {
    const [du, u] = pq.pop();
    if (du > d[u]) continue;
    for (const [v, w] of g[u])
      if (du + w < d[v]) { d[v] = du + w; pq.push([d[v], v]); }
  }
  return d[n-1] === Infinity ? -1 : d[n-1];
};
```

```

}
const mx = Math.max(...d);
return mx === Infinity ? -1 : mx;           // one Infinity poisons the max:
exactly right
};

```

Time $O((n + m) \log n)$ **Space** $O(n + m)$

Verified on 0→1 (1), 0→2 (4), 1→2 (2), 2→3 (1): answer 4 (distances 0 1 3 4). One unreachable node gives -1. $n = 1$ gives 0.

TRAP

`Math.max(...d)` spreads the whole array onto the call stack as arguments. Verified on this Node build: 10^5 entries are fine, 2×10^5 throws `RangeError: Maximum call stack size exceeded`. For a big array write `d.reduce((a, b) => a > b ? a : b, 0)` instead — same answer, no stack.

SHORTCUT

“When does **everyone** get it” is always max over the Dijkstra array. “When does the **first** one get it” is min. The graph work is identical.

Answer: 4

Example 19 [Sea / Shopee · pattern]

Doors and free corridors. A grid where stepping into a 0 cell is free and stepping into a 1 cell costs 1 (you open a door). Find the cheapest route.

Constraints: grid up to 1000×1000 , so 10^6 nodes. Dijkstra’s log factor is avoidable. Edge cases: every cell free (answer 0).

SOLUTION

When the only weights are 0 and 1, you do not need a heap. Use a **deque**: push a 0-weight neighbour to the **front**, a 1-weight neighbour to the **back**. The deque then holds at most two distinct distance values at any time, in order — exactly what a heap would give you, in $O(1)$ per push.

The toolkit Deque is a head-index **queue**: push at the back, shift from the front, pop from the back. It has no push-front, and 0-1 BFS is the one algorithm in this book that needs one. The eight-line two-stack deque below supplies it in amortised $O(1)$ — unshift on a plain array would be $O(n)$ per call and would turn this $O(n + m)$ algorithm into $O(nm)$.

0-1 BFS

JS

```

class Deque01 {
  constructor() { this.f = []; this.b = []; }    // f is reversed: f[last] is the
  front
  get size() { return this.f.length + this.b.length; }
  pushFront(v) { this.f.push(v); }
  pushBack(v) { this.b.push(v); }
  popFront() {
    if (!this.f.length) while (this.b.length) this.f.push(this.b.pop());
    return this.f.pop();
  }
}

```

```

const zeroOneBFS = (n, g, src) => {
  const d = new Array(n).fill(Infinity);
  d[src] = 0;
  const dq = new Deque01();
  dq.pushBack(src);
  while (dq.size) {
    const u = dq.popFront();
    for (const [v, w] of g[u])
      if (d[u] + w < d[v]) {
        d[v] = d[u] + w;
        if (w === 0) dq.pushFront(v);           // free step -> jump the queue
        else        dq.pushBack(v);
      }
  }
  return d;
};

```

Time $O(n + m)$ **Space** $O(n + m)$

No log factor. At 10^6 cells that matters.

Verified on 0->1 (0), 0->3 (1), 1->2 (1), 3->2 (0), 2->4 (1): result 0 0 1 1 2. An all-zero chain gives 0 0 0, an unreachable node stays Infinity, and $n = 1$ gives 0. Cross-checked against the heap Dijkstra on 300 random 0/1 graphs: zero mismatches.

The idea that unlocked it: the heap in Dijkstra exists only to keep the frontier sorted. With two possible weights the frontier is sorted by **which end you push to**.

Answer: 0 0 1 1 2

Example 20 [GIC · pattern]

Widest pipe. Links have capacities. The capacity of a route is its **smallest** link. Find the route from `src` to `dst` with the largest capacity. Edge cases: no route (-1); a single link.

SOLUTION

This is Dijkstra with two swaps:

- min over a path becomes `min(current, edge)` instead of `+`.
- “smallest distance wins” becomes “largest capacity wins”, so use a **max**-heap.

Widest (max-bottleneck) path

JS

```

const widestPath = (n, g, src, dst) => {
  const cap = new Array(n).fill(-1);
  cap[src] = Infinity;
  const pq = new MinHeap((a, b) => b[0] - a[0]); // MAX-heap: flip the comparator
  pq.push([cap[src], src]);
  while (pq.size) {
    const [c, u] = pq.pop();
    if (c < cap[u]) continue;
    if (u === dst) return c;
    for (const [v, w] of g[u]) {
      const nc = Math.min(c, w); // bottleneck, not sum
      if (nc > cap[v]) { cap[v] = nc; pq.push([nc, v]); }
    }
  }
};

```

```
    return -1;
};
```

Time $O((n + m) \log n)$ **Space** $O(n + m)$

Verified on 0-1:3, 1-3:4, 0-2:7, 2-3:2: answer 3. Route 0-1-3 bottlenecks at 3; route 0-2-3 bottlenecks at 2. Unreachable gives -1; a single link 0-1:9 gives 9.

SHORTCUT

There is no max-heap in JavaScript either — but you do not need one. The toolkit MinHeap takes a comparator, so `new MinHeap((a, b) => b[0] - a[0])` is a max-heap. This is the standard answer when an interviewer asks “JavaScript has no priority queue, what do you do?”

SHORTCUT

Dijkstra works for any “combine” rule that never **improves** as the path grows. Sum with non-negative weights qualifies. So does min. So does multiplying probabilities in $[0, 1]$ (with a max-heap). Recognise the shape and you get three problems for free.

Answer: 3

Example 21 [DBS · pattern]

How many terms to finish the degree? Courses with prerequisites. You may take **any** number of courses in one term, as long as every prerequisite is already done. Fewest terms? Return -1 if impossible. Edge cases: no prerequisites (1 term); a circular prerequisite (-1).

SOLUTION

Kahn’s algorithm, but peel a **whole level** at a time — exactly the ring trick from multi-source BFS.

Level-by-level Kahn

JS

```
const semesters = (n, g) => {
  const indeg = new Array(n).fill(0);
  for (let u = 0; u < n; u++) for (const v of g[u]) indeg[v]++;
  const q = new Deque(); // JS Toolkit appendix
  for (let i = 0; i < n; i++) if (indeg[i] === 0) q.push(i);
  let done = 0, terms = 0;
  while (q.size) {
    let sz = q.size; // one whole term
    terms++;
    while (sz-- > 0) {
      const u = q.shift();
      done++;
      for (const v of g[u]) if (--indeg[v] === 0) q.push(v);
    }
  }
  return done === n ? terms : -1;
};
```

Time $O(n + m)$ **Space** $O(n)$

Verified on 0->2, 1->2, 2->3, 3->4: 4 terms — take {0, 1}, then {2}, then {3}, then {4}. A 2-cycle gives -1. Five courses with no prerequisites give 1, and one course gives 1.

TRAP

Use the toolkit Deque, not a plain array with `shift()`. `Array.prototype.shift` moves every remaining element down one slot, so it is $O(n)$ — a queue of 10^5 nodes turns this $O(n + m)$ algorithm into $O(n^2)$. The toolkit Deque keeps a head index instead.

The answer equals the number of nodes on the **longest chain** of prerequisites. That is why this is also called the longest path in a DAG measured in nodes.

Answer: 4

Example 22 [SCB · pattern]

Count the cheapest itineraries. Weighted graph. How many **different** shortest routes from the source reach each node? Edge cases: an unreachable node (count 0); counts that grow past $2^{53} - 1$ — plain numbers stop being exact there, so use `BigInt` or the modulus the problem asks for.

SOLUTION

Dijkstra plus a `ways` array, exactly like the unweighted version in Chapter 15, but the comparison is on **distance**, not on “first seen”.

Dijkstra that counts

JS

```
const countShortest = (n, g, src) => {
  const d = new Array(n).fill(Infinity), ways = new Array(n).fill(0);
  const pq = new MinHeap((a, b) => a[0] - b[0]);
  d[src] = 0; ways[src] = 1; pq.push([0, src]);
  while (pq.size) {
    const [du, u] = pq.pop();
    if (du > d[u]) continue;
    for (const [v, w] of g[u]) {
      if (du + w < d[v]) { // strictly better
        d[v] = du + w;
        ways[v] = ways[u];
        pq.push([d[v], v]);
      } else if (du + w === d[v]) { // equally good
        ways[v] += ways[u];
      }
    }
  }
  return [d, ways];
};
```

Time $O((n + m) \log n)$ **Space** $O(n)$

Verified on 0->1 (1), 0->2 (1), 1->3 (1), 2->3 (1), 3->4 (1): distances 0 1 1 2 3, counts 1 1 1 2 2.

TRAP

The `if (du > d[u]) continue;` line must stay. Without it a stale heap entry re-adds its `ways` value and the counts double.

TRAP

`du + w === d[v]` uses `===`, which does **not** coerce. That is what you want. But watch the arithmetic: if the weights were fractions, `0.1 + 0.2 === 0.3` is `false` in JavaScript, and the “equally good” branch would never fire. Weighted-graph problems in interviews use integers; if yours does not, compare with a tolerance.

Answer: ways = 1 1 1 2 2

Example 23 [Razer · pattern]

Lay cable when some links already exist. Some offices are already connected (those links are free). Other links have a price. Cheapest total to connect everything. Edge cases: already fully connected (answer 0); impossible (−1).

SOLUTION

Feed the free links into DSU **first**, then run Kruskal on the priced ones. A free link costs nothing and can never be a bad choice.

MST with pre-merged groups

JS

```
const connectWithExisting = (n, built, opt) => {
  const d = new DSU(n);
  let comps = n;
  for (const [a, b] of built) if (d.union(a, b)) comps--;
  const sorted = [...opt].sort((x, y) => x[2] - y[2]);
  let total = 0;
  for (const [u, v, w] of sorted)
    if (d.union(u, v)) { total += w; comps--; }
  return comps === 1 ? total : -1;
};
```

Time $O(m \log m)$ **Space** $O(n)$

Verified: $n = 4$, built $\{0, 1\}$, options $\{1, 2, 5\}$, $\{0, 2, 7\}$, $\{2, 3, 2\} \rightarrow 7$ (take 2-3 for 2, then 1-2 for 5; 0-2 is rejected). With only $\{0, 1, 5\}$ as an option $\rightarrow -1$. Already fully connected gives 0, and $n = 1$ gives 0.

Answer: 7

Tier 3 — the hard ones

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Google · pattern]

The gentlest climb. A grid of heights. Moving between neighbouring cells costs the **height difference**. The cost of a route is its **largest** single step. Find the smallest possible largest step from the top-left to the bottom-right.

Constraints: grid up to 500×500 . Edge cases: a 1×1 grid (answer 0); a flat grid (answer 0).

SOLUTION

Approach 1 — Try every route (exponential)

Approach 2 — Binary search on the answer plus BFS ($O(RC \log H)$ — good)

Guess a limit L . Delete every step bigger than L and ask BFS whether the corner is still reachable. If yes, try a smaller L ; if no, a bigger one. Since the feasibility is **monotone** in L , binary search works.

Approach 3 — Dijkstra where 'distance' means 'worst step so far' ($O(RC \log(RC))$ — optimal and shorter)

This is the same swap as the widest-path problem: replace + with max.

Minimax-path Dijkstra on a grid

JS

```
const minEffort = (h) => {
  const R = h.length, C = h[0].length;
  const best = Array.from({ length: R }, () => new Array(C).fill(Infinity));
  const pq = new MinHeap((a, b) => a[0] - b[0]); // [worst step so far, row, col]
  best[0][0] = 0;
  pq.push([0, 0, 0]);
  const DR = [1, -1, 0, 0], DC = [0, 0, 1, -1];
  while (pq.size) {
    const [e, r, c] = pq.pop();
    if (e > best[r][c]) continue;
    if (r === R - 1 && c === C - 1) return e;
    for (let k = 0; k < 4; k++) {
      const nr = r + DR[k], nc = c + DC[k];
      if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
      const ne = Math.max(e, Math.abs(h[nr][nc] - h[r][c])); // max, not +
      if (ne < best[nr][nc]) { best[nr][nc] = ne; pq.push([ne, nr, nc]); }
    }
  }
  return -1;
};
```

Time $O(RC \log(RC))$ **Space** $O(RC)$

Verified:

Real runs

TEXT

```
1 3 5
2 8 3   -> 1   (route 1,2,3,4,5 down the left then across the bottom)
3 4 5

1 10 1
1 10 1  -> 0   (go straight down the left column, then across)
1  1 1

[[7]]   -> 0

1 2 3   -> 1   (single row)
```

Follow-up: “the same, but minimise the sum of steps instead of the maximum.” Change `max(e, ...)` back to `e + ...`. It is plain Dijkstra again. Point out that the **only** difference between the two problems is one operator.

Follow-up: “what if the grid is $10^5 \times 10^5$ but mostly flat?” Then the graph does not fit in memory. Say so, and describe compressing runs of equal height into single nodes before running the same algorithm.

Answer: 1

Example 25 [Amazon · pattern]

One free ride. Directed graph with prices. You hold one coupon that makes **one** edge free. Cheapest trip from `src` to `dst`?

Constraints: $n \leq 10^5$. Target $O((n + m) \log n)$. Edge cases: the coupon is worth nothing (the cheapest route is already free); `src == dst` (answer 0).

SOLUTION

Approach 1 — Try the coupon on each edge, re-run Dijkstra ($O(m(n+m) \log n)$ — too slow)

Approach 2 — Double the graph: state = (node, coupon used?) ($O((n+m) \log n)$ — optimal)

STATE EXPANSION — THE KEY TRICK FOR HARD DIJKSTRA

When a rule limits what you may do, add the rule to the node. Here the node becomes a **pair**: (city, have I spent the coupon yet).

That doubles the graph to $2n$ states. From state $(u, 0)$ an edge $u \rightarrow v$ of weight w gives two moves:

- pay it: go to $(v, 0)$ at cost w ;
- burn the coupon: go to $(v, 1)$ at cost 0.

From $(u, 1)$ you can only pay. The answer is $\min(d[(dst, 0)], d[(dst, 1)])$.

Dijkstra over doubled states

JS

```
const withOneFreeEdge = (n, g, src, dst) => {
  // d[u] = [best with the coupon still in hand, best with it already spent]
  const d = Array.from({ length: n }, () => [Infinity, Infinity]);
  const pq = new MinHeap((a, b) => a[0] - b[0]); // [dist, node, used]
  d[src][0] = 0;
  pq.push([0, src, 0]);
  while (pq.size) {
    const [du, u, used] = pq.pop();
    if (du > d[u][used]) continue;
    for (const [v, w] of g[u]) {
      if (du + w < d[v][used]) { // pay normally
        d[v][used] = du + w;
        pq.push([du + w, v, used]);
      }
      if (used === 0 && du < d[v][1]) { // spend the coupon
        d[v][1] = du;
        pq.push([du, v, 1]);
      }
    }
  }
  const b = Math.min(d[dst][0], d[dst][1]);
  return b === Infinity ? -1 : b;
};
```

TRAP

`Array.from({ length: n }, () => [Infinity, Infinity])` builds n **separate** pairs. `new Array(n).fill([Infinity, Infinity])` builds one pair shared by all n nodes, and the first relaxation corrupts every node at once. This is the shallow-copy trap in its most expensive form: the code runs, produces plausible numbers, and is wrong.

Time $O((n+m) \log n)$ **Space** $O(n)$

The graph doubled, so every cost doubled. Same big-O.

Verified on 0->1 (5), 1->2 (5), 0->2 (12), 2->3 (1), 3->4 (20): plain Dijkstra gives $d[4] = 31$; with one coupon the answer is **11** — pay 5 + 5 + 1 = 11 to reach node 3, then ride 3->4 free. `src == dst` gives 0, an unreachable target gives -1, and a graph whose cheapest route already costs 0 gives 0. Cross-checked against

an exhaustive brute force — try the coupon on every edge in turn, Bellman-Ford each time — on 400 random graphs: zero mismatches.

Follow-up: “now you have K coupons.” Make the state (node, coupons used) with $K + 1$ layers. Time becomes $O(K(n + m) \log(Kn))$, memory $O(Kn)$. Same code, one array dimension bigger.

Follow-up: “the coupon halves an edge instead of zeroing it.” Identical structure. Replace the “cost 0” transition with “cost $e.w / 2$ ”. Watch out for odd weights and rounding — ask the interviewer which way it rounds.

Answer: 11

Example 26 [Microsoft · pattern]

Which cables can never be replaced? Given a weighted undirected graph, split the edges into:

- **critical** — in every minimum spanning tree;
- **pseudo-critical** — in at least one MST, but not all.

Constraints: $n \leq 100$, $m \leq 200$. Edge cases: all weights distinct (then the MST is unique and every MST edge is critical); several equal weights.

SOLUTION

TWO TESTS, ONE HELPER

Let base = the weight of the minimum spanning tree.

- Edge e is **critical** if banning e makes the MST heavier (or breaks connectivity).
- Otherwise, edge e is **pseudo-critical** if forcing e in first still reaches base.
- If neither, the edge is in no MST at all.

MST with one edge banned or forced

JS

```
// E is sorted by weight; each entry is [u, v, w, original index].
// `ban` is an ORIGINAL index; `force` is a SORTED slot.
const mstWeight = (n, E, ban, force) => {
  const d = new DSU(n);
  let tot = 0, used = 0;
  if (force !== -1) {
    d.union(E[force][0], E[force][1]);
    tot += E[force][2];
    used++;
  }
  for (const [u, v, w, id] of E) {
    if (id === ban || id === force) continue;
    if (d.union(u, v)) { tot += w; used++; }
  }
  return used === n - 1 ? tot : Infinity; // Infinity means "cannot connect"
};
```

The two classifications

JS

```
const criticalEdges = (n, raw) => {
  const E = raw.map([u, v, w], i) => [u, v, w, i])
    .sort((a, b) => a[2] - b[2]);
  const posOf = new Map(E.map((e, i) => [e[3], i])); // original index -> sorted slot
  const base = mstWeight(n, E, -1, -1);
  const crit = [], pseudo = [];
```

```

for (let i = 0; i < raw.length; i++) {
  if (mstWeight(n, E, i, -1) > base) crit.push(i);
  else if (mstWeight(n, E, -1, posOf.get(i)) === base) pseudo.push(i);
}
return [crit, pseudo];
};

```

Time $O(m^2\alpha(n))$ **Space** $O(m)$
 $m = 200$ gives 40000 DSU runs. Instant.

TRAP

The array is **sorted**, but the answer must be reported in **original** indices. That is why every entry carries its original index in slot 3 and why `posOf` — a Map from original index to sorted slot — exists at all. `ban` is compared against slot 3, so it takes an original index; `force` indexes `E` directly, so it takes a sorted slot. Mixing the two is the classic bug here, and it only shows up when the input happens to arrive unsorted.

Use a Map rather than a plain object for `posOf`. Object keys are coerced to strings, so `obj[0]` and `obj["0"]` are the same slot — harmless for small integers, a real bug the moment your keys are anything else. Map keeps the key's type.

Verified on the square $0-1:1, 1-2:1, 2-3:1, 3-0:1, 0-2:2$: critical = none, pseudo-critical = edges 0, 1, 2, 3. Any three of the four weight-1 edges form an MST of weight 3, so no single one is required; the weight-2 diagonal is in no MST at all. On the chain $0-1:1, 1-2:2$: both edges are critical (removing either disconnects the graph). Two parallel edges of equal weight between the same pair: neither is critical, both are pseudo-critical. $n = 1$ with no edges gives two empty lists.

Follow-up: “is the MST unique?” Yes exactly when every MST edge is critical, that is when the pseudo-critical list is empty.

Answer: critical: none. pseudo-critical: 0, 1, 2, 3

Example 27 [Goldman Sachs · pattern]

The quietest branch. Cities joined by two-way roads of known length, plus a distance limit. For each city, count how many **other** cities lie within limit (using shortest road distance). Return the city with the smallest count; on a tie, return the largest city number.

Constraints: $n \leq 100$. Edge cases: `limit = 0` (every count is 0, so return $n - 1$); a huge limit (every count is $n - 1$).

SOLUTION

$n \leq 100$, and you need **all** pairs. Floyd-Warshall is the right shape: $100^3 = 10^6$.

Floyd, then count and tie-break

JS

```

const bestCity = (n, E, limit) => {
  const d = Array.from({ length: n }, () => new Array(n).fill(Infinity));
  for (let i = 0; i < n; i++) d[i][i] = 0;
  for (const [u, v, w] of E) {
    d[u][v] = Math.min(d[u][v], w);
    d[v][u] = d[u][v];
  }
  for (let k = 0; k < n; k++)
    for (let i = 0; i < n; i++) {
      if (d[i][k] === Infinity) continue;
      for (let j = 0; j < n; j++)

```

```

        d[i][j] = Math.min(d[i][j], d[i][k] + d[k][j]);
    }
    let best = -1, bestCount = Infinity;
    for (let i = 0; i < n; i++) {
        let c = 0;
        for (let j = 0; j < n; j++) if (i !== j && d[i][j] <= limit) c++;
        if (c < bestCount || (c === bestCount && i > best)) { bestCount = c; best = i; }
    }
    return best;
};

```

Time $O(n^3)$ **Space** $O(n^2)$

Verified on roads 0-1:2, 0-3:1, 1-3:3, 1-2:4, 2-3:1 with `limit = 3`: answer 2. City 1 reaches {0 at 2, 3 at 3} = 2 cities; city 2 reaches {3 at 1, 0 at 2} = 2 cities; cities 0 and 3 each reach 3. Tie between 1 and 2, so the larger number 2 wins. With `limit = 100` the answer is 3 (everyone ties at $n - 1$), and with `limit = 0` the answer is 3 as well (every count is 0, so the largest index wins). $n = 1$ gives 0.

TRAP

Read the tie-break rule carefully. “Largest index on a tie” is written as `c == bestCount && i > best`. If you loop upward and use plain `<`, you silently keep the **smallest** index and fail half the tests.

Follow-up: “ n is 10^5 now.” Floyd is dead (10^{15}). Run Dijkstra from each node instead: $O(n(n + m) \log n)$ — still far too slow at 10^5 . The honest answer is that the problem must change: either only a few query cities matter, or the graph has special structure (a tree, a planar road map) you can exploit.

Answer: 2

Example 28 [D. E. Shaw · pattern]

The second-best route. Undirected weighted graph. Find the length of the second shortest route from `src` to `dst`. It must be **strictly** longer than the shortest. It may repeat nodes.

Constraints: $n \leq 10^5$. Edge cases: no second route (-1).

SOLUTION

Keep **two** best values per node instead of one: `d1[v]` (best) and `d2[v]` (best that is strictly worse than `d1[v]`). Run Dijkstra, but do not stop at the first time a node is settled — let a node be popped up to twice.

Two-label Dijkstra

JS

```

const secondShortest = (n, g, s, t) => {
    const d1 = new Array(n).fill(Infinity), d2 = new Array(n).fill(Infinity);
    const pq = new MinHeap((a, b) => a[0] - b[0]);
    d1[s] = 0; pq.push([0, s]);
    while (pq.size) {
        const [du, u] = pq.pop();
        if (du > d2[u]) continue; // worse than both labels
        for (const [v, w] of g[u]) {
            const nd = du + w;
            if (nd < d1[v]) { // new best: old best slides
                d2[v] = d1[v];
                d1[v] = nd;
                pq.push([nd, v]);
            }
        }
    }
    return d1[t] === Infinity ? -1 : d1[t];
}

```

```

    } else if (nd > d1[v] && nd < d2[v]) {
        d2[v] = nd;
        pq.push([nd, v]);
    }
}
}
return d2[t] === Infinity ? -1 : d2[t];
};

```

Time $O((n + m) \log n)$ **Space** $O(n)$

Each node enters the heap at most twice, so the bound holds.

Verified on 0-1:1, 1-2:1, 0-2:3, 2-3:1, 3-4:1, 2-4:5:

- 0 → 4: shortest is 0, 1, 2, 3, 4 = 4; second is 0, 2, 3, 4 = 5. Code gives 5.
- 0 → 2: shortest 2, second 3. Code gives 3.
- unreachable gives -1.
- the single edge 0-1:4: the answer is **12**, walking $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$. Routes may repeat nodes, so “no second route” is rarer than it looks.

TRAP

The test must be $nd > d1[v]$, not $nd !== d1[v]$. With $>=$ or $!==$ you can let an equal-length route sneak in as “second”, which the problem forbids.

Follow-up: “now the K-th shortest.” Same idea with K labels per node, or the standard trick: keep a counter per node and allow each node to be popped K times. Time $O(K(n + m) \log n)$, and you should mention the memory cost of Kn before writing a line.

Answer: 5

Example 29 [Uber · pattern]

Round trip. Directed roads (one-way streets). Find the cheapest trip that goes from the depot to a customer **and back**. Edge cases: one direction impossible (−1).

SOLUTION

The naive move is to run Dijkstra from the customer as well — but there can be many customers, and that is one Dijkstra each.

Better: run Dijkstra **twice from the depot**. Once on the graph, once on the graph with every edge reversed. In the reversed graph, the distance from the depot to node t is exactly the distance from t back to the depot in the real graph.

Forward plus reverse Dijkstra

JS

```

const roundTrip = (n, g, s, t) => {
  const run = (gg, st) => {
    const d = new Array(n).fill(Infinity);
    const pq = new MinHeap((a, b) => a[0] - b[0]);
    d[st] = 0; pq.push([0, st]);
    while (pq.size) {
      const [du, u] = pq.pop();
      if (du > d[u]) continue;
      for (const [v, w] of gg[u])
        if (du + w < d[v]) { d[v] = du + w; pq.push([d[v], v]); }
    }
    return d;
  }

```

```

};
const rg = Array.from({ length: n }, () => []);
for (let u = 0; u < n; u++)
  for (const [v, w] of g[u]) rg[v].push([u, w]); // reverse every edge
const out = run(g, s), back = run(rg, s);
if (out[t] === Infinity || back[t] === Infinity) return -1;
return out[t] + back[t];
};

```

Time $O((n + m) \log n)$ **Space** $O(n + m)$

Two Dijkstras total, no matter how many customers you ask about.

Verified on 0->1 (3), 1->2 (4), 2->0 (2), 0->2 (10): round trip 0 <-> 2 costs **9** (out 0, 1, 2 = 7; back 2 → 0 = 2; the direct 0->2 at 10 is worse). Adding a one-way 0->3 (1) to a dead end gives -1, because back[3] stays Infinity. s == t gives 0.

Follow-up: “the answer for every customer at once.” You already have it: out[t] + back[t] for every t, from the same two runs. Say this — it is the whole point of reversing instead of re-running.

Follow-up: “the driver must pick up a parcel at one of a set S of depots on the way.” Add a virtual source joined to every depot in S with weight 0 and run one Dijkstra from it.

Answer: 9

Example 30 [Adobe · pattern]

Project deadline. n tasks, each with a duration, plus “A must end before B starts” arrows. Tasks with no unfinished prerequisite may run in parallel. When does the whole project finish? Return -1 if the arrows contain a loop. Edge cases: no arrows (answer = the longest single duration).

SOLUTION

This is the **critical path**. Relax with max in topological order, carrying finish times.

Earliest finish time of the whole project

JS

```

const earliestFinish = (n, g, dur) => {
  const indeg = new Array(n).fill(0);
  for (let u = 0; u < n; u++) for (const v of g[u]) indeg[v]++;
  const q = new Deque();
  const fin = new Array(n).fill(0);
  let done = 0, ans = 0;
  for (let i = 0; i < n; i++)
    if (indeg[i] === 0) { q.push(i); fin[i] = dur[i]; }
  while (q.size) {
    const u = q.shift();
    done++;
    ans = Math.max(ans, fin[u]);
    for (const v of g[u]) {
      fin[v] = Math.max(fin[v], fin[u] + dur[v]); // wait for the slowest input
      if (--indeg[v] === 0) q.push(v);
    }
  }
  return done === n ? ans : -1;
};

```

Time $O(n + m)$ **Space** $O(n)$

Verified on durations {3, 2, 4, 1} and arrows 0->1, 0->2, 1->3, 2->3: answer **8**. Task 0 ends at 3; task 1 at 5; task 2 at 7; task 3 must wait for both, so it starts at 7 and ends at 8. A 2-cycle gives -1.

Durations {5, 2, 9} with no arrows give 9 — the longest single task. One task gives its own duration.

TRAP

$\text{fin}[v] = \text{Math.max}(\text{fin}[v], \text{fin}[u] + \text{dur}[v])$ must run for **every** incoming arrow, not only the one that pushes the in-degree to zero. Task 3 above would finish at 6 instead of 8 if you only used the last arrow.

Follow-up: “which tasks can be delayed without moving the deadline?” Compute the latest allowed finish time by running the same relaxation backwards from the deadline on the reversed graph. Any task whose earliest and latest times differ has **slack**; tasks with zero slack form the critical path.

Answer: 8

Dry run — Dijkstra, pop by pop

Undirected weighted graph, 6 nodes:

edge	0-1	0-2	2-1	1-3	2-3	3-4	2-4	4-5	3-5
weight	4	1	2	5	8	3	10	2	6

Source is node 0. We print every pop, every relaxation, and the whole dist array.

Actual printed trace

TEXT

```

step 1: pop (0,0) settle node 0
  relax 0->1 w=4 : inf -> 4  push
  relax 0->2 w=1 : inf -> 1  push
  dist = [0,4,1,inf,inf,inf]
step 2: pop (1,2) settle node 2
  edge 2->0 w=1 gives 2 , not better than 0
  relax 2->1 w=2 : 4 -> 3  push
  relax 2->3 w=8 : inf -> 9  push
  relax 2->4 w=10 : inf -> 11 push
  dist = [0,3,1,9,11,inf]
step 3: pop (3,1) settle node 1
  edge 1->0 w=4 gives 7 , not better than 0
  edge 1->2 w=2 gives 5 , not better than 1
  relax 1->3 w=5 : 9 -> 8  push
  dist = [0,3,1,8,11,inf]
step 4: pop (4,1) STALE, skip
step 5: pop (8,3) settle node 3
  edge 3->1 w=5 gives 13 , not better than 3
  edge 3->2 w=8 gives 16 , not better than 1
  edge 3->4 w=3 gives 11 , not better than 11
  relax 3->5 w=6 : inf -> 14  push
  dist = [0,3,1,8,11,14]
step 6: pop (9,3) STALE, skip
step 7: pop (11,4) settle node 4
  edge 4->3 w=3 gives 14 , not better than 8
  edge 4->2 w=10 gives 21 , not better than 1
  relax 4->5 w=2 : 14 -> 13  push
  dist = [0,3,1,8,11,13]
step 8: pop (13,5) settle node 5
  edge 5->4 w=2 gives 15 , not better than 11

```

```

edge 5->3 w=6 gives 19 , not better than 8
dist = [0,3,1,8,11,13]
step 9: pop (14,5) STALE, skip
final: 0 3 1 8 11 13

```

The same thing as a table. “Heap after” lists the pairs (distance, node) still waiting.

step	pop	dist array after	heap after
1	(0,0)	0 4 1 - - -	(1,2) (4,1)
2	(1,2)	0 3 1 9 11 -	(3,1) (4,1) (9,3) (11,4)
3	(3,1)	0 3 1 8 11 -	(4,1) (8,3) (9,3) (11,4)
4	(4,1)	stale — 4 > dist[1] = 3, skip	(8,3) (9,3) (11,4)
5	(8,3)	0 3 1 8 11 14	(9,3) (11,4) (14,5)
6	(9,3)	stale — 9 > dist[3] = 8, skip	(11,4) (14,5)
7	(11,4)	0 3 1 8 11 13	(13,5) (14,5)
8	(13,5)	no change	(14,5)
9	(14,5)	stale, skip	empty

THREE THINGS THIS TRACE TEACHES

1. **Stale entries are normal.** Node 1 went into the heap twice (at 4 and at 3) and node 3 twice (at 9 and 8) and node 5 twice (at 14 and 13). The one-line guard `if (du > d[u]) continue;` throws the old copies away. You never need a decrease-key operation.
2. **A node’s value can drop before it is popped, never after.** `dist[1]` fell from 4 to 3 at step 2 — before node 1 was settled at step 3. After that it never moved.
3. **The heap can hold more than n items.** Up to m of them. That is why the bound is $O((n + m) \log n)$ and not $O(n \log n)$.

Check the final answer by hand for node 5: $0 \rightarrow 2(1) \rightarrow 1(2) \rightarrow 3(5) \rightarrow 4(3) \rightarrow 5(2)$ gives $1 + 2 + 5 + 3 + 2 = 13$. Any other route is longer. The trace is right.

Python, for the two signature problems

JavaScript is the primary language of this book; Python appears here only for the two problems every weighted-graph round comes back to. Both versions below were run.

Dijkstra and Kahn in Python

PYTHON

```

import heapq
from collections import deque

def dijkstra(n, g, src):
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        du, u = heapq.heappop(pq)
        if du > dist[u]:
            continue
        # g[u] = list of (v, w)

```



```

        for v, w in g[u]:
            if du + w < dist[v]:
                dist[v] = du + w
                heapq.heappush(pq, (dist[v], v))
    return dist

def topo_kahn(n, g):
    indeg = [0] * n
    for u in range(n):
        for v in g[u]:
            indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else []

```

Real output: `dijkstra` on the Warm-up 3 graph gives `[0, 3, 1, 4, inf]`; `dijkstra(1, [[]], 0)` gives `[0]`; `topo_kahn` on `0->2, 1->2, 2->3` with $n = 5$ gives `[0, 1, 4, 2, 3]`; on a 2-cycle it gives `[]`.

SHORTCUT

Python's `heapq` is a min-heap and has no `decrease-key` — and it is the one thing Python gives you that JavaScript does not give you at all. The lazy `if du > dist[u]: continue` pattern is the standard way in both languages; in JavaScript the same loop runs on the toolkit `MinHeap`.

Note that `heapq` orders **tuples** element by element for free, so Python can push `(dist, node)` with no comparator. The JavaScript `MinHeap` compares arrays only if you tell it how: `new MinHeap((a, b) => a[0] - b[0])`.

Practice

PRACTICE · WARM-UP target: 20 min

1. $d[u] = 10$, edge weight 3, $d[v] = 12$. What is $d[v]$ after relaxing $u \rightarrow v$?
2. A graph has 5 nodes. How many Bellman-Ford rounds are enough (with no negative loop)?
3. Which algorithm: one source, all weights equal to 7?
4. Which algorithm: every pair of nodes, $n = 300$, weights may be negative but no negative loop?

PRACTICE · TIER 1 target: 60 min

5. Given n and a weighted undirected edge list, return the total weight of the minimum spanning tree, or -1 if the graph is not connected.
6. Given a list of points on a plane, return the cheapest total cost to link them all, where the cost of a link is the Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Points up to 1000.
7. Given a DAG and a list you were handed, decide whether that list is a valid topological order.
8. Grid of positive cell costs. You pay the cost of every cell you **enter**, plus the starting cell. Cheapest route from the top-left to the bottom-right, 4 directions.

PRACTICE · TIER 2 target: 70 min

9. Weighted graph, a source, a target, and a node x you **must** pass through. Cheapest route.
10. Given a source and a limit T , count the nodes whose shortest distance is at most T .
11. Tasks with durations and dependencies. Report the earliest finish time of the whole project.
12. Is the shortest route from s to t unique?

PRACTICE · TIER 3 target: 60 min

13. Directed graph with possibly negative weights. List the nodes whose shortest distance is undefined because a negative cycle can reach them.
14. Undirected graph. For a given s and t , find the smallest possible **largest** edge on any route between them (the minimax edge).
15. Given an MST and one extra edge, decide in $O(n)$ whether the MST changes.

ANSWER KEY

1. $10 + 3 = 13$, and $13 < 12$ is false. $d[v]$ stays **12**.
2. $n - 1 = 4$ rounds.
3. All weights equal means the cheapest route is the one with the fewest edges: plain **BFS**, then multiply the hop count by 7.
4. **Floyd-Warshall**. $300^3 = 2.7 \times 10^7$, and it handles negative weights.
5. Kruskal, exactly as in Example 11. Verified: $0-1:1$, $1-2:2$, $0-2:4$, $2-3:3$, $3-0:10$ gives **6**; a disconnected graph gives -1 ; $n = 1$ gives 0.
6. Every pair of points is a possible link, so $m = \frac{n^2}{2}$ — the graph is **dense** and the $O(n^2)$ Prim beats Kruskal (which would need to sort half a million edges).

Q6 — dense Prim, no heap

JS

```
const connectPoints = (p) => {
  const n = p.length;
  if (n <= 1) return 0;
  const best = new Array(n).fill(Infinity);
  const inMst = new Array(n).fill(false);
  best[0] = 0;
  let total = 0;
  for (let it = 0; it < n; it++) {
    let u = -1;
    for (let i = 0; i < n; i++)
      if (!inMst[i] && (u === -1 || best[i] < best[u])) u = i;
    inMst[u] = true;
    total += best[u];
    for (let v = 0; v < n; v++)
      if (!inMst[v]) {
        const d = Math.abs(p[u][0] - p[v][0]) + Math.abs(p[u][1] - p[v][1]);
        best[v] = Math.min(best[v], d);
      }
  }
  return total;
};
```

Verified on $(0,0)$ $(2,2)$ $(3,10)$ $(5,2)$ $(7,0)$: **20** (links $0-1$ costs 4, $1-3$ costs 3, $3-4$ costs 4, $1-2$ costs 9). One point gives 0; zero points gives 0; two identical points give 0; negative coordinates work ($(-5,-5)$ $(-5,-1)$ gives 4).

Time $O(n^2)$ **Space** $O(n)$

7. Record each node's position in the given list, then check that every arrow points forward. Also reject a list with wrong length or repeats.

Q7

JS

```
const validTopo = (n, g, order) => {
  if (order.length !== n) return false;
  const pos = new Array(n).fill(-1);
  for (let i = 0; i < n; i++) {
    const u = order[i];
    if (u < 0 || u >= n || pos[u] !== -1) return false;
    pos[u] = i;
  }
  for (let u = 0; u < n; u++)
    for (const v of g[u]) if (pos[u] > pos[v]) return false;
  return true;
};
```

Verified on $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$: $0 \ 1 \ 2 \ 3 \rightarrow \text{true}$; $0 \ 3 \ 1 \ 2 \rightarrow \text{false}$; a short list $0 \ 1 \ 2 \rightarrow \text{false}$; a list with a repeat $0 \ 1 \ 1 \ 3 \rightarrow \text{false}$; $n = 0$ with an empty list $\rightarrow \text{true}$.

Time $O(n + m)$ **Space** $O(n)$

8. Dijkstra on the grid, where entering a cell costs that cell's number.

Q8

JS

```
const gridCost = (c) => {
  const R = c.length, C = c[0].length;
  const d = Array.from({ length: R }, () => new Array(C).fill(Infinity));
  const pq = new MinHeap((a, b) => a[0] - b[0]); // [cost, row, col]
  d[0][0] = c[0][0];
  pq.push([d[0][0], 0, 0]);
  const DR = [1, -1, 0, 0], DC = [0, 0, 1, -1];
  while (pq.size) {
    const [du, r, cc] = pq.pop();
    if (du > d[r][cc]) continue;
    if (r === R - 1 && cc === C - 1) return du;
    for (let k = 0; k < 4; k++) {
      const nr = r + DR[k], nc = cc + DC[k];
      if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
      if (du + c[nr][nc] < d[nr][nc]) {
        d[nr][nc] = du + c[nr][nc];
        pq.push([d[nr][nc], nr, nc]);
      }
    }
  }
  return -1;
};
```

Verified on $[[1, 9, 9], [1, 1, 9], [9, 1, 1]]$: 5 (down the left, across the bottom). $[[7]]$ gives 7 — the starting cell still costs. A single row $[[1, 2, 3]]$ gives 6.

Time $O(RC \log(RC))$ **Space** $O(RC)$

9. Any route through x splits at x . So the answer is $\text{dist}(s, x) + \text{dist}(x, t)$. Run Dijkstra from s on the graph, and Dijkstra from x on the graph (undirected) or on the **reversed** graph (directed). Two runs, then one addition. Report -1 if either half is INF.

10. One Dijkstra from the source, then count entries $\leq T$. $O((n + m) \log n)$. `d.filter(x => x <= T).length` reads well; just remember that unreachable entries are `Infinity` and are correctly excluded by $\leq$.

11. Exactly Example 30. Verified: durations $\{3, 2, 4, 1\}$ with $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$ gives **8**; a cycle gives -1 .

12. Count shortest routes as in Example 22 and ask whether `ways[t] == 1`.

Q12

JS

```
const uniqueShortest = (n, g, s, t) => {
  const d = new Array(n).fill(Infinity), ways = new Array(n).fill(0);
  const pq = new MinHeap((a, b) => a[0] - b[0]);
  d[s] = 0; ways[s] = 1; pq.push([0, s]);
  while (pq.size) {
    const [du, u] = pq.pop();
    if (du > d[u]) continue;
    for (const [v, w] of g[u]) {
      if (du + w < d[v]) {
        d[v] = du + w; ways[v] = ways[u];
        pq.push([d[v], v]);
      } else if (du + w === d[v]) ways[v] += ways[u];
    }
  }
  return d[t] !== Infinity && ways[t] === 1;
};
```

Verified: $0 \rightarrow 1$ (1), $0 \rightarrow 2$ (1), $1 \rightarrow 3$ (1), $2 \rightarrow 3$ (1) gives false (two routes); the chain $0 \rightarrow 1 \rightarrow 2$ gives true; an unreachable target gives false.

13. Run $n - 1$ Bellman-Ford rounds. Then run n more sweeps that mark a node bad if it still improves, **or** if any of its predecessors is already bad. The extra sweeps let “badness” spread along every arrow.

Q13

JS

```
const hitByNegCycle = (n, E, src) => {
  const d = new Array(n).fill(Infinity);
  d[src] = 0;
  for (let i = 0; i < n - 1; i++)
    for (const [u, v, w] of E)
      if (d[u] + w < d[v]) d[v] = d[u] + w;
  const bad = new Array(n).fill(false);
  for (let i = 0; i < n; i++) // n sweeps let "badness"
    spread
    for (const [u, v, w] of E)
      if (d[u] !== Infinity && (bad[u] || d[u] + w < d[v])) bad[v] = true;
  return { bad, d };
};
```

Verified on $0 \rightarrow 1$ (1), $1 \rightarrow 2$ (-1), $2 \rightarrow 1$ (-1), $2 \rightarrow 3$ (1), $0 \rightarrow 4$ (4): `bad = 0 1 1 1 0` and `d[4] = 4`. Node 4 is off the loop and keeps a real distance; node 0 is the source and nothing returns to it. A graph with no negative cycle marks nothing.

Here the `d[u] !== Infinity` guard is needed, because it also gates the `bad[u]` propagation — an unreachable node must not spread badness.

Time $O(nm)$ **Space** $O(n)$

14. This is the widest-path idea with **max** instead of **min**: use Dijkstra where the “distance” of a route is its largest edge, and keep the smallest such value. Change the minimax Dijkstra of Example 24 from a grid to an adjacency list. A second correct answer: run Kruskal, and the minimax edge between s and t is the **largest edge on the MST path** between them — worth saying out loud, because it shows you know the MST has that property.

15. Adding edge (u, v, w) to a spanning tree creates exactly one cycle. The MST changes if and only if w is **smaller** than the largest edge on the existing tree path from u to v . Walk that path with one DFS from u to v on the tree, tracking the maximum edge: $O(n)$. If w is smaller, swap it for that maximum edge.

ONE-PAGE REVISION CARD

One import covers the whole chapter.

Toolkit

JS

```
const { MinHeap, DSU, Deque } = require('./toolkit.js'); // JS Toolkit appendix
```

JavaScript ships no heap, no ordered map and no deque. MinHeap with a flipped comparator is your max-heap; DSU.union returns false when the ends were already joined; Deque is the $O(1)$ queue that `Array.prototype.shift` is not. Graph: `g[u]` is a list of `[to, weight]` pairs. Unreached: Infinity.

Dijkstra — non-negative weights

Dijkstra

JS

```
const d = new Array(n).fill(Infinity);
const pq = new MinHeap((a, b) => a[0] - b[0]); // [dist, node]
d[src] = 0; pq.push([0, src]);
while (pq.size) {
  const [du, u] = pq.pop();
  if (du > d[u]) continue;
  for (const [v, w] of g[u])
    if (du + w < d[v]) { d[v] = du + w; pq.push([d[v], v]); }
}
```

Bellman-Ford — negatives allowed

Bellman-Ford

JS

```
const d = new Array(n).fill(Infinity);
d[src] = 0;
for (let i = 0; i < n - 1; i++)
  for (const [u, v, w] of E) // e = [from, to, weight]
    if (d[u] + w < d[v]) d[v] = d[u] + w;
for (const [u, v, w] of E) // one more pass that still
  if (d[u] + w < d[v]) return null; // improves => negative cycle
return d;
```

Floyd-Warshall — every pair, k outermost

Floyd

JS

```
for (let k = 0; k < n; k++)
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++)
      if (d[i][k] + d[k][j] < d[i][j]) d[i][j] = d[i][k] + d[k][j];
```

Kruskal — MST

Kruskal

JS

```
const e = [...E].sort((a, b) => a[2] - b[2]); // numeric, never a bare sort()
const d = new DSU(n);
let tot = 0, used = 0;
```

```
for (const [u, v, w] of e)
  if (d.union(u, v)) { tot += w; used++; }
return used === n - 1 ? tot : -1;
```

Kahn — topological order + cycle test

Kahn

JS

```
const indeg = new Array(n).fill(0), order = [];
const q = new Deque();
for (let u = 0; u < n; u++) for (const v of g[u]) indeg[v]++;
for (let i = 0; i < n; i++) if (indeg[i] === 0) q.push(i);
while (q.size) {
  const u = q.shift(); order.push(u);
  for (const v of g[u]) if (--indeg[v] === 0) q.push(v);
}
return order.length === n ? order : [];
```

// short -> a cycle

Pick the algorithm

weights all 1	BFS, $O(n + m)$
weights 0 or 1	0-1 BFS with a deque, $O(n + m)$
weights ≥ 0	Dijkstra, $O((n + m) \log n)$
some negative	Bellman-Ford, $O(nm)$
graph is a DAG	relax in topological order, $O(n + m)$
every pair, $n \leq 400$	Floyd-Warshall, $O(n^3)$
cheapest way to link all	Kruskal / Prim, $O(m \log m)$
order of dependent jobs	Kahn or DFS topo, $O(n + m)$
route cost = worst edge	Dijkstra with max instead of +
a limited resource (K stops, K coupons)	add it to the state

Complexity table

algorithm	time	space
BFS / 0-1 BFS	$O(n + m)$	$O(n + m)$
Dijkstra with heap	$O((n + m) \log n)$	$O(n + m)$
Dijkstra, dense, no heap	$O(n^2 + m)$	$O(n)$
Bellman-Ford	$O(nm)$	$O(n)$
Floyd-Warshall	$O(n^3)$	$O(n^2)$
Kruskal	$O(m \log m)$	$O(n)$
Prim with heap	$O(m \log n)$	$O(n + m)$
Topological sort	$O(n + m)$	$O(n)$
DAG shortest / longest path	$O(n + m)$	$O(n)$

The JavaScript facts this chapter depends on

<code>arr.sort()</code>	lexicographic on stringified elements. A bare <code>.sort()</code> on the edge list <code>[[0,1,1], [1,2,2], [0,2,4], [2,3,3], [3,0,10]]</code> yields weight order 1, 4, 2, 3, 10. Always <code>sort((a, b) => a[2] - b[2])</code> , and copy with <code>[...E]</code> first because <code>sort</code> mutates.
no heap	MinHeap from the JS Toolkit appendix. Max-heap = flipped comparator.
no TreeMap	not needed here; when it is, a sorted array plus the toolkit <code>lowerBound</code> .
comparators	must return a number . <code>(a, b) => a[0] > b[0]</code> returns a boolean, coerces to 1/0, and the ordering collapses.
numbers are doubles	exact to $2^{53} - 1 \approx 9 \times 10^{15}$. A path of 10^5 edges of weight 10^9 caps at 10^{14} , so this chapter is safe. Past it, BigInt .
recursion depth	about 10^4 frames. The DFS topological sort throws RangeError on a 100000-node chain — use Kahn.
<code>Array(n).fill([])</code>	stores one array in all n slots. Use <code>Array.from({ length: n }, () => [])</code> ; a grid copy is <code>g.map(r => [...r])</code> .
<code>arr.shift()</code>	$O(n)$. Use the toolkit Deque for a queue.
<code>Math.max(...arr)</code>	throws RangeError past roughly 2×10^5 entries. Use <code>reduce</code> .

Top traps

1	A bare <code>.sort()</code> in Kruskal. Lexicographic, so the MST is wrong.
2	Forgetting the comparator on MinHeap when pushing <code>[dist, node]</code> pairs — the default <code>(a, b) => a - b</code> on arrays gives NaN .
3	Dropping <code>if (du > d[u]) continue;</code> . Stale entries then corrupt path counts.
4	Running Dijkstra on a graph with a negative edge. Use Bellman-Ford.
5	Mixing a finite sentinel with a missing <code>d[u] !== INF</code> guard. Seed with Infinity and the guard becomes unnecessary.
6	Floyd with k not in the outer loop. The answer is silently wrong.
7	Kruskal without the <code>used === n - 1</code> check. A disconnected graph returns a fake total.
8	The “at most K stops” round without <code>const nd = [...d]</code> first.
9	Returning Infinity instead of -1 for unreachable nodes — and note <code>JSON.stringify(Infinity)</code> is <code>null</code> .
10	Mixing the sorted position of an edge with its original index.
11	Recursive DFS on 10^5 nodes. Kahn, or an explicit stack.

The one sentence to remember Pick the algorithm from the **weights**, not from the story: all-ones means BFS, non-negative means Dijkstra, negative means Bellman-Ford, no-cycles means topological order, and “link everything cheaply” means MST.

Dynamic Programming I — 1D & Knapsack

Recursion that refuses to repeat itself

Pattern in one page

Dynamic programming (DP) is one idea: **solve a big problem by re-using answers to smaller versions of the same problem**. Nothing more.

You already use it. To find the 10th Fibonacci number you do not re-count from zero every time. You keep the last two answers and add. That is DP.

THE THREE QUESTIONS

Before you write one line of code, answer these.

1. **State** — what is the smallest set of numbers that describes “where I am”? Write it as `dp[...]` and say in words what it means.
2. **Transition** — how do I build `dp[i]` from smaller states?
3. **Base case** — which states do I know without computing?

If you cannot say the state in one English sentence, you do not have a DP yet.

The five-step recipe

RECIPE — USE IT EVERY SINGLE TIME

1. Write the **brute-force recursion**. Do not optimise. Just: “at each step I choose X or Y”.
2. Find the **changing arguments** of that recursion. They are your state.
3. Add a **memo** array indexed by those arguments. Now it is top-down DP.
4. Turn the memo into a **loop** (bottom-up table). The loop order must go from base cases outward.
5. Look at which rows the loop reads. If it only reads the previous row, **drop the rest of the table** and keep $O(1)$ or $O(W)$ space.

Steps 1 → 2 → 3 are mechanical. You never need cleverness there. Cleverness only appears in step 1 (choosing the recursion) and step 5 (shrinking memory).

Two DP problems are two different animals

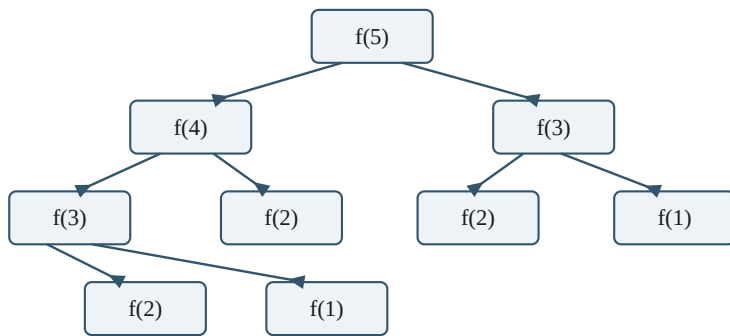
	Optimisation DP	Counting DP
Asks	max / min / “is it possible”	how many ways
Combiner	<code>Math.max</code> , <code>Math.min</code> , <code> </code>	<code>+</code>
Base	<code>dp[0] = 0</code> or <code>false</code>	<code>dp[0] = 1</code> (the empty way)
Precision	rare problem	common — watch 2^{53}
Order of loops	usually does not matter	often changes the answer

TRAP

The most common counting bug in placement tests: `dp[0] = 0` instead of `dp[0] = 1`. There is exactly **one** way to make the amount 0 — take nothing. If you set it to 0 every answer becomes 0.

Overlapping subproblems — the picture

Plain recursion for “ways to climb n stairs with steps of 1 or 2” re-computes the same call many times.



f(3) is computed twice, f(2) three times. A memo computes each node once.

Number of distinct nodes: about n . Number of calls without a memo: about 2^n . That single gap is the whole chapter.

Template 1 — top-down (memo)

Top-down skeleton — write this first when you are stuck

JAVASCRIPT

```
function solve(n) {
  const memo = new Array(n + 1).fill(-1);
  const go = (i) => {
    if (i < 0) return 0;          // base: impossible
    if (i === 0) return 1;        // base: known answer
    if (memo[i] !== -1) return memo[i];
    return memo[i] = go(i - 1) + go(i - 2);
  };
  return go(n);
}
```

TRAP

JavaScript recursion depth is roughly 10^4 frames, far below C++ or Java. A top-down DP over $n = 10^5$ will throw `RangeError: Maximum call stack size exceeded`. Two fixes: (a) convert to a bottom-up loop — always possible for a 1D DP; (b) run the recursion with an explicit stack. Prefer (a). Write top-down to **find** the recurrence, then ship the loop.

Template 2 — bottom-up (table)

Bottom-up skeleton — same recurrence, written as a loop

JAVASCRIPT

```
function solve(n) {
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;
  for (let i = 1; i <= n; i++) {
    dp[i] = dp[i - 1];
    if (i >= 2) dp[i] += dp[i - 2];
  }
  return dp[n];
}
```

TRAP

`new Array(n)` makes an array of **holes**, not zeros. `new Array(3).map(x => 1)` returns `[null, null, null]` — `map` skips holes. Always write `new Array(n).fill(0)`. And for a 2D table, `new Array(3).fill(new Array(4).fill(0))` stores **the same row three times**: setting `bad[0][1] = 9` gives `[[0,9,0,0],[0,9,0,0],[0,9,0,0]]`. The correct form is `Array.from({ length: 3 }, () => new Array(4).fill(0))`, which gives `[[0,9,0,0],[0,0,0,0],[0,0,0,0]]`.

Template 3 — 0/1 knapsack in one row

This is the single most tested DP shape in placement papers. Memorise the two-line core.

0/1 knapsack, one row, capacity loop BACKWARDS

JAVASCRIPT

```
const dp = new Array(cap + 1).fill(0);
for (let i = 0; i < w.length; i++)
  for (let c = cap; c >= w[i]; c--)
    dp[c] = Math.max(dp[c], v[i] + dp[c - w[i]]);
return dp[cap];
```

SHORTCUT

Backwards = each item used at most once (0/1). **Forwards** = each item used any number of times (**unbounded**). That one word — the direction of the inner loop — is the whole difference. Write it on your rough sheet before you start.

Use Infinity, not a big number

C++ programmers write `INT_MAX / 2` as a sentinel because `INT_MAX + 1` overflows. JavaScript has a real infinity and it behaves correctly under addition.

Sentinels in JS

JAVASCRIPT

```
const dp = new Array(n + 1).fill(Infinity);
dp[0] = 0;
// Infinity + 1 === Infinity -> no overflow, no wrap-around
// Infinity > 5 -> true
return dp[n] === Infinity ? -1 : dp[n];
```

Verified in Node: `Infinity + 1` prints `Infinity`, and `Infinity > 5` prints `true`. Use `-Infinity` for maximisation problems.

When to reach for 1D DP

The question says...	Reach for
count the ways to reach step n	$dp[i] = dp[i-1] + dp[i-2] + \dots$
minimum cost to reach the end	$dp[i] = \min \text{ over previous } j$
pick items, no two neighbours	take / skip pair of variables
fill a bag of capacity W exactly once each	0/1 knapsack, loop backwards
unlimited copies of each item	unbounded knapsack, loop forwards
split the array into two equal halves	subset sum on $\text{total}/2$
longest increasing / chain	LIS, $O(n^2)$ then $O(n \log n)$

The complexity rule for DP

COST OF A DP

time = (number of states) $\times$ (work per state)

space = (number of states you must keep alive)

For 0/1 knapsack: states = $n \times W$, work per state = $O(1)$, so time = $O(nW)$. With $n \leq 100$ and $W \leq 10^4$ that is 10^6 operations — fast. With $W \leq 10^9$ it is hopeless, and that is your hint to index the table by **value** instead (Example 27).

The number trap you must know before you start

TRAP

JavaScript numbers are doubles. Integers are exact only up to `Number.MAX_SAFE_INTEGER` = $2^{53} - 1 = 9007199254740991$.

Counting DPs cross that line fast. For the staircase count of Example 1:

`climbWays(45)` = 1836311903 — exact. `climbWays(77)` = 8944394323791464 — exact, just under the limit. `climbWays(79)` = 23416728348467684 — **wrong**. The true value is 23416728348467685.

Products bite even harder: `123456789 * 987654321` prints 121932631112635260, but the true product is 121932631112635269.

Two fixes: take the answer **modulo** $10^9 + 7$ (what the question usually asks for), or switch to `BigInt` — `123456789n * 987654321n` gives the exact 121932631112635269n.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

A staircase has n steps. You climb 1 or 2 steps at a time. How many different ways can you reach the top? Constraints: $0 \leq n \leq 77$ for an exact answer. Target: $O(n)$ time, $O(1)$ space. Edge cases: $n = 0$ (one way — stand still), $n = 79$ (the answer is no longer exact in a JS number).

SOLUTION

State: $\text{ways}[i]$ = number of ways to stand on step i .

To stand on step i your last move came from $i-1$ or from $i-2$. So

$$\text{ways}[i] = \text{ways}[i-1] + \text{ways}[i-2]$$

Base: $\text{ways}[0] = 1$ (do nothing), $\text{ways}[1] = 1$ (one single step).

Only two old values are ever needed, so keep two variables.

climbWays — $O(1)$ space

JAVASCRIPT

```
function climbWays(n) {
  if (n <= 1) return 1;
  let twoBack = 1, oneBack = 1;    // ways(0), ways(1)
  for (let i = 2; i <= n; i++) {
    const cur = oneBack + twoBack;
    twoBack = oneBack;
    oneBack = cur;
  }
  return oneBack;
}
```

Run for $n = 0..10$:

1 1 2 3 5 8 13 21 34 55 89

and $\text{climbWays}(45) = 1836311903$, $\text{climbWays}(77) = 8944394323791464$.

Time $O(n)$ **Space** $O(1)$

exact up to $n = 77$; beyond that use BigInt or a modulus

Answer: The Fibonacci numbers, shifted by one.

Example 2 [warm-up]

$\text{cost}[i]$ is the fee to **step on** stair i . You start on the ground, may begin at stair 0 or stair 1, and from stair i you move to $i+1$ or $i+2$. Find the cheapest way to get past the top. Constraints: $0 \leq n \leq 10^5$, $0 \leq \text{cost}[i] \leq 1000$. Edge cases: empty array, single stair.

SOLUTION

State: $\text{dp}[i]$ = cheapest total fee to **arrive at and pay for** stair i .

$$\text{dp}[i] = \text{cost}[i] + \min(\text{dp}[i-1], \text{dp}[i-2])$$

Base: $\text{dp}[0] = \text{cost}[0]$, $\text{dp}[1] = \text{cost}[1]$ (you may start at either).

The top is one step past the last stair, so the answer is $\min(\text{dp}[n-1], \text{dp}[n-2])$.

```
function minCostStairs(cost) {
  const n = cost.length;
  if (n === 0) return 0;
  if (n === 1) return cost[0];
  let prev2 = cost[0], prev1 = cost[1];
  for (let i = 2; i < n; i++) {
    const cur = cost[i] + Math.min(prev1, prev2);
    prev2 = prev1;
    prev1 = cur;
  }
  return Math.min(prev1, prev2);
}
```

Tested outputs:

input	output
[10, 15, 20]	15
[1, 100, 1, 1, 1, 100, 1, 1, 100, 1]	6
[] (empty)	0
[7]	7
[5, 3]	3

Time $O(n)$ **Space** $O(1)$

Answer: [10, 15, 20] → 15 (start at stair 1, pay 15, then jump over the top).

Example 3 [warm-up]

Pick numbers from an array so that no two picked numbers sit next to each other. Maximise the sum. Constraints: $0 \leq n \leq 10^5$, values may be negative. Edge cases: empty array, all negative.

SOLUTION

Two running answers are enough.

- take = best sum where the **last element was taken**
- skip = best sum where the **last element was skipped**

Moving to the next element x :

$$\text{take}' = \text{skip} + x, \quad \text{skip}' = \max(\text{skip}, \text{take})$$

```
function maxNonAdjacent(a) {
  let take = 0, skip = 0; // best if a[i-1] was taken / skipped
  for (const x of a) {
    const newTake = skip + x;
    const newSkip = Math.max(skip, take);
    take = newTake;
    skip = newSkip;
  }
  return Math.max(take, skip);
}
```

Outputs: $[3, 2, 7, 10] \rightarrow 13$ ($3 + 10$); $[5] \rightarrow 5$; $[] \rightarrow 0$; $[-4, -1, -9] \rightarrow 0$; $[2000000000, 1, 2000000000] \rightarrow 4000000000$.

Time $O(n)$ **Space** $O(1)$

Answer: 13 for $[3, 2, 7, 10]$.

TRAP

$[-4, -1, -9]$ gives **0**, not -1 , because “pick nothing” is a legal empty choice. If the question says **at least one element must be picked**, start with $\text{take} = a[0]$, $\text{skip} = -\text{Infinity}$ instead. Always ask the interviewer which one they want.

Example 4 [warm-up]

Same staircase, but steps of 1, 2 or 3. Count the ways. $0 \leq n \leq 60$. Edge cases: $n = 0$; $n = 2$ (only 2 ways, not 4).

SOLUTION

$$\text{dp}[i] = \text{dp}[i - 1] + \text{dp}[i - 2] + \text{dp}[i - 3]$$

Base: $\text{dp}[0] = 1$, $\text{dp}[1] = 1$, $\text{dp}[2] = 2$.

waysSteps123

JAVASCRIPT

```
function waysSteps123(n) {
  const dp = new Array(Math.max(n + 1, 3)).fill(0);
  dp[0] = 1; dp[1] = 1; dp[2] = 2;
  for (let i = 3; i <= n; i++) dp[i] = dp[i-1] + dp[i-2] + dp[i-3];
  return dp[n];
}
```

Output for $n = 0..8$:

1 1 2 4 7 13 24 44 81

and $\text{waysSteps123}(30) = 53798080$.

Time $O(n)$ **Space** $O(n)$
shrink to 3 variables for $O(1)$ space

Answer: These are the Tribonacci numbers.

Why $\text{Math.max}(n + 1, 3)$ in the array size? If a tester calls with $n = 0$, an array of length 1 would leave $\text{dp}[2] = 2$ writing past the end and silently growing the array. Sizing it to at least 3 removes that whole class of bug.

Example 5 [warm-up]

A frog is on stone 0 of n stones with heights $h[]$. From stone i it hops to $i+1$ or $i+2$. A hop from i to j costs $|h[i] - h[j]|$. Find the minimum total cost to reach the last stone. $1 \leq n \leq 10^5$. Edge cases: $n = 1$ (cost 0), all heights equal.

SOLUTION

State: $\text{dp}[i]$ = cheapest cost to **stand on** stone i .

$$\text{dp}[i] = \min(\text{dp}[i - 1] + |h[i] - h[i - 1]|, \text{dp}[i - 2] + |h[i] - h[i - 2]|)$$

frogMinCost

JAVASCRIPT

```
function frogMinCost(h) {
  const n = h.length;
  if (n <= 1) return 0;
  let prev2 = 0, prev1 = Math.abs(h[1] - h[0]);
  for (let i = 2; i < n; i++) {
    const one = prev1 + Math.abs(h[i] - h[i-1]);
    const two = prev2 + Math.abs(h[i] - h[i-2]);
    prev2 = prev1;
    prev1 = Math.min(one, two);
  }
  return prev1;
}
```

Outputs: [10,30,40,20] → 30; [30,10,60,10,60,50] → 40; [7] → 0; [] → 0; [1,1,1,1] → 0.

Time $O(n)$ **Space** $O(1)$

Answer:

30. Path 10 → 30 → 20 costs 20 + 10 = 30; the greedy path 10 → 30 → 40 → 20 costs 20 + 10 + 20 = 50.

TRAP

Greedy fails here. “Always hop to the closer height” picks 10 → 30 then 30 → 40 and loses. DP wins because it keeps **both** possibilities alive until the end.

Example 6 [warm-up]

Given non-negative numbers, can any subset add up exactly to target? $1 \leq n \leq 100$, $0 \leq \text{target} \leq 10^4$. Edge cases: empty array with target 0; array containing zeros.

SOLUTION

State: $\text{dp}[s]$ = true if some subset of the items seen so far sums to s .

Start with $\text{dp}[0] = \text{true}$. For each new item x , any sum s becomes reachable if $s - x$ was already reachable.

subsetSumExists — the 0/1 pattern

JAVASCRIPT

```
function subsetSumExists(a, target) {
  const dp = new Array(target + 1).fill(false);
  dp[0] = true;
  for (const x of a)
    for (let s = target; s >= x; s--)
      if (dp[s - x]) dp[s] = true;
  return dp[target];
}
```

Outputs: [3,34,4,12,5,2] target 9 → true, target 30 → false; [] target 0 → true, target 5 → false; [0,0,7] target 7 → true, target 1 → false; [2,2,2] target 4 → true, target 5 → false.

Time $O(n \cdot \text{target})$ **Space** $O(\text{target})$

Answer: true for 9 (4 + 5), false for 30.

TRAP

If the inner loop went **forwards** (`for (let s = x; s <= target; s++)`) the same item would be reused many times and [2] would reach target 6. Backwards is what makes it 0/1.

Tier 1 — the standard set

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Count the ways to climb n stairs with steps of 1 or 2 — but this time show the **full ladder** from the naive solution to the best one. $n \leq 77$.

SOLUTION

Approach 1 — Plain recursion ($O(2^n)$ — dies around $n = 40$)

Approach 1 — no memory at all

JAVASCRIPT

```
function waysBrute(n) {
  if (n === 0) return 1;
  if (n < 0) return 0;
  return waysBrute(n - 1) + waysBrute(n - 2);
}
```

Time $O(2^n)$ **Space** $O(n)$
call stack depth n

Approach 2 — Recursion + memo ($O(n)$ — just add an array)

Approach 2 — top-down

JAVASCRIPT

```
function waysMemo(n) {
  const memo = new Array(n + 1).fill(-1);
  const go = (k) => {
    if (k === 0) return 1;
    if (k < 0) return 0;
    if (memo[k] !== -1) return memo[k];
    return memo[k] = go(k - 1) + go(k - 2);
  };
  return go(n);
}
```

Time $O(n)$ **Space** $O(n)$
memo array + recursion stack — the stack is the risky part in JS

Approach 3 — Bottom-up table ($O(n)$, no stack)

Approach 3 — loop, not recursion

JAVASCRIPT

```
function waysTab(n) {
  const dp = new Array(n + 1).fill(0);
  dp[0] = 1;
  for (let i = 1; i <= n; i++) {
    dp[i] = dp[i - 1];
    if (i >= 2) dp[i] += dp[i - 2];
  }
}
```

```

    }
    return dp[n];
}

```

Time $O(n)$ **Space** $O(n)$

Approach 4 — Two variables ($O(n)$ time, $O(1)$ space — optimal)

Approach 4 — the table only ever reads 2 cells back

JAVASCRIPT

```

function ways01(n) {
  if (n === 0) return 1;
  let twoBack = 1, oneBack = 1;
  for (let i = 2; i <= n; i++) {
    const cur = oneBack + twoBack;
    twoBack = oneBack;
    oneBack = cur;
  }
  return oneBack;
}

```

Time $O(n)$ **Space** $O(1)$

All four agree on $n = 0..12$:

1 1 2 3 5 8 13 21 34 55 89 144 233

and all three fast ones give $\text{waysMemo}(50) = \text{waysTab}(50) = \text{ways01}(50) = 20365011074$.

Answer: The idea that unlocked it: the recursion's only changing argument is n , so there are only $n + 1$ distinct calls. Everything else was repeated work.

Notice that Approach 4 is not a new idea — it is Approach 3 with the useless part of the array deleted. In an interview, say out loud: “the recurrence only looks two cells back, so I can keep two variables.” That sentence is what they want to hear.

TRAP

Approach 2 is the one that fails in production JavaScript. $\text{waysMemo}(200000)$ throws `RangeError: Maximum call stack size exceeded` long before it runs out of time, because Node's stack holds only about 10^4 frames. Approaches 3 and 4 have no stack at all. Use top-down to **discover** the recurrence, then ship bottom-up.

Example 8 [Infosys · pattern]

Frog on stones again, but now it may hop up to k stones forward. Same cost $|h[i] - h[j]|$. Minimise the cost to reach the last stone. $1 \leq n \leq 10^5$, $1 \leq k \leq 100$. Target: $O(nk)$. Edge cases: $k = 1$ (forced path), $n = 1$.

SOLUTION

State is the same, only the transition widens.

$$\text{dp}[i] = \min_{1 \leq j \leq k, i-j \geq 0} (\text{dp}[i-j] + |h[i] - h[i-j]|)$$

frogK

JAVASCRIPT

```

function frogK(h, k) {
  const n = h.length;
  if (n <= 1) return 0;

```

```
const dp = new Array(n).fill(Infinity);
dp[0] = 0;
for (let i = 1; i < n; i++)
  for (let j = 1; j <= k && j <= i; j++)
    dp[i] = Math.min(dp[i], dp[i - j] + Math.abs(h[i] - h[i - j]));
return dp[n - 1];
}
```

Outputs: [10, 30, 40, 50, 20] with $k = 3 \rightarrow 30$; the same array with $k = 1 \rightarrow 70$; [40, 10, 20, 70, 80, 10] with $k = 4 \rightarrow 30$; [5] $\rightarrow 0$; [] $\rightarrow 0$.

Time $O(nk)$ **Space** $O(n)$

Answer: 30 with $k = 3$ (hop 10 $\rightarrow$ 50 $\rightarrow$ 20 is $40 + 30 = 70$; hop 10 $\rightarrow$ 30 $\rightarrow$ 20 is $20 + 10 = 30$).

SHORTCUT

In C++ this loop needs a hand-picked sentinel like `INT_MAX / 2`, because `INT_MAX + cost` overflows to a negative number and the `min` picks garbage. JavaScript's `Infinity` is a genuine infinity: `Infinity + 40` is still `Infinity`. One less bug class for free.

Example 9 [Wipro · pattern]

Shops stand in a **circle**. You may not pick two neighbouring shops, and shop 0 and shop $n - 1$ are neighbours. Maximise the sum. $1 \leq n \leq 10^5$, $0 \leq a[i] \leq 10^9$. Edge cases: $n = 1$, $n = 2$.

SOLUTION

Approach 1 — Try every subset ($O(2^n)$ — impossible past $n = 25$)

Approach 2 — Two straight-line runs ($O(n)$ — optimal)

The circular rule says one thing only: **you cannot take both index 0 and index $n - 1$** . So the best answer is either an answer that never touches the last shop, or one that never touches the first shop.

- Run the straight-line solver on `a[0 .. n-2]`
- Run it again on `a[1 .. n-1]`
- Take the larger

robCircle

JAVASCRIPT

```
function bestLine(a, lo, hi) { // inclusive range
  let take = 0, skip = 0;
  for (let i = lo; i <= hi; i++) {
    const nt = skip + a[i];
    const ns = Math.max(skip, take);
    take = nt; skip = ns;
  }
  return Math.max(take, skip);
}

function robCircle(a) {
  const n = a.length;
  if (n === 0) return 0;
  if (n === 1) return a[0];
  return Math.max(bestLine(a, 0, n - 2), bestLine(a, 1, n - 1));
}
```

Outputs: $[2, 3, 2] \rightarrow 3$; $[1, 2, 3, 1] \rightarrow 4$; $[9] \rightarrow 9$; $[4, 4] \rightarrow 4$; $[] \rightarrow 0$; $[10^9, 1, 10^9, 1, 10^9] \rightarrow 2000000000$.

Time $O(n)$ **Space** $O(1)$

Answer: 3 for $[2, 3, 2]$ — you cannot take both 2s because they are neighbours in a circle.

TRAP

The $n = 1$ case must be handled **before** the two runs. With $n = 1$ the range $0 \dots n-2$ is $0 \dots -1$, which is empty, and you would return 0 instead of $a[0]$.

Example 10 [Capgemini · pattern]

Coins of given values, unlimited supply. What is the **smallest number of coins** that makes exactly amount? Return -1 if impossible. $1 \leq \text{amount} \leq 10^4$, at most 12 coin types. Edge cases: amount 0; coin set that cannot make the amount.

SOLUTION

Approach 1 — Recursion — try every coin at every step (exponential)

Approach 1

JAVASCRIPT

```
function bruteCoins(coins, amt) {
  if (amt === 0) return 0;
  let best = Infinity;
  for (const c of coins)
    if (c <= amt) best = Math.min(best, 1 + bruteCoins(coins, amt - c));
  return best;
}
```

Time $O(k^{\text{amt}})$ **Space** $O(\text{amt})$

k = number of coin types

Approach 2 — Greedy — always take the biggest coin (WRONG)

With coins $[1, 3, 4]$ and amount 6, greedy takes $4 + 1 + 1 = 3$ coins. The real answer is $3 + 3 = 2$ coins. Greedy only works for special coin systems, never in general.

Approach 3 — Bottom-up table ($O(\text{amount} \times k)$ — optimal)

$$\text{dp}[s] = 1 + \min_{c \in \text{coins}, c \leq s} \text{dp}[s - c]$$

minCoins

JAVASCRIPT

```
function minCoins(coins, amt) {
  const dp = new Array(amt + 1).fill(Infinity);
  dp[0] = 0;
  for (let s = 1; s <= amt; s++)
    for (const c of coins)
      if (c <= s) dp[s] = Math.min(dp[s], dp[s - c] + 1);
  return dp[amt] === Infinity ? -1 : dp[amt];
}
```

With coins $[1, 3, 4]$, amounts $0..12$:

0 1 2 1 1 2 2 2 2 3 3 3 3

With coins [5, 7]: amount 11 $\rightarrow$ -1, amount 12 $\rightarrow$ 2, amount 3 $\rightarrow$ -1. With coins []: amount 0 $\rightarrow$ 0, amount 5 $\rightarrow$ -1. The brute force gives the same numbers for 0..12.

Time $O(\text{amount} \times k)$ **Space** $O(\text{amount})$

Answer: 2 coins for amount 6 with [1, 3, 4].

Python, same recurrence — a little shorter because of the list comprehension:

min_coins — the Python version

PYTHON

```
def min_coins(coins, amount):
    INF = float('inf')
    dp = [INF] * (amount + 1)
    dp[0] = 0
    for s in range(1, amount + 1):
        for c in coins:
            if c <= s and dp[s - c] + 1 < dp[s]:
                dp[s] = dp[s - c] + 1
    return -1 if dp[amount] == INF else dp[amount]
```

Output: [0, 1, 2, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3] for coins [1, 3, 4], then 2, -1, 0, -1 for the edge cases above.

Example 11 [Cognizant · pattern]

Same coins, unlimited supply — now **count the number of ways** to make amount, where [1, 3] and [3, 1] are the **same** way. Then also count them as **different**. $1 \leq \text{amount} \leq 5000$. Edge cases: amount 0; a coin larger than the amount.

SOLUTION

This is the one problem where **loop order is the answer**.

Combinations (order does not matter) — coin loop outside:

countCombinations — coin loop OUTSIDE

JAVASCRIPT

```
function countCombinations(coins, amt) {
    const dp = new Array(amt + 1).fill(0);
    dp[0] = 1;
    for (const c of coins)
        for (let s = c; s <= amt; s++)
            dp[s] += dp[s - c];
    return dp[amt];
}
```

Permutations (order matters) — amount loop outside:

countPermutations — amount loop OUTSIDE

JAVASCRIPT

```
function countPermutations(coins, amt) {
    const dp = new Array(amt + 1).fill(0);
    dp[0] = 1;
    for (let s = 1; s <= amt; s++)
        for (const c of coins)
            if (c <= s) dp[s] += dp[s - c];
    return dp[amt];
}
```

Measured outputs:

coins	amt	combinations	permutations
[1,2,5]	5	4	9
[1,3]	4	2	3
[2]	3	0	—
[2]	4	1	—
[]	0	1	—
[]	7	0	—
[1,2]	60	31	2504730781961

Answer: 4 combinations of [1, 2, 5] for 5: 5; 2 + 2 + 1; 2 + 1 + 1 + 1; 1 × 5.

SHORTCUT

Say it to yourself in words. **Coin outside** = “I finish deciding how many 1-rupee coins, then move on to 2-rupee coins.” Each coin is considered once, so order cannot repeat. **Amount outside** = “for this amount, the last coin I placed was any of them.” The last coin can differ, so orders are counted separately.

TRAP

`countPermutations([1,2], 60)` is 2504730781961 — safe. But the same count for amount 200 blows past 2^{53} and stops being exact. Any counting DP that is not asked modulo something should be checked against `Number.MAX_SAFE_INTEGER` before you trust it.

Example 12 [TCS Digital · pattern]

0/1 knapsack. n items, item i has weight $w[i]$ and value $v[i]$. A bag holds at most cap . Each item may be taken at most once. Maximise value. $1 \leq n \leq 100$, $1 \leq cap \leq 10^4$. Edge cases: no items; item heavier than the bag; $cap = 0$.

SOLUTION

Approach 1 — Try both choices for every item ($O(2^n)$)

Approach 1 — pure recursion

JAVASCRIPT

```
function knapBrute(w, v, i, cap) {
  if (i < 0 || cap === 0) return 0;
  const skip = knapBrute(w, v, i - 1, cap);
  let take = 0;
  if (w[i] <= cap) take = v[i] + knapBrute(w, v, i - 1, cap - w[i]);
  return Math.max(skip, take);
}
```

Time $O(2^n)$ **Space** $O(n)$

Approach 2 — Memo on (i, cap) ($O(n \times cap)$)

The recursion has exactly two changing arguments: i and cap . So memo on a 2D array.

Approach 2 — top-down

JAVASCRIPT

```
function knapMemo(w, v, cap) {
  const n = w.length;
  if (n === 0) return 0;
  const memo = Array.from({ length: n }, () => new Array(cap + 1).fill(-1));
  const go = (i, c) => {
    if (i < 0 || c === 0) return 0;
    if (memo[i][c] !== -1) return memo[i][c];
    const skip = go(i - 1, c);
    const take = w[i] <= c ? v[i] + go(i - 1, c - w[i]) : 0;
    return memo[i][c] = Math.max(skip, take);
  };
  return go(n - 1, cap);
}
```

Time $O(n \cdot \text{cap})$ **Space** $O(n \cdot \text{cap})$

Approach 3 — 2D table ($O(n \times \text{cap})$, no recursion)

Approach 3 — bottom-up, rows = items

JAVASCRIPT

```
function knapTable(w, v, cap) {
  const n = w.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(cap + 1).fill(0));
  for (let i = 1; i <= n; i++)
    for (let c = 0; c <= cap; c++) {
      dp[i][c] = dp[i-1][c];
      if (w[i-1] <= c) dp[i][c] = Math.max(dp[i][c], v[i-1] + dp[i-1][c - w[i-1]]);
    }
  return dp[n][cap];
}
```

Time $O(n \cdot \text{cap})$ **Space** $O(n \cdot \text{cap})$

Approach 4 — One row ($O(n \times \text{cap})$ time, $O(\text{cap})$ space — optimal)

Row i reads only row $i-1$. Keep one row and walk the capacity **backwards** so that $\text{dp}[c - w[i]]$ still holds the old row's value.

Approach 4 — the version to memorise

JAVASCRIPT

```
function knap1D(w, v, cap) {
  const dp = new Array(cap + 1).fill(0);
  for (let i = 0; i < w.length; i++)
    for (let c = cap; c >= w[i]; c--)
      dp[c] = Math.max(dp[c], v[i] + dp[c - w[i]]);
  return dp[cap];
}
```

Time $O(n \cdot \text{cap})$ **Space** $O(\text{cap})$

Verification with $w = [1,3,4,5]$, $v = [1,4,5,7]$, $\text{cap} = 7$: all four approaches return **9**. Edge cases: no items $\rightarrow 0$; single item of weight 10 with cap 5 $\rightarrow 0$, with cap 10 $\rightarrow 60$; $w=[2,2,2]$, $v=[3,3,3]$, $\text{cap}=4 \rightarrow 6$, with cap 0 $\rightarrow 0$. A randomised test comparing Approach 1 against Approach 4 on 200 random inputs printed 200 random cross-checks passed.

Answer: 9 — take items of weight 3 and 4, value $4 + 5 = 9$, total weight 7.

knapsack — Python, the same one row

PYTHON

```
def knapsack(weights, values, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(weights, values):
        for c in range(cap, w - 1, -1):
            dp[c] = max(dp[c], v + dp[c - w])
    return dp[cap]
```

Output: 9 for the example above; 0 for empty input; 0 and 60 for the single heavy item with cap 5 and 10.

TRAP

Writing the capacity loop **forwards** in Approach 4 silently turns 0/1 knapsack into unbounded knapsack. It still runs and gives a bigger (wrong) answer. Graders love this test.

TRAP

`Array.from({ length: n }, () => new Array(cap + 1).fill(-1))` in Approach 2 is not decoration. `new Array(n).fill(new Array(cap + 1).fill(-1))` would store the **same row** n times — every memo write would be visible from every item index, and the answer would be wrong in a way that is very hard to see.

Example 13 [Accenture · pattern]

Split an array of non-negative numbers into two groups with the **same sum**. Is it possible? $1 \leq n \leq 200$, $0 \leq a[i] \leq 100$. Edge cases: odd total (instantly false); a single element; all zeros.

SOLUTION

Approach 1 — Generate every subset ($O(2^n)$ — hopeless at $n = 200$)

Approach 2 — Subset sum on half the total ($O(n \times \text{total})$ — optimal here)

If the two groups have equal sums, each group sums to $\frac{\text{total}}{2}$. So:

1. If total is odd → impossible, stop.
2. Otherwise ask: can a subset reach exactly $\text{total} / 2$? That is Example 6.

canPartition

JAVASCRIPT

```
function canPartition(a) {
    const total = a.reduce((s, x) => s + x, 0);
    if (total % 2 !== 0) return false;
    const half = total / 2;
    const dp = new Array(half + 1).fill(false);
    dp[0] = true;
    for (const x of a)
        for (let s = half; s >= x; s--)
            if (dp[s - x]) dp[s] = true;
    return dp[half];
}
```

Tested outputs:

input	output	why
[1, 5, 11, 5]	true	$11 = 1 + 5 + 5$

[1, 2, 3, 5]	false	total 11 is odd
[]	true	two empty groups, both sum 0
[4]	false	one group would be empty, sums 4 and 0
[2, 2]	true	
[0, 0]	true	
[100, 100, 100, 100, 99, 1]	false	total 500, but no subset hits 250

Time $O(n \cdot \text{total})$ **Space** $O(\text{total})$
total $\leq 200 \times 100 = 20000$, so about 4 million steps

Answer: true for [1, 5, 11, 5].

Why is [100, 100, 100, 100, 99, 1] false? The total is 500 and half is 250. Every subset sum is built from 100s plus possibly 99 and 1. To reach 250 you need $100a + 99b + c = 250$ with $b, c \in \{0, 1\}$. $a = 2$ gives 200, 299, 201 or 300 — none is 250. So no split exists, even though the total is even. This is exactly why you cannot answer with a parity check alone.

Example 14 [TCS NQT · pattern]

Unbounded knapsack. Same bag, but each item type has unlimited copies. Maximise value. Then use it to solve **rod cutting**: a rod of length n , $\text{price}[L-1]$ is the money for a piece of length L ; cut it to maximise income. $1 \leq \text{cap}, n \leq 10^4$. Edge cases: cap 0; every item heavier than the bag.

SOLUTION

The only change from 0/1 knapsack is the **direction of the inner loop**. Going forwards means $\text{dp}[c - w[i]]$ may already include item i , so item i can be used again.

unboundedKnap — capacity loop FORWARDS

JAVASCRIPT

```
function unboundedKnap(w, v, cap) {
  const dp = new Array(cap + 1).fill(0);
  for (let c = 1; c <= cap; c++)
    for (let i = 0; i < w.length; i++)
      if (w[i] <= c) dp[c] = Math.max(dp[c], v[i] + dp[c - w[i]]);
  return dp[cap];
}
```

Rod cutting is the same code with $w[i] = i + 1$:

rodCut

JAVASCRIPT

```
function rodCut(price) {
  const n = price.length;
  const dp = new Array(n + 1).fill(0);
  for (let len = 1; len <= n; len++)
    for (let cut = 1; cut <= len; cut++)
      dp[len] = Math.max(dp[len], price[cut - 1] + dp[len - cut]);
  return dp[n];
}
```

Outputs: $w=[2,3,4]$, $v=[5,9,11]$, $\text{cap}=8 \rightarrow 23$; same with $\text{cap } 0 \rightarrow 0$; $w=[5]$, $v=[7]$, $\text{cap } 4 \rightarrow 0$, $\text{cap } 17 \rightarrow 21$; $\text{price} = [1,5,8,9,10,17,17,20]$ (rod length 8) $\rightarrow 22$; $\text{price}=[3] \rightarrow 3$; $\text{price}=[] \rightarrow 0$.

Time $O(n \cdot \text{cap})$ **Space** $O(\text{cap})$

Answer: 23 for the bag: take the weight-3 item twice (18) plus the weight-2 item once (5), total weight 8. For the rod of length 8: cut into 2 + 6 giving 5 + 17 = 22.

SHORTCUT

Two identical loops, one word apart:

for (let c = cap; c >= w[i]; c--) → each item once (0/1)

for (let c = w[i]; c <= cap; c++) → each item unlimited

Write both on your rough sheet at the start of the exam and never think about it again.

Example 15 [Infosys · pattern]

Longest Increasing Subsequence (LIS). Find the length of the longest strictly increasing subsequence, and also print one such subsequence. $1 \leq n \leq 2500$ for this $O(n^2)$ version. Edge cases: empty array; all equal; all negative.

SOLUTION

Approach 1 — Check every subsequence ($O(2^n)$)

Approach 2 — DP on 'LIS ending exactly at i' ($O(n^2)$ — good up to $n = 2500$)

State: $dp[i]$ = length of the longest increasing subsequence that **ends at index i**.

$$dp[i] = 1 + \max_{j < i, a[j] < a[i]} dp[j] \quad (\text{or } 1 \text{ if no such } j)$$

The answer is the maximum over all i — **not** $dp[n-1]$.

lisLength

JAVASCRIPT

```
function lisLength(a) {
  const n = a.length;
  if (n === 0) return 0;
  const dp = new Array(n).fill(1);
  let best = 1;
  for (let i = 1; i < n; i++) {
    for (let j = 0; j < i; j++)
      if (a[j] < a[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
    best = Math.max(best, dp[i]);
  }
  return best;
}
```

To **print** the subsequence, remember which j gave you the best value.

lisSequence — with parent pointers

JAVASCRIPT

```
function lisSequence(a) {
  const n = a.length;
  if (n === 0) return [];
  const dp = new Array(n).fill(1), parent = new Array(n).fill(-1);
  let best = 1, bestEnd = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < i; j++)
      if (a[j] < a[i] && dp[j] + 1 > dp[i]) { dp[i] = dp[j] + 1; parent[i] = j; }
    if (dp[i] > best) { best = dp[i]; bestEnd = i; }
  }
  const out = [];
  for (let i = bestEnd; i !== -1; i = parent[i]) out.push(a[i]);
}
```

```
    return out.reverse();
}
```

Outputs:

input	len	one LIS
[10,9,2,5,3,7,101,18]	4	2 5 7 101
[7,7,7]	1	7
[]	0	(nothing)
[5]	1	5
[-5, -3, -4, -1]	3	-5 -3 -1

Time $O(n^2)$ **Space** $O(n)$

Answer: 4, one witness being 2, 5, 7, 101.

TRAP

[7,7,7] has LIS 1 because **strictly** increasing means $a[j] < a[i]$, not $\leq$. If the problem says “non-decreasing”, change the test to $a[j] \leq a[i]$ — and the answer becomes 3.

Example 16 [Wipro · pattern]

A message is a string of digits. 1 decodes to A, ..., 26 decodes to Z. There is no letter for 0. How many ways can the string be decoded? $1 \leq |s| \leq 100$. Edge cases: leading 0; “100”; empty string.

SOLUTION

State: $dp[i]$ = number of ways to decode the first i characters.

Two moves land on position i :

- take one digit $s[i-1]$, legal unless it is ‘0’
- take two digits $s[i-2..i-1]$, legal only if the number is between 10 and 26

$$dp[i] = [s_{i-1} \neq 0] \cdot dp[i-1] + [10 \leq \overline{s_{i-2}s_{i-1}} \leq 26] \cdot dp[i-2]$$

Base: $dp[0] = 1$ — the empty prefix has one (empty) decoding.

decodeWays

JAVASCRIPT

```
function decodeWays(s) {
    const n = s.length;
    if (n === 0) return 0;
    const dp = new Array(n + 1).fill(0);
    dp[0] = 1;
    for (let i = 1; i <= n; i++) {
        if (s[i-1] !== '0') dp[i] += dp[i-1];
        if (i >= 2) {
            const two = Number(s.slice(i-2, i));
            if (two >= 10 && two <= 26) dp[i] += dp[i-2];
        }
    }
    return dp[n];
}
```

Outputs:

input	ways	input	ways
"12"	2	"0"	0
"226"	3	"10"	1
"06"	0	"100"	0
" "	0	"27"	1
"1111"	5	"2101"	1
30 ones	1346269		

Time $O(|s|)$ **Space** $O(|s|)$
two variables give $O(1)$ space

Answer: "226" → 3: BZ (2,26), VF (22,6), BBF (2,2,6).

TRAP

Three separate zero traps: "06" is 0 because a two-digit code may not start with 0; "10" is 1 because the 0 must pair with the 1; "100" is 0 because the second 0 has no legal partner ("00" is not in 10..26). Test all three. A fourth trap is pure JavaScript: `s[i-2] * 10 + s[i-1]` does **string** arithmetic and gives nonsense, because `s[i]` is a one-character string. `Number(s.slice(i-2, i))` is the safe form, and "06" converts to the number 6, which correctly fails the `>= 10` test.

Tier 2 — applied and two-step

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Shopee · pattern]

A warehouse has n parcels with integer weights. Count **how many different subsets** weigh exactly target kilograms. Answers can be huge, so report the count modulo $10^9 + 7$. $1 \leq n \leq 200$, $0 \leq \text{target} \leq 10^4$, weights may be **zero** (empty boxes). Edge cases: zero-weight parcels; target 0; empty warehouse.

SOLUTION

Counting DP, so the combiner is + and the base is $\text{dp}[0] = 1$.

$$\text{dp}[s] \text{ after adding item } x = \text{dp}[s] + \text{dp}[s - x]$$

countSubsetsWithSum

JAVASCRIPT

```
const MOD = 1000000007;

function countSubsetsWithSum(a, target) {
  if (target < 0) return 0;
  const dp = new Array(target + 1).fill(0);
  dp[0] = 1; // the empty subset
  for (const x of a)
    for (let s = target; s >= x; s--) // x may be 0: the loop still runs,
      // that is wanted
      dp[s] = (dp[s] + dp[s - x]) % MOD;
  return dp[target];
}
```

With $a = [1, 2, 3]$ and targets 0..6:

1 1 1 2 1 1 1

Other outputs: $[0, 0, 1]$ target 1 $\rightarrow$ 4, target 0 $\rightarrow$ 4; $[\]$ target 0 $\rightarrow$ 1, target 3 $\rightarrow$ 0; $[2, 2, 2]$ target 4 $\rightarrow$ 3.

Time $O(n \cdot \text{target})$ **Space** $O(\text{target})$

Answer: For $[1, 2, 3]$ target 3 the answer is 2: $\{3\}$ and $\{1, 2\}$.

TRAP

Zero-weight parcels double the count. With $[0, 0, 1]$ and target 1 there are four subsets: $\{1\}$, $\{0_a, 1\}$, $\{0_b, 1\}$, $\{0_a, 0_b, 1\}$. If your code skips items with $x === 0$ you will report 1 and fail. The loop `for (let s = target; s >= x; s--)` with $x = 0$ runs over every s and correctly doubles each count — leave it alone.

Why is the modulus safe here? Both $\text{dp}[s]$ and $\text{dp}[s - x]$ stay below $10^9 + 7$, so their sum stays below 2×10^9 — far under 2^{53} . If the modulus were near 2^{53} instead, the **addition** would still be fine but any **multiplication** would lose precision, and you would need `BigInt`.

Example 18 [Grab · pattern]

Split all n parcels into two trucks, group S_1 and group S_2 , so that $\text{sum}(S_1) - \text{sum}(S_2) = D$ exactly. Count the number of such splits, modulo $10^9 + 7$. $1 \leq n \leq 100$, $0 \leq a[i] \leq 1000$, $0 \leq D \leq \text{total}$. Edge cases: D larger than the total; total and D of different parity; zero weights.

SOLUTION

Turn it into Example 17 with algebra. Let T be the total.

$$\text{sum}(S_1) + \text{sum}(S_2) = T, \quad \text{sum}(S_1) - \text{sum}(S_2) = D$$

Subtracting: $2 \cdot \text{sum}(S_2) = T - D$, so

$$\text{sum}(S_2) = \frac{T - D}{2}$$

Two guards fall out immediately:

- if $T - D < 0 \rightarrow$ answer 0
- if $T - D$ is odd $\rightarrow$ answer 0 (a sum of integers cannot be a half-integer)

Otherwise just count subsets with that sum.

countWithDifference

JAVASCRIPT

```
function countWithDifference(a, D) {
  const total = a.reduce((s, x) => s + x, 0);
  const t2 = total - D; // 2 * sum(S2)
  if (t2 < 0 || t2 % 2 !== 0) return 0;
  return countSubsetsWithSum(a, t2 / 2);
}
```

Outputs: [1,1,2,3] with $D = 1 \rightarrow 3$; [1,2,3,1] with $D = 3 \rightarrow 2$; [1] with $D = 1 \rightarrow 1$ and with $D = 2 \rightarrow 0$; [0,0,1] with $D = 1 \rightarrow 4$; [] with $D = 0 \rightarrow 1$.

Time $O(n \cdot T)$ **Space** $O(T)$

Answer: 3 for [1,1,2,3], $D = 1$: the small truck can carry $\{3\}$, $\{1_a, 2\}$ or $\{1_b, 2\}$.

SHORTCUT

Any “difference equals D ” question is a “sum equals $\frac{T-D}{2}$ ” question. Do the algebra on paper **before** touching the keyboard — it converts a scary problem into one you already solved.

Example 19 [Agoda · pattern]

Split the parcels into two trucks so that the **difference of their weights is as small as possible**. Return that smallest difference. $1 \leq n \leq 100$, $0 \leq a[i] \leq 200$. Edge cases: empty array; one item; all zeros.

SOLUTION

Let T be the total. If one truck carries s , the other carries $T - s$, and the difference is $|T - 2s|$. To make it small, push s as close to $\frac{T}{2}$ as you can **from below**.

1. Mark every reachable subset sum with a boolean DP (Example 6).
2. Scan $s = 0.. \frac{T}{2}$ and take the best $T - 2s$ among reachable sums.

minSubsetDiff

JAVASCRIPT

```
function minSubsetDiff(a) {
  const total = a.reduce((s, x) => s + x, 0);
  const dp = new Array(total + 1).fill(false);
  dp[0] = true;
  for (const x of a)
    for (let s = total; s >= x; s--)
      if (dp[s - x]) dp[s] = true;
  let best = Infinity;
```

```

for (let s = 0; s <= Math.floor(total / 2); s++)
    if (dp[s]) best = Math.min(best, total - 2 * s);
return best;
}

```

Outputs: [1,6,11,5] → 1; [1,2,7] → 4; [8] → 8; [] → 0; [3,3] → 0; [0,0,0] → 0.

Time $O(n \cdot T)$ **Space** $O(T)$
 $T \leq 100 \times 200 = 20000$

Answer: 1 for [1, 6, 11, 5]: split into {1, 5, 6} = 12 and {11} = 11.

dp[0] is always true, so the loop always finds at least $s = 0$ and best is never left at Infinity. That is why no extra guard is needed for the empty array.

Example 20 [SCB · pattern]

A house painter must paint n houses in a row using 3 colours. $\text{cost}[i][c]$ is the price of painting house i in colour c . Neighbouring houses must differ in colour. Find the cheapest total. $1 \leq n \leq 10^5$, $0 \leq \text{cost} \leq 10^4$. Edge cases: $n = 0$; $n = 1$; all costs equal.

SOLUTION

The state must remember **the last colour used**, so it is a triple, not a single number.

- a = cheapest cost for houses $0..i$ where house i is colour 0
- b = same, colour 1
- c = same, colour 2

$$a' = \text{cost}[i][0] + \min(b, c)$$

and similarly for the other two.

minPaint

JAVASCRIPT

```

function minPaint(cost) {
    if (cost.length === 0) return 0;
    let [a, b, c] = cost[0];
    for (let i = 1; i < cost.length; i++) {
        const na = cost[i][0] + Math.min(b, c);
        const nb = cost[i][1] + Math.min(a, c);
        const nc = cost[i][2] + Math.min(a, b);
        a = na; b = nb; c = nc;
    }
    return Math.min(a, b, c);
}

```

Outputs: [[17,2,17],[16,16,5],[14,3,19]] → 10; [[7,6,2]] → 2; [] → 0; [[1,1,1],[1,1,1]] → 2.

Time $O(n)$ **Space** $O(1)$

Answer: 10 — colours 1, 2, 1 with costs 2 + 5 + 3.

TRAP

na , nb , nc must all be computed **before** any of a , b , c is overwritten. Writing $a = \text{cost}[i][0] + \text{Math.min}(b, c)$; $b = \text{cost}[i][1] + \text{Math.min}(a, c)$; uses the **new** a in the second line and gives a wrong (and very hard to spot) answer. The destructuring line `let [a, b, c] = cost[0];` is safe because the right-hand side is fully evaluated first.

Example 21 [Sea/Shopee · pattern]

Daily percentage changes are stored as integers. Find the **largest product** of any contiguous block. $1 \leq n \leq 10^5$, $|a[i]| \leq 10$ — and then the same question with much larger values, where the product stops being exact. Edge cases: zeros; all negative; a single element.

SOLUTION

Sum problems (Kadane) only need the best-so-far. Products need **two** running values, because a very negative product times a negative number becomes very positive.

- `hi` = largest product of a block ending here
- `lo` = smallest (most negative) product of a block ending here

$$hi' = \max(x, hi \cdot x, lo \cdot x), \quad lo' = \min(x, hi \cdot x, lo \cdot x)$$

maxProductSubarray

JAVASCRIPT

```
function maxProductSubarray(a) {
  if (a.length === 0) return 0;
  let best = a[0], hi = a[0], lo = a[0];
  for (let i = 1; i < a.length; i++) {
    const x = a[i];
    const nhi = Math.max(x, hi * x, lo * x);
    const nlo = Math.min(x, hi * x, lo * x);
    hi = nhi; lo = nlo;
    best = Math.max(best, hi);
  }
  return best;
}
```

Outputs: `[2,3,-2,4] → 6`; `[-2,0,-1] → 0`; `[] → 0`; `[-3] → -3`; `[-2,-3,-4] → 12`;
`[100000,100000,100000] → 1000000000000000`; `[0,0,0] → 0`.

Time $O(n)$ **Space** $O(1)$

Answer: 6 for `[2, 3, -2, 4]` (the block `2, 3`). Note `4` alone is only `4` and `2 · 3 · (-2) · 4 = -48`.

TRAP

`[100000,100000,100000]` gives 10^{15} , which is still exact — it is under $2^{53} \approx 9.007 \times 10^{15}$. But `maxProductSubarray([123456789, 987654321])` returns **121932631112635260** while the true product is **121932631112635269**. Nine wrong at the end, no warning, no exception.

The fix when products can be that large: switch the whole DP to `BigInt` (`123456789n * 987654321n` gives the exact value), or ask the interviewer whether the answer is wanted modulo something. Always ask “how big can this product get?” before you write `*`.

Example 22 [LINE MAN · pattern]

A menu has words. Given a string with no spaces, can it be split into a sequence of menu words? Then print one valid split. $1 \leq |s| \leq 300$, dictionary up to 1000 words. Edge cases: empty string; a word that is a prefix of another; impossible strings.

SOLUTION

Approach 1 — Try every cut position recursively ($O(2^n)$)

Approach 2 — DP over prefixes ($O(n^2 \times L)$) — optimal enough

State: `dp[i]` = true if the first `i` characters can be split.

$$dp[i] = \text{OR}_{0 \leq j < i} (dp[j] \text{ AND } s[j..i-1] \in \text{dict})$$

Base: $dp[0] = \text{true}$ — the empty prefix always splits (into zero words).

wordBreak — a Set, not a plain object

JAVASCRIPT

```
function wordBreak(s, dict) {
  const words = new Set(dict);
  const n = s.length;
  const dp = new Array(n + 1).fill(false);
  dp[0] = true; // the empty prefix always breaks
  for (let i = 1; i <= n; i++)
    for (let j = 0; j < i; j++)
      if (dp[j] && words.has(s.slice(j, i))) { dp[i] = true; break; }
  return dp[n];
}
```

To print a split, store **where the last word started**.

oneSentence — rebuild the split from cut[]

JAVASCRIPT

```
function oneSentence(s, dict) {
  const words = new Set(dict);
  const n = s.length;
  const dp = new Array(n + 1).fill(false), cut = new Array(n + 1).fill(-1);
  dp[0] = true;
  for (let i = 1; i <= n; i++)
    for (let j = 0; j < i; j++)
      if (dp[j] && words.has(s.slice(j, i))) { dp[i] = true; cut[i] = j; break; }
  if (!dp[n]) return '(impossible)';
  const parts = [];
  for (let i = n; i > 0; i = cut[i]) parts.push(s.slice(cut[i], i));
  return parts.reverse().join(' ');
}
```

Outputs:

string	dictionary	ok	split printed
carton	['cart', 'on', 'car', 'ton']	true	car ton
abcd	['ab', 'abc']	false	(impossible)
" "	['a']	true	(empty line)
aaaaa	['aa', 'aaa']	true	aa aaa
x	[]	false	

Time $O(n^2L)$ **Space** $O(n + \text{dict})$

L = average word length, from hashing each substring

Answer: carton splits as car ton (and also as cart on; either is accepted).

TRAP

Use a Set, not a plain object, for the dictionary. A plain object inherits keys from `Object.prototype`, so `dict['constructor']` is truthy even when the word "constructor" is not in your list — "constructor"

`in {}` is `true`. A `Set` has no inherited members, keeps the type of what you put in, and `has()` is the honest test. The same argument applies to frequency counting: prefer `Map` over `{}`.

`dp[0] = true` is not a special case you bolt on — it is the honest answer to “can an empty string be written as zero words?” Yes, it can. Every counting and feasibility DP starts from that same idea.

Example 23 [Razer · pattern]

Bounded knapsack. Item i has weight $w[i]$, value $v[i]$ and **exactly** $\text{cnt}[i]$ copies in stock. Fill a bag of capacity cap for maximum value. $1 \leq n \leq 100$, $0 \leq \text{cnt}[i] \leq 10^4$, $\text{cap} \leq 10^4$. Edge cases: $\text{cnt}[i] = 0$; no items; $\text{cap} = 0$.

SOLUTION

Approach 1 — Expand every copy into its own item ($O(\text{cap} \times \sum \text{cnt}) \sim 10^8$, too slow)

If $\text{cnt}[i]$ is 10000 for 100 items, you create a million 0/1 items. Too slow.

Approach 2 — Binary splitting ($O(\text{cap} \times \sum \log \text{cnt})$ — optimal)

Here is the trick. Any count k can be written using pieces of size 1, 2, 4, 8, ..., remainder. With $k = 11$ the pieces are 1, 2, 4, 4. Every number from 0 to 11 is the sum of some of those pieces, and no number above 11 is. So replace item i by about $\log_2 \text{cnt}[i]$ **fused** items and run plain 0/1 knapsack.

boundedKnap

JAVASCRIPT

```
function boundedKnap(w, v, cnt, cap) {
  const W = [], V = [];
  for (let i = 0; i < w.length; i++) {
    let k = cnt[i], piece = 1;
    while (k > 0) {
      const use = Math.min(piece, k);
      W.push(w[i] * use);
      V.push(v[i] * use);
      k -= use;
      piece *= 2;
    }
  }
  const dp = new Array(cap + 1).fill(0);
  for (let i = 0; i < W.length; i++)
    for (let c = cap; c >= W[i]; c--)
      dp[c] = Math.max(dp[c], V[i] + dp[c - W[i]]);
  return dp[cap];
}
```

Checked against the slow “expand every copy” version: $w=[2,3]$, $v=[4,5]$, $\text{cnt}=[3,2]$, $\text{cap}=10$ gives **18** from both. $\text{cnt} = [0]$ gives 0 from both. Empty input gives 0. A randomised comparison over 300 inputs printed 300 random cross-checks passed.

Time $O(\text{cap} \cdot \sum \log \text{cnt}[i])$ **Space** $O(\text{cap})$

Answer: 18 — three weight-2 items (12) plus two weight-3 items (10) is weight 12, too heavy; the best legal pack is two weight-2 (8) plus two weight-3 (10), weight 10, value 18.

SHORTCUT

Why 1, 2, 4, ..., r ? Because binary. Every integer $0 \leq m \leq k$ has a binary form, and the leftover piece r covers the top of the range. You get **exactly** the numbers $0..k$ and nothing more — which is precisely the “at most $\text{cnt}[i]$ copies” rule.

Tier 3 — insight needed

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Google · pattern]

LIS again, but n is up to 2×10^5 . The $O(n^2)$ solution will time out. Find the length in $O(n \log n)$. Edge cases: empty array; all equal; strictly decreasing.

SOLUTION

Approach 1 — DP ending at i ($O(n^2)$ — 4×10^{10} steps, far too slow)

Approach 2 — Keep the best tail for every length ($O(n \log n)$ — optimal)

Here is the insight. For each possible length L , we only care about **the smallest value that can end an increasing subsequence of length L** . Call it `tails[L-1]`.

Why smallest? Because a smaller ending value is never worse — anything that can extend a big ending can also extend a small one.

Two facts make this work:

1. `tails` is always sorted increasing. (A length-5 chain contains a length-4 chain ending at a smaller value.)
2. For a new value x , find the first tail that is $\geq x$ and **overwrite it** with x . If no tail is $\geq x$, then x extends the longest chain, so append it.

Step 2 is exactly a lower-bound search. JavaScript has no `lower_bound` in the standard library, so we use `lowerBound` from the JS Toolkit appendix.

lisFast — strictly increasing

JAVASCRIPT

```
const { lowerBound } = require('./toolkit.js');

function lisFast(a) {
  const tails = []; // tails[k] = smallest tail of a length
  k+1 chain
  for (const x of a) {
    const i = lowerBound(tails, x);
    if (i === tails.length) tails.push(x);
    else tails[i] = x;
  }
  return tails.length;
}
```

Trace on [10, 9, 2, 5, 3, 7, 101, 18]:

step	x	tails after	what happened
1	10	10	append (first value)
2	9	9	9 replaces 10 — cheaper length-1 ending
3	2	2	2 replaces 9
4	5	2 5	5 > 2, append: a length-2 chain exists
5	3	2 3	3 replaces 5 — cheaper length-2 ending
6	7	2 3 7	append: length 3

7	101	2 3 7 101	append: length 4
8	18	2 3 7 18	18 replaces 101

Final size 4 — matching the $O(n^2)$ answer from Example 15.

Outputs: [10,9,2,5,3,7,101,18] → 4; [7,7,7] → 1; [] → 0; [4,3,2,1] → 1. A randomised comparison of `lisFast` against the $O(n^2)$ version on 300 arrays printed 300 random cross-checks passed.

Time $O(n \log n)$ **Space** $O(n)$

Answer: 4.

The follow-up the interviewer asks next

“Now make it non-decreasing (ties allowed).”

Change `lowerBound` (first element $\geq x$) to `upperBound` (first element $> x$). That single word lets an equal value sit next to an equal value.

`lndsFast` — non-decreasing, one word changed

JAVASCRIPT

```
const { upperBound } = require('./toolkit.js');

function lndsFast(a) {
  const tails = [];
  for (const x of a) {
    const i = upperBound(tails, x);
    if (i === tails.length) tails.push(x);
    else tails[i] = x;
  }
  return tails.length;
}
```

Outputs: [10,9,2,5,3,7,101,18] → 4 (unchanged); [7,7,7] → 3 (was 1).

TRAP

Two JavaScript-specific points on this problem.

(1) There is no ordered set. C++ has an ordered set with `lower_bound`; Java has `TreeMap`. JavaScript has neither. The working pattern is exactly what this solution does: keep a **sorted plain array** and binary-search it with the toolkit’s `lowerBound` / `upperBound`.

(2) `tails` is not an LIS. After the trace above it holds 2 3 7 18. Only its **length** is meaningful; the contents are a mixture of different chains. To print an actual LIS you must store parent pointers as in Example 15.

Example 25 [Adobe · pattern]

Count **how many** longest increasing subsequences an array has. $1 \leq n \leq 2000$. Edge cases: all equal (answer is n , not 1); empty array; single element.

SOLUTION

Run the $O(n^2)$ LIS, but carry a second array.

- `len[i]` = length of the longest increasing subsequence ending at `i`
- `cnt[i]` = how many such subsequences there are

When we look at a smaller index `j` with `a[j] < a[i]`:

- if `len[j] + 1 > len[i]` → we found a **longer** chain. Reset: `len[i] = len[j]+1`, `cnt[i] = cnt[j]`.
- if `len[j] + 1 === len[i]` → we found **another** way to reach the same length. Add: `cnt[i] += cnt[j]`.

Finally sum `cnt[i]` over every `i` whose `len[i]` equals the overall best.

countLIS

JAVASCRIPT

```
function countLIS(a) {
  const n = a.length;
  if (n === 0) return 0;
  const len = new Array(n).fill(1), cnt = new Array(n).fill(1);
  let best = 1;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < i; j++) {
      if (a[j] < a[i]) {
        if (len[j] + 1 > len[i]) { len[i] = len[j] + 1; cnt[i] = cnt[j]; }
        else if (len[j] + 1 === len[i]) cnt[i] += cnt[j];
      }
    }
    best = Math.max(best, len[i]);
  }
  let total = 0;
  for (let i = 0; i < n; i++) if (len[i] === best) total += cnt[i];
  return total;
}
```

Outputs: [1,3,5,4,7] → 2; [2,2,2,2] → 4; [] → 0; [5] → 1; [1,2,4,3,5,4,7,2] → 3.

Time $O(n^2)$ **Space** $O(n)$

Answer: 2 for [1, 3, 5, 4, 7]: 1, 3, 5, 7 and 1, 3, 4, 7.

The follow-up the interviewer asks next

“`n` is now 10^5 — can you still do it?”

Yes, but you must replace the inner `j` loop with a data structure. Keep a Fenwick tree (or segment tree) indexed by **compressed value**, where each node stores the pair [best length, number of ways] for all values below it. Merging two such pairs uses the same “longer resets, equal adds” rule. That gives $O(n \log n)$.

Say this out loud even if you do not code it — recognising that the transition is a **prefix maximum with counts** is the point of the question.

TRAP

[2,2,2,2] answers 4, not 1. The LIS length is 1, and there are four different single-element subsequences. Many candidates return 1 here. Test equal values first.

Example 26 [Amazon · pattern]

You have `n` rectangular boxes given as [width, depth] pairs. Box `B` may sit on box `A` only if **both** $w_B < w_A$ and $d_B < d_A$ (strictly). What is the tallest stack (most boxes)? $1 \leq n \leq 5000$. Edge cases: identical boxes; boxes where one dimension ties; one box.

SOLUTION

Approach 1 — Try all orders ($O(n!)$ — never)

Approach 2 — Sort by width, then LIS on depth ($O(n^2)$, optimal for $n = 5000$)

Two conditions are hard to handle at once. So **remove one of them by sorting**. After sorting by width ascending, a box later in the array can only sit under a box earlier in the array. Now the problem reads: **find the longest chain where depth also increases** — which is LIS with a custom comparison.

We still check width strictly inside the loop, because sorting does not stop two boxes from sharing a width.

longestChain

JAVASCRIPT

```
function longestChain(box) {
  const n = box.length;
  if (n === 0) return 0;
  const b = box.map(p => [...p]); // deep enough copy of the
  pairs
  b.sort((p, q) => p[0] - q[0] || p[1] - q[1]); // NEVER b.sort()
  const dp = new Array(n).fill(1);
  let best = 1;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < i; j++)
      if (b[j][0] < b[i][0] && b[j][1] < b[i][1])
        dp[i] = Math.max(dp[i], dp[j] + 1);
    best = Math.max(best, dp[i]);
  }
  return best;
}
```

Outputs:

boxes	stack	why
[[5,4],[6,4],[6,7],[2,3]]	3	(2,3) < (5,4) < (6,7)
[[1,1],[1,1],[1,1]]	1	ties are not allowed
[]	0	
[[9,9]]	1	
[[1,5],[2,4],[3,3],[4,2],[5,1]]	1	width up means depth down
[[1,2],[2,3],[3,4],[4,5]]	4	everything nests

Time $O(n^2)$ Space $O(n)$

Answer: 3.

The follow-up the interviewer asks next

“Now boxes may be rotated 90°, and n is 10^5 .”

Two separate changes.

1. **Rotation:** for each box push both $[w, d]$ and $[d, w]$ into the list, then solve the same problem on $2n$ boxes. Normalising each box so that $w \leq d$ first removes exact duplicates.
2. **Speed:** sort by width ascending, and for equal widths sort by depth **descending**. Now two boxes with the same width can never both be picked — because their depths are in decreasing order, they can never form an increasing depth pair. That makes the width check unnecessary, and the problem becomes plain LIS on depth, solvable in $O(n \log n)$ with the tails method of Example 24. The comparator becomes $(p, q) \Rightarrow p[0] - q[0] || q[1] - p[1]$.

That tie-breaking trick — **sort the second key downwards so ties self-destruct** — is worth memorising. It appears in many “2D chain” questions.

TRAP

This is the single most common JavaScript interview bug. `Array.prototype.sort()` with no comparator converts every element to a **string** and sorts lexicographically.

`[10, 9, 1].sort()` returns `[1, 10, 9]`.

It is worse for pairs. `[[6,4],[5,4],[10,1],[2,3]].sort()` returns `[[10,1],[2,3],[5,4],[6,4]]` — because `"10,1" < "2,3"` as strings. Your box DP would then run on garbage order and quietly return a wrong number.

Always pass a comparator: $(a, b) \Rightarrow a - b$ for numbers, and $(p, q) \Rightarrow p[0] - q[0] \mid\mid p[1] - q[1]$ for pairs. The `||` trick works because a `0` first-key difference is falsy, so the second key decides.

Example 27 [D. E. Shaw · pattern]

0/1 knapsack, but the weights are huge: $n \leq 100$, $w[i] \leq 10^9$, $\text{cap} \leq 10^9$, and $v[i] \leq 100$. The classic `dp[cap]` table would need 10^9 cells. Edge cases: no items; capacity 0; a single item that does not fit.

SOLUTION

Approach 1 — dp indexed by capacity ($O(n \times \text{cap}) = 10^{11}$ time and 10^9 cells — impossible)

Approach 2 — Flip the table: index by VALUE ($O(n \times \Sigma v) = 10^6$ — optimal)

The insight is a swap of question. Instead of

> “with capacity `c`, what is the best value?”

ask

> “to reach value `val`, what is the **least weight** I need?”

Total value is at most $100 \times 100 = 10^4$, so the table is tiny. Then walk down from the largest value and return the first one whose least weight fits in the bag.

$$\text{dp}[\text{val}] = \min(\text{dp}[\text{val}], \text{dp}[\text{val} - v_i] + w_i)$$

knapsackByValue

JAVASCRIPT

```
function knapsackByValue(w, v, cap) {
  const V = v.reduce((s, x) => s + x, 0);
  const dp = new Array(V + 1).fill(Infinity); // dp[val] = least weight that
  reaches val
  dp[0] = 0;
  for (let i = 0; i < w.length; i++)
    for (let val = V; val >= v[i]; val--)
      if (dp[val - v[i]] < Infinity)
        dp[val] = Math.min(dp[val], dp[val - v[i]] + w[i]);
  for (let val = V; val >= 0; val--)
    if (dp[val] <= cap) return val;
  return 0;
}
```

Outputs with `w = [1000000000, 999999999, 3]` and `v = [3, 4, 2]`:

capacity	best value	chosen items
1000000002	6	weights 999999999 + 3
1999999999	7	weights 1000000000 + 999999999
2	0	nothing fits

Also: empty input with `cap 10` → 0; `w=[4,5,6]`, `v=[1,2,3]` with `cap 11` → 5.

Time $O(n \cdot \sum v_i)$ **Space** $O(\sum v_i)$

Answer: 6 at capacity 1000000002.

The follow-up the interviewer asks next

“What if both the weights and the values are up to 10^9 ?”

Then neither table fits and no polynomial-in-the-input algorithm is known — knapsack is NP-hard in general. The honest answers are:

- **meet in the middle:** split the n items into two halves, list all $2^{\frac{n}{2}}$ subset [weight, value] pairs of each half, sort one half and binary search. Works up to about $n = 40$: $2 \times 2^{20} \approx 2 \times 10^6$ pairs.
- **approximation:** scale all values down by a factor and run the value-indexed DP. You lose a controllable small percentage of the optimum.

Naming “NP-hard, so here are the two standard escapes” is the expected answer. Do not pretend a polynomial algorithm exists.

SHORTCUT

Whenever a DP dimension is too big, look at the **other** quantity in the problem. If the answer range is small, index the table by the answer and store the resource. This “invert the DP” move shows up again in scheduling and in shortest-path problems with small edge weights.

TRAP

Weights here reach 10^9 and sums reach 2×10^9 . In C++ that overflows a 32-bit `int`; in JavaScript it does not, because a `Number` is exact to 2^{53} . This is one of the rare places where JS is **safer** than C++. But do not relax: a **sum of 100 such weights** is 10^{11} , still exact, while a **product** of two of them is 10^{18} and is not. Addition is safe far longer than multiplication.

Example 28 [Goldman Sachs · pattern]

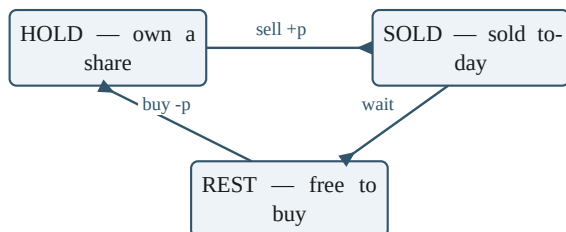
Daily prices of one stock. You may buy and sell any number of times, but you hold at most one share, and **after selling you must rest one full day** before buying again. Maximise profit. $1 \leq n \leq 10^5$. Edge cases: empty; one day; prices only falling.

SOLUTION

Approach 1 — Recursion over (day, holding?, resting?) ($O(2^n)$ without a memo)

Approach 2 — Three-state machine, $O(1)$ space (optimal)

The state is not a number — it is **which situation you are in today**. There are three:



The three states and the moves between them. Buying is only allowed from REST.

- hold = best profit while owning a share
- sold = best profit on a day you just sold (tomorrow you must rest)
- rest = best profit while free and not in cooldown

Transitions for day i with price p :

$$\text{hold}' = \max(\text{hold}, \text{rest} - p), \quad \text{sold}' = \text{hold} + p, \quad \text{rest}' = \max(\text{rest}, \text{sold})$$

maxProfitCooldown

JAVASCRIPT

```
function maxProfitCooldown(p) {
  const n = p.length;
  if (n === 0) return 0;
  let hold = -p[0];           // holding a share
  let sold = -Infinity;       // sold today, so tomorrow is a rest day
  let rest = 0;               // free, not in cooldown
  for (let i = 1; i < n; i++) {
    const nHold = Math.max(hold, rest - p[i]);
    const nSold = hold + p[i];
    const nRest = Math.max(rest, sold);
    hold = nHold; sold = nSold; rest = nRest;
  }
  return Math.max(rest, sold);
}
```

Outputs (second number is the profit with **no** cooldown, for comparison):

prices	cooldown	no rule
[1,2,3,0,2]	3	4
[5,4,3,2,1]	0	0
[]	0	—
[7]	0	—
[1,5]	4	—
[2,1,4,5,2,9,7]	10	11

Time $O(n)$ Space $O(1)$

Answer: 3 for [1,2,3,0,2] — buy at 1, sell at 3, rest, buy at 0, sell at 2.

The follow-up the interviewer asks next

“Drop the cooldown, but now allow at most k transactions.”

The state becomes “how many sales are done so far”, so keep two arrays of size $k + 1$.

maxProfitK — at most k buy/sell pairs

JAVASCRIPT

```
function maxProfitK(p, k) {
  const n = p.length;
  if (n === 0 || k === 0) return 0;
  const hold = new Array(k + 1).fill(-Infinity);
  const sold = new Array(k + 1).fill(0); // sold[j] = free, j sales done
  for (const price of p)
    for (let j = 1; j <= k; j++) {
      hold[j] = Math.max(hold[j], sold[j-1] - price);
      sold[j] = Math.max(sold[j], hold[j] + price);
    }
  return sold[k];
}
```

Outputs: [2, 4, 1] with $k = 2 \rightarrow 2$; [3, 2, 6, 5, 0, 3] with $k = 2 \rightarrow 7$, with $k = 1 \rightarrow 4$, with $k = 9 \rightarrow 7$ (same as unlimited); [] $\rightarrow 0$; [5] $\rightarrow 0$; [5, 4, 3, 2] $\rightarrow 0$; [1, 2, 3, 4, 5] with $k = 1 \rightarrow 4$ and with $k = 2 \rightarrow 4$.

Time $O(nk)$ **Space** $O(k)$

TRAP

Inside the j loop, `hold[j]` is updated **before** `sold[j]` uses it. That is not a bug — it means “buy and sell on the same day”, which earns 0 and never hurts. But if you reversed the two lines you would be using yesterday’s `hold`, and the answer would drop. Know which order you wrote and why.

Also note `-Infinity` as the “impossible” marker. `-Infinity - price` stays `-Infinity`, so an impossible state can never accidentally win a `Math.max`.

Example 29 [Uber · pattern]

An array of scores. You start at index 0 and must reach index $n - 1$. From index i you may jump to any index in $[i + 1, i + k]$. Landing on i adds `a[i]` to your score (including the first and last). Maximise the total score. $1 \leq n \leq 10^5$, $1 \leq k \leq n$, $-10^4 \leq a[i] \leq 10^4$. Edge cases: $n = 1$; all negatives; $k = n - 1$.

SOLUTION

Approach 1 — DP over all previous k positions ($O(nk)$ — $10^4 \cdot 10$ when k is large)

$$dp[i] = a[i] + \max_{i-k \leq j < i} dp[j]$$

Approach 1 — correct but slow (used here as the reference)

JAVASCRIPT

```
function slow(a, k) {
  const n = a.length; if (n === 0) return 0;
  const dp = new Array(n).fill(-Infinity); dp[0] = a[0];
  for (let i = 1; i < n; i++)
    for (let j = Math.max(0, i - k); j < i; j++)
      dp[i] = Math.max(dp[i], dp[j] + a[i]);
  return dp[n-1];
}
```

Time $O(nk)$ **Space** $O(n)$

Approach 2 — Sliding-window maximum with a deque ($O(n)$ — optimal)

The inner loop is **a maximum over a window of the last k values of `dp`**. That is exactly the monotonic deque from Chapter 9. The deque stores indices whose `dp` values are decreasing from front to back, so the front is always the window maximum.

Two rules per step:

1. drop the front while it is older than $i - k$ (out of the window);
2. before pushing i , drop from the back every index whose `dp` is $\leq dp[i]$ — it can never be the maximum again.

maxScoreJumpK

JAVASCRIPT

```
const { Deque } = require('./toolkit.js');

function maxScoreJumpK(a, k) {
  const n = a.length;
  if (n === 0) return 0;
  const dp = new Array(n).fill(-Infinity);
```

```

const dq = new Deque();
dp[0] = a[0]; dq.push(0);
for (let i = 1; i < n; i++) {
  while (dq.size && dq.front() < i - k) dq.shift();
  dp[i] = dp[dq.front()] + a[i];
  while (dq.size && dp[dq.back()] <= dp[i]) dq.pop();
  dq.push(i);
}
return dp[n-1];
}

```

Each index enters and leaves the deque once, so the total deque work is $O(n)$.

Outputs (both methods agree): $[1, -1, -2, 4, -7, 3]$ with $k = 2 \rightarrow 7$; $[10, -5, -2, 4, 0, 3]$ with $k = 3 \rightarrow 17$; $[5] \rightarrow 5$; $[] \rightarrow 0$. A randomised comparison of the two versions over 300 inputs printed 300 random cross-checks passed.

Time $O(n)$ **Space** $O(n)$
 $O(k)$ extra for the deque

Answer: 7 for $[1, -1, -2, 4, -7, 3]$, $k = 2$: indices $0 \rightarrow 1 \rightarrow 3 \rightarrow 5$ giving $1 - 1 + 4 + 3 = 7$.

The follow-up the interviewer asks next

“The array arrives as a stream — you see $a[i]$ only at step i .”

Nothing changes. The deque solution already reads each element once, in order, and keeps only the last k useful entries. Memory is $O(k)$, not $O(n)$, if you also store dp in a circular buffer of size $k + 1$. Streaming-friendliness is a direct consequence of the window being **bounded**; say that, and you have answered the question.

TRAP

Do not use a plain array with `shift()` as a deque. `Array.prototype.shift()` is $O(n)$ in the worst case because every remaining element moves down one slot, which turns this $O(n)$ algorithm back into $O(n^2)$. The toolkit `Deque` keeps a head index and only compacts occasionally, giving $O(1)$ amortised `shift`. JavaScript also has no built-in priority queue; when a problem needs one, use the toolkit `MinHeap`.

SHORTCUT

Whenever a DP transition is “max or min over a contiguous window of previous states”, the fix is always the same: monotonic deque for a fixed window, or a heap / prefix-max array if the window grows. Recognising the shape is worth more than memorising the code.

ONE-PAGE REVISION CARD

The recipe

FIVE STEPS, EVERY TIME

1. Write the brute-force recursion: “at this index I take it or I skip it”.
2. The **changing arguments** of that recursion are the state.
3. Memoise on those arguments → top-down DP.
4. Rewrite as a loop from the base cases outward → bottom-up table.
5. If the loop only reads the previous row, throw the rest of the table away.

The templates

T1 — top-down memo (use it to FIND the recurrence)

JS

```
const climbMemo = (n) => {  
  const memo = new Array(n + 1).fill(-1);  
  const go = (i) => {  
    if (i < 0) return 0;           // impossible  
    if (i === 0) return 1;        // the one empty way  
    if (memo[i] !== -1) return memo[i];  
    return memo[i] = go(i - 1) + go(i - 2);  
  };  
  return go(n);  
};
```

T2 — bottom-up table (SHIP this one)

JS

```
const climb = (n) => {  
  const dp = new Array(n + 1).fill(0);           // never new Array(n) alone  
  dp[0] = 1;                                     // counting base is 1, not 0  
  for (let i = 1; i <= n; i++) {  
    dp[i] = dp[i - 1];  
    if (i >= 2) dp[i] += dp[i - 2];  
  }  
  return dp[n];  
};
```

T3 — 0/1 knapsack, one row, capacity loop BACKWARDS

JS

```
const knap01 = (w, v, cap) => {  
  const dp = new Array(cap + 1).fill(0);  
  for (let i = 0; i < w.length; i++)  
    for (let c = cap; c >= w[i]; c--)           // BACKWARDS = each item once  
      dp[c] = Math.max(dp[c], v[i] + dp[c - w[i]]);  
  return dp[cap];  
};
```

T3' — unbounded knapsack: the same loop, FORWARDS

JS

```
const knapU = (w, v, cap) => {
  const dp = new Array(cap + 1).fill(0);
  for (let i = 0; i < w.length; i++)
    for (let c = w[i]; c <= cap; c++) // FORWARDS = unlimited copies
      dp[c] = Math.max(dp[c], v[i] + dp[c - w[i]]);
  return dp[cap];
};
```

SHORTCUT

One word is the whole difference. Run both on $w = [3, 4]$, $v = [4, 5]$, $cap = 8$: backwards gives **9** (one of each), forwards gives **10** (two copies of item 1). Verified in Node. Write the direction on your rough sheet before you write the loop.

T4 — min-cost with an Infinity sentinel

JS

```
const coinMin = (coins, amt) => {
  const dp = new Array(amt + 1).fill(Infinity);
  dp[0] = 0;
  for (const c of coins)
    for (let x = c; x <= amt; x++) dp[x] = Math.min(dp[x], dp[x - c] + 1);
  return dp[amt] === Infinity ? -1 : dp[amt];
};
```

T5 — take / skip in two variables, O(1) space

JS

```
const rob = (a) => { // no two neighbours
  let take = 0, skip = 0;
  for (const x of a) {
    const t = skip + x;
    skip = Math.max(skip, take);
    take = t;
  }
  return Math.max(take, skip);
};
```

T6 — LIS in O(n log n) with the toolkit lowerBound

JS

```
const { lowerBound } = require('./toolkit.js'); // JS Toolkit appendix

const lis = (a) => {
  const tails = []; // tails[k] = smallest tail of
  for (const x of a) { // an increasing run of length k+1
    const i = lowerBound(tails, x);
    if (i === tails.length) tails.push(x); else tails[i] = x;
  }
  return tails.length; // LENGTH only – not the
  subsequence
};
```

All six were run in Node. `climb(5) = 8`; `knap01([1,3,4,5],[1,4,5,7],7) = 9`; `coinMin([1,3,4],6) = 2` and `coinMin([2],3) = -1`; `rob([2,7,9,3,1]) = 12`; `lis([10,9,2,5,3,7,101,18]) = 4`. Empty input returns 0 in every one of them.

When to reach for which

The question says	Template	Time
count the ways to reach step n	T2, + combiner, <code>dp[0] = 1</code>	$O(n)$
minimum cost to reach the end	T4, Infinity sentinel	$O(n)$ or $O(nk)$
pick items, no two neighbours	T5	$O(n)$
same, but in a circle	T5 twice: drop first, drop last	$O(n)$
fill a bag, each item once	T3 — backwards	$O(nW)$
unlimited copies of each item	T3' — forwards	$O(nW)$
exactly <code>cnt[i]</code> copies	binary-split into powers of two, then T3	$O(nW \log \text{cnt})$
split the array into two equal halves	subset sum on <code>total/2</code> (T3, boolean)	$O(nS)$
smallest difference between two groups	subset sum, then scan from the middle	$O(nS)$
count subsets with a given weight	T3 with + instead of max	$O(nS)$
can this string be cut into dictionary words	T2 over prefixes + a Set	$O(n^2)$
longest increasing subsequence	T6 (or $O(n^2)$ T2 if n is small)	$O(n \log n)$
weights up to 10^9 , values small	index the table by value , not weight	$O(n \sum v)$
max / min over a sliding window of states	T2 + monotonic Deque	$O(n)$

Complexity you should be able to quote

THE ONLY DP COST FORMULA

$$\text{time} = (\text{number of states}) \times (\text{work per state})$$

$$\text{space} = (\text{number of states you must keep alive})$$

Problem	Time	Space
staircase / Fibonacci-shaped	$O(n)$	$O(1)$
house robber, straight or circular	$O(n)$	$O(1)$
frog with k jumps	$O(nk)$	$O(n)$
coin change, min coins or count of ways	$O(nA)$	$O(A)$
0/1 knapsack	$O(nW)$	$O(W)$
unbounded knapsack	$O(nW)$	$O(W)$
bounded knapsack (binary splitting)	$O(nW \log \text{cnt})$	$O(W)$
subset sum / equal partition	$O(nS)$	$O(S)$
decode-ways / word-break	$O(n) / O(n^2)$	$O(n)$
LIS, table version	$O(n^2)$	$O(n)$

LIS, tails + binary search	$O(n \log n)$	$O(n)$
knapsack indexed by value	$O(n \sum v)$	$O(\sum v)$

A is the amount, W the capacity, S the target sum. None of these are polynomial in the **input size** — they are polynomial in the **value** of a number in the input. That is what “pseudo-polynomial” means, and interviewers ask for the word.

Top traps, in the order they bite

1. **dp[0] = 0 in a counting DP.** There is exactly **one** way to make 0 — take nothing. Set `dp[0] = 1` or every answer collapses to 0.
2. **Knapsack loop direction.** Backwards for 0/1, forwards for unbounded. Forwards on a 0/1 problem silently reuses an item and the answer comes out too large.
3. **new Array(n) without .fill(0).** That is an array of **holes**. `map` skips holes, and `dp[i] += x` on a hole gives NaN.
4. **new Array(3).fill(new Array(4).fill(0))** stores **the same row three times**. Setting `grid[0][1] = 9` changes all three. Use `Array.from({ length: 3 }, () => new Array(4).fill(0))`.
5. **Recursion depth.** JavaScript gives you roughly 10^4 frames. A top-down DP over $n = 10^5$ throws `RangeError: Maximum call stack size exceeded`. Write top-down to find the recurrence, then ship the loop.
6. **Counting past $2^{53} - 1$.** Measured: `climb(77) = 8944394323791464` is exact; `climb(79) = 23416728348467684` is **wrong** by one. Take the answer modulo $10^9 + 7$, or switch the whole table to `BigInt`.
7. **A finite sentinel instead of Infinity.** A large finite number can be beaten by a real answer, or be relaxed by accident. `Infinity + cost` is still `Infinity`; it cannot be picked by a `Math.min` unless nothing else exists.
8. **Loop order in a counting DP.** Coins outside, amount inside counts **combinations** (`{1, 2}` once). Amount outside, coins inside counts **permutations** (`1, 2` and `2, 1` separately). The question tells you which — read it twice.
9. **tails in the LIS template is not the LIS.** Only its length is meaningful. To recover the actual subsequence you must store predecessor indices as well.
10. **Forgetting the “no solution” check.** `dp[amt] === Infinity ? -1 : dp[amt]`. Without it you print `Infinity` and lose the mark.
11. **A plain array with shift() as a deque** in a windowed DP. `shift()` is $O(n)$ worst case and turns an $O(n)$ DP back into $O(n^2)$. Use the toolkit `Deque`.

The 30-second checklist before you code

1. What is the state? Name it in one sentence: “`dp[i]` is the best answer using the first `i` items.”
2. What is the base case, and is it 0 or 1?
3. Optimisation or counting? That fixes the combiner: `Math.max/Math.min` or `+`.
4. Which earlier states does `dp[i]` read? That fixes the loop order.
5. Number of states times work per state — is that under 10^8 ?
6. Can the answer pass $2^{53} - 1$? If yes, modulo or `BigInt` from the start.
7. Does the table only read the previous row? Then collapse it to one row.

Dynamic Programming II — 2D, Strings, Trees & Bitmask

Same recipe, bigger state

Pattern in one page

Chapter 17 had one moving index. This chapter has two, or three, or a whole set. Nothing about the method changes — only the **shape of the state**.

THE SAME THREE QUESTIONS, EVERY TIME

1. **State** — the changing arguments of the brute-force recursion.
2. **Transition** — how a bigger state is built from smaller ones.
3. **Base case** — the states you know for free.

Then: memo → table → shrink the memory.

The five state shapes in this chapter

Shape	State	Typical size	Example
Grid	$dp[i][j]$ = cell	mn	count paths, min path sum
Two strings	$dp[i][j]$ = prefixes	nm	LCS, edit distance
One string, interval	$dp[i][j]$ = the piece $i..j$	n^2 , work $O(n)$	palindrome cuts, matrix chain
Tree	$dp[u][0/1]$ = subtree of u	n	robber on a tree
Subsets	$dp[mask]$	2^n	assignment, TSP

Template 1 — grid

Grid DP, one row of memory

JS

```
const row = new Array(n).fill(1);
for (let i = 1; i < m; i++)
  for (let j = 1; j < n; j++)
    row[j] += row[j-1];      // row[j] is still row i-1; row[j-1] is already row i
return row[n-1];
```

Read the rolling-row trick carefully. When the loop reaches $row[j]$, that cell still holds the value from **the previous row** (it has not been written yet this pass), while $row[j-1]$ was already updated **this row**. So $row[j] += row[j-1]$ is exactly $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. No extra array is needed.

Template 2 — two strings

The LCS / edit-distance skeleton

JS

```
const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
for (let i = 1; i <= n; i++)
  for (let j = 1; j <= m; j++)
    if (a[i-1] === b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;      // characters agree
    else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
```

TRAP

Build a 2D table with `Array.from`, never with `fill`. `new Array(n+1).fill(new Array(m+1).fill(0))` puts the **same** row object in all $n + 1$ slots, so `dp[3][2] = 5` writes row 0, row 1 and every other row at once. The code runs and the numbers are quietly wrong. `Array.from({ length: n+1 }, () => new Array(m+1).fill(0))` calls the factory once per row.

The same rule governs copying: `[...grid]` shares every row. A real 2D copy is `grid.map(r => [...r])`. Every table in this chapter is built with `Array.from`.

Indexing a JavaScript string with `a[i-1]` gives a **one-character string**, and `===` compares those by value — so `a[i-1] === b[j-1]` is exactly the character comparison you want. There is no `char` type and no need for `charAt` unless you are doing arithmetic on letters.

TRAP

Index shift. `dp[i][j]` talks about the **first i** characters, so the character is `a[i-1]`, not `a[i]`. Mixing these two up is the number-one 2D DP bug. Pick the i means “count of characters” convention and never change it.

Template 3 — interval (partition) DP

Used when you must **split a sequence at some point k** and the cost of the split depends on the whole piece.

Interval DP skeleton — always loop by LENGTH

JS

```
for (let len = 2; len <= n; len++)
  for (let i = 0; i + len - 1 < n; i++) {
    const j = i + len - 1;
    dp[i][j] = Infinity;
    for (let k = i; k < j; k++)
      dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k+1][j] + costOfJoining(i, k, j));
  }
```

`Infinity` replaces `INT_MAX` throughout this chapter. It is a real number in JavaScript: `Math.min(Infinity, 5)` is 5, `Infinity + 1` is `Infinity`, and no sentinel can ever overflow into a smaller value the way `INT_MAX + 1` does in C+. Use `-Infinity` for a “smallest so far” accumulator.

SHORTCUT

Why loop by length? Because `dp[i][j]` needs `dp[i][k]` and `dp[k+1][j]`, and both are **shorter** intervals. Going by length guarantees every shorter interval is already finished. Looping i then j in the usual order does not.

Template 4 — tree DP

A tree has no cycles, so “the answer for a subtree” is well defined. Do one DFS and return the numbers the parent needs.

Tree DP skeleton — return a pair, one value per situation

JS

```
const dfs = (u) => {
  let withU = val[u], withoutU = 0;
  for (const v of children[u]) {
    const [t, s] = dfs(v);           // destructure the pair the child returned
    withU      += s;                 // if u is taken, children must be skipped
    withoutU += Math.max(t, s);      // if u is skipped, children are free
  }
  return [withU, withoutU];
};
```

TRAP

Node’s call stack is about 10^4 frames deep — far shallower than C++ or Java. Every tree DP in this chapter recurses once per node, and the constraints say $n \leq 10^5$. A tree that is one long chain of 100000 nodes makes this dfs throw `RangeError: Maximum call stack size exceeded`; that was run, not guessed.

The fix is an explicit stack: push `[node, childIndex]` frames, do the child work on the way down, and combine on the way up. Say this out loud before you write the recursive version — “recursive here for clarity; on a 10^5 chain I would convert it to an iterative post-order” — and you have answered the follow-up before it is asked.

Template 5 — bitmask DP

When $n \leq 20$ and the state is “which items are already used”, the state is a bitmask.

JavaScript has no `__builtin_popcount`, so write the two-line Kernighan loop once and reuse it. `m &= m - 1` clears the lowest set bit, so the loop runs once per set bit.

Bitmask skeleton — fill positions in a fixed order

JS

```
const popcount = (m) => { let c = 0; while (m) { m &= m - 1; c++; } return c; };

const dp = new Array(1 << n).fill(Infinity);
dp[0] = 0;
for (let mask = 0; mask < (1 << n); mask++) {
  if (dp[mask] === Infinity) continue;
  const i = popcount(mask);           // the next slot to fill
  if (i === n) continue;
  for (let j = 0; j < n; j++)
    if (!(mask & (1 << j)))
      dp[mask | (1 << j)] = Math.min(dp[mask | (1 << j)], dp[mask] + cost[i][j]);
}
```

TRAP

JavaScript bitwise operators work on 32-bit signed integers, even though numbers are doubles. `1 << 31` is negative and `1 << 32` is 1, not 2^{32} . This chapter caps masks at $n \leq 18$, so `1 << n` is at most 262144 and everything is safe — but if a problem ever pushes past 31 bits, bit operations stop working and you need `BigInt` or a pair of masks. Say the limit out loud; it is a favourite follow-up.

BIT TRICKS YOU WILL NEED

- `mask & (1 << j)` — is item `j` in the set?
- `mask | (1 << j)` — add item `j`
- `mask ^ (1 << j)` — flip item `j`

- `mask & -mask` — the lowest set bit, on its own
- `popcount(mask)` — how many bits are set (the Kernighan loop above; JavaScript has no built-in)
- `31 - Math.clz32(lowBit)` — index of the lowest set bit, the `__builtin_ctz` of other languages
- loop over all sub-sets of `mask`: `for (let s = mask; s; s = (s-1) & mask)`

All verified: `popcount(0b1011)` is 3, `popcount(0)` is 0, `31 - Math.clz32(0b1000)` is 3, and the sub-set loop over `0b1011` yields exactly its 7 non-empty subsets.

Choosing the shape from the question

The question says...	Shape	Cost
move only right/down on a grid	grid	$O(mn)$
compare / align two sequences	two strings	$O(nm)$
insert brackets, cut, merge, burst	interval	$O(n^3)$
subtree, parent, child, no two adjacent nodes	tree	$O(n)$
$n \leq 20$ and “assign each to each”	bitmask	$O(2^n n)$ or $O(2^n n^2)$
$n \leq 20$ and “split into groups”	bitmask + submask	$O(3^n)$

3^n comes from summing $2^{|\text{mask}|}$ over all masks. For $n = 15$ that is about 14 million — fine. For $n = 20$ it is 3.5 billion — too slow.

Warm-up

WARM-UP

build the reflex

Example 1 [warm-up]

A robot starts at the top-left of an $m \times n$ grid and may move only right or down. How many paths reach the bottom-right? $1 \leq m, n \leq 18$. Edge cases: a single row; a single cell; a zero-sized grid.

SOLUTION

To stand on cell (i, j) the robot arrived from above or from the left.

$$\text{dp}[i][j] = \text{dp}[i-1][j] + \text{dp}[i][j-1]$$

The whole first row and first column are 1 — there is only one way along an edge.

uniquePaths — one row of memory

JS

```
const uniquePaths = (m, n) => {
  if (m <= 0 || n <= 0) return 0;
  const row = new Array(n).fill(1);
  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      row[j] += row[j-1];
  return row[n-1];
};
```

Outputs: $(3, 7) \rightarrow 28$; $(3, 2) \rightarrow 3$; $(1, 1) \rightarrow 1$; $(1, 10) \rightarrow 1$; $(0, 5) \rightarrow 0$; $(2, 2) \rightarrow 2$; $(17, 17) \rightarrow 601080390$.

Time $O(mn)$ **Space** $O(n)$

Answer: 28.

TRAP

Counting DP is where JavaScript numbers run out. There is no `int` to overflow — numbers are doubles, exact only to `Number.MAX_SAFE_INTEGER =  $2^{53} - 1 \approx 9.0 \times 10^{15}$` . Everything was measured:

<code>uniquePaths(17, 17)</code>	601080390	exact
<code>uniquePaths(29, 29)</code>	7648690600760440	exact, still a safe integer
<code>uniquePaths(32, 32)</code>	465428353255261060	wrong — the true value is 465428353255261088

Past that line the answer is `BigInt`: seed with `new Array(n).fill(1n)` and the identical loop is exact at any size. The cost is roughly a 10x slowdown and the rule that `BigInt` and `Number` never mix in one expression — `1n + 1` throws a `TypeError`.

Most problems dodge the issue by asking for the count **modulo** $10^9 + 7$. That makes **addition** safe, because two residues sum to under 2×10^9 . It does **not** make multiplication safe: $(999999937 * 999999893) \% (10^9 + 7)$ gives 8023 in plain numbers and 7980 with `BigInt` — the plain answer is simply wrong, because the product is about 10^{18} . A modular DP that multiplies needs `BigInt` for the multiply, or a modulus small enough that the product stays under 2^{53} (the 1003 used later in this chapter is such a modulus: $1002 * 1002 * 4$ is tiny).

Example 2 [warm-up]

Same grid, but some cells hold a wall (1). Walls cannot be entered. Count the paths. Edge cases: the start cell is a wall; the finish cell is a wall; a full row of walls.

SOLUTION

One extra rule: a wall cell has **zero** ways.

pathsWithObstacles

JS

```
const pathsWithObstacles = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  const dp = new Array(n).fill(0);
  dp[0] = g[0][0] === 1 ? 0 : 1;
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
      if (g[i][j] === 1) { dp[j] = 0; continue; }
      if (j > 0) dp[j] += dp[j-1];
    }
  return dp[n-1];
};
```

Outputs:

grid	paths	note
[[0,0,0],[0,1,0],[0,0,0]]	2	wall in the middle
[[0,1],[0,0]]	1	only down-then-right
[[1]]	0	start is a wall
[[0]]	1	start = finish
[[0,0],[1,1],[0,0]]	0	a full wall row blocks every route
[] (no rows)	0	

Time $O(mn)$ **Space** $O(n)$

Answer: 2.

Example 3 [warm-up]

Each cell of the grid holds a non-negative cost. Moving only right or down, find the cheapest path from top-left to bottom-right. Edge cases: single cell; single row; single column.

SOLUTION

$$dp[i][j] = g[i][j] + \min(dp[i-1][j], dp[i][j-1])$$

Using one rolling row, $dp[j]$ is the cell above and $dp[j-1]$ is the cell to the left.

minPathSum

JS

```
const minPathSum = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  const dp = new Array(n).fill(Infinity);
  dp[0] = 0;
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
```

```

    if (j > 0) dp[j] = Math.min(dp[j], dp[j-1]);
    dp[j] += g[i][j];
  }
  return dp[n-1];
};

```

Outputs: $[[1,3,1],[1,5,1],[4,2,1]] \rightarrow 7$; $[[5]] \rightarrow 5$; $[[1,2,3]] \rightarrow 6$; $[[1],[2],[3]] \rightarrow 6$; $[] \rightarrow 0$.

Time $O(mn)$ **Space** $O(n)$

Answer: 7 — the path $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$.

TRAP

Re-setting $dp[0] = \text{Infinity}$ at the start of each row would be wrong for column 0, where there is no “left”. The code sets $dp[0] = 0$ once, **before** the loop, and never touches $dp[j-1]$ when $j === 0$. Check that guard exists in your own version — and note that $\text{Infinity} + 5$ is still Infinity , so a missing guard shows up as an Infinity answer rather than as a wrapped-around negative number. That is a mercy: the bug is visible.

Example 4 [warm-up]

A triangle of numbers: row i has $i+1$ entries. From entry j of row i you move to entry j or $j+1$ of row $i+1$. Find the smallest top-to-bottom sum. Edge cases: one row; negative numbers; empty triangle.

SOLUTION

Going **bottom-up** removes all boundary checks, because the last row is the answer for itself.

$$dp[j] = t[i][j] + \min(dp[j], dp[j+1])$$

triangleMin

JS

```

const triangleMin = (t) => {
  const m = t.length; if (!m) return 0;
  const dp = [...t[m-1]]; // COPY the last row, do not alias it
  for (let i = m - 2; i >= 0; i--)
    for (let j = 0; j <= i; j++)
      dp[j] = t[i][j] + Math.min(dp[j], dp[j+1]);
  return dp[0];
};

```

Outputs: $[[2],[3,4],[6,5,7],[4,1,8,3]] \rightarrow 11$; $[[-10]] \rightarrow -10$; $[] \rightarrow 0$; $[[1],[2,3]] \rightarrow 3$; $[[-1],[2,3],[1,-1,-3]] \rightarrow -1$.

TRAP

`const dp = t[m-1]` would not copy anything — it would make `dp` **another name for** the caller’s last row, and the loop would overwrite the caller’s data. The `[...t[m-1]]` spread is what makes this function safe to call twice. A test confirms the caller’s triangle comes back unchanged.

Time $O(n^2)$ **Space** $O(n)$

Answer: 11 — the path $2 \rightarrow 3 \rightarrow 5 \rightarrow 1$.

SHORTCUT

Top-down needs “what if $j - 1$ does not exist” checks at both edges. Bottom-up needs none, because $dp[j]$ and $dp[j+1]$ always exist in the row below. When a grid or triangle DP feels fiddly, try reversing the direction first.

Example 5 [warm-up]

Is string s a subsequence of string t ? $|s|, |t| \leq 10^5$. Edge cases: empty s ; empty t ; s longer than t .

SOLUTION

The DP table for this is $O(nm)$, but the DP collapses to a single pointer: scan t once and advance a pointer into s on every match.

isSubsequence

JS

```
const isSubsequence = (s, t) => {
  let i = 0;
  for (const ch of t) if (i < s.length && s[i] === ch) i++;
  return i === s.length;
};
```

Outputs: ("abc", "ahbgdc") → true; ("axc", "ahbgdc") → false; ("", "xyz") → true; ("a", "") → false; ("aaa", "aa") → false; ("aa", "aaa") → true.

Time $O(|t|)$ **Space** $O(1)$

Answer: true.

The empty string is a subsequence of everything — the loop ends with $i === 0 === s.length$. That matches $dp[0][j] = true$, the base case you would write in the table version. The two views always agree.

`for (const ch of t)` iterates a string by **code point**, so an emoji or an accented character arrives as one unit rather than as two surrogate halves. $t[i]$ would split them. For ASCII interview inputs the two are identical; say the difference if asked about Unicode.

Example 6 [warm-up]

A rooted tree is given as a list of children. Find its height (edges on the longest root-to-leaf path). Edge cases: a single node (height 0); a straight chain.

SOLUTION

The smallest “subproblem” of a tree is a subtree.

$$\text{height}(u) = \begin{cases} 0 & \text{if } u \text{ has no children} \\ 1 + \max_{v \in \text{children}(u)} \text{height}(v) & \text{otherwise} \end{cases}$$

heightOf

JS

```
const heightOf = (u, ch) => {
  let best = 0;
  for (const v of ch[u]) best = Math.max(best, heightOf(v, ch) + 1);
  return best;
};
```

Tested on the tree $0 \rightarrow 1, 2; 1 \rightarrow 3, 4; 4 \rightarrow 5$: height 3. A single node: 0. A chain $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$: 3.

Time $O(n)$ **Space** $O(h)$

h = recursion depth. In Node the stack gives out near 10^4 , so a 10^5 chain needs an explicit stack.

| **Answer: 3.**

Tier 1 — the standard set

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS NQT · pattern]

Count the right/down paths in an $m \times n$ grid, with the **full ladder** from naive recursion to the best version. $1 \leq m, n \leq 18$.

SOLUTION

Approach 1 — Plain recursion ($O(2^{m+n})$ — dies around $m = n = 15$)

Approach 1

JS

```
const pathsBrute = (i, j, m, n) => {
  if (i === m-1 && j === n-1) return 1;
  if (i >= m || j >= n) return 0;
  return pathsBrute(i+1, j, m, n) + pathsBrute(i, j+1, m, n);
};
```

Time $O(2^{m+n})$ **Space** $O(m+n)$

Approach 2 — Memo on (i, j) ($O(mn)$)

Approach 2

JS

```
const pathsMemo = (m, n) => {
  if (m <= 0 || n <= 0) return 0;
  const memo = Array.from({ length: m }, () => new Array(n).fill(-1));
  const go = (i, j) => { // closure: m, n, memo are already in
    scope
    if (i === m-1 && j === n-1) return 1;
    if (i >= m || j >= n) return 0;
    if (memo[i][j] !== -1) return memo[i][j];
    return memo[i][j] = go(i+1, j) + go(i, j+1);
  };
  return go(0, 0);
};
```

SHORTCUT

Defining **go** **inside** **pathsMemo** is the idiomatic JavaScript memo. The inner arrow function closes over **m**, **n** and **memo**, so the recursion carries two arguments instead of five, and the table cannot leak between calls the way a module-level cache would. Use a Map keyed by $\{\{i\}, \{j\}\}$ only when the state is not a small pair of integers — an array of arrays is far faster when it is.

Time $O(mn)$ **Space** $O(mn)$

Approach 3 — Full 2D table ($O(mn)$, no recursion)

Approach 3

JS

```
const pathsTable = (m, n) => {
  if (m <= 0 || n <= 0) return 0;
  const dp = Array.from({ length: m }, () => new Array(n).fill(0));
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
      if (i === 0 && j === 0) { dp[i][j] = 1; continue; }
      const up = i > 0 ? dp[i-1][j] : 0;
      const left = j > 0 ? dp[i][j-1] : 0;
      dp[i][j] = up + left;
    }
  return dp[m-1][n-1];
};
```

Time $O(mn)$ **Space** $O(mn)$

Approach 4 — One rolling row ($O(mn)$ time, $O(n)$ space — optimal)

Approach 4

JS

```
const pathsRow = (m, n) => {
  if (m <= 0 || n <= 0) return 0;
  const row = new Array(n).fill(1);
  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      row[j] += row[j-1];
  return row[n-1];
};
```

Time $O(mn)$ **Space** $O(n)$

All four agree. The 4×4 block of answers (rows $m = 1..4$, columns $n = 1..4$):

```
1  1  1  1
1  2  3  4
1  3  6 10
1  4 10 20
```

and $\text{pathsMemo}(3,7) = 28$, $\text{pathsRow}(17,17) = 601080390$, $\text{pathsRow}(0,3) = 0$. All four were also compared cell by cell for every m, n from 0 to 8: zero mismatches.

Answer: The idea that unlocked it: only (i, j) changes in the recursion, so there are at most mn distinct calls. Everything past that was duplicated work.

Those numbers are Pascal's triangle — the answer also equals $\binom{m+n-2}{m-1}$. Say that in an interview, then add: “but the DP version generalises to obstacles and costs, and the formula does not.” That is the real reason to know the DP.

Example 8 [Infosys · pattern]

A grid of numbers. Start anywhere in the top row, and from (i, j) fall to $(i+1, j-1)$, $(i+1, j)$ or $(i+1, j+1)$. Maximise the sum collected. $1 \leq m, n \leq 500$. Edge cases: one row; one column; all negative.

SOLUTION

The state is the cell; the transition looks at three cells above.

$$\text{dp}[i][j] = g[i][j] + \max(\text{dp}[i-1][j-1], \text{dp}[i-1][j], \text{dp}[i-1][j+1])$$

Answer: the maximum over the last row — **not** $\text{dp}[m-1][n-1]$.

```
const maxFallingPath = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  let prev = [...g[0]], cur = new Array(n).fill(0);
  for (let i = 1; i < m; i++) {
    for (let j = 0; j < n; j++) {
      let best = prev[j];
      if (j > 0) best = Math.max(best, prev[j-1]);
      if (j < n - 1) best = Math.max(best, prev[j+1]);
      cur[j] = g[i][j] + best;
    }
    [prev, cur] = [cur, prev]; // swap the two row references, no
    copying
  }
  return prev.reduce((a, b) => a > b ? a : b);
};
```

Outputs: $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow 18$; $[[-1, -2], [-3, -4]] \rightarrow -4$; $[[5]] \rightarrow 5$; $[] \rightarrow 0$; $[[2,1,3],[6,5,4],[7,8,9]] \rightarrow 17$. The caller's grid comes back untouched.

SHORTCUT

`[prev, cur] = [cur, prev]` is JavaScript's swap. It rebinds two **references** — no array is copied — so it is $O(1)$ however wide the row is. It needs `let`, not `const`, and it needs the rows to be separate arrays, which is why `prev` starts as `[...g[0]]` and not as `g[0]`.

TRAP

`prev.reduce((a, b) => a > b ? a : b)` rather than `Math.max(...prev)`. The spread form passes every element as a separate argument and throws `RangeError: Maximum call stack size exceeded` somewhere past 10^5 entries. `reduce` has no such limit.

Time $O(mn)$ **Space** $O(n)$

Answer: 18 — the path $3 \rightarrow 6 \rightarrow 9$.

TRAP

$[[-1, -2], [-3, -4]]$ gives -4 , not 0 . You **must** fall all the way down; there is no “stop early” option. If your code initialises `best = 0` you will silently allow skipping.

Example 9 [Accenture · pattern]

Longest Common Subsequence. Find the length of the longest subsequence present in both strings, then print one such subsequence. $1 \leq n, m \leq 1000$. Edge cases: no common character; identical strings; empty string.

SOLUTION

Approach 1 — Recursion from the two ends ($O(2^{(n+m)})$)

Compare the last characters. If they match, both must be in the LCS (there is never a reason to drop a matching pair). If not, throw away one of them and take the better result.

Approach 1

```
const lcsBrute = (a, b, i, j) => {
  if (i < 0 || j < 0) return 0;
  if (a[i] === b[j]) return 1 + lcsBrute(a, b, i-1, j-1);
  return Math.max(lcsBrute(a, b, i-1, j), lcsBrute(a, b, i, j-1));
};
```

Time $O(2^{n+m})$ **Space** $O(n + m)$

Approach 3 — 2D table ($O(nm)$ — the standard answer)

Approach 3 — dp[i][j] uses the FIRST i and FIRST j characters

JS

```
const lcsTable = (a, b) => {
  const n = a.length, m = b.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++)
      dp[i][j] = a[i-1] === b[j-1] ? dp[i-1][j-1] + 1
                                   : Math.max(dp[i-1][j], dp[i][j-1]);

  return dp[n][m];
};
```

Time $O(nm)$ **Space** $O(nm)$

Approach 4 — Two rows ($O(nm)$ time, $O(m)$ space)

Approach 4

JS

```
const lcsTwoRows = (a, b) => {
  const n = a.length, m = b.length;
  let prev = new Array(m+1).fill(0), cur = new Array(m+1).fill(0);
  for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++)
      cur[j] = a[i-1] === b[j-1] ? prev[j-1] + 1 : Math.max(prev[j], cur[j-1]);
    [prev, cur] = [cur, prev];
  }
  return prev[m];
};
```

Time $O(nm)$ **Space** $O(m)$

To **print** the subsequence you need the full table — walk backwards from $dp[n][m]$.

lcsString — backtracking through the table

JS

```
const lcsString = (a, b) => {
  const n = a.length, m = b.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++)
      dp[i][j] = a[i-1] === b[j-1] ? dp[i-1][j-1] + 1 : Math.max(dp[i-1][j], dp[i][j-1]);
  const out = [];
  // collect into an ARRAY, join at the
```

```

end
let i = n, j = m;
while (i > 0 && j > 0) {
  if (a[i-1] === b[j-1]) { out.push(a[i-1]); i--; j--; }
  else if (dp[i-1][j] >= dp[i][j-1]) i--;
  else j--;
}
return out.reverse().join('');
};

```

TRAP

JavaScript strings are immutable. `out += ch` in a loop allocates a fresh string every time, so building an n -character answer one character at a time is $O(n^2)$ in the worst case. Push the characters into an array and `join('')` once. For the same reason, reversing a string is `[...s].reverse().join('')` — there is no `s.reverse()`.

Outputs:

a	b	len	one LCS
abcde	ace	3	ace
abc	abc	3	abc
abc	def	0	(empty)
''	abc	0	(empty)
aggtab	gxtxayb	4	gtab
aaaa	aa	2	aa

All three versions agree on every row above, and `lcsBrute` agrees on the small ones.

Answer: 3 for abcde / ace.

lcs — Python, two rows

PYTHON

```

def lcs(a, b):
    n, m = len(a), len(b)
    prev = [0] * (m + 1)
    for i in range(1, n + 1):
        cur = [0] * (m + 1)
        for j in range(1, m + 1):
            if a[i-1] == b[j-1]:
                cur[j] = prev[j-1] + 1
            else:
                cur[j] = max(prev[j], cur[j-1])
        prev = cur
    return prev[m]

```

Output: 3 0 0 4 for ("abcde", "ace"), ("abc", "def"), ("", "abc"), ("aggtab", "gxtxayb").

TRAP

If the characters match, do **not** also try `Math.max(dp[i-1][j], dp[i][j-1])`. It cannot help, and adding it makes people think the match case is optional. It is not: there is always an optimal LCS that uses a matching pair of last characters.

Example 10 [Wipro · pattern]

Longest Common Substring — the pieces must now be **contiguous**. $1 \leq n, m \leq 1000$. Edge cases: no common character; one string empty; repeated letters.

SOLUTION

The state is the same, but the meaning changes, and that changes everything.

$dp[i][j]$ = length of the longest common **suffix** of $a[0..i-1]$ and $b[0..j-1]$.

- characters match $\rightarrow dp[i][j] = dp[i-1][j-1] + 1$
- characters differ $\rightarrow dp[i][j] = 0$ (**the run is broken**)

The answer is the maximum cell, not the corner cell.

longestCommonSubstring

JS

```
const longestCommonSubstring = (a, b) => {
  const n = a.length, m = b.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
  let best = 0;
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++)
      if (a[i-1] === b[j-1]) { dp[i][j] = dp[i-1][j-1] + 1; best = Math.max(best, dp[i][j]); }
      else dp[i][j] = 0;
  return best;
};
```

Outputs: ('abcdxyz', 'xyzabcd') $\rightarrow$ 4; ('abcde', 'ace') $\rightarrow$ 1; ('', 'abc') $\rightarrow$ 0; ('a', 'a') $\rightarrow$ 1; ('aaaa', 'aa') $\rightarrow$ 2; ('abc', 'xyz') $\rightarrow$ 0.

Time $O(nm)$ **Space** $O(nm)$

two rows give $O(m)$ if you only need the length

Answer: 4 — abcd.

SHORTCUT

Subsequence vs substring, in one line: **subsequence** $\rightarrow$ on mismatch take `Math.max` of the neighbours (the run may skip); **substring** $\rightarrow$ on mismatch write 0 (the run must restart). The rest of the code is identical.

Example 11 [Capgemini · pattern]

Edit distance. The fewest single-character insert / delete / replace operations that turn a into b . $1 \leq n, m \leq 1000$. Edge cases: one string empty; identical strings; two swapped letters.

SOLUTION

Approach 1 — Recursion on the two prefix lengths ($O(3^n)$)

Approach 1

JS

```
const edBrute = (a, b, i, j) => {
  if (i === 0) return j; // insert the rest of b
  if (j === 0) return i; // delete the rest of a
  if (a[i-1] === b[j-1]) return edBrute(a, b, i-1, j-1);
  return 1 + Math.min(edBrute(a, b, i-1, j), edBrute(a, b, i, j-1), edBrute(a, b, i-1, j-1));
};
```

`Math.min` takes any number of arguments, so C++'s `min({x, y, z})` is simply `Math.min(x, y, z)`. Beware only the array form: `Math.min(...arr)` spreads and hits the argument-count limit on a long array. Three named values are always fine.

Time $O(3^{n+m})$ **Space** $O(n + m)$

Approach 3 — 2D table ($O(nm)$ — the standard answer)

Read the three options as **operations on the last character**:

term	operation	meaning
<code>dp[i-1][j] + 1</code>	delete <code>a[i-1]</code>	drop one char of <code>a</code>
<code>dp[i][j-1] + 1</code>	insert <code>b[j-1]</code>	add one char to <code>a</code>
<code>dp[i-1][j-1] + 1</code>	replace <code>a[i-1]</code> by <code>b[j-1]</code>	swap one char
<code>dp[i-1][j-1]</code>	nothing	only when the characters already match

editDistance

JS

```
const editDistance = (a, b) => {
  const n = a.length, m = b.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
  for (let i = 0; i <= n; i++) dp[i][0] = i;
  for (let j = 0; j <= m; j++) dp[0][j] = j;
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++)
      dp[i][j] = a[i-1] === b[j-1] ? dp[i-1][j-1]
        : 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
  return dp[n][m];
};
```

Time $O(nm)$ **Space** $O(nm)$

Approach 4 — Two rows ($O(nm)$ time, $O(m)$ space — optimal)

editTwoRows

JS

```
const editTwoRows = (a, b) => {
  const n = a.length, m = b.length;
  let prev = Array.from({ length: m+1 }, (_, j) => j); // base row: 0, 1, 2, ...
  let cur = new Array(m+1).fill(0);
  for (let i = 1; i <= n; i++) {
    cur[0] = i; // base case for this row
    for (let j = 1; j <= m; j++)
      cur[j] = a[i-1] === b[j-1] ? prev[j-1]
        : 1 + Math.min(prev[j], cur[j-1], prev[j-1]);
    [prev, cur] = [cur, prev];
  }
  return prev[m];
};
```

SHORTCUT

`Array.from({ length: m+1 }, (_, j) => j)` is the JavaScript way to write “0, 1, 2, ...”. The second argument is a map function that receives the index — the same mechanism that makes `Array.from({ length: n }, () => [])` the correct way to build n distinct rows.

Time $O(nm)$ **Space** $O(m)$

Outputs: ('horse', 'ros') → 3 from all three versions; ('kitten', 'sitting') → 3; ('', 'abc') → 3; ('abc', '') → 3; ('', '') → 0; ('same', 'same') → 0; ('ab', 'ba') → 2. A randomised comparison of the three versions on 200 random string pairs reported 200 random cross-checks passed.

Answer: 3 for horse → ros: replace h → r, delete r, delete e.

edit_distance — Python, two rows

PYTHON

```
def edit_distance(a, b):
    n, m = len(a), len(b)
    prev = list(range(m + 1))
    for i in range(1, n + 1):
        cur = [i] + [0] * m
        for j in range(1, m + 1):
            if a[i-1] == b[j-1]:
                cur[j] = prev[j-1]
            else:
                cur[j] = 1 + min(prev[j], cur[j-1], prev[j-1])
        prev = cur
    return prev[m]
```

Output: 3 3 and 3 3 0 for the cases above.

TRAP

`cur[0] = i` must be written **inside** the `i` loop, before the `j` loop. It is the base case “turning the first `i` characters of `a` into the empty string costs `i` deletions”. Forgetting it leaves stale data from two rows ago — and because `[prev, cur] = [cur, prev]` **reuses** the same two arrays rather than allocating fresh ones, that stale data really is there waiting for you.

Example 12 [Cognizant · pattern]

Longest palindromic subsequence. $1 \leq n \leq 1000$. Edge cases: empty string; all identical letters; a string with no repeats.

SOLUTION

The insight is one line: **a palindromic subsequence of s is a common subsequence of s and $\text{reverse}(s)$.**

Reading a palindrome forwards and backwards gives the same letters, so every palindromic subsequence appears in both. And the longest common subsequence of s and its reverse is always palindromic.

So: reverse the string and call LCS.

longestPalinSubseq — one line of new code

JS

```
const longestPalinSubseq = (s) => lcsTable(s, [...s].reverse().join(''));
```

Outputs: 'bbbab' → 4; 'abcd' → 1; '' → 0; 'a' → 1; 'aaaa' → 4; 'agbcba' → 5.

TRAP

There is no `String.prototype.reverse`. `[...s].reverse().join('')` is the idiom, and the spread — not `s.split('')` — is what keeps surrogate pairs intact. `s.split('').reverse()` would tear an emoji in half.

Time $O(n^2)$ **Space** $O(n^2)$

$O(n)$ space with the two-row LCS if you only need the length

Answer: 4 for "bbbab" — the subsequence bbbb.

Example 13 [TCS Digital · pattern]

What is the **fewest insertions** needed to make a string a palindrome? And the fewest **deletions**? $1 \leq n \leq 1000$.

SOLUTION

Let L be the longest palindromic subsequence. Every character **not** in that subsequence must be dealt with, and each one costs exactly one operation.

- **Insertions:** mirror each unmatched character on the other side $\rightarrow n - L$.
- **Deletions:** throw each unmatched character away $\rightarrow n - L$.

They are the same number. That is a good sentence to say out loud.

`minInsertionsToPalindrome / minDeletionsToPalindrome`

JS

```
const minInsertionsToPalindrome = (s) => s.length - longestPalinSubseq(s);
const minDeletionsToPalindrome = (s) => s.length - longestPalinSubseq(s);
```

Outputs: 'bbbab' $\rightarrow$ LPS 4, so 1; 'abcd' $\rightarrow$ LPS 1, so 3; 'aaaa' $\rightarrow$ LPS 4, so 0; 'agbcba' $\rightarrow$ LPS 5, so 1.

Time $O(n^2)$ **Space** $O(n^2)$

Answer: 1 insertion for "bbbab": make it "babbab" — or delete the a to get "bbbb".

SHORTCUT

Any question of the form “fewest edits to reach property P” where P has a “largest sub-thing with property P” version, is usually answer = $n - (\text{largest sub-thing})$. Spotting that saves you from designing a new DP.

Example 14 [Infosys · pattern]

A binary grid. (a) Count **all** square blocks made only of 1s. (b) Report the **area** of the largest such square. $1 \leq m, n \leq 300$. Edge cases: all zeros; all ones; a single cell.

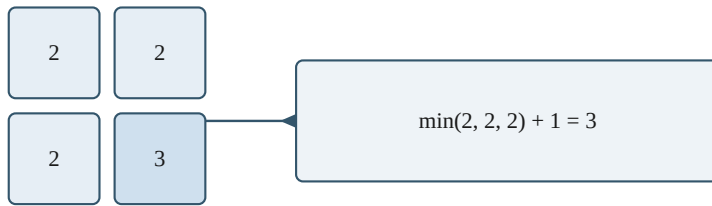
SOLUTION

State: $dp[i][j]$ = the side of the largest all-ones square whose **bottom-right corner** is (i, j) .

$$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) \quad \text{if } g[i][j] = 1$$

Why the minimum of three? A square of side $k+1$ at (i, j) needs squares of side at least k at the three neighbouring corners. The weakest of the three limits you.

And the count: a cell with $dp[i][j] = k$ is the bottom-right corner of exactly k squares (sides 1, 2, ..., k). So the total is the sum of all cells.



$dp[i][j] = 3$ needs side-2 squares at the three marked corners.

countSquares — every all-ones square

JS

```
const countSquares = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  const dp = Array.from({ length: m }, () => new Array(n).fill(0));
  let total = 0;
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
      if (!g[i][j]) { dp[i][j] = 0; continue; }
      dp[i][j] = (i === 0 || j === 0) ? 1
        : 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
      total += dp[i][j];
    }
  return total;
};
```

maximalSquareArea — same table, different reading

JS

```
const maximalSquareArea = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  const dp = Array.from({ length: m }, () => new Array(n).fill(0));
  let best = 0;
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
      if (!g[i][j]) continue;
      dp[i][j] = (i === 0 || j === 0) ? 1
        : 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
      best = Math.max(best, dp[i][j]);
    }
  return best * best;
};
```

Outputs:

grid	count	max area
[[0,1,1,1],[1,1,1,1],[0,1,1,1]]	15	9
[[1,0,1],[1,1,0],[1,1,0]]	7	4
[[0]]	0	0
[[1]]	1	1

[]

0

0

Time $O(mn)$ **Space** $O(mn)$
 two rows give $O(n)$ space

Answer: 15 squares, largest area 9.

Example 15 [Wipro · pattern]

Shortest common supersequence. The shortest string that contains both a and b as subsequences. Report its length and print one such string. $1 \leq n, m \leq 1000$. Edge cases: one string empty; identical strings; no shared letters.

SOLUTION

Length first. Write both strings, then merge the shared part once instead of twice:

$$|SCS| = n + m - |LCS(a, b)|$$

To build the string, backtrack through the LCS table. At each step:

- characters match $\rightarrow$ write it once, move diagonally
- otherwise $\rightarrow$ write the character from the side the table says to move from

shortestCommonSupersequence

JS

```
const shortestCommonSupersequence = (a, b) => {
  const n = a.length, m = b.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++)
      dp[i][j] = a[i-1] === b[j-1] ? dp[i-1][j-1] + 1 : Math.max(dp[i-1][j], dp[i][j-1]);
  const out = [];
  let i = n, j = m;
  while (i > 0 && j > 0) {
    if (a[i-1] === b[j-1]) { out.push(a[i-1]); i--; j--; }
    else if (dp[i-1][j] >= dp[i][j-1]) { out.push(a[i-1]); i--; }
    else { out.push(b[j-1]); j--; }
  }
  while (i > 0) { out.push(a[i-1]); i--; } // leftovers of a
  while (j > 0) { out.push(b[j-1]); j--; } // leftovers of b
  return out.reverse().join('');
};
```

Outputs: ('abac', 'cab') $\rightarrow$ cabac, length 5; ('', 'abc') $\rightarrow$ abc; ('abc', '') $\rightarrow$ abc; ('', '') $\rightarrow$ empty; ('geek', 'eke') $\rightarrow$ gekek, length 5.

Check the formula on the last one: $4 + 3 - |LCS(\text{geek}, \text{eke})| = 7 - 2 = 5$. ✓

Time $O(nm)$ **Space** $O(nm)$

Answer: cabac — it contains abac (positions 2,3,4,5) and cab (positions 1,2,3).

TRAP

The two while loops **after** the main loop are not optional. When one string runs out first, the remaining characters of the other must still be written. Leaving them out produces a string that is too short and does not contain both inputs.

Tier 2 — applied and two-step

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 16 [Shopee · pattern]

A long log string s and a short code t . How many **different subsequences** of s are exactly equal to t ? Report the count modulo $10^9 + 7$. $1 \leq |s| \leq 1000$, $1 \leq |t| \leq 100$. Edge cases: t empty; t longer than s ; repeated letters in s .

SOLUTION

State: $dp[j]$ = number of ways to build the first j characters of t using the part of s processed so far.

Processing one more character $s[i-1]$: it can only help positions j where $t[j-1] == s[i-1]$, and it adds the ways that already had $j-1$ characters matched.

$$dp[j] += dp[j-1] \quad \text{when } s[i-1] = t[j-1]$$

Base $dp[0] = 1$ — there is one way to match nothing.

The j loop must run backwards, exactly as in 0/1 knapsack, so that the same s character is not used twice in one pass.

distinctSubseq

JS

```
const MOD = 1000000007;

const distinctSubseq = (s, t) => {
  const n = s.length, m = t.length;
  const dp = new Array(m + 1).fill(0);
  dp[0] = 1;
  for (let i = 1; i <= n; i++)
    for (let j = m; j >= 1; j--) // BACKWARDS
      if (s[i-1] === t[j-1]) dp[j] = (dp[j] + dp[j-1]) % MOD;
  return dp[m];
};
```

Outputs: ('rabbbit', 'rabbit') $\rightarrow$ 3; ('babgbag', 'bag') $\rightarrow$ 5; ('abc', '') $\rightarrow$ 1; ('', 'abc') $\rightarrow$ 0; ('aaa', 'aa') $\rightarrow$ 3; ('abc', 'abcd') $\rightarrow$ 0.

This DP only **adds**, so plain numbers are safe under $MOD = 1e9+7$: two residues sum to under 2×10^9 , well inside $2^{53} - 1$. A DP that **multiplies** residues would not be safe — see the trap after Warm-up 1.

Time $O(|s| \cdot |t|)$ **Space** $O(|t|)$

Answer: 3 for rabbbit $\rightarrow$ rabbit: the three ways choose a different one of the three bs to drop.

TRAP

If the j loop runs forwards, one s character can satisfy two positions of t in the same pass, and ('aaa', 'aa') returns 6 instead of 3 — verified by running the broken version. Backwards. Always backwards when one input item may be used once.

Example 17 [Grab · pattern]

Wildcard matching. In the pattern, ? matches exactly one character and * matches any run of characters, including none. Does the pattern match the whole string? $0 \leq |s| \leq 2000$, $0 \leq |p| \leq 2000$. Edge cases: empty string; a pattern of only stars; empty pattern.

SOLUTION

State: $dp[i][j]$ = do the first i characters of s match the first j of p ?

Three cases for $p[j-1]$:

*	$dp[i][j] = dp[i-1][j] \ \ dp[i][j-1]$ — the star eats one more character, or the star matches nothing
? or equal	$dp[i][j] = dp[i-1][j-1]$ — consume one from each
else	false

Base: $dp[0][0] = \text{true}$, and $dp[0][j]$ is true only while the pattern is all stars.

wildcardMatch

JS

```
const wildcardMatch = (s, p) => {
  const n = s.length, m = p.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(false));
  dp[0][0] = true;
  for (let j = 1; j <= m; j++) dp[0][j] = dp[0][j-1] && p[j-1] === '*';
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++) {
      if (p[j-1] === '*') dp[i][j] = dp[i-1][j] || dp[i][j-1];
      else if (p[j-1] === '?' || p[j-1] === s[i-1]) dp[i][j] = dp[i-1][j-1];
      else dp[i][j] = false;
    }
  return dp[n][m];
};
```

A boolean table stores real `true/false` here, not `0/1`. That matters because `dp[0][j-1] && p[j-1] === '*'` returns a **boolean**, and later `dp[i-1][j] || dp[i][j-1]` returns one too — no coercion anywhere. If you do store `0/1`, remember that `0` is falsy but the **string** `'0'` is truthy, which is the kind of mix-up that costs a round.

Outputs:

s	p	ok	s	p	ok
aa	a	no	' '	' '	yes
aa	*	yes	' '	*	yes
cb	?a	no	' '	***	yes
adceb	*a*b	yes	' '	?	no
acdcb	a*c?b	no	a	' '	no
abc	a?c	yes	abc	*c	yes

Time $O(|s| \cdot |p|)$ **Space** $O(|s| \cdot |p|)$
two rows give $O(|p|)$ space

Answer: adceb matches *a*b; acdcb does not match a*c?b.

Why is `acdcb` vs `a*c?b` a **no**? After `a`, the star must cover some prefix, then a literal `c`, then one any-character, then `b`. The string ends `...c b`, so the `?` would have to be the `c` and the `b` the `b` — but then the literal `c` of the pattern must match the `d`. It fails. The table finds this without you tracing it; that is the point.

Example 18 [Sea/Shopee · pattern]

Interleaving strings. Given a , b and c , can c be formed by taking the characters of a and b in order, mixing them freely? $0 \leq |a|, |b| \leq 500$. Edge cases: length mismatch; both empty; repeated letters.

SOLUTION

State: $dp[i][j]$ = can the first i of a plus the first j of b build the first $i + j$ of c ?

The last character of $c[0..i+j-1]$ came from a or from b :

$$dp[i][j] = (dp[i-1][j] \text{ and } a_{i-1} = c_{i+j-1}) \text{ or } (dp[i][j-1] \text{ and } b_{j-1} = c_{i+j-1})$$

isInterleave

JS

```
const isInterleave = (a, b, c) => {
  const n = a.length, m = b.length;
  if (c.length !== n + m) return false; // length check FIRST
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(false));
  dp[0][0] = true;
  for (let i = 0; i <= n; i++)
    for (let j = 0; j <= m; j++) {
      if (i > 0 && dp[i-1][j] && a[i-1] === c[i+j-1]) dp[i][j] = true;
      if (j > 0 && dp[i][j-1] && b[j-1] === c[i+j-1]) dp[i][j] = true;
    }
  return dp[n][m];
};
```

Outputs: ('abc', 'def', 'adbecf') → yes; ('abc', 'def', 'abcdef') → yes; ('abc', 'def', 'abdecf') → yes; ('abc', 'def', 'abcdfe') → **no**; ('', '', '') → yes; ('a', '', 'a') → yes; ('', 'a', 'a') → yes; ('a', 'b', 'ab') → yes; ('a', 'b', 'ba') → yes; ('aa', 'ab', 'aaba') → yes; ('aa', 'ab', 'abaa') → yes.

Time $O(nm)$ **Space** $O(nm)$
one row gives $O(m)$

Answer: abcdfe fails — after abcd the only letters left are e then f, in that order.

TRAP

The length check $c.length === a.length + b.length$ must come first. In C++ reading $c[i+j-1]$ past the end is undefined behaviour; in JavaScript it is worse in a quieter way — indexing a string out of range returns undefined, every comparison against it is false, and the function returns a **plausible-looking** false for the wrong reason. Bugs that never crash are the expensive kind. Verified: `isInterleave('a', 'b', 'abc')` is rejected by the length check, not by the table.

Example 19 [Agoda · pattern]

Two collectors start on the top row of an $m \times n$ grid — collector A at column 0, collector B at column $n - 1$. Every turn **both** move down one row and may shift the column by $-1, 0$ or $+1$. They collect the value of the cell they land on; if both land on the same cell it is collected once. Maximise the total. $1 \leq m \leq 70, 1 \leq n \leq 70$. Edge cases: a single column; a single row; a one-cell grid.

SOLUTION

Approach 1 — Run the best path for A, then for B (WRONG)

Greedy fails: A taking the richest path can force B onto a poor one. The two choices are not independent.

Approach 2 — Move both at once — state is (row, colA, colB) ($O(m \cdot n^2 \cdot 9)$ — optimal)

Because both robots always sit on the **same row**, the row is a single index. The state is (i, j_1, j_2) and each step tries $3 \times 3 = 9$ moves.

twoRobots

JS

```
const twoRobots = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  let dp = Array.from({ length: n }, () => new Array(n).fill(-Infinity));
  dp[0][n-1] = n === 1 ? g[0][0] : g[0][0] + g[0][n-1];
  for (let i = 1; i < m; i++) {
    const nxt = Array.from({ length: n }, () => new Array(n).fill(-Infinity));
    for (let j1 = 0; j1 < n; j1++)
      for (let j2 = 0; j2 < n; j2++) {
        if (dp[j1][j2] === -Infinity) continue;
        for (let d1 = -1; d1 <= 1; d1++)
          for (let d2 = -1; d2 <= 1; d2++) {
            const a = j1 + d1, b = j2 + d2;
            if (a < 0 || a >= n || b < 0 || b >= n) continue;
            const gain = a === b ? g[i][a] : g[i][a] + g[i][b];
            nxt[a][b] = Math.max(nxt[a][b], dp[j1][j2] + gain);
          }
      }
    dp = nxt;
  }
  let best = 0;
  for (let j1 = 0; j1 < n; j1++)
    for (let j2 = 0; j2 < n; j2++)
      best = Math.max(best, dp[j1][j2] === -Infinity ? 0 : dp[j1][j2]);
  return best;
};
```

SHORTCUT

-Infinity replaces the INT_MIN / 4 dodge entirely. The / 4 existed only so that INT_MIN + gain would not wrap around to a large positive number; -Infinity + gain is still -Infinity, so the “unreachable” marker cannot be faked by arithmetic. One fewer thing to get wrong.

Outputs:

grid	best
[[3,1,1],[2,5,1],[1,5,5],[2,1,1]]	24
[[1,0,0,0,0,0,1],[2,0,0,0,0,3,0],[2,0,9,0,0,0,0], [0,3,0,5,4,0,0],[1,0,2,3,0,0,6]]	28
[[5]]	5
[[1,1],[1,1]]	4
[[2,3,4]]	6
[]	0

Time $O(mn^2)$ **Space** $O(n^2)$

9 transitions per state, a constant

Answer: 24 for the first grid.

TRAP

The one-column case: $n === 1$ means both robots start on the **same** cell, so the starting value must be counted once. The line `dp[0][n-1] = n === 1 ? g[0][0] : ...` handles it. Miss it and a 1-wide grid doubles every cell.

Example 20 [GIC · pattern]

Matrix chain multiplication. Matrix i has size $d[i] \times d[i+1]$. Multiplying an $p \times q$ by a $q \times r$ matrix costs pqr scalar multiplications. Where do you put the brackets to make the total smallest? $2 \leq |d| \leq 100$. Edge cases: one matrix (cost 0); two matrices (only one order).

SOLUTION

This is the first **interval DP**. The last multiplication splits the chain somewhere:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + d_i d_{k+1} d_{j+1})$$

$dp[i][k]$ produces a $d_i \times d_{k+1}$ matrix and $dp[k+1][j]$ produces a $d_{k+1} \times d_{j+1}$ matrix — multiplying them is the last term.

Loop by interval length, so shorter pieces are always ready.

matrixChain

JS

```
const matrixChain = (d) => {
  const n = d.length - 1;           // number of matrices
  if (n <= 1) return 0;
  const dp = Array.from({ length: n }, () => new Array(n).fill(0));
  for (let len = 2; len <= n; len++)
    for (let i = 0; i + len - 1 < n; i++) {
      const j = i + len - 1;
      dp[i][j] = Infinity;
      for (let k = i; k < j; k++)
        dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k+1][j] + d[i]*d[k+1]*d[j+1]);
    }
  return dp[0][n-1];
};
```

Outputs: $[10, 30, 5, 60] \rightarrow 4500$; $[40, 20, 30, 10, 30] \rightarrow 26000$; $[5, 6] \rightarrow 0$; $[2, 3, 4] \rightarrow 24$; $[1, 2, 3, 4, 5] \rightarrow 38$.

Time $O(n^3)$ **Space** $O(n^2)$

Answer: 4500 for {10, 30, 5, 60}. Bracketing $(AB)C$ costs $10 \cdot 30 \cdot 5 + 10 \cdot 5 \cdot 60 = 1500 + 3000 = 4500$; $A(BC)$ costs $30 \cdot 5 \cdot 60 + 10 \cdot 30 \cdot 60 = 9000 + 18000 = 27000$.

SHORTCUT

Every interval DP has the same three lines: loop `len`, loop start `i`, loop split `k`. If you can write those three lines from memory, half of the hardest DP questions are already framed. The only thing that changes is the cost term.

Example 21 [DBS · pattern]

Maximum weight independent set on a tree. Each node has a value. Pick a set of nodes with no two picked nodes joined by an edge, maximising the total. $1 \leq n \leq 10^5$. Edge cases: a single node; a chain; all values zero.

SOLUTION

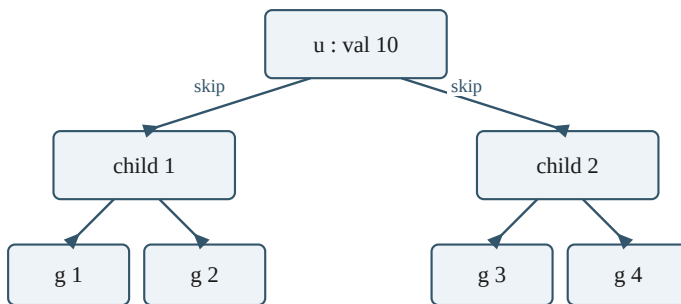
On an array this was “no two neighbours”. On a tree it is the same idea with one DFS.

For each node u return **two** numbers:

- **withU** — the best total in u 's subtree **when u is taken**
- **withoutU** — the best total in u 's subtree **when u is not taken**

$$\text{withU}(u) = \text{val}[u] + \sum_{v \in \text{children}} \text{withoutU}(v)$$

$$\text{withoutU}(u) = \sum_{v \in \text{children}} \max(\text{withU}(v), \text{withoutU}(v))$$



If a node is taken, every child must be skipped; if it is skipped, each child chooses freely.

robTree — returns [take u, skip u]

JS

```
const robTree = (u, ch, val) => {
  let take = val[u], skip = 0;
  for (const v of ch[u]) {
    const [t, s] = robTree(v, ch, val);
    take += s;
    skip += Math.max(t, s);
  }
  return [take, skip];
};
```

The answer is `Math.max(...robTree(root, ch, val))` at the root — a two-element spread, so no stack risk there.

Outputs on the tree $0 \rightarrow 1, 2; 1 \rightarrow 3, 4$:

values	best	chosen
[3,2,3,1,1]	5	nodes 0, 3, 4 (or nodes 2, 3, 4)
[3,4,5,1,1]	9	nodes 1 and 2
single node [7]	7	that node
chain 0-1-2 with [1,100,1]	100	node 1
[0,0,0,0,0]	0	nothing

Time $O(n)$ **Space** $O(h)$

Answer: 5 for {3, 2, 3, 1, 1}: either take node 0 (3) plus grandchildren 3 and 4 (1+1) = 5, or take nodes 2, 3, 4 (3+1+1) = 5. Both reach 5; node 0 and node 2 cannot both be taken because they are joined by an edge.

TRAP

Recursion depth — and in Node this is not hypothetical. A tree that is one long chain of 100000 nodes makes this exact function throw `RangeError: Maximum call stack size exceeded`. That was run; the chapter is not guessing. C++ and Java survive the same depth, JavaScript does not, and the constraint here says $n \leq 10^5$.

Two fixes, both worth naming:

1. **Explicit stack.** Push `[node, childIndex]` frames, advance the index on each visit, and fold the child's `[take, skip]` into the parent's when a frame is popped. Same algorithm, no recursion.
2. **Iterative post-order by reverse BFS.** Order the nodes by BFS from the root, then walk that order **backwards** — every node is then processed after all of its children. It is four lines and it is what most people write under time pressure.

`node --stack-size=...` exists but is not something you can rely on in a judge.

Example 22 [SCB · pattern]

Best path in a weighted tree. A path may start and end at any two nodes and bends at most once (it goes up from one node and down to another). Node values may be negative. Find the largest path sum. $1 \leq n \leq 10^5$. Edge cases: a single node with a negative value; all values negative.

SOLUTION

For each node u , define $\text{down}(u)$ = the best sum of a path that **starts at u and only goes downward**. A child contributes $\max(0, \text{down}(\text{child}))$ — a negative branch is simply not used.

At u there are two things to do:

1. **answer candidate:** the path bends at u and uses the two best child branches: $\text{val}[u] + \text{best1} + \text{best2}$.
2. **return value:** the path continues up through u , so it can use only one branch: $\text{val}[u] + \text{best1}$.

maxPathSum — one DFS, two jobs per node

JS

```
const maxPathSum = (root, ch, val) => {
  let bestAns = -Infinity; // a local, not a global
  const downFrom = (u) => {
    let b1 = 0, b2 = 0; // two largest child contributions
    for (const v of ch[u]) {
      const d = Math.max(0, downFrom(v)); // a negative branch is simply
    }
    if (d > b1) { b2 = b1; b1 = d; }
    else if (d > b2) b2 = d;
  }
  bestAns = Math.max(bestAns, val[u] + b1 + b2); // path bends at u
  return val[u] + b1; // path continues upward
};
downFrom(root);
return bestAns;
};
```

Outputs: root with children, values `[1,2,3]` → 6 (the whole thing); the tree `0 → 1,2; 2 → 3,4` with values `[-10,9,20,15,7]` → 42 (the path `15 → 20 → 7`); single node `[-5]` → -5; `[-1,-2,-3]` → -1; chain `0-1-2` with `[5,-100,5]` → 5.

Time $O(n)$ **Space** $O(h)$

Answer: 42.

TRAP

Three separate traps in one problem. (1) `Math.max(0, ...)` on the child, so a negative branch is dropped — but (2) the node's **own** value is never dropped, which is why `[-5]` correctly returns `-5` and not `0`. (3) `bestAns` must start at `-Infinity`, not `0`, or an all-negative tree returns `0`.

And a fourth, specific to JavaScript: keep `bestAns` as a `let` **inside** `maxPathSum` rather than as a module-level variable. A global accumulator survives between calls, so the second test in your own test file silently reads the first test's answer. The closure makes that impossible.

Tier 3 — insight needed

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 23 [Google · pattern]

Palindrome partitioning — minimum cuts. Cut a string into pieces so that every piece is a palindrome. What is the fewest number of cuts? $1 \leq n \leq 2000$. Edge cases: already a palindrome (0 cuts); no repeats ($n-1$ cuts); empty string.

SOLUTION

Approach 1 — Try every set of cut positions ($O(2^n)$)

Approach 2 — DP with a palindrome check inside ($O(n^3)$ — the check costs $O(n)$)

Approach 3 — Precompute a palindrome table first ($O(n^2)$ — optimal)

Two DPs, run one after the other.

DP 1 — is $s[i..j]$ a palindrome?

$$\text{pal}[i][j] = (s_i = s_j) \text{ and } (\text{length} = 2 \text{ or } \text{pal}[i+1][j-1])$$

Fill it by increasing length, so $\text{pal}[i+1][j-1]$ is always ready.

DP 2 — fewest cuts for the prefix ending at i .

$$\text{dp}[i] = \begin{cases} 0 & \text{if } s[0..i] \text{ is a palindrome} \\ \min_{1 \leq j \leq i, \text{pal}[j][i]} (\text{dp}[j-1] + 1) & \text{otherwise} \end{cases}$$

minPalindromeCuts

JS

```
const minPalindromeCuts = (s) => {
  const n = s.length;
  if (n <= 1) return 0;
  const pal = Array.from({ length: n }, () => new Array(n).fill(false));
  for (let i = 0; i < n; i++) pal[i][i] = true;
  for (let len = 2; len <= n; len++)
    for (let i = 0; i + len - 1 < n; i++) {
      const j = i + len - 1;
      pal[i][j] = s[i] === s[j] && (len === 2 || pal[i+1][j-1]);
    }
  const dp = new Array(n).fill(Infinity);
  for (let i = 0; i < n; i++) {
    if (pal[0][i]) { dp[i] = 0; continue; }
    for (let j = 1; j <= i; j++)
      if (pal[j][i] && dp[j-1] !== Infinity) dp[i] = Math.min(dp[i], dp[j-1] + 1);
  }
  return dp[n-1];
};
```

Outputs: 'aab' → 1; 'a' → 0; '' → 0; 'abcba' → 0; 'abcd' → 3; 'aaaa' → 0; 'abbab' → 1; 'noonabbad' → 2.

Time $O(n^2)$ **Space** $O(n^2)$

Answer: "noonabbad" needs 2 cuts: noon | abba | d.

The follow-up the interviewer asks next

"Now print the pieces."

Store `from[i]` = the `j` that gave the minimum, i.e. the start index of the last piece ending at `i`. Then walk backwards from `n-1`, slicing off `s[from[i] .. i]` each time, and reverse the list. This is the same parent-pointer idea as Chapter 17, Example 15.

"And can you get the memory down to $O(n)$?"

Yes. Replace the `pal` table by the **expand-around-centre** method: for each of the $2n - 1$ centres, grow outward while the characters match, and relax `dp` as you grow. That keeps only the `dp` array — $O(n)$ space, still $O(n^2)$ time.

Example 24 [Amazon · pattern]

Cutting a stick. A stick has length `L`. You must cut it at each of the given positions. A single cut costs the **current length of the piece being cut**. The order of cuts is yours to choose. Minimise the total cost. Number of cut positions ≤ 100 . Edge cases: no cuts at all; one cut; evenly spaced cuts.

SOLUTION

Approach 1 — Try every order of cuts ($O(k!)$ — 100! is beyond hopeless)

Approach 2 — Interval DP over cut positions ($O(k^3)$ — optimal)

Here is the insight. Do not think about **which cut happens first in time**. Think about **which cut is the first one made inside a given piece**, because that single choice splits the piece into two independent pieces.

Add the two ends `0` and `L` to the list and sort it. Then

$$dp[i][j] = \min_{i < k < j} (dp[i][k] + dp[k][j]) + (cuts[j] - cuts[i])$$

The last term is the length of the piece `i .. j`, paid once for the first cut inside it.

minCutCost

JS

```
const minCutCost = (L, cuts) => {
  const c = [...cuts, 0, L].sort((x, y) => x - y); // COPY, then sort NUMERICALLY
  const m = c.length;
  const dp = Array.from({ length: m }, () => new Array(m).fill(0));
  for (let len = 2; len < m; len++)
    for (let i = 0; i + len < m; i++) {
      const j = i + len;
      dp[i][j] = Infinity;
      for (let k = i + 1; k < j; k++)
        dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k][j] + c[j] - c[i]);
    }
  return dp[0][m-1];
};
```

Outputs: `L=7, cuts=[1,3,4,5] → 16`; `L=9, cuts=[5,6,1,4,2] → 22`; `L=10, cuts=[] → 0`; `L=10, cuts=[5] → 10`; `L=4, cuts=[1,2,3] → 8`. The caller's `cuts` array comes back in its original order.

TRAP

This is the only sort in the chapter, and it is the most dangerous line in it. `arr.sort()` with empty brackets is **lexicographic**: it stringifies every element first, so `[10, 9, 1].sort()` gives `[1, 10, 9]`. Here that is not theoretical — `[5,6,1,4,2,0,9].sort()` happens to come out right because every value is a single digit, but change the stick length to 10 and `[5,6,1,4,2,0,10].sort()` gives `[0, 1, 10, 2, 4, 5, 6]`. Both were run. Numbers **always** need `sort((a, b) => a - b)`.

Two companions: a comparator must return a **number**, never a boolean — $(a, b) \Rightarrow a > b$ coerces to $1/0$, the sort never sees a negative, and the array stays essentially unsorted. And `sort` mutates in place and returns the same array, which is why this code sorts a fresh `[...cuts, 0, L]` rather than the caller's list.

Time $O(k^3)$ **Space** $O(k^2)$

Answer:

16. One optimal order for $L=7$, $\text{cuts}=\{1, 3, 4, 5\}$: cut at 3 (cost 7), then at 5 (cost 4), then 1 (cost 3) and 4 (cost 2): $7 + 4 + 3 + 2 = 16$.

The follow-up the interviewer asks next

“What if the cost of a cut is a fixed fee instead of the piece length?”

Then every order costs the same, and there is nothing to optimise. Say that. Recognising when a problem **collapses** is as valuable as solving a hard one.

“What if each cut has both a fee and a piece-length cost?”

The recurrence barely changes: $\text{dp}[i][j] = \min(\text{dp}[i][k] + \text{dp}[k][j] + \text{fee}[k]) + (\text{cuts}[j] - \text{cuts}[i])$. The $\text{fee}[k]$ moves inside the min because it depends on which cut you choose first.

SHORTCUT

The whole family — matrix chain, stick cutting, merging piles, bursting balloons — shares one move: **pick the element that is processed LAST (or FIRST) inside the interval**, because that choice makes the two sides independent. When an interval problem feels stuck, ask “what if I decide which one is last?”

Example 25 [Microsoft · pattern]

Bursting balloons. Balloons in a row each hold a number. Bursting balloon i earns $\text{left} \times a[i] \times \text{right}$, where left and right are the nearest **surviving** neighbours (treat a missing neighbour as 1). Burst them all. Maximise the earnings. $1 \leq n \leq 300$. Edge cases: one balloon; empty list; all equal values.

SOLUTION

Approach 1 — Try every burst order ($O(n!)$)

Approach 2 — Interval DP on 'which balloon bursts FIRST' (WRONG — the pieces are not independent)

If you burst k first, the two sides still touch each other afterwards, so they are not separate problems. This is the trap the question is built around.

Approach 3 — Interval DP on 'which balloon bursts LAST' ($O(n^3)$ — optimal)

Flip it. If balloon k is the **last** one burst in the open interval (i, j) , then at that moment its neighbours are exactly $a[i]$ and $a[j]$ — the boundary balloons, which are still alive. Everything strictly between i and k was burst without ever touching anything beyond k , and the same on the right. Now the two sides **are** independent.

$$\text{dp}[i][j] = \max_{i < k < j} (\text{dp}[i][k] + \text{dp}[k][j] + a_i \cdot a_k \cdot a_j)$$

Pad the array with a 1 on each end so the boundaries always exist.

burst

JS

```
const burst = (input) => {
  if (input.length === 0) return 0;
  const a = [1, ...input, 1];           // pad both ends, without touching
  the caller
  const m = a.length;
```

```
const dp = Array.from({ length: m }, () => new Array(m).fill(0));
for (let len = 2; len < m; len++)
  for (let i = 0; i + len < m; i++) {
    const j = i + len;
    for (let k = i + 1; k < j; k++)
      dp[i][j] = Math.max(dp[i][j], dp[i][k] + dp[k][j] + a[i]*a[k]*a[j]);
  }
return dp[0][m-1];
};
```

Outputs: [3,1,5,8] → 167; [1,5] → 10; [7] → 7; [] → 0; [2,2,2] → 14; [9,76,64,21] → 116718. The caller's array is unchanged.

SHORTCUT

[1, ...input, 1] does the padding, the copy and the boundary sentinels in one expression. C++ needs an insert at the front — an $O(n)$ shift — plus a push_back; the spread form builds the new array once and never mutates the input.

Time $O(n^3)$ **Space** $O(n^2)$

Answer:

167. One optimal order for {3, 1, 5, 8}: burst 1 (earning $3 \cdot 1 \cdot 5 = 15$), then 5 ($3 \cdot 5 \cdot 8 = 120$), then 3 ($1 \cdot 3 \cdot 8 = 24$), then 8 ($1 \cdot 8 \cdot 1 = 8$): $15 + 120 + 24 + 8 = 167$.

The follow-up the interviewer asks next

“Why does ‘burst first’ fail but ‘burst last’ work? Say it precisely.”

Because the recurrence needs the two sub-intervals to be **solvable without knowing anything about each other**. After bursting k first, the left and right pieces become neighbours, so the left piece's final earnings depend on what survives on the right. After declaring k **last**, the left piece never sees past k and the right piece never sees before k . Independence is the whole requirement of DP, and that is the sentence the interviewer is listening for.

“What if values can be negative?”

Then “burst them all” may be worse than an empty burst — but the problem still forces every balloon to pop, so the DP is unchanged. In JavaScript there is no int to overflow; the question is only whether the total stays inside $2^{53} - 1$. Here it does easily: $300 \times 100^3 = 3 \times 10^8$.

Example 26 [Goldman Sachs · pattern]

Counting bracketings. An expression like $T|T\&F^{\wedge}T$ uses T, F and the operators $\&, |, ^$ (XOR). In how many ways can you bracket it so the result is **True**? Report the answer modulo 1003. Operands ≤ 50 . Edge cases: a single T ; a single F ; an expression that is never true.

SOLUTION

This is interval DP with **two** tables, because to know how many bracketings give True you must also know how many give False.

- $T[i][j]$ = bracketings of operands $i..j$ that evaluate to True
- $F[i][j]$ = the same, evaluating to False

Split at operator m (between operand m and $m+1$). Let $lt = T[i][m]$, $lf = F[i][m]$, $rt = T[m+1][j]$, $rf = F[m+1][j]$, and let $tot = (lt + lf)(rt + rf)$ be the total number of bracketings of this split.

op	True count	False count
&	$lt \cdot rt$	$tot - lt \cdot rt$

	$\text{tot} - \text{lf} \cdot \text{rf}$	$\text{lf} \cdot \text{rf}$
^	$\text{lt} \cdot \text{rf} + \text{lf} \cdot \text{rt}$	$\text{lt} \cdot \text{rt} + \text{lf} \cdot \text{rf}$

Using $\text{tot} -$ (the easy one) saves writing three products and is less error-prone.

countTrueWays

JS

```
const countTrueWays = (e) => {
  const MOD = 1003;
  const n = e.length;
  if (n === 0) return 0;
  const k = (n + 1) >> 1; // number of operands
  const T = Array.from({ length: k }, () => new Array(k).fill(0));
  const F = Array.from({ length: k }, () => new Array(k).fill(0));
  for (let i = 0; i < k; i++) {
    const c = e[2*i];
    T[i][i] = c === 'T' ? 1 : 0;
    F[i][i] = c === 'F' ? 1 : 0;
  }
  for (let len = 2; len <= k; len++)
    for (let i = 0; i + len - 1 < k; i++) {
      const j = i + len - 1;
      for (let m = i; m < j; m++) {
        const op = e[2*m + 1];
        const lt = T[i][m], lf = F[i][m], rt = T[m+1][j], rf = F[m+1][j];
        const tot = (lt + lf) * (rt + rf);
        if (op === '&') { T[i][j] += lt * rt; F[i][j] += tot - lt * rt; }
        else if (op === '|') { F[i][j] += lf * rf; T[i][j] += tot - lf * rf; }
        else { T[i][j] += lt * rf + lf * rt; F[i][j] += lt * rt + lf * rf; }
        T[i][j] %= MOD; F[i][j] %= MOD;
      }
    }
  return T[0][k-1] % MOD;
};
```

Outputs: 'T|T&F^T' → 4; 'T^F|F' → 2; 'T' → 1; 'F' → 0; '' → 0; 'T&F' → 0; 'T|F' → 1; 'T^T' → 0; 'T^T^F' → 0.

TRAP

$T[i][i] = c === 'T' ? 1 : 0$, not $T[i][i] = c === 'T'$. C++ silently converts `bool` to `0/1`; JavaScript would store real booleans, and `true * true` does coerce to `1` but `tot - lt * rt` with booleans in it reads terribly and breaks the moment a sum exceeds 1. Store numbers in a numeric table.

Also note $(n + 1) \gg 1$ rather than $(n + 1) / 2$: JavaScript division produces a **fraction**, and `dp[2.5]` is a silently different property from `dp[2]`. Any index computed by division needs `>> 1`, `Math.floor`, or `| 0`. This is one of the most common C++-to-JavaScript porting bugs.

This DP **does** multiply, but the modulus is 1003, so every product stays under $1003^2 \cdot 4 \approx 4 \times 10^6$ — nowhere near 2^{53} . Plain numbers are exact here. Had the modulus been $10^9 + 7$, the same code would be wrong and would need `BigInt`.

Time $O(k^3)$ **Space** $O(k^2)$

Answer: 4 for "T|T&F^T".

The follow-up the interviewer asks next

“Why did you keep a False table too?”

Because the operators mix the two. `|` produces True from **any** pair that is not False-False, so the True count needs the False counts of both sides. A DP state must be “closed” — you must be able to compute it from the states you kept. Whenever an answer depends on a property you did not store, add that property to the state.

“The modulus 1003 is not prime. Does that matter?”

Not for this problem, because we only add and multiply — no division, no modular inverse. If the problem asked for a ratio or used division you would need a prime modulus or a different technique.

Example 27 [Adobe · pattern]

Assignment. n workers, n jobs, $\text{cost}[w][j]$ is what it costs worker w to do job j . Each worker gets exactly one job and each job exactly one worker. Minimise the total cost. $1 \leq n \leq 18$. Edge cases: $n = 1$; all costs equal; empty input.

SOLUTION

Approach 1 — Try every permutation ($O(n!)$ — $18!$ is about 6×10^{15})

Approach 2 — Bitmask DP ($O(2^n \cdot n)$ — $18 \times 2^{18} \approx 5 \times 10^6$, instant)

The insight: while filling jobs one worker at a time, **the only thing that matters about the past is which jobs are already used** — not which worker took which. So the state is a set of used jobs, and a set of $n \leq 18$ items is an 18-bit integer.

Even better: the number of workers already placed equals `popcount(mask)`, so the current worker index is free — no second dimension needed.

```
assignMinCost JS

const popcount = (m) => { let c = 0; while (m) { m &= m - 1; c++; } return c; };

const assignMinCost = (cost) => {
  const n = cost.length;
  if (n === 0) return 0;
  const dp = new Array(1 << n).fill(Infinity);
  dp[0] = 0;
  for (let mask = 0; mask < (1 << n); mask++) {
    if (dp[mask] === Infinity) continue;
    const w = popcount(mask); // next worker to place
    if (w === n) continue;
    for (let j = 0; j < n; j++)
      if (!(mask & (1 << j)))
        dp[mask | (1 << j)] = Math.min(dp[mask | (1 << j)], dp[mask] + cost[w][j]);
  }
  return dp[(1 << n) - 1];
};
```

Outputs: `[[9,2,7],[6,4,3],[5,8,1]]` → 9 (matching the brute-force permutation search); single worker `[[5]]` → 5; empty → 0; `[[1,1],[1,1]]` → 2. A randomised comparison against the $O(n!)$ version over 200 inputs reported 200 random cross-checks passed.

Time $O(2^n \cdot n)$ **Space** $O(2^n)$

Answer: 9 — worker 0 takes job 1 (2), worker 1 takes job 0 (6), worker 2 takes job 2 (1).

The follow-up the interviewer asks next

“Now n is 2000.”

Bitmask is dead (2^{2000} — and JavaScript bit operations gave up at 31 bits long before that). The right answer is the **Hungarian algorithm**, which solves the assignment problem in $O(n^3)$, or a min-cost max-flow formulation. You are not expected to code it — you are expected to **name** it and say why bitmask stops working. The boundary is roughly $n \leq 20$ for $O(2^n n)$ and $n \leq 15$ for $O(3^n)$.

“What if a worker can take up to two jobs?”

Keep the mask of used jobs but add a small second dimension — how many jobs the current worker already has — or iterate over pairs of free jobs for each worker. The state grows, the shape does not.

Example 28 [Uber · pattern]

Shortest tour (TSP). n cities with a full distance matrix. Start at city 0, visit every city exactly once, return to city 0. Minimise the distance. $2 \leq n \leq 18$. Edge cases: $n = 1$ (tour length 0); $n = 2$; a symmetric matrix.

SOLUTION

Approach 1 — Try all orders ($O(n!)$ — $18! \approx 6 \times 10^{15}$)

Approach 2 — Bitmask DP with a 'current city' dimension ($O(2^n \cdot n^2)$ — optimal known)

Unlike the assignment problem, **where you are standing matters** — the next distance depends on it. So the state needs two parts:

$\text{dp}[\text{mask}][u]$ = cheapest way to have visited exactly the set mask and be standing at city u .

$$\text{dp}[\text{mask} \mid (1 \ll v)][v] = \min(\text{dp}[\text{mask}][u] + d[u][v]) \quad \text{for } v \notin \text{mask}$$

Base: $\text{dp}[1][0] = 0$ — only city 0 visited, standing at city 0. Answer: $\min_{u \neq 0}(\text{dp}[\text{full}][u] + d[u][0])$ — close the loop.

tsp

JS

```
const tsp = (d) => {
  const n = d.length;
  if (n <= 1) return 0;
  const dp = Array.from({ length: 1 << n }, () => new Array(n).fill(Infinity));
  dp[1][0] = 0; // only city 0 visited, at city 0
  for (let mask = 1; mask < (1 << n); mask++)
    for (let u = 0; u < n; u++) {
      if (dp[mask][u] === Infinity || !(mask & (1 << u))) continue;
      for (let v = 0; v < n; v++) {
        if (mask & (1 << v)) continue;
        const nm = mask | (1 << v);
        dp[nm][v] = Math.min(dp[nm][v], dp[mask][u] + d[u][v]);
      }
    }
  const full = (1 << n) - 1;
  let best = Infinity;
  for (let u = 1; u < n; u++) best = Math.min(best, dp[full][u] + d[u][0]);
  return best;
};
```

Outputs: the classic four-city matrix $[[0, 10, 15, 20], [10, 0, 35, 25], [15, 35, 0, 30], [20, 25, 30, 0]] \rightarrow 80$; $[[0]] \rightarrow 0$; $[[0, 5], [5, 0]] \rightarrow 10$. A randomised comparison against the $O(n!)$ version on 100 random symmetric matrices reported 100 random cross-checks passed.

TRAP

`Array.from({ length: 1 << n }, () => new Array(n).fill(Infinity))` allocates 2^n **separate** rows. At $n = 18$ that is 262144 ordinary arrays — about 4.7 million numbers, and each JavaScript array carries its own header. It works, but it is heavy. If memory is the constraint, flatten to a single new `Float64Array((1 << n) * n)` and index it as `dp[mask * n + u]`: one allocation, no per-row overhead, and typed arrays initialise to 0 so you `fill(Infinity)` yourself.

Time $O(2^n \cdot n^2)$ **Space** $O(2^n \cdot n)$

$n = 18$ gives about 8×10^7 steps and 4.7 M ints — tight but fine

Answer: 80 — the tour $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$ costs $10 + 25 + 30 + 15 = 80$.

tsp — Python version

PYTHON

```
def tsp(d):
    n = len(d)
    if n <= 1:
        return 0
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1, 1 << n):
        for u in range(n):
            if dp[mask][u] == INF or not (mask >> u) & 1:
                continue
            for v in range(n):
                if (mask >> v) & 1:
                    continue
                nm = mask | (1 << v)
                if dp[mask][u] + d[u][v] < dp[nm][v]:
                    dp[nm][v] = dp[mask][u] + d[u][v]
    full = (1 << n) - 1
    return min(dp[full][u] + d[u][0] for u in range(1, n))
```

Output: 80, then 10 0.

The follow-up the interviewer asks next

“Memory is 4.7 million numbers at $n = 18$. Can you cut it?”

Yes — process masks in increasing order of popcount, keeping only two popcount layers. Or flatten the table into one `Float64Array` (or `Int32Array`, if the distances fit in 32 bits) and index it as `dp[mask * n + u]`: that removes 262144 array headers and keeps the values packed. Also note that any mask without bit 0 set is unreachable, so half the table is wasted; skipping those halves the memory again.

“ $n = 1000$, and you only need a good tour, not the best one.”

Say: nearest-neighbour construction, then 2-opt improvement, or Christofides for metric instances with a $1.5 \times$ guarantee. TSP is NP-hard; nobody expects exactness at 1000 cities.

TRAP

The difference between Example 27 and Example 28 is worth memorising. **Assignment** needs only `dp[mask]` because the cost of the next pick does not depend on the previous pick. **TSP** needs `dp[mask][u]` because the next edge starts at u . Ask yourself: “does the cost of the next step depend on which item I picked last?” If yes, add that dimension.

Dry run — edit distance, cell by cell

Turn horse into ros. Rows are the characters of horse, columns the characters of ros. $dp[i][j]$ = the fewest operations that turn the first i letters of horse into the first j letters of ros.

Step 1 — the base row and column

$dp[i][0] = i$ — to reach the empty string you delete every letter. $dp[0][j] = j$ — from the empty string you insert every letter.

		r	o	s
	0	1	2	3
h	1	.	.	.
o	2	.	.	.
r	3	.	.	.
s	4	.	.	.
e	5	.	.	.

Step 2 — row h

- $dp[1][1]$: h vs r, different $\rightarrow 1 + \min(dp[0][1], dp[1][0], dp[0][0]) = 1 + \min(1, 1, 0) = 1$
- $dp[1][2]$: h vs o, different $\rightarrow 1 + \min(2, 1, 1) = 2$
- $dp[1][3]$: h vs s, different $\rightarrow 1 + \min(3, 2, 2) = 3$

Step 3 — row o

- $dp[2][1]$: o vs r, different $\rightarrow 1 + \min(dp[1][1], dp[2][0], dp[1][0]) = 1 + \min(1, 2, 1) = 2$
- $dp[2][2]$: o vs o, **same** $\rightarrow$ copy the diagonal: $dp[1][1] = 1$
- $dp[2][3]$: o vs s, different $\rightarrow 1 + \min(3, 1, 2) = 2$

Step 4 — row r

- $dp[3][1]$: r vs r, **same** $\rightarrow$ diagonal $dp[2][0] = 2$
- $dp[3][2]$: r vs o, different $\rightarrow 1 + \min(dp[2][2], dp[3][1], dp[2][1]) = 1 + \min(1, 2, 2) = 2$
- $dp[3][3]$: r vs s, different $\rightarrow 1 + \min(2, 2, 1) = 2$

Step 5 — rows s and e

- $dp[4][1]$: s vs r $\rightarrow 1 + \min(2, 3, 3) = 3$
- $dp[4][2]$: s vs o $\rightarrow 1 + \min(2, 3, 2) = 3$
- $dp[4][3]$: s vs s, **same** $\rightarrow$ diagonal $dp[3][2] = 2$
- $dp[5][1]$: e vs r $\rightarrow 1 + \min(3, 4, 4) = 4$
- $dp[5][2]$: e vs o $\rightarrow 1 + \min(3, 4, 3) = 4$
- $dp[5][3]$: e vs s $\rightarrow 1 + \min(2, 4, 3) = 3$

The finished table (printed by the program)

		r	o	s
	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

Answer: $dp[5][3] = 3$.

Reading the operations backwards

Start at $dp[5][3] = 3$ and move to whichever neighbour justified the value.

cell	val	came from	operation
dp[5][3]	3	dp[4][3] = 2	delete e
dp[4][3]	2	dp[3][2] = 2 (match s)	keep s
dp[3][2]	2	dp[2][2] = 1	delete r
dp[2][2]	1	dp[1][1] = 1 (match o)	keep o
dp[1][1]	1	dp[0][0] = 0	replace h with r

Reading downwards: replace h → r, keep o, delete r, keep s, delete e. Three operations. horse → rorse → rose → ros. ✓

The same table as a two-row computation

The two-row version keeps only the previous row and the row being built. At the moment cur[2] is computed for row r:

	' '	r	o	s	meaning
prev (row o)	2	2	1	2	finished
cur (row r)	3	2	?	—	being built

The three numbers cur[2] needs are prev[2] = 1 (delete), cur[1] = 2 (insert) and prev[1] = 2 (replace). All three are available. That is the whole proof that two rows are enough.

TRAP

In the two-row version, prev[j - 1] must be read **before** cur[j - 1] overwrites anything you still need. Because cur and prev are separate arrays, this is automatic. If you try to squeeze it into a single array you must save the diagonal in a temporary variable first — a very common source of wrong answers.

Second dry run — the LCS table

Same idea, different rule. abcde against ace:

	' '	a	c	e
' '	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

Row b: b never matches a, c or e, so every cell copies the better of “above” and “left” — the row is unchanged from row a. Row c: at column c the characters match, so $dp[3][2] = dp[2][1] + 1 = 1 + 1 = 2$. Row e: at column e they match, so $dp[5][3] = dp[4][2] + 1 = 2 + 1 = 3$.

Backtracking from dp[5][3]: match e → go to dp[4][2]; d vs c differ and $dp[3][2] = 2 \geq dp[4][1] = 1$, go up to dp[3][2]; match c → dp[2][1]; b vs a differ, go up to dp[1][1]; match a → done. Collected backwards: e, c, a → reversed, ace.

Practice

PRACTICE · WARM-UP target: 15 min

- P1.** A grid of costs. Fall from the top row to the bottom row, but you may **not** land in the same column as the row before. Minimise the sum. $1 \leq m, n \leq 200$.
- P2.** Count **all** palindromic substrings of a string (single letters count). $1 \leq n \leq 1000$.
- P3.** Given a and b, report the fewest deletions from a and insertions into a that turn a into b. $1 \leq |a|, |b| \leq 1000$.

PRACTICE · TIER 1 target: 30 min

- P4.** Longest common subsequence of **three** strings. $1 \leq \text{each length} \leq 100$.
- P5.** n houses, k colours, $\text{cost}[i][c]$ to paint house i colour c, no two neighbouring houses share a colour. Minimise the cost in $O(nk)$ — not $O(nk^2)$. $1 \leq n \leq 10^5, 2 \leq k \leq 20$.
- P6.** Height of a rooted tree is easy. Now find its **diameter**: the largest number of edges on any path between two nodes. $1 \leq n \leq 10^5$.
- P7.** Every node of a rooted tree has a (possibly negative) value. Find the largest sum of any **subtree**. $1 \leq n \leq 10^5$.

PRACTICE · TIER 2 target: 40 min

- P8.** Regular expression matching with `.` (any single character) and `*` (zero or more of the **previous** character). Must match the whole string. $0 \leq |s|, |p| \leq 1000$.
- P9.** Piles of stones in a row. Repeatedly merge two **adjacent** piles; the cost of a merge is the number of stones merged. Merge everything into one pile at minimum cost. $1 \leq n \leq 300$.
- P10.** Count the right/down paths in a grid with walls, where you may **remove at most one wall**. $1 \leq m, n \leq 500$.

PRACTICE · TIER 3 target: 40 min

- P11.** Maximum weight independent set on a tree — and **print** the chosen nodes. $1 \leq n \leq 10^5$.
- P12.** $2m$ people and a table saying which pairs may be partners. Count the ways to split everybody into partner pairs. $2m \leq 16$.
- P13.** $n \leq 15$ items with weights. Put them into groups so that every group's total weight is at most C. Use the fewest groups.
- P14.** A tree with n nodes and k colours. Count the ways to colour the nodes so that no edge joins two nodes of the same colour, modulo $10^9 + 7$.

ANSWER KEY

P1. The forbidden column means “take the best of the previous row **excluding one column**”. Keep the two smallest values of the previous row and their position: if the forbidden column is where the smallest sits, use the second smallest. That is $O(mn)$, not $O(mn^2)$.

P1

JS

```
const minFallingNoSameCol = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  if (n === 1) return m === 1 ? g[0][0] : Infinity; // no legal move
  let prev = [...g[0]], cur = new Array(n).fill(0);
  for (let i = 1; i < m; i++) {
    let b1 = Infinity, b2 = Infinity, bli = -1;
    for (let j = 0; j < n; j++) {
      if (prev[j] < b1) { b2 = b1; b1 = prev[j]; bli = j; }
      else if (prev[j] < b2) b2 = prev[j];
    }
    for (let j = 0; j < n; j++) cur[j] = g[i][j] + (j === bli ? b2 : b1);
    [prev, cur] = [cur, prev];
  }
  return prev.reduce((a, b) => a < b ? a : b);
};
```

Outputs: $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow 13$; $[[7]] \rightarrow 7$; $[[1,99],[99,1]] \rightarrow 2$; $[[2,2],[2,2],[2,2]] \rightarrow 6$; $[] \rightarrow 0$. **Edge case:** one column and more than one row has no legal move at all, and returns *Infinity* — which is honest, and prints as *Infinity* rather than as a huge fake number.

P2. Fill a `pal[i][j]` table by increasing length and count the `true` cells.

```
P2 JS
const countPalindromicSubstrings = (s) => {
  const n = s.length; if (!n) return 0;
  const pal = Array.from({ length: n }, () => new Array(n).fill(false));
  let total = 0;
  for (let len = 1; len <= n; len++)
    for (let i = 0; i + len - 1 < n; i++) {
      const j = i + len - 1;
      if (len === 1) pal[i][j] = true;
      else if (len === 2) pal[i][j] = s[i] === s[j];
      else pal[i][j] = s[i] === s[j] && pal[i+1][j-1];
      if (pal[i][j]) total++;
    }
  return total;
};
```

Outputs: `'abc' → 3`; `'aaa' → 6`; `'' → 0`; `'a' → 1`; `'abba' → 6`; `'abcba' → 7`. Time $O(n^2)$, space $O(n^2)$. The expand-around-centre method gets $O(1)$ space.

P3. Everything outside the LCS must go or be added: deletions = $|a| - L$, insertions = $|b| - L$, where $L = |\text{LCS}(a, b)|$.

```
P3 JS
const convertCost = (a, b) => {
  const l = lcsTable(a, b);
  return [a.length - l, b.length - l]; // [deletions, insertions]
};
```

Outputs: $(\text{'heap'}, \text{'pea'}) \rightarrow 2 \text{ deletions}, 1 \text{ insertion (LCS is ea)}$; $(\text{'abc'}, \text{'abc'}) \rightarrow 0, 0$; $(\text{''}, \text{'abc'}) \rightarrow 0, 3$.

P4. Three indices, three-way match, three-way max.

```
P4 JS
```



```
const lcs3 = (a, b, c) => {
  const x = a.length, y = b.length, z = c.length;
  const dp = Array.from({ length: x+1 }, () =>           // three nested factories,
    Array.from({ length: y+1 }, () =>                   // never a nested fill
      new Array(z+1).fill(0)));
  for (let i = 1; i <= x; i++)
    for (let j = 1; j <= y; j++)
      for (let k = 1; k <= z; k++)
        dp[i][j][k] = (a[i-1] === b[j-1] && b[j-1] === c[k-1])
          ? dp[i-1][j-1][k-1] + 1
          : Math.max(dp[i-1][j][k], dp[i][j-1][k], dp[i][j][k-1]);
  return dp[x][y][z];
};
```

Outputs: ('abcd', 'acbd', 'aacd') → 3; ('abc', 'abc', 'abc') → 3; ('abc', 'def', 'ghi') → 0; ('', 'abc', 'abc') → 0; ('geeks', 'geeksfor', 'geeksforgeeks') → 5. Time $O(xyz)$, space $O(xyz)$ — at 100 each that is 10^6 cells, fine.

P5. Naive is $O(nk^2)$: for every colour, scan every other colour. Instead keep the **two smallest** values of the previous row, exactly as in P1.

```
P5 JS
const paintKColours = (cost) => {
  const n = cost.length; if (!n) return 0;
  const k = cost[0].length; if (!k) return 0;
  let prev = [...cost[0]], cur = new Array(k).fill(0);
  for (let i = 1; i < n; i++) {
    let b1 = Infinity, b2 = Infinity, bli = -1;
    for (let c = 0; c < k; c++) {
      if (prev[c] < b1) { b2 = b1; b1 = prev[c]; bli = c; }
      else if (prev[c] < b2) b2 = prev[c];
    }
    for (let c = 0; c < k; c++) cur[c] = cost[i][c] + (c === bli ? b2 : b1);
    [prev, cur] = [cur, prev];
  }
  return prev.reduce((a, b) => a < b ? a : b);
};
```

Outputs: [[17,2,17],[16,16,5],[14,3,19]] → 10; [[1,5,3],[2,9,4]] → 5; [[9,4,7]] → 4; [] → 0; [[1,1],[1,1],[1,1]] → 3.

P6. One DFS. At each node keep the two deepest child depths; the diameter through that node is their sum.

```
P6 JS
const treeDiameter = (root, ch) => {
  let diameter = 0; // local, so repeated calls cannot
  pollute it
  const depthOf = (u) => {
    let b1 = 0, b2 = 0;
    for (const v of ch[u]) {
      const d = depthOf(v) + 1;
      if (d > b1) { b2 = b1; b1 = d; }
      else if (d > b2) b2 = d;
    }
    diameter = Math.max(diameter, b1 + b2);
  }
  depthOf(root);
  return diameter;
};
```

```

    return b1;
};
depthOf(root);
return diameter;
};

```

Outputs: the tree $0 \rightarrow 1, 2$; $2 \rightarrow 3, 4 \rightarrow 3$; a single node $\rightarrow 0$; a chain of $4 \rightarrow 3$; a star with 4 leaves $\rightarrow 2$. **Note:** the answer counts **edges**. If the question counts nodes, add 1. **JavaScript note:** wrapping the accumulator in a closure is not decoration — a module-level `let diameter` would carry the previous call's value into the next test.

P7. Post-order sum, taking the maximum as you go.

P7

JS

```

const bestSubtreeSum = (root, ch, val) => {
  let best = -Infinity;
  const subSum = (u) => {
    let s = val[u];
    for (const v of ch[u]) s += subSum(v);
    best = Math.max(best, s);
    return s;
  };
  const total = subSum(root);
  return [total, best];
};

```

Outputs on $0 \rightarrow 1, 2$; $1 \rightarrow 3, 4$: values $[1, -5, 4, 3, 3] \rightarrow$ total 6, best subtree 6; values $[-1, -2, -3, -4, -5] \rightarrow$ total -15 , best subtree -3 (the single node 2); a single node $[7] \rightarrow 7$ and 7. **Trap:** best must start at $-Infinity$. Starting at 0 returns 0 for an all-negative tree.

P8. * refers to the character **before** it, so a star always occupies two pattern slots.

P8

JS

```

const regexMatch = (s, p) => {
  const n = s.length, m = p.length;
  const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(false));
  dp[0][0] = true;
  for (let j = 2; j <= m; j++) if (p[j-1] === '*') dp[0][j] = dp[0][j-2];
  for (let i = 1; i <= n; i++)
    for (let j = 1; j <= m; j++) {
      if (p[j-1] === '*') {
        if (j < 2) { dp[i][j] = false; continue; }
        const zero = dp[i][j-2]; // "x*"
        matches nothing
        const more = (p[j-2] === '.' || p[j-2] === s[i-1]) && dp[i-1][j]; // one
        more x
        dp[i][j] = zero || more;
      } else dp[i][j] = (p[j-1] === '.' || p[j-1] === s[i-1]) && dp[i-1][j-1];
    }
  return dp[n][m];
};

```

Outputs: ('aa', 'a') $\rightarrow$ no; ('aa', 'a*') $\rightarrow$ yes; ('ab', '.*') $\rightarrow$ yes; ('aab', 'c*a*b') $\rightarrow$ yes; ('mississippi', 'mis*is*p*.') $\rightarrow$ no; ('', '') $\rightarrow$ yes; ('', 'a*') $\rightarrow$ yes; ('', 'a') $\rightarrow$ no; ('a', '') $\rightarrow$ no; ('abc', 'a.c') $\rightarrow$ yes; ('aaa', 'a*a') $\rightarrow$ yes; ('aaa', 'ab*a*c*a') $\rightarrow$ yes. **JavaScript note:** do not reach for the built-in RegExp here. `new RegExp(p)` would match a **substring** unless you anchor it, '.' and '*' mean different things

in a real regular expression, and the interviewer is asking you to write the DP, not to call a library. **Difference from wildcard matching:** here * needs a partner character; there * stood alone. Read the question twice before choosing a recurrence.

P9. Interval DP. A prefix-sum array gives the cost of the final merge of any interval in $O(1)$.

P9

JS

```
const mergeStones = (a) => {
  const n = a.length;
  if (n <= 1) return 0;
  const pre = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) pre[i+1] = pre[i] + a[i];
  const dp = Array.from({ length: n }, () => new Array(n).fill(0));
  for (let len = 2; len <= n; len++)
    for (let i = 0; i + len - 1 < n; i++) {
      const j = i + len - 1;
      dp[i][j] = Infinity;
      for (let k = i; k < j; k++)
        dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k+1][j]);
      dp[i][j] += pre[j+1] - pre[i];
    }
  return dp[0][n-1];
};
```

Outputs: [4,3,3,4] → 28; [1,2,3] → 9; [5] → 0; [] → 0; [1,1,1,1] → 8; [10,1,1,10] → 36. **Why += pre[j+1] - pre[i] outside the k loop?** Because the final merge of the interval always costs the whole interval's weight, whatever the split point is.

P10. Add one bit to the state: how many walls have been removed (0 or 1).

P10

JS

```
const pathsWithOneRemoval = (g) => {
  const m = g.length; if (!m) return 0;
  const n = g[0].length; if (!n) return 0;
  // dp[i][j] = [ways with no wall removed, ways with exactly one removed]
  const dp = Array.from({ length: m }, () => Array.from({ length: n }, () => [0, 0]));
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++) {
      let in0 = 0, in1 = 0;
      if (i === 0 && j === 0) { in0 = 1; in1 = 0; }
      else {
        if (i > 0) { in0 += dp[i-1][j][0]; in1 += dp[i-1][j][1]; }
        if (j > 0) { in0 += dp[i][j-1][0]; in1 += dp[i][j-1][1]; }
      }
      if (g[i][j] === 1) { dp[i][j][0] = 0; dp[i][j][1] = in0; }
      else { dp[i][j][0] = in0; dp[i][j][1] = in1; }
    }
  return dp[m-1][n-1][0] + dp[m-1][n-1][1];
};
```

Outputs: [[0,0,0],[0,1,0],[0,0,0]] → 6 (all six paths become legal); [[0,1],[1,0]] → 2; [[0,1,0],[1,1,1],[0,1,0]] → 0 (every route needs two removals); [[1]] → 1 (remove the start wall); [[0]] → 1; [[0,0],[1,1],[0,0]] → 2. **Note the doubly nested Array.from.** The inner pair [0, 0] must be a fresh array per cell; new Array(n).fill([0, 0]) would give every cell in a row the same pair.

P11. Run the tree DP of Example 21 storing both values per node, then walk down from the root deciding at each node, remembering whether the parent was taken.

P11 — the DFS

JS

```
const dfs = (u) => {
  withU[u] = val[u];
  withoutU[u] = 0;
  for (const v of ch[u]) {
    dfs(v);
    withU[u] += withoutU[v];
    withoutU[u] += Math.max(withU[v], withoutU[v]);
  }
};
```

P11 — the reconstruction

JS

```
const collect = (u, allowed, out) => {
  const takeU = allowed && withU[u] >= withoutU[u];
  if (takeU) out.push(u);
  for (const v of ch[u]) collect(v, !takeU, out);
};
```

Both live inside one `mwisTree(root, ch, val)` wrapper that owns `withU` and `withoutU`, runs `dfs(root)`, then `collect(root, true, out)`, and returns `[Math.max(withU[root], withoutU[root]), out]`. On the tree $0 \rightarrow 1, 2; 1 \rightarrow 3, 4; 2 \rightarrow 5, 6$: values `[10, 5, 5, 4, 4, 4, 4]` $\rightarrow$ total 26, nodes `0 3 4 5 6`; values `[1, 50, 1, 1, 1, 1, 1]` $\rightarrow$ total 52, nodes `1 5 6`; a single node `[9]` $\rightarrow$ 9.

P12. Bitmask over people. Always pair the **lowest unpaired** person — that removes the double counting automatically.

P12

JS

```
const countPairings = (ok) => {
  const n = ok.length;
  if (n === 0) return 1;
  if (n % 2) return 0;
  const dp = new Array(1 << n).fill(0);
  dp[0] = 1;
  for (let mask = 0; mask < (1 << n); mask++) {
    if (!dp[mask]) continue;
    let i = 0;
    while (i < n && (mask & (1 << i))) i++; // lowest unpaired person
    if (i === n) continue;
    for (let j = i + 1; j < n; j++)
      if (!(mask & (1 << j)) && ok[i][j])
        dp[mask | (1 << i) | (1 << j)] += dp[mask];
  }
  return dp[(1 << n) - 1];
};
```

Outputs: 4 people, everyone compatible $\rightarrow$ 3; 6 people, everyone compatible $\rightarrow$ 15; 2 people who may **not** pair $\rightarrow$ 0; 0 people $\rightarrow$ 1; 4 people where only 0-1 and 2-3 are allowed $\rightarrow$ 1. **Why fix the lowest unpaired person?** Otherwise the pairing $\{(0, 1), (2, 3)\}$ is counted once for each order in which the pairs are formed.

P13. Bitmask plus a submask loop. `sum[mask]` is precomputed with the lowest-bit trick.

P13

JS

```
const minGroups = (w, C) => {
  const n = w.length;
```

```

if (n === 0) return 0;
const full = (1 << n) - 1;
const sum = new Array(1 << n).fill(0);
for (let mask = 1; mask <= full; mask++) {
    const lb = mask & -mask, i = 31 - Math.clz32(lb); // lowest set bit, and its
index
    sum[mask] = sum[mask ^ lb] + w[i];
}
const dp = new Array(1 << n).fill(Infinity);
dp[0] = 0;
for (let mask = 0; mask <= full; mask++) {
    if (dp[mask] === Infinity) continue;
    const rest = full ^ mask;
    for (let sub = rest; sub; sub = (sub - 1) & rest) // every sub-set of `rest`
        if (sum[sub] <= C) dp[mask | sub] = Math.min(dp[mask | sub], dp[mask] + 1);
}
return dp[full] === Infinity ? -1 : dp[full];
};

```

Outputs: [3,3,3,3] with $C=6 \rightarrow 2$, $C=5 \rightarrow 4$, $C=12 \rightarrow 1$; [10] with $C=5 \rightarrow -1$, $C=10 \rightarrow 1$; [] $\rightarrow 0$; [1,2,3,4,5] with $C=5 \rightarrow 3$, $C=15 \rightarrow 1$. **JavaScript note:** `31 - Math.clz32(lb)` is the `__builtin_ctz` of C++ — `Math.clz32` counts leading zeros in a 32-bit word, and for a value with exactly one bit set that gives the bit's index directly. Verified: `31 - Math.clz32(0b1000)` is 3. Time $O(3^n)$ — safe to $n = 15$, too slow at $n = 20$.

P14. Tree DP, but the answer collapses to a formula: the root has k choices and every other node has $k-1$ (anything except its parent's colour).

P14 — the tree DP (used here to verify the formula)

JS

```

const MOD = 1000000007;

const dpColour = (u, parentColour, ch, k) => {
    let total = 0;
    for (let c = 1; c <= k; c++) {
        if (c === parentColour) continue;
        let ways = 1;
        for (const v of ch[u]) ways = ways * dpColour(v, c, ch, k) % MOD;
        total = (total + ways) % MOD;
    }
    return total;
};

```

The modular-multiply caveat applies here. `ways * dpColour(...)` can reach $(10^9)^2 = 10^{18}$, past $2^{53} - 1$, so on a large tree this multiply loses precision. It is exact for the small trees below, and the closed form makes it moot — but if you ever need this DP at scale, do the multiply in `BigInt` (or with a `mulmod` helper that splits the operands). Naming that limitation is the point of the exercise. The closed form is $k(k-1)^{n-1}$. On the tree $0 \rightarrow 1, 2; 1 \rightarrow 3, 4$ ($n = 5$) the two methods agree exactly (both were run): $k = 1 \rightarrow 0$, $k = 2 \rightarrow 2$, $k = 3 \rightarrow 48$, $k = 4 \rightarrow 324$. A single node with $k = 3 \rightarrow 3$. A star with 4 nodes and $k = 3 \rightarrow 24$. **Interview point:** notice that the answer does not depend on the **shape** of the tree at all, only on n . That is because every node has exactly one parent.

ONE-PAGE REVISION CARD

The four lines every DP in this chapter starts from.

Setup

JS

```
const grid2D = (r, c, v) =>
  Array.from({ length: r }, () => new Array(c).fill(v));
const popcount = (m) => {
  let c = 0; while (m) { m &= m - 1; c++; } return c;
};
const lowBitIndex = (m) => 31 - Math.clz32(m & -m); // __builtin_ctz
const deepCopy = (g) => g.map(r => [...r]); // [...g] is NOT enough
```

Unreached is Infinity, worst-so-far is -Infinity, swap two rows with `[prev, cur] = [cur, prev]`. No toolkit import is needed in this chapter — dynamic programming needs arrays, not a heap, a DSU or a deque. When a DP problem **does** need a priority queue, that is the **JS Toolkit** appendix's MinHeap.

Template 1 — grid. One rolling row.

Grid

JS

```
const row = new Array(n).fill(1);
for (let i = 1; i < m; i++)
  for (let j = 1; j < n; j++)
    row[j] += row[j-1]; // row[j] = above, row[j-1] = left
return row[n-1];
```

Template 2 — two strings. $dp[i][j]$ = the FIRST i and the FIRST j characters.

Two strings

JS

```
const dp = Array.from({ length: n+1 }, () => new Array(m+1).fill(0));
for (let i = 1; i <= n; i++)
  for (let j = 1; j <= m; j++)
    dp[i][j] = a[i-1] === b[j-1] ? dp[i-1][j-1] + 1 // LCS: match
    : Math.max(dp[i-1][j], dp[i][j-1]);
// substring? write 0 on mismatch and track the max cell instead
return dp[n][m];
```

Template 3 — interval. Loop by LENGTH, never by i then j .

Interval

JS

```
for (let len = 2; len <= n; len++)
  for (let i = 0; i + len - 1 < n; i++) {
    const j = i + len - 1;
    dp[i][j] = Infinity;
    for (let k = i; k < j; k++)
      dp[i][j] = Math.min(dp[i][j], dp[i][k] + dp[k+1][j] + cost(i, k, j));
  }
```

Template 4 — tree. One DFS, return a pair.

Tree

JS

```

const go = (u) => {
  let take = val[u], skip = 0;
  for (const v of ch[u]) {
    const [t, s] = go(v);
    take += s; // u taken -> children skipped
    skip += Math.max(t, s); // u skipped -> children free
  }
  return [take, skip];
};
return Math.max(...go(root)); // depth ~10^4: then an explicit stack

```

Template 5 — bitmask. Two shapes; pick by one question.

Bitmask

JS

```

// Shape A – dp[mask] only. Use when the next cost does NOT
// depend on which item was picked last.
const dp = new Array(1 << n).fill(Infinity);
dp[0] = 0;
for (let mask = 0; mask < (1 << n); mask++) {
  if (dp[mask] === Infinity) continue;
  const i = popcount(mask); // next slot, free of charge
  if (i === n) continue;
  for (let j = 0; j < n; j++)
    if (!(mask & (1 << j)))
      dp[mask | (1 << j)] = Math.min(dp[mask | (1 << j)], dp[mask] + cost(i, j));
}

// Shape B – dp[mask][u]. Use when WHERE YOU STAND changes the next cost.
dp2[1][0] = 0;
for (let mask = 1; mask < (1 << n); mask++)
  for (let u = 0; u < n; u++) {
    if (dp2[mask][u] === Infinity || !(mask & (1 << u))) continue;
    for (let v = 0; v < n; v++)
      if (!(mask & (1 << v)))
        dp2[mask | (1 << v)][v] =
          Math.min(dp2[mask | (1 << v)][v], dp2[mask][u] + d[u][v]);
  }

// Sub-sets of a mask, for "split into groups": O(3^n) overall.
for (let sub = rest; sub; sub = (sub - 1) & rest) { /* ... */ }

```

When to use which shape.

The question says...	Shape	State
move right / down on a grid	grid	$dp[i][j]$ = cell
compare, align, transform two sequences	two strings	$dp[i][j]$ = prefixes
insert brackets, cut, merge, burst	interval	$dp[i][j]$ = the piece $i..j$
subtree, parent, child, no two adjacent	tree	$dp[u]$ = a pair per node
$n \leq 20$, “assign each to each”	bitmask A	$dp[mask]$
$n \leq 20$ and the order matters (a tour)	bitmask B	$dp[mask][u]$

$n \leq 15$, “split into groups”	bitmask + submask	dp[mask], sub-set loop
“at most one exception / one removal”	any of the above + 1 bit	dp[...][0..1]

Complexity table.

Problem	Time	Space	Note
grid paths / min path sum	$O(mn)$	$O(n)$	one rolling row
falling path, two robots	$O(mn) / O(mn^2)$	$O(n) / O(n^2)$	9 moves is a constant
LCS, edit distance, interleaving	$O(nm)$	$O(m)$	two rows; full table only to print the answer
longest common substring	$O(nm)$	$O(m)$	answer is the max cell, not the corner
longest palindromic subsequence	$O(n^2)$	$O(n)$	LCS of s and its reverse
distinct subsequences	$O(nm)$	$O(m)$	j loop BACKWARDS
count / largest all-ones square	$O(mn)$	$O(n)$	1 + min of three neighbours
wildcard / regex matching	$O(nm)$	$O(m)$	
palindrome cuts	$O(n^2)$	$O(n^2)$	$O(n)$ space with expand-around-centre
matrix chain, stick cut, merge piles	$O(n^3)$	$O(n^2)$	interval; $n \leq 300$
bursting balloons	$O(n^3)$	$O(n^2)$	decide who bursts last
bracketings to True	$O(k^3)$	$O(k^2)$	keep a False table too
tree DP (rob, diameter, best path)	$O(n)$	$O(h)$	h is the recursion depth
assignment (bitmask A)	$O(2^n n)$	$O(2^n)$	$n \leq 20$
TSP (bitmask B)	$O(2^n n^2)$	$O(2^n n)$	$n \leq 18$
split into groups (submask)	$O(3^n)$	$O(2^n)$	$n \leq 15$

The JavaScript facts this chapter depends on.

Array(n).fill([])	stores one array in all n slots, so writing dp[0][1] writes every row. Always Array.from({ length: n }, () => ...).
[...grid]	a SHALLOW copy — the rows are shared. A real 2D copy is grid.map(r => [...r]).
Infinity	replaces INT_MAX. Infinity + w stays Infinity, so a sentinel can never be relaxed by accident. Use -Infinity for a max accumulator.
numbers are doubles	exact only to $2^{53} - 1$. uniquePaths(29,29) is exact; uniquePaths(32,32) is wrong . Past that, BigInt.
modular multiply	(a * b) % (10 ⁹ +7) is WRONG in plain numbers — the product reaches 10 ¹⁸ . Additive counting mod 10 ⁹ + 7 is fine. A small modulus like 1003 is fine either way.
recursion depth	about 10 ⁴ frames. Every tree DP here throws RangeError on a 100000-node chain — use an explicit stack.
bitwise ops	32-bit signed. 1 << 31 is negative; bitmask DP is capped near $n = 30$ regardless of time.
integer division	(n+1) / 2 gives a fraction . Index arithmetic needs >> 1, Math.floor or 0.

strings	immutable: build answers in an array and <code>join('')</code> . There is no <code>s.reverse()</code> — use <code>[...s].reverse().join('')</code> .
<code>arr.sort()</code>	LEXICOGRAPHIC. <code>[10,9,1].sort()</code> gives <code>[1,10,9]</code> . Numbers always need <code>sort((a, b) => a - b)</code> , and a comparator must return a number , not a boolean.
<code>Math.max(...arr)</code>	spreads every element as an argument and throws <code>RangeError</code> past roughly 2×10^5 . Use <code>arr.reduce((a, b) => a > b ? a : b)</code> .
no popcount	write the Kernighan loop. No <code>__builtin_ctz</code> either — <code>31 - Math.clz32(m & -m)</code> .

The top traps, in the order they bite.

1. **Index shift.** `dp[i][j]` means the **first i** characters, so the character is `a[i-1]`. Pick that convention once and never change it. This is the number-one 2D DP bug.
2. **Interval loops.** `len` outermost. `i` then `j` reads cells that are not ready yet.
3. **Burst LAST, not first.** The two sides are only independent once you fix which element survives longest. Say the sentence; it is what the interviewer is listening for.
4. **Backwards j loop** in distinct-subsequences, exactly as in 0/1 knapsack. Forwards turns `('aaa', 'aa')` from 3 into 6.
5. **The base case written in the wrong place.** `cur[0] = i` belongs inside the `i` loop, before the `j` loop — the rolling arrays are reused, so stale data really is waiting.
6. **The accumulator.** Start it at `-Infinity`, not 0, or an all-negative tree returns 0 — and keep it in a closure, because a module-level `let` carries one test's answer into the next.
7. **The length check first.** `isInterleave` must reject `|c| != |a| + |b|` before indexing, because an out-of-range string index is undefined, not a crash.

The one sentence to remember. Nothing about the method changed from Chapter 17 — only the **shape of the state**. Name the state, name the transition, name the base case, and the code writes itself; everything else on this page is JavaScript bookkeeping.

Greedy & Intervals

Sort by the right key, take the best local move, prove it with an exchange argument

Pattern in one page

A **greedy** algorithm builds the answer one step at a time. At every step it takes the move that looks best **right now** and never takes it back. That is all. The whole skill is deciding **what “best right now” means**, and proving that the local choice cannot hurt the final answer.

THE GREEDY CHECKLIST

Step 1 — find the sort key. Almost every greedy problem in a coding round starts with a sort. The interview is really asking: sort by **what**?

Goal	Sort key that works
Pick the most non-overlapping intervals	earliest finish time
Merge / union of intervals	earliest start time
Minimum rooms, maximum overlap	starts and ends separately (sweep)
Fractional knapsack	value / weight, descending
Job with deadline + profit	profit descending, then fill the latest free day
Pair two lists to satisfy the most	both lists ascending, two pointers
Minimum groups of size ≤ 2 under a cap	ascending, then lightest with heaviest
Join pieces to make the biggest number	custom: $a + b > b + a$ as strings

Step 2 — state the invariant. One sentence that stays true after every step. Example, activity selection: “after processing k intervals, `freeAt` is the earliest possible finishing time among all ways to pick this many non-overlapping intervals.”

Step 3 — the exchange argument (this is the proof). Assume some optimal answer O differs from the greedy answer G . Look at the **first** place they differ. Swap the optimal’s choice for the greedy’s choice. Show the answer is still valid and **no worse**. Repeat. After enough swaps O has turned into G , so G is optimal too.

Step 4 — sanity-check with a counterexample hunt. Before you code, spend 30 seconds trying to break your own rule with a tiny 3-element input. If you cannot break it in 30 seconds, code it.

The two template loops you will reuse all chapter

Template A — merge / union (sort by START)

JS

```
v.sort((a, b) => a[0] - b[0] || a[1] - b[1]);           // by start, then by end
const out = [v[0]];
for (let i = 1; i < v.length; i++) {
  const last = out[out.length - 1];
  if (v[i][0] <= last[1])                             // touches the open block
    last[1] = Math.max(last[1], v[i][1]);
  else
    out.push(v[i]);                                   // clean gap -> new block
}
```

```

v.sort((a, b) => a[1] - b[1]); // by END, numerically
let cnt = 0, freeAt = -Infinity;
for (const [s, e] of v)
  if (s >= freeAt) { cnt++; freeAt = e; } // fits -> take it, never undo

```

TRAP

arr.sort() in JavaScript is LEXICOGRAPHIC, not numeric. `[10, 9, 1].sort()` gives `[1, 10, 9]`, because the default comparator turns every element into a string first. Numbers **always** need an explicit comparator: `arr.sort((a, b) => a - b)`. This chapter sorts in almost every solution, so the rule is: if you typed `.sort()` with empty brackets, you have a bug.

Two more comparator rules that go with it:

- A comparator must return a **number**, not a boolean. `(a, b) => a[1] > b[1]` returns `true/false`, which coerce to `1/0` — the sort never sees a negative and leaves the array essentially untouched. Write `(a, b) => a[1] - b[1]`.
- Chain tie-breaks with `||`, because `0` is falsy: `(a, b) => a[0] - b[0] || a[1] - b[1]`.
- `arr.sort()` sorts **in place** and returns the same array. Every solution in this chapter starts with `[...v]` or `v.map(iv => [...iv])` so the caller's array is not silently reordered.

SHORTCUT

Almost every interval question is Template A or Template B with a different word problem wrapped around it. Learn to see which one it is in the first 20 seconds:

- The answer is a **list of intervals** → Template A (sort by start).
- The answer is a **count** → Template B (sort by end).
- The answer is **how many at once** → sweep line (starts and ends separately).

Open or closed? Decide before you code

An interval $[s, e]$ can mean two different things and the code changes by one character.

Meaning	Overlap test	Typical wording
Closed $[s, e]$ — both ends count	$\max(s1, s2) \leq \min(e1, e2)$	a balloon spans $x = 2$ to $x = 5$
Half-open $[s, e)$ — end does not count	$\max(s1, s2) < \min(e1, e2)$	a meeting from 2 pm to 5 pm

TRAP

The single most common wrong answer in an interval question is treating “meeting ends at 5, next starts at 5” as a clash. A meeting that **ends** at 5 frees the room at 5. Write the rule on your rough sheet before you write the first line of code.

When greedy is WRONG

Greedy fails the moment a local best move can block a bigger future gain. Two examples you must be able to produce on demand in an interview:

1. **Coin change with a strange coin set.** Coins $\{25, 10, 1\}$, amount 30. Greedy takes 25 and then five 1s = 6 coins. The best answer is $10 + 10 + 10 = 3$ coins.
2. **Intervals with weights.** Meetings $(1, 4)$ worth 30, $(2, 6)$ worth 90, $(5, 8)$ worth 20. “Earliest finish” greedy takes $(1, 4)$ and $(5, 8)$ for 50. The best is the single meeting $(2, 6)$ for 90.

Both need dynamic programming. We solve both later in this chapter.

Warm-up — build the reflex

WARM-UP

build the reflex

Example 1 [TCS NQT pattern]

You have notes of value 1, 2, 5, 10, 20, 50, 100, 200, 500. Pay an amount with the **fewest** notes. Print that count, or -1 if the amount cannot be paid exactly.

Constraints: amount up to 10^9 . **Target:** $O(k \log k)$ where k is the number of distinct notes. **Edge cases:** amount = 0; a coin set where exact payment is impossible.

SOLUTION

Sort the notes biggest first. Take as many of each as fit. This **particular** note set is “canonical”, which means greedy is provably correct on it.

minCoins — greedy on a canonical note set

JS

```
const minCoins = (amount, coins) => {
  const sorted = [...coins].sort((a, b) => b - a); // biggest first
  let used = 0;
  for (const c of sorted) {
    used += Math.floor(amount / c); // take as many as fit
    amount %= c; // keep the leftover
  }
  return amount === 0 ? used : -1; // -1 = cannot pay exactly
};
```

Trace for amount = 287:

Note	How many fit	Used so far	Left
500	0	0	287
200	1	1	87
100	0	1	87
50	1	2	37
20	1	3	17
10	1	4	7
5	1	5	2
2	1	6	0
1	0	6	0

Tested: minCoins(287, indian) prints 6, minCoins(0, indian) prints 0, minCoins(3, [2, 4]) prints -1, and minCoins(1e9, indian) prints 2000000.

Answer: 6 notes: 200 + 50 + 20 + 10 + 5 + 2

Example 2 [Infosys pattern]

Two closed intervals $[a_1, a_2]$ and $[b_1, b_2]$. Do they share at least one point?

Constraints: values fit in a 32-bit range. **Target:** $O(1)$. **Edge cases:** they touch at exactly one point; one sits fully inside the other.

SOLUTION

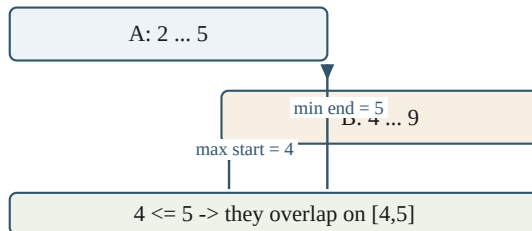
Do **not** write four `if` cases. There is a one-line rule.

Two intervals overlap when the later of the two starts is not after the earlier of the two ends.

overlap — the one-line rule

JS

```
const overlap = (a1, a2, b1, b2) => Math.max(a1, b1) <= Math.min(a2, b2);
```



max of the starts vs min of the ends

Tested: `overlap(2,5,4,9) → true`, `overlap(2,5,5,9) → true` (they touch at 5), `overlap(2,5,6,9) → false`.

Answer: `max(a1,b1) <= min(a2,b2)`

Example 3 [Capgemini pattern]

Prices of n items and a budget B . Buy the **most items** possible. You may buy each item at most once.

Constraints: $n \leq 10^5$, prices up to 10^9 , B up to 10^{14} . **Target:** $O(n \log n)$. **Edge cases:** empty price list; $B = 0$.

SOLUTION

Wanting the **count** and not the **value** is the key. Cheap items are strictly better than dear ones because every item adds exactly 1 to the count. So sort ascending and buy until the money runs out.

Exchange argument. Suppose an optimal basket skips the cheapest item c but includes some item d with $d \geq c$. Swap d for c . The count stays the same and the money spent does not go up. So there is always an optimal basket that contains the cheapest item. Repeat on the rest.

maxItems — cheapest first

JS

```
const maxItems = (price, budget) => {  
  const p = [...price].sort((a, b) => a - b); // cheapest first  
  let cnt = 0;  
  for (const x of p) {  
    if (budget < x) break;  
    budget -= x;  
    cnt++;  
  }  
  return cnt;  
};
```

Tested with prices $\{40, 10, 25, 5, 90\}$ and $B = 60$: sorted is 5, 10, 25, 40, 90. Buy 5 (left 55), 10 (left 45), 25 (left 20), stop at 40. Output 3. `maxItems([40, 10], 0)` prints 0; `maxItems([], 100)` prints 0.

Answer: 3 items

TRAP

JavaScript numbers are doubles — exact only up to $\text{Number.MAX_SAFE_INTEGER} = 2^{53} - 1 \approx 9.007 \times 10^{15}$. Here you are safe: 10^5 items at 10^9 each is 10^{14} , comfortably inside that. Run `Number.isSafeInteger(total)` if you are unsure. The moment a running total can pass 9×10^{15} — a product of two values near 10^9 , for example — the answer silently loses its low digits and you must switch to `BigInt`: `items.reduce((s, x) => s + BigInt(x), 0n)`.

Example 4 [Wipro pattern]

Given a string of digits, rearrange them to make the largest possible number.

Constraints: length up to 10^5 . **Target:** $O(n \log n)$, or $O(n)$ with counting sort. **Edge cases:** all digits equal; a single digit.

SOLUTION

Largest number = biggest digit first. Sort descending.

biggestNumber

JS

```
const biggestNumber = (digits) => [...digits].sort().reverse().join('');
```

Tested: `biggestNumber("40921")` prints 94210; `biggestNumber("7")` prints 7; `biggestNumber("222")` prints 222.

This is the one place a bare `.sort()` is **safe**, and only because every element is a single character: for one-digit strings the lexicographic order and the numeric order agree. The moment a piece has two digits that stops being true — see Example 25.

Answer: 94210

Careful — this works only because all pieces are **single digits**. Joining multi-digit pieces needs a different comparator. That is Tier 2, Example 25.

Example 5 [Cognizant pattern]

Sort a list of intervals by finishing time, breaking ties by starting time.

Constraints: $n \leq 10^5$. **Target:** $O(n \log n)$. **Edge cases:** two intervals with the same end; an empty list.

SOLUTION

This is the exact comparator Template B needs, so write it once and keep it.

byEnd — the comparator you will reuse all chapter

JS

```
const byEnd = (v) => [...v].sort((a, b) => a[1] - b[1] || a[0] - b[0]);
```

Tested with (1, 9), (2, 4), (6, 7), (3, 4): output is (2, 4) (3, 4) (6, 7) (1, 9).

Answer: (2,4) (3,4) (6,7) (1,9)

TRAP

A JS comparator must return a **number**: negative if *a* comes first, positive if *b* does, 0 for a tie. Writing `(a, b) => a[1] > b[1]` returns a boolean, which coerces to 1 or 0 — the engine never sees a negative number, so it concludes nothing is ever out of order. Run it on (1, 9), (2, 4), (6, 7), (3, 4) and it prints the input back unchanged: (1, 9) (2, 4) (6, 7) (3, 4). No crash, no warning, just a wrong answer.

Example 6 [Accenture pattern]

An array of integers and a number k . You must flip the sign of **exactly** k numbers (you may flip the same position more than once). Make the total as large as possible.

Constraints: $n \leq 10^5$, $k \leq 10^4$, values in $[-100, 100]$. **Target:** $O(n \log n)$. **Edge cases:** no negatives at all; k larger than the count of negatives; a single element.

SOLUTION

Two greedy moves, in this order:

1. Flipping a negative always increases the total. Do that first, smallest (most negative) first, while flips remain.
2. If flips are still left, they must be spent. Flipping a number twice is a no-op, so an **even** leftover costs nothing. An **odd** leftover forces one real flip — waste it on the smallest remaining value.

maxSumAfterFlips

JS

```
const maxSumAfterFlips = (a, k) => {
  const v = [...a].sort((x, y) => x - y);
  for (let i = 0; i < v.length && k > 0 && v[i] < 0; i++) {
    v[i] = -v[i]; // flipping a negative always helps
    k--;
  }
  let sum = 0, small = Infinity;
  for (const x of v) { sum += x; small = Math.min(small, x); }
  return k % 2 === 1 ? sum - 2 * small : sum; // one flip left: waste it on the
  smallest
};
```

Input	k	After flipping negatives	Output
-4, -2, 3	2	4, 2, 3, k left 0	9
-4, -2, 3	3	4, 2, 3, k left 1, smallest is 2	$9 - 4 = 5$
5	1	nothing to flip, k left 1, smallest is 5	$5 - 10 = -5$

All three lines were run and matched.

Answer: 9, 5 and -5 for the three cases

TRAP

After the flipping loop you must recompute the smallest value. The smallest value of the **new** array may not be $a[0]$ any more — flipping -9 into 9 moves it to the far end. The code above scans for `small` after the flips, which is why it is correct.

Tier 1 — the standard interval toolkit

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS Digital pattern]

You are given n intervals. Some overlap. Merge every group that overlaps (or touches) into a single interval and return the merged list, sorted.

Constraints: $n \leq 10^5$, endpoints up to 10^9 . **Target:** $O(n \log n)$ time, $O(n)$ extra space. **Edge cases:** an empty list; one interval; several copies of the same interval; an interval fully inside another.

SOLUTION

Approach 1 — Fuse any two that touch, then start over $O(n^3)$ worst case, too slow

The first idea most students have: scan every pair, and whenever two touch, replace them with their union and restart the scan.

mergeBrute — correct but slow

JS

```
const mergeBrute = (input) => {
  const v = input.map(iv => [...iv]); // per-row copy, see the
  trap below
  let changed = true;
  while (changed) {
    changed = false;
    for (let i = 0; i < v.length && !changed; i++)
      for (let j = i + 1; j < v.length && !changed; j++)
        if (Math.max(v[i][0], v[j][0]) <= Math.min(v[i][1], v[j][1])) {
          v[i] = [Math.min(v[i][0], v[j][0]), Math.max(v[i][1], v[j][1])];
          v.splice(j, 1);
          changed = true;
        }
  }
  return v.sort((a, b) => a[0] - b[0]);
};
```

Time $O(n^3)$ **Space** $O(1)$ extra

Each fuse costs a full $O(n^2)$ rescan, and there can be n fuses. It is a fine reference implementation to test against, never a submission.

Approach 2 — Sort by start, then sweep once $O(n \log n)$, optimal

Here is the unlock. **After sorting by start time, any interval can only ever merge with the block that is currently open.** It can never reach backwards, because every earlier interval started earlier and has already been folded in.

So keep one open block. For each new interval: if it begins at or before the open block's end, stretch the block's end; otherwise the open block is finished, push it and open a new one.

mergeSorted — Template A

JS

```
const mergeSorted = (input) => {
  if (input.length === 0) return [];
  const v = input.map(iv => [...iv]); // copy every row, not just
  the outer array
  v.sort((a, b) => a[0] - b[0] || a[1] - b[1]);
  const out = [v[0]];
  for (let i = 1; i < v.length; i++) {
    const last = out[out.length - 1];
    if (v[i][0] <= last[1]) // touches the last block
      last[1] = Math.max(last[1], v[i][1]);
    else
      out.push(v[i]); // clean gap, start new
  }
  return out;
};
```

Time $O(n \log n)$ **Space** $O(n)$

The sort dominates. The sweep itself is $O(n)$.

Trace

Input (7, 9), (1, 4), (2, 5), (9, 11), (15, 18). After sorting by start: (1, 4), (2, 5), (7, 9), (9, 11), (15, 18).

i	Interval	Open block	What happens
0	(1,4)	(1,4)	first block
1	(2,5)	(1,5)	$2 \leq 4$ so stretch the end to $\max(4, 5) = 5$
2	(7,9)	(7,9)	$7 > 5$, gap. Push (1,5), open (7,9)
3	(9,11)	(7,11)	$9 \leq 9$, they touch. Stretch to 11
4	(15,18)	(15,18)	$15 > 11$, gap. Push (7,11), open (15,18)

Both versions were run on this input. Both printed `[[1, 5], [7, 11], [15, 18]]`. Extra runs: `mergeSorted([])` prints `[]`, `mergeSorted([[3, 3]])` prints `[[3, 3]]`, and three copies of (1, 2) collapse to `[[1, 2]]`. A fourth run confirmed the **caller's** array comes back in its original order.

Answer: `[1, 5] [7, 11] [15, 18]`

Python version — same idea, 8 lines

PYTHON

```
def merge_intervals(v):
    if not v:
        return []
    v.sort()
    out = [list(v[0])]
    for s, e in v[1:]:
        if s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Run: `merge_intervals([(7,9),(1,4),(2,5),(9,11),(15,18)])` prints `[[1, 5], [7, 11], [15, 18]]`.

TRAP

`last[1] = v[i][1]`; is wrong. Use `Math.max`. Input (1, 10), (2, 3) would otherwise shrink the block from [1, 10] to [1, 3] and silently lose half the range.

TRAP

`[...v]` is a **SHALLOW** copy. It makes a new outer array whose slots still point at the **same** inner `[s, e]` arrays. Since this solution mutates `last[1]` in place, a shallow copy would rewrite the caller's data. Copy every row: `const v = input.map(iv => [...iv])`; The same applies to any 2D grid — `grid.map(r => [...r])`, never `[...grid]`.

Example 8 [Infosys pattern]

A single room. You have n meeting requests, each $[s, e]$ — the room is free again at time e . Book the **largest number** of meetings.

Constraints: $n \leq 10^5$. **Target:** $O(n \log n)$. **Edge cases:** an empty list; a zero-length meeting (4, 4); every meeting identical.

SOLUTION

Approach 1 — Try every subset ($O(2^n \cdot n \log n)$, dies past $n = 20$)

```
const maxMeetBrute = (v) => {
  const n = v.length;
  let best = 0;
  for (let mask = 0; mask < (1 << n); mask++) {
    const pick = v.filter((_, i) => mask >> i & 1).sort((a, b) => a[0] - b[0]);
    let ok = true;
    for (let i = 1; i < pick.length; i++)
      if (pick[i][0] < pick[i - 1][1]) ok = false;
    if (ok) best = Math.max(best, pick.length);
  }
  return best;
};
```

Time $O(2^n \cdot n \log n)$ **Space** $O(n)$

Useful only to check the greedy on random small inputs — which is exactly what I did.

Approach 2 — Sort by earliest finish, take whatever fits $(O(n \log n)$, optimal)

Why **finish** and not **start** or **shortest**? Because finishing early is the only thing that helps the future. A meeting that ends at 4 leaves the room free from 4 onward; a meeting that ends at 9 blocks everything until 9. Nothing else about a meeting matters once it is chosen.

```
const maxMeetGreedy = (v) => {
  const a = [...v].sort((x, y) => x[1] - y[1]); // earliest FINISH first
  let cnt = 0, freeAt = -Infinity;
  for (const [s, e] of a)
    if (s >= freeAt) { cnt++; freeAt = e; }
  return cnt;
};
```

Time $O(n \log n)$ **Space** $O(1)$ extra

One sort, one pass.

The exchange argument, written out in full

Let the greedy picks in order be $g_1, g_2, \dots, g_k$ and let some optimal solution, sorted by finish time, be $o_1, o_2, \dots, o_m$ with $m \geq k$. We prove by induction that $\text{finish}(g_i) \leq \text{finish}(o_i)$ for every i .

- $i = 1$: greedy picked the globally earliest finishing interval, so $\text{finish}(g_1) \leq \text{finish}(o_1)$.
- Suppose $\text{finish}(g_{i-1}) \leq \text{finish}(o_{i-1})$. The optimal's next interval o_i starts at or after $\text{finish}(o_{i-1})$, which is at or after $\text{finish}(g_{i-1})$. So o_i was still available to greedy at step i . Greedy takes the earliest finishing available interval, so $\text{finish}(g_i) \leq \text{finish}(o_i)$.

Now suppose $m > k$. Then o_{k+1} exists and starts at or after $\text{finish}(o_k)$, which is at or after $\text{finish}(g_k)$. So o_{k+1} was still available when greedy stopped — but greedy only stops when nothing fits. Contradiction. Therefore $m = k$ and greedy is optimal.

Trace

Meetings (1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9), (6, 10), (8, 11). Sorted by end:
 (1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9), (6, 10), (8, 11).

Meeting	Start $\geq$ freeAt?	Take?	freeAt
---------	----------------------	-------	--------

(1,4)	$1 \geq -\infty$ yes	yes	4
(3,5)	$3 \geq 4$ no	skip	4
(0,6)	$0 \geq 4$ no	skip	4
(5,7)	$5 \geq 4$ yes	yes	7
(3,9)	$3 \geq 7$ no	skip	7
(5,9)	$5 \geq 7$ no	skip	7
(6,10)	$6 \geq 7$ no	skip	7
(8,11)	$8 \geq 7$ yes	yes	11

Both versions were run on this input: brute force printed 3, greedy printed 3. `maxMeetGreedy([])` prints 0; `maxMeetGreedy([[4,4]])` prints 1.

Answer: 3 meetings: (1, 4), (5, 7), (8, 11)

Python version

PYTHON

```
def max_meetings(v):
    v.sort(key=lambda p: p[1])
    cnt, free_at = 0, float("-inf")
    for s, e in v:
        if s >= free_at:
            cnt += 1
            free_at = e
    return cnt
```

Run: on the eight meetings above it prints 3; on [] it prints 0.

TRAP

“Pick the shortest meeting first” **feels** right and is wrong. Meetings (1, 10), (9, 11), (10, 20): shortest-first takes (9, 11) and then nothing else fits, giving 1. Earliest-finish takes (1, 10) and (10, 20), giving 2.

Example 9 [Wipro pattern]

A railway station. Train i arrives at a_i and departs at d_i . A train occupies a platform for $[a_i, d_i)$. What is the **minimum number of platforms** so no train waits?

Constraints: $n \leq 10^5$, times given as 24-hour numbers like 900 and 1145. **Target:** $O(n \log n)$. **Edge cases:** one train departs exactly when the next arrives; all trains identical.

SOLUTION

Approach 1 — For each train, count how many are live at its arrival ($O(n^2)$)

roomsBrute

JS

```
const roomsBrute = (v) => {
    let best = 0;
    for (const [s] of v) {
        let live = 0;
        for (const [a, d] of v) if (a <= s && s < d) live++;
        best = Math.max(best, live);
    }
}
```

```
    return best;
};
```

Time $O(n^2)$ **Space** $O(1)$

Correct, because the busiest moment is always some train's arrival time. $n = 10^5$ means 10^{10} operations. Too slow.

Approach 2 — Sweep line: split into arrival events and departure events $O(n \log n)$, optimal

The unlock: **a platform count only changes at an arrival (+1) or a departure (−1), and which train an event belongs to does not matter**. So throw away the pairing. Sort all arrivals into one array and all departures into another. Walk both with two pointers and watch the running count.

roomsSweep — two sorted event lists

JS

```
const roomsSweep = (v) => {
  const n = v.length;
  const st = v.map([s]) => s).sort((a, b) => a - b);
  const en = v.map([, e]) => e).sort((a, b) => a - b);
  let i = 0, j = 0, live = 0, best = 0;
  while (i < n) {
    if (st[i] < en[j]) { live++; i++; best = Math.max(best, live); }
    else { live--; j++; }
  }
  return best;
};
```

Time $O(n \log n)$ **Space** $O(n)$

Two sorts, one linear merge walk.

Trace

Trains (900, 1010), (940, 1200), (950, 1120), (1100, 1130), (1500, 1900), (1800, 2000).

Arrivals: 900 940 950 1100 1500 1800. Departures: 1010 1120 1130 1200 1900 2000.

st[i]	en[j]	st < en?	live	best
900	1010	yes	1	1
940	1010	yes	2	2
950	1010	yes	3	3
1100	1010	no (a train left)	2	3
1100	1120	yes	3	3
1500	1120	no	2	3
1500	1130	no	1	3
1500	1200	no	0	3
1500	1900	yes	1	3
1800	1900	yes	2	3

Both versions were run: both printed 3. For (1, 5), (5, 9), (9, 12) — a perfect relay — both printed 1, which confirms the strict < handles the touching case correctly.

Answer: 3 platforms

TRAP

Use `st[i] < en[j]`, not `<=`. With `<=`, a train departing at 1010 and another arriving at 1010 would be counted as needing two platforms. Flip this to `<=` only if the problem says a platform needs cleaning time.

Example 10 [Accenture pattern]

You already have a sorted list of **non-overlapping** intervals. Insert one new interval and re-merge, keeping the list sorted and non-overlapping.

Constraints: $n \leq 10^5$. **Target:** $O(n)$ — no re-sorting allowed. **Edge cases:** the list is empty; the new interval lands after everything; the new interval swallows the whole list.

SOLUTION

Approach 1 — Throw it on the pile and re-merge everything ($O(n \log n)$ — correct, but it re-sorts data that was already sorted)

insertBrute

JS

```
const insertBrute = (v, nw) => {
  const a = [...v.map(iv => [...iv]), [...nw]];
  a.sort((x, y) => x[0] - y[0] || x[1] - y[1]);
  const out = [a[0]];
  for (let i = 1; i < a.length; i++) {
    const last = out[out.length - 1];
    if (a[i][0] <= last[1]) last[1] = Math.max(last[1], a[i][1]);
    else out.push(a[i]);
  }
  return out;
};
```

Time $O(n \log n)$ **Space** $O(n)$

It works — the run printed [1,5][6,9], [2,5] and [0,20] on the three test lists, matching the fast version. But sorting already-sorted data is wasted work.

Approach 2 — Three zones, one linear pass ($O(n)$, optimal)

Because the list is already sorted and clean, you can split it into three zones in one pass: intervals entirely to the **left** of the new one, intervals that **touch** it, and intervals entirely to the **right**.

insertInterval — three zones, one pass

JS

```
const insertInterval = (v, nw) => {
  const out = [];
  let [lo, hi] = nw, i = 0;
  while (i < v.length && v[i][1] < lo) out.push(v[i++]); // fully left
  while (i < v.length && v[i][0] <= hi) { // overlapping
    lo = Math.min(lo, v[i][0]);
    hi = Math.max(hi, v[i][1]);
    i++;
  }
  out.push([lo, hi]);
  while (i < v.length) out.push(v[i++]); // fully right
  return out;
};
```

Time $O(n)$ **Space** $O(n)$

Every interval is looked at exactly once by exactly one of the three loops.

Runs, all checked:

List	Insert	Output
[1,3] [6,9]	[2,5]	[1,5] [6,9]
(empty)	[2,5]	[2,5]
[1,3] [6,9]	[10,12]	[1,3] [6,9] [10,12]
[1,3] [6,9]	[0,20]	[0,20]

Answer: [1,5] [6,9] for the first case

Example 11 [TCS NQT pattern]

Fractional knapsack. Items have weight w_i and value v_i . The bag holds W total weight. You **may cut an item** and take part of it, getting that fraction of its value. Maximise the value carried.

Constraints: $n \leq 10^5$, $W \leq 10^9$. **Target:** $O(n \log n)$. **Edge cases:** $W = 0$; total weight of all items less than W .

SOLUTION

Every unit of bag space should carry as much value as possible. So rank items by **value per unit weight** and fill from the top. The last item may be cut.

Exchange argument. Take any optimal packing. If a lower-ratio item is in the bag while some of a higher-ratio item is left outside, remove one kilogram of the lower one and put in one kilogram of the higher one. Weight is unchanged and value went up or stayed equal. Repeat until the packing is the greedy one.

fracKnap — sort by value/weight

JS

```
const fracKnap = (item, cap) => { // item = [weight, value]
  const a = [...item].sort((x, y) => y[1] / y[0] - x[1] / x[0]); // ratio,
  descending
  let got = 0;
  for (const [w, val] of a) {
    if (cap === 0) break;
    const take = Math.min(w, cap);
    got += val * take / w;
    cap -= take;
  }
  return got;
};
```

Time $O(n \log n)$ **Space** $O(1)$ extra

The sort dominates.

Trace

Items (10, 60), (20, 100), (30, 120), bag = 50.

Item (w, v)	Ratio	Take	Value added	Cap left
(10, 60)	6.0	all 10	60	40
(20, 100)	5.0	all 20	100	20
(30, 120)	4.0	20 of 30	$120 \times 20/30 = 80$	0

Run output: 240.00. `fracKnap([[10,60]], 0)` prints 0.00, and `fracKnap([], 50)` prints 0.00.

Answer: 240

TRAP

This greedy is **only** valid because you may cut items. If items are all-or-nothing (0/1 knapsack) the same greedy is wrong. Items (10, 60) and (20, 100) with a bag of 20: the ratios are 6.0 and 5.0, so greedy grabs the 10-weight item for 60 and the other no longer fits — total 60. Taking the 20-weight item alone gives 100. This was checked against an exhaustive search over all subsets, which printed 100 while the ratio greedy printed 60. 0/1 knapsack needs DP.

Example 12 [Capgemini pattern]

An array `a` where `a[i]` is the maximum number of steps you may jump forward from index i . Starting at index 0, can you reach the last index?

Constraints: $n \leq 10^5$, $0 \leq a[i] \leq 10^5$. **Target:** $O(n)$. **Edge cases:** a single element (already at the end); a zero that blocks the path.

SOLUTION

Approach 1 — Backward DP — mark each index reachable or not ($O(n \cdot \text{maxJump})$, too slow)

canJumpDP

JS

```
const canJumpDP = (a) => {
  const n = a.length;
  if (n === 0) return false;
  const ok = new Array(n).fill(false);
  ok[n - 1] = true;
  for (let i = n - 2; i >= 0; i--)
    for (let s = 1; s <= a[i] && i + s < n; s++)
      if (ok[i + s]) { ok[i] = true; break; }
  return ok[0];
};
```

Time $O(n^2)$ worst case **Space** $O(n)$

With every `a[i] = n` the inner loop is length `n`.

Approach 2 — Carry one number: the farthest index reached so far ($O(n)$, $O(1)$ space, optimal)

The unlock: you never need to know **which** index you jumped from. You only need the farthest index any jump so far could land on. Walk left to right. If you ever stand at an index beyond that farthest reach, there is a hole you cannot cross.

canJumpGreedy

JS

```
const canJumpGreedy = (a) => {
  let reach = 0;
  for (let i = 0; i < a.length; i++) {
    if (i > reach) return false;           // a hole we can never cross
    reach = Math.max(reach, i + a[i]);
  }
  return true;
};
```

Time $O(n)$ **Space** $O(1)$

One pass, one integer of state.

Trace on 2, 3, 1, 1, 4

i	a[i]	i > reach?	reach after
0	2	no	$\max(0, 0 + 2) = 2$
1	3	no	$\max(2, 1 + 3) = 4$
2	1	no	$\max(4, 3) = 4$
3	1	no	4
4	4	no	8

Reached the end. Output `true`.

Trace on 3, 2, 1, 0, 4

reach goes 3, 3, 3, 3; at $i = 4$ we have $4 > 3$, so `false`. Output `false`.

Both versions were run on both inputs and on the single-element input `[0]`; all three pairs agreed (`true true`, `false false`, `true true`).

Answer: `true` for 2, 3, 1, 1, 4; `false` for 3, 2, 1, 0, 4

Example 13 [Cognizant pattern]

Given n intervals, delete the **fewest** of them so that no two of the survivors overlap. Return how many you deleted.

Constraints: $n \leq 10^5$. **Target:** $O(n \log n)$. **Edge cases:** empty list; no overlaps at all.

SOLUTION

This is Example 8 wearing a disguise. “Delete the fewest” = “keep the most”, and “keep the most non-overlapping” is exactly Template B.

$$\text{deleted} = n - \text{kept}$$

minRemovals — keep the most, subtract

JS

```
const minRemovals = (v) => {
  if (v.length === 0) return 0;
  const a = [...v].sort((x, y) => x[1] - y[1]);
  let kept = 0, freeAt = -Infinity;
  for (const [s, e] of a)
    if (s >= freeAt) { kept++; freeAt = e; }
  return v.length - kept;
};
```

Time $O(n \log n)$ **Space** $O(1)$ extra

Run on (1, 4), (2, 5), (3, 6), (7, 8): sorted by end it is the same order. Keep (1, 4), skip (2, 5), skip (3, 6), keep (7, 8). Kept 2 out of 4, so the output is 2. `minRemovals([])` prints 0.

Answer: 2 deletions

SHORTCUT

The **same** function also answers “how many arrows to pop all the balloons” and “how many machines to run all the jobs on one shift” — with one twist: for arrows, intervals that merely **touch** can share an arrow, so the test becomes $s > \text{freeAt}$ and you count the kept groups instead of subtracting.

Example 14 [Infosys pattern]

m children, child i needs a biscuit of size at least $\text{need}[i]$. You have k biscuits with sizes $\text{size}[j]$. Each child gets at most one biscuit. Make the most children happy.

Constraints: $m, k \leq 10^5$. **Target:** $O(m \log m + k \log k)$. **Edge cases:** no children; every biscuit too small.

SOLUTION

Approach 1 — For each child, search the whole biscuit list ($O(m \cdot k)$, too slow)

```
childrenBrute
JS

const childrenBrute = (need, size) => {
  const used = new Array(size.length).fill(false);
  let happy = 0;
  for (const nd of need) {
    let pick = -1;
    for (let j = 0; j < size.length; j++)
      if (!used[j] && size[j] >= nd && (pick === -1 || size[j] < size[pick])) pick = j;
    if (pick !== -1) { used[pick] = true; happy++; }
  }
  return happy;
};
```

Time $O(mk)$ **Space** $O(k)$

Run on the same three tests as the fast version, it printed 2, 0 and 0 — identical answers, 10^{10} operations at the stated limits.

Approach 2 — Sort both, then two pointers ($O(m \log m + k \log k)$, optimal)

Sort both lists ascending and walk them with two pointers. Always feed the **least demanding unfed child** with the **smallest biscuit that is big enough**.

Why it is safe: if a bigger biscuit satisfies the fussiest child, a smaller one that also fits satisfies them too — so spending the small one first never loses an option.

```
contentChildren — two sorted lists, two pointers
JS

const contentChildren = (need, size) => {
  const nd = [...need].sort((a, b) => a - b);
  const sz = [...size].sort((a, b) => a - b);
  let i = 0, j = 0, happy = 0;
  while (i < nd.length && j < sz.length) {
    if (sz[j] >= nd[i]) { happy++; i++; }
    j++;
  }
  return happy;
};
```

Time $O(m \log m + k \log k)$ **Space** $O(1)$ extra

Runs: `contentChildren([1,2,7], [1,1,3,5])` prints 2 (biscuit 1 feeds child 1, biscuit 3 feeds child 2, biscuit 5 is too small for child 7). `contentChildren([9], [1])` prints 0. `contentChildren([], [1,2])` prints 0. `contentChildren([1,1,1], [1,1,1])` prints 3.

Answer: 2 happy children

Notice that `j` advances on **every** loop turn but `i` advances only on a match. That is the whole algorithm: a biscuit that cannot feed the current least-demanding child cannot feed anybody, so throw it away.

Example 15 [TCS NQT pattern]

Greedy coin change is not always right. Show that with coins $\{25, 10, 1\}$ and amount 30 the greedy answer is worse than the true best, and give the algorithm that is always right.

Constraints: amount up to 10^4 , up to 20 coin types. **Target:** $O(\text{amount} \times \text{coins})$. **Edge cases:** amount 0; a set with no way to pay.

SOLUTION

Greedy: $25 \rightarrow 5 \text{ left} \rightarrow \text{five 1s}$. That is $1 + 5 = 6$ coins. True best: $10 + 10 + 10 = 3$ coins. Greedy is off by a factor of two.

The safe algorithm is a small DP. `dp[a]` is the fewest coins that make exactly `a`.

coinsDP — always correct

JS

```
const coinsDP = (amount, coins) => {
  const dp = new Array(amount + 1).fill(Infinity);
  dp[0] = 0;
  for (let a = 1; a <= amount; a++)
    for (const c of coins)
      if (c <= a && dp[a - c] + 1 < dp[a]) dp[a] = dp[a - c] + 1;
  return dp[amount] === Infinity ? -1 : dp[amount];
};
```

Time $O(\text{amount} \times \text{coins})$ **Space** $O(\text{amount})$

Verified side by side: `minCoins(30, [25,10,1])` from Example 1 prints 6 while `coinsDP(30, [25,10,1])` prints 3. On a set where greedy happens to be right, `minCoins(7, [5,2])` and `coinsDP(7, [5,2])` both print 2. `coinsDP(0, [2])` prints 0 and `coinsDP(3, [2,4])` prints -1.

Answer: Greedy 6 coins, true best 3 coins. Use DP.

TRAP

In an interview, never say “greedy” for coin change without adding “if the coin system is canonical”. Indian and most world currencies are canonical, so greedy happens to work — but the interviewer is testing whether you know **why**, and the $\{25, 10, 1\}$ example is the fastest way to show you do.

Tier 2 — greedy with a business wrapper

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 16 [Grab · pattern]

A delivery hub runs n one-hour jobs. Job i earns `profit[i]` but must be **finished by the end of hour** `deadline[i]`. Only one job can run in any hour, and hours are 1, 2, 3, Schedule jobs to earn the most money. Report how many jobs you ran and the total earned.

Constraints: $n \leq 10^5$, deadlines up to n , profit up to 10^5 . **Target:** $O(n \log n)$. **Edge cases:** one job; every job with deadline 1 (only one can run).

SOLUTION

Approach 1 — Take the richest job first, park it as late as it may legally run ($O(n \cdot \text{maxDeadline})$, passes most tests)

Why “as late as possible”? Because an early hour is more useful than a late hour: an early hour can host a job with a tight deadline, a late hour cannot. So spend the **latest legal hour** and keep the early hours free for jobs you have not seen yet.

jobSeqSlots — greedy by profit, linear slot search

JS

```
// a job is { id, deadline, profit }
const jobSeqSlots = (job) => {
  const a = [...job].sort((x, y) => y.profit - x.profit);
  const last = a.reduce((m, j) => Math.max(m, j.deadline), 0);
  const slot = new Array(last + 1).fill(-1); // slot[t] = job sitting in
  hour t
  let cnt = 0, money = 0;
  for (const j of a)
    for (let t = Math.min(j.deadline, last); t >= 1; t--)
      if (slot[t] === -1) { slot[t] = j.id; cnt++; money += j.profit; break; }
  return [cnt, money];
};
```

Time $O(n \cdot D)$ **Space** $O(D)$

D is the largest deadline. If deadlines are up to n this is $O(n^2)$ in the worst case — all jobs with deadline n .

Approach 2 — Same order, but a DSU jumps straight to the free hour ($O(n \log n)$ effectively, optimal)

The inner loop wastes time walking over hours that are already full. A disjoint-set union fixes that: `find(t)` returns the **latest free hour at or before t** . When you fill hour t , point t at $t - 1$, so the next `find` skips it instantly.

Use the tested DSU from the **JS Toolkit** appendix rather than writing one here. Note we call `d.find` and then set `d.p[t]` by hand instead of calling `d.union` — union-by-rank would attach the trees whichever way is shallower, and this trick needs the parent to point **specifically** at $t - 1$.

jobSeqDSU — the same greedy, with a skip pointer

JS

```
const { DSU } = require('./toolkit.js'); // JS Toolkit appendix

const jobSeqDSU = (job) => {
  const a = [...job].sort((x, y) => y.profit - x.profit);
  const last = a.reduce((m, j) => Math.max(m, j.deadline), 0);
  const d = new DSU(last + 1); // d.find(t) = latest free hour at
  or before t
  let cnt = 0, money = 0;
  for (const j of a) {
    const t = d.find(Math.min(j.deadline, last));
    if (t > 0) {
      d.p[t] = t - 1; // skip pointer, NOT union-by-rank
      cnt++; money += j.profit;
    }
  }
}
```

```

    }
  }
  return [cnt, money];
};

```

Time $O(n \log n)$ **Space** $O(D)$

Sorting dominates; every find is near-constant after path compression.

Trace

Jobs (id, deadline, profit): (1, 4, 20), (2, 1, 10), (3, 1, 40), (4, 1, 30), (5, 2, 60). Sorted by profit: 60, 40, 30, 20, 10.

Profit	Deadline	Latest free hour	Placed?	Money
60	2	2	yes, hour 2	60
40	1	1	yes, hour 1	100
30	1	none (hour 1 full)	no	100
20	4	4	yes, hour 4	120
10	1	none	no	120

Both versions were run on this input. Both printed 3 120. A one-job input (1, 2, 100) printed 1 100.

Answer: 3 jobs, 120 profit

TRAP

10^5 jobs at 10^5 profit is 10^{10} . In C++ or Java that overflows a 32-bit int and you would reach for a 64-bit integer; in JavaScript a plain number holds it exactly, because 10^{10} is far below $2^{53} - 1$. The run confirmed 100000 100000000000 with Number.isSafeInteger returning true. Only start worrying past about 9×10^{15} — then switch the accumulator to BigInt.

Example 17 [Shopee · pattern]

Now do more than count rooms. Given n meetings, actually **assign a room number** to each meeting, using as few rooms as possible. Return the room number for each meeting **in the original input order**.

Constraints: $n \leq 10^5$. **Target:** $O(n \log n)$. **Edge cases:** meetings that touch end-to-start must share a room; all meetings at once.

SOLUTION

JavaScript has no built-in priority queue. Use the MinHeap from the **JS Toolkit** appendix — it takes an optional comparator, so one class covers min-heaps, max-heaps and heaps of tuples.

Counting used a sweep. Assigning needs to remember **which** room freed up. Two heaps:

- busy: min-heap of (free-at-time, room number) — rooms in use.
- spare: min-heap of room numbers that are free right now.

Process meetings in start order. Move every room whose meeting has ended into spare. Reuse the lowest-numbered spare room if any, else open a brand new room.

assignRooms — two heaps

JS

```

const { MinHeap } = require('./toolkit.js'); // JS has no priority queue

const assignRooms = (v) => {
  const room = new Array(v.length).fill(-1);

```

```

const order = [...v.keys()].sort((a, b) => v[a][0] - v[b][0] || v[a][1] - v[b][1]);
const busy = new MinHeap((a, b) => a[0] - b[0]); // [freeAt, room]
const spare = new MinHeap(); // free room numbers
let made = 0;
for (const i of order) {
  while (busy.size && busy.peek()[0] <= v[i][0]) spare.push(busy.pop()[1]);
  const r = spare.size ? spare.pop() : made++;
  room[i] = r;
  busy.push([v[i][1], r]);
}
return room;
};

```

Time $O(n \log n)$ **Space** $O(n)$

Each meeting enters and leaves each heap at most once.

Trace

Meetings (0, 30), (5, 10), (15, 20), (35, 40) — already in start order.

Meeting	What frees up first	Room given
(0,30)	nothing busy, no spares	new room 0
(5,10)	room 0 busy until 30	new room 1
(15,20)	room 1 freed at 10, goes to spare	reuse room 1
(35,40)	rooms 0 and 1 both freed by 30	reuse room 0 (lowest spare)

Run output: 0 1 1 0. The maximum room number used is 1, so 2 rooms — which matches what the sweep-line count of Example 9 would say. `assignRooms([])` prints nothing, (1, 5), (5, 9), (9, 12) gives 0 0 0 (one room, reused twice) and three copies of (1, 9) give 0 1 2.

Answer: Rooms 0 1 1 0 — two rooms are enough

Example 18 [Agoda · pattern]

A ring road has n fuel pumps. Pump i gives `gas[i]` litres, and driving from pump i to pump $i + 1$ burns `cost[i]` litres. The tank starts empty and has no limit. Find a pump you can start from and complete the whole loop, or report -1 . The answer is unique if it exists.

Constraints: $n \leq 10^5$, values up to 10^4 (a running total reaches 10^9 — well inside a JS number). **Target:** $O(n)$.

Edge cases: one pump; total gas less than total cost.

SOLUTION

Approach 1 — Start at every pump and drive the loop ($O(n^2)$)

startBrute

JS

```

const startBrute = (gas, cost) => {
  const n = gas.length;
  for (let s = 0; s < n; s++) {
    let tank = 0, ok = true;
    for (let k = 0; k < n; k++) {
      const i = (s + k) % n;
      tank += gas[i] - cost[i];
      if (tank < 0) { ok = false; break; }
    }
  }
};

```

```

    }
    if (ok) return s;
  }
  return -1;
};

```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — One pass, restart the moment the tank goes negative ($O(n)$, optimal)

Two facts unlock it.

1. **If the total of all $(\text{gas}[i] - \text{cost}[i])$ is negative, no start works.** You cannot create fuel. If it is zero or positive, some start works.
2. **If you start at s and die somewhere between s and j , then none of $s, s + 1, \dots, j$ works either.** Reason: starting at s you arrived at every pump in between with a tank that was zero or better. Starting **later** means you arrive with less, so you die no later. So skip straight past j and try $j + 1$.

startGreedy — single pass

JS

```

const startGreedy = (gas, cost) => {
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < gas.length; i++) {
    const d = gas[i] - cost[i];
    total += d; tank += d;
    if (tank < 0) { start = i + 1; tank = 0; }
  }
  return total < 0 ? -1 : start;
};

```

Time $O(n)$ **Space** $O(1)$

One pass. Two running sums, nothing else.

Trace

gas = 1 2 3 4 5, cost = 3 4 5 1 2, so d = -2 -2 -2 3 3.

i	d	tank after	tank < 0?	start
0	-2	-2	yes, reset	1
1	-2	-2	yes, reset	2
2	-2	-2	yes, reset	3
3	3	3	no	3
4	3	6	no	3

total = $-2 - 2 - 2 + 3 + 3 = 0 \geq 0$, so the answer is pump 3.

Both versions were run: both printed 3. With gas = 2 3 4, cost = 3 4 3 the total is -1 and both printed -1 .
With one pump gas = 5, cost = 4 both printed 0.

Answer: Start at pump index 3

Python version

PYTHON

```
def fuel_start(gas, cost):
    total = tank = 0
    start = 0
    for i, (g, c) in enumerate(zip(gas, cost)):
        total += g - c
        tank += g - c
        if tank < 0:
            start, tank = i + 1, 0
    return start if total >= 0 else -1
```

Run: prints 3 and -1 for the two cases above.

TRAP

Do not return start without the total check. On gas = 2 3 4, cost = 3 4 3 the loop would leave start = 1 even though no start works at all.

Example 19 [Sea · pattern]

A small lift carries at most **two** people and at most limit kilograms per trip. Given everybody's weight (each $\leq$ limit), find the fewest trips.

Constraints: $n \leq 10^5$. **Target:** $O(n \log n)$. **Edge cases:** one person; everybody so heavy that nobody can share.

SOLUTION

Sort ascending. Point i at the lightest and j at the heaviest. The heaviest person **must** travel now in some trip — so send them. If the lightest can ride along without breaking the limit, send them too; otherwise the heaviest rides alone.

Why pairing the lightest with the heaviest is safe: if the lightest person cannot ride with the heaviest, nobody can, so the heaviest is alone in every possible solution. If the lightest **can** ride with the heaviest, then swapping whoever the optimal put with the heaviest for the lightest keeps the trip legal and frees a heavier person for later.

minBoats — two pointers from both ends

JS

```
const minBoats = (w, limit) => {
    const a = [...w].sort((x, y) => x - y);
    let i = 0, j = a.length - 1, boats = 0;
    while (i <= j) {
        if (a[i] + a[j] <= limit) i++; // lightest rides with heaviest
        j--; boats++; // heaviest always leaves now
    }
    return boats;
};
```

Time $O(n \log n)$ **Space** $O(1)$ extra

Trace

Weights 3, 5, 3, 4 with limit = 5. Sorted: 3, 3, 4, 5.

i	j	w[i] + w[j]	Decision	Trips
0	3	3 + 5 = 8 > 5	5 rides alone	1
0	2	3 + 4 = 7 > 5	4 rides alone	2
0	1	3 + 3 = 6 > 5	one 3 rides alone	3

0	0	$3 + 3 = 6 > 5$	last 3 rides alone	4
---	---	-----------------	--------------------	---

Run output: 4. `minBoats([1,2], 3)` prints 1; `minBoats([9], 9)` prints 1; `minBoats([], 5)` prints 0.

Answer: 4 trips

TRAP

`while (i <= j)` — not `i < j`. When one person is left, `i == j` and that person still needs a trip. Using `<` silently loses them.

Example 20 [LINE MAN · pattern]

A lowercase string. Cut it into as many pieces as possible so that **every letter appears in only one piece**. Return the piece lengths in order.

Constraints: length up to 10^5 . **Target:** $O(n)$ time, $O(1)$ extra space (26 slots). **Edge cases:** one repeated letter; all letters distinct.

SOLUTION

Record the **last position** of every letter. Then walk the string carrying `reach`, the furthest index any letter seen so far must still reach. The moment `i == reach`, every letter inside the current piece has finished — so cut there.

partitionSizes — last-position + reach

JS

```
const partitionSizes = (s) => {
  const lastAt = new Map(); // Map, not a 26-slot array
  for (let i = 0; i < s.length; i++) lastAt.set(s[i], i);
  const out = [];
  let start = 0, reach = 0;
  for (let i = 0; i < s.length; i++) {
    reach = Math.max(reach, lastAt.get(s[i]));
    if (i === reach) { out.push(i - start + 1); start = i + 1; }
  }
  return out;
};
```

Time $O(n)$ **Space** $O(1)$

Two passes, a fixed 26-slot table.

Trace on abacdefegh

Last positions: a $\rightarrow$ 2, b $\rightarrow$ 1, c $\rightarrow$ 3, d $\rightarrow$ 4, e $\rightarrow$ 7, f $\rightarrow$ 6, g $\rightarrow$ 8, h $\rightarrow$ 9.

i	char	lastAt	reach	cut?
0	a	2	2	no
1	b	1	2	no
2	a	2	2	yes $\rightarrow$ piece aba, length 3
3	c	3	3	yes $\rightarrow$ piece c, length 1
4	d	4	4	yes $\rightarrow$ piece d, length 1
5	e	7	7	no
6	f	6	7	no

7	e	7	7	yes → piece efe, length 3
8	g	8	8	yes → piece g, length 1
9	h	9	9	yes → piece h, length 1

Run output: 3 1 1 3 1 1. For zzzz it prints 4.

Answer: 3 1 1 3 1 1 — pieces aba | c | d | efe | g | h

Example 21 [Razer · pattern]

You have n ropes. Joining two ropes of length a and b costs $a + b$ and produces one rope of length $a + b$. Join all of them into one rope at the least total cost.

Constraints: $n \leq 10^5$, each length up to 10^5 . The total cost reaches 10^{10} . **Target:** $O(n \log n)$. **Edge cases:** zero ropes; one rope.

SOLUTION

Approach 1 — Scan the list for the two shortest, every single time ($O(n^2)$)

joinRopesBrute

JS

```
const joinRopesBrute = (len) => {
  const v = [...len];
  let cost = 0;
  while (v.length > 1) {
    let i = 0;
    for (let t = 1; t < v.length; t++) if (v[t] < v[i]) i = t;
    const a = v.splice(i, 1)[0];
    let j = 0;
    for (let t = 1; t < v.length; t++) if (v[t] < v[j]) j = t;
    const b = v.splice(j, 1)[0];
    cost += a + b;
    v.push(a + b);
  }
  return cost;
};
```

Time $O(n^2)$ **Space** $O(n)$

$n-1$ joins, each with two linear scans and two erases. It printed 29, 0 and 0 on the tests below — the same answers, just slowly.

Approach 2 — Let a min-heap hand you the two shortest ($O(n \log n)$, optimal)

Every time a rope takes part in a join, its whole length is paid again. So a rope joined early gets charged many times. Therefore: **always join the two shortest ropes**. A min-heap gives them to you in $O(\log n)$ — and since JavaScript ships no heap, that is the toolkit MinHeap again.

joinRopes — min-heap greedy

JS

```
const { MinHeap } = require('./toolkit.js');

const joinRopes = (len) => {
  if (len.length <= 1) return 0;
  const pq = new MinHeap();
  for (const x of len) pq.push(x);
  let cost = 0;
  while (pq.size > 1) {
```

```

    const a = pq.pop(), b = pq.pop();
    cost += a + b;
    pq.push(a + b);
  }
  return cost;
};

```

Time $O(n \log n)$ **Space** $O(n)$
 n-1 joins, each two pops and one push.

Trace

Ropes 4, 3, 2, 6.

Heap	Two smallest	Cost added	Total
2 3 4 6	2 and 3	5	5
4 5 6	4 and 5	9	14
6 9	6 and 9	15	29
15	done	—	29

Run output: 29. `joinRopes([7])` prints 0, `joinRopes([])` prints 0.

Size check. With 10^5 ropes of length 20000 the answer is 33378560000. The run printed exactly that, and `Number.isSafeInteger` on it returned `true` — 3.3×10^{10} is nowhere near $2^{53} - 1$.

Answer: 29

TRAP

Even with small individual lengths, the **total** explodes because every join re-pays the whole combined length. In JavaScript that is fine up to $2^{53} - 1$; raise the constraints to 10^6 ropes of length 10^9 and the answer passes 10^{16} , at which point plain numbers start rounding and you need `BigInt` — including inside the heap, with new `MinHeap((a, b) => (a < b ? -1 : a > b ? 1 : 0))`, because `a - b` on `BigInts` returns a `BigInt` and the heap wants a plain number.

Example 22 [DBS · pattern]

Several employees each have a sorted list of busy intervals. Find every **free window common to everybody** that is at least `minLen` long.

Constraints: total intervals up to 10^5 . **Target:** $O(n \log n)$. **Edge cases:** one employee only; no common gap at all.

SOLUTION

The insight: a window is free for everybody exactly when it lies in **no** busy interval. So dump every interval from every person into one list, merge it (Template A), and read off the gaps between merged blocks.

`commonFree` — merge everything, report the holes

JS

```

const commonFree = (people, minLen) => {
  const all = people.flat().sort((a, b) => a[0] - b[0] || a[1] - b[1]);
  const gaps = [];
  if (all.length === 0) return gaps;
  let reach = all[0][1];
  for (let i = 1; i < all.length; i++) {
    if (all[i][0] > reach) {
      if (all[i][0] - reach >= minLen) gaps.push([reach, all[i][0]]);
    }
  }
};

```

```

    reach = all[i][1];
  } else reach = Math.max(reach, all[i][1]);
}
return gaps;
};

```

Time $O(n \log n)$ **Space** $O(n)$

Trace

Person A busy (1, 3), (6, 7); person B busy (2, 4); person C busy (2, 5), (9, 12), with `minLen = 1`.

Pooled and sorted: (1, 3), (2, 4), (2, 5), (6, 7), (9, 12).

Interval	reach	Gap found
(1,3)	3	—
(2,4)	4	$2 \leq 3$, overlaps, extend
(2,5)	5	$2 \leq 4$, overlaps, extend
(6,7)	7	$6 > 5 \rightarrow$ gap [5, 6]
(9,12)	12	$9 > 7 \rightarrow$ gap [7, 9]

Run output: [5,6][7,9].

Answer: [5,6] and [7,9]

Example 23 [GIC · pattern]

Given non-negative integers, join them end to end to make the **largest possible number**. Return it as a string.

Constraints: $n \leq 10^4$, values up to 10^9 . **Target:** $O(n \log n \cdot L)$ where L is the digit length. **Edge cases:** every number is 0 (answer is "0", not "000"); a single number.

SOLUTION

Approach 1 — Try every ordering ($O(n! \cdot L)$, only useful as a checker)

joinBiggestBrute

JS

```

const permutations = (a) => a.length <= 1 ? [a] :
  a.flatMap((x, i) =>
    permutations([...a.slice(0, i), ...a.slice(i + 1)]).map(p => [x, ...p]));

const joinBiggestBrute = (a) => {
  if (a.length === 0) return '0';
  let best = '';
  for (const p of permutations(a.map(String))) {
    const cur = p.join('');
    if (best === '' || cur.length > best.length ||
      (cur.length === best.length && cur > best)) best = cur;
  }
  return best[0] === '0' ? '0' : best;
};

```

Time $O(n! \cdot L)$ **Space** $O(nL)$

Run on the three test inputs it printed 9534330, 0 and 12 — exactly what the comparator version gives. That agreement is the evidence the comparator is right.

Approach 2 — One sort with the right comparator $O(n \log n \cdot L)$, optimal

Sorting by value fails: 3 vs 30 — is 3 bigger? As numbers yes, but $330 > 303$, so 3 must come first. Sorting by first digit fails too: 34 vs 3 — $343 > 334$, so 34 comes first even though the first digits tie.

The correct rule compares the two **possible joins**:

a comes before b when $(a + b) > (b + a)$ as strings

joinBiggest — the a+b vs b+a comparator

JS

```
const joinBiggest = (a) => {
  const s = a.map(String)
  .sort((x, y) => (x + y > y + x ? -1 : x + y < y + x ? 1 : 0));
  if (s.length === 0 || s[0] === '0') return '0';
  return s.join('');
};
```

Time $O(n \log n \cdot L)$ **Space** $O(nL)$

Every comparison builds two strings of length at most $2L$.

Why this comparator is a valid ordering

It is **transitive** — that is the part people doubt. Sketch: writing a followed by b as a number is $a \cdot 10^{|b|} + b$. The test $a + b > b + a$ becomes $a \cdot 10^{|b|} + b > b \cdot 10^{|a|} + a$, i.e. $a(10^{|b|} - 1) > b(10^{|a|} - 1)$, i.e.

$$\frac{a}{10^{|a|} - 1} > \frac{b}{10^{|b|} - 1}$$

That is a comparison of two plain numbers, and plain number comparison is transitive.

Run: `joinBiggest([3, 30, 34, 5, 9])` prints 9534330; `joinBiggest([0, 0])` prints 0; `joinBiggest([12])` prints 12; `joinBiggest([])` prints 0; `joinBiggest([10, 2])` prints 210.

Note the comparator shape. $(x, y) \Rightarrow x + y > y + x$ would be a boolean and would not sort at all; the three-way $? -1 : \dots ? 1 : 0$ is what a JS comparator has to return. The plain $>$ on strings is safe here because $x + y$ and $y + x$ always have the **same length**, so a character-by-character comparison is exactly a numeric one.

Answer: 9534330

TRAP

Miss the all-zero guard and you print 00. Checking only `s[0] === '0'` is enough: if the biggest piece after sorting is '0', every piece is '0'. Note `a.map(String)`, not `a.map(x => x.toString())` inside a callback that also receives an index — `map(Number)` and `map(String)` are safe, `map(parseInt)` famously is not.

Tier 3 — insight, then the follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 24 [Google · pattern]

A road runs from position 0 to position n . At every integer position i there is a sprinkler with radius $r[i]$; it wets the closed range $[i - r[i], i + r[i]]$. A radius of 0 means the sprinkler is broken. Use the **fewest** sprinklers to wet the whole road, or report -1 .

Constraints: $n \leq 10^4$. **Target:** $O(n)$ after an $O(n)$ preparation pass. **Edge cases:** $n = 0$ (already wet, answer 0); every sprinkler broken.

Follow-up the interviewer asks next: “the sprinklers are now at arbitrary real positions, not integers — what changes?”

SOLUTION

Approach 1 — DP over positions ($O(n^2)$, fine for $n = 10^4$ but not the answer they want)

$dp[x]$ = fewest sprinklers to wet everything from 0 up to x .

coverDP — the reference answer

JS

```
const coverDP = (n, r) => {
  const dp = new Array(n + 1).fill(Infinity);
  dp[0] = 0;
  for (let i = 0; i <= n; i++) {
    if (r[i] === 0) continue;
    const lo = Math.max(0, i - r[i]), hi = Math.min(n, i + r[i]);
    let best = Infinity;
    for (let j = lo; j <= hi; j++) best = Math.min(best, dp[j]);
    if (best === Infinity) continue;
    for (let j = lo; j <= hi; j++) dp[j] = Math.min(dp[j], best + 1);
  }
  return dp[n] === Infinity ? -1 : dp[n];
};
```

Time $O(n^2)$ **Space** $O(n)$

Approach 2 — Convert to intervals, then jump greedily ($O(n)$, optimal)

Two steps, and the first one is the whole trick.

Step 1 — forget the sprinklers, keep the reach. A sprinkler at i with radius r says: “if the wet part already reaches position $i - r$, I can push it to $i + r$.” So build $far[L]$ = the furthest right any single sprinkler can push you, given you have already covered up to L . Several sprinklers may share the same L ; keep the best.

Step 2 — jump. Stand at reach. Look at every start from 0 to reach that you have not looked at yet, and remember the best far among them. Jump there. One sprinkler used. Repeat. If the best jump does not move you, there is a dry gap and the answer is -1 .

coverGreedy — interval cover in one pass

JS

```
const coverGreedy = (n, r) => {
  const far = new Array(n + 1).fill(0);
  for (let i = 0; i <= n; i++) {
    if (r[i] === 0) continue;
    const lo = Math.max(0, i - r[i]);
    far[lo] = Math.max(far[lo], Math.min(n, i + r[i]));
  }
  let used = 0, reach = 0, nxt = 0, i = 0;
  while (reach < n) {
    while (i <= reach) { nxt = Math.max(nxt, far[i]); i++; }
    if (nxt <= reach) return -1; // stuck, a dry gap
    reach = nxt; used++;
  }
  return used;
};
```

Time $O(n)$ **Space** $O(n)$

The pointer i never moves backwards, so the inner while loop runs n times in total across the whole algorithm.

Trace

$n = 7$, radii 1, 2, 1, 0, 2, 1, 0, 1 at positions 0...7.

Reaches: position 0 $\rightarrow$ [0, 1], 1 $\rightarrow$ [0, 3], 2 $\rightarrow$ [1, 3], 4 $\rightarrow$ [2, 6], 5 $\rightarrow$ [4, 6], 7 $\rightarrow$ [6, 7].

So $\text{far}[0] = 3$ (from position 1), $\text{far}[1] = 3$, $\text{far}[2] = 6$, $\text{far}[4] = 6$, $\text{far}[6] = 7$.

reach	Starts scanned	Best far	Used
0	$\text{far}[0] = 3$	3	1
3	$\text{far}[1], \text{far}[2], \text{far}[3] \rightarrow 3, 6, 0$	6	2
6	$\text{far}[4], \text{far}[5], \text{far}[6] \rightarrow 6, 0, 7$	7	3
7	done	—	3

Both versions were run on four inputs and agreed every time: $n = 5$ with radii 3, 4, 1, 1, 0, 0 $\rightarrow$ 1; $n = 3$ all broken $\rightarrow$ -1; $n = 7$ above $\rightarrow$ 3; $n = 0 \rightarrow \emptyset$.

Answer: 3 sprinklers

Follow-up: arbitrary real positions

Sprinklers are now pairs (L_i, R_i) in any order with real endpoints. The $\text{far}[]$ array no longer works because the index is not an integer. Fix: sort the pairs by L , then sweep with a pointer that admits every interval whose $L \leq$ reach and tracks the largest R among them. Same loop, same proof; the cost becomes $O(m \log m)$ for the sort instead of $O(n)$.

Example 25 [Amazon · pattern]

A worker has a list of tasks, each labelled A..Z. Two tasks with the **same** label must be separated by at least `cool` idle-or-different slots. Each task takes one slot. Find the shortest total number of slots.

Constraints: up to 10^5 tasks, `cool` up to 100. **Target:** $O(n + 26)$. **Edge cases:** `cool` = 0; a single task; two labels tied for most frequent.

Follow-up the interviewer asks next: “now print an actual valid schedule, not just its length.”

SOLUTION

Think about the **most frequent** label. Say it appears m times. Those m copies force $m - 1$ gaps, and each gap is at least `cool` slots wide, so those copies alone stretch the timeline to

$$(m - 1) \times (\text{cool} + 1) + 1 \text{ slots}$$

If several labels tie at frequency m , each tie adds one more slot at the very end. With **ties** labels at the maximum:

$$\text{frame} = (m - 1) \times (\text{cool} + 1) + \text{ties}$$

Every other task is **poured into the idle holes** of that frame. If there are enough holes, the frame is the answer. If there are more tasks than holes, the holes all fill up and the schedule has no idle time at all, so the answer is just n . Hence:

$$\text{answer} = \max(\text{frame}, n)$$

scheduleLength — the frame formula

JS

```
const scheduleLength = (task, cool) => {
  const cnt = new Map();
  for (const c of task) cnt.set(c, (cnt.get(c) ?? 0) + 1);
  const mx = Math.max(0, ...cnt.values());
  // at most 26 values
```

```
const ties = [...cnt.values()].filter(v => v === mx).length;
const frame = (mx - 1) * (cool + 1) + ties;
return Math.max(frame, task.length);
};
```

Time $O(n)$ **Space** $O(1)$
One counting pass plus a fixed 26-slot scan.



frame = (3-1)(2+1) + 2 ties = 8, and n = 6, so answer = 8

AAA with cool = 2: three copies, two gaps of width 2, frame = 8

Tasks	cool	m	ties	frame	Answer
AAABBB	2	3	2	$2 \times 3 + 2 = 8$	8
AAABBB	0	3	2	$2 \times 1 + 2 = 4$	$\max(4, 6) = 6$
ABCDEABC	2	2	3	$1 \times 3 + 3 = 6$	$\max(6, 8) = 8$
A	5	1	1	$0 + 1 = 1$	1

All four rows were run and printed 8 6 8 1.

Answer: 8, 6, 8, 1 for the four rows

Follow-up: print a real schedule

Use a max-heap of (remaining count, label). Repeat: pop up to $\text{cool} + 1$ distinct labels, write them out (writing `idle` if the heap runs dry but tasks remain), decrement each count, and push back the ones still above zero. That is $O(n \log 26)$ and produces a schedule whose length matches the formula.

TRAP

With $m = 10^5$ and $\text{cool} = 100$ the frame is about 10^7 — fine anywhere. Interviewers often raise cool to 10^9 to see whether you noticed: $(m - 1)(\text{cool} + 1)$ is then about 10^{14} , which overflows a 32-bit int in C++ but is still **exact** in a JavaScript number. The run printed 99999000100000 with `Number.isSafeInteger` true. Push cool to 10^{12} and you are past $2^{53} - 1$ — that is when `BigInt` becomes the answer, not before.

Second, smaller trap in the same function: `Math.max(0, ...cnt.values())` spreads the values as arguments, and spreading a **large** array blows the call stack around 10^5 elements. It is safe here only because there are at most 26 labels. For a general array write `arr.reduce((m, x) => Math.max(m, x), -Infinity)`.

Example 26 [Microsoft · pattern]

Children stand in a line, each with a rating. Give every child at least one sweet, and any child whose rating is **strictly higher** than a neighbour must get strictly more sweets than that neighbour. Use the fewest sweets.

Constraints: $n \leq 10^5$, ratings up to 10^9 . The total fits a JS number easily. **Target:** $O(n)$ time. **Edge cases:** empty line; all ratings equal; a strictly increasing run of length n .

Follow-up the interviewer asks next: “now do it in $O(1)$ extra space.”

SOLUTION

Approach 1 — Keep fixing broken neighbours until nothing changes $(O(n^2)$ worst case)

sweetsBrute

JS

```
const sweetsBrute = (rank) => {
  const n = rank.length;
  if (n === 0) return 0;
  const c = new Array(n).fill(1);
  let changed = true;
  while (changed) {
    changed = false;
    for (let i = 0; i < n; i++) {
      if (i > 0 && rank[i] > rank[i-1] && c[i] <= c[i-1]) { c[i] = c[i-1] + 1;
changed = true; }
      if (i + 1 < n && rank[i] > rank[i+1] && c[i] <= c[i+1]) { c[i] = c[i+1] + 1;
changed = true; }
    }
  }
  return c.reduce((s, x) => s + x, 0);
};
```

Time $O(n^2)$ **Space** $O(n)$

A strictly rising line of n children needs n sweeps. It printed 5, 4, 0, 4 and 15 on the five tests — the same as the two-pass version.

Approach 2 — One pass each way $(O(n)$, optimal)

The constraint has **two directions** and one pass can only satisfy one of them. So do two passes and take the maximum.

1. Left to right: if $\text{rank}[i] > \text{rank}[i-1]$ then $c[i] = c[i-1] + 1$.
2. Right to left: if $\text{rank}[i] > \text{rank}[i+1]$ then $c[i] = \max(c[i], c[i+1] + 1)$.

The max in pass 2 is essential — it protects the work pass 1 already did.

sweets — two passes

JS

```
const sweets = (rank) => {
  const n = rank.length;
  if (n === 0) return 0;
  const c = new Array(n).fill(1);
  for (let i = 1; i < n; i++)
    if (rank[i] > rank[i-1]) c[i] = c[i-1] + 1;
  for (let i = n - 2; i >= 0; i--)
    if (rank[i] > rank[i+1]) c[i] = Math.max(c[i], c[i+1] + 1);
  return c.reduce((s, x) => s + x, 0);
};
```

Time $O(n)$ **Space** $O(n)$

Trace on ratings 1, 0, 2

Pass	i	c after	Why
start	—	1, 1, 1	everyone gets at least one
L to R	1	1, 1, 1	$0 > 1$? no
L to R	2	1, 1, 2	$2 > 0$? yes, so $c[1] + 1 = 2$

R to L	1	1, 1, 2	$0 > 2$? no
R to L	0	2, 1, 2	$1 > 0$? yes, so $\max(1, c[1] + 1) = 2$

Total = $2 + 1 + 2 = 5$.

Runs: $(1, 0, 2) \rightarrow 5$; $(1, 2, 2) \rightarrow 4$; empty $\rightarrow 0$; $(5, 5, 5, 5) \rightarrow 4$; $(1, 2, 3, 4, 5) \rightarrow 15$. All five matched.

Answer: 5 sweets

Follow-up: $O(1)$ extra space

Walk once and measure **runs**. Track up = the length of the current rising run and down = the length of the current falling run. Each rising run of length k contributes $1 + 2 + \dots + k$; each falling run likewise; the peak between them is counted once, in whichever run is longer. Keep a running total and reset up or down at each turn. No array, one pass, $O(1)$ memory. The two-pass version is easier to get right under pressure — offer the $O(1)$ version as the follow-up answer, not the first answer.

TRAP

max in the second pass, never plain assignment. Ratings 1, 3, 2: pass 1 gives 1, 2, 1. A plain assignment in pass 2 would set $c[1] = c[2] + 1 = 2$, which is right here — but ratings 1, 5, 4, 3 give pass 1 = 1, 2, 1, 1 and pass 2 must lift $c[1]$ to 3, not drop it. Use max and you are safe in both.

Example 27 [Adobe · pattern]

Event i is open on every day from $\text{start}[i]$ to $\text{end}[i]$ inclusive. You may attend at most one event per day, and attending an event takes exactly one day. Attend the most events.

Constraints: $n \leq 10^5$, days up to 10^5 . **Target:** $O(n \log n)$. **Edge cases:** several one-day events on the same day; an empty list.

Follow-up the interviewer asks next: “now each event has a value — maximise total value instead of count.”

SOLUTION

Greedy rule: **on each day, attend the open event that expires soonest**. An event with a late deadline can wait; one expiring today cannot.

A min-heap keyed on end gives the soonest-expiring open event in $O(\log n)$.

maxEvents — day sweep plus a min-heap of deadlines

JS

```
const { MinHeap } = require('./toolkit.js');

const maxEvents = (ev) => {
  const a = [...ev].sort((x, y) => x[0] - y[0] || x[1] - y[1]);
  const live = new MinHeap(); // end days of open events
  const n = a.length;
  let i = 0, got = 0, day = n ? a[0][0] : 0;
  while (i < n || live.size) {
    if (live.size === 0 && i < n) day = Math.max(day, a[i][0]);
    while (i < n && a[i][0] <= day) live.push(a[i++][1]);
    while (live.size && live.peek() < day) live.pop(); // already expired
    if (live.size) { live.pop(); got++; }
    day++;
  }
  return got;
};
```

Time $O(n \log n)$ **Space** $O(n)$

The jump `day = max(day, ev[i].first)` when the heap is empty keeps the loop from crawling through empty days one at a time.

Trace

Events (1, 2), (2, 3), (3, 4), (1, 2).

day	Heap (end days)	Attend	Total
1	push 2, push 2 $\rightarrow$ {2, 2}	an event ending 2	1
2	push 3 $\rightarrow$ {2, 3}	the other ending 2	2
3	push 4 $\rightarrow$ {3, 4}	the one ending 3	3
4	{4}	the one ending 4	4

Runs: (1, 2)(2, 3)(3, 4) $\rightarrow$ 3; the four events above $\rightarrow$ 4; three copies of (1, 1) $\rightarrow$ 1; empty list $\rightarrow$ 0.

Answer: 4 events

Follow-up: events have values

The count version is greedy because every event is worth 1. Add values and the greedy breaks — a cheap event expiring today may be worth skipping for an expensive one tomorrow. The fix is the **job sequencing with a heap** pattern: process events in order of deadline, push each value into a **min**-heap, and whenever the heap holds more events than the days available, pop the smallest value. That is $O(n \log n)$ and is the same trick as Example 16 run in the opposite direction.

Example 28 [Goldman Sachs · pattern]

Now every interval carries a **weight**. Pick non-overlapping intervals with the maximum total weight. First show that “earliest finish time” greedy is wrong, then give the correct algorithm.

Constraints: $n \leq 10^5$, weights up to 10^9 ; the total reaches 10^{14} , still exact in a JS number. **Target:** $O(n \log n)$.

Edge cases: one interval; all intervals identical.

Follow-up the interviewer asks next: “also print which intervals you chose.”

SOLUTION

Approach 1 — Earliest-finish greedy (WRONG — and knowing why is the point)

weightedGreedyByEnd — the tempting wrong answer

JS

```
// an interval is { s, e, w }
const weightedGreedyByEnd = (v) => {
  const a = [...v].sort((x, y) => x.e - y.e);
  let got = 0, freeAt = -Infinity;
  for (const j of a) if (j.s >= freeAt) { got += j.w; freeAt = j.e; }
  return got;
};
```

Counterexample: (1, 4) worth 30, (2, 6) worth 90, (5, 8) worth 20. Greedy takes (1, 4) then (5, 8) for 50. Taking (2, 6) alone gives 90. The exchange argument collapses because swapping an interval now changes the **weight**, not just the finishing time — so “no worse” is no longer guaranteed.

Approach 2 — DP over sorted ends, with binary search for the previous compatible job ($O(n \log n)$, correct)

Sort by end. Let $dp[i]$ be the best total using only the first i intervals. For interval i you either skip it ($dp[i]$) or take it, in which case you may add the best answer that finishes at or before $a[i].s$. Binary search finds that index — JavaScript has no `lower_bound`/`upper_bound`, so use the toolkit’s `upperBound` from the **JS Toolkit** appendix.

```
const { upperBound } = require('./toolkit.js');

const weightedDP = (v) => {
  const a = [...v].sort((x, y) => x.e - y.e);
  const n = a.length;
  const ends = a.map(j => j.e);
  const dp = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) {
    const p = Math.min(upperBound(ends, a[i].s), i); // jobs finishing at or
    before s
    dp[i + 1] = Math.max(dp[i], dp[p] + a[i].w);
  }
  return dp[n];
};
```

Time $O(n \log n)$ **Space** $O(n)$
 One sort plus one binary search per interval.

Trace

Sorted by end: (1, 4, 30), (2, 6, 90), (5, 8, 20). ends = 4, 6, 8.

i	Interval	p (first end > s)	Take = dp[p] + w	dp[i+1]
0	(1,4,30)	upperBound(ends, 1) = 0	0 + 30 = 30	max(0, 30) = 30
1	(2,6,90)	upperBound(ends, 2) = 0	0 + 90 = 90	max(30, 90) = 90
2	(5,8,20)	upperBound(ends, 5) = 1	30 + 20 = 50	max(90, 50) = 90

Runs: greedy printed 50, DP printed 90. On a case where greedy happens to be right — (1, 3, 5), (3, 5, 5), (5, 7, 5) — both printed 15. A randomised cross-check over 400 arrays of up to 8 intervals against an exhaustive 2^n search printed random check PASS.

Answer: Greedy 50, correct answer 90

Follow-up: print the chosen intervals

Keep a `from[i+1]` array recording which branch won. Walk backwards from n : if `dp[i+1] == dp[i]` the interval was skipped, so move to i ; otherwise record interval i and jump to p . Reverse the list at the end. Costs $O(n)$ extra memory and $O(n)$ time, and changes nothing about the forward pass.

TRAP

The toolkit `upperBound` searches the **whole** ends array, so the `Math.min(..., i)` is not decoration — it clamps the result to the already-computed prefix of `dp`. Drop it and a degenerate zero-length interval can return an index past i and read a `dp` entry that is still 0, silently giving a wrong answer on some inputs. Slicing the array instead (`upperBound(ends.slice(0, i), s)`) is also wrong for a different reason: it copies $O(n)$ elements per step and quietly turns the $O(n \log n)$ solution into $O(n^2)$.

Example 29 [D. E. Shaw · pattern]

Numbers arrive one at a time in an endless stream. After each arrival, be able to report the current set as a list of merged, disjoint ranges.

Constraints: up to 10^6 arrivals, values up to 10^9 , duplicates allowed. **Target:** $O(\log k)$ per arrival where k is the number of ranges. **Edge cases:** a number already inside a range; a number that glues two ranges into one.

Follow-up the interviewer asks next: “and if the stream is split across ten machines?”

SOLUTION

In C++ or Java you would reach for an ordered map (C++’s `map`, Java’s `TreeMap`) keyed on the range’s **start**, and get $O(\log k)$ per insert. **JavaScript has neither.** `Map` keeps insertion order, not sorted order, and has no “give me the next key after x ” operation.

The standard workaround is a **sorted array plus binary search**, and that is what we do here: two parallel arrays, `starts` and `ends`, kept sorted, with `upperBound` from the toolkit locating the insertion point. A new value x then needs only two lookups: the range starting just after x , and the range just before it.

Four cases:

1. x already sits inside the previous range $\rightarrow$ do nothing.
2. The previous range ends at $x - 1 \rightarrow$ extend it left-side, absorb it.
3. The next range starts at $x + 1 \rightarrow$ absorb it too.
4. Neither $\rightarrow$ insert the singleton $[x, x]$.

Cases 2 and 3 can both fire at once, which is exactly how two ranges get glued.

RangeStream — an ordered map of start $\rightarrow$ end

JS

```
const { upperBound } = require('./toolkit.js');

class RangeStream {
  constructor() { this.starts = []; this.ends = []; } // two parallel sorted
arrays
  add(x) {
    const i = upperBound(this.starts, x); // first range starting
    AFTER x
    const lo = i - 1; // the only range that
    could hold x
    if (lo >= 0 && this.ends[lo] >= x) return; // already inside
    let s = x, e = x, from = i, drop = 0;
    if (lo >= 0 && this.ends[lo] === x - 1) { s = this.starts[lo]; from = lo; drop
    = 1; }
    if (i < this.starts.length && this.starts[i] === x + 1) { e = this.ends[i];
    drop++; }
    this.starts.splice(from, drop, s);
    this.ends.splice(from, drop, e);
  }
  ranges() { return this.starts.map((s, i) => [s, this.ends[i]]); }
}
```

Time $O(\log k)$ to locate, $O(k)$ to splice **Space** $O(k)$

k is the number of disjoint ranges, never more than the number of distinct values seen. Say the honest cost out loud: the search is logarithmic, the array shift is linear but is a single memmove the engine does very fast.

TRAP

JavaScript has no `TreeMap`, no ordered set, and no `map.upperBound`. Say this before you write code, then name the two workarounds: (1) a sorted array plus binary search — $O(\log k)$ search, $O(k)$ insert, which is what almost every interview accepts; (2) a hand-written balanced BST or skip list if the interviewer insists on a true $O(\log k)$ insert. Pretending `Map` is ordered is the answer that loses the round.

Trace

Adding 1, 3, 7, 2, 6:

Add	Case	Ranges after
1	nothing near it	[1, 1]
3	nothing near it	[1, 1] [3, 3]
7	nothing near it	[1, 1] [3, 3] [7, 7]
2	previous ends at 1 and next starts at 3 — glue both	[1, 3] [7, 7]
6	next starts at 7	[1, 3] [6, 7]

Run output after those five: `[[1, 3], [6, 7]]`. Then adding 3 (already inside) and 5: `[[1, 3], [5, 7]]`. An empty stream reports `[]`, and adding 10, 10, `-1` gives `[[-1, -1], [10, 10]]`. A cross-check drove 10^5 random adds through both this class and a plain `Set`, then expanded the ranges back out: `oracle match: true`.

Answer: `[1, 3] [6, 7]`, and `[1, 3] [5, 7]` after the two extra adds

Follow-up: ten machines

Range sets **merge**. Each machine keeps its own map. To combine, pour all ranges into one list, sort by start and run Template A — the merge of two disjoint range sets is a normal interval merge. Because merging is associative and order-independent, the machines can combine in a tree rather than one by one, which is what makes this pattern fit a map-reduce job. Each machine’s local map is a **partial** answer, never a wrong one.

Dry run — one problem, every single step

We trace `minBoats` (Example 19) completely: every variable, after every line, with the weights 7, 2, 3, 6, 4, 1 and `limit = 8`.

Step 0 — sort. `w` becomes 1, 2, 3, 4, 6, 7. Indices 0...5.

Step 1 — set up. `i = 0`, `j = 5`, `boats = 0`.

Now the loop. Read each row as “state **before** the turn, then what the turn does”.

Turn	i	j	w[i]	w[j]	Sum vs 8	Action, then new state
1	0	5	1	7	$8 \leq 8$ fits	both ride. <code>i</code> $\rightarrow$ 1, <code>j</code> $\rightarrow$ 4, <code>boats</code> $\rightarrow$ 1
2	1	4	2	6	$8 \leq 8$ fits	both ride. <code>i</code> $\rightarrow$ 2, <code>j</code> $\rightarrow$ 3, <code>boats</code> $\rightarrow$ 2
3	2	3	3	4	$7 \leq 8$ fits	both ride. <code>i</code> $\rightarrow$ 3, <code>j</code> $\rightarrow$ 2, <code>boats</code> $\rightarrow$ 3
4	3	2	—	—	—	<code>i > j</code> , loop ends

Answer: 3 trips. Passengers are (1, 7), (2, 6), (3, 4).

The same trace when the pairing fails

Weights 3, 5, 3, 4, `limit = 5`. Sorted: 3, 3, 4, 5. `i = 0`, `j = 3`, `boats = 0`.

Turn	i	j	w[i]	w[j]	Sum vs 5	Action, then new state
1	0	3	3	5	$8 > 5$ no	5 rides alone. <code>i</code> stays 0, <code>j</code> $\rightarrow$ 2, <code>boats</code> $\rightarrow$ 1
2	0	2	3	4	$7 > 5$ no	4 rides alone. <code>i</code> stays 0, <code>j</code> $\rightarrow$ 1, <code>boats</code> $\rightarrow$ 2
3	0	1	3	3	$6 > 5$ no	one 3 rides alone. <code>i</code> stays 0, <code>j</code> $\rightarrow$ 0, <code>boats</code> $\rightarrow$ 3
4	0	0	3	3	$6 > 5$ no	<code>i == j</code> , the last person rides alone. <code>j</code> $\rightarrow$ <code>-1</code> , <code>boats</code> $\rightarrow$ 4
5	0	<code>-1</code>	—	—	—	<code>i > j</code> , loop ends

Answer: 4 trips. Both traces matched what node printed, exactly.

Look carefully at turn 4 of the second trace. i and j point at the **same person**. The condition $i \leq j$ lets the turn happen, the pair sum test fails (a person cannot ride with themselves anyway), j drops below i , and the trip is counted. That single $=$ is the whole reason the answer is 4 and not 3.

Practice

PRACTICE · TIER 1 target: 12 min

1. Given $2n$ numbers, split them into n pairs so the **largest** pair sum is as small as possible. Return that largest pair sum. Test with $\{3, 5, 2, 3\}$.
2. Can one person attend every meeting in a list? Return true or false.
3. A van has slots box positions. Each box type has a count and a “units per box” value. Load the most units. Test with types $(1, 3), (2, 2), (3, 1)$ and $\text{slots} = 4$.
4. Two **already sorted, already disjoint** interval lists. Return their intersection.
5. Notes of 5, 10 and 20 arrive at a counter selling a 5-rupee item. Each customer pays with one note and you must give correct change from notes you already have. Can you serve everybody?

ANSWER KEY

1. Sort, then pair the smallest with the largest, second smallest with second largest, and so on. Any other pairing puts two large numbers together, which can only raise the maximum.

minMaxPairSum

JS

```
const minMaxPairSum = (a) => {
  const v = [...a].sort((x, y) => x - y);
  let i = 0, j = v.length - 1, best = -Infinity;
  while (i < j) { best = Math.max(best, v[i] + v[j]); i++; j--; }
  return v.length === 0 ? 0 : best;
};
```

Run: $[3, 5, 2, 3]$ sorts to $2, 3, 3, 5$; pairs $(2, 5) = 7$ and $(3, 3) = 6$; output 7. $[1, 1]$ gives 2, $[]$ gives 0, and $[-5, -1, 0, 4]$ gives -1 — which is why `best` starts at `-Infinity` and not at 0.

Time $O(n \log n)$ **Space** $O(1)$ extra

2. Sort by start and look for one clash.

noClash

JS

```
const noClash = (v) => {
  const a = [...v].sort((x, y) => x[0] - y[0] || x[1] - y[1]);
  for (let i = 1; i < a.length; i++)
    if (a[i][0] < a[i-1][1]) return false;
  return true;
};
```

Run: $(0, 30), (5, 10) \rightarrow \text{false}$; $(7, 10), (2, 4) \rightarrow \text{true}$; empty $\rightarrow \text{true}$; $(1, 5), (5, 9) \rightarrow \text{true}$.

3. Sort box types by units-per-box, descending, and fill.

maxUnits

JS

```
const maxUnits = (box, slots) => { // box = [count, unitsPerBox]
  const a = [...box].sort((x, y) => y[1] - x[1]);
  let got = 0;
```

```

for (const [cnt, u] of a) {
  const take = Math.min(cnt, slots);
  got += take * u; slots -= take;
  if (slots === 0) break;
}
return got;
};

```

Run: 1 box of 3 units, then 2 boxes of 2 units, then 1 box of 1 unit = $3 + 4 + 1 = 8$. Output 8. With `slots = 0` the output is 0.

4. Two pointers. Intersect the two fronts; then **drop whichever ends first**, because it can never reach any later interval of the other list.

intersect

JS

```

const intersect = (A, B) => {
  const out = [];
  let i = 0, j = 0;
  while (i < A.length && j < B.length) {
    const lo = Math.max(A[i][0], B[j][0]), hi = Math.min(A[i][1], B[j][1]);
    if (lo <= hi) out.push([lo, hi]);
    if (A[i][1] < B[j][1]) i++; else j++;
  }
  return out;
};

```

Run on `[0,2] [5,10] [13,23]` and `[1,5] [8,12] [15,24]`: output `[1,2][5,5][8,10][15,23]`. With an empty first list: nothing.

5. Hoard the small notes. A 20 should be changed with $10 + 5$ when possible, because three 5s are more flexible than one 10 plus one 5.

canServe

JS

```

const canServe = (pay) => {
  let five = 0, ten = 0;
  for (const p of pay) {
    if (p === 5) five++;
    else if (p === 10) { if (!five) return false; five--; ten++; }
    else {
      if (ten && five) { ten--; five--; }
      else if (five >= 3) five -= 3;
      else return false;
    }
  }
  return true;
};

```

Run: `5,5,5,10,20` → true; `5,5,10,10,20` → false; a lone 10 → false; empty → true.

PRACTICE · TIER 2 target: 15 min

- Delete exactly k digits from a digit string so the remaining number is the smallest possible. Keep the digit order. Strip leading zeros. Test "1432219" with $k = 3$.
- Share prices day by day. You may buy and sell as many times as you like, but you may hold at most one share. Find the maximum total profit.

8. $2n$ candidates, each with a cost to fly to city A and a cost to fly to city B. Send exactly n to each city at the least total cost.
9. Rearrange a lowercase string so no two identical letters sit next to each other, or report that it is impossible.
10. Given an array, make it **strictly increasing** using the fewest total $+1$ increments. Return the number of increments.

ANSWER KEY

6. Walk left to right keeping a stack. A digit that is **smaller** than the top of the stack makes the number smaller if we drop that top — so pop while we still have deletions left. This is a monotonic-stack greedy.

dropK

JS

```
const dropK = (s, k) => {
  const st = [];
  for (const c of s) {
    while (st.length && k > 0 && st[st.length - 1] > c) { st.pop(); k--; }
    st.push(c);
  }
  while (k-- > 0 && st.length) st.pop();
  let p = 0;
  while (p + 1 < st.length && st[p] === '0') p++;
  const out = st.slice(p).join('');
  return out === '' ? '0' : out;
};
```

Run: `dropK("1432219", 3) → 1219`; `dropK("10200", 1) → 200`; `dropK("10", 2) → 0`. The trailing while matters for an already-increasing string like "12345", where nothing is ever popped inside the loop.

Time $O(n)$ **Space** $O(n)$

7. Collect every rise. If tomorrow is dearer than today, buy today and sell tomorrow; chains of rises add up to the same profit as one long hold, so this is never worse.

stockProfit

JS

```
const stockProfit = (p) => {
  let got = 0;
  for (let i = 1; i < p.length; i++)
    if (p[i] > p[i-1]) got += p[i] - p[i-1];
  return got;
};
```

Run: `7, 1, 5, 3, 6, 4 → 7`; a falling series `9, 8, 7 → 0`; one day `→ 0`.

8. Sort by (cost to A — cost to B). The most negative differences are the people who are relatively cheapest in A, so send the first half to A and the rest to B.

twoCity

JS

```
const twoCity = (c) => {
  const a = [...c].sort((x, y) => (x[0] - x[1]) - (y[0] - y[1]));
  const n = a.length / 2;
  return a.reduce((tot, [ca, cb], i) => tot + (i < n ? ca : cb), 0);
};
```


Run with $(10, 20), (30, 200), (400, 50), (30, 20)$: differences are $-10, -170, 350, 10$. Sorted order is $(30, 200), (10, 20), (30, 20), (400, 50)$. First two fly to A for $30 + 10 = 40$, last two fly to B for $20 + 50 = 70$. Output 110.

9. Always place the letter with the most copies left, but never the one you just placed. Hold the last-used letter aside for one turn, then push it back. JavaScript has no max-heap either — the toolkit `MinHeap` with a flipped comparator is one.

```
spreadOut JS

const { MinHeap } = require('./toolkit.js');

const spreadOut = (s) => {
  const cnt = new Map();
  for (const ch of s) cnt.set(ch, (cnt.get(ch) ?? 0) + 1);
  // no max-heap either: a MinHeap with a flipped comparator IS the max-heap
  const pq = new MinHeap((a, b) => b[0] - a[0] || (a[1] < b[1] ? -1 : 1));
  for (const [ch, c] of cnt) pq.push([c, ch]);
  const out = [];
  let hold = null;
  while (pq.size) {
    const [c, ch] = pq.pop();
    out.push(ch);
    if (hold && hold[0] > 0) pq.push(hold);
    hold = [c - 1, ch];
  }
  return out.length === s.length ? out.join('') : '';
};
```

Run: "aab" → aba; "aaab" → the empty string (impossible, because one letter has more than half the copies); "a" → a; "aaabbbccc" → abcabcabc, and a neighbour check on that output confirmed no two equal letters touch.

10. One pass. If an element is not already larger than its left neighbour, raise it to exactly `left + 1` — never higher, because a higher value only makes the next element more expensive.

```
makeIncreasing JS

const makeIncreasing = (a) => {
  const v = [...a];
  let cost = 0;
  for (let i = 1; i < v.length; i++)
    if (v[i] <= v[i-1]) { cost += v[i-1] + 1 - v[i]; v[i] = v[i-1] + 1; }
  return cost;
};
```

Run: 1, 1, 1 → 3 (raise to 1, 2, 3); 3, 2, 1 → 6 (raise to 3, 4, 5); 1, 5, 9 → 0.

PRACTICE · TIER 3 target: 18 min

- Split items into the fewest groups such that no group contains the same item twice. Prove your answer is a lower bound as well as achievable.
- You are given n intervals. Answer q queries of the form “how many intervals cover point x ?” Preprocess once, then answer each query fast.
- Explain, with a 3-interval counterexample, why sorting by **start** time breaks “pick the most non-overlapping intervals”.
- Given intervals and an integer k , find the smallest number of points such that every interval contains at least k of the chosen points. Solve it for $k = 1$ and describe the change for general k .

15. You run `mergeSorted` on a list that is **already sorted by end time** instead of start time. Give an input where the output is wrong, and say exactly which line breaks.

ANSWER KEY

11. The answer is the largest frequency. **Lower bound:** if some item appears f times, those f copies must sit in f different groups, so at least f groups are needed. **Achievable:** number the copies of each item $0, 1, 2, \dots$ and put copy number t into group t . No group ever receives two copies of the same item.

fewestGroups

JS

```
const fewestGroups = (a) => {  
  const f = new Map(); // Map, not {} -- keys keep their number  
  type  
  let best = 0;  
  for (const x of a) {  
    f.set(x, (f.get(x) ?? 0) + 1);  
    best = Math.max(best, f.get(x));  
  }  
  return best;  
};
```

Run: $[4, 4, 4, 2, 2, 7] \rightarrow 3$; $[1, 2, 3] \rightarrow 1$; $[] \rightarrow 0$; $[-1, -1] \rightarrow 2$ — a `Map` keeps `-1` as the **number** `-1`, where a plain object would stringify it to `"-1"`.

Time $O(n)$ **Space** $O(n)$

12. Difference array plus prefix sums. For every interval $[s, e]$ do $d[s] += 1$ and $d[e+1] -= 1$. The prefix sum of d at position x is the number of intervals covering x . Preprocessing is $O(n + M)$ where M is the coordinate range; each query is $O(1)$. If coordinates are up to 10^9 , first compress them: collect all endpoints, sort, remove duplicates, and index into that list — then M becomes $2n$ and a query is a binary search, $O(\log n)$.

13. Intervals $(0, 100), (1, 2), (3, 4)$. Sorted by start, greedy takes $(0, 100)$ and then nothing fits: answer 1. Sorted by end, greedy takes $(1, 2)$ and $(3, 4)$: answer 2. Starting early tells you nothing about when the room frees up; only the finish time does.

14. For $k = 1$: sort by end and sweep. Keep `last = -infinity`. For each interval, if its start is greater than `last`, place a point at its **end** and set `last` to that end. Placing the point at the end is the greedy move — it is the position that can also serve the largest number of future intervals. For general k : sort by end, and for each interval count how many already-placed points lie inside it; if that count is $c < k$, add the $k - c$ largest unused positions inside the interval, starting from its right end and walking left. The “place as far right as legally possible” rule is unchanged; only the number of points per step changes.

15. Input $(1, 10), (2, 3), (4, 5)$ is sorted by end as $(2, 3), (4, 5), (1, 10)$. `mergeSorted` walks it and produces $[2, 3]$ $[4, 5]$ $[1, 10]$ — not merged and not even sorted. The line that breaks is `if (v[i][0] <= last[1])`. Its correctness rests on the guarantee that every later interval starts at or after the current one, which only start-order provides. With end-order, $(1, 10)$ arrives last and reaches backwards over both earlier blocks, and the single open block cannot represent that.

ONE-PAGE REVISION CARD

The four JavaScript facts this chapter depends on.

<code>arr.sort()</code>	lexicographic. Always <code>arr.sort((a, b) => a - b)</code> , and the comparator returns a number .
no heap	MinHeap from the JS Toolkit appendix; flip the comparator for a max-heap.
no TreeMap	sorted array + <code>upperBound</code> , or a hand-rolled balanced tree. <code>Map</code> is insertion-ordered, not sorted.
numbers are doubles	exact to $2^{53} - 1 \approx 9 \times 10^{15}$. Everything in this chapter fits; past it, <code>BigInt</code> .

Two templates, memorised.

Sort by START	merge / union / insert / free-gaps. Keep one open block; stretch its end with <code>max</code> , or push it and open a new one.
Sort by END	count the most non-overlapping / minimum deletions / minimum arrows. Keep <code>freeAt</code> ; take an interval only when <code>s >= freeAt</code> .
Sweep (split events)	minimum rooms / maximum overlap. Sort starts and ends separately, walk with two pointers, <code>+1</code> on a start, <code>-1</code> on an end.
Heap	when you must know which resource freed up (room assignment), or repeatedly need the current smallest/largest (ropes, events, reorganise string).

Sort key cheat sheet.

Problem shape	Sort key	Cost
merge / union of intervals	start ascending	$O(n \log n)$
most non-overlapping	end ascending	$O(n \log n)$
minimum rooms	starts and ends separately	$O(n \log n)$
fractional knapsack	value/weight descending	$O(n \log n)$
job + deadline + profit	profit descending + DSU	$O(n \log n)$
pair two lists	both ascending	$O(n \log n)$
lift / boat with 2 seats	ascending, two pointers	$O(n \log n)$
biggest joined number	<code>a+b > b+a</code> on strings	$O(nL \log n)$
cheapest rope joins	min-heap	$O(n \log n)$
reach the end (jump)	none — carry <code>reach</code>	$O(n)$
circular fuel	none — carry <code>tank</code>	$O(n)$
cut string by last letter position	none — carry <code>reach</code>	$O(n)$

When greedy is NOT allowed.

Looks greedy but is not	Use instead
coin change, non-canonical coins	DP over amount, $O(A \cdot C)$
intervals with weights	DP sorted by end + binary search, $O(n \log n)$
0/1 knapsack (no cutting)	DP over capacity, $O(nW)$
maximise the value of events attended	deadline order + min-heap of values

The five traps that cost marks.

1. **Touch is not overlap.** Decide `<` versus `<=` before writing code, and write it down.
2. **Math.max when stretching a block.** `last[1] = Math.max(last[1], e)`, never plain assignment.
3. **Never a bare .sort().** It is lexicographic. Numbers need `(a, b) => a - b`, and a comparator must return a **number**, never a boolean.
4. **[...v] is shallow.** Intervals are arrays; copy each row with `v.map(iv => [...iv])` before mutating.
5. **i <= j in two-pointer pairing.** The last lone element still needs its own group.

The exchange argument in one sentence. Take any optimal answer, find the first place it differs from greedy, swap in greedy's choice, show the answer stays valid and no worse — repeat until optimal **is** greedy.

Thirty-second pre-flight check before you code any greedy. Try to break your rule with a 3-element input. If you cannot break it in 30 seconds, write the code. If you can, you just found the reason the problem is really DP.

Bit Manipulation, Math & Number Theory

Bits are just switches; number theory is just counting, done once and reused

Pattern in one page

A JavaScript number is a double, but the moment you write `&`, `|`, `^`, `<<` or `>>` the engine converts it to a **signed 32-bit integer**, does the operation, and converts back. So a bitwise value in JS is 32 switches in a row. A `BigInt` is as many switches as you like. Every bit trick in this chapter is one of three things: **read** a switch, **change** a switch, or **count** the switches that are on. Nothing more.

THE BIT TABLE — LEARN THESE BY HEART

Let k be a bit position, counted from the right starting at 0.

$x \gg k \ \& \ 1$	read bit k — gives 0 or 1
$x \mid (1 \ll k)$	turn bit k on
$x \ \& \ \sim(1 \ll k)$	turn bit k off
$x \ ^{(1 \ll k)}$	flip bit k
$x \ \& \ (x - 1)$	turn off the lowest bit that is on
$x \ \& \ -x$	keep only the lowest bit that is on
$(x \ \& \ (x - 1)) === 0$	x is 0 or a power of two
$x \ ^x$	always 0 — this is why XOR cancels pairs
$x \ ^0$	always x
$(a \ ^b) < 0$	a and b have opposite signs (32-bit)
$1 \ll n$	2^n — only while $n \leq 30$. Past that use <code>1n << BigInt(n)</code>

Why $x \ \& \ (x - 1)$ clears the lowest set bit. Subtracting 1 flips the lowest 1 into a 0 and turns every 0 below it into a 1. ANDing keeps only the bits both have — the lowest 1 is gone and the bits below it were 0 in x anyway.

x	binary	$x - 1$	$x \ \& \ (x-1)$
12	1100	1011	1000 = 8
8	1000	0111	0000 = 0
7	0111	0110	0110 = 6

The three XOR laws that solve half of these problems.

$$x \oplus x = 0, \quad x \oplus 0 = x, \quad a \oplus b = b \oplus a$$

Together they mean: XOR every element of an array, and every value that appears an **even** number of times vanishes.

THE NUMBER-THEORY TABLE

gcd and lcm.

$$\gcd(a, 0) = a, \quad \gcd(a, b) = \gcd(b, a \bmod b), \quad \text{lcm}(a, b) = \frac{a}{\gcd(a, b)} \times b$$

Divide **first**. Writing $a \times b / \gcd$ loses precision once the product passes 2^{53} .

Primes. To test one number, trial-divide up to $\sqrt{n}$. To get **all** primes up to n , use the sieve: cost $O(n \log \log n)$, which is nearly $O(n)$.

Factorisation. $N = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$. Then

$$\text{number of divisors} = \prod (a_i + 1), \quad \text{sum of divisors} = \prod \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

Factorials. The power of prime p in $n!$ is

$$\left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \dots$$

Trailing zeros of $n!$ is the power of 5 (there are always more 2s than 5s).

Modular arithmetic. With $M = 10^9 + 7$ (a prime):

$$(a + b) \bmod M, \quad (a - b + M) \bmod M, \quad (a \times b) \bmod M$$

Division is **not** allowed directly. Instead multiply by the modular inverse. By Fermat's little theorem, when M is prime and a is not a multiple of M :

$$a^{M-1} \equiv 1 \pmod{M} \Rightarrow a^{-1} \equiv a^{M-2} \pmod{M}$$

Binomial. $\binom{n}{r} = \frac{n!}{r!(n-r)!}$. Modulo a prime, precompute factorials and their inverses once, then every query is $O(1)$.

The two templates you will reuse all chapter

powMod is the one place where a JavaScript number is simply not good enough. With $M \approx 10^9$, the product $r * \text{base}$ reaches 10^{18} , and doubles are exact only to $2^{53} - 1 \approx 9 \times 10^{15}$. Every multiplication under a large modulus in this chapter therefore happens in BigInt.

Template A — fast exponentiation (square and multiply)

JS

```
const powMod = (base, exp, mod) => {
  const m = BigInt(mod);
  let b = BigInt(base) % m, e = BigInt(exp), r = 1n % m;
  if (b < 0n) b += m; // BigInt % keeps the sign, like C
  while (e > 0n) {
    if (e & 1n) r = r * b % m; // BigInt: never loses a digit
    b = b * b % m;
    e >>= 1n;
  }
  return Number(r); // the result is below mod, so it fits
};
```

Template B — sieve of Eratosthenes

JS

```
const sieve = (n) => {
  const composite = new Uint8Array(n + 1); // 1 byte per number, zero-filled
  const primes = [];
  for (let i = 2; i <= n; i++) {
    if (!composite[i]) {
      primes.push(i);
    }
  }
}
```

```

    for (let j = i * i; j <= n; j += i) composite[j] = 1;
  }
}
return primes;
};

```

Both were run. `powMod(2, 10, 1e9+7)` prints 1024, `powMod(2, 62, 1e9+7)` prints 145586002, `powMod(3, 10n**18n, 1e9+7)` prints 246336683. `sieve(30)` prints the ten primes below 30 and `sieve(1e7).length` prints 664579.

TRAP

The three JavaScript number bugs that fail more submissions than any algorithm mistake.

1. **`1 << 40` is not 2^{40} — it is 256.** JavaScript masks the shift count to its bottom 5 bits, so `1 << 40` computes `1 << 8`. Even worse, `1 << 32` gives 1, not 0, and `1 << 31` gives `−2147483648` because bit 31 is the sign bit. Safe shifts are `1 << k` for $k \leq 30$ only. Beyond that use `2 ** k` (a plain number, exact to $k = 53$) or `1n << BigInt(k)` (a `BigInt`, exact forever).
2. **`a * b % mod` silently rounds.** With `a = 123456789` and `b = 987654321`, `a * b` gives 121932631112635260 but the true product is 121932631112635269. The modulus then comes out as 259106854 instead of 259106859. Convert to `BigInt` **before** multiplying.
3. **A big literal is already wrong before you use it.** Typing 999999998000000002 in source produces the number 999999998000000000. If a value can exceed $2^{53} - 1$, it must arrive and stay as a `BigInt`.

TRAP

`>>` and `>>>` are different operators. `>>` keeps the sign bit, so `-1 >> 1` is still `−1` and a `while (x) { x >=> 1; }` loop over a negative number never ends. `>>>` shifts in zeros and treats the value as unsigned 32-bit: `-1 >>> 1` is 2147483647. Whenever a problem says “unsigned 32-bit”, the loop needs `>>>`, and `x >>> 0` is the idiom that turns a negative 32-bit result back into its unsigned reading.

Warm-up — build the reflex

WARM-UP

build the reflex

Example 1 [TCS NQT pattern]

Write four one-line functions: read bit k , set bit k , clear bit k , flip bit k . Test them all on $x = 22$.

Constraints: $0 \leq k \leq 30$. **Target:** $O(1)$ each. **Edge cases:** $k = 0$; a bit that is already in the wanted state.

SOLUTION

22 in binary is 10110. Positions from the right: bit 0 is 0, bit 1 is 1, bit 2 is 1, bit 3 is 0, bit 4 is 1.

the four one-bit operations

JS

```

const getBit   = (x, k) => (x >> k) & 1;
const setBit   = (x, k) => x | (1 << k);
const clearBit = (x, k) => x & ~(1 << k);
const flipBit  = (x, k) => x ^ (1 << k);

```

Call	Working	Binary	Result
<code>getBit(22,0)</code>	<code>10110 >> 0 & 1</code>	<code>...0</code>	0

getBit(22,1)	10110 >> 1 = 1011, and 1	...1	1
setBit(22,0)	10110 00001	10111	23
clearBit(22,1)	10110 & 11101	10100	20
flipBit(22,2)	10110 ^ 00100	10010	18

Run output: 0 1 1 1 for the four getBit calls (bits 0, 1, 2, 4), then 23 20 18.

Answer: 0, 1, 1, 1 then 23, 20, 18

setBit on a bit that is already 1 leaves it at 1 — OR is **idempotent**. The run confirms it: setBit(23, 0) prints 23 and clearBit(20, 1) prints 20. Only flipBit changes something every single time.

Example 2 [Accenture pattern]

Decide whether a number is a power of two, in $O(1)$, with no loop and no log.

Constraints: values up to 10^{18} , so the arithmetic must be BigInt. **Edge cases:** 0; negative numbers; 2^{40} .

SOLUTION

A power of two has **exactly one** bit on. And $x \& (x - 1)$ removes the lowest bit that is on. So if the result is 0, there was only one bit.

The $x > 0$ guard is essential: $x = 0$ gives $0n \& -1n === 0n$ (wrong answer “yes”), and a negative BigInt behaves as if it had an infinite run of sign bits.

isPowerOfTwo

JS

```
const isPowerOfTwo = (x) => {
  const b = BigInt(x);
  return b > 0n && (b & (b - 1n)) === 0n;
};
```

Run on 1, 2, 3, 64, 0, -8, 2^{40} : output 1101001. Exactly right — 3 is not a power of two, 0 is not, -8 is not, and 2^{40} is.

Answer: 1101001

TRAP

Do **not** write this with plain & on a number: $1 \ll 40$ is 256, so the number version would call 2^{40} a non-power-of-two and 256 a power of two. BigInt bitwise operators are the only ones in JavaScript that are not capped at 32 bits.

Example 3 [Infosys pattern]

Return the lowest set bit of a number **as a value**, and also its **index**.

Constraints: values up to 10^{18} . **Target:** $O(1)$ value, $O(63)$ index. **Edge cases:** 0 (no bits at all); 2^{40} .

SOLUTION

$x \& -x$ is the whole trick. In two’s complement, $-x$ is $\sim x + 1$, which flips every bit above the lowest 1 and leaves the lowest 1 alone. ANDing keeps only that bit. BigInt follows the same two’s-complement rule, with an unlimited sign extension.

lowest set bit, value and index

JS

```
const lowestSetBit = (x) => { const b = BigInt(x); return b & -b; };
```



```
const lowestSetIndex = (x) => {
  let b = BigInt(x);
  if (b === 0n) return -1;           // no bits at all
  b &= -b;                          // keep only the lowest one
  let i = 0;
  while (b > 1n) { b >>= 1n; i++; }
  return i;
};
```

x	binary	$-x$ binary (low part)	$x \& -x$	index
12	1100	...0100	4	2
1	0001	...1111	1	0
0	0000	0000	0	-1

Run output: 4n 1n 0n for the values and 2 -1 40 for the indices.

Answer: 4, 1, 0 and indices 2, -1, 40

TRAP

JavaScript has no `__builtin_ctz`, no `__builtin_popcount`, and no `Math.log2` you can trust for this — `Math.log2` returns a double and rounds. The explicit shift loop above is the honest answer, and 63 turns is free. For a **32-bit** value there is one built-in worth knowing: `Math.clz32(x)` counts **leading** zeros, so the index of the highest set bit is `31 - Math.clz32(x)`.

Example 4 [Wipro pattern]

Count how many bits of an unsigned 32-bit number are 1, the plain way.

Constraints: the full unsigned 32-bit range. **Target:** $O(32)$. **Edge cases:** 0; all bits on.

SOLUTION

countBitsLoop — one turn per bit position

JS

```
const countBitsLoop = (x) => {
  let v = x >>> 0, c = 0;           // >>> 0 reads the value as unsigned 32-bit
  while (v) { c += v & 1; v >>= 1; }
  return c;
};
```

Run: 0 → 0; 7 → 3; 255 → 8; 4294967295 → 32.

Answer: 0, 3, 8, 32

TRAP

Use `>>=`, not `>>=`. `>>` copies the sign bit, so on a negative input `v >>= 1` settles on `-1` forever and the loop never ends. The run proves it: `-1 >> 1` prints `-1` while `-1 >>= 1` prints `2147483647`.

Example 5 [Capgemini pattern]

Swap two values without a third variable.

Edge cases: the two names refer to the **same** storage slot.

SOLUTION

JavaScript passes numbers by value, so there is no `int&` to swap. The question really lives on two **slots** — two array elements — and that is where the classic bug lives too.

Three XORs. Each line uses the fact that $a \oplus b \oplus b = a$.

swapXor — with the guard nobody remembers

JS

```
const swapXor = (a, i, j) => {
  if (i === j) return a; // the guard everybody forgets
  a[i] ^= a[j]; a[j] ^= a[i]; a[i] ^= a[j];
  return a;
};
```

Line	a[i]	a[j]	Reasoning
start	3	9	—
a[i]^=a[j]	$3 \oplus 9 = 10$	9	slot i now holds the combination
a[j]^=a[i]	10	$9 \oplus 10 = 3$	slot j recovers the old a[i]
a[i]^=a[j]	$10 \oplus 3 = 9$	3	slot i recovers the old a[j]

Run: `swapXor([3, 9], 0, 1)` prints 9 3. `swapXor([5], 0, 0)` prints 5 thanks to the guard — without it the first line would set the slot to 0 and destroy the value.

Answer: 9 and 3; and 5 stays 5 when swapped with itself

SHORTCUT

In real JavaScript you never write this. `[a, b] = [b, a]` swaps two variables in one line, has no aliasing bug, and works on any type — the run prints 9 3 for that too. The XOR version is an interview question about two’s complement, not a technique you ship.

Example 6 [Cognizant pattern]

Turn off the lowest set bit and report what is left. Also say how many times you can repeat this before reaching 0.

Edge cases: 0; a power of two.

SOLUTION

dropLowestBit

JS

```
const dropLowestBit = (x) => { const b = BigInt(x); return b & (b - 1n); };
```

Run: `12 → 8n; 8 → 0n; 0 → 0n`.

The number of repeats before reaching 0 is exactly the number of 1 bits — which is the whole idea behind the next section’s fastest popcount.

Answer: 8, 0, 0

Tier 1 — the standard toolkit

TIER 1

TCS NQT · Accenture · Infosys · Wipro · Capgemini

Example 7 [TCS Digital pattern]

Every element of an array appears **twice**, except one which appears once. Find that one.

Constraints: $n \leq 10^6$, values can be negative. **Target:** $O(n)$ time, $O(1)$ space. **Edge cases:** a one-element array; negative values.

SOLUTION

Approach 1 — Count occurrences of each element by scanning again ($O(n^2)$, too slow)

singleBrute

JS

```
const singleBrute = (a) => {
  for (const v of a) {
    let c = 0;
    for (const x of a) if (x === v) c++;
    if (c === 1) return v;
  }
  return -1;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — Hash map of counts ($O(n)$ time but $O(n)$ space)

singleMap

JS

```
const singleMap = (a) => {
  const f = new Map(); // Map, not {} -- keys keep their type
  for (const x of a) f.set(x, (f.get(x) ?? 0) + 1);
  for (const [k, v] of f) if (v === 1) return k;
  return -1;
};
```

Time $O(n)$ **Space** $O(n)$

Approach 3 — XOR everything together ($O(n)$ time, $O(1)$ space, optimal)

The unlock: $x \oplus x = 0$ and $x \oplus 0 = x$, and XOR does not care about order. So XORing the whole array makes every **pair** cancel to 0, and 0 XOR the lonely value is the lonely value.

singleXor

JS

```
const singleXor = (a) => a.reduce((r, x) => r ^ x, 0);
```

Time $O(n)$ **Space** $O(1)$

One pass, one integer of state. Works on negatives without any change, because JS converts each operand to a signed 32-bit pattern and XOR operates on the raw bits.

Trace on 4, 7, 4, 9, 7

x	r before	r after	Binary
4	0	4	$0000 \oplus 0100 = 0100$
7	4	3	$0100 \oplus 0111 = 0011$
4	3	7	$0011 \oplus 0100 = 0111$

9	7	14	$0111 \oplus 1001 = 1110$
7	14	9	$1110 \oplus 0111 = 1001$

All three versions were run on this array; all three printed 9. The running values of `r` printed 4 3 7 14 9, matching the table row for row. `singleXor([-3, 5, -3])` prints 5 and `singleXor([8])` prints 8.

Answer: 9

TRAP

`singleXor` only works while every value fits in 32 signed bits. $2^{40} \oplus 0$ is 0, because the XOR converts 2^{40} to a 32-bit pattern first and every one of those bits is zero. If values can exceed $2^{31} - 1$, seed the reduce with 0n and map the array through `BigInt`.

Example 8 [Accenture pattern]

Count the set bits of a 64-bit number, but make the loop run once per **set** bit instead of once per bit position.

Constraints: values up to $2^{64} - 1$, so `BigInt`. **Target:** $O(\text{popcount})$. **Edge cases:** 0; all 64 bits on.

SOLUTION

Each turn, `x &= x - 1` deletes exactly one 1. Count the turns.

popcount — Kernighan's loop, `BigInt` so it survives past 32 bits

JS

```
const popcount = (x) => {
  let v = BigInt(x), c = 0;
  while (v) { v &= v - 1n; c++; } // one turn per SET bit
  return c;
};
```

Time $O(s)$ where s is the number of set bits **Space** $O(1)$

For a sparse number like 2^{40} this is 1 turn instead of 41 — the run confirms `popcount(2n ** 40n)` is 1.

Trace on $x = 12$ (1100)

Turn	x	x - 1	x & (x-1)
1	1100 = 12	1011	1000 = 8
2	1000 = 8	0111	0000 = 0

Two turns, so two set bits.

Run: $0 \rightarrow 0$; $7 \rightarrow 3$; $1023 \rightarrow 10$; $2^{64} - 1 \rightarrow 64$.

Answer: 0, 3, 10, 64

SHORTCUT

When the value is known to fit in 32 bits, do the same loop on a plain number — it is several times faster than `BigInt` and the code is identical apart from the literals:

popcount32 — same loop, no `BigInt`

JS

```
const popcount32 = (x) => {
  let v = x >>> 0, c = 0;
  while (v) { v &= v - 1; c++; }
}
```

```
return c;
};
```

Run: 0 3 10 32 for 0, 7, 1023, $2^{32} - 1$. There is **no** built-in popcount in JavaScript, so one of these two is what you write.

Python version

PYTHON

```
def popcount(x):
    c = 0
    while x:
        x &= x - 1
        c += 1
    return c
```

Run: `popcount(0) → 0`, `popcount(7) → 3`, `popcount(1023) → 10`. Python integers are arbitrary precision, so the same three lines handle 64-bit and 640-bit values alike — and `(1023).bit_count()` is the built-in shortcut JavaScript does not have.

Example 9 [Infosys pattern]

List every subset of a set of n distinct items, using bit masks.

Constraints: $n \leq 20$ (else 2^n is too big). **Target:** $O(2^n \cdot n)$. **Edge cases:** the empty set (one subset — itself).

SOLUTION

Number the items $0 \dots n - 1$. A subset is a number from 0 to $2^n - 1$ where bit i means “item i is in”. Counting from 0 to $2^n - 1$ therefore visits every subset exactly once.

allSubsets — counting is enumerating

JS

```
const allSubsets = (a) => {
    const n = a.length, out = [];
    for (let mask = 0; mask < (1 << n); mask++) {
        const cur = [];
        for (let i = 0; i < n; i++)
            if ((mask >> i) & 1) cur.push(a[i]);
        out.push(cur);
    }
    return out;
};
```

Time $O(2^n \cdot n)$ **Space** $O(2^n \cdot n)$

mask	binary	subset of {1, 2, 3}
0	000	{}
1	001	{1}
2	010	{2}
3	011	{1, 2}
4	100	{3}
5	101	{1, 3}

6	110	{2, 3}
7	111	{1, 2, 3}

Run output: $\{\}\{1\}\{2\}\{12\}\{3\}\{13\}\{23\}\{123\}$. On an empty input the function returns exactly one subset (the empty one), which the run confirmed by printing 1.

Answer: 8 subsets, listed above

TRAP

`mask < (1 << n)` is a **silent** disaster at $n = 31$ and above. $1 << 31$ is -2147483648 , so the loop never starts and you get zero subsets with no error at all. At $n = 32$, $1 << 32$ is 1 and you get one subset. In practice $n > 25$ is already too slow, so hitting this means you have misread the constraints — but write `mask < 2 ** n` if you want the loop to be honestly impossible rather than silently empty.

Example 10 [Wipro pattern]

Write gcd and lcm for two numbers up to 10^9 without losing precision.

Constraints: inputs up to 10^9 ; the lcm can reach 10^{18} , which is past $2^{53} - 1$. **Target:** $O(\log \min(a, b))$. **Edge cases:** one input is 0; negative inputs.

SOLUTION

Euclid's rule: $\gcd(a, b) = \gcd(b, a \bmod b)$, stopping when b is 0. Inputs up to 10^9 are far below 2^{53} , so gcd needs no BigInt — but the lcm does.

gcd and lcmSafe

JS

```
const gcd = (a, b) => {
  a = Math.abs(a); b = Math.abs(b);
  while (b) { [a, b] = [b, a % b]; } // destructuring: no temp variable
  return a;
};

const lcmSafe = (a, b) => {
  if (a === 0 || b === 0) return 0n;
  const g = BigInt(gcd(a, b));
  return BigInt(Math.abs(a)) / g * BigInt(Math.abs(b)); // divide FIRST
};
```

Time $O(\log \min(a, b))$ **Space** $O(1)$

Euclid halves the pair roughly every two steps, so it is logarithmic, never linear.

Trace of gcd(84, 132)

a	b	a mod b
84	132	84
132	84	48
84	48	36
48	36	12
36	12	0
12	0	stop

Answer 12. Note the first step politely swaps the pair for you: $84 \bmod 132 = 84$.

Runs: $\text{gcd}(84, 132) \rightarrow 12$; $\text{gcd}(0, 7) \rightarrow 7$; $\text{gcd}(-12, 18) \rightarrow 6$. $\text{lcmSafe}(4, 6) \rightarrow 12n$; $\text{lcmSafe}(0, 5) \rightarrow 0n$; $\text{lcmSafe}(1000000000, 999999999) \rightarrow 999999999000000000n$.

Answer: gcd 12; lcm of the two big numbers is 999999999000000000

TRAP

$a * b / \text{gcd}$ on plain numbers is the bug, and JavaScript hides it better than C++ does. With $a = 999999937$ and $b = 99999893$ (both prime, so the gcd is 1), the number version prints 99999830000006800. The true product is 99999830000006741 — the answer is wrong in its last **two** digits and nothing warned you. $a / \text{gcd} * b$ in BigInt never forms a rounded value at all.

Example 11 [Capgemini pattern]

Is n prime?

Constraints: n up to 10^{12} . **Target:** $O(\sqrt{n})$. **Edge cases:** 0 and 1 (not prime); 2 (the only even prime); a large prime.

SOLUTION

Approach 1 — Try every divisor from 2 to $n-1$ ($O(n)$, hopeless past 10^9)

isPrimeSlow

JS

```
const isPrimeSlow = (n) => {
  if (n < 2) return false;
  for (let d = 2; d < n; d++) if (n % d === 0) return false;
  return true;
};
```

Time $O(n)$ **Space** $O(1)$

Approach 2 — Stop at the square root, and skip even numbers ($O(\sqrt{n})$, optimal for a single test)

The unlock: divisors come in **pairs**. If $n = d \times e$ with $d \leq e$, then $d \leq \sqrt{n}$. So if no divisor up to $\sqrt{n}$ exists, none exists at all — the partner would have been found first.

isPrimeFast

JS

```
const isPrimeFast = (n) => {
  if (n < 2) return false;
  if (n % 2 === 0) return n === 2;
  for (let d = 3; d * d <= n; d += 2) if (n % d === 0) return false;
  return true;
};
```

Time $O(\sqrt{n}/2)$ **Space** $O(1)$

$d * d$ reaches $n = 10^{12}$ at most, and 10^{12} is well under 2^{53} , so plain numbers stay exact here. Use $d * d <= n$, never $d <= \text{Math.sqrt}(n)$.

Runs: both versions printed the same for 97 (1), 1 (0) and 2 (1). The fast version alone handled 1000000007 and printed 1; the slow one would need a billion turns. At the top of the range isPrimeFast(1000000000039) prints 1 and isPrimeFast(1000000000000) prints 0.

Answer: 97 prime, 1 not prime, 2 prime, 1000000007 prime

TRAP

`Math.sqrt` returns a double. At n near 10^{12} it is still exact, but the habit is what kills you later: above 2^{53} the square root is rounded and the loop bound is off by one in either direction. `d * d <= n` has no such failure mode while $n \leq 2^{53}$, and past that the whole function must move to `BigInt`.

Example 12 [Cognizant pattern]

Print every prime up to n .

Constraints: $n \leq 10^7$. **Target:** $O(n \log \log n)$, $O(n)$ memory. **Edge cases:** $n = 1$ (no primes); $n = 10^6$.

SOLUTION

Trial division per number would cost $O(n\sqrt{n})$. The sieve flips the work around: instead of asking “who divides me?”, each prime **marks** its own multiples.

Start the inner loop at $i \times i$, not $2i$. Every smaller multiple of i already has a smaller prime factor and was marked long ago.

sieve — Template B

JS

```
const sieve = (n) => {
  const composite = new Uint8Array(n + 1);
  const primes = [];
  for (let i = 2; i <= n; i++) {
    if (!composite[i]) {
      primes.push(i);
      for (let j = i * i; j <= n; j += i) composite[j] = 1;
    }
  }
  return primes;
};
```

Time $O(n \log \log n)$ **Space** $O(n)$

$\log \log 10^7$ is under 4, so in practice this is a small constant times n .

Trace for $n = 30$

i = 2	prime. Mark 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30
i = 3	not marked → prime. Mark 9, 12, 15, 18, 21, 24, 27, 30
i = 4	already marked, skip
i = 5	prime. Mark 25, 30
i = 6	marked, skip
i = 7	prime. $7 \times 7 = 49 > 30$, nothing to mark
8 to 30	every composite is already marked; the rest are collected as primes

Run output: 2 3 5 7 11 13 17 19 23 29, count 10. `sieve(1)` returns nothing; `sieve(1000000)` returns 78498 primes and `sieve(10000000)` returns 664579.

Answer: 10 primes below 30; 78498 primes below 10^6

Python version

PYTHON

```
def sieve(n):
    composite = [False] * (n + 1)
    primes = []
```



```

for i in range(2, n + 1):
    if not composite[i]:
        primes.append(i)
        for j in range(i * i, n + 1, i):
            composite[j] = True
return primes

```

Run: `sieve(30)` prints the same ten primes; `len(sieve(1000000))` prints 78498.

TRAP

Use a `Uint8Array`, not `new Array(n + 1).fill(false)`. A plain JavaScript array of 10^7 booleans is 10^7 tagged values — tens of megabytes and a cache miss on every write. `Uint8Array` is one contiguous byte per entry, 10 MB for $n = 10^7$, and it starts zero-filled so you do not even pay for the `fill`. `i * i` needs no special care here: 10^7 squared is 10^{14} , comfortably inside the exact-integer range.

Example 13 [TCS NQT pattern]

How many zeros does $n!$ end with?

Constraints: n up to 10^{18} . **Target:** $O(\log_5 n)$. **Edge cases:** $n = 0$ (answer 0); $n = 5$; $n = 10^9$.

SOLUTION

A trailing zero comes from a factor of 10, and $10 = 2 \times 5$. In $n!$ there are far more 2s than 5s, so the answer is **the number of 5s**.

Count them in layers: $\lfloor \frac{n}{5} \rfloor$ numbers give at least one 5, $\lfloor \frac{n}{25} \rfloor$ give a second one, $\lfloor \frac{n}{125} \rfloor$ give a third, and so on.

$$Z(n) = \left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{n}{25} \right\rfloor + \left\lfloor \frac{n}{125} \right\rfloor + \dots$$

n can be 10^{18} , past $2^{53} - 1$, so both the input and the running total are `BigInt`. `BigInt` division truncates toward zero, which is exactly the floor we want for positive values.

trailingZeros

JS

```

const trailingZeros = (n) => {
    const N = BigInt(n);
    let z = 0n;
    for (let p = 5n; p <= N; p *= 5n) z += N / p;    // BigInt / truncates = floor
    return z;
};

```

Time $O(\log_5 n)$ **Space** $O(1)$

For $n = 10^{18}$ the loop runs 26 times.

n	n/5	n/25	n/125	Total
5	1	0	0	1
25	5	1	0	6
100	20	4	0	24

Run output: `1n 6n 24n 0n 249999998n` for $n = 5, 25, 100, 0, 10^9$, and `trailingZeros(10n ** 18n)` prints `24999999999999995n`.

Answer: 1, 6, 24, 0, 249999998

TRAP

Had this been written on plain numbers, `p *= 5` would still **run** — `BigInt` is the only thing that makes the 10^{18} case correct. The number version fails twice over: the argument $10^{18} + 3$ cannot even be written down exactly, and `z` itself reaches 2.5×10^{17} , which is past $2^{53} - 1$. A count that can exceed 9×10^{15} is a `BigInt` count.

Example 14 [Accenture pattern]

Compute $a^b \bmod m$ where b can be 10^{18} .

Constraints: $a, m \leq 10^9$, $b \leq 10^{18}$. **Target:** $O(\log b)$. **Edge cases:** $b = 0$; $m = 1$; a negative base.

SOLUTION

Multiplying b times is 10^{18} operations. Instead use the binary expansion of b .

$$a^{13} = a^{8+4+1} = a^8 \times a^4 \times a^1$$

13 in binary is 1101. So: square the base at every step and multiply it into the answer whenever the current bit of b is 1.

powMod — Template A

JS

```
const powMod = (base, exp, mod) => {
  const m = BigInt(mod);
  let b = BigInt(base) % m, e = BigInt(exp), r = 1n % m;
  if (b < 0n) b += m;
  while (e > 0n) {
    if (e & 1n) r = r * b % m;
    b = b * b % m;
    e >>= 1n;
  }
  return Number(r);
};
```

Time $O(\log b)$ **Space** $O(1)$

At most 60 turns for an exponent of 10^{18} . `BigInt` ops on numbers this small are a few nanoseconds each.

Trace of `powMod(2, 10, 1000000007)`

10 in binary is 1010.

Turn	exp	bit	r after	base after
1	10 = 1010	0	1	$2^2 = 4$
2	5 = 101	1	$1 \times 4 = 4$	$4^2 = 16$
3	2 = 10	0	4	$16^2 = 256$
4	1 = 1	1	$4 \times 256 = 1024$	256^2
5	0	—	stop	—

Run output: 1024 1 145586002 0 for $2^{10} \bmod (10^9 + 7)$, $3^0 \bmod 7$, $2^{62} \bmod (10^9 + 7)$, and $5^3 \bmod 1$. A negative base works too: `powMod(-2, 3, 1e9+7)` prints 999999999, which is -8 folded back into range.

Answer: 1024; 1; 145586002; 0

TRAP

let $r = 1n \% m$; — not $= 1n$. When mod is 1 every answer must be 0, and starting at 1 with an exponent of 0 would wrongly return 1. This is a favourite hidden test case.

The `BigInt(exp)` in the signature is doing quiet work as well: it accepts $10n ** 18n$ **and** the number 1000000000. If you let a caller pass the **number** $10^{18} + 1$, the exponent is wrong before your function ever sees it.

Python version — the whole thing is a built-in

PYTHON

```
def pow_mod(base, exp, mod):  
    return pow(base, exp, mod)      # three-argument pow IS modular  
exponentiation
```

Run: `pow_mod(2, 10, 1000000007)` prints 1024, `pow_mod(2, 62, 1000000007)` prints 145586002, `pow_mod(5, 3, 1)` prints 0, and `pow_mod(3, 10 ** 18, 1000000007)` prints 246336683 — the same four values the JavaScript version produced. Python integers are arbitrary precision, so there is no `BigInt` decision to make and no overflow to guard against. This is the clearest case in the book of Python being shorter for the wrong reason: you still have to be able to **write** square-and-multiply on a whiteboard.

Example 15 [Infosys pattern]

Reverse the 32 bits of an unsigned number: bit 0 becomes bit 31, bit 1 becomes bit 30, and so on.

Constraints: exactly 32 bits. **Target:** $O(32)$. **Edge cases:** 0; 1; all bits on.

SOLUTION

Pull one bit off the right of the input and push it onto the right of the answer. After 32 turns the order is reversed.

reverseBits

JS

```
const reverseBits = (x) => {  
    let v = x >>> 0, r = 0;  
    for (let i = 0; i < 32; i++) {  
        r = ((r << 1) | (v & 1)) >>> 0;      // >>> 0 keeps r unsigned  
        v >>= 1;  
    }  
    return r >>> 0;  
};
```

Time $O(32)$ **Space** $O(1)$

Run: $1 \rightarrow 2147483648$ (that is 2^{31} — bit 0 moved to bit 31); $0 \rightarrow 0$; all 32 bits on $\rightarrow 4294967295$ (unchanged, as expected). `reverseBits(2147483648)` prints 1, which is the round trip.

Answer: 2147483648, 0, 4294967295

TRAP

Two failures hide in this five-line function.

1. The loop must run a fixed **32** times, not `while (v)`. Reversing 1 with a `while` loop would stop after one turn and return 1 instead of 2^{31} .
2. Every `>>> 0` is load-bearing. Once the answer reaches bit 31, `r << 1` produces a **negative** number, and without the `>>> 0` the function returns -1 instead of 4294967295. Any time a 32-bit answer is meant to be read as unsigned, the last thing you do to it is `>>> 0`.

Example 16 [Wipro pattern]

Digit sum and digit reversal of a possibly negative integer.

Constraints: values within the exact-integer range. **Target:** $O(d)$ where d is the digit count. **Edge cases:** 0; a negative number; a number ending in zeros.

SOLUTION

There is no integer division in JavaScript, so $n / 10$ would give a fraction and the loop would never terminate. `Math.floor(v / 10)` is the replacement, and the loop condition is $v > 0$ rather than a truthiness test.

digitSum and reverseDigits

JS

```
const digitSum = (n) => {
  let v = Math.abs(n), s = 0;
  while (v > 0) { s += v % 10; v = Math.floor(v / 10); }
  return s;
};

const reverseDigits = (n) => {
  const sign = n < 0 ? -1 : 1;
  let v = Math.abs(n), r = 0;
  while (v > 0) { r = r * 10 + (v % 10); v = Math.floor(v / 10); }
  return sign * r;
};
```

Runs: `digitSum(98765) → 35`; `digitSum(0) → 0`; `digitSum(-407) → 11`. `reverseDigits(1200) → 21`; `reverseDigits(-95) → -59`; `reverseDigits(0) → 0`.

Answer: 35, 0, 11; then 21, -59, 0

`reverseDigits(1200)` gives 21, not 0021. Leading zeros simply do not exist in a number — this is a favourite “gotcha” question and the honest answer is that nothing special needs to be done.

SHORTCUT

For a value beyond $2^{53} - 1$, stop doing arithmetic and use the string: `[...String(n)].reduce((s, c) => s + +c, 0)` for the digit sum, and `[...String(n)].reverse().join('')` for the reversal. Strings have no precision limit, and this is the version to reach for when the constraint says 10^{18} . Note the string reversal keeps the leading zeros — 1200 comes back as “0021”, the run confirms it — so wrap it in `BigInt(...)` if you want the number 21 back.

Tier 2 — two ideas stitched together

TIER 2

Singapore & Thailand · Grab · Shopee · GIC · DBS · Agoda · SCB

Example 17 [Grab · pattern]

In an array, **two** different values appear exactly once and every other value appears exactly twice. Find both, in ascending order.

Constraints: $n \leq 10^6$, values may be negative. **Target:** $O(n)$ time, $O(1)$ space. **Edge cases:** an array of exactly two elements; negative values.

SOLUTION

Approach 1 — Count every value in a map ($O(n \log n)$ time and $O(n)$ space)

```
const twoSinglesMap = (a) => {
  const f = new Map();
  for (const x of a) f.set(x, (f.get(x) ?? 0) + 1);
  return [...f].filter(([, c]) => c === 1)
    .map(([k]) => k)
    .sort((p, q) => p - q); // NEVER a bare .sort()
};
```

Time $O(n \log n)$ **Space** $O(n)$

Run on the two tests below it printed 4 9 and 3 7 — the same as the optimal version. The problem statement's $O(1)$ space rule is what rules it out.

TRAP

arr.sort() in JavaScript is LEXICOGRAPHIC, not numeric. [10, 9, 1].sort() gives [1, 10, 9], because the default comparator turns every element into a string first. This problem asks for the answer **in ascending order**, so the bare sort is a wrong answer, not a style issue — the run proves it: sorting the keys 4, 9, 10 with .sort() prints 10 4 9, while .sort((p, q) => p - q) prints 4 9 10.

Two rules travel with it:

- A comparator must return a **number**, never a boolean. (p, q) => p > q yields true/false, which coerce to 1/0, so the sort never sees a negative and leaves the array essentially untouched.
- Chain tie-breaks with ||, because 0 is falsy: (p, q) => p[0] - q[0] || p[1] - q[1].

Approach 2 — XOR everything, then split on a differing bit ($O(n)$ time, $O(1)$ space, optimal)

XOR the whole array. Every pair cancels, so you are left with $u \oplus v$ — the XOR of the two lonely values. That is not enough on its own.

The unlock: **u and v are different, so $u \oplus v$ has at least one bit set, and at that bit position u and v disagree.** Pick any such bit — all & -all gives the lowest one. Now split the array into “bit set” and “bit clear”. Each half contains exactly one lonely value and complete pairs. XOR each half separately.

```
const twoSingles = (a) => {
  let all = 0;
  for (const x of a) all ^= x;
  const bit = all & -all; // any bit where the two differ
  let g1 = 0, g2 = 0;
  for (const x of a) {
    if (x & bit) g1 ^= x;
    else g2 ^= x;
  }
  return [Math.min(g1, g2), Math.max(g1, g2)];
};
```

Time $O(n)$ **Space** $O(1)$

Two passes, three integers of state, and Math.min/Math.max instead of a sort.

Trace on 4, 1, 2, 1, 2, 9

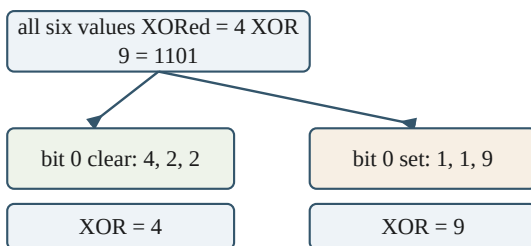
Pass 1. $4 \oplus 1 \oplus 2 \oplus 1 \oplus 2 \oplus 9 = 4 \oplus 9 = 13$ (1101). So bit = $13 \& -13 = 1$ (0001).

Pass 2. Split on bit 0 (odd or even).

x	x & 1	Goes to	Group XOR after
4	0	g2 (even)	g2 = 4
1	1	g1 (odd)	g1 = 1
2	0	g2	g2 = 6
1	1	g1	g1 = 0
2	0	g2	g2 = 4
9	1	g1	g1 = 9

Run output: 4 9. `twoSingles([-5, 7, -5, 3])` prints 3 7 and `twoSingles([3, 8])` prints 3 8.

Answer: 4 and 9



the differing bit splits the array into two clean halves

Example 18 [Shopee · pattern]

An array of length n holds values from 1 to n . Exactly one value is **missing** and exactly one value appears **twice**. Find both.

Constraints: $n \leq 10^6$. **Target:** $O(n)$ time, $O(1)$ space. **Edge cases:** $n = 2$; the repeated value is 1 or n .

SOLUTION

Approach 1 — Tally the values in a counting array $(O(n)$ time but $O(n)$ space)

```

missRepCount
JS

const missRepCount = (a) => {
  const n = a.length;
  const seen = new Array(n + 2).fill(0);
  for (const x of a) seen[x]++;
  let rep = -1, mis = -1;
  for (let v = 1; v <= n; v++) {
    if (seen[v] === 2) rep = v;
    if (seen[v] === 0) mis = v;
  }
  return [rep, mis];
};

```

Time $O(n)$ **Space** $O(n)$

Printed 3 2 and 2 1 on the two tests — the same answers. Say this one out loud first in an interview, then offer the $O(1)$ -space version.

Approach 2 — XOR, then split on a differing bit $(O(n)$ time, $O(1)$ space, optimal)

XOR the array **and** XOR $1\dots n$. Everything that appears the right number of times cancels, leaving repeated XOR missing. Then split on a differing bit exactly as in Example 17 — but this time each half must be swept over **both** the array and the range $1\dots n$.

At the end you have the two values but not which is which. One extra count pass settles it: if the array contains the candidate twice, that one is the repeat.

missingAndRepeated — returns [repeated, missing]

JS

```
const missingAndRepeated = (a) => {
  const n = a.length;
  let x = 0;
  for (const v of a) x ^= v;
  for (let v = 1; v <= n; v++) x ^= v;      // x = repeated XOR missing
  const bit = x & -x;
  let p = 0, q = 0;
  for (const v of a) { if (v & bit) p ^= v; else q ^= v; }
  for (let v = 1; v <= n; v++) { if (v & bit) p ^= v; else q ^= v; }
  let cnt = 0;
  for (const v of a) if (v === p) cnt++;
  return cnt === 2 ? [p, q] : [q, p];
};
```

Time $O(n)$ **Space** $O(1)$

Four linear passes, no extra array.

Runs: `missingAndRepeated([3,1,3,4,5])` prints 3 2 — 3 is repeated, 2 is missing.
`missingAndRepeated([2,2])` prints 2 1. The two edge cases named in the statement both hold: `[1,2,3,3]` prints 3 4 (the repeat is n) and `[1,1,3,4]` prints 1 2 (the repeat is 1).

Answer: repeated 3, missing 2

SHORTCUT

A shorter but riskier alternative uses sums: $S = \sum a_i - \sum_1^n i = \text{rep} - \text{miss}$ and $Q = \sum a_i^2 - \sum i^2 = \text{rep}^2 - \text{miss}^2$. Solve the two equations. It works, but $\sum i^2$ for $n = 10^6$ is about 3.3×10^{17} — past $2^{53} - 1$, so in JavaScript the whole computation must be BigInt and the “shorter” version stops being shorter. The XOR version never leaves 32-bit integers at all.

Example 19 [GIC · pattern]

Compute $\frac{a}{b} \bmod M$ where $M = 10^9 + 7$ is prime. Then use it to compute $\binom{n}{r} \bmod M$ for up to 10^5 queries with $n \leq 2 \times 10^5$.

Target: $O(n)$ preprocessing, $O(1)$ per query. **Edge cases:** $r < 0$ or $r > n$ (answer 0); $r = 0$; $n = r$.

SOLUTION

Why you cannot just divide. $\left(\frac{a}{b}\right) \bmod M$ is not $\frac{a \bmod M}{b \bmod M}$. In JavaScript it is worse than in a language with integer division: `7 / 2` is 3.5, and `3.5 % 5` is 3.5. A modulus applied to a fraction is meaningless.

The fix. Find the number b^{-1} with $b \times b^{-1} \equiv 1 \pmod{M}$, then multiply by it. Fermat’s little theorem gives it for free when M is prime:

$$b^{M-1} \equiv 1 \pmod{M} \Rightarrow b^{-1} \equiv b^{M-2} \pmod{M}$$

inverse by Fermat

JS

```
const inverse = (a, p) => powMod(a, p - 2, p);
```

For binomials, precompute all factorials and **all** their inverses. The clever part: do **one** expensive inverse at the top and walk backwards, because $(\text{inv})(i-1)! = (\text{inv})i! \times i$.

Every multiplication here is $10^9 \times 10^9 \approx 10^{18}$, far past $2^{53} - 1$, so the tables hold `BigInt` and only the returned value is converted back.

makeComb — $O(n)$ setup, $O(1)$ queries

JS

```
const makeComb = (n, mod) => {
  const p = BigInt(mod);
  const f = new Array(n + 1).fill(1n);
  for (let i = 1; i <= n; i++) f[i] = f[i - 1] * BigInt(i) % p;

  const inv = new Array(n + 1).fill(1n);
  inv[n] = BigInt(powMod(Number(f[n]), mod - 2, mod)); // one expensive step
  for (let i = n; i >= 1; i--) inv[i - 1] = inv[i] * BigInt(i) % p;

  return (N, r) => (r < 0 || r > N)
    ? 0
    : Number(f[N] * inv[r] % p * inv[N - r] % p);
};
```

Time $O(n + \log p)$ setup, $O(1)$ per query **Space** $O(n)$

Only one modular exponentiation in the whole setup. The closure keeps `f` and `inv` alive without a class.

Runs: `inverse(3, MOD)` prints 333333336, and `3n * 333333336n % MOD` prints 1n — which is the proof it is correct. With `const C = makeComb(200000, MOD)`: `C(5, 2) → 10`; `C(10, 0) → 1`; `C(5, 9) → 0`; `C(200000, 100000) → 879467333`. That last one was checked against an exact `BigInt` product of $\binom{200000}{100000}$ reduced mod M , which printed 879467333 too.

Answer: $3^{-1} = 333333336$; $\binom{200000}{100000} \equiv 879467333$

TRAP

Fermat needs M **prime**. For a composite modulus the inverse exists only when $\gcd(a, M) = 1$, and you must use the extended Euclid method instead — that is Example 30.

A second, JavaScript-only trap: never verify an inverse with plain numbers. `3 * 333333336 % 10000000007` happens to be right because the product is small, but the same check on a pair near 10^9 would round and print a wrong “proof”. Verify in `BigInt`.

Example 20 [Sea · pattern]

Answer up to 10^5 queries of the form “give me the prime factorisation of x ”, where $x \leq 10^6$.

Target: $O(n \log \log n)$ setup, then $O(\log x)$ per query. **Edge cases:** $x = 1$ (no factors); x prime.

SOLUTION

Approach 1 — Trial division, once per query ($O(\sqrt{x})$ per query, 10^8 total — too slow)

factoriseTrial

JS

```
const factoriseTrial = (x) => {
  const out = [];
  let v = x;
  for (let d = 2; d * d <= v; d++)
    if (v % d === 0) {
      let c = 0;
      while (v % d === 0) { v /= d; c++; } // exact: d divides v, so no fraction
      out.push([d, c]);
    }
```



```

    }
    if (v > 1) out.push([v, 1]);
    return out;
};

```

Time $O(\sqrt{x})$ per query **Space** $O(1)$

Printed $2^3 3^2 5^1$ for 360 and 999983^1 for the prime — the same as the SPF version. Use this when there is only ONE query.

Approach 2 — Precompute the smallest prime factor of everything $O(\log x)$ per query, optimal for many queries

Trial division per query is $O(\sqrt{x})$, which is 1000 steps each — 10^8 total. Too slow. Instead precompute, for every number, its **smallest prime factor** (SPF). Then factorising is: divide by the SPF, look up the SPF of what remains, repeat. Each division at least halves the number, so at most $\log_2 x$ steps.

spfTable and factorise

JS

```

const spfTable = (n) => {
  const spf = new Int32Array(n + 1);
  for (let i = 0; i <= n; i++) spf[i] = i;
  for (let i = 2; i * i <= n; i++)
    if (spf[i] === i)
      for (let j = i * i; j <= n; j += i)
        if (spf[j] === j) spf[j] = i;
  return spf;
};

const factorise = (x, spf) => {
  const out = [];
  let v = x;
  while (v > 1) {
    const p = spf[v];
    let c = 0;
    while (v % p === 0) { v /= p; c++; }
    out.push([p, c]);
  }
  return out;
};

```

Time $O(n \log \log n)$ setup, $O(\log x)$ per query **Space** $O(n)$

Int32Array, not a plain array: 4 MB flat instead of a tagged-value array many times that size.

Trace of factorise(360)

x	spf[x]	Divide out	Recorded
360	2	$360 \rightarrow 180 \rightarrow 90 \rightarrow 45$	2^3
45	3	$45 \rightarrow 15 \rightarrow 5$	3^2
5	5	$5 \rightarrow 1$	5^1

Run output: $2^3 3^2 5^1$. For the prime 999983 the output is 999983^1 , and `factorise(1, spf)` returns an empty list.

Bonus — divisor sum from the factorisation.

sumOfDivisors

JS

```
const sumOfDivisors = (x, spf) => {
  let s = 1;
  for (const [p, c] of factorise(x, spf)) {
    let term = 1, pw = 1;
    for (let i = 0; i < c; i++) { pw *= p; term += pw; }
    s *= term; // (1 + p + ... + p^c) for each prime
  }
  return s;
};
```

Run: `sumOfDivisors(12, spf) → 28` (that is $1 + 2 + 3 + 4 + 6 + 12$); `sumOfDivisors(1, spf) → 1`;
`sumOfDivisors(97, spf) → 98`.

Answer: $360 = 2^3 \cdot 3^2 \cdot 5$; divisor sum of 12 is 28

TRAP

$v \neq d$ is safe **only** because d divides v exactly at that moment — this is one of the few places in JavaScript where $/$ on integers is not a bug. Move that line anywhere else in the loop and you get a fraction, $v \% p$ stops being 0, and the inner while spins forever. When in doubt, $v = \text{Math.floor}(v / d)$.

Example 21 [Agoda · pattern]

Count the total number of 1 bits across **every** number from 0 to n .

Constraints: $n \leq 10^{18}$. **Target:** $O(63)$. **Edge cases:** $n = 0$; $n = 10^9$ (the answer exceeds 32 bits).

SOLUTION

Approach 1 — Popcount every number from 0 to n ($O(n)$, impossible at 10^{18})

```
setBitsBrute JS

const popcount32 = (x) => { let v = x >>> 0, c = 0; while (v) { v &= v - 1; c++; }
return c; };

const setBitsBrute = (n) => {
  let t = 0;
  for (let i = 0; i <= n; i++) t += popcount32(i);
  return t;
};
```

Time $O(n)$ **Space** $O(1)$

Printed 0, 4, 12 and 17 for $n = 0, 3, 7, 10$ — the same as the fast version. At $n = 10^{18}$ it would run for centuries.

Approach 2 — Count each bit column separately ($O(63)$, optimal)

Looping and calling popcount is $O(n)$ — impossible at 10^{18} . Count **per bit position** instead.

Look at bit b across $0, 1, 2, 3, \dots$. It follows a perfectly regular pattern: 2^b zeros, then 2^b ones, then 2^b zeros, and so on. The block length is 2^{b+1} .

So among the first $n + 1$ numbers:

- complete blocks: $\lfloor (n + 1) / 2^{b+1} \rfloor$, each contributing 2^b ones;
- a leftover of $(n + 1) \bmod 2^{b+1}$ numbers, of which the ones beyond the first 2^b are set: $\max(0, \text{rest} - 2^b)$.

Both n and the **answer** pass 2^{53} , and the shifts go up to bit 62, so this is BigInt end to end.

setBitsUpTo — count column by column

JS

```
const setBitsUpTo = (n) => {
  const N = BigInt(n) + 1n;
  let total = 0n;
  for (let b = 0n; b < 63n; b++) {
    const half = 1n << b, block = half * 2n;
    const full = N / block * half;
    const rest = N % block;
    total += full + (rest > half ? rest - half : 0n); // BigInt has no Math.max
  }
  return total;
};
```

Time $O(63)$ **Space** $O(1)$

Trace for $n = 10$, so 11 numbers $0 \dots 10$

bit b	block	full blocks	from full	rest	from rest
0	2	5	5	1	$\max(0, 1 - 1) = 0$
1	4	2	4	3	$\max(0, 3 - 2) = 1$
2	8	1	4	3	$\max(0, 3 - 4) = 0$
3	16	0	0	11	$\max(0, 11 - 8) = 3$

Total = $5 + 0 + 4 + 1 + 4 + 0 + 0 + 3 = 17$.

A brute-force loop over $0 \dots 10$ was run alongside and also printed 17, and the two versions were compared for every n from 0 to 2000 — the run printed PASS. Other runs: $n = 0 \rightarrow 0n$; $n = 3 \rightarrow 4n$; $n = 7 \rightarrow 12n$; $n = 10^9 \rightarrow 14846928141n$; $n = 10^{18} \rightarrow 29761222783429247000n$.

Answer: 17 for $n = 10$; 14846928141 for $n = 10^9$

TRAP

`Math.max` does not accept `BigInt` — `Math.max(0n, 1n)` throws `TypeError: Cannot convert a BigInt value to a number`. Nor can you mix the two in arithmetic: `1n + 1` throws as well. Inside a `BigInt` routine every literal needs its `n`, and a maximum is written as a ternary. That is why the line above reads `rest > half ? rest - half : 0n`.

Example 22 [DBS · pattern]

Add up the XOR of every subset of an array (including the empty subset, whose XOR is 0).

Constraints: $n \leq 20$ for a brute-force check, $n \leq 10^5$ for the real answer. **Target:** $O(n)$. **Edge cases:** empty array; a single element.

SOLUTION

Approach 1 — Walk all 2^n subsets ($O(2^n \cdot n)$, dies past $n = 25$)

subsetXorBrute

JS

```
const subsetXorBrute = (a) => {
  const n = a.length;
  let s = 0n;
  for (let m = 0; m < (1 << n); m++) {
    let x = 0;
```

```

    for (let i = 0; i < n; i++) if ((m >> i) & 1) x ^= a[i];
    s += BigInt(x);
  }
  return s;
};

```

Time $O(2^n \cdot n)$ **Space** $O(1)$

Printed $6n$, $5n$, $0n$ and $56n$ on the four tests — exactly what the formula gives. That agreement is how I checked the formula.

Approach 2 — Count per bit position $(O(n), \text{optimal})$

Count **per bit position** again, and one fact does all the work.

Fix a bit b . If **no** element has bit b set, no subset XOR has it either — contributes

0. If **at least one** element has it, then exactly **half** of all 2^n subsets have an odd number of such elements, so exactly 2^{n-1} subsets have bit b set in their XOR.

Why exactly half: pick one element e that has bit b . Pair every subset S with S -with- e -toggled. The two partners differ in bit b of their XOR, so exactly one of each pair has it on. The pairing covers all 2^n subsets.

So the answer is (OR of every element) $\times 2^{n-1}$.

subsetXorSum — one line after the insight

JS

```

const subsetXorSum = (a) => {
  const n = a.length;
  if (n === 0) return 0n;
  let orAll = 0;
  for (const x of a) orAll |= x;
  return BigInt(orAll) * (1n << BigInt(n - 1)); // each bit is 1 in half the
subsets
};

```

Time $O(n)$ **Space** $O(1)$

The answer is astronomically large, so it is a BigInt by necessity, not by taste.

Check on $\{1, 3\}$

Subset	XOR
$\{\}$	0
$\{1\}$	1
$\{3\}$	3
$\{1, 3\}$	2

Total = 6. Formula: OR = 3, $2^{2-1} = 2$, so $3 \times 2 = 6$. Match.

Runs: $\{1, 3\} \rightarrow 6n$; $\{5\} \rightarrow 5n$; empty $\rightarrow 0n$. A full brute force over all 16 subsets of $\{1, 3, 5, 7\}$ printed $56n$, and the formula printed $56n$ too. For the 100 values 1...100 the formula printed 80495813114492566995040653541376n — a 32-digit number, which is the whole reason this function returns a BigInt.

Answer: 6; and 56 for $\{1, 3, 5, 7\}$

TRAP

$1 \ll (n - 1)$ on plain numbers is the trap this problem is built out of. At $n = 32$ it gives 1, at $n = 33$ it gives 2, and at $n = 100$ it gives 8 — all with no error. The moment an answer is 2^{n-1} for anything but a toy n , it is a BigInt.

Example 23 [SCB · pattern]

How many bit flips turn a into b ?

Constraints: 32-bit unsigned. **Target:** $O(\text{popcount})$. **Edge cases:** $a = b$; every bit different.

SOLUTION

$a \oplus b$ has a 1 exactly where a and b disagree. Count those 1s.

flipsNeeded

JS

```
const flipsNeeded = (a, b) => {
  let d = (a ^ b) >>> 0, c = 0;
  while (d) { d &= d - 1; c++; }
  return c;
};
```

10 is 01010, 20 is 10100, so $10 \oplus 20$ is 11110 — four bits differ.

Run: `flipsNeeded(10,20) → 4`; `flipsNeeded(7,7) → 0`; `flipsNeeded(0, 4294967295) → 32`.

Answer: 4, 0, 32

TRAP

The `>>> 0` is what makes the third test work. $0 \wedge 4294967295$ is -1 as a signed 32-bit value, and `while (d)` on -1 with `d &= d - 1` still terminates — but any code that later compares `d` against a positive bound, or prints it, sees -1 . Convert once, at the point of the XOR.

Example 24 [Razer · pattern]

Produce the n -bit **Gray code**: a list of all 2^n numbers where each number differs from the previous one in exactly one bit, starting at 0.

Constraints: $n \leq 20$. **Target:** $O(2^n)$. **Edge cases:** $n = 0$ (one entry, 0); $n = 1$.

SOLUTION

Approach 1 — Build it by reflecting the previous list ($O(2^n)$ — correct, and it explains WHY the code works)

Take the $(n - 1)$ -bit list, write it out, then write it again **backwards** with a 1 added in front. Neighbours inside each half already differ by one bit, and the two middle entries are the same number with only the new top bit different.

grayReflect

JS

```
const grayReflect = (n) => {
  const g = [0];
  for (let b = 0; b < n; b++)
    for (let i = g.length - 1; i >= 0; i--) g.push(g[i] | (1 << b));
  return g;
};
```

Time $O(2^n)$ **Space** $O(2^n)$

Printed 0 1 3 2 6 7 5 4 for $n = 3$ — bit for bit the same list the one-liner produces. The two were compared for every n from 0 to 12 and the run printed PASS.

TRAP

The inner loop reads `g.length` **once**, into `i`, and then only pushes. If you write `for (let i = 0; i < g.length; i++) g.push(...)` the array grows while you walk it and the loop never ends. Snapshotting the length before a loop that appends is a JavaScript reflex worth building.

Approach 2 — The closed form ($O(1)$ per entry, optimal)

There is a closed form, and it is one line: the i -th Gray code is $i \oplus (i \gg 1)$.

Why it works. Going from i to $i + 1$ flips a suffix of bits: a run of trailing 1s becomes 0s and the bit above becomes 1. XORing with the shifted copy cancels that whole run and leaves exactly one changed bit.

grayCode

JS

```
const grayCode = (n) => Array.from({ length: 1 << n }, (_, i) => i ^ (i >> 1));
```

i	i binary	i >> 1	XOR	value
0	000	000	000	0
1	001	000	001	1
2	010	001	011	3
3	011	001	010	2
4	100	010	110	6
5	101	010	111	7
6	110	011	101	5
7	111	011	100	4

Run output: 0 1 3 2 6 7 5 4. Check the neighbours: $0 \rightarrow 1$ (bit 0), $1 \rightarrow 3$ (bit 1), $3 \rightarrow 2$ (bit 0), $2 \rightarrow 6$ (bit 2), and so on — always exactly one bit. `grayCode(0)` prints 0 and `grayCode(1)` prints 1.

Answer: 0 1 3 2 6 7 5 4

Example 25 [LINE MAN · pattern]

Compute Euler's totient $\varphi(n)$: how many numbers in $1 \dots n$ share no factor with n .

Constraints: $n \leq 10^{12}$. **Target:** $O(\sqrt{n})$. **Edge cases:** $n = 1$ (answer 1); n prime (answer $n - 1$).

SOLUTION

Approach 1 — Test gcd with every number up to n ($O(n \log n)$, hopeless at 10^{12})

phiBrute

JS

```
const gcd = (a, b) => { while (b) { [a, b] = [b, a % b]; } return a; };

const phiBrute = (n) => {
  let c = 0;
  for (let i = 1; i <= n; i++) if (gcd(i, n) === 1) c++;
  return c;
};
```

Time $O(n \log n)$ **Space** $O(1)$

Printed 1, 4, 96 and 96 for $n = 1, 10, 97, 360$ — the same as the formula. Useful only as a checker on small n , which is exactly how the formula below was verified for every n from 1 to 3000.

Approach 2 — Use the product formula over distinct prime factors ($O(\sqrt{n})$, optimal)

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$$

The product runs over **distinct** prime factors only, so once you divide a prime out completely you never touch it again. Written as integer arithmetic, $r -= r / p$ is exactly $r \times (1 - 1/p)$ with no rounding error, because p divides r at that moment — the one situation where JavaScript's $/$ on integers gives an integer.

eulerPhi

JS

```
const eulerPhi = (n) => {
  let r = n, v = n;
  for (let p = 2; p * p <= v; p++)
    if (v % p === 0) {
      while (v % p === 0) v /= p;
      r -= r / p; // exact: p divides r right now
    }
  if (v > 1) r -= r / v; // one large prime factor left over
  return r;
};
```

Time $O(\sqrt{n})$ **Space** $O(1)$

The trailing if catches one large prime factor left above $\sqrt{n}$. At $n = 10^{12}$ the loop runs 10^6 times and every value stays inside the exact-integer range.

n	Factors	φ
1	none	1
10	$2 \cdot 5$	$10 \times \frac{1}{2} \times \frac{4}{5} = 4$
97	prime	96
360	$2^3 \cdot 3^2 \cdot 5$	$360 \times \frac{1}{2} \times \frac{2}{3} \times \frac{4}{5} = 96$

Run output: 1 4 96 96. `eulerPhi(1000000000000)` prints 400000000000, and `Number.isInteger` on it prints `true` — no fraction ever crept in.

Answer: 1, 4, 96, 96

φ is the exponent in Euler's theorem: $a^{\varphi(n)} \equiv 1 \pmod{n}$ whenever $\gcd(a, n) = 1$. That is how you reduce a huge exponent modulo a **composite** number — Fermat's $p - 1$ is just the special case where n is prime.

Tier 3 — insight, then the follow-up

TIER 3

Google · Amazon · Microsoft · Goldman Sachs · D. E. Shaw · Adobe

Example 26 [Amazon · pattern]

Every element of an array appears **three** times except one, which appears once. Find it.

Constraints: $n \leq 10^6$, values fit in 32 bits. **Target:** $O(n)$ time, $O(1)$ space. **Edge cases:** a one-element array; the lonely value is 0.

Follow-up the interviewer asks next: “now every other element appears k times, for a general k .”

SOLUTION

XOR alone fails: $x \oplus x \oplus x = x$, so triples do not vanish.

Approach 1 — Count bits column by column, modulo 3 ($O(32n)$, easy to explain, usually accepted)

Look at bit position b across the whole array. Every value that appears three times contributes 0 or 3 to that column. So the column total modulo 3 is exactly the lonely value's bit.

singleOfThreeCount

JS

```
const singleOfThreeCount = (a) => {
  let r = 0;
  for (let b = 0; b < 32; b++) {
    let c = 0;
    for (const x of a) c += (x >> b) & 1;
    if (c % 3) r |= (1 << b);
  }
  return r;
};
```

Time $O(32n)$ **Space** $O(1)$

Approach 2 — Two variables acting as a base-3 counter ($O(n)$ with one pass, optimal)

Keep two masks. ones holds the bits that have been seen a number of times $\equiv 1 \pmod{3}$; twos holds the bits seen $\equiv 2 \pmod{3}$. A bit seen a third time must disappear from both.

singleOfThreeState — a 32-lane mod-3 counter

JS

```
const singleOfThreeState = (a) => {
  let ones = 0, twos = 0;
  for (const x of a) {
    ones = (ones ^ x) & ~twos;
    twos = (twos ^ x) & ~ones; // uses the freshly updated ones
  }
  return ones;
};
```

Time $O(n)$ **Space** $O(1)$

Two integers do 32 independent mod-3 counters in parallel. The order of the two lines matters: twos uses the freshly updated ones.

Trace of one bit column, values 6, 1, 6, 6, 2, 1, 1

Follow bit 1 (value 2). The elements with bit 1 set are 6, 6, 6, and 2 — four of them. $4 \bmod 3 = 1$, so the answer has bit 1 set. Follow bit 0: set in 1, 1, 1 — three of them, $3 \bmod 3 = 0$, so the answer has bit 0 clear. Follow bit 2: set in 6, 6, 6 — again 0. Answer $010 = 2$.

Both versions were run on this array and both printed 2. `singleOfThreeCount([9])` and the state version both printed 9; `[0,0,0,13]` gave 13 from both; and `[-7,5,5,5]` gave -7 from both.

Answer: 2

Follow-up: every other element appears k times

The counting version generalises immediately — replace $\% 3$ with $\% k$. Cost stays $O(32n)$ and the code is one parameter different; the run on $[4, 4, 4, 4, 4, 11]$ with $k = 5$ printed 11. The two-variable trick does **not** generalise cleanly: you would need $\lceil \log_2 k \rceil$ masks and a hand-derived update rule per k . In an interview, say exactly that: the mod- k column count is the right general answer, and the two-mask version is a $k = 3$ special case worth knowing because it is asked so often.

TRAP

if $(c \% 3) \text{ then } r \mid= (1 \ll b)$ runs with $b = 31$, where $1 \ll 31$ is -2147483648 — so r comes back **negative**. In this problem that is exactly right: a negative lonely value in two's complement has bit 31 set, and the run on $[-7, 5, 5, 5]$ returns -7 . Do not “fix” it. But remember that this is the one place in the chapter where $1 \ll 31$ is wanted, and everywhere else it is a bug.

Example 27 [Google · pattern]

Given an array, find the largest value of $a_i \oplus a_j$ over all pairs.

Constraints: $n \leq 10^5$, values under 2^{31} . **Target:** $O(32n)$. **Edge cases:** a one-element array (answer 0); all values equal.

Follow-up the interviewer asks next: “now answer queries — for a given x , what is the largest $x \oplus a_i$?”

SOLUTION

Approach 1 — Try every pair ($O(n^2)$, dies at $n = 10^5$)

maxXorBrute

JS

```
const maxXorBrute = (a) => {
  let best = 0;
  for (let i = 0; i < a.length; i++)
    for (let j = i + 1; j < a.length; j++)
      best = Math.max(best, a[i] ^ a[j]);
  return best;
};
```

Time $O(n^2)$ **Space** $O(1)$

Approach 2 — A binary trie: greedily take the opposite bit at every level ($O(32n)$, optimal)

Here is the unlock. XOR is decided bit by bit from the **top**: a single 1 at bit 30 beats every possible combination of the 30 bits below it. So to maximise $x \oplus y$, walk from the highest bit down and, at each level, go to the child with the **opposite** bit if that child exists. That is a plain greedy walk on a binary trie of the numbers.

The trie is stored as a flat array of $[\text{child0}, \text{child1}]$ pairs rather than as linked objects — one array, no allocation per node beyond the pair, and node ids are plain integers.

XorTrie — insert, and the greedy best-partner walk

JS

```
class XorTrie {
  constructor(bits = 31) { this.bits = bits; this.ch = [[-1, -1]]; }

  insert(x) {
    let cur = 0;
    for (let b = this.bits - 1; b >= 0; b--) {
      const d = (x >> b) & 1;
      if (this.ch[cur][d] === -1) {
        this.ch.push([-1, -1]);
        this.ch[cur][d] = this.ch.length - 1;
      }
    }
  }
}
```

```

    }
    cur = this.ch[cur][d];
  }
}

best(x) {
    // largest x ^ (something already inserted)
    let cur = 0, res = 0;
    for (let b = this.bits - 1; b >= 0; b--) {
        const d = (x >> b) & 1;
        if (this.ch[cur][1 - d] !== -1) { res |= 1 << b; cur = this.ch[cur][1 - d]; }
        else cur = this.ch[cur][d];
    }
    return res;
}

const maxXorTrie = (a, bits = 31) => {
    if (a.length < 2) return 0;
    const t = new XorTrie(bits);
    t.insert(a[0]);
    let best = 0;
    for (let i = 1; i < a.length; i++) {
        best = Math.max(best, t.best(a[i]));
        t.insert(a[i]);
    }
    return best;
};

```

Time $O(32n)$ **Space** $O(32n)$

Every insert and every query walks exactly 31 levels, and the walk is iterative — no recursion, so no stack depth to worry about.

Why the greedy walk is right

At the top bit, any partner with the opposite bit gives $x \oplus y$ a 1 there, worth 2^{30} . Every partner **without** the opposite bit gives 0 there, and even if all 30 lower bits come out as 1 the total is $2^{30} - 1 < 2^{30}$. So the top bit dominates completely, and the same argument repeats one level down inside the chosen subtree.

Trace on {3, 10, 5, 25, 2, 8}, showing the winning pair

The answer is $5 \oplus 25$. In 5 bits: $5 = 00101$, $25 = 11001$, $XOR = 11100 = 28$. When 25 arrives, the walk from the top asks for a partner starting 0 (25 starts 1), finds the subtree holding 3, 10, 5, 2, keeps steering to opposite bits, and lands on 5.

Runs: brute and trie both printed 28. A single element gave 0 from both; [0, 0] gave 0 from both. A randomised cross-check over 200 arrays of up to 12 random values printed PASS — every trie answer equalled the brute-force answer.

Answer: 28, from the pair (5, 25)

Follow-up: online queries

Nothing changes. Build the trie once over the array, then every query x is one `t.best(x)` call in $O(32)$. If values may also be **removed**, store a counter at each trie node and treat “count = 0” as “child does not exist”; insert increments along the path and remove decrements.

TRAP

bits = 31 covers values below 2^{31} , and the deepest res |= 1 << b is 1 << 30 — still positive. Push it to bits = 32 and 1 << 31 makes res negative, so Math.max compares the wrong way and the answer collapses. For values up to 10^{18} the whole trie must move to BigInt: (x >> BigInt(b)) & 1n for the descent and res |= 1n << BigInt(b) for the answer, with 63 levels instead of 31.

Example 28 [D. E. Shaw · pattern]

Count the subarrays whose XOR equals exactly k .

Constraints: $n \leq 10^5$, values under 2^{20} . The count can reach 5×10^9 . **Target:** $O(n)$. **Edge cases:** empty array; $k = 0$ with an array of zeros.

Follow-up the interviewer asks next: “now count subarrays whose XOR is **less than** k .”

SOLUTION

Approach 1 — Every subarray, extending the XOR as you go $(O(n^2))$

xorSubBrute

JS

```
const xorSubBrute = (a, k) => {
  let c = 0;
  for (let i = 0; i < a.length; i++) {
    let x = 0;
    for (let j = i; j < a.length; j++) { x ^= a[j]; if (x === k) c++; }
  }
  return c;
};
```

Time $O(n^2)$ **Space** $O(1)$

Printed 4, 2, 0 and 6 on the four tests — identical to the hash-map version. 10^{10} operations at $n = 10^5$.

Approach 2 — Prefix XOR plus a hash map $(O(n), \text{optimal})$

Define the prefix XOR $P_i = a_0 \oplus a_1 \oplus \dots \oplus a_{i-1}$, with $P_0 = 0$. Then

$$\text{XOR of the subarray } a_i \dots a_{j-1} = P_i \oplus P_j$$

So “this subarray XORs to k ” becomes $P_i \oplus P_j = k$, which rearranges to $P_i = P_j \oplus k$. Sweep j from left to right and ask a hash map how many earlier prefixes equal $P_j \oplus k$.

countXorSubarrays — prefix XOR plus a Map

JS

```
const countXorSubarrays = (a, k) => {
  const seen = new Map([[0, 1]]); // the empty prefix, seeded
  let pref = 0, ans = 0;
  for (const x of a) {
    pref ^= x;
    ans += seen.get(pref ^ k) ?? 0;
    seen.set(pref, (seen.get(pref) ?? 0) + 1);
  }
  return ans;
};
```

Time $O(n)$ average **Space** $O(n)$

seen starts with $[0, 1]$ for the empty prefix, which is what lets a subarray starting at index 0 be counted.

Trace on 4, 2, 2, 6, 4 **with** $k = 6$

x	pref	Looking for $\text{pref} \oplus k$	Found	Running count
—	0	—	seed seen with [0, 1]	0
4	4	$4 \oplus 6 = 2$	0 times	0
2	6	$6 \oplus 6 = 0$	1 time	1
2	4	$4 \oplus 6 = 2$	0 times	1
6	2	$2 \oplus 6 = 4$	2 times	3
4	6	$6 \oplus 6 = 0$	1 time	4

Run output: 4. Other runs: [5, 6, 7, 8, 9] with $k = 5$ gave 2; an empty array gave 0; [0, 0, 0] with $k = 0$ gave 6 — which is right, since all $3 + 2 + 1 = 6$ subarrays XOR to 0.

Answer: 4 subarrays

Follow-up: XOR strictly less than k

The hash map cannot answer a range question. Swap it for the binary trie of Example 27, storing a **count** at every node. To count partners P_i with $P_j \oplus P_i < k$, walk down the trie comparing against k 's bits: whenever k has a 1 at the current bit, the entire subtree that produces a 0 there is **all** strictly smaller — add its stored count and move into the other subtree. Cost becomes $O(32n)$.

TRAP

Use a Map, not a plain object. An object stringifies every key, so `seen[4]` and `seen["4"]` are the same slot — harmless here, but the moment a key can be negative or a `BigInt` you get silent collisions, and `Object.keys` hands them back as strings. Map keeps the number **as a number**, keeps insertion order, and is faster under heavy insert-and-lookup traffic, which is exactly this loop.

The answer itself can reach about $n^2/2 = 5 \times 10^9$. That still fits a double exactly (it is far below $2^{53} - 1$), so ans may stay a plain number here — but check that bound before you assume it in another problem.

Example 29 [Microsoft · pattern]

Given positive integers and a target $T \leq 10^5$, decide whether **some subset** sums to exactly T .

Constraints: $n \leq 10^3$, $T \leq 10^5$. **Target:** fast enough that $nT = 10^8$ passes. **Edge cases:** $T = 0$ (the empty subset works); one element equal to T .

Follow-up the interviewer asks next: “now also print the subset.”

SOLUTION

The classic DP is a boolean array `can[0..T]`, updated once per element: `can[s] |= can[s - x]`. That is 10^8 boolean operations — borderline.

The unlock: those booleans are **bits**. C++ has a `bitset` type; JavaScript has something better suited to it than you might expect — a `BigInt` is an arbitrary-width bit vector, and `|`, `<<` and `&` on it are implemented as word-at-a-time machine loops. So the entire inner DP collapses into one line, `can |= can << BigInt(x)`, and the engine does 64 of the updates per machine word for you.

The `& mask` keeps the number from growing past T bits; without it the `BigInt` doubles in width on every element and the whole thing becomes quadratic.

subsetReaches — the DP as a single shift-OR on a `BigInt`

JS

```
const subsetReaches = (a, target) => {
  const T = BigInt(target);
  const mask = (1n << (T + 1n)) - 1n;           // keep only sums 0..T
  let can = 1n;                                  // bit 0 set: sum 0 is reachable
```

```

for (const x of a) {
  if (x < 0 || x > target) continue;
  can = (can | (can << BigInt(x))) & mask;
}
return ((can >> T) & 1n) === 1n;
};

```

Time $O(nT/64)$ **Space** $O(T/8)$ bytes

Measured: $n = 1000$, $T = 10^5$ finishes in about 6 ms. The mask is what keeps it there.

Trace on {3, 4, 2} with the BigInt printed as reachable sums

start	{0}
after 3	{0, 3}
after 4	{0, 3, 4, 7}
after 2	{0, 2, 3, 4, 5, 6, 7, 9}

Each step is the old set plus a copy of the old set shifted **up** by x . Those four lines are the literal run output.

Runs: [3, 34, 4, 12, 5, 2] with target 9 $\rightarrow$ true (that is $3 + 4 + 2$); target 30 $\rightarrow$ false; an empty array with target 0 $\rightarrow$ true; [7] with target 7 $\rightarrow$ true. A randomised cross-check against a Set-based brute force over 300 small cases printed PASS.

Answer: 9 reachable, 30 not reachable

Follow-up: print the subset

A bit vector gives you the answer but not the path. Keep a second structure: for each element index i , store the BigInt **after** processing element i . Then walk backwards from T : if bit s is already set in `after[i-1]` the element was not needed, so move to $i - 1$; otherwise record element i and move to $(i - 1, s - a_i)$. That is $O(nT/8)$ bytes of memory — acceptable at $n = 10^3$, $T = 10^5$ only if you are careful, so in an interview offer the ordinary $O(nT)$ boolean DP over a Uint8Array with a parent array as the print-the-subset version.

TRAP

`can << BigInt(x)` — the shift amount must be a BigInt too. `can << 3` throws `TypeError: Cannot mix BigInt and other types`. And `1n << (T + 1n)` with $T = 10^5$ is a 100001-bit number: perfectly legal, about 12 KB, but build it **once** outside the loop. Rebuilding the mask each iteration is the difference between 6 ms and several seconds.

Example 30 [Goldman Sachs · pattern]

Solve $ax \equiv 1 \pmod{m}$ for x , where m is **not necessarily prime**. Report that no solution exists when $\gcd(a, m) \neq 1$.

Constraints: $a, m \leq 10^{18}$. **Target:** $O(\log m)$. **Edge cases:** $\gcd(a, m) > 1$; a larger than m ; a negative.

Follow-up the interviewer asks next: “use it to solve a pair of simultaneous congruences.”

SOLUTION

Fermat is unavailable — it needs a prime modulus. Use the **extended** Euclid algorithm, which returns not just $g = \gcd(a, b)$ but also integers x, y with

$$ax + by = g$$

When $g = 1$ this reads $ax + my = 1$, so $ax \equiv 1 \pmod{m}$, and x is the inverse. Reduce x into $[0, m)$ at the end because it may come back negative.

With m up to 10^{18} everything is `BigInt`. Two `BigInt` behaviours make this work without any extra code: / truncates toward zero (integer division), and % keeps the sign of the left operand — both exactly as C and C++ do.

extGcd and invAny

JS

```
const extGcd = (a, b) => {
  if (b === 0n) return [a, 1n, 0n];
  const [g, x1, y1] = extGcd(b, a % b); // destructuring instead of out-params
  return [g, y1, x1 - (a / b) * y1];
};

const invAny = (a, m) => { // -1n when no inverse exists
  const M = BigInt(m);
  const A = ((BigInt(a) % M) + M) % M; // normalise a negative a
  const [g, x] = extGcd(A, M);
  if (g !== 1n) return -1n;
  return ((x % M) + M) % M;
};
```

Time $O(\log m)$ **Space** $O(\log m)$ recursion

About 87 levels at $m = 10^{18}$, far under JavaScript's $\sim 10^4$ stack limit. Same step count as plain Euclid; only the bookkeeping is extra.

Trace of extGcd(3n, 10n)

Going down: $(3, 10) \rightarrow (10, 3) \rightarrow (3, 1) \rightarrow (1, 0)$.

Level	a, b	returns (g, x, y)	Check $ax + by$
4	1, 0	(1, 1, 0)	$1(1) + 0(0) = 1$
3	3, 1	(1, 0, 1)	$3(0) + 1(1) = 1$
2	10, 3	(1, 1, -3)	$10(1) + 3(-3) = 1$
1	3, 10	(1, -3, 1)	$3(-3) + 10(1) = 1$

Those four rows are the run output, level by level. So $x = -3$, and $-3 \bmod 10 = 7$.

Runs: $\text{invAny}(3, 10) \rightarrow 7n$, and $3n * 7n \% 10n$ printed $1n$. $\text{invAny}(4, 10) \rightarrow -1n$ (since $\text{gcd}(4, 10) = 2$). $\text{invAny}(-7, 10) \rightarrow 7n$, so the normalisation handles a negative a . $\text{invAny}(3, 1000000007) \rightarrow 333333336n$, the same value Fermat gave in Example 19. At the top of the range, $\text{invAny}(123456789123456789n, 10000000000000000003n) \rightarrow 244581051350687566n$, and multiplying the two back together modulo m printed $1n$.

Answer: $3^{-1} \equiv 7 \pmod{10}$; **no inverse of 4 modulo 10**

Follow-up: two simultaneous congruences

To solve $x \equiv r_1 \pmod{m_1}$ and $x \equiv r_2 \pmod{m_2}$ with $\text{gcd}(m_1, m_2) = 1$: write $x = r_1 + m_1 t$, substitute into the second congruence to get $m_1 t \equiv r_2 - r_1 \pmod{m_2}$, and multiply both sides by $\text{invAny}(m_1, m_2)$. That gives t , hence x modulo $m_1 m_2$. This is the Chinese Remainder Theorem, and `invAny` is the only new ingredient it needs.

TRAP

$((a \% m) + m) \% m$ is not optional. `BigInt %` keeps the sign of the left operand, so $-7n \% 10n$ is $-3n$, not $7n$ — the same rule C++ uses and the same bug it causes. Every subtraction and every input normalisation under a modulus in this chapter carries that $+$ m .

The other half of the trap is mixing types. `invAny(3, 10)` takes plain numbers and converts them; `invAny(3n, 10)` also works; but writing `x % m` where one side is a number and the other a `BigInt` throws immediately. Convert at the boundary, then stay in one world.

Example 31 [Adobe · pattern]

Count the integers in $[1, N]$ whose digit sum is divisible by k .

Constraints: $N \leq 10^{18}$, $k \leq 100$. **Target:** $O(d \times 9d \times 2 \times 10)$ where d is the digit count. **Edge cases:** N with a leading digit larger than the rest; $k = 1$ (every number qualifies).

Follow-up the interviewer asks next: “count them for every k from 1 to 100.”

SOLUTION

Approach 1 — Loop over every number and add its digits ($O(N \cdot d)$, fine to $N = 10^7$, impossible at 10^{18})

digitSumBrute

JS

```
const digitSumBrute = (N, k) => {
  let c = 0;
  for (let i = 1; i <= N; i++) {
    let s = 0, t = i;
    while (t) { s += t % 10; t = Math.floor(t / 10); }
    if (s % k === 0) c++;
  }
  return c;
};
```

Time $O(Nd)$ **Space** $O(1)$

Printed 6, 9 and 19 for (20,3), (9,1) and (100,5) — the same as the DP. It was also run against the DP for every N from 1 to 600 and every k from 1 to 12; the run printed PASS.

Approach 2 — Digit DP ($O(d \cdot 9d \cdot 2 \cdot 10)$, optimal)

Looping to 10^{18} is impossible. **Digit DP** builds the number one digit at a time, carrying only what matters: the position, the digit sum so far (capped at $9d$), and whether we are still “tight” against N ’s prefix.

“Tight” means every digit chosen so far equalled N ’s digit, so the next digit is capped at N ’s digit. Once you choose anything smaller, you are free and the cap becomes 9.

N can be 10^{18} , so it arrives as a `BigInt` or a string — never as a number literal. `String(N)` handles all three, and from then on the DP works on characters, so precision is not a question.

countDigitSum — digit DP

JS

```
const countDigitSum = (N, k) => {
  const s = String(N), d = s.length;
  const memo = new Map(); // fresh per call -- see the trap

  const dig = (pos, sum, tight) => {
    if (pos === d) return sum % k === 0 ? 1 : 0;
    const key = (pos * 200 + sum) * 2 + tight;
    if (memo.has(key)) return memo.get(key);
    const hi = tight ? s[pos] : 9;
    let tot = 0;
    for (let x = 0; x <= hi; x++)
      tot += dig(pos + 1, sum + x, tight && x === hi ? 1 : 0);
    memo.set(key, tot);
  };
  return dig(0, 0, 1);
};
```

```

    return tot;
};

return dig(0, 0, 1) - 1;           // subtract the number 0
};

```

Time $O(d \times 9d \times 2 \times 10)$ **Space** $O(d \times 9d \times 2)$

For $d = 19$ that is about 65 thousand states, each with a 10-way loop. Recursion is 19 deep, nowhere near JavaScript's limit.

Trace for $N = 20, k = 3$

$s = "20"$. Position 0 is tight, so the first digit may be 0, 1 or 2.

First digit	Still tight?	Second digit choices and which qualify
0	no	0..9; sums 0..9; divisible by 3: 0, 3, 6, 9 $\rightarrow$ 4 numbers
1	no	0..9; sums 1..10; divisible by 3: 2, 5, 8 $\rightarrow$ 3 numbers (12, 15, 18)
2	yes	0 only (N's last digit); sum 2, not divisible $\rightarrow$ 0 numbers

Total 7, minus 1 for the number 0 itself, gives 6: they are 3, 6, 9, 12, 15, 18.

Run output: 6 9 19 142857049 for $(N, k) = (20, 3), (9, 1), (100, 5), (10^9, 7)$. A brute-force loop up to 100 with $k = 5$ was run alongside and also printed 19.

Answer: 6 for $N = 20, k = 3$

Follow-up: every k from 1 to 100

Do not rerun the DP 100 times. Run it **once** with the state $(pos, sum, tight)$ and no k at all, producing $cnt[s]$ = how many numbers in $[1, N]$ have digit sum exactly s , for s up to $9 \times 19 = 171$. Then each k is one $O(171)$ sum over the multiples of k . One DP, 100 cheap answers.

TRAP

`const memo = new Map()` must be created **inside** `countDigitSum`, on every call, because the cached values belong to one particular N and k . Hoisting it to module scope “for speed” gives the previous query’s answer — the sort of bug that passes the sample and fails everything else. The run proves the version above is clean: back-to-back calls `countDigitSum(20, 3)` and `countDigitSum(20, 7)` print 6 and 2, not 6 and 6.

A second, arithmetic trap: the **count** can overflow the exact-integer range. At $N = 10^{18}$ with $k = 1$ every number qualifies, so the true answer is 10^{18} — well past $2^{53} - 1$, and the returned number is only approximately right. If the constraints allow a count above 9×10^{15} , `tot` must accumulate in `BigInt`.

Dry run — one problem, every single step

We trace `powMod(3, 13, 100)` completely. Every variable, after every turn.

Why 13? Because $13 = 8 + 4 + 1$, so $3^{13} = 3^8 \times 3^4 \times 3^1$. In binary $13 = 1101$ — read right to left, the bits that are 1 are positions 0, 2 and 3, which are the powers $3^1, 3^4$ and 3^8 .

Setup. $r = 1n \% 100n = 1n, b = 3n \% 100n = 3n, e = 13n$.

Turn	e	e in binary	e & 1n	r update	b update
1	13	1101	1	$r = 1 \times 3 \bmod 100 = 3$	$b = 3^2 = 9$
2	6	110	0	r stays 3	$b = 9^2 = 81$
3	3	11	1	$r = 3 \times 81 = 243 \bmod 100 = 43$	$b = 81^2 = 6561 \bmod 100 = 61$

4	1	1	1	$r = 43 \times 61 = 2623 \bmod 100 = 23$	$b = 61^2 = 3721 \bmod 100 = 21$
5	0	0	—	loop ends	—

Answer 23. The run prints exactly that.

Check it by hand. $3^{13} = 1594323$. And $1594323 \bmod 100 = 23$. Match.

The same numbers, read the other way

Look at which b values got multiplied into r :

Turn	b at that moment	Which power of 3?	Multiplied in?
1	3	3^1	yes — bit 0 of 13 is 1
2	9	3^2	no — bit 1 of 13 is 0
3	81	3^4	yes — bit 2 of 13 is 1
4	61	$3^8 \bmod 100$	yes — bit 3 of 13 is 1

So the algorithm literally computes $3^1 \times 3^4 \times 3^8 = 3^{13}$, one binary digit at a time. That is the whole idea, and it is why the loop runs $\log_2 b$ times instead of b times.

Trace of the edge cases

Call	What happens	Result
<code>powMod(3, 0, 7)</code>	e is 0n, the loop body never runs, r is still $1n \% 7n$	1
<code>powMod(5, 3, 1)</code>	$r = 1n \% 1n = 0n$, and 0 times anything stays 0	0
<code>powMod(2, 62, 1e9+7)</code>	62 turns' worth of squaring, largest intermediate about $(10^9)^2 \approx 10^{18}$ — exact only because it is a <code>BigInt</code>	145586002
<code>powMod(-2, 3, 1e9+7)</code>	b starts at $-2n$, the guard adds the modulus, so it is $(10^9 + 5)^3$	999999999

All four were run and printed exactly these values.

Watch the largest intermediate value. $b * b$ with b just under $10^9 + 7$ is about 10^{18} . In C++ that fits a 64-bit integer with room to spare; in JavaScript a **number** would have rounded it away eight digits earlier. This is the single reason every modular multiplication in this chapter is `BigInt`, and it is also the reason `BigInt` costs you nothing to reach for: the alternative is not “slower but correct”, it is “faster and wrong”.

Practice

PRACTICE · TIER 1 target: 12 min

- Given a number, return the index of its only set bit, or -1 if it does not have exactly one. Test with 1, 1024, 12, 0 and 2^{40} .
- Compute $0 \oplus 1 \oplus 2 \oplus \dots \oplus n$ in $O(1)$. Test with $n = 10$ against a loop.
- Decide whether two integers have opposite signs, using no comparison operator on the values themselves.
- Add two integers without using $+$, $-$, $*$ or $/$. It must work on negatives.
- Count the decimal digits of a possibly negative integer, including one past 2^{53} .

ANSWER KEY

- Exactly one bit means $x > 0n \ \&\& \ (x \ \& \ (x - 1n)) === 0n$. The index is then found by shifting the surviving bit down. 2^{40} is in the test set, so this cannot be a 32-bit function.

onlyBitIndex

JS

```
const onlyBitIndex = (x) => {
  const v = BigInt(x);
  if (v <= 0n || (v & (v - 1n)) !== 0n) return -1;
  let i = 0, w = v;
  while (w > 1n) { w >>= 1n; i++; }
  return i;
};
```

Run: $1 \rightarrow 0$; $1024 \rightarrow 10$; $12 \rightarrow -1$; $0 \rightarrow -1$; $2^{40} \rightarrow 40$.

Time $O(\log x)$ **Space** $O(1)$

2. XOR of $0 \dots n$ has period 4. Pair up $(4t, 4t + 1, 4t + 2, 4t + 3)$: their XOR is always 0, because $4t \oplus (4t + 1) = 1$ and $(4t + 2) \oplus (4t + 3) = 1$, and $1 \oplus 1 = 0$. So only the remainder matters — and in JavaScript the four cases are just an array lookup.

xorUpTo

JS

```
const xorUpTo = (n) => [n, 1, n + 1, 0][n % 4];
```

Run: $0, 1, 2, 3 \rightarrow 0$; $10 \rightarrow 11$, which a brute-force loop confirmed; $10^9 \rightarrow 1000000000$. The formula was compared against a running XOR for every n from 0 to 5000 and the run printed PASS.

SHORTCUT

To XOR a **range** $[l, r]$ use $\text{xorUpTo}(r) \oplus \text{xorUpTo}(l - 1)$ — the same prefix trick as prefix sums, because XOR is its own inverse. Run: $\text{xorUpTo}(9) \oplus \text{xorUpTo}(3)$ prints 1, and a loop over $4 \dots 9$ prints 1 too.

3. The sign lives in the top bit. XOR makes the top bit 1 exactly when the two signs differ, and JavaScript's bitwise operators produce a **signed** 32-bit result — so a negative result means exactly that.

oppositeSigns

JS

```
const oppositeSigns = (a, b) => (a ^ b) < 0;
```

Run: $(3, -4) \rightarrow \text{true}$; $(-3, -4) \rightarrow \text{false}$; $(0, 5) \rightarrow \text{false}$.

4. $a \oplus b$ is the sum with every carry ignored. $(a \& b) \ll 1$ is exactly the carries. Keep folding the carries back in until there are none left.

addNoPlus

JS

```
const addNoPlus = (a, b) => {
  let x = a, y = b;
  while (y !== 0) {
    const carry = (x & y) << 1;
    x = x ^ y;
    y = carry;
  }
  return x;
};
```

Run: $(13, 29) \rightarrow 42$; $(-7, 3) \rightarrow -4$; $(0, 0) \rightarrow 0$; $(-5, -6) \rightarrow -11$. Every pair (i, j) with i, j from -50 to 50 was checked against $i + j$; the run printed PASS.

TRAP

This is one of the rare places where JavaScript is **easier** than C++. Left-shifting a negative signed integer is undefined behaviour in C++, so the C++ answer has to launder everything through `unsigned int`. In JavaScript `<<` is **defined** to operate on a signed 32-bit two's-complement value, so the loop above is correct as written. The price is the ceiling: this function is exact only for inputs inside the 32-bit signed range. Feed it 2^{40} and it answers about the low 32 bits, silently.

5. Divide by 10 until nothing is left, and handle 0 as a special case.

digitCount

JS

```
const digitCount = (n) => {
  if (n === 0) return 1;
  let v = Math.abs(n), c = 0;
  while (v > 0) { c++; v = Math.floor(v / 10); }
  return c;
};
```

Run: $0 \rightarrow 1$; $7 \rightarrow 1$; $-1205 \rightarrow 4$.

SHORTCUT

For anything past $2^{53} - 1$ the arithmetic version cannot even receive the input correctly. Count the characters instead:

digitCountBig

JS

```
const digitCountBig = (n) => {
  const v = BigInt(n);
  return (v < 0n ? -v : v).toString().length;
};
```

Run: `digitCountBig(9223372036854775807n)` $\rightarrow$ 19; `digitCountBig(-1205)` $\rightarrow$ 4; `digitCountBig(0)` $\rightarrow$ 1.

PRACTICE · TIER 2 target: 15 min

- Build Pascal's triangle and read $\binom{n}{r}$ off it, exactly, with no modulus, for $n \leq 30$.
- Decide whether an integer up to 10^{18} is a perfect square, without `Math.sqrt` and without any floating point.
- Count the divisors of n in $O(\sqrt{n})$. Test with 10^{12} .
- Compute $(1 + 2 + \dots + n) \bmod (10^9 + 7)$ for n up to 10^{12} , using the modular inverse of 2.
- Compute the n -th Fibonacci number modulo $10^9 + 7$ for n up to 10^{18} .

ANSWER KEY

- $\binom{i}{j} = \binom{i-1}{j-1} + \binom{i-1}{j}$. No multiplication means no precision risk for moderate n ; $\binom{30}{15}$ is about 1.5×10^8 , which fits easily.

pascalC

JS

```
const pascalC = (n, r) => {
  if (r < 0 || r > n) return 0;
  const t = Array.from({ length: n + 1 }, () => new Array(n + 1).fill(0));
  for (let i = 0; i <= n; i++) {
    t[i][0] = 1;
    for (let j = 1; j <= i; j++) t[i][j] = t[i-1][j-1] + t[i-1][j];
  }
};
```

```
    }  
    return t[n][r];  
};
```

Run: $\binom{5}{2} \rightarrow 10$; $\binom{30}{15} \rightarrow 155117520$; $\binom{4}{7} \rightarrow 0$; $\binom{0}{0} \rightarrow 1$.

Time $O(n^2)$ **Space** $O(n^2)$
 Exact to n around 50 in doubles. Beyond that use BigInt cells, or the modular factorial method of Example 19.

Exact to n around 50 in doubles. Beyond that use BigInt cells, or the modular factorial method of Example 19.

TRAP

Building the 2D table is where this one goes wrong. `Array.from({length: n+1}, () => new Array(n+1).fill(0))` calls the factory once per row, so every row is its own array. Write it as `new Array(n+1).fill(new Array(n+1).fill(0))` and all $n + 1$ “rows” are the **same** array — `t[0][0] = 1` sets `t[1][0]` too. The run demonstrates it: aliasing two rows and writing to one prints 1 when reading the other. The same rule covers copying: `[...grid]` is a shallow copy, and a real 2D copy is `grid.map(row => [...row])`.

Building the 2D table is where this one goes wrong. `Array.from({length: n+1}, () => new Array(n+1).fill(0))` calls the factory once per row, so every row is its own array. Write it as `new Array(n+1).fill(new Array(n+1).fill(0))` and all $n + 1$ “rows” are the **same** array — `t[0][0] = 1` sets `t[1][0]` too. The run demonstrates it: aliasing two rows and writing to one prints 1 when reading the other. The same rule covers copying: `[...grid]` is a shallow copy, and a real 2D copy is `grid.map(row => [...row])`.

7. Binary search on the root, in `BigInt`. There is no overflow guard to write: a `BigInt` product is exact however large it gets, so `mid * mid` can simply be compared.

isPerfectSquare JS

JS

```
const isPerfectSquare = (n) => {
  const N = BigInt(n);
  if (N < 0n) return false;
  let lo = 0n, hi = 3037000499n; // floor(sqrt(2^63 - 1))
  while (lo <= hi) {
    const mid = (lo + hi) >> 1n;
    const sq = mid * mid;
    if (sq === N) return true;
    if (sq < N) lo = mid + 1n; else hi = mid - 1n;
  }
  return false;
};
```

Run: $0 \rightarrow \text{true}$; $1 \rightarrow \text{true}$; $144 \rightarrow \text{true}$; $145 \rightarrow \text{false}$; $-4 \rightarrow \text{false}$; $9999999999999999n \rightarrow \text{false}$; $999999998000000001n \rightarrow \text{true}$ (it is 999999999^2). Every k^2 and $k^2 + 1$ for k up to 3000 was checked; the run printed PASS.

TRAP

`Math.round(Math.sqrt(n)) ** 2 == n` is the version everybody writes, and it is wrong in **both** directions past 2^{53} . The run shows a false positive: 10000000000000000001 is not a square, but the double round trip says it is, because $10^{18} + 1$ is not even representable — it becomes 10^{18} on the way in. Never use floating point for an exact integer question, and never type a literal above 9007199254740991 without an `n` on the end.

`Math.round(Math.sqrt(n)) ** 2 == n` is the version everybody writes, and it is wrong in **both** directions past 2^{53} . The run shows a false positive: `1000000000000000001` is not a square, but the double round trip says it is, because $10^{18} + 1$ is not even representable — it becomes 10^{18} on the way in. Never use floating point for an exact integer question, and never type a literal above `9007199254740991` without an `n` on the end.

8. Divisors come in pairs $(d, \frac{n}{d})$. Walk d up to $\sqrt{n}$ and add 2 each time, except when $d = \frac{n}{d}$.

divisorCount JS

JS

```
const divisorCount = (n) => {
  if (n <= 0) return 0;
  let c = 0;
  for (let d = 1; d * d <= n; d++)
    if (n % d === 0) c += (d === n / d) ? 1 : 2;
}
```

```
    return c;
};
```

Run: $1 \rightarrow 1$; $36 \rightarrow 9$; $97 \rightarrow 2$; $10^{12} \rightarrow 169$. Check: $10^{12} = 2^{12} \cdot 5^{12}$, so $(12 + 1)(12 + 1) = 169$. Match. Note `d === n / d` is a safe comparison only because `d` divides `n` exactly at that point; everywhere else that expression would be a float compare.

9. $1 + \dots + n = n(n + 1)/2$. You may not divide under a modulus, so multiply by 2^{-1} . Reduce n and $n + 1$ **before** multiplying, and keep the whole chain in `BigInt` — the product of two reduced values is about 10^{18} .

sumUpToMod

JS

```
const MOD = 1000000007;

const sumUpToMod = (n) => {
  const M = BigInt(MOD), N = BigInt(n);
  const inv2 = BigInt(powMod(2, MOD - 2, MOD));
  return Number(N % M * ((N + 1n) % M) % M * inv2 % M);
};
```

Run: $n = 10 \rightarrow 55$, which matches $10 \times 11/2$. $n = 10^{12} \rightarrow 24496500$. $n = 0 \rightarrow 0$ and $n = 1 \rightarrow 1$.

10. Fast doubling. From $F(k)$ and $F(k + 1)$ you get the pair at $2k$ in two multiplications:

$$F(2k) = F(k) \cdot (2F(k + 1) - F(k)), \quad F(2k + 1) = F(k)^2 + F(k + 1)^2$$

So halving n each step gives $O(\log n)$. Returning a **pair** is natural in JavaScript — an array, destructured at the call site.

fib by fast doubling

JS

```
const M = 1000000007n;

const fibPair = (n) => { // returns [F(n), F(n+1)]
  if (n === 0n) return [0n, 1n];
  const [a, b] = fibPair(n >> 1n);
  const c = a * ((2n * b % M - a + M) % M) % M;
  const d = (a * a % M + b * b % M) % M;
  return (n & 1n) ? [d, (c + d) % M] : [c, d];
};

const fib = (n) => Number(fibPair(BigInt(n))[0]);
```

Run: $F(0) \rightarrow 0$; $F(1) \rightarrow 1$; $F(10) \rightarrow 55$; $F(90) \rightarrow 210345902$; $F(10^9) \rightarrow 21$. Every value from $F(0)$ to $F(500)$ was cross-checked against a plain `BigInt` iteration and the run printed `PASS`; the exact $F(90) = 2880067194370816120$ reduces to 210345902 , which the fast version also printed. Recursion is $\log_2 10^{18} \approx 60$ deep — nowhere near a stack problem.

TRAP

$2n * b \% M - a$ can go negative. The $+ M$ before the final $\% M$ is what keeps it in range. Subtraction under a modulus **always** needs that $+ M$, in every language.

And $(n \ \& \ 1n)$ is a `BigInt`: `0n` is falsy and `1n` is truthy, so the ternary works — but `n & 1` would throw, because you cannot mix a `BigInt` with a number.

PRACTICE · TIER 3 target: 18 min

11. n people stand in a circle and every **second** person is removed until one remains. Find the survivor's position (1-based) using only bit operations. Test $n = 41$.
12. Build a table of the set-bit count of every number from 0 to n in $O(n)$ total, with no popcount call.
13. Explain in your own words why the sieve's inner loop may start at $i \times i$ rather than $2i$, and what breaks if $n < i \times i$.
14. You must compute $\binom{n}{r} \bmod p$ where p is a **small** prime, say 13, and n is up to 10^{18} . Fermat's method needs $n < p$. What do you do?
15. Someone writes `const mask = 1 << k;` and their program works for $k = 20$ but produces garbage for $k = 35$. Explain exactly what happens at $k = 31$, at $k = 32$ and at $k = 35$.

ANSWER KEY

11. Write $n = 2^m + l$ where 2^m is the largest power of two not above n . The survivor is $2l + 1$. Reason: once l people have been removed, exactly 2^m remain and the count restarts at a known person; with a power-of-two crowd the person who starts the round always survives.

josephusStep2

JS

```
const josephusStep2 = (n) => {
  if (n <= 0) return -1;
  let high = 1;
  while (high * 2 <= n) high *= 2;
  return 2 * (n - high) + 1;
};
```

Run: $n = 1 \rightarrow 1$; $n = 5 \rightarrow 3$; $n = 8 \rightarrow 1$; $n = 41 \rightarrow 19$. Check $n = 5$: remove 2, 4, 1, 5 — the survivor is 3. Match. An explicit circle simulation was run for every n from 1 to 200 and agreed every time: the run printed PASS.

SHORTCUT

In bits: the answer is n with its highest bit moved to the **bottom**. 41 is 101001; drop the leading 1 to get 01001 and append a 1 to get 010011 = 19. Note the loop above uses `high *= 2` rather than `high <= 1` — at n near 2^{31} the shift would go negative and the loop would never end, while `*= 2` stays exact up to 2^{53} .

12. `dp[i] = dp[i >> 1] + (i & 1)`. Dropping the last bit gives a strictly smaller number whose answer is already known, and the last bit adds 0 or 1.

bitsTable

JS

```
const bitsTable = (n) => {
  const dp = new Int32Array(n + 1); // zero-filled, one word per entry
  for (let i = 1; i <= n; i++) dp[i] = dp[i >> 1] + (i & 1);
  return dp;
};
```

Run for $n = 10$: 0 1 1 2 1 2 2 3 1 2 2. Check $i = 7$: 111, three bits — the table says 3.

Time $O(n)$ **Space** $O(n)$

13. When i is prime and $j = i \times c$ with $c < i$, the number j has a prime factor c or smaller, so some earlier prime already marked it. Only from $i \times i$ upward is i possibly the **smallest** factor. If $n < i \times i$, the inner loop simply never runs — which is exactly right, since i has no multiples inside the range other than itself. That is also why the outer loop can stop at $\sqrt{n}$ for **marking**, though it must keep going to n to **collect** the remaining primes.

14. Use **Lucas' theorem**. Write n and r in base p : $n = n_k \dots n_1 n_0$ and $r = r_k \dots r_1 r_0$. Then

$$\binom{n}{r} \equiv \prod_{i=0}^k \binom{n_i}{r_i} \pmod{p}$$

Each small binomial has $n_i, r_i < p \leq 13$, so a tiny Pascal table answers it, and if any $r_i > n_i$ the whole product is 0. The number of digits is $\log_p n$, about 17 for $p = 13$ and $n = 10^{18}$, so each query is essentially free. In JavaScript, extract the base- p digits with `BigInt`: `n % BigInt(p)` then `n /= BigInt(p)`, because n is past 2^{53} and a number-based loop would corrupt the digits before it started.

15. `1` is converted to a signed 32-bit integer holding the value 1 in bit 0, and JavaScript then masks the shift count to its bottom five bits.

- At $k = 31$ the shift moves that 1 into the **sign** bit, so `mask` is `-2147483648`. Every `x & mask` test still reads the right bit, but any comparison, print, or `Math.max` involving `mask` behaves backwards.
- At $k = 32$ the count masks to 0, so `1 << 32` computes `1 << 0` and gives `1`, not 0. The program silently tests bit 0.
- At $k = 35$ the count masks to 3, so `1 << 35` gives `8`. Again, no crash, no warning, just a wrong answer.

This is not undefined behaviour, as it is in C++ — it is **defined** to be wrong, which means it is perfectly reproducible on every machine and every judge, and therefore completely invisible until the test data reaches bit 31. The fix is one of two: `2 ** k` for a plain number (exact to $k = 53$), or `1n << BigInt(k)` for a `BigInt` (exact forever). The run prints `1 << 40 = 256`, `1 << 32 = 1`, `2 ** 40 = 1099511627776`.

ONE-PAGE REVISION CARD

The bit table.

$x \gg k \ \& \ 1$	read bit k	$x \ \& \ (x-1)$	clear the lowest set bit
$x \ \ (1 \ll k)$	set bit k	$x \ \& \ -x$	keep only the lowest set bit
$x \ \& \ \sim(1 \ll k)$	clear bit k	$i \ ^ \wedge \ (i \gg 1)$	the i -th Gray code
$x \ ^ \wedge \ (1 \ll k)$	flip bit k	$(a \ ^ \wedge b) < 0$	a and b have opposite signs
$x \ ^ \wedge x = 0$	pairs cancel	$x \ \gg \gg \ 0$	read a 32-bit result as unsigned
$x \ ^ \wedge 0 = x$	zero is the identity	$31 - \text{Math.clz32}(x)$	index of the highest set bit

The number-theory table.

$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$	Euclid, $O(\log \min(a, b))$
$\text{lcm} = a / \text{gcd} \times b$	divide first — never $a \times b / \text{gcd}$
trial division to $\sqrt{n}$	one primality test, $O(\text{sqrt } n)$
sieve, inner loop from i^2	all primes to n , $O(n \log \log n)$
smallest-prime-factor sieve	factorise any $x \leq n$ in $O(\log x)$
$\lfloor \frac{n}{5} \rfloor + \lfloor \frac{n}{25} \rfloor + \dots$	trailing zeros of $n!$
$a^{-1} \equiv a^{p-2} \pmod{p}$	inverse, prime modulus only
extended Euclid	inverse for any modulus with $\text{gcd} = 1$
$\varphi(n) = n \prod (1 - 1/p)$	Euler totient, $O(\text{sqrt } n)$
$\binom{n}{r}$ with factorial + inverse tables	$O(n)$ setup, $O(1)$ per query

Pattern to technique.

What the problem says	Reach for	Cost
one value appears once, others twice	XOR everything	$O(n)$
two values appear once	XOR, then split on <code>all & -all</code>	$O(n)$
others appear three times	count bits mod 3	$O(32n)$
largest XOR of a pair	binary trie, greedy walk	$O(32n)$
count subarrays with $\text{XOR} = k$	prefix XOR + Map	$O(n)$
total set bits over a range	count column by column	$O(63)$
huge exponent	square and multiply, BigInt	$O(\log b)$
division under a modulus	multiply by the inverse	$O(\log p)$
many factorisation queries	smallest-prime-factor sieve	$O(\log x)$ each
count numbers up to 10^{18} with a digit property	digit DP on <code>String(N)</code>	$O(d \cdot \text{state})$
subset sum with a small target	BigInt shift-OR	$O(nT/64)$

The JavaScript facts this chapter is built on.

bitwise is 32-bit	<code>&</code> , <code> </code> , <code>^</code> , <code><<</code> , <code>>></code> convert to a signed 32-bit integer first. <code>1 << 31</code> is negative, <code>1 << 32</code> is 1, <code>1 << 40</code> is 256.
-------------------	---

>> vs >>>	>> copies the sign bit, so <code>-1 >> 1</code> is <code>-1</code> and the loop hangs. <code>>>></code> shifts in zeros. End an unsigned computation with <code>>>> 0</code> .
numbers are doubles	exact only to $2^{53} - 1 \approx 9 \times 10^{15}$. A literal past it is already the wrong value.
BigInt or nothing	any product under a modulus near 10^9 , any 2^k with $k > 30$, any count that can pass 9×10^{15} .
no mixing	<code>1n + 1</code> throws; <code>Math.max(0n, 1n)</code> throws. Convert at the boundary, then stay in one world.
no built-ins	no popcount, no <code>ctz</code> , no 128-bit type, no <code>bitset</code> . A BigInt is the <code>bitset</code> ; the Kernighan loop is the popcount.
<code>arr.sort()</code>	lexicographic. Always <code>arr.sort((a, b) => a - b)</code> , and a comparator must return a number .
Map over <code>{}</code>	object keys stringify; Map keeps the key's type and insertion order and is faster under heavy churn.
typed arrays	<code>Uint8Array</code> / <code>Int32Array</code> for sieves and DP tables — flat memory, zero-filled, no tagged values.
shallow copy	<code>[...grid]</code> shares every row. A 2D copy is <code>grid.map(row => [...row])</code> .

This is the one chapter in the book that borrows **nothing** from the **JS Toolkit** appendix. Bits and number theory need no heap, no DSU, no deque and no `lowerBound` — they need **BigInt** and a clear head about 32-bit wraparound. Everywhere else in this book, start with `const { MinHeap, DSU, Deque, lowerBound, upperBound } = require('./toolkit.js')` rather than rewriting a structure; here, there is nothing to import.

The six traps that cost marks.

1. **`1 << k` for $k \geq 31$.** The shift count is masked to 5 bits, so `1 << 32` is 1, not 0, and nothing warns you. Use `2 ** k` or `1n << BigInt(k)`.
2. **`a * b` before the modulus.** Doubles round past $2^{53} - 1$. Convert to **BigInt** first.
3. **Subtraction under a modulus.** Always `(a - b + M) % M`.
4. **`r = 1n % m`, not `r = 1n`.** The modulus can be 1.
5. **`>>` where you meant `>>>`.** A right-shift loop on a negative number never terminates.
6. **`Math.sqrt` on a big integer.** Above 2^{53} it lies in both directions. Use `d * d <= n` while the values are small, and **BigInt** binary search once they are not.

Two checks before you submit any bit or math solution. Run your code on 0, on 1, on the maximum allowed input, and on a negative if negatives are allowed. Then ask: what is the **largest intermediate value** my code ever forms — and is it under $2^{53} - 1$, or does this function need to be **BigInt** from end to end?

The JS Toolkit

Five things every other interview language gives you for free, and JavaScript does not.

JavaScript’s standard library is small. It has no heap, no ordered map, no real queue and no binary search. Those four gaps cost marks, because the algorithms that need them are exactly the ones interviews ask about.

This appendix holds the five pieces that fill the gaps. Every one of them is short enough to type from memory in an interview, and every one is tested — the test file lives beside the source and reports **15 passed, 0 failed**.

HOW TO USE THIS IN A REAL CODING ROUND

Put all five in one file and start every solution with one line:

```
const { MinHeap, DSU, Deque, lowerBound, upperBound } = require('./toolkit.js');
```

In a whiteboard or online-judge round where you cannot import anything, type in **only the piece you need**. MinHeap is about 25 lines, DSU is 8, Deque is 8, and each binary search is one line. Say out loud what you are doing: “JavaScript has no priority queue, so I will write a small binary heap — about twenty lines — and then the algorithm.” That sentence earns marks. Silently writing a slow array-scan instead loses them.

1 · MinHeap — the priority queue JavaScript does not have

Replaces	C++ <code>priority_queue</code> , Java <code>PriorityQueue</code> , Python <code>heapq</code>
Reach for it when	the question says top-K, K-th largest, “merge K sorted lists”, “running median”, Dijkstra, Prim, “schedule the next task”, or any loop whose next step is “take the current smallest / largest”
Do not reach for it when	you only need the single minimum (<code>Math.min</code>), or the data is already sorted (two pointers is cheaper)

MinHeap — verbatim from the shared toolkit

JS

```
// ---- MinHeap / priority queue (JS has none built in) ----
class MinHeap {
  constructor(cmp = (a, b) => a - b) { this.a = []; this.cmp = cmp; }
  get size() { return this.a.length; }
  peek() { return this.a[0]; }
  push(v) {
    this.a.push(v);
    let i = this.a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (this.cmp(this.a[i], this.a[p]) >= 0) break;
      [this.a[i], this.a[p]] = [this.a[p], this.a[i]]; i = p;
    }
  }
  pop() {
    const top = this.a[0], last = this.a.pop();
    if (this.a.length) {
      this.a[0] = last;
      let i = 0;
      for (;;) {
        const l = 2*i+1, r = l+1; let m = i;
```

```

    if (l < this.a.length && this.cmp(this.a[l], this.a[m]) < 0) m = l;
    if (r < this.a.length && this.cmp(this.a[r], this.a[m]) < 0) m = r;
    if (m === i) break;
    [this.a[i], this.a[m]] = [this.a[m], this.a[i]]; i = m;
  }
}
return top;
}
}

```

How to use it

three shapes you will need

JS

```

const h = new MinHeap(); // smallest comes out first
[5, 3, 8, 1].forEach(v => h.push(v));
h.pop(); // 1

const mx = new MinHeap((a, b) => b - a); // MAX-heap: flip the comparator
[5, 3, 8].forEach(v => mx.push(v));
mx.pop(); // 8

const pq = new MinHeap((a, b) => a[1] - b[1]); // heap of [name, priority] pairs
pq.push(['a', 3]); pq.push(['b', 1]);
pq.pop(); // ['b', 1]

```

Operation	Time	Note
push	$O(\log n)$	one swap chain up the tree
pop	$O(\log n)$	one swap chain down the tree
peek	$O(1)$	returns <code>undefined</code> on an empty heap
size	$O(1)$	a getter, not a method — no brackets
build from n items	$O(n \log n)$	pushing one at a time
heapify an existing array	$O(n)$	not in this toolkit; sift down from $n/2$ to 0

TRAP

`h.size` is a **getter**. Write `h.size`, not `h.size()`. Calling it throws `TypeError: h.size is not a function`, and it always happens inside a `while` condition where it is hardest to see.

TRAP

The comparator returns a **number**, like `Array.prototype.sort`: negative means “a comes first”. A comparator that returns `true/false` coerces to `1/0`, can never say “a first”, and the heap silently returns items in the wrong order.

SHORTCUT

There is no decrease-key. For Dijkstra, do not try to update an entry in place — push the new, smaller distance as a **second** entry and skip stale entries when they pop: `if (d > dist[u]) continue;`. This is the standard “lazy deletion” heap and it is what every accepted solution does.

2 · DSU — disjoint set union (union-find)

Replaces	nothing in any standard library — C++, Java and Python do not ship one either. Everyone writes it.
Reach for it when	the question is “are these two connected?”, “how many groups?”, “merge these accounts”, Kruskal’s MST, cycle detection in an undirected graph, or “process the edges in some order and keep the components”
Do not reach for it when	you need to remove an edge, or you need the actual path between two nodes. DSU only ever merges, and it never remembers routes.

DSU — verbatim from the shared toolkit

JS

```
// ---- Disjoint Set Union ----
class DSU {
  constructor(n) { this.p = Array.from({length: n}, (_, i) => i); this.r = new
Array(n).fill(0); }
  find(x) { while (this.p[x] !== x) { this.p[x] = this.p[this.p[x]]; x = this.p[x]; }
return x; }
  union(a, b) {
    a = this.find(a); b = this.find(b);
    if (a === b) return false;
    if (this.r[a] < this.r[b]) [a, b] = [b, a];
    this.p[b] = a; if (this.r[a] === this.r[b]) this.r[a]++;
    return true;
  }
}
```

How to use it

count the groups, and detect a cycle

JS

```
const d = new DSU(6);
d.union(0, 1);           // true  – they were separate, now merged
d.union(1, 2);           // true
d.union(0, 2);           // false – already together: this edge closes a CYCLE
d.find(2) === d.find(0); // true
d.find(3) === d.find(0); // false

let groups = 0;           // number of components
for (let i = 0; i < 6; i++) if (d.find(i) === i) groups++; // 4
```

Operation	Time	Note
find	$O(\alpha(n))$	α is the inverse Ackermann function
union	$O(\alpha(n))$	returns false if they were already together
build for n items	$O(n)$	two arrays
m operations on n items	$O(m\alpha(n))$	quote this — “effectively constant”

$\alpha(n)$ is below 5 for every n that fits in a computer. In an interview say “amortised almost constant, $O(\alpha(n))$ ”, never plain $O(1)$ — the distinction is exactly what the question is testing.

SHORTCUT

union returning false is the whole cycle-detection algorithm. In Kruskal's MST you write `if (d.union(u, v)) total += w;` — the boolean both merges the sets and tells you whether the edge was safe to take.

TRAP

find here does **path halving**, so it flattens the tree as it walks. Do not “simplify” it to `while (p[x] !== x) x = p[x];` — that version keeps the tree tall and turns a chain of 10^5 unions into $O(n^2)$.

3 · Deque — an $O(1)$ queue

Replaces	C++ deque, Java ArrayDeque, Python <code>collections.deque</code>
Reach for it when	BFS, level-order traversal, monotonic-window problems (window maximum, shortest subarray with a condition), 0-1 BFS, or any loop that pops from the front
Do not reach for it when	you only push and pop at the back — a plain array is already a perfect stack

WHY THIS EXISTS AT ALL

A plain array **looks** like a queue: `q.push(x)` and `q.shift()`. But `shift()` removes the first element and then moves every remaining element down one slot. On a queue of 10^5 items that is 10^{10} element moves over the run — an $O(n)$ BFS silently becomes $O(n^2)$ and times out.

Measured in Node on 100000 items: `shift()` in a loop took **971 ms**; the same loop with a head index took **2 ms**.

The fix is to never actually remove the front. Keep a head index, step it forward, and compact the array only when more than half of it is dead.

Deque — verbatim from the shared toolkit

JS

```
// ---- Deque backed by a plain array with head index (O(1) amortised shift) ----
class Deque {
  constructor() { this.a = []; this.h = 0; }
  get size() { return this.a.length - this.h; }
  push(v) { this.a.push(v); }
  pop() { return this.a.pop(); }
  shift() { const v = this.a[this.h++]; if (this.h * 2 > this.a.length) { this.a = this.a.slice(this.h); this.h = 0; } return v; }
  front() { return this.a[this.h]; }
  back() { return this.a[this.a.length - 1]; }
}
```

How to use it

BFS, and a monotonic window maximum

JS

```
const q = new Deque();
q.push(1); q.push(2); q.push(3);
q.shift();           // 1  — from the FRONT
q.back();            // 3
q.size;              // 2  — a getter, no brackets

// window maximum: the deque holds INDEXES, values decreasing
const windowMax = (a, k) => {
```

```
const dq = new Deque(), out = [];
for (let i = 0; i < a.length; i++) {
  while (dq.size && a[dq.back()] <= a[i]) dq.pop();           // drop the useless
  dq.push(i);
  if (dq.front() <= i - k) dq.shift();                     // drop the expired
  if (i >= k - 1) out.push(a[dq.front()]);
}
return out;
};
```

Operation	Time	Note
push (back)	$O(1)$	plain array push
pop (back)	$O(1)$	plain array pop
shift (front)	$O(1)$ amortised	head index; compacts when half is dead
front / back / size	$O(1)$	front and back are methods, size is a getter
memory	$O(n)$	holds dead slots until the next compaction

TRAP

This Deque has no unshift — you cannot push to the **front**. Plain BFS and monotonic windows never need to. If a problem genuinely does (0-1 BFS with zero-weight edges), keep two arrays: one you push to and read backwards for the front, one for the back. Say that out loud rather than reaching for `Array.prototype.unshift`, which is $O(n)$ for the same reason shift is.

TRAP

`dq.size` is a getter and `dq.front()` is a method. `while (dq.size() ...)` throws; `if (dq.front <= i - k)` compares a **function** with a number and is always false. Both mistakes compile and both are silent.

4 · lowerBound / upperBound — the binary search JS is missing

Replaces	C++ <code>lower_bound / upper_bound</code> , Python <code>bisect_left / bisect_right</code> , Java <code>Collections.binarySearch</code>
Stands in for	an ordered map — C++'s <code>map</code> , Java's <code>TreeMap</code> , Python's <code>sortedcontainers</code> . JavaScript has none of them. A sorted array plus these two searches is the standard workaround.
Reach for it when	the array is sorted and you need: an insert position, the first element $\geq x$, the count of a value, the number of elements below a threshold, LIS in $O(n \log n)$, or “the next booking after time t ”

lowerBound / upperBound — verbatim from the shared toolkit

JS

```
// ---- binary search helpers (JS has no lower_bound) ----
const lowerBound = (a, x) => { let lo = 0, hi = a.length; while (lo < hi) { const m =
(lo+hi)>>1; a[m] < x ? lo = m+1 : hi = m; } return lo; };
const upperBound = (a, x) => { let lo = 0, hi = a.length; while (lo < hi) { const m =
(lo+hi)>>1; a[m] <= x ? lo = m+1 : hi = m; } return lo; };
```

How to use it

the five things these two lines give you

JS

```

const a = [1, 3, 3, 5, 7];

lowerBound(a, 3);           // 1  first index with a[i] >= 3
upperBound(a, 3);           // 3  first index with a[i] > 3
upperBound(a, 3) - lowerBound(a, 3); // 2  how many 3s
lowerBound(a, 0);           // 0  x is below everything
lowerBound(a, 9);           // 5  == a.length: x is above everything

a[lowerBound(a, 4)] === 4;   // false - 4 is ABSENT. This is "binary search".
a.splice(lowerBound(a, 4), 0, 4); // insert 4 and keep the array sorted

```

Operation	Time	Note
lowerBound / upperBound	$O(\log n)$	the array must already be sorted
membership test	$O(\log n)$	<code>a[lowerBound(a, x)] === x</code> — check the value!
count of a value	$O(\log n)$	<code>upperBound - lowerBound</code>
insert, keeping order	$O(n)$	the <code>splice</code> shift dominates the search
delete one copy	$O(n)$	<code>splice(i, 1)</code> after <code>lowerBound</code>
n inserts into a sorted array	$O(n^2)$	say this out loud before you use it

TRAP

These return an **insert position**, never -1. `lowerBound(a, 4)` on `[1,3,3,5,7]` returns 3, and `a[3]` is 5. Always check the value before you trust the index: `const i = lowerBound(a, x); const found = i < a.length && a[i] === x;`

TRAP

`(lo + hi) >> 1` is safe **here** because `lo` and `hi` are array indexes, which never reach 2^{31} . Do not copy this midpoint into a binary search **on the answer**, where the bounds can be 10^{15} ; `>>` truncates to 32 bits and the search returns nonsense. There, use `Math.floor(lo + (hi - lo) / 2)`.

SHORTCUT

Sorted array plus these two searches is your answer to **every** “JavaScript has no TreeMap” moment. Quote the trade-off: lookups are $O(\log n)$, the same as a balanced tree, but each insert is $O(n)$ instead of $O(\log n)$. That is fine for up to a few tens of thousands of elements, and an interviewer who hears you state the limit is satisfied.

JS gotchas for interviews — all seven on one page

Every line below was run in Node. The output is what actually printed.

1	sort() is lexicographic. It compares numbers as text . <code>[10,9,1].sort()</code> → <code>[1,10,9]</code> <code>[10,9,1].sort((a,b)=>a-b)</code> → <code>[1,9,10]</code>
2	There is no priority queue. <code>typeof PriorityQueue</code> prints "undefined". <code>arr.sort((a,b)=>a-b)[0]</code> inside a loop → $O(n^2 \log n)$ <code>const h = new MinHeap(); h.push(x); h.pop();</code> → $O(\log n)$ each
3	There is no TreeMap or ordered set. <code>typeof TreeMap</code> prints "undefined". <code>[...map.keys()].sort()</code> inside the loop → $O(n^2 \log n)$ <code>sorted array + lowerBound(a, x)</code> → $O(\log n)$ to find
4	Numbers are doubles — exact only to $2^{53} - 1$. <code>123456789 * 987654321</code> → <code>121932631112635260</code> (wrong) <code>123456789n * 987654321n</code> → <code>121932631112635269n</code> (exact)
5	Recursion depth is about 10^4 frames. Measured: RangeError after 12545 frames. <code>const dfs = (u) => { ... dfs(v) ... }</code> on 10^5 nodes → stack overflow <code>const st = [s]; while (st.length) { const u = st.pop(); ... }</code> → no limit
6	Object keys are strings; Map keys are not. <code>o[1]="num"; o["1"]="str";</code> → one entry, <code>o[1]</code> is "str" <code>m.set(1,"num"); m.set("1","str");</code> → <code>m.size</code> is 2
7	Array copy is shallow. <code>const b = [...g]; b[0][0] = 9;</code> → <code>g[0][0]</code> is now 9 <code>const b = g.map(r => [...r]); b[0][0] = 9;</code> → <code>g[0][0]</code> is still 0

Three more that cost marks just as often

8	new Array(n) makes holes, not zeros. <code>map</code> skips holes. <code>new Array(3).map(x => 1)</code> → <code>[null,null,null]</code> <code>new Array(3).fill(0).map(x => 1)</code> → <code>[1,1,1]</code>
9	fill with an array stores one shared row. <code>new Array(2).fill(new Array(2).fill(0))</code> then <code>g[0][1]=9</code> → <code>[[0,9],[0,9]]</code> <code>Array.from({length: 2}, () => new Array(2).fill(0))</code> → <code>[[0,9],[0,0]]</code>
10	shift() on an array is $O(n)$. Measured on 100000 items: 971 ms against 2 ms. <code>const q = [s]; while (q.length) q.shift();</code> → $O(n^2)$ <code>const q = new Deque(); q.push(s); while (q.size) q.shift();</code> → $O(n)$

Four of these ten — 1, 4, 6 and 7 — produce a **wrong answer with no error message**. Those are the dangerous ones. Before you say “done”, check: did I sort numbers with a comparator, can any value pass 2^{53} , am I using a Map, and did I deep-copy the grid?

JavaScript against Python — the quick table

Python is the other language interviewers accept everywhere. If you write JavaScript, know the Python line too: it is often the shorter way to **explain** what you are doing.

Arrays and lists

You want	JavaScript	Python
length	<code>a.length</code>	<code>len(a)</code>
add at the end	<code>a.push(x)</code>	<code>a.append(x)</code>
remove from the end	<code>a.pop()</code>	<code>a.pop()</code>
remove from the front	<code>Deque.shift()</code> — array <code>shift()</code> is $O(n)$	<code>collections.deque.popleft()</code>
last element	<code>a.at(-1)</code>	<code>a[-1]</code>
slice	<code>a.slice(1, 4)</code>	<code>a[1:4]</code>
reverse a copy	<code>[...a].reverse()</code>	<code>a[::-1]</code>
copy (one level)	<code>[...a]</code>	<code>a[:]</code> or <code>list(a)</code>
copy a 2D grid	<code>a.map(r => [...r])</code>	<code>[r[:] for r in a]</code>
n zeros	<code>new Array(n).fill(0)</code>	<code>[0] * n</code>
an $r \times c$ grid	<code>Array.from({length: r}, () => new Array(c).fill(0))</code>	<code>[[0]*c for _ in range(r)]</code>
range 0..n-1	<code>[...Array(n).keys()]</code>	<code>range(n)</code>
index and value together	<code>a.forEach((x, i) => ...)</code>	<code>for i, x in enumerate(a)</code>
sum	<code>a.reduce((p, q) => p + q, 0)</code>	<code>sum(a)</code>
max	<code>Math.max(...a)</code> — breaks past 100k items	<code>max(a)</code>
filter	<code>a.filter(x => x > 0)</code>	<code>[x for x in a if x > 0]</code>
does it contain x	<code>a.includes(x)</code> — $O(n)$	<code>x in a</code> — $O(n)$

Maps and sets

You want	JavaScript	Python
empty map	<code>new Map()</code>	<code>{}</code>
set a value	<code>m.set(k, v)</code>	<code>m[k] = v</code>
read a value	<code>m.get(k)</code> — undefined if absent	<code>m[k]</code> — raises <code>KeyError</code>
read with a default	<code>m.get(k) ?? 0</code>	<code>m.get(k, 0)</code>
count into it	<code>m.set(k, (m.get(k) ?? 0) + 1)</code>	<code>c[k] += 1</code> with <code>Counter()</code>
does the key exist	<code>m.has(k)</code>	<code>k in m</code>
delete a key	<code>m.delete(k)</code>	<code>del m[k]</code>
number of keys	<code>m.size</code> — a getter	<code>len(m)</code>
walk keys and values	<code>for (const [k, v] of m)</code>	<code>for k, v in m.items()</code>

keys in sorted order	<code>[...m.keys()].sort((a,b)=>a-b)</code>	<code>sorted(m)</code>
empty set	<code>new Set()</code>	<code>set()</code>
add / test / remove	<code>s.add(x)</code> · <code>s.has(x)</code> · <code>s.delete(x)</code>	<code>s.add(x)</code> · <code>x in s</code> · <code>s.discard(x)</code>
set from an array	<code>new Set(a)</code>	<code>set(a)</code>
back to an array	<code>[...s]</code>	<code>list(s)</code>
most common element	count in a Map, then scan	<code>Counter(a).most_common(1)</code>

TRAP

The row that matters most: `m.get(k)` returns undefined for a missing key, and `undefined + 1` is NaN. Python raises a loud `KeyError` instead. JavaScript fails **quietly**, so write `?? 0` every single time you count.

Sorting

You want	JavaScript	Python
sort numbers	<code>a.sort((x, y) => x - y)</code> — the comparator is mandatory	<code>a.sort()</code>
descending	<code>a.sort((x, y) => y - x)</code>	<code>a.sort(reverse=True)</code>
sort a copy	<code>[...a].sort((x, y) => x - y)</code>	<code>sorted(a)</code>
by a key	<code>a.sort((x, y) => x.age - y.age)</code>	<code>a.sort(key=lambda p: p.age)</code>
two keys	<code>a.sort((x, y) => x.age - y.age x.name.localeCompare(y.name))</code>	<code>a.sort(key=lambda p: (p.age, p.name))</code>
strings	<code>a.sort()</code> works, or <code>localeCompare</code>	<code>a.sort()</code>
stable?	yes, guaranteed since ES2019	yes, always
in place?	yes — <code>sort</code> mutates and returns the same array	<code>sort()</code> mutates, <code>sorted()</code> does not
k smallest	toolkit <code>MinHeap</code> , pop k times	<code>heapq.nsmallest(k, a)</code>

Strings

You want	JavaScript	Python
character at i	<code>s[i]</code> (read only)	<code>s[i]</code>
change one character	<code>const t = [...s]; t[i] = 'x'; t.join('')</code>	<code>s[:i] + 'x' + s[i+1:]</code>
build a long string	push into an array, then <code>.join('')</code>	append to a list, then <code>' '.join(...)</code>
concatenate in a loop	safe in V8 (ropes), but <code>join</code> is clearer	$O(n^2)$ — always use <code>join</code>
code of a character	<code>s.charCodeAt(i)</code>	<code>ord(s[i])</code>
character from a code	<code>String.fromCharCode(n)</code>	<code>chr(n)</code>
letter index a..z	<code>s.charCodeAt(i) - 97</code>	<code>ord(s[i]) - ord('a')</code>
split / join	<code>s.split(' ').arr.join(' ')</code>	<code>s.split(' ') · ' '.join(arr)</code>
reverse	<code>[...s].reverse().join('')</code>	<code>s[::-1]</code>

substring	<code>s.slice(i, j)</code>	<code>s[i:j]</code>
starts with	<code>s.startsWith(p)</code>	<code>s.startswith(p)</code>
repeat	<code>'ab'.repeat(3)</code>	<code>'ab' * 3</code>

TRAP

Strings are immutable in **both** languages. `s[i] = 'x'` in JavaScript changes nothing and raises no error at all — Python at least throws `TypeError`. Spread into an array, edit, `join('')`.

Numbers, and where the two languages really differ

You want	JavaScript	Python
integer division	<code>Math.floor(a / b)</code>	<code>a // b</code>
remainder, never negative	<code>((a % b) + b) % b</code>	<code>a % b</code> — already non-negative
big integers	<code>BigInt</code> : <code>2n ** 100n</code>	built in: <code>2 ** 100</code>
exact integer range	$\pm(2^{53} - 1)$, then rounding	unlimited
infinity	<code>Infinity</code> / <code>-Infinity</code>	<code>float('inf')</code> / <code>-float('inf')</code>
bit operations	32-bit only; use <code>>>></code> to stay unsigned	unlimited width
max / min of two	<code>Math.max(a, b)</code>	<code>max(a, b)</code>

The single biggest real difference: **Python integers never overflow and never round**. JavaScript's stop being exact at $2^{53} - 1 \approx 9.0 \times 10^{15}$. If a problem's answer can pass that — counting DPs, products of two large numbers, $n(n+1)/2$ for huge n — either take the answer modulo $10^9 + 7$, or move that whole computation to `BigInt`. This is the one place where the Python solution is genuinely simpler, and it is worth saying so in an interview.